O'REILLY®

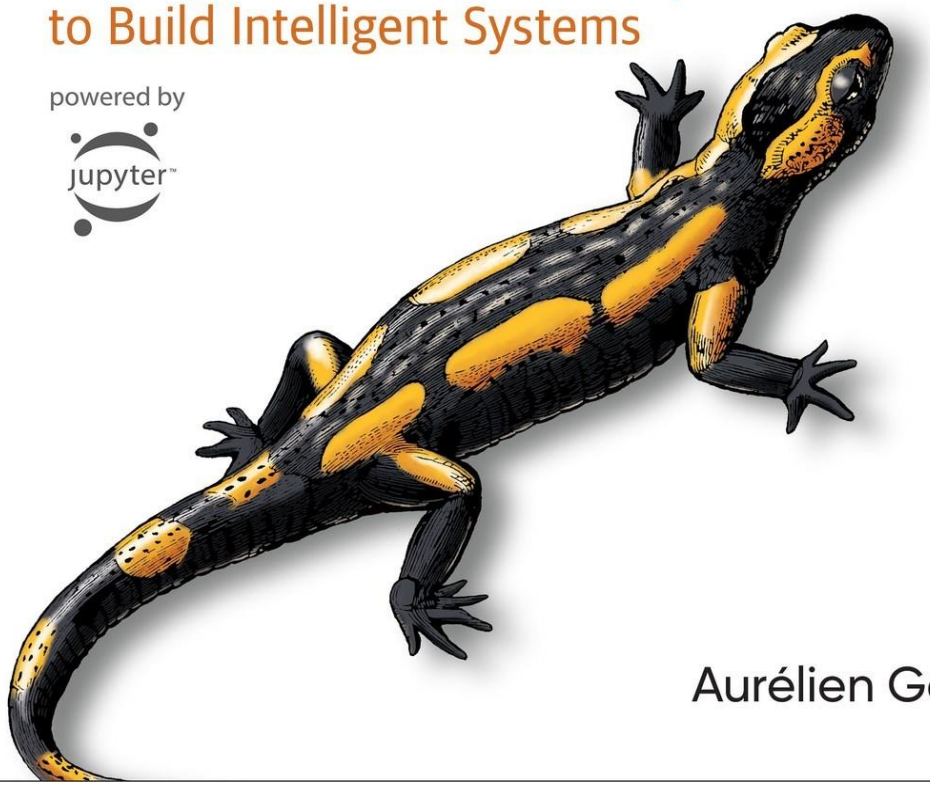
Third Edition

Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow

Concepts, Tools, and Techniques to Build Intelligent Systems

powered by

jupyter™



Aurélien Géron

Cuốn sách “Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow” (Thực hành Học máy với Scikit-Learn, Keras và TensorFlow) Phiên bản thứ ba Các khái niệm, công cụ và kỹ thuật để xây dựng hệ thống thông minh

Aurélien Géron

Thực hành Học máy với Scikit-Learn, Keras và TensorFlow bởi Aurélien Géron Bản quyền © 2023 Aurélien Géron. Bảo lưu mọi quyền. In tại Hợp chủng quốc Hoa Kỳ. Được xuất bản bởi O’Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472. Sách của O’Reilly có thể được mua cho mục đích giáo dục, kinh doanh hoặc khuyến mại. Các phiên bản trực tuyến cũng có sẵn cho hầu hết các tựa sách (https://oreilly.com). Để biết thêm thông tin, hãy liên hệ với bộ phận bán hàng của công ty/tổ chức của chúng tôi: 800-998-9938 hoặc corporate@oreilly.com.

Biên tập viên mua lại: Nicole Butterfield

Biên tập viên phát triển: Nicole Taché và Michele Cronin

Biên tập viên sản xuất: Beth Kelly Biên tập viên bản quyền: Kim Cofer Người đọc soát lỗi: Rachel Head Người lập chỉ mục: Potomac Indexing, LLC Thiết kế nội thất: David Futato

Thiết kế bìa: Karen Montgomery Minh họa: Kate Dullea

Tháng 3 năm 2017: Phiên bản đầu tiên Tháng 9 năm 2019: Phiên bản thứ hai Tháng 10 năm 2022: Phiên bản thứ ba Lịch sử sửa đổi cho Phiên bản thứ ba

2022-10-03:

Phát hành lần đầu Xem https://oreilly.com/catalog/errata.csp?isbn=9781492032649 để biết chi tiết phát hành. Logo của O’Reilly là nhãn hiệu đã đăng ký của O’Reilly Media, Inc. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, hình ảnh bìa và nhãn hiệu liên quan là các nhãn hiệu của O’Reilly Media, Inc. Các quan điểm thể hiện trong tác phẩm này là của tác giả, và không đại diện cho quan điểm của nhà xuất bản. Mặc dù nhà xuất bản và tác giả đã nỗ lực hết sức để đảm bảo rằng thông tin và hướng dẫn trong tác phẩm này là chính xác, nhưng nhà xuất bản và tác giả từ chối mọi trách nhiệm về các lỗi hoặc thiếu sót, bao gồm nhưng không giới hạn trách nhiệm về các thiệt hại do việc sử dụng hoặc dựa vào tác phẩm này. Việc sử dụng thông tin và hướng dẫn trong tác phẩm này là rủi ro của riêng bạn. Nếu bất kỳ mẫu mã hoặc công nghệ nào khác mà tác phẩm này chứa đựng hoặc mô tả phải tuân theo giấy phép mã nguồn mở hoặc quyền sở hữu trí tuệ của người khác, bạn có trách nhiệm đảm bảo rằng việc sử dụng của bạn tuân thủ các giấy phép và/hoặc quyền đó. 978-1-098-12597-4 [LSI]

Lời nói đầu

Cơn sóng thần học máy Năm 2006, Geoffrey Hinton và cộng sự đã xuất bản một bài báo1 trình bày cách đào tạo một mạng lưới thần kinh sâu có khả năng nhận diện chữ viết tay với độ chính xác tiên tiến (>98%). Họ gọi kỹ thuật này là "học sâu". Mạng lưới thần kinh sâu là một mô hình (rất) đơn giản hóa của vỏ não, bao gồm một chồng các lớp nơ-ron nhân tạo. Việc đào tạo một mạng lưới thần kinh sâu được nhiều người coi là không thể vào thời điểm đó,2 và hầu hết các nhà nghiên cứu đã từ bỏ ý tưởng này vào cuối những năm 1990. Bài báo này đã khơi dậy sự quan tâm của cộng đồng khoa học, và chẳng bao lâu sau nhiều bài báo mới đã chứng minh rằng học sâu không chỉ khả thi, mà còn có khả năng đạt được những thành tựu đáng kinh ngạc mà không kỹ thuật học máy (ML) nào khác có thể sánh được (với sự trợ giúp của sức mạnh tính toán khổng lồ và lượng dữ liệu lớn). Sự nhiệt tình này nhanh chóng lan rộng sang nhiều lĩnh vực khác của học máy. Một thập kỷ sau, học máy đã chinh phục ngành công nghiệp, và ngày nay nó là cốt lõi của phần lớn phép màu trong các sản phẩm công nghệ cao, xếp hạng kết quả tìm kiếm trên web của bạn, cung cấp năng lượng cho nhận dạng giọng nói của điện thoại thông minh, đề xuất video và thậm chí có thể lái xe của bạn.

Học máy trong các dự án của bạn Vì vậy, tự nhiên bạn rất hào hứng với học máy và muốn tham gia vào cuộc chơi! Có lẽ bạn muốn cung cấp cho robot tự chế của mình một bộ não riêng? Làm cho nó nhận diện khuôn mặt? Hoặc học cách đi lại? Hoặc có thể công ty của bạn có hàng tấn dữ liệu (nhật ký người dùng, dữ liệu tài chính, dữ liệu sản xuất, dữ liệu cảm biến máy, số liệu thống kê đường dây nóng, báo cáo nhân sự, v.v.) và rất có thể bạn có thể khám phá một số viên ngọc ẩn nếu bạn biết nơi để tìm. Với học máy, bạn có thể hoàn thành những điều sau đây và nhiều hơn nữa:

- Phân khúc khách hàng và tìm chiến lược tiếp thị tốt nhất cho từng nhóm.
- Đề xuất sản phẩm cho mỗi khách hàng dựa trên những gì khách hàng tương tự đã mua.
- Phát hiện các giao dịch có khả năng gian lận.
- Dự báo doanh thu năm tới.

Dù lý do là gì, bạn đã quyết định học máy và triển khai nó vào các dự án của mình. Ý tưởng tuyệt vời!

Mục tiêu và phương pháp tiếp cận Cuốn sách này giả định rằng bạn không biết gì nhiều về học máy. Mục tiêu của nó là cung cấp cho bạn các khái niệm, công cụ và trực giác bạn cần để triển khai các chương trình có khả năng học từ dữ liệu. Chúng ta sẽ bao gồm một số lượng lớn các kỹ thuật, từ đơn giản nhất và được sử dụng phổ biến nhất (như hồi quy tuyến tính) đến một số kỹ thuật học sâu thường xuyên chiến thắng các cuộc thi. Để làm được điều này, chúng ta sẽ sử dụng các khung công tác Python sẵn sàng cho sản xuất:

- **Scikit-Learn** rất dễ sử dụng, nhưng nó triển khai nhiều thuật toán học máy một cách hiệu quả, vì vậy nó là một điểm khởi đầu tuyệt vời để học học máy. Nó được David

Cournapeau tạo ra vào năm 2007, và hiện được dẫn dắt bởi một nhóm các nhà nghiên cứu tại Viện Nghiên cứu Khoa học Máy tính và Tự động hóa Pháp (Inria).

- **TensorFlow** là một thư viện phức tạp hơn để tính toán số phân tán. Nó giúp đào tạo và chạy các mạng lưới thần kinh rất lớn một cách hiệu quả bằng cách phân phối các phép tính trên hàng trăm máy chủ đa GPU (đơn vị xử lý đồ họa). TensorFlow (TF) được tạo ra tại Google và hỗ trợ nhiều ứng dụng học máy quy mô lớn của nó. Nó được mã nguồn mở vào tháng 11 năm 2015 và phiên bản 2.0 được phát hành vào tháng 9 năm 2019.
- **Keras** là một API học sâu cấp cao giúp việc đào tạo và chạy mạng lưới thần kinh trở nên rất đơn giản. Keras được tích hợp sẵn với TensorFlow và nó dựa vào TensorFlow cho tất cả các phép tính chuyên sâu.

Cuốn sách ưu tiên phương pháp tiếp cận thực hành, phát triển sự hiểu biết trực quan về học máy thông qua các ví dụ thực tế và một chút lý thuyết.

Mã ví dụ

Tất cả các ví dụ mã trong cuốn sách này đều là mã nguồn mở và có sẵn trực tuyến tại https://github.com/ageron/handson-ml3 , dưới dạng sổ ghi chép Jupyter. Đây là các tài liệu tương tác chứa văn bản, hình ảnh và các đoạn mã có thể thực thi (trong trường hợp của chúng tôi là Python). Cách dễ nhất và nhanh nhất để bắt đầu là chạy các sổ ghi chép này bằng Google Colab: đây là một dịch vụ miễn phí cho phép bạn chạy bất kỳ sổ ghi chép Jupyter nào trực tiếp trực tuyến mà không cần cài đặt bất cứ thứ gì trên máy của bạn. Tất cả những gì bạn cần là một trình duyệt web và tài khoản Google.

Cuốn sách này ở đây để giúp bạn hoàn thành công việc của mình. Nếu bạn muốn sử dụng nội dung bổ sung ngoài các ví dụ mã, và việc sử dụng đó nằm ngoài phạm vi hướng dẫn sử dụng hợp pháp (chẳng hạn như bán hoặc phân phối nội dung từ sách của O'Reilly, hoặc kết hợp một lượng đáng kể tài liệu từ cuốn sách này vào tài liệu sản phẩm của bạn), vui lòng liên hệ với chúng tôi để xin phép, tại permissions@oreilly.com. Chúng tôi đánh giá cao, nhưng không yêu cầu, ghi công. Một ghi công thường bao gồm tiêu đề, tác giả, nhà xuất bản và ISBN. Ví dụ: "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow của Aurélien Géron. Bản quyền 2023 Aurélien Géron, 978-1-098-12597-4."

Yêu cầu tiên quyết

Cuốn sách này giả định rằng bạn có một số kinh nghiệm lập trình Python. Nếu bạn chưa biết Python, https://learnpython.org là một nơi tuyệt vời để bắt đầu. Hướng dẫn chính thức trên Python.org cũng khá tốt. Cuốn sách này cũng giả định rằng bạn đã quen thuộc với các thư viện khoa học chính của Python — đặc biệt là NumPy, Pandas và Matplotlib. Nếu bạn chưa bao giờ sử dụng các thư viện này, đừng lo lắng; chúng dễ học, và tôi đã tạo một hướng dẫn cho mỗi thư viện. Bạn có thể truy cập chúng trực tuyến tại https://homl.info/tutorials . Hơn nữa, nếu bạn muốn hiểu đầy đủ cách hoạt động của các thuật toán học máy (không chỉ cách sử dụng chúng), thì bạn nên có ít nhất một sự hiểu biết cơ bản về một vài khái niệm toán học, đặc biệt là đại số tuyến tính. Cụ thể, bạn nên biết vector và ma trận là gì, và

cách thực hiện một số phép toán đơn giản như cộng vector, hoặc chuyển vị và nhân ma trận. Nếu bạn cần một giới thiệu nhanh về đại số tuyến tính (nó thực sự không phải là khoa học tên lửa!), tôi cung cấp một hướng dẫn tại https://homl.info/tutorials . Bạn cũng sẽ tìm thấy một hướng dẫn về giải tích vi phân, có thể hữu ích để hiểu cách các mạng thần kinh được đào tạo, nhưng nó không hoàn toàn cần thiết để nắm bắt các khái niệm quan trọng. Cuốn sách này cũng sử dụng các khái niệm toán học khác một cách không thường xuyên, chẳng hạn như số mũ và logarit, một chút lý thuyết xác suất và một số khái niệm thống kê cơ bản, nhưng không có gì quá nâng cao. Nếu bạn cần trợ giúp về bất kỳ điều nào trong số này, vui lòng kiểm tra https://khanacademy.org , nơi cung cấp nhiều khóa học toán học trực tuyến miễn phí và xuất sắc.

Lộ trình

Cuốn sách này được tổ chức thành hai phần. Phần I, “Những kiến thức cơ bản về học máy”, bao gồm các chủ đề sau:

- Học máy là gì, nó cố gắng giải quyết những vấn đề gì, và các loại chính cũng như các khái niệm cơ bản của các hệ thống của nó.
- Các bước trong một dự án học máy điển hình.
- Học bằng cách điều chỉnh một mô hình cho dữ liệu.
- Tối ưu hóa hàm chi phí.
- Xử lý, làm sạch và chuẩn bị dữ liệu.
- Chọn và thiết kế các đặc trưng.
- Chọn một mô hình và điều chỉnh siêu tham số bằng cách sử dụng kiểm định chéo.
- Những thách thức của học máy, đặc biệt là hiện tượng thiếu khớp và quá khớp (sự đánh đổi giữa độ chệch/phương sai).
- Các thuật toán học phổ biến nhất: hồi quy tuyến tính và đa thức, hồi quy logistic, k-hàng xóm gần nhất, máy vector hỗ trợ, cây quyết định, rừng ngẫu nhiên và các phương pháp kết hợp.
- Giảm số chiều của dữ liệu huấn luyện để chống lại “lời nguyền của số chiều”.
- Các kỹ thuật học không giám sát khác, bao gồm phân cụm, ước lượng mật độ và phát hiện bất thường.

Phần II, “Mạng nơ-ron và học sâu”, bao gồm các chủ đề sau:

- Mạng nơ-ron là gì và chúng dùng để làm gì.
- Xây dựng và huấn luyện mạng nơ-ron bằng TensorFlow và Keras.
- Các kiến trúc mạng nơ-ron quan trọng nhất: mạng nơ-ron truyền thẳng cho dữ liệu dạng bảng, mạng tích chập cho thị giác máy tính, mạng hồi quy và mạng bộ nhớ dài ngắn hạn (LSTM) cho xử lý chuỗi, bộ mã hóa–giải mã và transformer cho xử lý ngôn ngữ tự nhiên (và nhiều hơn nữa!), bộ mã hóa tự động, mạng tạo sinh đối nghịch (GAN) và mô hình khuếch tán cho học tạo sinh.
- Các kỹ thuật để huấn luyện mạng nơ-ron sâu.

- Cách xây dựng một tác nhân (ví dụ: một bot trong trò chơi) có thể học các chiến lược tốt thông qua thử và lỗi, sử dụng học tăng cường.
- Tải và tiền xử lý lượng lớn dữ liệu một cách hiệu quả.
- Huấn luyện và triển khai mô hình TensorFlow ở quy mô lớn.

Phần đầu tiên chủ yếu dựa trên Scikit-Learn, trong khi phần thứ hai sử dụng TensorFlow và Keras.

Những thay đổi giữa Phiên bản đầu tiên và Phiên bản thứ hai Nếu bạn đã đọc phiên bản đầu tiên, đây là những thay đổi chính giữa phiên bản đầu tiên và phiên bản thứ hai:

- Tất cả mã đã được chuyển từ TensorFlow 1.x sang TensorFlow 2.x, và tôi đã thay thế hầu hết mã TensorFlow cấp thấp (đồ thị, phiên, cột đặc trưng, bộ ước lượng, v.v.) bằng mã Keras đơn giản hơn nhiều.
- Phiên bản thứ hai giới thiệu API Dữ liệu để tải và tiền xử lý các tập dữ liệu lớn, API chiến lược phân phối để huấn luyện và triển khai các mô hình TF ở quy mô lớn, TF Serving và Google Cloud AI Platform để đưa các mô hình vào sản xuất, và (ngắn gọn) TF Transform, TFLite, TF Addons/Seq2Seq, TensorFlow.js và TF Agents.
- Nó cũng giới thiệu nhiều chủ đề ML bổ sung, bao gồm một chương mới về học không giám sát, các kỹ thuật thị giác máy tính để phát hiện đối tượng và phân đoạn ngữ nghĩa, xử lý chuỗi bằng mạng nơ-ron tích chập (CNN), xử lý ngôn ngữ tự nhiên (NLP) bằng mạng nơ-ron hồi quy (RNN), CNN và transformer, GAN, v.v.

Xem https://homl.info/changes2 để biết thêm chi tiết.

Những thay đổi giữa Phiên bản thứ hai và Phiên bản thứ ba

Nếu bạn đã đọc phiên bản thứ hai, đây là những thay đổi chính giữa phiên bản thứ hai và phiên bản thứ ba:

- Tất cả mã đã được cập nhật lên các phiên bản thư viện mới nhất. Đặc biệt, phiên bản thứ ba này giới thiệu nhiều bổ sung mới cho Scikit-Learn (ví dụ: theo dõi tên đặc trưng, tăng cường gradient dựa trên biểu đồ, lan truyền nhãn, v.v.). Nó cũng giới thiệu thư viện Keras Tuner để điều chỉnh siêu tham số, thư viện Transformers của Hugging Face cho xử lý ngôn ngữ tự nhiên và các lớp tiền xử lý và tăng cường dữ liệu mới của Keras.
- Một số mô hình thị giác đã được thêm vào (ResNeXt, DenseNet, MobileNet, CSPNet và EfficientNet), cũng như các hướng dẫn để chọn mô hình phù hợp.
- Chương 15 hiện phân tích dữ liệu hành khách xe buýt và đường sắt Chicago thay vì chuỗi thời gian được tạo, và nó giới thiệu mô hình ARMA và các biến thể của nó.
- Chương 16 về xử lý ngôn ngữ tự nhiên hiện xây dựng một mô hình dịch Anh-Tây Ban Nha, đầu tiên sử dụng bộ mã hóa-giải mã RNN, sau đó sử dụng mô hình transformer. Chương này cũng đề cập đến các mô hình ngôn ngữ như Switch Transformers, DistilBERT, T5 và PaLM (với chuỗi suy nghĩ). Ngoài ra, nó giới thiệu các transformer thị giác (ViT) và đưa ra một cái nhìn tổng quan về một số mô hình thị

giác dựa trên transformer, chẳng hạn như transformer hình ảnh hiệu quả dữ liệu (DeiT), Perceiver và DINO, cũng như một cái nhìn tổng quan ngắn gọn về một số mô hình đa phương thức lớn, bao gồm CLIP, DALL·E, Flamingo và GATO.

- Chương 17 về học tạo sinh hiện giới thiệu các mô hình khuếch tán, và cho thấy cách triển khai một mô hình khuếch tán xác suất khử nhiễu (DDPM) từ đầu.
- Chương 19 đã chuyển từ Google Cloud AI Platform sang Google Vertex AI, và sử dụng Keras Tuner phân tán để tìm kiếm siêu tham số quy mô lớn. Chương này hiện bao gồm mã TensorFlow.js mà bạn có thể thử nghiệm trực tuyến. Nó cũng giới thiệu các kỹ thuật huấn luyện phân tán bổ sung, bao gồm PipeDream và Pathways.

Để có chỗ cho tất cả nội dung mới, một số phần đã được chuyển trực tuyến, bao gồm hướng dẫn cài đặt, phân tích thành phần chính hạt nhân (PCA), chi tiết toán học của hỗn hợp Gaussian Bayes, TF Agents và các phụ lục cũ A (giải bài tập), C (toán học máy vector hỗ trợ) và E (kiến trúc mạng nơ-ron bổ sung). Xem https://homl.info/changes3 để biết thêm chi tiết.

Các tài nguyên khác

Nhiều tài nguyên tuyệt vời có sẵn để tìm hiểu về học máy. Ví dụ, khóa học ML của Andrew Ng trên Coursera rất tuyệt vời, mặc dù nó đòi hỏi một khoản đầu tư thời gian đáng kể. Cũng có nhiều trang web thú vị về học máy, bao gồm Hướng dẫn sử dụng đặc biệt của Scikit-Learn. Bạn cũng có thể thích Dataquest, cung cấp các hướng dẫn tương tác rất hay và các blog ML như những blog được liệt kê trên Quora. Có nhiều cuốn sách giới thiệu khác về học máy. Đặc biệt:

- **Data Science from Scratch, 2nd edition** (O'Reilly) của Joel Grus, trình bày các nguyên tắc cơ bản của học máy và triển khai một số thuật toán chính bằng Python thuần túy (từ đầu, như tên gọi).
- **Machine Learning: An Algorithmic Perspective, 2nd edition** (Chapman & Hall) của Stephen Marsland, là một giới thiệu tuyệt vời về học máy, bao gồm nhiều chủ đề chuyên sâu với các ví dụ mã bằng Python (cũng từ đầu, nhưng sử dụng NumPy).
- **Python Machine Learning, 3rd edition** (Packt Publishing) của Sebastian Raschka, cũng là một giới thiệu tuyệt vời về học máy và tận dụng các thư viện mã nguồn mở Python (Pylearn 2 và Theano).
- **Deep Learning with Python, 2nd edition** (Manning) của François Chollet, là một cuốn sách rất thực tế bao gồm nhiều chủ đề một cách rõ ràng và súc tích, như bạn có thể mong đợi từ tác giả của thư viện Keras tuyệt vời. Nó ưu tiên các ví dụ mã hơn lý thuyết toán học.
- **The Hundred-Page Machine Learning Book** (tự xuất bản) của Andriy Burkov rất ngắn nhưng bao gồm một loạt các chủ đề ấn tượng, giới thiệu chúng bằng các thuật ngữ dễ tiếp cận mà không né tránh các phương trình toán học.
- **Learning from Data** (AMLBook) của Yaser S. Abu-Mostafa, Malik Magdon-Ismail và Hsuan-Tien Lin là một cách tiếp cận khá lý thuyết về ML cung cấp những hiểu biết sâu sắc, đặc biệt về sự đánh đổi giữa độ chệch/phương sai (xem Chương 4).

- **Artificial Intelligence: A Modern Approach, 4th edition** (Pearson) của Stuart Russell và Peter Norvig, là một cuốn sách tuyệt vời (và đồ sộ) bao gồm một lượng lớn các chủ đề đáng kinh ngạc, bao gồm học máy. Nó giúp đặt ML vào đúng góc nhìn.
- **Deep Learning for Coders with fastai and PyTorch** (O'Reilly) của Jeremy Howard và Sylvain Gugger cung cấp một giới thiệu rõ ràng và thực tế tuyệt vời về học sâu bằng cách sử dụng các thư viện fastai và PyTorch.

Cuối cùng, việc tham gia các trang web thi đấu ML như Kaggle.com sẽ cho phép bạn thực hành các kỹ năng của mình trên các vấn đề trong thế giới thực, với sự giúp đỡ và hiểu biết từ một số chuyên gia ML giỏi nhất hiện có.

Quy ước được sử dụng trong sách này

Các quy ước kiểu chữ sau được sử dụng trong sách này: **Chữ nghiêng** Chỉ ra các thuật ngữ mới, URL, địa chỉ email, tên tệp và phần mở rộng tệp. **Chữ cố định** Được sử dụng cho các danh sách chương trình, cũng như trong các đoạn văn để chỉ các phần tử chương trình như tên biến hoặc hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa. **Chữ cố định in đậm** Hiển thị các lệnh hoặc văn bản khác mà người dùng nên nhập theo nghĩa đen. **Chữ cố định in nghiêng** Hiển thị văn bản mà người dùng nên thay thế bằng các giá trị do người dùng cung cấp hoặc bằng các giá trị được xác định theo ngữ cảnh. **Dấu câu** Để tránh bất kỳ sự nhầm lẫn nào, dấu câu xuất hiện bên ngoài dấu ngoặc kép trong suốt cuốn sách. Thành thật xin lỗi những người theo chủ nghĩa thuần túy.

O'Reilly Online Learning

Mạng lưới các chuyên gia và nhà đổi mới độc đáo của chúng tôi chia sẻ kiến thức và chuyên môn của họ thông qua sách, bài viết và nền tảng học trực tuyến của chúng tôi. Nền tảng học trực tuyến của O'Reilly cung cấp cho bạn quyền truy cập theo yêu cầu vào các khóa đào tạo trực tiếp, các lộ trình học chuyên sâu, môi trường mã hóa tương tác và một bộ sưu tập lớn văn bản và video từ O'Reilly và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, hãy truy cập https://oreilly.com .

Cách liên hệ với chúng tôi

Vui lòng gửi bình luận và câu hỏi liên quan đến cuốn sách này cho nhà xuất bản:

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (tại Hoa Kỳ hoặc Canada) 707-829-0515 (quốc tế hoặc địa phương) 707-829-0104 (fax)

Chúng tôi có một trang web dành cho cuốn sách này, nơi chúng tôi liệt kê các lỗi, ví dụ và bất kỳ thông tin bổ sung nào. Bạn có thể truy cập trang này tại https://homl.info/oreilly3 . Gửi email tới bookquestions@oreilly.com để bình luận hoặc đặt câu hỏi kỹ thuật về cuốn sách này. Để biết tin tức và thông tin về sách và khóa học của chúng tôi, hãy truy cập https://oreilly.com . Tìm chúng tôi trên LinkedIn: https://linkedin.com/company/oreilly-media Theo dõi chúng tôi trên Twitter: https://twitter.com/oreillymedia Xem chúng tôi trên YouTube: https://youtube.com/oreillymedia

Lời cảm ơn

Trong những giấc mơ hoang dại nhất của mình, tôi chưa bao giờ tưởng tượng rằng các phiên bản đầu tiên và thứ hai của cuốn sách này lại có được một lượng độc giả lớn đến vậy. Tôi đã nhận được rất nhiều tin nhắn từ độc giả, nhiều người đặt câu hỏi, một số vui lòng chỉ ra lỗi, và hầu hết gửi cho tôi những lời động viên. Tôi không thể diễn tả lòng biết ơn của mình đối với tất cả những độc giả này vì sự hỗ trợ to lớn của họ. Xin chân thành cảm ơn tất cả các bạn! Xin đừng ngần ngại gửi vấn đề trên GitHub nếu bạn tìm thấy lỗi trong các ví dụ mã (hoặc chỉ để đặt câu hỏi), hoặc gửi lỗi nếu bạn tìm thấy lỗi trong văn bản. Một số độc giả cũng chia sẻ cách cuốn sách này đã giúp họ có được công việc đầu tiên, hoặc cách nó giúp họ giải quyết một vấn đề cụ thể mà họ đang thực hiện. Tôi thấy phản hồi như vậy vô cùng động lực. Nếu bạn thấy cuốn sách này hữu ích, tôi rất muốn bạn chia sẻ câu chuyện của mình với tôi, dù là riêng tư (ví dụ: qua LinkedIn) hay công khai (ví dụ: tweet cho tôi tại @aureliengeron hoặc viết một bài đánh giá trên Amazon).

Xin chân thành cảm ơn tất cả những người tuyệt vời đã dành thời gian và chuyên môn của mình để xem xét phiên bản thứ ba này, sửa lỗi và đưa ra vô số gợi ý. Phiên bản này tốt hơn rất nhiều nhờ họ: Olzhas Akpambetov, George Bonner, François Chollet, Siddha Gangju, Sam Goodman, Matt Harrison, Sasha Sobran, Lewis Tunstall, Leandro von Werra, và người anh trai yêu quý của tôi Sylvain. Tất cả các bạn thật tuyệt vời! Tôi cũng rất biết ơn nhiều người đã hỗ trợ tôi trong suốt quá trình, bằng cách trả lời các câu hỏi của tôi, đề xuất cải tiến và đóng góp mã trên GitHub: đặc biệt là Yannick Assogba, Ian Beauregard, Ulf Bissbort, Rick Chao, Peretz Cohen, Kyle Gallatin, Hannes Hapke, Victor Khaustov, Soonson Kwon, Eric Lebigot, Jason Mayes, Laurence Moroney, Sara Robinson, Joaquín Ruales và Yuefeng Zhou. Cuốn sách này sẽ không tồn tại nếu không có đội ngũ nhân viên tuyệt vời của O'Reilly, đặc biệt là Nicole Taché, người đã cho tôi những phản hồi sâu sắc và luôn vui vẻ, khuyến khích và hữu ích: tôi không thể mơ ước có một biên tập viên nào tốt hơn. Cảm ơn Michele Cronin rất nhiều, người đã cổ vũ tôi qua các chương cuối cùng và giúp tôi vượt qua vạch đích. Cảm ơn toàn bộ đội ngũ sản xuất, đặc biệt là Elizabeth Kelly và Kristen Brown. Cũng xin cảm ơn Kim Cofer vì đã biên tập kỹ lưỡng, và Johnny O'Toole, người đã quản lý mối quan hệ với Amazon và trả lời nhiều câu hỏi của tôi. Cảm ơn Kate Dullea vì đã cải thiện đáng kể các hình minh họa của tôi. Cảm ơn Marie Beaugureau, Ben Lorica, Mike Loukides và Laurel Ruma vì đã tin tưởng vào dự án này và giúp tôi xác định phạm vi của nó. Cảm ơn Matt Hacker và toàn bộ nhóm Atlas vì đã trả lời tất cả các câu hỏi kỹ thuật của tôi liên quan đến định dạng, AsciiDoc, MathML và LaTeX, và cảm ơn Nick Adams, Rebecca Demarest, Rachel Head, Judith McConville, Helen Monroe, Karen Montgomery, Rachel Roumeliotis, và tất cả những người khác tại O'Reilly đã đóng góp vào cuốn sách này. Tôi sẽ không bao giờ quên tất cả những người tuyệt vời đã giúp tôi với các phiên bản đầu tiên và thứ hai của cuốn sách này: bạn bè, đồng nghiệp, chuyên gia, bao gồm nhiều thành viên của nhóm TensorFlow. Danh sách rất dài: Olzhas Akpambetov, Karmel Allison, Martin Andrews, David Andrzejewski, Paige Bailey, Lukas Biewald, Eugene Brevdo, William Chargin, François Chollet, Clément Courbet, Robert Crowe, Mark Daoust, Daniel "Wolff" Dobson, Julien Dubois, Mathias Kende, Daniel Kitachewsky, Nick Felt, Bruce Fontaine, Justin Francis, Goldie Gadde, Irene Giannoumis, Ingrid von Glehn, Vincent Guilbeau,

Sandeep Gupta, Priya Gupta, Kevin Haas, Eddy Hung, Konstantinos Katsiapis, Viacheslav Kovalevskyi, Jon Krohn, Allen Lavoie, Karim Matrah, Grégoire Mesnil, Clemens Mewald, Dan Moldovan, Dominic Monn, Sean Morgan, Tom O'Malley, James Pack, Alexander Pak, Haesun Park, Alexandre Passos, Ankur Patel, Josh Patterson, André Susano Pinto, Anthony Platanios, Anosh Raj, Oscar Ramirez, Anna Revinskaya, Saurabh Saxena, Salim Sémaoune, Ryan Sepassi, Vitor Sessak, Jiri Simsa, Iain Smears, Xiaodan Song, Christina Sorokin, Michel Tessier, Wiktor Tomczak, Dustin Tran, Todd Wang, Pete Warden, Rich Washington, Martin Wicke, Edd Wilder-James, Sam Witteveen, Jason Zaman, Yuefeng Zhou, và anh trai tôi Sylvain. Cuối cùng nhưng không kém phần quan trọng, tôi vô cùng biết ơn người vợ yêu quý của tôi, Emmanuelle, và ba đứa con tuyệt vời của chúng tôi, Alexandre, Rémi và Gabrielle, vì đã khuyến khích tôi làm việc chăm chỉ cho cuốn sách này. Sự tò mò không ngừng của chúng thật vô giá: việc giải thích một số khái niệm khó nhất trong cuốn sách này cho vợ và các con tôi đã giúp tôi làm rõ suy nghĩ của mình và trực tiếp cải thiện nhiều phần của nó. Thêm vào đó, chúng không ngừng mang bánh quy và cà phê cho tôi, ai có thể đòi hỏi gì hơn nữa?

1 Geoffrey E. Hinton và cộng sự, "A Fast Learning Algorithm for Deep Belief Nets", Neural Computation 18 (2006): 1527–1554. 2 Mặc dù mạng nơ-ron tích chập sâu của Yann LeCun đã hoạt động tốt để nhận dạng hình ảnh từ những năm 1990, mặc dù chúng không có mục đích chung.

Mục lục

Lời nói đầu 3
Mã ví dụ 4
Yêu cầu tiên quyết 4
Lộ trình 5
Những thay đổi giữa Phiên bản thứ hai và Phiên bản thứ ba 6
Các tài nguyên khác 7
Quy ước được sử dụng trong sách này 8
O'Reilly Online Learning 8
Cách liên hệ với chúng tôi 8
Lời cảm ơn 9
PHẦN I. NHỮNG KIẾN THỨC CƠ BẢN VỀ HỌC MÁY 26
CHƯƠNG 1. BỨC TRANH TỔNG QUAN VỀ HỌC MÁY 26
Học máy là gì? 26
Tại sao sử dụng học máy? 27
Ví dụ về các ứng dụng 30
Các loại hệ thống học máy 31
Giám sát huấn luyện 31
Học có giám sát 32
Học không giám sát 33
Học bán giám sát 35
Học tự giám sát 36
Học tăng cường 37
Học theo lô so với học trực tuyến 38
Học theo lô 38
Học trực tuyến 39
Học dựa trên thực thể và học dựa trên mô hình 41
Học dựa trên thực thể 41
Học dựa trên mô hình và quy trình làm việc học máy điển hình 42
Những thách thức chính của học máy 47
Số lượng dữ liệu huấn luyện không đủ 47

Dữ liệu huấn luyện không đại diện 48
Dữ liệu chất lượng kém 49
Đặc trưng không liên quan 50
Quá khớp dữ liệu huấn luyện (Overfitting the Training Data) 50
Dưới khớp dữ liệu huấn luyện (Underfitting the Training Data) 52
Kiểm thử và xác thực 53
Điều chỉnh siêu tham số và chọn mô hình 53
Bài tập 59
CHƯƠNG 2. DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI 61
2.1 Làm việc với dữ liệu thực 61
.2.2 Nhìn vào bức tranh lớn 62
2.2.1 Xác định vấn đề 62
2.2.2 Chọn một Thước đo Hiệu suất 64
CÁC KÝ HIỆU 64
2.2.3 Kiểm tra các Giả định 66
2.3 Lấy Dữ liệu 66
Chạy các Ví dụ về Mã nguồn bằng Google Colab 67
2.3.1 Tải Dữ liệu 71
2.3.2 Xem nhanh Cấu trúc Dữ liệu 71
2.3.3 Tạo tập kiểm thử 75
2.4 Tự do khám phá và trực quan hóa dữ liệu để thu được thông tin chi tiết 79
2.4.1 Trực quan hóa dữ liệu địa lý 80
2.4.2 Tìm kiếm các tương quan 82
2.4.3 Thử nghiệm với các kết hợp thuộc tính 85
2.5 Chuẩn bị dữ liệu cho các thuật toán học máy 86
2.5.1 Làm sạch dữ liệu 86
2.5.2 Xử lý các thuộc tính văn bản và phân loại 89
2.5.3 Co giãn và Biến đổi Đặc trưng 92
2.5.4 Bộ biến đổi tùy chỉnh 96
2.5.5 Các pipeline biến đổi 100

2.6 Chọn và Huấn luyện Mô hình 104
2.6.1 Huấn luyện và Đánh giá trên Tập huấn luyện 104
2.6.2 Đánh giá tốt hơn bằng kiểm định chéo 106
2.7 Tinh chỉnh mô hình của bạn 108
2.7.1 Tìm kiếm theo lưới (Grid Search) 108
2.7.2 Tìm kiếm ngẫu nhiên 110
2.7.3 Các phương pháp tập hợp (Ensemble Methods) 111
2.7.4 Phân tích các mô hình tốt nhất và lỗi của chúng 111
2.7.5 Đánh giá hệ thống của bạn trên tập kiểm thử 112
2.8 Triển khai, Giám sát và Bảo trì hệ thống của bạn 113
2.9 Bài tập 116
Bài tập 118
CHƯƠNG 3. PHÂN LOẠI 120
3.1 MNIST 120
3.2 Huấn luyện bộ phân loại nhị phân 123
3.3 Các thước đo hiệu suất 123
3.3.1 Đo độ chính xác bằng kiểm định chéo 123
3.3.2 Ma trận nhầm lẫn 125
3.3.3 Độ chính xác và độ nhạy 127
3.3.4 Sự đánh đổi Độ chính xác/Độ nhạy 128
3.3.5 Đường cong ROC 131
3.4 Phân loại đa lớp 134
3.5 Phân tích lỗi 137
3.6 Phân loại đa nhãn 141
3.7 Phân loại đa đầu ra 143
3.8 Bài tập 145
CHƯƠNG 4. HUẤN LUYỆN MÔ HÌNH 147
4.1 Hồi quy tuyến tính 148
4.1.1 Phương trình chuẩn tắc 149
4.1.2 Độ phức tạp tính toán 152
4.2 Gradient Descent 152

4.2.1 Gradient Descent theo lô 155
4.2.2 Gradient Descent ngẫu nhiên 157
4.2.3 Gradient Descent theo Mini-Batch 160
4.3 Hồi quy đa thức 162
4.4 Đường cong học tập 163
4.5 Mô hình tuyến tính được chính quy hóa 167
4.5.1 Hồi quy Ridge 167
4.5.2 Hồi quy Lasso 169
Hồi quy Elastic Net 172
4.5.4 Dừng sớm (Early Stopping) 172
4.6 Hồi quy Logistic 174
4.6.1 Ước tính xác suất 174
Hàm huấn luyện và Hàm chi phí 175
Đạo hàm riêng của hàm chi phí 176
4.6.3 Các đường ranh giới quyết định 176
Hồi quy Softmax 179
4.8 Bài tập 182
CHƯƠNG 5. MÁY HỌC VÉC-TƠ HỖ TRỢ 185
5.1. Phân loại SVM tuyến tính 185
5.1.1 Phân loại biên mềm 186
Phân loại SVM phi tuyến 188
Kernel đa thức 189
Các đặc trưng tương tự 190
Kernel Gaussian RBF 191
Các lớp SVM và độ phức tạp tính toán 192
Hồi quy SVM 192
Bên trong các bộ phân loại SVM tuyến tính 194
Bài toán đối ngẫu 196
SVM sử dụng Kernel (Kernelized SVMs) 197
Bài tập 199
CHƯƠNG 6. CÂY QUYẾT ĐỊNH 201

Huấn luyện và trực quan hóa cây quyết định 201
Đưa ra dự đoán 202
Ước tính xác suất lớp 204
Thuật toán huấn luyện CART 204
Độ phức tạp tính toán 205
Độ không tinh khiết Gini hay Entropy? 205
Siêu tham số chính quy hóa 205
Hồi quy 207
Độ nhạy với hướng trục 209
Cây quyết định có phương sai cao 210
Bài tập 211
CHƯƠNG 7. HỌC TỔ HỢP VÀ RỪNG NGẪU NHIÊN 213
Bộ phân loại biểu quyết 213
Túi hóa (Bagging) và Dán nhãn (Pasting) 217
Túi hóa và Dán nhãn trong Scikit-Learn 218
Đánh giá ngoài mẫu (Out-of-Bag Evaluation) 219
Random Patches và Random Subspaces 220
Rừng ngẫu nhiên 220
Cây ngẫu nhiên cực đại (Extra-Trees) 221
Tầm quan trọng của đặc trưng 221
Boosting 222
AdaBoost 223
Gradient Boosting 225
Tăng cường Gradient dựa trên Biểu đồ (Histogram-Based Gradient Boosting) 229
Phân lớp xếp chồng (Stacking) 230
Bài tập 233
CHƯƠNG 8. GIẢM CHIỀU DỮ LIỆU 235
Lời nguyền của số chiều 235
Các cách tiếp cận chính để giảm chiều 237
Phép chiếu 237
Học đa tạp (Manifold Learning) 238
PCA 240

Bảo toàn phương sai 240
Các thành phần chính (Principal Components) 241
Chiếu xuống d chiều 242
Sử dụng Scikit-Learn 242
Tỷ lệ phương sai giải thích (Explained Variance Ratio) 243
Chọn số chiều phù hợp 243
PCA để nén 245
PCA ngẫu nhiên (Randomized PCA) 246
PCA tăng dần (Incremental PCA) 246
Phép chiếu ngẫu nhiên (Random Projection) 247
Nhúng tuyến tính cục bộ 249
Bước 1: Mô hình hóa tuyến tính các mối quan hệ cục bộ 250
Bước 2: Giảm chiều dữ liệu trong khi vẫn giữ các mối quan hệ 251
Các kỹ thuật giảm chiều khác 251
Bài tập 252
CHƯƠNG 9. CÁC KỸ THUẬT HỌC KHÔNG GIÁM SÁT 254
Các thuật toán phân cụm: k-means và DBSCAN 255
k-means 257
Thuật toán k-means 259
Sử dụng phân cụm cho phân đoạn hình ảnh 267
Sử dụng phân cụm cho học bán giám sát 269
DBSCAN 272
Các thuật toán phân cụm khác 275
Hỗn hợp Gaussian (Gaussian Mixtures) 276
Sử dụng Hỗn hợp Gaussian để phát hiện dị thường 280
Chọn số lượng cụm 281
Mô hình hỗn hợp Gaussian Bayes (Bayesian Gaussian Mixture Models) 284
Các thuật toán khác để phát hiện dị thường và điểm mới 285
Bài tập 286
PHẦN II. MẠNG NƠ-RON VÀ HỌC SÂU 288
CHƯƠNG 10. GIỚI THIỆU VỀ MẠNG NƠ-RON NHÂN TẠO VỚI KERAS 288

Từ Nơ-ron Sinh học đến Nơ-ron Nhân tạo 288
Nơ-ron Sinh học 289
Các phép tính logic với nơ-ron 291
Perceptron 292
Mạng perceptron đa lớp và Backpropagation 296
MLP hồi quy 299
MLP phân loại 301
Triển khai MLP với Keras 302
Xây dựng bộ phân loại hình ảnh bằng API tuần tự 303
Xây dựng mô hình phức tạp bằng API chức năng 313
Sử dụng Subclassing API để xây dựng mô hình động 319
Lưu và khôi phục mô hình 320
Sử dụng Callbacks 321
Sử dụng TensorBoard để trực quan hóa 322
Tinh chỉnh các siêu tham số của mạng nơ-ron 326
Số lượng lớp ẩn 331
Số lượng nơ-ron trên mỗi lớp ẩn 332
Tốc độ học, kích thước Batch và các siêu tham số khác 332
Bài tập 334
CHƯƠNG 11. HUẤN LUYỆN MẠNG NƠ-RON SÂU 336
Vấn đề Gradient biến mất/bùng nổ 336
Khởi tạo Glorot và He 337
Các hàm kích hoạt tốt hơn 339
Chuẩn hóa theo Batch 344
Cắt Gradient (Gradient Clipping) 349
Tái sử dụng các lớp đã được huấn luyện trước 349
Học chuyển giao với Keras 351
Tiền huấn luyện không giám sát 352
Tiền huấn luyện (Pretraining) trên một Tác vụ Phụ trợ 354
Các Trình tối ưu hóa Nhanh hơn 355
Momentum (Động lượng) 355

Nesterov Accelerated Gradient (NAG) 356
AdaGrad 357
RMSProp 358
Adam 359
Lập lịch tốc độ học (Learning Rate Scheduling) 360
Tránh Overfitting thông qua chính quy hóa 365
Chính quy hóa $l1$ và $l2$ 365
Dropout 366
Monte Carlo (MC) Dropout 368
Chính quy hóa Max-Norm 370
Tóm tắt và hướng dẫn thực hành 371
Bài tập 372
CHƯƠNG 12. MÔ HÌNH TÙY CHỈNH VÀ HUẤN LUYỆN VỚI TENSORFLOW 374
Giới thiệu nhanh về TensorFlow 374
Sử dụng TensorFlow như NumPy 377
Tensor và các phép toán 377
Tensor và NumPy 378
Chuyển đổi kiểu dữ liệu 379
Biến (Variables) 380
Các cấu trúc dữ liệu khác 380
Tùy chỉnh mô hình và thuật toán huấn luyện 381
Các hàm mất mát tùy chỉnh 381
Lưu và tải mô hình chứa các thành phần tùy chỉnh 382
Các hàm kích hoạt, trình khởi tạo, trình chính quy hóa và ràng buộc tùy chỉnh 384
Các lớp tùy chỉnh 388
Mô hình tùy chỉnh 390
Các hàm mất mát và số liệu dựa trên các chi tiết nội bộ của mô hình 392
Tính toán Gradient bằng Autodiff 394
Vòng lặp huấn luyện tùy chỉnh 398
Hàm và biểu đồ TensorFlow 401

AutoGraph và Tracing 403
Quy tắc hàm TF 404
Bài tập 406
CHƯƠNG 13. TẢI VÀ TIỀN XỬ LÝ DỮ LIỆU VỚI TENSORFLOW 408
API `tf.data` 409
Xếp chuỗi các phép biến đổi 410
Xáo trộn dữ liệu (Shuffling the Data) 412
Xen kẽ các dòng từ nhiều tệp (Interleaving Lines from Multiple Files) 413
Tiền xử lý dữ liệu (Preprocessing the Data) 415
Prefetching 417
Sử dụng Dataset với Keras 419
Định dạng TFRecord 420
Tệp TFRecord được nén 421
Giới thiệu ngắn gọn về Protocol Buffers 421
Các Protobuf của TensorFlow 422
Tải và Phân tích cú pháp các Example 423
Xử lý Danh sách của các Danh sách bằng SequenceExample Protobuf 424
Các Lớp Tiền xử lý của Keras 424
Lớp Normalization 424
Lớp Discretization 426
Lớp CategoryEncoding 426
Lớp StringLookup 426
Lớp Hashing 427
Mã hóa đặc trưng phân loại bằng Embedding 427
Tiền xử lý văn bản 431
Sử dụng các thành phần mô hình ngôn ngữ được huấn luyện trước 433
Các lớp tiền xử lý hình ảnh 434
Dự án TensorFlow Datasets 435
Bài tập 436
CHƯƠNG 14. THỊ GIÁC MÁY TÍNH CHUYÊN SÂU SỬ DỤNG MẠNG NƠ-RON TÍCH CHẬP 438

Kiến trúc của vỏ não thị giác 438
Các lớp tích chập 439
Bộ lọc 441
Triển khai các lớp tích chập với Keras 443
Yêu cầu về bộ nhớ 447
Các lớp gộp 447
Triển khai các lớp gộp với Keras 449
Kiến trúc CNN 451
LeNet-5 453
AlexNet 454
GoogLeNet 456
VGGNet 459
ResNet 460
Xception 463
SENet 464
Các kiến trúc đáng chú ý khác 466
ResNeXt 466
DenseNet 466
MobileNet 466
CSPNet 467
EfficientNet 467
Chọn kiến trúc CNN phù hợp 467
Triển khai một CNN ResNet-34 bằng Keras 468
Sử dụng các mô hình đã được huấn luyện sẵn từ Keras 470
Học chuyển đổi với các mô hình đã được huấn luyện sẵn 471
Phân loại và định vị 473
Phát hiện đối tượng 475
Mạng hoàn toàn tích chập 477
You Only Look Once 479
Theo dõi đối tượng 482

Phân đoạn ngữ nghĩa 482
Bài tập 485
CHƯƠNG 15. XỬ LÝ CHUỖI BẰNG RNN VÀ CNN 487
15.1 Các nơ-ron và lớp hồi quy 487
Các ô bộ nhớ 489
Các chuỗi đầu vào và đầu ra 490
15.2 Huấn luyện RNN 491
15.3 Dự báo chuỗi thời gian 492
Các mô hình họ ARMA 497
Chuẩn bị dữ liệu cho các mô hình học máy 500
Dự báo bằng mô hình tuyến tính 503
Dự báo bằng RNN đơn giản 504
Dự báo bằng RNN sâu 505
Dự báo chuỗi thời gian đa biến 507
Dự báo nhiều bước thời gian phía trước 508
Dự báo bằng mô hình Sequence-to-Sequence 514
Xử lý các chuỗi dài 516
Giải quyết vấn đề gradient không ổn định 516
Nghĩ lại vấn đề bộ nhớ ngắn hạn 519
WaveNet 525
Bài tập 529
CHƯƠNG 16. XỬ LÝ NGÔN NGỮ TỰ NHIÊN VỚI RNN VÀ CƠ CHẾ CHÚ Ý 532
Tạo văn bản kiểu Shakespeare bằng cách sử dụng RNN ký tự 533
Tạo tập dữ liệu huấn luyện 533
Xây dựng và huấn luyện mô hình Char-RNN 535
Tạo văn bản Shakespeare giả 537
RNN có trạng thái 538
Phân tích cảm xúc 540
Mặt nạ (Masking) 543
Tái sử dụng Embedding và Mô hình ngôn ngữ đã được huấn luyện trước 546

Mạng mã hóa-giải mã cho dịch máy thần kinh 547
RNN hai chiều 553
Tìm kiếm chùm (Beam Search) 554
Cơ chế chú ý (Attention Mechanisms) 555
Attention Is All You Need: Kiến trúc Transformer gốc 559
Mã hóa vị trí 561
Chú ý đa đầu (Multi-head attention) 563
Tuyết lở các mô hình Transformer 568
Vision Transformers 572
Thư viện Transformers của Hugging Face 575
Bài tập 579
CHƯƠNG 17. AUTOENCODER, MẠNG ĐỐI KHÁNG SINH (GAN) VÀ MÔ HÌNH KHUẾCH TÁN 583
Biểu diễn dữ liệu hiệu quả 584
Thực hiện PCA bằng Autoencoder tuyến tính thiếu hoàn chỉnh 586
Autoencoder xếp chồng (Stacked Autoencoders) 587
Triển khai Autoencoder xếp chồng bằng Keras 588
Trực quan hóa các bản tái tạo 589
Trực quan hóa tập dữ liệu Fashion MNIST 590
Tiền huấn luyện không giám sát bằng Autoencoder xếp chồng 591
Ràng buộc trọng số (Tying Weights) 592
Huấn luyện từng Autoencoder một 594
Autoencoder tích chập (Convolutional Autoencoders) 595
Autoencoder khử nhiễu (Denoising Autoencoders) 596
Autoencoder thưa (Sparse Autoencoders) 598
Autoencoder biến phân (Variational Autoencoders - VAEs) 601
Tạo ảnh Fashion MNIST 605
Mạng đối kháng sinh (Generative Adversarial Networks - GANs) 606
Những khó khăn khi huấn luyện GAN 611
GAN tích chập sâu (Deep Convolutional GANs - DCGANs) 612
Sự phát triển tăng dần của GANs (Progressive Growing of GANs) 615

StyleGANs ... 617
Mô hình khuếch tán (Diffusion Models) ... 620
Bài tập ... 628
CHƯƠNG 18. HỌC TĂNG CƯỜNG ... 630
Học để tối ưu hóa phần thưởng ... 630
Tìm kiếm chính sách (Policy Search) ... 632
Giới thiệu về OpenAI Gym ... 633
Các chính sách mạng nơ-ron ... 636
Đánh giá hành động: Bài toán phân bổ tín nhiệm ... 638
Độ dốc chính sách (Policy Gradients) ... 639
Các quá trình quyết định Markov (Markov Decision Processes) ... 644
Học chênh lệch thời gian (Temporal Difference Learning) ... 648
Học Q (Q-Learning) ... 649
Các chính sách khám phá (Exploration Policies) ... 651
Học Q xấp xỉ và Học Q sâu (Approximate Q-Learning and Deep Q-Learning) ... 651
Triển khai học Q sâu ... 653
Các biến thể của học Q sâu (Deep Q-Learning Variants) ... 657
Tổng quan về một số thuật toán RL phổ biến ... 660
Bài tập ... 662
CHƯƠNG 19. HUẤN LUYỆN VÀ TRIỂN KHAI MÔ HÌNH TENSORFLOW QUY MÔ LỚN ... 664
Phục vụ một mô hình TensorFlow ... 665
Sử dụng TensorFlow Serving ... 665
Tạo dịch vụ dự đoán trên Vertex AI ... 674
Chạy các Công việc Dự đoán Hàng loạt trên Vertex AI ... 681
Triển khai Mô hình lên Thiết bị Di động hoặc Thiết bị Nhúng ... 684
Chạy Mô hình trong Trang Web ... 688
Sử dụng GPU để Tăng tốc Tính toán ... 689
Có được GPU của riêng bạn ... 690
Quản lý RAM GPU ... 692
Đặt các Phép toán và Biến trên các Thiết bị ... 695
Thực thi song song trên nhiều thiết bị ... 696

Huấn luyện mô hình trên nhiều thiết bị 698
Mô hình song song (Model Parallelism) 699
Song song Dữ liệu (Data Parallelism) 701
Huấn luyện Quy mô lớn bằng API Chiến lược Phân phối (Distribution Strategies API) 708
Huấn luyện Mô hình trên Cụm TensorFlow 710
Chạy các Tác vụ Huấn luyện Lớn trên Vertex AI 714
Điều chỉnh Siêu Tham số trên Vertex AI 717
Bài tập 721
PHỤ LỤC A. DANH SÁCH KIỂM TRA DỰ ÁN HỌC MÁY 724
Xác định Vấn đề và Nhìn vào Bức tranh lớn 724
Lấy Dữ liệu 724
Khám phá Dữ liệu 725
Chuẩn bị Dữ liệu 725
Lập danh sách rút gọn các Mô hình Tiềm năng 726
Tinh chỉnh Hệ thống 727
Trình bày Giải pháp của bạn 727
Khởi chạy! 727
PHỤ LỤC B. TỰ ĐỘNG VI PHÂN (AUTODIFF) 730
Vi Phân Thủ Công (Manual Differentiation) 730
Xấp Xỉ Sai Phân Hữu Hạn (Finite Difference Approximation) 730
Autodiff Chế Độ Xuôi (Forward-Mode Autodiff) 732
Autodiff Chế Độ Ngược (Reverse-Mode Autodiff) 734
PHỤ LỤC C. CÁC CẤU TRÚC DỮ LIỆU ĐẶC BIỆT 737
Chuỗi (Strings) 737
Tensor Răng Cưa (Ragged Tensors) 738
Tensor Thưa (Sparse Tensors) 739
Mảng Tensor (Tensor Arrays) 740
Tập hợp (Sets) 740
Hàng đợi (Queues) 741
Phụ lục D. Đồ thị TensorFlow 745
Hàm TF và Hàm Cụ thể (Concrete Functions) 745
Khám phá Định nghĩa Hàm và Đồ thị 746

Xem xét kỹ hơn về Tracing 748
Sử dụng AutoGraph để Ghi lại Luồng Điều khiển (Control Flow) 750
Xử lý Biến và các Tài nguyên Khác trong Hàm TF 751
Sử dụng Hàm TF với Keras (hoặc không)..... 752

PHẦN I. NHỮNG KIẾN THỨC CƠ BẢN VỀ HỌC MÁY

CHƯƠNG 1. BỨC TRANH TỔNG QUAN VỀ HỌC MÁY

Cách đây không lâu, nếu bạn nhấc điện thoại lên và hỏi đường về nhà, nó sẽ phớt lờ bạn — và mọi người sẽ nghi ngờ sự tỉnh táo của bạn. Nhưng học máy không còn là khoa học viễn tưởng nữa: hàng tỷ người sử dụng nó mỗi ngày. Và sự thật là nó đã thực sự tồn tại trong nhiều thập kỷ ở một số ứng dụng chuyên biệt, chẳng hạn như nhận dạng ký tự quang học (OCR). Ứng dụng ML đầu tiên thực sự trở thành xu hướng chủ đạo, cải thiện cuộc sống của hàng trăm triệu người, đã chiếm lĩnh thế giới vào những năm 1990: bộ lọc thư rác. Nó không hoàn toàn là một robot có ý thức, nhưng về mặt kỹ thuật, nó đủ điều kiện là học máy: nó đã học tốt đến mức bạn hiếm khi cần đánh dấu một email là thư rác nữa. Sau đó là hàng trăm ứng dụng ML hiện đang lặng lẽ cung cấp sức mạnh cho hàng trăm sản phẩm và tính năng mà bạn sử dụng thường xuyên: lời nhắc bằng giọng nói, dịch tự động, tìm kiếm hình ảnh, đề xuất sản phẩm, và nhiều hơn nữa.

Học máy bắt đầu từ đâu và kết thúc ở đâu? Chính xác thì một cỗ máy học hỏi điều gì? Nếu tôi tải một bản sao của tất cả các bài viết trên Wikipedia, máy tính của tôi có thực sự học được điều gì không? Nó có đột nhiên thông minh hơn không? Trong chương này, tôi sẽ bắt đầu bằng cách làm rõ học máy là gì và tại sao bạn có thể muốn sử dụng nó. Sau đó, trước khi chúng ta bắt đầu khám phá lục địa học máy, chúng ta sẽ xem xét bản đồ và tìm hiểu về các khu vực chính và các địa danh đáng chú ý nhất: học có giám sát so với học không giám sát và các biến thể của chúng, học trực tuyến so với học theo lô, học dựa trên thể hiện so với học dựa trên mô hình. Sau đó, chúng ta sẽ xem xét quy trình làm việc của một dự án ML điển hình, thảo luận về những thách thức chính mà bạn có thể gặp phải và đề cập đến cách đánh giá và tinh chỉnh một hệ thống học máy.

Chương này giới thiệu rất nhiều khái niệm cơ bản (và thuật ngữ) mà mọi nhà khoa học dữ liệu nên thuộc lòng. Đây sẽ là một cái nhìn tổng quan cấp cao (đây là chương duy nhất không có nhiều mã), tất cả khá đơn giản, nhưng mục tiêu của tôi là đảm bảo mọi thứ rõ ràng đối với bạn trước khi chúng ta tiếp tục các phần còn lại của cuốn sách. Vậy thì hãy pha một ly cà phê và bắt đầu thôi!

Học máy là gì?

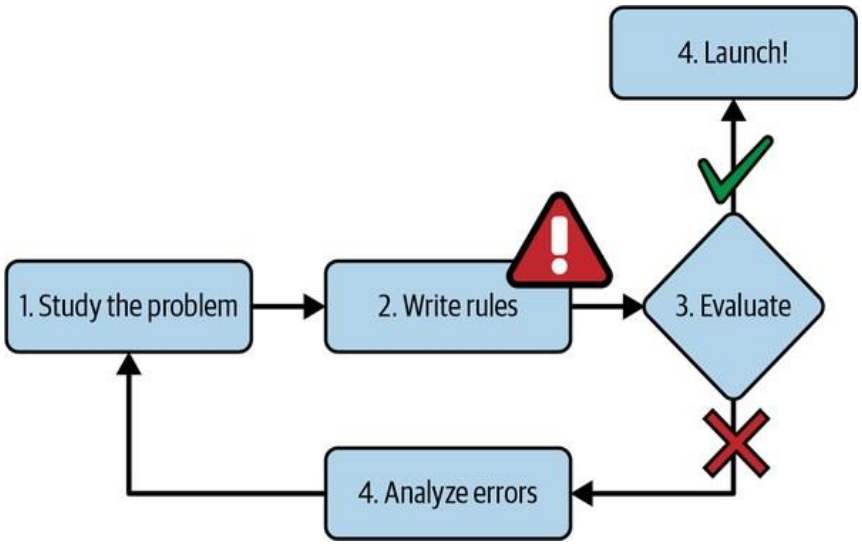
Học máy là khoa học (và nghệ thuật) lập trình máy tính để chúng có thể học từ dữ liệu. Đây là một định nghĩa tổng quát hơn một chút: [Học máy là] lĩnh vực nghiên cứu cung cấp cho máy tính khả năng học hỏi mà không cần được lập trình rõ ràng. —Arthur Samuel, 1959 Và một định nghĩa hướng đến kỹ thuật hơn: Một chương trình máy tính được cho là học từ kinh nghiệm E đối với một nhiệm vụ T và một thước đo hiệu suất P, nếu hiệu suất của nó trên T, được đo bằng P, cải thiện theo kinh nghiệm E. —Tom Mitchell, 1997 Bộ lọc thư rác của bạn là một chương trình học máy mà, với các ví dụ về email rác (được người dùng đánh dấu) và các ví dụ về email thông thường (không phải thư rác, còn được gọi là “ham”), có thể học

cách đánh dấu thư rác. Các ví dụ mà hệ thống sử dụng để học được gọi là tập huấn luyện. Mỗi ví dụ huấn luyện được gọi là một trường hợp huấn luyện (hoặc mẫu). Phần của một hệ thống học máy học và đưa ra dự đoán được gọi là một mô hình. Mạng thần kinh và rừng ngẫu nhiên là các ví dụ về mô hình. Trong trường hợp này, nhiệm vụ T là đánh dấu thư rác cho các email mới, kinh nghiệm E là dữ liệu huấn luyện, và thước đo hiệu suất P cần được xác định; ví dụ, bạn có thể sử dụng tỷ lệ email được phân loại đúng. Thước đo hiệu suất cụ thể này được gọi là độ chính xác (accuracy), và nó thường được sử dụng trong các nhiệm vụ phân loại. Nếu bạn chỉ tải một bản sao của tất cả các bài viết trên Wikipedia, máy tính của bạn có nhiều dữ liệu hơn, nhưng nó không đột nhiên giỏi hơn trong bất kỳ nhiệm vụ nào. Đây không phải là học máy.

Tại sao sử dụng học máy?

Hãy xem xét cách bạn sẽ viết một bộ lọc thư rác bằng các kỹ thuật lập trình truyền thống (Hình 1-1):

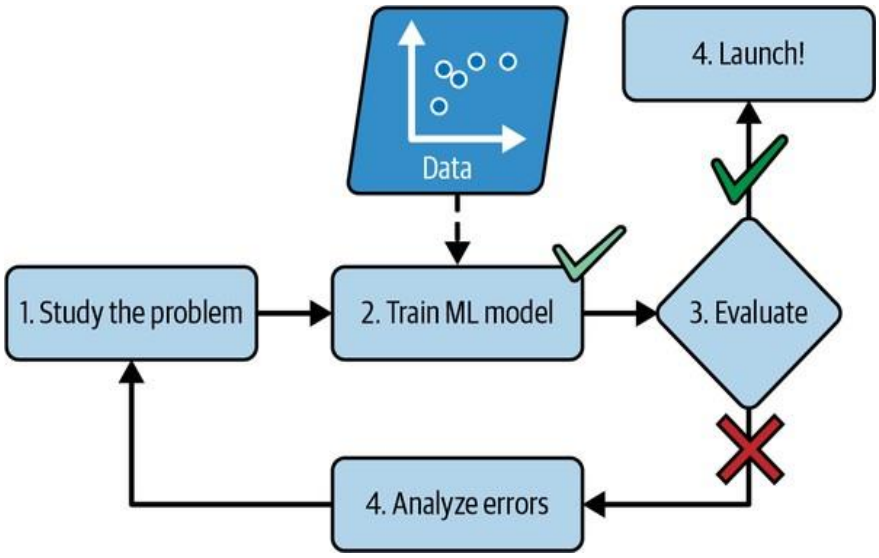
- Đầu tiên, bạn sẽ kiểm tra xem thư rác thường trông như thế nào. Bạn có thể nhận thấy rằng một số từ hoặc cụm từ (như “4U”, “credit card”, “free” và “amazing”) có xu hướng xuất hiện nhiều trong dòng chủ đề. Có lẽ bạn cũng sẽ nhận thấy một vài mẫu khác trong tên người gửi, nội dung email và các phần khác của email.
- Bạn sẽ viết một thuật toán phát hiện cho mỗi mẫu mà bạn nhận thấy, và chương trình của bạn sẽ đánh dấu email là thư rác nếu một số mẫu này được phát hiện.
- Bạn sẽ kiểm tra chương trình của mình và lặp lại các bước 1 và 2 cho đến khi nó đủ tốt để ra mắt.



Hình 1-1. Phương pháp truyền thống

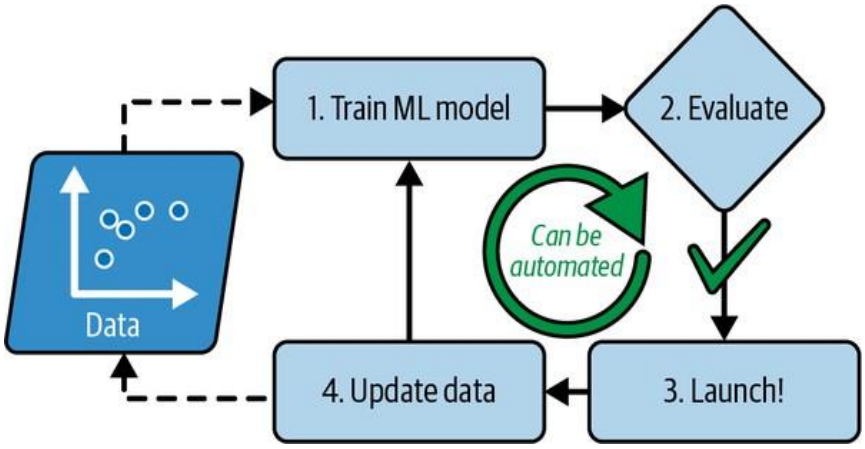
Vì vấn đề rất khó, chương trình của bạn có thể sẽ trở thành một danh sách dài các quy tắc phức tạp — khá khó bảo trì. Ngược lại, một bộ lọc thư rác dựa trên các kỹ thuật học máy tự động học các từ và cụm từ nào là những yếu tố dự báo tốt về thư rác bằng cách phát hiện

các mẫu từ có tần suất bất thường trong các ví dụ thư rác so với các ví dụ thư hợp lệ (Hình 1-2). Chương trình ngắn hơn nhiều, dễ bảo trì hơn và rất có thể chính xác hơn.



Hình 1-2. Phương pháp học máy

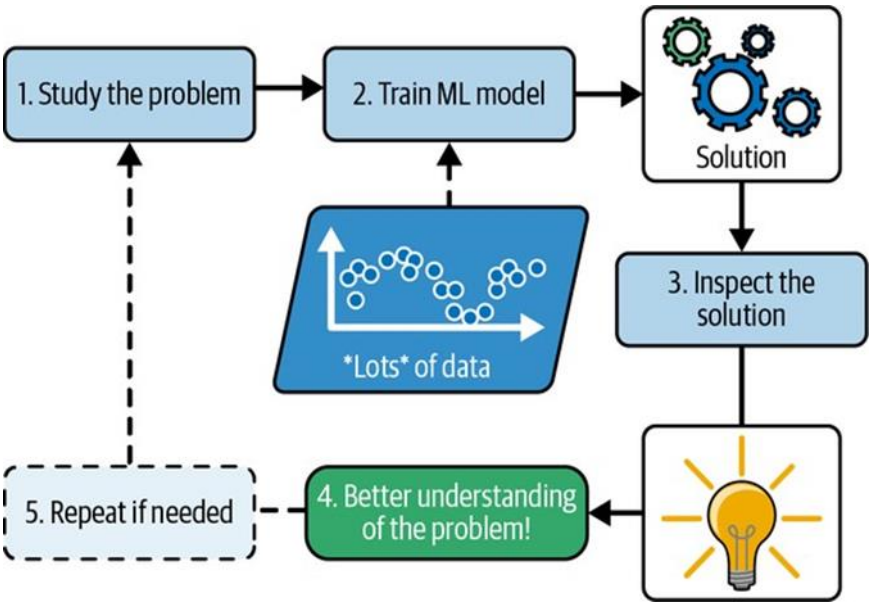
Điều gì sẽ xảy ra nếu những kẻ gửi thư rác nhận thấy rằng tất cả các email của chúng chứa "4U" đều bị chặn? Chúng có thể bắt đầu viết "For U" thay vào đó. Một bộ lọc thư rác sử dụng các kỹ thuật lập trình truyền thống sẽ cần được cập nhật để đánh dấu các email "For U". Nếu những kẻ gửi thư rác tiếp tục vượt qua bộ lọc thư rác của bạn, bạn sẽ cần phải tiếp tục viết các quy tắc mới mãi mãi. Ngược lại, một bộ lọc thư rác dựa trên các kỹ thuật học máy tự động nhận thấy rằng "For U" đã trở nên thường xuyên một cách bất thường trong thư rác được người dùng đánh dấu, và nó bắt đầu đánh dấu chúng mà không cần sự can thiệp của bạn (Hình 1-3).



Hình 1-3. Tự động thích nghi với sự thay đổi

Một lĩnh vực khác mà học máy tỏa sáng là đối với các vấn đề quá phức tạp đối với các phương pháp truyền thống hoặc không có thuật toán nào được biết đến. Ví dụ, hãy xem xét nhận dạng giọng nói. Giả sử bạn muốn bắt đầu đơn giản và viết một chương trình có khả

năng phân biệt các từ “một” và “hai”. Bạn có thể nhận thấy rằng từ “hai” bắt đầu bằng âm thanh cao (“T”), vì vậy bạn có thể mã hóa cứng một thuật toán đo cường độ âm thanh cao và sử dụng nó để phân biệt một và hai — nhưng rõ ràng kỹ thuật này sẽ không mở rộng cho hàng nghìn từ được nói bởi hàng triệu người rất khác nhau trong môi trường ồn ào và trong hàng chục ngôn ngữ. Giải pháp tốt nhất (ít nhất là ngày nay) là viết một thuật toán tự học, với nhiều ví dụ ghi âm cho mỗi từ. Cuối cùng, học máy có thể giúp con người học hỏi (Hình 1-4). Các mô hình ML có thể được kiểm tra để xem chúng đã học được gì (mặc dù đối với một số mô hình điều này có thể khó). Chẳng hạn, một khi bộ lọc thư rác đã được huấn luyện trên đủ thư rác, nó có thể dễ dàng được kiểm tra để tiết lộ danh sách các từ và sự kết hợp các từ mà nó tin là những yếu tố dự báo tốt nhất về thư rác. Đôi khi điều này sẽ tiết lộ những mối tương quan không ngờ hoặc những xu hướng mới, và từ đó dẫn đến sự hiểu biết tốt hơn về vấn đề. Việc đào sâu vào lượng lớn dữ liệu để khám phá các mẫu ẩn được gọi là khai phá dữ liệu, và học máy rất xuất sắc trong lĩnh vực này.



Hình 1-4. Học máy có thể giúp con người học hỏi

Tóm lại, học máy rất tuyệt vời cho:

- Các vấn đề mà các giải pháp hiện có yêu cầu rất nhiều tinh chỉnh hoặc danh sách dài các quy tắc (một mô hình học máy thường có thể đơn giản hóa mã và thực hiện tốt hơn phương pháp truyền thống).
- Các vấn đề phức tạp mà việc sử dụng phương pháp truyền thống không mang lại giải pháp tốt (các kỹ thuật học máy tốt nhất có thể tìm ra giải pháp).
- Môi trường biến động (một hệ thống học máy có thể dễ dàng được huấn luyện lại trên dữ liệu mới, luôn cập nhật).
- Thu được những hiểu biết sâu sắc về các vấn đề phức tạp và lượng lớn dữ liệu.

Ví dụ về các ứng dụng

Hãy xem xét một số ví dụ cụ thể về các tác vụ học máy, cùng với các kỹ thuật có thể xử lý chúng:

- **Phân tích hình ảnh sản phẩm trên dây chuyền sản xuất để tự động phân loại chúng:** Đây là phân loại hình ảnh, thường được thực hiện bằng cách sử dụng mạng nơ-ron tích chập (CNNs; xem Chương 14) hoặc đôi khi là transformer (xem Chương 16).
- **Phát hiện khối u trong ảnh quét não:** Đây là phân đoạn hình ảnh ngữ nghĩa, trong đó mỗi pixel trong ảnh được phân loại (vì chúng ta muốn xác định vị trí và hình dạng chính xác của khối u), thường sử dụng CNNs hoặc transformer.
- **Tự động phân loại các bài báo tin tức:** Đây là xử lý ngôn ngữ tự nhiên (NLP), và cụ thể hơn là phân loại văn bản, có thể được xử lý bằng cách sử dụng mạng nơ-ron hồi quy (RNNs) và CNNs, nhưng transformer hoạt động tốt hơn nữa (xem Chương 16).
- **Tự động gắn cờ các bình luận xúc phạm trên diễn đàn thảo luận:** Đây cũng là phân loại văn bản, sử dụng cùng các công cụ NLP.
- **Tự động tóm tắt các tài liệu dài:** Đây là một nhánh của NLP được gọi là tóm tắt văn bản, cũng sử dụng các công cụ tương tự.
- **Tạo chatbot hoặc trợ lý cá nhân:** Việc này liên quan đến nhiều thành phần NLP, bao gồm hiểu ngôn ngữ tự nhiên (NLU) và các mô-đun trả lời câu hỏi.
- **Dự báo doanh thu của công ty bạn vào năm tới, dựa trên nhiều chỉ số hiệu suất:** Đây là một tác vụ hồi quy (tức là dự đoán giá trị) có thể được giải quyết bằng bất kỳ mô hình hồi quy nào, chẳng hạn như hồi quy tuyến tính hoặc hồi quy đa thức (xem Chương 4), máy vector hỗ trợ hồi quy (xem Chương 5), rừng ngẫu nhiên hồi quy (xem Chương 7) hoặc mạng nơ-ron nhân tạo (xem Chương 10). Nếu bạn muốn tính đến các chuỗi chỉ số hiệu suất trong quá khứ, bạn có thể muốn sử dụng RNNs, CNNs hoặc transformer (xem Chương 15 và 16).
- **Làm cho ứng dụng của bạn phản ứng với lệnh thoại:** Đây là nhận dạng giọng nói, yêu cầu xử lý các mẫu âm thanh: vì chúng là các chuỗi dài và phức tạp, chúng thường được xử lý bằng RNNs, CNNs hoặc transformer (xem Chương 15 và 16).
- **Phát hiện gian lận thẻ tín dụng:** Đây là phát hiện bất thường, có thể được xử lý bằng cách sử dụng rừng cô lập (isolation forests), mô hình hỗn hợp Gaussian (xem Chương 9) hoặc bộ mã hóa tự động (autoencoders) (xem Chương 17).
- **Phân khúc khách hàng dựa trên giao dịch mua hàng của họ để bạn có thể thiết kế một chiến lược tiếp thị khác nhau cho mỗi phân khúc:** Đây là phân cụm, có thể đạt được bằng cách sử dụng k-means, DBSCAN và nhiều phương pháp khác (xem Chương 9).
- **Biểu diễn một tập dữ liệu phức tạp, nhiều chiều trong một biểu đồ rõ ràng và sâu sắc:** Đây là trực quan hóa dữ liệu, thường liên quan đến các kỹ thuật giảm chiều (xem Chương 8).

- **Đề xuất một sản phẩm mà khách hàng có thể quan tâm, dựa trên các giao dịch mua hàng trước đây:** Đây là một hệ thống đề xuất. Một cách tiếp cận là đưa các giao dịch mua hàng trước đây (và các thông tin khác về khách hàng) vào một mạng nơ-ron nhân tạo (xem Chương 10), và để nó xuất ra sản phẩm tiếp theo có khả năng nhất. Mạng nơ-ron này thường được huấn luyện trên các chuỗi mua hàng trong quá khứ của tất cả các khách hàng.
- **Xây dựng một bot thông minh cho trò chơi:** Điều này thường được giải quyết bằng cách sử dụng học tăng cường (RL; xem Chương 18), đây là một nhánh của học máy huấn luyện các tác nhân (như bot) để chọn các hành động sẽ tối đa hóa phần thưởng của chúng theo thời gian (ví dụ: một bot có thể nhận được phần thưởng mỗi khi người chơi mất một số điểm sinh lực), trong một môi trường nhất định (như trò chơi). Chương trình AlphaGo nổi tiếng đã đánh bại nhà vô địch thế giới trong trò chơi cờ vây được xây dựng bằng RL.

Danh sách này có thể kéo dài mãi, nhưng hy vọng nó mang lại cho bạn cảm nhận về phạm vi và độ phức tạp đáng kinh ngạc của các tác vụ mà học máy có thể giải quyết, và các loại kỹ thuật mà bạn sẽ sử dụng cho mỗi tác vụ.

Các loại hệ thống học máy

Có rất nhiều loại hệ thống học máy khác nhau nên việc phân loại chúng thành các danh mục lớn, dựa trên các tiêu chí sau là rất hữu ích:

- Cách chúng được giám sát trong quá trình huấn luyện (có giám sát, không giám sát, bán giám sát, tự giám sát và các loại khác)
- Liệu chúng có thể học tăng cường ngay lập tức hay không (học trực tuyến so với học theo lô)
- Liệu chúng hoạt động bằng cách chỉ đơn giản so sánh các điểm dữ liệu mới với các điểm dữ liệu đã biết, hay thay vào đó bằng cách phát hiện các mẫu trong dữ liệu huấn luyện và xây dựng một mô hình dự đoán, giống như các nhà khoa học làm (học dựa trên trường hợp so với học dựa trên mô hình)

Các tiêu chí này không loại trừ lẫn nhau; bạn có thể kết hợp chúng theo bất kỳ cách nào bạn muốn. Ví dụ, một bộ lọc thư rác hiện đại có thể học ngay lập tức bằng cách sử dụng mô hình mạng nơ-ron sâu được huấn luyện bằng các ví dụ về thư rác và thư hợp lệ do con người cung cấp; điều này làm cho nó trở thành một hệ thống học trực tuyến, dựa trên mô hình, có giám sát.

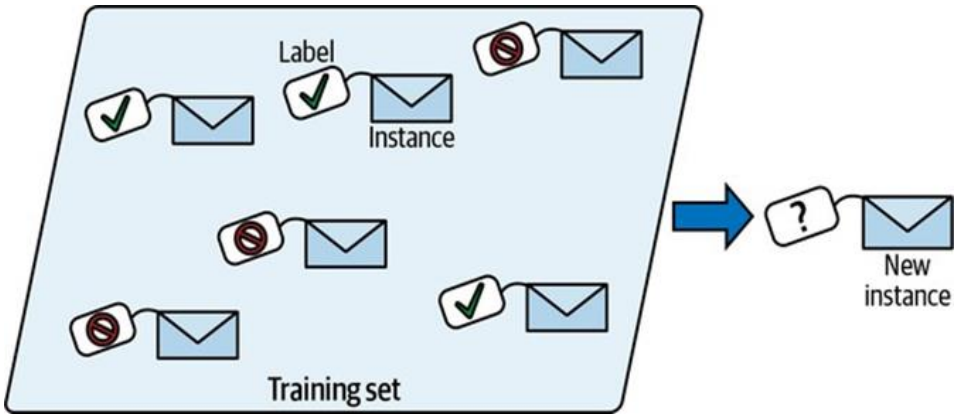
Hãy cùng xem xét từng tiêu chí này kỹ hơn một chút.

Giám sát huấn luyện

Các hệ thống ML có thể được phân loại theo lượng và loại giám sát mà chúng nhận được trong quá trình huấn luyện. Có nhiều danh mục, nhưng chúng ta sẽ thảo luận về những danh mục chính: học có giám sát, học không giám sát, học tự giám sát, học bán giám sát và học tăng cường.

Học có giám sát

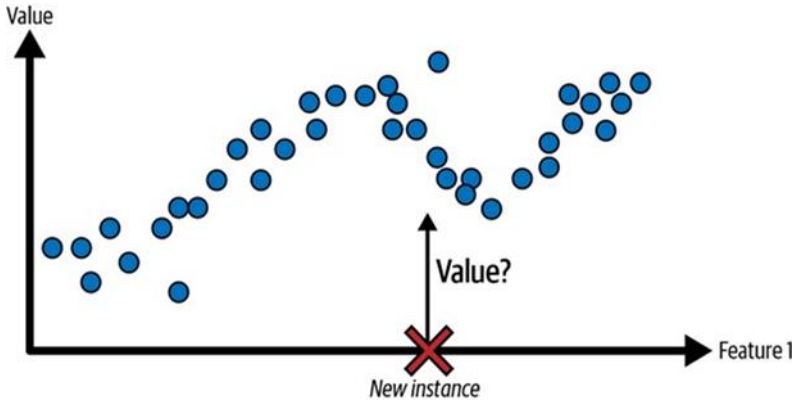
Trong học có giám sát, tập huấn luyện mà bạn cung cấp cho thuật toán bao gồm các giải pháp mong muốn, được gọi là nhãn (Hình 1-5).



Hình 1-5. Một tập huấn luyện có nhãn để phân loại thư rác (một ví dụ về học có giám sát)

Một tác vụ học có giám sát điển hình là phân loại. Bộ lọc thư rác là một ví dụ điển hình về điều này: nó được huấn luyện với nhiều email mẫu cùng với lớp của chúng (thư rác hoặc thư hợp lệ), và nó phải học cách phân loại các email mới.

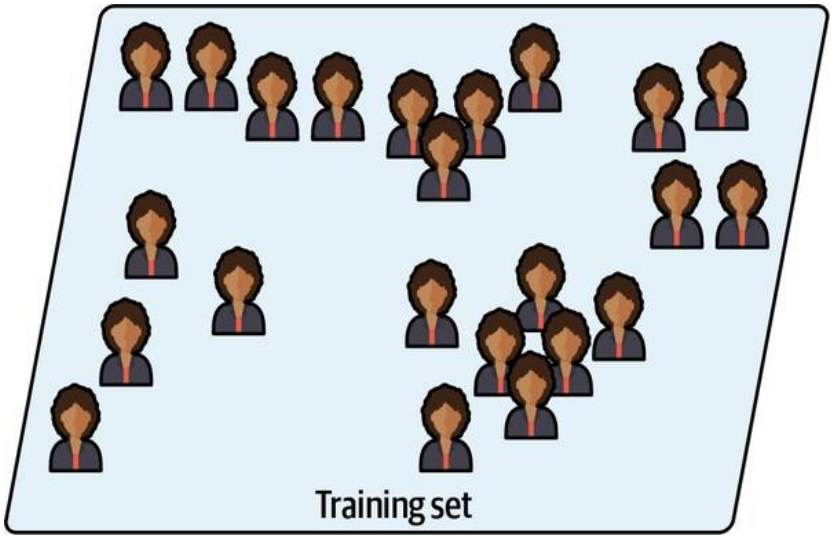
Một tác vụ điển hình khác là dự đoán một giá trị số mục tiêu, chẳng hạn như giá của một chiếc ô tô, với một tập hợp các đặc trưng (số dặm, tuổi, thương hiệu, v.v.). Loại tác vụ này được gọi là hồi quy (Hình 1-6).1 Để huấn luyện hệ thống, bạn cần cung cấp cho nó nhiều ví dụ về ô tô, bao gồm cả các đặc trưng và mục tiêu của chúng (tức là giá của chúng). Lưu ý rằng một số mô hình hồi quy cũng có thể được sử dụng để phân loại, và ngược lại. Ví dụ, hồi quy logistic thường được sử dụng để phân loại, vì nó có thể xuất ra một giá trị tương ứng với xác suất thuộc về một lớp nhất định (ví dụ: 20% khả năng là thư rác).



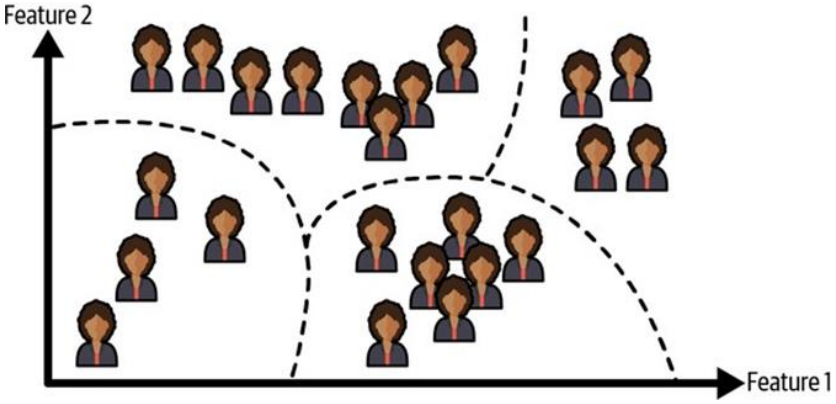
Hình 1-6. Một vấn đề hồi quy: dự đoán một giá trị, với một đặc trưng đầu vào (thường có nhiều đặc trưng đầu vào, và đôi khi nhiều giá trị đầu ra)

Học không giám sát

Trong học không giám sát, như bạn có thể đoán, dữ liệu huấn luyện không có nhãn (Hình 1-7). Hệ thống cố gắng học mà không có giáo viên. Ví dụ, giả sử bạn có rất nhiều dữ liệu về khách truy cập blog của mình. Bạn có thể muốn chạy một thuật toán phân cụm để cố gắng phát hiện các nhóm khách truy cập tương tự (Hình 1-8). Bạn không hề nói cho thuật toán biết khách truy cập thuộc nhóm nào: nó tìm ra những kết nối đó mà không cần sự giúp đỡ của bạn. Ví dụ, nó có thể nhận thấy rằng 40% khách truy cập của bạn là thanh thiếu niên yêu thích truyện tranh và thường đọc blog của bạn sau giờ học, trong khi 20% là người lớn thích khoa học viễn tưởng và truy cập vào cuối tuần. Nếu bạn sử dụng thuật toán phân cụm phân cấp, nó cũng có thể chia mỗi nhóm thành các nhóm nhỏ hơn. Điều này có thể giúp bạn nhắm mục tiêu bài đăng của mình cho từng nhóm.



Hình 1-7. Một tập huấn luyện không nhãn cho học không giám sát

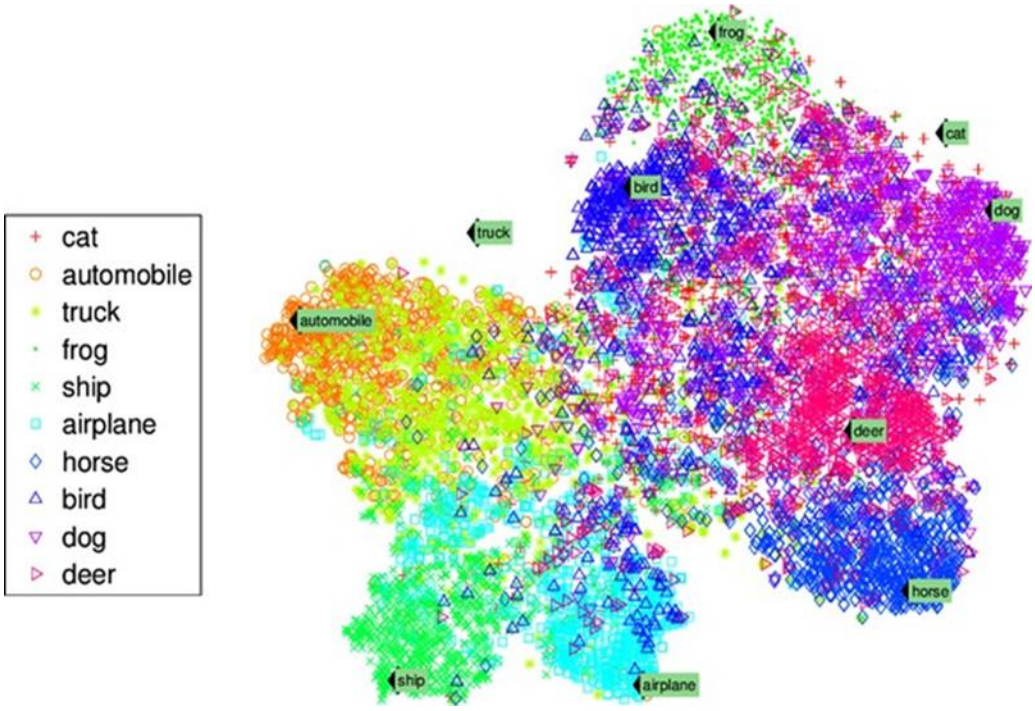


Hình 1-8. Phân cụm

Các thuật toán trực quan hóa cũng là những ví dụ điển hình về học không giám sát: bạn cung cấp cho chúng rất nhiều dữ liệu phức tạp và không nhãn, và chúng xuất ra một biểu diễn 2D hoặc 3D của dữ liệu của bạn có thể dễ dàng vẽ (Hình 1-9). Các thuật toán này cố

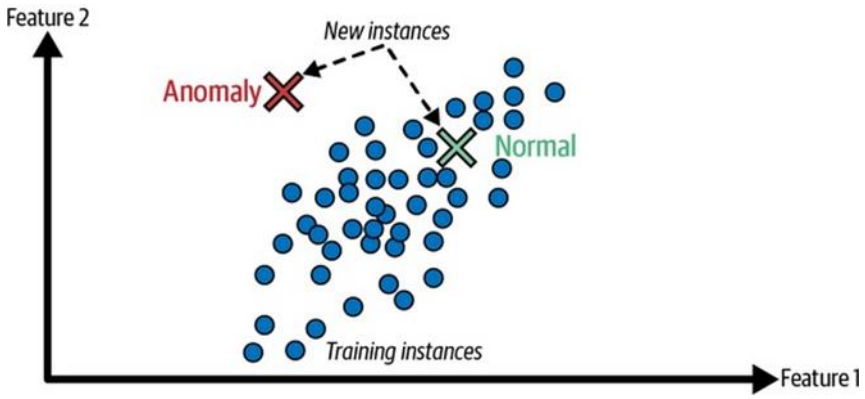
gắng bảo tồn cấu trúc càng nhiều càng tốt (ví dụ: cố gắng giữ các cụm riêng biệt trong không gian đầu vào không chồng chéo trong hình ảnh trực quan) để bạn có thể hiểu cách dữ liệu được tổ chức và có thể xác định các mẫu không ngờ tới.

Một tác vụ liên quan là giảm chiều, trong đó mục tiêu là đơn giản hóa dữ liệu mà không làm mất quá nhiều thông tin. Một cách để làm điều này là hợp nhất một số đặc trưng tương quan thành một. Ví dụ, số dặm của một chiếc ô tô có thể tương quan mạnh mẽ với tuổi của nó, vì vậy thuật toán giảm chiều sẽ hợp nhất chúng thành một đặc trưng đại diện cho sự hao mòn của ô tô. Điều này được gọi là trích xuất đặc trưng.



Hình 1-9. Ví dụ về trực quan hóa t-SNE làm nổi bật các cụm ngữ nghĩa2

Một tác vụ không giám sát quan trọng khác là phát hiện bất thường — ví dụ, phát hiện các giao dịch thẻ tín dụng bất thường để ngăn chặn gian lận, phát hiện lỗi sản xuất hoặc tự động loại bỏ các giá trị ngoại lai khỏi một tập dữ liệu trước khi đưa nó vào một thuật toán học khác. Hệ thống được hiển thị chủ yếu các trường hợp bình thường trong quá trình huấn luyện, vì vậy nó học cách nhận dạng chúng; sau đó, khi nó thấy một trường hợp mới, nó có thể biết liệu nó trông giống như một trường hợp bình thường hay nó có khả năng là một bất thường (xem Hình 1-10). Một tác vụ rất giống là phát hiện sự mới lạ: nó nhằm mục đích phát hiện các trường hợp mới trông khác với tất cả các trường hợp trong tập huấn luyện. Điều này yêu cầu có một tập huấn luyện rất “sạch”, không có bất kỳ trường hợp nào mà bạn muốn thuật toán phát hiện. Ví dụ, nếu bạn có hàng nghìn bức ảnh chó, và 1% trong số đó là chó Chihuahua, thì thuật toán phát hiện sự mới lạ không nên coi những bức ảnh Chihuahua mới là mới lạ. Mặt khác, các thuật toán phát hiện bất thường có thể coi những con chó này rất hiếm và rất khác so với những con chó khác đến mức chúng có thể phân loại chúng là bất thường (không có ý xúc phạm chó Chihuahua).

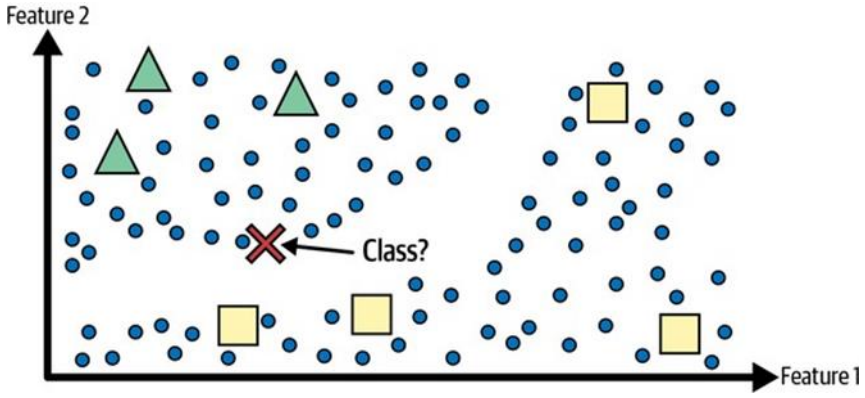


Hình 1-10. Phát hiện bất thường

Cuối cùng, một tác vụ không giám sát phổ biến khác là học luật kết hợp, trong đó mục tiêu là đào sâu vào lượng lớn dữ liệu và khám phá các mối quan hệ thú vị giữa các thuộc tính. Ví dụ, giả sử bạn sở hữu một siêu thị. Chạy một luật kết hợp trên nhật ký bán hàng của bạn có thể tiết lộ rằng những người mua sốt thịt nướng và khoai tây chiên cũng có xu hướng mua bít tết. Do đó, bạn có thể muốn đặt những mặt hàng này gần nhau.

Học bán giám sát

Vì việc gắn nhãn dữ liệu thường tốn thời gian và chi phí, bạn thường sẽ có rất nhiều trường hợp không nhãn, và ít trường hợp có nhãn. Một số thuật toán có thể xử lý dữ liệu được gắn nhãn một phần. Điều này được gọi là học bán giám sát (Hình 1-11).



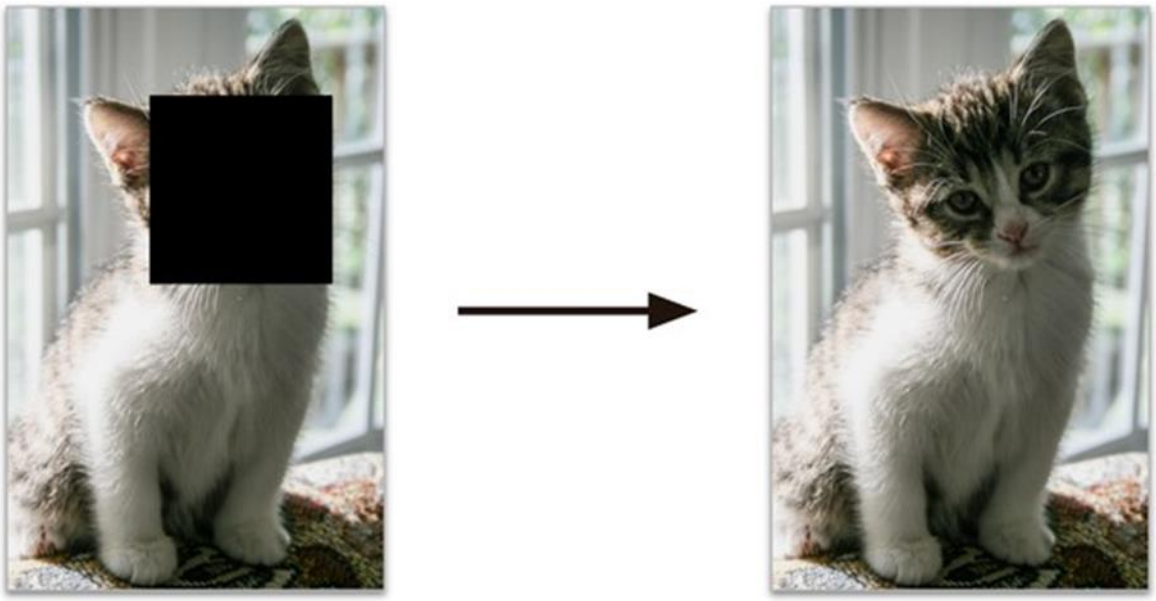
Hình 1-11. Học bán giám sát với hai lớp (tam giác và hình vuông): các ví dụ không nhãn (vòng tròn) giúp phân loại một trường hợp mới (dấu chữ thập) vào lớp tam giác thay vì lớp hình vuông, mặc dù nó gần với các hình vuông có nhãn hơn

Một số dịch vụ lưu trữ ảnh, chẳng hạn như Google Photos, là những ví dụ điển hình về điều này. Khi bạn tải tất cả ảnh gia đình của mình lên dịch vụ, nó sẽ tự động nhận ra rằng cùng một người A xuất hiện trong ảnh 1, 5 và 11, trong khi một người B khác xuất hiện trong ảnh 2, 5 và 7. Đây là phần không giám sát của thuật toán (phân cụm). Bây giờ tất cả những gì hệ thống cần là bạn cho nó biết những người này là ai. Chỉ cần thêm một nhãn cho mỗi người3 và nó có thể đặt tên cho mọi người trong mọi bức ảnh, điều này hữu ích cho việc tìm kiếm ảnh. Hầu hết các thuật toán học bán giám sát là sự kết hợp của các thuật toán không giám

sát và có giám sát. Ví dụ, một thuật toán phân cụm có thể được sử dụng để nhóm các trường hợp tương tự lại với nhau, và sau đó mỗi trường hợp không nhãn có thể được gắn nhãn với nhãn phổ biến nhất trong cụm của nó. Khi toàn bộ tập dữ liệu được gắn nhãn, có thể sử dụng bất kỳ thuật toán học có giám sát nào.

Học tự giám sát

Một cách tiếp cận khác đối với học máy liên quan đến việc thực sự tạo ra một tập dữ liệu được gắn nhãn đầy đủ từ một tập dữ liệu hoàn toàn không được gắn nhãn. Một lần nữa, khi toàn bộ tập dữ liệu được gắn nhãn, bất kỳ thuật toán học có giám sát nào cũng có thể được sử dụng. Cách tiếp cận này được gọi là học tự giám sát. Ví dụ, nếu bạn có một tập dữ liệu lớn các hình ảnh không được gắn nhãn, bạn có thể che ngẫu nhiên một phần nhỏ của mỗi hình ảnh và sau đó huấn luyện một mô hình để khôi phục hình ảnh gốc (Hình 1-12). Trong quá trình huấn luyện, các hình ảnh bị che được sử dụng làm đầu vào cho mô hình, và các hình ảnh gốc được sử dụng làm nhãn.



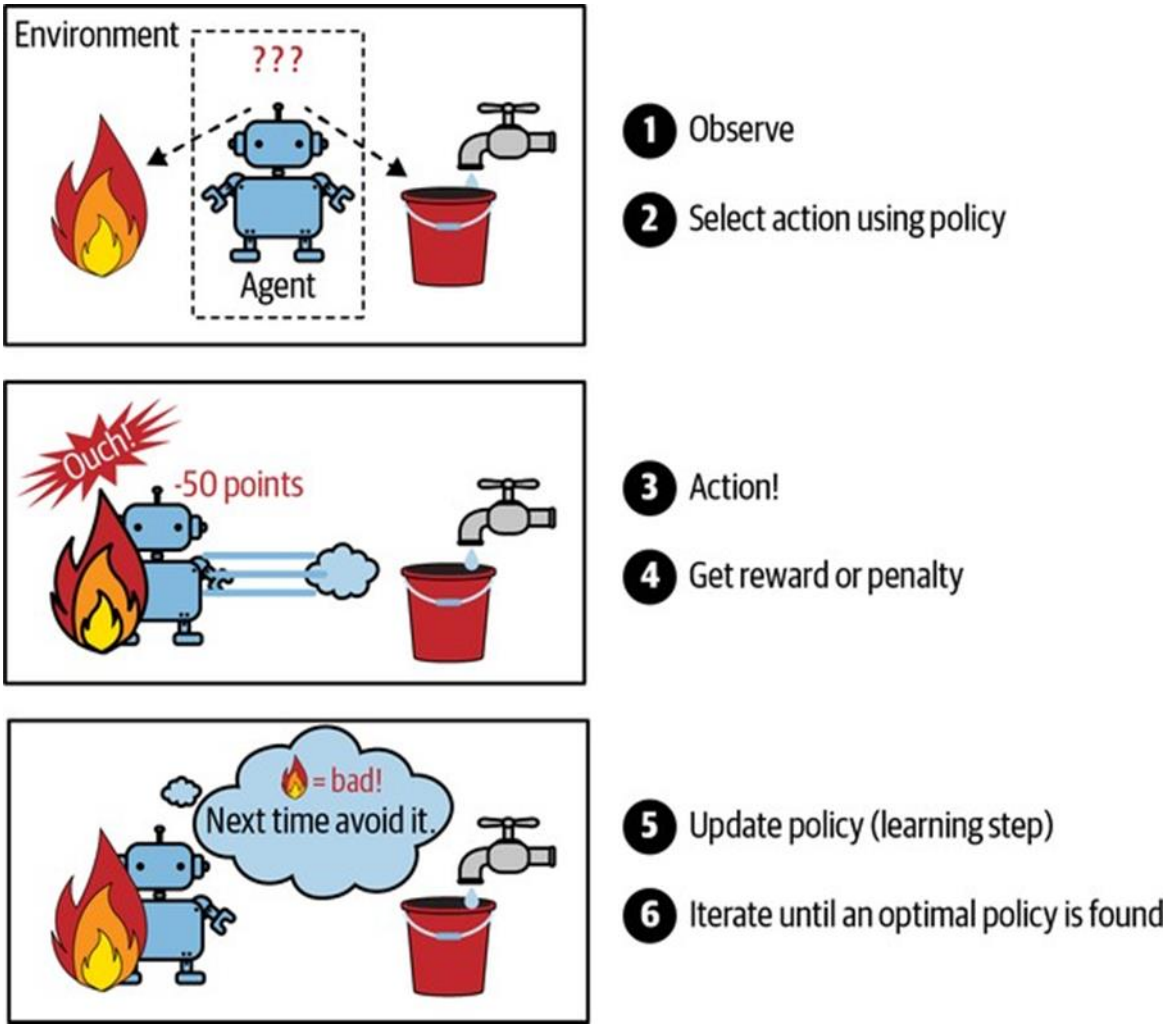
Hình 1-12. Ví dụ học tự giám sát: đầu vào (trái) và mục tiêu (phải)

Mô hình thu được có thể khá hữu ích, ví dụ, để sửa chữa các hình ảnh bị hỏng hoặc xóa các đối tượng không mong muốn khỏi hình ảnh. Nhưng thường thì một mô hình được huấn luyện bằng học tự giám sát không phải là mục tiêu cuối cùng. Bạn thường muốn tinh chỉnh và điều chỉnh mô hình cho một tác vụ hơi khác — một tác vụ mà bạn thực sự quan tâm. Ví dụ, giả sử điều bạn thực sự muốn là có một mô hình phân loại vật nuôi: với một bức ảnh của bất kỳ vật nuôi nào, nó sẽ cho bạn biết nó thuộc loài nào. Nếu bạn có một tập dữ liệu lớn các bức ảnh vật nuôi không được gắn nhãn, bạn có thể bắt đầu bằng cách huấn luyện một mô hình sửa chữa hình ảnh bằng học tự giám sát. Một khi nó hoạt động tốt, nó sẽ có thể phân biệt các loài vật nuôi khác nhau: khi nó sửa chữa một hình ảnh mèo bị che mặt, nó phải biết không thêm mặt chó vào. Giả sử kiến trúc mô hình của bạn cho phép (và hầu hết các kiến trúc mạng nơ-ron đều làm vậy), thì có thể tinh chỉnh mô hình để nó dự đoán

loài vật nuôi thay vì sửa chữa hình ảnh. Bước cuối cùng bao gồm việc tinh chỉnh mô hình trên một tập dữ liệu được gắn nhãn: mô hình đã biết mèo, chó và các loài vật nuôi khác trông như thế nào, vì vậy bước này chỉ cần thiết để mô hình có thể học ánh xạ giữa các loài mà nó đã biết và các nhãn mà chúng ta mong đợi từ nó.

Một số người coi học tự giám sát là một phần của học không giám sát, vì nó xử lý các tập dữ liệu hoàn toàn không được gắn nhãn. Nhưng học tự giám sát sử dụng các nhãn (được tạo) trong quá trình huấn luyện, vì vậy về mặt đó nó gần với học có giám sát hơn. Và thuật ngữ "học không giám sát" thường được sử dụng khi xử lý các tác vụ như phân cụm, giảm chiều hoặc phát hiện bất thường, trong khi học tự giám sát tập trung vào các tác vụ giống như học có giám sát: chủ yếu là phân loại và hồi quy. Nói tóm lại, tốt nhất nên coi học tự giám sát là một loại riêng.

Học tăng cường



Hình 1-13. Học tăng cường

Học tăng cường là một loại khác biệt. Hệ thống học, được gọi là tác nhân trong ngữ cảnh này, có thể quan sát môi trường, chọn và thực hiện các hành động, và nhận lại phần thưởng

(hoặc hình phạt dưới dạng phần thưởng tiêu cực, như được hiển thị trong Hình 1-13). Sau đó, nó phải tự học chiến lược tốt nhất, được gọi là chính sách, để đạt được phần thưởng cao nhất theo thời gian. Một chính sách xác định hành động mà tác nhân nên chọn khi nó ở trong một tình huống nhất định.

Ví dụ, nhiều robot triển khai các thuật toán học tăng cường để học cách đi bộ. Chương trình AlphaGo của DeepMind cũng là một ví dụ điển hình về học tăng cường: nó đã gây chú ý vào tháng 5 năm 2017 khi đánh bại Ke Jie, người chơi cờ vây số một thế giới vào thời điểm đó. Nó đã học được chính sách chiến thắng của mình bằng cách phân tích hàng triệu ván đấu, và sau đó tự chơi nhiều ván đấu với chính mình. Lưu ý rằng quá trình học đã bị tắt trong các ván đấu với nhà vô địch; AlphaGo chỉ đơn thuần áp dụng chính sách mà nó đã học. Như bạn sẽ thấy trong phần tiếp theo, đây được gọi là học ngoại tuyến.

Học theo lô so với học trực tuyến

Một tiêu chí khác được sử dụng để phân loại các hệ thống học máy là liệu hệ thống có thể học tăng cường từ một luồng dữ liệu đến hay không.

Học theo lô

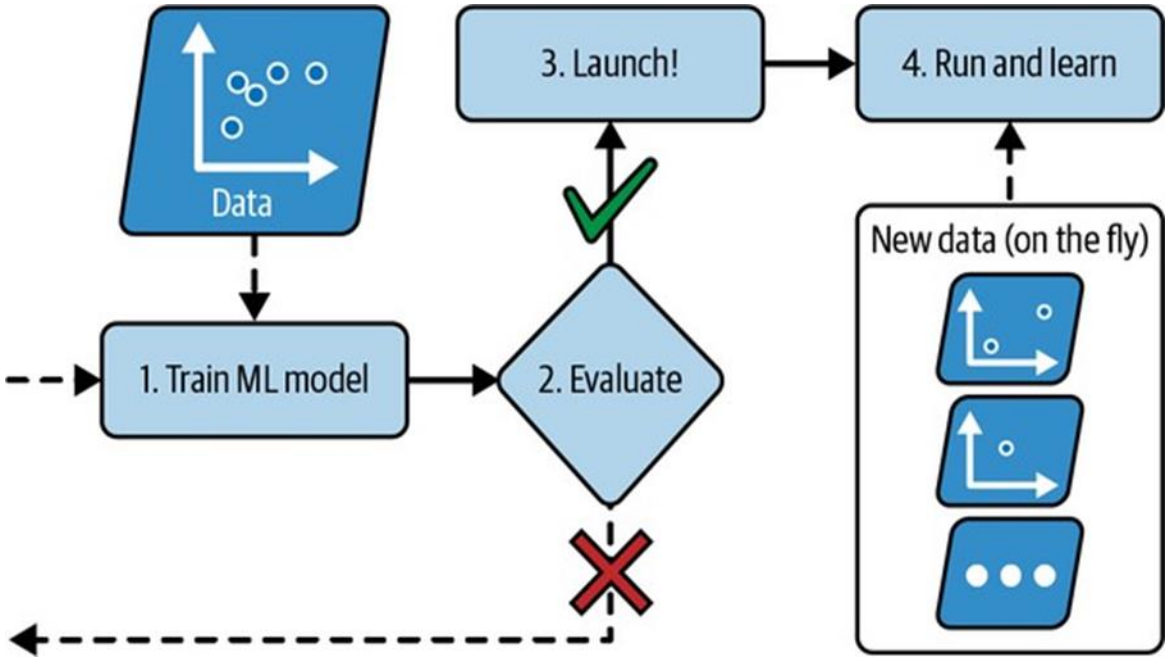
Trong học theo lô, hệ thống không thể học tăng cường: nó phải được huấn luyện bằng cách sử dụng tất cả dữ liệu có sẵn. Điều này thường sẽ tốn rất nhiều thời gian và tài nguyên tính toán, vì vậy nó thường được thực hiện ngoại tuyến. Đầu tiên hệ thống được huấn luyện, và sau đó nó được đưa vào sản xuất và chạy mà không học thêm nữa; nó chỉ áp dụng những gì nó đã học. Điều này được gọi là học ngoại tuyến. Thật không may, hiệu suất của mô hình có xu hướng suy giảm từ từ theo thời gian, đơn giản vì thế giới tiếp tục phát triển trong khi mô hình vẫn không thay đổi. Hiện tượng này thường được gọi là phân rã mô hình (model rot) hoặc trôi dữ liệu (data drift). Giải pháp là thường xuyên huấn luyện lại mô hình trên dữ liệu cập nhật. Tần suất bạn cần làm điều đó phụ thuộc vào trường hợp sử dụng: nếu mô hình phân loại hình ảnh mèo và chó, hiệu suất của nó sẽ suy giảm rất chậm, nhưng nếu mô hình xử lý các hệ thống phát triển nhanh, ví dụ như đưa ra dự đoán trên thị trường tài chính, thì nó có thể suy giảm khá nhanh.

Nếu bạn muốn một hệ thống học theo lô biết về dữ liệu mới (chẳng hạn như một loại thư rác mới), bạn cần huấn luyện một phiên bản mới của hệ thống từ đầu trên toàn bộ tập dữ liệu (không chỉ dữ liệu mới mà còn cả dữ liệu cũ), sau đó thay thế mô hình cũ bằng mô hình mới. May mắn thay, toàn bộ quá trình huấn luyện, đánh giá và triển khai một hệ thống học máy có thể được tự động hóa khá dễ dàng (như chúng ta đã thấy trong Hình 1-3), vì vậy ngay cả một hệ thống học theo lô cũng có thể thích nghi với sự thay đổi. Chỉ cần cập nhật dữ liệu và huấn luyện một phiên bản mới của hệ thống từ đầu thường xuyên khi cần. Giải pháp này đơn giản và thường hoạt động tốt, nhưng việc huấn luyện bằng cách sử dụng toàn bộ tập dữ liệu có thể mất nhiều giờ, vì vậy bạn thường chỉ huấn luyện một hệ thống mới mỗi 24 giờ hoặc thậm chí chỉ hàng tuần. Nếu hệ thống của bạn cần thích nghi với dữ liệu thay đổi nhanh chóng (ví dụ: để dự đoán giá cổ phiếu), thì bạn cần một giải pháp phản ứng nhanh hơn. Ngoài ra, việc huấn luyện trên toàn bộ tập dữ liệu đòi hỏi rất nhiều tài nguyên tính toán (CPU, không gian bộ nhớ, không gian đĩa, I/O đĩa, I/O mạng, v.v.). Nếu bạn có

nhiều dữ liệu và bạn tự động hóa hệ thống của mình để huấn luyện lại từ đầu mỗi ngày, nó sẽ tốn rất nhiều tiền. Nếu lượng dữ liệu quá lớn, thậm chí có thể không thể sử dụng thuật toán học theo lô. Cuối cùng, nếu hệ thống của bạn cần có khả năng học tự động và nó có tài nguyên hạn chế (ví dụ: một ứng dụng điện thoại thông minh hoặc một robot trên sao Hỏa), thì việc mang theo lượng lớn dữ liệu huấn luyện và tốn nhiều tài nguyên để huấn luyện hàng giờ mỗi ngày là một trở ngại. Một lựa chọn tốt hơn trong tất cả các trường hợp này là sử dụng các thuật toán có khả năng học tăng dần.

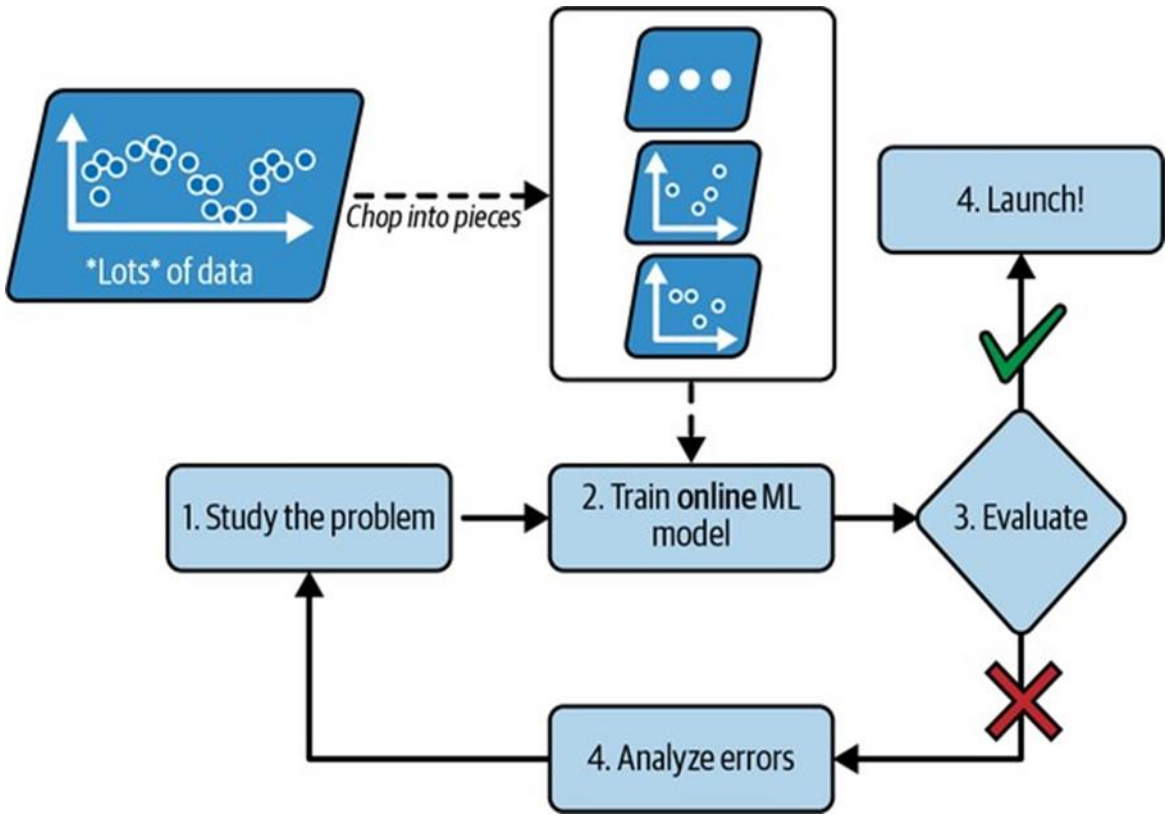
Học trực tuyến

Trong học trực tuyến, bạn huấn luyện hệ thống tăng dần bằng cách cung cấp cho nó các phiên bản dữ liệu một cách tuần tự, riêng lẻ hoặc theo các nhóm nhỏ gọi là mini-batches. Mỗi bước học nhanh và rẻ, vì vậy hệ thống có thể học về dữ liệu mới ngay lập tức, khi nó đến (xem Hình 1-14).



Hình 1-14. Trong học trực tuyến, một mô hình được huấn luyện và triển khai vào sản xuất, và sau đó nó tiếp tục học khi dữ liệu mới đến

Học trực tuyến hữu ích cho các hệ thống cần thích nghi với sự thay đổi cực kỳ nhanh chóng (ví dụ: để phát hiện các mẫu mới trên thị trường chứng khoán). Nó cũng là một lựa chọn tốt nếu bạn có tài nguyên tính toán hạn chế; ví dụ, nếu mô hình được huấn luyện trên thiết bị di động. Ngoài ra, các thuật toán học trực tuyến có thể được sử dụng để huấn luyện các mô hình trên các tập dữ liệu khổng lồ không thể chứa trong bộ nhớ chính của một máy (đây được gọi là học ngoài bộ nhớ). Thuật toán tải một phần dữ liệu, chạy một bước huấn luyện trên dữ liệu đó, và lặp lại quá trình cho đến khi nó đã chạy trên tất cả dữ liệu (xem Hình 1-15).



Hình 1-15. Sử dụng học trực tuyến để xử lý các tập dữ liệu khổng lồ

Một tham số quan trọng của các hệ thống học trực tuyến là tốc độ chúng nên thích nghi với dữ liệu thay đổi: đây được gọi là tốc độ học (learning rate). Nếu bạn đặt tốc độ học cao, thì hệ thống của bạn sẽ nhanh chóng thích nghi với dữ liệu mới, nhưng nó cũng sẽ có xu hướng nhanh chóng quên dữ liệu cũ (và bạn không muốn một bộ lọc thư rác chỉ gắn cờ những loại thư rác mới nhất mà nó được hiển thị). Ngược lại, nếu bạn đặt tốc độ học thấp, hệ thống sẽ có quán tính hơn; tức là nó sẽ học chậm hơn, nhưng nó cũng sẽ ít nhạy cảm hơn với nhiễu trong dữ liệu mới hoặc với các chuỗi điểm dữ liệu không đại diện (các giá trị ngoại lai).

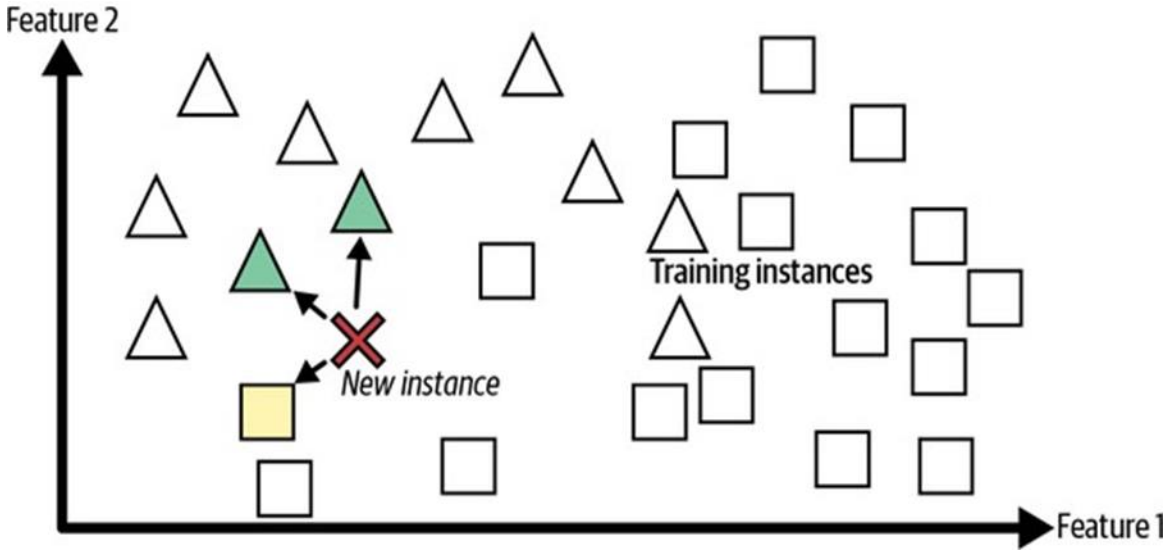
Một thách thức lớn với học trực tuyến là nếu dữ liệu xấu được đưa vào hệ thống, hiệu suất của hệ thống sẽ suy giảm, có thể nhanh chóng (tùy thuộc vào chất lượng dữ liệu và tốc độ học). Nếu đó là một hệ thống đang hoạt động, khách hàng của bạn sẽ nhận thấy. Ví dụ, dữ liệu xấu có thể đến từ một lỗi (ví dụ: một cảm biến bị trục trặc trên một robot), hoặc nó có thể đến từ một người nào đó đang cố gắng gian lận hệ thống (ví dụ: gửi thư rác vào công cụ tìm kiếm để cố gắng xếp hạng cao trong kết quả tìm kiếm). Để giảm thiểu rủi ro này, bạn cần theo dõi hệ thống của mình chặt chẽ và nhanh chóng tắt chế độ học (và có thể quay lại trạng thái hoạt động trước đó) nếu bạn phát hiện hiệu suất giảm sút. Bạn cũng có thể muốn theo dõi dữ liệu đầu vào và phản ứng với dữ liệu bất thường; ví dụ, sử dụng thuật toán phát hiện bất thường (xem Chương 9).

Học dựa trên thực thể và học dựa trên mô hình

Một cách khác để phân loại các hệ thống học máy là dựa vào cách chúng tổng quát hóa. Hầu hết các tác vụ học máy đều liên quan đến việc đưa ra dự đoán. Điều này có nghĩa là, với một số ví dụ huấn luyện, hệ thống cần có khả năng đưa ra dự đoán tốt cho các ví dụ mà nó chưa từng thấy trước đây (tổng quát hóa). Có một thước đo hiệu suất tốt trên dữ liệu huấn luyện là tốt, nhưng không đủ; mục tiêu thực sự là thực hiện tốt trên các trường hợp mới. Có hai cách tiếp cận chính để tổng quát hóa: học dựa trên trường hợp và học dựa trên mô hình.

Học dựa trên thực thể

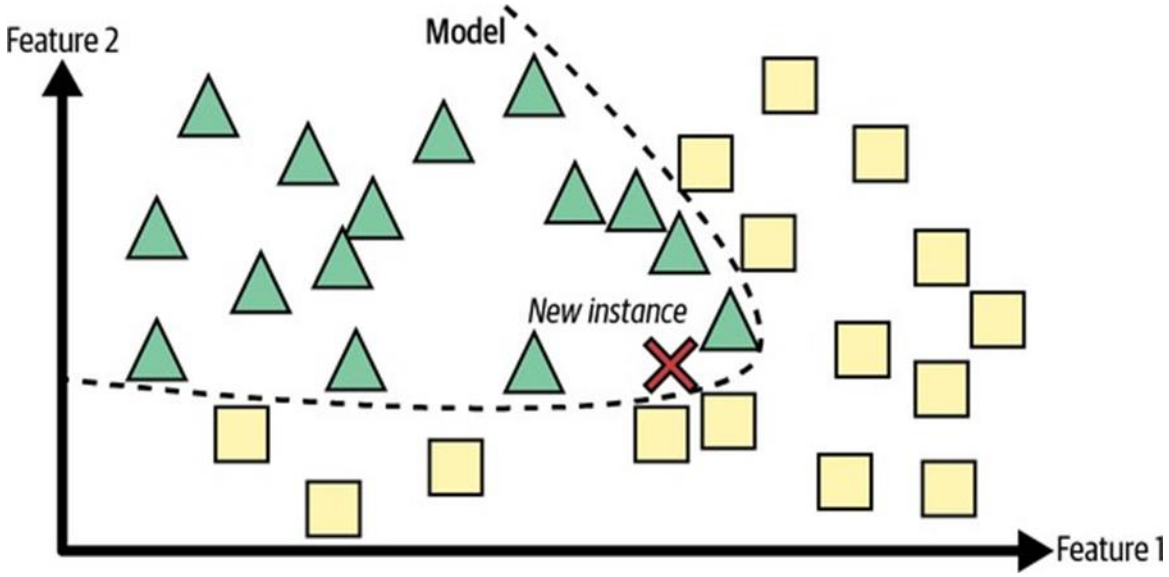
Có lẽ hình thức học đơn giản nhất là học thuộc lòng. Nếu bạn tạo một bộ lọc thư rác theo cách này, nó sẽ chỉ đánh dấu tất cả các email giống hệt với các email đã được người dùng đánh dấu — không phải là giải pháp tồi tệ nhất, nhưng chắc chắn không phải là tốt nhất. Thay vì chỉ đánh dấu các email giống hệt với các email thư rác đã biết, bộ lọc thư rác của bạn có thể được lập trình để cũng đánh dấu các email rất giống với các email thư rác đã biết. Điều này yêu cầu một thước đo sự tương đồng giữa hai email. Một thước đo sự tương đồng (rất cơ bản) giữa hai email có thể là đếm số từ chúng có chung. Hệ thống sẽ đánh dấu một email là thư rác nếu nó có nhiều từ chung với một email thư rác đã biết. Đây được gọi là học dựa trên trường hợp: hệ thống học thuộc lòng các ví dụ, sau đó tổng quát hóa sang các trường hợp mới bằng cách sử dụng một thước đo sự tương đồng để so sánh chúng với các ví dụ đã học (hoặc một tập hợp con của chúng). Ví dụ, trong Hình 1-16, trường hợp mới sẽ được phân loại là một tam giác vì đa số các trường hợp tương tự nhất thuộc về lớp đó.



Hình 1-16. Học dựa trên trường hợp

Học dựa trên mô hình và quy trình làm việc học máy điển hình

Một cách khác để tổng quát hóa từ một tập hợp các ví dụ là xây dựng một mô hình từ các ví dụ này và sau đó sử dụng mô hình đó để đưa ra dự đoán. Đây được gọi là học dựa trên mô hình (Hình 1-17).



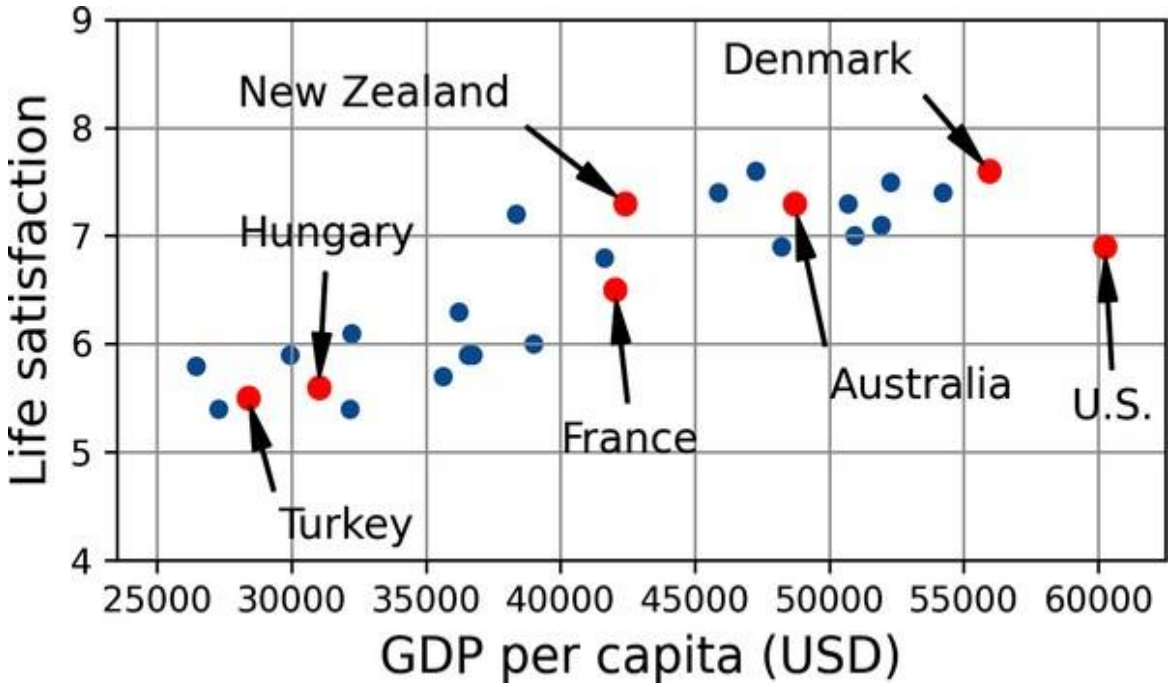
Hình 1-17. Học dựa trên mô hình

Ví dụ, giả sử bạn muốn biết liệu tiền có làm cho con người hạnh phúc hay không, vì vậy bạn tải dữ liệu Chỉ số Cuộc sống Tốt đẹp từ trang web của OECD và số liệu thống kê của Ngân hàng Thế giới về tổng sản phẩm quốc nội (GDP) trên đầu người. Sau đó, bạn nối các bảng và sắp xếp theo GDP trên đầu người. Bảng 1-1 hiển thị một phần trích dẫn những gì bạn nhận được.

Bảng 1-1. Tiền có làm con người hạnh phúc hơn không?

Quốc gia	GDP trên đầu người (USD)	Hài lòng cuộc sống
Thổ Nhĩ Kỳ	28.384	5.5
Hungary	31.008	5.6
Pháp	42.026	6.5
Hoa Kỳ	60.236	6.9
New Zealand	42.404	7.3
Úc	48.698	7.3
Đan Mạch	55.938	7.6

Hãy vẽ dữ liệu cho các quốc gia này (Hình 1-18).



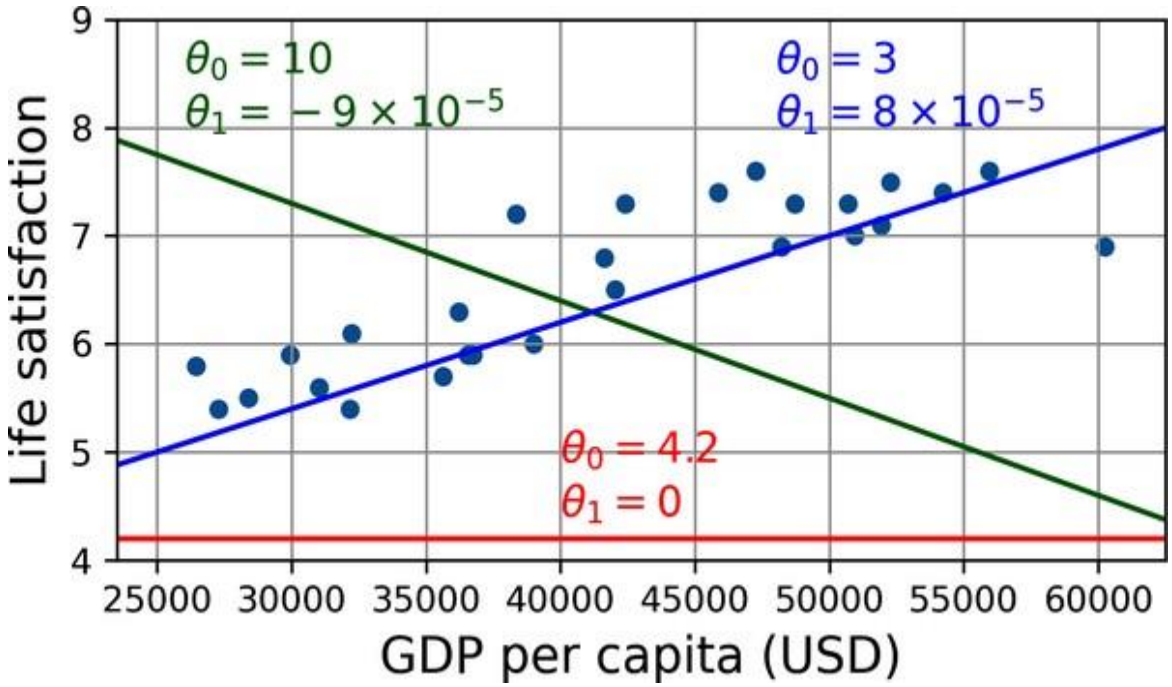
Hình 1-18. Bạn có thấy xu hướng nào ở đây không?

Có vẻ như có một xu hướng ở đây! Mặc dù dữ liệu nhiễu (tức là một phần ngẫu nhiên), có vẻ như sự hài lòng cuộc sống tăng lên gần như tuyến tính khi GDP trên đầu người của quốc gia tăng lên. Vì vậy, bạn quyết định mô hình hóa sự hài lòng cuộc sống như một hàm tuyến tính của GDP trên đầu người. Bước này được gọi là chọn mô hình: bạn đã chọn một mô hình tuyến tính về sự hài lòng cuộc sống chỉ với một thuộc tính, GDP trên đầu người (Phương trình 1-1).

Phương trình 1-1. A simple linear model

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

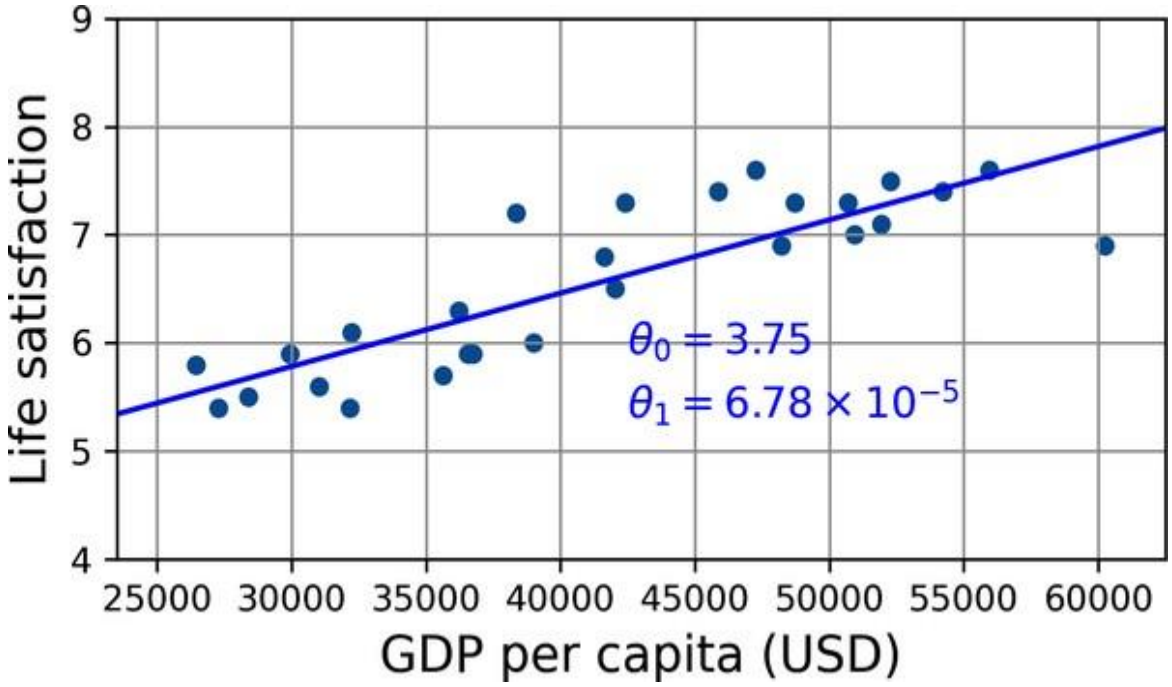
Mô hình này có hai tham số mô hình, θ_0 và θ_1. Bằng cách điều chỉnh các tham số này, bạn có thể làm cho mô hình của mình đại diện cho bất kỳ hàm tuyến tính nào, như được hiển thị trong Hình 1-19.



Hình 1-19. Một vài mô hình tuyến tính có thể

Trước khi bạn có thể sử dụng mô hình của mình, bạn cần xác định các giá trị tham số θ_0 và θ_1. Làm thế nào bạn có thể biết giá trị nào sẽ làm cho mô hình của bạn hoạt động tốt nhất? Để trả lời câu hỏi này, bạn cần chỉ định một thước đo hiệu suất. Bạn có thể định nghĩa một hàm tiện ích (hoặc hàm phù hợp) đo lường mức độ tốt của mô hình, hoặc bạn có thể định nghĩa một hàm chi phí đo lường mức độ tồi tệ của nó. Đối với các vấn đề hồi quy tuyến tính, mọi người thường sử dụng một hàm chi phí đo lường khoảng cách giữa các dự đoán của mô hình tuyến tính và các ví dụ huấn luyện; mục tiêu là giảm thiểu khoảng cách này. Đây là lúc thuật toán hồi quy tuyến tính xuất hiện: bạn cung cấp cho nó các ví dụ huấn luyện của mình, và nó tìm ra các tham số làm cho mô hình tuyến tính phù hợp nhất với dữ liệu của bạn. Đây được gọi là huấn luyện mô hình. Trong trường hợp của chúng ta, thuật toán tìm thấy rằng các giá trị tham số tối ưu là $\theta_0 = 3.75$ và $\theta_1 = 6.78 \times 10–5$.

Bây giờ mô hình phù hợp với dữ liệu huấn luyện càng gần càng tốt (đối với một mô hình tuyến tính), như bạn có thể thấy trong Hình 1-20.



Hình 1-20. Mô hình tuyến tính phù hợp nhất với dữ liệu huấn luyện

Cuối cùng, bạn đã sẵn sàng chạy mô hình để đưa ra dự đoán. Ví dụ, giả sử bạn muốn biết người Síp hạnh phúc đến mức nào, và dữ liệu của OECD không có câu trả lời. May mắn thay, bạn có thể sử dụng mô hình của mình để đưa ra một dự đoán tốt: bạn tra cứu GDP trên đầu người của Síp, tìm thấy $37.655, và sau đó áp dụng mô hình của bạn và tìm thấy rằng sự hài lòng cuộc sống có khả năng nằm đâu đó khoảng $3.75 + 37.655 \times 6.78 \times 10^{-5} = 6.30$. Để kích thích sự tò mò của bạn, Ví dụ 1-1 hiển thị mã Python tải dữ liệu, tách đầu vào X khỏi nhãn y, tạo biểu đồ phân tán để trực quan hóa, sau đó huấn luyện một mô hình tuyến tính và đưa ra dự đoán.5

Ví dụ 1-1. Huấn luyện và chạy một mô hình tuyến tính bằng Scikit-Learn

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Tải và chuẩn bị dữ liệu
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Trực quan hóa dữ liệu
lifesat.plot(kind='scatter', grid=True,
```

```
            x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Chọn một mô hình tuyến tính
model = LinearRegression()

# Huấn luyện mô hình
model.fit(X, y)

# Dự đoán cho Síp
X_new = [[37_655.2]] # GDP trên đầu người của Síp năm 2020
print(model.predict(X_new)) # đầu ra: [[6.30165767]]
```

LƯU Ý Nếu bạn sử dụng thuật toán học dựa trên trường hợp thay vào đó, bạn sẽ thấy rằng Israel có GDP trên đầu người gần nhất với Síp ($38.341), và vì dữ liệu OECD cho chúng ta biết rằng sự hài lòng cuộc sống của người Israel là 7.2, bạn sẽ dự đoán sự hài lòng cuộc sống là 7.2 cho Síp. Nếu bạn phóng to một chút và nhìn vào hai quốc gia gần nhất tiếp theo, bạn sẽ tìm thấy Litva và Slovenia, cả hai đều có mức độ hài lòng cuộc sống là 5.9. Trung bình cộng ba giá trị này, bạn nhận được 6.33, khá gần với dự đoán dựa trên mô hình của bạn. Thuật toán đơn giản này được gọi là hồi quy k-láng giềng gần nhất (trong ví dụ này, k = 3). Thay thế mô hình hồi quy tuyến tính bằng hồi quy k-láng giềng gần nhất trong đoạn mã trước dễ dàng như việc thay thế các dòng này:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

bằng hai dòng này:

```
from sklearn.neighbors import KNeighborsRegressor
model = KNeighborsRegressor(n_neighbors=3)
```

Nếu mọi thứ diễn ra tốt đẹp, mô hình của bạn sẽ đưa ra những dự đoán tốt. Nếu không, bạn có thể cần sử dụng thêm các thuộc tính (tỷ lệ việc làm, sức khỏe, ô nhiễm không khí, v.v.), nhận được nhiều dữ liệu huấn luyện hơn hoặc chất lượng tốt hơn, hoặc có thể chọn một mô hình mạnh hơn (ví dụ: mô hình hồi quy đa thức). Tóm lại:

- Bạn đã nghiên cứu dữ liệu.
- Bạn đã chọn một mô hình.
- Bạn đã huấn luyện nó trên dữ liệu huấn luyện (tức là thuật toán học đã tìm kiếm các giá trị tham số mô hình để giảm thiểu hàm chi phí).
- Cuối cùng, bạn đã áp dụng mô hình để đưa ra dự đoán về các trường hợp mới (đây được gọi là suy luận), hy vọng rằng mô hình này sẽ tổng quát hóa tốt.

Đây là những gì một dự án học máy điển hình trông giống như vậy. Trong Chương 2, bạn sẽ trải nghiệm điều này trực tiếp bằng cách thực hiện một dự án từ đầu đến cuối. Chúng ta đã đi được một chặng đường dài cho đến nay: bạn đã biết học máy thực sự là gì, tại sao nó

hữu ích, một số loại hệ thống ML phổ biến nhất là gì và quy trình làm việc của một dự án điển hình trông như thế nào. Bây giờ hãy xem điều gì có thể xảy ra sai sót trong quá trình học và ngăn bạn đưa ra dự đoán chính xác.

Những thách thức chính của học máy

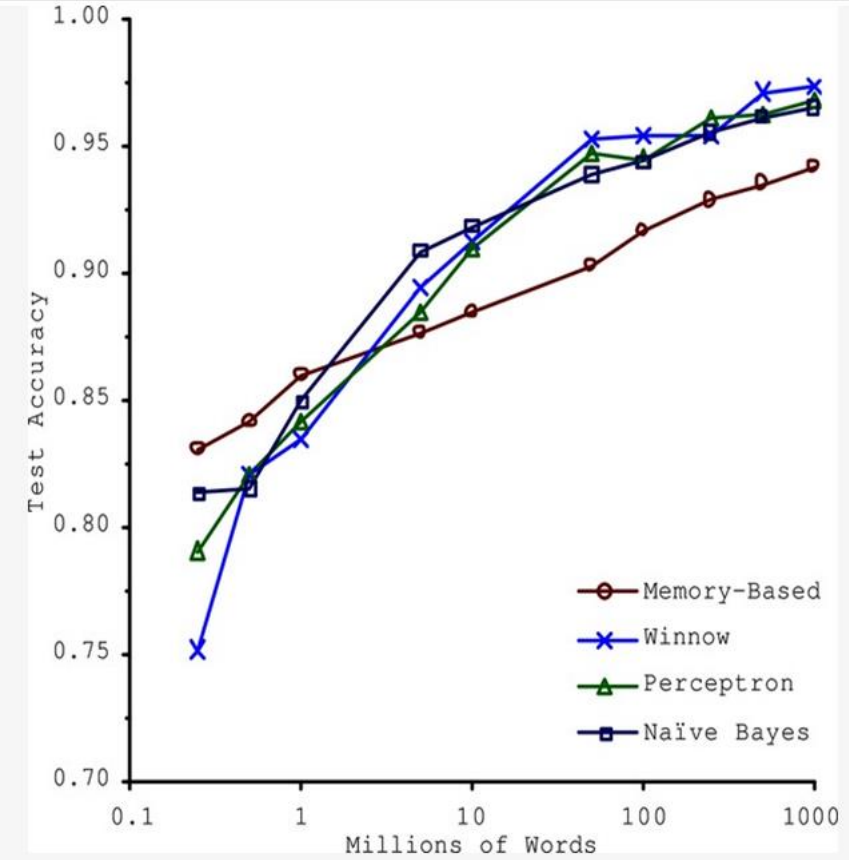
Tóm lại, vì nhiệm vụ chính của bạn là chọn một mô hình và huấn luyện nó trên một số dữ liệu, hai điều có thể sai là "mô hình kém" và "dữ liệu kém". Hãy bắt đầu với các ví dụ về dữ liệu kém.

Số lượng dữ liệu huấn luyện không đủ

Để một đứa trẻ mới biết đi học thế nào là một quả táo, tất cả những gì bạn cần làm là chỉ vào một quả táo và nói "quả táo" (có thể lặp lại thủ tục này vài lần). Bây giờ đứa trẻ có thể nhận ra những quả táo với đủ màu sắc và hình dạng. Thật thiên tài. Học máy chưa đạt đến trình độ đó; nó cần rất nhiều dữ liệu để hầu hết các thuật toán học máy hoạt động đúng cách. Ngay cả đối với các vấn đề rất đơn giản, bạn thường cần hàng nghìn ví dụ, và đối với các vấn đề phức tạp như nhận dạng hình ảnh hoặc giọng nói, bạn có thể cần hàng triệu ví dụ (trừ khi bạn có thể sử dụng lại các phần của một mô hình hiện có).

HIỆU QUẢ KHÔNG HỢP LÝ CỦA DỮ LIỆU

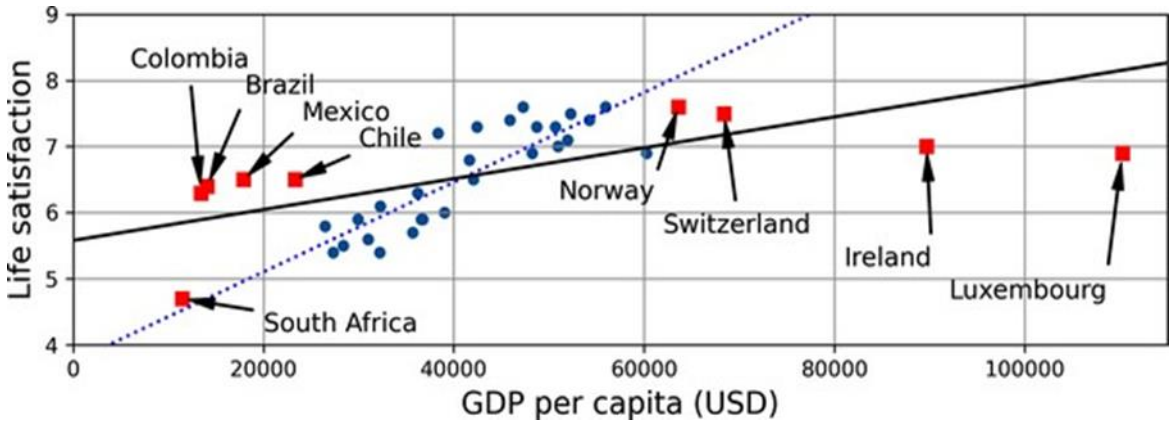
Trong một bài báo nổi tiếng được xuất bản năm 2001, các nhà nghiên cứu của Microsoft Michele Banko và Eric Brill đã chỉ ra rằng các thuật toán học máy rất khác nhau, bao gồm cả những thuật toán khá đơn giản, hoạt động gần như tốt như nhau trên một vấn đề phức tạp về giải mơ hồ ngôn ngữ tự nhiên6 khi chúng được cung cấp đủ dữ liệu (như bạn có thể thấy trong Hình 1-21). Như các tác giả đã nói, "những kết quả này gợi ý rằng chúng ta có thể muốn xem xét lại sự đánh đổi giữa việc dành thời gian và tiền bạc cho phát triển thuật toán so với việc dành nó cho phát triển kho ngữ liệu". Ý tưởng rằng dữ liệu quan trọng hơn thuật toán đối với các vấn đề phức tạp đã được Peter Norvig và cộng sự phổ biến hơn nữa trong một bài báo có tựa đề "The Unreasonable Effectiveness of Data", được xuất bản năm 2009.7 Tuy nhiên, cần lưu ý rằng các tập dữ liệu nhỏ và vừa vẫn rất phổ biến, và không phải lúc nào cũng dễ dàng hoặc rẻ để có được dữ liệu huấn luyện bổ sung — vì vậy đừng từ bỏ thuật toán vội.



Hình 1.21. Tầm quan trọng của dữ liệu so với thuật toán.

Dữ liệu huấn luyện không đại diện

Để tổng quát hóa tốt, điều quan trọng là dữ liệu huấn luyện của bạn phải đại diện cho các trường hợp mới mà bạn muốn tổng quát hóa. Điều này đúng cho dù bạn sử dụng học dựa trên trường hợp hay học dựa trên mô hình. Ví dụ, tập hợp các quốc gia bạn đã sử dụng trước đó để huấn luyện mô hình tuyến tính không hoàn toàn đại diện; nó không chứa bất kỳ quốc gia nào có GDP trên đầu người thấp hơn 23.500 đô la hoặc cao hơn 62.500 đô la. Hình 1-22 cho thấy dữ liệu trông như thế nào khi bạn thêm các quốc gia như vậy. Nếu bạn huấn luyện một mô hình tuyến tính trên dữ liệu này, bạn sẽ nhận được đường liền nét, trong khi mô hình cũ được biểu thị bằng đường chấm chấm. Như bạn có thể thấy, việc thêm một vài quốc gia bị thiếu không chỉ làm thay đổi đáng kể mô hình mà còn làm rõ rằng một mô hình tuyến tính đơn giản như vậy có lẽ sẽ không bao giờ hoạt động tốt. Có vẻ như các quốc gia rất giàu không hạnh phúc hơn các quốc gia giàu vừa phải (thực tế, họ dường như hơi bất hạnh hơn một chút!), và ngược lại, một số quốc gia nghèo có vẻ hạnh phúc hơn nhiều quốc gia giàu. Bằng cách sử dụng một tập huấn luyện không đại diện, bạn đã huấn luyện một mô hình không có khả năng đưa ra dự đoán chính xác, đặc biệt đối với các quốc gia rất nghèo và rất giàu.



Hình 1-22. Một mẫu huấn luyện đại diện hơn

Điều quan trọng là phải sử dụng một tập huấn luyện đại diện cho các trường hợp bạn muốn tổng quát hóa. Điều này thường khó hơn nghe có vẻ: nếu mẫu quá nhỏ, bạn sẽ có nhiễu lấy mẫu (tức là dữ liệu không đại diện do ngẫu nhiên), nhưng ngay cả các mẫu rất lớn cũng có thể không đại diện nếu phương pháp lấy mẫu bị lỗi. Điều này được gọi là sai lệch lấy mẫu.

VÍ DỤ VỀ SAI LỆCH LẤY MẪU

Có lẽ ví dụ nổi tiếng nhất về sai lệch lấy mẫu xảy ra trong cuộc bầu cử tổng thống Hoa Kỳ năm 1936, đối đầu giữa Landon và Roosevelt: The Literary Digest đã tiến hành một cuộc thăm dò rất lớn, gửi thư đến khoảng 10 triệu người. Nó nhận được 2,4 triệu câu trả lời, và dự đoán với độ tin cậy cao rằng Landon sẽ nhận được 57% số phiếu. Thay vào đó, Roosevelt đã giành chiến thắng với 62% số phiếu. Sai sót nằm ở phương pháp lấy mẫu của The Literary Digest: Đầu tiên, để có được địa chỉ gửi các cuộc thăm dò, The Literary Digest đã sử dụng danh bạ điện thoại, danh sách người đăng ký tạp chí, danh sách thành viên câu lạc bộ và những thứ tương tự. Tất cả các danh sách này có xu hướng ưu tiên những người giàu có hơn, những người có nhiều khả năng bỏ phiếu cho Đảng Cộng hòa (do đó là Landon). Thứ hai, chưa đến 25% số người được thăm dò đã trả lời. Điều này lại gây ra sai lệch lấy mẫu, bằng cách có khả năng loại trừ những người không quan tâm nhiều đến chính trị, những người không thích The Literary Digest và các nhóm quan trọng khác. Đây là một loại sai lệch lấy mẫu đặc biệt được gọi là sai lệch không phản hồi. Đây là một ví dụ khác: giả sử bạn muốn xây dựng một hệ thống để nhận dạng video nhạc funk. Một cách để xây dựng tập huấn luyện của bạn là tìm kiếm "nhạc funk" trên YouTube và sử dụng các video kết quả. Nhưng điều này giả định rằng công cụ tìm kiếm của YouTube trả về một tập hợp các video đại diện cho tất cả các video nhạc funk trên YouTube. Trong thực tế, kết quả tìm kiếm có khả năng bị sai lệch theo hướng các nghệ sĩ nổi tiếng (và nếu bạn sống ở Brazil, bạn sẽ nhận được rất nhiều video "funk carioca", không giống James Brown chút nào). Mặt khác, làm thế nào khác bạn có thể có một tập huấn luyện lớn?

Dữ liệu chất lượng kém

Rõ ràng, nếu dữ liệu huấn luyện của bạn đầy lỗi, giá trị ngoại lai và nhiễu (ví dụ: do đo lường kém chất lượng), nó sẽ khiến hệ thống khó phát hiện các mẫu cơ bản, do đó hệ thống của

CHƯƠNG 2. DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

Trong chương này, bạn sẽ thực hiện một dự án ví dụ từ đầu đến cuối, đóng vai một nhà khoa học dữ liệu mới được tuyển dụng tại một công ty bất động sản. Ví dụ này là hư cấu; mục tiêu là minh họa các bước chính của một dự án học máy, không phải để tìm hiểu bất cứ điều gì về ngành kinh doanh bất động sản. Dưới đây là các bước chính mà chúng ta sẽ thực hiện:

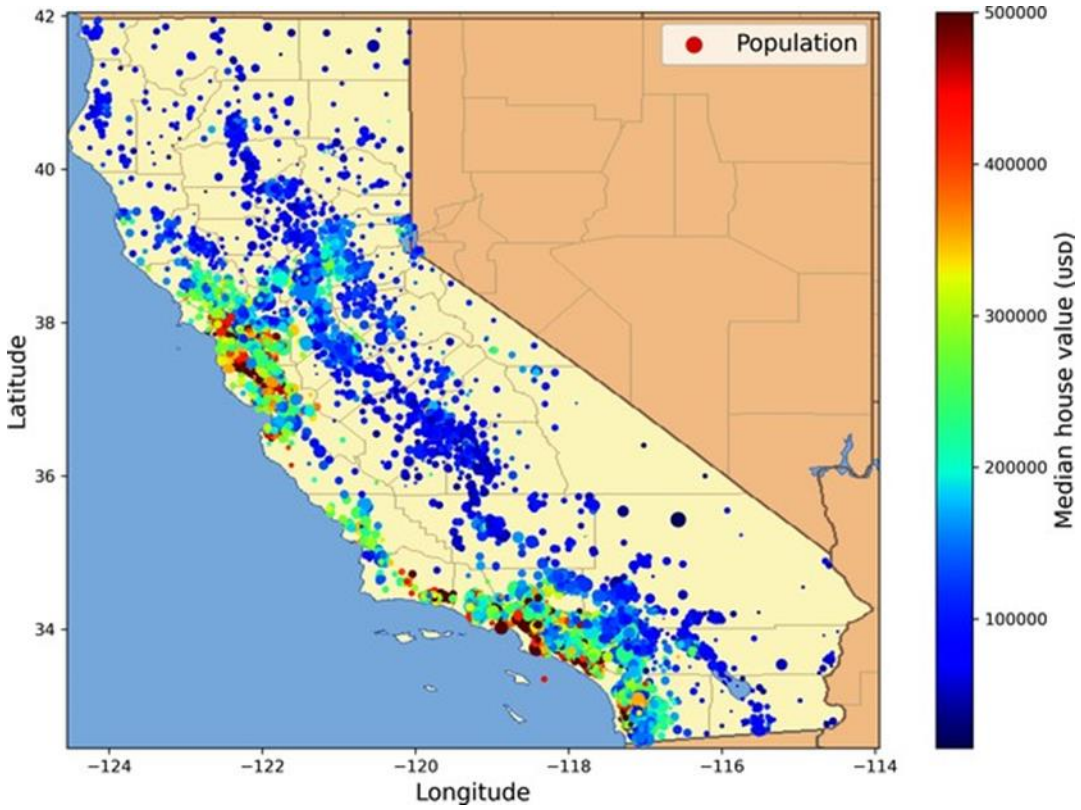
- Nhìn vào bức tranh lớn.
- Lấy dữ liệu.
- Khám phá và trực quan hóa dữ liệu để có được cái nhìn sâu sắc.
- Chuẩn bị dữ liệu cho các thuật toán học máy.
- Chọn một mô hình và huấn luyện nó.
- Tinh chỉnh mô hình của bạn.
- Trình bày giải pháp của bạn.
- Triển khai, giám sát và duy trì hệ thống của bạn.

2.1 Làm việc với dữ liệu thực

Khi bạn đang học về học máy, tốt nhất là nên thử nghiệm với dữ liệu thực tế, không phải tập dữ liệu nhân tạo. May mắn thay, có hàng nghìn tập dữ liệu mở để lựa chọn, trải rộng trên tất cả các loại lĩnh vực. Dưới đây là một vài nơi bạn có thể tìm để lấy dữ liệu:

- Kho lưu trữ dữ liệu mở phổ biến:
    - OpenML.org
    - Kaggle.com
    - PapersWithCode.com
    - UC Irvine Machine Learning Repository
    - Bộ dữ liệu AWS của Amazon
    - Bộ dữ liệu TensorFlow
- Các cổng thông tin Meta (chúng liệt kê các kho lưu trữ dữ liệu mở):
    - DataPortals.org
    - OpenDataMonitor.eu
- Các trang khác liệt kê nhiều kho lưu trữ dữ liệu mở phổ biến:
    - Danh sách các tập dữ liệu học máy của Wikipedia
    - Quora.com
    - Subreddit datasets

Trong chương này, chúng ta sẽ sử dụng tập dữ liệu Giá nhà California từ kho StatLib1 (xem Hình 2-1). Tập dữ liệu này dựa trên dữ liệu từ cuộc điều tra dân số California năm 1990. Nó không hoàn toàn mới (một ngôi nhà đẹp ở Vùng Vịnh vẫn có giá phải chăng vào thời điểm đó), nhưng nó có nhiều phẩm chất để học, vì vậy chúng ta sẽ giả vờ đó là dữ liệu gần đây. Với mục đích giảng dạy, tôi đã thêm một thuộc tính phân loại và loại bỏ một vài tính năng.



Hình 2-1. Giá nhà California

.2.2 Nhìn vào bức tranh lớn

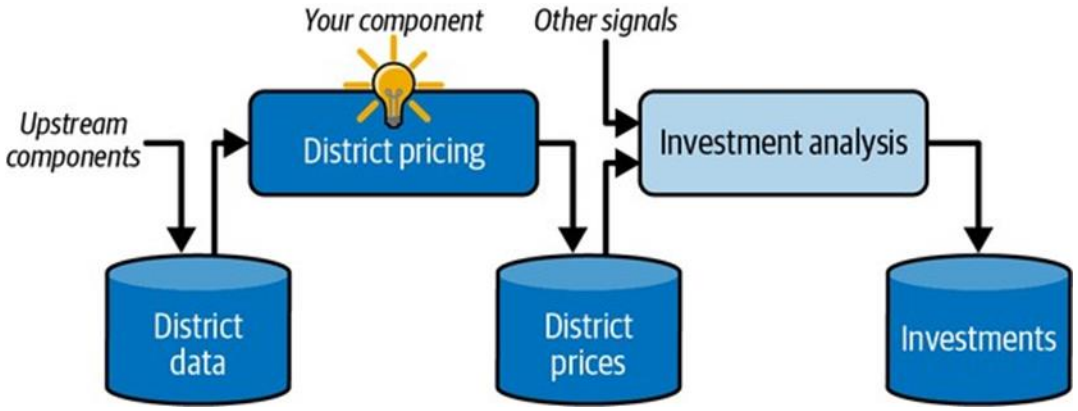
Chào mừng bạn đến với Công ty Học máy Bất động sản! Nhiệm vụ đầu tiên của bạn là sử dụng dữ liệu điều tra dân số California để xây dựng mô hình giá nhà ở tiểu bang. Dữ liệu này bao gồm các số liệu như dân số, thu nhập trung bình và giá nhà trung bình cho mỗi nhóm khu vực ở California. Nhóm khu vực là đơn vị địa lý nhỏ nhất mà Cục Thống kê Hoa Kỳ công bố dữ liệu mẫu (một nhóm khu vực thường có dân số từ 600 đến 3.000 người). Tôi sẽ gọi chúng là "quận" cho ngắn gọn.

Mô hình của bạn nên học từ dữ liệu này và có thể dự đoán giá nhà trung bình ở bất kỳ quận nào, với tất cả các số liệu khác.

2.2.1 Xác định vấn đề

Câu hỏi đầu tiên cần hỏi sếp của bạn là mục tiêu kinh doanh chính xác là gì. Xây dựng một mô hình có lẽ không phải là mục tiêu cuối cùng. Công ty mong đợi sử dụng và hưởng lợi từ mô hình này như thế nào? Biết mục tiêu là quan trọng vì nó sẽ quyết định cách bạn xác định vấn đề, thuật toán bạn sẽ chọn, thước đo hiệu suất bạn sẽ sử dụng để đánh giá mô hình của bạn và bạn sẽ dành bao nhiêu nỗ lực để điều chỉnh nó. Sếp của bạn trả lời rằng đầu ra của mô hình của bạn (dự đoán giá nhà trung bình của một quận) sẽ được đưa vào một hệ thống học máy khác (xem Hình 2-2), cùng với nhiều tín hiệu khác.2 Hệ thống hạ nguồn này sẽ xác định xem có đáng để đầu tư vào một khu vực nhất định hay không. Làm đúng điều

này là rất quan trọng, vì nó ảnh hưởng trực tiếp đến doanh thu. Câu hỏi tiếp theo cần hỏi sếp của bạn là giải pháp hiện tại trông như thế nào (nếu có). Tình hình hiện tại thường sẽ cung cấp cho bạn một tham chiếu về hiệu suất, cũng như những hiểu biết sâu sắc về cách giải quyết vấn đề. Sếp của bạn trả lời rằng giá nhà quận hiện đang được ước tính thủ công bởi các chuyên gia: một nhóm thu thập thông tin cập nhật về một quận, và khi họ không thể có được giá nhà trung bình, họ ước tính nó bằng các quy tắc phức tạp.



Hình 2-2. Một quy trình học máy cho đầu tư bất động sản

Điều này tốn kém và mất thời gian, và các ước tính của họ không tốt; trong những trường hợp họ tìm ra được giá nhà trung bình thực tế, họ thường nhận ra rằng ước tính của họ sai lệch hơn 30%. Đây là lý do tại sao công ty cho rằng việc huấn luyện một mô hình để dự đoán giá nhà trung bình của một quận, với dữ liệu khác về quận đó, sẽ hữu ích. Dữ liệu điều tra dân số có vẻ là một tập dữ liệu tuyệt vời để khai thác cho mục đích này, vì nó bao gồm giá nhà trung bình của hàng nghìn quận, cũng như các dữ liệu khác.

Đường ống (Pipelines)

Một chuỗi các thành phần xử lý dữ liệu được gọi là đường ống dữ liệu. Đường ống rất phổ biến trong các hệ thống học máy, vì có rất nhiều dữ liệu để thao tác và nhiều phép biến đổi dữ liệu cần áp dụng. Các thành phần thường chạy không đồng bộ. Mỗi thành phần kéo một lượng lớn dữ liệu vào, xử lý nó và xuất kết quả vào một kho dữ liệu khác. Sau đó, một thời gian sau, thành phần tiếp theo trong đường ống kéo dữ liệu này vào và xuất ra đầu ra của riêng nó. Mỗi thành phần khá khép kín: giao diện giữa các thành phần chỉ đơn giản là kho dữ liệu. Điều này làm cho hệ thống dễ nắm bắt (với sự trợ giúp của biểu đồ luồng dữ liệu), và các nhóm khác nhau có thể tập trung vào các thành phần khác nhau. Hơn nữa, nếu một thành phần bị hỏng, các thành phần hạ nguồn thường có thể tiếp tục chạy bình thường (ít nhất là trong một thời gian) bằng cách chỉ sử dụng đầu ra cuối cùng từ thành phần bị hỏng. Điều này làm cho kiến trúc khá mạnh mẽ.

Mặt khác, một thành phần bị hỏng có thể không được chú ý trong một thời gian nếu không có giám sát phù hợp. Dữ liệu trở nên cũ và hiệu suất tổng thể của hệ thống giảm.

Với tất cả thông tin này, bây giờ bạn đã sẵn sàng bắt đầu thiết kế hệ thống của mình. Đầu tiên, xác định loại giám sát huấn luyện mà mô hình sẽ cần: đó là tác vụ học có giám sát,

không giám sát, bán giám sát, tự giám sát hay học tăng cường? Và đó là tác vụ phân loại, tác vụ hồi quy, hay một thứ khác? Bạn nên sử dụng kỹ thuật học theo lô hay học trực tuyến? Trước khi bạn đọc tiếp, hãy tạm dừng và tự mình trả lời những câu hỏi này.

Bạn đã tìm ra câu trả lời chưa? Hãy xem. Đây rõ ràng là một tác vụ học có giám sát điển hình, vì mô hình có thể được huấn luyện bằng các ví dụ có nhãn (mỗi trường hợp đi kèm với đầu ra mong muốn, tức là giá nhà trung bình của quận). Đây là một tác vụ hồi quy điển hình, vì mô hình sẽ được yêu cầu dự đoán một giá trị. Cụ thể hơn, đây là một vấn đề hồi quy đa biến, vì hệ thống sẽ sử dụng nhiều đặc trưng để đưa ra dự đoán (dân số của quận, thu nhập trung bình, v.v.). Nó cũng là một vấn đề hồi quy đơn biến, vì chúng ta chỉ cố gắng dự đoán một giá trị duy nhất cho mỗi quận. Nếu chúng ta cố gắng dự đoán nhiều giá trị cho mỗi quận, đó sẽ là một vấn đề hồi quy đa biến. Cuối cùng, không có luồng dữ liệu liên tục đổ vào hệ thống, không có nhu cầu đặc biệt nào để điều chỉnh dữ liệu thay đổi nhanh chóng, và dữ liệu đủ nhỏ để vừa trong bộ nhớ, vì vậy học theo lô đơn giản sẽ ổn.

2.2.2 Chọn một Thước đo Hiệu suất

Bước tiếp theo của bạn là chọn một thước đo hiệu suất. Một thước đo hiệu suất điển hình cho các bài toán hồi quy là **sai số toàn phương trung bình (RMSE)**. Nó cho biết mức độ lỗi mà hệ thống thường mắc phải trong các dự đoán của mình, với trọng số cao hơn được đặt cho các sai số lớn. Công thức 2-1 trình bày công thức toán học để tính RMSE.

Công thức 2-1. Sai số toàn phương trung bình (RMSE)

$$RMSE(X,h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(h(x^{(i)}) - y^{(i)})^2}$$

CÁC KÝ HIỆU

Phần này giới thiệu một số ký hiệu rất phổ biến trong học máy mà tôi sẽ sử dụng xuyên suốt cuốn sách này:

- **m** là **số lượng mẫu** trong tập dữ liệu mà bạn đang đo lường.
    - Ví dụ, nếu bạn đang đánh giá trên một tập kiểm định gồm 2.000 quận, thì **m = 2.000**.
- $\mathbf{x^{(i)}}$ là một **vector** chứa tất cả các giá trị đặc trưng (không bao gồm nhãn) của mẫu thứ *i* trong tập dữ liệu, và $\mathbf{y^{(i)}}$ là **nhãn** của nó (giá trị đầu ra mong muốn cho mẫu đó).
    - Ví dụ, nếu quận đầu tiên trong tập dữ liệu có kinh độ –118.29°, vĩ độ 33.91°, có 1.416 cư dân với thu nhập trung vị là $38.372, và giá nhà trung vị là $156.400 (tạm bỏ qua các đặc trưng khác), thì:

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118.29 \\ 33.91 \\ 1,416 \\ 38,372 \end{pmatrix} \quad \text{và} \quad y^{(1)} = 156,400$$

- **X** là một **ma trận** chứa tất cả các giá trị đặc trưng (không bao gồm nhãn) của tất cả các mẫu trong tập dữ liệu. Mỗi hàng tương ứng với một mẫu, và hàng thứ *i* bằng với ma trận chuyển vị của $\mathbf{x}^{(i)}$, ký hiệu là $(\mathbf{x}^{(i)})^T$.
    - Ví dụ, nếu quận đầu tiên như mô tả ở trên, thì ma trận **X** sẽ trông như sau:

$$\mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^T \\ (\mathbf{x}^{(2)})^T \\ \vdots \\ (\mathbf{x}^{(1999)})^T \\ (\mathbf{x}^{(2000)})^T \end{pmatrix} = \begin{pmatrix} -118.29 & 33.91 & 1,416 & 38,372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

- **h** là **hàm dự đoán** của hệ thống, còn được gọi là một **giả thuyết** (hypothesis). Khi hệ thống của bạn nhận một vector đặc trưng của một mẫu là $\mathbf{x}^{(i)}$, nó sẽ đưa ra một giá trị dự đoán $\hat{\mathbf{y}}^{(i)}$ cho mẫu đó ($\hat{y}$ được phát âm là "y-mũ").
    - Ví dụ, nếu hệ thống của bạn dự đoán rằng giá nhà trung vị ở quận đầu tiên là \$158.400, thì $\hat{\mathbf{y}}^{(1)} = \mathbf{h}(\mathbf{x}^{(1)}) = \mathbf{158.400}$. Sai số dự đoán cho quận này là $\hat{\mathbf{y}}^{(1)} - \mathbf{y}^{(1)} = \mathbf{2.000}$.
- **RMSE(X, h)** là **hàm chi phí** được đo lường trên tập hợp các ví dụ bằng cách sử dụng giả thuyết **h** của bạn.

Chúng tôi sử dụng chữ in nghiêng viết thường cho các giá trị vô hướng (chẳng hạn như m hoặc $y^{(i)}$) và tên hàm (chẳng hạn như h), chữ in đậm viết thường cho các vector (chẳng hạn như $x^{(i)}$), và chữ in đậm viết hoa cho các ma trận (chẳng hạn như X).

Mặc dù RMSE thường là thước đo hiệu suất được ưa chuộng cho các tác vụ hồi quy, trong một số bối cảnh, bạn có thể muốn sử dụng một hàm khác. Ví dụ, nếu có nhiều quận là **giá trị ngoại lai (outlier)**. Trong trường hợp đó, bạn có thể cân nhắc sử dụng **sai số tuyệt đối trung bình (MAE)**, còn được gọi là **độ lệch tuyệt đối trung bình**, được trình bày trong Công thức 2-2:

Công thức 2-2. Sai số tuyệt đối trung bình (MAE)

$$MAE(X, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(x^{(i)}\right) - y^{(i)}\right|$$

Cả RMSE và MAE đều là các cách để đo **khoảng cách** giữa hai vector: vector các giá trị dự đoán và vector các giá trị mục tiêu. Có nhiều cách đo khoảng cách, hay còn gọi là **chuẩn (norm)**, khác nhau:

- Việc tính căn bậc hai của tổng các bình phương (RMSE) tương ứng với **chuẩn Euclid**: đây là khái niệm khoảng cách quen thuộc với tất cả chúng ta. Nó còn được gọi là **chuẩn** ℓ_2 , ký hiệu là $\|\cdot\|_2$ (hoặc chỉ là $\|\cdot\|$).
- Việc tính tổng các giá trị tuyệt đối (MAE) tương ứng với **chuẩn** ℓ_1 , ký hiệu là $\|\cdot\|_1$. Đôi khi nó được gọi là **chuẩn Manhattan** vì nó đo khoảng cách giữa hai điểm trong một thành phố nếu bạn chỉ có thể đi dọc theo các dãy nhà vuông góc với nhau.
- Tổng quát hơn, **chuẩn** ℓ_k của một vector v chứa n phần tử được định nghĩa là $\| v \|_k = (| v_1 |^k + | v_2 |^k + \cdots + | v_n |^k)^{1/k}$. ℓ_0 cho biết số lượng phần tử khác không trong vector, và ℓ_∞ cho biết giá trị tuyệt đối lớn nhất trong vector.

Chỉ số chuẩn càng cao, nó càng tập trung vào các giá trị lớn và bỏ qua các giá trị nhỏ. Đây là lý do tại sao RMSE nhạy cảm với các giá trị ngoại lai hơn MAE. Nhưng khi các giá trị ngoại lai hiếm theo cấp số nhân (như trong đường cong hình chuông), RMSE hoạt động rất tốt và thường được ưu tiên sử dụng.

2.2.3 Kiểm tra các Giả định

Cuối cùng, một thông lệ tốt là liệt kê và xác minh các giả định đã được đưa ra (bởi bạn hoặc những người khác); điều này có thể giúp bạn phát hiện sớm các vấn đề nghiêm trọng.

Ví dụ, giá của các quận mà hệ thống của bạn đưa ra sẽ được cung cấp cho một hệ thống máy học ở khâu sau, và bạn giả định rằng những mức giá này sẽ được sử dụng nguyên trạng. Nhưng điều gì sẽ xảy ra nếu hệ thống ở khâu sau chuyển đổi các mức giá này thành các hạng mục (ví dụ: "rẻ", "trung bình" hoặc "đắt") và sau đó sử dụng các hạng mục đó thay vì chính các mức giá? Trong trường hợp này, việc dự đoán giá một cách chính xác hoàn toàn không còn quan trọng; hệ thống của bạn chỉ cần xác định đúng hạng mục. Nếu vậy, thì bài toán lẽ ra nên được xác định là một **bài toán phân loại**, chứ không phải là một **bài toán hồi quy**. Bạn sẽ không muốn phát hiện ra điều này sau nhiều tháng làm việc trên một hệ thống hồi quy.

May mắn thay, sau khi nói chuyện với đội ngũ phụ trách hệ thống ở khâu sau, bạn tin chắc rằng họ thực sự cần các mức giá thực tế chứ không chỉ là các hạng mục. Tuyệt vời! Mọi thứ đã sẵn sàng, đèn xanh đã bật, và bạn có thể bắt đầu lập trình ngay bây giờ!

2.3 Lấy Dữ liệu

Đã đến lúc bắt tay vào việc.

Đừng ngần ngại lấy máy tính xách tay của bạn và thực hành theo các ví dụ về mã nguồn. Như tôi đã đề cập trong lời nói đầu, tất cả các ví dụ về mã trong cuốn sách này đều là **mã nguồn mở** và có sẵn trực tuyến dưới dạng **Jupyter notebook**, là các tài liệu tương tác chứa văn bản, hình ảnh và các đoạn mã có thể thực thi (trong trường hợp của chúng ta là Python). Trong cuốn sách này, tôi sẽ giả định rằng bạn đang chạy các notebook này trên **Google Colab**, một dịch vụ miễn phí cho phép bạn chạy bất kỳ Jupyter notebook nào trực tiếp trên trình duyệt mà không cần phải cài đặt bất cứ thứ gì trên máy của mình. Nếu bạn muốn sử dụng một nền tảng trực tuyến khác (ví dụ: Kaggle) hoặc nếu bạn muốn cài đặt mọi

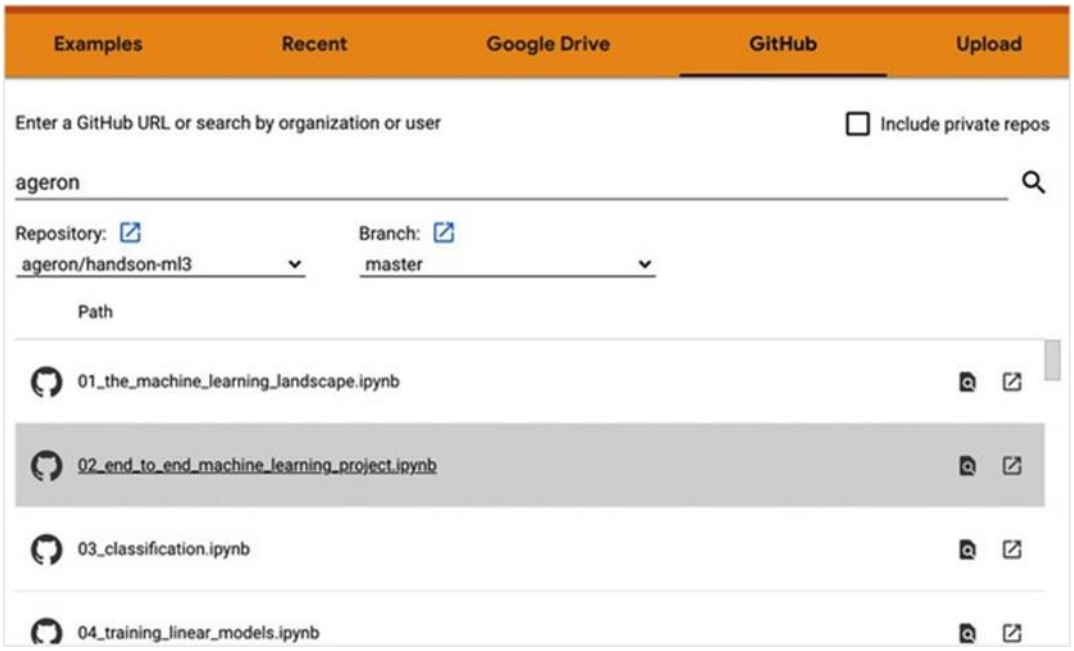
thứ cục bộ trên máy của riêng mình, vui lòng xem hướng dẫn trên trang GitHub của cuốn sách.

Chạy các Ví dụ về Mã nguồn bằng Google Colab

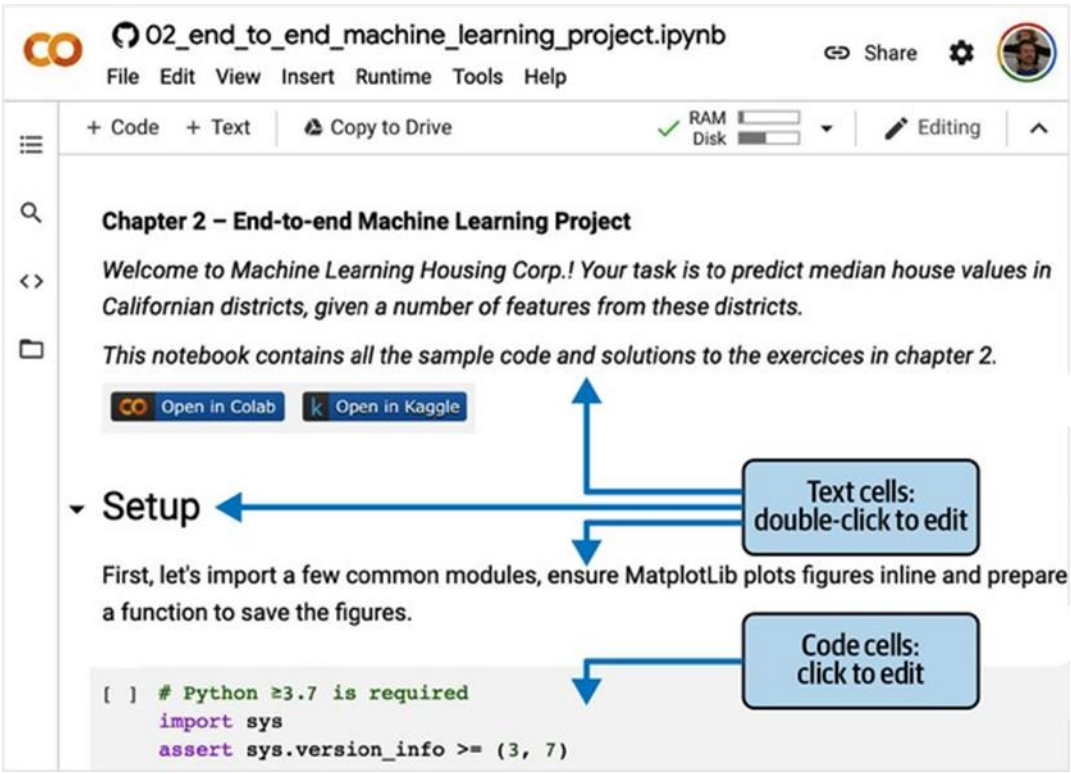
Đầu tiên, hãy mở một trình duyệt web và truy cập https://homl.info/colab3 : đường link này sẽ dẫn bạn đến Google Colab và hiển thị danh sách các Jupyter notebook cho cuốn sách này (xem Hình 2-3).

Bạn sẽ tìm thấy một notebook cho mỗi chương, cộng với một vài notebook và bài hướng dẫn bổ sung về NumPy, Matplotlib, Pandas, đại số tuyến tính và vi tích phân. Ví dụ, nếu bạn nhấp vào `02_end_to_end_machine_learning_project.ipynb`, notebook của Chương 2 sẽ được mở ra trong Google Colab (xem Hình 2-4).

Một Jupyter notebook bao gồm một danh sách các **ô (cell)**. Mỗi ô chứa mã có thể thực thi hoặc văn bản. Hãy thử nhấp đúp vào ô văn bản đầu tiên (chứa câu "Welcome to Machine Learning Housing Corp.!"). Thao tác này sẽ mở ô đó ra để chỉnh sửa. Lưu ý rằng Jupyter notebook sử dụng cú pháp **Markdown** để định dạng (ví dụ: **chữ đậm**, *chữ nghiêng*, `# Tiêu đề`, `[url](văn bản liên kết)`, v.v.). Hãy thử sửa đổi văn bản này, sau đó nhấn **Shift-Enter** để xem kết quả.



Hình 2-3. Danh sách các notebook trong Google Colab



Hình 2-4. Notebook của bạn trong Google Colab

Tiếp theo, hãy tạo một ô mã mới bằng cách chọn **Insert → "Code cell"** từ menu. Ngoài ra, bạn có thể nhấp vào nút **+ Code** trên thanh công cụ, hoặc di chuột qua cuối một ô cho đến khi bạn thấy **+ Code** và **+ Text** xuất hiện, sau đó nhấp vào **+ Code**. Trong ô mã mới, hãy gõ một đoạn mã Python, chẳng hạn như `print("Hello World")`, sau đó nhấn **Shift-Enter** để chạy mã này (hoặc nhấp vào nút ▶ ở bên trái của ô).

Nếu bạn chưa đăng nhập vào tài khoản Google của mình, bạn sẽ được yêu cầu đăng nhập ngay bây giờ (nếu bạn chưa có tài khoản Google, bạn sẽ cần tạo một tài khoản). Sau khi đăng nhập, khi bạn cố gắng chạy mã, bạn sẽ thấy một cảnh báo bảo mật cho biết rằng notebook này không phải do Google tạo ra. Một người có ý đồ xấu có thể tạo ra một notebook để lừa bạn nhập thông tin đăng nhập Google của mình để họ có thể truy cập dữ liệu cá nhân của bạn, vì vậy trước khi chạy một notebook, **hãy luôn đảm bảo rằng bạn tin tưởng tác giả của nó** (hoặc kiểm tra kỹ xem mỗi ô mã sẽ làm gì trước khi chạy). Giả sử bạn tin tưởng tôi (hoặc bạn dự định kiểm tra mọi ô mã), bây giờ bạn có thể nhấp vào "Run anyway".

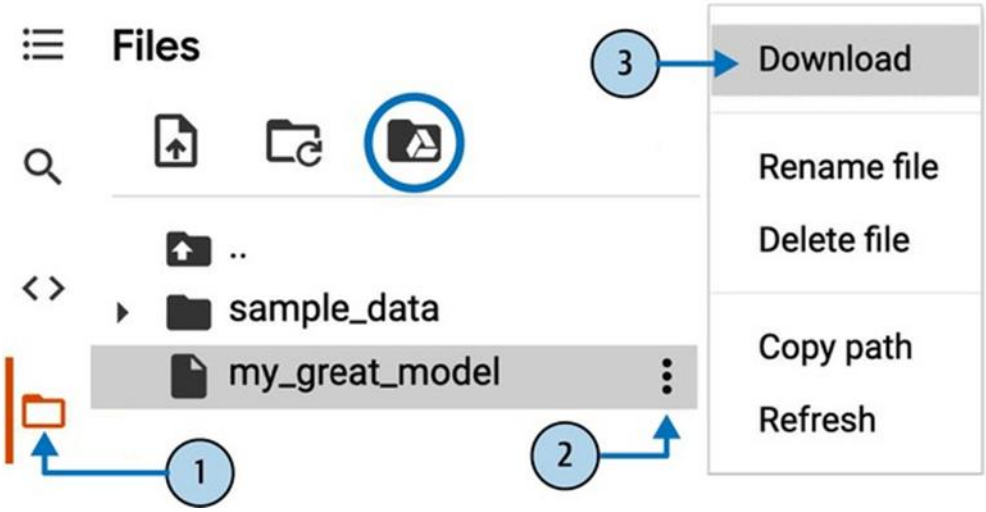
Colab sau đó sẽ cấp phát một **môi trường thực thi (runtime)** mới cho bạn: đây là một **máy ảo** miễn phí đặt trên máy chủ của Google, chứa một loạt các công cụ và thư viện Python, bao gồm mọi thứ bạn cần cho hầu hết các chương (trong một số chương, bạn sẽ cần chạy một lệnh để cài đặt các thư viện bổ sung). Quá trình này sẽ mất vài giây. Tiếp theo, Colab sẽ tự động kết nối với môi trường thực thi này và sử dụng nó để thực thi ô mã mới của bạn. Điều quan trọng là, mã chạy trên môi trường thực thi chứ không phải trên máy của bạn.

Đầu ra của mã sẽ được hiển thị bên dưới ô. Chúc mừng, bạn đã chạy thành công một đoạn mã Python trên Colab! 🥳

Lưu các Thay đổi Mã nguồn và Dữ liệu của bạn

Bạn có thể thay đổi một notebook Colab, và các thay đổi đó sẽ được duy trì miễn là bạn giữ tab trình duyệt mở. Nhưng một khi bạn đóng nó, các thay đổi sẽ bị mất. Để tránh điều này, hãy đảm bảo bạn lưu một bản sao của notebook vào Google Drive của mình bằng cách chọn **File → "Save a copy in Drive"**. Ngoài ra, bạn có thể tải notebook về máy tính của mình bằng cách chọn **File → Download → "Download .ipynb"**. Sau đó, bạn có thể truy cập lại https://colab.research.google.com và mở notebook đó lên (từ Google Drive hoặc bằng cách tải lên từ máy tính của bạn).

Nếu notebook tạo ra dữ liệu mà bạn quan tâm, hãy chắc chắn rằng bạn đã tải dữ liệu này về trước khi môi trường thực thi tắt. Để làm điều này, hãy nhấp vào biểu tượng **Files** (xem bước 1 trong Hình 2-5), tìm tệp bạn muốn tải xuống, nhấp vào dấu ba chấm dọc bên cạnh nó (bước 2), và nhấp vào **Download** (bước 3). Ngoài ra, bạn có thể **gắn (mount)** Google Drive của mình vào môi trường thực thi, cho phép notebook đọc và ghi tệp trực tiếp vào Google Drive như thể đó là một thư mục cục bộ. Để làm điều này, hãy nhấp vào biểu tượng **Files** (bước 1), sau đó nhấp vào biểu tượng **Google Drive** (được khoanh tròn trong Hình 2-5) và làm theo hướng dẫn trên màn hình.



Hình 2-5. Tải xuống một tệp từ môi trường thực thi Google Colab (các bước từ 1 đến 3), hoặc gắn Google Drive của bạn (biểu tượng được khoanh tròn)

Theo mặc định, Google Drive của bạn sẽ được gắn tại `/content/drive/MyDrive`. Nếu bạn muốn sao lưu một tệp dữ liệu, chỉ cần sao chép nó vào thư mục này bằng cách chạy `!cp /content/my_great_model /content/drive/MyDrive`. Bất kỳ lệnh nào bắt đầu bằng dấu chấm than (`!`) đều được coi là một **lệnh shell**, không phải là mã Python: `cp` là lệnh shell của

Linux để sao chép một tệp từ đường dẫn này sang đường dẫn khác. Lưu ý rằng các môi trường thực thi của Colab chạy trên Linux (cụ thể là Ubuntu).

Sức mạnh và Mối nguy của Tính tương tác

Jupyter notebook có tính tương tác, và đó là một điều tuyệt vời: bạn có thể chạy từng ô một, dừng lại ở bất kỳ điểm nào, chèn một ô, thử nghiệm với mã, quay lại và chạy lại cùng một ô, v.v., và tôi rất khuyến khích bạn làm như vậy. Nếu bạn chỉ chạy từng ô một mà không bao giờ thử nghiệm với chúng, bạn sẽ không học nhanh được. Tuy nhiên, sự linh hoạt này đi kèm với một cái giá: rất dễ **chạy các ô sai thứ tự**, hoặc **quên chạy một ô nào đó**. Nếu điều này xảy ra, các ô mã tiếp theo có khả năng sẽ bị lỗi. Ví dụ, ô mã đầu tiên trong mỗi notebook chứa mã thiết lập (chẳng hạn như các lệnh import), vì vậy hãy đảm bảo bạn chạy nó trước tiên, nếu không sẽ không có gì hoạt động.

Mã trong Sách và Mã trong Notebook

Đôi khi bạn có thể nhận thấy một số khác biệt nhỏ giữa mã trong cuốn sách này và mã trong các notebook. Điều này có thể xảy ra vì nhiều lý do:

- Một **thư viện có thể đã thay đổi** một chút vào thời điểm bạn đọc những dòng này, hoặc có lẽ dù đã cố gắng hết sức tôi vẫn mắc lỗi trong sách. Đáng buồn là, tôi không thể sửa mã trong bản sách của bạn một cách thần kỳ (trừ khi bạn đang đọc bản điện tử và có thể tải về phiên bản mới nhất), nhưng tôi có thể sửa các notebook. Vì vậy, nếu bạn gặp lỗi sau khi sao chép mã từ cuốn sách này, vui lòng tìm mã đã được sửa trong các notebook: tôi sẽ cố gắng giữ chúng không có lỗi và cập nhật với các phiên bản thư viện mới nhất.
- Các notebook chứa một số **mã bổ sung để làm đẹp các biểu đồ** (thêm nhãn, đặt kích thước phông chữ, v.v.) và để lưu chúng ở độ phân giải cao cho cuốn sách này. Bạn có thể yên tâm bỏ qua phần mã bổ sung này nếu muốn.
- Tôi đã **tối ưu hóa mã cho tính dễ đọc và đơn giản**: Tôi đã làm cho nó tuyến tính và phẳng nhất có thể, định nghĩa rất ít hàm hoặc lớp. Mục tiêu là đảm bảo rằng mã bạn đang chạy thường ở ngay trước mắt bạn, và không bị lồng trong nhiều lớp trừu tượng mà bạn phải tìm kiếm. Điều này cũng giúp bạn dễ dàng thử nghiệm với mã hơn. Để đơn giản, có **ít xử lý lỗi**, và tôi đã đặt một số lệnh import ít phổ biến ngay tại nơi chúng cần thiết (thay vì đặt chúng ở đầu tệp, như được khuyến nghị bởi hướng dẫn về phong cách Python PEP 8). Nói như vậy, mã nguồn cho sản phẩm thực tế của bạn sẽ không khác nhiều: chỉ cần module hóa hơn một chút, và có thêm các bài kiểm thử và xử lý lỗi.

OK! Một khi bạn đã thoải mái với Colab, bạn đã sẵn sàng để tải xuống dữ liệu.

2.3.1 Tải Dữ liệu

Trong các môi trường điển hình, dữ liệu của bạn sẽ có sẵn trong một cơ sở dữ liệu quan hệ hoặc một kho dữ liệu phổ biến nào đó, và được phân tán trên nhiều bảng/tài liệu/tệp. Để truy cập nó, trước tiên bạn sẽ cần lấy thông tin xác thực và quyền truy cập và làm quen với lược đồ dữ liệu. Tuy nhiên, trong dự án này, mọi thứ đơn giản hơn nhiều: bạn sẽ chỉ cần tải xuống một tệp nén duy nhất, `housing.tgz`, chứa một tệp giá trị được phân tách bằng dấu phẩy (CSV) có tên là `housing.csv` với tất cả dữ liệu.

Thay vì tải xuống và giải nén dữ liệu theo cách thủ công, thường thì tốt hơn là **viết một hàm để làm việc đó cho bạn**. Điều này đặc biệt hữu ích nếu dữ liệu thay đổi thường xuyên: bạn có thể viết một kịch bản nhỏ sử dụng hàm để lấy dữ liệu mới nhất (hoặc bạn có thể thiết lập một công việc theo lịch trình để tự động làm điều đó vào các khoảng thời gian đều đặn). Tự động hóa quá trình lấy dữ liệu cũng hữu ích nếu bạn cần cài đặt tập dữ liệu trên nhiều máy.

Đây là hàm để lấy và tải dữ liệu:

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing = load_housing_data()
```

Khi `load_housing_data()` được gọi, nó sẽ tìm tệp `datasets/housing.tgz`. Nếu không tìm thấy, nó sẽ tạo thư mục `datasets` bên trong thư mục hiện tại (mặc định là `/content` trong Colab), tải tệp `housing.tgz` từ kho lưu trữ GitHub `ageron/data`, và giải nén nội dung của nó vào thư mục `datasets`; thao tác này sẽ tạo ra thư mục `datasets/housing` chứa tệp `housing.csv` bên trong. Cuối cùng, hàm này tải tệp CSV đó vào một đối tượng **DataFrame của Pandas** chứa tất cả dữ liệu, và trả về nó.

2.3.2 Xem nhanh Cấu trúc Dữ liệu

Bạn bắt đầu bằng cách xem năm hàng dữ liệu đầu tiên bằng phương thức `head()` của DataFrame (xem Hình 2-6).

```
housing.head()
```

	longitude	latitude	housing_median_age	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41.0	8.3252	NEAR BAY	452600.0
1	-122.22	37.86	21.0	8.3014	NEAR BAY	358500.0
2	-122.24	37.85	52.0	7.2574	NEAR BAY	352100.0
3	-122.25	37.85	52.0	5.6431	NEAR BAY	341300.0
4	-122.25	37.85	52.0	3.8462	NEAR BAY	342200.0

Hình 2-6. Năm hàng đầu tiên trong tập dữ liệu

Mỗi hàng đại diện cho một quận. Có 10 thuộc tính (không phải tất cả đều được hiển thị trong ảnh chụp màn hình): `longitude`, `latitude`, `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`, `median_house_value`, và `ocean_proximity`.

Phương thức `info()` rất hữu ích để có được một mô tả nhanh về dữ liệu, đặc biệt là tổng số hàng, kiểu của mỗi thuộc tính và số lượng giá trị không rỗng (non-null):

```
>>> housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

Có 20.640 mẫu trong tập dữ liệu, có nghĩa là nó khá nhỏ theo tiêu chuẩn học máy, nhưng nó hoàn hảo để bắt đầu. Bạn nhận thấy rằng thuộc tính `total_bedrooms` chỉ có 20.433 giá trị không rỗng, nghĩa là có 207 quận bị thiếu đặc trưng này. Bạn sẽ cần phải xử lý vấn đề này sau.

Tất cả các thuộc tính đều là số, ngoại trừ `ocean_proximity`. Kiểu của nó là `object`, vì vậy nó có thể chứa bất kỳ loại đối tượng Python nào. Nhưng vì bạn đã tải dữ liệu này từ một tệp CSV, bạn biết rằng nó phải là một thuộc tính văn bản. Khi bạn xem năm hàng đầu tiên, bạn có thể đã nhận thấy rằng các giá trị trong cột `ocean_proximity` lặp đi lặp lại, điều đó có nghĩa là nó có thể là một **thuộc tính phân loại (categorical attribute)**. Bạn có thể tìm ra

các loại (category) tồn tại và có bao nhiêu quận thuộc về mỗi loại bằng cách sử dụng phương thức `value_counts()`:

```
>>> housing["ocean_proximity"].value_counts()
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: ocean_proximity, dtype: int64
```

Hãy xem các trường khác. Phương thức `describe()` hiển thị một bản tóm tắt các thuộc tính số (Hình 2-7).

```
housing.describe()
```

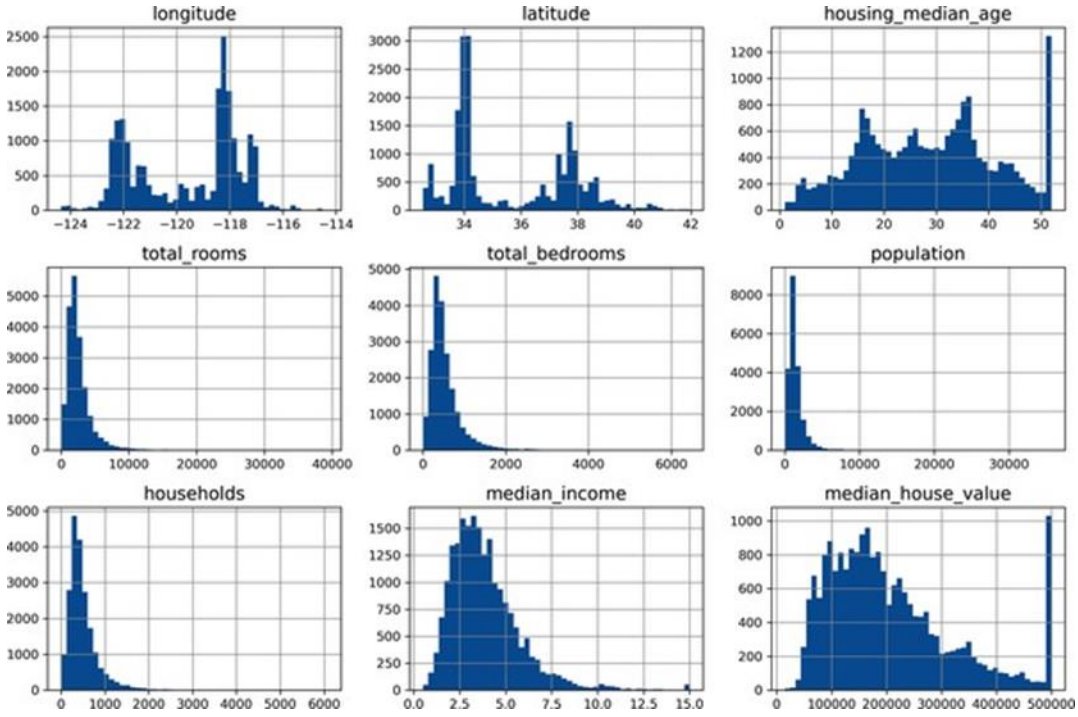
	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	500001.000000

Hình 2-7. Tóm tắt của từng thuộc tính số

Các hàng `count`, `mean`, `min`, và `max` khá dễ hiểu. Lưu ý rằng các giá trị rỗng bị bỏ qua (vì vậy, ví dụ, `count` của `total_bedrooms` là 20.433 chứ không phải 20.640). Hàng `std` hiển thị **độ lệch chuẩn**, đo lường mức độ phân tán của các giá trị. Các hàng `25%`, `50%`, và `75%` hiển thị các **phân vị (percentile)** tương ứng: một phân vị cho biết giá trị mà dưới nó có một tỷ lệ phần trăm nhất định các quan sát trong một nhóm quan sát. Ví dụ, 25% các quận có `housing_median_age` thấp hơn 18, trong khi 50% thấp hơn 29 và 75% thấp hơn 37. Chúng thường được gọi là **phân vị thứ 25** (hoặc **tứ phân vị thứ nhất**), **trung vị**, và **phân vị thứ 75** (hoặc **tứ phân vị thứ ba**).

Một cách nhanh chóng khác để có cảm nhận về loại dữ liệu bạn đang xử lý là vẽ một **biểu đồ tần suất (histogram)** cho mỗi thuộc tính số. Biểu đồ tần suất hiển thị số lượng mẫu (trên trục tung) có một phạm vi giá trị nhất định (trên trục hoành). Bạn có thể vẽ biểu đồ này cho từng thuộc tính một, hoặc bạn có thể gọi phương thức `hist()` trên toàn bộ tập dữ liệu (như được hiển thị trong ví dụ mã sau), và nó sẽ vẽ một biểu đồ tần suất cho mỗi thuộc tính số (xem Hình 2-8):

```
import matplotlib.pyplot as plt

housing.hist(bins=50, figsize=(12, 8))
plt.show()
```



Hình 2-8. Một biểu đồ tần suất cho mỗi thuộc tính số

Nhìn vào các biểu đồ tần suất này, bạn nhận thấy một vài điều:

- Trước hết, thuộc tính **thu nhập trung vị (`median_income`)** có vẻ không được biểu thị bằng đô la Mỹ (USD). Sau khi kiểm tra với đội ngũ đã thu thập dữ liệu, bạn được cho biết rằng dữ liệu đã được **co giãn (scaled)** và **giới hạn trên (capped)** ở mức 15 (thực ra là 15.0001) cho các mức thu nhập trung vị cao hơn, và giới hạn dưới ở mức 0.5 (thực ra là 0.4999) cho các mức thu nhập trung vị thấp hơn. Các con số đại diện cho khoảng hàng chục nghìn đô la (ví dụ: 3 thực sự có nghĩa là khoảng $30.000). Làm việc với các thuộc tính đã được tiền xử lý là phổ biến trong học máy, và không nhất thiết là một vấn đề, nhưng bạn nên cố gắng hiểu dữ liệu đã được tính toán như thế nào.
- **Tuổi trung vị của nhà (`housing_median_age`)** và **giá trị nhà trung vị (`median_house_value`)** cũng bị giới hạn trên. Điều sau có thể là một vấn đề nghiêm trọng vì nó là thuộc tính mục tiêu của bạn (nhãn của bạn). Các thuật toán học máy của bạn có thể học rằng giá không bao giờ vượt quá giới hạn đó. Bạn cần kiểm tra với nhóm khách hàng của mình (nhóm sẽ sử dụng đầu ra của hệ thống của bạn) để xem đây có phải là vấn đề hay không. Nếu họ nói rằng họ cần những dự đoán chính xác ngay cả khi vượt quá $500.000, thì bạn có hai lựa chọn:
    - Thu thập các nhãn phù hợp cho các quận có nhãn bị giới hạn trên.
    - Loại bỏ các quận đó khỏi tập huấn luyện (và cả tập kiểm tra, vì hệ thống của bạn không nên bị đánh giá kém nếu nó dự đoán các giá trị vượt quá $500.000).

- Các thuộc tính này có **thang đo rất khác nhau**. Chúng ta sẽ thảo luận về điều này sau trong chương này, khi chúng ta khám phá về co giãn đặc trưng (feature scaling).
- Cuối cùng, nhiều biểu đồ tần suất bị **lệch phải (skewed right)**: chúng kéo dài về phía bên phải của trung vị xa hơn nhiều so với bên trái. Điều này có thể làm cho một số thuật toán học máy khó phát hiện các mẫu hơn một chút. Sau này, bạn sẽ thử biến đổi các thuộc tính này để có các phân phối đối xứng hơn và có dạng hình chuông.

Bây giờ bạn đã có một sự hiểu biết tốt hơn về loại dữ liệu mà bạn đang làm việc.

2.3.3 Tạo tập kiểm thử

Có vẻ lạ khi tự nguyện gạt một phần dữ liệu sang một bên ở giai đoạn này. Rốt cuộc, bạn mới chỉ xem lướt qua dữ liệu, và chắc chắn bạn nên tìm hiểu nhiều hơn về nó trước khi quyết định sử dụng thuật toán nào, phải không? Điều này đúng, nhưng bộ não của bạn là một hệ thống phát hiện mẫu tuyệt vời, điều đó cũng có nghĩa là nó rất dễ bị quá khớp: nếu bạn nhìn vào tập kiểm thử, bạn có thể tình cờ tìm thấy một số mẫu dường như thú vị trong dữ liệu kiểm thử, dẫn bạn đến việc chọn một loại mô hình học máy cụ thể. Khi bạn ước tính lỗi tổng quát hóa bằng cách sử dụng tập kiểm thử, ước tính của bạn sẽ quá lạc quan, và bạn sẽ triển khai một hệ thống không hoạt động tốt như mong đợi. Điều này được gọi là sai lệch do rò rỉ dữ liệu (data snooping bias). Tạo một tập kiểm thử về mặt lý thuyết rất đơn giản; chọn ngẫu nhiên một số trường hợp, thường là 20% tập dữ liệu (hoặc ít hơn nếu tập dữ liệu của bạn rất lớn), và đặt chúng sang một bên:

```
import numpy as np

def shuffle_and_split_data(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

Bạn có thể sử dụng hàm này như sau:

```
>>> train_set, test_set = shuffle_and_split_data(housing, 0.2)
>>> len(train_set)
16512
>>> len(test_set)
4128
```

Vâng, cách này hoạt động, nhưng nó không hoàn hảo: nếu bạn chạy lại chương trình, nó sẽ tạo ra một tập kiểm thử khác! Theo thời gian, bạn (hoặc các thuật toán học máy của bạn) sẽ được thấy toàn bộ tập dữ liệu, đây là điều bạn muốn tránh.

Một giải pháp là lưu tập kiểm thử trong lần chạy đầu tiên và sau đó tải nó trong các lần chạy tiếp theo. Một lựa chọn khác là đặt hạt giống của bộ tạo số ngẫu nhiên (ví dụ: với `np.random.seed(42)`)6 trước khi gọi `np.random.permutation()` để nó luôn tạo ra cùng một

chỉ số được xáo trộn. Tuy nhiên, cả hai giải pháp này sẽ bị hỏng vào lần tới khi bạn lấy một tập dữ liệu đã cập nhật. Để có một phân tách huấn luyện/kiểm thử ổn định ngay cả sau khi cập nhật tập dữ liệu, một giải pháp phổ biến là sử dụng định danh của mỗi trường hợp để quyết định xem nó có nên nằm trong tập kiểm thử hay không (giả sử các trường hợp có định danh duy nhất và bất biến). Ví dụ, bạn có thể tính hàm băm (hash) của định danh mỗi trường hợp và đưa trường hợp đó vào tập kiểm thử nếu hàm băm nhỏ hơn hoặc bằng 20% giá trị hàm băm tối đa. Điều này đảm bảo rằng tập kiểm thử sẽ vẫn nhất quán qua nhiều lần chạy, ngay cả khi bạn làm mới tập dữ liệu. Tập kiểm thử mới sẽ chứa 20% các trường hợp mới, nhưng nó sẽ không chứa bất kỳ trường hợp nào đã có trong tập huấn luyện trước đó. Dưới đây là một triển khai khả thi:

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

Thật không may, tập dữ liệu nhà ở không có cột định danh. Giải pháp đơn giản nhất là sử dụng chỉ mục hàng làm ID:

```
housing_with_id = housing.reset_index() #adds an `index` column
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

Nếu bạn sử dụng chỉ mục hàng làm định danh duy nhất, bạn cần đảm bảo rằng dữ liệu mới được thêm vào cuối tập dữ liệu và không có hàng nào bị xóa. Nếu điều này không thể, thì bạn có thể cố gắng sử dụng các đặc trưng ổn định nhất để xây dựng một định danh duy nhất. Ví dụ, vĩ độ và kinh độ của một quận được đảm bảo ổn định trong vài triệu năm, vì vậy bạn có thể kết hợp chúng thành một ID như sau:7

```
housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

Scikit-Learn cung cấp một vài hàm để chia tập dữ liệu thành nhiều tập con theo nhiều cách khác nhau. Hàm đơn giản nhất là `train_test_split()`, thực hiện khá giống với hàm `shuffle_and_split_data()` mà chúng ta đã định nghĩa trước đó, với một vài tính năng bổ sung. Thứ nhất, có tham số `random_state` cho phép bạn đặt hạt giống bộ tạo ngẫu nhiên. Thứ hai, bạn có thể truyền cho nó nhiều tập dữ liệu với số hàng giống hệt nhau, và nó sẽ chia chúng theo cùng các chỉ mục (điều này rất hữu ích, ví dụ, nếu bạn có một DataFrame riêng cho các nhãn):

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2,
random_state=42)
```

Cho đến nay, chúng ta đã xem xét các phương pháp lấy mẫu hoàn toàn ngẫu nhiên. Điều này nói chung là ổn nếu tập dữ liệu của bạn đủ lớn (đặc biệt là so với số lượng thuộc tính), nhưng nếu không, bạn có nguy cơ gây ra một sai lệch lấy mẫu đáng kể. Khi nhân viên tại một công ty khảo sát quyết định gọi 1.000 người để hỏi họ một vài câu hỏi, họ không chỉ chọn ngẫu nhiên 1.000 người trong danh bạ điện thoại. Họ cố gắng đảm bảo rằng 1.000 người này đại diện cho toàn bộ dân số, liên quan đến các câu hỏi họ muốn hỏi. Ví dụ, dân số Hoa Kỳ có 51.1% nữ và 48.9% nam, vì vậy một cuộc khảo sát được thực hiện tốt ở Hoa Kỳ sẽ cố gắng duy trì tỷ lệ này trong mẫu: 511 nữ và 489 nam (ít nhất là nếu có vẻ khả thi rằng câu trả lời có thể thay đổi theo giới tính). Đây được gọi là lấy mẫu phân tầng: dân số được chia thành các nhóm con đồng nhất gọi là tầng (strata), và số lượng trường hợp đúng được lấy mẫu từ mỗi tầng để đảm bảo rằng tập kiểm thử đại diện cho toàn bộ dân số. Nếu những người thực hiện khảo sát sử dụng lấy mẫu hoàn toàn ngẫu nhiên, sẽ có khoảng 10.7% khả năng lấy mẫu một tập kiểm thử bị lệch với ít hơn 48.5% nữ hoặc hơn 53.5% nữ tham gia. Dù bằng cách nào, kết quả khảo sát có thể sẽ khá sai lệch.

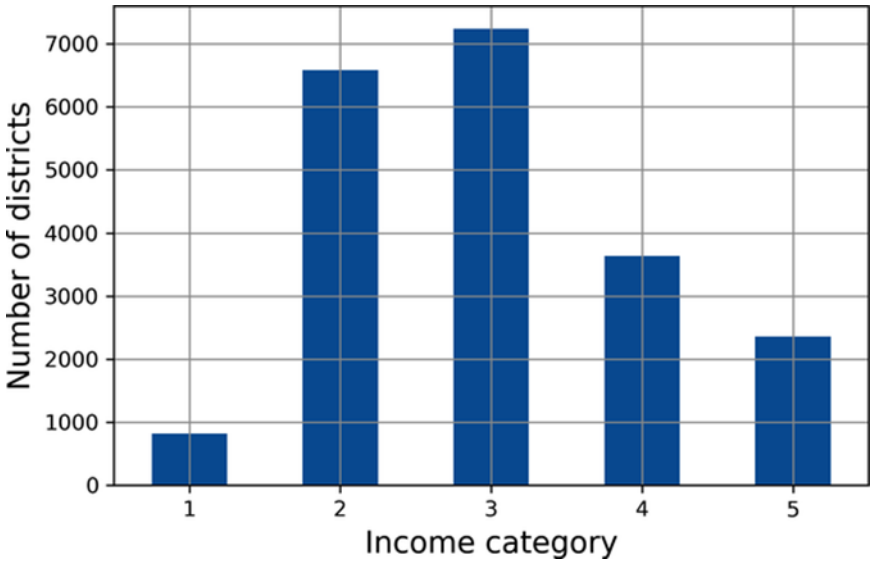
Giả sử bạn đã trò chuyện với một số chuyên gia đã nói với bạn rằng thu nhập trung bình là một thuộc tính rất quan trọng để dự đoán giá nhà trung bình. Bạn có thể muốn đảm bảo rằng tập kiểm thử đại diện cho các loại thu nhập khác nhau trong toàn bộ tập dữ liệu. Vì thu nhập trung bình là một thuộc tính số liên tục, trước tiên bạn cần tạo một thuộc tính danh mục thu nhập. Hãy xem xét kỹ hơn biểu đồ thu nhập trung bình (quay lại Hình 2-8): hầu hết các giá trị thu nhập trung bình được nhóm quanh 1.5 đến 6 (tức là $15.000–$60.000), nhưng một số thu nhập trung bình vượt xa 6. Điều quan trọng là phải có đủ số lượng trường hợp trong tập dữ liệu của bạn cho mỗi tầng, nếu không ước tính tầm quan trọng của một tầng có thể bị sai lệch. Điều này có nghĩa là bạn không nên có quá nhiều tầng, và mỗi tầng phải đủ lớn. Đoạn mã sau sử dụng hàm `pd.cut()` để tạo một thuộc tính danh mục thu nhập với năm loại (được gắn nhãn từ 1 đến 5); loại 1 nằm trong khoảng từ 0 đến 1.5 (tức là dưới $15.000), loại 2 từ 1.5 đến 3, v.v.:

```
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])
```

Các danh mục thu nhập này được biểu diễn trong Hình 2-9:

```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0, grid=True)
plt.xlabel("Income category")
plt.ylabel("Number of districts")
plt.show()
```

Bây giờ bạn đã sẵn sàng thực hiện lấy mẫu phân tầng dựa trên danh mục thu nhập. Scikit-Learn cung cấp một số lớp chia tách trong gói `sklearn.model_selection` triển khai các chiến lược khác nhau để chia tập dữ liệu của bạn thành một tập huấn luyện và một tập kiểm thử. Mỗi bộ chia tách có một phương thức `split()` trả về một trình lặp trên các phân tách huấn luyện/kiểm thử khác nhau của cùng một dữ liệu.



Hình 2-9. Biểu đồ danh mục thu nhập

Chính xác hơn, phương thức `split()` trả về các chỉ mục huấn luyện và kiểm thử, chứ không phải bản thân dữ liệu. Việc có nhiều phân tách có thể hữu ích nếu bạn muốn ước tính tốt hơn hiệu suất của mô hình, như bạn sẽ thấy khi chúng ta thảo luận về xác thực chéo sau trong chương này. Ví dụ, đoạn mã sau tạo 10 phân tách phân tầng khác nhau của cùng một tập dữ liệu:

```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2,
random_state=42)
strat_splits = []

for train_index, test_index in splitter.split(housing,
housing["income_cat"]):
    strat_train_set_n = housing.iloc[train_index]
    strat_test_set_n = housing.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

Hiện tại, bạn có thể chỉ sử dụng phân tách đầu tiên:

```
strat_train_set, strat_test_set = strat_splits[0]
```

Hoặc, vì lấy mẫu phân tầng khá phổ biến, có một cách ngắn gọn hơn để có được một phân tách duy nhất bằng cách sử dụng hàm `train_test_split()` với đối số `stratify`:

```
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"], random_state=42)
```

Hãy xem liệu điều này có hoạt động như mong đợi không. Bạn có thể bắt đầu bằng cách xem xét tỷ lệ danh mục thu nhập trong tập kiểm thử:

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
3    0.350533
2    0.318798
4    0.176357
5    0.114341
1    0.039971
Name: income_cat, dtype: float64
```

Với mã tương tự, bạn có thể đo lường tỷ lệ danh mục thu nhập trong toàn bộ tập dữ liệu. Hình 2-10 so sánh tỷ lệ danh mục thu nhập trong toàn bộ tập dữ liệu, trong tập kiểm thử được tạo bằng lấy mẫu phân tầng và trong tập kiểm thử được tạo bằng lấy mẫu hoàn toàn ngẫu nhiên. Như bạn có thể thấy, tập kiểm thử được tạo bằng lấy mẫu phân tầng có tỷ lệ danh mục thu nhập gần như giống hệt với tập dữ liệu đầy đủ, trong khi tập kiểm thử được tạo bằng lấy mẫu hoàn toàn ngẫu nhiên bị lệch.

Income Category	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

Hình 2-10. So sánh sai lệch lấy mẫu giữa lấy mẫu phân tầng và lấy mẫu hoàn toàn ngẫu nhiên

Bạn sẽ không sử dụng cột `income_cat` nữa, vì vậy bạn có thể loại bỏ nó, đưa dữ liệu về trạng thái ban đầu:

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

Chúng ta đã dành khá nhiều thời gian cho việc tạo tập kiểm thử vì một lý do chính đáng: đây là một phần thường bị bỏ qua nhưng lại rất quan trọng của một dự án học máy. Hơn nữa, nhiều ý tưởng này sẽ hữu ích sau này khi chúng ta thảo luận về kiểm định chéo. Bây giờ là lúc chuyển sang giai đoạn tiếp theo: khám phá dữ liệu.

2.4 Tự do khám phá và trực quan hóa dữ liệu để thu được thông tin chi tiết

Cho đến nay bạn mới chỉ lướt qua dữ liệu để có được cái nhìn tổng quan về loại dữ liệu bạn đang thao tác. Bây giờ mục tiêu là đi sâu hơn một chút. Đầu tiên, hãy đảm bảo bạn đã để riêng tập kiểm thử và bạn chỉ đang khám phá tập huấn luyện. Ngoài ra, nếu tập huấn luyện rất lớn, bạn có thể muốn lấy mẫu một tập khám phá, để thao tác dễ dàng và nhanh chóng trong giai đoạn khám phá. Trong trường hợp này, tập huấn luyện khá nhỏ, vì vậy bạn có thể làm việc trực tiếp trên toàn bộ tập. Vì bạn sẽ thử nghiệm với các biến đổi khác nhau của

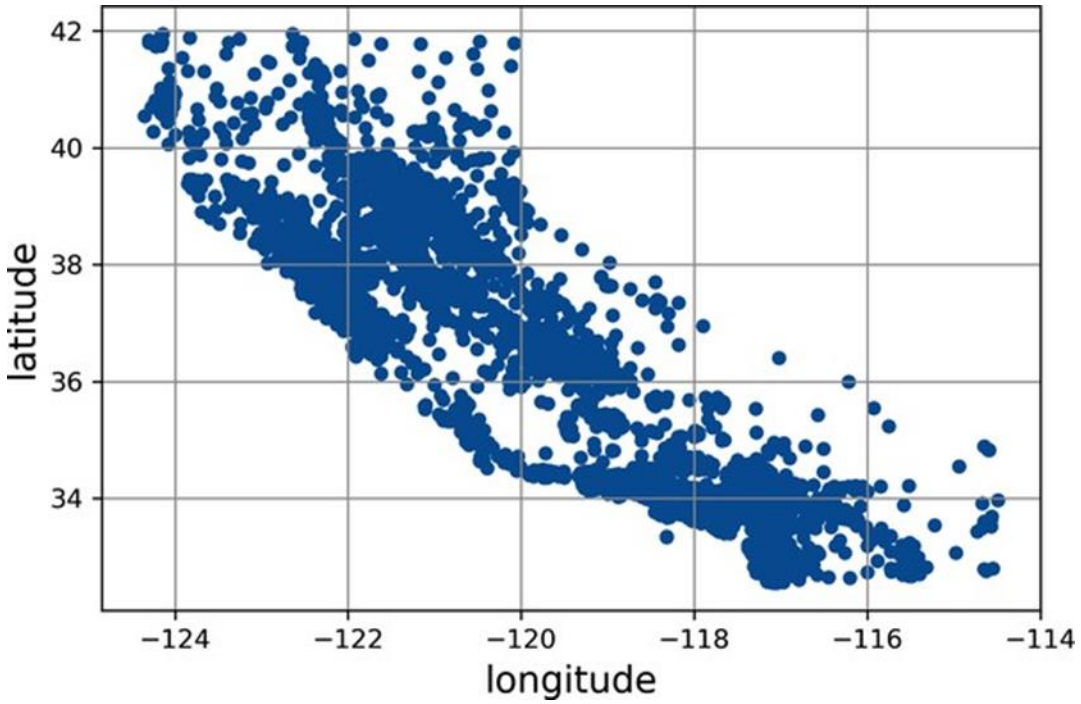
toàn bộ tập huấn luyện, bạn nên tạo một bản sao của bản gốc để bạn có thể hoàn nguyên về nó sau này:

```
housing = strat_train_set.copy()
```

2.4.1 Trực quan hóa dữ liệu địa lý

Vì tập dữ liệu bao gồm thông tin địa lý (vĩ độ và kinh độ), nên việc tạo biểu đồ phân tán của tất cả các quận để trực quan hóa dữ liệu là một ý tưởng hay (Hình 2-11):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
plt.show()
```

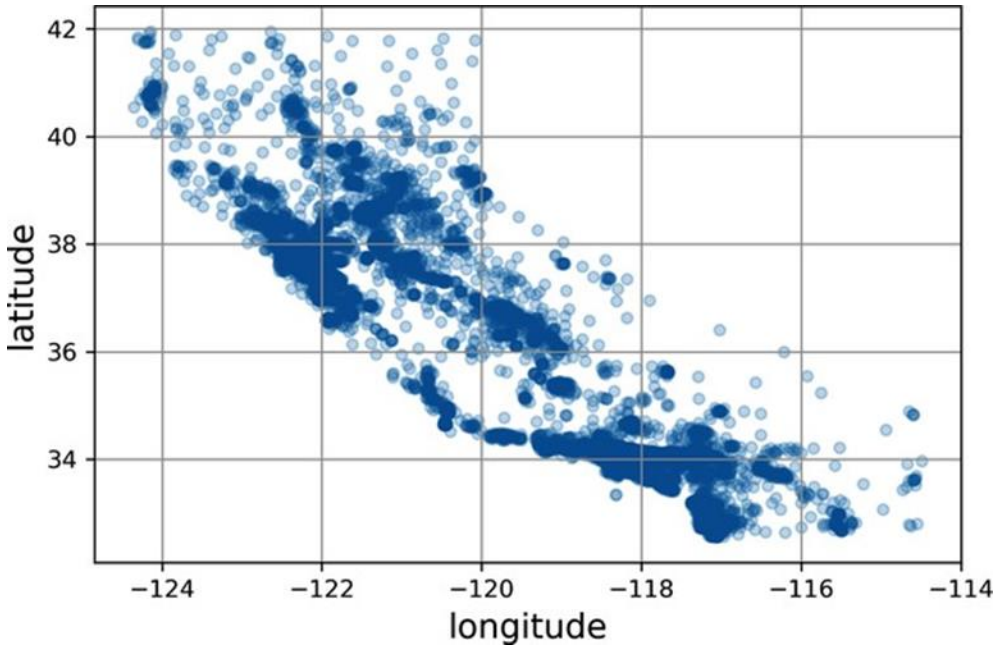


Hình 2-11. Biểu đồ phân tán địa lý của dữ liệu

Điều này trông giống California rồi, nhưng ngoài ra rất khó để thấy bất kỳ mẫu cụ thể nào. Đặt tùy chọn `alpha` thành 0.2 giúp dễ dàng trực quan hóa các nơi có mật độ điểm dữ liệu cao hơn (Hình 2-12):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
alpha=0.2)
plt.show()
```

Bây giờ thì tốt hơn nhiều: bạn có thể thấy rõ các khu vực mật độ cao, cụ thể là Vùng Vịnh và xung quanh Los Angeles và San Diego, cộng thêm một dải dài các khu vực mật độ khá cao ở Thung lũng Trung tâm (đặc biệt là xung quanh Sacramento và Fresno). Bộ não của chúng ta rất giỏi trong việc phát hiện các mẫu trong hình ảnh, nhưng bạn có thể cần thử nghiệm với các tham số trực quan hóa để làm cho các mẫu nổi bật hơn.

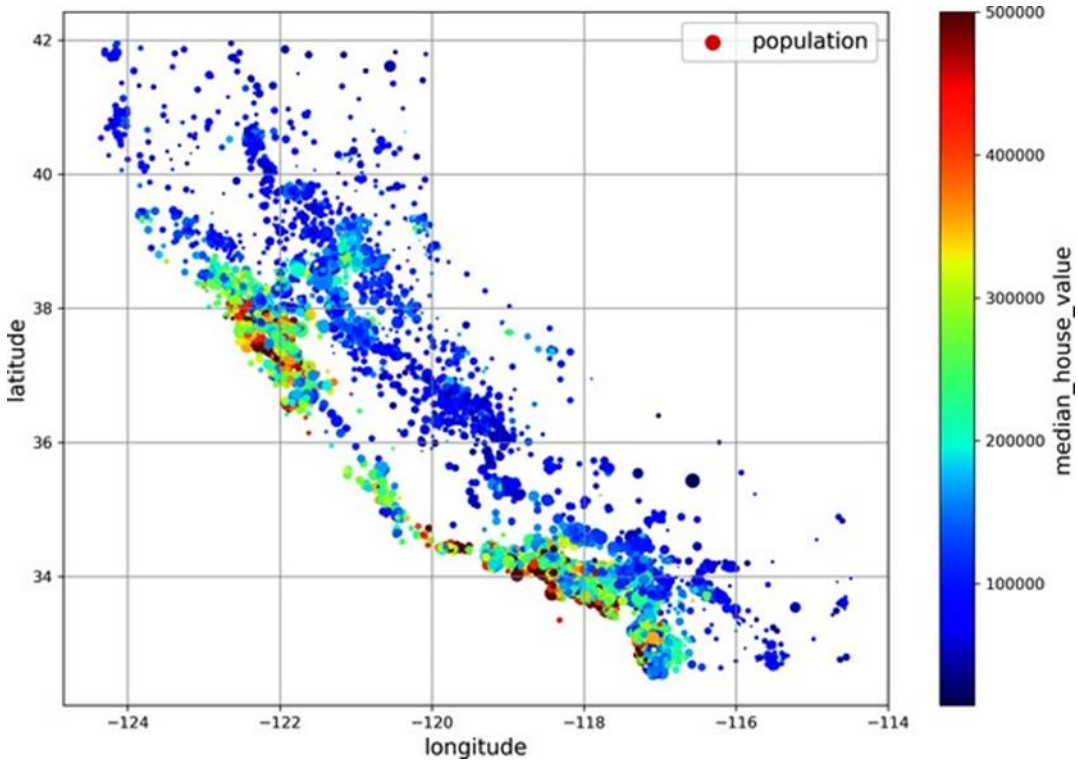


Hình 2-12. Một hình ảnh trực quan tốt hơn làm nổi bật các khu vực mật độ cao

Tiếp theo, bạn xem xét giá nhà (Hình 2-13). Bán kính của mỗi vòng tròn đại diện cho dân số của quận (tùy chọn `s`), và màu sắc đại diện cho giá (tùy chọn `c`). Ở đây bạn sử dụng một bản đồ màu được xác định trước (tùy chọn `cmap`) có tên là `jet`, từ màu xanh lam (giá trị thấp) đến màu đỏ (giá cao):8

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))
plt.show()
```

Hình ảnh này cho bạn biết rằng giá nhà rất liên quan đến vị trí (ví dụ: gần biển) và mật độ dân số, như bạn có lẽ đã biết. Một thuật toán phân cụm sẽ hữu ích để phát hiện cụm chính và để thêm các đặc trưng mới đo lường sự gần gũi với các trung tâm cụm. Thuộc tính gần biển cũng có thể hữu ích, mặc dù ở Bắc California, giá nhà ở các quận ven biển không quá cao, vì vậy nó không phải là một quy tắc đơn giản.



Hình 2-13. Giá nhà California: màu đỏ là đắt, màu xanh lam là rẻ, vòng tròn lớn hơn cho biết các khu vực có dân số lớn hơn

2.4.2 Tìm kiếm các tương quan

Vì tập dữ liệu không quá lớn, bạn có thể dễ dàng tính toán hệ số tương quan chuẩn (còn gọi là r của Pearson) giữa mỗi cặp thuộc tính bằng cách sử dụng phương thức `corr()`:

```
corr_matrix = housing.corr()
```

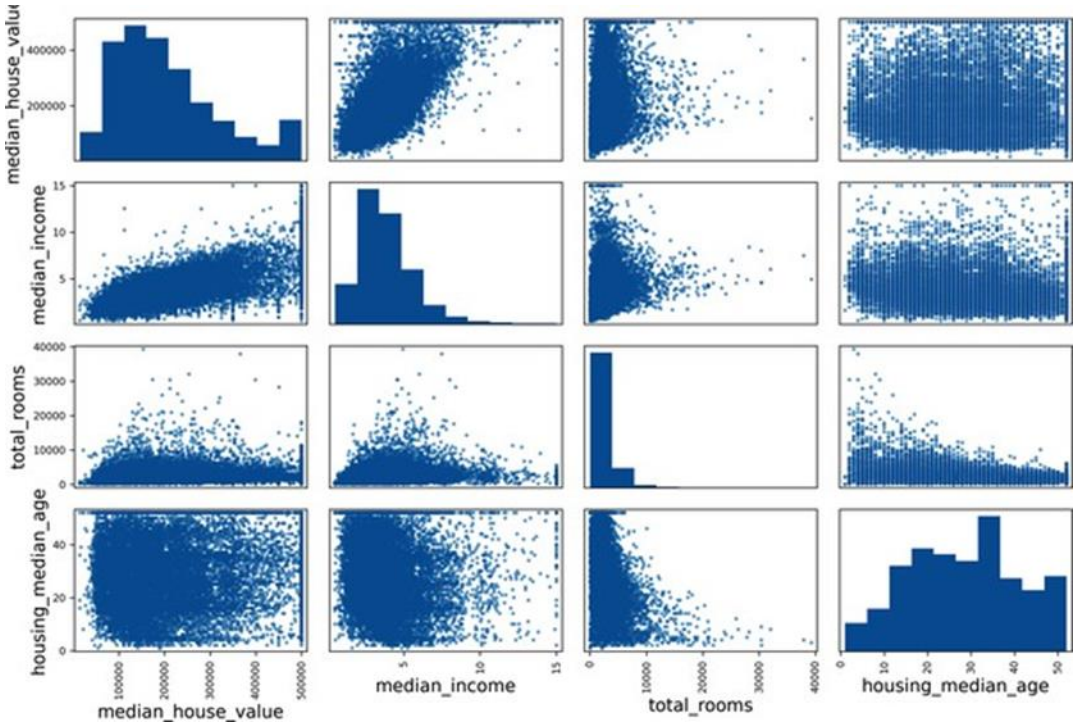
Bây giờ bạn có thể xem xét mức độ mỗi thuộc tính tương quan với giá trị nhà trung bình:

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
longitude            -0.050859
latitude             -0.139584
Name: median_house_value, dtype: float64
```

Hệ số tương quan nằm trong khoảng từ -1 đến 1. Khi nó gần 1, có nghĩa là có mối tương quan dương mạnh; ví dụ, giá trị nhà trung bình có xu hướng tăng khi thu nhập trung bình tăng. Khi hệ số gần -1, có nghĩa là có mối tương quan âm mạnh; bạn có thể thấy mối tương quan âm nhỏ giữa vĩ độ và giá trị nhà trung bình (tức là giá có xu hướng giảm nhẹ khi bạn đi

về phía bắc). Cuối cùng, các hệ số gần 0 có nghĩa là không có mối tương quan tuyến tính. Một cách khác để kiểm tra mối tương quan giữa các thuộc tính là sử dụng hàm `scatter_matrix()` của Pandas, hàm này vẽ biểu đồ mỗi thuộc tính số với mọi thuộc tính số khác. Vì hiện có 11 thuộc tính số, bạn sẽ nhận được $11^2 = 121$ biểu đồ, sẽ không vừa trên một trang — vì vậy bạn quyết định tập trung vào một vài thuộc tính đầy hứa hẹn dường như tương quan nhất với giá trị nhà trung bình (Hình 2-14):

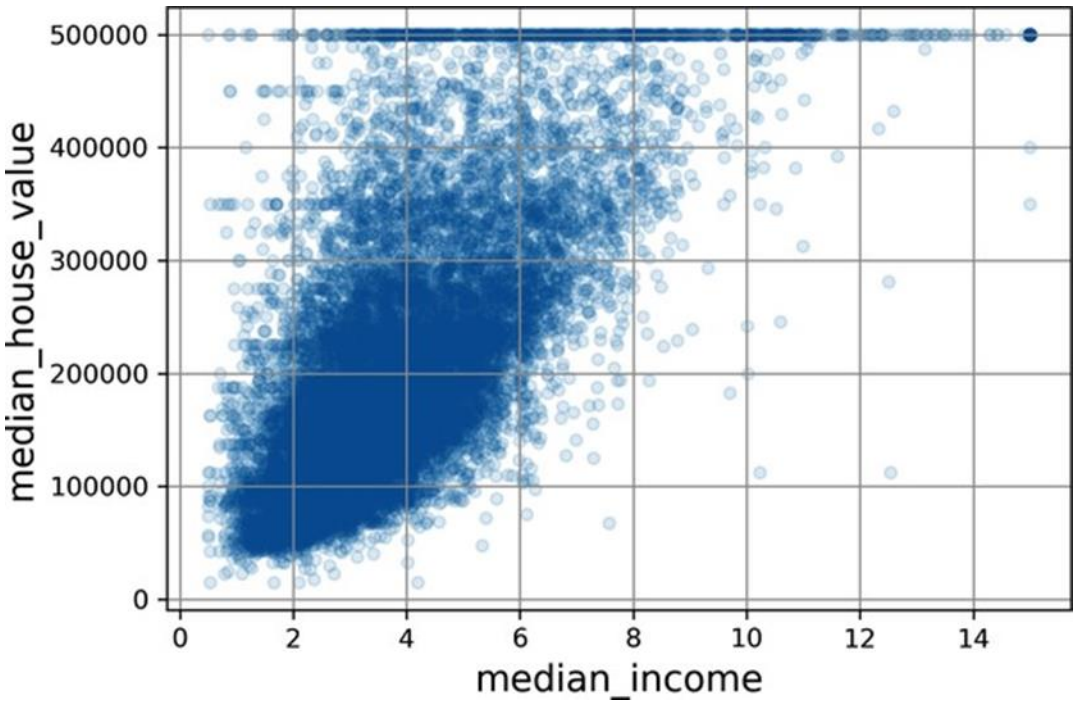
```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
"housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
plt.show()
```



Hình 2-14. Ma trận phân tán này vẽ biểu đồ mọi thuộc tính số với mọi thuộc tính số khác, cộng với biểu đồ tần suất giá trị của mỗi thuộc tính số trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải)

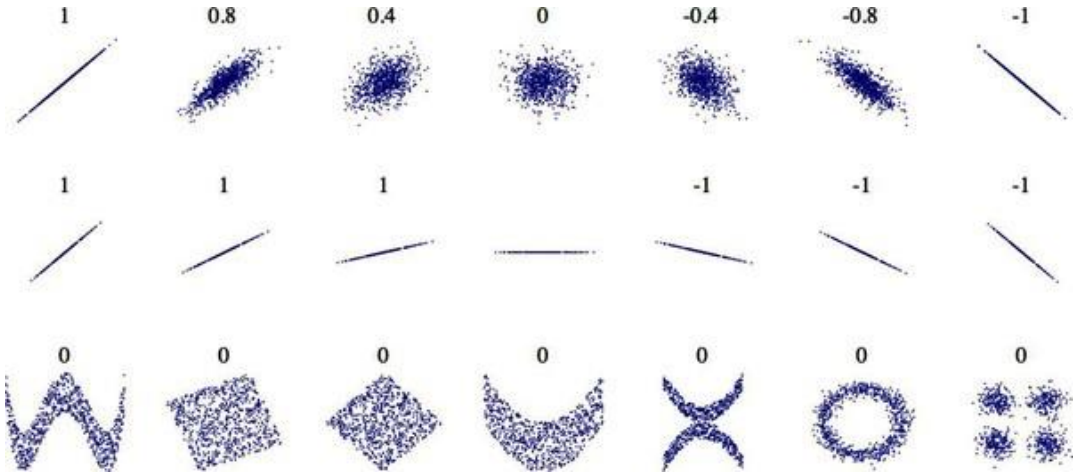
Đường chéo chính sẽ đầy các đường thẳng nếu Pandas vẽ mỗi biến với chính nó, điều đó sẽ không hữu ích lắm. Thay vào đó, Pandas hiển thị biểu đồ tần suất của mỗi thuộc tính (các tùy chọn khác có sẵn; xem tài liệu Pandas để biết thêm chi tiết). Nhìn vào các biểu đồ phân tán tương quan, có vẻ như thuộc tính hứa hẹn nhất để dự đoán giá trị nhà trung bình là thu nhập trung bình, vì vậy bạn phóng to biểu đồ phân tán của chúng (Hình 2-15):

```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
plt.show()
```



Hình 2-15. Thu nhập trung bình so với giá trị nhà trung bình

Biểu đồ này tiết lộ một vài điều. Đầu tiên, mối tương quan thực sự khá mạnh; bạn có thể thấy rõ xu hướng tăng, và các điểm không quá phân tán. Thứ hai, giới hạn giá mà bạn nhận thấy trước đó rõ ràng hiển thị dưới dạng một đường ngang ở mức $500.000. Nhưng biểu đồ cũng tiết lộ các đường thẳng ít rõ ràng hơn khác: một đường ngang quanh $450.000, một đường khác quanh $350.000, có lẽ một đường quanh $280.000, và một vài đường nữa bên dưới đó. Bạn có thể muốn thử loại bỏ các quận tương ứng để ngăn thuật toán của bạn học cách tái tạo những điểm đặc biệt của dữ liệu này.



Hình 2-16. Hệ số tương quan chuẩn của các tập dữ liệu khác nhau (nguồn: Wikipedia; hình ảnh phạm vi công cộng)

2.4.3 Thử nghiệm với các kết hợp thuộc tính

Hy vọng các phần trước đã cho bạn ý tưởng về một vài cách bạn có thể khám phá dữ liệu và thu được thông tin chi tiết. Bạn đã xác định được một vài điểm đặc biệt của dữ liệu mà bạn có thể muốn làm sạch trước khi cung cấp dữ liệu cho thuật toán học máy, và bạn đã tìm thấy những tương quan thú vị giữa các thuộc tính, đặc biệt là với thuộc tính mục tiêu. Bạn cũng nhận thấy rằng một số thuộc tính có phân phối lệch phải, vì vậy bạn có thể muốn biến đổi chúng (ví dụ: bằng cách tính logarit hoặc căn bậc hai của chúng). Dĩ nhiên, hiệu quả của bạn sẽ khác nhau đáng kể với mỗi dự án, nhưng những ý tưởng chung là tương tự.

Một điều cuối cùng bạn có thể muốn làm trước khi chuẩn bị dữ liệu cho các thuật toán học máy là thử các kết hợp thuộc tính khác nhau. Ví dụ, tổng số phòng trong một quận không hữu ích lắm nếu bạn không biết có bao nhiêu hộ gia đình. Điều bạn thực sự muốn là số phòng trên mỗi hộ gia đình. Tương tự, tổng số phòng ngủ tự nó không hữu ích lắm: bạn có thể muốn so sánh nó với tổng số phòng. Và dân số trên mỗi hộ gia đình cũng có vẻ là một sự kết hợp thuộc tính thú vị để xem xét. Bạn tạo các thuộc tính mới này như sau:

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] /
housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

Và sau đó bạn xem lại ma trận tương quan:

```
>>> corr_matrix = housing.corr()
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
rooms_per_house       0.143663
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
people_per_house     -0.038224
longitude            -0.050859
latitude             -0.139584
bedrooms_ratio       -0.256397
Name: median_house_value, dtype: float64
```

Chà, không tệ! Thuộc tính `bedrooms_ratio` mới tương quan với giá trị nhà trung bình nhiều hơn đáng kể so với tổng số phòng hoặc phòng ngủ. Rõ ràng, những ngôi nhà có tỷ lệ phòng ngủ/phòng thấp hơn có xu hướng đắt hơn. Số phòng trên mỗi hộ gia đình cũng cung cấp nhiều thông tin hơn tổng số phòng trong một quận — rõ ràng nhà càng lớn thì càng đắt.

Vòng khám phá này không nhất thiết phải hoàn toàn kỹ lưỡng; mục đích là để bắt đầu đúng hướng và nhanh chóng thu được những hiểu biết sâu sắc sẽ giúp bạn có được một nguyên mẫu đầu tiên khá tốt. Nhưng đây là một quy trình lặp đi lặp lại: một khi bạn có một nguyên

mẫu hoạt động, bạn có thể phân tích đầu ra của nó để có thêm thông tin chi tiết và quay lại bước khám phá này.

2.5 Chuẩn bị dữ liệu cho các thuật toán học máy

Đã đến lúc chuẩn bị dữ liệu cho các thuật toán học máy của bạn. Thay vì thực hiện thủ công, bạn nên viết các hàm cho mục đích này, vì một số lý do chính đáng:

- Điều này sẽ cho phép bạn tái tạo các phép biến đổi này một cách dễ dàng trên bất kỳ tập dữ liệu nào (ví dụ: lần tới khi bạn có một tập dữ liệu mới).
- Bạn sẽ dần dần xây dựng một thư viện các hàm biến đổi mà bạn có thể tái sử dụng trong các dự án tương lai.
- Bạn có thể sử dụng các hàm này trong hệ thống trực tiếp của mình để biến đổi dữ liệu mới trước khi cung cấp cho các thuật toán của bạn.
- Điều này sẽ giúp bạn dễ dàng thử các phép biến đổi khác nhau và xem sự kết hợp các phép biến đổi nào hoạt động tốt nhất.

Nhưng trước tiên, hãy quay lại tập huấn luyện sạch (bằng cách sao chép `strat_train_set` một lần nữa). Bạn cũng nên tách các biến dự đoán và nhãn, vì bạn không nhất thiết muốn áp dụng cùng các phép biến đổi cho các biến dự đoán và giá trị mục tiêu (lưu ý rằng `drop()` tạo một bản sao của dữ liệu và không ảnh hưởng đến `strat_train_set`):

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

2.5.1 Làm sạch dữ liệu

Hầu hết các thuật toán học máy không thể hoạt động với các đặc trưng bị thiếu, vì vậy bạn sẽ cần phải xử lý chúng. Ví dụ, bạn đã nhận thấy trước đó rằng thuộc tính `total_bedrooms` có một số giá trị bị thiếu. Bạn có ba lựa chọn để khắc phục điều này:

- Loại bỏ các quận tương ứng.
- Loại bỏ toàn bộ thuộc tính.
- Đặt các giá trị bị thiếu thành một giá trị nào đó (không, giá trị trung bình, giá trị trung vị, v.v.). Điều này được gọi là điền khuyết (imputation).

Bạn có thể dễ dàng thực hiện những điều này bằng các phương thức `dropna()`, `drop()` và `fillna()` của Pandas DataFrame:

```
housing.dropna(subset=["total_bedrooms"], inplace=True) # option 1
housing.drop("total_bedrooms", axis=1) # option 2
median = housing["total_bedrooms"].median() # option 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

Bạn quyết định chọn tùy chọn 3 vì nó ít phá hủy nhất, nhưng thay vì đoạn mã trên, bạn sẽ sử dụng một lớp tiện dụng của Scikit-Learn: `SimpleImputer`. Lợi ích là nó sẽ lưu trữ giá trị trung vị của mỗi đặc trưng: điều này sẽ giúp điền khuyết các giá trị bị thiếu không chỉ trên

tập huấn luyện mà còn trên tập xác thực, tập kiểm thử và bất kỳ dữ liệu mới nào được đưa vào mô hình. Để sử dụng nó, trước tiên bạn cần tạo một thể hiện `SimpleImputer`, chỉ định rằng bạn muốn thay thế các giá trị bị thiếu của mỗi thuộc tính bằng giá trị trung vị của thuộc tính đó:

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
```

Vì giá trị trung vị chỉ có thể được tính trên các thuộc tính số, bạn cần tạo một bản sao của dữ liệu chỉ với các thuộc tính số (điều này sẽ loại trừ thuộc tính văn bản `ocean_proximity`):

```
housing_num = housing.select_dtypes(include=[np.number])
```

Bây giờ bạn có thể điều chỉnh thể hiện `imputer` cho dữ liệu huấn luyện bằng phương thức `fit()`:

```
imputer.fit(housing_num)
```

`imputer` đơn giản đã tính toán giá trị trung vị của mỗi thuộc tính và lưu trữ kết quả trong biến thể hiện `statistics_` của nó. Chỉ thuộc tính

`total_bedrooms` có giá trị bị thiếu, nhưng bạn không thể chắc chắn rằng sẽ không có bất kỳ giá trị bị thiếu nào trong dữ liệu mới sau khi hệ thống hoạt động, vì vậy an toàn hơn là áp dụng `imputer` cho tất cả các thuộc tính số:

```
>>> imputer.statistics_
array([-118.51, 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
>>> housing_num.median().values
array([-118.51, 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
```

Bây giờ bạn có thể sử dụng `imputer` "đã huấn luyện" này để biến đổi tập huấn luyện bằng cách thay thế các giá trị bị thiếu bằng các giá trị trung vị đã học:

```
X = imputer.transform(housing_num)
```

Các giá trị bị thiếu cũng có thể được thay thế bằng giá trị trung bình (`strategy="mean"`), hoặc với giá trị thường xuyên nhất (`strategy="most_frequent"`), hoặc với một giá trị không đổi (`strategy="constant"`, `fill_value=…`). Hai chiến lược cuối cùng hỗ trợ dữ liệu không phải số.

THIẾT KẾ SCIKIT-LEARN

API của Scikit-Learn được thiết kế đặc biệt tốt. Đây là các nguyên tắc thiết kế chính:

Tính nhất quán

Tất cả các đối tượng đều chia sẻ một giao diện nhất quán và đơn giản:

- **Estimators** Bất kỳ đối tượng nào có thể ước tính một số tham số dựa trên tập dữ liệu được gọi là estimator (ví dụ: `SimpleImputer` là một estimator). Bản thân quá trình ước tính được thực hiện bằng phương thức `fit()`, và nó nhận một tập dữ liệu làm

tham số, hoặc hai cho các thuật toán học có giám sát — tập dữ liệu thứ hai chứa các nhãn. Bất kỳ tham số nào khác cần thiết để hướng dẫn quá trình ước tính được coi là một siêu tham số (chẳng hạn như chiến lược của `SimpleImputer`), và nó phải được đặt dưới dạng biến thể hiện (thường thông qua tham số của hàm tạo).

- **Transformers** Một số estimators (chẳng hạn như `SimpleImputer`) cũng có thể biến đổi một tập dữ liệu; chúng được gọi là transformer. Một lần nữa, API rất đơn giản: phép biến đổi được thực hiện bằng phương thức `transform()` với tập dữ liệu cần biến đổi làm tham số. Nó trả về tập dữ liệu đã biến đổi. Phép biến đổi này thường dựa trên các tham số đã học, như trường hợp của `SimpleImputer`. Tất cả các transformer cũng có một phương thức tiện lợi được gọi là `fit_transform()`, tương đương với việc gọi `fit()` và sau đó `transform()` (nhưng đôi khi `fit_transform()` được tối ưu hóa và chạy nhanh hơn nhiều).
- **Predictors** Cuối cùng, một số estimators, khi được cung cấp một tập dữ liệu, có khả năng đưa ra dự đoán; chúng được gọi là predictor. Ví dụ, mô hình `LinearRegression` trong chương trước là một predictor: khi được cung cấp GDP trên đầu người của một quốc gia, nó đã dự đoán sự hài lòng cuộc sống. Một predictor có phương thức `predict()` nhận một tập dữ liệu các trường hợp mới và trả về một tập dữ liệu các dự đoán tương ứng. Nó cũng có phương thức `score()` đo lường chất lượng của các dự đoán, với một tập kiểm thử (và các nhãn tương ứng, trong trường hợp các thuật toán học có giám sát). **Kiểm tra** Tất cả các siêu tham số của estimator có thể truy cập trực tiếp thông qua các biến thể hiện công khai (ví dụ: `imputer.strategy`), và tất cả các tham số đã học của estimator có thể truy cập thông qua các biến thể hiện công khai với hậu tố gạch dưới (ví dụ: `imputer.statistics_`). **Không mở rộng lớp** Các tập dữ liệu được biểu diễn dưới dạng mảng NumPy hoặc ma trận thưa SciPy, thay vì các lớp tự tạo. Siêu tham số chỉ là các chuỗi hoặc số Python thông thường. **Thành phần** Các khối xây dựng hiện có được tái sử dụng càng nhiều càng tốt. Ví dụ, rất dễ dàng tạo một `Pipeline` estimator từ một chuỗi bất kỳ các transformer được theo sau bởi một estimator cuối cùng, như bạn sẽ thấy. **Giá trị mặc định hợp lý** Scikit-Learn cung cấp các giá trị mặc định hợp lý cho hầu hết các tham số, giúp dễ dàng nhanh chóng tạo một hệ thống hoạt động cơ bản.

Transformer của Scikit-Learn xuất ra mảng NumPy (hoặc đôi khi là ma trận thưa SciPy) ngay cả khi chúng được cung cấp Pandas DataFrames làm đầu vào. Vì vậy, đầu ra của `imputer.transform(housing_num)` là một mảng NumPy: `X` không có tên cột cũng như chỉ mục. May mắn thay, không quá khó để bao bọc `X` trong một DataFrame và khôi phục tên cột và chỉ mục từ `housing_num`:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

2.5.2 Xử lý các thuộc tính văn bản và phân loại

Cho đến nay chúng ta mới chỉ xử lý các thuộc tính số, nhưng dữ liệu của bạn cũng có thể chứa các thuộc tính văn bản. Trong tập dữ liệu này, chỉ có một: thuộc tính `ocean_proximity`. Hãy xem giá trị của nó cho một vài trường hợp đầu tiên:

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(8)
      ocean_proximity
13096        NEAR BAY
14973       <1H OCEAN
3785           INLAND
14689          INLAND
20507       NEAR OCEAN
1286           INLAND
18078       <1H OCEAN
4396         NEAR BAY
```

Nó không phải là văn bản tùy ý: có một số lượng giới hạn các giá trị có thể, mỗi giá trị đại diện cho một danh mục. Vì vậy, thuộc tính này là một thuộc tính phân loại. Hầu hết các thuật toán học máy thích làm việc với số, vì vậy hãy chuyển đổi các danh mục này từ văn bản sang số. Để làm điều này, chúng ta có thể sử dụng lớp

`OrdinalEncoder` của Scikit-Learn:

```
from sklearn.preprocessing import OrdinalEncoder
ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

Dưới đây là một vài giá trị được mã hóa đầu tiên trong `housing_cat_encoded` trông như thế nào:

```
>>> housing_cat_encoded[:8]
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

Bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thể hiện `categories_`. Nó là một danh sách chứa một mảng 1D các danh mục cho mỗi thuộc tính phân loại (trong trường hợp này, một danh sách chứa một mảng duy nhất vì chỉ có một thuộc tính phân loại):

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

Một vấn đề với biểu diễn này là các thuật toán ML sẽ giả định rằng hai giá trị gần nhau thì tương tự nhau hơn hai giá trị xa nhau. Điều này có thể ổn trong một số trường hợp (ví dụ: đối với các danh mục có thứ tự như "xấu", "trung bình", "tốt" và "xuất sắc"), nhưng rõ ràng không phải là trường hợp đối với cột `ocean_proximity` (ví dụ: các danh mục 0 và 4 rõ ràng tương tự nhau hơn các danh mục 0 và 1). Để khắc phục vấn đề này, một giải pháp phổ biến là tạo một thuộc tính nhị phân cho mỗi danh mục: một thuộc tính bằng 1 khi danh mục là "<1H OCEAN" (và 0 nếu không), một thuộc tính khác bằng 1 khi danh mục là "INLAND" (và 0 nếu không), v.v. Điều này được gọi là mã hóa một-nóng (one-hot encoding), vì chỉ một thuộc tính sẽ bằng 1 (hot), trong khi các thuộc tính khác sẽ bằng 0 (cold). Các thuộc tính mới đôi khi được gọi là thuộc tính giả (dummy attributes). Scikit-Learn cung cấp lớp `OneHotEncoder` để chuyển đổi các giá trị phân loại thành các vector một-nóng:

```
from sklearn.preprocessing import OneHotEncoder
cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

Theo mặc định, đầu ra của

`OneHotEncoder` là một ma trận thưa SciPy, thay vì một mảng NumPy:

```
>>> housing_cat_1hot
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
    with 16512 stored elements in Compressed Sparse Row format>
```

Ma trận thưa là một biểu diễn rất hiệu quả cho các ma trận chủ yếu chứa các số 0. Thực tế, bên trong nó chỉ lưu trữ các giá trị khác 0 và vị trí của chúng. Khi một thuộc tính phân loại có hàng trăm hoặc hàng nghìn danh mục, việc mã hóa một-nóng nó sẽ tạo ra một ma trận rất lớn chứa đầy các số 0 ngoại trừ một số 1 duy nhất trên mỗi hàng. Trong trường hợp này, ma trận thưa chính xác là thứ bạn cần: nó sẽ tiết kiệm nhiều bộ nhớ và tăng tốc tính toán. Bạn có thể sử dụng ma trận thưa gần giống như một mảng 2D thông thường, nhưng nếu bạn muốn chuyển đổi nó thành một mảng NumPy (đậm đặc), chỉ cần gọi phương thức `toarray()`:

```
>>> housing_cat_1hot.toarray()
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

Thay vào đó, bạn có thể đặt `sparse=False` khi tạo `OneHotEncoder`, trong trường hợp đó phương thức `transform()` sẽ trả về trực tiếp một mảng NumPy thông thường (đậm đặc).

Cũng giống như `OrdinalEncoder`, bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thể hiện `categories_` của bộ mã hóa:

```
>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

Pandas có một hàm gọi là `get_dummies()`, cũng chuyển đổi mỗi đặc trưng phân loại thành một biểu diễn một-nóng, với một đặc trưng nhị phân cho mỗi danh mục:

```
>>> df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]})
>>> pd.get_dummies(df_test)
   ocean_proximity_INLAND  ocean_proximity_NEAR BAY
0                       1                         0
1                       0                         1
```

Nó trông đẹp và đơn giản, vậy tại sao không sử dụng nó thay vì `OneHotEncoder`? Chà, lợi thế của `OneHotEncoder` là nó ghi nhớ các danh mục mà nó đã được huấn luyện. Điều này rất quan trọng vì một khi mô hình của bạn đã được đưa vào sản xuất, nó phải được cung cấp chính xác các đặc trưng giống như trong quá trình huấn luyện: không hơn, không kém. Hãy xem đầu ra của `cat_encoder` đã được huấn luyện khi chúng ta yêu cầu nó biến đổi cùng `df_test` (sử dụng `transform()`, không phải `fit_transform()`):

```
>>> cat_encoder.transform(df_test)
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

Bạn có thấy sự khác biệt không?

`get_dummies()` chỉ thấy hai danh mục, vì vậy nó xuất ra hai cột, trong khi `OneHotEncoder` xuất ra một cột cho mỗi danh mục đã học, theo đúng thứ tự. Hơn nữa, nếu bạn cung cấp cho `get_dummies()` một DataFrame chứa một danh mục không xác định (ví dụ: “<2H OCEAN”), nó sẽ vui vẻ tạo một cột cho nó:

```
>>> df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN",
"ISLAND"]})
>>> pd.get_dummies(df_test_unknown)
   ocean_proximity_<2H OCEAN  ocean_proximity_ISLAND
0                          1                       0
1                          0                       1
```

Nhưng `OneHotEncoder` thông minh hơn: nó sẽ phát hiện danh mục không xác định và đưa ra một ngoại lệ. Nếu bạn thích, bạn có thể đặt siêu tham số `handle_unknown` thành “ignore”, trong trường hợp đó nó sẽ chỉ biểu diễn danh mục không xác định bằng các số 0:

```
>>> cat_encoder.handle_unknown = "ignore"
>>> cat_encoder.transform(df_test_unknown)
array([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

Khi bạn điều chỉnh bất kỳ ước lượng viên Scikit-Learn nào bằng DataFrame, ước lượng viên sẽ lưu trữ tên cột trong thuộc tính `feature_names_in_`. Sau đó, Scikit-Learn đảm bảo rằng bất kỳ DataFrame nào được cung cấp cho ước lượng viên này sau đó (ví dụ: để

transform() hoặc predict()) đều có cùng tên cột. Transformer cũng cung cấp phương thức get_feature_names_out() mà bạn có thể sử dụng để xây dựng DataFrame xung quanh đầu ra của transformer:

```
>>> cat_encoder.feature_names_in_
array(['ocean_proximity'], dtype=object)

>>> cat_encoder.get_feature_names_out()
array(['ocean_proximity_<1H OCEAN', 'ocean_proximity_INLAND',
       'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
       'ocean_proximity_NEAR OCEAN'], dtype=object)

>>> df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
...                          columns=cat_encoder.get_feature_names_out(),
...                          index=df_test_unknown.index)
...
```

2.5.3 Co giãn và Biến đổi Đặc trưng

Một trong những phép biến đổi quan trọng nhất bạn cần áp dụng cho dữ liệu của mình là phân loại đặc trưng. Trừ một số trường hợp ngoại lệ, các thuật toán học máy không hoạt động tốt khi các thuộc tính số đầu vào có các thang đo rất khác nhau. Đây là trường hợp của dữ liệu nhà ở: tổng số phòng dao động từ khoảng 6 đến 39.320, trong khi thu nhập trung bình chỉ dao động từ 0 đến 15. Nếu không có bất kỳ sự điều chỉnh nào, hầu hết các mô hình sẽ có xu hướng bỏ qua thu nhập trung bình và tập trung hơn vào số lượng phòng.

Có hai cách phổ biến để làm cho tất cả các thuộc tính có cùng thang đo: điều chỉnh min-max (min-max scaling) và chuẩn hóa (standardization).

Điều chỉnh min-max (nhiều người gọi đây là chuẩn hóa) là cách đơn giản nhất: đối với mỗi thuộc tính, các giá trị được dịch chuyển và thay đổi tỷ lệ sao cho chúng nằm trong khoảng từ 0 đến 1. Điều này được thực hiện bằng cách trừ đi giá trị nhỏ nhất và chia cho sự khác biệt giữa giá trị nhỏ nhất và giá trị lớn nhất. Scikit-Learn cung cấp một bộ biến đổi gọi là MinMaxScaler cho việc này. Nó có một siêu tham số feature_range cho phép bạn thay đổi phạm vi nếu, vì một lý do nào đó, bạn không muốn phạm vi từ 0–1 (ví dụ: mạng nơ-ron hoạt động tốt nhất với đầu vào có giá trị trung bình bằng 0, vì vậy phạm vi từ –1 đến 1 được ưu tiên hơn). Nó khá dễ sử dụng:

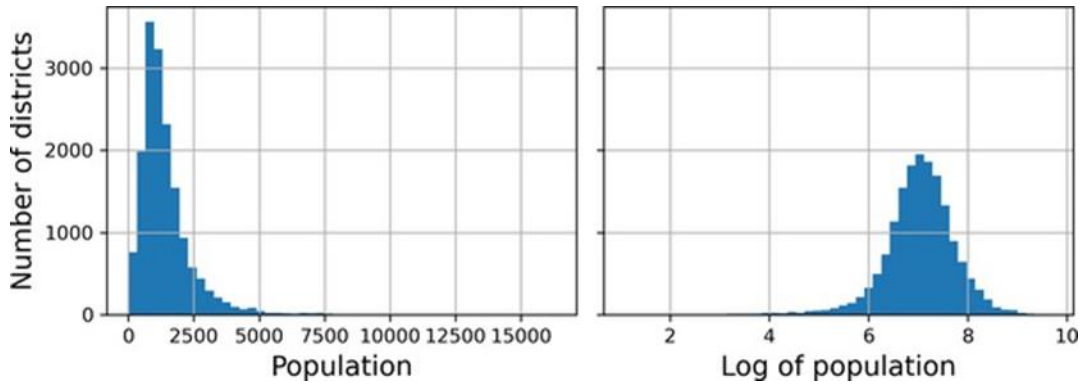
```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

Chuẩn hóa khác: đầu tiên nó trừ đi giá trị trung bình (do đó các giá trị đã được chuẩn hóa có giá trị trung bình bằng 0), sau đó nó chia kết quả cho độ lệch chuẩn (do đó các giá trị đã được chuẩn hóa có độ lệch chuẩn bằng 1). Không giống như điều chỉnh min-max, chuẩn hóa không giới hạn các giá trị trong một phạm vi cụ thể. Tuy nhiên, chuẩn hóa ít bị ảnh hưởng bởi các giá trị ngoại lai hơn nhiều. Ví dụ, giả sử một quận có thu nhập trung bình

bằng 100 (do nhầm lẫn), thay vì từ 0–15 thông thường. Điều chỉnh min-max thành phạm vi 0–1 sẽ ánh xạ giá trị ngoại lai này xuống 1 và nén tất cả các giá trị khác xuống 0–0.15, trong khi chuẩn hóa sẽ không bị ảnh hưởng nhiều. Scikit-Learn cung cấp một bộ biến đổi gọi là `StandardScaler` để chuẩn hóa:

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

Khi phân phối của một đặc trưng có "đuôi nặng" (heavy tail) (tức là khi các giá trị cách xa giá trị trung bình không hiếm một cách theo cấp số mũ), cả điều chỉnh min-max và chuẩn hóa sẽ nén hầu hết các giá trị vào một phạm vi nhỏ. Các mô hình học máy nói chung không thích điều này chút nào. Vì vậy, trước khi bạn điều chỉnh đặc trưng, bạn nên biến đổi nó để làm co hẹp "đuôi nặng" và nếu có thể, làm cho phân phối gần đối xứng. Ví dụ, một cách phổ biến để làm điều này đối với các đặc trưng dương có "đuôi nặng" về phía bên phải là thay thế đặc trưng bằng căn bậc hai của nó (hoặc nâng đặc trưng lên một lũy thừa giữa 0 và 1). Nếu đặc trưng có một "đuôi nặng" và dài thực sự, chẳng hạn như phân phối luật lũy thừa, thì việc thay thế đặc trưng bằng logarit của nó có thể hữu ích. Ví dụ, đặc trưng dân số gần như tuân theo luật lũy thừa: các quận có 10.000 dân chỉ ít thường xuyên hơn 10 lần so với các quận có 1.000 dân, chứ không phải ít thường xuyên hơn theo cấp số mũ. Hình 2-17 cho thấy đặc trưng này trông tốt hơn nhiều khi bạn tính logarit của nó: nó rất gần với phân phối Gaussian (tức là hình chuông).



Hình 2-17. Biến đổi một đặc trưng để làm cho nó gần hơn với phân phối Gaussian.

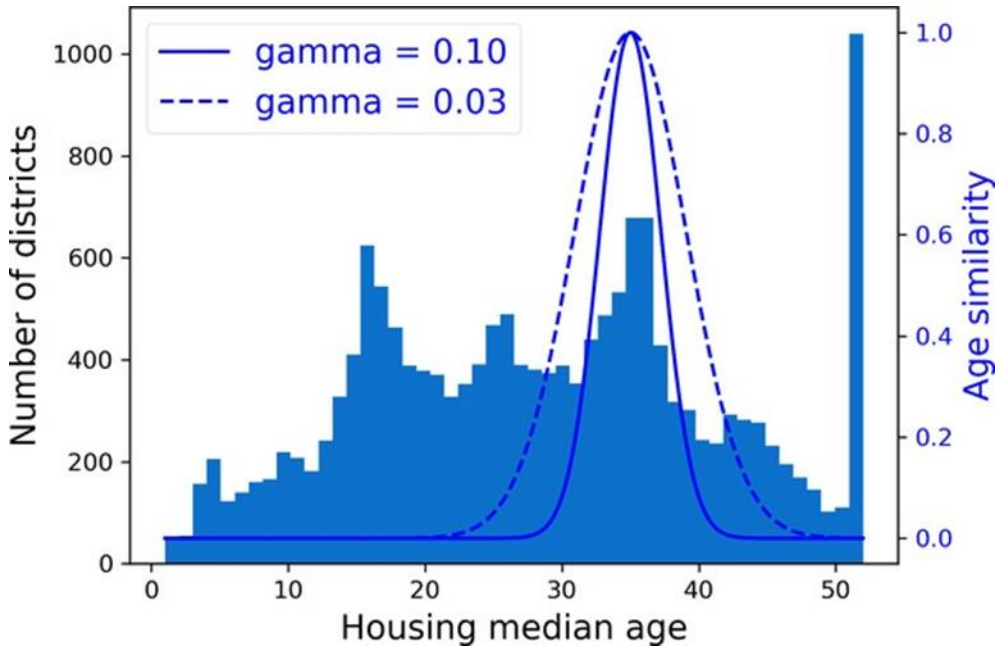
Một cách tiếp cận khác để xử lý các đặc trưng có "đuôi nặng" là phân nhóm (bucketizing) đặc trưng. Điều này có nghĩa là chia phân phối của nó thành các nhóm có kích thước gần bằng nhau, và thay thế mỗi giá trị đặc trưng bằng chỉ số của nhóm mà nó thuộc về, giống như cách chúng ta đã làm để tạo đặc trưng `income_cat` (mặc dù chúng ta chỉ sử dụng nó để lấy mẫu phân tầng). Ví dụ, bạn có thể thay thế mỗi giá trị bằng phân vị của nó. Phân nhóm bằng các nhóm có kích thước bằng nhau tạo ra một đặc trưng với phân phối gần như đồng đều, vì vậy không cần điều chỉnh thêm, hoặc bạn có thể chỉ cần chia cho số lượng nhóm để buộc các giá trị nằm trong khoảng 0–1.

Khi một đặc trưng có phân phối đa phương thức (tức là có hai hoặc nhiều đỉnh rõ ràng, được gọi là các mode), chẳng hạn như đặc trưng `housing_median_age`, việc phân nhóm nó cũng có thể hữu ích, nhưng lần này xử lý các ID nhóm như các danh mục, thay vì các giá trị số. Điều này có nghĩa là các chỉ số nhóm phải được mã hóa, ví dụ bằng cách sử dụng `OneHotEncoder` (vì vậy bạn thường không muốn sử dụng quá nhiều nhóm). Cách tiếp cận này sẽ cho phép mô hình hồi quy dễ dàng học các quy tắc khác nhau cho các phạm vi khác nhau của giá trị đặc trưng này. Ví dụ, có thể những ngôi nhà được xây dựng cách đây khoảng 35 năm có một phong cách đặc biệt đã lỗi thời, và do đó chúng rẻ hơn so với tuổi của chúng.

Một cách tiếp cận khác để biến đổi phân phối đa phương thức là thêm một đặc trưng cho mỗi mode (ít nhất là những mode chính), đại diện cho sự tương đồng giữa tuổi trung bình của nhà và mode cụ thể đó. Thước đo sự tương đồng thường được tính bằng hàm cơ sở xuyên tâm (RBF) — bất kỳ hàm nào chỉ phụ thuộc vào khoảng cách giữa giá trị đầu vào và một điểm cố định. RBF Gaussian được sử dụng phổ biến nhất, với giá trị đầu ra giảm theo cấp số mũ khi giá trị đầu vào di chuyển ra xa điểm cố định. Ví dụ, sự tương đồng RBF Gaussian giữa tuổi nhà x và 35 được cho bởi phương trình $\exp(-\gamma(x - 35)^2)$. Siêu tham số γ (gamma) xác định tốc độ suy giảm của thước đo sự tương đồng khi x di chuyển ra xa 35. Sử dụng hàm `rbf_kernel()` của Scikit-Learn, bạn có thể tạo một đặc trưng RBF Gaussian mới đo lường sự tương đồng giữa tuổi trung bình của nhà và 35:

```
from sklearn.metrics.pairwise import rbf_kernel

age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```

Hình 2-18 cho thấy đặc trưng mới này dưới dạng hàm của tuổi trung bình của nhà (đường liền nét). Nó cũng cho thấy đặc trưng sẽ trông như thế nào nếu bạn sử dụng giá trị gamma nhỏ hơn. Như biểu đồ cho thấy, đặc trưng tương đồng tuổi mới đạt đỉnh tại 35, ngay quanh đỉnh phân phối tuổi trung bình của nhà: nếu nhóm tuổi cụ thể này tương quan tốt với giá thấp hơn, có nhiều khả năng đặc trưng mới này sẽ giúp ích.



Hình 2-18. Đặc trưng RBF Gaussian đo lường sự tương đồng giữa tuổi trung bình của nhà và 35.

Cho đến nay chúng ta mới chỉ xem xét các đặc trưng đầu vào, nhưng các giá trị mục tiêu cũng có thể cần được biến đổi. Ví dụ, nếu phân phối mục tiêu có "đuôi nặng", bạn có thể chọn thay thế mục tiêu bằng logarit của nó. Nhưng nếu bạn làm vậy, mô hình hồi quy bây giờ sẽ dự đoán logarit của giá trị nhà trung bình, chứ không phải bản thân giá trị nhà trung bình. Bạn sẽ cần tính toán lũy thừa của dự đoán của mô hình nếu bạn muốn giá trị nhà trung bình dự đoán.

May mắn thay, hầu hết các bộ biến đổi của Scikit-Learn đều có phương thức `inverse_transform()`, giúp dễ dàng tính toán nghịch đảo của các phép biến đổi của chúng. Ví dụ, đoạn mã sau đây cho thấy cách điều chỉnh các nhãn bằng `StandardScaler` (giống như chúng ta đã làm cho đầu vào), sau đó huấn luyện một mô hình hồi quy tuyến tính đơn giản trên các nhãn đã điều chỉnh và sử dụng nó để đưa ra dự đoán trên một số dữ liệu mới, mà chúng ta biến đổi trở lại thang đo gốc bằng phương thức `inverse_transform()` của bộ điều chỉnh đã huấn luyện. Lưu ý rằng chúng ta chuyển đổi các nhãn từ Pandas Series sang DataFrame, vì `StandardScaler` yêu cầu đầu vào 2D. Ngoài ra, trong ví dụ này chúng ta chỉ huấn luyện mô hình trên một đặc trưng đầu vào thô duy nhất (thu nhập trung bình), cho đơn giản:

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)
```

```
some_new_data = housing[["median_income"]].iloc[:5] # pretend this is new
data

scaled_predictions = model.predict(some_new_data)
predictions = target_scaler.inverse_transform(scaled_predictions)
```

Cách này hoạt động tốt, nhưng một lựa chọn đơn giản hơn là sử dụng `TransformedTargetRegressor`. Chúng ta chỉ cần xây dựng nó, cung cấp cho nó mô hình hồi quy và bộ biến đổi nhãn, sau đó điều chỉnh nó trên tập huấn luyện, sử dụng các nhãn gốc chưa điều chỉnh. Nó sẽ tự động sử dụng bộ biến đổi để điều chỉnh các nhãn và huấn luyện mô hình hồi quy trên các nhãn đã điều chỉnh, giống như chúng ta đã làm trước đây. Sau đó, khi chúng ta muốn đưa ra dự đoán, nó sẽ gọi phương thức `predict()` của mô hình hồi quy và sử dụng phương thức `inverse_transform()` của bộ điều chỉnh để tạo ra dự đoán:

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())

model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

2.5.4 Bộ biến đổi tùy chỉnh

Mặc dù Scikit-Learn cung cấp nhiều bộ biến đổi hữu ích, bạn sẽ cần phải viết bộ biến đổi của riêng mình cho các tác vụ như biến đổi tùy chỉnh, hoạt động làm sạch hoặc kết hợp các thuộc tính cụ thể.

Đối với các phép biến đổi không yêu cầu bất kỳ quá trình huấn luyện nào, bạn chỉ cần viết một hàm nhận một mảng NumPy làm đầu vào và xuất ra mảng đã biến đổi. Ví dụ, như đã thảo luận trong phần trước, việc biến đổi các đặc trưng có phân phối "đuôi nặng" bằng cách thay thế chúng bằng logarit của chúng thường là một ý tưởng hay (giả sử đặc trưng là dương và "đuôi" nằm ở bên phải). Hãy tạo một bộ biến đổi logarit và áp dụng nó cho đặc trưng `population`:

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

Đối số `inverse_func` là tùy chọn. Nó cho phép bạn chỉ định một hàm biến đổi ngược, ví dụ, nếu bạn định sử dụng bộ biến đổi của mình trong một `TransformedTargetRegressor`.

Hàm biến đổi của bạn có thể nhận các siêu tham số làm đối số bổ sung. Ví dụ, đây là cách tạo một bộ biến đổi tính toán cùng thước đo sự tương đồng RBF Gaussian như trước:

```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                      kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

Lưu ý rằng không có hàm nghịch đảo cho hạt nhân RBF, vì luôn có hai giá trị ở một khoảng cách nhất định từ một điểm cố định (ngoại trừ ở khoảng cách 0). Cũng lưu ý rằng `rbf_kernel()` không xử lý riêng các đặc trưng. Nếu bạn truyền cho nó một mảng có hai đặc trưng, nó sẽ đo khoảng cách 2D (Euclidean) để đo sự tương đồng. Ví dụ, đây là cách thêm một đặc trưng sẽ đo sự tương đồng địa lý giữa mỗi quận và San Francisco:

```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords], gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude", "longitude"]])
```

Các bộ biến đổi tùy chỉnh cũng hữu ích để kết hợp các đặc trưng. Ví dụ, đây là một `FunctionTransformer` tính toán tỷ lệ giữa các đặc trưng đầu vào 0 và 1:

```
>>> ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]])
>>> ratio_transformer.transform(np.array([[1., 2.], [3., 4.]]))
array([[0.5 ],
       [0.75]])
```

`FunctionTransformer` rất tiện lợi, nhưng điều gì sẽ xảy ra nếu bạn muốn bộ biến đổi của mình có thể huấn luyện được, học một số tham số trong phương thức `fit()` và sử dụng chúng sau này trong phương thức `transform()`? Để làm được điều này, bạn cần viết một lớp tùy chỉnh. Scikit-Learn dựa trên duck typing (kiểu vịt), vì vậy lớp này không cần kế thừa từ bất kỳ lớp cơ sở cụ thể nào. Tất cả những gì nó cần là ba phương thức:

`fit()` (phải trả về `self`), `transform()`, và `fit_transform()`.

Bạn có thể có `fit_transform()` miễn phí bằng cách đơn giản thêm `TransformerMixin` làm lớp cơ sở: triển khai mặc định sẽ chỉ gọi `fit()` và sau đó `transform()`. Nếu bạn thêm `BaseEstimator` làm lớp cơ sở (và tránh sử dụng `*args` và `**kwargs` trong hàm tạo của bạn), bạn cũng sẽ nhận được hai phương thức bổ sung: `get_params()` và `set_params()`. Những điều này sẽ hữu ích cho việc điều chỉnh siêu tham số tự động.

Ví dụ, đây là một bộ biến đổi tùy chỉnh hoạt động rất giống `StandardScaler`:

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_array, check_is_fitted

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True): # no *args or **kwargs!
        self.with_mean = with_mean

    def fit(self, X, y=None): # y is required even though we don't use it
        X = check_array(X) # checks that X is an array with finite float
        # values
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1] # every estimator stores this in
                                         # fit()
```

```
        return self # always return self!

    def transform(self, X):
        check_is_fitted(self) # looks for learned attributes (with trailing
                              # _)
        X = check_array(X)
        assert self.n_features_in_ == X.shape[1]
        if self.with_mean:
            X = X - self.mean_
        return X / self.scale_
```

Dưới đây là một vài điều cần lưu ý:

- Gói `sklearn.utils.validation` chứa một số hàm mà chúng ta có thể sử dụng để xác thực đầu vào. Để đơn giản, chúng ta sẽ bỏ qua các kiểm thử này trong phần còn lại của cuốn sách này, nhưng mã sản xuất nên có chúng.
- Các pipeline của Scikit-Learn yêu cầu phương thức `fit()` phải có hai đối số `X` và `y`, đó là lý do tại sao chúng ta cần đối số `y=None` mặc dù chúng ta không sử dụng `y`.
- Tất cả các ước lượng của Scikit-Learn đều đặt `n_features_in_` trong phương thức `fit()`, và chúng đảm bảo rằng dữ liệu được truyền đến `transform()` hoặc `predict()` có số lượng đặc trưng này.
- Phương thức `fit()` phải trả về `self`.
- Triển khai này chưa hoàn chỉnh 100%: tất cả các ước lượng nên đặt `feature_names_in_` trong phương thức `fit()` khi chúng được truyền một DataFrame. Hơn nữa, tất cả các bộ biến đổi nên cung cấp phương thức `get_feature_names_out()`, cũng như phương thức `inverse_transform()` khi phép biến đổi của chúng có thể được đảo ngược.

Một bộ biến đổi tùy chỉnh có thể (và thường xuyên) sử dụng các ước lượng viên khác trong triển khai của nó. Ví dụ, đoạn mã sau đây minh họa bộ biến đổi tùy chỉnh sử dụng bộ phân cụm `KMeans` trong phương thức `fit()` để xác định các cụm chính trong dữ liệu huấn luyện, và sau đó sử dụng `rbf_kernel()` trong phương thức `transform()` để đo lường mức độ tương đồng của mỗi mẫu với mỗi trung tâm cụm:

```
from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters,
                              random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self # always return self!
```

```
    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in range(self.n_clusters)]
```

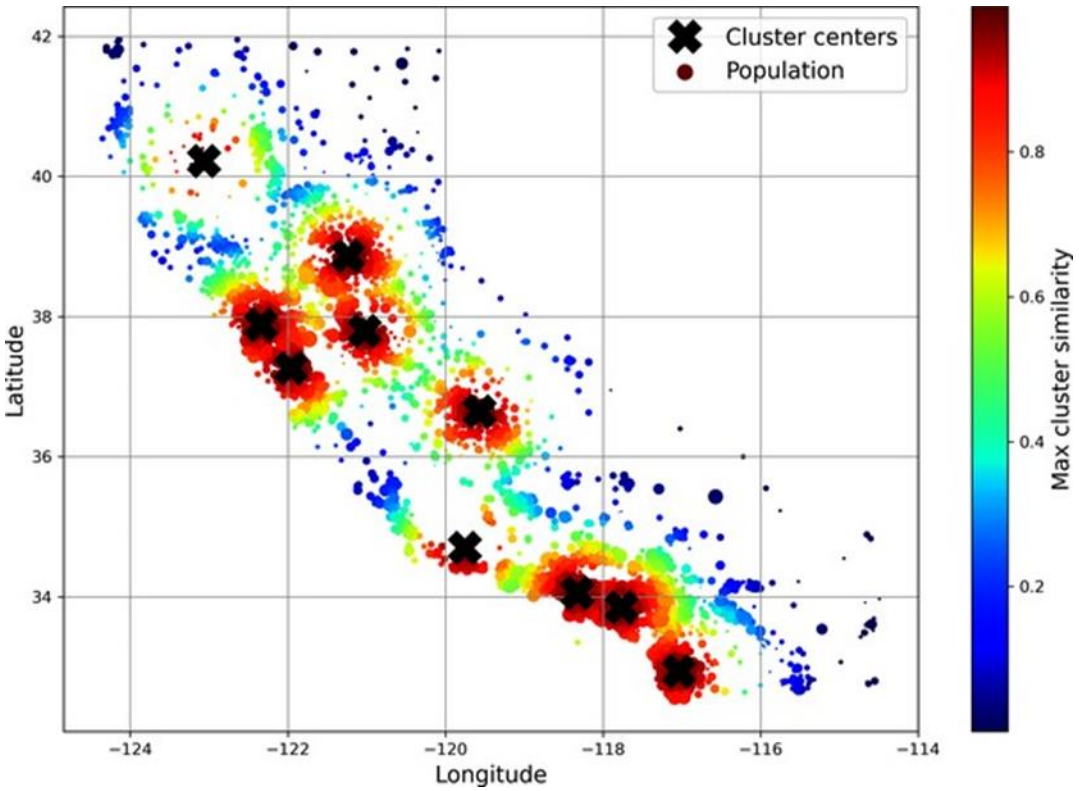
Như bạn sẽ thấy trong Chương 9, k-means là một thuật toán phân cụm tìm các cụm trong dữ liệu. Số lượng cụm mà nó tìm kiếm được kiểm soát bởi siêu tham số `n_clusters`. Sau khi huấn luyện, các tâm cụm có sẵn thông qua thuộc tính `cluster_centers_`. Phương thức `fit()` của `KMeans` hỗ trợ một đối số tùy chọn `sample_weight`, cho phép người dùng chỉ định trọng số tương đối của các mẫu . k-means là một thuật toán ngẫu nhiên, nghĩa là nó dựa vào sự ngẫu nhiên để định vị các cụm, vì vậy nếu bạn muốn kết quả có thể tái tạo, bạn phải đặt tham số `random_state`. Như bạn có thể thấy, bất chấp sự phức tạp của tác vụ, mã khá đơn giản. Bây giờ hãy sử dụng bộ biến đổi tùy chỉnh này:

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude",
"longitude"]],
                                           sample_weight=housing_labels)
```

Đoạn mã này tạo ra một bộ biến đổi `ClusterSimilarity`, đặt số cụm là 10. Sau đó, nó gọi `fit_transform()` với vĩ độ và kinh độ của mỗi quận trong tập huấn luyện, trọng số mỗi quận theo giá trị nhà trung bình của nó. Bộ biến đổi sử dụng k-means để định vị các cụm, sau đó đo lường độ tương đồng RBF Gaussian giữa mỗi quận và tất cả 10 tâm cụm. Kết quả là một ma trận với một hàng cho mỗi quận và một cột cho mỗi cụm. Hãy xem ba hàng đầu tiên, làm tròn đến hai chữ số thập phân:

```
>>> similarities[:3].round(2)

array([[0.  , 0.14, 0.  , 0.  , 0.  , 0.08, 0.  , 0.99, 0.  , 0.6 ],
       [0.63, 0.  , 0.99, 0.  , 0.  , 0.  , 0.04, 0.  , 0.11, 0.  ],
       [0.  , 0.29, 0.  , 0.  , 0.01, 0.44, 0.  , 0.7 , 0.  , 0.3 ]])
```

Hình 2-19 cho thấy 10 tâm cụm được tìm thấy bởi k-means. Các quận được tô màu theo sự tương đồng địa lý của chúng với tâm cụm gần nhất. Như bạn có thể thấy, hầu hết các cụm nằm ở các khu vực đông dân cư và đắt đỏ.



Hình 2-19. Độ tương đồng RBF Gaussian với tâm cụm gần nhất.

2.5.5 Các pipeline biến đổi

Như bạn có thể thấy, có nhiều bước biến đổi dữ liệu cần được thực hiện theo đúng thứ tự. May mắn thay, Scikit-Learn cung cấp lớp `Pipeline` để giúp với các chuỗi biến đổi như vậy. Dưới đây là một pipeline nhỏ cho các thuộc tính số, sẽ đầu tiên điền khuyết sau đó chuẩn hóa các đặc trưng đầu vào:

```
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

Hàm tạo

`Pipeline` nhận một danh sách các cặp tên/ước lượng viên (bộ 2 phần tử) định nghĩa một chuỗi các bước. Tên có thể là bất cứ thứ gì bạn muốn, miễn là chúng là duy nhất và không chứa dấu gạch dưới kép (__). Chúng sẽ hữu ích sau này, khi chúng ta thảo luận về điều chỉnh siêu tham số. Tất cả các ước lượng viên phải là bộ biến đổi (tức là chúng phải có phương thức `fit_transform()`), ngoại trừ ước lượng viên cuối cùng, có thể là bất cứ thứ gì: bộ biến đổi, bộ dự đoán hoặc bất kỳ loại ước lượng viên nào khác.

Nếu bạn không muốn đặt tên cho các bộ biến đổi, bạn có thể sử dụng hàm `make_pipeline()` thay vào đó ; nó nhận các bộ biến đổi làm đối số vị trí và tạo một `Pipeline` sử dụng tên các lớp của bộ biến đổi, viết thường và không có dấu gạch dưới (ví dụ: “simpleimputer”).

```
from sklearn.pipeline import make_pipeline

num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
StandardScaler())
```

Nếu nhiều bộ biến đổi có cùng tên, một chỉ mục sẽ được thêm vào tên của chúng (ví dụ: “foo-1”, “foo-2”, v.v.).

Khi bạn gọi phương thức `fit()` của pipeline, nó sẽ gọi `fit_transform()` tuần tự trên tất cả các bộ biến đổi, chuyển đầu ra của mỗi lần gọi làm tham số cho lần gọi tiếp theo cho đến khi nó đạt đến ước lượng cuối cùng, mà nó chỉ gọi phương thức `fit()`.

Pipeline hiển thị các phương thức tương tự như ước lượng cuối cùng. Trong ví dụ này, ước lượng cuối cùng là một `StandardScaler`, đây là một bộ biến đổi, vì vậy pipeline cũng hoạt động như một bộ biến đổi. Nếu bạn gọi phương thức `transform()` của pipeline, nó sẽ áp dụng tuần tự tất cả các phép biến đổi cho dữ liệu. Nếu ước lượng cuối cùng là một bộ dự đoán thay vì một bộ biến đổi, thì pipeline sẽ có phương thức `predict()` thay vì phương thức `transform()`. Gọi nó sẽ áp dụng tuần tự tất cả các phép biến đổi cho dữ liệu và chuyển kết quả cho phương thức `predict()` của bộ dự đoán.

Hãy gọi phương thức `fit_transform()` của pipeline và xem hai hàng đầu tiên của đầu ra, làm tròn đến hai chữ số thập phân:

```
>>> housing_num_prepared = num_pipeline.fit_transform(housing_num)
>>> housing_num_prepared[:2].round(2)
array([[-1.42, 1.01, 1.86, 0.31, 1.37, 0.14, 1.39, -0.94],
       [ 0.6 , -0.7 , 0.91, -0.31, -0.44, -0.69, -0.37, 1.17]])
```

Như bạn đã thấy trước đó, nếu bạn muốn khôi phục một DataFrame đẹp, bạn có thể sử dụng phương thức `get_feature_names_out()` của pipeline:

```
df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared, columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)
```

Pipelines hỗ trợ lập chỉ mục; ví dụ, `pipeline[1]` trả về ước lượng thứ hai trong pipeline, và `pipeline[:-1]` trả về một đối tượng `Pipeline` chứa tất cả các ước lượng ngoại trừ ước lượng cuối cùng. Bạn cũng có thể truy cập các ước lượng thông qua thuộc tính `steps`, đây là danh sách các cặp tên/ước lượng viên, hoặc thông qua thuộc tính từ điển `named_steps`, ánh xạ các tên đến các ước lượng viên. Ví dụ, `num_pipeline["simpleimputer"]` trả về ước lượng viên có tên “simpleimputer”.

Cho đến nay, chúng ta đã xử lý riêng các cột phân loại và các cột số. Sẽ thuận tiện hơn nếu có một bộ biến đổi duy nhất có khả năng xử lý tất cả các cột, áp dụng các phép biến đổi thích hợp cho từng cột. Để làm được điều này, bạn có thể sử dụng

`ColumnTransformer`. Ví dụ, `ColumnTransformer` sau đây sẽ áp dụng `num_pipeline` (cái chúng ta vừa định nghĩa) cho các thuộc tính số và `cat_pipeline` cho thuộc tính phân loại:

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",
               "total_bedrooms", "population", "households",
               "median_income"]
cat_attribs = ["ocean_proximity"]

cat_pipeline = make_pipeline(SimpleImputer(strategy="most_frequent"),
                             OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])
```

Đầu tiên, chúng ta import lớp `ColumnTransformer` , sau đó chúng ta định nghĩa danh sách tên cột số và phân loại và xây dựng một pipeline đơn giản cho các thuộc tính phân loại. Cuối cùng, chúng ta xây dựng một `ColumnTransformer`. Hàm tạo của nó yêu cầu một danh sách các bộ ba (bộ 3 phần tử), mỗi bộ ba chứa một tên (phải là duy nhất và không chứa dấu gạch dưới kép), một bộ biến đổi và một danh sách tên (hoặc chỉ mục) của các cột mà bộ biến đổi nên được áp dụng.

Vì việc liệt kê tất cả các tên cột không thuận tiện lắm, Scikit-Learn cung cấp hàm `make_column_selector()` trả về một hàm chọn mà bạn có thể sử dụng để tự động chọn tất cả các đặc trưng của một loại nhất định, chẳng hạn như số hoặc phân loại. Bạn có thể truyền hàm chọn này cho `ColumnTransformer` thay vì tên hoặc chỉ mục cột. Hơn nữa, nếu bạn không quan tâm đến việc đặt tên cho các bộ biến đổi, bạn có thể sử dụng `make_column_transformer()`, hàm này sẽ chọn tên cho bạn, giống như `make_pipeline()`. Ví dụ, đoạn mã sau tạo cùng một `ColumnTransformer` như trước, ngoại trừ các bộ biến đổi được tự động đặt tên “pipeline-1” và “pipeline-2” thay vì “num” và “cat”:

```
from sklearn.compose import make_column_selector, make_column_transformer

preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

Bây giờ chúng ta đã sẵn sàng áp dụng `ColumnTransformer` này cho dữ liệu nhà ở:

```
housing_prepared = preprocessing.fit_transform(housing)
```

Tuyệt vời! Chúng ta có một pipeline tiền xử lý nhận toàn bộ tập dữ liệu huấn luyện và áp dụng từng bộ biến đổi cho các cột thích hợp, sau đó nối các cột đã biến đổi theo chiều ngang (các bộ biến đổi không được thay đổi số lượng hàng). Một lần nữa, điều này trả về một mảng NumPy, nhưng bạn có thể lấy tên cột bằng `preprocessing.get_feature_names_out()` và gói dữ liệu trong một DataFrame đẹp như chúng ta đã làm trước đó.

Dự án của bạn đang tiến triển rất tốt và bạn gần như đã sẵn sàng để huấn luyện một số mô hình! Bây giờ bạn muốn tạo một pipeline duy nhất sẽ thực hiện tất cả các phép biến đổi mà bạn đã thử nghiệm cho đến nay. Hãy tóm tắt những gì pipeline sẽ làm và tại sao:

- Các giá trị bị thiếu trong các đặc trưng số sẽ được điền khuyết bằng cách thay thế chúng bằng giá trị trung vị, vì hầu hết các thuật toán ML không mong đợi các giá trị bị thiếu. Trong các đặc trưng phân loại, các giá trị bị thiếu sẽ được thay thế bằng danh mục thường xuyên nhất.
- Đặc trưng phân loại sẽ được mã hóa một-nóng (one-hot encoded), vì hầu hết các thuật toán ML chỉ chấp nhận đầu vào số.
- Một vài đặc trưng tỷ lệ sẽ được tính toán và thêm vào: `bedrooms_ratio`, `rooms_per_house` và `people_per_house`. Hy vọng những đặc trưng này sẽ tương quan tốt hơn với giá trị nhà trung bình, và do đó giúp các mô hình ML.
- Một vài đặc trưng tương đồng cụm cũng sẽ được thêm vào. Những đặc trưng này có khả năng hữu ích cho mô hình hơn là vĩ độ và kinh độ.
- Các đặc trưng có "đuôi dài" sẽ được thay thế bằng logarit của chúng, vì hầu hết các mô hình thích các đặc trưng có phân phối gần như đồng đều hoặc Gaussian.
- Tất cả các đặc trưng số sẽ được chuẩn hóa, vì hầu hết các thuật toán ML thích khi tất cả các đặc trưng có cùng thang đo.

Đoạn mã xây dựng pipeline để thực hiện tất cả những điều này bây giờ trông quen thuộc với bạn:

```
def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"] # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())
```

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())

preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline,
     ["total_bedrooms", "total_rooms", "population",
      "households", "median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
],
    remainder=default_num_pipeline) # one column remaining:
housing_median_age
```

Nếu bạn chạy `ColumnTransformer` này, nó sẽ thực hiện tất cả các phép biến đổi và xuất ra một mảng NumPy với 24 đặc trưng:

```
>>> housing_prepared = preprocessing.fit_transform(housing)
>>> housing_prepared.shape
(16512, 24)
>>> preprocessing.get_feature_names_out()
array(['bedrooms__ratio', 'rooms_per_house__ratio',
       'people_per_house__ratio', 'log__total_bedrooms',
       'log__total_rooms', 'log__population', 'log__households',
       'log__median_income', 'geo__Cluster 0 similarity',
       'geo__Cluster 1 similarity', 'geo__Cluster 2 similarity',
       'geo__Cluster 3 similarity', 'geo__Cluster 4 similarity',
       'geo__Cluster 5 similarity', 'geo__Cluster 6 similarity',
       'geo__Cluster 7 similarity', 'geo__Cluster 8 similarity',
       'geo__Cluster 9 similarity', 'cat__ocean_proximity_<1H OCEAN',
       'cat__ocean_proximity_INLAND', 'cat__ocean_proximity_ISLAND',
       'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR OCEAN',
       'remainder__housing_median_age'], dtype=object)
```

2.6 Chọn và Huấn luyện Mô hình

Cuối cùng! Bạn đã xác định vấn đề, có được dữ liệu và khám phá nó, lấy mẫu tập huấn luyện và tập kiểm thử, và viết một pipeline tiền xử lý để tự động làm sạch và chuẩn bị dữ liệu cho các thuật toán học máy. Bây giờ bạn đã sẵn sàng chọn và huấn luyện một mô hình học máy.

2.6.1 Huấn luyện và Đánh giá trên Tập huấn luyện

Tin tốt là nhờ tất cả các bước trước đó, mọi thứ bây giờ sẽ dễ dàng! Bạn quyết định huấn luyện một mô hình hồi quy tuyến tính rất cơ bản để bắt đầu.

```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)
```

Hoàn tất! Bây giờ bạn đã có một mô hình hồi quy tuyến tính hoạt động. Bạn thử nó trên tập huấn luyện, xem năm dự đoán đầu tiên và so sánh chúng với các nhãn:

```
>>> housing_predictions = lin_reg.predict(housing)
>>> housing_predictions[:5].round(-2) # -2 = rounded to the nearest hundred
array([243700., 372400., 128800.,  94400., 328300.])

>>> housing_labels.iloc[:5].values
array([458300., 483800., 101700.,  96100., 361800.])
```

Vâng, nó hoạt động, nhưng không phải lúc nào cũng vậy: dự đoán đầu tiên sai lệch rất nhiều (hơn 200.000 đô la!), trong khi các dự đoán khác tốt hơn: hai dự đoán sai lệch khoảng 25%, và hai dự đoán sai lệch dưới 10%. Hãy nhớ rằng bạn đã chọn sử dụng RMSE làm thước đo hiệu suất của mình, vì vậy bạn muốn đo RMSE của mô hình hồi quy này trên toàn bộ tập huấn luyện bằng hàm

`mean_squared_error()` của Scikit-Learn, với đối số `squared` được đặt thành `False`:

```
>>> from sklearn.metrics import mean_squared_error

>>> lin_rmse = mean_squared_error(housing_labels, housing_predictions,
...                               squared=False)
...
>>> lin_rmse
68687.89176589991
```

Điều này tốt hơn không có gì, nhưng rõ ràng không phải là một điểm số tuyệt vời:

`median_housing_values` của hầu hết các quận nằm trong khoảng từ 120.000 đô la đến 265.000 đô la, vì vậy lỗi dự đoán điển hình là 68.628 đô la thực sự không thỏa mãn lắm. Đây là một ví dụ về mô hình dưới khớp dữ liệu huấn luyện. Khi điều này xảy ra, nó có thể có nghĩa là các đặc trưng không cung cấp đủ thông tin để đưa ra dự đoán tốt, hoặc mô hình không đủ mạnh. Như chúng ta đã thấy trong chương trước, các cách chính để khắc phục dưới khớp là chọn một mô hình mạnh hơn, cung cấp cho thuật toán huấn luyện các đặc trưng tốt hơn, hoặc giảm các ràng buộc trên mô hình. Mô hình này không được chính quy hóa, điều này loại trừ lựa chọn cuối cùng. Bạn có thể thử thêm các đặc trưng, nhưng trước tiên bạn muốn thử một mô hình phức tạp hơn để xem nó hoạt động như thế nào.

Bạn quyết định thử

`DecisionTreeRegressor`, vì đây là một mô hình khá mạnh mẽ có khả năng tìm ra các mối quan hệ phi tuyến phức tạp trong dữ liệu (cây quyết định được trình bày chi tiết hơn trong Chương 6).

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing,
DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

Bây giờ mô hình đã được huấn luyện, bạn đánh giá nó trên tập huấn luyện:

```
>>> housing_predictions = tree_reg.predict(housing)
>>> tree_rmse = mean_squared_error(housing_labels, housing_predictions,
...                                squared=False)
...
>>> tree_rmse
0.0
```

Khoan đã!? Không có lỗi nào cả? Liệu mô hình này có thực sự hoàn hảo tuyệt đối không? Tất nhiên, nhiều khả năng mô hình đã quá khớp dữ liệu một cách nghiêm trọng. Làm thế nào bạn có thể chắc chắn? Như bạn đã thấy trước đây, bạn không muốn chạm vào tập kiểm thử cho đến khi bạn sẵn sàng triển khai một mô hình mà bạn tự tin, vì vậy bạn cần sử dụng một phần tập huấn luyện để huấn luyện và một phần để xác thực mô hình.

2.6.2 Đánh giá tốt hơn bằng kiểm định chéo

Một cách để đánh giá mô hình cây quyết định là sử dụng hàm `train_test_split()` để chia tập huấn luyện thành một tập huấn luyện nhỏ hơn và một tập xác thực, sau đó huấn luyện các mô hình của bạn trên tập huấn luyện nhỏ hơn và đánh giá chúng trên tập xác thực. Điều này hơi tốn công sức, nhưng không quá khó, và nó sẽ hoạt động khá tốt.

Một lựa chọn thay thế tuyệt vời là sử dụng tính năng kiểm định chéo k-fold của Scikit-Learn. Đoạn mã sau đây ngẫu nhiên chia tập huấn luyện thành 10 tập con không chồng lấn được gọi là “folds”, sau đó nó huấn luyện và đánh giá mô hình cây quyết định 10 lần, chọn một fold khác để đánh giá mỗi lần và sử dụng 9 folds còn lại để huấn luyện. Kết quả là một mảng chứa 10 điểm đánh giá:

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
```

Hãy xem kết quả:

```
>>> pd.Series(tree_rmses).describe()
count       10.000000
mean     66868.027288
std       2060.966425
min      63649.536493
25%      65338.078316
50%      66801.953094
75%      68229.934454
```

```
max      70094.778246
dtype: float64
```

Bây giờ cây quyết định không còn tốt như trước nữa. Trên thực tế, nó dường như hoạt động kém gần như mô hình hồi quy tuyến tính! Lưu ý rằng kiểm định chéo cho phép bạn không chỉ có được ước tính hiệu suất của mô hình, mà còn cả thước đo độ chính xác của ước tính này (tức là độ lệch chuẩn của nó). Cây quyết định có RMSE khoảng 66.868, với độ lệch chuẩn khoảng 2.061. Bạn sẽ không có thông tin này nếu bạn chỉ sử dụng một tập xác thực. Nhưng kiểm định chéo phải trả giá bằng việc huấn luyện mô hình nhiều lần, vì vậy không phải lúc nào cũng khả thi.

Nếu bạn tính toán cùng một số liệu cho mô hình hồi quy tuyến tính, bạn sẽ thấy rằng RMSE trung bình là 69.858 và độ lệch chuẩn là 4.182. Vì vậy, mô hình cây quyết định dường như hoạt động tốt hơn một chút so với mô hình tuyến tính, nhưng sự khác biệt là rất nhỏ do hiện tượng quá khớp nghiêm trọng. Chúng ta biết có vấn đề quá khớp vì lỗi huấn luyện thấp (thực tế là bằng 0) trong khi lỗi xác thực cao.

Hãy thử một mô hình cuối cùng: `RandomForestRegressor`. Như bạn sẽ thấy trong Chương 7, rừng ngẫu nhiên hoạt động bằng cách huấn luyện nhiều cây quyết định trên các tập con ngẫu nhiên của các đặc trưng, sau đó lấy trung bình các dự đoán của chúng. Các mô hình như vậy được tạo thành từ nhiều mô hình khác được gọi là tập hợp (ensembles): chúng có khả năng tăng cường hiệu suất của mô hình cơ bản (trong trường hợp này là cây quyết định). Đoạn mã cũng tương tự như trước:

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                           RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error",
                                cv=10)
```

Hãy xem điểm số:

```
>>> pd.Series(forest_rmses).describe()
count       10.000000
mean     47019.561281
std       1033.957120
min      45458.112527
25%      46464.031184
50%      46967.596354
75%      47325.694987
max      49243.765795
dtype: float64
```

Chà, điều này tốt hơn nhiều: rừng ngẫu nhiên thực sự trông rất hứa hẹn cho nhiệm vụ này! Tuy nhiên, nếu bạn huấn luyện một `RandomForest` và đo RMSE trên tập huấn luyện, bạn sẽ thấy khoảng 17.474: con số đó thấp hơn nhiều, có nghĩa là vẫn còn khá nhiều hiện tượng

quá khớp đang diễn ra. Các giải pháp khả thi là đơn giản hóa mô hình, ràng buộc nó (tức là chính quy hóa nó), hoặc thu thập thêm nhiều dữ liệu huấn luyện. Tuy nhiên, trước khi đi sâu hơn vào rừng ngẫu nhiên, bạn nên thử nhiều mô hình khác từ các loại thuật toán học máy khác nhau (ví dụ: một số máy vector hỗ trợ với các hạt nhân khác nhau, và có thể là một mạng nơ-ron), mà không tốn quá nhiều thời gian điều chỉnh các siêu tham số. Mục tiêu là chọn lọc một vài mô hình (hai đến năm) hứa hẹn.

2.7 Tinh chỉnh mô hình của bạn

Giả sử bạn hiện có một danh sách ngắn các mô hình đầy hứa hẹn. Bây giờ bạn cần tinh chỉnh chúng. Hãy xem một vài cách bạn có thể làm điều đó.

2.7.1 Tìm kiếm theo lưới (Grid Search)

Một lựa chọn là tự điều chỉnh các siêu tham số thủ công, cho đến khi bạn tìm thấy một sự kết hợp tuyệt vời của các giá trị siêu tham số. Đây sẽ là một công việc rất tẻ nhạt, và bạn có thể không có thời gian để khám phá nhiều sự kết hợp. Thay vào đó, bạn có thể sử dụng lớp `GridSearchCV` của Scikit-Learn để tìm kiếm giúp bạn. Tất cả những gì bạn cần làm là cho nó biết bạn muốn nó thử nghiệm với siêu tham số nào và giá trị nào để thử, và nó sẽ sử dụng kiểm định chéo để đánh giá tất cả các sự kết hợp có thể của các giá trị siêu tham số. Ví dụ, đoạn mã sau tìm kiếm sự kết hợp tốt nhất của các giá trị siêu tham số cho

`RandomForestRegressor`:

```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])

param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],
     'random_forest__max_features': [6, 8, 10]},
]

grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

Lưu ý rằng bạn có thể tham chiếu đến bất kỳ siêu tham số nào của bất kỳ ước lượng viên nào trong một pipeline, ngay cả khi ước lượng viên này nằm sâu bên trong nhiều pipeline và bộ biến đổi cột. Ví dụ, khi Scikit-Learn thấy “preprocessing__geo__n_clusters”, nó sẽ chia chuỗi này tại dấu gạch dưới kép, sau đó nó tìm kiếm một ước lượng viên có tên “preprocessing” trong pipeline và tìm thấy `ColumnTransformer` tiền xử lý. Tiếp theo, nó tìm kiếm một bộ biến đổi có tên “geo” bên trong `ColumnTransformer` này và tìm thấy bộ biến

đổi ClusterSimilarity mà chúng ta đã sử dụng trên các thuộc tính vĩ độ và kinh độ. Sau đó, nó tìm thấy siêu tham số n_clusters của bộ biến đổi này. Tương tự, random_forest__max_features đề cập đến siêu tham số max_features của ước lượng viên có tên "random_forest", đây tất nhiên là mô hình RandomForest (siêu tham số max_features sẽ được giải thích trong Chương 7).

Có hai từ điển trong param_grid này, vì vậy GridSearchCV sẽ đầu tiên đánh giá tất cả 3 × 3 = 9 sự kết hợp của các giá trị siêu tham số n_clusters và max_features được chỉ định trong từ điển đầu tiên, sau đó nó sẽ thử tất cả 2 × 3 = 6 sự kết hợp của các giá trị siêu tham số trong từ điển thứ hai. Vì vậy, tổng cộng tìm kiếm lưới sẽ khám phá 9 + 6 = 15 sự kết hợp của các giá trị siêu tham số, và nó sẽ huấn luyện pipeline 3 lần cho mỗi sự kết hợp, vì chúng ta đang sử dụng kiểm định chéo 3-fold. Điều này có nghĩa là sẽ có tổng cộng 15 × 3 = 45 vòng huấn luyện! Nó có thể mất một lúc, nhưng khi hoàn tất, bạn có thể nhận được sự kết hợp tham số tốt nhất như sau:

```
>>> grid_search.best_params_
{'preprocessing__geo__n_clusters': 15, 'random_forest__max_features': 6}
```

Trong ví dụ này, mô hình tốt nhất được lấy bằng cách đặt n_clusters thành 15 và đặt max_features thành 8.

Bạn có thể truy cập ước lượng viên tốt nhất bằng cách sử dụng grid_search.best_estimator_. Nếu GridSearchCV được khởi tạo với refit=True (là giá trị mặc định), thì một khi nó tìm thấy ước lượng viên tốt nhất bằng cách sử dụng kiểm định chéo, nó sẽ huấn luyện lại trên toàn bộ tập huấn luyện. Điều này thường là một ý tưởng hay, vì việc cung cấp thêm dữ liệu có khả năng cải thiện hiệu suất của nó.

Các điểm đánh giá có sẵn bằng grid_search.cv_results_. Đây là một từ điển, nhưng nếu bạn gói nó trong một DataFrame, bạn sẽ nhận được một danh sách đẹp của tất cả các điểm kiểm thử cho mỗi sự kết hợp siêu tham số và cho mỗi phân tách kiểm định chéo, cũng như điểm kiểm thử trung bình trên tất cả các phân tách:

```
>>> cv_res = pd.DataFrame(grid_search.cv_results_)
>>> cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
>>> [...] # change column names to fit on this page, and show rmse = -score
>>> cv_res.head() # note: the 1st column is the row ID
    n_clusters  max_features  split0  split1  split2  mean_test_rmse
12          15             6   43460   43919   44748           44042
13          15             8   44132   44075   45010           44406
14          15            10   44374   44286   45316           44659
7           10             6   44683   44655   45657           44999
9           10             6   44683   44655   45657           44999
```

Điểm RMSE kiểm thử trung bình cho mô hình tốt nhất là 44.042, tốt hơn điểm bạn nhận được trước đó bằng cách sử dụng các giá trị siêu tham số mặc định (là 47.019). Chúc mừng, bạn đã tinh chỉnh thành công mô hình tốt nhất của mình!

2.7.2 Tìm kiếm ngẫu nhiên

Cách tiếp cận tìm kiếm theo lưới (grid search) phù hợp khi bạn khám phá tương đối ít các sự kết hợp, như trong ví dụ trước, nhưng `RandomizedSearchCV` thường được ưu tiên hơn, đặc biệt khi không gian tìm kiếm siêu tham số lớn. Lớp này có thể được sử dụng theo cách tương tự như lớp `GridSearchCV`, nhưng thay vì thử tất cả các sự kết hợp có thể, nó đánh giá một số lượng cố định các sự kết hợp, chọn một giá trị ngẫu nhiên cho mỗi siêu tham số ở mỗi lần lặp. Điều này nghe có vẻ đáng ngạc nhiên, nhưng cách tiếp cận này có một số lợi ích:

- Nếu một số siêu tham số của bạn là liên tục (hoặc rời rạc nhưng có nhiều giá trị có thể), và bạn cho tìm kiếm ngẫu nhiên chạy trong, giả sử, 1.000 lần lặp, thì nó sẽ khám phá 1.000 giá trị khác nhau cho mỗi siêu tham số này, trong khi tìm kiếm theo lưới sẽ chỉ khám phá một vài giá trị bạn đã liệt kê cho mỗi siêu tham số.
- Giả sử một siêu tham số thực sự không tạo ra nhiều khác biệt, nhưng bạn chưa biết điều đó. Nếu nó có 10 giá trị có thể và bạn thêm nó vào tìm kiếm theo lưới của mình, thì quá trình huấn luyện sẽ mất thời gian gấp 10 lần. Nhưng nếu bạn thêm nó vào tìm kiếm ngẫu nhiên, nó sẽ không tạo ra bất kỳ khác biệt nào.
- Nếu có 6 siêu tham số cần khám phá, mỗi siêu tham số có 10 giá trị có thể, thì tìm kiếm theo lưới không có lựa chọn nào khác ngoài việc huấn luyện mô hình một triệu lần, trong khi tìm kiếm ngẫu nhiên luôn có thể chạy với bất kỳ số lần lặp nào bạn chọn.

Đối với mỗi siêu tham số, bạn phải cung cấp một danh sách các giá trị có thể, hoặc một phân phối xác suất:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                  'random_forest__max_features': randint(low=2, high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distribs, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

Scikit-Learn cũng có các lớp tìm kiếm siêu tham số `HalvingRandomSearchCV` và `HalvingGridSearchCV`. Mục tiêu của chúng là sử dụng tài nguyên tính toán hiệu quả hơn, để huấn luyện nhanh hơn hoặc để khám phá không gian siêu tham số lớn hơn. Đây là cách chúng hoạt động: trong vòng đầu tiên, nhiều sự kết hợp siêu tham số (được gọi là “ứng cử viên”) được tạo ra bằng cách sử dụng cách tiếp cận lưới hoặc cách tiếp cận ngẫu nhiên. Các ứng cử viên này sau đó được sử dụng để huấn luyện các mô hình được đánh giá bằng kiểm định chéo, như thường lệ. Tuy nhiên, việc huấn luyện sử dụng tài nguyên hạn chế, điều này làm tăng tốc vòng đầu tiên đáng kể. Theo mặc định, “tài nguyên hạn chế” có nghĩa

là các mô hình được huấn luyện trên một phần nhỏ của tập huấn luyện. Tuy nhiên, các giới hạn khác cũng có thể, chẳng hạn như giảm số lần lặp huấn luyện nếu mô hình có một siêu tham số để đặt nó. Sau khi mỗi ứng cử viên đã được đánh giá, chỉ những ứng cử viên tốt nhất mới tiếp tục sang vòng thứ hai, nơi chúng được phép sử dụng nhiều tài nguyên hơn để cạnh tranh. Sau vài vòng, các ứng cử viên cuối cùng được đánh giá bằng cách sử dụng tài nguyên đầy đủ. Điều này có thể giúp bạn tiết kiệm thời gian điều chỉnh siêu tham số.

2.7.3 Các phương pháp tập hợp (Ensemble Methods)

Một cách khác để tinh chỉnh hệ thống của bạn là cố gắng kết hợp các mô hình hoạt động tốt nhất. Nhóm (hoặc "tập hợp") thường sẽ hoạt động tốt hơn mô hình riêng lẻ tốt nhất — giống như rừng ngẫu nhiên hoạt động tốt hơn các cây quyết định riêng lẻ mà chúng dựa vào — đặc biệt nếu các mô hình riêng lẻ mắc các loại lỗi rất khác nhau. Ví dụ, bạn có thể huấn luyện và tinh chỉnh mô hình k-nearest neighbors, sau đó tạo một mô hình tập hợp chỉ dự đoán trung bình của dự đoán rừng ngẫu nhiên và dự đoán của mô hình đó. Chúng ta sẽ đề cập chi tiết hơn về chủ đề này trong Chương 7.

2.7.4 Phân tích các mô hình tốt nhất và lỗi của chúng

Bạn sẽ thường có được cái nhìn sâu sắc về vấn đề bằng cách kiểm tra các mô hình tốt nhất. Ví dụ, `RandomForestRegressor` có thể chỉ ra tầm quan trọng tương đối của mỗi thuộc tính để đưa ra dự đoán chính xác:

```
>>> final_model = rnd_search.best_estimator_ # includes preprocessing

>>> feature_importances = final_model["random_forest"].feature_importances_

>>> feature_importances.round(2)
array([0.07, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.19, [...], 0.01])
```

Hãy sắp xếp các điểm tầm quan trọng này theo thứ tự giảm dần và hiển thị chúng bên cạnh tên thuộc tính tương ứng của chúng:

```
>>> sorted(zip(feature_importances,
...              final_model["preprocessing"].get_feature_names_out()),
...              reverse=True)
...
[(0.18694559869103852, 'log__median_income'),
(0.0748194905715524, 'cat__ocean_proximity_INLAND'),
(0.06926417748515576, 'bedrooms__ratio'),
(0.05446998753775219, 'rooms_per_house__ratio'),
(0.05262301809680712, 'people_per_house__ratio'),
(0.03819415873915732, 'geo__Cluster 0 similarity'),
[...]
(0.00015061247730531558, 'cat__ocean_proximity_NEAR BAY'),
(7.301686597099842e-05, 'cat__ocean_proximity_ISLAND')]
```

Với thông tin này, bạn có thể muốn thử loại bỏ một số đặc trưng ít hữu ích hơn (ví dụ: rõ ràng chỉ có một danh mục `ocean_proximity` thực sự hữu ích, vì vậy bạn có thể thử loại bỏ những cái khác).

Bạn cũng nên xem xét các lỗi cụ thể mà hệ thống của bạn mắc phải, sau đó cố gắng hiểu tại sao nó mắc lỗi và cách khắc phục vấn đề: thêm các đặc trưng bổ sung hoặc loại bỏ những đặc trưng không cung cấp thông tin, làm sạch các giá trị ngoại lai, v.v.

Bây giờ cũng là thời điểm tốt để đảm bảo rằng mô hình của bạn không chỉ hoạt động tốt ở mức trung bình, mà còn trên tất cả các danh mục quận, dù là nông thôn hay thành thị, giàu hay nghèo, miền bắc hay miền nam, thiểu số hay không, v.v. Việc tạo các tập con của tập xác thực của bạn cho mỗi danh mục tốn một chút công sức, nhưng điều đó quan trọng: nếu mô hình của bạn hoạt động kém trên toàn bộ một danh mục quận, thì có lẽ nó không nên được triển khai cho đến khi vấn đề được giải quyết, hoặc ít nhất nó không nên được sử dụng để đưa ra dự đoán cho danh mục đó, vì nó có thể gây hại nhiều hơn lợi.

2.7.5 Đánh giá hệ thống của bạn trên tập kiểm thử

Sau khi điều chỉnh mô hình của bạn một thời gian, cuối cùng bạn có một hệ thống hoạt động đủ tốt. Bạn đã sẵn sàng đánh giá mô hình cuối cùng trên tập kiểm thử. Không có gì đặc biệt về quy trình này; chỉ cần lấy các biến dự đoán và nhãn từ tập kiểm thử của bạn và chạy `final_model` của bạn để biến đổi dữ liệu và đưa ra dự đoán, sau đó đánh giá các dự đoán này:

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)

final_rmse = mean_squared_error(y_test, final_predictions, squared=False)
print(final_rmse) # prints 41424.40026462184
```

Trong một số trường hợp, ước tính điểm lỗi tổng quát hóa như vậy sẽ không đủ để thuyết phục bạn triển khai: điều gì sẽ xảy ra nếu nó chỉ tốt hơn 0.1% so với mô hình hiện đang được sản xuất? Bạn có thể muốn có một ý tưởng về độ chính xác của ước tính này. Để làm được điều này, bạn có thể tính toán khoảng tin cậy 95% cho lỗi tổng quát hóa bằng cách sử dụng `scipy.stats.t.interval()`. Bạn nhận được một khoảng khá rộng từ 39.275 đến 43.467, và ước tính điểm trước đó của bạn là 41.424 nằm gần giữa khoảng đó:

```
>>> from scipy import stats

>>> confidence = 0.95

>>> squared_errors = (final_predictions - y_test) ** 2

>>> np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
...                          loc=squared_errors.mean(),
...                          scale=stats.sem(squared_errors)))
```

```
...
array([39275.40861216, 43467.27680583])
```

Nếu bạn đã điều chỉnh siêu tham số rất nhiều, hiệu suất thường sẽ hơi tệ hơn so với những gì bạn đã đo bằng kiểm định chéo. Điều đó là do hệ thống của bạn đã được tinh chỉnh để hoạt động tốt trên dữ liệu xác thực và có khả năng sẽ không hoạt động tốt trên các tập dữ liệu không xác định. Điều đó không đúng trong ví dụ này vì RMSE kiểm thử thấp hơn RMSE xác thực, nhưng khi điều đó xảy ra, bạn phải chống lại sự cám dỗ để điều chỉnh siêu tham số để làm cho các con số trông đẹp trên tập kiểm thử; những cải tiến đó sẽ không có khả năng tổng quát hóa sang dữ liệu mới.

Bây giờ là giai đoạn trước khi triển khai dự án: bạn cần trình bày giải pháp của mình (làm nổi bật những gì bạn đã học được, những gì hoạt động và những gì không, những giả định đã đưa ra và những hạn chế của hệ thống của bạn), ghi lại mọi thứ và tạo các bài thuyết trình đẹp mắt với các hình ảnh trực quan rõ ràng và các tuyên bố dễ nhớ (ví dụ: “thu nhập trung bình là yếu tố dự đoán số một về giá nhà ở”). Trong ví dụ về giá nhà ở California này, hiệu suất cuối cùng của hệ thống không tốt hơn nhiều so với ước tính giá của các chuyên gia, vốn thường sai lệch 30%, nhưng vẫn có thể là một ý tưởng hay để triển khai, đặc biệt nếu điều này giúp các chuyên gia có thêm thời gian để làm những nhiệm vụ thú vị và hiệu quả hơn.

2.8 Triển khai, Giám sát và Bảo trì hệ thống của bạn

Hoàn hảo, bạn đã được chấp thuận triển khai! Bây giờ bạn cần chuẩn bị giải pháp của mình cho môi trường sản xuất (ví dụ: làm sạch mã, viết tài liệu và kiểm thử, v.v.). Sau đó, bạn có thể triển khai mô hình của mình vào môi trường sản xuất. Cách cơ bản nhất để làm điều này là chỉ cần lưu mô hình tốt nhất mà bạn đã huấn luyện, chuyển tệp sang môi trường sản xuất của bạn và tải nó. Để lưu mô hình, bạn có thể sử dụng thư viện `joblib` như sau:

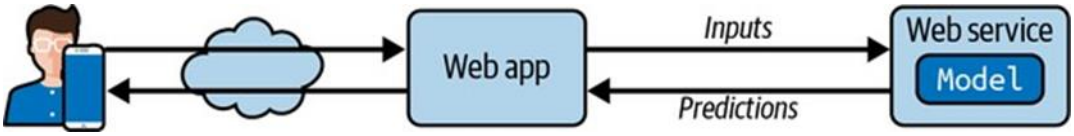
```
import joblib
joblib.dump(final_model, "my_california_housing_model.pkl")
```

Khi mô hình của bạn được chuyển sang môi trường sản xuất, bạn có thể tải nó và sử dụng nó. Để làm điều này, trước tiên bạn phải import bất kỳ lớp và hàm tùy chỉnh nào mà mô hình phụ thuộc vào (nghĩa là chuyển mã sang môi trường sản xuất), sau đó tải mô hình bằng `joblib` và sử dụng nó để đưa ra dự đoán:

```
[...] # import KMeans, BaseEstimator, TransformerMixin, rbf_kernel, etc.
def column_ratio(X): [...]
def ratio_name(function_transformer, feature_names_in): [...]
class ClusterSimilarity(BaseEstimator, TransformerMixin): [...]

final_model_reloaded = joblib.load("my_california_housing_model.pkl")
new_data = [...] # some new districts to make predictions for
predictions = final_model_reloaded.predict(new_data)
```

Ví dụ, mô hình có thể được sử dụng trong một trang web: người dùng sẽ nhập một số dữ liệu về một quận mới và nhấp vào nút “Estimate Price”. Thao tác này sẽ gửi một truy vấn

chứa dữ liệu đến máy chủ web, máy chủ này sẽ chuyển tiếp đến ứng dụng web của bạn và cuối cùng mã của bạn sẽ chỉ đơn giản gọi phương thức `predict()` của mô hình (bạn muốn tải mô hình khi máy chủ khởi động, thay vì mỗi khi mô hình được sử dụng). Ngoài ra, bạn có thể gói mô hình trong một dịch vụ web chuyên dụng mà ứng dụng web của bạn có thể truy vấn thông qua API REST (xem Hình 2-20). Điều này giúp dễ dàng nâng cấp mô hình của bạn lên các phiên bản mới mà không làm gián đoạn ứng dụng chính. Nó cũng đơn giản hóa việc mở rộng quy mô, vì bạn có thể khởi động nhiều dịch vụ web khi cần và cân bằng tải các yêu cầu đến từ ứng dụng web của bạn trên các dịch vụ web này. Hơn nữa, nó cho phép ứng dụng web của bạn sử dụng bất kỳ ngôn ngữ lập trình nào, không chỉ Python.



Hình 2-20. Một mô hình được triển khai dưới dạng dịch vụ web và được sử dụng bởi ứng dụng web.

Một chiến lược phổ biến khác là triển khai mô hình của bạn lên đám mây, ví dụ trên Google Vertex AI (trước đây được gọi là Google Cloud AI Platform và Google Cloud ML Engine): chỉ cần lưu mô hình của bạn bằng `joblib` và tải nó lên Google Cloud Storage (GCS), sau đó truy cập Vertex AI và tạo một phiên bản mô hình mới, trỏ nó đến tệp GCS. Vậy là xong! Điều này mang lại cho bạn một dịch vụ web đơn giản tự động cân bằng tải và mở rộng quy mô cho bạn. Nó nhận các yêu cầu JSON chứa dữ liệu đầu vào (ví dụ: của một quận) và trả về các phản hồi JSON chứa các dự đoán. Sau đó, bạn có thể sử dụng dịch vụ web này trong trang web của mình (hoặc bất kỳ môi trường sản xuất nào bạn đang sử dụng). Như bạn sẽ thấy trong Chương 19, việc triển khai các mô hình TensorFlow trên Vertex AI không khác nhiều so với việc triển khai các mô hình Scikit-Learn.

Nhưng việc triển khai không phải là kết thúc của câu chuyện. Bạn cũng cần viết mã giám sát để kiểm tra hiệu suất trực tiếp của hệ thống theo định kỳ và kích hoạt cảnh báo khi hiệu suất giảm. Nó có thể giảm rất nhanh, ví dụ nếu một thành phần bị hỏng trong cơ sở hạ tầng của bạn, nhưng hãy lưu ý rằng nó cũng có thể suy giảm rất chậm, điều này có thể dễ dàng không được chú ý trong một thời gian dài. Điều này khá phổ biến do “sự phân rã mô hình” (model rot): nếu mô hình được huấn luyện bằng dữ liệu năm ngoái, nó có thể không phù hợp với dữ liệu ngày nay.

Vì vậy, bạn cần giám sát hiệu suất trực tiếp của mô hình của bạn. Nhưng làm thế nào để bạn làm điều đó? Chà, điều đó phụ thuộc. Trong một số trường hợp, hiệu suất của mô hình có thể được suy ra từ các chỉ số hạ nguồn. Ví dụ, nếu mô hình của bạn là một phần của hệ thống đề xuất và nó đề xuất các sản phẩm mà người dùng có thể quan tâm, thì việc giám sát số lượng sản phẩm được đề xuất bán ra mỗi ngày là dễ dàng. Nếu con số này giảm (so với các sản phẩm không được đề xuất), thì nghi phạm chính là mô hình. Điều này có thể là do pipeline dữ liệu bị hỏng, hoặc có lẽ mô hình cần được huấn luyện lại trên dữ liệu mới (như chúng ta sẽ thảo luận ngắn gọn). Tuy nhiên, bạn cũng có thể cần phân tích thủ công để đánh giá hiệu suất của mô hình. Ví dụ, giả sử bạn đã huấn luyện một mô hình phân loại

hình ảnh (chúng ta sẽ xem xét các mô hình này trong Chương 3) để phát hiện các lỗi sản phẩm khác nhau trên dây chuyền sản xuất. Làm thế nào bạn có thể nhận được cảnh báo nếu hiệu suất của mô hình giảm, trước khi hàng nghìn sản phẩm bị lỗi được vận chuyển đến khách hàng của bạn? Một giải pháp là gửi một mẫu tất cả các hình ảnh mà mô hình đã phân loại (đặc biệt là những hình ảnh mà mô hình không chắc chắn lắm) cho những người đánh giá thủ công. Tùy thuộc vào tác vụ, những người đánh giá có thể cần phải là chuyên gia, hoặc họ có thể là những người không chuyên, chẳng hạn như những người làm việc trên nền tảng crowdsourcing (ví dụ: Amazon Mechanical Turk). Trong một số ứng dụng, họ thậm chí có thể là chính người dùng, phản hồi, ví dụ, thông qua các cuộc khảo sát hoặc các captcha được sửa đổi.

Bằng cách này hay cách khác, bạn cần thiết lập một hệ thống giám sát (có hoặc không có người đánh giá thủ công để đánh giá mô hình trực tiếp), cũng như tất cả các quy trình liên quan để xác định phải làm gì trong trường hợp lỗi và cách chuẩn bị cho chúng. Thật không may, điều này có thể tốn rất nhiều công sức. Trên thực tế, nó thường tốn nhiều công sức hơn cả việc xây dựng và huấn luyện một mô hình.

Nếu dữ liệu tiếp tục thay đổi, bạn sẽ cần cập nhật tập dữ liệu của mình và huấn luyện lại mô hình của bạn thường xuyên. Bạn có thể nên tự động hóa toàn bộ quá trình càng nhiều càng tốt. Dưới đây là một vài điều bạn có thể tự động hóa:

- Thu thập dữ liệu mới thường xuyên và gắn nhãn cho nó (ví dụ: sử dụng người đánh giá thủ công).
- Viết một script để huấn luyện mô hình và tinh chỉnh các siêu tham số tự động. Script này có thể chạy tự động, ví dụ mỗi ngày hoặc mỗi tuần, tùy thuộc vào nhu cầu của bạn.
- Viết một script khác sẽ đánh giá cả mô hình mới và mô hình trước đó trên tập kiểm thử đã cập nhật, và triển khai mô hình vào sản xuất nếu hiệu suất không giảm (nếu có, hãy đảm bảo bạn điều tra lý do). Script có lẽ nên kiểm thử hiệu suất của mô hình của bạn trên các tập con khác nhau của tập kiểm thử, chẳng hạn như các quận nghèo hoặc giàu, các quận nông thôn hoặc thành thị, v.v.

Bạn cũng nên đảm bảo rằng bạn đánh giá chất lượng dữ liệu đầu vào của mô hình. Đôi khi hiệu suất sẽ suy giảm nhẹ do tín hiệu kém chất lượng (ví dụ: một cảm biến bị trục trặc gửi các giá trị ngẫu nhiên, hoặc đầu ra của một nhóm khác trở nên cũ), nhưng có thể mất một thời gian trước khi hiệu suất của hệ thống của bạn suy giảm đủ để kích hoạt cảnh báo. Nếu bạn giám sát đầu vào của mô hình, bạn có thể phát hiện điều này sớm hơn. Ví dụ, bạn có thể kích hoạt cảnh báo nếu ngày càng nhiều đầu vào thiếu một đặc trưng, hoặc giá trị trung bình hoặc độ lệch chuẩn trôi quá xa so với tập huấn luyện, hoặc một đặc trưng phân loại bắt đầu chứa các danh mục mới. Cuối cùng, hãy đảm bảo bạn giữ bản sao lưu của mọi mô hình bạn tạo và có quy trình và công cụ để khôi phục nhanh chóng về mô hình trước đó, trong trường hợp mô hình mới bắt đầu hoạt động kém vì lý do nào đó. Việc có bản sao lưu cũng giúp dễ dàng so sánh các mô hình mới với các mô hình trước đó. Tương tự, bạn nên giữ bản sao lưu của mọi phiên bản tập dữ liệu của mình để bạn có thể khôi phục về tập dữ liệu trước đó nếu tập dữ liệu mới bị hỏng (ví dụ: nếu dữ liệu mới được thêm vào hóa ra đầy

các giá trị ngoại lai). Việc có bản sao lưu tập dữ liệu cũng cho phép bạn đánh giá bất kỳ mô hình nào với bất kỳ tập dữ liệu trước đó nào.

Như bạn có thể thấy, học máy liên quan khá nhiều đến cơ sở hạ tầng. Chương 19 thảo luận một số khía cạnh của điều này, nhưng đây là một chủ đề rất rộng được gọi là Hoạt động ML (MLOps), xứng đáng có một cuốn sách riêng. Vì vậy, đừng ngạc nhiên nếu dự án ML đầu tiên của bạn tốn nhiều công sức và thời gian để xây dựng và triển khai vào sản xuất. May mắn thay, một khi tất cả cơ sở hạ tầng đã được thiết lập, việc chuyển đổi từ ý tưởng sang sản xuất sẽ nhanh hơn nhiều.

Hãy thử ngay!

Hy vọng chương này đã cho bạn một cái nhìn tổng quan về một dự án học máy trông như thế nào, cũng như giới thiệu một số công cụ bạn có thể sử dụng để xây dựng một hệ thống tuyệt vời. Như bạn có thể thấy, phần lớn công việc nằm ở bước chuẩn bị dữ liệu: xây dựng công cụ giám sát, thiết lập các pipeline đánh giá thủ công và tự động hóa quá trình huấn luyện mô hình thường xuyên. Các thuật toán học máy tất nhiên là quan trọng, nhưng có lẽ tốt hơn hết là bạn nên nắm vững quy trình tổng thể và hiểu rõ ba hoặc bốn thuật toán thay vì dành tất cả thời gian của mình để khám phá các thuật toán nâng cao.

Vì vậy, nếu bạn chưa làm, bây giờ là thời điểm tốt để cầm máy tính xách tay lên, chọn một tập dữ liệu mà bạn quan tâm và cố gắng thực hiện toàn bộ quá trình từ A đến Z. Một nơi tốt để bắt đầu là trên một trang web thi đấu như Kaggle: bạn sẽ có một tập dữ liệu để thử nghiệm, một mục tiêu rõ ràng và những người để chia sẻ kinh nghiệm. Chúc bạn vui vẻ!

2.9 Bài tập

Các bài tập sau đây dựa trên tập dữ liệu nhà ở của chương này:

- Hãy thử bộ hồi quy máy vector hỗ trợ (`sklearn.svm.SVR`) với các siêu tham số khác nhau, chẳng hạn như `kernel="linear"` (với các giá trị khác nhau cho siêu tham số `C`) hoặc `kernel="rbf"` (với các giá trị khác nhau cho siêu tham số `C` và `gamma`). Lưu ý rằng máy vector hỗ trợ không mở rộng tốt với các tập dữ liệu lớn, vì vậy bạn có thể chỉ nên huấn luyện mô hình của mình trên 5.000 trường hợp đầu tiên của tập huấn luyện và chỉ sử dụng kiểm định chéo 3-fold, nếu không sẽ mất hàng giờ. Đừng lo lắng về ý nghĩa của các siêu tham số bây giờ; chúng ta sẽ thảo luận chúng trong Chương 5. Bộ dự đoán SVR tốt nhất hoạt động như thế nào?
- Hãy thử thay thế `GridSearchCV` bằng `RandomizedSearchCV`.
- Hãy thử thêm bộ biến đổi `SelectFromModel` vào pipeline chuẩn bị để chỉ chọn các thuộc tính quan trọng nhất.
- Hãy thử tạo một bộ biến đổi tùy chỉnh mà huấn luyện một bộ hồi quy k-nearest neighbors (`sklearn.neighbors.KNeighborsRegressor`) trong phương thức `fit()` của nó, và xuất ra các dự đoán của mô hình trong phương thức `transform()` của nó. Sau đó thêm đặc trưng này vào pipeline tiền xử lý, sử dụng vĩ độ và kinh độ làm đầu vào cho bộ biến đổi này. Điều này sẽ thêm một đặc trưng vào mô hình tương ứng với giá nhà trung bình của các quận gần nhất.

- Tự động khám phá một số tùy chọn chuẩn bị bằng cách sử dụng `GridSearchCV`.
- Hãy thử triển khai lại lớp `StandardScalerClone` từ đầu, sau đó thêm hỗ trợ cho phương thức `inverse_transform()`: việc thực thi `scaler.inverse_transform(scaler.fit_transform(X))` sẽ trả về một mảng rất gần với X. Sau đó thêm hỗ trợ cho tên đặc trưng: đặt `feature_names_in_` trong phương thức `fit()` nếu đầu vào là DataFrame. Thuộc tính này phải là một mảng NumPy các tên cột. Cuối cùng, triển khai phương thức `get_feature_names_out()`: nó phải có một đối số tùy chọn `input_features=None`. Nếu được truyền, phương thức phải kiểm tra xem độ dài của nó có khớp với `n_features_in_` không, và nó phải khớp với `feature_names_in_` nếu nó được định nghĩa; sau đó `input_features` sẽ được trả về. Nếu `input_features` là `None`, thì phương thức phải trả về `feature_names_in_` nếu nó được định nghĩa hoặc `np.array(["x0", "x1", ...])` với độ dài `n_features_in_` nếu không.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại https://homl.info/colab3 .

- Bộ dữ liệu gốc xuất hiện trong R. Kelley Pace và Ronald Barry, “Sparse Spatial Autoregressions”, Statistics & Probability Letters 33, số 3 (1997): 291–297.
- Một mảnh thông tin được đưa vào hệ thống học máy thường được gọi là tín hiệu, theo lý thuyết thông tin của Claude Shannon, mà ông đã phát triển tại Bell Labs để cải thiện viễn thông. Lý thuyết của ông: bạn muốn tỷ lệ tín hiệu trên nhiễu cao.
- Nhắc lại rằng toán tử chuyển vị lật một vector cột thành một vector hàng (và ngược lại).
- Bạn cũng có thể cần kiểm tra các ràng buộc pháp lý, chẳng hạn như các trường riêng tư không bao giờ được sao chép vào các kho dữ liệu không an toàn.
- Độ lệch chuẩn thường được ký hiệu là σ (chữ cái Hy Lạp sigma), và nó là căn bậc hai của phương sai, là trung bình của độ lệch bình phương so với giá trị trung bình. Khi một đặc trưng có phân phối chuẩn hình chuông (còn gọi là phân phối Gaussian), rất phổ biến, quy tắc “68-95-99.7” được áp dụng: khoảng 68% các giá trị nằm trong 1σ của giá trị trung bình, 95% trong 2σ, và 99.7% trong 3σ.
- Bạn sẽ thường thấy mọi người đặt hạt giống ngẫu nhiên thành 42. Con số này không có thuộc tính đặc biệt nào, ngoài việc là Câu trả lời cho Câu hỏi Cuối cùng về Sự sống, Vũ trụ và Mọi thứ.
- Thông tin vị trí thực tế khá thô, và kết quả là nhiều quận sẽ có cùng ID chính xác, vì vậy chúng sẽ nằm trong cùng một tập hợp (kiểm thử hoặc huấn luyện). Điều này gây ra một số sai lệch lấy mẫu không mong muốn.
- Nếu bạn đang đọc bằng thang độ xám, hãy lấy một cây bút đỏ và gạch lên phần lớn bờ biển từ Vùng Vịnh xuống San Diego (như bạn có thể mong đợi). Bạn cũng có thể thêm một mảng màu vàng quanh Sacramento.
- Để biết thêm chi tiết về các nguyên tắc thiết kế, xem Lars Buitinck et al., “API Design for Machine Learning Software: Experiences from the Scikit-Learn Project”, arXiv preprint arXiv:1309.0238 (2013).

- Một số bộ dự đoán cũng cung cấp các phương thức để đo độ tin cậy của dự đoán của chúng.
- Khi bạn đọc những dòng này, có thể tất cả các bộ biến đổi sẽ xuất ra Pandas DataFrames khi chúng nhận một DataFrame làm đầu vào: Pandas vào, Pandas ra. Có khả năng sẽ có một tùy chọn cấu hình chung cho việc này: `sklearn.set_config(pandas_in_out=True)`.
- Xem tài liệu của SciPy để biết thêm chi tiết.
- Tóm lại, API REST (hoặc RESTful) là một API dựa trên HTTP tuân theo một số quy ước, chẳng hạn như sử dụng các động từ HTTP chuẩn để đọc, cập nhật, tạo hoặc xóa tài nguyên (GET, POST, PUT và DELETE) và sử dụng JSON cho đầu vào và đầu ra.
- Captcha là một bài kiểm tra để đảm bảo người dùng không phải là robot. Các bài kiểm tra này thường được sử dụng như một cách rẻ tiền để gắn nhãn dữ liệu huấn luyện.

Bài tập

Trong chương này, chúng ta đã đề cập đến một số khái niệm quan trọng nhất trong học máy. Trong các chương tiếp theo, chúng ta sẽ đi sâu hơn và viết nhiều mã hơn, nhưng trước khi làm điều đó, hãy đảm bảo bạn có thể trả lời các câu hỏi sau:

- Bạn sẽ định nghĩa học máy như thế nào?
- Bạn có thể kể tên bốn loại ứng dụng mà nó tỏa sáng không?
- Tập huấn luyện có nhãn là gì?
- Hai tác vụ có giám sát phổ biến nhất là gì?
- Bạn có thể kể tên bốn tác vụ không giám sát phổ biến không?
- Bạn sẽ sử dụng loại thuật toán nào để cho phép một robot đi trên nhiều địa hình không xác định khác nhau?
- Bạn sẽ sử dụng loại thuật toán nào để phân khúc khách hàng của mình thành nhiều nhóm?
- Bạn sẽ coi vấn đề phát hiện thư rác là một vấn đề học có giám sát hay một vấn đề học không giám sát?
- Hệ thống học trực tuyến là gì?
- Học ngoài lõi (out-of-core learning) là gì?
- Loại thuật toán nào dựa vào một thước đo tương tự để đưa ra dự đoán?
- Sự khác biệt giữa tham số mô hình và siêu tham số mô hình là gì?
- Các thuật toán dựa trên mô hình tìm kiếm điều gì? Chiến lược phổ biến nhất mà chúng sử dụng để thành công là gì? Chúng đưa ra dự đoán như thế nào?
- Bạn có thể kể tên bốn trong số những thách thức chính trong học máy không?
- Nếu mô hình của bạn hoạt động tốt trên dữ liệu huấn luyện nhưng tổng quát hóa kém cho các trường hợp mới, điều gì đang xảy ra? Bạn có thể kể tên ba giải pháp khả thi không?

- Tập kiểm thử là gì, và tại sao bạn muốn sử dụng nó?
- Mục đích của tập xác thực là gì?
- Tập train-dev là gì, khi nào bạn cần nó, và bạn sử dụng nó như thế nào?
- Điều gì có thể sai nếu bạn điều chỉnh siêu tham số bằng cách sử dụng tập kiểm thử?

Lời giải cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại https://homl.info/colab3 .

1 Thực tế thú vị: tên nghe có vẻ kỳ lạ này là một thuật ngữ thống kê được Francis Galton giới thiệu khi ông nghiên cứu thực tế là con cái của những người cao có xu hướng thấp hơn cha mẹ chúng. Vì con cái thấp hơn, ông gọi đây là hồi quy về giá trị trung bình. Tên này sau đó được áp dụng cho các phương pháp ông sử dụng để phân tích mối tương quan giữa các biến. 2 Lưu ý cách động vật được tách biệt khá tốt với xe cộ và ngựa gần với hươu nhưng xa chim. Hình ảnh được tái bản với sự cho phép của Richard Socher et al., “Zero-Shot Learning Through Cross-Modal Transfer”, Proceedings of the 26th International Conference on Neural Information Processing Systems 1 (2013): 935–943. 3 Đó là khi hệ thống hoạt động hoàn hảo. Trong thực tế, nó thường tạo ra một vài cụm cho mỗi người, và đôi khi trộn lẫn hai người trông giống nhau, vì vậy bạn có thể cần cung cấp một vài nhãn cho mỗi người và làm sạch thủ công một số cụm. 4 Theo quy ước, chữ cái Hy Lạp θ (theta) thường được sử dụng để biểu thị các tham số mô hình. 5 Không sao nếu bạn chưa hiểu tất cả mã; tôi sẽ trình bày Scikit-Learn trong các chương tiếp theo. 6 Ví dụ, biết nên viết “to”, “two”, hay “too”, tùy thuộc vào ngữ cảnh. 7 Peter Norvig et al., “The Unreasonable Effectiveness of Data”, IEEE Intelligent Systems 24, no. 2 (2009): 8–12. 8 Hình ảnh được tái bản với sự cho phép của Michele Banko và Eric Brill, “Scaling to Very Very Large Corpora for Natural Language Disambiguation”, Proceedings of the 39th Annual Meeting of the Association for Computational Linguistics (2001): 26–33. 9 David Wolpert, “The Lack of A Priori Distinctions Between Learning Algorithms”, Neural Computation 8, no. 7 (1996): 1341–1390.

CHƯƠNG 3. PHÂN LOẠI

Trong Chương 1, tôi đã đề cập rằng các tác vụ học có giám sát phổ biến nhất là hồi quy (dự đoán giá trị) và phân loại (dự đoán lớp). Trong Chương 2, chúng ta đã khám phá một tác vụ hồi quy, dự đoán giá trị nhà ở, sử dụng các thuật toán khác nhau như hồi quy tuyến tính, cây quyết định và rừng ngẫu nhiên (sẽ được giải thích chi tiết hơn trong các chương sau). Bây giờ chúng ta sẽ chuyển sự chú ý sang các hệ thống phân loại.

3.1 MNIST

Trong chương này, chúng ta sẽ sử dụng bộ dữ liệu MNIST , đây là một tập hợp 70.000 hình ảnh nhỏ của các chữ số viết tay bởi học sinh trung học và nhân viên Cục Thống kê Hoa Kỳ. Mỗi hình ảnh được gắn nhãn với chữ số mà nó đại diện. Tập hợp này đã được nghiên cứu nhiều đến nỗi nó thường được gọi là "hello world" của học máy : bất cứ khi nào mọi người đưa ra một thuật toán phân loại mới, họ đều tò mò muốn xem nó sẽ hoạt động như thế nào trên MNIST, và bất cứ ai học học máy đều xử lý bộ dữ liệu này sớm hay muộn.

Scikit-Learn cung cấp nhiều hàm trợ giúp để tải xuống các bộ dữ liệu phổ biến. MNIST là một trong số đó. Đoạn mã sau lấy bộ dữ liệu MNIST từ OpenML.org:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

Gói `sklearn.datasets` chủ yếu chứa ba loại hàm: các hàm `fetch_*` như `fetch_openml()` để tải xuống các bộ dữ liệu thực tế, các hàm `load_*` để tải các bộ dữ liệu đồ chơi nhỏ được đóng gói cùng với Scikit-Learn (vì vậy chúng không cần được tải xuống qua internet), và các hàm `make_*` để tạo các bộ dữ liệu giả, hữu ích cho các kiểm thử. Các bộ dữ liệu được tạo thường được trả về dưới dạng một bộ (X, y) chứa dữ liệu đầu vào và các mục tiêu, cả hai đều dưới dạng mảng NumPy. Các bộ dữ liệu khác được trả về dưới dạng đối tượng `sklearn.utils.Bunch`, đây là các từ điển mà các mục nhập của chúng cũng có thể được truy cập dưới dạng thuộc tính. Chúng thường chứa các mục nhập sau:

- "DESCR": Mô tả của bộ dữ liệu
- "data": Dữ liệu đầu vào, thường là mảng NumPy 2D
- "target": Các nhãn, thường là mảng NumPy 1D

Hàm `fetch_openml()` hơi bất thường vì theo mặc định, nó trả về đầu vào dưới dạng Pandas DataFrame và nhãn dưới dạng Pandas Series (trừ khi bộ dữ liệu thưa thớt). Nhưng bộ dữ liệu MNIST chứa hình ảnh, và DataFrame không lý tưởng cho điều đó, vì vậy tốt hơn là đặt `as_frame=False` để lấy dữ liệu dưới dạng mảng NumPy. Hãy xem các mảng này:

```
>>> X, y = mnist.data, mnist.target
>>> X
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
```

```
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
>>> X.shape
(70000, 784)
>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
>>> y.shape
(70000,)
```

Có 70.000 hình ảnh, và mỗi hình ảnh có 784 đặc trưng. Điều này là do mỗi hình ảnh là 28 × 28 pixel, và mỗi đặc trưng chỉ đơn giản đại diện cho cường độ của một pixel, từ 0 (trắng) đến 255 (đen). Hãy xem một chữ số từ bộ dữ liệu (Hình 3-1). Tất cả những gì chúng ta cần làm là lấy vector đặc trưng của một trường hợp, định hình lại nó thành một mảng 28 × 28 và hiển thị nó bằng hàm `imshow()` của Matplotlib. Chúng ta sử dụng `cmap="binary"` để có bản đồ màu xám trong đó 0 là trắng và 255 là đen:

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()
```

Hình 3-1. Ví dụ về hình ảnh MNIST

Điều này trông giống số 5, và đúng là nhãn cũng cho chúng ta biết điều đó:

```
>>> y[0]
'5'
```

Để bạn cảm nhận được sự phức tạp của tác vụ phân loại, Hình 3-2 cho thấy một vài hình ảnh khác từ bộ dữ liệu MNIST.

Nhưng chờ đã! Bạn nên luôn tạo một tập kiểm thử và đặt nó sang một bên trước khi kiểm tra dữ liệu kỹ lưỡng. Bộ dữ liệu MNIST được trả về bởi `fetch_openml()` thực tế đã được chia thành một tập huấn luyện (60.000 hình ảnh đầu tiên) và một tập kiểm thử (10.000 hình ảnh cuối cùng):

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

Tập huấn luyện đã được xáo trộn sẵn cho chúng ta, điều này tốt vì nó đảm bảo rằng tất cả các fold kiểm định chéo sẽ tương tự nhau (chúng ta không muốn một fold bị thiếu một số chữ số). Hơn nữa, một số thuật toán học nhạy cảm với thứ tự của các trường hợp huấn luyện, và chúng hoạt động kém nếu chúng nhận được nhiều trường hợp tương tự liên tiếp. Việc xáo trộn bộ dữ liệu đảm bảo rằng điều này sẽ không xảy ra.

Hình 3-2. Các chữ số từ bộ dữ liệu MNIST

3.2 Huấn luyện bộ phân loại nhị phân

Bây giờ, hãy đơn giản hóa vấn đề và chỉ cố gắng xác định một chữ số — ví dụ, số 5. "Bộ phát hiện 5" này sẽ là một ví dụ về bộ phân loại nhị phân, có khả năng phân biệt giữa chỉ hai lớp, 5 và không-5. Đầu tiên, chúng ta sẽ tạo các vector mục tiêu cho tác vụ phân loại này:

```
y_train_5 = (y_train == '5') # True for all 5s, False for all other digits
y_test_5 = (y_test == '5')
```

Bây giờ hãy chọn một bộ phân loại và huấn luyện nó. Một nơi tốt để bắt đầu là với bộ phân loại xuống dốc ngẫu nhiên (SGD, hoặc SGD ngẫu nhiên), sử dụng lớp

`SGDClassifier` của Scikit-Learn.

Bộ phân loại này có khả năng xử lý các tập dữ liệu rất lớn một cách hiệu quả. Một phần là do SGD xử lý các trường hợp huấn luyện độc lập, từng trường hợp một, điều này cũng làm cho SGD rất phù hợp cho học trực tuyến, như bạn sẽ thấy sau này. Hãy tạo một `SGDClassifier` và huấn luyện nó trên toàn bộ tập huấn luyện:

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

Bây giờ chúng ta có thể sử dụng nó để phát hiện hình ảnh của số 5:

```
>>> sgd_clf.predict([some_digit])
array([ True])
```

Bộ phân loại đoán rằng hình ảnh này đại diện cho một số 5 (True). Có vẻ như nó đã đoán đúng trong trường hợp cụ thể này! Bây giờ, hãy đánh giá hiệu suất của mô hình này.

3.3 Các thước đo hiệu suất

Đánh giá một bộ phân loại thường phức tạp hơn đáng kể so với đánh giá một bộ hồi quy, vì vậy chúng ta sẽ dành một phần lớn chương này cho chủ đề này. Có nhiều thước đo hiệu suất có sẵn, vì vậy hãy pha một ly cà phê nữa và sẵn sàng tìm hiểu một loạt các khái niệm và từ viết tắt mới!

3.3.1 Đo độ chính xác bằng kiểm định chéo

Một cách hay để đánh giá một mô hình là sử dụng kiểm định chéo, giống như bạn đã làm trong Chương 2. Hãy sử dụng hàm `cross_val_score()` để đánh giá mô hình `SGDClassifier` của chúng ta, sử dụng kiểm định chéo k-fold với ba fold. Nhớ rằng kiểm định chéo k-fold có nghĩa là chia tập huấn luyện thành k fold (trong trường hợp này là ba), sau đó huấn luyện mô hình k lần, mỗi lần giữ lại một fold khác để đánh giá (xem Chương 2):

```
>>> from sklearn.model_selection import cross_val_score
```

```
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.95035, 0.96035, 0.9604 ])
```

Wow! Độ chính xác (tỷ lệ dự đoán đúng) trên 95% trên tất cả các fold kiểm định chéo? Điều này có vẻ tuyệt vời, phải không? Chà, trước khi bạn quá phấn khích, hãy xem một bộ phân loại giả (dummy classifier) mà chỉ phân loại mọi hình ảnh duy nhất vào lớp thường xuyên nhất, trong trường hợp này là lớp phủ định (tức là không phải 5):

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train))) # prints False: no 5s detected
```

Bạn có thể đoán độ chính xác của mô hình này không? Hãy cùng tìm hiểu:

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```

Đúng vậy, nó có độ chính xác trên 90%! Điều này đơn giản là vì chỉ khoảng 10% hình ảnh là số 5, vì vậy nếu bạn luôn đoán rằng một hình ảnh không phải là số 5, bạn sẽ đúng khoảng 90% thời gian. Điều này vượt qua Nostradamus.

Điều này chứng tỏ tại sao độ chính xác nói chung không phải là thước đo hiệu suất được ưu tiên cho các bộ phân loại, đặc biệt khi bạn đang xử lý các tập dữ liệu bị lệch (tức là khi một số lớp thường xuyên hơn nhiều so với các lớp khác). Một cách tốt hơn nhiều để đánh giá hiệu suất của bộ phân loại là xem xét ma trận nhầm lẫn (CM).

THỰC HIỆN KIỂM ĐỊNH CHÉO Đôi khi bạn sẽ cần kiểm soát nhiều hơn đối với quy trình kiểm định chéo so với những gì Scikit-Learn cung cấp sẵn. Trong những trường hợp này, bạn có thể tự mình thực hiện kiểm định chéo. Đoạn mã sau đây thực hiện gần giống với hàm `cross_val_score()` của Scikit-Learn, và nó in ra cùng kết quả:

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3) # add shuffle=True if the dataset is
                                      # not already shuffled
for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred)) # prints 0.95035, 0.96035, and 0.9604
```

Lớp `StratifiedKFold` thực hiện lấy mẫu phân tầng (như đã giải thích trong Chương 2) để tạo ra các fold chứa tỷ lệ đại diện của mỗi lớp. Ở mỗi lần lặp, mã tạo ra một bản sao của bộ phân loại, huấn luyện bản sao đó trên các fold huấn luyện và đưa ra dự đoán trên fold kiểm thử. Sau đó, nó đếm số lượng dự đoán đúng và xuất ra tỷ lệ dự đoán đúng.

3.3.2 Ma trận nhầm lẫn

Ý tưởng chung của ma trận nhầm lẫn là đếm số lần các trường hợp của lớp A được phân loại là lớp B, cho tất cả các cặp A/B. Ví dụ, để biết số lần bộ phân loại nhầm lẫn hình ảnh số 8 với số 0, bạn sẽ xem hàng #8, cột #0 của ma trận nhầm lẫn.

Để tính toán ma trận nhầm lẫn, trước tiên bạn cần có một tập hợp các dự đoán để có thể so sánh chúng với các mục tiêu thực tế. Bạn có thể đưa ra dự đoán trên tập kiểm thử, nhưng tốt nhất là giữ nguyên tập đó tạm thời (hãy nhớ rằng bạn chỉ muốn sử dụng tập kiểm thử vào cuối dự án của mình, một khi bạn đã có một bộ phân loại mà bạn sẵn sàng triển khai). Thay vào đó, bạn có thể sử dụng hàm `cross_val_predict()`:

```
from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

Giống như hàm `cross_val_score()`, `cross_val_predict()` thực hiện kiểm định chéo k-fold, nhưng thay vì trả về các điểm đánh giá, nó trả về các dự đoán được đưa ra trên mỗi fold kiểm thử. Điều này có nghĩa là bạn nhận được một dự đoán sạch cho mỗi trường hợp trong tập huấn luyện (bởi “sạch” tôi muốn nói “ngoài mẫu”: mô hình đưa ra dự đoán trên dữ liệu mà nó chưa từng thấy trong quá trình huấn luyện).

Bây giờ bạn đã sẵn sàng lấy ma trận nhầm lẫn bằng cách sử dụng hàm `confusion_matrix()`. Chỉ cần chuyển cho nó các lớp mục tiêu (`y_train_5`) và các lớp được dự đoán (`y_train_pred`):

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,   687],
       [ 1891,  3530]])
```

Mỗi hàng trong ma trận nhầm lẫn biểu thị một lớp thực tế, trong khi mỗi cột biểu thị một lớp được dự đoán. Hàng đầu tiên của ma trận này xem xét các hình ảnh không phải là 5 (lớp phủ định): 53.892 trong số đó được phân loại đúng là không phải 5 (chúng được gọi là true negatives), trong khi 687 còn lại bị phân loại sai là 5 (false positives, còn gọi là lỗi loại I). Hàng thứ hai xem xét các hình ảnh số 5 (lớp tích cực): 1.891 bị phân loại sai là không phải 5 (false negatives, còn gọi là lỗi loại II), trong khi 3.530 còn lại được phân loại đúng là số 5 (true positives). Một bộ phân loại hoàn hảo sẽ chỉ có true positives và true negatives, vì vậy ma trận nhầm lẫn của nó sẽ chỉ có các giá trị khác 0 trên đường chéo chính của nó (từ trên cùng bên trái xuống dưới cùng bên phải):

```
>>> y_train_perfect_predictions = y_train_5 # pretend we reached perfection
>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
```

```
array([[54579,     0],
       [    0,  5421]])
```

Ma trận nhầm lẫn cung cấp cho bạn rất nhiều thông tin, nhưng đôi khi bạn có thể muốn một chỉ số ngắn gọn hơn. Một chỉ số thú vị đáng xem xét là độ chính xác của các dự đoán tích cực; đây được gọi là độ chính xác (precision) của bộ phân loại (Phương trình 3-1).

Phương trình 3-1. Độ chính xác

$$precision = \frac{TP}{TP + FP}$$

TP là số lượng true positives, và FP là số lượng false positives.

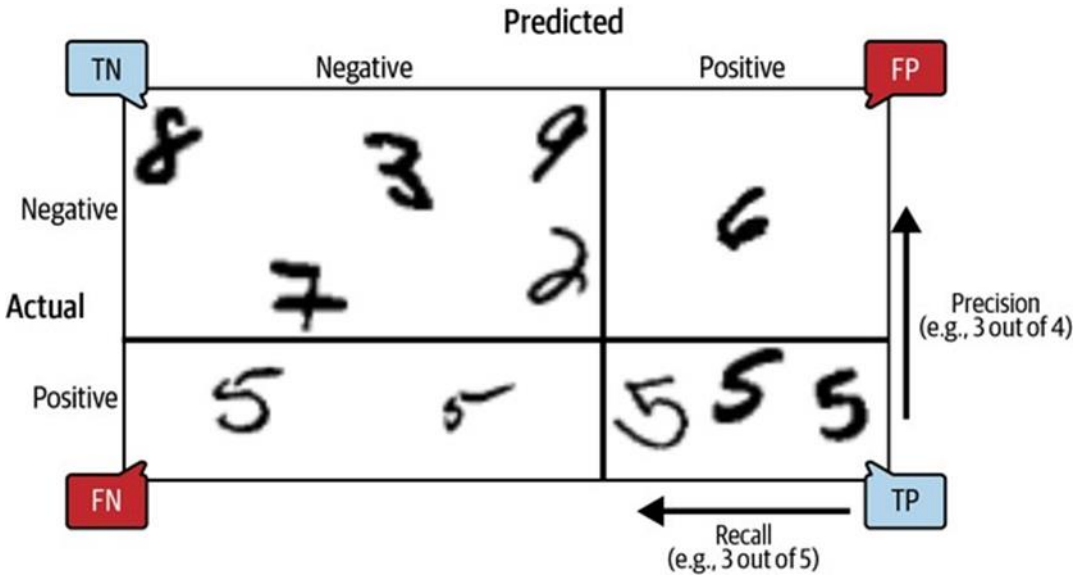
Một cách đơn giản để có độ chính xác hoàn hảo là tạo một bộ phân loại luôn đưa ra dự đoán tiêu cực, ngoại trừ một dự đoán tích cực duy nhất trên trường hợp mà nó tự tin nhất. Nếu dự đoán này là đúng, thì bộ phân loại có độ chính xác 100% (precision = 1/1 = 100%). Rõ ràng, một bộ phân loại như vậy sẽ không hữu ích lắm, vì nó sẽ bỏ qua tất cả ngoại trừ một trường hợp tích cực. Vì vậy, độ chính xác thường được sử dụng cùng với một chỉ số khác có tên là độ nhạy (recall), còn được gọi là độ nhạy (sensitivity) hoặc tỷ lệ dương tính đúng (TPR): đây là tỷ lệ các trường hợp tích cực được bộ phân loại phát hiện đúng (Phương trình 3-2).

Phương trình 3-2. Độ nhạy

$$recall = \frac{TP}{TP + FN}$$

FN tất nhiên là số lượng false negatives.

Nếu bạn bối rối về ma trận nhầm lẫn, Hình 3-3 có thể giúp ích.



Hình 3-3. Ma trận nhầm lẫn minh họa hiển thị các ví dụ về true negatives (trên cùng bên trái), false positives (trên cùng bên phải), false negatives (dưới cùng bên trái) và true positives (dưới cùng bên phải).

3.3.3 Độ chính xác và độ nhạy

Scikit-Learn cung cấp một số hàm để tính toán các chỉ số phân loại, bao gồm độ chính xác và độ nhạy:

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
0.8370879772350012
>>> recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
0.6511713705958311
```

Bây giờ bộ phát hiện số 5 của chúng ta không còn sáng bóng như khi chúng ta nhìn vào độ chính xác của nó nữa. Khi nó cho rằng một hình ảnh đại diện cho số 5, nó chỉ đúng 83.7% thời gian. Hơn nữa, nó chỉ phát hiện được 65.1% số 5.

Thường thì thuận tiện để kết hợp độ chính xác và độ nhạy vào một chỉ số duy nhất được gọi là điểm F1, đặc biệt khi bạn cần một chỉ số duy nhất để so sánh hai bộ phân loại. Điểm F1 là trung bình điều hòa của độ chính xác và độ nhạy (Phương trình 3-3). Trong khi trung bình thông thường coi tất cả các giá trị như nhau, trung bình điều hòa lại ưu tiên các giá trị thấp hơn nhiều. Kết quả là, bộ phân loại sẽ chỉ đạt được điểm F1 cao nếu cả độ nhạy và độ chính xác đều cao.

Phương trình 3-3. Điểm F1

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN+FP}{2}}$$

Để tính điểm F1, chỉ cần gọi hàm `f1_score()`:

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

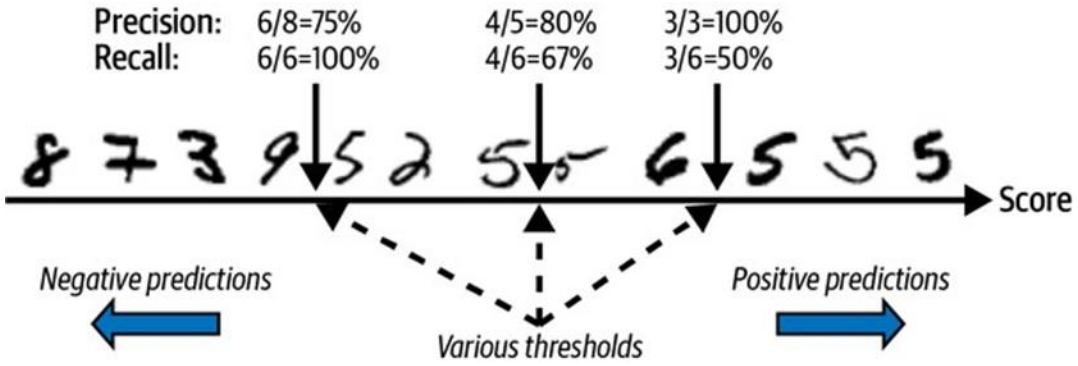
Điểm F1 ưu tiên các bộ phân loại có độ chính xác và độ nhạy tương tự nhau. Điều này không phải lúc nào cũng là điều bạn muốn: trong một số ngữ cảnh, bạn chủ yếu quan tâm đến độ chính xác, và trong các ngữ cảnh khác, bạn thực sự quan tâm đến độ nhạy. Ví dụ, nếu bạn huấn luyện một bộ phân loại để phát hiện các video an toàn cho trẻ em, bạn có thể sẽ thích một bộ phân loại từ chối nhiều video tốt (độ nhạy thấp) nhưng chỉ giữ lại những video an toàn (độ chính xác cao), hơn là một bộ phân loại có độ nhạy cao hơn nhiều nhưng lại để một vài video thực sự tệ xuất hiện trong sản phẩm của bạn (trong những trường hợp như vậy, bạn thậm chí có thể muốn thêm một pipeline thủ công để kiểm tra việc lựa chọn video của bộ phân loại). Mặt khác, giả sử bạn huấn luyện một bộ phân loại để phát hiện những kẻ trộm trong hình ảnh giám sát: có lẽ ổn nếu bộ phân loại của bạn chỉ có độ chính

xác 30% miễn là nó có độ nhạy 99% (chắc chắn, nhân viên bảo vệ sẽ nhận được một vài cảnh báo sai, nhưng gần như tất cả những kẻ trộm sẽ bị bắt).

Thật không may, bạn không thể có cả hai: tăng độ chính xác sẽ làm giảm độ nhạy, và ngược lại. Điều này được gọi là sự đánh đổi độ chính xác/độ nhạy (precision/recall trade-off).

3.3.4 Sự đánh đổi Độ chính xác/Độ nhạy

Để hiểu sự đánh đổi này, hãy xem cách `SGDClassifier` đưa ra các quyết định phân loại của nó. Đối với mỗi trường hợp, nó tính toán một điểm số dựa trên một hàm quyết định. Nếu điểm số đó lớn hơn một ngưỡng, nó gán trường hợp đó vào lớp dương; nếu không, nó gán vào lớp âm. Hình 3-4 cho thấy một vài chữ số được đặt từ điểm thấp nhất bên trái đến điểm cao nhất bên phải. Giả sử ngưỡng quyết định được đặt ở mũi tên trung tâm (giữa hai số 5): bạn sẽ tìm thấy 4 dự đoán đúng (số 5 thực tế) ở bên phải ngưỡng đó, và 1 dự đoán sai (thực tế là số 6). Do đó, với ngưỡng đó, độ chính xác là 80% (4 trên 5). Nhưng trong số 6 số 5 thực tế, bộ phân loại chỉ phát hiện 4, vì vậy độ nhạy là 67% (4 trên 6). Nếu bạn tăng ngưỡng (di chuyển nó đến mũi tên bên phải), dự đoán sai (số 6) trở thành một dự đoán đúng tiêu cực, do đó tăng độ chính xác (lên đến 100% trong trường hợp này), nhưng một dự đoán đúng tích cực trở thành một dự đoán sai tiêu cực, làm giảm độ nhạy xuống 50%. Ngược lại, việc giảm ngưỡng sẽ làm tăng độ nhạy và giảm độ chính xác.



Hình 3-4. Sự đánh đổi độ chính xác/độ nhạy: hình ảnh được xếp hạng theo điểm số bộ phân loại của chúng, và những hình ảnh trên ngưỡng quyết định được chọn được coi là tích cực; ngưỡng càng cao thì độ nhạy càng thấp, nhưng (nói chung) độ chính xác càng cao.

Scikit-Learn không cho phép bạn đặt trực tiếp ngưỡng, nhưng nó cung cấp cho bạn quyền truy cập vào các điểm quyết định mà nó sử dụng để đưa ra dự đoán. Thay vì gọi phương thức `predict()` của bộ phân loại, bạn có thể gọi phương thức `decision_function()` của nó, phương thức này trả về một điểm số cho mỗi trường hợp, và sau đó sử dụng bất kỳ ngưỡng nào bạn muốn để đưa ra dự đoán dựa trên các điểm số đó.

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2164.22030239])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
array([ True])
```

`SGDClassifier` sử dụng ngưỡng bằng 0, vì vậy đoạn mã trên trả về kết quả tương tự như phương thức `predict()` (tức là `True`). Hãy tăng ngưỡng:

```
>>> threshold = 3000
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([False])
```

Điều này xác nhận rằng việc tăng ngưỡng làm giảm độ nhạy. Hình ảnh thực tế đại diện cho số 5, và bộ phân loại phát hiện nó khi ngưỡng là 0, nhưng nó bỏ sót khi ngưỡng tăng lên 3.000.

Làm thế nào để bạn quyết định sử dụng ngưỡng nào? Đầu tiên, sử dụng hàm `cross_val_predict()` để lấy điểm của tất cả các trường hợp trong tập huấn luyện, nhưng lần này chỉ định rằng bạn muốn trả về điểm quyết định thay vì dự đoán:

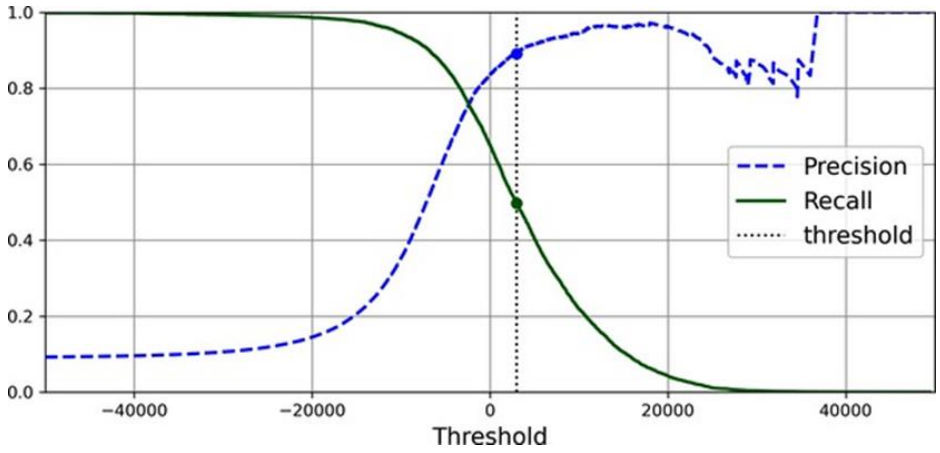
```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

Với các điểm số này, sử dụng hàm `precision_recall_curve()` để tính toán độ chính xác và độ nhạy cho tất cả các ngưỡng có thể (hàm này thêm một độ chính xác cuối cùng là 0 và một độ nhạy cuối cùng là 1, tương ứng với một ngưỡng vô hạn):

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

Cuối cùng, sử dụng Matplotlib để vẽ biểu đồ độ chính xác và độ nhạy dưới dạng hàm của giá trị ngưỡng (Hình 3-5). Hãy hiển thị ngưỡng 3.000 mà chúng ta đã chọn:

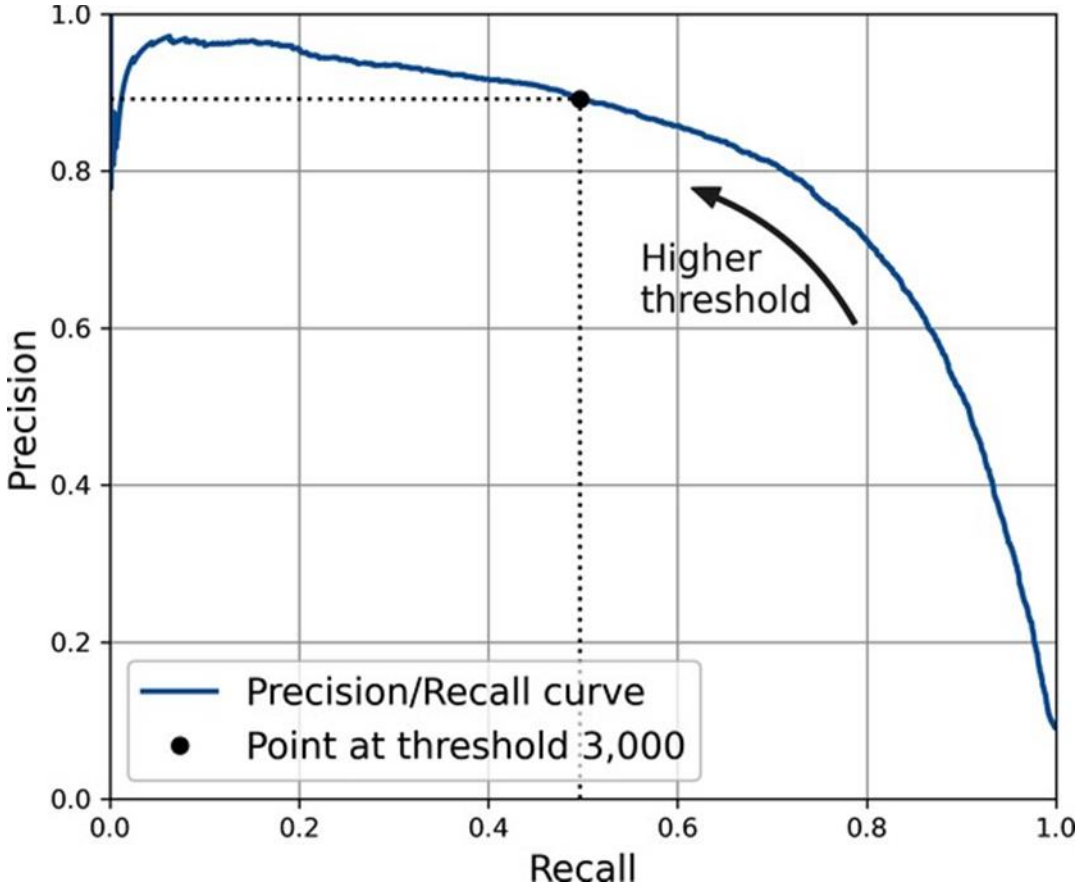
```
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")
[...] # beautify the figure: add grid, legend, axis, labels, and circles
plt.show()
```



Hình 3-5. Độ chính xác và độ nhạy so với ngưỡng quyết định.

Tại giá trị ngưỡng này, độ chính xác gần 90% và độ nhạy khoảng 50%. Một cách khác để chọn một sự đánh đổi độ chính xác/độ nhạy tốt là vẽ trực tiếp độ chính xác so với độ nhạy, như được hiển thị trong Hình 3-6 (cùng ngưỡng được hiển thị).

```
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")
[...] # beautify the figure: add labels, grid, legend, arrow, and text
plt.show()
```



Hình 3-6. Độ chính xác so với độ nhạy.

Bạn có thể thấy rằng độ chính xác thực sự bắt đầu giảm mạnh ở khoảng 80% độ nhạy. Bạn có lẽ sẽ muốn chọn một sự đánh đổi độ chính xác/độ nhạy ngay trước khi xảy ra sự giảm đó — ví dụ, ở khoảng 60% độ nhạy. Nhưng tất nhiên, lựa chọn phụ thuộc vào dự án của bạn.

Giả sử bạn quyết định đạt được độ chính xác 90%. Bạn có thể sử dụng biểu đồ đầu tiên để tìm ngưỡng bạn cần sử dụng, nhưng điều đó không chính xác lắm. Thay vào đó, bạn có thể tìm kiếm ngưỡng thấp nhất mang lại cho bạn ít nhất 90% độ chính xác. Đối với điều này, bạn có thể sử dụng phương thức `argmax()` của mảng NumPy. Phương thức này trả về chỉ mục đầu tiên của giá trị lớn nhất, trong trường hợp này có nghĩa là giá trị `True` đầu tiên.

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax()
>>> threshold_for_90_precision = thresholds[idx_for_90_precision]
```

```
>>> threshold_for_90_precision
3370.0194991439557
```

Để đưa ra dự đoán (trên tập huấn luyện hiện tại), thay vì gọi phương thức `predict()` của bộ phân loại, bạn có thể chạy đoạn mã này:

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

Hãy kiểm tra độ chính xác và độ nhạy của các dự đoán này:

```
>>> precision_score(y_train_5, y_train_pred_90)
0.9000345901072293
>>> recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)
>>> recall_at_90_precision
0.4799852425751706
```

Tuyệt vời, bạn có một bộ phân loại với độ chính xác 90%! Như bạn có thể thấy, việc tạo ra một bộ phân loại với độ chính xác hầu như bất kỳ nào bạn muốn là khá dễ dàng: chỉ cần đặt ngưỡng đủ cao, và bạn đã hoàn thành. Nhưng chờ đã, đừng vội vàng – một bộ phân loại có độ chính xác cao sẽ không hữu ích lắm nếu độ nhạy của nó quá thấp! Đối với nhiều ứng dụng, độ nhạy 48% sẽ không tốt chút nào.

3.3.5 Đường cong ROC

Đường cong đặc trưng hoạt động của bộ thu (ROC) là một công cụ phổ biến khác được sử dụng với các bộ phân loại nhị phân. Nó rất giống với đường cong độ chính xác/độ nhạy, nhưng thay vì vẽ biểu đồ độ chính xác so với độ nhạy, đường cong ROC vẽ biểu đồ tỷ lệ dương tính đúng (một tên khác của độ nhạy) so với tỷ lệ dương tính giả (FPR).

FPR (còn được gọi là fall-out) là tỷ lệ các trường hợp âm tính bị phân loại sai là dương tính. Nó bằng 1 trừ đi tỷ lệ âm tính đúng (TNR), là tỷ lệ các trường hợp âm tính được phân loại đúng là âm tính. TNR cũng được gọi là độ đặc hiệu (specificity). Do đó, đường cong ROC vẽ biểu đồ độ nhạy (recall) so với 1 – độ đặc hiệu.

Để vẽ đường cong ROC, trước tiên bạn sử dụng hàm `roc_curve()` để tính toán TPR và FPR cho các giá trị ngưỡng khác nhau:

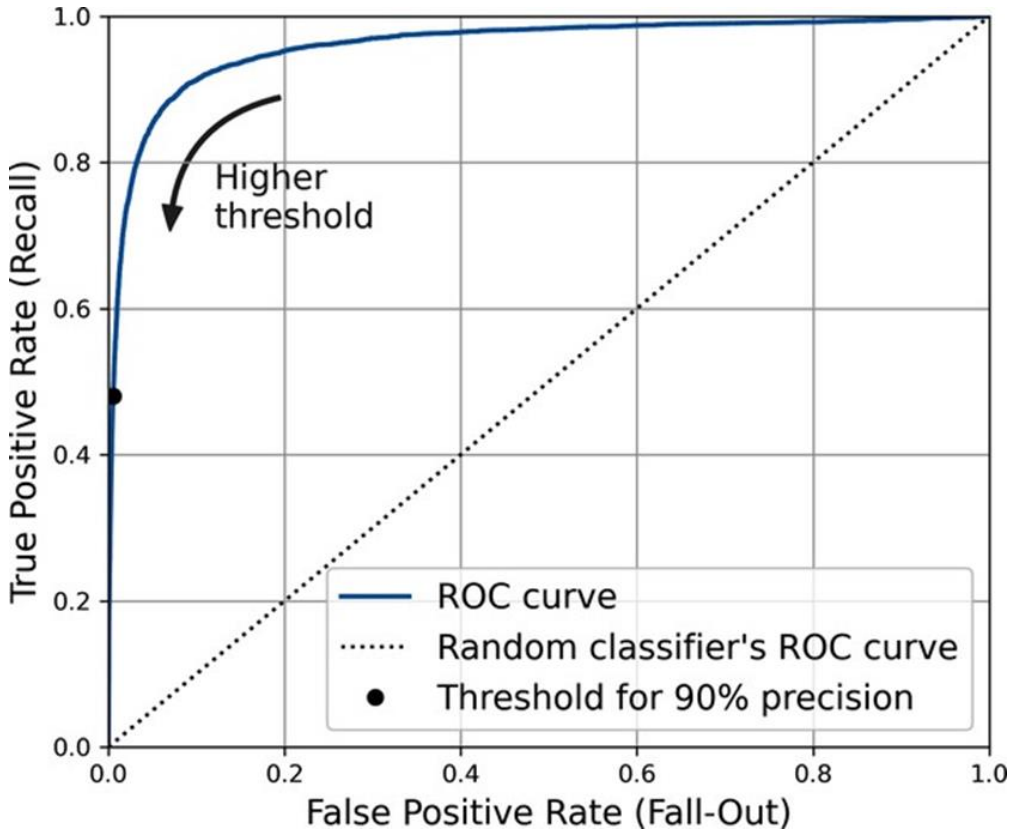
```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

Sau đó, bạn có thể vẽ FPR so với TPR bằng Matplotlib. Đoạn mã sau tạo ra biểu đồ trong Hình 3-7.

Để tìm điểm tương ứng với độ chính xác 90%, chúng ta cần tìm chỉ mục của ngưỡng mong muốn. Vì các ngưỡng được liệt kê theo thứ tự giảm dần trong trường hợp này, chúng ta sử dụng <= thay vì >= trên dòng đầu tiên:

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]
```

```
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")
[...] # beautify the figure: add labels, grid, legend, arrow, and text
plt.show()
```



Hình 3-7. Đường cong ROC vẽ tỷ lệ dương tính giả so với tỷ lệ dương tính đúng cho tất cả các ngưỡng có thể; vòng tròn đen làm nổi bật tỷ lệ đã chọn (ở độ chính xác 90% và độ nhạy 48%).

Một lần nữa lại có một sự đánh đổi: độ nhạy (TPR) càng cao, bộ phân loại tạo ra càng nhiều dương tính giả (FPR). Đường chấm chấm đại diện cho đường cong ROC của một bộ phân loại hoàn toàn ngẫu nhiên; một bộ phân loại tốt nằm càng xa đường đó càng tốt (hướng về góc trên bên trái).

Một cách để so sánh các bộ phân loại là đo diện tích dưới đường cong (AUC). Một bộ phân loại hoàn hảo sẽ có ROC AUC bằng 1, trong khi một bộ phân loại hoàn toàn ngẫu nhiên sẽ có ROC AUC bằng 0.5. Scikit-Learn cung cấp một hàm để ước tính ROC AUC:

```
>>> from sklearn.metrics import roc_auc_score

>>> roc_auc_score(y_train_5, y_scores)
0.9604938554008616
```

Bây giờ hãy tạo một RandomForestClassifier, mà chúng ta có thể so sánh đường cong PR và điểm F1 của nó với SGDClassifier.

```
from sklearn.ensemble import RandomForestClassifier

forest_clf = RandomForestClassifier(random_state=42)
```

Hàm precision_recall_curve() mong đợi nhãn và điểm cho mỗi trường hợp, vì vậy chúng ta cần huấn luyện bộ phân loại rừng ngẫu nhiên và yêu cầu nó gán điểm cho mỗi trường hợp. Nhưng lớp RandomForestClassifier không có phương thức decision_function(), do cách nó hoạt động. May mắn thay, nó có phương thức predict_proba() trả về xác suất lớp cho mỗi trường hợp, và chúng ta có thể chỉ cần sử dụng xác suất của lớp dương tính làm điểm, vì vậy nó sẽ hoạt động tốt. Chúng ta có thể gọi hàm cross_val_predict() để huấn luyện RandomForestClassifier bằng cách sử dụng kiểm định chéo và yêu cầu nó dự đoán xác suất lớp cho mọi hình ảnh như sau:

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

Hãy xem xác suất lớp cho hai hình ảnh đầu tiên trong tập huấn luyện:

```
>>> y_probas_forest[:2]
array([[0.11, 0.89],
       [0.99, 0.01]])
```

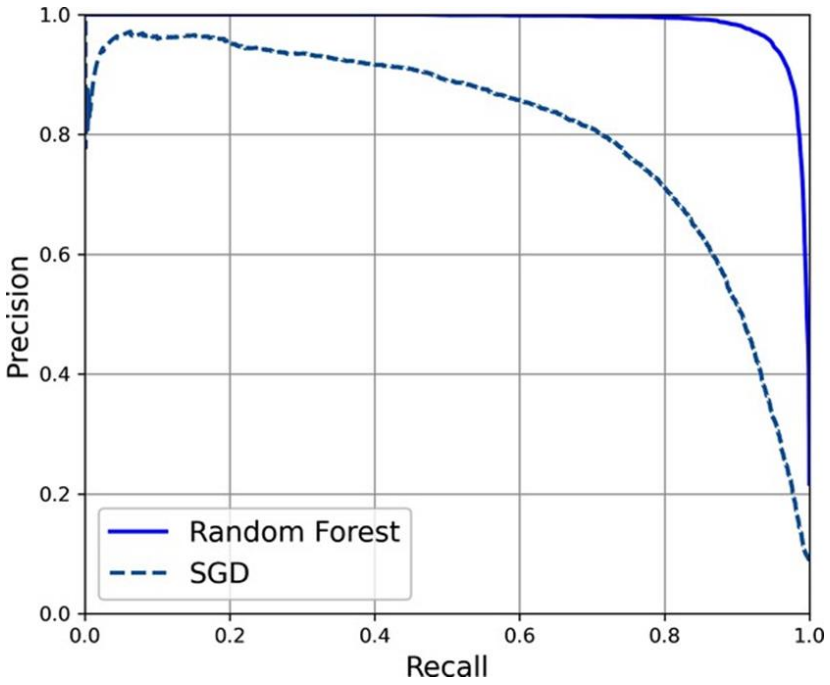
Mô hình dự đoán rằng hình ảnh đầu tiên là tích cực với xác suất 89%, và nó dự đoán rằng hình ảnh thứ hai là tiêu cực với xác suất 99%. Vì mỗi hình ảnh hoặc là tích cực hoặc là tiêu cực, xác suất trong mỗi hàng cộng lại bằng 100%.

Cột thứ hai chứa xác suất ước tính cho lớp dương tính, vì vậy hãy truyền chúng cho hàm precision_recall_curve():

```
y_scores_forest = y_probas_forest[:, 1]
precisions_forest, recalls_forest, thresholds_forest = \
    precision_recall_curve(y_train_5, y_scores_forest)
```

Bây giờ chúng ta đã sẵn sàng vẽ đường cong PR. Việc vẽ đường cong PR đầu tiên cũng hữu ích để xem chúng so sánh như thế nào (Hình 3-8):

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,
         label="Random Forest")
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")
[...] # beautify the figure: add labels, grid, and legend
plt.show()
```



Hình 3-8. So sánh đường cong PR: bộ phân loại rừng ngẫu nhiên vượt trội hơn bộ phân loại SGD vì đường cong PR của nó gần góc trên bên phải hơn nhiều, và nó có AUC lớn hơn.

Như bạn có thể thấy trong Hình 3-8, đường cong PR của `RandomForestClassifier` trông tốt hơn nhiều so với `SGDClassifier`: nó gần góc trên bên phải hơn nhiều. Điểm F1 và điểm ROC AUC của nó cũng tốt hơn đáng kể:

```
>>> y_train_pred_forest = y_probas_forest[:, 1] >= 0.5 # positive proba ≥ 50%
>>> f1_score(y_train_5, y_train_pred_forest)
0.9242275142688446
>>> roc_auc_score(y_train_5, y_scores_forest)
0.9983436731328145
```

Hãy thử đo điểm độ chính xác và độ nhạy: bạn sẽ tìm thấy khoảng 99.1% độ chính xác và 86.6% độ nhạy. Không tệ chút nào!

Bây giờ bạn đã biết cách huấn luyện các bộ phân loại nhị phân, chọn chỉ số phù hợp cho tác vụ của bạn, đánh giá các bộ phân loại của bạn bằng cách sử dụng kiểm định chéo, chọn sự đánh đổi độ chính xác/độ nhạy phù hợp với nhu cầu của bạn, và sử dụng một số chỉ số và đường cong để so sánh các mô hình khác nhau. Bạn đã sẵn sàng thử phát hiện nhiều hơn chỉ số 5.

3.4 Phân loại đa lớp

Trong khi các bộ phân loại nhị phân phân biệt giữa hai lớp, các bộ phân loại đa lớp (còn được gọi là bộ phân loại đa thức) có thể phân biệt giữa nhiều hơn hai lớp.

Một số bộ phân loại của Scikit-Learn (ví dụ: `LogisticRegression`, `RandomForestClassifier` và `GaussianNB`) có khả năng xử lý nhiều lớp một cách tự nhiên.

Các bộ phân loại khác lại là bộ phân loại nhị phân nghiêm ngặt (ví dụ: `SGDClassifier` và `SVC`). Tuy nhiên, có nhiều chiến lược khác nhau mà bạn có thể sử dụng để thực hiện phân loại đa lớp với nhiều bộ phân loại nhị phân.

Một cách để tạo một hệ thống có thể phân loại hình ảnh chữ số thành 10 lớp (từ 0 đến 9) là huấn luyện 10 bộ phân loại nhị phân, mỗi bộ cho một chữ số (một bộ phát hiện số 0, một bộ phát hiện số 1, một bộ phát hiện số 2, v.v.). Sau đó, khi bạn muốn phân loại một hình ảnh, bạn sẽ lấy điểm quyết định từ mỗi bộ phân loại cho hình ảnh đó và chọn lớp có bộ phân loại xuất ra điểm cao nhất. Điều này được gọi là chiến lược một-đối-phần-còn-lại (OvR), hoặc đôi khi là một-đối-tất-cả (OvA).

Một chiến lược khác là huấn luyện một bộ phân loại nhị phân cho mỗi cặp chữ số: một để phân biệt số 0 và số 1, một để phân biệt số 0 và số 2, một để phân biệt số 1 và số 2, v.v. Điều này được gọi là chiến lược một-đối-một (OvO). Nếu có N lớp, bạn cần huấn luyện N × (N – 1) / 2 bộ phân loại. Đối với vấn đề MNIST, điều này có nghĩa là huấn luyện 45 bộ phân loại nhị phân! Khi bạn muốn phân loại một hình ảnh, bạn phải chạy hình ảnh đó qua tất cả 45 bộ phân loại và xem lớp nào thắng nhiều trận đấu nhất. Ưu điểm chính của OvO là mỗi bộ phân loại chỉ cần được huấn luyện trên phần tập huấn luyện chứa hai lớp mà nó phải phân biệt.

Một số thuật toán (chẳng hạn như bộ phân loại máy vector hỗ trợ) mở rộng kém với kích thước của tập huấn luyện. Đối với các thuật toán này, OvO được ưu tiên vì huấn luyện nhiều bộ phân loại trên các tập huấn luyện nhỏ nhanh hơn so với huấn luyện ít bộ phân loại trên các tập huấn luyện lớn. Tuy nhiên, đối với hầu hết các thuật toán phân loại nhị phân, OvR được ưu tiên hơn.

Scikit-Learn phát hiện khi bạn cố gắng sử dụng thuật toán phân loại nhị phân cho tác vụ phân loại đa lớp, và nó tự động chạy OvR hoặc OvO, tùy thuộc vào thuật toán. Hãy thử điều này với bộ phân loại máy vector hỗ trợ sử dụng lớp `sklearn.svm.SVC` (xem Chương 5). Chúng ta sẽ chỉ huấn luyện trên 2.000 hình ảnh đầu tiên, nếu không sẽ mất rất nhiều thời gian:

```
from sklearn.svm import SVC

svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train, not y_train_5
```

Thật dễ dàng! Chúng ta đã huấn luyện `SVC` bằng cách sử dụng các lớp mục tiêu gốc từ 0 đến 9 (`y_train`), thay vì các lớp mục tiêu 5-đối-phần-còn-lại (`y_train_5`).

Vì có 10 lớp (tức là hơn 2), Scikit-Learn đã sử dụng chiến lược OvO và huấn luyện 45 bộ phân loại nhị phân. Bây giờ hãy đưa ra dự đoán trên một hình ảnh:

```
>>> svm_clf.predict([some_digit])
array(['5'], dtype=object)
```

Đúng rồi! Mã này thực sự đã đưa ra 45 dự đoán — một cho mỗi cặp lớp — và nó đã chọn lớp thắng nhiều "trận đấu" nhất. Nếu bạn gọi phương thức `decision_function()`, bạn sẽ thấy

nó trả về 10 điểm cho mỗi trường hợp: một cho mỗi lớp. Mỗi lớp nhận được một điểm bằng số "trận đấu" thắng được cộng hoặc trừ một điều chỉnh nhỏ (tối đa ±0.33) để phá vỡ các trường hợp hòa, dựa trên điểm số của bộ phân loại:

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])
>>> some_digit_scores.round(2)
array([[ 3.79,  0.73,  6.06,  8.3 , -0.29,  9.3 ,  1.75,  2.77,  7.21,
         4.82]])
```

Điểm cao nhất là 9.3, và đó thực sự là điểm tương ứng với lớp 5:

```
>>> class_id = some_digit_scores.argmax()
>>> class_id
5
```

Khi một bộ phân loại được huấn luyện, nó lưu trữ danh sách các lớp mục tiêu trong thuộc tính `classes_` của nó, được sắp xếp theo giá trị. Trong trường hợp MNIST, chỉ mục của mỗi lớp trong mảng `classes_` trùng khớp với chính lớp đó (ví dụ: lớp ở chỉ mục 5 tình cờ là lớp '5'), nhưng nói chung bạn sẽ không may mắn như vậy; bạn sẽ cần tra cứu nhãn lớp như thế này:

```
>>> svm_clf.classes_
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
>>> svm_clf.classes_[class_id]
'5'
```

Nếu bạn muốn buộc Scikit-Learn sử dụng chiến lược một-đối-một hoặc một-đối-phần-còn-lại, bạn có thể sử dụng các lớp `OneVsOneClassifier` hoặc `OneVsRestClassifier`. Chỉ cần tạo một thể hiện và truyền một bộ phân loại vào hàm tạo của nó (nó thậm chí không cần phải là bộ phân loại nhị phân). Ví dụ, mã này tạo một bộ phân loại đa lớp sử dụng chiến lược OvR, dựa trên một `SVC`:

```
from sklearn.multiclass import OneVsRestClassifier

ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

Hãy đưa ra dự đoán và kiểm tra số lượng bộ phân loại đã được huấn luyện:

```
>>> ovr_clf.predict([some_digit])
array(['5'], dtype='<U1')
>>> len(ovr_clf.estimators_)
10
```

Huấn luyện một `SGDClassifier` trên một tập dữ liệu đa lớp và sử dụng nó để đưa ra dự đoán cũng dễ dàng như vậy:

```
>>> sgd_clf = SGDClassifier(random_state=42)
>>> sgd_clf.fit(X_train, y_train)
```

```
>>> sgd_clf.predict([some_digit])
array(['3'], dtype='<U1')
```

Ối, sai rồi. Lỗi dự đoán vẫn xảy ra! Lần này Scikit-Learn đã sử dụng chiến lược OvR ẩn bên trong: vì có 10 lớp, nó đã huấn luyện 10 bộ phân loại nhị phân. Phương thức `decision_function()` bây giờ trả về một giá trị cho mỗi lớp. Hãy xem các điểm mà bộ phân loại SGD đã gán cho mỗi lớp:

```
>>> sgd_clf.decision_function([some_digit]).round()
array([[-31893., -34420.,  -9531.,   1824., -22320.,  -1386., -26189.,
        -16148.,  -4604., -12051.]])
```

Bạn có thể thấy rằng bộ phân loại không tự tin lắm về dự đoán của mình: hầu hết các điểm đều rất âm, trong khi lớp 3 có điểm +1.824, và lớp 5 không quá xa phía sau ở -1.386. Tất nhiên, bạn sẽ muốn đánh giá bộ phân loại này trên nhiều hơn một hình ảnh. Vì có khoảng cùng số lượng hình ảnh trong mỗi lớp, chỉ số độ chính xác là ổn. Như thường lệ, bạn có thể sử dụng hàm `cross_val_score()` để đánh giá mô hình:

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
array([0.87365, 0.85835, 0.8689 ])
```

Nó đạt hơn 85.8% trên tất cả các fold kiểm thử. Nếu bạn sử dụng một bộ phân loại ngẫu nhiên, bạn sẽ đạt độ chính xác 10%, vì vậy đây không phải là một điểm số quá tệ, nhưng bạn vẫn có thể làm tốt hơn nhiều. Đơn giản là điều chỉnh tỷ lệ đầu vào (như đã thảo luận trong Chương 2) làm tăng độ chính xác lên trên 89.1%:

```
from sklearn.preprocessing import StandardScaler

>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3,
scoring="accuracy")
array([0.8983, 0.891 , 0.9018])
```

3.5 Phân tích lỗi

Nếu đây là một dự án thực tế, bây giờ bạn sẽ thực hiện các bước trong danh sách kiểm tra dự án học máy của mình (xem Phụ lục A). Bạn sẽ khám phá các tùy chọn chuẩn bị dữ liệu, thử nhiều mô hình, chọn lọc những mô hình tốt nhất, tinh chỉnh siêu tham số của chúng bằng `GridSearchCV` và tự động hóa càng nhiều càng tốt. Ở đây, chúng ta sẽ giả định rằng bạn đã tìm thấy một mô hình hứa hẹn và bạn muốn tìm cách cải thiện nó. Một cách để làm điều này là phân tích các loại lỗi mà nó mắc phải.

Đầu tiên, hãy xem ma trận nhầm lẫn. Để làm điều này, trước tiên bạn cần tạo dự đoán bằng hàm `cross_val_predict()`; sau đó bạn có thể truyền nhãn và dự đoán cho hàm `confusion_matrix()`, giống như bạn đã làm trước đây. Tuy nhiên, vì bây giờ có 10 lớp thay vì 2, ma trận nhầm lẫn sẽ chứa khá nhiều số, và có thể khó đọc.

Một biểu đồ màu của ma trận nhầm lẫn dễ phân tích hơn nhiều. Để vẽ biểu đồ như vậy, hãy sử dụng hàm `ConfusionMatrixDisplay.from_predictions()` như sau:

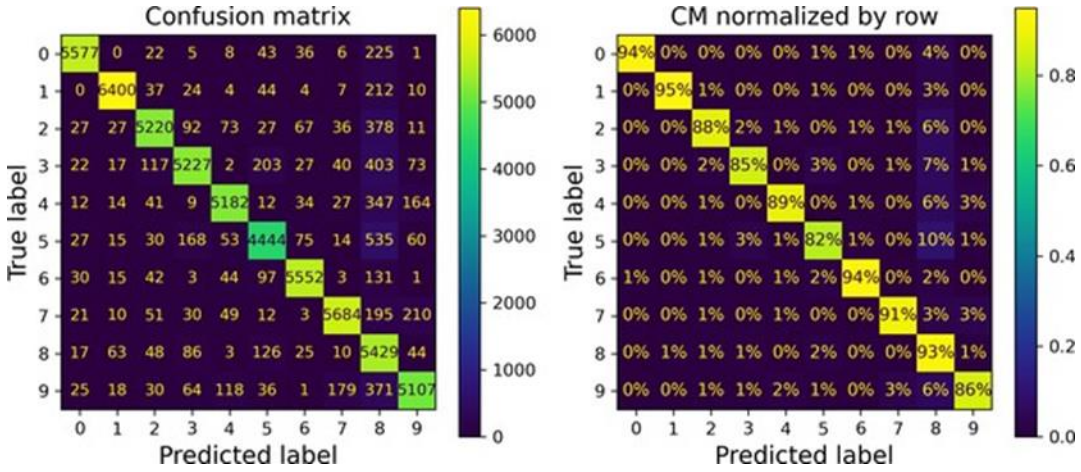
```
from sklearn.metrics import ConfusionMatrixDisplay

y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```

Điều này tạo ra biểu đồ bên trái trong Hình 3-9. Ma trận nhầm lẫn này trông khá tốt: hầu hết các hình ảnh nằm trên đường chéo chính, có nghĩa là chúng được phân loại đúng. Lưu ý rằng ô trên đường chéo ở hàng #5 và cột #5 trông hơi tối hơn các chữ số khác. Điều này có thể là do mô hình mắc nhiều lỗi hơn với các số 5, hoặc vì có ít số 5 hơn trong tập dữ liệu so với các chữ số khác. Đó là lý do tại sao điều quan trọng là phải chuẩn hóa ma trận nhầm lẫn bằng cách chia mỗi giá trị cho tổng số hình ảnh trong lớp (thực tế) tương ứng (tức là chia cho tổng hàng). Điều này có thể được thực hiện đơn giản bằng cách đặt `normalize="true"`. Chúng ta cũng có thể chỉ định đối số `values_format=".0%"` để hiển thị phần trăm không có số thập phân. Đoạn mã sau tạo ra biểu đồ bên phải trong Hình 3-9:

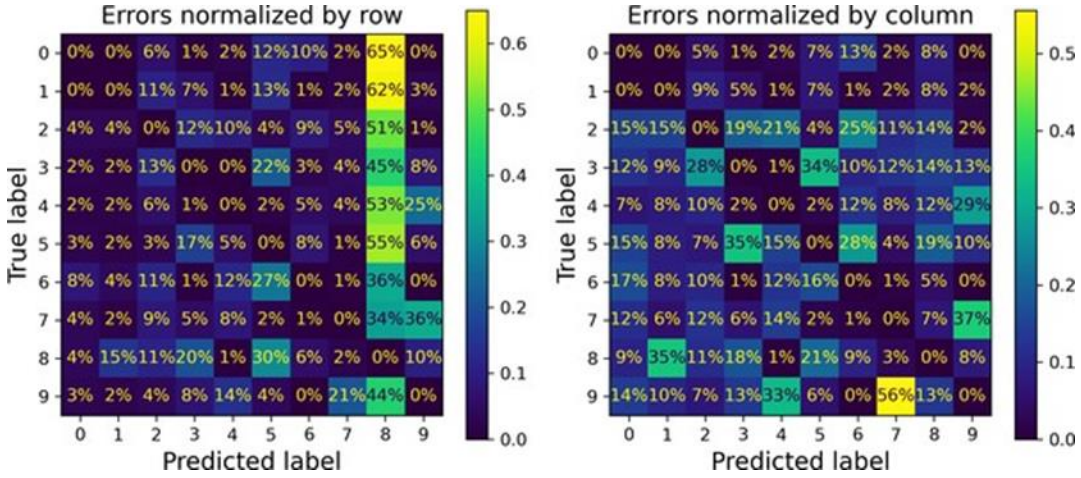
```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        normalize="true",
                                        values_format=".0%")
plt.show()
```

Bây giờ chúng ta có thể dễ dàng thấy rằng chỉ có 82% hình ảnh số 5 được phân loại đúng. Lỗi phổ biến nhất mà mô hình mắc phải với hình ảnh số 5 là phân loại sai chúng thành số 8: điều này xảy ra với 10% tổng số số 5. Nhưng chỉ 2% số 8 bị phân loại sai thành số 5; ma trận nhầm lẫn nói chung không đối xứng! Nếu bạn nhìn kỹ, bạn sẽ nhận thấy rằng nhiều chữ số đã bị phân loại sai thành số 8, nhưng điều này không hiển thị rõ ngay lập tức từ biểu đồ này. Nếu bạn muốn làm cho các lỗi nổi bật hơn, bạn có thể thử đặt trọng số 0 cho các dự đoán đúng. Đoạn mã sau đây thực hiện điều đó và tạo ra biểu đồ bên trái trong Hình 3-10:

```
sample_weight = (y_train_pred != y_train)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        sample_weight=sample_weight,
                                        normalize="true",
                                        values_format=".0%")
plt.show()
```



Hình 3-9. Ma trận nhầm lẫn (trái) và cùng ma trận nhầm lẫn được chuẩn hóa theo hàng (phải).



Hình 3-10. Ma trận nhầm lẫn chỉ hiển thị lỗi, được chuẩn hóa theo hàng (trái) và theo cột (phải).

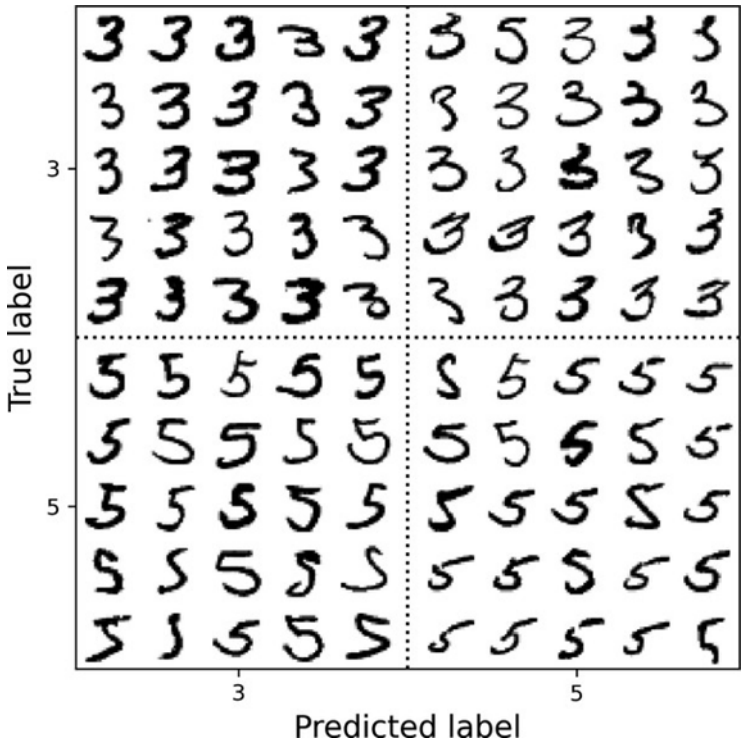
Bây giờ bạn có thể thấy rõ hơn nhiều các loại lỗi mà bộ phân loại mắc phải. Cột dành cho lớp 8 bây giờ thực sự sáng, điều này xác nhận rằng nhiều hình ảnh bị phân loại sai là số 8. Trên thực tế, đây là lỗi phân loại sai phổ biến nhất đối với hầu hết các lớp. Nhưng hãy cẩn thận cách bạn diễn giải tỷ lệ phần trăm trong biểu đồ này: hãy nhớ rằng chúng ta đã loại trừ các dự đoán đúng. Ví dụ, 36% ở hàng #7, cột #9 không có nghĩa là 36% tất cả các hình ảnh số 7 bị phân loại sai là số 9. Nó có nghĩa là 36% các lỗi mà mô hình mắc phải trên hình ảnh số 7 là phân loại sai là số 9. Trong thực tế, chỉ 3% hình ảnh số 7 bị phân loại sai là số 9, như bạn có thể thấy trong biểu đồ bên phải trong Hình 3-9.

Cũng có thể chuẩn hóa ma trận nhầm lẫn theo cột thay vì theo hàng: nếu bạn đặt `normalize="pred"`, bạn sẽ nhận được biểu đồ ở bên phải trong Hình 3-10. Ví dụ, bạn có thể thấy rằng 56% số 7 bị phân loại sai thực sự là số 9.

Phân tích ma trận nhầm lẫn thường cung cấp cho bạn cái nhìn sâu sắc về các cách để cải thiện bộ phân loại của bạn. Nhìn vào các biểu đồ này, có vẻ như nỗ lực của bạn nên tập trung vào việc giảm các số 8 sai. Ví dụ, bạn có thể thử thu thập thêm dữ liệu huấn luyện cho các chữ số trông giống số 8 (nhưng không phải) để bộ phân loại có thể học cách phân biệt chúng với số 8 thực. Hoặc bạn có thể thiết kế các đặc trưng mới sẽ giúp bộ phân loại — ví dụ, viết một thuật toán để đếm số vòng tròn kín (ví dụ: số 8 có hai, số 6 có một, số 5 không có). Hoặc bạn có thể tiền xử lý hình ảnh (ví dụ: sử dụng Scikit-Image, Pillow hoặc OpenCV) để làm cho một số mẫu, chẳng hạn như vòng tròn kín, nổi bật hơn.

Phân tích các lỗi riêng lẻ cũng có thể là một cách tốt để hiểu rõ hơn về những gì bộ phân loại của bạn đang làm và tại sao nó lại thất bại. Ví dụ, hãy vẽ biểu đồ các ví dụ về số 3 và số 5 theo kiểu ma trận nhầm lẫn (Hình 3-11):

```
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
[...] # plot all images in X_aa, X_ab, X_ba, X_bb in a confusion matrix style
```



Hình 3-11. Một số hình ảnh số 3 và số 5 được tổ chức như một ma trận nhầm lẫn.

Như bạn có thể thấy, một số chữ số mà bộ phân loại mắc lỗi (tức là trong các khối dưới cùng bên trái và trên cùng bên phải) được viết quá tệ đến nỗi ngay cả con người cũng gặp khó khăn khi phân loại chúng. Tuy nhiên, hầu hết các hình ảnh bị phân loại sai dường như là lỗi hiển nhiên đối với chúng ta. Có thể khó hiểu tại sao bộ phân loại lại mắc lỗi, nhưng hãy nhớ rằng bộ não con người là một hệ thống nhận dạng mẫu tuyệt vời, và hệ thống thị

giác của chúng ta thực hiện rất nhiều tiền xử lý phức tạp trước khi bất kỳ thông tin nào đến được ý thức của chúng ta. Vì vậy, việc tác vụ này có vẻ đơn giản không có nghĩa là nó thực sự đơn giản. Hãy nhớ rằng chúng ta đã sử dụng một `SGDClassifier` đơn giản, đây chỉ là một mô hình tuyến tính: tất cả những gì nó làm là gán một trọng số cho mỗi lớp cho mỗi pixel, và khi nó thấy một hình ảnh mới, nó chỉ đơn giản là tổng hợp cường độ pixel có trọng số để có được điểm số cho mỗi lớp. Vì số 3 và số 5 chỉ khác nhau vài pixel, mô hình này sẽ dễ dàng nhầm lẫn chúng.

Sự khác biệt chính giữa số 3 và số 5 là vị trí của đường nhỏ nối đường trên cùng với cung dưới cùng. Nếu bạn vẽ số 3 với điểm nối hơi lệch sang trái, bộ phân loại có thể phân loại nó là số 5, và ngược lại. Nói cách khác, bộ phân loại này khá nhạy cảm với sự dịch chuyển và xoay hình ảnh.

Một cách để giảm nhầm lẫn giữa số 3/5 là tiền xử lý hình ảnh để đảm bảo chúng được căn giữa tốt và không bị xoay quá nhiều. Tuy nhiên, điều này có thể không dễ dàng vì nó yêu cầu dự đoán hướng xoay đúng của mỗi hình ảnh. Một cách tiếp cận đơn giản hơn nhiều là tăng cường tập huấn luyện bằng cách thêm các biến thể hình ảnh huấn luyện đã bị dịch chuyển và xoay nhẹ. Điều này sẽ buộc mô hình học cách chịu đựng tốt hơn các biến thể như vậy. Điều này được gọi là tăng cường dữ liệu (data augmentation) (chúng ta sẽ đề cập đến điều này trong Chương 14; cũng xem bài tập 2 ở cuối chương này).

3.6 Phân loại đa nhãn

Cho đến nay, mỗi trường hợp luôn được gán cho chỉ một lớp. Nhưng trong một số trường hợp, bạn có thể muốn bộ phân loại của mình xuất ra nhiều lớp cho mỗi trường hợp. Hãy xem xét một bộ phân loại nhận dạng khuôn mặt: nó nên làm gì nếu nó nhận ra một số người trong cùng một bức ảnh? Nó nên gắn một thẻ cho mỗi người mà nó nhận ra. Giả sử bộ phân loại đã được huấn luyện để nhận dạng ba khuôn mặt: Alice, Bob và Charlie. Sau đó, khi bộ phân loại được hiển thị một bức ảnh của Alice và Charlie, nó sẽ xuất ra `[True, False, True]` (nghĩa là “Alice có, Bob không, Charlie có”). Một hệ thống phân loại xuất ra nhiều thẻ nhị phân như vậy được gọi là hệ thống phân loại đa nhãn.

Chúng ta sẽ chưa đi sâu vào nhận dạng khuôn mặt, nhưng hãy xem một ví dụ đơn giản hơn, chỉ để minh họa:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

# Giả định X_train và y_train đã được định nghĩa từ dữ liệu MNIST hoặc tương
tự
# Ví dụ:
# from sklearn.datasets import fetch_openml
# mnist = fetch_openml('mnist_784', as_frame=False)
# X, y = mnist.data, mnist.target
# X_train = X[:60000]
# y_train = y[:60000]
```

```
y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

Đoạn mã này tạo ra một mảng `y_multilabel` chứa hai nhãn mục tiêu cho mỗi hình ảnh chữ số: nhãn đầu tiên cho biết chữ số đó có lớn (7, 8 hoặc 9) hay không, và nhãn thứ hai cho biết chữ số đó có lẻ hay không. Sau đó, đoạn mã tạo một thể hiện `KNeighborsClassifier`, hỗ trợ phân loại đa nhãn (không phải tất cả các bộ phân loại đều hỗ trợ), và huấn luyện mô hình này bằng cách sử dụng mảng mục tiêu đa nhãn. Bây giờ bạn có thể đưa ra dự đoán và nhận thấy rằng nó xuất ra hai nhãn:

```
>>> knn_clf.predict([some_digit]) # some_digit là một ví dụ từ tập X_train
array([[False, True]])
```

Và nó đoán đúng! Chữ số 5 thực sự không lớn (False) và lẻ (True).

Có nhiều cách để đánh giá một bộ phân loại đa nhãn, và việc chọn chỉ số phù hợp thực sự phụ thuộc vào dự án của bạn. Một cách tiếp cận là đo điểm F1 cho từng nhãn riêng lẻ (hoặc bất kỳ chỉ số bộ phân loại nhị phân nào khác đã thảo luận trước đó), sau đó chỉ cần tính điểm trung bình. Đoạn mã sau tính điểm F1 trung bình trên tất cả các nhãn:

```
# Giả định cross_val_predict và f1_score đã được import
# from sklearn.model_selection import cross_val_predict
# from sklearn.metrics import f1_score
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
>>> f1_score(y_multilabel, y_train_knn_pred, average="macro") [cite: 5]
0.976410265560605
```

Cách tiếp cận này giả định rằng tất cả các nhãn đều quan trọng như nhau, điều này có thể không đúng. Đặc biệt, nếu bạn có nhiều ảnh của Alice hơn Bob hoặc Charlie, bạn có thể muốn ưu tiên điểm số của bộ phân loại trên ảnh của Alice. Một lựa chọn đơn giản là gán cho mỗi nhãn một trọng số bằng với *hỗ trợ* của nó (tức là số lượng trường hợp có nhãn mục tiêu đó). Để làm điều này, chỉ cần đặt `average="weighted"` khi gọi hàm `f1_score()`.

Nếu bạn muốn sử dụng một bộ phân loại không hỗ trợ phân loại đa nhãn một cách tự nhiên, chẳng hạn như SVC, một chiến lược khả thi là huấn luyện một mô hình cho mỗi nhãn. Tuy nhiên, chiến lược này có thể gặp khó khăn trong việc nắm bắt các phụ thuộc giữa các nhãn. Ví dụ, một chữ số lớn (7, 8 hoặc 9) có khả năng lẻ gấp đôi so với chẵn, nhưng bộ phân loại cho nhãn “lẻ” không biết bộ phân loại cho nhãn “lớn” đã dự đoán gì. Để giải quyết vấn đề này, các mô hình có thể được tổ chức thành một chuỗi: khi một mô hình đưa ra dự đoán, nó sử dụng các đặc trưng đầu vào cộng với tất cả các dự đoán của các mô hình đi trước nó trong chuỗi.

Tin tốt là Scikit-Learn có một lớp tên là `ChainClassifier` làm được điều đó! Theo mặc định, nó sẽ sử dụng các nhãn đúng để huấn luyện, cung cấp cho mỗi mô hình các nhãn thích hợp

tùy thuộc vào vị trí của chúng trong chuỗi. Nhưng nếu bạn đặt siêu tham số `cv`, nó sẽ sử dụng kiểm định chéo để có được các dự đoán "sạch" (ngoài mẫu) từ mỗi mô hình đã huấn luyện cho mọi trường hợp trong tập huấn luyện, và những dự đoán này sau đó sẽ được sử dụng để huấn luyện tất cả các mô hình sau đó trong chuỗi. Dưới đây là một ví dụ cho thấy cách tạo và huấn luyện `ChainClassifier` bằng cách sử dụng chiến lược kiểm định chéo. Như trước, chúng ta sẽ chỉ sử dụng 2.000 hình ảnh đầu tiên trong tập huấn luyện để tăng tốc:

```
from sklearn.multioutput import ClassifierChain
from sklearn.svm import SVC # Import SVC as it's used in the example

chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

Bây giờ chúng ta có thể sử dụng `ChainClassifier` này để đưa ra dự đoán:

```
>>> chain_clf.predict([some_digit])
array([[0., 1.]])
```

3.7 Phân loại đa đầu ra

Loại tác vụ phân loại cuối cùng chúng ta sẽ thảo luận ở đây được gọi là phân loại đa đầu ra – đa lớp (hoặc chỉ phân loại đa đầu ra). Đây là một khái quát hóa của phân loại đa nhãn, trong đó mỗi nhãn có thể là đa lớp (tức là nó có thể có nhiều hơn hai giá trị có thể).

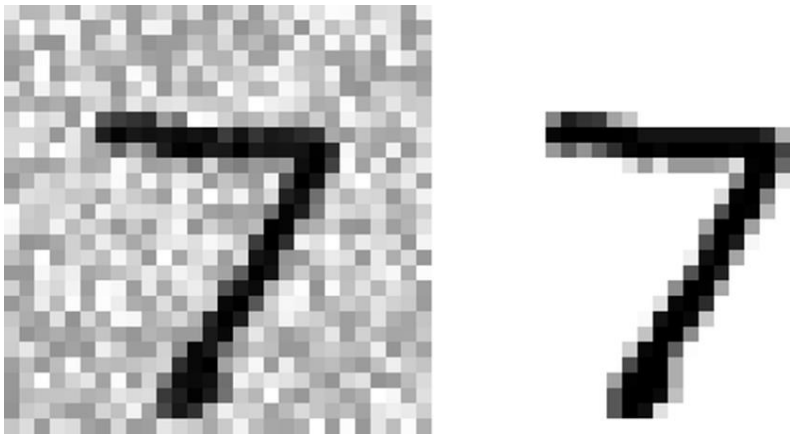
Để minh họa điều này, hãy xây dựng một hệ thống loại bỏ nhiễu khỏi hình ảnh. Nó sẽ nhận một hình ảnh chữ số bị nhiễu làm đầu vào, và nó sẽ (hy vọng) xuất ra một hình ảnh chữ số sạch, được biểu diễn dưới dạng một mảng cường độ pixel, giống như các hình ảnh MNIST. Lưu ý rằng đầu ra của bộ phân loại là đa nhãn (một nhãn cho mỗi pixel) và mỗi nhãn có thể có nhiều giá trị (cường độ pixel dao động từ 0 đến 255). Do đó, đây là một ví dụ về hệ thống phân loại đa đầu ra.

Hãy bắt đầu bằng cách tạo các tập huấn luyện và kiểm thử bằng cách lấy các hình ảnh MNIST và thêm nhiễu vào cường độ pixel của chúng bằng hàm `randint()` của NumPy. Các hình ảnh mục tiêu sẽ là các hình ảnh gốc:

```
import numpy as np
# Giả định X_train, X_test đã được định nghĩa từ dữ liệu MNIST hoặc tương tự
# Ví dụ:
# from sklearn.datasets import fetch_openml
# mnist = fetch_openml('mnist_784', as_frame=False)
# X, y = mnist.data, mnist.target
# X_train = X[:60000]
# X_test = X[60000:]

np.random.seed(42) # to make this code example reproducible
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
```

```
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

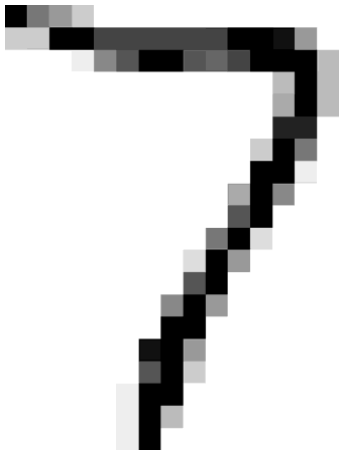
Hãy xem lướt qua hình ảnh đầu tiên từ tập kiểm thử (Hình 3-12). Vâng, chúng ta đang "rình mò" dữ liệu kiểm thử, vì vậy bạn nên nhíu mày ngay bây giờ.



Hình 3-12. Một hình ảnh bị nhiễu (trái) và hình ảnh mục tiêu sạch (phải).

Bên trái là hình ảnh đầu vào bị nhiễu, và bên phải là hình ảnh mục tiêu sạch. Bây giờ hãy huấn luyện bộ phân loại và làm cho nó làm sạch hình ảnh này (Hình 3-13):

```
from sklearn.neighbors import KNeighborsClassifier
# Giả định plot_digit đã được định nghĩa
# import matplotlib.pyplot as plt
# def plot_digit(image_data):
#     image = image_data.reshape(28, 28)
#     plt.imshow(image, cmap="binary")
#     plt.axis("off")

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```



Hình 3-13. Hình ảnh đã được làm sạch.

Trông khá giống với mục tiêu! Điều này kết thúc chuyến tham quan của chúng ta về phân loại. Bây giờ bạn đã biết cách chọn các chỉ số tốt cho các tác vụ phân loại, chọn sự đánh đổi độ chính xác/độ nhạy phù hợp, so sánh các bộ phân loại và tổng quát hơn là xây dựng các hệ thống phân loại tốt cho nhiều tác vụ khác nhau. Trong các chương tiếp theo, bạn sẽ tìm hiểu cách tất cả các mô hình học máy mà bạn đã sử dụng thực sự hoạt động.

3.8 Bài tập

- Hãy thử xây dựng một bộ phân loại cho tập dữ liệu MNIST đạt độ chính xác trên 97% trên tập kiểm thử. Gợi ý: `KNeighborsClassifier` hoạt động khá tốt cho tác vụ này; bạn chỉ cần tìm các giá trị siêu tham số tốt (hãy thử tìm kiếm lưới trên các siêu tham số `weights` và `n_neighbors`).
- Viết một hàm có thể dịch chuyển một hình ảnh MNIST theo bất kỳ hướng nào (trái, phải, lên hoặc xuống) một pixel. Sau đó, đối với mỗi hình ảnh trong tập huấn luyện, tạo bốn bản sao đã dịch chuyển (một cho mỗi hướng) và thêm chúng vào tập huấn luyện. Cuối cùng, huấn luyện mô hình tốt nhất của bạn trên tập huấn luyện đã mở rộng này và đo độ chính xác của nó trên tập kiểm thử. Bạn sẽ quan sát thấy rằng mô hình của bạn hoạt động tốt hơn nữa! Kỹ thuật này của việc tăng cường tập huấn luyện một cách nhân tạo được gọi là tăng cường dữ liệu (data augmentation) hoặc mở rộng tập huấn luyện.
- Xử lý tập dữ liệu Titanic. Một nơi tuyệt vời để bắt đầu là trên Kaggle. Ngoài ra, bạn có thể tải dữ liệu từ https://homl.info/titanic.tgz và giải nén tarball này giống như bạn đã làm với dữ liệu nhà ở trong Chương 2. Điều này sẽ cung cấp cho bạn hai tệp CSV, `train.csv` và `test.csv`, mà bạn có thể tải bằng `pandas.read_csv()`. Mục tiêu là huấn luyện một bộ phân loại có thể dự đoán cột `Survived` dựa trên các cột khác.
- Xây dựng bộ phân loại thư rác (một bài tập thử thách hơn): a. Tải xuống các ví dụ về thư rác và thư hợp lệ từ các tập dữ liệu công khai của Apache SpamAssassin. b. Giải nén các tập dữ liệu và làm quen với định dạng dữ liệu. c. Chia dữ liệu thành tập huấn luyện và tập kiểm thử. d. Viết một pipeline chuẩn bị dữ liệu để chuyển đổi mỗi email thành một vector đặc trưng. Pipeline chuẩn bị của bạn nên biến đổi một email

thành một vector (thưa) cho biết sự có mặt hoặc vắng mặt của mỗi từ có thể. Ví dụ, nếu tất cả các email chỉ chứa bốn từ, “Hello”, “how”, “are”, “you”, thì email “Hello you Hello Hello you” sẽ được chuyển đổi thành một vector `[1, 0, 0, 1]` (nghĩa là [“Hello” có mặt, “how” vắng mặt, “are” vắng mặt, “you” có mặt]), hoặc `[3, 0, 0, 2]` nếu bạn muốn đếm số lần xuất hiện của mỗi từ. Bạn có thể muốn thêm các siêu tham số vào pipeline chuẩn bị của mình để kiểm soát việc có loại bỏ tiêu đề email hay không, chuyển đổi mỗi email thành chữ thường, loại bỏ dấu câu, thay thế tất cả URL bằng “URL”, thay thế tất cả các số bằng “NUMBER”, hoặc thậm chí thực hiện phân tách từ gốc (stemming) (tức là cắt bỏ các đuôi từ; có các thư viện Python có sẵn để làm điều này). e. Cuối cùng, hãy thử một vài bộ phân loại và xem liệu bạn có thể xây dựng một bộ phân loại thư rác tuyệt vời, với cả độ nhạy cao và độ chính xác cao. Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại https://homl.info/colab3 .

- Theo mặc định, Scikit-Learn lưu trữ các tập dữ liệu đã tải xuống trong một thư mục có tên `scikit_learn_data` trong thư mục chính của bạn.
- Các tập dữ liệu được trả về bởi `fetch_openml()` không phải lúc nào cũng được xáo trộn hoặc chia.
- Việc xáo trộn có thể là một ý tưởng tồi trong một số ngữ cảnh — ví dụ, nếu bạn đang làm việc với dữ liệu chuỗi thời gian (chẳng hạn như giá thị trường chứng khoán hoặc điều kiện thời tiết). Chúng ta sẽ khám phá điều này trong Chương 15.
- Các bộ phân loại của Scikit-Learn luôn có phương thức `decision_function()` hoặc phương thức `predict_proba()`, hoặc đôi khi cả hai.
- Scikit-Learn cung cấp một vài tùy chọn trung bình hóa khác và các chỉ số bộ phân loại đa nhãn; xem tài liệu để biết thêm chi tiết.
- Bạn có thể sử dụng hàm `shift()` từ mô-đun `scipy.ndimage.interpolation`. Ví dụ, `shift(image, [2, 1], cval=0)` dịch chuyển hình ảnh xuống hai pixel và sang phải một pixel.

CHƯƠNG 4. HUẤN LUYỆN MÔ HÌNH

Cho đến nay, chúng ta đã coi các mô hình học máy và thuật toán huấn luyện của chúng chủ yếu như "hộp đen". Nếu bạn đã thực hiện một số bài tập trong các chương trước, bạn có thể đã ngạc nhiên về mức độ bạn có thể làm được mà không cần biết bất cứ điều gì về "những gì bên dưới": bạn đã tối ưu hóa một hệ thống hồi quy, cải thiện bộ phân loại hình ảnh chữ số, và thậm chí xây dựng một bộ phân loại thư rác từ đầu, tất cả mà không biết chúng thực sự hoạt động như thế nào. Thật vậy, trong nhiều tình huống, bạn thực sự không cần biết chi tiết triển khai.

Tuy nhiên, việc hiểu rõ cách mọi thứ hoạt động có thể giúp bạn nhanh chóng chọn được mô hình phù hợp, thuật toán huấn luyện đúng để sử dụng và một tập hợp siêu tham số tốt cho tác vụ của bạn. Hiểu biết về "những gì bên dưới" cũng sẽ giúp bạn gỡ lỗi các vấn đề và thực hiện phân tích lỗi hiệu q

uả hơn. Cuối cùng, hầu hết các chủ đề được thảo luận trong chương này sẽ rất cần thiết để hiểu, xây dựng và huấn luyện mạng nơ-ron (được thảo luận trong Phần II của cuốn sách này).

Trong chương này, chúng ta sẽ bắt đầu bằng cách xem xét mô hình hồi quy tuyến tính, một trong những mô hình đơn giản nhất hiện có. Chúng ta sẽ thảo luận hai cách rất khác nhau để huấn luyện nó:

- Sử dụng một phương trình "dạng đóng" trực tiếp tính toán các tham số mô hình phù hợp nhất với tập huấn luyện (tức là các tham số mô hình giảm thiểu hàm chi phí trên tập huấn luyện).
- Sử dụng một phương pháp tối ưu hóa lặp lại được gọi là **gradient descent (GD)**, dần dần điều chỉnh các tham số mô hình để giảm thiểu hàm chi phí trên tập huấn luyện, cuối cùng hội tụ đến cùng một tập hợp các tham số như phương pháp đầu tiên. Chúng ta sẽ xem xét một vài biến thể của gradient descent mà chúng ta sẽ sử dụng lặp đi lặp lại khi nghiên cứu mạng nơ-ron trong Phần II: GD theo lô (batch GD), GD theo mini-batch và GD ngẫu nhiên (stochastic GD).

Tiếp theo, chúng ta sẽ xem xét hồi quy đa thức, một mô hình phức tạp hơn có thể phù hợp với các tập dữ liệu phi tuyến tính. Vì mô hình này có nhiều tham số hơn hồi quy tuyến tính, nó dễ bị quá khớp dữ liệu huấn luyện hơn. Chúng ta sẽ khám phá cách phát hiện liệu điều này có phải là trường hợp hay không bằng cách sử dụng các đường cong học tập, và sau đó chúng ta sẽ xem xét một số kỹ thuật chính quy hóa có thể giảm nguy cơ quá khớp tập huấn luyện.

Cuối cùng, chúng ta sẽ xem xét hai mô hình nữa thường được sử dụng cho các tác vụ phân loại: hồi quy logistic và hồi quy softmax.

4.1 Hồi quy tuyến tính

Trong Chương 1, chúng ta đã xem xét một mô hình hồi quy đơn giản về sự hài lòng cuộc sống: `life_satisfaction = θ₀ + θ₁ × GDP_per_capita` Mô hình này chỉ là một hàm tuyến tính của đặc trưng đầu vào `GDP_per_capita`. `θ0` và `θ1` là các tham số của mô hình.

Tổng quát hơn, một mô hình tuyến tính đưa ra dự đoán bằng cách đơn giản tính toán tổng có trọng số của các đặc trưng đầu vào, cộng với một hằng số được gọi là **hệ số chệch (bias term)** (còn gọi là **hệ số chặn (intercept term)**), như được hiển thị trong Phương trình 4-1.

Phương trình 4-1. Dự đoán mô hình hồi quy tuyến tính

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Trong phương trình này:

- $\hat{y}$ là giá trị được dự đoán.
- n là số lượng đặc trưng.
- x_i là giá trị đặc trưng thứ i .
- θ_j là tham số mô hình thứ j, bao gồm hệ số chệch θ_0 và trọng số đặc trưng $\theta_1, \theta_2, \cdots, \theta_n$.
- Điều này có thể được viết gọn hơn nhiều bằng cách sử dụng dạng vector hóa, như được hiển thị trong Phương trình 4-2.

Phương trình 4-2. Dự đoán mô hình hồi quy tuyến tính (dạng vector hóa)

$$y = h_\theta(x) = \theta \cdot x$$

Trong phương trình này:

- h_θ là hàm giả thuyết, sử dụng các tham số mô hình θ .
- θ là vector tham số của mô hình, bao gồm số hạng độ lệch θ_0 và các trọng số đặc trưng từ θ_1 đến θ_n .
- x là vector đặc trưng của một trường hợp (instance), bao gồm các giá trị từ x_0 đến x_n , với x_0 luôn bằng 1.
- $\theta \cdot x$ là tích vô hướng (dot product) của hai vector θ và x , bằng $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$.

OK, đó là mô hình hồi quy tuyến tính—nhưng làm thế nào để chúng ta huấn luyện nó? Chà, hãy nhớ rằng huấn luyện một mô hình có nghĩa là thiết lập các tham số của nó sao cho mô hình phù hợp nhất với tập huấn luyện. Với mục đích này, trước tiên chúng ta cần một thước đo mức độ phù hợp (hoặc không phù hợp) của mô hình với dữ liệu huấn luyện. Trong Chương 2, chúng ta đã thấy rằng thước đo hiệu suất phổ biến nhất của một mô hình hồi quy là lỗi trung bình bình phương gốc (RMSE) (Phương trình 2-1). Do đó, để huấn luyện một mô hình hồi quy tuyến tính, chúng ta cần tìm giá trị của $theta$ làm giảm thiểu RMSE. Trong thực

tế, việc giảm thiểu lỗi trung bình bình phương (MSE) đơn giản hơn RMSE và nó dẫn đến cùng một kết quả (vì giá trị giảm thiểu một hàm dương cũng giảm thiểu căn bậc hai của nó).

MSE của một giả thuyết hồi quy tuyến tính h_theta trên một tập huấn luyện X được tính bằng Phương trình 4-3.

Phương trình 4-3. Hàm chi phí MSE cho mô hình hồi quy tuyến tính $MSE(X, h_theta) = frac1msum_i = 1^m\left(theta^T x^{(i)} - y^{(i)}\right)^2$

Hầu hết các ký hiệu này đã được trình bày trong Chương 2 (xem "Ký hiệu"). Sự khác biệt duy nhất là chúng ta viết h_theta thay vì chỉ h để làm rõ rằng mô hình được tham số hóa bằng vector $theta$. Để đơn giản hóa ký hiệu, chúng ta sẽ chỉ viết $MSE(theta)$ thay vì $MSE(X, h_theta)$.

4.1.1 Phương trình chuẩn tắc

Để tìm giá trị của $theta$ làm giảm thiểu MSE, tồn tại một nghiệm dạng đóng — nói cách khác, một phương trình toán học đưa ra kết quả trực tiếp. Điều này được gọi là **phương trình chuẩn tắc (Normal equation)** (Phương trình 4-4).

Phương trình 4-4. Phương trình chuẩn tắc

$$MSE(X, h_\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(\theta^\intercal x^{(i)} - y^{(i)}\right)^2$$

Hầu hết các ký hiệu này đã được trình bày trong Chương 2 (xem "Các ký hiệu"). Điểm khác biệt duy nhất là chúng ta viết h_θ thay vì chỉ h để làm rõ rằng mô hình được tham số hóa bởi vector θ . Để đơn giản hóa ký hiệu, chúng ta sẽ chỉ viết $MSE(\theta)$ thay vì $MSE(X, h_\theta)$.

Để tìm giá trị của θ làm giảm thiểu MSE, có một nghiệm dạng đóng (closed-form solution)— nói cách khác, một phương trình toán học cho ra kết quả trực tiếp. Phương trình này được gọi là **Phương trình chuẩn tắc** (Công thức 4-4).

Công thức 4-4: Phương trình chuẩn tắc

$$\theta^= (X^\intercal X)^{-1} X^\intercal y$$

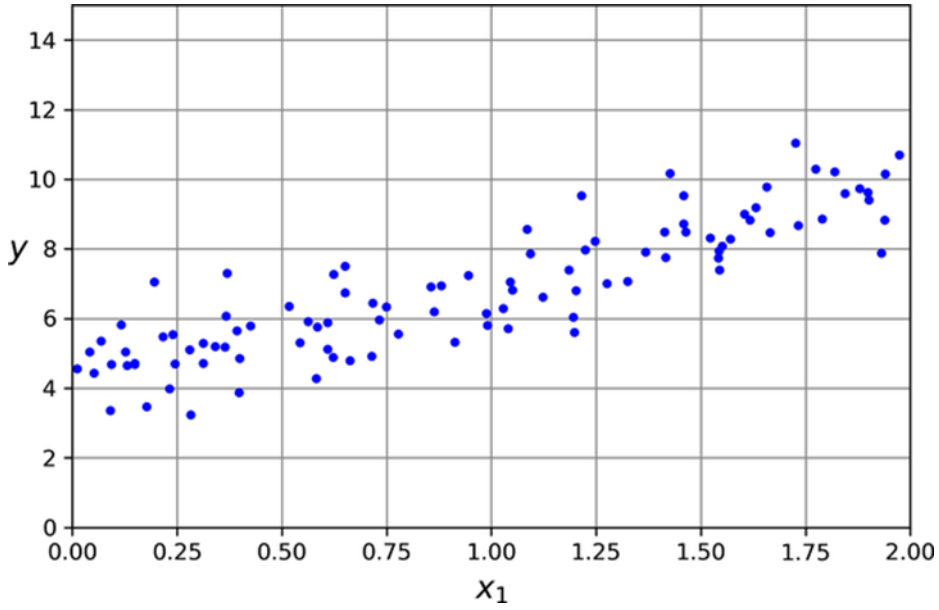
Trong phương trình này:

- $\hat{\theta}$ là giá trị của θ làm giảm thiểu hàm chi phí.
- y là vector các giá trị mục tiêu, chứa các giá trị từ $y^{(1)}$ đến $y^{(m)}$.

Hãy tạo một số dữ liệu có dạng tuyến tính để kiểm tra phương trình này (Hình 4-1):

```
import numpy as np

np.random.seed(42) # to make this code example reproducible
m = 100 # number of instances
```

```
X = 2 * np.random.rand(m, 1) # column vector
y = 4 + 3 * X + np.random.randn(m, 1) # column vector
```



Hình 4-1. Một tập dữ liệu tuyến tính được tạo ngẫu nhiên.

Bây giờ chúng ta hãy tính $hattheta$ bằng Phương trình chuẩn tắc. Chúng ta sẽ sử dụng hàm `inv()` từ mô-đun đại số tuyến tính của NumPy (`np.linalg`) để tính nghịch đảo của một ma trận, và phương thức `dot()` để nhân ma trận:

```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X) # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

Hàm mà chúng ta đã sử dụng để tạo dữ liệu là $y = 4 + 3x_1 + Gaussiannoise$. Hãy cùng xem phương trình đã tìm được gì:

```
>>> theta_best array([[4.21509616], [2.77011339]])
```

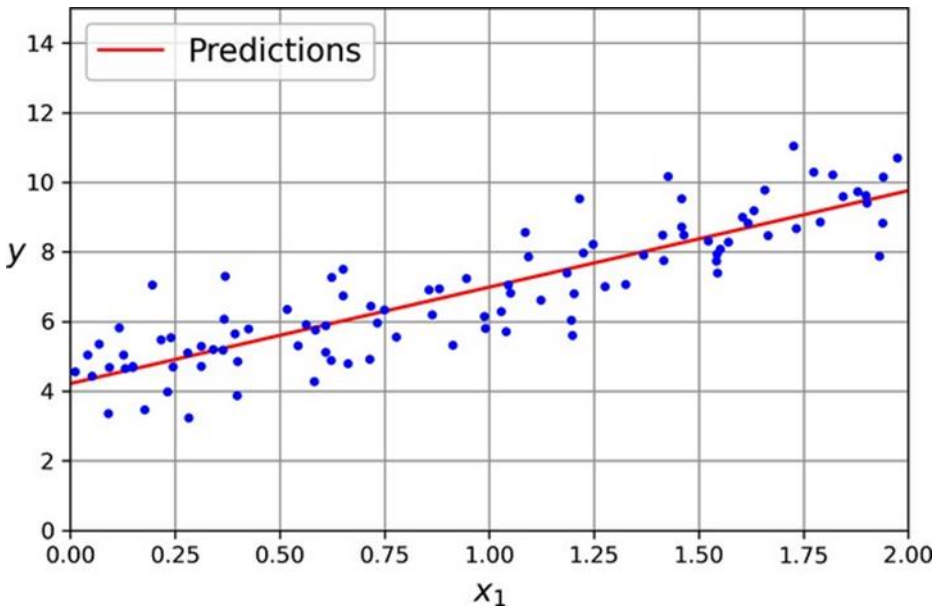
Chúng ta đã kỳ vọng $\theta_0 = 4$ và $\theta_1 = 3$, thay vì $\theta_0 = 4.215$ và $\theta_1 = 2.770$. Kết quả khá gần, nhưng nhiễu (noise) đã khiến chúng ta không thể khôi phục chính xác các tham số ban đầu của hàm gốc. Tập dữ liệu càng nhỏ và nhiều nhiễu thì việc này càng khó.

Bây giờ chúng ta có thể đưa ra các dự đoán bằng cách sử dụng $\hat{\theta}$:

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = add_dummy_feature(X_new) # add x0 = 1 to each instance
>>> y_predict = X_new_b @ theta_best
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```

Hãy vẽ biểu đồ các dự đoán của mô hình này (Hình 4-2):

```
import matplotlib.pyplot as plt

plt.plot(X_new, y_predict, "r-", label="Predictions")
plt.plot(X, y, "b.")
# [...]  # beautify the figure: add labels, axis, grid, and legend
plt.show()
```



Hình 4-2. Dự đoán mô hình hồi quy tuyến tính.

Thực hiện hồi quy tuyến tính bằng Scikit-Learn tương đối đơn giản:

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

Lưu ý rằng Scikit-Learn tách biệt hệ số chệch (`intercept_`) khỏi trọng số đặc trưng (`coef_`). Lớp `LinearRegression` dựa trên hàm `scipy.linalg.lstsq()` (tên viết tắt của "least squares"), mà bạn có thể gọi trực tiếp:

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

Hàm này tính toán $\theta^{=}X^{+}y$, trong đó X^{+} là **nghịch đảo giả** (pseudoinverse) của X (cụ thể là nghịch đảo Moore-Penrose). Bạn có thể sử dụng `np.linalg.pinv()` để tính toán nghịch đảo giả một cách trực tiếp.

```
>>> np.linalg.pinv(X_b) @ y
array([[4.21509616],
       [2.77011339]])
```

Nghịch đảo giả được tính bằng một kỹ thuật phân tách ma trận tiêu chuẩn gọi là **Phân tách giá trị suy biến (Singular Value Decomposition - SVD)**. SVD có thể phân tách ma trận tập huấn luyện X thành tích của ba ma trận: $U\Sigma V^{\intercal}$ (xem `numpy.linalg.svd()`).

Nghịch đảo giả được tính bằng công thức $X^{+} = V\Sigma^{+}U^{\intercal}$. Để tính ma trận Σ^{+} , thuật toán sẽ lấy ma trận Σ , đặt tất cả các giá trị nhỏ hơn một ngưỡng nhỏ bằng 0, sau đó thay thế tất cả các giá trị khác 0 bằng nghịch đảo của chúng, và cuối cùng chuyển vị ma trận kết quả. Phương pháp này hiệu quả hơn việc tính Phương trình chuẩn tắc, đồng thời xử lý tốt các trường hợp đặc biệt. Thật vậy, Phương trình chuẩn tắc có thể không hoạt động nếu ma trận $X^{\intercal}X$ không khả nghịch (chẳng hạn như khi $m < n$ hoặc một số đặc trưng bị thừa), nhưng nghịch đảo giả luôn được xác định.

4.1.2 Độ phức tạp tính toán

Phương trình chuẩn tắc tính nghịch đảo của $X^{\intercal}X$, một ma trận $(n+1) \times (n+1)$ (trong đó n là số đặc trưng). **Độ phức tạp tính toán** của việc nghịch đảo một ma trận như vậy thường là khoảng $O(n^{2.4})$ đến $O(n^{3})$, tùy thuộc vào cách triển khai. Nói cách khác, nếu bạn tăng gấp đôi số đặc trưng, thời gian tính toán sẽ tăng lên khoảng $2^{2.4} \approx 5.3$ đến $2^{3} = 8$ lần.

Cách tiếp cận SVD được sử dụng bởi `Scikit-Learn's LinearRegression` có độ phức tạp khoảng $O(n^{2})$. Nếu bạn tăng gấp đôi số đặc trưng, thời gian tính toán sẽ tăng lên khoảng 4 lần

Ngoài ra, một khi bạn đã huấn luyện mô hình hồi quy tuyến tính của mình (bằng cách sử dụng Phương trình chuẩn tắc hoặc bất kỳ thuật toán nào khác), các dự đoán rất nhanh: độ phức tạp tính toán là tuyến tính đối với cả số lượng trường hợp bạn muốn đưa ra dự đoán và số lượng đặc trưng. Nói cách khác, việc đưa ra dự đoán trên số lượng trường hợp gấp đôi (hoặc số lượng đặc trưng gấp đôi) sẽ mất khoảng thời gian gấp đôi.

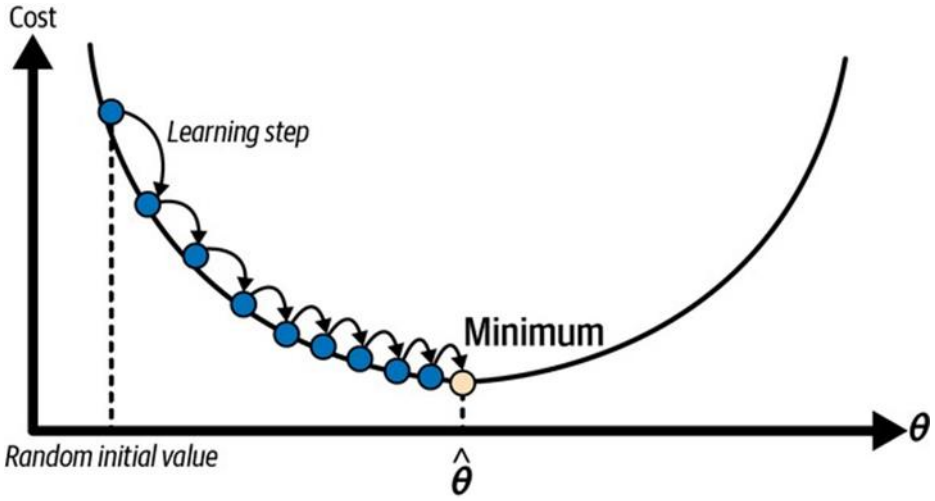
Bây giờ chúng ta sẽ xem xét một cách rất khác để huấn luyện mô hình hồi quy tuyến tính, phù hợp hơn cho các trường hợp có số lượng đặc trưng lớn hoặc quá nhiều trường hợp huấn luyện không thể nằm vừa trong bộ nhớ.

4.2 Gradient Descent

Gradient descent là một thuật toán tối ưu hóa chung có khả năng tìm ra các giải pháp tối ưu cho nhiều vấn đề khác nhau. Ý tưởng chung của gradient descent là điều chỉnh các tham số lặp đi lặp lại để giảm thiểu một hàm chi phí.

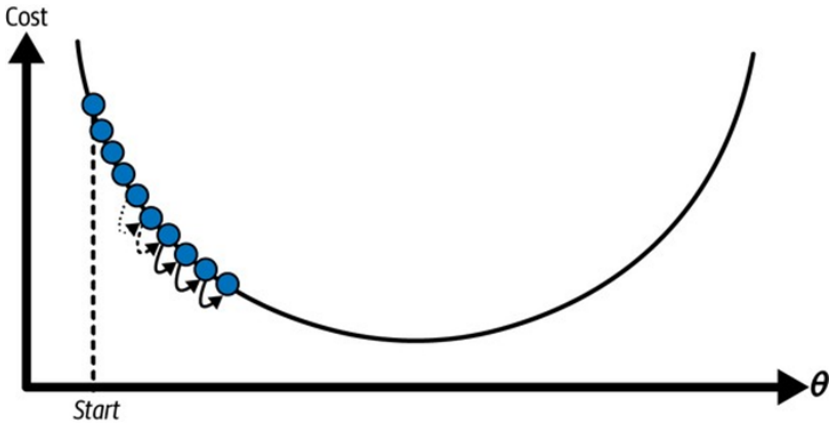
Giả sử bạn bị lạc trong núi trong một lớp sương mù dày đặc, và bạn chỉ có thể cảm nhận độ dốc của mặt đất dưới chân mình. Một chiến lược tốt để xuống đáy thung lũng nhanh chóng là đi xuống dốc theo hướng dốc nhất. Đây chính xác là những gì gradient descent làm: nó đo gradient cục bộ của hàm lỗi đối với vector tham số θ, và nó đi theo hướng gradient giảm dần. Khi gradient bằng 0, bạn đã đạt đến một cực tiểu!

Trong thực tế, bạn bắt đầu bằng cách điền các giá trị ngẫu nhiên vào θ(điều này được gọi là khởi tạo ngẫu nhiên). Sau đó, bạn cải thiện nó dần dần, từng bước nhỏ, mỗi bước cố gắng giảm hàm chi phí (ví dụ: MSE), cho đến khi thuật toán hội tụ đến một cực tiểu (xem Hình 4-3).



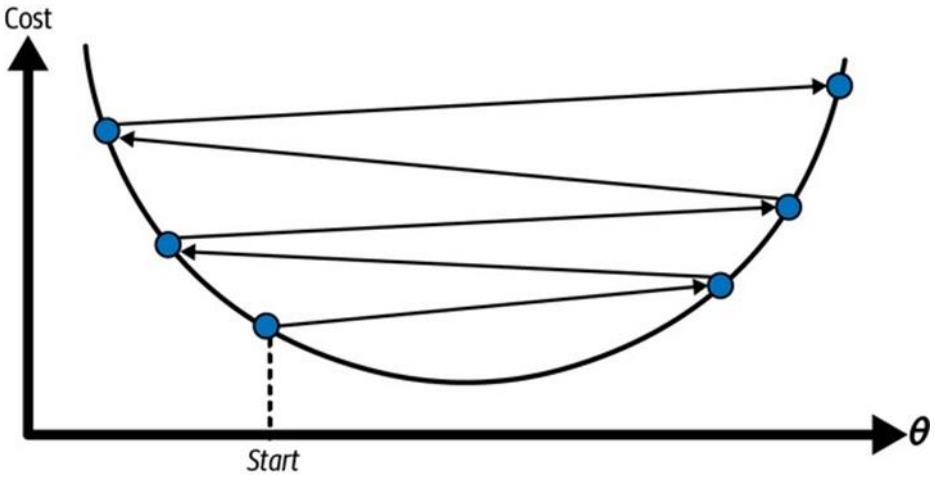
Hình 4-3. Trong mô tả gradient descent này, các tham số mô hình được khởi tạo ngẫu nhiên và được điều chỉnh lặp đi lặp lại để giảm thiểu hàm chi phí; kích thước bước học tỷ lệ thuận với độ dốc của hàm chi phí, vì vậy các bước dần dần nhỏ hơn khi chi phí tiến gần đến cực tiểu.

Một tham số quan trọng trong gradient descent là kích thước của các bước, được xác định bởi siêu tham số tốc độ học (learning rate). Nếu tốc độ học quá nhỏ, thì thuật toán sẽ phải trải qua nhiều lần lặp để hội tụ, điều này sẽ mất nhiều thời gian (xem Hình 4-4).



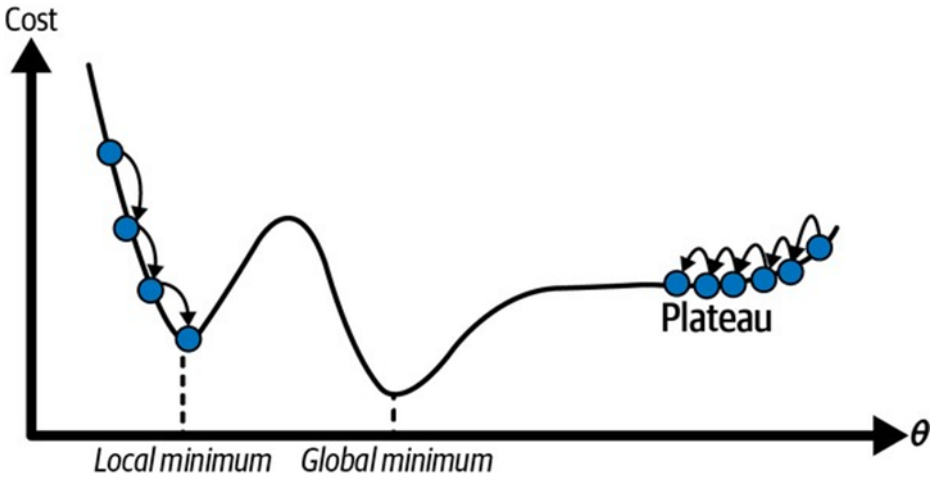
Hình 4-4. Tốc độ học quá nhỏ.

Mặt khác, nếu tốc độ học quá cao, bạn có thể nhảy qua thung lũng và kết thúc ở phía bên kia, thậm chí có thể cao hơn bạn trước đó. Điều này có thể khiến thuật toán phân kỳ, với các giá trị ngày càng lớn hơn, không tìm được giải pháp tốt (xem Hình 4-5).



Hình 4-5. Tốc độ học quá cao.

Ngoài ra, không phải tất cả các hàm chi phí đều trông giống như những cái bát đẹp, đều đặn. Có thể có các lỗ, rãnh, cao nguyên và đủ loại địa hình không đều, khiến việc hội tụ đến cực tiểu trở nên khó khăn. Hình 4-6 cho thấy hai thách thức chính với gradient descent. Nếu khởi tạo ngẫu nhiên bắt đầu thuật toán ở bên trái, thì nó sẽ hội tụ đến một cực tiểu cục bộ, không tốt bằng cực tiểu toàn cục. Nếu nó bắt đầu ở bên phải, thì nó sẽ mất rất nhiều thời gian để vượt qua cao nguyên. Và nếu bạn dừng quá sớm, bạn sẽ không bao giờ đạt đến cực tiểu toàn cục.

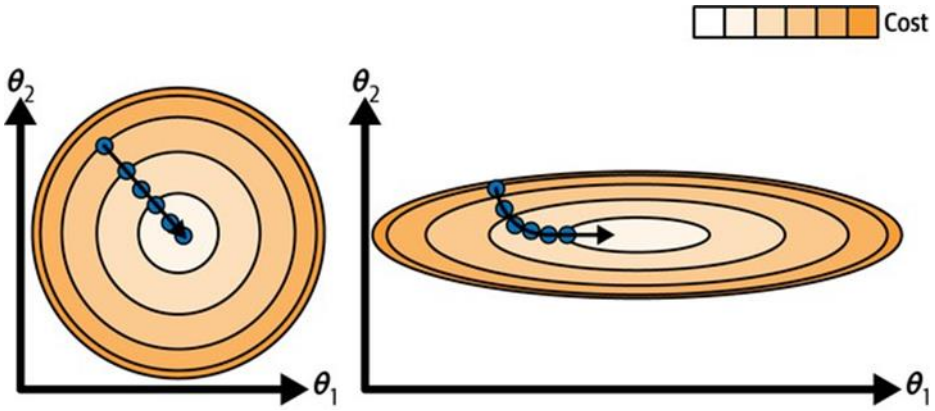


Hình 4-6. Những cạm bẫy của gradient descent.

May mắn thay, hàm chi phí MSE cho mô hình hồi quy tuyến tính tình cờ là một hàm lồi (convex function), điều đó có nghĩa là nếu bạn chọn bất kỳ hai điểm nào trên đường cong, đoạn thẳng nối chúng sẽ không bao giờ nằm dưới đường cong. Điều này ngụ ý rằng không có cực tiểu cục bộ nào, chỉ có một cực tiểu toàn cục. Nó cũng là một hàm liên tục với độ

dốc không bao giờ thay đổi đột ngột. Hai sự thật này có một hệ quả lớn: gradient descent được đảm bảo sẽ tiến gần một cách tùy ý đến cực tiểu toàn cục (nếu bạn đợi đủ lâu và nếu tốc độ học không quá cao).

Trong khi hàm chi phí có hình dạng một cái bát, nó có thể là một cái bát kéo dài nếu các đặc trưng có các thang đo rất khác nhau. Hình 4-7 cho thấy gradient descent trên một tập huấn luyện nơi đặc trưng 1 và 2 có cùng thang đo (bên trái), và trên một tập huấn luyện nơi đặc trưng 1 có giá trị nhỏ hơn nhiều so với đặc trưng 2 (bên phải).



Hình 4-7. Gradient descent có (trái) và không có (phải) điều chỉnh đặc trưng.

Như bạn có thể thấy, ở bên trái thuật toán gradient descent đi thẳng về phía cực tiểu, do đó đạt được nó nhanh chóng, trong khi ở bên phải, nó ban đầu đi theo hướng gần như trực giao với hướng của cực tiểu toàn cục, và kết thúc bằng một hành trình dài xuống một thung lũng gần như phẳng. Cuối cùng nó sẽ đạt đến cực tiểu, nhưng sẽ mất nhiều thời gian.

Biểu đồ này cũng minh họa thực tế là việc huấn luyện một mô hình có nghĩa là tìm kiếm một sự kết hợp các tham số mô hình làm giảm thiểu một hàm chi phí (trên tập huấn luyện). Đây là một tìm kiếm trong không gian tham số của mô hình. Mô hình càng có nhiều tham số, không gian này càng có nhiều chiều, và việc tìm kiếm càng khó khăn hơn: tìm kim trong đống rơm 300 chiều khó hơn nhiều so với trong 3 chiều. May mắn thay, vì hàm chi phí là lồi trong trường hợp hồi quy tuyến tính, kim chỉ đơn giản nằm ở đáy cái bát.

4.2.1 Gradient Descent theo lô

Để triển khai gradient descent, bạn cần tính toán gradient của hàm chi phí đối với từng tham số mô hình θ_j . Nói cách khác, bạn cần tính toán hàm chi phí sẽ thay đổi bao nhiêu nếu bạn thay đổi θ_j một chút. Đây được gọi là đạo hàm riêng (partial derivative). Nó giống như hỏi, “Độ dốc của ngọn núi dưới chân tôi là bao nhiêu nếu tôi đối mặt về phía đông”? và sau đó hỏi cùng câu hỏi khi đối mặt về phía bắc (và cứ thế cho tất cả các chiều khác, nếu bạn có thể tưởng tượng một vũ trụ có nhiều hơn ba chiều). Phương trình 4-5 tính toán đạo hàm riêng của MSE đối với tham số θ_j , được ký hiệu là $\partial MSE(\theta)/\partial\theta_j$.

Phương trình 4-5. Đạo hàm riêng của hàm chi phí

$$fracpartialMSE(theta)partialtheta_j = frac2msum_i = 1^m\left(theta^T x^{(i)} - y^{(i)}\right)x_j^{(i)}$$

Thay vì tính toán các đạo hàm riêng này một cách riêng lẻ, bạn có thể sử dụng Phương trình 4-6 để tính toán tất cả chúng cùng một lúc. Vector gradient, được ký hiệu là $nabla_thetaMSE(theta)$, chứa tất cả các đạo hàm riêng của hàm chi phí (một cho mỗi tham số mô hình).

Phương trình 4-6. Vector gradient của hàm chi phí

Công thức 4-5: Đạo hàm riêng của hàm chi phí

$$\frac{\partial}{\partial\theta_j}MSE(\theta) = \frac{2}{m}\sum_{i=1}^{m}\left(\theta^\intercal x^{(i)} - y^{(i)}\right)x_{j(i)}$$

Thay vì tính các đạo hàm riêng này một cách riêng lẻ, bạn có thể sử dụng **Công thức 4-6** để tính tất cả cùng một lúc. Vector gradient, được ký hiệu là $\nabla_\theta MSE(\theta)$, chứa tất cả các đạo hàm riêng của hàm chi phí (mỗi đạo hàm tương ứng với một tham số mô hình).

Phương trình 4-7. Bước gradient descent

$$\nabla_{\boldsymbol{\theta}}\,\mathrm{MSE}\left(\boldsymbol{\theta}\right) = \begin{pmatrix} \frac{\partial}{\partial\theta_0}\mathrm{MSE}\left(\boldsymbol{\theta}\right) \\ \frac{\partial}{\partial\theta_1}\mathrm{MSE}\left(\boldsymbol{\theta}\right) \\ \vdots \\ \frac{\partial}{\partial\theta_n}\mathrm{MSE}\left(\boldsymbol{\theta}\right) \end{pmatrix} = \frac{2}{m}\mathbf{X}^\intercal\left(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\right)$$

Một khi bạn có vector gradient, nó sẽ chỉ theo hướng dốc lên. Để đi xuống, bạn chỉ cần đi theo hướng ngược lại. Điều này có nghĩa là bạn sẽ trừ $\nabla_\theta MSE(\theta)$ khỏi θ . Đây là lúc tốc độ học η (eta) phát huy tác dụng: nhân vector gradient với η để xác định kích thước của bước đi xuống (Công thức 4-7).

Công thức 4-7: Bước hạ Gradient

$$\theta^{(bướctieptheo)} = \theta - \eta\nabla_\theta MSE(\theta)$$

Hãy xem một cách triển khai nhanh chóng của thuật toán này:

```
eta = 0.1 # learning rate
n_epochs = 1000
m = len(X_b) # number of instances

np.random.seed(42)
theta = np.random.randn(2, 1) # randomly initialized model parameters

for epoch in range(n_epochs):
```

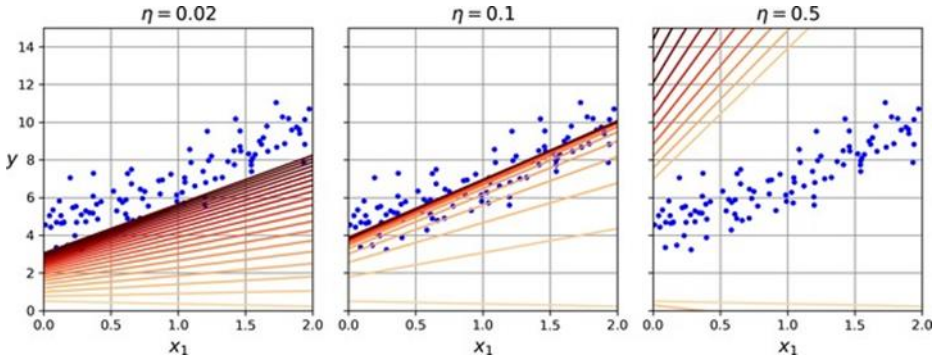
```
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
```

Không quá khó! Mỗi lần lặp trên tập huấn luyện được gọi là một epoch. Hãy xem kết quả `theta`:

```
>>> theta
array([[4.21509616],
       [2.77011339]])
```

Này, đó chính xác là những gì Phương trình chuẩn tắc tìm thấy! Gradient descent hoạt động hoàn hảo. Nhưng điều gì sẽ xảy ra nếu bạn sử dụng một tốc độ học khác (

`eta`)? Hình 4-8 cho thấy 20 bước đầu tiên của gradient descent sử dụng ba tốc độ học khác nhau. Đường ở dưới cùng của mỗi biểu đồ đại diện cho điểm bắt đầu ngẫu nhiên, sau đó mỗi epoch được biểu diễn bằng một đường ngày càng tối hơn.



Hình 4-8. Gradient descent với các tốc độ học khác nhau.

Ở bên trái, tốc độ học quá thấp: thuật toán cuối cùng sẽ đạt được giải pháp, nhưng sẽ mất nhiều thời gian. Ở giữa, tốc độ học trông khá tốt: chỉ trong vài epoch, nó đã hội tụ đến giải pháp. Ở bên phải, tốc độ học quá cao: thuật toán phân kỳ, nhảy loạn xạ và thực sự ngày càng rời xa giải pháp ở mỗi bước.

Để tìm một tốc độ học tốt, bạn có thể sử dụng tìm kiếm lưới (xem Chương 2). Tuy nhiên, bạn có thể muốn giới hạn số epoch để tìm kiếm lưới có thể loại bỏ các mô hình mất quá nhiều thời gian để hội tụ.

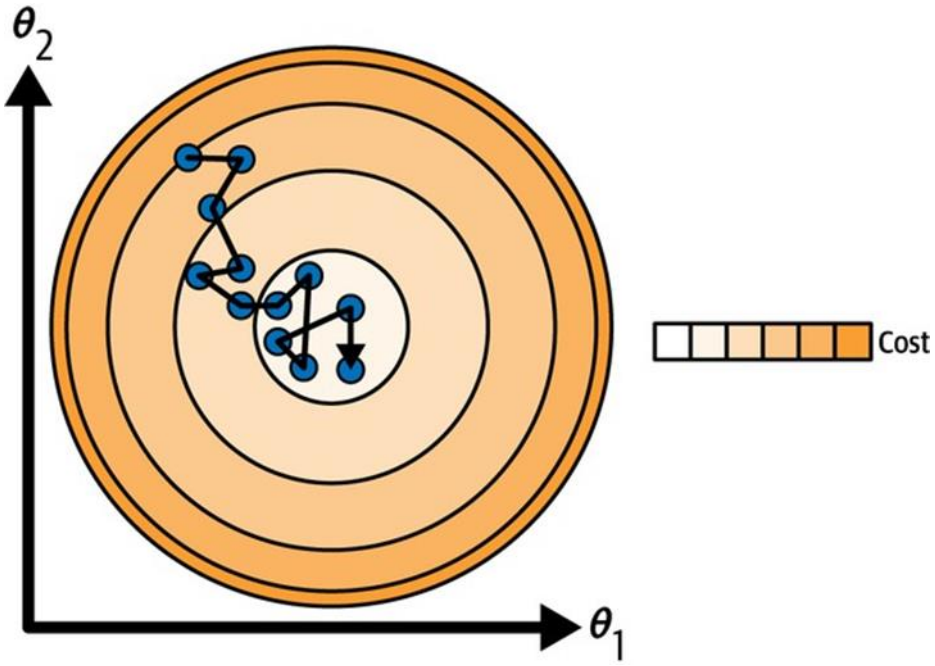
Bạn có thể tự hỏi làm thế nào để đặt số epoch. Nếu nó quá thấp, bạn sẽ vẫn còn cách xa giải pháp tối ưu khi thuật toán dừng lại; nhưng nếu nó quá cao, bạn sẽ lãng phí thời gian trong khi các tham số mô hình không còn thay đổi nữa. Một giải pháp đơn giản là đặt một số lượng epoch rất lớn nhưng ngắt thuật toán khi vector gradient trở nên rất nhỏ — tức là khi chuẩn của nó trở nên nhỏ hơn một số rất nhỏ $epsilon$ (được gọi là độ dung sai) — bởi vì điều này xảy ra khi gradient descent đã (gần như) đạt đến cực tiểu.

4.2.2 Gradient Descent ngẫu nhiên

Vấn đề chính với gradient descent theo lô là nó sử dụng toàn bộ tập huấn luyện để tính toán gradient ở mỗi bước, điều này khiến nó rất chậm khi tập huấn luyện lớn. Ở thái cực đối lập,

gradient descent ngẫu nhiên (stochastic gradient descent) chọn một trường hợp ngẫu nhiên trong tập huấn luyện ở mỗi bước và tính toán gradient chỉ dựa trên trường hợp đó. Rõ ràng, việc làm việc trên một trường hợp tại một thời điểm làm cho thuật toán nhanh hơn nhiều vì nó có rất ít dữ liệu để thao tác ở mỗi lần lặp. Nó cũng giúp có thể huấn luyện trên các tập huấn luyện khổng lồ, vì chỉ cần một trường hợp nằm trong bộ nhớ ở mỗi lần lặp (GD ngẫu nhiên có thể được triển khai như một thuật toán out-of-core; xem Chương 1).

Mặt khác, do tính chất ngẫu nhiên của nó, thuật toán này ít đều đặn hơn nhiều so với gradient descent theo lô: thay vì giảm dần nhẹ nhàng cho đến khi đạt đến cực tiểu, hàm chi phí sẽ dao động lên xuống, chỉ giảm trung bình. Theo thời gian, nó sẽ tiến rất gần đến cực tiểu, nhưng một khi đã đến đó, nó sẽ tiếp tục dao động xung quanh, không bao giờ ổn định (xem Hình 4-9). Khi thuật toán dừng lại, các giá trị tham số cuối cùng sẽ tốt, nhưng không tối ưu.



Hình 4-9. Với stochastic gradient descent, mỗi bước huấn luyện nhanh hơn nhiều nhưng cũng ngẫu nhiên hơn nhiều so với khi sử dụng batch gradient descent.

Khi hàm chi phí rất không đều (như trong Hình 4-6), điều này thực sự có thể giúp thuật toán thoát khỏi các cực tiểu cục bộ, vì vậy gradient descent ngẫu nhiên có cơ hội tìm thấy cực tiểu toàn cục tốt hơn so với gradient descent theo lô.

Do đó, tính ngẫu nhiên là tốt để thoát khỏi các cực tiểu cục bộ, nhưng xấu vì nó có nghĩa là thuật toán không bao giờ có thể ổn định ở cực tiểu. Một giải pháp cho tình huống khó xử này là giảm dần tốc độ học. Các bước bắt đầu lớn (giúp tiến bộ nhanh chóng và thoát khỏi các cực tiểu cục bộ), sau đó ngày càng nhỏ hơn, cho phép thuật toán ổn định ở cực tiểu toàn cục. Quá trình này giống như mô phỏng ủ kim loại, một thuật toán lấy cảm hứng từ quá trình ủ kim loại, nơi kim loại nóng chảy được làm nguội từ từ. Hàm xác định tốc độ học ở mỗi lần lặp được gọi là lịch học. Nếu tốc độ học giảm quá nhanh, bạn có thể bị kẹt trong

một cực tiểu cục bộ, hoặc thậm chí bị đóng băng giữa chừng đến cực tiểu. Nếu tốc độ học giảm quá chậm, bạn có thể nhảy xung quanh cực tiểu trong một thời gian dài và kết thúc với một giải pháp không tối ưu nếu bạn dừng huấn luyện quá sớm.

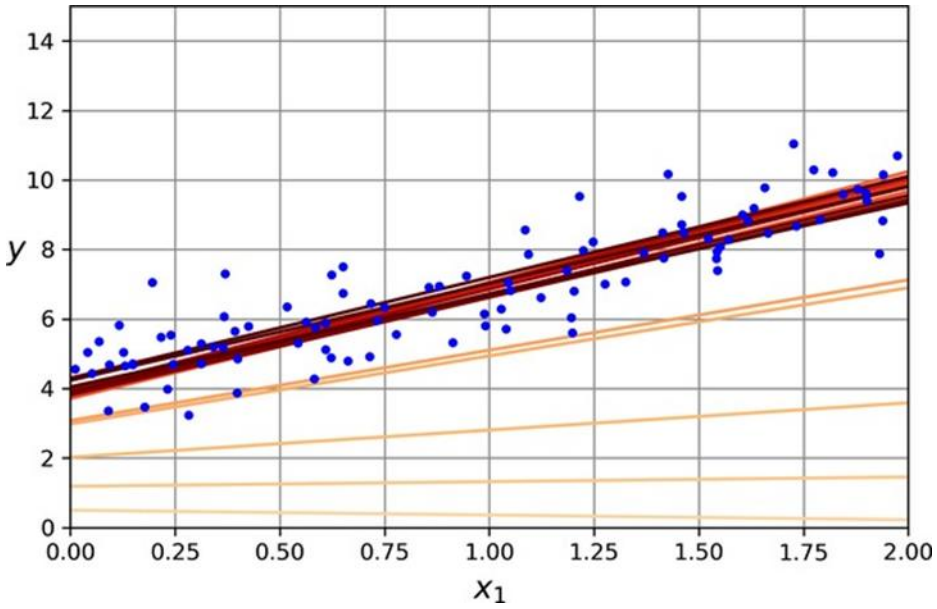
Đoạn mã này triển khai gradient descent ngẫu nhiên bằng cách sử dụng một lịch học đơn giản:

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index : random_index + 1]
        yi = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # for SGD, do not divide by
m
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
```

Theo quy ước, chúng ta lặp theo các vòng lặp m lần lặp; mỗi vòng lặp được gọi là một epoch, như trước. Trong khi mã gradient descent theo lô lặp 1.000 lần qua toàn bộ tập huấn luyện, mã này chỉ đi qua tập huấn luyện 50 lần và đạt được một giải pháp khá tốt:

```
>>> theta
array([[4.21076011],
       [2.74856079]])
```

Hình 4-10 cho thấy 20 bước huấn luyện đầu tiên (lưu ý các bước không đều đặn như thế nào). Lưu ý rằng vì các trường hợp được chọn ngẫu nhiên, một số trường hợp có thể được chọn nhiều lần trong một epoch, trong khi những trường hợp khác có thể không được chọn. Nếu bạn muốn đảm bảo rằng thuật toán đi qua mọi trường hợp trong mỗi epoch, một cách tiếp cận khác là xáo trộn tập huấn luyện (đảm bảo xáo trộn đồng thời các đặc trưng đầu vào và nhãn), sau đó đi qua từng trường hợp, sau đó xáo trộn lại, v.v. Tuy nhiên, cách tiếp cận này phức tạp hơn và nói chung không cải thiện kết quả.



Hình 4-10. 20 bước đầu tiên của stochastic gradient descent.

Để thực hiện hồi quy tuyến tính bằng SGD với Scikit-Learn, bạn có thể sử dụng lớp `SGDRegressor`, mặc định sẽ tối ưu hóa hàm chi phí MSE. Đoạn mã sau chạy tối đa 1.000 epoch (`max_iter`) hoặc cho đến khi lỗi giảm xuống dưới 10^-5 (`tol`) trong 100 epoch (`n_iter_no_change`). Nó bắt đầu với tốc độ học là 0.01 (`eta0`), sử dụng lịch học mặc định (khác với lịch chúng ta đã sử dụng). Cuối cùng, nó không sử dụng bất kỳ chính quy hóa nào (`penalty=None`; chi tiết hơn về điều này sẽ sớm được đề cập):

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel()) # y.ravel() because fit() expects 1D targets
```

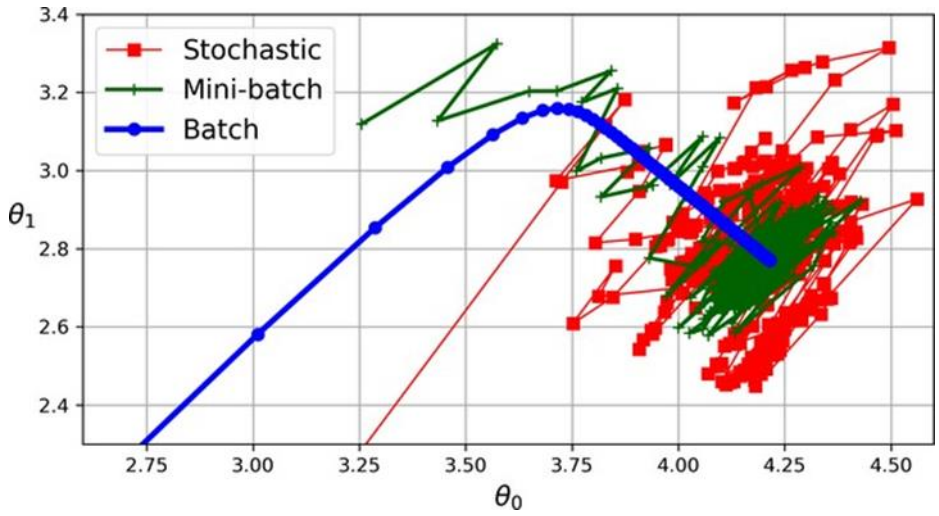
Một lần nữa, bạn tìm thấy một giải pháp khá gần với giải pháp được trả về bởi phương trình chuẩn tắc:

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.21278812]), array([2.77270267]))
```

4.2.3 Gradient Descent theo Mini-Batch

Thuật toán gradient descent cuối cùng mà chúng ta sẽ xem xét được gọi là gradient descent theo mini-batch. Nó khá đơn giản một khi bạn đã hiểu gradient descent theo lô và ngẫu nhiên: ở mỗi bước, thay vì tính toán gradient dựa trên toàn bộ tập huấn luyện (như trong GD theo lô) hoặc chỉ dựa trên một trường hợp (như trong GD ngẫu nhiên), GD theo mini-batch tính toán gradient trên các tập hợp nhỏ ngẫu nhiên các trường hợp được gọi là mini-batch. Ưu điểm chính của GD theo mini-batch so với GD ngẫu nhiên là bạn có thể tăng hiệu suất từ việc tối ưu hóa phần cứng của các phép toán ma trận, đặc biệt khi sử dụng GPU.

Sự tiến bộ của thuật toán trong không gian tham số ít thất thường hơn so với GD ngẫu nhiên, đặc biệt với các mini-batch khá lớn. Kết quả là, GD theo mini-batch sẽ đi lại gần cực tiểu hơn một chút so với GD ngẫu nhiên—nhưng nó có thể khó thoát khỏi các cực tiểu cục bộ hơn (trong trường hợp các vấn đề gặp phải cực tiểu cục bộ, không giống như hồi quy tuyến tính với hàm chi phí MSE). Hình 4-11 cho thấy các đường đi của ba thuật toán gradient descent trong không gian tham số trong quá trình huấn luyện. Chúng đều kết thúc gần cực tiểu, nhưng đường đi của GD theo lô thực sự dừng ở cực tiểu, trong khi cả GD ngẫu nhiên và GD theo mini-batch tiếp tục đi lại xung quanh. Tuy nhiên, đừng quên rằng GD theo lô mất rất nhiều thời gian để thực hiện mỗi bước, và GD ngẫu nhiên và GD theo mini-batch cũng sẽ đạt đến cực tiểu nếu bạn sử dụng một lịch học tốt.



Hình 4-11. Đường đi của Gradient Descent trong không gian tham số.

Bảng 4-1 so sánh các thuật toán chúng ta đã thảo luận cho đến nay cho hồi quy tuyến tính (hãy nhớ rằng m là số lượng trường hợp huấn luyện và n là số lượng đặc trưng).

Bảng 4-1. So sánh các thuật toán cho hồi quy tuyến tính

Thuật toán	Lớn m	Hỗ trợ ngoài lõi	Lớn n	Siêu tham số	Yêu cầu điều chỉnh tỉ lệ	SciPy
Phương trình chuẩn tắc	Nhanh	Không	Chậm	0	Không	N/A
SVD	Nhanh	Không	Chậm	0	Không	Lin
GD theo lô	Chậm	Không	Nhanh	2	Có	N/A
GD ngẫu nhiên	Nhanh	Có	Nhanh	$ge2$	Có	SGD
GD theo mini-batch	Nhanh	Có	Nhanh	$ge2$	Có	N/A

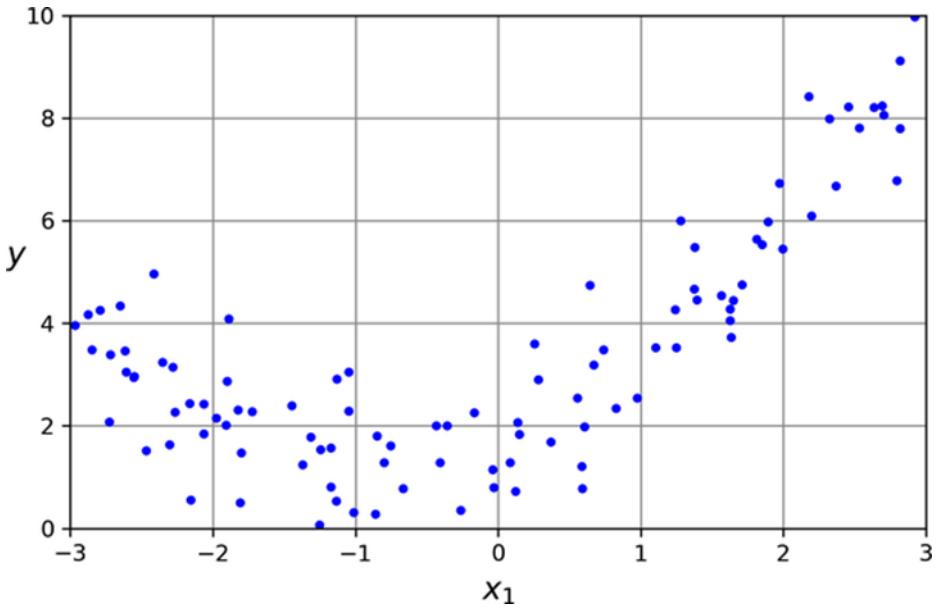
Sau khi huấn luyện, hầu như không có sự khác biệt: tất cả các thuật toán này đều tạo ra các mô hình rất giống nhau và đưa ra dự đoán theo cùng một cách.

4.3 Hồi quy đa thức

Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng? Thật đáng ngạc nhiên, bạn có thể sử dụng một mô hình tuyến tính để phù hợp với dữ liệu phi tuyến tính. Một cách đơn giản để làm điều này là thêm các lũy thừa của mỗi đặc trưng làm đặc trưng mới, sau đó huấn luyện một mô hình tuyến tính trên tập đặc trưng mở rộng này. Kỹ thuật này được gọi là **hồi quy đa thức**.

Hãy xem một ví dụ. Đầu tiên, chúng ta sẽ tạo một số dữ liệu phi tuyến tính (xem Hình 4-12), dựa trên một phương trình bậc hai đơn giản—đó là một phương trình có dạng $y = ax^2 + bx + c$ —cộng thêm một số nhiễu:

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
```



Hình 4-12. Tập dữ liệu phi tuyến tính và nhiễu được tạo.

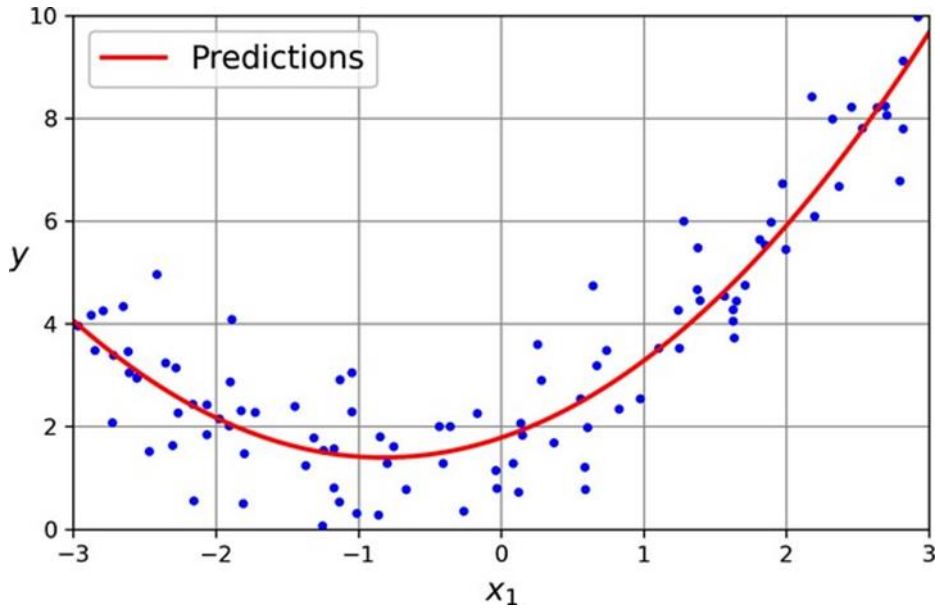
Rõ ràng, một đường thẳng sẽ không bao giờ phù hợp với dữ liệu này một cách chính xác. Vì vậy, hãy sử dụng lớp `PolynomialFeatures` của Scikit-Learn để biến đổi dữ liệu huấn luyện của chúng ta, thêm bình phương (đa thức bậc hai) của mỗi đặc trưng trong tập huấn luyện làm đặc trưng mới (trong trường hợp này chỉ có một đặc trưng):

```
from sklearn.preprocessing import PolynomialFeatures

poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
```

```
>>> X[0]
array([-0.75275929])

>>> X_poly[0]
array([-0.75275929,  0.56664654])
```

`X_poly` bây giờ chứa đặc trưng gốc của `X` cộng với bình phương của đặc trưng này. Bây giờ chúng ta có thể điều chỉnh mô hình `LinearRegression` cho dữ liệu huấn luyện mở rộng này (Hình 4-13):

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```



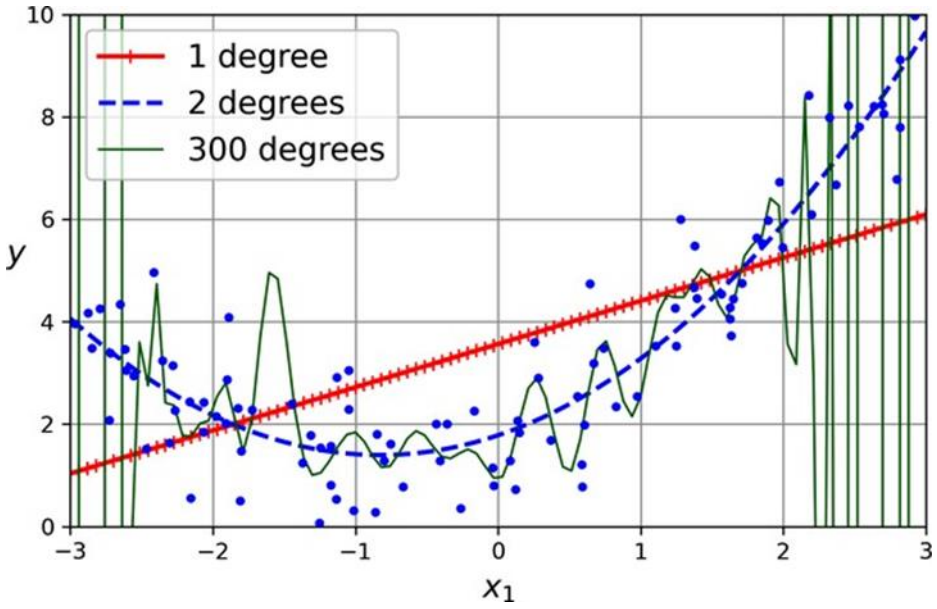
Hình 4-13. Dự đoán mô hình hồi quy đa thức.

Không tệ: mô hình ước tính $haty = 0.56x_1^2 + 0.93x_1 + 1.78$ trong khi hàm gốc thực tế là $y = 0.5x_1^2 + 1.0x_1 + 2.0 +$ nhiễu $Gaussian$. Lưu ý rằng khi có nhiều đặc trưng, hồi quy đa thức có khả năng tìm ra mối quan hệ giữa các đặc trưng, điều mà mô hình hồi quy tuyến tính thông thường không thể làm được. Điều này có thể thực hiện được nhờ việc `PolynomialFeatures` cũng thêm tất cả các sự kết hợp của các đặc trưng lên đến bậc đã cho. Ví dụ, nếu có hai đặc trưng a và b , `PolynomialFeatures` với `degree=3` sẽ không chỉ thêm các đặc trưng $a^2, a^3, b^2,$ và b^3 , mà còn cả các sự kết hợp $ab, a^2b,$ và ab^2 .

4.4 Đường cong học tập

Nếu bạn thực hiện hồi quy đa thức bậc cao, bạn có thể sẽ khớp dữ liệu huấn luyện tốt hơn nhiều so với hồi quy tuyến tính thông thường. Ví dụ, Hình 4-14 áp dụng mô hình đa thức bậc

300 cho dữ liệu huấn luyện trước đó, và so sánh kết quả với một mô hình tuyến tính thuần túy và một mô hình bậc hai (đa thức bậc hai). Lưu ý cách mô hình đa thức bậc 300 uốn lượn để gần nhất có thể với các trường hợp huấn luyện.



Hình 4-14. Hồi quy đa thức bậc cao.

Mô hình hồi quy đa thức bậc cao này đang quá khớp dữ liệu huấn luyện một cách nghiêm trọng, trong khi mô hình tuyến tính đang dưới khớp. Mô hình sẽ tổng quát hóa tốt nhất trong trường hợp này là mô hình bậc hai, điều này có lý vì dữ liệu được tạo ra bằng một mô hình bậc hai. Nhưng nói chung bạn sẽ không biết hàm nào đã tạo ra dữ liệu, vậy làm thế nào bạn có thể quyết định mô hình của mình nên phức tạp đến mức nào? Làm thế nào bạn có thể biết mô hình của mình đang quá khớp hay dưới khớp dữ liệu?

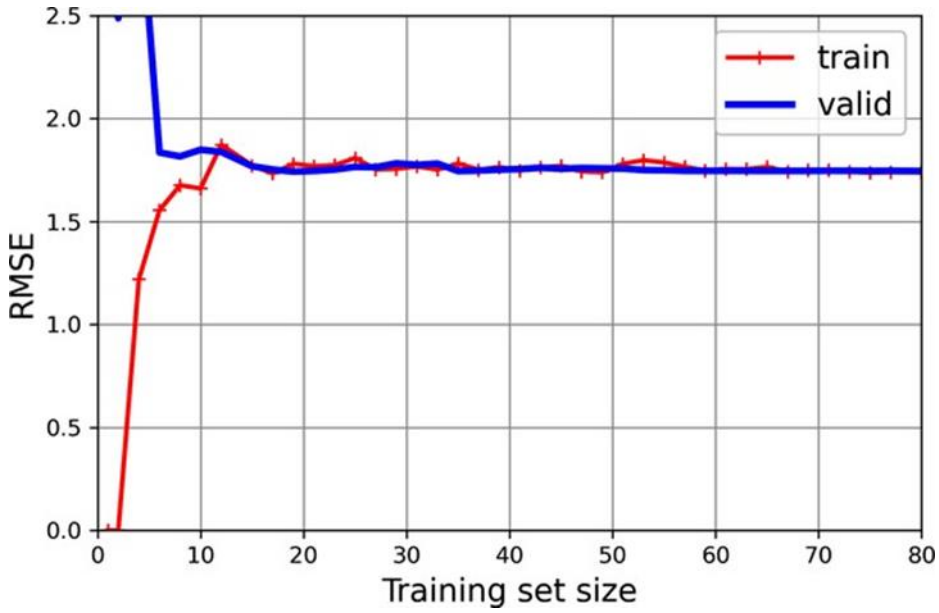
Trong Chương 2, bạn đã sử dụng kiểm định chéo để có được ước tính hiệu suất tổng quát hóa của một mô hình. Nếu một mô hình hoạt động tốt trên dữ liệu huấn luyện nhưng tổng quát hóa kém theo các chỉ số kiểm định chéo, thì mô hình của bạn đang quá khớp. Nếu nó hoạt động kém trên cả hai, thì nó đang dưới khớp. Đây là một cách để biết khi nào một mô hình quá đơn giản hoặc quá phức tạp.

Một cách khác để nhận biết là nhìn vào các đường cong học tập, là các biểu đồ thể hiện lỗi huấn luyện và lỗi xác thực của mô hình dưới dạng hàm của lần lặp huấn luyện: chỉ cần đánh giá mô hình theo các khoảng đều đặn trong quá trình huấn luyện trên cả tập huấn luyện và tập xác thực, và vẽ biểu đồ kết quả. Nếu mô hình không thể được huấn luyện tăng dần (tức là nếu nó không hỗ trợ `partial_fit()` hoặc `warm_start`), thì bạn phải huấn luyện nó nhiều lần trên các tập con tăng dần của tập huấn luyện.

Scikit-Learn có một hàm `learning_curve()` hữu ích để giúp với điều này: nó huấn luyện và đánh giá mô hình bằng cách sử dụng kiểm định chéo. Theo mặc định, nó huấn luyện lại mô hình trên các tập con đang tăng dần của tập huấn luyện, nhưng nếu mô hình hỗ trợ học tăng dần, bạn có thể đặt `exploit_incremental_learning=True` khi gọi `learning_curve()`

và nó sẽ huấn luyện mô hình tăng dần thay vào đó. Hàm này trả về kích thước tập huấn luyện mà nó đã đánh giá mô hình, và các điểm huấn luyện và xác thực mà nó đã đo cho mỗi kích thước và cho mỗi fold kiểm định chéo. Hãy sử dụng hàm này để xem các đường cong học tập của mô hình hồi quy tuyến tính thông thường (xem Hình 4-15):

```
from sklearn.model_selection import learning_curve
from sklearn.linear_model import LinearRegression # Import LinearRegression
import matplotlib.pyplot as plt # Import matplotlib.pyplot
import numpy as np # Import numpy

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")
# [...] # beautify the figure: add labels, axis, grid, and legend
plt.show()
```



Hình 4-15. Đường cong học tập.

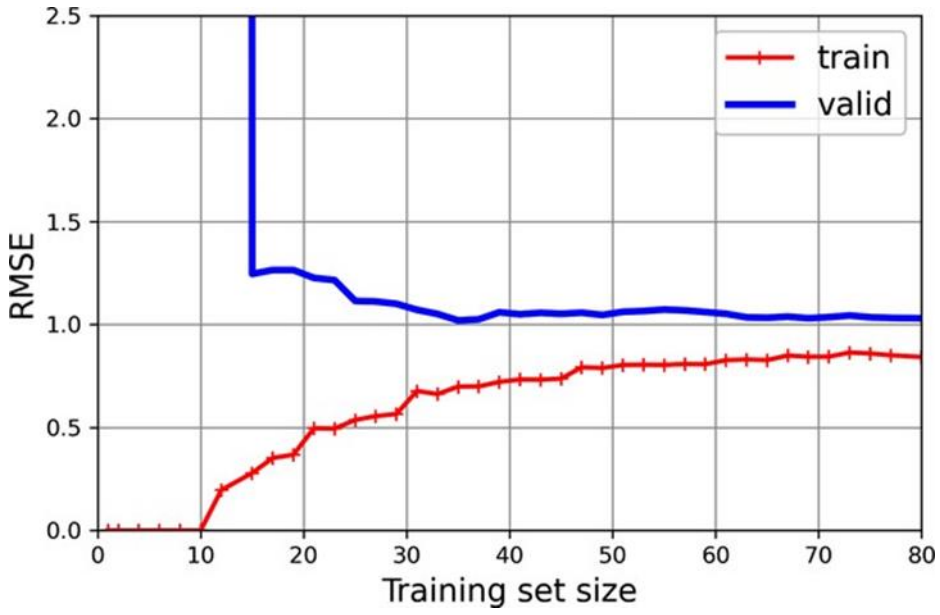
Mô hình này đang bị dưới khớp. Để hiểu tại sao, trước tiên hãy nhìn vào lỗi huấn luyện. Khi chỉ có một hoặc hai trường hợp trong tập huấn luyện, mô hình có thể khớp chúng hoàn hảo, đó là lý do tại sao đường cong bắt đầu từ 0. Nhưng khi các trường hợp mới được thêm vào tập huấn luyện, mô hình không thể khớp dữ liệu huấn luyện một cách hoàn hảo, cả vì dữ liệu nhiễu và vì nó không hề tuyến tính. Vì vậy, lỗi trên dữ liệu huấn luyện tăng lên cho đến khi đạt đến một cao nguyên, tại thời điểm đó việc thêm các trường hợp mới vào tập huấn luyện không làm cho lỗi trung bình tốt hơn hoặc tệ hơn nhiều. Bây giờ hãy nhìn vào lỗi xác

thực. Khi mô hình được huấn luyện trên rất ít trường hợp huấn luyện, nó không có khả năng tổng quát hóa đúng cách, đó là lý do tại thái lỗi xác thực ban đầu khá lớn. Sau đó, khi mô hình được hiển thị nhiều ví dụ huấn luyện hơn, nó học, và do đó lỗi xác thực dần dần giảm xuống. Tuy nhiên, một lần nữa một đường thẳng không thể làm tốt việc mô hình hóa dữ liệu, vì vậy lỗi kết thúc ở một cao nguyên, rất gần với đường cong kia.

Các đường cong học tập này điển hình cho một mô hình đang dưới khớp. Cả hai đường cong đều đã đạt đến một cao nguyên; chúng gần nhau và khá cao.

Bây giờ chúng ta hãy nhìn vào các đường cong học tập của mô hình đa thức bậc 10 trên cùng dữ liệu (Hình 4-16):

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures # Import
PolynomialFeatures
from sklearn.linear_model import LinearRegression # Import LinearRegression

polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())

train_sizes, train_scores, valid_scores = learning_curve(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40),
cv=5,
    scoring="neg_root_mean_squared_error")
# [...] # same as earlier
```



Hình 4-16. Đường cong học tập cho mô hình đa thức bậc 10.

Những đường cong học tập này trông hơi giống những đường cong trước, nhưng có hai điểm khác biệt rất quan trọng:

- Lỗi trên dữ liệu huấn luyện thấp hơn nhiều so với trước đây.
- Có một khoảng cách giữa các đường cong. Điều này có nghĩa là mô hình hoạt động tốt hơn đáng kể trên dữ liệu huấn luyện so với dữ liệu xác thực, đây là đặc điểm của một mô hình quá khớp. Tuy nhiên, nếu bạn sử dụng một tập huấn luyện lớn hơn nhiều, hai đường cong sẽ tiếp tục gần nhau hơn.

Sự đánh đổi giữa độ chệch/phương sai Một kết quả lý thuyết quan trọng của thống kê và học máy là thực tế lỗi tổng quát hóa của một mô hình có thể được biểu diễn dưới dạng tổng của ba loại lỗi rất khác nhau:

- **Độ chệch (Bias)** Phần lỗi tổng quát hóa này là do các giả định sai, chẳng hạn như giả định rằng dữ liệu là tuyến tính khi thực tế nó là bậc hai. Một mô hình có độ chệch cao rất có khả năng dưới khớp dữ liệu huấn luyện.
- **Phương sai (Variance)** Phần này là do mô hình quá nhạy cảm với những biến đổi nhỏ trong dữ liệu huấn luyện. Một mô hình có nhiều bậc tự do (chẳng hạn như mô hình đa thức bậc cao) có khả năng có phương sai cao và do đó quá khớp dữ liệu huấn luyện.
- **Lỗi không thể giảm (Irreducible error)** Phần này là do tính nhiễu của chính dữ liệu. Cách duy nhất để giảm phần lỗi này là làm sạch dữ liệu (ví dụ: sửa các nguồn dữ liệu, chẳng hạn như cảm biến bị hỏng, hoặc phát hiện và loại bỏ các giá trị ngoại lai).

Tăng độ phức tạp của mô hình thường sẽ làm tăng phương sai và giảm độ chệch của nó. Ngược lại, giảm độ phức tạp của mô hình sẽ làm tăng độ chệch và giảm phương sai của nó. Đây là lý do tại sao nó được gọi là một sự đánh đổi.

4.5 Mô hình tuyến tính được chính quy hóa

Như bạn đã thấy trong Chương 1 và 2, một cách tốt để giảm quá khớp là chính quy hóa mô hình (tức là ràng buộc nó): nó càng có ít bậc tự do, càng khó để nó quá khớp dữ liệu. Một cách đơn giản để chính quy hóa một mô hình đa thức là giảm số bậc đa thức. Đối với một mô hình tuyến tính, chính quy hóa thường đạt được bằng cách ràng buộc trọng số của mô hình. Chúng ta sẽ xem xét hồi quy Ridge, hồi quy Lasso và hồi quy Elastic Net, đây là ba cách khác nhau để ràng buộc trọng số.

4.5.1 Hồi quy Ridge

Hồi quy Ridge (còn được gọi là **chuẩn hóa Tikhonov**) là một phiên bản được chuẩn hóa của hồi quy tuyến tính: một số hạng chuẩn hóa bằng $\frac{\alpha}{m}\sum_{i=1}^{n}\theta_{i2}$ được thêm vào MSE. Điều này buộc thuật toán học không chỉ phải khớp với dữ liệu mà còn phải giữ cho các trọng số của mô hình càng nhỏ càng tốt. Lưu ý rằng số hạng chuẩn hóa chỉ nên được thêm vào hàm chi phí trong quá trình huấn luyện. Sau khi mô hình được huấn luyện, bạn chỉ nên sử dụng MSE (hoặc RMSE) không được chuẩn hóa để đánh giá hiệu suất của mô hình.

Siêu tham số (hyperparameter) α kiểm soát mức độ bạn muốn chuẩn hóa mô hình. Nếu $\alpha = 0$, hồi quy Ridge chỉ là hồi quy tuyến tính. Nếu α rất lớn, tất cả các trọng số cuối cùng sẽ gần bằng 0 và kết quả là một đường thẳng đi qua giá trị trung bình của dữ liệu.

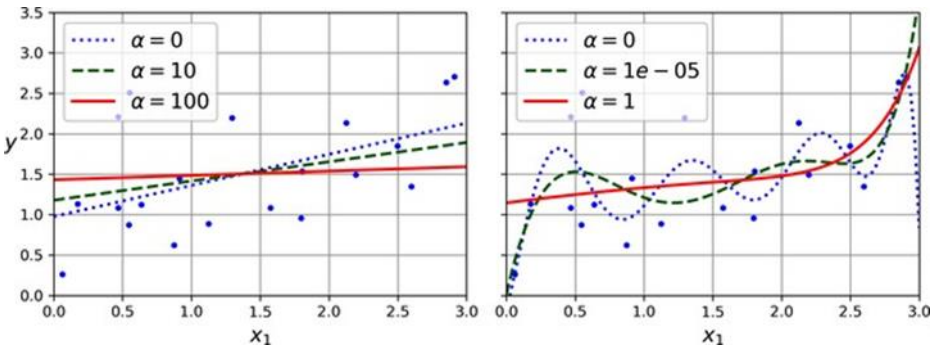
Công thức 4-8 trình bày hàm chi phí hồi quy Ridge.

Công thức 4-8: Hàm chi phí hồi quy Ridge

$$J(\theta) = MSE(\theta) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_{i2}$$

Lưu ý rằng số hạng độ lệch θ_0 không được chuẩn hóa (tổng bắt đầu từ $i = 1$, không phải 0). Nếu chúng ta định nghĩa w là vector trọng số đặc trưng (θ_1 đến θ_n), thì số hạng chuẩn hóa bằng $\alpha \parallel w \parallel_{22}/m$, trong đó $\parallel w \parallel_2$ là chuẩn l_2 của vector trọng số. Đối với hạ gradient theo lô (batch gradient descent), chỉ cần thêm $2\alpha w/m$ vào phần của vector gradient MSE tương ứng với các trọng số đặc trưng, mà không thêm gì vào phần của số hạng độ lệch (xem Công thức 4-6).

Hình 4-17 cho thấy một số mô hình Ridge được huấn luyện trên một số dữ liệu tuyến tính rất nhiễu bằng cách sử dụng các giá trị $alpha$ khác nhau. Ở bên trái, các mô hình Ridge thông thường được sử dụng, dẫn đến các dự đoán tuyến tính. Ở bên phải, dữ liệu đầu tiên được mở rộng bằng `PolynomialFeatures(degree=10)`, sau đó được chuẩn hóa bằng `StandardScaler`, và cuối cùng các mô hình Ridge được áp dụng cho các đặc trưng kết quả: đây là hồi quy đa thức với chính quy hóa Ridge. Lưu ý cách tăng $alpha$ dẫn đến các dự đoán phẳng hơn (tức là ít cực đoan hơn, hợp lý hơn), do đó giảm phương sai của mô hình nhưng tăng độ chệch của nó.



Hình 4-17. Mô hình tuyến tính (trái) và đa thức (phải), cả hai đều có các mức chính quy hóa Ridge khác nhau.

Tương tự như hồi quy tuyến tính, chúng ta có thể thực hiện hồi quy Ridge bằng cách tính một phương trình dạng đóng hoặc bằng cách thực hiện hạ gradient. Ưu và nhược điểm của hai cách này là tương tự nhau.

Công thức 4-9 trình bày nghiệm dạng đóng cho hồi quy Ridge, trong đó A là ma trận đơn vị $(n + 1) \times (n + 1)$, nhưng ô trên cùng bên trái bằng 0, tương ứng với số hạng độ lệch (bias term).

Công thức 4-9: Nghiệm dạng đóng của hồi quy Ridge

$$\theta^{=}(X^{\intercal}X + \alpha A)^{-1}X^{\intercal}y$$

Dưới đây là cách thực hiện hồi quy Ridge với Scikit-Learn bằng cách sử dụng giải pháp dạng đóng (một biến thể của Phương trình 4-9 sử dụng kỹ thuật phân tách ma trận của André-Louis Cholesky):

```
from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
# Output: array([[1.55325833]])
```

Và sử dụng stochastic gradient descent:

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
                       max_iter=1000, eta0=0.01, random_state=42)
sgd_reg.fit(X, y.ravel()) # y.ravel() because fit() expects 1D targets
sgd_reg.predict([[1.5]])
# Output: array([1.55302613])
```

Siêu tham số `penalty` đặt loại thuật ngữ chính quy hóa sẽ sử dụng. Việc chỉ định "l2" cho biết bạn muốn SGD thêm một thuật ngữ chính quy hóa vào hàm chi phí MSE bằng `alpha` nhân với bình phương chuẩn ell_2 của vector trọng số. Điều này giống như hồi quy Ridge, ngoại trừ trong trường hợp này không có phép chia cho `m`; đó là lý do tại sao chúng ta truyền `alpha=0.1 / m`, để có cùng kết quả như `Ridge(alpha=0.1)`.

4.5.2 Hồi quy Lasso

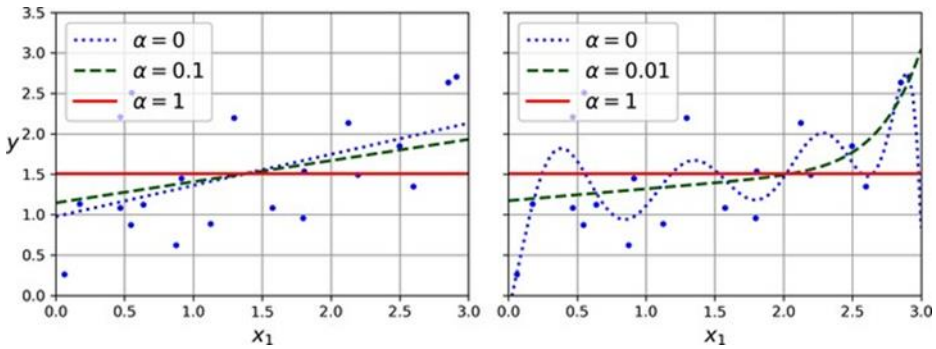
Hồi quy Lasso (tên đầy đủ là **Least absolute shrinkage and selection operator regression**) là một phiên bản hồi quy tuyến tính có chuẩn hóa khác: giống như hồi quy Ridge, nó thêm một số hạng chuẩn hóa vào hàm chi phí, nhưng sử dụng chuẩn l_1 của vector trọng số thay vì bình phương của chuẩn l_2 (xem Công thức 4-10).

Lưu ý rằng chuẩn l_1 được nhân với 2α , trong khi chuẩn l_2 được nhân với α/m trong hồi quy Ridge. Các hệ số này được chọn để đảm bảo rằng giá trị α tối ưu là độc lập với kích thước của tập huấn luyện. Các chuẩn khác nhau dẫn đến các hệ số khác nhau.

Công thức 4-10: Hàm chi phí hồi quy Lasso

$$J(\theta) = MSE(\theta) + 2\alpha \sum_{i=1}^{n} | \theta_i |$$

Hình 4-18 cho thấy điều tương tự như Hình 4-17 nhưng thay thế các mô hình Ridge bằng các mô hình Lasso và sử dụng các giá trị $alpha$ khác nhau.

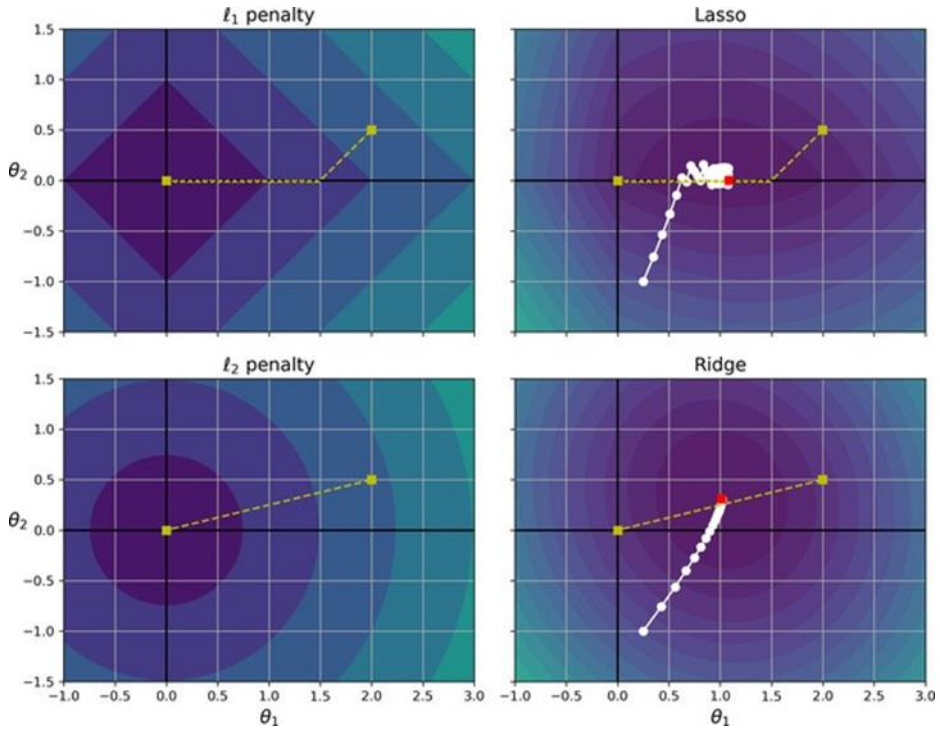


Hình 4-18. Mô hình tuyến tính (trái) và đa thức (phải), cả hai đều sử dụng các mức chính quy hóa Lasso khác nhau.

Một đặc điểm quan trọng của hồi quy Lasso là nó có xu hướng loại bỏ trọng số của các đặc trưng ít quan trọng nhất (nghĩa là, đặt chúng bằng không). Do đó, hồi quy Lasso tự động thực hiện **lựa chọn đặc trưng** và cho ra một mô hình **thưa** (sparse) với ít trọng số đặc trưng khác không.

Bạn có thể hiểu lý do tại sao lại như vậy khi nhìn vào Hình 4-19: các trục đại diện cho hai tham số mô hình và các đường viền nền biểu thị các hàm mất mát khác nhau. Trong biểu đồ phía trên bên trái, các đường viền đại diện cho hàm mất mát l_1 (nghĩa là $|\theta_1| + |\theta_2|$), giảm tuyến tính khi bạn tiến gần đến bất kỳ trục nào. Ví dụ, nếu bạn khởi tạo các tham số mô hình tại $\theta_1 = 2$ và $\theta_2 = 0.5$, việc chạy hạ gradient sẽ làm giảm cả hai tham số như nhau (được biểu thị bằng đường đứt nét màu vàng); do đó θ_2 sẽ về 0 trước (vì nó gần 0 hơn ngay từ đầu). Sau đó, việc hạ gradient sẽ lăn xuống "máng xối" cho đến khi nó đạt đến $\theta_1 = 0$.

Trong biểu đồ phía trên bên phải, các đường viền đại diện cho hàm mất mát MSE cộng với một hàm mất mát l_1. Các vòng tròn nhỏ màu trắng cho thấy đường đi mà hạ gradient thực hiện để tối ưu hóa một số tham số mô hình được khởi tạo xung quanh $\theta_1 = 0.25$ và $\theta_2 = -1$. Lưu ý một lần nữa cách đường đi nhanh chóng đạt đến $\theta_2 = 0$, sau đó lăn xuống "máng xối" và kết thúc ở xung quanh điểm tối ưu toàn cục (được biểu thị bằng hình vuông màu đỏ). Nếu chúng ta tăng α, điểm tối ưu toàn cục sẽ di chuyển sang trái dọc theo đường đứt nét màu vàng, trong khi nếu chúng ta giảm α, điểm tối ưu toàn cục sẽ di chuyển sang phải (trong ví dụ này, các tham số tối ưu cho MSE không được chuẩn hóa là $\theta_1 = 2$ và $\theta_2 = 0.5$).



Hình 4-19. Chính quy hóa Lasso so với Ridge.

Hai biểu đồ phía dưới cho thấy điều tương tự nhưng với hình phạt ell_2 thay thế. Trong biểu đồ dưới cùng bên trái, bạn có thể thấy rằng lỗi ell_2 giảm khi chúng ta đến gần gốc tọa độ hơn, vì vậy gradient descent chỉ đi một đường thẳng về phía điểm đó. Trong biểu đồ dưới cùng bên phải, các đường đồng mức đại diện cho hàm chi phí của hồi quy Ridge (tức là hàm chi phí MSE cộng với lỗi ell_2). Như bạn có thể thấy, các gradient trở nên nhỏ hơn khi các tham số tiếp cận cực tối ưu toàn cục, vì vậy gradient descent tự nhiên chậm lại. Điều này hạn chế sự dao động, giúp Ridge hội tụ nhanh hơn hồi quy Lasso. Cũng lưu ý rằng các tham số tối ưu (được biểu thị bằng hình vuông màu đỏ) ngày càng gần gốc tọa độ hơn khi bạn tăng $alpha$, nhưng chúng không bao giờ bị loại bỏ hoàn toàn.

Hàm chi phí của Lasso không khả vi tại $theta_i = 0$ (với $i = 1,2,\ldots,n$), nhưng hạ gradient vẫn hoạt động tốt nếu bạn sử dụng một **vector hạ gradient** g thay vì gradient thông thường khi một trong các $theta_i$ bằng 0.

Công thức 4-11 trình bày một vector hạ gradient của Lasso mà bạn có thể sử dụng cho hạ gradient với hàm chi phí Lasso.

Công thức 4-11: Vector hạ gradient của Lasso

$$g(\boldsymbol{\theta}, J) = \nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta}) + 2\alpha \begin{pmatrix} \operatorname{sign}(\theta_1) \\ \operatorname{sign}(\theta_2) \\ \vdots \\ \operatorname{sign}(\theta_n) \end{pmatrix} \quad \text{where } \operatorname{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

Sau đây là một ví dụ đơn giản với Scikit-Learn sử dụng lớp `Lasso`:

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

Lưu ý rằng bạn cũng có thể sử dụng `SGDRegressor(penalty="l1", alpha=0.1)` thay thế.

Hồi quy Elastic Net

Hồi quy Elastic Net là một điểm trung gian giữa hồi quy Ridge và hồi quy Lasso. Số hạng chuẩn hóa của nó là tổng hợp của cả hai số hạng chuẩn hóa của Ridge và Lasso, và bạn có thể kiểm soát tỷ lệ pha trộn bằng tham số `r`. Khi $r = 0$, Elastic Net tương đương với hồi quy Ridge, và khi $r = 1$, nó tương đương với hồi quy Lasso (xem **Công thức 4-12**).

Công thức 4-12: Hàm chi phí Elastic Net

$$J(\theta) = MSE(\theta) + r\left(2\alpha \sum_{i=1}^{n} |\theta_i|\right) + (1-r)\left(\frac{\alpha}{m}\sum_{i=1}^{n}\theta_{i2}\right)$$

Vậy khi nào bạn nên sử dụng hồi quy Elastic Net, Ridge, Lasso, hay hồi quy tuyến tính thông thường? Tốt nhất là bạn nên luôn ưu tiên sử dụng một chút chuẩn hóa, vì vậy bạn nên tránh sử dụng hồi quy tuyến tính thông thường. Ridge là một lựa chọn mặc định tốt. Nếu bạn nghi ngờ rằng chỉ một vài đặc trưng là hữu ích, bạn nên ưu tiên Lasso hoặc Elastic Net vì chúng có xu hướng đẩy trọng số của các đặc trưng vô ích về 0, như đã thảo luận trước đó. Nhìn chung, Elastic Net được ưa chuộng hơn Lasso vì Lasso có thể hoạt động thất thường khi số lượng đặc trưng lớn hơn số lượng trường hợp huấn luyện, hoặc khi một số đặc trưng có tương quan mạnh với nhau.

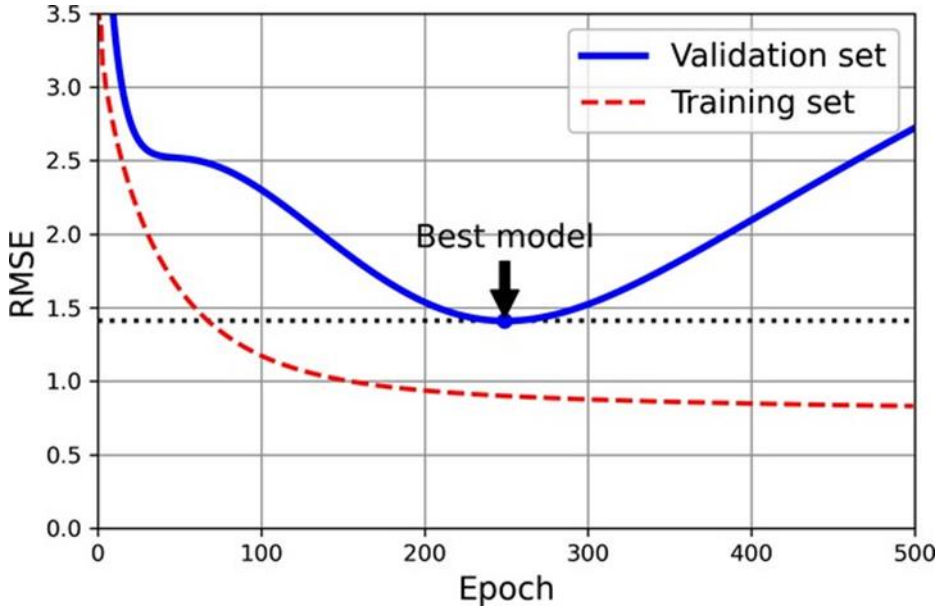
Dưới đây là một ví dụ đơn giản sử dụng `ElasticNet` của Scikit-Learn (`l1_ratio` tương ứng với tỷ lệ pha trộn r):

```
from sklearn.linear_model import ElasticNet

elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
elastic_net.predict([[1.5]]) # Output: array([1.54333232])
```

4.5.4 Dừng sớm (Early Stopping)

Một cách rất khác để chính quy hóa các thuật toán học lặp như gradient descent là dừng huấn luyện ngay khi lỗi xác thực đạt đến cực tiểu. Điều này được gọi là **dừng sớm**. Hình 4-20 cho thấy một mô hình phức tạp (trong trường hợp này, một mô hình hồi quy đa thức bậc cao) đang được huấn luyện bằng gradient descent theo lô trên tập dữ liệu bậc hai chúng ta đã sử dụng trước đó. Khi các epoch trôi qua, thuật toán học, và lỗi dự đoán (RMSE) trên tập huấn luyện giảm xuống, cùng với lỗi dự đoán trên tập xác thực. Tuy nhiên, sau một thời

gian, lỗi xác thực ngừng giảm và bắt đầu tăng trở lại. Điều này cho thấy mô hình đã bắt đầu quá khớp dữ liệu huấn luyện. Với dừng sớm, bạn chỉ cần dừng huấn luyện ngay khi lỗi xác thực đạt đến cực tiểu. Đây là một kỹ thuật chính quy hóa đơn giản và hiệu quả đến mức Geoffrey Hinton gọi nó là "bữa trưa miễn phí tuyệt vời".



Hình 4-20. Chính quy hóa dừng sớm.

Dưới đây là một triển khai cơ bản của dừng sớm:

```
from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline # Import make_pipeline
from sklearn.preprocessing import PolynomialFeatures # Import
PolynomialFeatures
from sklearn.linear_model import SGDRegressor # Import SGDRegressor

# Assuming X_train, y_train, X_valid, y_valid are already defined as
quadratic dataset
# For example:
# np.random.seed(42)
# m = 100
# X = 6 * np.random.rand(m, 1) - 3
# y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
# X_train, X_valid, y_train, y_valid = train_test_split(X, y,
random_state=42)

# X_train, y_train, X_valid, y_valid = [...] #split the quadratic dataset #
Placeholder for actual data split

preprocessing = make_pipeline(PolynomialFeatures(degree=90,
```

```
include_bias=False),
                                 StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)

sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = mean_squared_error(y_valid, y_valid_predict, squared=False)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)
```

Đoạn mã này đầu tiên thêm các đặc trưng đa thức và điều chỉnh tất cả các đặc trưng đầu vào, cả cho tập huấn luyện và cho tập xác thực (mã giả định rằng bạn đã chia tập huấn luyện gốc thành một tập huấn luyện nhỏ hơn và một tập xác thực). Sau đó, nó tạo một mô hình `SGDRegressor` không có chính quy hóa và tốc độ học nhỏ. Trong vòng lặp huấn luyện, nó gọi `partial_fit()` thay vì `fit()`, để thực hiện học tăng dần. Ở mỗi epoch, nó đo RMSE trên tập xác thực. Nếu nó thấp hơn RMSE thấp nhất được thấy cho đến nay, nó sẽ lưu một bản sao của mô hình vào biến `best_model`. Triển khai này thực tế không dừng huấn luyện, nhưng nó cho phép bạn quay lại mô hình tốt nhất sau khi huấn luyện. Lưu ý rằng mô hình được sao chép bằng `copy.deepcopy()`, bởi vì nó sao chép cả siêu tham số và tham số đã học của mô hình. Ngược lại, `sklearn.base.clone()` chỉ sao chép siêu tham số của mô hình.

4.6 Hồi quy Logistic

Như đã thảo luận trong Chương 1, một số thuật toán hồi quy có thể được sử dụng để phân loại (và ngược lại). Hồi quy Logistic (còn được gọi là hồi quy logit) thường được sử dụng để ước tính xác suất một trường hợp thuộc về một lớp cụ thể (ví dụ: xác suất email này là thư rác là bao nhiêu?). Nếu xác suất ước tính lớn hơn một ngưỡng nhất định (thường là 50%), thì mô hình dự đoán rằng trường hợp đó thuộc về lớp đó (được gọi là lớp dương, được gắn nhãn "1"), và ngược lại nó dự đoán rằng nó không thuộc về lớp đó (tức là nó thuộc về lớp âm, được gắn nhãn "0"). Điều này làm cho nó trở thành một bộ phân loại nhị phân.

4.6.1 Ước tính xác suất

Mô hình hồi quy Logistic hoạt động tương tự như mô hình hồi quy tuyến tính, cũng tính tổng trọng số của các đặc trưng đầu vào (cộng với một số hạng độ lệch). Tuy nhiên, thay vì xuất ra kết quả trực tiếp, nó xuất ra **hàm logistic** của kết quả đó.

Công thức 4-13: Xác suất ước lượng của hồi quy Logistic (dạng vector)

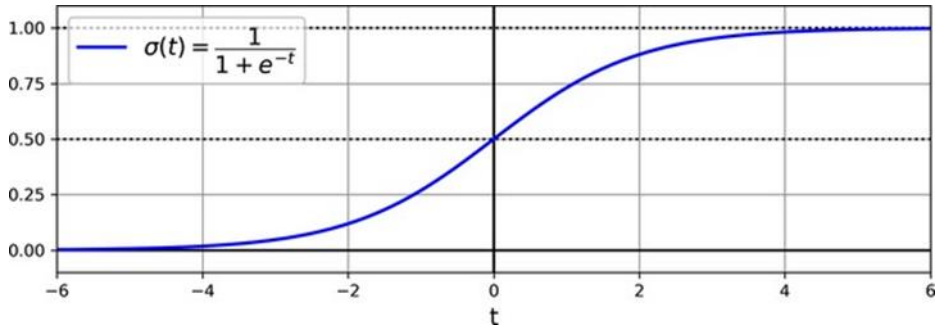
$$p = h_\theta(x) = \sigma(\theta^\intercal x)$$

Hàm logistic ($\sigma(\cdot)$) là một hàm sigmoid (có hình chữ S) cho ra một số giữa 0 và 1. Nó được định nghĩa như sau:

Công thức 4-14: Hàm Logistic

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

Khi mô hình hồi quy Logistic đã ước lượng xác suất $p = h_\theta(x)$ rằng một trường hợp x thuộc về lớp dương, nó có thể dễ dàng đưa ra dự đoán y (xem Công thức 4-15).



Hình 4-21. Hàm logistic.

Công thức 4-15: Dự đoán mô hình hồi quy Logistic sử dụng ngưỡng xác suất 50%

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

Bạn có thể thấy rằng $\sigma(t) < 0.5$ khi $t < 0$, và $\sigma(t) \geq 0.5$ khi $t \geq 0$. Do đó, một mô hình hồi quy logistic sử dụng ngưỡng mặc định 50% sẽ dự đoán 1 nếu $\theta^T x$ dương và 0 nếu nó âm.

Hàm huấn luyện và Hàm chi phí

Mục tiêu của việc huấn luyện là thiết lập vector tham số θ sao cho mô hình ước lượng xác suất cao cho các trường hợp dương ($y = 1$) và xác suất thấp cho các trường hợp âm ($y = 0$). Ý tưởng này được thể hiện trong hàm chi phí sau:

Công thức 4-16: Hàm chi phí cho một trường hợp huấn luyện đơn lẻ x

$$c(\boldsymbol{\theta}) = \begin{cases} -\log\left(\hat{p}\right) & \text{if } y = 1 \\ -\log\left(1 - \hat{p}\right) & \text{if } y = 0 \end{cases}$$

Hàm chi phí này có ý nghĩa vì $-\log(t)$ sẽ trở nên rất lớn khi t tiến gần đến 0. Do đó, chi phí sẽ rất lớn nếu mô hình ước lượng xác suất gần 0 cho một trường hợp dương, hoặc ước lượng xác suất gần 1 cho một trường hợp âm. Điều này chính xác là những gì chúng ta muốn.

Hàm chi phí trên toàn bộ tập huấn luyện là giá trị trung bình của chi phí trên tất cả các trường hợp huấn luyện. Nó có thể được viết trong một biểu thức duy nhất gọi là **log loss**, được thể hiện trong Công thức 4-17.

Công thức 4-17: Hàm chi phí hồi quy Logistic (log loss)

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(p^{(i)}\right) + \left(1 - y^{(i)}\right)\log\left(1 - p^{(i)}\right)\right]$$

Đạo hàm riêng của hàm chi phí

Tin xấu là không có phương trình dạng đóng nào để tính giá trị của θ làm giảm thiểu hàm chi phí này (tức là không có phương trình tương đương với Phương trình chuẩn tắc). Nhưng tin tốt là hàm chi phí này là hàm lồi, vì vậy hạ gradient (hoặc bất kỳ thuật toán tối ưu hóa nào khác) được đảm bảo sẽ tìm thấy cực tiểu toàn cục (nếu tốc độ học không quá lớn và bạn chờ đủ lâu).

Đạo hàm riêng của hàm chi phí đối với tham số mô hình thứ j , θ_j , được cho bởi Công thức 4-18.

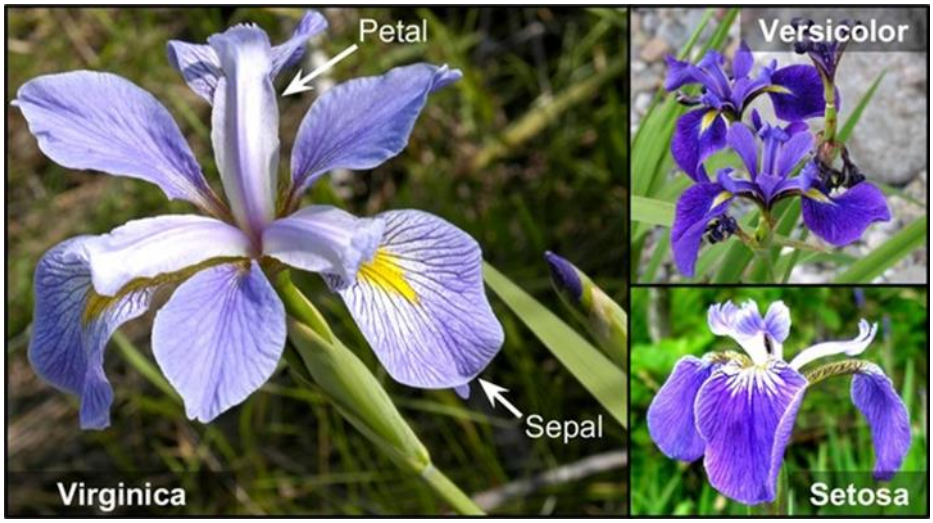
Công thức 4-18: Đạo hàm riêng của hàm chi phí Logistic

$$\frac{\partial}{\partial\theta_j}J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(\sigma\left(\theta^{\intercal}x^{(i)}\right) - y^{(i)}\right)x_{j(i)}$$

Phương trình này rất giống với Công thức 4-5: đối với mỗi trường hợp, nó tính sai số dự đoán và nhân với giá trị đặc trưng thứ j , sau đó tính trung bình trên tất cả các trường hợp huấn luyện. Khi bạn đã có vector gradient chứa tất cả các đạo hàm riêng, bạn có thể sử dụng nó trong thuật toán hạ gradient theo lô.

4.6.3 Các đường ranh giới quyết định

Chúng ta có thể sử dụng tập dữ liệu iris để minh họa hồi quy logistic. Đây là một tập dữ liệu nổi tiếng chứa chiều dài và chiều rộng đài hoa (sepal) và cánh hoa (petal) của 150 bông hoa diên vĩ thuộc ba loài khác nhau: *Iris setosa*, *Iris versicolor* và *Iris virginica* (xem Hình 4-22).



Hình 4-22. Hoa của ba loài thực vật diên vĩ.

Hãy thử xây dựng một bộ phân loại để phát hiện loài *Iris virginica* chỉ dựa trên đặc trưng chiều rộng cánh hoa. Bước đầu tiên là tải dữ liệu và xem nhanh:

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> list(iris)
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
'filename', 'data_module']
>>> iris.data.head(3)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
>>> iris.target.head(3)  # note that the instances are not shuffled
0    0
1    0
2    0
Name: target, dtype: int64
>>> iris.target_names
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

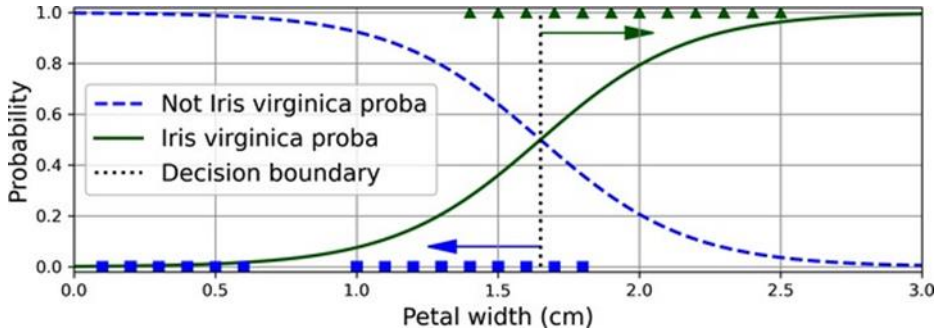
Tiếp theo, chúng ta sẽ chia dữ liệu và huấn luyện một mô hình hồi quy logistic trên tập huấn luyện:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

```
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

Hãy xem các xác suất ước tính của mô hình cho các bông hoa có chiều rộng cánh hoa dao động từ 0 cm đến 3 cm (Hình 4-23).

```
import numpy as np
import matplotlib.pyplot as plt

X_new = np.linspace(0, 3, 1000).reshape(-1, 1) # reshape to get a column
vector
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]

plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2, label="Not Iris virginica
proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris virginica
proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
label="Decision boundary")
# [...] # beautify the figure: add grid, labels, axis, legend, arrows, and
samples
plt.show()
```

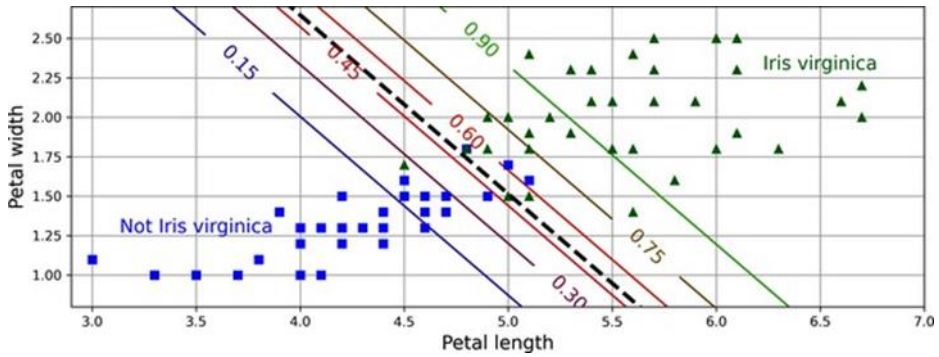


Hình 4-23. Xác suất ước tính và đường ranh giới quyết định.

Chiều rộng cánh hoa của các bông *Iris virginica* (được biểu thị bằng hình tam giác) dao động từ 1.4 cm đến 2.5 cm, trong khi các bông diên vĩ khác (được biểu thị bằng hình vuông) thường có chiều rộng cánh hoa nhỏ hơn, dao động từ 0.1 cm đến 1.8 cm. Lưu ý rằng có một chút chồng chéo. Trên khoảng 2 cm, bộ phân loại rất tự tin rằng bông hoa là một *Iris virginica* (nó xuất ra xác suất cao cho lớp đó), trong khi dưới 1 cm, nó rất tự tin rằng đó không phải là một *Iris virginica* (xác suất cao cho lớp "Không phải *Iris virginica*"). Giữa hai thái cực này, bộ phân loại không chắc chắn. Tuy nhiên, nếu bạn yêu cầu nó dự đoán lớp (bằng cách sử dụng phương thức `predict()` thay vì phương thức `predict_proba()`), nó sẽ trả về lớp có khả năng nhất. Do đó, có một đường ranh giới quyết định ở khoảng 1.6 cm nơi cả hai xác suất đều bằng 50%: nếu chiều rộng cánh hoa lớn hơn 1.6 cm, bộ phân loại sẽ dự đoán rằng bông hoa là một *Iris virginica*, và nếu không, nó sẽ dự đoán rằng nó không phải (ngay cả khi nó không quá tự tin):

```
>>> decision_boundary
1.6516516516516517
>>> log_reg.predict([[1.7], [1.5]])
array([ True, False])
```

Hình 4-24 cho thấy cùng một tập dữ liệu, nhưng lần này hiển thị hai đặc trưng: chiều rộng và chiều dài cánh hoa. Sau khi được huấn luyện, bộ phân loại hồi quy logistic có thể, dựa trên hai đặc trưng này, ước tính xác suất một bông hoa mới là *Iris virginica*. Đường chấm chấm biểu thị các điểm mà mô hình ước tính xác suất 50%: đây là đường ranh giới quyết định của mô hình. Lưu ý rằng nó là một đường ranh giới tuyến tính. Mỗi đường thẳng song song biểu thị các điểm mà mô hình xuất ra một xác suất cụ thể, từ 15% (dưới cùng bên trái) đến 90% (trên cùng bên phải). Tất cả các bông hoa nằm ngoài đường trên cùng bên phải có hơn 90% khả năng là *Iris virginica*, theo mô hình.



Hình 4-24. Đường ranh giới quyết định tuyến tính.

Cũng như các mô hình tuyến tính khác, mô hình hồi quy logistic có thể được chính quy hóa bằng cách sử dụng hình phạt ell_1 hoặc ell_2 . Scikit-Learn thực tế thêm hình phạt ell_2 theo mặc định.

Hồi quy Softmax

Hồi quy logistic có thể được tổng quát hóa để hỗ trợ trực tiếp nhiều lớp mà không cần phải huấn luyện và kết hợp nhiều bộ phân loại nhị phân. Phương pháp này được gọi là **hồi quy softmax** hoặc **hồi quy logistic đa thức**.

Ý tưởng rất đơn giản: với một trường hợp x , mô hình hồi quy softmax tính một điểm số $s_k(x)$ cho mỗi lớp k , sau đó ước lượng xác suất của mỗi lớp bằng cách áp dụng **hàm softmax** (còn gọi là hàm lũy thừa chuẩn hóa) cho các điểm số. Phương trình tính $s_k(x)$ sẽ quen thuộc vì nó giống hệt phương trình dự đoán hồi quy tuyến tính.

Công thức 4-19: Điểm số Softmax cho lớp k

$$s_k(x) = \left(\theta^{(k)}\right)^{\intercal} x$$

Lưu ý rằng mỗi lớp có vector tham số riêng biệt $\theta^{(k)}$. Tất cả các vector này thường được lưu trữ dưới dạng các hàng trong một ma trận tham số θ .

Khi bạn đã tính điểm số của mỗi lớp cho trường hợp x , bạn có thể ước lượng xác suất $hatp_k$ rằng trường hợp đó thuộc về lớp k bằng cách chạy các điểm số qua hàm softmax.

Công thức 4-20: Hàm Softmax

$$p_k = \sigma\big(s(x)\big)_k = \frac{\exp\big(s_k(x)\big)}{\sum_{j=1}^{K}\exp\Big(s_j(x)\Big)}$$

Trong phương trình này:

- K là số lượng lớp.
- $s(x)$ là vector chứa điểm số của mỗi lớp cho trường hợp x .
- $sigma\big(s(x)\big)_k$ là xác suất ước lượng rằng trường hợp x thuộc về lớp k , cho trước điểm số của mỗi lớp cho trường hợp đó.

Giống như bộ phân loại hồi quy logistic, bộ phân loại hồi quy softmax mặc định dự đoán lớp có xác suất ước lượng cao nhất (đơn giản là lớp có điểm số cao nhất).

Công thức 4-21: Dự đoán của bộ phân loại Softmax

$$y = argmax_k\sigma\big(s(x)\big)_k = argmax_k s_k(x) = argmax_k\Big(\big(\theta^{(k)}\big)^{\mathrm{T}}x\Big)$$

Toán tử **argmax** trả về giá trị của một biến làm tối đa hóa một hàm. Trong phương trình này, nó trả về giá trị của k làm tối đa hóa xác suất ước lượng $\sigma\big(s(x)\big)_k$.

Bây giờ bạn đã biết cách mô hình ước lượng xác suất và đưa ra dự đoán, hãy xem xét việc huấn luyện. Mục tiêu là để mô hình ước lượng xác suất cao cho lớp mục tiêu (và do đó, xác suất thấp cho các lớp khác). Việc tối thiểu hóa hàm chi phí trong Công thức 4-22 sẽ đạt được mục tiêu này vì nó phạt mô hình khi ước lượng xác suất thấp cho một lớp mục tiêu. **Cross entropy** (Hàm mất mát chéo) thường được sử dụng để đo lường mức độ phù hợp của một tập hợp các xác suất lớp ước lượng với các giá trị mục tiêu.

Công thức 4-22: Hàm chi phí Cross entropy

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_{k(i)}\log\big(p_{k(i)}\big)$$

Trong phương trình này, $y_k^{(i)}$ là mục tiêu của trường hợp thứ i thuộc về lớp k . Nói chung, nó bằng 1 hoặc 0, tùy thuộc vào việc trường hợp đó có thuộc về lớp đó hay không.

Lưu ý rằng khi chỉ có hai lớp (tức là $K = 2$), hàm chi phí này tương đương với hàm chi phí hồi quy logistic (log loss; xem Công thức 4-17).

Vector gradient của hàm chi phí đối với $boldsymboltheta^{(k)}$ được cho bởi **Công thức 4-23**.

Công thức 4-23: Vector gradient Cross entropy cho lớp k

$$\nabla_{\theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^{m} \left(p_{k(i)} - y_{k(i)} \right) x^{(i)}$$

Bây giờ bạn có thể tính vector gradient cho mỗi lớp, sau đó sử dụng hạ gradient (hoặc bất kỳ thuật toán tối ưu hóa nào khác) để tìm ma trận tham số Θ làm tối thiểu hóa hàm chi phí.

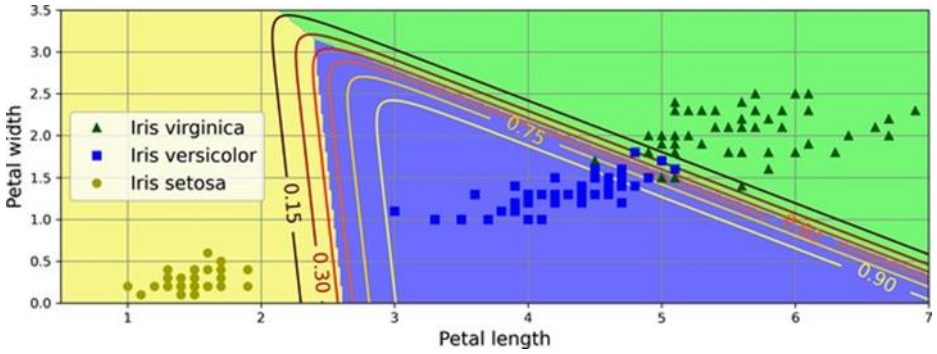
Bây giờ bạn có thể tính toán vector gradient cho mỗi lớp, sau đó sử dụng gradient descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) để tìm ma trận tham số Θ làm giảm thiểu hàm chi phí. Hãy sử dụng hồi quy softmax để phân loại các loài diên vĩ thành cả ba lớp. Bộ phân loại `LogisticRegression` của Scikit-Learn tự động sử dụng hồi quy softmax khi bạn huấn luyện nó trên nhiều hơn hai lớp (giả sử bạn sử dụng `solver="lbfgs"`, đây là mặc định). Nó cũng áp dụng chính quy hóa l_2 theo mặc định, mà bạn có thể kiểm soát bằng siêu tham số `C`, như đã đề cập trước đó:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split # Import
train_test_split

# Assuming iris.data and iris.target are loaded
# from sklearn.datasets import load_iris
# iris = load_iris(as_frame=True)

X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

Vậy lần tới khi bạn tìm thấy một bông diên vĩ có cánh hoa dài 5 cm và rộng 2 cm, bạn có thể yêu cầu mô hình của mình cho bạn biết đó là loại diên vĩ nào, và nó sẽ trả lời *Iris virginica* (lớp 2) với xác suất 96% (hoặc *Iris versicolor* với xác suất 4%):

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]]).round(2)
array([[0.  , 0.04, 0.96]])
```

Hình 4-25 cho thấy các đường ranh giới quyết định thu được, được biểu thị bằng màu nền. Lưu ý rằng các đường ranh giới quyết định giữa bất kỳ hai lớp nào đều là tuyến tính. Hình cũng cho thấy các xác suất cho lớp *Iris versicolor*, được biểu thị bằng các đường cong (ví dụ: đường được gắn nhãn 0.30 biểu thị ranh giới xác suất 30%). Lưu ý rằng mô hình có thể dự đoán một lớp có xác suất ước tính dưới 50%. Ví dụ, tại điểm mà tất cả các đường ranh giới quyết định gặp nhau, tất cả các lớp đều có xác suất ước tính bằng nhau là 33%.



Hình 4-25. Đường ranh giới quyết định của hồi quy Softmax.

Trong chương này, bạn đã tìm hiểu các cách khác nhau để huấn luyện các mô hình tuyến tính, cả cho hồi quy và phân loại. Bạn đã sử dụng một phương trình dạng đóng để giải hồi quy tuyến tính, cũng như gradient descent, và bạn đã học cách thêm các hình phạt khác nhau vào hàm chi phí trong quá trình huấn luyện để chính quy hóa mô hình. Trên đường đi, bạn cũng đã học cách vẽ và phân tích các đường cong học tập, và cách triển khai dừng sớm. Cuối cùng, bạn đã học cách hồi quy logistic và hồi quy softmax hoạt động. Chúng ta đã mở ra những "hộp đen" học máy đầu tiên! Trong các chương tiếp theo, chúng ta sẽ mở ra nhiều hơn nữa, bắt đầu với máy vector hỗ trợ.

4.8 Bài tập

- Bạn có thể sử dụng thuật toán huấn luyện hồi quy tuyến tính nào nếu bạn có một tập huấn luyện với hàng triệu đặc trưng?
- Giả sử các đặc trưng trong tập huấn luyện của bạn có các thang đo rất khác nhau. Thuật toán nào có thể bị ảnh hưởng bởi điều này, và như thế nào? Bạn có thể làm gì để khắc phục nó?
- Gradient descent có thể bị kẹt ở cực tiểu cục bộ khi huấn luyện mô hình hồi quy logistic không?
- Tất cả các thuật toán gradient descent có dẫn đến cùng một mô hình không, miễn là bạn để chúng chạy đủ lâu?
- Giả sử bạn sử dụng gradient descent theo lô và bạn vẽ biểu đồ lỗi xác thực ở mỗi epoch. Nếu bạn nhận thấy rằng lỗi xác thực liên tục tăng lên, điều gì có thể đang xảy ra? Làm thế nào bạn có thể khắc phục điều này?
- Có phải là một ý tưởng hay để dừng gradient descent theo mini-batch ngay lập tức khi lỗi xác thực tăng lên?
- Thuật toán gradient descent nào (trong số những thuật toán chúng ta đã thảo luận) sẽ đạt đến vùng lân cận của giải pháp tối ưu nhanh nhất? Thuật toán nào sẽ thực sự hội tụ? Làm thế nào bạn có thể làm cho các thuật toán khác cũng hội tụ?
- Giả sử bạn đang sử dụng hồi quy đa thức. Bạn vẽ biểu đồ các đường cong học tập và bạn nhận thấy có một khoảng cách lớn giữa lỗi huấn luyện và lỗi xác thực. Điều gì đang xảy ra? Ba cách để giải quyết vấn đề này là gì?

- Giả sử bạn đang sử dụng hồi quy Ridge và bạn nhận thấy rằng lỗi huấn luyện và lỗi xác thực gần như bằng nhau và khá cao. Bạn có nói rằng mô hình bị độ chệch cao hay phương sai cao không? Bạn nên tăng siêu tham số chính quy hóa $alpha$ hay giảm nó?
- Tại sao bạn muốn sử dụng: a. Hồi quy Ridge thay vì hồi quy tuyến tính thông thường (tức là không có bất kỳ chính quy hóa nào)? b. Lasso thay vì hồi quy Ridge? c. Mạng đàn hồi thay vì hồi quy Lasso?
- Giả sử bạn muốn phân loại hình ảnh là ngoài trời/trong nhà và ban ngày/ban đêm. Bạn nên triển khai hai bộ phân loại hồi quy logistic hay một bộ phân loại hồi quy softmax?
- Triển khai gradient descent theo lô với dừng sớm cho hồi quy softmax mà không sử dụng Scikit-Learn, chỉ sử dụng NumPy. Sử dụng nó trên một tác vụ phân loại như tập dữ liệu iris.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại https://homl.info/colab3 .

- Một phương trình dạng đóng chỉ bao gồm một số hữu hạn các hằng số, biến và các phép toán tiêu chuẩn: ví dụ, $a = sin(b - c)$. Không có tổng vô hạn, không có giới hạn, không có tích phân, v.v.
- Nói về mặt kỹ thuật, đạo hàm của nó là liên tục Lipschitz.
- Vì đặc trưng 1 nhỏ hơn, cần một sự thay đổi lớn hơn trong $theta_1$ để ảnh hưởng đến hàm chi phí, đó là lý do tại sao hình bát bị kéo dài dọc theo trục $theta_1$.
- Eta (eta) là chữ cái thứ bảy của bảng chữ cái Hy Lạp.
- Trong khi phương trình chuẩn tắc chỉ có thể thực hiện hồi quy tuyến tính, các thuật toán gradient descent có thể được sử dụng để huấn luyện nhiều mô hình khác, như bạn sẽ thấy.
- Khái niệm độ chệch này không nên bị nhầm lẫn với hệ số chệch của các mô hình tuyến tính.
- Người ta thường sử dụng ký hiệu $J(theta)$ cho các hàm chi phí không có tên ngắn; tôi sẽ thường xuyên sử dụng ký hiệu này trong phần còn lại của cuốn sách này. Ngữ cảnh sẽ làm rõ hàm chi phí nào đang được thảo luận.
- Các chuẩn đã được thảo luận trong Chương 2.
- Một ma trận vuông đầy số 0 ngoại trừ các số 1 trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải).
- Ngoài ra, bạn có thể sử dụng lớp `Ridge` với `solver="sag"`. SGD trung bình ngẫu nhiên là một biến thể của SGD ngẫu nhiên. Để biết thêm chi tiết, xem bài thuyết trình “Minimizing Finite Sums with the Stochastic Average Gradient Algorithm” của Mark Schmidt và cộng sự từ Đại học British Columbia.
- Bạn có thể coi vector cận gradient tại một điểm không khả vi là một vector trung gian giữa các vector gradient xung quanh điểm đó.

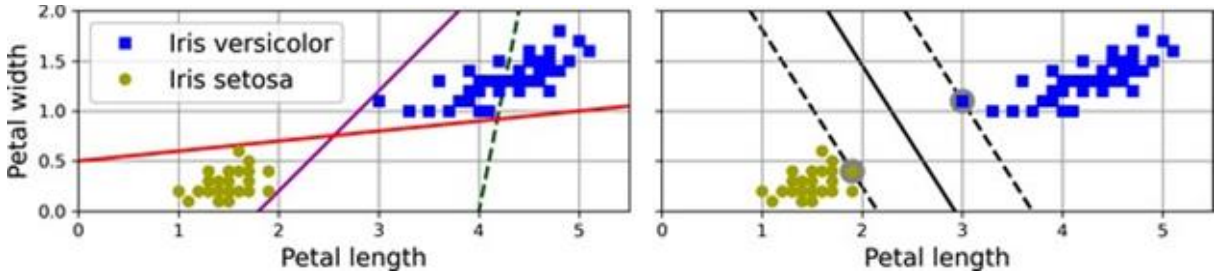
- Ảnh được tái bản từ các trang Wikipedia tương ứng. Ảnh *Iris virginica* của Frank Mayfield (Creative Commons BY-SA 2.0), ảnh *Iris versicolor* của D. Gordon E. Robertson (Creative Commons BY-SA 3.0), ảnh *Iris setosa* thuộc phạm vi công cộng.
- Hàm `reshape()` của NumPy cho phép một chiều là –1, nghĩa là "tự động": giá trị được suy ra từ độ dài của mảng và các chiều còn lại.
- Đây là tập hợp các điểm **x** sao cho $theta_0 + theta_1 x_1 + theta_2 x_2 = 0$, định nghĩa một đường thẳng.

CHƯƠNG 5. MÁY HỌC VÉC-TƠ HỖ TRỢ

Một máy học véc-tơ hỗ trợ (SVM) là một mô hình học máy mạnh mẽ và linh hoạt, có khả năng thực hiện phân loại tuyến tính hoặc phi tuyến, hồi quy, và thậm chí cả phát hiện điểm mới. SVM đặc biệt hiệu quả với các tập dữ liệu phi tuyến có kích thước nhỏ đến trung bình (tức là hàng trăm đến hàng nghìn trường hợp), đặc biệt cho các tác vụ phân loại. Tuy nhiên, chúng không mở rộng tốt với các tập dữ liệu rất lớn, như bạn sẽ thấy. Chương này sẽ giải thích các khái niệm cốt lõi của SVM, cách sử dụng chúng và cách chúng hoạt động. Hãy cùng tìm hiểu ngay!

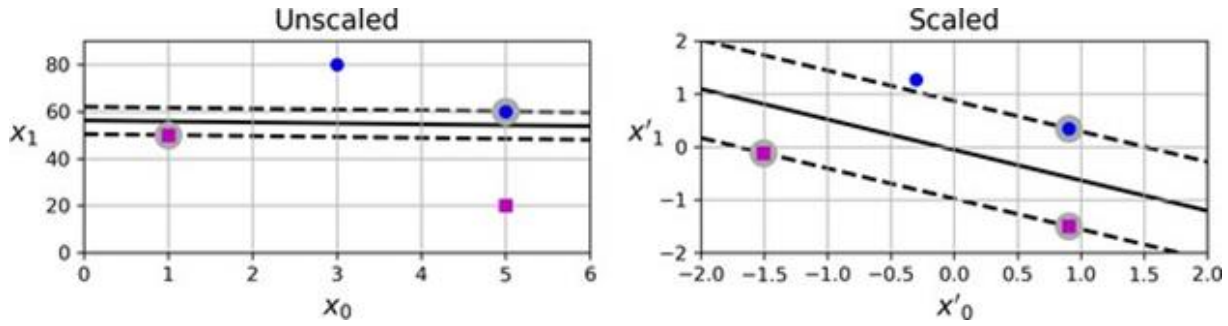
5.1. Phân loại SVM tuyến tính

Ý tưởng cơ bản đằng sau SVM được giải thích tốt nhất bằng một số hình ảnh. Hình 5-1 cho thấy một phần của tập dữ liệu iris đã được giới thiệu ở cuối Chương 4. Hai lớp có thể được phân tách rõ ràng một cách dễ dàng bằng một đường thẳng (chúng có thể phân tách tuyến tính). Biểu đồ bên trái cho thấy các đường biên quyết định của ba bộ phân loại tuyến tính có thể có. Mô hình có đường biên quyết định được biểu thị bằng đường nét đứt tệ đến mức nó thậm chí không phân tách các lớp một cách đúng đắn. Hai mô hình còn lại hoạt động hoàn hảo trên tập huấn luyện này, nhưng các đường biên quyết định của chúng quá gần với các trường hợp khiến các mô hình này có thể không hoạt động tốt trên các trường hợp mới. Ngược lại, đường nét liền trong biểu đồ bên phải đại diện cho đường biên quyết định của một bộ phân loại SVM; đường này không chỉ phân tách hai lớp mà còn giữ khoảng cách xa nhất có thể so với các trường hợp huấn luyện gần nhất. Bạn có thể coi một bộ phân loại SVM như việc khớp một "con đường" rộng nhất có thể (được biểu thị bằng các đường nét đứt song song) giữa các lớp. Đây được gọi là phân loại biên lớn (large margin classification).



Hình 5-1. Phân loại biên lớn

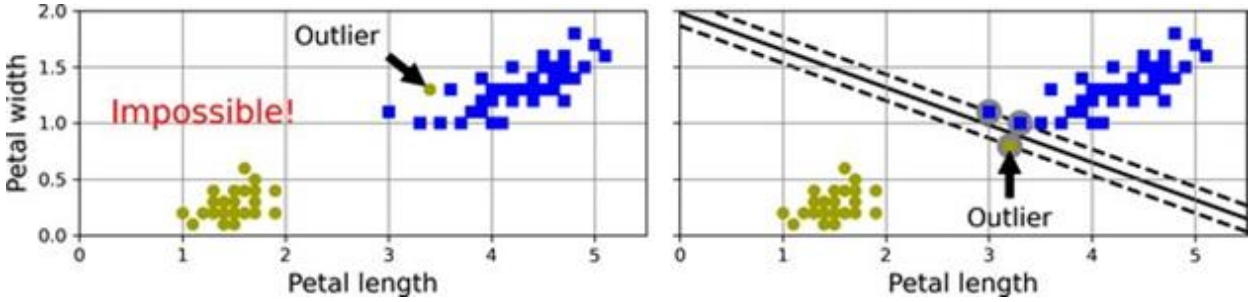
Lưu ý rằng việc thêm nhiều trường hợp huấn luyện "ngoài lề" sẽ không ảnh hưởng đến đường biên quyết định: nó hoàn toàn được xác định (hoặc "hỗ trợ") bởi các trường hợp nằm trên rìa của "con đường". Các trường hợp này được gọi là các véc-tơ hỗ trợ (chúng được khoanh tròn trong Hình 5-1).



Hình 5-2. Độ nhạy với thang đo đặc trưng

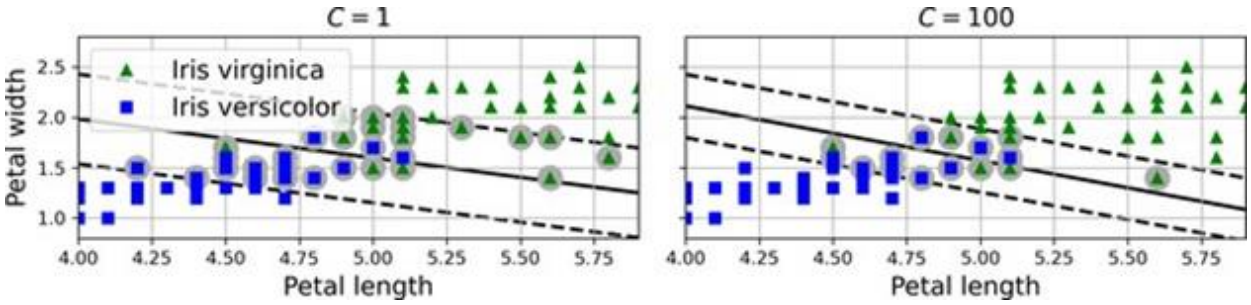
5.1.1 Phân loại biên mềm

Nếu chúng ta áp đặt nghiêm ngặt rằng tất cả các trường hợp phải nằm ngoài "con đường" và ở đúng phía, đây được gọi là phân loại biên cứng (hard margin classification). Có hai vấn đề chính với phân loại biên cứng. Thứ nhất, nó chỉ hoạt động nếu dữ liệu có thể phân tách tuyến tính. Thứ hai, nó nhạy cảm với các ngoại lệ. Hình 5-3 cho thấy tập dữ liệu iris chỉ với một ngoại lệ bổ sung: ở bên trái, không thể tìm thấy biên cứng; ở bên phải, đường biên quyết định rất khác so với đường chúng ta thấy trong Hình 5-1 mà không có ngoại lệ, và mô hình có thể sẽ không tổng quát hóa tốt.



Hình 5-3. Độ nhạy của biên cứng với các ngoại lệ

Để tránh những vấn đề này, chúng ta cần sử dụng một mô hình linh hoạt hơn. Mục tiêu là tìm sự cân bằng tốt giữa việc giữ "con đường" càng lớn càng tốt và hạn chế các vi phạm biên (tức là các trường hợp nằm ở giữa "con đường" hoặc thậm chí ở sai phía). Đây được gọi là phân loại biên mềm (soft margin classification). Khi tạo một mô hình SVM bằng Scikit-Learn, bạn có thể chỉ định một số siêu tham số, bao gồm siêu tham số chính quy hóa C. Nếu bạn đặt nó thành giá trị thấp, bạn sẽ nhận được mô hình ở bên trái Hình 5-4. Với giá trị cao, bạn sẽ nhận được mô hình ở bên phải. Như bạn có thể thấy, giảm C làm cho "con đường" lớn hơn, nhưng nó cũng dẫn đến nhiều vi phạm biên hơn. Nói cách khác, giảm C dẫn đến nhiều trường hợp hỗ trợ "con đường" hơn, do đó ít rủi ro về việc quá khớp (overfitting). Nhưng nếu bạn giảm nó quá nhiều, mô hình sẽ bị dưới khớp (underfitting), như trường hợp này: mô hình với C=100 dường như sẽ tổng quát hóa tốt hơn mô hình với C=1.



Hình 5-4. Biên lớn (trái) so với ít vi phạm biên hơn (phải)

Đoạn mã Scikit-Learn sau đây tải tập dữ liệu iris và huấn luyện một bộ phân loại SVM tuyến tính để phát hiện hoa Iris virginica. Pipeline đầu tiên chuẩn hóa các đặc trưng, sau đó sử dụng LinearSVC với C=1:

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

# Tải dữ liệu Iris
iris = load_iris(as_frame=True)

# Chọn đặc trưng và nhãn
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2)  # Nhãn cho Iris virginica

# Tạo pipeline với StandardScaler và LinearSVC
svm_clf = make_pipeline(
  StandardScaler(),
  LinearSVC(C=1, random_state=42)
)

# Huấn luyện mô hình
svm_clf.fit(X, y)
```

Mô hình thu được được biểu thị ở bên trái trong Hình 5-4. Sau đó, như thường lệ, bạn có thể sử dụng mô hình để đưa ra dự đoán:

X_new = [[5.5, 1.7], [5.0, 1.5]] svm_clf.predict(X_new) array([True, False])

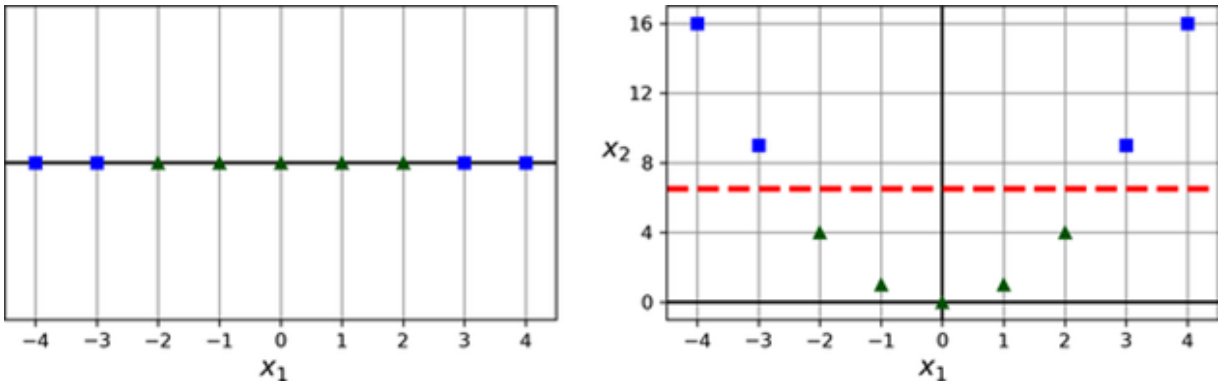
Cây đầu tiên được phân loại là Iris virginica, trong khi cây thứ hai thì không. Hãy xem xét các điểm mà SVM đã sử dụng để đưa ra các dự đoán này. Chúng đo khoảng cách có dấu giữa mỗi trường hợp và đường biên quyết định:

svm_clf.decision_function(X_new) array([0.66163411, -0.22036063])

Không giống như LogisticRegression, LinearSVC không có phương thức predict_proba() để ước tính xác suất lớp. Điều đó nói lên rằng, nếu bạn sử dụng lớp SVC (sẽ được thảo luận sau) thay vì LinearSVC, và nếu bạn đặt siêu tham số probability của nó thành True, thì mô hình sẽ khớp một mô hình bổ sung ở cuối quá trình huấn luyện để ánh xạ các điểm chức năng quyết định của SVM sang xác suất ước tính. Về cơ bản, điều này yêu cầu sử dụng xác thực chéo 5 lần để tạo ra các dự đoán ngoài mẫu cho mọi trường hợp trong tập huấn luyện, sau đó huấn luyện một mô hình LogisticRegression, do đó sẽ làm chậm quá trình huấn luyện đáng kể. Sau đó, các phương thức predict_proba() và predict_log_proba() sẽ có sẵn.

Phân loại SVM phi tuyến

Mặc dù các bộ phân loại SVM tuyến tính hiệu quả và thường hoạt động đáng ngạc nhiên, nhiều tập dữ liệu thậm chí còn không gần với việc có thể phân tách tuyến tính. Một cách tiếp cận để xử lý các tập dữ liệu phi tuyến là thêm nhiều đặc trưng hơn, chẳng hạn như các đặc trưng đa thức (như chúng ta đã làm trong Chương 4); trong một số trường hợp, điều này có thể dẫn đến một tập dữ liệu có thể phân tách tuyến tính. Hãy xem xét biểu đồ bên trái trong Hình 5-5: nó biểu thị một tập dữ liệu đơn giản chỉ với một đặc trưng, x1. Tập dữ liệu này không thể phân tách tuyến tính, như bạn có thể thấy. Nhưng nếu bạn thêm một đặc trưng thứ hai x2 = (x1)2, tập dữ liệu 2D thu được hoàn toàn có thể phân tách tuyến tính.

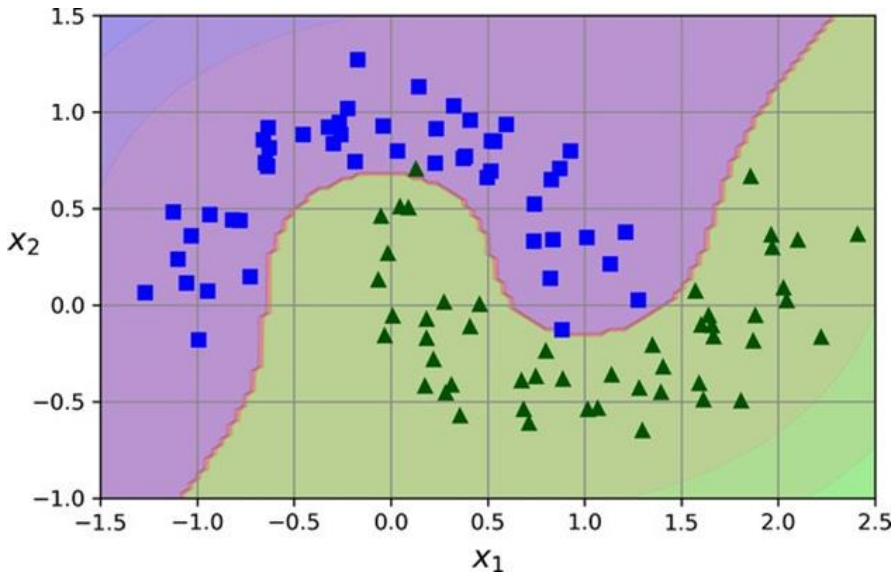


Hình 5-5. Thêm đặc trưng để làm cho một tập dữ liệu có thể phân tách tuyến tính

Để triển khai ý tưởng này bằng Scikit-Learn, bạn có thể tạo một pipeline chứa bộ biến đổi PolynomialFeatures (được thảo luận trong “Hồi quy đa thức”), theo sau là StandardScaler và một bộ phân loại LinearSVC. Hãy thử nghiệm điều này trên tập dữ liệu moons, một tập dữ liệu đồ chơi để phân loại nhị phân trong đó các điểm dữ liệu có hình dạng hai lưỡi liềm xen kẽ (xem Hình 5-6). Bạn có thể tạo tập dữ liệu này bằng cách sử dụng hàm make_moons():

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline
```

```
# Tạo dữ liệu dạng hình mặt trăng
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

# Tạo pipeline: thêm đặc trưng đa thức bậc 3, chuẩn hóa, sau đó huấn luyện SVM tuyến tính
polynomial_svm_clf = make_pipeline(
  PolynomialFeatures(degree=3),
  StandardScaler(),
  LinearSVC(C=10, max_iter=10_000, random_state=42)
)

# Huấn luyện mô hình
polynomial_svm_clf.fit(X, y)
```



Hình 5-6. Bộ phân loại SVM tuyến tính sử dụng các đặc trưng đa thức

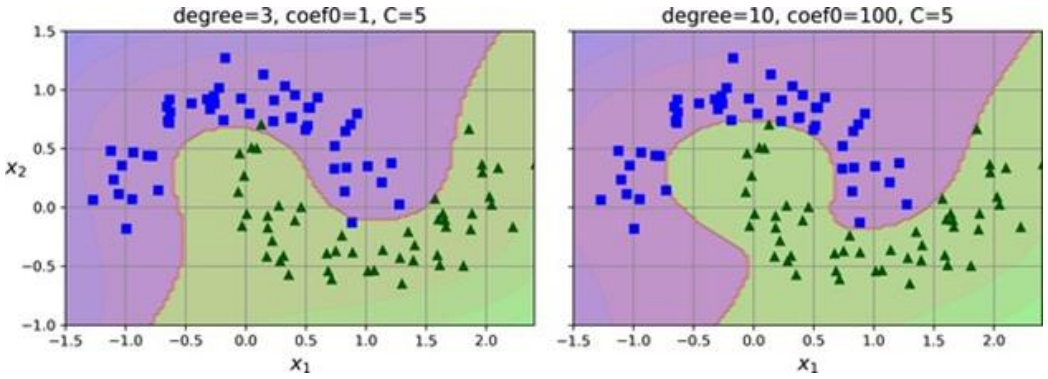
Kernel đa thức

Việc thêm các đặc trưng đa thức rất đơn giản để triển khai và có thể hoạt động tuyệt vời với tất cả các loại thuật toán học máy (không chỉ SVM). Điều đó nói lên rằng, ở bậc đa thức thấp, phương pháp này không thể xử lý các tập dữ liệu rất phức tạp, và ở bậc đa thức cao, nó tạo ra một số lượng lớn các đặc trưng, làm cho mô hình quá chậm. May mắn thay, khi sử dụng SVM, bạn có thể áp dụng một kỹ thuật toán học gần như kỳ diệu được gọi là kernel trick (sẽ được giải thích sau trong chương này). Kernel trick giúp có thể đạt được kết quả tương tự như khi bạn đã thêm nhiều đặc trưng đa thức, ngay cả với bậc rất cao, mà không thực sự phải thêm chúng. Điều này có nghĩa là không có sự bùng nổ tổ hợp về số lượng đặc trưng. Thủ thuật này được triển khai bởi lớp SVC. Hãy thử nghiệm nó trên tập dữ liệu moons:

SVC

poly_kernel_svm_clf = make_pipeline(StandardScaler(), SVC(kernel="poly",degree=3, coef0=1, C=5)) poly_kernel_svm_clf.fit(X, y)

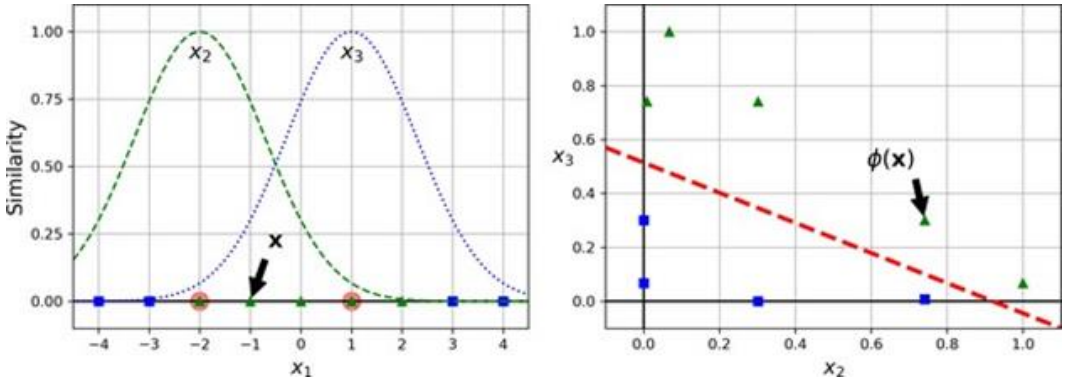
Đoạn mã này huấn luyện một bộ phân loại SVM sử dụng kernel đa thức bậc ba, được biểu thị ở bên trái trong Hình 5-7. Ở bên phải là một bộ phân loại SVM khác sử dụng kernel đa thức bậc 10. Rõ ràng, nếu mô hình của bạn đang bị quá khớp, bạn có thể muốn giảm bậc đa thức. Ngược lại, nếu nó bị dưới khớp, bạn có thể thử tăng nó. Siêu tham số coef0 kiểm soát mức độ ảnh hưởng của các số hạng bậc cao so với các số hạng bậc thấp đến mô hình.



Hình 5-7. Các bộ phân loại SVM với kernel đa thức

Các đặc trưng tương tự

Một kỹ thuật khác để giải quyết các vấn đề phi tuyến là thêm các đặc trưng được tính toán bằng hàm tương tự (similarity function), đo lường mức độ mỗi trường hợp giống một điểm mốc cụ thể, như chúng ta đã làm trong Chương 2 khi thêm các đặc trưng tương tự địa lý. Ví dụ, hãy lấy tập dữ liệu 1D từ trước và thêm hai điểm mốc vào đó tại x1 = –2 và x1 = 1 (xem biểu đồ bên trái trong Hình 5-8). Tiếp theo, chúng ta sẽ định nghĩa hàm tương tự là Gaussian RBF với γ = 0.3. Đây là một hàm hình chuông thay đổi từ 0 (rất xa điểm mốc) đến 1 (tại điểm mốc). Bây giờ chúng ta đã sẵn sàng để tính toán các đặc trưng mới. Ví dụ, hãy xem xét trường hợp x1 = –1: nó nằm ở khoảng cách 1 từ điểm mốc đầu tiên và 2 từ điểm mốc thứ hai. Do đó, các đặc trưng mới của nó là x2 = exp(–0.3 × 12) ≈ 0.74 và x3 = exp(–0.3 × 22) ≈ 0.30. Biểu đồ bên phải trong Hình 5-8 cho thấy tập dữ liệu đã được biến đổi (bỏ qua các đặc trưng gốc). Như bạn có thể thấy, nó hiện có thể phân tách tuyến tính.



Hình 5-8. Các đặc trưng tương tự sử dụng Gaussian RBF

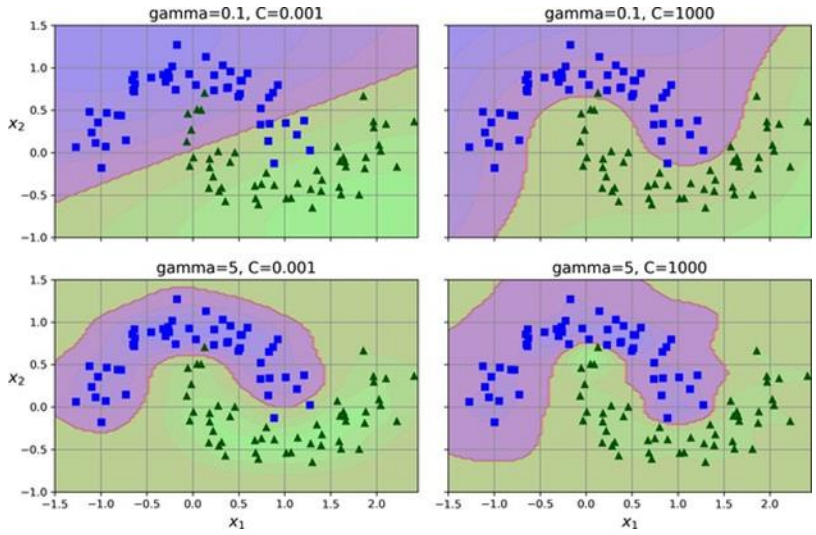
Bạn có thể tự hỏi làm thế nào để chọn các điểm mốc. Cách tiếp cận đơn giản nhất là tạo một điểm mốc tại vị trí của mỗi và mọi trường hợp trong tập dữ liệu. Làm như vậy tạo ra nhiều chiều và do đó tăng khả năng tập huấn luyện đã được biến đổi sẽ có thể phân tách tuyến tính. Nhược điểm là một tập huấn luyện với m trường hợp và n đặc trưng sẽ được biến đổi thành một tập huấn luyện với m trường hợp và m đặc trưng (giả sử bạn bỏ qua các đặc trưng gốc). Nếu tập huấn luyện của bạn rất lớn, bạn sẽ có một số lượng đặc trưng tương đương.

Kernel Gaussian RBF

Cũng giống như phương pháp đặc trưng đa thức, phương pháp đặc trưng tương tự có thể hữu ích với bất kỳ thuật toán học máy nào, nhưng việc tính toán tất cả các đặc trưng bổ sung có thể tốn kém về mặt tính toán (đặc biệt trên các tập huấn luyện lớn). Một lần nữa, kernel trick lại thực hiện phép thuật SVM của nó, giúp có thể đạt được kết quả tương tự như khi bạn đã thêm nhiều đặc trưng tương tự, nhưng không thực sự làm như vậy. Hãy thử lớp SVC với kernel Gaussian RBF:

rbf_kernel_svm_clf = make_pipeline(StandardScaler(), SVC(kernel="rbf",gamma=5, C=0.001)) rbf_kernel_svm_clf.fit(X, y)

Mô hình này được biểu thị ở phía dưới bên trái trong Hình 5-9. Các biểu đồ khác cho thấy các mô hình được huấn luyện với các giá trị khác nhau của các siêu tham số gamma (γ) và C. Tăng gamma làm cho đường cong hình chuông hẹp hơn (xem các biểu đồ bên trái trong Hình 5-8). Kết quả là, phạm vi ảnh hưởng của mỗi trường hợp nhỏ hơn: đường biên quyết định trở nên bất thường hơn, uốn lượn quanh từng trường hợp riêng lẻ. Ngược lại, giá trị gamma nhỏ làm cho đường cong hình chuông rộng hơn: các trường hợp có phạm vi ảnh hưởng lớn hơn, và đường biên quyết định trở nên mượt mà hơn. Vì vậy, γ hoạt động giống như một siêu tham số chính quy hóa: nếu mô hình của bạn bị quá khớp, bạn nên giảm γ; nếu nó bị dưới khớp, bạn nên tăng γ (tương tự như siêu tham số C).



Hình 5-9. Các bộ phân loại SVM sử dụng kernel RBF

Các kernel khác tồn tại nhưng được sử dụng ít thường xuyên hơn nhiều. Một số kernel được chuyên biệt cho các cấu trúc dữ liệu cụ thể. Các kernel chuỗi đôi khi được sử dụng khi phân loại tài liệu văn bản hoặc chuỗi DNA (ví dụ: sử dụng kernel chuỗi con hoặc các kernel dựa trên khoảng cách Levenshtein).

Các lớp SVM và độ phức tạp tính toán

Lớp LinearSVC dựa trên thư viện liblinear, triển khai một thuật toán tối ưu hóa cho SVM tuyến tính. Nó không hỗ trợ kernel trick, nhưng nó mở rộng gần như tuyến tính với số lượng trường hợp huấn luyện và số lượng đặc trưng. Độ phức tạp thời gian huấn luyện của nó xấp xỉ O(m × n). Thuật toán mất nhiều thời gian hơn nếu bạn yêu cầu độ chính xác rất cao. Điều này được kiểm soát bởi siêu tham số dung sai ϵ (được gọi là tol trong Scikit-Learn). Trong hầu hết các tác vụ phân loại, dung sai mặc định là tốt. Lớp SVC dựa trên thư viện libsvm, triển khai một thuật toán hỗ trợ kernel trick. Độ phức tạp thời gian huấn luyện thường nằm trong khoảng từ O(m2 × n) đến O(m3 × n). Thật không may, điều này có nghĩa là nó trở nên cực kỳ chậm khi số lượng trường hợp huấn luyện trở nên lớn (ví dụ: hàng trăm nghìn trường hợp), vì vậy thuật toán này tốt nhất cho các tập huấn luyện phi tuyến có kích thước nhỏ hoặc trung bình. Nó mở rộng tốt với số lượng đặc trưng, đặc biệt với các đặc trưng thưa thớt (tức là khi mỗi trường hợp có ít đặc trưng khác không). Trong trường hợp này, thuật toán mở rộng xấp xỉ với số lượng trung bình các đặc trưng khác không trên mỗi trường hợp. Lớp SGDClassifier cũng thực hiện phân loại biên lớn theo mặc định, và các siêu tham số của nó – đặc biệt là các siêu tham số chính quy hóa (alpha và penalty) và learning_rate – có thể được điều chỉnh để tạo ra kết quả tương tự như SVM tuyến tính. Để huấn luyện, nó sử dụng giảm độ dốc ngẫu nhiên (xem Chương 4), cho phép học tăng dần và sử dụng ít bộ nhớ, vì vậy bạn có thể sử dụng nó để huấn luyện một mô hình trên một tập dữ liệu lớn không phù hợp với RAM (tức là cho học ngoài lõi). Hơn nữa, nó mở rộng rất tốt, vì độ phức tạp tính toán của nó là O(m × n). Bảng 5-1 so sánh các lớp phân loại SVM của Scikit-Learn.

Bảng 5-1. So sánh các lớp Scikit-Learn cho phân loại SVM

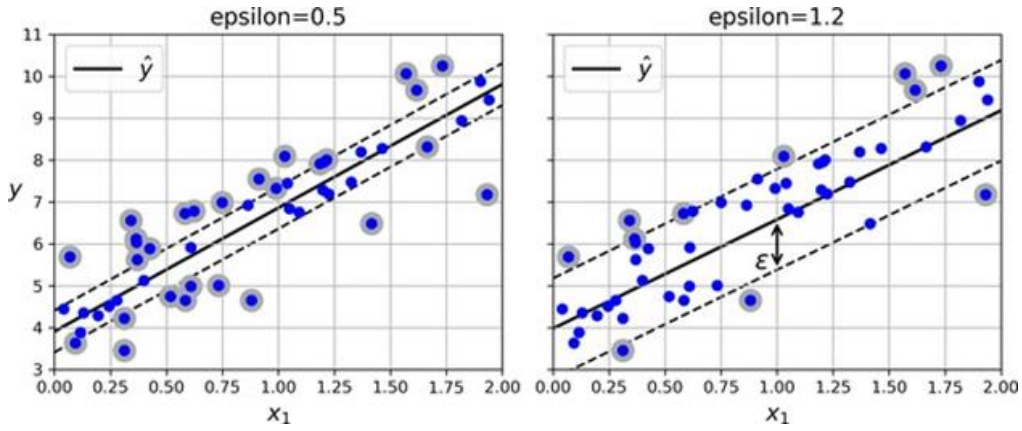
Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	No	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes
SGDClassifier	$O(m \times n)$	Yes	Yes	No

Bây giờ chúng ta hãy xem các thuật toán SVM cũng có thể được sử dụng cho hồi quy tuyến tính và phi tuyến như thế nào.

Hồi quy SVM

Để sử dụng SVM cho hồi quy thay vì phân loại, mẹo là điều chỉnh mục tiêu: thay vì cố gắng khớp "con đường" lớn nhất có thể giữa hai lớp trong khi hạn chế vi phạm biên, hồi quy SVM cố gắng khớp càng nhiều trường hợp càng tốt trên "con đường" trong khi hạn chế vi phạm biên (tức là các trường hợp nằm ngoài "con đường"). Chiều rộng của "con đường" được kiểm soát bởi một siêu tham số, ϵ. Hình 5-10 cho thấy hai mô hình hồi quy SVM tuyến tính

được huấn luyện trên một số dữ liệu tuyến tính, một với biên nhỏ (ϵ= 0.5) và một với biên lớn hơn (ϵ = 1.2).



Hình 5-10. Hồi quy SVM

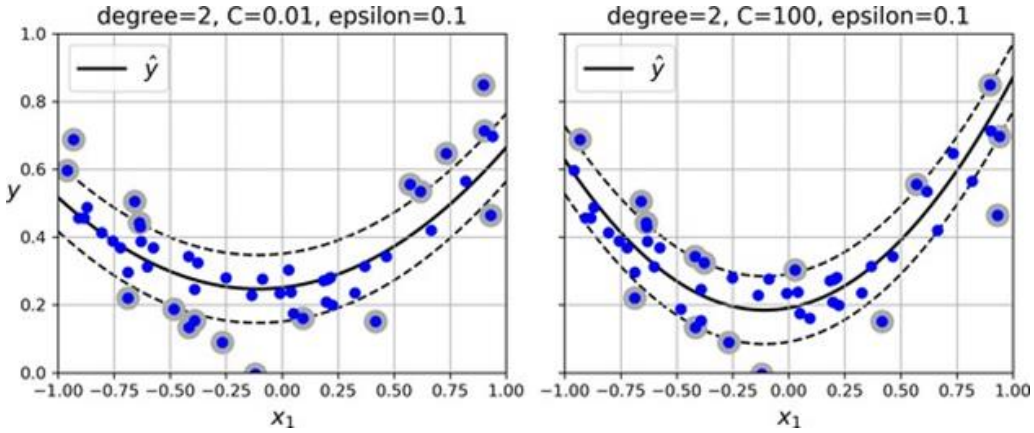
Giảm ϵ làm tăng số lượng véc-tơ hỗ trợ, điều này chuẩn hóa mô hình. Hơn nữa, nếu bạn thêm nhiều trường hợp huấn luyện hơn trong biên, nó sẽ không ảnh hưởng đến dự đoán của mô hình; do đó, mô hình được gọi là không nhạy cảm với ϵ.

Bạn có thể sử dụng lớp LinearSVR của Scikit-Learn để thực hiện hồi quy SVM tuyến tính. Đoạn mã sau đây tạo ra mô hình được biểu thị ở bên trái trong Hình 5-10:

LinearSVR

X, y = [...] # một tập dữ liệu tuyến tính svm_reg = make_pipeline(StandardScaler(), LinearSVR(epsilon=0.5, random_state=42)) svm_reg.fit(X, y)

Để giải quyết các tác vụ hồi quy phi tuyến, bạn có thể sử dụng mô hình SVM với kernel. Hình 5-11 cho thấy hồi quy SVM trên một tập huấn luyện bậc hai ngẫu nhiên, sử dụng kernel đa thức bậc hai. Có một số chính quy hóa trong biểu đồ bên trái (tức là giá trị C nhỏ), và ít hơn nhiều ở biểu đồ bên phải (tức là giá trị C lớn).



Hình 5-11. Hồi quy SVM sử dụng kernel đa thức bậc hai

Đoạn mã sau đây sử dụng lớp SVR của Scikit-Learn (hỗ trợ kernel trick) để tạo ra mô hình được biểu thị ở bên trái trong Hình 5-11:

SVR

X, y = [...] # một tập dữ liệu bậc hai svm_poly_reg = make_pipeline(StandardScaler(), SVR(kernel="poly",degree=2, C=0.01, epsilon=0.1)) svm_poly_reg.fit(X, y)

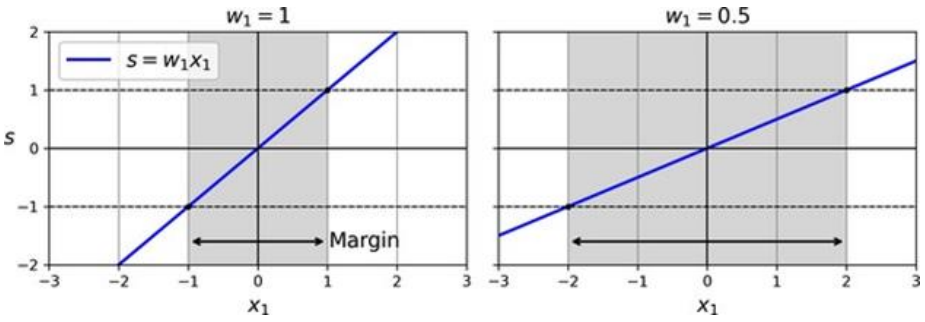
Lớp SVR là tương đương hồi quy của lớp SVC, và lớp LinearSVR là tương đương hồi quy của lớp LinearSVC. Lớp LinearSVR mở rộng tuyến tính với kích thước của tập huấn luyện (giống như lớp LinearSVC), trong khi lớp SVR trở nên quá chậm khi tập huấn luyện trở nên rất lớn (giống như lớp SVC).

Phần còn lại của chương này giải thích cách SVM đưa ra dự đoán và cách các thuật toán huấn luyện của chúng hoạt động, bắt đầu với các bộ phân loại SVM tuyến tính. Nếu bạn mới bắt đầu với học máy, bạn có thể bỏ qua phần này và đi thẳng đến các bài tập ở cuối chương này, và quay lại sau khi bạn muốn hiểu sâu hơn về SVM.

Bên trong các bộ phân loại SVM tuyến tính

Bộ phân loại SVM tuyến tính dự đoán lớp của một trường hợp mới x bằng cách đầu tiên tính toán hàm quyết định $\theta^T x = \theta_0 x_0 + \cdots + \theta n xn$, trong đó x_0 là đặc trưng bias (luôn bằng 1). Nếu kết quả là dương, thì lớp dự đoán ŷ là lớp dương (1); nếu không thì là lớp âm (0). Điều này giống hệt như Hồi quy Logistic (được thảo luận trong Chương 4).

Dự đoán bằng bộ phân loại SVM tuyến tính khá đơn giản. Vậy còn việc huấn luyện thì sao? Điều này đòi hỏi phải tìm vector trọng số w và số hạng bias b sao cho "con đường", hay biên, rộng nhất có thể trong khi hạn chế số lượng vi phạm biên. Hãy bắt đầu với chiều rộng của "con đường": để làm cho nó lớn hơn, chúng ta cần làm cho w nhỏ hơn. Điều này có thể dễ hình dung hơn trong không gian 2D, như trong Hình 5-12. Hãy định nghĩa các đường biên của "con đường" là các điểm mà hàm quyết định bằng -1 hoặc +1. Trong biểu đồ bên trái, trọng số w_1 là 1, vì vậy các điểm mà $w_1 x_1 = -1$ hoặc +1 là $x_1 = -1$ và +1: do đó kích thước của biên là 2. Trong biểu đồ bên phải, trọng số là 0.5, vì vậy các điểm mà $w_1 x_1 = -1$ hoặc +1 là $x_1 = -2$ và +2: kích thước của biên là 4. Vì vậy, chúng ta cần giữ w càng nhỏ càng tốt. Lưu ý rằng số hạng bias b không ảnh hưởng đến kích thước của biên: việc điều chỉnh nó chỉ làm dịch chuyển biên mà không ảnh hưởng đến kích thước của nó.



Hình 5-12. Một vector trọng số nhỏ hơn dẫn đến một biên lớn hơn

Chúng ta cũng muốn tránh các vi phạm biên, vì vậy chúng ta cần hàm quyết định lớn hơn 1 cho tất cả các trường hợp huấn luyện dương và nhỏ hơn -1 cho các trường hợp huấn luyện âm. Nếu chúng ta định nghĩa $t^{(i)} = -1$ cho các trường hợp âm (khi $y^{(i)} = 0$) và $t^{(i)} = 1$ cho các trường hợp dương (khi $y^{(i)} = 1$), thì chúng ta có thể viết ràng buộc này là $t^{(i)}\left(w^T x^{(i)} + b\right) \geq 1$ cho tất cả các trường hợp. Do đó, chúng ta có thể biểu diễn mục tiêu của bộ phân loại SVM tuyến tính biên cứng dưới dạng bài toán tối ưu hóa có ràng buộc trong Phương trình 5-1. Phương trình 5-1. Mục tiêu của bộ phân loại SVM tuyến tính biên cứng tối thiểu hóa w, b

$$\text{minimize}_{w,b} \frac{1}{2} w^\intercal w$$

$$\text{subject to} \quad t^{(i)}\left(w^\intercal x^{(i)} + b\right) \geq 1 \; với \; i = 1,2, \dots, m$$

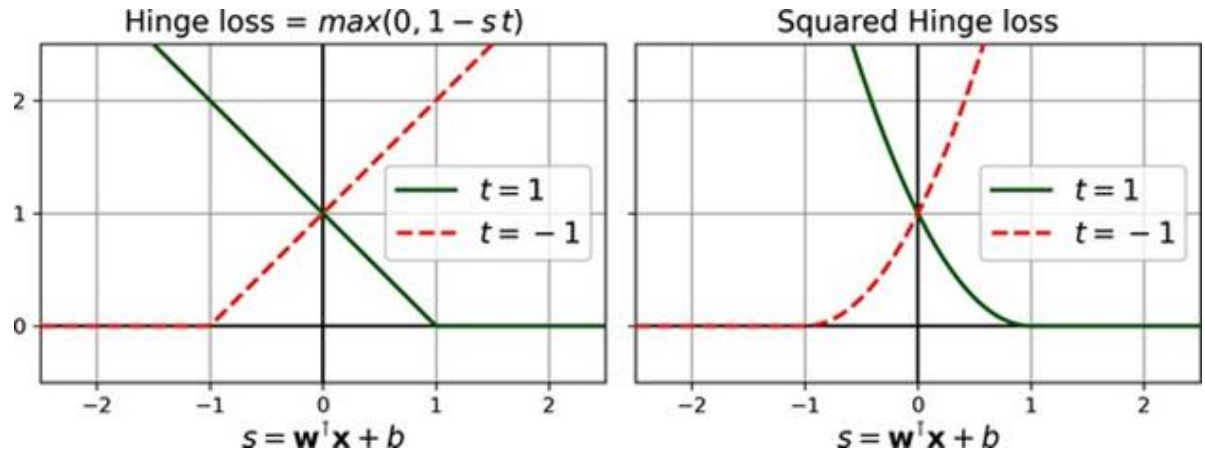
Để có được mục tiêu biên mềm, chúng ta cần đưa vào một biến trượt $\zeta^{(i)} \geq 0$ cho mỗi trường hợp: $\zeta^{(i)}$ đo lường mức độ trường hợp thứ i được phép vi phạm biên. Bây giờ chúng ta có hai mục tiêu xung đột: làm cho các biến trượt càng nhỏ càng tốt để giảm vi phạm biên, và làm cho $\frac{1}{2} w^T w$ càng nhỏ càng tốt để tăng biên. Đây là lúc siêu tham số C xuất hiện: nó cho phép chúng ta định nghĩa sự cân bằng giữa hai mục tiêu này. Điều này cho chúng ta bài toán tối ưu hóa có ràng buộc trong Phương trình 5-2. Phương trình 5-2. Mục tiêu của bộ phân loại SVM tuyến tính biên mềm tối thiểu hóa w, b, ζ

$$\frac{1}{2} w^T w + C \sum_{i=1}^{m} \zeta^{(i)}$$

$$\text{thỏa mãn } t^{(i)}\left(w^T x^{(i)} + b\right) \geq 1 - \zeta^{(i)} \text{ và } \zeta^{(i)} \geq 0 \text{ cho } i = 1,2, \dots, m$$

Cả bài toán biên cứng và biên mềm đều là các bài toán tối ưu hóa bậc hai lồi với các ràng buộc tuyến tính. Các bài toán như vậy được gọi là bài toán quy hoạch bậc hai (QP). Nhiều bộ giải có sẵn để giải các bài toán QP bằng cách sử dụng nhiều kỹ thuật khác nhau nằm ngoài phạm vi của cuốn sách này. Sử dụng bộ giải QP là một cách để huấn luyện SVM. Một cách khác là sử dụng giảm độ dốc để tối thiểu hóa hàm mất mát bản lề (hinge loss) hoặc hàm mất mát bản lề bình phương (squared hinge loss) (xem Hình 5-13). Với một trường hợp x của lớp dương (tức là với $t = 1$), hàm mất mát là 0 nếu đầu ra s của hàm quyết định ($s = w^T x + b$) lớn hơn hoặc bằng 1. Điều này xảy ra khi trường hợp nằm ngoài "con đường" và ở phía dương. Với một trường hợp của lớp âm (tức là với $t = -1$), hàm mất mát là 0 nếu $s \leq -1$. Điều này xảy ra khi trường hợp nằm ngoài "con đường" và ở phía âm. Trường hợp càng xa phía đúng của biên thì hàm mất mát càng cao: nó tăng tuyến tính đối với hàm mất mát bản lề, và bậc hai đối với hàm mất mát bản lề bình phương. Điều này làm cho hàm mất mát bản lề bình phương nhạy cảm hơn với các ngoại lệ. Tuy nhiên, nếu tập dữ liệu sạch, nó có xu hướng hội tụ nhanh hơn. Theo mặc định, LinearSVC sử dụng hàm mất mát bản lề bình phương, trong khi SGDClassifier sử dụng hàm mất mát bản lề. Cả hai lớp đều cho phép bạn chọn hàm mất mát bằng cách đặt siêu tham số loss thành "hinge" hoặc "squared_hinge".

Thuật toán tối ưu hóa của lớp SVC tìm kiếm một giải pháp tương tự như việc tối thiểu hóa hàm mất mát bản lề.



Hình 5-13. Hàm mất mát bản lề (trái) và hàm mất mát bản lề bình phương (phải)

Tiếp theo, chúng ta sẽ xem xét một cách khác để huấn luyện bộ phân loại SVM tuyến tính: giải quyết bài toán đối ngẫu.

Bài toán đối ngẫu

Cho một bài toán tối ưu hóa có ràng buộc, được gọi là bài toán gốc (primal problem), có thể biểu diễn một bài toán khác nhưng có liên quan chặt chẽ, được gọi là bài toán đối ngẫu của nó. Giải pháp cho bài toán đối ngẫu thường đưa ra một cận dưới cho giải pháp của bài toán gốc, nhưng trong một số điều kiện, nó có thể có cùng giải pháp với bài toán gốc. May mắn thay, bài toán SVM đáp ứng các điều kiện này, vì vậy bạn có thể chọn giải bài toán gốc hoặc bài toán đối ngẫu; cả hai sẽ có cùng giải pháp.

Phương trình 5-3 cho thấy dạng đối ngẫu của mục tiêu SVM tuyến tính. Nếu bạn muốn biết cách dẫn xuất bài toán đối ngẫu từ bài toán gốc, hãy xem phần tài liệu bổ sung trong sổ tay của chương này. Phương trình 5-3. Dạng đối ngẫu của mục tiêu SVM tuyến tính tối thiểu hóa α

$$\underset{\alpha}{\text{minimize}} \ \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)\intercal}\mathbf{x}^{(j)} \quad - \quad \sum_{i=1}^{m}\alpha^{(i)} \text{subject to } \alpha^{(i)} \geq 0$$

$$\text{cho tất cả } i = 1, \ldots, m$$

Một khi bạn tìm thấy vector $\hat{\alpha}$ tối thiểu hóa phương trình này (sử dụng bộ giải QP), hãy sử dụng Phương trình 5-4 để tính toán $\hat{w}$ và $\hat{b}$ tối thiểu hóa bài toán gốc. Trong phương trình này, n_s biểu thị số lượng véc-tơ hỗ trợ. Phương trình 5-4. Từ giải pháp đối ngẫu đến giải pháp gốc

$$w = \sum_{i=1}^{m} \alpha^{(i)} t^{(i)} x^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\alpha^{(i)}>0}^{m} \left(t^{(i)} - \hat{w}^T x^{(i)} \right)$$

Bài toán đối ngẫu nhanh hơn để giải hơn bài toán gốc khi số lượng trường hợp huấn luyện nhỏ hơn số lượng đặc trưng. Quan trọng hơn, bài toán đối ngẫu giúp kernel trick trở nên khả thi, trong khi bài toán gốc thì không. Vậy kernel trick là gì?

SVM sử dụng Kernel (Kernelized SVMs)

Giả sử bạn muốn áp dụng phép biến đổi đa thức bậc hai lên một tập huấn luyện hai chiều, sau đó huấn luyện một bộ phân loại SVM tuyến tính trên tập dữ liệu đã được biến đổi. **Công thức 5-5** trình bày hàm ánh xạ đa thức bậc hai φ mà bạn muốn áp dụng.

Công thức 5-5: Ánh xạ đa thức bậc hai

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} {x_1}^2 \\ \sqrt{2}\, x_1 x_2 \\ {x_2}^2 \end{pmatrix}$$

Lưu ý rằng vector đã biến đổi có 3 chiều thay vì 2. Bây giờ, hãy xem điều gì sẽ xảy ra với một cặp vector 2D, a và b , nếu chúng ta áp dụng ánh xạ đa thức bậc hai này rồi tính tích vô hướng của các vector đã biến đổi.

Công thức 5-6: Kỹ thuật Kernel cho ánh xạ đa thức bậc hai

$$\begin{aligned} \phi(\mathbf{a})^\intercal \phi(\mathbf{b}) &= \begin{pmatrix} {a_1}^2 \\ \sqrt{2}\, a_1 a_2 \\ {a_2}^2 \end{pmatrix}^\intercal \begin{pmatrix} {b_1}^2 \\ \sqrt{2}\, b_1 b_2 \\ {b_2}^2 \end{pmatrix} = {a_1}^2 {b_1}^2 + 2 a_1 b_1 a_2 b_2 + {a_2}^2 {b_2}^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\intercal \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^\intercal \mathbf{b})^2 \end{aligned}$$

Tích vô hướng của các vector đã biến đổi bằng với bình phương của tích vô hướng của các vector ban đầu. Đây là mấu chốt quan trọng: nếu bạn áp dụng phép biến đổi φ cho tất cả các trường hợp huấn luyện, thì bài toán đối ngẫu sẽ chứa tích vô hướng $\varphi\left(x^{(i)}\right)^\intercal \varphi\left(x^{(j)}\right)$. Nhưng nếu φ là phép biến đổi đa thức bậc hai được định nghĩa trong Công thức 5-5, thì bạn có thể thay thế tích vô hướng của các vector đã biến đổi bằng $\left(x^{(i)\intercal} x^{(j)}\right)^2$. Vì vậy, bạn không cần phải biến đổi tập huấn luyện; chỉ cần thay thế tích vô hướng bằng bình phương của nó. Kết quả sẽ giống hệt như khi bạn đã thực hiện toàn bộ quá trình biến đổi tập dữ liệu

và sau đó huấn luyện thuật toán SVM tuyến tính, nhưng cách này hiệu quả hơn về mặt tính toán.

Trong học máy, một hàm có khả năng tính toán tích vô hướng $\varphi(a)^{\intercal}\varphi(b)$, chỉ dựa trên các vector gốc a và b mà không cần phải tính toán phép biến đổi φ , được gọi là **kernel**. **Công thức 5-7** liệt kê một số kernel thường được sử dụng nhất.

Công thức 5-7: Các Kernel thông dụng

- **Tuyến tính:** $K(a,b) = a^{\intercal}b$
- **Đa thức:** $K(a,b) = (\gamma a^{\intercal}b + r)^{d}$
- **RBF Gauss:** $K(a,b) = \exp(-\gamma \parallel a - b \parallel^{2})$
- **Sigmoid:** $K(a,b) = \tanh(\gamma a^{\intercal}b + r)$

Vẫn còn một vấn đề cần giải quyết. **Công thức 5-4** cho thấy cách chuyển từ nghiệm đối ngẫu sang nghiệm sơ cấp trong trường hợp bộ phân loại SVM tuyến tính. Nhưng nếu bạn áp dụng kỹ thuật kernel, bạn sẽ có các phương trình bao gồm $\varphi\left(x^{(i)}\right)$. Trên thực tế, $w^$ phải có cùng số chiều với $\varphi\left(x^{(i)}\right)$, có thể rất lớn hoặc thậm chí vô hạn, nên bạn không thể tính toán nó. Nhưng làm thế nào bạn có thể đưa ra dự đoán mà không biết $w^$? Tin tốt là bạn có thể thay công thức của $w^$ từ Công thức 5-4 vào hàm quyết định cho một trường hợp mới $x^{(n)}$, và bạn sẽ có một phương trình chỉ chứa các tích vô hướng giữa các vector đầu vào. Điều này giúp bạn có thể sử dụng kỹ thuật kernel.

Công thức 5-8: Đưa ra dự đoán bằng Kernelized SVM

$$
\begin{aligned}
h_{\hat{\mathbf{w}},\hat{b}}\left(\phi\left(\mathbf{x}^{(n)}\right)\right) &= \hat{\mathbf{w}}^{\intercal}\phi\left(\mathbf{x}^{(n)}\right) + \hat{b} = \left(\sum_{i=1}^{m}\widehat{\alpha}^{(i)}t^{(i)}\phi\left(\mathbf{x}^{(i)}\right)\right)^{\intercal}\phi\left(\mathbf{x}^{(n)}\right) + \hat{b} \\
&= \sum_{i=1}^{m}\widehat{\alpha}^{(i)}t^{(i)}\left(\phi\left(\mathbf{x}^{(i)}\right)^{\intercal}\phi\left(\mathbf{x}^{(n)}\right)\right) + \hat{b} \\
&= \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)}>0}}^{m}\widehat{\alpha}^{(i)}t^{(i)}K\left(\mathbf{x}^{(i)},\mathbf{x}^{(n)}\right) + \hat{b}
\end{aligned}
$$

Lưu ý rằng vì $\alpha^{(i)} \neq 0$ chỉ đối với các vector hỗ trợ, việc đưa ra dự đoán chỉ liên quan đến việc tính tích vô hướng của vector đầu vào mới $x^{(n)}$ với các vector hỗ trợ, chứ không phải với tất cả các trường hợp huấn luyện. Tất nhiên, bạn cũng phải sử dụng cùng một kỹ thuật để tính số hạng độ lệch $b^$.

Công thức 5-9: Sử dụng kỹ thuật Kernel để tính số hạng độ lệch

$$\hat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)}>0}}^{m} \left(t^{(i)} - \widehat{\mathbf{w}}^{\mathsf{T}} \phi\left(\mathbf{x}^{(i)}\right)\right) = \frac{1}{n_s} \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)}>0}}^{m} \left(t^{(i)} - \left(\sum_{j=1}^{m} \widehat{\alpha}^{(j)} t^{(j)} \phi\left(\mathbf{x}^{(j)}\right)\right)^{\mathsf{T}} \phi\left(\mathbf{x}^{(i)}\right)\right)$$

$$= \frac{1}{n_s} \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)}>0}}^{m} \left(t^{(i)} - \sum_{\substack{j=1 \\ \widehat{\alpha}^{(j)}>0}}^{m} \widehat{\alpha}^{(j)} t^{(j)} K\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)\right)$$

Nếu bạn cảm thấy đau đầu, điều đó hoàn toàn bình thường: đây là một tác dụng phụ không may của kỹ thuật kernel.

Bài tập

1. Ý tưởng cơ bản đằng sau máy học véc-tơ hỗ trợ là gì?
2. Véc-tơ hỗ trợ là gì?
3. Tại sao việc chuẩn hóa đầu vào lại quan trọng khi sử dụng SVM?
4. Một bộ phân loại SVM có thể đưa ra điểm tin cậy khi phân loại một trường hợp không? Còn xác suất thì sao?
5. Làm thế nào để bạn chọn giữa LinearSVC, SVC và SGDClassifier?
6. Giả sử bạn đã huấn luyện một bộ phân loại SVM với kernel RBF, nhưng nó dường như bị dưới khớp trên tập huấn luyện. Bạn nên tăng hay giảm γ (gamma)? Còn C thì sao?
7. Một mô hình được gọi là không nhạy cảm với ϵ có nghĩa là gì?
8. Mục đích của việc sử dụng kernel trick là gì?
9. Huấn luyện một LinearSVC trên một tập dữ liệu có thể phân tách tuyến tính. Sau đó huấn luyện một SVC và một SGDClassifier trên cùng tập dữ liệu đó. Xem liệu bạn có thể làm cho chúng tạo ra cùng một mô hình hay không.
10. Huấn luyện một bộ phân loại SVM trên tập dữ liệu wine, bạn có thể tải bằng cách sử dụng sklearn.datasets.load_wine(). Tập dữ liệu này chứa các phân tích hóa học của 178 mẫu rượu được sản xuất bởi 3 nhà sản xuất khác nhau: mục tiêu là huấn luyện một mô hình phân loại có khả năng dự đoán nhà sản xuất dựa trên phân tích hóa học của rượu. Vì bộ phân loại SVM là bộ phân loại nhị phân, bạn sẽ cần sử dụng chiến lược "một đối tất cả" (one-versus-all) để phân loại cả ba lớp. Bạn có thể đạt được độ chính xác bao nhiêu?
11. Huấn luyện và tinh chỉnh một hồi quy SVM trên tập dữ liệu California housing. Bạn có thể sử dụng tập dữ liệu gốc thay vì phiên bản đã được điều chỉnh mà chúng ta đã sử dụng trong Chương 2, bạn có thể tải bằng cách sử dụng sklearn.datasets.fetch_california_housing(). Các mục tiêu đại diện cho hàng trăm nghìn đô la. Vì có hơn 20.000 trường hợp, SVM có thể chậm, vì vậy để tinh chỉnh siêu tham số, bạn nên sử dụng ít trường hợp hơn (ví dụ: 2.000) để kiểm tra nhiều kết hợp siêu tham số hơn. RMSE của mô hình tốt nhất của bạn là bao nhiêu? Các giải pháp

cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 6. CÂY QUYẾT ĐỊNH

Cây quyết định là các thuật toán học máy linh hoạt có thể thực hiện cả tác vụ phân loại và hồi quy, và thậm chí cả các tác vụ đa đầu ra. Chúng là các thuật toán mạnh mẽ, có khả năng khớp các tập dữ liệu phức tạp. Ví dụ, trong Chương 2 bạn đã huấn luyện một mô hình DecisionTreeRegressor trên tập dữ liệu California housing, khớp nó một cách hoàn hảo (thực ra, quá khớp nó). Cây quyết định cũng là các thành phần cơ bản của rừng ngẫu nhiên (xem Chương 7), là một trong những thuật toán học máy mạnh mẽ nhất hiện có. Trong chương này, chúng ta sẽ bắt đầu bằng cách thảo luận về cách huấn luyện, trực quan hóa và đưa ra dự đoán bằng cây quyết định. Sau đó, chúng ta sẽ đi qua thuật toán huấn luyện CART được sử dụng bởi Scikit-Learn, và chúng ta sẽ khám phá cách chính quy hóa cây và sử dụng chúng cho các tác vụ hồi quy. Cuối cùng, chúng ta sẽ thảo luận về một số hạn chế của cây quyết định.

Huấn luyện và trực quan hóa cây quyết định

Để hiểu cây quyết định, hãy xây dựng một cây và xem cách nó đưa ra dự đoán. Đoạn mã sau đây huấn luyện một DecisionTreeClassifier trên tập dữ liệu iris (xem Chương 4):

sklearn.datasets import load_iris sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True) X_iris = iris.data[[“petal length (cm)”, “petal width (cm)”]].values y_iris = iris.target

tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42) tree_clf.fit(X_iris, y_iris)

Bạn có thể trực quan hóa cây quyết định đã được huấn luyện bằng cách trước tiên sử dụng hàm export_graphviz() để xuất một tệp định nghĩa đồ thị có tên iris_tree.dot:

from sklearn.tree import export_graphviz

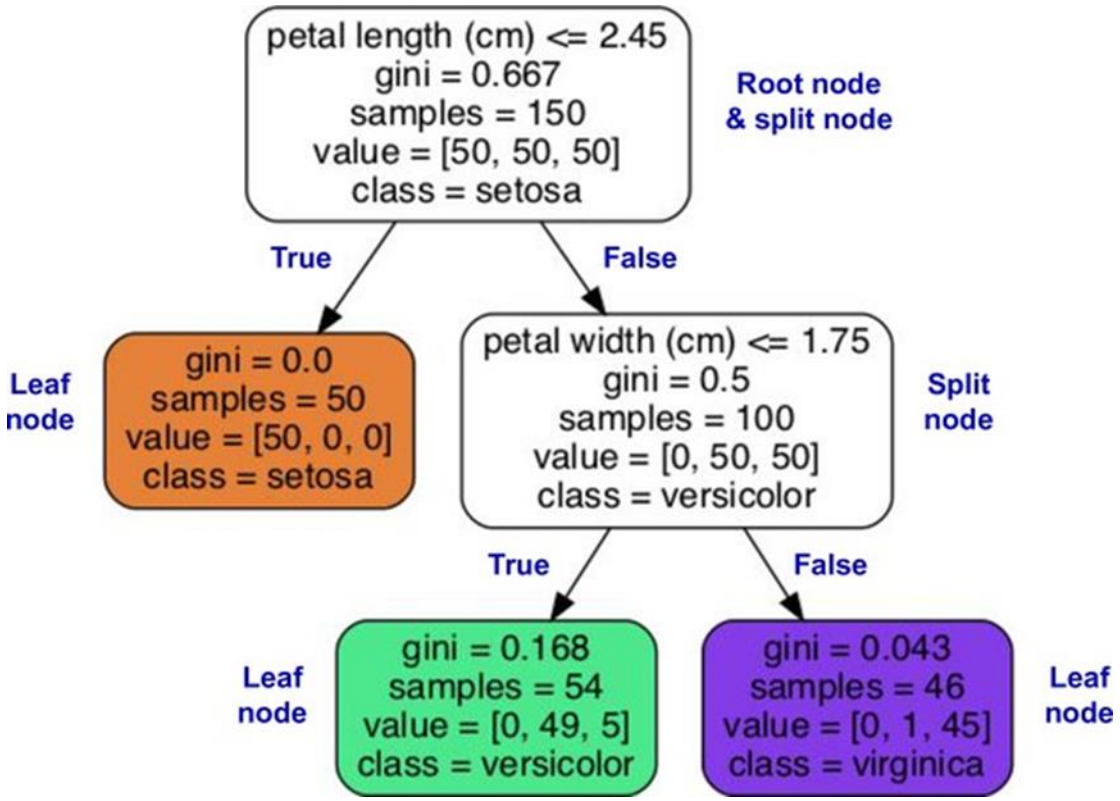
export_graphviz(tree_clf, out_file=“iris_tree.dot”, feature_names=[“petal length (cm)”, “petal width (cm)”], class_names=iris.target_names, rounded=True, filled=True)

Sau đó, bạn có thể sử dụng graphviz.Source.from_file() để tải và hiển thị tệp trong sổ tay Jupyter:

from graphviz import Source

Source.from_file(“iris_tree.dot”)

Graphviz là một gói phần mềm trực quan hóa đồ thị mã nguồn mở. Nó cũng bao gồm một công cụ dòng lệnh dot để chuyển đổi các tệp .dot sang nhiều định dạng khác nhau, chẳng hạn như PDF hoặc PNG. Cây quyết định đầu tiên của bạn trông giống như Hình 6-1.



Hình 6-1. Cây quyết định Iris

Đưa ra dự đoán

Hãy xem cây được biểu thị trong Hình 6-1 đưa ra dự đoán như thế nào. Giả sử bạn tìm thấy một bông hoa iris và bạn muốn phân loại nó dựa trên cánh hoa của nó. Bạn bắt đầu ở nút gốc (độ sâu 0, ở trên cùng): nút này hỏi liệu chiều dài cánh hoa của bông hoa có nhỏ hơn 2.45 cm hay không. Nếu có, thì bạn di chuyển xuống nút con bên trái của gốc (độ sâu 1, bên trái). Trong trường hợp này, đó là một nút lá (tức là nó không có bất kỳ nút con nào), vì vậy nó không hỏi bất kỳ câu hỏi nào: chỉ cần nhìn vào lớp dự đoán cho nút đó, và cây quyết định dự đoán rằng bông hoa của bạn là một Iris setosa (class=setosa). Bây giờ giả sử bạn tìm thấy một bông hoa khác, và lần này chiều dài cánh hoa lớn hơn 2.45 cm. Bạn lại bắt đầu ở nút gốc nhưng bây giờ di chuyển xuống nút con bên phải của nó (độ sâu 1, bên phải). Đây không phải là một nút lá, nó là một nút phân tách, vì vậy nó hỏi một câu hỏi khác: chiều rộng cánh hoa có nhỏ hơn 1.75 cm không? Nếu có, thì bông hoa của bạn rất có thể là một Iris versicolor (độ sâu 2, bên trái). Nếu không, nó có thể là một Iris virginica (độ sâu 2, bên phải). Nó thực sự đơn giản như vậy.

Thuộc tính samples của một nút đếm số lượng trường hợp huấn luyện mà nó áp dụng. Ví dụ, 100 trường hợp huấn luyện có chiều dài cánh hoa lớn hơn 2.45 cm (độ sâu 1, bên phải), và trong số 100 trường hợp đó, 54 trường hợp có chiều rộng cánh hoa nhỏ hơn 1.75 cm (độ sâu 2, bên trái). Thuộc tính value của một nút cho bạn biết có bao nhiêu trường hợp huấn luyện của mỗi lớp mà nút này áp dụng: ví dụ, nút dưới cùng bên phải áp dụng cho 0 Iris setosa, 1 Iris versicolor và 45 Iris virginica. Cuối cùng, thuộc tính gini của một nút đo độ

không tinh khiết Gini của nó: một nút là “thuần khiết” (gini=0) nếu tất cả các trường hợp huấn luyện mà nó áp dụng thuộc cùng một lớp. Ví dụ, vì nút bên trái ở độ sâu 1 chỉ áp dụng cho các trường hợp huấn luyện Iris setosa, nó là thuần khiết và độ không tinh khiết Gini của nó là 0. Phương trình 6-1 cho thấy cách thuật toán huấn luyện tính toán độ không tinh khiết Gini Gi của nút thứ i. Nút bên trái ở độ sâu 2 có độ không tinh khiết Gini bằng $1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0.168$.

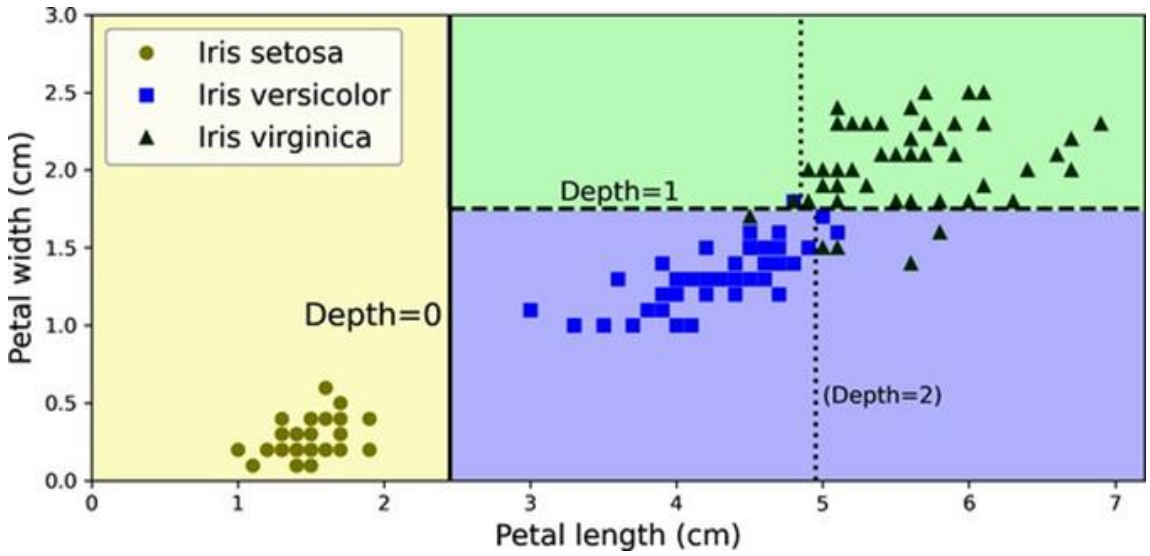
Phương trình 6-1. Độ không tinh khiết Gini

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k^2}$$

Trong phương trình này:

- G_i là độ không tinh khiết Gini của nút thứ i.
- $p_{i,k}$ là tỷ lệ các trường hợp thuộc lớp k trong số các trường hợp huấn luyện ở nút thứ i.

Hình 6-2 cho thấy các đường biên quyết định của cây quyết định này. Đường dọc dày biểu thị đường biên quyết định của nút gốc (độ sâu 0): chiều dài cánh hoa = 2.45 cm. Vì khu vực bên trái là thuần khiết (chỉ Iris setosa), nó không thể được phân chia thêm nữa. Tuy nhiên, khu vực bên phải là không thuần khiết, vì vậy nút bên phải ở độ sâu 1 phân chia nó tại chiều rộng cánh hoa = 1.75 cm (được biểu thị bằng đường nét đứt). Vì max_depth được đặt thành 2, cây quyết định dừng lại ở đó. Nếu bạn đặt max_depth thành 3, thì hai nút ở độ sâu 2 sẽ thêm một đường biên quyết định khác (được biểu thị bằng hai đường chấm chấm dọc).



Hình 6-2. Đường biên quyết định của cây quyết định

Ước tính xác suất lớp

Một cây quyết định cũng có thể ước tính xác suất một trường hợp thuộc về một lớp k cụ thể. Đầu tiên, nó duyệt cây để tìm nút lá cho trường hợp này, sau đó nó trả về tỷ lệ các trường hợp huấn luyện thuộc lớp k trong nút này. Ví dụ, giả sử bạn tìm thấy một bông hoa có cánh dài 5 cm và rộng 1.5 cm. Nút lá tương ứng là nút bên trái ở độ sâu 2, vì vậy cây quyết định xuất ra các xác suất sau: 0% cho Iris setosa (0/54), 90.7% cho Iris versicolor (49/54), và 9.3% cho Iris virginica (5/54). Và nếu bạn yêu cầu nó dự đoán lớp, nó sẽ xuất ra Iris versicolor (lớp 1) vì nó có xác suất cao nhất. Hãy kiểm tra điều này:

```
>>> tree_clf.predict_proba([[5, 1.5]]).round(3)
array([[0. , 0.907, 0.093]])

>>> tree_clf.predict([[5, 1.5]])
array([1])
```

Hoàn hảo! Lưu ý rằng các xác suất ước tính sẽ giống hệt nhau ở bất kỳ vị trí nào khác trong hình chữ nhật dưới cùng bên phải của Hình 6-2—ví dụ, nếu cánh hoa dài 6 cm và rộng 1.5 cm (mặc dù rõ ràng là trong trường hợp này nó rất có thể là một Iris virginica).

Thuật toán huấn luyện CART

Scikit-Learn sử dụng thuật toán Cây phân loại và hồi quy (CART) để huấn luyện cây quyết định (còn gọi là "phát triển" cây). Thuật toán hoạt động bằng cách đầu tiên chia tập huấn luyện thành hai tập con bằng cách sử dụng một đặc trưng k duy nhất và một ngưỡng t_k (ví dụ: "chiều dài cánh hoa le 2.45 cm"). Làm thế nào để nó chọn k và t_k ? Nó tìm kiếm cặp (k, t_k) tạo ra các tập con thuần khiết nhất, có trọng số theo kích thước của chúng. Phương trình 6-2 đưa ra hàm chi phí mà thuật toán cố gắng tối thiểu hóa.

Phương trình 6-2. Hàm chi phí CART cho phân loại

$$J\left(k, t_k\right) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

Trong đó:

- $G_{left/right}$ đo độ không tinh khiết của tập con trái/phải
- $m_{left/right}$ là số lượng trường hợp trong tập con trái/phải

Một khi thuật toán CART đã chia tập huấn luyện thành hai thành công, nó sẽ chia các tập con bằng cách sử dụng logic tương tự, sau đó là các tập con-con, cứ thế tiếp tục một cách đệ quy. Nó dừng đệ quy khi đạt đến độ sâu tối đa (được định nghĩa bởi siêu tham số `max_depth`), hoặc nếu nó không thể tìm thấy một cách chia nào làm giảm độ không tinh khiết. Một vài siêu tham số khác (sẽ được mô tả ngay sau đây) kiểm soát các điều kiện dừng bổ sung: `min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf`, và `max_leaf_nodes`.

Độ phức tạp tính toán

Việc đưa ra dự đoán đòi hỏi phải duyệt cây quyết định từ gốc đến một nút lá. Cây quyết định thường cân bằng gần đúng, vì vậy việc duyệt cây quyết định đòi hỏi phải đi qua khoảng $O(log_2(m))$ nút, trong đó $log_2(m)$ là logarit nhị phân của m , bằng $log(m)/log(2)$. Vì mỗi nút chỉ yêu cầu kiểm tra giá trị của một đặc trưng, độ phức tạp dự đoán tổng thể là $O(log_2(m))$, không phụ thuộc vào số lượng đặc trưng. Vì vậy, dự đoán rất nhanh, ngay cả khi xử lý các tập huấn luyện lớn. Thuật toán huấn luyện so sánh tất cả các đặc trưng (hoặc ít hơn nếu `max_features` được đặt) trên tất cả các mẫu tại mỗi nút. So sánh tất cả các đặc trưng trên tất cả các mẫu tại mỗi nút dẫn đến độ phức tạp huấn luyện là $O(ntimesmlog_2(m))$.

Độ không tinh khiết Gini hay Entropy?

Theo mặc định, lớp `DecisionTreeClassifier` sử dụng độ đo độ không tinh khiết Gini, nhưng bạn có thể chọn độ đo độ không tinh khiết entropy thay thế bằng cách đặt siêu tham số `criterion` thành "entropy". Khái niệm entropy bắt nguồn từ nhiệt động lực học như một độ đo sự hỗn loạn phân tử: entropy tiến gần về 0 khi các phân tử đứng yên và được sắp xếp tốt. Entropy sau đó lan rộng sang nhiều lĩnh vực khác nhau, bao gồm lý thuyết thông tin của Shannon, nơi nó đo lượng thông tin trung bình của một thông điệp, như chúng ta đã thấy trong Chương 4. Entropy bằng 0 khi tất cả các thông điệp giống hệt nhau. Trong học máy, entropy thường được sử dụng làm độ đo độ không tinh khiết: entropy của một tập hợp bằng 0 khi nó chỉ chứa các trường hợp của một lớp. Phương trình 6-3 cho thấy định nghĩa entropy của nút thứ i . Ví dụ, nút bên trái ở độ sâu 2 trong Hình 6-1 có entropy bằng $-(49/54)log_2(49/54)-(5/54)log_2(5/54)approx0.445$. Phương trình 6-3. Entropy

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k}\neq 0}}^{n} p_{i,k} \log_2 (p_{i,k})$$

Vậy, bạn nên sử dụng độ không tinh khiết Gini hay entropy? Sự thật là, hầu hết thời gian nó không tạo ra sự khác biệt lớn: chúng dẫn đến các cây tương tự. Độ không tinh khiết Gini nhanh hơn một chút để tính toán, vì vậy đó là một lựa chọn mặc định tốt. Tuy nhiên, khi chúng khác nhau, độ không tinh khiết Gini có xu hướng cô lập lớp phổ biến nhất trong nhánh riêng của cây, trong khi entropy có xu hướng tạo ra các cây cân bằng hơn một chút.

Siêu tham số chính quy hóa

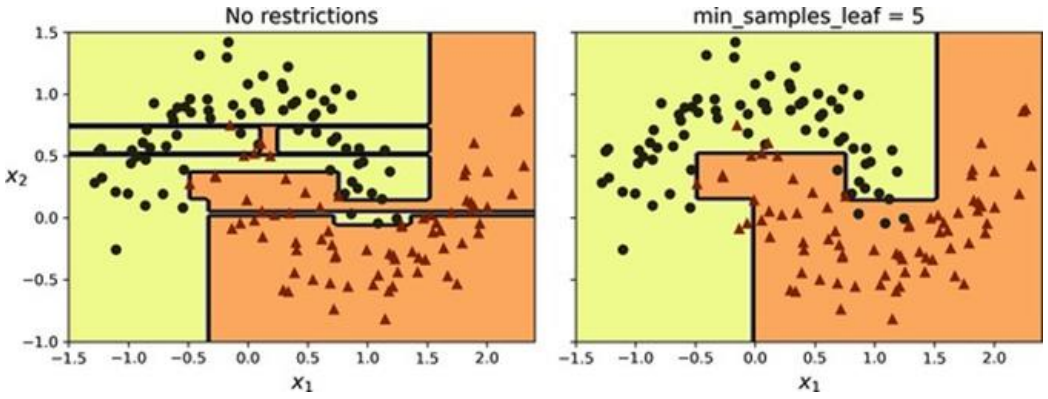
Cây quyết định đưa ra rất ít giả định về dữ liệu huấn luyện (ngược lại với các mô hình tuyến tính, giả định rằng dữ liệu là tuyến tính, chẳng hạn). Nếu không bị ràng buộc, cấu trúc cây sẽ tự điều chỉnh theo dữ liệu huấn luyện, khớp nó rất chặt chẽ—thực tế, rất có thể là quá khớp. Một mô hình như vậy thường được gọi là mô hình phi tham số, không phải vì nó không có bất kỳ tham số nào (nó thường có rất nhiều) mà vì số lượng tham số không được xác định trước khi huấn luyện, vì vậy cấu trúc mô hình có thể tự do bám sát dữ liệu. Ngược lại, một mô hình tham số, chẳng hạn như mô hình tuyến tính, có số lượng tham số được xác

định trước, vì vậy mức độ tự do của nó bị hạn chế, giảm nguy cơ quá khớp (nhưng tăng nguy cơ dưới khớp). Để tránh quá khớp dữ liệu huấn luyện, bạn cần hạn chế sự tự do của cây quyết định trong quá trình huấn luyện. Như bạn đã biết, điều này được gọi là chính quy hóa. Các siêu tham số chính quy hóa phụ thuộc vào thuật toán được sử dụng, nhưng nói chung bạn có thể ít nhất hạn chế độ sâu tối đa của cây quyết định. Trong Scikit-Learn, điều này được kiểm soát bởi siêu tham số `max_depth`. Giá trị mặc định là `None`, có nghĩa là không giới hạn. Giảm `max_depth` sẽ chính quy hóa mô hình và do đó giảm nguy cơ quá khớp. Lớp `DecisionTreeClassifier` có một vài tham số khác tương tự hạn chế hình dạng của cây quyết định:

- `max_features`: Số lượng đặc trưng tối đa được đánh giá để phân chia tại mỗi nút.
- `max_leaf_nodes`: Số lượng nút lá tối đa.
- `min_samples_split`: Số lượng mẫu tối thiểu mà một nút phải có trước khi nó có thể được phân chia.
- `min_samples_leaf`: Số lượng mẫu tối thiểu mà một nút lá phải có để được tạo.
- `min_weight_fraction_leaf`: Tương tự như `min_samples_leaf` nhưng được biểu thị dưới dạng một phần của tổng số lượng trường hợp có trọng số. Tăng các siêu tham số `min_*` hoặc giảm các siêu tham số `max_*` sẽ chính quy hóa mô hình.

Hãy kiểm tra chính quy hóa trên tập dữ liệu moons, được giới thiệu trong Chương 5. Chúng ta sẽ huấn luyện một cây quyết định không có chính quy hóa, và một cây khác với `min_samples_leaf=5`. Đây là đoạn mã; Hình 6-3 cho thấy các đường biên quyết định của mỗi cây:

```
from sklearn.datasets import make_moons

X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)

tree_clf1 = DecisionTreeClassifier(random_state=42)
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_clf1.fit(X_moons, y_moons)
tree_clf2.fit(X_moons, y_moons)
```



Hình 6-3. Đường biên quyết định của một cây không chính quy hóa (trái) và một cây chính quy hóa (phải)

Mô hình không chính quy hóa ở bên trái rõ ràng đang bị quá khớp, và mô hình chính quy hóa ở bên phải có thể sẽ tổng quát hóa tốt hơn. Chúng ta có thể xác minh điều này bằng cách đánh giá cả hai cây trên một tập kiểm tra được tạo bằng cách sử dụng một hạt giống ngẫu nhiên khác:

```
>>> X_moons_test, y_moons_test = make_moons(n_samples=1000, noise=0.2,
random_state=43)

>>> tree_clf1.score(X_moons_test, y_moons_test)
0.898

>>> tree_clf2.score(X_moons_test, y_moons_test)
0.92
```

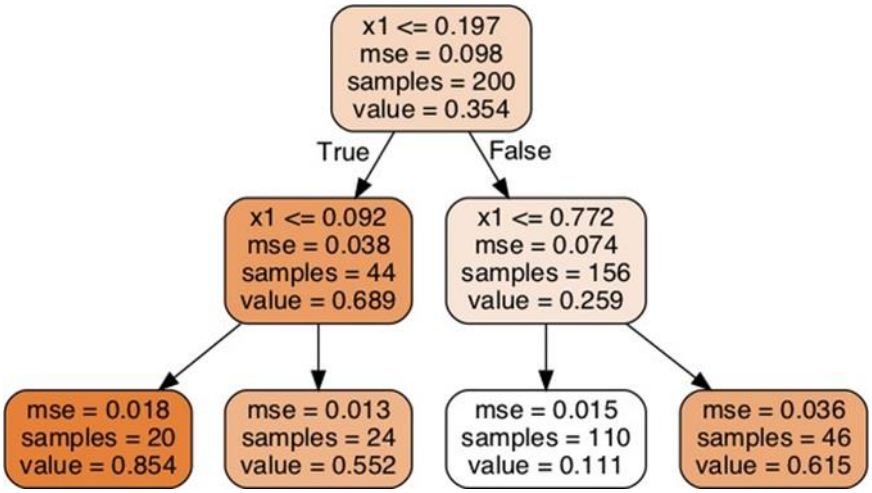
Thực vậy, cây thứ hai có độ chính xác tốt hơn trên tập kiểm tra.

Hồi quy

Cây quyết định cũng có khả năng thực hiện các tác vụ hồi quy. Hãy xây dựng một cây hồi quy sử dụng lớp `DecisionTreeRegressor` của Scikit-Learn, huấn luyện nó trên một tập dữ liệu bậc hai nhiễu với `max_depth=2`:

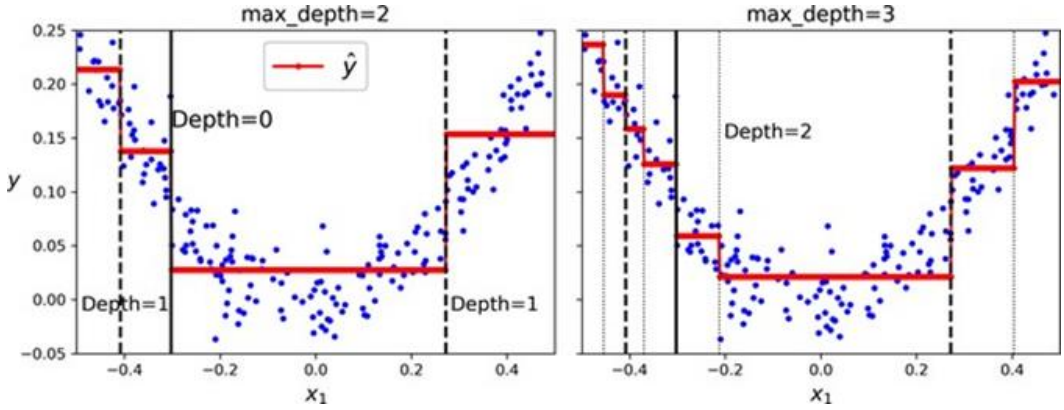
```
DecisionTreeRegressor

np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5 # một đặc trưng đầu vào ngẫu nhiên duy
nhất
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

Cây kết quả được biểu thị trong Hình 6-4.



Hình 6-4. Một cây quyết định cho hồi quy

Cây này trông rất giống với cây phân loại mà bạn đã xây dựng trước đó. Sự khác biệt chính là thay vì dự đoán một lớp trong mỗi nút, nó dự đoán một giá trị. Ví dụ, giả sử bạn muốn đưa ra dự đoán cho một trường hợp mới với $x_1 = 0.2$. Nút gốc hỏi liệu $x_1 le 0.197$ hay không. Vì không phải, thuật toán đi đến nút con bên phải, nút này hỏi liệu $x_1 le 0.772$. Vì có, thuật toán đi đến nút con bên trái. Đây là một nút lá, và nó dự đoán giá trị = 0.111. Dự đoán này là giá trị mục tiêu trung bình của 110 trường hợp huấn luyện liên quan đến nút lá này, và nó dẫn đến một sai số bình phương trung bình bằng 0.015 trên 110 trường hợp này. Các dự đoán của mô hình này được biểu thị ở bên trái trong Hình 6-5. Nếu bạn đặt `max_depth=3`, bạn sẽ nhận được các dự đoán được biểu thị ở bên phải. Lưu ý rằng giá trị dự đoán cho mỗi vùng luôn là giá trị mục tiêu trung bình của các trường hợp trong vùng đó. Thuật toán chia mỗi vùng theo cách làm cho hầu hết các trường hợp huấn luyện gần nhất có thể với giá trị dự đoán đó.

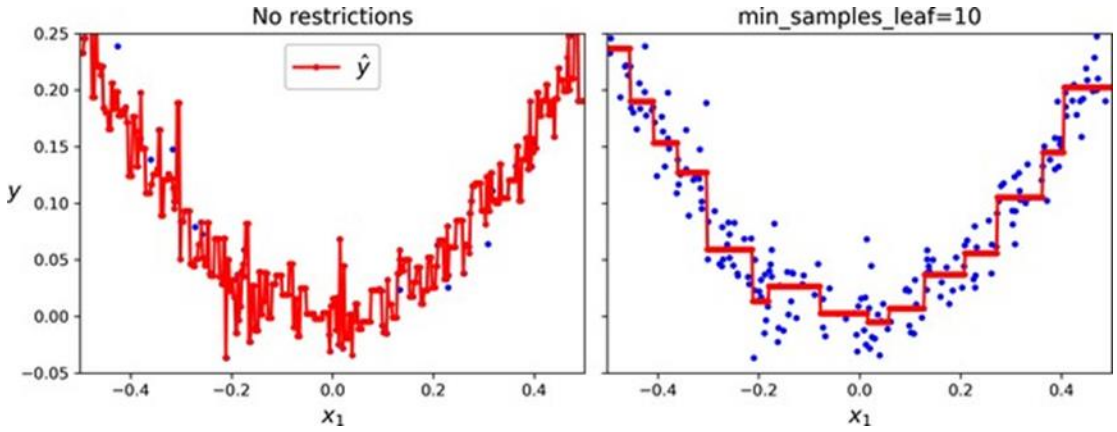


Hình 6-5. Dự đoán của hai mô hình hồi quy cây quyết định

Thuật toán CART hoạt động như đã mô tả trước đó, ngoại trừ việc thay vì cố gắng chia tập huấn luyện theo cách giảm thiểu độ không tinh khiết, giờ đây nó cố gắng chia tập huấn luyện theo cách giảm thiểu MSE. Phương trình 6-4 cho thấy hàm chi phí mà thuật toán cố gắng tối thiểu hóa. Phương trình 6-4. Hàm chi phí CART cho hồi quy

$$J(k, t_k) = \frac{m_{\text{left}}}{m}\text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m}\text{MSE}_{\text{right}} \quad \text{where} \begin{cases} \text{MSE}_{\text{node}} = \dfrac{\sum_{i \in \text{node}} \left(\hat{y}_{\text{node}} - y^{(i)}\right)^2}{m_{\text{node}}} \\ \hat{y}_{\text{node}} = \dfrac{\sum_{i \in \text{node}} y^{(i)}}{m_{\text{node}}} \end{cases}$$

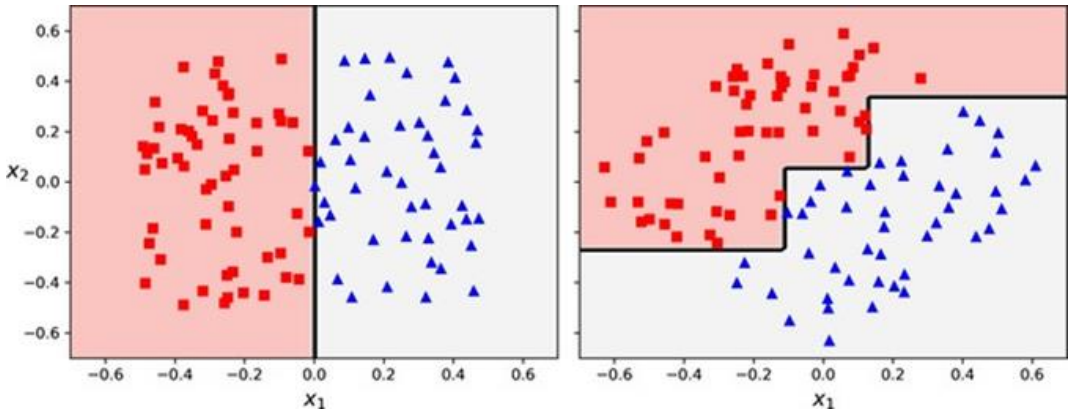
Cũng giống như đối với các tác vụ phân loại, cây quyết định dễ bị quá khớp khi xử lý các tác vụ hồi quy. Nếu không có bất kỳ chính quy hóa nào (tức là sử dụng các siêu tham số mặc định), bạn sẽ nhận được các dự đoán ở bên trái trong Hình 6-6. Những dự đoán này rõ ràng đang quá khớp tập huấn luyện rất tệ. Chỉ cần đặt `min_samples_leaf=10` sẽ tạo ra một mô hình hợp lý hơn nhiều, được biểu thị ở bên phải trong Hình 6-6.



Hình 6-6. Dự đoán của một cây hồi quy không chính quy hóa (trái) và một cây được chính quy hóa (phải)

Độ nhạy với hướng trục

Hy vọng bây giờ bạn đã bị thuyết phục rằng cây quyết định có rất nhiều ưu điểm: chúng tương đối dễ hiểu và giải thích, đơn giản để sử dụng, linh hoạt và mạnh mẽ. Tuy nhiên, chúng có một vài hạn chế. Đầu tiên, như bạn có thể đã nhận thấy, cây quyết định thích các đường biên quyết định trực giao (tất cả các phân chia đều vuông góc với một trục), điều này làm cho chúng nhạy cảm với hướng của dữ liệu. Ví dụ, Hình 6-7 cho thấy một tập dữ liệu có thể phân tách tuyến tính đơn giản: ở bên trái, một cây quyết định có thể chia nó dễ dàng, trong khi ở bên phải, sau khi tập dữ liệu được xoay 45°, đường biên quyết định trông không cần thiết. Mặc dù cả hai cây quyết định đều khớp tập huấn luyện hoàn hảo, rất có khả năng mô hình bên phải sẽ không tổng quát hóa tốt.

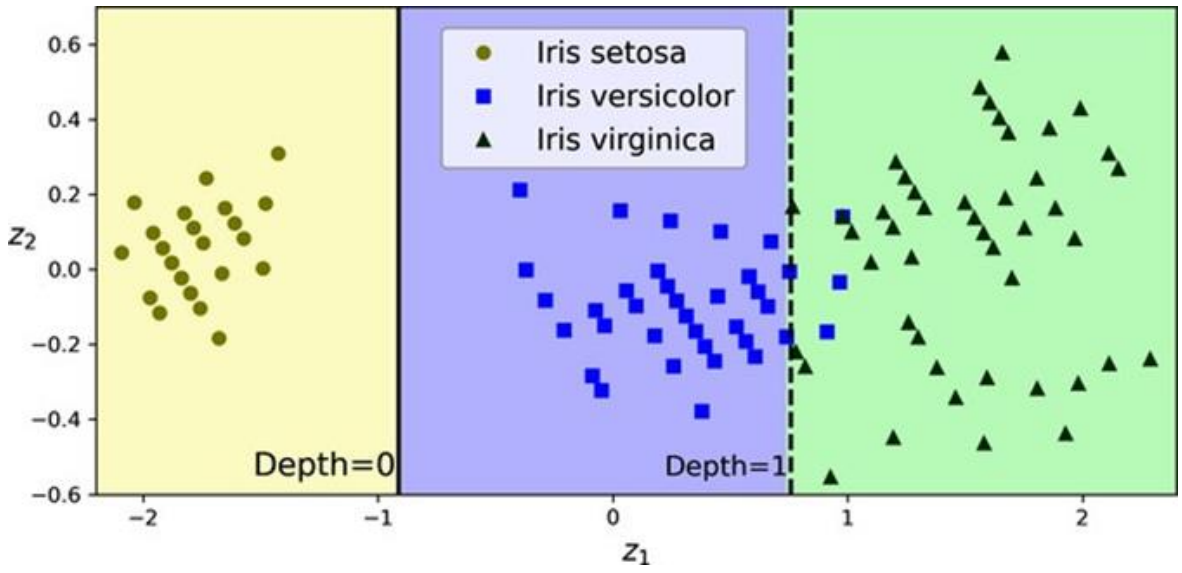


Hình 6-7. Độ nhạy với việc xoay tập huấn luyện

Một cách để hạn chế vấn đề này là chuẩn hóa dữ liệu, sau đó áp dụng phép biến đổi phân tích thành phần chính (PCA). Chúng ta sẽ xem xét PCA chi tiết trong Chương 8, nhưng hiện tại bạn chỉ cần biết rằng nó xoay dữ liệu theo cách giảm sự tương quan giữa các đặc trưng, điều này thường (không phải luôn luôn) làm cho mọi thứ dễ dàng hơn cho cây. Hãy tạo một pipeline nhỏ để chuẩn hóa dữ liệu và xoay nó bằng PCA, sau đó huấn luyện một `DecisionTreeClassifier` trên dữ liệu đó. Hình 6-8 cho thấy các đường biên quyết định của

cây đó: như bạn có thể thấy, việc xoay giúp có thể khớp tập dữ liệu khá tốt chỉ bằng một đặc trưng, z_1 , là một hàm tuyến tính của chiều dài và chiều rộng cánh hoa gốc. Đây là đoạn mã:

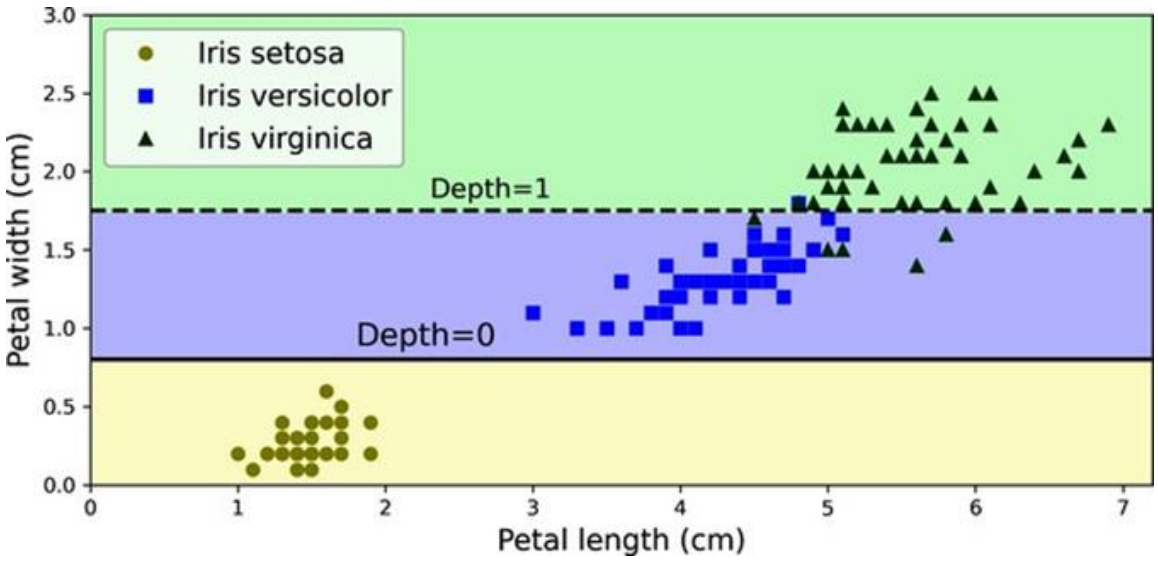
```
sklearn.decomposition import PCA
sklearn.pipeline import make_pipeline
sklearn.preprocessing import StandardScaler

pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```



Hình 6-8. Đường biên quyết định của cây trên tập dữ liệu iris đã được chuẩn hóa và xoay PCA

Cây quyết định có phương sai cao

Tổng quát hơn, vấn đề chính với cây quyết định là chúng có phương sai khá cao: những thay đổi nhỏ đối với các siêu tham số hoặc dữ liệu có thể tạo ra các mô hình rất khác nhau. Trên thực tế, vì thuật toán huấn luyện được sử dụng bởi Scikit-Learn là ngẫu nhiên—nó chọn ngẫu nhiên tập hợp các đặc trưng để đánh giá tại mỗi nút—ngay cả việc huấn luyện lại cùng một cây quyết định trên cùng một dữ liệu cũng có thể tạo ra một mô hình rất khác, chẳng hạn như mô hình được biểu thị trong Hình 6-9 (trừ khi bạn đặt siêu tham số `random_state`). Như bạn có thể thấy, nó trông rất khác so với cây quyết định trước đó (Hình 6-2).



Hình 6-9. Huấn luyện lại cùng một mô hình trên cùng dữ liệu có thể tạo ra một mô hình rất khác

May mắn thay, bằng cách tính trung bình các dự đoán trên nhiều cây, có thể giảm đáng kể phương sai. Một tập hợp các cây như vậy được gọi là rừng ngẫu nhiên, và nó là một trong những loại mô hình mạnh mẽ nhất hiện có, như bạn sẽ thấy trong chương tiếp theo.

Bài tập

1. Độ sâu xấp xỉ của một cây quyết định được huấn luyện (không giới hạn) trên một tập huấn luyện với một triệu trường hợp là bao nhiêu?
2. Độ không tinh khiết Gini của một nút thường thấp hơn hay cao hơn so với nút cha của nó? Nó thường thấp hơn/cao hơn, hay luôn luôn thấp hơn/cao hơn?
3. Nếu một cây quyết định đang quá khớp tập huấn luyện, có phải là một ý hay khi thử giảm `max_depth` không?
4. Nếu một cây quyết định đang dưới khớp tập huấn luyện, có phải là một ý hay khi thử chuẩn hóa các đặc trưng đầu vào không?
5. Nếu mất một giờ để huấn luyện một cây quyết định trên một tập huấn luyện chứa một triệu trường hợp, thì sẽ mất khoảng bao nhiêu thời gian để huấn luyện một cây quyết định khác trên một tập huấn luyện chứa mười triệu trường hợp? Gợi ý: xem xét độ phức tạp tính toán của thuật toán CART.
6. Nếu mất một giờ để huấn luyện một cây quyết định trên một tập huấn luyện nhất định, thì sẽ mất khoảng bao nhiêu thời gian nếu bạn tăng gấp đôi số lượng đặc trưng?
7. Huấn luyện và tinh chỉnh một cây quyết định cho tập dữ liệu moons bằng cách làm theo các bước sau: a. Sử dụng `make_moons(n_samples=10000, noise=0.4)` để tạo một tập dữ liệu moons. b. Sử dụng `train_test_split()` để chia tập dữ liệu thành một tập huấn luyện và một tập kiểm tra. c. Sử dụng tìm kiếm lưới với xác thực chéo (với sự trợ giúp của lớp `GridSearchCV`) để tìm các giá trị siêu tham số tốt cho một `DecisionTreeClassifier`. Gợi ý: thử các giá trị khác nhau cho `max_leaf_nodes`.

d. Huấn luyện nó trên toàn bộ tập huấn luyện bằng cách sử dụng các siêu tham số này, và đo hiệu suất của mô hình trên tập kiểm tra. Bạn sẽ đạt được độ chính xác khoảng 85% đến 87%.

8. Phát triển một rừng bằng cách làm theo các bước sau: a. Tiếp tục bài tập trước, tạo 1.000 tập con của tập huấn luyện, mỗi tập con chứa 100 trường hợp được chọn ngẫu nhiên. Gợi ý: bạn có thể sử dụng lớp `ShuffleSplit` của Scikit-Learn cho việc này. b. Huấn luyện một cây quyết định trên mỗi tập con, sử dụng các giá trị siêu tham số tốt nhất được tìm thấy trong bài tập trước. Đánh giá 1.000 cây quyết định này trên tập kiểm tra. Vì chúng được huấn luyện trên các tập nhỏ hơn, các cây quyết định này có thể sẽ hoạt động kém hơn cây quyết định đầu tiên, chỉ đạt độ chính xác khoảng 80%. c. Bây giờ đến phần kỳ diệu. Đối với mỗi trường hợp tập kiểm tra, tạo các dự đoán của 1.000 cây quyết định, và chỉ giữ lại dự đoán phổ biến nhất (bạn có thể sử dụng hàm `mode()` của SciPy cho việc này). Cách tiếp cận này cho bạn các dự đoán bỏ phiếu đa số trên tập kiểm tra. d. Đánh giá các dự đoán này trên tập kiểm tra: bạn sẽ đạt được độ chính xác cao hơn một chút so với mô hình đầu tiên của bạn (cao hơn khoảng 0.5 đến 1.5%). Chúc mừng, bạn đã huấn luyện một bộ phân loại rừng ngẫu nhiên! Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 7. HỌC TỔ HỢP VÀ RỪNG NGẪU NHIÊN

Giả sử bạn đặt một câu hỏi phức tạp cho hàng nghìn người ngẫu nhiên, sau đó tổng hợp câu trả lời của họ. Trong nhiều trường hợp, bạn sẽ thấy rằng câu trả lời tổng hợp này tốt hơn câu trả lời của một chuyên gia. Đây được gọi là "sự khôn ngoan của đám đông". Tương tự, nếu bạn tổng hợp các dự đoán của một nhóm các bộ dự đoán (chẳng hạn như bộ phân loại hoặc bộ hồi quy), bạn thường sẽ nhận được các dự đoán tốt hơn so với bộ dự đoán riêng lẻ tốt nhất. Một nhóm các bộ dự đoán được gọi là một tập hợp (ensemble); do đó, kỹ thuật này được gọi là học tổ hợp (ensemble learning), và một thuật toán học tổ hợp được gọi là một phương pháp tổ hợp (ensemble method).

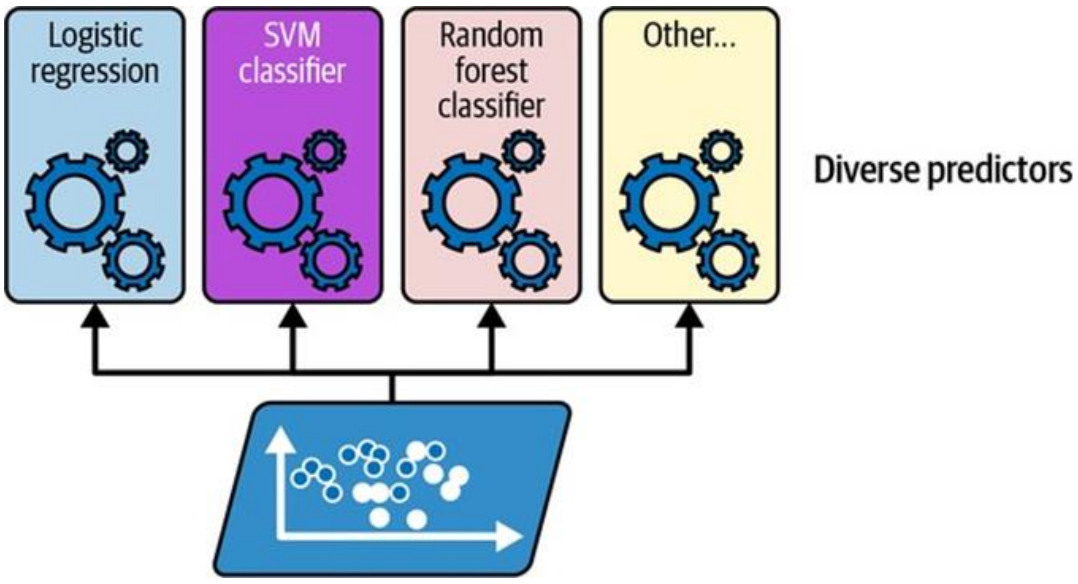
Ví dụ về một phương pháp tổ hợp, bạn có thể huấn luyện một nhóm các bộ phân loại cây quyết định, mỗi bộ trên một tập con ngẫu nhiên khác nhau của tập huấn luyện. Sau đó, bạn có thể lấy các dự đoán của tất cả các cây riêng lẻ, và lớp nhận được nhiều phiếu bầu nhất là dự đoán của tập hợp (xem bài tập cuối cùng trong Chương 6). Một tập hợp các cây quyết định như vậy được gọi là rừng ngẫu nhiên, và mặc dù đơn giản, đây là một trong những thuật toán học máy mạnh mẽ nhất hiện có.

Như đã thảo luận trong Chương 2, bạn thường sẽ sử dụng các phương pháp tổ hợp gần cuối một dự án, một khi bạn đã xây dựng được một vài bộ dự đoán tốt, để kết hợp chúng thành một bộ dự đoán thậm chí còn tốt hơn. Trên thực tế, các giải pháp chiến thắng trong các cuộc thi học máy thường liên quan đến một số phương pháp tổ hợp—nổi tiếng nhất trong cuộc thi Netflix Prize.

Trong chương này, chúng ta sẽ xem xét các phương pháp tổ hợp phổ biến nhất, bao gồm bộ phân loại bỏ phiếu, các tập hợp bagging và pasting, rừng ngẫu nhiên, và các tập hợp boosting và stacking.

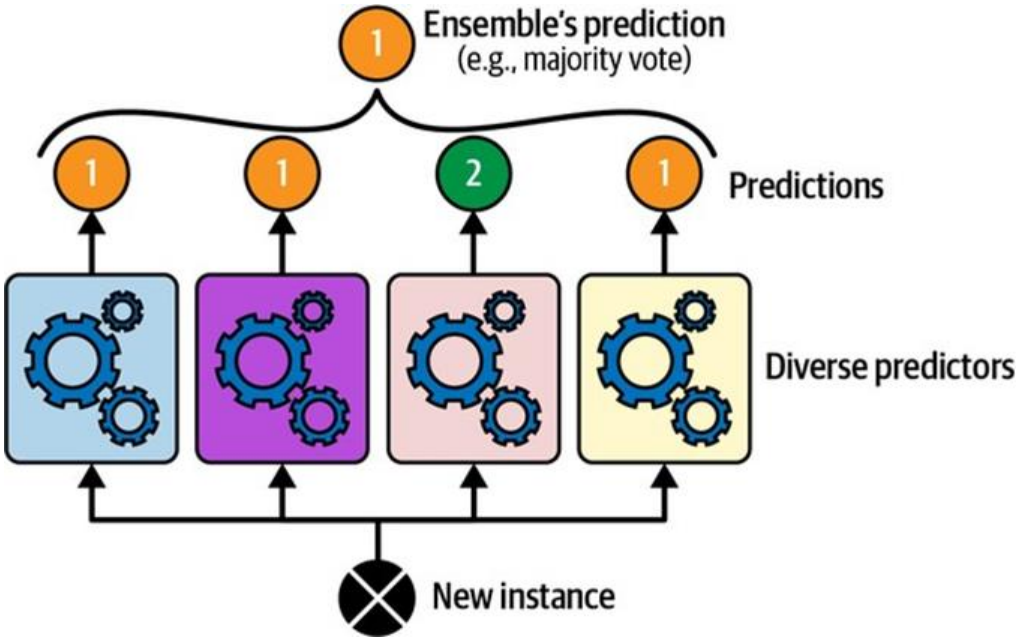
Bộ phân loại biểu quyết

Giả sử bạn đã huấn luyện một vài bộ phân loại, mỗi bộ đạt độ chính xác khoảng 80%. Bạn có thể có một bộ phân loại hồi quy logistic, một bộ phân loại SVM, một bộ phân loại rừng ngẫu nhiên, một bộ phân loại k-láng giềng gần nhất, và có lẽ một vài bộ nữa (xem Hình 7-1).



Hình 7-1. Huấn luyện các bộ phân loại đa dạng

Một cách rất đơn giản để tạo ra một bộ phân loại thậm chí còn tốt hơn là tổng hợp các dự đoán của từng bộ phân loại: lớp nhận được nhiều phiếu bầu nhất là dự đoán của tập hợp. Bộ phân loại bỏ phiếu đa số này được gọi là bộ phân loại bỏ phiếu cứng (hard voting classifier) (xem Hình 7-2).

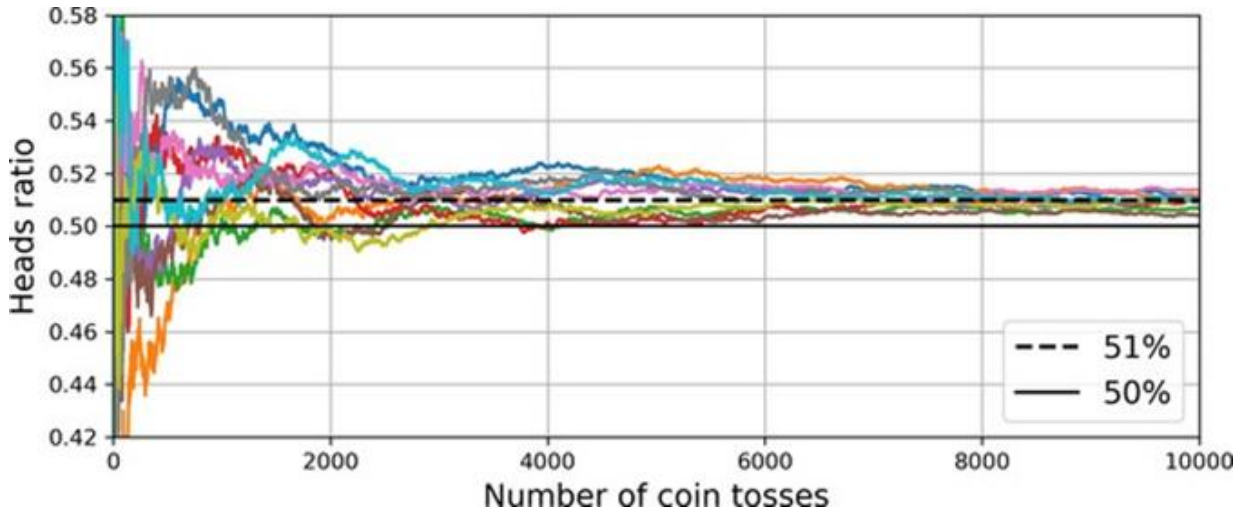


Hình 7-2. Dự đoán của bộ phân loại bỏ phiếu cứng

Hơi đáng ngạc nhiên, bộ phân loại bỏ phiếu này thường đạt được độ chính xác cao hơn so với bộ phân loại tốt nhất trong tập hợp. Trên thực tế, ngay cả khi mỗi bộ phân loại là một bộ học yếu (weak learner) (nghĩa là nó chỉ tốt hơn một chút so với đoán ngẫu nhiên), tập hợp

vẫn có thể là một bộ học mạnh (strong learner) (đạt độ chính xác cao), miễn là có đủ số lượng bộ học yếu trong tập hợp và chúng đủ đa dạng.

Làm thế nào điều này có thể xảy ra? Phép tương tự sau đây có thể giúp làm sáng tỏ bí ẩn này. Giả sử bạn có một đồng xu hơi lệch, có 51% cơ hội ra mặt sấp và 49% cơ hội ra mặt ngửa. Nếu bạn tung nó 1.000 lần, bạn thường sẽ nhận được ít nhiều 510 mặt sấp và 490 mặt ngửa, và do đó đa số là mặt sấp. Nếu bạn tính toán, bạn sẽ thấy rằng xác suất thu được đa số mặt sấp sau 1.000 lần tung là gần 75%. Bạn càng tung đồng xu nhiều, xác suất càng cao (ví dụ: với 10.000 lần tung, xác suất tăng lên hơn 97%). Điều này là do luật số lớn: khi bạn tiếp tục tung đồng xu, tỷ lệ mặt sấp ngày càng gần với xác suất mặt sấp (51%). Hình 7-3 cho thấy 10 chuỗi tung đồng xu lệch. Bạn có thể thấy rằng khi số lần tung tăng, tỷ lệ mặt sấp tiến gần đến 51%. Cuối cùng, tất cả 10 chuỗi đều kết thúc gần 51% đến mức chúng luôn cao hơn 50%.



Hình 7-3. Luật số lớn

Tương tự, giả sử bạn xây dựng một tập hợp chứa 1.000 bộ phân loại mà từng bộ chỉ đúng 51% thời gian (chỉ tốt hơn một chút so với đoán ngẫu nhiên). Nếu bạn dự đoán lớp được bỏ phiếu đa số, bạn có thể hy vọng đạt độ chính xác lên đến 75%! Tuy nhiên, điều này chỉ đúng nếu tất cả các bộ phân loại hoàn toàn độc lập, mắc lỗi không tương quan, điều này rõ ràng không phải là trường hợp vì chúng được huấn luyện trên cùng một dữ liệu. Chúng có khả năng mắc cùng loại lỗi, vì vậy sẽ có nhiều phiếu bầu đa số cho lớp sai, làm giảm độ chính xác của tập hợp.

Scikit-Learn cung cấp một lớp `VotingClassifier` khá dễ sử dụng: chỉ cần cung cấp cho nó một danh sách các cặp tên/bộ dự đoán, và sử dụng nó như một bộ phân loại thông thường. Hãy thử nó trên tập dữ liệu moons (được giới thiệu trong Chương 5). Chúng ta sẽ tải và chia tập dữ liệu moons thành tập huấn luyện và tập kiểm tra, sau đó chúng ta sẽ tạo và huấn luyện một bộ phân loại bỏ phiếu gồm ba bộ phân loại đa dạng:

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)
```

Khi bạn huấn luyện một `VotingClassifier`, nó sẽ sao chép mọi bộ ước lượng và huấn luyện các bản sao. Các bộ ước lượng gốc có sẵn thông qua thuộc tính `estimators`, trong khi các bản sao đã được huấn luyện có sẵn thông qua thuộc tính `estimators_`. Nếu bạn thích một từ điển hơn là một danh sách, bạn có thể sử dụng `named_estimators` hoặc `named_estimators_` thay thế. Để bắt đầu, hãy xem độ chính xác của từng bộ phân loại đã được huấn luyện trên tập kiểm tra:

```
>>> for name, clf in voting_clf.named_estimators_.items():
...     print(name, "=", clf.score(X_test, y_test))
...
lr = 0.864
rf = 0.896
svc = 0.896
```

Khi bạn gọi phương thức `predict()` của bộ phân loại bỏ phiếu, nó thực hiện bỏ phiếu cứng. Ví dụ, bộ phân loại bỏ phiếu dự đoán lớp 1 cho trường hợp đầu tiên của tập kiểm tra, bởi vì hai trong số ba bộ phân loại dự đoán lớp đó:

```
>>> voting_clf.predict(X_test[:1])
array([1])

>>> [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
[array([1]), array([1]), array([0])]
```

Bây giờ hãy xem hiệu suất của bộ phân loại bỏ phiếu trên tập kiểm tra:

```
>>> voting_clf.score(X_test, y_test)
0.912
```

Đó! Bộ phân loại bỏ phiếu vượt trội hơn tất cả các bộ phân loại riêng lẻ. Nếu tất cả các bộ phân loại đều có khả năng ước tính xác suất lớp (tức là nếu tất cả chúng đều có phương thức `predict_proba()`), thì bạn có thể yêu cầu Scikit-Learn dự đoán lớp có xác suất lớp cao nhất, được tính trung bình trên tất cả các bộ phân loại riêng lẻ. Đây được gọi là bỏ phiếu mềm (soft voting). Nó thường đạt được hiệu suất cao hơn bỏ phiếu cứng vì nó ưu tiên

hơn các phiếu bầu có độ tin cậy cao. Tất cả những gì bạn cần làm là đặt siêu tham số `voting` của bộ phân loại bỏ phiếu thành “soft”, và đảm bảo rằng tất cả các bộ phân loại đều có thể ước tính xác suất lớp. Điều này không đúng với lớp SVC theo mặc định, vì vậy bạn cần đặt siêu tham số `probability` của nó thành `True` (điều này sẽ làm cho lớp SVC sử dụng cross-validation để ước tính xác suất lớp, làm chậm quá trình huấn luyện, và nó sẽ thêm một phương thức `predict_proba()`). Hãy thử điều đó:

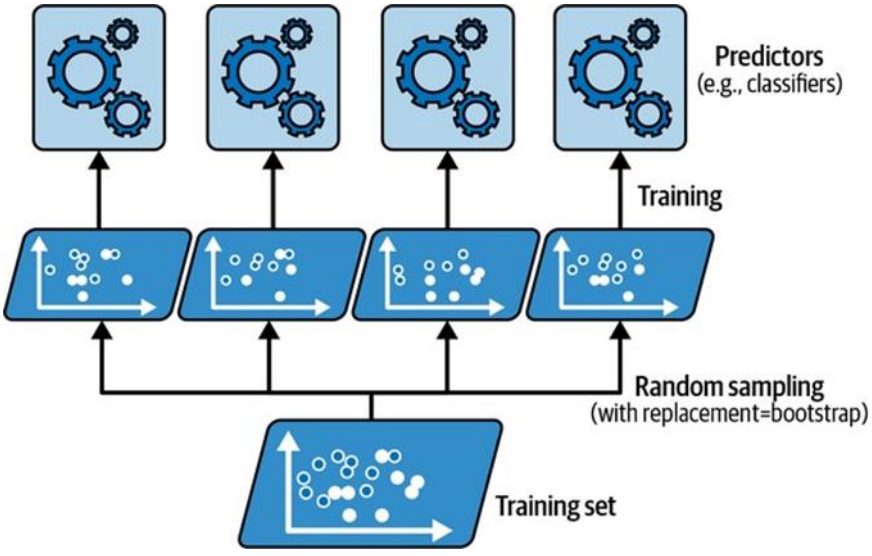
```
>>> voting_clf.voting = "soft"
>>> voting_clf.named_estimators["svc"].probability = True
>>> voting_clf.fit(X_train, y_train)
>>> voting_clf.score(X_test, y_test)
0.92
```

Chúng ta đạt được độ chính xác 92% chỉ bằng cách sử dụng bỏ phiếu mềm—không tệ!

Túi hóa (Bagging) và Dán nhãn (Pasting)

Một cách để có được một tập hợp các bộ phân loại đa dạng là sử dụng các thuật toán huấn luyện rất khác nhau, như vừa thảo luận. Một cách tiếp cận khác là sử dụng cùng một thuật toán huấn luyện cho mỗi bộ dự đoán nhưng huấn luyện chúng trên các tập con ngẫu nhiên khác nhau của tập huấn luyện. Khi việc lấy mẫu được thực hiện **có thay thế** , phương pháp này được gọi là **túi hóa (bagging)** (viết tắt của bootstrap aggregating). Khi việc lấy mẫu được thực hiện **không thay thế**, nó được gọi là **dán nhãn (pasting)**.

Nói cách khác, cả túi hóa và dán nhãn đều cho phép các trường hợp huấn luyện được lấy mẫu nhiều lần trên nhiều bộ dự đoán, nhưng chỉ túi hóa mới cho phép các trường hợp huấn luyện được lấy mẫu nhiều lần cho cùng một bộ dự đoán. Quá trình lấy mẫu và huấn luyện này được biểu thị trong Hình 7-4.



Hình 7-4. Túi hóa và dán nhãn bao gồm việc huấn luyện nhiều bộ dự đoán trên các mẫu ngẫu nhiên khác nhau của tập huấn luyện

Sau khi tất cả các bộ dự đoán được huấn luyện, tập hợp có thể đưa ra dự đoán cho một trường hợp mới bằng cách đơn giản tổng hợp các dự đoán của tất cả các bộ dự đoán. Hàm tổng hợp thường là **chế độ thống kê (statistical mode)** cho phân loại (tức là dự đoán thường xuyên nhất, giống như với bộ phân loại bỏ phiếu cứng), hoặc **giá trị trung bình** cho hồi quy. Mỗi bộ dự đoán riêng lẻ có độ lệch cao hơn nếu nó được huấn luyện trên tập huấn luyện gốc, nhưng tổng hợp giúp giảm cả độ lệch và phương sai. Nói chung, kết quả cuối cùng là tập hợp có độ lệch tương tự nhưng phương sai thấp hơn so với một bộ dự đoán duy nhất được huấn luyện trên tập huấn luyện gốc.

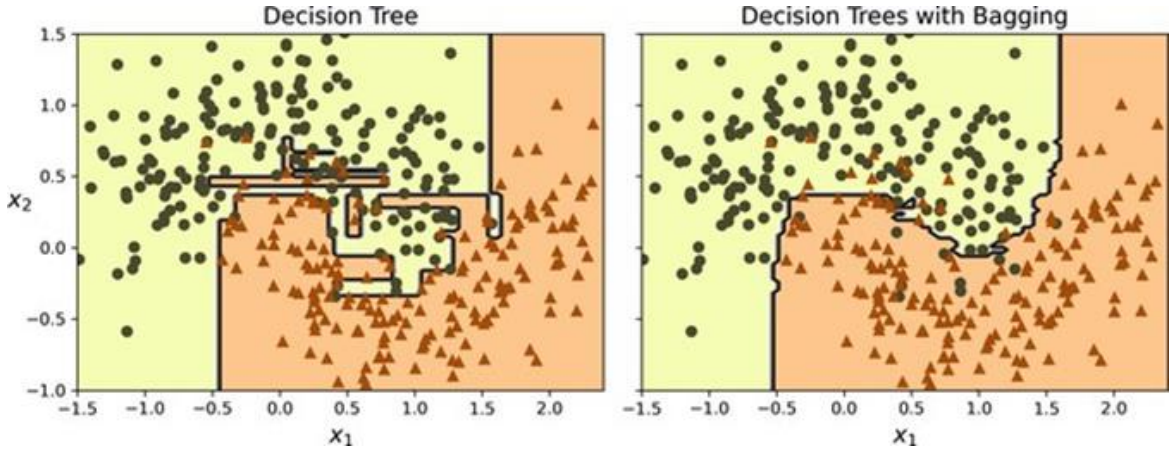
Như bạn có thể thấy trong Hình 7-4, các bộ dự đoán đều có thể được huấn luyện song song, thông qua các lõi CPU khác nhau hoặc thậm chí các máy chủ khác nhau. Tương tự, các dự đoán có thể được đưa ra song song. Đây là một trong những lý do tại sao túi hóa và dán nhãn là những phương pháp phổ biến như vậy: chúng mở rộng rất tốt.

Túi hóa và Dán nhãn trong Scikit-Learn

Scikit-Learn cung cấp một API đơn giản cho cả túi hóa và dán nhãn: lớp `BaggingClassifier` (hoặc `BaggingRegressor` cho hồi quy). Đoạn mã sau đây huấn luyện một tập hợp gồm 500 bộ phân loại cây quyết định: mỗi bộ được huấn luyện trên 100 trường hợp huấn luyện được lấy mẫu ngẫu nhiên từ tập huấn luyện có thay thế (đây là một ví dụ về túi hóa, nhưng nếu bạn muốn sử dụng dán nhãn thay vào đó, chỉ cần đặt `bootstrap=False`). Tham số `n_jobs` cho Scikit-Learn biết số lượng lõi CPU để sử dụng cho việc huấn luyện và dự đoán, và –1 cho Scikit-Learn biết sử dụng tất cả các lõi có sẵn:

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                            max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```

Hình 7-5 so sánh đường biên quyết định của một cây quyết định đơn lẻ với đường biên quyết định của một tập hợp túi hóa gồm 500 cây (từ đoạn mã trước), cả hai đều được huấn luyện trên tập dữ liệu moons. Như bạn có thể thấy, các dự đoán của tập hợp có khả năng tổng quát hóa tốt hơn nhiều so với dự đoán của cây quyết định đơn lẻ: tập hợp có độ lệch tương đương nhưng phương sai nhỏ hơn (nó mắc cùng số lỗi trên tập huấn luyện, nhưng đường biên quyết định ít bất thường hơn). Túi hóa đưa vào một chút đa dạng hơn trong các tập con mà mỗi bộ dự đoán được huấn luyện, vì vậy túi hóa cuối cùng có độ lệch cao hơn một chút so với dán nhãn; nhưng sự đa dạng bổ sung cũng có nghĩa là các bộ dự đoán cuối cùng ít tương quan hơn, do đó phương sai của tập hợp được giảm. Nhìn chung, túi hóa thường mang lại các mô hình tốt hơn, điều này giải thích tại sao nó thường được ưu tiên. Nhưng nếu bạn có thời gian rảnh và sức mạnh CPU, bạn có thể sử dụng xác thực chéo để đánh giá cả túi hóa và dán nhãn và chọn cái nào hoạt động tốt nhất.



Hình 7-5. Một cây quyết định đơn lẻ (trái) so với một tập hợp túi hóa gồm 500 cây (phải)

Đánh giá ngoài mẫu (Out-of-Bag Evaluation)

Với túi hóa, một số trường hợp huấn luyện có thể được lấy mẫu nhiều lần cho bất kỳ bộ dự đoán nhất định nào, trong khi những trường hợp khác có thể không được lấy mẫu chút nào. Theo mặc định, một `BaggingClassifier` lấy mẫu m trường hợp huấn luyện có thay thế (`bootstrap=True`), trong đó m là kích thước của tập huấn luyện. Với quy trình này, có thể chứng minh toán học rằng trung bình chỉ có khoảng 63% các trường hợp huấn luyện được lấy mẫu cho mỗi bộ dự đoán. 37% còn lại của các trường hợp huấn luyện không được lấy mẫu được gọi là các **trường hợp ngoài mẫu (out-of-bag - OOB)**. Lưu ý rằng chúng không phải là cùng 37% đối với tất cả các bộ dự đoán. Một tập hợp túi hóa có thể được đánh giá bằng cách sử dụng các trường hợp OOB, không cần một tập xác thực riêng biệt: thực tế, nếu có đủ các bộ ước lượng, thì mỗi trường hợp trong tập huấn luyện có thể sẽ là một trường hợp OOB của một số bộ ước lượng, vì vậy các bộ ước lượng này có thể được sử dụng để đưa ra một dự đoán tập hợp công bằng cho trường hợp đó. Một khi bạn có dự đoán cho mỗi trường hợp, bạn có thể tính toán độ chính xác dự đoán của tập hợp (hoặc bất kỳ số liệu nào khác). Trong Scikit-Learn, bạn có thể đặt `oob_score=True` khi tạo một `BaggingClassifier` để yêu cầu đánh giá OOB tự động sau khi huấn luyện. Đoạn mã sau đây minh họa điều này. Điểm đánh giá kết quả có sẵn trong thuộc tính `oob_score_`:

```
>>> bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
...                             oob_score=True, n_jobs=-1, random_state=42)
...
>>> bag_clf.fit(X_train, y_train)
>>> bag_clf.oob_score_
0.896
```

Theo đánh giá OOB này, `BaggingClassifier` này có khả năng đạt độ chính xác khoảng 89.6% trên tập kiểm tra. Hãy xác minh điều này:

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = bag_clf.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.92
```

Chúng ta đạt 92% độ chính xác trên tập kiểm tra. Đánh giá OOB hơi quá bi quan, thấp hơn hơn 2%. Hàm quyết định OOB cho mỗi trường hợp huấn luyện cũng có sẵn thông qua thuộc tính `oob_decision_function_`. Vì bộ ước lượng cơ sở có phương thức `predict_proba()`, hàm quyết định trả về xác suất lớp cho mỗi trường hợp huấn luyện. Ví dụ, đánh giá OOB ước tính rằng trường hợp huấn luyện đầu tiên có 67.6% xác suất thuộc về lớp dương và 32.4% xác suất thuộc về lớp âm:

```
>>> bag_clf.oob_decision_function_[:3] # probas for the first 3 instances
array([[0.32352941, 0.67647059],
       [0.3375    , 0.6625    ],
       [1.        , 0.        ]])
```

Random Patches và Random Subspaces

Lớp `BaggingClassifier` cũng hỗ trợ lấy mẫu các đặc trưng. Việc lấy mẫu được kiểm soát bởi hai siêu tham số: `max_features` và `bootstrap_features`. Chúng hoạt động tương tự như `max_samples` và `bootstrap`, nhưng là để lấy mẫu đặc trưng thay vì lấy mẫu trường hợp. Do đó, mỗi bộ dự đoán sẽ được huấn luyện trên một tập con ngẫu nhiên của các đặc trưng đầu vào. Kỹ thuật này đặc biệt hữu ích khi bạn đang xử lý các đầu vào có chiều cao (chẳng hạn như hình ảnh), vì nó có thể tăng tốc đáng kể quá trình huấn luyện. Lấy mẫu cả các trường hợp huấn luyện và đặc trưng được gọi là phương pháp **random patches**. Giữ tất cả các trường hợp huấn luyện (bằng cách đặt `bootstrap=False` và `max_samples=1.0`) nhưng lấy mẫu đặc trưng (bằng cách đặt `bootstrap_features` thành `True` và/hoặc `max_features` thành một giá trị nhỏ hơn 1.0) được gọi là phương pháp **random subspaces**.

Việc lấy mẫu đặc trưng dẫn đến sự đa dạng của bộ dự đoán thậm chí còn lớn hơn, đổi một chút độ lệch cao hơn lấy phương sai thấp hơn.

Rừng ngẫu nhiên

Như chúng ta đã thảo luận, **rừng ngẫu nhiên** là một tập hợp các cây quyết định, thường được huấn luyện thông qua phương pháp túi hóa (hoặc đôi khi là dán nhãn), thường với `max_samples` được đặt bằng kích thước của tập huấn luyện. Thay vì xây dựng một `BaggingClassifier` và truyền cho nó một `DecisionTreeClassifier`, bạn có thể sử dụng lớp `RandomForestClassifier`, tiện lợi hơn và được tối ưu hóa cho cây quyết định (tương tự, có một lớp `RandomForestRegressor` cho các tác vụ hồi quy). Đoạn mã sau đây huấn luyện một bộ phân loại rừng ngẫu nhiên với 500 cây, mỗi cây giới hạn tối đa 16 nút lá, sử dụng tất cả các lõi CPU có sẵn:

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                 n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
y_pred_rf = rnd_clf.predict(X_test)
```

Với một vài ngoại lệ, một `RandomForestClassifier` có tất cả các siêu tham số của một `DecisionTreeClassifier` (để kiểm soát cách cây được phát triển), cộng với tất cả các siêu

tham số của một `BaggingClassifier` để kiểm soát chính tập hợp. Thuật toán rừng ngẫu nhiên đưa thêm sự ngẫu nhiên khi phát triển cây; thay vì tìm kiếm đặc trưng tốt nhất khi chia một nút (xem Chương 6), nó tìm kiếm đặc trưng tốt nhất trong một tập con ngẫu nhiên của các đặc trưng. Theo mặc định, nó lấy mẫu $sqrtn$ đặc trưng (trong đó n là tổng số đặc trưng). Thuật toán dẫn đến sự đa dạng cây lớn hơn, điều này (một lần nữa) đổi một độ lệch cao hơn lấy phương sai thấp hơn, thường mang lại một mô hình tổng thể tốt hơn. Vì vậy, `BaggingClassifier` sau đây tương đương với `RandomForestClassifier` trước đó:

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, n_jobs=-1, random_state=42
)
```

Cây ngẫu nhiên cực đại (Extra-Trees)

Khi bạn phát triển một cây trong rừng ngẫu nhiên, tại mỗi nút chỉ một tập con ngẫu nhiên của các đặc trưng được xem xét để phân chia (như đã thảo luận trước đó). Có thể làm cho cây thậm chí còn ngẫu nhiên hơn bằng cách cũng sử dụng các ngưỡng ngẫu nhiên cho mỗi đặc trưng thay vì tìm kiếm các ngưỡng tốt nhất có thể (như các cây quyết định thông thường làm). Để làm điều này, chỉ cần đặt `splitter="random"` khi tạo một `DecisionTreeClassifier`. Một rừng các cây cực kỳ ngẫu nhiên như vậy được gọi là tập hợp **cây ngẫu nhiên cực đại** (hoặc viết tắt là extra-trees). Một lần nữa, kỹ thuật này đổi nhiều độ lệch hơn lấy phương sai thấp hơn. Nó cũng làm cho các bộ phân loại extra-trees nhanh hơn nhiều để huấn luyện so với rừng ngẫu nhiên thông thường, bởi vì việc tìm kiếm ngưỡng tốt nhất có thể cho mỗi đặc trưng tại mỗi nút là một trong những tác vụ tốn thời gian nhất của việc phát triển cây.

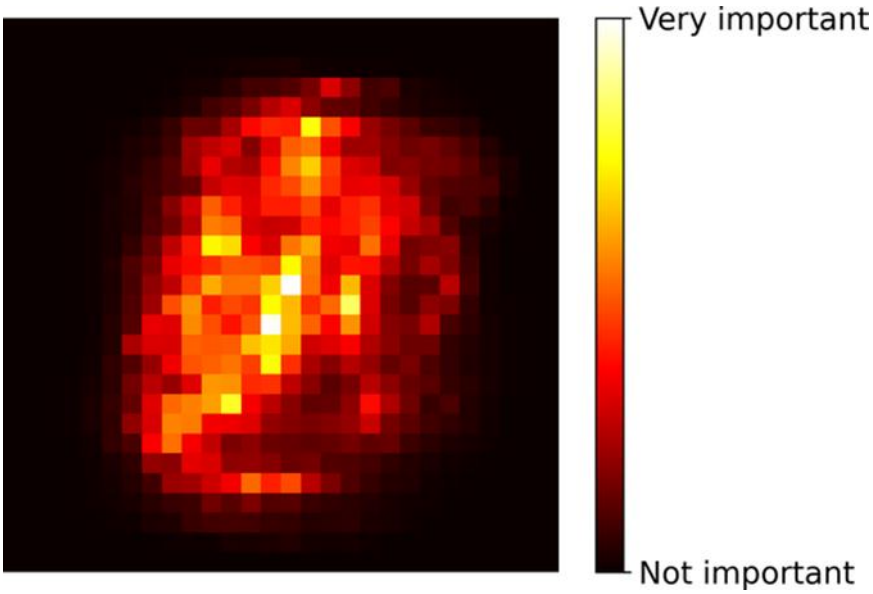
Bạn có thể tạo một bộ phân loại extra-trees bằng cách sử dụng lớp `ExtraTreesClassifier` của Scikit-Learn. API của nó giống hệt với lớp `RandomForestClassifier`, ngoại trừ `bootstrap` mặc định là `False`. Tương tự, lớp `ExtraTreesRegressor` có cùng API với lớp `RandomForestRegressor`, ngoại trừ `bootstrap` mặc định là `False`.

Tầm quan trọng của đặc trưng

Một phẩm chất tuyệt vời khác của rừng ngẫu nhiên là chúng giúp dễ dàng đo lường tầm quan trọng tương đối của mỗi đặc trưng. Scikit-Learn đo tầm quan trọng của một đặc trưng bằng cách xem xét mức độ các nút cây sử dụng đặc trưng đó làm giảm độ không tinh khiết trung bình, trên tất cả các cây trong rừng. Chính xác hơn, đó là một trung bình có trọng số, trong đó trọng số của mỗi nút bằng số lượng mẫu huấn luyện liên quan đến nó (xem Chương 6). Scikit-Learn tự động tính toán điểm này cho mỗi đặc trưng sau khi huấn luyện, sau đó nó chuẩn hóa kết quả để tổng tất cả các độ quan trọng bằng 1. Bạn có thể truy cập kết quả bằng biến `feature_importances_`. Ví dụ, đoạn mã sau đây huấn luyện một `RandomForestClassifier` trên tập dữ liệu iris (được giới thiệu trong Chương 4) và xuất ra tầm quan trọng của mỗi đặc trưng. Dường như các đặc trưng quan trọng nhất là chiều dài cánh hoa (44%) và chiều rộng (42%), trong khi chiều dài và chiều rộng đài hoa khá không quan trọng so với (lần lượt là 11% và 2%):

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
>>> rnd_clf.fit(iris.data, iris.target)

>>> for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
...     print(round(score, 2), name)
...
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm)
0.42 petal width (cm)
```

Tương tự, nếu bạn huấn luyện một bộ phân loại rừng ngẫu nhiên trên tập dữ liệu MNIST (được giới thiệu trong Chương 3) và vẽ tầm quan trọng của mỗi pixel, bạn sẽ nhận được hình ảnh được biểu thị trong Hình 7-6.



Hình 7-6. Tầm quan trọng của pixel MNIST (theo bộ phân loại rừng ngẫu nhiên)

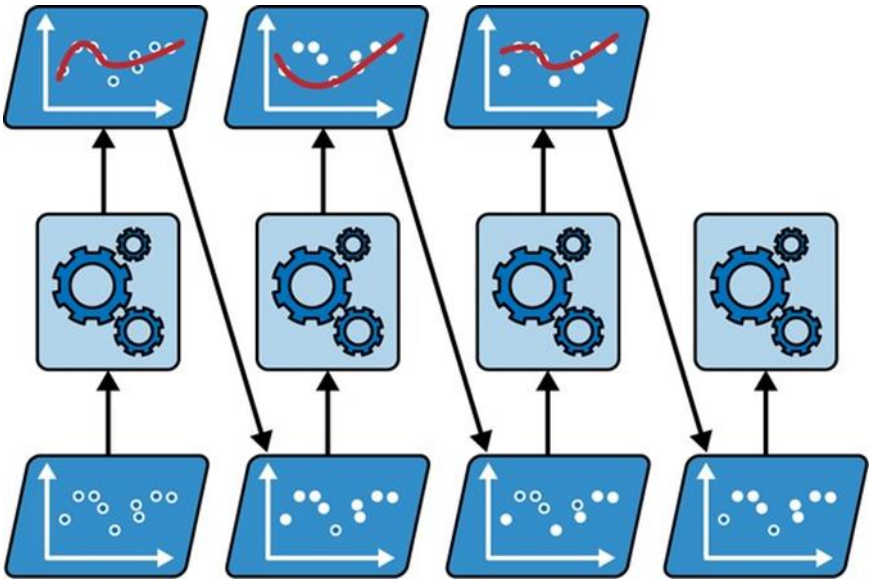
Rừng ngẫu nhiên rất tiện lợi để nhanh chóng hiểu được những đặc trưng nào thực sự quan trọng, đặc biệt nếu bạn cần thực hiện lựa chọn đặc trưng.

Boosting

Boosting (ban đầu được gọi là **tăng cường giả thuyết**) đề cập đến bất kỳ phương pháp tổ hợp nào có thể kết hợp nhiều bộ học yếu thành một bộ học mạnh. Ý tưởng chung của hầu hết các phương pháp boosting là huấn luyện các bộ dự đoán theo trình tự, mỗi bộ cố gắng sửa lỗi của bộ tiền nhiệm. Có nhiều phương pháp boosting có sẵn, nhưng cho đến nay phổ biến nhất là AdaBoost (viết tắt của adaptive boosting) và gradient boosting. Hãy bắt đầu với AdaBoost.

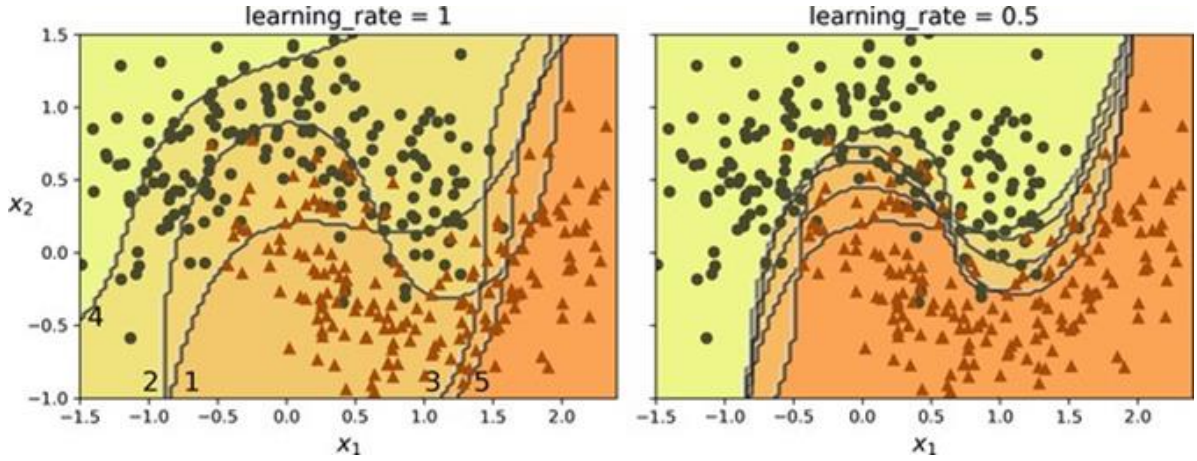
AdaBoost

Một cách để một bộ dự đoán mới điều chỉnh bộ dự đoán tiền nhiệm của nó là tập trung hơn một chút vào các trường hợp huấn luyện mà bộ dự đoán tiền nhiệm đã dưới khớp. Điều này dẫn đến việc các bộ dự đoán mới ngày càng tập trung vào các trường hợp khó. Đây là kỹ thuật được sử dụng bởi AdaBoost. Ví dụ, khi huấn luyện một bộ phân loại AdaBoost, thuật toán đầu tiên huấn luyện một bộ phân loại cơ sở (chẳng hạn như cây quyết định) và sử dụng nó để đưa ra dự đoán trên tập huấn luyện. Sau đó, thuật toán tăng trọng số tương đối của các trường hợp huấn luyện bị phân loại sai. Sau đó, nó huấn luyện một bộ phân loại thứ hai, sử dụng các trọng số đã cập nhật, và lại đưa ra dự đoán trên tập huấn luyện, cập nhật trọng số trường hợp, v.v. (xem Hình 7-7). Hình 7-8 cho thấy các đường biên quyết định của năm bộ dự đoán liên tiếp trên tập dữ liệu moons (trong ví dụ này, mỗi bộ dự đoán là một bộ phân loại SVM được chính quy hóa cao với kernel RBF). Bộ phân loại đầu tiên mắc lỗi với nhiều trường hợp, vì vậy trọng số của chúng được tăng cường. Bộ phân loại thứ hai do đó làm tốt hơn trên các trường hợp này, và cứ thế tiếp tục. Biểu đồ bên phải thể hiện cùng một chuỗi các bộ dự đoán, ngoại trừ tốc độ học bị giảm một nửa (tức là trọng số của trường hợp bị phân loại sai được tăng cường ít hơn nhiều ở mỗi lần lặp). Như bạn có thể thấy, kỹ thuật học tuần tự này có một số điểm tương đồng với giảm độ dốc, ngoại trừ việc thay vì điều chỉnh các tham số của một bộ dự đoán duy nhất để tối thiểu hóa hàm chi phí, AdaBoost thêm các bộ dự đoán vào tập hợp, dần dần làm cho nó tốt hơn.



Hình 7-7. Huấn luyện AdaBoost tuần tự với các cập nhật trọng số trường hợp

Sau khi tất cả các bộ dự đoán được huấn luyện, tập hợp đưa ra dự đoán rất giống với túi hóa hoặc dán nhãn, ngoại trừ việc các bộ dự đoán có trọng số khác nhau tùy thuộc vào độ chính xác tổng thể của chúng trên tập huấn luyện có trọng số.



Hình 7-8. Đường biên quyết định của các bộ dự đoán liên tiếp

Hãy xem xét kỹ hơn thuật toán AdaBoost. Mỗi trọng số trường hợp $w^{(i)}$ ban đầu được đặt thành $1/m$. Một bộ dự đoán đầu tiên được huấn luyện, và tỷ lệ lỗi có trọng số r_j của nó được tính toán trên tập huấn luyện; xem Phương trình 7-1.

Phương trình 7-1. Tỷ lệ lỗi có trọng số của bộ dự đoán thứ j

$$r_j = \sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)} \quad \text{where } \hat{y}_j^{(i)} \text{ is the } j^{\text{th}} \text{ predictor's prediction for the } i^{\text{th}} \text{ instance}$$

Trọng số $alpha_j$ của bộ dự đoán sau đó được tính toán bằng Phương trình 7-2, trong đó eta là siêu tham số tốc độ học (mặc định là 1). Bộ dự đoán càng chính xác, trọng số của nó càng cao. Nếu nó chỉ đoán ngẫu nhiên, thì trọng số của nó sẽ gần bằng không. Tuy nhiên, nếu nó thường xuyên sai (tức là kém chính xác hơn đoán ngẫu nhiên), thì trọng số của nó sẽ là âm.

Phương trình 7-2. Trọng số bộ dự đoán

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

Tiếp theo, thuật toán AdaBoost cập nhật trọng số trường hợp, sử dụng Phương trình 7-3, làm tăng trọng số của các trường hợp bị phân loại sai. Phương trình 7-3. Quy tắc cập nhật trọng số cho $i = 1,2,..,m$

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

Sau đó tất cả các trọng số trường hợp được chuẩn hóa (tức là chia cho $\sum_{i=1}^{m} w^{(i)}$).

Cuối cùng, một bộ dự đoán mới được huấn luyện bằng cách sử dụng các trọng số đã cập nhật, và toàn bộ quá trình được lặp lại: trọng số của bộ dự đoán mới được tính toán, trọng số trường hợp được cập nhật, sau đó một bộ dự đoán khác được huấn luyện, v.v. Thuật toán dừng lại khi đạt được số lượng bộ dự đoán mong muốn, hoặc khi tìm thấy một bộ dự đoán hoàn hảo. Để đưa ra dự đoán, AdaBoost chỉ đơn giản tính toán các dự đoán của tất cả các bộ dự đoán và trọng số chúng bằng cách sử dụng các trọng số bộ dự đoán $alpha_j$. Lớp dự đoán là lớp nhận được đa số phiếu có trọng số (xem Phương trình 7-4). Phương trình 7-4. Dự đoán của AdaBoost

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x})=k}}^{N} \alpha_j$$

Trong đó N là số lượng bộ dự đoán.

Scikit-Learn sử dụng một phiên bản đa lớp của AdaBoost được gọi là SAMME (viết tắt của Stagewise Additive Modeling using a Multiclass Exponential loss function). Khi chỉ có hai lớp, SAMME tương đương với AdaBoost. Nếu các bộ dự đoán có thể ước tính xác suất lớp (tức là nếu chúng có phương thức `predict_proba()`), Scikit-Learn có thể sử dụng một biến thể của SAMME được gọi là SAMME.R (chữ R là viết tắt của “Real”), dựa vào xác suất lớp thay vì dự đoán và thường hoạt động tốt hơn. Đoạn mã sau đây huấn luyện một bộ phân loại AdaBoost dựa trên 30 cây quyết định một nút (decision stumps) bằng cách sử dụng lớp `AdaBoostClassifier` của Scikit-Learn (như bạn có thể mong đợi, cũng có một lớp `AdaBoostRegressor`). Một cây quyết định một nút là một cây quyết định với `max_depth=1`—nói cách khác, một cây bao gồm một nút quyết định duy nhất cộng với hai nút lá. Đây là bộ ước lượng cơ sở mặc định cho lớp `AdaBoostClassifier`:

```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42
)
ada_clf.fit(X_train, y_train)
```

Gradient Boosting

Một thuật toán boosting rất phổ biến khác là gradient boosting. Giống như AdaBoost, gradient boosting hoạt động bằng cách tuần tự thêm các bộ dự đoán vào một tập hợp, mỗi bộ sửa lỗi của bộ tiền nhiệm. Tuy nhiên, thay vì điều chỉnh trọng số trường hợp ở mỗi lần lặp như AdaBoost, phương pháp này cố gắng khớp bộ dự đoán mới với các lỗi còn lại do bộ dự đoán trước đó tạo ra.

Hãy xem xét một ví dụ hồi quy đơn giản, sử dụng cây quyết định làm bộ dự đoán cơ sở; điều này được gọi là **tăng cường cây gradient** (gradient tree boosting), hoặc **cây hồi quy tăng**

cường gradient (GBRT). Đầu tiên, hãy tạo một tập dữ liệu bậc hai nhiễu và khớp một `DecisionTreeRegressor` với nó:

```
DecisionTreeRegressor

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100) # y = 3x² + nhiễu Gaussian

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

Tiếp theo, chúng ta sẽ huấn luyện một `DecisionTreeRegressor` thứ hai trên các lỗi còn lại do bộ dự đoán đầu tiên tạo ra:

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)
```

Và sau đó chúng ta sẽ huấn luyện một bộ hồi quy thứ ba trên các lỗi còn lại do bộ dự đoán thứ hai tạo ra:

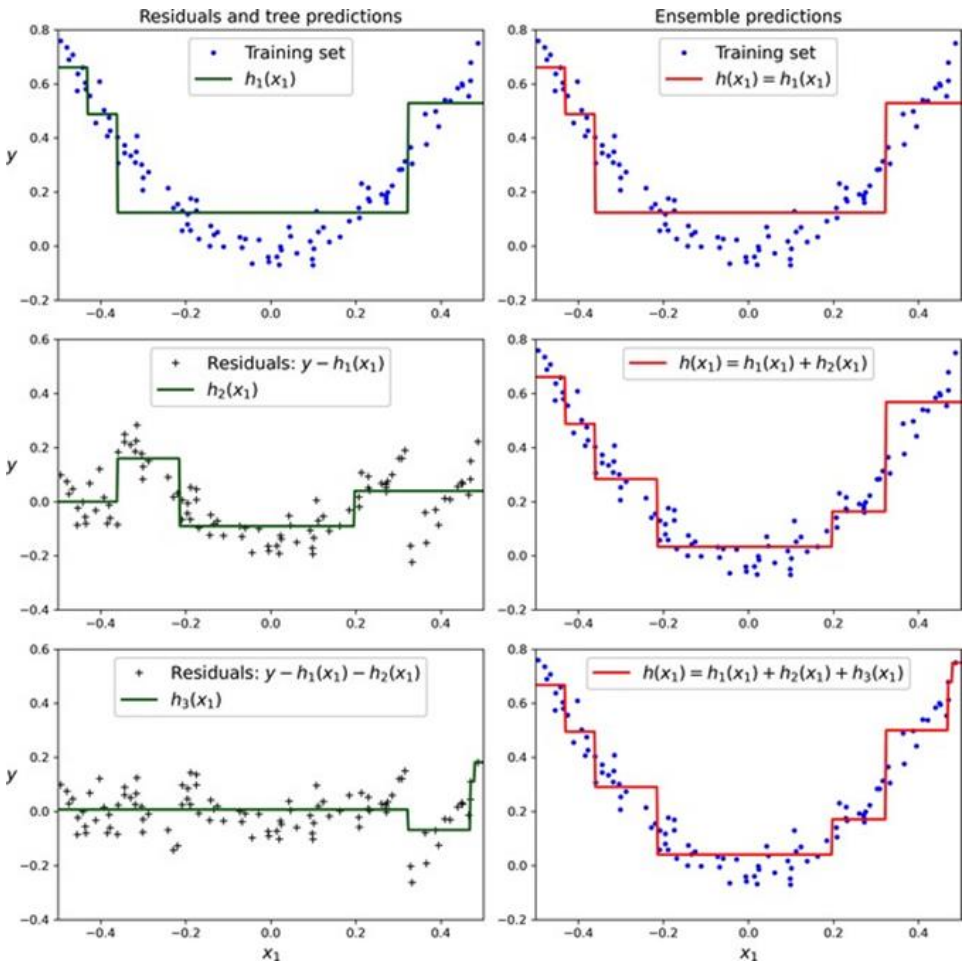
```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)
```

Bây giờ chúng ta có một tập hợp chứa ba cây. Nó có thể đưa ra dự đoán trên một trường hợp mới chỉ bằng cách cộng các dự đoán của tất cả các cây:

```
>>> X_new = np.array([[-0.4], [0.], [0.5]])

>>> sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
array([0.49484029, 0.04021166, 0.75026781])
```

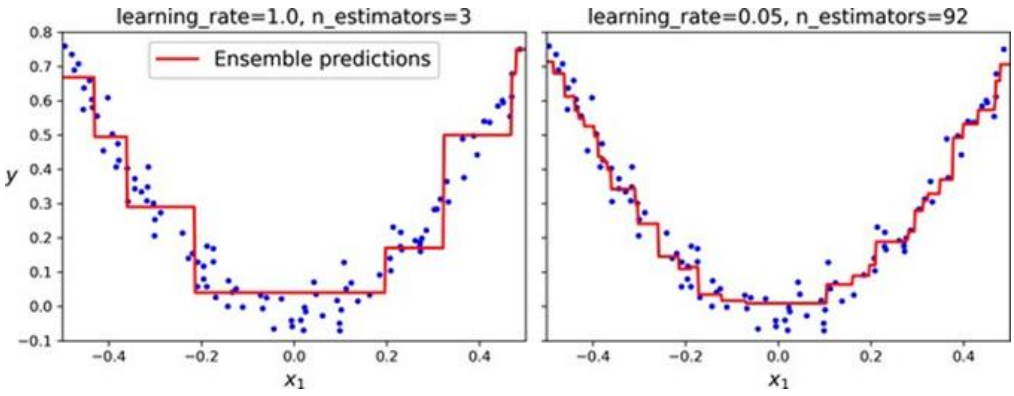
Hình 7-9 biểu thị các dự đoán của ba cây này ở cột bên trái, và các dự đoán của tập hợp ở cột bên phải. Trong hàng đầu tiên, tập hợp chỉ có một cây, vì vậy các dự đoán của nó chính xác là các dự đoán của cây đầu tiên. Trong hàng thứ hai, một cây mới được huấn luyện trên các lỗi còn lại của cây đầu tiên. Ở bên phải, bạn có thể thấy rằng các dự đoán của tập hợp bằng tổng các dự đoán của hai cây đầu tiên. Tương tự, trong hàng thứ ba, một cây khác được huấn luyện trên các lỗi còn lại của cây thứ hai. Bạn có thể thấy rằng các dự đoán của tập hợp dần dần tốt hơn khi các cây được thêm vào tập hợp. Bạn có thể sử dụng lớp `GradientBoostingRegressor` của Scikit-Learn để huấn luyện các tập hợp GBRT dễ dàng hơn (cũng có một lớp `GradientBoostingClassifier` cho phân loại). Giống như lớp `RandomForestRegressor`, nó có các siêu tham số để kiểm soát sự phát triển của cây quyết định (ví dụ: `max_depth`, `min_samples_leaf`), cũng như các siêu tham số để kiểm soát việc huấn luyện tập hợp, chẳng hạn như số lượng cây (`n_estimators`). Đoạn mã sau đây tạo ra cùng một tập hợp như đoạn mã trước đó:

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                 learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```



Hình 7-9. Trong mô tả này về gradient boosting, bộ dự đoán đầu tiên (trên cùng bên trái) được huấn luyện bình thường, sau đó mỗi bộ dự đoán liên tiếp (giữa bên trái và dưới bên trái) được huấn luyện trên các lỗi còn lại của bộ dự đoán trước đó; cột bên phải hiển thị các dự đoán của tập hợp kết quả

Siêu tham số `learning_rate` điều chỉnh sự đóng góp của mỗi cây. Nếu bạn đặt nó thành một giá trị thấp, chẳng hạn như 0.05, bạn sẽ cần nhiều cây hơn trong tập hợp để khớp tập huấn luyện, nhưng các dự đoán thường sẽ tổng quát hóa tốt hơn. Đây là một kỹ thuật chính quy hóa được gọi là **shrinkage**. Hình 7-10 cho thấy hai tập hợp GBRT được huấn luyện với các siêu tham số khác nhau: tập hợp bên trái không có đủ cây để khớp tập huấn luyện, trong khi tập hợp bên phải có số lượng cây vừa đủ. Nếu chúng ta thêm nhiều cây hơn, GBRT sẽ bắt đầu quá khớp tập huấn luyện.



Hình 7-10. Các tập hợp GBRT với không đủ bộ dự đoán (trái) và vừa đủ (phải)

Để tìm số lượng cây tối ưu, bạn có thể thực hiện xác thực chéo bằng cách sử dụng `GridSearchCV` hoặc `RandomizedSearchCV`, như thường lệ, nhưng có một cách đơn giản hơn: nếu bạn đặt siêu tham số `n_iter_no_change` thành một giá trị số nguyên, chẳng hạn 10, thì `GradientBoostingRegressor` sẽ tự động ngừng thêm cây trong quá trình huấn luyện nếu nó thấy rằng 10 cây cuối cùng không giúp ích. Đây đơn giản là dừng sớm (được giới thiệu trong Chương 4), nhưng với một chút kiên nhẫn: nó chịu đựng việc không có tiến triển trong một vài lần lặp trước khi dừng lại. Hãy huấn luyện tập hợp bằng cách sử dụng dừng sớm:

```
gbrt_best = GradientBoostingRegressor(
    max_depth=2, learning_rate=0.05, n_estimators=500,
    n_iter_no_change=10, random_state=42
)
gbrt_best.fit(X, y)
```

Nếu bạn đặt `n_iter_no_change` quá thấp, quá trình huấn luyện có thể dừng quá sớm và mô hình sẽ bị dưới khớp. Nhưng nếu bạn đặt nó quá cao, nó sẽ quá khớp thay vào đó. Chúng tôi cũng đặt tốc độ học khá nhỏ và số lượng bộ ước lượng cao, nhưng số lượng bộ ước lượng thực tế trong tập hợp đã huấn luyện thấp hơn nhiều, nhờ vào dừng sớm:

```
>>> gbrt_best.n_estimators_
92
```

Khi `n_iter_no_change` được đặt, phương thức `fit()` tự động chia tập huấn luyện thành một tập huấn luyện nhỏ hơn và một tập xác thực: điều này cho phép nó đánh giá hiệu suất của mô hình mỗi khi nó thêm một cây mới. Kích thước của tập xác thực được kiểm soát bởi siêu tham số `validation_fraction`, mặc định là 10%. Siêu tham số `tol` xác định cải thiện hiệu suất tối đa vẫn được coi là không đáng kể. Nó mặc định là 0.0001. Lớp `GradientBoostingRegressor` cũng hỗ trợ siêu tham số `subsample`, chỉ định tỷ lệ các trường hợp huấn luyện được sử dụng để huấn luyện mỗi cây. Ví dụ, nếu `subsample=0.25`, thì mỗi cây được huấn luyện trên 25% các trường hợp huấn luyện, được chọn ngẫu nhiên. Như bạn có thể đoán được, kỹ thuật này đổi một độ lệch cao hơn lấy phương sai thấp hơn. Nó cũng tăng tốc đáng kể quá trình huấn luyện. Đây được gọi là **tăng cường gradient ngẫu nhiên (stochastic gradient boosting)**.

Tăng cường Gradient dựa trên Biểu đồ (Histogram-Based Gradient Boosting)

Scikit-Learn cũng cung cấp một triển khai GBRT khác, được tối ưu hóa cho các tập dữ liệu lớn: tăng cường gradient dựa trên biểu đồ (HGB). Nó hoạt động bằng cách nhóm các đặc trưng đầu vào lại, thay thế chúng bằng các số nguyên. Số lượng nhóm được kiểm soát bởi siêu tham số `max_bins`, mặc định là 255 và không thể đặt cao hơn giá trị này. Việc nhóm có thể giảm đáng kể số lượng ngưỡng khả thi mà thuật toán huấn luyện cần đánh giá. Hơn nữa, làm việc với các số nguyên giúp sử dụng các cấu trúc dữ liệu nhanh hơn và hiệu quả bộ nhớ hơn. Và cách các nhóm được xây dựng giúp loại bỏ nhu cầu sắp xếp các đặc trưng khi huấn luyện mỗi cây.

Kết quả là, triển khai này có độ phức tạp tính toán là $O(btimesm)$ thay vì $O\left(ntimesmtimeslog(m)\right)$, trong đó b là số lượng nhóm, m là số lượng trường hợp huấn luyện, và n là số lượng đặc trưng. Trong thực tế, điều này có nghĩa là HGB có thể huấn luyện nhanh hơn hàng trăm lần so với GBRT thông thường trên các tập dữ liệu lớn. Tuy nhiên, việc nhóm gây ra mất mát độ chính xác, hoạt động như một bộ chính quy hóa: tùy thuộc vào tập dữ liệu, điều này có thể giúp giảm quá khớp, hoặc có thể gây dưới khớp.

Scikit-Learn cung cấp hai lớp cho HGB: `HistGradientBoostingRegressor` và `HistGradientBoostingClassifier`. Chúng tương tự như `GradientBoostingRegressor` và `GradientBoostingClassifier`, với một vài khác biệt đáng chú ý:

- Dừng sớm được tự động kích hoạt nếu số lượng trường hợp lớn hơn 10.000. Bạn có thể bật hoặc tắt dừng sớm bằng cách đặt siêu tham số `early_stopping` thành `True` hoặc `False`.
- Lấy mẫu phụ (subsampling) không được hỗ trợ.
- `n_estimators` được đổi tên thành `max_iter`.
- Các siêu tham số cây quyết định duy nhất có thể được điều chỉnh là `max_leaf_nodes`, `min_samples_leaf` và `max_depth`.

Các lớp HGB cũng có hai tính năng hay: chúng hỗ trợ cả các đặc trưng phân loại và các giá trị bị thiếu. Điều này giúp đơn giản hóa đáng kể việc tiền xử lý. Tuy nhiên, các đặc trưng phân loại phải được biểu diễn dưới dạng số nguyên từ 0 đến một số nhỏ hơn `max_bins`. Bạn có thể sử dụng `OrdinalEncoder` cho việc này. Ví dụ, đây là cách xây dựng và huấn luyện một pipeline hoàn chỉnh cho tập dữ liệu California housing được giới thiệu trong Chương 2:

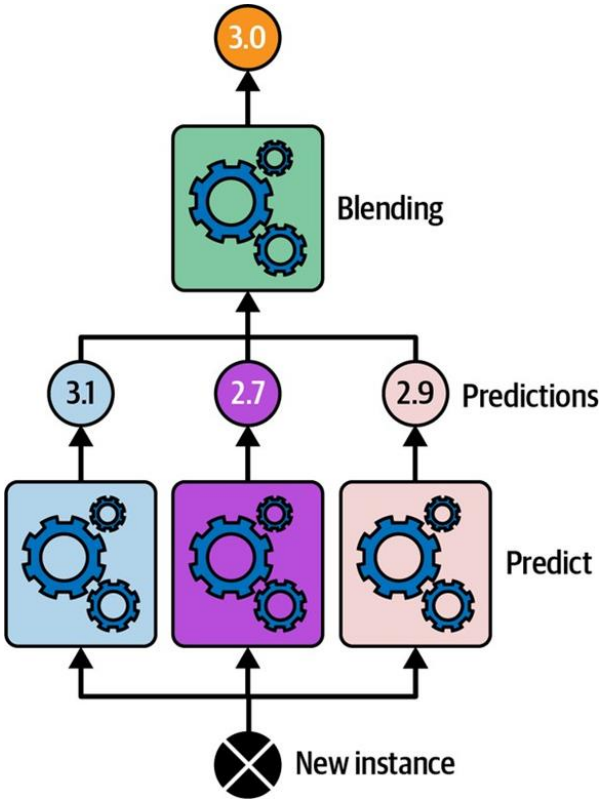
```
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.preprocessing import OrdinalEncoder

hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
                            remainder="passthrough"),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
)
hgb_reg.fit(housing, housing_labels)
```

Toàn bộ pipeline chỉ ngắn gọn như các lệnh import! Không cần bộ điền thiếu (imputer), bộ chuẩn hóa (scaler) hoặc bộ mã hóa một-nóng (one-hot encoder), vì vậy nó thực sự tiện lợi. Lưu ý rằng `categorical_features` phải được đặt thành các chỉ số cột phân loại (hoặc một mảng Boolean). Không cần điều chỉnh siêu tham số, mô hình này cho RMSE khoảng 47.600, không quá tệ.

Phân lớp xếp chồng (Stacking)

Phương pháp tập hợp cuối cùng chúng ta sẽ thảo luận trong chương này được gọi là

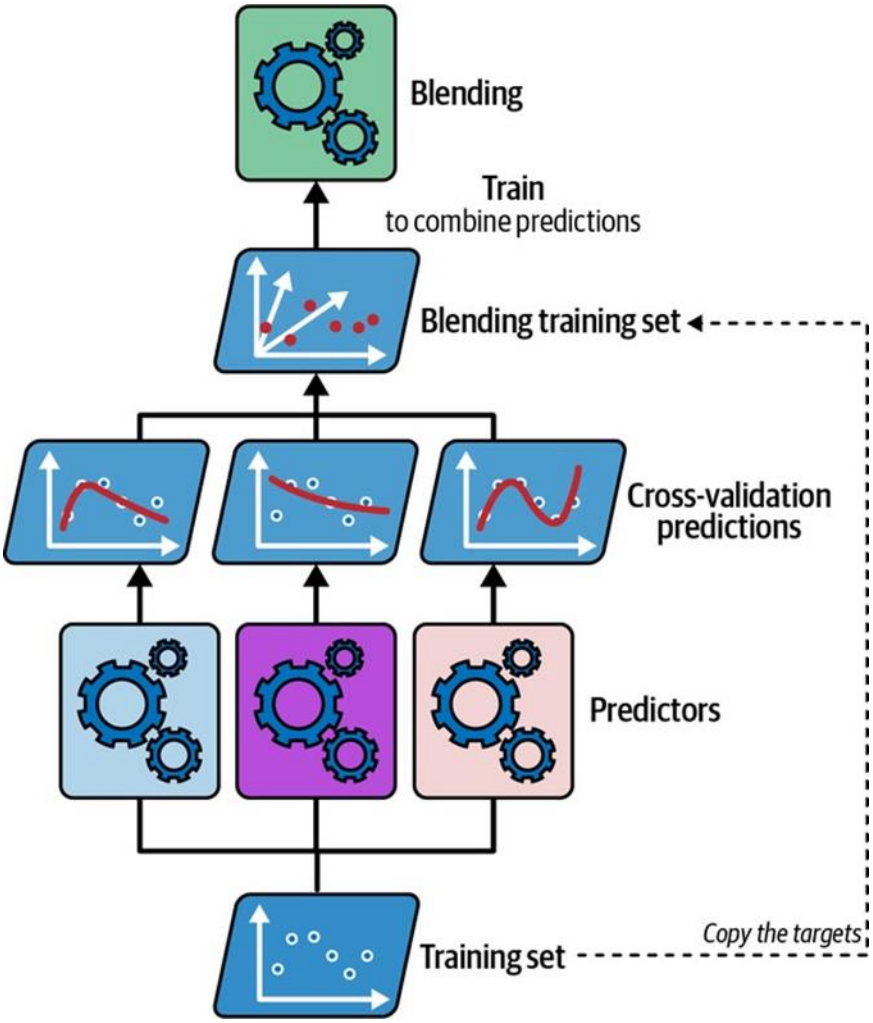
phân lớp xếp chồng (stacking) (viết tắt của stacked generalization). Nó dựa trên một ý tưởng đơn giản: thay vì sử dụng các hàm tầm thường (chẳng hạn như bỏ phiếu cứng) để tổng hợp các dự đoán của tất cả các bộ dự đoán trong một tập hợp, tại sao chúng ta không huấn luyện một mô hình để thực hiện việc tổng hợp này? Hình 7-11 cho thấy một tập hợp như vậy thực hiện một tác vụ hồi quy trên một trường hợp mới. Mỗi trong ba bộ dự đoán phía dưới dự đoán một giá trị khác nhau (3.1, 2.7 và 2.9), và sau đó bộ dự đoán cuối cùng (được gọi là **bộ trộn (blender)**, hoặc **meta learner**) lấy các dự đoán này làm đầu vào và đưa ra dự đoán cuối cùng (3.0).



Hình 7-11. Tổng hợp các dự đoán bằng cách sử dụng bộ dự đoán pha trộn

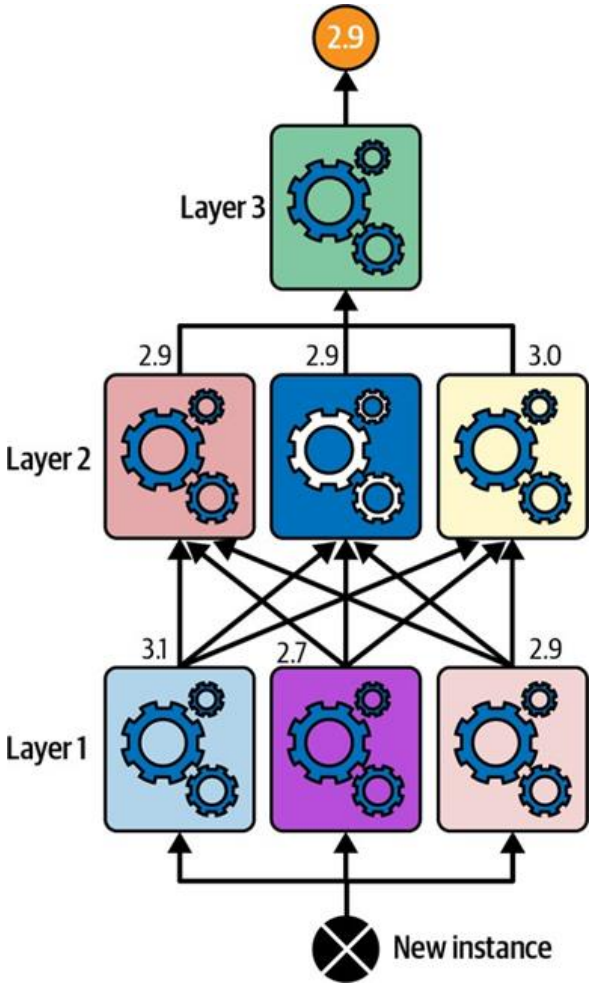
Để huấn luyện bộ trộn, trước tiên bạn cần xây dựng tập huấn luyện pha trộn. Bạn có thể sử dụng `cross_val_predict()` trên mỗi bộ dự đoán trong tập hợp để nhận các dự đoán ngoài mẫu (out-of-sample predictions) cho mỗi trường hợp trong tập huấn luyện gốc (Hình 7-12), và sử dụng các dự đoán này làm đặc trưng đầu vào để huấn luyện bộ trộn; và các mục tiêu

có thể đơn giản được sao chép từ tập huấn luyện gốc. Lưu ý rằng bất kể số lượng đặc trưng trong tập huấn luyện gốc (chỉ một trong ví dụ này), tập huấn luyện pha trộn sẽ chứa một đặc trưng đầu vào cho mỗi bộ dự đoán (ba trong ví dụ này). Một khi bộ trộn được huấn luyện, các bộ dự đoán cơ sở sẽ được huấn luyện lại lần cuối trên toàn bộ tập huấn luyện gốc.



Hình 7-12. Huấn luyện bộ trộn trong một tập hợp xếp chồng

Thực tế có thể huấn luyện một số bộ trộn khác nhau theo cách này (ví dụ: một bộ sử dụng hồi quy tuyến tính, một bộ khác sử dụng hồi quy rừng ngẫu nhiên) để có được toàn bộ một lớp các bộ trộn, và sau đó thêm một bộ trộn khác lên trên đó để tạo ra dự đoán cuối cùng, như trong Hình 7-13. Bạn có thể tận dụng thêm một chút hiệu suất bằng cách này, nhưng nó sẽ tốn kém cả về thời gian huấn luyện và độ phức tạp của hệ thống.



Hình 7-13. Dự đoán trong một tập hợp xếp chồng đa lớp

Scikit-Learn cung cấp hai lớp cho các tập hợp xếp chồng: `StackingClassifier` và `StackingRegressor`. Ví dụ, chúng ta có thể thay thế `VotingClassifier` mà chúng ta đã sử dụng ở đầu chương này trên tập dữ liệu moons bằng một `StackingClassifier`:

```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # số lần cross-validation
)
stacking_clf.fit(X_train, y_train)
```

Đối với mỗi bộ dự đoán, bộ phân loại xếp chồng sẽ gọi `predict_proba()` nếu có; nếu không, nó sẽ quay lại `decision_function()` hoặc, là phương án cuối cùng, gọi `predict()`.

Nếu bạn không cung cấp một bộ ước lượng cuối cùng, `StackingClassifier` sẽ sử dụng `LogisticRegression` và `StackingRegressor` sẽ sử dụng `RidgeCV`. Nếu bạn đánh giá mô hình xếp chồng này trên tập kiểm tra, bạn sẽ thấy độ chính xác 92.8%, tốt hơn một chút so với bộ phân loại bỏ phiếu sử dụng bỏ phiếu mềm, đã đạt 92%.

Tóm lại, các phương pháp tập hợp rất linh hoạt, mạnh mẽ và khá đơn giản để sử dụng. Rừng ngẫu nhiên, AdaBoost và GBRT là một trong những mô hình đầu tiên bạn nên thử cho hầu hết các tác vụ học máy, và chúng đặc biệt hiệu quả với dữ liệu dạng bảng không đồng nhất. Hơn nữa, vì chúng yêu cầu rất ít tiền xử lý, chúng rất tuyệt vời để nhanh chóng xây dựng và chạy một nguyên mẫu. Cuối cùng, các phương pháp tập hợp như bộ phân loại bỏ phiếu và bộ phân loại xếp chồng có thể giúp đẩy hiệu suất hệ thống của bạn đến giới hạn.

Bài tập

1. Nếu bạn đã huấn luyện năm mô hình khác nhau trên cùng một dữ liệu huấn luyện, và tất cả chúng đều đạt độ chính xác 95%, liệu có cơ hội nào bạn có thể kết hợp các mô hình này để có được kết quả tốt hơn không? Nếu có, bằng cách nào? Nếu không, tại sao?
2. Sự khác biệt giữa bộ phân loại bỏ phiếu cứng và bỏ phiếu mềm là gì?
3. Có thể tăng tốc quá trình huấn luyện của một tập hợp bagging bằng cách phân tán nó trên nhiều máy chủ không? Còn các tập hợp pasting, boosting, random forests, hay stacking thì sao?
4. Lợi ích của đánh giá out-of-bag là gì?
5. Điều gì làm cho các tập hợp extra-trees ngẫu nhiên hơn rừng ngẫu nhiên thông thường? Sự ngẫu nhiên bổ sung này có thể giúp ích như thế nào? Bộ phân loại extra-trees chậm hơn hay nhanh hơn rừng ngẫu nhiên thông thường?
6. Nếu tập hợp AdaBoost của bạn dưới khớp dữ liệu huấn luyện, bạn nên điều chỉnh siêu tham số nào và như thế nào?
7. Nếu tập hợp gradient boosting của bạn quá khớp tập huấn luyện, bạn nên tăng hay giảm tốc độ học?
8. Tải tập dữ liệu MNIST (được giới thiệu trong Chương 3), và chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra (ví dụ: sử dụng 50.000 trường hợp để huấn luyện, 10.000 cho xác thực và 10.000 cho kiểm tra). Sau đó, huấn luyện các bộ phân loại khác nhau, chẳng hạn như bộ phân loại rừng ngẫu nhiên, bộ phân loại extra-trees và bộ phân loại SVM. Tiếp theo, thử kết hợp chúng thành một tập hợp vượt trội hơn mỗi bộ phân loại riêng lẻ trên tập xác thực, sử dụng bỏ phiếu mềm hoặc cứng. Một khi bạn đã tìm thấy một tập hợp, hãy thử nó trên tập kiểm tra. Nó hoạt động tốt hơn bao nhiêu so với các bộ phân loại riêng lẻ?
9. Chạy các bộ phân loại riêng lẻ từ bài tập trước để đưa ra dự đoán trên tập xác thực, và tạo một tập huấn luyện mới với các dự đoán thu được: mỗi trường hợp huấn luyện là một vector chứa tập hợp các dự đoán từ tất cả các bộ phân loại của bạn cho một hình ảnh, và mục tiêu là lớp của hình ảnh đó. Huấn luyện một bộ phân loại trên tập huấn luyện mới này. Chúc mừng — bạn vừa huấn luyện một bộ trộn, và cùng với các bộ phân loại, nó tạo thành một tập hợp xếp chồng! Bây giờ hãy đánh giá

tập hợp trên tập kiểm tra. Đối với mỗi hình ảnh trong tập kiểm tra, đưa ra dự đoán với tất cả các bộ phân loại của bạn, sau đó đưa các dự đoán đó cho bộ trộn để nhận được các dự đoán của tập hợp. Nó so sánh như thế nào với bộ phân loại bỏ phiếu mà bạn đã huấn luyện trước đó? Bây giờ hãy thử lại bằng cách sử dụng `StackingClassifier` thay thế. Bạn có nhận được hiệu suất tốt hơn không? Nếu có, tại sao?

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 8. GIẢM CHIỀU DỮ LIỆU

Nhiều bài toán học máy liên quan đến hàng nghìn hoặc thậm chí hàng triệu đặc trưng cho mỗi trường hợp huấn luyện. Tất cả các đặc trưng này không chỉ làm cho quá trình huấn luyện cực kỳ chậm mà còn có thể làm cho việc tìm ra một giải pháp tốt trở nên khó khăn hơn nhiều, như bạn sẽ thấy. Vấn đề này thường được gọi là **lời nguyền của số chiều**.

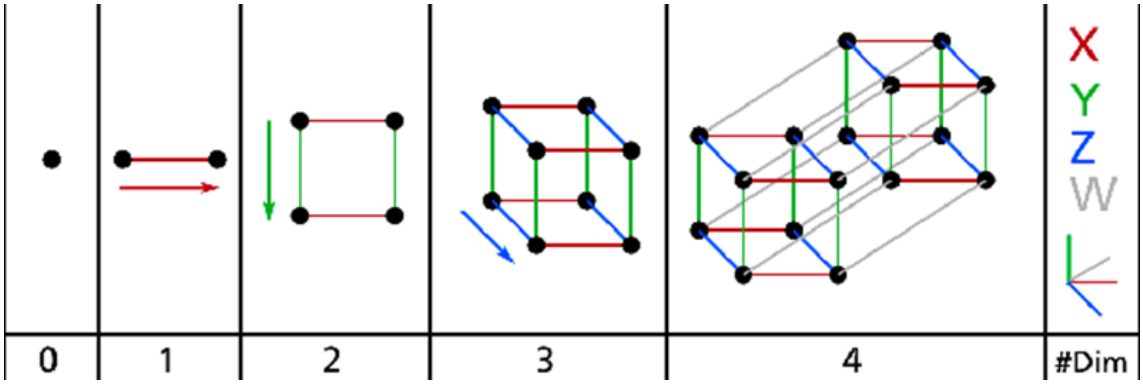
May mắn thay, trong các bài toán thực tế, thường có thể giảm đáng kể số lượng đặc trưng, biến một bài toán không thể giải quyết được thành một bài toán có thể giải quyết được. Ví dụ, hãy xem xét các hình ảnh MNIST (được giới thiệu trong Chương 3): các pixel ở đường biên hình ảnh hầu như luôn luôn trắng, vì vậy bạn có thể loại bỏ hoàn toàn các pixel này khỏi tập huấn luyện mà không mất nhiều thông tin. Như chúng ta đã thấy trong chương trước (Hình 7-6), điều này xác nhận rằng các pixel này hoàn toàn không quan trọng đối với tác vụ phân loại. Ngoài ra, hai pixel lân cận thường có tương quan cao: nếu bạn hợp nhất chúng thành một pixel duy nhất (ví dụ: bằng cách lấy trung bình cường độ của hai pixel), bạn sẽ không mất nhiều thông tin.

Ngoài việc tăng tốc quá trình huấn luyện, giảm chiều dữ liệu cũng cực kỳ hữu ích cho việc trực quan hóa dữ liệu. Giảm số lượng chiều xuống còn hai (hoặc ba) giúp có thể vẽ một cái nhìn cô đọng về một tập huấn luyện có chiều cao trên một biểu đồ và thường thu được một số hiểu biết quan trọng bằng cách phát hiện các mẫu một cách trực quan, chẳng hạn như các cụm. Hơn nữa, trực quan hóa dữ liệu là điều cần thiết để truyền đạt kết luận của bạn cho những người không phải là nhà khoa học dữ liệu—đặc biệt là những người ra quyết định sẽ sử dụng kết quả của bạn.

Trong chương này, chúng ta sẽ thảo luận đầu tiên về lời nguyền của số chiều và có được cảm nhận về những gì xảy ra trong không gian có chiều cao. Sau đó, chúng ta sẽ xem xét hai cách tiếp cận chính để giảm chiều (phép chiếu và học đa tạp), và chúng ta sẽ đi qua ba trong số các kỹ thuật giảm chiều phổ biến nhất: PCA, phép chiếu ngẫu nhiên và nhúng tuyến tính cục bộ (LLE).

Lời nguyền của số chiều

Chúng ta đã quá quen với việc sống trong không gian ba chiều đến nỗi trực giác của chúng ta thất bại khi chúng ta cố gắng tưởng tượng một không gian có chiều cao. Ngay cả một siêu hình lập phương 4D cơ bản cũng cực kỳ khó hình dung trong tâm trí chúng ta (xem Hình 8-1), chứ đừng nói đến một hình elip 200 chiều bị uốn cong trong không gian 1.000 chiều.



Hình 8-1. Điểm, đoạn thẳng, hình vuông, hình lập phương và tesseract (siêu hình lập phương từ 0D đến 4D)

Hóa ra nhiều thứ hoạt động rất khác nhau trong không gian có chiều cao. Ví dụ, nếu bạn chọn một điểm ngẫu nhiên trong một hình vuông đơn vị (một hình vuông 1 × 1), nó sẽ chỉ có khoảng 0.4% cơ hội nằm cách đường biên dưới 0.001 (nói cách khác, rất khó có khả năng một điểm ngẫu nhiên sẽ "cực đoan" dọc theo bất kỳ chiều nào). Nhưng trong một siêu hình lập phương đơn vị 10.000 chiều, xác suất này lớn hơn 99.999999%. Hầu hết các điểm trong một siêu hình lập phương có chiều cao đều rất gần đường biên.

Đây là một sự khác biệt rắc rối hơn: nếu bạn chọn hai điểm ngẫu nhiên trong một hình vuông đơn vị, khoảng cách giữa hai điểm này sẽ, trung bình, khoảng 0.52. Nếu bạn chọn hai điểm ngẫu nhiên trong một hình lập phương đơn vị 3D, khoảng cách trung bình sẽ là khoảng 0.66. Nhưng còn hai điểm được chọn ngẫu nhiên trong một siêu hình lập phương đơn vị 1.000.000 chiều thì sao? Khoảng cách trung bình, dù bạn tin hay không, sẽ là khoảng 408.25 (khoảng $\sqrt{1.000.000/6}$)! Điều này phản trực giác: làm thế nào hai điểm có thể cách xa nhau đến vậy khi cả hai đều nằm trong cùng một siêu hình lập phương đơn vị? Vâng, có rất nhiều không gian trong các chiều cao. Kết quả là, các tập dữ liệu có chiều cao có nguy cơ rất thưa thớt: hầu hết các trường hợp huấn luyện có thể sẽ cách xa nhau. Điều này cũng có nghĩa là một trường hợp mới có thể sẽ cách xa bất kỳ trường hợp huấn luyện nào, làm cho dự đoán kém tin cậy hơn nhiều so với trong các chiều thấp hơn, vì chúng sẽ dựa trên các phép ngoại suy lớn hơn nhiều. Tóm lại, tập huấn luyện có càng nhiều chiều, nguy cơ quá khớp càng lớn.

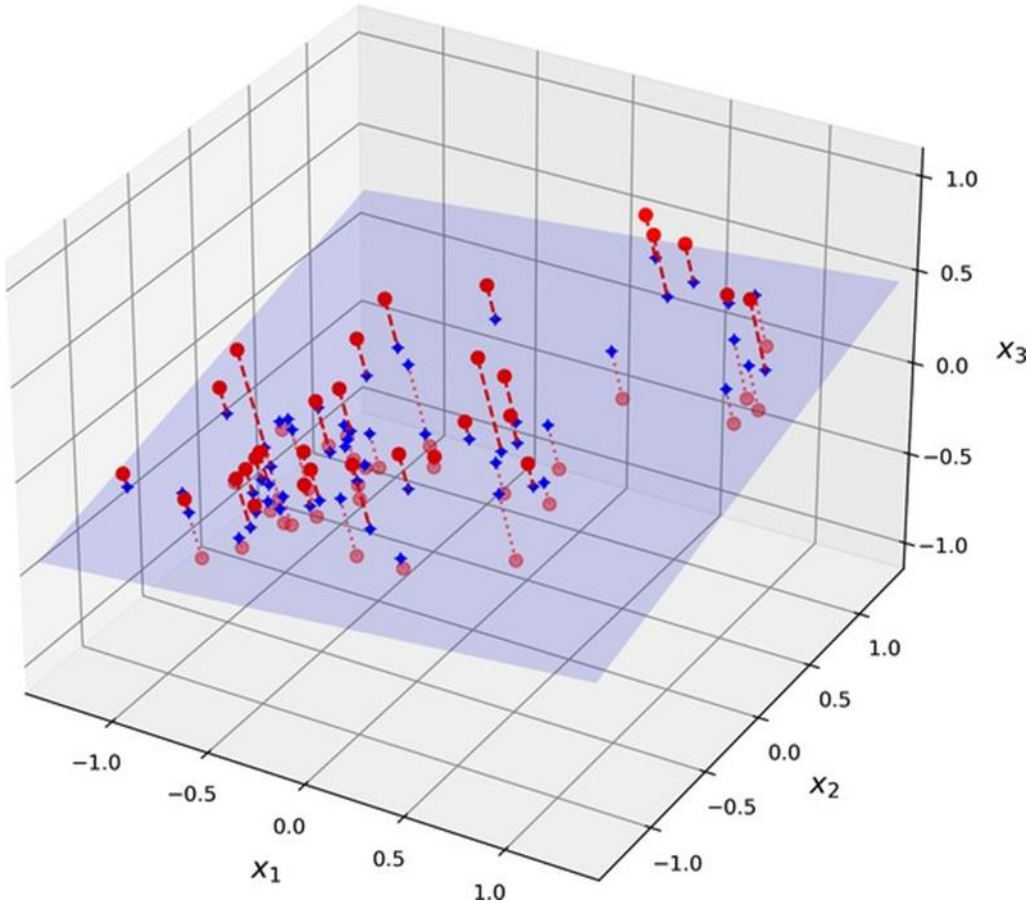
Về lý thuyết, một giải pháp cho lời nguyền của số chiều có thể là tăng kích thước của tập huấn luyện để đạt được mật độ đủ của các trường hợp huấn luyện. Thật không may, trong thực tế, số lượng trường hợp huấn luyện cần thiết để đạt được mật độ nhất định tăng theo cấp số mũ với số lượng chiều. Với chỉ 100 đặc trưng—ít hơn đáng kể so với trong bài toán MNIST—tất cả đều có giá trị từ 0 đến 1, bạn sẽ cần nhiều trường hợp huấn luyện hơn số nguyên tử trong vũ trụ quan sát được để các trường hợp huấn luyện nằm trong khoảng 0.1 của nhau trung bình, giả sử chúng được trải đều trên tất cả các chiều.

Các cách tiếp cận chính để giảm chiều

Trước khi chúng ta đi sâu vào các thuật toán giảm chiều cụ thể, hãy xem xét hai cách tiếp cận chính để giảm chiều: phép chiếu và học đa tạp.

Phép chiếu

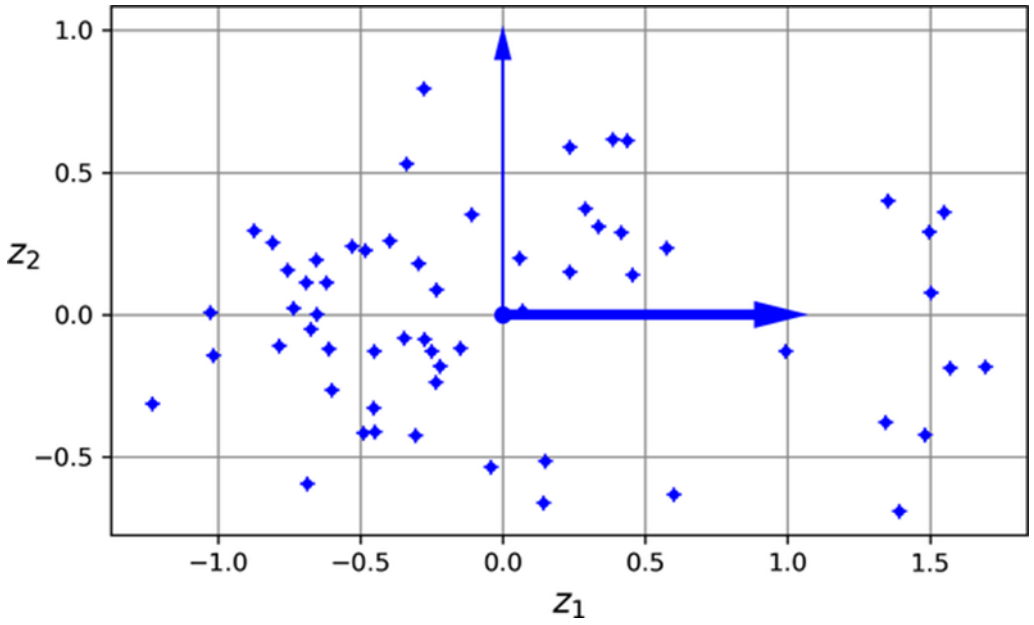
Trong hầu hết các bài toán thực tế, các trường hợp huấn luyện không được trải đều trên tất cả các chiều. Nhiều đặc trưng gần như không đổi, trong khi những đặc trưng khác có tương quan cao (như đã thảo luận trước đó cho MNIST). Kết quả là, tất cả các trường hợp huấn luyện nằm trong (hoặc gần) một không gian con có chiều thấp hơn nhiều của không gian có chiều cao. Điều này nghe có vẻ rất trừu tượng, vì vậy hãy xem một ví dụ. Trong Hình 8-2, bạn có thể thấy một tập dữ liệu 3D được biểu diễn bằng các hình cầu nhỏ.



Hình 8-2. Một tập dữ liệu 3D nằm gần một không gian con 2D

Lưu ý rằng tất cả các trường hợp huấn luyện nằm gần một mặt phẳng: đây là một không gian con có chiều thấp hơn (2D) của không gian có chiều cao hơn (3D). Nếu chúng ta chiếu mọi trường hợp huấn luyện một cách vuông góc lên không gian con này (như được biểu diễn bằng các đường nét đứt ngắn nối các trường hợp với mặt phẳng), chúng ta sẽ nhận được tập dữ liệu 2D mới được hiển thị trong Hình 8-3. Ta-da! Chúng ta vừa giảm chiều của

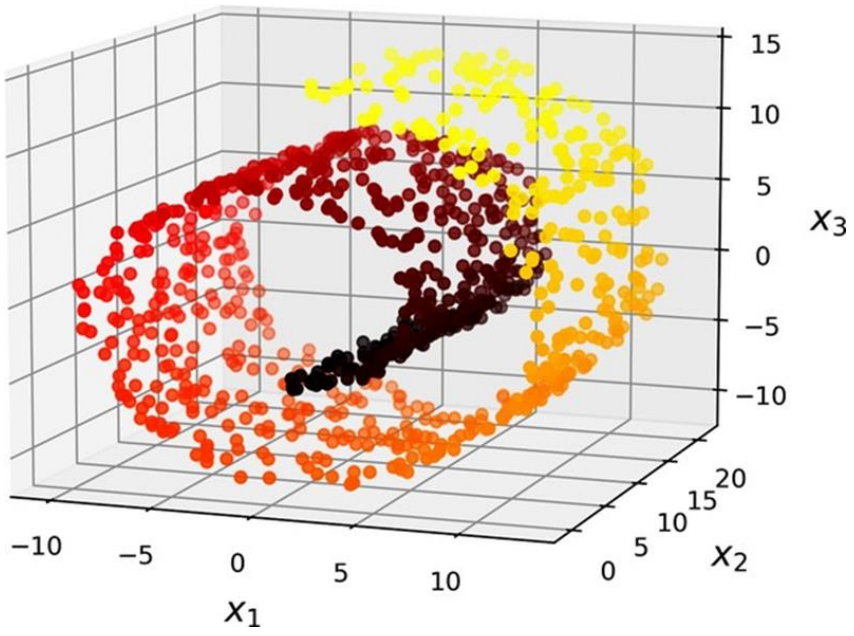
tập dữ liệu từ 3D xuống 2D. Lưu ý rằng các trục tương ứng với các đặc trưng mới z_1 và z_2 : chúng là tọa độ của các phép chiếu trên mặt phẳng.



Hình 8-3. Tập dữ liệu 2D mới sau phép chiếu

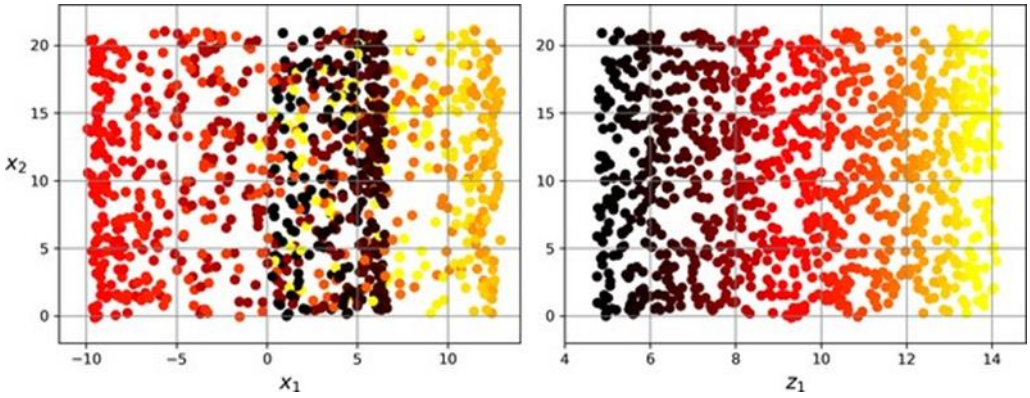
Học đa tạp (Manifold Learning)

Tuy nhiên, phép chiếu không phải lúc nào cũng là cách tiếp cận tốt nhất để giảm chiều. Trong nhiều trường hợp, không gian con có thể xoắn và uốn cong, như trong tập dữ liệu đồ chơi Swiss roll nổi tiếng được biểu thị trong Hình 8-4.



Hình 8-4. Tập dữ liệu Swiss roll

Đơn giản là chiếu lên một mặt phẳng (ví dụ, bằng cách bỏ đi x_3) sẽ làm dẹt các lớp khác nhau của Swiss roll lại với nhau, như trong hình bên trái của Hình 8-5. Điều bạn có lẽ muốn thay vào đó là mở cuộn Swiss roll để có được tập dữ liệu 2D ở phía bên phải của Hình 8-5.

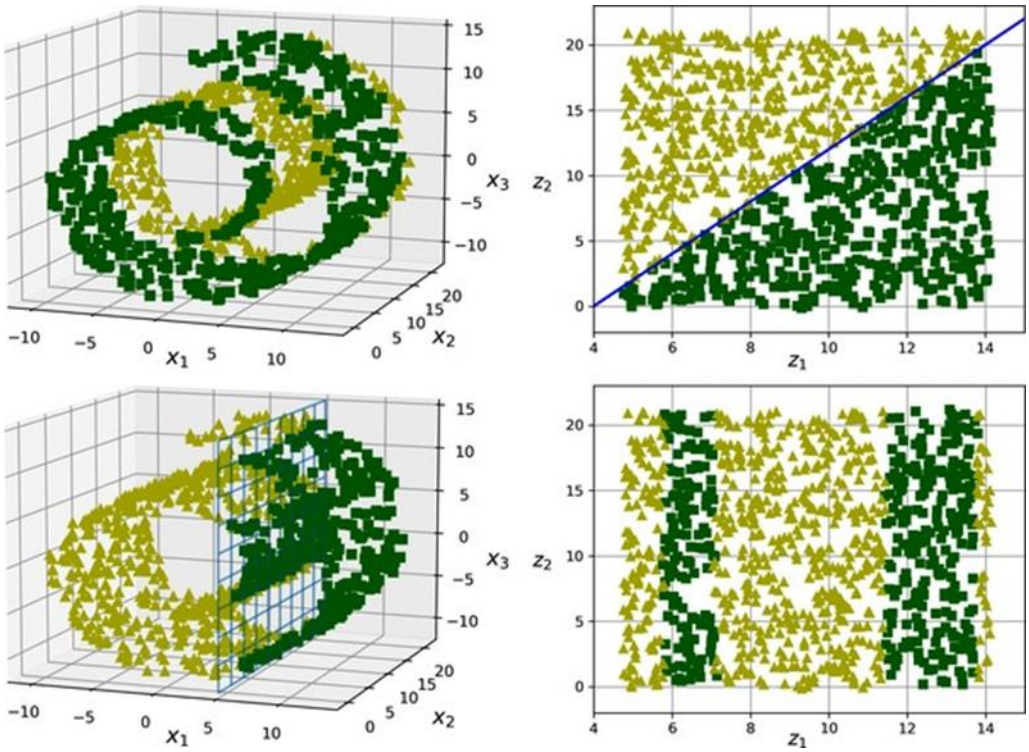


Hình 8-5. Làm dẹt bằng cách chiếu lên một mặt phẳng (trái) so với mở cuộn Swiss roll (phải)

Swiss roll là một ví dụ về một **đa tạp 2D**. Nói một cách đơn giản, một đa tạp 2D là một hình dạng 2D có thể bị uốn cong và xoắn trong một không gian có chiều cao hơn. Tổng quát hơn, một **đa tạp d -chiều** là một phần của không gian n -chiều (trong đó $d < n$) mà cục bộ giống một siêu phẳng d -chiều. Trong trường hợp của Swiss roll, $d = 2$ và $n = 3$: nó cục bộ giống một mặt phẳng 2D, nhưng nó được cuộn trong chiều thứ ba. Nhiều thuật toán giảm chiều hoạt động bằng cách mô hình hóa đa tạp mà các trường hợp huấn luyện nằm trên đó; đây được gọi là **học đa tạp**. Nó dựa trên **giả định đa tạp**, còn được gọi là **giả thuyết đa tạp**, cho rằng hầu hết các tập dữ liệu thực tế có chiều cao đều nằm gần một đa tạp có chiều thấp hơn nhiều. Giả định này rất thường được quan sát thực nghiệm. Một lần nữa, hãy nghĩ về tập dữ liệu MNIST: tất cả các hình ảnh chữ số viết tay đều có một số điểm tương đồng. Chúng được tạo thành từ các đường nối liền, các đường biên màu trắng và chúng ít nhiều được căn giữa. Nếu bạn tạo hình ảnh ngẫu nhiên, chỉ một phần rất nhỏ trong số chúng sẽ trông giống các chữ số viết tay. Nói cách khác, số bậc tự do có sẵn cho bạn nếu bạn cố gắng tạo một hình ảnh chữ số thấp hơn đáng kể so với số bậc tự do bạn có nếu bạn được phép tạo bất kỳ hình ảnh nào bạn muốn. Những ràng buộc này có xu hướng nén tập dữ liệu vào một đa tạp có chiều thấp hơn. Giả định đa tạp thường đi kèm với một giả định ngầm khác: rằng tác vụ đang thực hiện (ví dụ: phân loại hoặc hồi quy) sẽ đơn giản hơn nếu được biểu diễn trong không gian chiều thấp hơn của đa tạp. Ví dụ, trong hàng trên cùng của Hình 8-6, Swiss roll được chia thành hai lớp: trong không gian 3D (bên trái) đường biên quyết định sẽ khá phức tạp, nhưng trong không gian đa tạp 2D đã mở cuộn (bên phải) đường biên quyết định là một đường thẳng.

Tuy nhiên, giả định ngầm này không phải lúc nào cũng đúng. Ví dụ, trong hàng dưới cùng của Hình 8-6, đường biên quyết định nằm ở $x_1 = 5$. Đường biên quyết định này trông rất đơn giản trong không gian 3D gốc (một mặt phẳng dọc), nhưng nó trông phức tạp hơn trong đa tạp đã mở cuộn (một tập hợp bốn đoạn thẳng độc lập). Tóm lại, giảm chiều của tập huấn luyện trước khi huấn luyện một mô hình thường sẽ tăng tốc quá trình huấn luyện, nhưng nó

không phải lúc nào cũng dẫn đến một giải pháp tốt hơn hoặc đơn giản hơn; tất cả phụ thuộc vào tập dữ liệu. Hy vọng bạn bây giờ đã hiểu rõ về lời nguyền của số chiều là gì và các thuật toán giảm chiều có thể chống lại nó như thế nào, đặc biệt khi giả định đa tạp đúng. Phần còn lại của chương này sẽ đi qua một số thuật toán phổ biến nhất để giảm chiều.



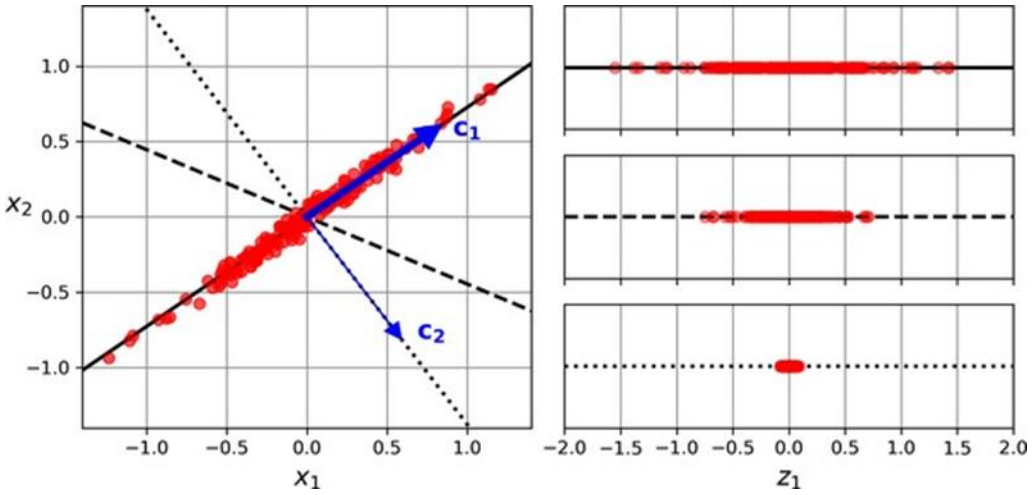
Hình 8-6. Đường biên quyết định không phải lúc nào cũng đơn giản hơn với các chiều thấp hơn

PCA

Phân tích thành phần chính (PCA) cho đến nay là thuật toán giảm chiều phổ biến nhất. Đầu tiên, nó xác định siêu phẳng nằm gần dữ liệu nhất, và sau đó nó chiếu dữ liệu lên đó, giống như trong Hình 8-2.

Bảo toàn phương sai

Trước khi bạn có thể chiếu tập huấn luyện lên một siêu phẳng có chiều thấp hơn, trước tiên bạn cần chọn siêu phẳng phù hợp. Ví dụ, một tập dữ liệu 2D đơn giản được biểu diễn ở bên trái trong Hình 8-7, cùng với ba trục khác nhau (tức là siêu phẳng 1D). Ở bên phải là kết quả của phép chiếu tập dữ liệu lên mỗi trục này. Như bạn có thể thấy, phép chiếu lên đường liền nét bảo toàn phương sai tối đa (trên cùng), trong khi phép chiếu lên đường chấm chấm bảo toàn rất ít phương sai (dưới cùng) và phép chiếu lên đường nét đứt bảo toàn một lượng phương sai trung bình (giữa).



Hình 8-7. Lựa chọn không gian con để chiếu

Có vẻ hợp lý khi chọn trục bảo toàn lượng phương sai tối đa, vì nó rất có thể sẽ mất ít thông tin hơn các phép chiếu khác. Một cách khác để biện minh cho lựa chọn này là đó là trục giảm thiểu khoảng cách bình phương trung bình giữa tập dữ liệu gốc và phép chiếu của nó lên trục đó. Đây là ý tưởng khá đơn giản đằng sau PCA.

Các thành phần chính (Principal Components)

PCA xác định trục có phương sai lớn nhất trong tập huấn luyện. Trong Hình 8-7, đó là đường liền nét. Nó cũng tìm một trục thứ hai, trực giao với trục thứ nhất, chiếm phần lớn phương sai còn lại. Trong ví dụ 2D này không có lựa chọn nào khác: đó là đường chấm chấm. Nếu là một tập dữ liệu có chiều cao hơn, PCA cũng sẽ tìm một trục thứ ba, trực giao với cả hai trục trước, và một trục thứ tư, thứ năm, v.v. — nhiều trục bằng số chiều trong tập dữ liệu. Trục thứ i được gọi là **thành phần chính (PC)** thứ i của dữ liệu. Trong Hình 8-7, PC thứ nhất là trục mà vector c_1 nằm trên đó, và PC thứ hai là trục mà vector c_2 nằm trên đó. Trong Hình 8-2, hai PC đầu tiên nằm trên mặt phẳng chiếu, và PC thứ ba là trục trực giao với mặt phẳng đó. Sau phép chiếu, trong Hình 8-3, PC thứ nhất tương ứng với trục z_1 , và PC thứ hai tương ứng với trục z_2 .

Vậy làm thế nào bạn có thể tìm các thành phần chính của một tập huấn luyện? May mắn thay, có một kỹ thuật phân tách ma trận tiêu chuẩn được gọi là **phân tách giá trị số ít (SVD)** có thể phân tách ma trận tập huấn luyện X thành phép nhân ma trận của ba ma trận $USigmaV^T$, trong đó V chứa các vector đơn vị định nghĩa tất cả các thành phần chính mà bạn đang tìm kiếm, như trong Phương trình 8-1. Phương trình 8-1. Ma trận các thành phần chính

$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

Đoạn mã Python sau sử dụng hàm `svd()` của NumPy để lấy tất cả các thành phần chính của tập huấn luyện 3D được biểu diễn trong Hình 8-2, sau đó nó trích xuất hai vector đơn vị định nghĩa hai PC đầu tiên:

```
import numpy as np

X = [...] # tạo một tập dữ liệu 3D nhỏ
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt[0]
c2 = Vt[1]
```

Chiếu xuống d chiều

Khi bạn đã xác định tất cả các thành phần chính, bạn có thể giảm chiều của tập dữ liệu xuống d chiều bằng cách chiếu nó lên siêu phẳng được định nghĩa bởi d thành phần chính đầu tiên. Việc chọn siêu phẳng này đảm bảo rằng phép chiếu sẽ bảo toàn càng nhiều phương sai càng tốt. Ví dụ, trong Hình 8-2, tập dữ liệu 3D được chiếu xuống mặt phẳng 2D được định nghĩa bởi hai thành phần chính đầu tiên, bảo toàn một phần lớn phương sai của tập dữ liệu. Kết quả là, phép chiếu 2D trông rất giống tập dữ liệu 3D gốc. Để chiếu tập huấn luyện lên siêu phẳng và thu được tập dữ liệu giảm chiều $X_d - proj$ có chiều d , hãy tính phép nhân ma trận của ma trận tập huấn luyện X với ma trận W_d , được định nghĩa là ma trận chứa d cột đầu tiên của V , như trong Phương trình 8-2. Phương trình 8-2. Chiếu tập huấn luyện xuống d chiều

$$X_{d-proj} = XW_d$$

Đoạn mã Python sau chiếu tập huấn luyện lên mặt phẳng được định nghĩa bởi hai thành phần chính đầu tiên:

```
W2 = Vt[:2].T
X2D = X_centered @ W2
```

Vậy là xong! Bây giờ bạn đã biết cách giảm chiều của bất kỳ tập dữ liệu nào bằng cách chiếu nó xuống bất kỳ số chiều nào, trong khi vẫn bảo toàn càng nhiều phương sai càng tốt.

Sử dụng Scikit-Learn

Lớp `PCA` của Scikit-Learn sử dụng SVD để triển khai PCA, giống như chúng ta đã làm ở đầu chương này. Đoạn mã sau đây áp dụng PCA để giảm chiều của tập dữ liệu xuống hai chiều (lưu ý rằng nó tự động xử lý việc căn giữa dữ liệu):

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

Sau khi khớp bộ biến đổi PCA với tập dữ liệu, thuộc tính `components_` của nó chứa ma trận chuyển vị của W_d : nó chứa một hàng cho mỗi trong số d thành phần chính đầu tiên.

Tỷ lệ phương sai giải thích (Explained Variance Ratio)

Một thông tin hữu ích khác là **tỷ lệ phương sai giải thích** của mỗi thành phần chính, có sẵn thông qua biến `explained_variance_ratio_`. Tỷ lệ này chỉ ra tỷ lệ phương sai của tập dữ liệu nằm dọc theo mỗi thành phần chính. Ví dụ, hãy xem tỷ lệ phương sai giải thích của hai thành phần đầu tiên của tập dữ liệu 3D được biểu diễn trong Hình 8-2:

```
>>> pca.explained_variance_ratio_
array([0.7578477 , 0.15186921])
```

Kết quả này cho chúng ta biết rằng khoảng 76% phương sai của tập dữ liệu nằm dọc theo PC thứ nhất, và khoảng 15% nằm dọc theo PC thứ hai. Điều này để lại khoảng 9% cho PC thứ ba, vì vậy có lý khi giả định rằng PC thứ ba có lẽ mang rất ít thông tin.

Chọn số chiều phù hợp

Thay vì tùy ý chọn số chiều để giảm xuống, đơn giản hơn là chọn số chiều mà tổng hợp lại chiếm một phần đủ lớn của phương sai—ví dụ, 95% (Một ngoại lệ của quy tắc này, tất nhiên, là nếu bạn đang giảm chiều để trực quan hóa dữ liệu, trong trường hợp đó bạn sẽ muốn giảm chiều xuống 2 hoặc 3). Đoạn mã sau đây tải và chia tập dữ liệu MNIST (được giới thiệu trong Chương 3) và thực hiện PCA mà không giảm chiều, sau đó tính toán số chiều tối thiểu cần thiết để bảo toàn 95% phương sai của tập huấn luyện:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]

pca = PCA()
pca.fit(X_train)

cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1 # d bằng 154
```

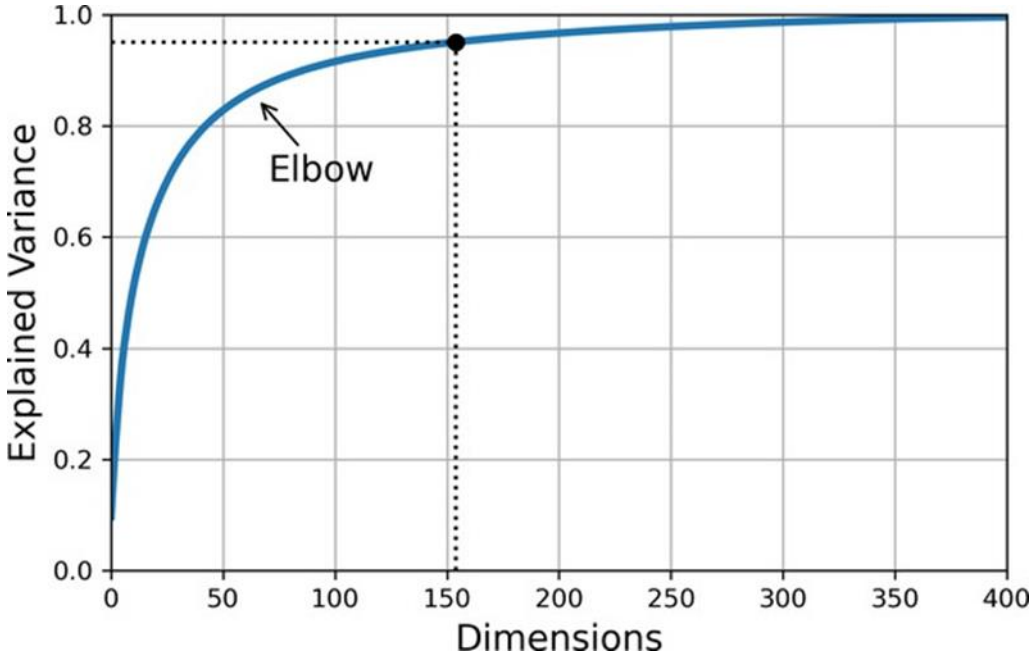
Sau đó, bạn có thể đặt `n_components=d` và chạy lại PCA, nhưng có một lựa chọn tốt hơn. Thay vì chỉ định số lượng thành phần chính bạn muốn bảo toàn, bạn có thể đặt `n_components` là một số thực từ 0.0 đến 1.0, cho biết tỷ lệ phương sai bạn muốn bảo toàn:

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

Số lượng thành phần thực tế được xác định trong quá trình huấn luyện, và nó được lưu trữ trong thuộc tính `n_components_`:

```
>>> pca.n_components_
154
```

Một lựa chọn khác là vẽ biểu đồ phương sai giải thích như một hàm của số chiều (chỉ cần vẽ `cumsum`; xem Hình 8-8). Thường sẽ có một điểm uốn trên đường cong, nơi phương sai giải thích dừng tăng nhanh. Trong trường hợp này, bạn có thể thấy rằng việc giảm chiều xuống khoảng 100 chiều sẽ không mất quá nhiều phương sai giải thích.



Hình 8-8. Phương sai giải thích như một hàm của số chiều

Cuối cùng, nếu bạn đang sử dụng giảm chiều như một bước tiền xử lý cho một tác vụ học có giám sát (ví dụ: phân loại), thì bạn có thể điều chỉnh số lượng chiều như bất kỳ siêu tham số nào khác (xem Chương 2). Ví dụ, đoạn mã sau đây tạo một pipeline hai bước, đầu tiên giảm chiều bằng PCA, sau đó phân loại bằng rừng ngẫu nhiên. Tiếp theo, nó sử dụng `RandomizedSearchCV` để tìm một sự kết hợp tốt các siêu tham số cho cả PCA và bộ phân loại rừng ngẫu nhiên. Ví dụ này thực hiện một tìm kiếm nhanh, chỉ điều chỉnh 2 siêu tham số, huấn luyện trên chỉ 1.000 trường hợp và chạy trong chỉ 10 lần lặp, nhưng bạn có thể thoải mái thực hiện một tìm kiếm kỹ lưỡng hơn nếu có thời gian:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

Hãy xem các siêu tham số tốt nhất được tìm thấy:

```
>>> print(rnd_search.best_params_)
{'randomforestclassifier__n_estimators': 465, 'pca__n_components': 23}
```

Thật thú vị khi lưu ý số lượng thành phần tối ưu thấp như thế nào: chúng ta đã giảm một tập dữ liệu 784 chiều xuống chỉ còn 23 chiều! Điều này liên quan đến việc chúng ta đã sử dụng rừng ngẫu nhiên, một mô hình khá mạnh mẽ. Nếu chúng ta sử dụng một mô hình tuyến tính thay thế, chẳng hạn như `SGDClassifier`, tìm kiếm sẽ cho thấy chúng ta cần bảo toàn nhiều chiều hơn (khoảng 70).

PCA để nén

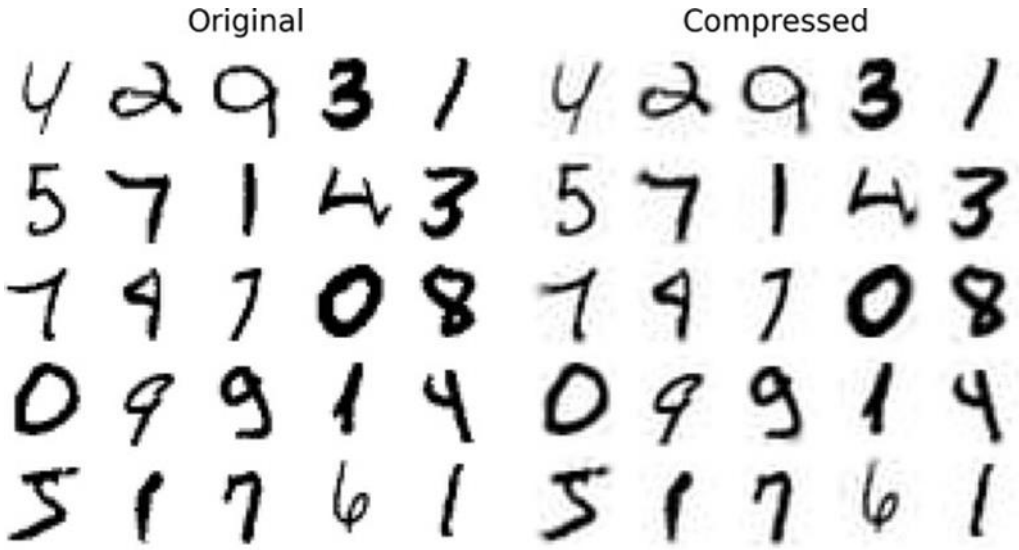
Sau khi giảm chiều dữ liệu, tập huấn luyện chiếm ít không gian hơn nhiều. Ví dụ, sau khi áp dụng PCA cho tập dữ liệu MNIST trong khi vẫn bảo toàn 95% phương sai của nó, chúng ta còn lại 154 đặc trưng, thay vì 784 đặc trưng ban đầu. Do đó, tập dữ liệu giờ đây nhỏ hơn 20% kích thước ban đầu của nó, và chúng ta chỉ mất 5% phương sai của nó! Đây là một tỷ lệ nén hợp lý, và dễ dàng nhận thấy việc giảm kích thước như vậy sẽ tăng tốc thuật toán phân loại đáng kể.

Cũng có thể giải nén tập dữ liệu đã giảm chiều trở lại 784 chiều bằng cách áp dụng phép biến đổi ngược của phép chiếu PCA. Điều này sẽ không trả lại dữ liệu gốc cho bạn, vì phép chiếu đã mất một chút thông tin (trong phạm vi 5% phương sai đã bị loại bỏ), nhưng nó có thể sẽ gần với dữ liệu gốc. Khoảng cách bình phương trung bình giữa dữ liệu gốc và dữ liệu được tái tạo (nén và sau đó giải nén) được gọi là **sai số tái tạo**.

Phương thức `inverse_transform()` cho phép chúng ta giải nén tập dữ liệu MNIST đã giảm chiều trở lại 784 chiều:

```
X_recovered = pca.inverse_transform(X_reduced)
```

Hình 8-9 cho thấy một vài chữ số từ tập huấn luyện gốc (bên trái), và các chữ số tương ứng sau khi nén và giải nén. Bạn có thể thấy rằng có một chút mất mát chất lượng hình ảnh, nhưng các chữ số vẫn hầu hết nguyên vẹn.



Hình 8-9. Nén MNIST bảo toàn 95% phương sai

Phương trình cho phép biến đổi ngược được hiển thị trong Phương trình 8-3.

Phương trình 8-3. Phép biến đổi ngược PCA, trở lại số chiều gốc

$$X_{recovered} = X_{d-proj} W_d^T$$

PCA ngẫu nhiên (Randomized PCA)

Nếu bạn đặt siêu tham số `svd_solver` thành "randomized", Scikit-Learn sử dụng một thuật toán ngẫu nhiên được gọi là **randomized PCA** nhanh chóng tìm thấy một xấp xỉ của d thành phần chính đầu tiên. Độ phức tạp tính toán của nó là $O(m \times d^2) + O(d^3)$, thay vì $O(m \times n^2) + O(n^3)$ cho phương pháp SVD đầy đủ, vì vậy nó nhanh hơn đáng kể so với SVD đầy đủ khi d nhỏ hơn nhiều so với n :

```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
X_reduced = rnd_pca.fit_transform(X_train)
```

PCA tăng dần (Incremental PCA)

Một vấn đề với các triển khai PCA trước đó là chúng yêu cầu toàn bộ tập huấn luyện phải nằm trong bộ nhớ để thuật toán chạy. May mắn thay, các thuật toán **incremental PCA (IPCA)** đã được phát triển cho phép bạn chia tập huấn luyện thành các mini-batch và cung cấp chúng từng mini-batch một. Điều này hữu ích cho các tập huấn luyện lớn và để áp dụng PCA trực tuyến (tức là ngay lập tức, khi các trường hợp mới đến).

Đoạn mã sau chia tập huấn luyện MNIST thành 100 mini-batch (sử dụng hàm `array_split()` của NumPy) và cung cấp chúng cho lớp `IncrementalPCA` của Scikit-Learn để giảm chiều của tập dữ liệu MNIST xuống 154 chiều, giống như trước đây. Lưu ý rằng bạn phải gọi phương thức `partial_fit()` với mỗi mini-batch, thay vì phương thức `fit()` với toàn bộ tập huấn luyện:

```
from sklearn.decomposition import IncrementalPCA

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

Thay vào đó, bạn có thể sử dụng lớp `memmap` của NumPy, cho phép bạn thao tác một mảng lớn được lưu trữ trong một tệp nhị phân trên đĩa như thể nó hoàn toàn nằm trong bộ nhớ; lớp này chỉ tải dữ liệu nó cần vào bộ nhớ, khi nó cần. Để chứng minh điều này, trước tiên hãy tạo một tệp ánh xạ bộ nhớ (memmap) và sao chép tập huấn luyện MNIST vào đó, sau đó gọi `flush()` để đảm bảo rằng bất kỳ dữ liệu nào còn trong bộ đệm đều được lưu vào đĩa. Trong thực tế, `X_train` thường sẽ không vừa trong bộ nhớ, vì vậy bạn sẽ tải nó từng đoạn và lưu mỗi đoạn vào đúng phần của mảng memmap:

```
filename = "my_mnist.mmap"
X_mmap = np.memmap(filename, dtype='float32', mode='write',
shape=X_train.shape)
X_mmap[:] = X_train #có thể là một vòng lặp thay vào đó, lưu dữ liệu từng
đoạn
X_mmap.flush()
```

Tiếp theo, chúng ta có thể tải tệp memmap và sử dụng nó như một mảng NumPy thông thường. Hãy sử dụng lớp `IncrementalPCA` để giảm chiều của nó. Vì thuật toán này chỉ sử dụng một phần nhỏ của mảng tại bất kỳ thời điểm nào, việc sử dụng bộ nhớ vẫn được kiểm soát. Điều này giúp có thể gọi phương thức `fit()` thông thường thay vì `partial_fit()`, điều này khá tiện lợi:

```
X_mmap = np.memmap(filename, dtype="float32", mode="readonly").reshape(-1,
784)
batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
inc_pca.fit(X_mmap)
```

Đối với các tập dữ liệu có chiều rất cao, PCA có thể quá chậm. Như bạn đã thấy trước đó, ngay cả khi bạn sử dụng randomized PCA, độ phức tạp tính toán của nó vẫn là $O(mtimesd^2) + O(d^3)$, vì vậy số chiều mục tiêu d không được quá lớn. Nếu bạn đang xử lý một tập dữ liệu với hàng chục nghìn đặc trưng trở lên (ví dụ: hình ảnh), thì quá trình huấn luyện có thể trở nên quá chậm: trong trường hợp này, bạn nên xem xét sử dụng phép chiếu ngẫu nhiên thay thế.

Phép chiếu ngẫu nhiên (Random Projection)

Như tên gọi của nó, thuật toán **phép chiếu ngẫu nhiên** chiếu dữ liệu vào một không gian có chiều thấp hơn bằng cách sử dụng phép chiếu tuyến tính ngẫu nhiên. Điều này nghe có vẻ

điên rồ, nhưng hóa ra một phép chiếu ngẫu nhiên như vậy thực sự rất có khả năng bảo toàn khoảng cách khá tốt, như đã được chứng minh toán học bởi William B. Johnson và Joram Lindenstrauss trong một định lý nổi tiếng. Do đó, hai trường hợp tương tự sẽ vẫn tương tự sau phép chiếu, và hai trường hợp rất khác nhau sẽ vẫn rất khác nhau.

Rõ ràng, bạn càng giảm nhiều chiều, càng mất nhiều thông tin và khoảng cách càng bị biến dạng. Vậy làm thế nào bạn có thể chọn số chiều tối ưu? Vâng, Johnson và Lindenstrauss đã đưa ra một phương trình xác định số chiều tối thiểu cần bảo toàn để đảm bảo—với xác suất cao—rằng khoảng cách sẽ không thay đổi quá một dung sai nhất định. Ví dụ, nếu bạn có một tập dữ liệu chứa $m = 5.000$ trường hợp với $n = 20.000$ đặc trưng mỗi trường hợp, và bạn không muốn khoảng cách bình phương giữa bất kỳ hai trường hợp nào thay đổi quá $epsilon = 10$, thì bạn nên chiếu dữ liệu xuống d chiều, với $dge4log(m)/(frac12epsilon^2 - frac13epsilon^3)$, là 7.300 chiều. Đó là một sự giảm chiều khá đáng kể!

Lưu ý rằng phương trình không sử dụng n , nó chỉ dựa vào m và $epsilon$. Phương trình này được triển khai bởi hàm `johnson_lindenstrauss_min_dim()`:

```
>>> from sklearn.random_projection import johnson_lindenstrauss_min_dim
>>> m, ε = 5_000, 0.1
>>> d = johnson_lindenstrauss_min_dim(m, eps=ε)
>>> d
7300
```

Bây giờ chúng ta chỉ có thể tạo một ma trận ngẫu nhiên P có hình dạng $[d, n]$, trong đó mỗi phần tử được lấy mẫu ngẫu nhiên từ phân phối Gaussian với giá trị trung bình 0 và phương sai $1/d$, và sử dụng nó để chiếu một tập dữ liệu từ n chiều xuống d :

```
n = 20_000
np.random.seed(42)
P = np.random.randn(d, n) / np.sqrt(d) # độ lệch chuẩn = căn bậc hai của
phương sai

X = np.random.randn(m, n) # tạo một tập dữ liệu giả
X_reduced = X @ P.T
```

Chỉ có thế thôi! Nó đơn giản và hiệu quả, và không yêu cầu huấn luyện: điều duy nhất thuật toán cần để tạo ma trận ngẫu nhiên là hình dạng của tập dữ liệu. Bản thân dữ liệu không được sử dụng chút nào.

Scikit-Learn cung cấp lớp `GaussianRandomProjection` để làm chính xác những gì chúng ta vừa làm : khi bạn gọi phương thức `fit()` của nó, nó sử dụng `johnson_lindenstrauss_min_dim()` để xác định chiều đầu ra, sau đó nó tạo một ma trận ngẫu nhiên, mà nó lưu trữ trong thuộc tính `components_`. Sau đó, khi bạn gọi `transform()`, nó sử dụng ma trận này để thực hiện phép chiếu. Khi tạo bộ biến đổi, bạn có thể đặt `eps` nếu bạn muốn điều chỉnh $epsilon$ (mặc định là 0.1), và `n_components` nếu bạn muốn buộc một chiều mục tiêu d cụ thể. Đoạn mã sau đây cho kết quả tương tự như đoạn mã trước đó (bạn cũng có thể xác minh rằng `gaussian_rnd_proj.components_` bằng P):

```
from sklearn.random_projection import GaussianRandomProjection

gaussian_rnd_proj = GaussianRandomProjection(eps=ε, random_state=42)
X_reduced = gaussian_rnd_proj.fit_transform(X) # cùng kết quả như trên
```

Scikit-Learn cũng cung cấp một bộ biến đổi phép chiếu ngẫu nhiên thứ hai, được gọi là `SparseRandomProjection`. Nó xác định chiều mục tiêu theo cùng một cách, tạo một ma trận ngẫu nhiên có cùng hình dạng và thực hiện phép chiếu giống hệt nhau. Sự khác biệt chính là ma trận ngẫu nhiên thưa thớt. Điều này có nghĩa là nó sử dụng ít bộ nhớ hơn nhiều: khoảng 25 MB thay vì gần 1.2 GB trong ví dụ trước đó! Và nó cũng nhanh hơn nhiều, cả để tạo ma trận ngẫu nhiên và để giảm chiều: nhanh hơn khoảng 50% trong trường hợp này. Hơn nữa, nếu đầu vào thưa thớt, phép biến đổi vẫn giữ nó thưa thớt (trừ khi bạn đặt `dense_output=True`). Cuối cùng, nó có cùng thuộc tính bảo toàn khoảng cách như cách tiếp cận trước, và chất lượng giảm chiều có thể so sánh được. Tóm lại, thường nên sử dụng bộ biến đổi này thay vì bộ biến đổi đầu tiên, đặc biệt đối với các tập dữ liệu lớn hoặc thưa thớt.

Tỷ lệ r các phần tử khác không trong ma trận ngẫu nhiên thưa thớt được gọi là **mật độ** của nó. Theo mặc định, nó bằng $1/sqrtn$. Với 20.000 đặc trưng, điều này có nghĩa là chỉ 1 trong khoảng 141 ô trong ma trận ngẫu nhiên là khác không: điều đó khá thưa thớt! Bạn có thể đặt siêu tham số `density` thành một giá trị khác nếu bạn muốn. Mỗi ô trong ma trận ngẫu nhiên thưa thớt có xác suất r là khác không, và mỗi giá trị khác không là $-v$ hoặc $+v$ (cả hai đều có khả năng như nhau), trong đó $v = 1/sqrtdr$.

Nếu bạn muốn thực hiện phép biến đổi ngược, trước tiên bạn cần tính nghịch đảo giả của ma trận thành phần bằng hàm `pinv()` của SciPy, sau đó nhân dữ liệu đã giảm chiều với ma trận chuyển vị của nghịch đảo giả:

```
components_pinv = np.linalg.pinv(gaussian_rnd_proj.components_)
X_recovered = X_reduced @ components_pinv.T
```

Tóm lại, phép chiếu ngẫu nhiên là một thuật toán giảm chiều đơn giản, nhanh, hiệu quả bộ nhớ và mạnh mẽ đáng ngạc nhiên mà bạn nên ghi nhớ, đặc biệt khi bạn xử lý các tập dữ liệu có chiều cao.

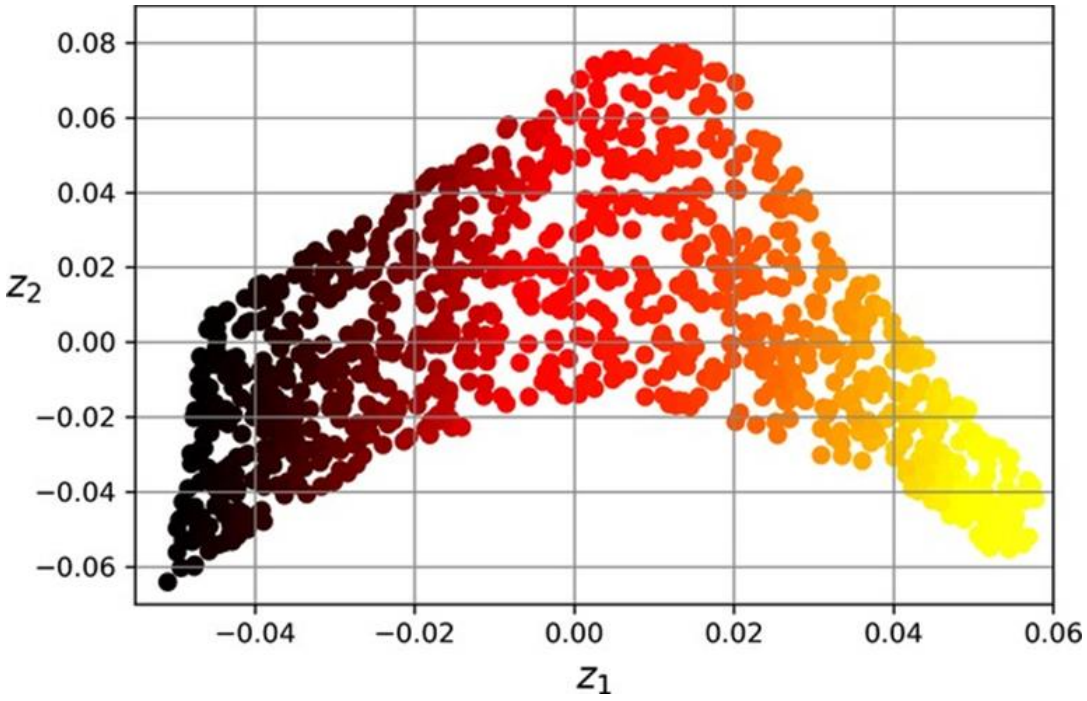
Nhúng tuyến tính cục bộ

Nhúng tuyến tính cục bộ (LLE) là một kỹ thuật giảm chiều phi tuyến tính (NLDR). Đây là một kỹ thuật học đa tạp không dựa vào phép chiếu, không giống như PCA và phép chiếu ngẫu nhiên. Tóm lại, LLE hoạt động bằng cách đầu tiên đo lường cách mỗi trường hợp huấn luyện liên quan tuyến tính đến các láng giềng gần nhất của nó, sau đó tìm kiếm một biểu diễn chiều thấp của tập huấn luyện nơi các mối quan hệ cục bộ này được bảo toàn tốt nhất (chi tiết hơn sẽ được trình bày ngay sau đây). Cách tiếp cận này làm cho nó đặc biệt tốt trong việc mở cuộn các đa tạp bị xoắn, đặc biệt khi không có quá nhiều nhiễu.

Đoạn mã sau tạo một Swiss roll, sau đó sử dụng lớp `LocallyLinearEmbedding` của Scikit-Learn để mở cuộn nó:

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```

Biến t là một mảng NumPy 1D chứa vị trí của mỗi trường hợp dọc theo trục cuộn của Swiss roll. Chúng ta không sử dụng nó trong ví dụ này, nhưng nó có thể được sử dụng làm mục tiêu cho một tác vụ hồi quy phi tuyến. Tập dữ liệu 2D thu được được hiển thị trong Hình 8-10. Như bạn có thể thấy, Swiss roll được mở cuộn hoàn toàn và khoảng cách giữa các trường hợp được bảo toàn tốt cục bộ. Tuy nhiên, khoảng cách không được bảo toàn ở quy mô lớn hơn: Swiss roll đã mở cuộn phải là một hình chữ nhật, không phải loại dải bị kéo dài và xoắn này. Tuy nhiên, LLE đã làm khá tốt việc mô hình hóa đa tạp.



Hình 8-10. Swiss roll đã mở cuộn bằng LLE

Bước 1: Mô hình hóa tuyến tính các mối quan hệ cục bộ

Đối với mỗi trường hợp huấn luyện $x^{(i)}$, thuật toán xác định k láng giềng gần nhất của nó. Sau đó, nó cố gắng tái tạo $x^{(i)}$ dưới dạng một hàm tuyến tính của các láng giềng này. Cụ thể hơn, thuật toán tìm các trọng số $w_{i,j}$ sao cho khoảng cách bình phương giữa $x^{(i)}$ và $\sum_{j=1}^{m} w_{i,j}\, x^{(j)}$ là nhỏ nhất có thể. Giả sử rằng $w_{i,j} = 0$ nếu $x^{(j)}$ không phải là một trong k láng giềng gần nhất của $x^{(i)}$.

Công thức 8-4: Bước 1 của LLE: Mô hình hóa tuyến tính các mối quan hệ cục bộ

$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^{m} \left(\mathbf{x}^{(i)} - \sum_{j=1}^{m} w_{i,j} \mathbf{x}^{(j)} \right)^2$$
$$\text{subject to} \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ n.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^{m} w_{i,j} = 1 & \text{for } i = 1, 2, \cdots, m \end{cases}$$

Sau bước này, ma trận trọng số $W^$ mã hóa các mối quan hệ tuyến tính cục bộ giữa các trường hợp huấn luyện.

Bước 2: Giảm chiều dữ liệu trong khi vẫn giữ các mối quan hệ

Bước thứ hai là ánh xạ các trường hợp huấn luyện vào một không gian d chiều (với $d < n$) trong khi vẫn bảo toàn các mối quan hệ cục bộ này càng nhiều càng tốt. Nếu $z^{(i)}$ là hình ảnh của $x^{(i)}$ trong không gian d chiều, thì chúng ta muốn khoảng cách bình phương giữa $z^{(i)}$ và $\sum_{j=1}^{m} w^{i,j} z^{(j)}$ nhỏ nhất có thể. Ý tưởng này dẫn đến bài toán tối ưu hóa không ràng buộc được mô tả trong Công thức 8-5.

Công thức 8-5: Bước 2 của LLE: Giảm chiều dữ liệu trong khi vẫn giữ các mối quan hệ

$$\widehat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^{m} \left(\mathbf{z}^{(i)} - \sum_{j=1}^{m} \widehat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$

Lưu ý rằng Z là ma trận chứa tất cả các $z^{(i)}$.

Việc triển khai LLE của Scikit-Learn có độ phức tạp tính toán như sau: $O\left(m\log(m)n\log(k)\right)$ để tìm các láng giềng gần nhất, $O(mnk^3)$ để tối ưu hóa trọng số, và $O(dm^2)$ để xây dựng các biểu diễn chiều thấp. Thật không may, số hạng m^2 trong thuật ngữ cuối cùng làm cho thuật toán này mở rộng kém đối với các tập dữ liệu rất lớn.

Các kỹ thuật giảm chiều khác

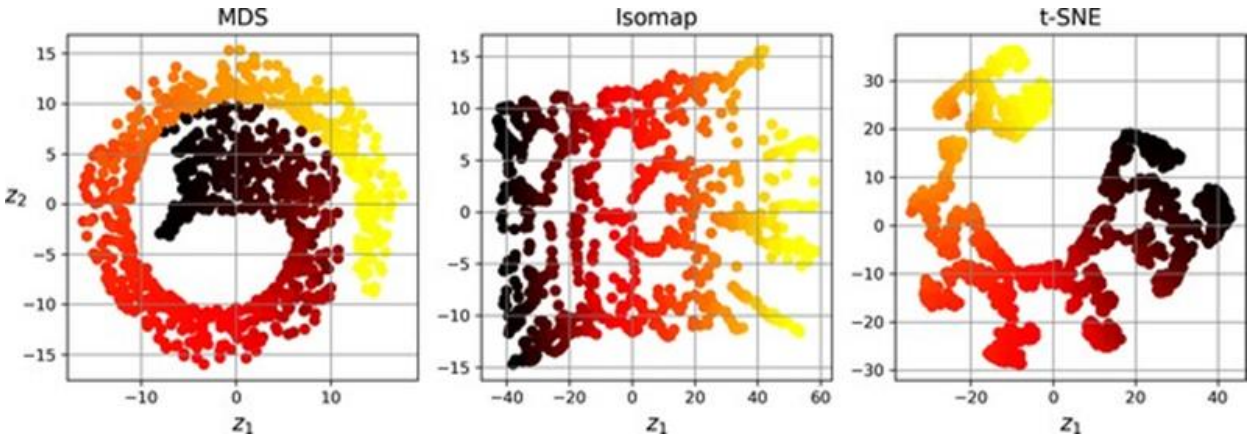
Trước khi chúng ta kết thúc chương này, hãy xem nhanh một vài kỹ thuật giảm chiều phổ biến khác có sẵn trong Scikit-Learn:

- `sklearn.manifold.MDS` **Multidimensional scaling (MDS)** giảm chiều dữ liệu trong khi cố gắng bảo toàn khoảng cách giữa các trường hợp. Phép chiếu ngẫu nhiên làm điều đó cho dữ liệu có chiều cao, nhưng nó không hoạt động tốt trên dữ liệu có chiều thấp.
- `sklearn.manifold.Isomap` **Isomap** tạo một đồ thị bằng cách nối mỗi trường hợp với các láng giềng gần nhất của nó, sau đó giảm chiều trong khi cố gắng bảo toàn

khoảng cách trắc địa giữa các trường hợp. Khoảng cách trắc địa giữa hai nút trong một đồ thị là số nút trên đường đi ngắn nhất giữa các nút này.

- `sklearn.manifold.TSNE` **t-distributed stochastic neighbor embedding (t-SNE)** giảm chiều trong khi cố gắng giữ các trường hợp tương tự gần nhau và các trường hợp không tương tự xa nhau. Nó chủ yếu được sử dụng để trực quan hóa, đặc biệt để trực quan hóa các cụm trường hợp trong không gian chiều cao. Ví dụ, trong các bài tập ở cuối chương này, bạn sẽ sử dụng t-SNE để trực quan hóa một bản đồ 2D của các hình ảnh MNIST.
- `sklearn.discriminant_analysis.LinearDiscriminantAnalysis` **Linear discriminant analysis (LDA)** là một thuật toán phân loại tuyến tính mà, trong quá trình huấn luyện, học các trục phân biệt nhất giữa các lớp. Các trục này sau đó có thể được sử dụng để định nghĩa một siêu phẳng để chiếu dữ liệu lên đó. Lợi ích của cách tiếp cận này là phép chiếu sẽ giữ các lớp cách xa nhau nhất có thể, vì vậy LDA là một kỹ thuật tốt để giảm chiều trước khi chạy một thuật toán phân loại khác (trừ khi LDA một mình là đủ).

Hình 8-11 cho thấy kết quả của MDS, Isomap và t-SNE trên Swiss roll. MDS quản lý để làm phẳng Swiss roll mà không làm mất độ cong tổng thể của nó, trong khi Isomap loại bỏ hoàn toàn nó. Tùy thuộc vào tác vụ tiếp theo, việc bảo toàn cấu trúc quy mô lớn có thể tốt hoặc xấu. t-SNE thực hiện một công việc khá tốt trong việc làm phẳng Swiss roll, bảo toàn một chút độ cong, và nó cũng khuếch đại các cụm, xé cuộn ra. Một lần nữa, điều này có thể tốt hoặc xấu, tùy thuộc vào tác vụ tiếp theo.



Hình 8-11. Sử dụng các kỹ thuật khác nhau để giảm Swiss roll xuống 2D

Bài tập

1. Động lực chính để giảm chiều dữ liệu là gì? Những hạn chế chính là gì?
2. Lời nguyền của số chiều là gì?
3. Một khi chiều của tập dữ liệu đã được giảm, có thể đảo ngược thao tác không? Nếu có, bằng cách nào? Nếu không, tại sao?
4. PCA có thể được sử dụng để giảm chiều của một tập dữ liệu phi tuyến tính cao không?

5. Giả sử bạn thực hiện PCA trên một tập dữ liệu 1.000 chiều, đặt tỷ lệ phương sai giải thích là 95%. Tập dữ liệu kết quả sẽ có bao nhiêu chiều?
6. Trong những trường hợp nào bạn sẽ sử dụng PCA thông thường, PCA tăng dần, PCA ngẫu nhiên, hoặc phép chiếu ngẫu nhiên?
7. Làm thế nào bạn có thể đánh giá hiệu suất của một thuật toán giảm chiều trên tập dữ liệu của bạn?
8. Có ý nghĩa gì khi nối chuỗi hai thuật toán giảm chiều khác nhau không?
9. Tải tập dữ liệu MNIST (được giới thiệu trong Chương 3) và chia nó thành một tập huấn luyện và một tập kiểm tra (lấy 60.000 trường hợp đầu tiên để huấn luyện, và 10.000 còn lại để kiểm tra). Huấn luyện một bộ phân loại rừng ngẫu nhiên trên tập dữ liệu và tính thời gian mất bao lâu, sau đó đánh giá mô hình kết quả trên tập kiểm tra. Tiếp theo, sử dụng PCA để giảm chiều của tập dữ liệu, với tỷ lệ phương sai giải thích là 95%. Huấn luyện một bộ phân loại rừng ngẫu nhiên mới trên tập dữ liệu đã giảm chiều và xem mất bao lâu. Quá trình huấn luyện có nhanh hơn nhiều không? Tiếp theo, đánh giá bộ phân loại trên tập kiểm tra. Nó so sánh như thế nào với bộ phân loại trước đó? Thử lại với một `SGDClassifier`. PCA giúp được bao nhiêu bây giờ?
10. Sử dụng t-SNE để giảm 5.000 hình ảnh đầu tiên của tập dữ liệu MNIST xuống 2 chiều và vẽ kết quả bằng Matplotlib. Bạn có thể sử dụng một biểu đồ phân tán (scatterplot) với 10 màu khác nhau để biểu thị lớp mục tiêu của mỗi hình ảnh. Thay vào đó, bạn có thể thay thế mỗi chấm trong biểu đồ phân tán bằng lớp của trường hợp tương ứng (một chữ số từ 0 đến 9), hoặc thậm chí vẽ các phiên bản thu nhỏ của chính các hình ảnh chữ số (nếu bạn vẽ tất cả các chữ số, trực quan hóa sẽ quá lộn xộn, vì vậy bạn nên vẽ một mẫu ngẫu nhiên hoặc chỉ vẽ một trường hợp nếu không có trường hợp nào khác đã được vẽ ở khoảng cách gần). Bạn sẽ nhận được một hình ảnh trực quan đẹp với các cụm chữ số được phân tách tốt. Thử sử dụng các thuật toán giảm chiều khác, chẳng hạn như PCA, LLE hoặc MDS, và so sánh các hình ảnh trực quan thu được. Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 9. CÁC KỸ THUẬT HỌC KHÔNG GIÁM SÁT

Mặc dù hầu hết các ứng dụng học máy ngày nay đều dựa trên **học có giám sát** (và do đó, đây là nơi phần lớn các khoản đầu tư được đổ vào), nhưng phần lớn dữ liệu có sẵn là không được gắn nhãn: chúng ta có các đặc trưng đầu vào X , nhưng chúng ta không có nhãn y . Nhà khoa học máy tính Yann LeCun đã nổi tiếng nói rằng "nếu trí thông minh là một chiếc bánh, thì học không giám sát sẽ là chiếc bánh, học có giám sát sẽ là lớp kem trên bánh, và học tăng cường sẽ là quả anh đào trên bánh." Nói cách khác, có một tiềm năng rất lớn trong học không giám sát mà chúng ta mới chỉ bắt đầu khai thác.

Giả sử bạn muốn tạo một hệ thống sẽ chụp vài bức ảnh của mỗi mặt hàng trên dây chuyền sản xuất và phát hiện mặt hàng nào bị lỗi. Bạn có thể khá dễ dàng tạo một hệ thống tự động chụp ảnh, và điều này có thể cung cấp cho bạn hàng nghìn bức ảnh mỗi ngày. Sau đó, bạn có thể xây dựng một tập dữ liệu khá lớn chỉ trong vài tuần. Nhưng đợi đã, không có nhãn! Nếu bạn muốn huấn luyện một bộ phân loại nhị phân thông thường sẽ dự đoán một mặt hàng có bị lỗi hay không, bạn sẽ cần gắn nhãn từng bức ảnh là "bị lỗi" hoặc "bình thường". Điều này thường sẽ yêu cầu các chuyên gia con người ngồi xuống và xem xét thủ công tất cả các bức ảnh. Đây là một công việc dài, tốn kém và tẻ nhạt, vì vậy nó thường chỉ được thực hiện trên một tập con nhỏ của các bức ảnh có sẵn. Kết quả là, tập dữ liệu được gắn nhãn sẽ khá nhỏ, và hiệu suất của bộ phân loại sẽ đáng thất vọng. Hơn nữa, mỗi khi công ty thay đổi sản phẩm của mình, toàn bộ quá trình sẽ cần được bắt đầu lại từ đầu. Sẽ không tuyệt vời sao nếu thuật toán có thể khai thác dữ liệu không được gắn nhãn mà không cần con người gắn nhãn từng bức ảnh? Hãy đến với học không giám sát.

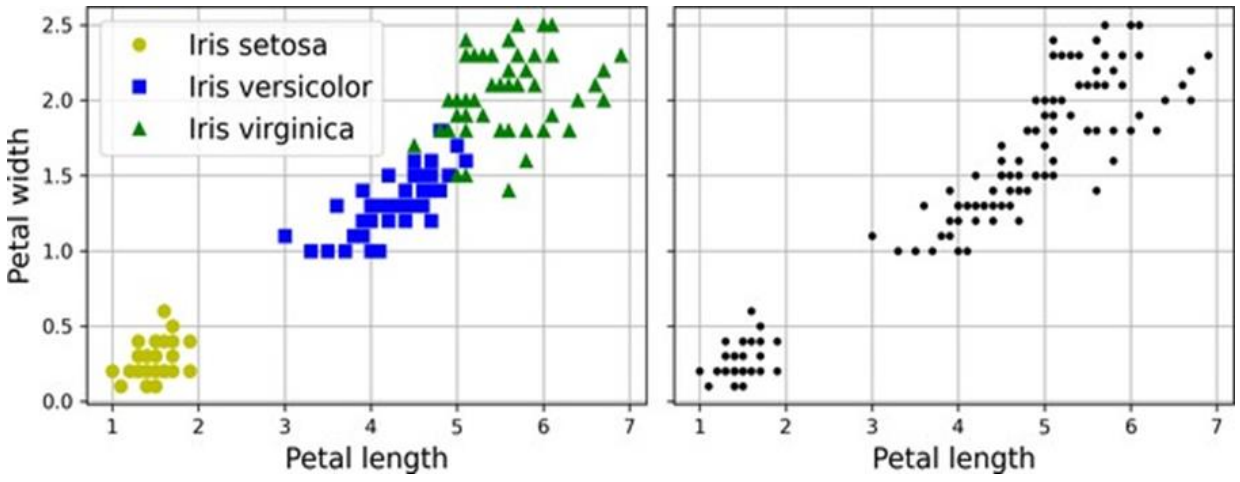
Trong Chương 8, chúng ta đã xem xét tác vụ học không giám sát phổ biến nhất: giảm chiều dữ liệu. Trong chương này, chúng ta sẽ xem xét thêm một vài tác vụ không giám sát nữa:

- **Phân cụm (Clustering)** Mục tiêu là nhóm các trường hợp tương tự lại với nhau thành các **cụm**. Phân cụm là một công cụ tuyệt vời để phân tích dữ liệu, phân khúc khách hàng, hệ thống đề xuất, công cụ tìm kiếm, phân đoạn hình ảnh, học bán giám sát, giảm chiều dữ liệu, và nhiều hơn nữa.
- **Phát hiện dị thường (Anomaly detection)** (còn gọi là phát hiện ngoại lệ - outlier detection) Mục tiêu là học cách dữ liệu "bình thường" trông như thế nào, và sau đó sử dụng điều đó để phát hiện các trường hợp bất thường. Các trường hợp này được gọi là **dị thường**, hoặc **ngoại lệ**, trong khi các trường hợp bình thường được gọi là **nội lệ (inliers)**. Phát hiện dị thường hữu ích trong nhiều ứng dụng khác nhau, chẳng hạn như phát hiện gian lận, phát hiện sản phẩm bị lỗi trong sản xuất, xác định các xu hướng mới trong chuỗi thời gian, hoặc loại bỏ các ngoại lệ khỏi tập dữ liệu trước khi huấn luyện một mô hình khác, điều này có thể cải thiện đáng kể hiệu suất của mô hình kết quả.
- **Ước tính mật độ (Density estimation)** Đây là tác vụ ước tính hàm mật độ xác suất (PDF) của quá trình ngẫu nhiên đã tạo ra tập dữ liệu. Ước tính mật độ thường được sử dụng để phát hiện dị thường: các trường hợp nằm trong các vùng mật độ rất thấp có khả năng là dị thường. Nó cũng hữu ích cho phân tích và trực quan hóa dữ liệu.

Sẵn sàng cho một ít bánh chưa? Chúng ta sẽ bắt đầu với hai thuật toán phân cụm, k-means và DBSCAN, sau đó chúng ta sẽ thảo luận về các mô hình hỗn hợp Gaussian và xem cách chúng có thể được sử dụng để ước tính mật độ, phân cụm và phát hiện dị thường.

Các thuật toán phân cụm: k-means và DBSCAN

Khi bạn đang đi bộ đường dài trên núi, bạn tình cờ gặp một loài thực vật mà bạn chưa từng thấy trước đây. Bạn nhìn xung quanh và nhận thấy thêm một vài cây nữa. Chúng không giống hệt nhau, nhưng chúng đủ giống nhau để bạn biết rằng chúng rất có thể thuộc cùng một loài (hoặc ít nhất là cùng một chi). Bạn có thể cần một nhà thực vật học để cho bạn biết đó là loài gì, nhưng bạn chắc chắn không cần một chuyên gia để xác định các nhóm vật thể trông tương tự. Đây được gọi là **phân cụm**: đó là nhiệm vụ xác định các trường hợp tương tự và gán chúng vào các **cụm**, hoặc các nhóm trường hợp tương tự. Cũng giống như trong phân loại, mỗi trường hợp được gán vào một nhóm. Tuy nhiên, không giống như phân loại, phân cụm là một tác vụ không giám sát. Xem xét Hình 9-1: ở bên trái là tập dữ liệu iris (được giới thiệu trong Chương 4), trong đó loài của mỗi trường hợp (tức là lớp của nó) được biểu thị bằng một ký hiệu khác nhau. Đây là một tập dữ liệu được gắn nhãn, rất phù hợp cho các thuật toán phân loại như hồi quy logistic, SVM hoặc bộ phân loại rừng ngẫu nhiên. Ở bên phải là cùng một tập dữ liệu, nhưng không có nhãn, vì vậy bạn không thể sử dụng thuật toán phân loại nữa. Đây là lúc các thuật toán phân cụm xuất hiện: nhiều thuật toán trong số chúng có thể dễ dàng phát hiện cụm dưới cùng bên trái. Cũng khá dễ nhìn thấy bằng mắt thường của chúng ta, nhưng không rõ ràng rằng cụm trên cùng bên phải bao gồm hai phân cụm riêng biệt. Điều đó nói rằng, tập dữ liệu có thêm hai đặc trưng (chiều dài và chiều rộng đài hoa) không được biểu thị ở đây, và các thuật toán phân cụm có thể tận dụng tốt tất cả các đặc trưng, vì vậy trên thực tế chúng xác định ba cụm khá tốt (ví dụ, sử dụng mô hình hỗn hợp Gaussian, chỉ có 5 trong số 150 trường hợp bị gán sai cụm).



Hình 9-1. Phân loại (trái) so với phân cụm (phải)

Phân cụm được sử dụng trong nhiều ứng dụng khác nhau, bao gồm:

- **Phân khúc khách hàng** Bạn có thể phân cụm khách hàng của mình dựa trên các giao dịch mua hàng và hoạt động của họ trên trang web của bạn. Điều này hữu ích

để hiểu khách hàng của bạn là ai và họ cần gì, để bạn có thể điều chỉnh sản phẩm và chiến dịch tiếp thị của mình cho từng phân khúc. Ví dụ, phân khúc khách hàng có thể hữu ích trong các hệ thống đề xuất để gợi ý nội dung mà những người dùng khác trong cùng cụm đã thích.

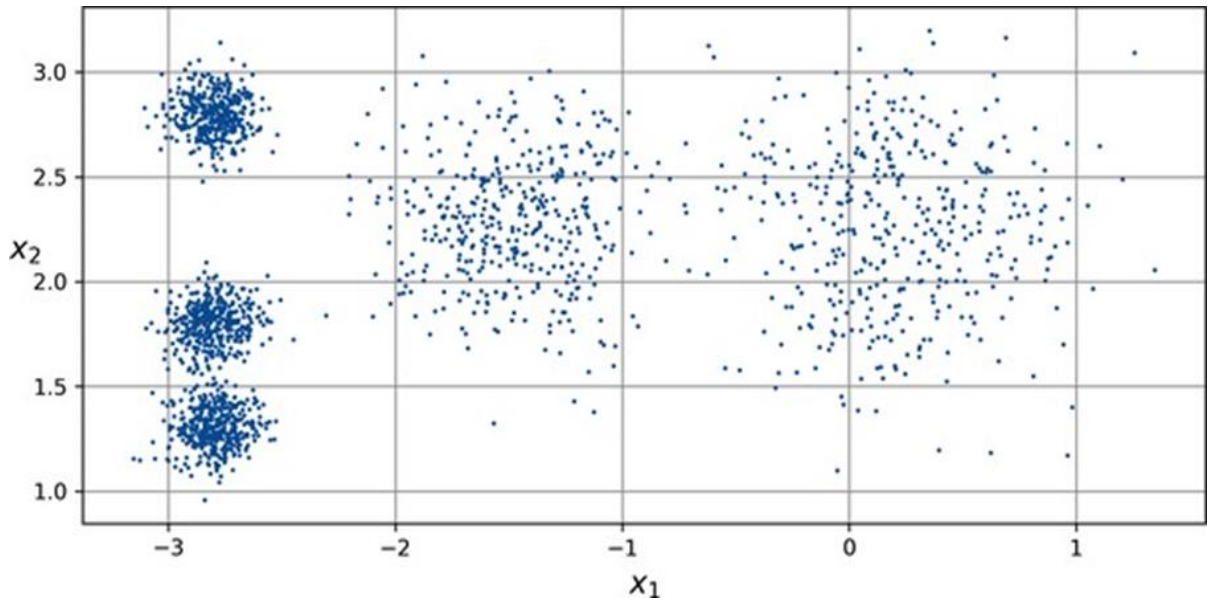
- **Phân tích dữ liệu** Khi bạn phân tích một tập dữ liệu mới, việc chạy một thuật toán phân cụm và sau đó phân tích từng cụm riêng biệt có thể hữu ích.
- **Giảm chiều dữ liệu** Khi một tập dữ liệu đã được phân cụm, thường có thể đo lường độ tương đồng của mỗi trường hợp với mỗi cụm; độ tương đồng là bất kỳ độ đo nào về mức độ một trường hợp phù hợp với một cụm. Vector đặc trưng x của mỗi trường hợp sau đó có thể được thay thế bằng vector các độ tương đồng cụm của nó. Nếu có k cụm, thì vector này có k chiều. Vector mới thường có chiều thấp hơn nhiều so với vector đặc trưng gốc, nhưng nó có thể bảo toàn đủ thông tin để xử lý tiếp.
- **Kỹ thuật đặc trưng** Các độ tương đồng cụm thường có thể hữu ích như các đặc trưng bổ sung. Ví dụ, chúng ta đã sử dụng k-means trong Chương 2 để thêm các đặc trưng độ tương đồng cụm địa lý vào tập dữ liệu California housing, và chúng đã giúp chúng ta đạt được hiệu suất tốt hơn.
- **Phát hiện dị thường** (còn gọi là phát hiện ngoại lệ) Bất kỳ trường hợp nào có độ tương đồng thấp với tất cả các cụm có khả năng là một dị thường. Ví dụ, nếu bạn đã phân cụm người dùng trang web của mình dựa trên hành vi của họ, bạn có thể phát hiện những người dùng có hành vi bất thường, chẳng hạn như số lượng yêu cầu mỗi giây bất thường.
- **Học bán giám sát** Nếu bạn chỉ có một vài nhãn, bạn có thể thực hiện phân cụm và truyền nhãn cho tất cả các trường hợp trong cùng cụm. Kỹ thuật này có thể làm tăng đáng kể số lượng nhãn có sẵn cho một thuật toán học có giám sát tiếp theo, và do đó cải thiện hiệu suất của nó.
- **Công cụ tìm kiếm** Một số công cụ tìm kiếm cho phép bạn tìm kiếm hình ảnh tương tự một hình ảnh tham chiếu. Để xây dựng một hệ thống như vậy, bạn sẽ phải áp dụng một thuật toán phân cụm cho tất cả các hình ảnh trong cơ sở dữ liệu của bạn; các hình ảnh tương tự sẽ nằm trong cùng một cụm. Sau đó, khi người dùng cung cấp một hình ảnh tham chiếu, tất cả những gì bạn cần làm là sử dụng mô hình phân cụm đã được huấn luyện để tìm cụm của hình ảnh này, và sau đó bạn có thể chỉ cần trả về tất cả các hình ảnh từ cụm này.
- **Phân đoạn hình ảnh** Bằng cách phân cụm các pixel theo màu sắc của chúng, sau đó thay thế màu sắc của mỗi pixel bằng màu trung bình của cụm của nó, có thể giảm đáng kể số lượng màu sắc khác nhau trong một hình ảnh. Phân đoạn hình ảnh được sử dụng trong nhiều hệ thống phát hiện và theo dõi đối tượng, vì nó giúp dễ dàng phát hiện đường viền của mỗi đối tượng.

Không có định nghĩa phổ quát nào về một cụm là gì: nó thực sự phụ thuộc vào ngữ cảnh, và các thuật toán khác nhau sẽ nắm bắt các loại cụm khác nhau. Một số thuật toán tìm kiếm các trường hợp tập trung xung quanh một điểm cụ thể, được gọi là **tâm cụm (centroid)**. Những thuật toán khác tìm kiếm các vùng liên tục của các trường hợp được đóng gói dày đặc: các cụm này có thể có bất kỳ hình dạng nào. Một số thuật toán là phân cấp, tìm kiếm

các cụm của các cụm. Và danh sách cứ thế tiếp tục. Trong phần này, chúng ta sẽ xem xét hai thuật toán phân cụm phổ biến, k-means và DBSCAN, và khám phá một số ứng dụng của chúng, chẳng hạn như giảm chiều phi tuyến tính, học bán giám sát và phát hiện dị thường.

k-means

Xem xét tập dữ liệu không được gắn nhãn được biểu thị trong Hình 9-2: bạn có thể thấy rõ năm khối trường hợp. Thuật toán k-means là một thuật toán đơn giản có khả năng phân cụm loại tập dữ liệu này rất nhanh chóng và hiệu quả, thường chỉ trong vài lần lặp. Nó được đề xuất bởi Stuart Lloyd tại Bell Labs vào năm 1957 như một kỹ thuật điều chế mã xung, nhưng mãi đến năm 1982 nó mới được công bố bên ngoài công ty. Năm 1965, Edward W. Forgy đã công bố hầu như cùng một thuật toán, vì vậy k-means đôi khi được gọi là thuật toán Lloyd–Forgy.



Hình 9-2. Một tập dữ liệu không được gắn nhãn bao gồm năm khối trường hợp

Hãy huấn luyện một bộ phân cụm k-means trên tập dữ liệu này. Nó sẽ cố gắng tìm tâm của mỗi khối và gán mỗi trường hợp vào khối gần nhất:

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y = make_blobs([...]) # tạo các khối: y chứa ID cụm, nhưng chúng ta
# sẽ không sử dụng chúng; đó là những gì chúng ta muốn dự đoán
k = 5
kmeans = KMeans(n_clusters=k, random_state=42)
y_pred = kmeans.fit_predict(X)
```

Lưu ý rằng bạn phải chỉ định số lượng cụm k mà thuật toán phải tìm. Trong ví dụ này, rõ ràng khi nhìn vào dữ liệu rằng k nên được đặt thành 5, nhưng nói chung điều đó không dễ

dàng như vậy. Chúng ta sẽ thảo luận điều này ngay sau đây. Mỗi trường hợp sẽ được gán vào một trong năm cụm. Trong bối cảnh phân cụm, nhãn của một trường hợp là chỉ số của cụm mà thuật toán gán trường hợp này; điều này không nên nhầm lẫn với các nhãn lớp trong phân loại, được sử dụng làm mục tiêu (hãy nhớ rằng phân cụm là một tác vụ học không giám sát). Thể hiện `KMeans` giữ lại các nhãn dự đoán của các trường hợp mà nó đã được huấn luyện, có sẵn thông qua biến thể hiện `labels_`:

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)

>>> y_pred is kmeans.labels_
True
```

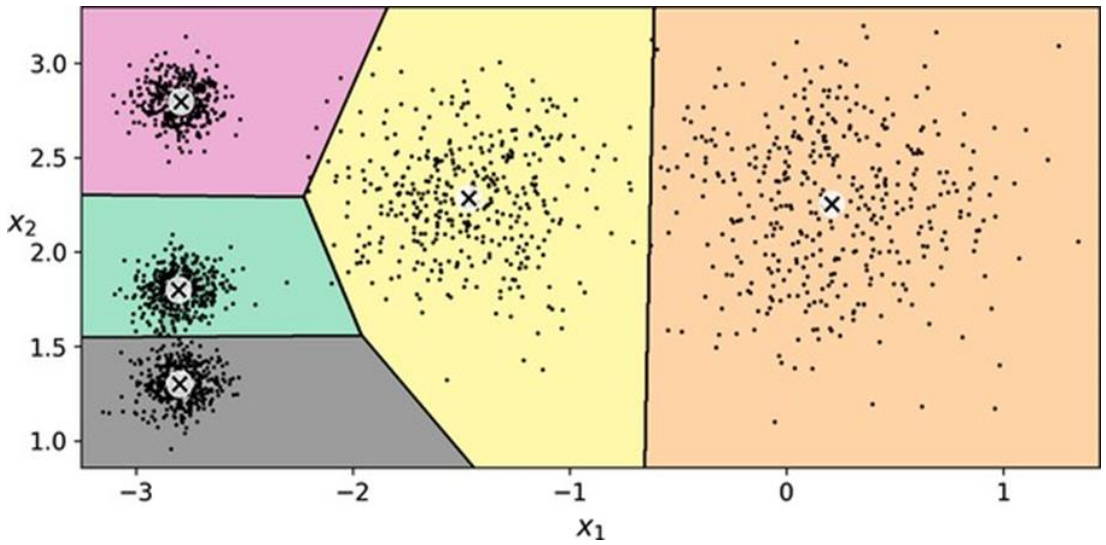
Chúng ta cũng có thể xem xét năm tâm cụm mà thuật toán đã tìm thấy:

```
>>> kmeans.cluster_centers_

array([[-2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

Bạn có thể dễ dàng gán các trường hợp mới vào cụm có tâm cụm gần nhất:

```
>>> import numpy as np
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
>>> kmeans.predict(X_new)
array([1, 1, 2, 2], dtype=int32)
```

Nếu bạn vẽ các đường biên quyết định của cụm, bạn sẽ có một phép phân vùng Voronoi: xem Hình 9-3, trong đó mỗi tâm cụm được biểu thị bằng một chữ X.



Hình 9-3. Đường biên quyết định của k-means (phép phân vùng Voronoi)

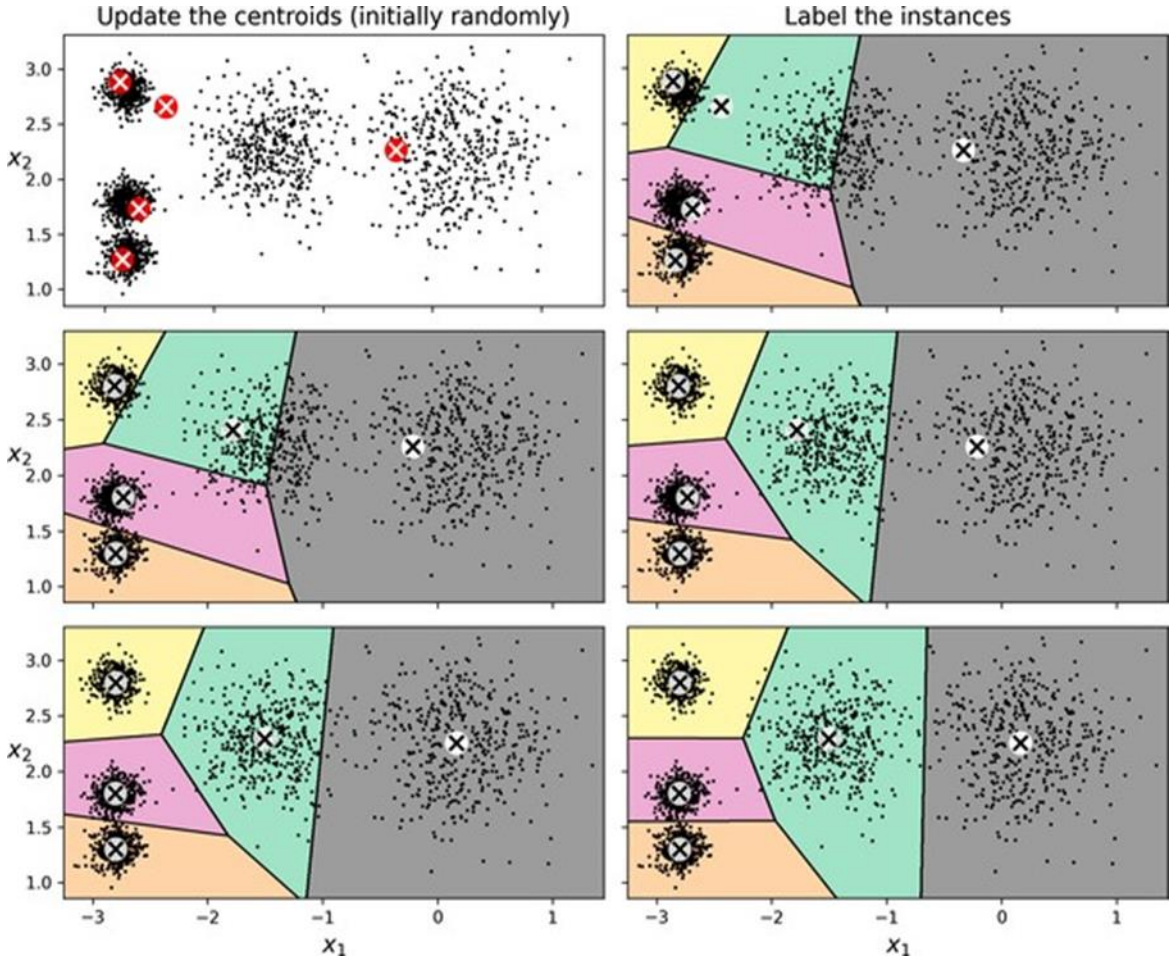
Phần lớn các trường hợp rõ ràng được gán vào cụm thích hợp, nhưng một vài trường hợp có lẽ bị gán sai nhãn, đặc biệt gần ranh giới giữa cụm trên cùng bên trái và cụm trung tâm. Thật vậy, thuật toán k-means hoạt động không tốt lắm khi các khối có đường kính rất khác nhau bởi vì tất cả những gì nó quan tâm khi gán một trường hợp vào một cụm là khoảng cách đến tâm cụm. Thay vì gán mỗi trường hợp vào một cụm duy nhất, được gọi là **phân cụm cứng (hard clustering)**, có thể hữu ích khi cho mỗi trường hợp một điểm số cho mỗi cụm, được gọi là **phân cụm mềm (soft clustering)**. Điểm số có thể là khoảng cách giữa trường hợp và tâm cụm hoặc một điểm số tương tự (hoặc độ tương đồng), chẳng hạn như hàm cơ sở xuyên tâm Gaussian mà chúng ta đã sử dụng trong Chương 2. Trong lớp `KMeans`, phương thức `transform()` đo khoảng cách từ mỗi trường hợp đến mọi tâm cụm:

```
>>> kmeans.transform(X_new).round(2)

array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])
```

Trong ví dụ này, trường hợp đầu tiên trong `X_new` nằm ở khoảng cách khoảng 2.81 từ tâm cụm đầu tiên, 0.33 từ tâm cụm thứ hai, 2.90 từ tâm cụm thứ ba, 1.49 từ tâm cụm thứ tư và 2.89 từ tâm cụm thứ năm. Nếu bạn có một tập dữ liệu có chiều cao và bạn biến đổi nó theo cách này, bạn sẽ có một tập dữ liệu k -chiều: phép biến đổi này có thể là một kỹ thuật giảm chiều phi tuyến tính rất hiệu quả. Ngoài ra, bạn có thể sử dụng các khoảng cách này làm các đặc trưng bổ sung để huấn luyện một mô hình khác, như trong Chương 2.

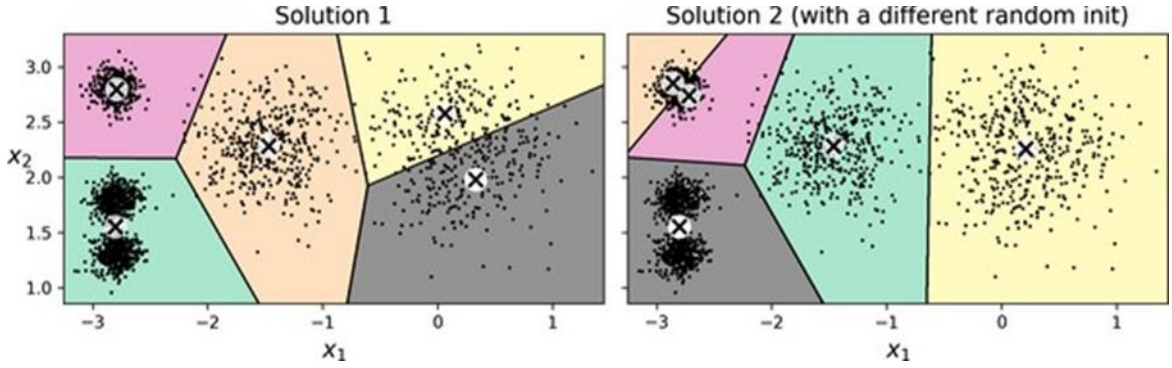
Thuật toán k-means

Vậy, thuật toán hoạt động như thế nào? Chà, giả sử bạn được cung cấp các tâm cụm. Bạn có thể dễ dàng gắn nhãn tất cả các trường hợp trong tập dữ liệu bằng cách gán mỗi trường hợp vào cụm có tâm cụm gần nhất. Ngược lại, nếu bạn được cung cấp tất cả các nhãn trường hợp, bạn có thể dễ dàng xác định vị trí tâm cụm của mỗi cụm bằng cách tính trung bình các trường hợp trong cụm đó. Nhưng bạn không được cung cấp cả nhãn lẫn tâm cụm, vậy làm thế nào bạn có thể tiếp tục? Bắt đầu bằng cách đặt các tâm cụm một cách ngẫu nhiên (ví dụ: bằng cách chọn k trường hợp ngẫu nhiên từ tập dữ liệu và sử dụng vị trí của chúng làm tâm cụm). Sau đó gắn nhãn các trường hợp, cập nhật các tâm cụm, gắn nhãn các trường hợp, cập nhật các tâm cụm, v.v. cho đến khi các tâm cụm ngừng di chuyển. Thuật toán được đảm bảo hội tụ trong một số bước hữu hạn (thường khá nhỏ). Điều đó là do khoảng cách bình phương trung bình giữa các trường hợp và các tâm cụm gần nhất của chúng chỉ có thể giảm xuống ở mỗi bước, và vì nó không thể là số âm, nó được đảm bảo hội tụ. Bạn có thể thấy thuật toán hoạt động trong Hình 9-4: các tâm cụm được khởi tạo ngẫu nhiên (trên cùng bên trái), sau đó các trường hợp được gắn nhãn (trên cùng bên phải), sau đó các tâm cụm được cập nhật (giữa bên trái), các trường hợp được gắn nhãn lại (giữa bên phải), v.v. Như bạn có thể thấy, chỉ trong ba lần lặp, thuật toán đã đạt được một phân cụm dường như gần tối ưu.



Hình 9-4. Thuật toán k-means

Mặc dù thuật toán được đảm bảo hội tụ, nó có thể không hội tụ đến giải pháp đúng (tức là nó có thể hội tụ đến một tối ưu cục bộ): việc nó có hội tụ hay không phụ thuộc vào việc khởi tạo tâm cụm. Hình 9-5 cho thấy hai giải pháp dưới tối ưu mà thuật toán có thể hội tụ nếu bạn không may mắn với bước khởi tạo ngẫu nhiên.



Hình 9-5. Các giải pháp dưới tối ưu do khởi tạo tâm cụm không may mắn

Hãy cùng xem xét một vài cách bạn có thể giảm thiểu rủi ro này bằng cách cải thiện việc khởi tạo tâm cụm. Các phương pháp khởi tạo tâm cụm Nếu bạn tình cờ biết xấp xỉ vị trí các tâm cụm (ví dụ: nếu bạn đã chạy một thuật toán phân cụm khác trước đó), thì bạn có thể đặt siêu tham số `init` thành một mảng NumPy chứa danh sách các tâm cụm, và đặt `n_init` thành 1:

```
good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)
kmeans.fit(X)
```

Một giải pháp khác là chạy thuật toán nhiều lần với các khởi tạo ngẫu nhiên khác nhau và giữ lại giải pháp tốt nhất. Số lần khởi tạo ngẫu nhiên được kiểm soát bởi siêu tham số `n_init`: theo mặc định, nó bằng 10, có nghĩa là toàn bộ thuật toán được mô tả trước đó chạy 10 lần khi bạn gọi `fit()`, và Scikit-Learn giữ lại giải pháp tốt nhất. Nhưng chính xác thì làm thế nào nó biết giải pháp nào là tốt nhất? Nó sử dụng một thước đo hiệu suất! Thước đo đó được gọi là **quán tính (inertia)** của mô hình, là tổng khoảng cách bình phương giữa các trường hợp và tâm cụm gần nhất của chúng. Nó xấp xỉ bằng 219.4 cho mô hình ở bên trái trong Hình 9-5, 258.6 cho mô hình ở bên phải trong Hình 9-5, và chỉ 211.6 cho mô hình trong Hình 9-3. Lớp `KMeans` chạy thuật toán `n_init` lần và giữ lại mô hình có quán tính thấp nhất. Trong ví dụ này, mô hình trong Hình 9-3 sẽ được chọn (trừ khi chúng ta rất không may mắn với `n_init` khởi tạo ngẫu nhiên liên tiếp). Nếu bạn tò mò, quán tính của mô hình có thể truy cập được thông qua biến thể hiện `inertia_`:

```
>>> kmeans.inertia_
211.59853725816836
```

Phương thức `score()` trả về quán tính âm (nó âm vì phương thức `score()` của một bộ dự đoán phải luôn tuân thủ quy tắc "lớn hơn là tốt hơn" của Scikit-Learn: nếu một bộ dự đoán tốt hơn một bộ dự đoán khác, phương thức `score()` của nó phải trả về một điểm số lớn hơn):

```
>>> kmeans.score(X)
-211.5985372581684
```

Một cải tiến quan trọng cho thuật toán k-means, **k-means++**, đã được đề xuất trong một bài báo năm 2006 của David Arthur và Sergei Vassilvitskii. Họ đã giới thiệu một bước khởi tạo thông minh hơn có xu hướng chọn các tâm cụm cách xa nhau, và cải tiến này làm cho thuật toán k-means ít có khả năng hội tụ đến một giải pháp dưới tối ưu hơn nhiều. Bài báo đã chỉ ra rằng việc tính toán bổ sung cần thiết cho bước khởi tạo thông minh hơn rất đáng giá vì nó giúp giảm đáng kể số lần thuật toán cần được chạy để tìm ra giải pháp tối ưu. Thuật toán khởi tạo k-means++ hoạt động như sau:

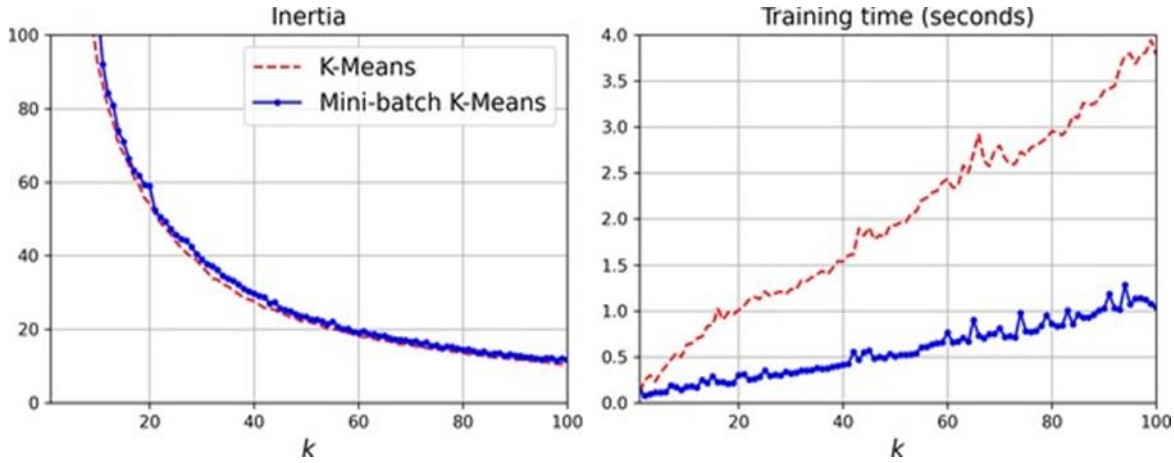
1. Lấy một tâm cụm $c^{(1)}$, được chọn ngẫu nhiên đồng đều từ tập dữ liệu.
2. Lấy một tâm cụm mới $c^{(i)}$, chọn một trường hợp $x^{(i)}$ với xác suất $D\left(x^{(i)}\right)^2 / sum_j = 1^m D\left(x^{(j)}\right)^2$, trong đó $D\left(x^{(i)}\right)$ là khoảng cách giữa trường hợp $x^{(i)}$ và tâm cụm gần

nhất đã được chọn. Phân phối xác suất này đảm bảo rằng các trường hợp ở xa các tâm cụm đã chọn có nhiều khả năng được chọn làm tâm cụm hơn.

3. Lặp lại bước trước cho đến khi tất cả k tâm cụm đã được chọn. Lớp `KMeans` sử dụng phương pháp khởi tạo này theo mặc định. K-means tăng tốc và k-means theo mini-batch Một cải tiến quan trọng khác cho thuật toán k-means đã được đề xuất trong một bài báo năm 2003 của Charles Elkan. Trên một số tập dữ liệu lớn với nhiều cụm, thuật toán có thể được tăng tốc bằng cách tránh nhiều phép tính khoảng cách không cần thiết. Elkan đã đạt được điều này bằng cách khai thác bất đẳng thức tam giác (tức là một đường thẳng luôn là khoảng cách ngắn nhất giữa hai điểm) và bằng cách theo dõi các cận dưới và cận trên cho khoảng cách giữa các trường hợp và tâm cụm. Tuy nhiên, thuật toán của Elkan không phải lúc nào cũng tăng tốc quá trình huấn luyện, và đôi khi nó thậm chí có thể làm chậm quá trình huấn luyện đáng kể; điều đó phụ thuộc vào tập dữ liệu. Tuy nhiên, nếu bạn muốn thử, hãy đặt `algorithm="elkan"`. Một biến thể quan trọng khác của thuật toán k-means đã được đề xuất trong một bài báo năm 2010 của David Sculley. Thay vì sử dụng toàn bộ tập dữ liệu ở mỗi lần lặp, thuật toán có khả năng sử dụng các mini-batch, di chuyển các tâm cụm chỉ một chút ở mỗi lần lặp. Điều này tăng tốc thuật toán (thường gấp ba đến bốn lần) và giúp có thể phân cụm các tập dữ liệu khổng lồ không vừa trong bộ nhớ. Scikit-Learn triển khai thuật toán này trong lớp `MiniBatchKMeans`, mà bạn có thể sử dụng giống như lớp `KMeans`:

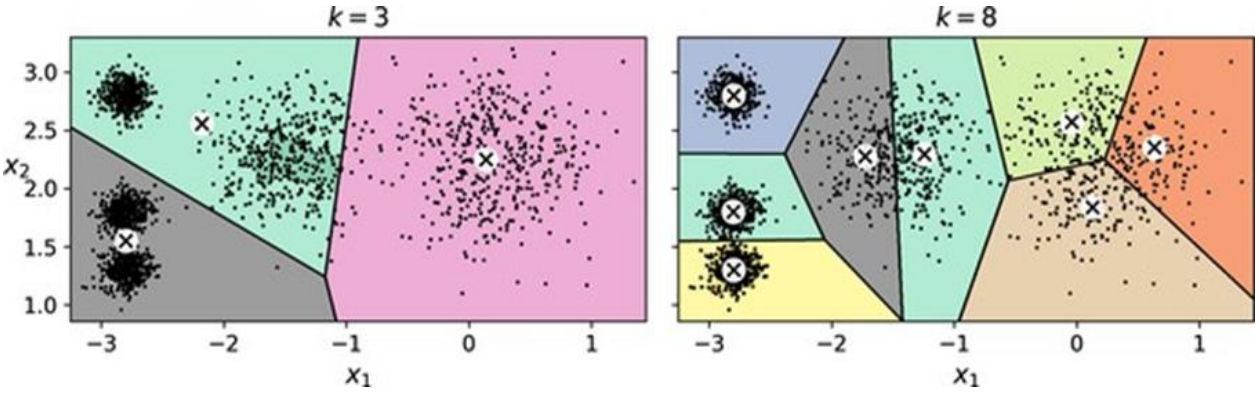
```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=5, random_state=42)
minibatch_kmeans.fit(X)
```

Nếu tập dữ liệu không vừa trong bộ nhớ, lựa chọn đơn giản nhất là sử dụng lớp `memmap`, như chúng ta đã làm cho PCA tăng dần trong Chương 8. Ngoài ra, bạn có thể truyền từng mini-batch một cho phương thức `partial_fit()`, nhưng điều này sẽ yêu cầu nhiều công việc hơn, vì bạn sẽ cần thực hiện nhiều lần khởi tạo và tự mình chọn giải pháp tốt nhất. Mặc dù thuật toán k-means theo mini-batch nhanh hơn nhiều so với thuật toán k-means thông thường, nhưng quán tính của nó nói chung hơi tệ hơn. Bạn có thể thấy điều này trong Hình 9-6: biểu đồ bên trái so sánh quán tính của các mô hình k-means theo mini-batch và k-means thông thường được huấn luyện trên tập dữ liệu năm khối trước đó bằng cách sử dụng các số lượng cụm k khác nhau. Sự khác biệt giữa hai đường cong là nhỏ, nhưng có thể nhìn thấy. Trong biểu đồ bên phải, bạn có thể thấy rằng k-means theo mini-batch nhanh hơn k-means thông thường khoảng 3.5 lần trên tập dữ liệu này.

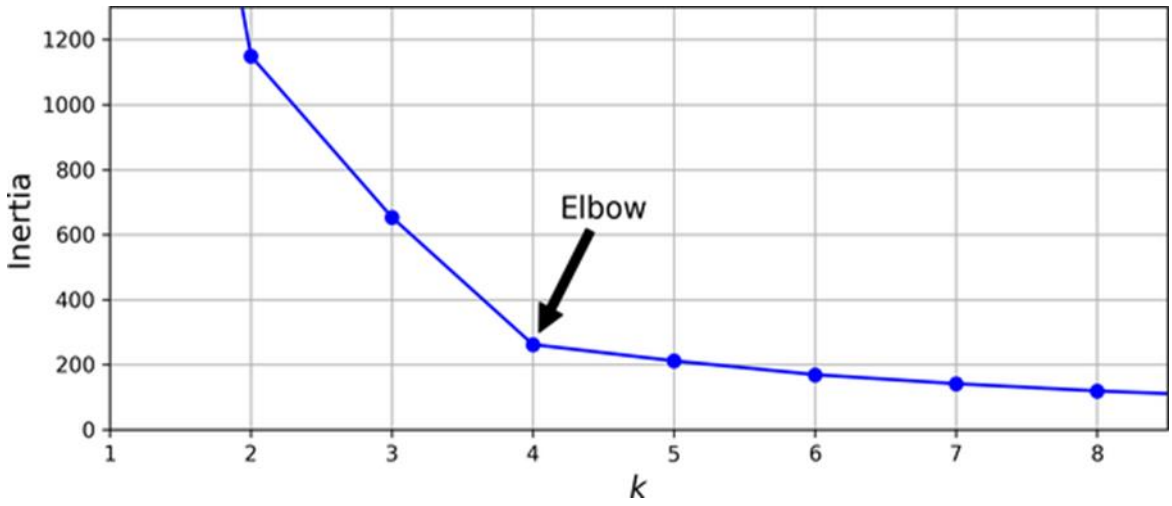


Hình 9-6. K-means theo mini-batch có quán tính cao hơn k-means (trái) nhưng nó nhanh hơn nhiều (phải), đặc biệt khi k tăng

Tìm số lượng cụm tối ưu Cho đến nay, chúng ta đã đặt số lượng cụm k là 5 vì rõ ràng khi nhìn vào dữ liệu rằng đây là số lượng cụm chính xác. Nhưng nói chung, sẽ không dễ dàng để biết cách đặt k , và kết quả có thể khá tệ nếu bạn đặt nó thành một giá trị sai. Như bạn có thể thấy trong Hình 9-7, đối với tập dữ liệu này, việc đặt k thành 3 hoặc 8 dẫn đến các mô hình khá tệ. Bạn có thể nghĩ rằng bạn có thể chỉ cần chọn mô hình có quán tính thấp nhất. Thật không may, điều đó không đơn giản. Quán tính cho $k = 3$ là khoảng 653.2, cao hơn nhiều so với $k = 5$ (211.6). Nhưng với $k = 8$, quán tính chỉ là 119.1. Quán tính không phải là một thước đo hiệu suất tốt khi cố gắng chọn k vì nó tiếp tục giảm khi chúng ta tăng k . Thật vậy, càng có nhiều cụm, mỗi trường hợp sẽ càng gần với tâm cụm gần nhất của nó, và do đó quán tính sẽ càng thấp. Hãy vẽ quán tính như một hàm của k . Khi chúng ta làm điều này, đường cong thường chứa một điểm uốn được gọi là **khuỷu tay (elbow)** (xem Hình 9-8).



Hình 9-7. Lựa chọn số lượng cụm kém: khi k quá nhỏ, các cụm riêng biệt bị hợp nhất (trái), và khi k quá lớn, một số cụm bị chia thành nhiều mảnh (phải)

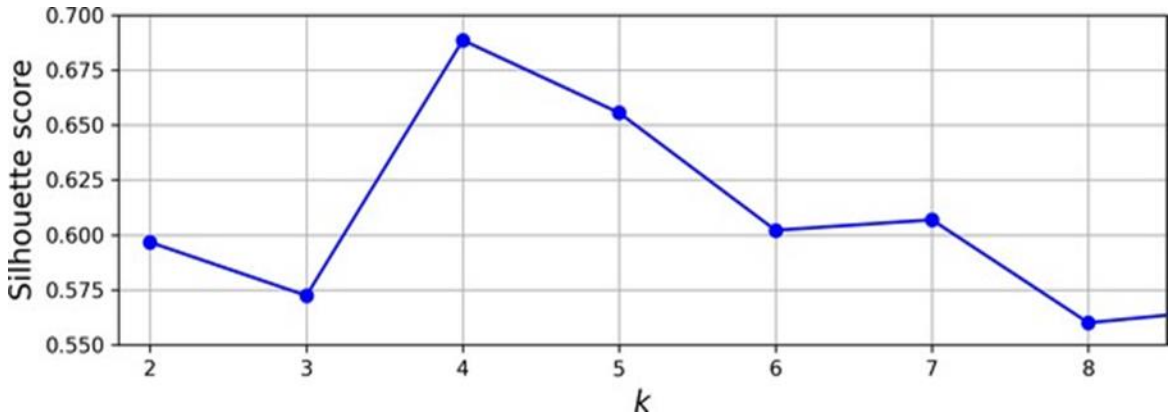


Hình 9-8. Vẽ quán tính như một hàm của số lượng cụm k

Như bạn có thể thấy, quán tính giảm rất nhanh khi chúng ta tăng k lên đến 4, nhưng sau đó nó giảm chậm hơn nhiều khi chúng ta tiếp tục tăng k . Đường cong này gần giống hình cánh tay, và có một khuỷu tay tại $k = 4$. Vì vậy, nếu chúng ta không có cách nào tốt hơn, chúng ta có thể nghĩ 4 là một lựa chọn tốt: bất kỳ giá trị thấp hơn nào cũng sẽ rất tệ, trong khi bất kỳ giá trị cao hơn nào cũng sẽ không giúp ích nhiều, và chúng ta có thể chỉ đang chia các cụm tốt thành hai mà không có lý do chính đáng. Kỹ thuật này để chọn giá trị tốt nhất cho số lượng cụm khá thô. Một cách tiếp cận chính xác hơn (nhưng cũng tốn kém tính toán hơn) là sử dụng **điểm silhouette (silhouette score)**, là hệ số silhouette trung bình trên tất cả các trường hợp. Hệ số silhouette của một trường hợp bằng $(b- a)/max(a, b)$, trong đó a là khoảng cách trung bình đến các trường hợp khác trong cùng cụm (tức là khoảng cách nội cụm trung bình) và b là khoảng cách cụm gần nhất trung bình (tức là khoảng cách trung bình đến các trường hợp của cụm gần nhất tiếp theo, được định nghĩa là cụm làm tối thiểu b , không bao gồm cụm của chính trường hợp). Hệ số silhouette có thể thay đổi từ –1 đến +1. Hệ số gần +1 có nghĩa là trường hợp nằm sâu trong cụm của chính nó và cách xa các cụm khác, trong khi hệ số gần 0 có nghĩa là nó gần ranh giới cụm; cuối cùng, hệ số gần –1 có nghĩa là trường hợp có thể đã bị gán sai cụm. Để tính điểm silhouette, bạn có thể sử dụng hàm `silhouette_score()` của Scikit-Learn, cung cấp cho nó tất cả các trường hợp trong tập dữ liệu và các nhãn mà chúng được gán:

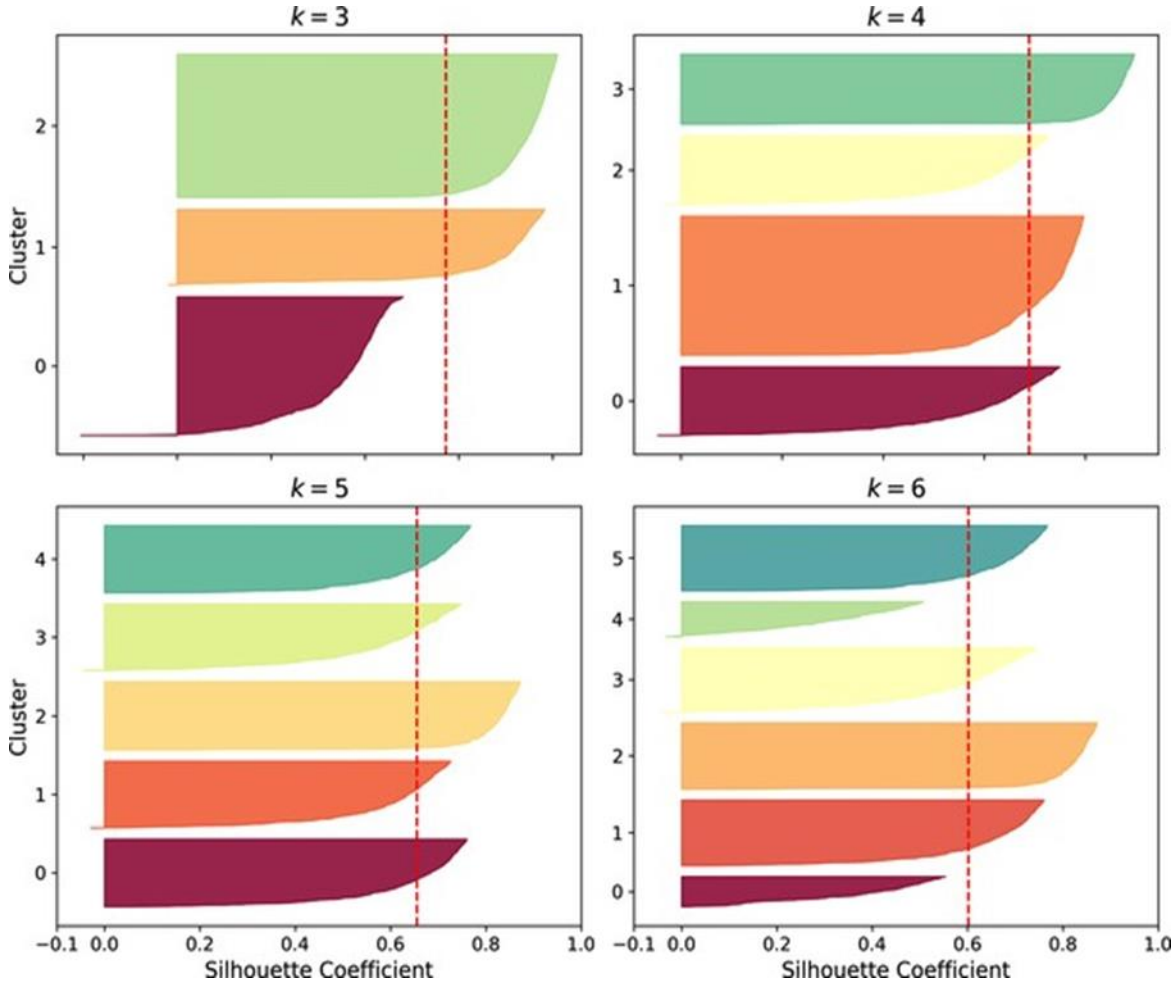
```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
0.655517642572828
```

Hãy so sánh các điểm silhouette cho các số lượng cụm khác nhau (xem Hình 9-9).



Hình 9-9. Chọn số lượng cụm k bằng cách sử dụng điểm silhouette

Như bạn có thể thấy, trực quan hóa này phong phú hơn nhiều so với trực quan hóa trước đó: mặc dù nó xác nhận rằng $k = 4$ là một lựa chọn rất tốt, nó cũng làm nổi bật thực tế rằng $k = 5$ cũng khá tốt, và tốt hơn nhiều so với $k = 6$ hoặc 7. Điều này không thể nhìn thấy khi so sánh quán tính. Một trực quan hóa mang lại nhiều thông tin hơn nữa được tạo ra khi chúng ta vẽ hệ số silhouette của mỗi trường hợp, được sắp xếp theo các cụm mà chúng được gán và theo giá trị của hệ số. Đây được gọi là **biểu đồ silhouette** (xem Hình 9-10). Mỗi biểu đồ chứa một hình dao cho mỗi cụm. Chiều cao của hình dạng cho biết số lượng trường hợp trong cụm, và chiều rộng của nó biểu thị các hệ số silhouette đã được sắp xếp của các trường hợp trong cụm (càng rộng càng tốt). Các đường nét đứt dọc biểu thị điểm silhouette trung bình cho mỗi số lượng cụm. Khi hầu hết các trường hợp trong một cụm có hệ số thấp hơn điểm này (tức là nếu nhiều trường hợp dừng lại trước đường nét đứt, kết thúc ở bên trái của nó), thì cụm đó khá tệ vì điều này có nghĩa là các trường hợp của nó quá gần các cụm khác. Ở đây chúng ta có thể thấy rằng khi $k = 3$ hoặc 6, chúng ta có các cụm tệ. Nhưng khi $k = 4$ hoặc 5, các cụm trông khá tốt: hầu hết các trường hợp mở rộng ra ngoài đường nét đứt, sang bên phải và gần 1.0 hơn. Khi $k = 4$, cụm ở chỉ số 1 (thứ hai từ dưới lên) khá lớn. Khi $k = 5$, tất cả các cụm có kích thước tương tự nhau. Vì vậy, mặc dù điểm silhouette tổng thể từ $k = 4$ hơi lớn hơn so với $k = 5$, có vẻ như nên sử dụng $k = 5$ để có các cụm có kích thước tương tự.

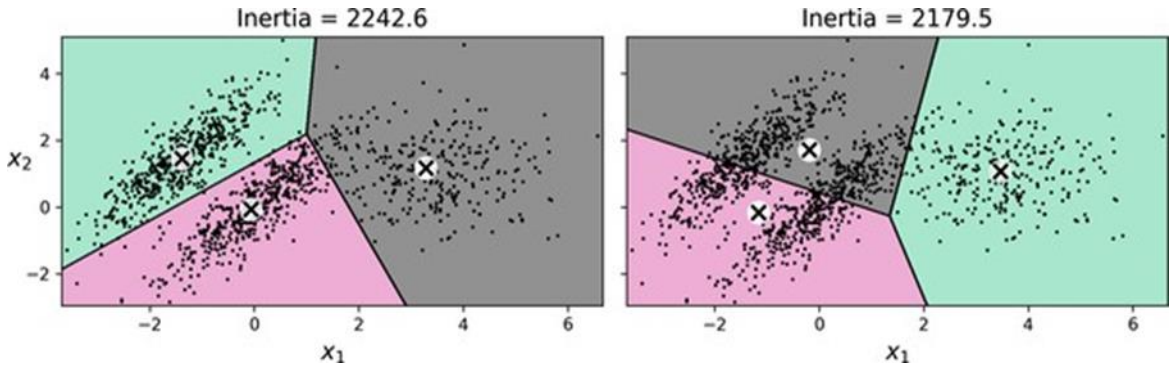


Hình 9-10. Phân tích các biểu đồ silhouette cho các giá trị khác nhau của k

Giới hạn của k-means

Mặc dù có nhiều ưu điểm, đáng chú ý nhất là nhanh và có khả năng mở rộng, k-means không hoàn hảo. Như chúng ta đã thấy, cần phải chạy thuật toán nhiều lần để tránh các giải pháp dưới tối ưu, cộng với việc bạn cần chỉ định số lượng cụm, điều này có thể khá phiền phức. Hơn nữa, k-means không hoạt động tốt lắm khi các cụm có kích thước khác nhau, mật độ khác nhau hoặc hình dạng không cầu. Ví dụ, Hình 9-11 cho thấy cách k-means phân cụm một tập dữ liệu chứa ba cụm hình elip có kích thước, mật độ và định hướng khác nhau.

Như bạn có thể thấy, không giải pháp nào trong số này là tốt cả. Giải pháp bên trái tốt hơn, nhưng nó vẫn cắt bỏ 25% cụm giữa và gán nó cho cụm bên phải. Giải pháp bên phải thì tệ hại, mặc dù quán tính của nó thấp hơn. Vì vậy, tùy thuộc vào dữ liệu, các thuật toán phân cụm khác nhau có thể hoạt động tốt hơn. Trên các loại cụm hình elip này, các mô hình hỗn hợp Gaussian hoạt động rất tốt.



Hình 9-11. k-means không phân cụm các khối hình elip này đúng cách

Bây giờ chúng ta hãy xem xét một vài cách chúng ta có thể hưởng lợi từ việc phân cụm. Chúng ta sẽ sử dụng k-means, nhưng bạn có thể thoải mái thử nghiệm với các thuật toán phân cụm khác.

Sử dụng phân cụm cho phân đoạn hình ảnh

Phân đoạn hình ảnh là nhiệm vụ phân chia một hình ảnh thành nhiều phân đoạn. Có một số biến thể:

- Trong **phân đoạn màu sắc**, các pixel có màu tương tự được gán vào cùng một phân đoạn. Điều này đủ trong nhiều ứng dụng. Ví dụ, nếu bạn muốn phân tích hình ảnh vệ tinh để đo tổng diện tích rừng trong một khu vực, phân đoạn màu sắc có thể là đủ.
- Trong **phân đoạn ngữ nghĩa**, tất cả các pixel là một phần của cùng một loại đối tượng được gán vào cùng một phân đoạn. Ví dụ, trong hệ thống thị giác của một chiếc xe tự lái, tất cả các pixel là một phần của hình ảnh người đi bộ có thể được gán vào phân đoạn "người đi bộ" (sẽ có một phân đoạn chứa tất cả những người đi bộ).
- Trong **phân đoạn thể hiện**, tất cả các pixel là một phần của cùng một đối tượng riêng lẻ được gán vào cùng một phân đoạn. Trong trường hợp này, sẽ có một phân đoạn khác cho mỗi người đi bộ. Hiện trạng của phân đoạn ngữ nghĩa hoặc thể hiện ngày nay đạt được bằng cách sử dụng các kiến trúc phức tạp dựa trên mạng lưới thần kinh tích chập (xem Chương 14). Trong chương này, chúng ta sẽ tập trung vào tác vụ phân đoạn màu sắc (đơn giản hơn nhiều), sử dụng k-means. Chúng ta sẽ bắt đầu bằng cách nhập gói `Pillow` (kế thừa từ Thư viện hình ảnh Python, PIL), sau đó chúng ta sẽ sử dụng nó để tải hình ảnh `ladybug.png` (xem hình ảnh trên cùng bên trái trong Hình 9-12), giả sử nó nằm ở đường dẫn tệp:

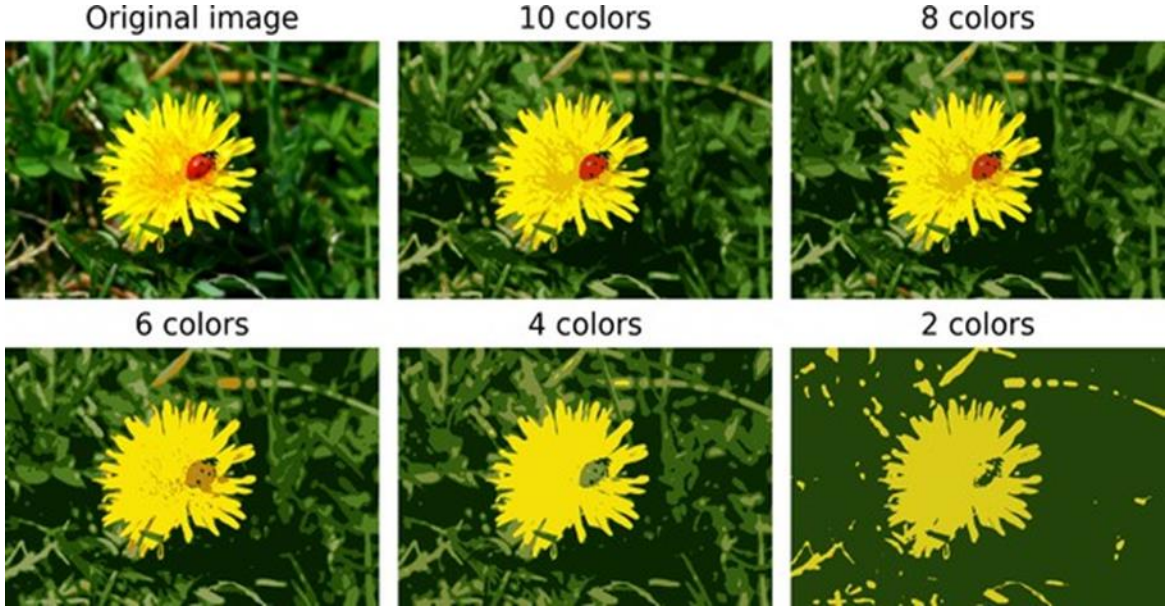
```
>>> import numpy as np
>>> import PIL.Image
>>> filepath = "path/to/ladybug.png" # Placeholder for the actual file path
>>> image = np.asarray(PIL.Image.open(filepath))
>>> image.shape
(533, 800, 3)
```

Hình ảnh được biểu diễn dưới dạng một mảng 3D. Kích thước của chiều thứ nhất là chiều cao; chiều thứ hai là chiều rộng; và chiều thứ ba là số kênh màu, trong trường hợp này là

đỏ, xanh lá cây và xanh lam (RGB). Nói cách khác, đối với mỗi pixel có một vector 3D chứa cường độ của màu đỏ, xanh lá cây và xanh lam dưới dạng số nguyên 8 bit không dấu từ 0 đến 255. Một số hình ảnh có thể có ít kênh hơn (chẳng hạn như hình ảnh thang độ xám, chỉ có một), và một số hình ảnh có thể có nhiều kênh hơn (chẳng hạn như hình ảnh có kênh alpha bổ sung để trong suốt, hoặc hình ảnh vệ tinh, thường chứa các kênh cho các tần số ánh sáng bổ sung (như hồng ngoại). Đoạn mã sau định hình lại mảng để có được một danh sách dài các màu RGB, sau đó nó phân cụm các màu này bằng k-means với tám cụm. Nó tạo một mảng `segmented_img` chứa tâm cụm gần nhất cho mỗi pixel (tức là màu trung bình của cụm của mỗi pixel), và cuối cùng nó định hình lại mảng này thành hình dạng hình ảnh gốc. Dòng thứ ba sử dụng lập chỉ mục NumPy nâng cao; ví dụ, nếu 10 nhãn đầu tiên trong `kmeans.labels_` bằng 1, thì 10 màu đầu tiên trong `segmented_img` bằng `kmeans.cluster_centers_[1]`:

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8, random_state=42, n_init=10).fit(X) # Thêm
n_init=10 để tránh lỗi cảnh báo
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

Điều này xuất ra hình ảnh được hiển thị ở phía trên bên phải của Hình 9-12. Bạn có thể thử nghiệm với các số lượng cụm khác nhau, như trong hình. Khi bạn sử dụng ít hơn tám cụm, hãy lưu ý rằng màu đỏ rực rỡ của bọ rùa không có cụm riêng: nó bị hợp nhất với các màu từ môi trường. Điều này là do k-means ưu tiên các cụm có kích thước tương tự. Bọ rùa nhỏ—nhỏ hơn nhiều so với phần còn lại của hình ảnh—vì vậy ngay cả khi màu của nó rực rỡ, k-means không phân bổ một cụm cho nó.



Hình 9-12. Phân đoạn hình ảnh sử dụng k-means với các số lượng cụm màu khác nhau

Điều đó không quá khó, phải không? Bây giờ chúng ta hãy xem một ứng dụng khác của phân cụm.

Sử dụng phân cụm cho học bán giám sát

Một trường hợp sử dụng khác cho phân cụm là trong **học bán giám sát**, khi chúng ta có rất nhiều trường hợp không được gắn nhãn và rất ít trường hợp được gắn nhãn. Trong phần này, chúng ta sẽ sử dụng tập dữ liệu chữ số, là một tập dữ liệu đơn giản giống MNIST chứa 1.797 hình ảnh thang độ xám 8 × 8 biểu thị các chữ số từ 0 đến 9. Đầu tiên, hãy tải và chia tập dữ liệu (nó đã được xáo trộn):

```
from sklearn.datasets import load_digits

X_digits, y_digits = load_digits(return_X_y=True)
X_train, y_train = X_digits[:1400], y_digits[:1400]
X_test, y_test = X_digits[1400:], y_digits[1400:]
```

Chúng ta sẽ giả vờ rằng chúng ta chỉ có nhãn cho 50 trường hợp. Để có hiệu suất cơ sở, hãy huấn luyện một mô hình hồi quy logistic trên 50 trường hợp được gắn nhãn này:

```
from sklearn.linear_model import LogisticRegression

n_labeled = 50
log_reg = LogisticRegression(max_iter=10_000, random_state=42) # Thêm
random_state để có kết quả tái tạo được
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

Sau đó, chúng ta có thể đo độ chính xác của mô hình này trên tập kiểm tra (lưu ý rằng tập kiểm tra phải được gắn nhãn):

```
>>> log_reg.score(X_test, y_test)
0.7481108312342569
```

Độ chính xác của mô hình chỉ là 74.8%. Điều đó không tốt: thực tế, nếu bạn thử huấn luyện mô hình trên toàn bộ tập huấn luyện, bạn sẽ thấy rằng nó sẽ đạt độ chính xác khoảng 90.7%. Hãy xem cách chúng ta có thể làm tốt hơn. Đầu tiên, hãy phân cụm tập huấn luyện thành 50 cụm. Sau đó, đối với mỗi cụm, chúng ta sẽ tìm hình ảnh gần nhất với tâm cụm. Chúng ta sẽ gọi những hình ảnh này là các hình ảnh đại diện:

```
k = 50
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10) # Thêm n_init=10
kmeans.fit(X_train)
X_digits_dist = kmeans.transform(X_train)
representative_digit_idx = np.argmin(X_digits_dist, axis=0)
X_representative_digits = X_train[representative_digit_idx]
```

Hình 9-13 cho thấy 50 hình ảnh đại diện.

Hình 9-13. Năm mươi hình ảnh chữ số đại diện (một hình ảnh cho mỗi cụm)

Hãy xem xét từng hình ảnh và gắn nhãn thủ công cho chúng:

```
y_representative_digits = np.array([
    4, 9, 2, 0, 1, 6, 8, 0, 7, 8, 3, 5, 0, 4, 6, 2, 7, 5, 1, 9,
    1, 2, 4, 3, 9, 8, 4, 7, 6, 3, 0, 2, 5, 7, 1, 9, 3, 8, 6, 4,
    2, 0, 7, 5, 1, 9, 3, 8, 6, 4
]) # Đây là một mảng ví dụ, bạn cần tự gắn nhãn dựa trên Hình 9-13
```

Bây giờ chúng ta có một tập dữ liệu chỉ với 50 trường hợp được gắn nhãn, nhưng thay vì là các trường hợp ngẫu nhiên, mỗi trường hợp là một hình ảnh đại diện của cụm của nó. Hãy xem hiệu suất có tốt hơn không:

```
>>> log_reg = LogisticRegression(max_iter=10_000, random_state=42)
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.8488664987405542
```

Tuyệt vời! Chúng ta đã nhảy từ độ chính xác 74.8% lên 84.9%, mặc dù chúng ta vẫn chỉ huấn luyện mô hình trên 50 trường hợp. Vì việc gắn nhãn các trường hợp thường tốn kém và đau đớn, đặc biệt khi phải thực hiện thủ công bởi các chuyên gia, nên việc gắn nhãn các trường hợp đại diện thay vì chỉ các trường hợp ngẫu nhiên là một ý hay.

Nhưng có lẽ chúng ta có thể tiến thêm một bước nữa: điều gì sẽ xảy ra nếu chúng ta truyền nhãn cho tất cả các trường hợp khác trong cùng cụm? Điều này được gọi là **truyền nhãn (label propagation)**:

```
y_train_propagated = np.empty(len(X_train), dtype=np.int64)
for i in range(k):
    y_train_propagated[kmeans.labels_ == i] = y_representative_digits[i]
```

Bây giờ hãy huấn luyện lại mô hình và xem hiệu suất của nó:

```
>>> log_reg = LogisticRegression(max_iter=10_000, random_state=42)
>>> log_reg.fit(X_train, y_train_propagated)
>>> log_reg.score(X_test, y_test)
0.8942065491183879
```

Chúng ta đã nhận được một sự tăng cường độ chính xác đáng kể khác! Hãy xem liệu chúng ta có thể làm tốt hơn nữa bằng cách bỏ qua 1% các trường hợp cách xa tâm cụm của

chúng nhất: điều này sẽ loại bỏ một số ngoại lệ. Đoạn mã sau đây đầu tiên tính khoảng cách từ mỗi trường hợp đến tâm cụm gần nhất của nó, sau đó đối với mỗi cụm, nó đặt 1% khoảng cách lớn nhất thành –1. Cuối cùng, nó tạo một tập hợp không có các trường hợp này được đánh dấu bằng khoảng cách –1:

```
percentile_closest = 99

X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]

for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1

partially_propagated = (X_cluster_dist != -1)
X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train_propagated[partially_propagated]
```

Bây giờ hãy huấn luyện lại mô hình trên tập dữ liệu đã được truyền nhãn một phần này và xem độ chính xác chúng ta nhận được:

```
>>> log_reg = LogisticRegression(max_iter=10_000, random_state=42)
>>> log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
>>> log_reg.score(X_test, y_test)
0.9093198992443325
```

Tuyệt vời! Chỉ với 50 trường hợp được gắn nhãn (trung bình chỉ 5 ví dụ cho mỗi lớp!) chúng ta đã đạt độ chính xác 90.9%, thực tế cao hơn một chút so với hiệu suất chúng ta nhận được trên tập dữ liệu chữ số đã được gắn nhãn đầy đủ (90.7%). Điều này một phần nhờ vào việc chúng ta đã loại bỏ một số ngoại lệ, và một phần vì các nhãn được truyền thực sự khá tốt—độ chính xác của chúng khoảng 97.5%, như đoạn mã sau cho thấy:

```
>>> (y_train_partially_propagated == y_train[partially_propagated]).mean()
0.9755555555555555
```

HỌC TÍCH CỰC (ACTIVE LEARNING) Để tiếp tục cải thiện mô hình và tập huấn luyện của bạn, bước tiếp theo có thể là thực hiện một vài vòng **học tích cực (active learning)**, là khi một chuyên gia con người tương tác với thuật toán học, cung cấp nhãn cho các trường hợp cụ thể khi thuật toán yêu cầu chúng. Có nhiều chiến lược khác nhau cho học tích cực, nhưng một trong những chiến lược phổ biến nhất được gọi là **lấy mẫu không chắc chắn (uncertainty sampling)**. Cách hoạt động của nó như sau:

4. Mô hình được huấn luyện trên các trường hợp đã được gắn nhãn thu thập được cho đến nay, và mô hình này được sử dụng để đưa ra dự đoán trên tất cả các trường hợp không được gắn nhãn.

5. Các trường hợp mà mô hình không chắc chắn nhất (tức là nơi xác suất ước tính của nó thấp nhất) được đưa cho chuyên gia để gắn nhãn.
6. Bạn lặp lại quá trình này cho đến khi cải thiện hiệu suất không còn đáng để nỗ lực gắn nhãn nữa. Các chiến lược học tích cực khác bao gồm gắn nhãn các trường hợp sẽ dẫn đến thay đổi mô hình lớn nhất hoặc giảm lỗi xác thực của mô hình lớn nhất, hoặc các trường hợp mà các mô hình khác nhau không đồng ý (ví dụ: một SVM và một rừng ngẫu nhiên).

Trước khi chúng ta chuyển sang các mô hình hỗn hợp Gaussian, hãy xem DBSCAN, một thuật toán phân cụm phổ biến khác minh họa một cách tiếp cận rất khác dựa trên ước tính mật độ cục bộ. Cách tiếp cận này cho phép thuật toán xác định các cụm có hình dạng tùy ý.

DBSCAN

Thuật toán **phân cụm không gian dựa trên mật độ cho các ứng dụng có nhiễu (DBSCAN)** định nghĩa các cụm là các vùng liên tục có mật độ cao. Cách hoạt động của nó như sau:

- Đối với mỗi trường hợp, thuật toán đếm xem có bao nhiêu trường hợp nằm trong khoảng cách nhỏ $epsilon$ (epsilon) từ nó. Vùng này được gọi là **vùng lân cận** $epsilon$ của trường hợp.
- Nếu một trường hợp có ít nhất `min_samples` trường hợp trong vùng lân cận $epsilon$ của nó (bao gồm chính nó), thì nó được coi là một **trường hợp lõi**. Nói cách khác, các trường hợp lõi là những trường hợp nằm trong các vùng dày đặc.
- Tất cả các trường hợp trong vùng lân cận của một trường hợp lõi thuộc cùng một cụm. Vùng lân cận này có thể bao gồm các trường hợp lõi khác; do đó, một chuỗi dài các trường hợp lõi lân cận tạo thành một cụm duy nhất.
- Bất kỳ trường hợp nào không phải là trường hợp lõi và không có trường hợp lõi nào trong vùng lân cận của nó được coi là một **dị thường**.

Thuật toán này hoạt động tốt nếu tất cả các cụm được phân tách tốt bởi các vùng mật độ thấp. Lớp `DBSCAN` trong Scikit-Learn đơn giản như bạn mong đợi. Hãy thử nghiệm nó trên tập dữ liệu moons, được giới thiệu trong Chương 5:

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.05, random_state=42) # Thêm
random_state để có kết quả tái tạo được
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

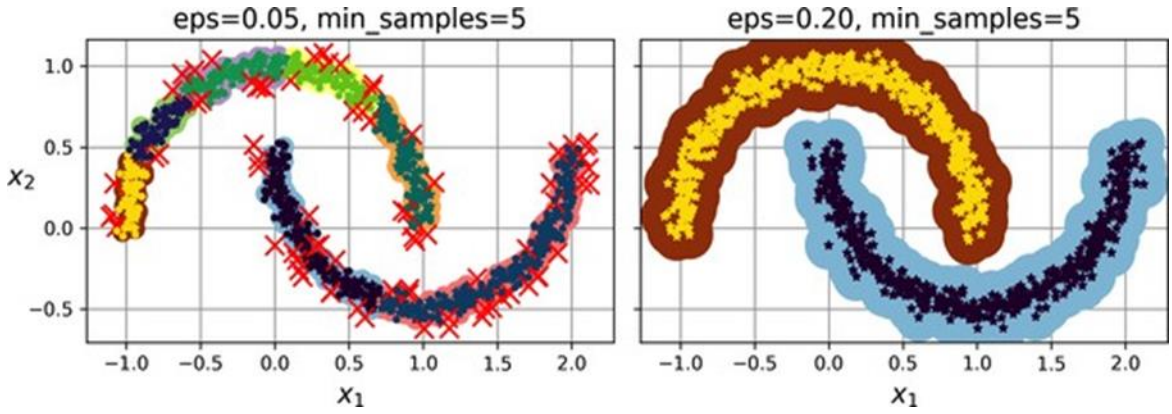
Các nhãn của tất cả các trường hợp hiện có sẵn trong biến thể hiện `labels_`:

```
>>> dbscan.labels_
array([ 0,  2, -1, ..., -1,  0,  0]) # Mảng thực tế có thể khác do tính ngẫu
nhiên.
```

Lưu ý rằng một số trường hợp có chỉ số cụm bằng –1, có nghĩa là chúng được thuật toán coi là dị thường. Các chỉ số của các trường hợp lõi có sẵn trong biến thể hiện `core_sample_indices_`, và chính các trường hợp lõi có sẵn trong biến thể hiện `components_`:

```
>>> dbscan.core_sample_indices_
array([  0,   4,   5,   6,   7,   8,  10,  11, ..., 993, 995, 997, 998, 999])
# Mảng thực tế có thể khác

>>> dbscan.components_
array([[-0.02137124,  0.40618608],
       [-0.84192557,  0.53058695],
       ...,
       [ 0.79419406,  0.60777171]]) # Mảng thực tế có thể khác
```

Phân cụm này được biểu thị trong biểu đồ bên trái của Hình 9-14. Như bạn có thể thấy, nó đã xác định khá nhiều dị thường, cộng với bảy cụm khác nhau. Thật đáng thất vọng! May mắn thay, nếu chúng ta mở rộng vùng lân cận của mỗi trường hợp bằng cách tăng `eps` lên 0.2, chúng ta sẽ nhận được phân cụm bên phải, trông hoàn hảo. Hãy tiếp tục với mô hình này.



Hình 9-14. Phân cụm DBSCAN sử dụng hai bán kính vùng lân cận khác nhau

Đáng ngạc nhiên, lớp `DBSCAN` không có phương thức `predict()`, mặc dù nó có phương thức `fit_predict()`. Nói cách khác, nó không thể dự đoán một trường hợp mới thuộc cụm nào. Quyết định này được đưa ra vì các thuật toán phân loại khác nhau có thể tốt hơn cho các tác vụ khác nhau, vì vậy các tác giả đã quyết định để người dùng chọn cái nào sẽ sử dụng. Hơn nữa, việc triển khai không khó. Ví dụ, hãy huấn luyện một `KNeighborsClassifier`:

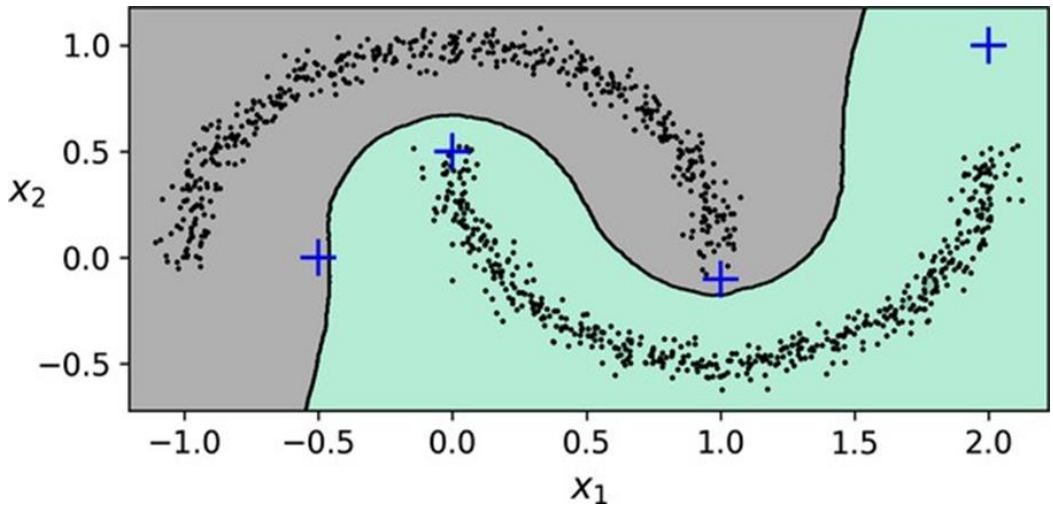
```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=50)
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
```

Bây giờ, với một vài trường hợp mới, chúng ta có thể dự đoán chúng có khả năng thuộc cụm nào nhất và thậm chí ước tính xác suất cho mỗi cụm:

```
>>> X_new = np.array([[-0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])

>>> knn.predict(X_new)
array([1, 0, 1, 0])

>>> knn.predict_proba(X_new)

array([[0.18, 0.82],
       [1.  , 0.  ],
       [0.12, 0.88],
       [1.  , 0.  ]])
```

Lưu ý rằng chúng ta chỉ huấn luyện bộ phân loại trên các trường hợp lõi, nhưng chúng ta cũng có thể chọn huấn luyện nó trên tất cả các trường hợp, hoặc tất cả trừ các dị thường: lựa chọn này phụ thuộc vào tác vụ cuối cùng. Đường biên quyết định được biểu thị trong Hình 9-15 (các dấu thập biểu thị bốn trường hợp trong `X_new`). Lưu ý rằng vì không có dị thường nào trong tập huấn luyện, bộ phân loại luôn chọn một cụm, ngay cả khi cụm đó ở xa. Khá đơn giản để đưa ra một khoảng cách tối đa, trong trường hợp đó hai trường hợp cách xa cả hai cụm sẽ được phân loại là dị thường. Để làm điều này, hãy sử dụng phương thức `kneighbors()` của `KNeighborsClassifier`. Với một tập hợp các trường hợp, nó trả về khoảng cách và chỉ số của k láng giềng gần nhất trong tập huấn luyện (hai ma trận, mỗi ma trận có k cột):

```
>>> y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
>>> y_pred = dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
>>> y_pred[y_dist > 0.2] = -1
>>> y_pred.ravel()
array([-1,  0,  1, -1])
```



Hình 9-15. Đường biên quyết định giữa hai cụm

Tóm lại, DBSCAN là một thuật toán rất đơn giản nhưng mạnh mẽ có khả năng xác định bất kỳ số lượng cụm nào có hình dạng bất kỳ. Nó mạnh mẽ với các ngoại lệ, và nó chỉ có hai

siêu tham số (`eps` và `min_samples`). Tuy nhiên, nếu mật độ thay đổi đáng kể giữa các cụm, hoặc nếu không có vùng mật độ đủ thấp xung quanh một số cụm, DBSCAN có thể gặp khó khăn trong việc nắm bắt tất cả các cụm một cách đúng đắn. Hơn nữa, độ phức tạp tính toán của nó xấp xỉ $O(m^2n)$, vì vậy nó không mở rộng tốt với các tập dữ liệu lớn.

Các thuật toán phân cụm khác

Scikit-Learn triển khai một số thuật toán phân cụm khác mà bạn nên xem xét. Tôi không thể trình bày chi tiết tất cả ở đây, nhưng đây là một cái nhìn tổng quan ngắn gọn:

- **Phân cụm phân cấp (Agglomerative clustering)** Một hệ thống phân cấp các cụm được xây dựng từ dưới lên. Hãy nghĩ về nhiều bong bóng nhỏ nổi trên mặt nước và dần dần gắn vào nhau cho đến khi có một nhóm bong bóng lớn. Tương tự, ở mỗi lần lặp, phân cụm phân cấp kết nối cặp cụm gần nhất (bắt đầu bằng các trường hợp riêng lẻ). Nếu bạn vẽ một cây với một nhánh cho mỗi cặp cụm đã hợp nhất, bạn sẽ nhận được một cây nhị phân các cụm, trong đó các lá là các trường hợp riêng lẻ. Cách tiếp cận này có thể nắm bắt các cụm có hình dạng khác nhau; nó cũng tạo ra một cây cụm linh hoạt và nhiều thông tin thay vì buộc bạn phải chọn một thang đo cụm cụ thể, và nó có thể được sử dụng với bất kỳ khoảng cách cặp nào. Nó có thể mở rộng tốt với số lượng lớn các trường hợp nếu bạn cung cấp một ma trận kết nối, là một ma trận $mtimesm$ thưa thớt cho biết những cặp trường hợp nào là láng giềng (ví dụ: được trả về bởi `sklearn.neighbors.kneighbors_graph()`). Không có ma trận kết nối, thuật toán không mở rộng tốt với các tập dữ liệu lớn.
- **BIRCH** Thuật toán **balanced iterative reducing and clustering using hierarchies (BIRCH)** được thiết kế đặc biệt cho các tập dữ liệu rất lớn, và nó có thể nhanh hơn k-means theo lô, với kết quả tương tự, miễn là số lượng đặc trưng không quá lớn (<20). Trong quá trình huấn luyện, nó xây dựng một cấu trúc cây chứa đủ thông tin để nhanh chóng gán mỗi trường hợp mới vào một cụm, mà không cần phải lưu trữ tất cả các trường hợp trong cây: cách tiếp cận này cho phép nó sử dụng bộ nhớ hạn chế trong khi xử lý các tập dữ liệu khổng lồ.
- **Mean-shift** Thuật toán này bắt đầu bằng cách đặt một vòng tròn có tâm tại mỗi trường hợp; sau đó đối với mỗi vòng tròn, nó tính toán giá trị trung bình của tất cả các trường hợp nằm trong đó, và nó dịch chuyển vòng tròn sao cho nó có tâm tại giá trị trung bình. Tiếp theo, nó lặp lại bước dịch chuyển trung bình này cho đến khi tất cả các vòng tròn ngừng di chuyển (tức là cho đến khi mỗi vòng tròn có tâm tại giá trị trung bình của các trường hợp nó chứa). Mean-shift dịch chuyển các vòng tròn theo hướng mật độ cao hơn, cho đến khi mỗi vòng tròn tìm thấy một cực đại mật độ cục bộ. Cuối cùng, tất cả các trường hợp có vòng tròn đã ổn định ở cùng một vị trí (hoặc đủ gần) được gán vào cùng một cụm. Mean-shift có một số tính năng giống như DBSCAN, chẳng hạn như cách nó có thể tìm thấy bất kỳ số lượng cụm nào có hình dạng bất kỳ, nó có rất ít siêu tham số (chỉ một—bán kính của các vòng tròn, được gọi là bandwidth), và nó dựa vào ước tính mật độ cục bộ. Nhưng không giống như DBSCAN, mean-shift có xu hướng cắt các cụm thành nhiều mảnh khi chúng có sự

biến đổi mật độ bên trong. Thật không may, độ phức tạp tính toán của nó là $O(m^2 n)$, vì vậy nó không phù hợp cho các tập dữ liệu lớn.

- **Lan truyền tương đồng (Affinity propagation)** Trong thuật toán này, các trường hợp liên tục trao đổi thông điệp với nhau cho đến khi mỗi trường hợp đã bầu một trường hợp khác (hoặc chính nó) để đại diện cho nó. Các trường hợp được bầu này được gọi là **exemplar**. Mỗi exemplar và tất cả các trường hợp đã bầu nó tạo thành một cụm. Trong chính trị đời thực, bạn thường muốn bỏ phiếu cho một ứng cử viên có quan điểm tương tự bạn, nhưng bạn cũng muốn họ thắng cử, vì vậy bạn có thể chọn một ứng cử viên mà bạn không hoàn toàn đồng ý, nhưng người đó phổ biến hơn. Bạn thường đánh giá sự phổ biến thông qua các cuộc thăm dò. Lan truyền tương đồng hoạt động theo cách tương tự, và nó có xu hướng chọn các exemplar nằm gần tâm của các cụm, tương tự k-means. Nhưng không giống như k-means, bạn không phải chọn số lượng cụm trước: nó được xác định trong quá trình huấn luyện. Hơn nữa, lan truyền tương đồng có thể xử lý tốt các cụm có kích thước khác nhau. Đáng buồn thay, thuật toán này có độ phức tạp tính toán là $O(m^2)$, vì vậy nó không phù hợp cho các tập dữ liệu lớn.
- **Phân cụm quang phổ (Spectral clustering)** Thuật toán này lấy một ma trận tương đồng giữa các trường hợp và tạo ra một nhúng chiều thấp từ đó (tức là nó giảm chiều của ma trận), sau đó nó sử dụng một thuật toán phân cụm khác trong không gian chiều thấp này (triển khai của Scikit-Learn sử dụng k-means). Phân cụm quang phổ có thể nắm bắt các cấu trúc cụm phức tạp, và nó cũng có thể được sử dụng để cắt đồ thị (ví dụ: để xác định các cụm bạn bè trên mạng xã hội). Nó không mở rộng tốt với số lượng lớn các trường hợp, và nó không hoạt động tốt khi các cụm có kích thước rất khác nhau.

Bây giờ chúng ta hãy đi sâu vào các mô hình hỗn hợp Gaussian, có thể được sử dụng để ước tính mật độ, phân cụm và phát hiện dị thường.

Hỗn hợp Gaussian (Gaussian Mixtures)

Một **mô hình hỗn hợp Gaussian (GMM)** là một mô hình xác suất giả định rằng các trường hợp được tạo ra từ một hỗn hợp của một số phân phối Gaussian với các tham số chưa biết. Tất cả các trường hợp được tạo ra từ một phân phối Gaussian duy nhất tạo thành một cụm thường trông giống một hình elip. Mỗi cụm có thể có hình dạng, kích thước, mật độ và hướng elip khác nhau, giống như trong Hình 9-11. Khi bạn quan sát một trường hợp, bạn biết nó được tạo ra từ một trong các phân phối Gaussian, nhưng bạn không được cho biết là cái nào, và bạn không biết các tham số của các phân phối này là gì.

Có một số biến thể GMM. Trong biến thể đơn giản nhất, được triển khai trong lớp `GaussianMixture`, bạn phải biết trước số lượng k phân phối Gaussian. Tập dữ liệu X được giả định đã được tạo ra thông qua quá trình xác suất sau:

- Đối với mỗi trường hợp, một cụm được chọn ngẫu nhiên từ k cụm. Xác suất chọn cụm thứ j là trọng số $phi^{(j)}$ của cụm. Chỉ số của cụm được chọn cho trường hợp thứ i được ghi là $z^{(i)}$.

- Nếu trường hợp thứ i được gán cho cụm thứ j (tức là $z^{(i)} = j$), thì vị trí $x^{(i)}$ của trường hợp này được lấy mẫu ngẫu nhiên từ phân phối Gaussian với giá trị trung bình $mu^{(j)}$ và ma trận hiệp phương sai $Sigma^{(j)}$. Điều này được ghi là $x^{(i)}simmathcalN\left(mu^{(j)}, Sigma^{(j)}\right)$.

Vậy bạn có thể làm gì với một mô hình như vậy? Chà, với tập dữ liệu X , bạn thường muốn bắt đầu bằng cách ước tính các trọng số phi và tất cả các tham số phân phối $mu^{(1)}$ đến $mu^{(k)}$ và $Sigma^{(1)}$ đến $Sigma^{(k)}$. Lớp `GaussianMixture` của Scikit-Learn làm cho việc này trở nên siêu dễ dàng:

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10, random_state=42) # Thêm
random_state để có kết quả tái tạo được
gm.fit(X)
```

Hãy xem xét các tham số mà thuật toán đã ước tính:

```
>>> gm.weights_
array([0.39025715, 0.40007391, 0.20966893])

>>> gm.means_
array([[ 0.05131611,  0.07521837],
       [-1.40763156,  1.42708225],
       [ 3.39893794,  1.05928897]])

>>> gm.covariances_
array([[[0.68799922, 0.79606357],
        [0.79606357, 1.21236106]],

       [[0.63479409, 0.72970799],
        [0.72970799, 1.1610351 ]],

       [[1.14833585, -0.03256179],
        [-0.03256179, 0.95490931]]])
```

Tuyệt vời, nó hoạt động tốt! Thật vậy, hai trong số ba cụm được tạo ra với 500 trường hợp mỗi cụm, trong khi cụm thứ ba chỉ chứa 250 trường hợp. Vì vậy, trọng số cụm thực sự lần lượt là 0.4, 0.4 và 0.2, và đó là xấp xỉ những gì thuật toán đã tìm thấy. Tương tự, các giá trị trung bình và ma trận hiệp phương sai thực sự khá gần với những gì thuật toán tìm thấy. Nhưng bằng cách nào? Lớp này dựa vào thuật toán **mong đợi-tối đa hóa (EM)**, có nhiều điểm tương đồng với thuật toán k-means: nó cũng khởi tạo các tham số cụm một cách ngẫu nhiên, sau đó lặp lại hai bước cho đến khi hội tụ, đầu tiên gán các trường hợp vào các cụm (đây được gọi là bước mong đợi) và sau đó cập nhật các cụm (đây được gọi là bước tối đa hóa). Nghe quen thuộc phải không? Trong bối cảnh phân cụm, bạn có thể nghĩ EM như một sự tổng quát hóa của k-means mà không chỉ tìm thấy các tâm cụm ($mu^{(1)}$ đến $mu^{(k)}$), mà còn cả kích thước, hình dạng và hướng của chúng ($Sigma^{(1)}$ đến $Sigma^{(k)}$), cũng như

trọng số tương đối của chúng ($phi^{(1)}$ đến $phi^{(k)}$). Tuy nhiên, không giống như k-means, EM sử dụng gán cụm mềm, không phải gán cứng. Đối với mỗi trường hợp, trong bước mong đợi, thuật toán ước tính xác suất nó thuộc về mỗi cụm (dựa trên các tham số cụm hiện tại). Sau đó, trong bước tối đa hóa, mỗi cụm được cập nhật bằng cách sử dụng tất cả các trường hợp trong tập dữ liệu, với mỗi trường hợp được trọng số bằng xác suất ước tính rằng nó thuộc về cụm đó. Các xác suất này được gọi là **trách nhiệm (responsibilities)** của các cụm đối với các trường hợp. Trong bước tối đa hóa, việc cập nhật mỗi cụm sẽ chủ yếu bị ảnh hưởng bởi các trường hợp mà nó chịu trách nhiệm nhiều nhất.

Bạn có thể kiểm tra xem thuật toán có hội tụ hay không và mất bao nhiêu lần lặp:

```
>>> gm.converged_
True

>>> gm.n_iter_
4
```

Bây giờ bạn đã có một ước tính về vị trí, kích thước, hình dạng, hướng và trọng số tương đối của mỗi cụm, mô hình có thể dễ dàng gán mỗi trường hợp vào cụm có khả năng nhất (phân cụm cứng) hoặc ước tính xác suất nó thuộc về một cụm cụ thể (phân cụm mềm). Chỉ cần sử dụng phương thức `predict()` cho phân cụm cứng, hoặc phương thức `predict_proba()` cho phân cụm mềm:

```
>>> gm.predict(X)
array([0, 0, 1, ..., 2, 2, 2])

>>> gm.predict_proba(X).round(3)
array([[0.977, 0.   , 0.023],
       [0.983, 0.001, 0.016],
       [0.   , 1.   , 0.   ],
       ...,
       [0.   , 0.   , 1.   ],
       [0.   , 0.   , 1.   ],
       [0.   , 0.   , 1.   ]])
```

Một mô hình hỗn hợp Gaussian là một **mô hình sinh thành**, có nghĩa là bạn có thể lấy mẫu các trường hợp mới từ nó (lưu ý rằng chúng được sắp xếp theo chỉ số cụm):

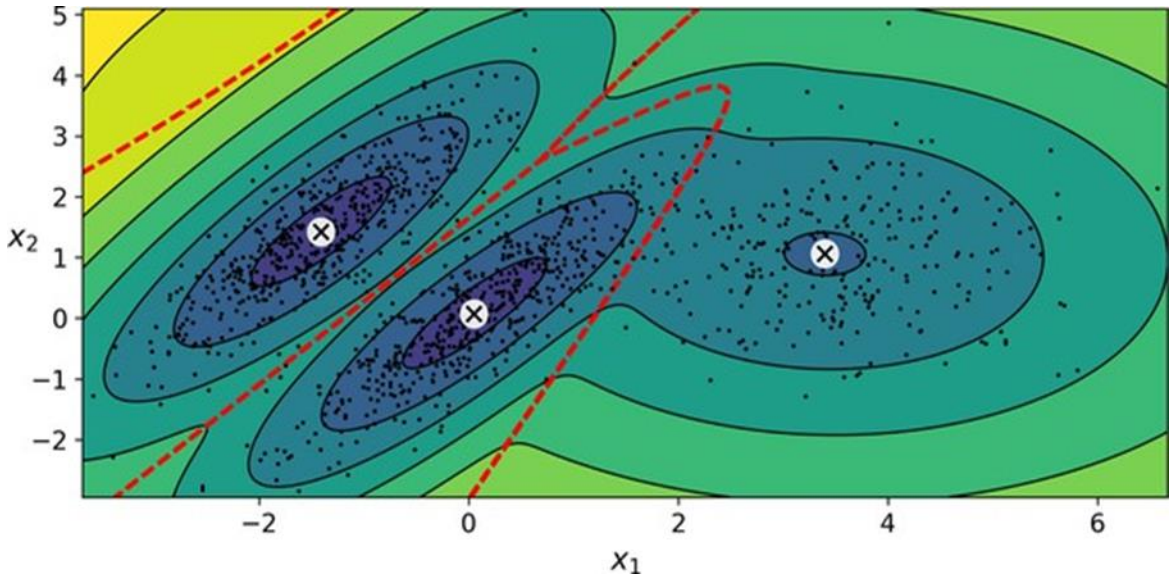
```
>>> X_new, y_new = gm.sample(6)

>>> X_new
array([[-0.86944074, -0.32767626],
       [ 0.29836051,  0.28297011],
       [-2.8014927 , -0.09047309],
       [ 3.98203732,  1.49951491],
       [ 3.81677148,  0.53095244],
       [ 2.84104923, -0.73858639]])
```

```
>>> y_new
array([0, 0, 1, 2, 2, 2])
```

Cũng có thể ước tính mật độ của mô hình tại bất kỳ vị trí nào. Điều này được thực hiện bằng phương thức `score_samples()`: đối với mỗi trường hợp được cung cấp, phương thức này ước tính log của hàm mật độ xác suất (PDF) tại vị trí đó. Điểm càng lớn, mật độ càng cao:

```
>>> gm.score_samples(X).round(2)
array([-2.61, -3.57, -3.33, ..., -3.51, -4.4 , -3.81])
```

Nếu bạn tính lũy thừa của các điểm này, bạn sẽ nhận được giá trị của PDF tại vị trí của các trường hợp đã cho. Đây không phải là xác suất, mà là mật độ xác suất: chúng có thể nhận bất kỳ giá trị dương nào, không chỉ giá trị từ 0 đến 1. Để ước tính xác suất một trường hợp sẽ nằm trong một vùng cụ thể, bạn sẽ phải tích phân PDF trên vùng đó (nếu bạn làm như vậy trên toàn bộ không gian các vị trí trường hợp có thể có, kết quả sẽ là 1). Hình 9-16 cho thấy các giá trị trung bình của cụm, các đường biên quyết định (đường nét đứt), và các đường đồng mật độ của mô hình này.

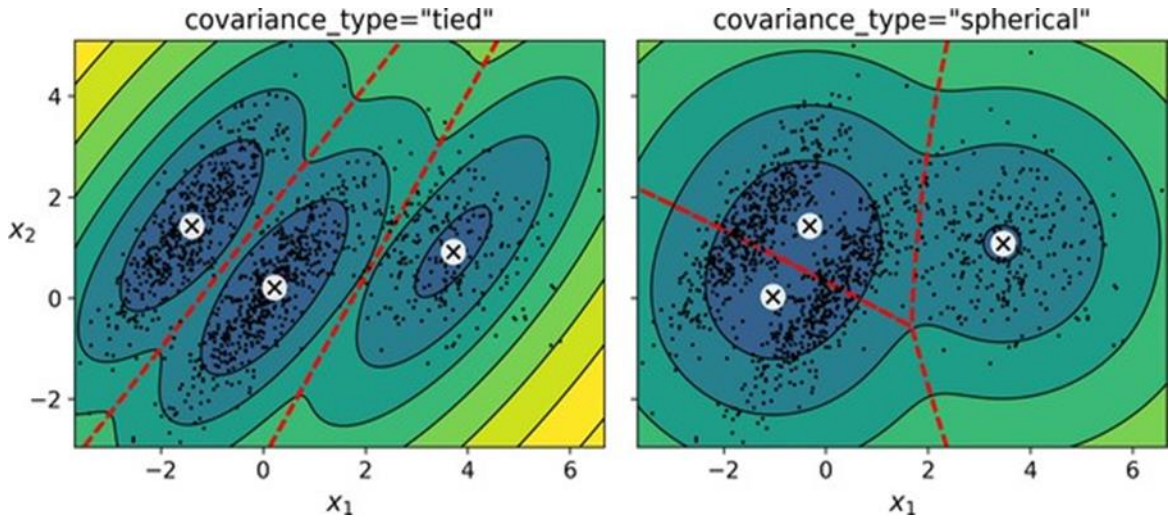


Hình 9-16. Các giá trị trung bình của cụm, đường biên quyết định và đường đồng mật độ của một mô hình hỗn hợp Gaussian đã được huấn luyện

Tuyệt vời! Thuật toán rõ ràng đã tìm thấy một giải pháp xuất sắc. Tất nhiên, chúng ta đã làm cho nhiệm vụ của nó dễ dàng bằng cách tạo dữ liệu bằng cách sử dụng một tập hợp các phân phối Gaussian 2D (thật không may, dữ liệu thực tế không phải lúc nào cũng Gaussian và có chiều thấp như vậy). Chúng ta cũng đã cung cấp cho thuật toán số lượng cụm chính xác. Khi có nhiều chiều, hoặc nhiều cụm, hoặc ít trường hợp, EM có thể gặp khó khăn trong việc hội tụ đến giải pháp tối ưu. Bạn có thể cần giảm độ khó của tác vụ bằng cách giới hạn số lượng tham số mà thuật toán phải học. Một cách để làm điều này là giới hạn phạm vi hình dạng và hướng mà các cụm có thể có. Điều này có thể đạt được bằng cách áp đặt các ràng buộc lên các ma trận hiệp phương sai. Để làm điều này, đặt siêu tham số `covariance_type` thành một trong các giá trị sau:

- “spherical” Tất cả các cụm phải là hình cầu, nhưng chúng có thể có đường kính khác nhau (tức là phương sai khác nhau).
- “diag” Các cụm có thể có bất kỳ hình dạng elip nào với bất kỳ kích thước nào, nhưng các trục của hình elip phải song song với các trục tọa độ (tức là các ma trận hiệp phương sai phải là đường chéo).
- “tied” Tất cả các cụm phải có cùng hình dạng, kích thước và hướng elip (tức là tất cả các cụm chia sẻ cùng một ma trận hiệp phương sai).

Theo mặc định, `covariance_type` bằng “full”, có nghĩa là mỗi cụm có thể có bất kỳ hình dạng, kích thước và hướng nào (nó có ma trận hiệp phương sai không bị ràng buộc riêng). Hình 9-17 vẽ các giải pháp được tìm thấy bởi thuật toán EM khi `covariance_type` được đặt thành “tied” hoặc “spherical”.



Hình 9-17. Hỗn hợp Gaussian cho các cụm liên kết (trái) và các cụm hình cầu (phải)

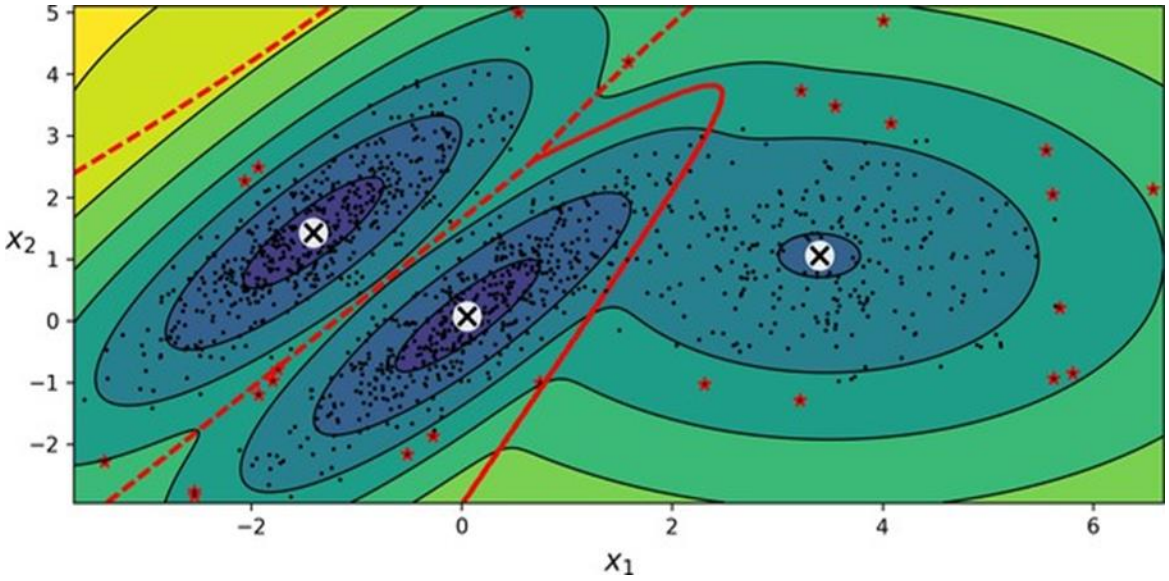
Các mô hình hỗn hợp Gaussian cũng có thể được sử dụng để phát hiện dị thường. Chúng ta sẽ xem cách thực hiện trong phần tiếp theo.

Sử dụng Hỗn hợp Gaussian để phát hiện dị thường

Sử dụng mô hình hỗn hợp Gaussian để phát hiện dị thường khá đơn giản: bất kỳ trường hợp nào nằm trong vùng mật độ thấp đều có thể được coi là một dị thường. Bạn phải định nghĩa ngưỡng mật độ bạn muốn sử dụng. Ví dụ, trong một công ty sản xuất cố gắng phát hiện sản phẩm bị lỗi, tỷ lệ sản phẩm bị lỗi thường được biết rõ. Giả sử nó bằng 2%. Sau đó, bạn đặt ngưỡng mật độ là giá trị dẫn đến 2% số trường hợp nằm trong các khu vực dưới ngưỡng mật độ đó. Nếu bạn nhận thấy có quá nhiều sai số dương (tức là sản phẩm hoàn toàn tốt bị gắn cờ là bị lỗi), bạn có thể hạ thấp ngưỡng. Ngược lại, nếu bạn có quá nhiều sai số âm (tức là sản phẩm bị lỗi mà hệ thống không gắn cờ là bị lỗi), bạn có thể tăng ngưỡng. Đây là sự đánh đổi độ chính xác/độ thu hồi thông thường (xem Chương 3). Đây là cách bạn xác định các ngoại lệ bằng cách sử dụng mật độ thấp nhất ở phân vị thứ tư làm ngưỡng (tức là khoảng 4% số trường hợp sẽ bị gắn cờ là dị thường):

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 2)
anomalies = X[densities < density_threshold]
```

Hình 9-18 biểu thị các dị thường này dưới dạng các ngôi sao. Một tác vụ liên quan chặt chẽ là **phát hiện điểm mới (novelty detection)**: nó khác với phát hiện dị thường ở chỗ thuật toán được giả định được huấn luyện trên một tập dữ liệu "sạch", không bị nhiễm ngoại lệ, trong khi phát hiện dị thường không đưa ra giả định này. Thật vậy, phát hiện ngoại lệ thường được sử dụng để làm sạch tập dữ liệu.



Hình 9-18. Phát hiện dị thường sử dụng mô hình hỗn hợp Gaussian

Giống như k-means, thuật toán `GaussianMixture` yêu cầu bạn chỉ định số lượng cụm. Vậy làm thế nào bạn có thể tìm thấy số đó?

Chọn số lượng cụm

Với k-means, bạn có thể sử dụng quán tính hoặc điểm silhouette để chọn số lượng cụm phù hợp. Nhưng với các hỗn hợp Gaussian, không thể sử dụng các chỉ số này vì chúng không đáng tin cậy khi các cụm không có hình cầu hoặc có kích thước khác nhau. Thay vào đó, bạn có thể cố gắng tìm mô hình tối thiểu hóa một tiêu chí thông tin lý thuyết, chẳng hạn như **tiêu chí thông tin Bayesian (BIC)** hoặc **tiêu chí thông tin Akaike (AIC)**, được định nghĩa trong Phương trình 9-1.

Phương trình 9-1. Tiêu chí thông tin Bayesian (BIC) và tiêu chí thông tin Akaike (AIC)

$$BIC = \log(m)p - 2\log\left(\hat{\mathcal{L}}\right)$$

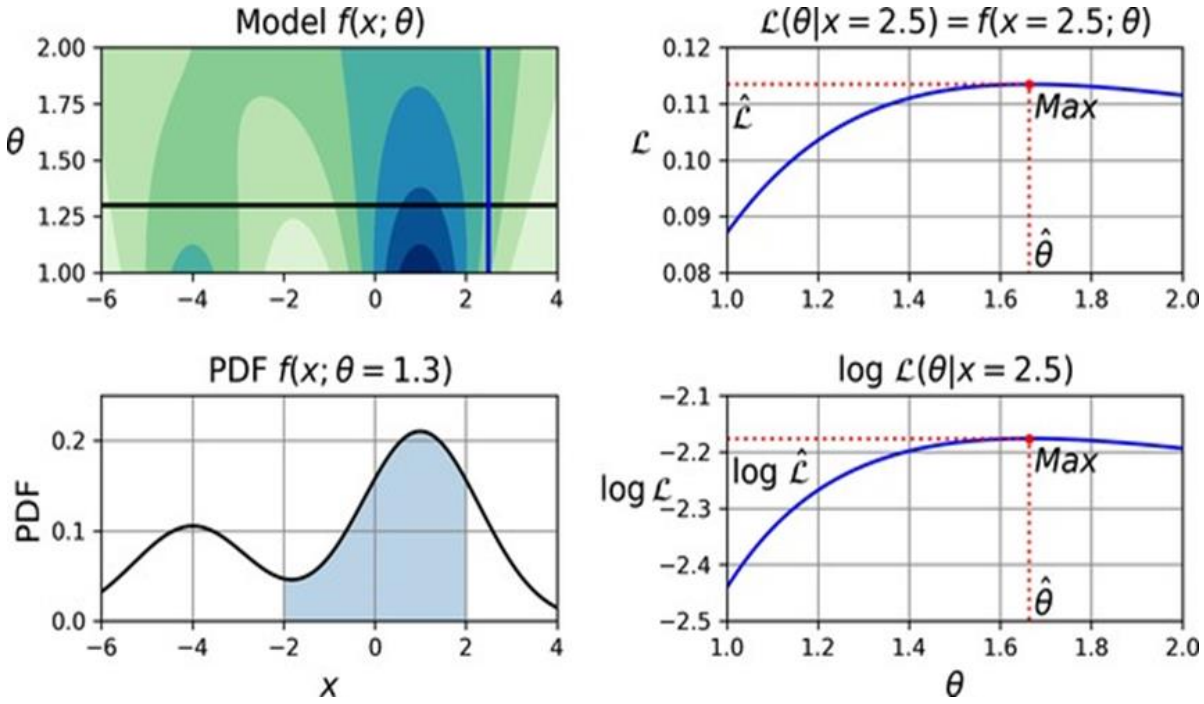
$$AIC = 2p - 2\log\left(\hat{\mathcal{L}}\right)$$

Trong các phương trình này:

- m là số lượng trường hợp, như mọi khi.
- p là số lượng tham số được học bởi mô hình.
- $\widehat{\mathscr{L}}$ là giá trị tối đa của hàm khả năng xảy ra (likelihood function) của mô hình.

Cả BIC và AIC đều phạt các mô hình có nhiều tham số cần học (ví dụ: nhiều cụm hơn) và thưởng các mô hình khớp dữ liệu tốt. Chúng thường chọn cùng một mô hình. Khi chúng khác nhau, mô hình được chọn bởi BIC có xu hướng đơn giản hơn (ít tham số hơn) so với mô hình được chọn bởi AIC, nhưng có xu hướng không khớp dữ liệu tốt bằng (điều này đặc biệt đúng đối với các tập dữ liệu lớn hơn).

HÀM KHẢ NĂNG XẢY RA (LIKELIHOOD FUNCTION) Các thuật ngữ “xác suất” (probability) và “khả năng xảy ra” (likelihood) thường được sử dụng thay thế cho nhau trong ngôn ngữ hàng ngày, nhưng chúng có ý nghĩa rất khác nhau trong thống kê. Cho một mô hình thống kê với một số tham số $theta$, từ “xác suất” được sử dụng để mô tả một kết quả x trong tương lai có thể xảy ra đến mức nào (biết giá trị tham số $theta$), trong khi từ “khả năng xảy ra” được sử dụng để mô tả một tập hợp giá trị tham số $theta$ cụ thể có thể xảy ra đến mức nào, sau khi kết quả x đã được biết. Xem xét một mô hình hỗn hợp 1D của hai phân phối Gaussian có tâm tại –4 và +1. Để đơn giản, mô hình đồ chơi này có một tham số duy nhất $theta$ kiểm soát độ lệch chuẩn của cả hai phân phối. Biểu đồ đường đồng mức trên cùng bên trái trong Hình 9-19 hiển thị toàn bộ mô hình $f(x; theta)$ như một hàm của cả x và $theta$. Để ước tính phân phối xác suất của một kết quả x trong tương lai, bạn cần đặt tham số mô hình $theta$. Ví dụ, nếu bạn đặt $theta$ thành 1.3 (đường ngang), bạn sẽ nhận được hàm mật độ xác suất $f(x; theta = 1.3)$ được hiển thị trong biểu đồ dưới cùng bên trái. Giả sử bạn muốn ước tính xác suất x sẽ nằm giữa –2 và +2. Bạn phải tính tích phân của PDF trên phạm vi này (tức là diện tích của vùng được tô bóng). Nhưng điều gì sẽ xảy ra nếu bạn không biết $theta$, và thay vào đó bạn đã quan sát một trường hợp duy nhất $x = 2.5$ (đường dọc trong biểu đồ trên cùng bên trái)? Trong trường hợp này, bạn nhận được hàm khả năng xảy ra $mathcalL(theta \mid x = 2.5) = f(x = 2.5; theta)$, được biểu diễn trong biểu đồ trên cùng bên phải.



Hình 9-19. Hàm tham số của mô hình (trên cùng bên trái), và một số hàm dẫn xuất: PDF (dưới cùng bên trái), hàm khả năng xảy ra (trên cùng bên phải), và hàm log khả năng xảy ra (dưới cùng bên phải)

Tóm lại, PDF là một hàm của x (với $theta$ cố định), trong khi hàm khả năng xảy ra là một hàm của $theta$ (với x cố định). Điều quan trọng là phải hiểu rằng hàm khả năng xảy ra **không phải** là một phân phối xác suất: nếu bạn tích phân một phân phối xác suất trên tất cả các giá trị có thể có của x , bạn luôn nhận được 1, nhưng nếu bạn tích phân hàm khả năng xảy ra trên tất cả các giá trị có thể có của $theta$, kết quả có thể là bất kỳ giá trị dương nào. Cho một tập dữ liệu X , một tác vụ phổ biến là cố gắng ước tính các giá trị có khả năng nhất cho các tham số mô hình. Để làm điều này, bạn phải tìm các giá trị tối đa hóa hàm khả năng xảy ra, cho X . Trong ví dụ này, nếu bạn đã quan sát một trường hợp duy nhất $x = 2.5$, ước tính khả năng xảy ra tối đa (MLE) của $theta$ là $hattheta = 1.5$. Nếu tồn tại một phân phối xác suất tiên nghiệm g trên $theta$, có thể tính đến nó bằng cách tối đa hóa $mathcalL(theta \mid x)g(theta)$ thay vì chỉ tối đa hóa $\hat{\mathcal{L}}=\mathcal{L}\left(\hat{\boldsymbol{\theta}},\mathbf{X}\right)$. Đây được gọi là ước tính hậu nghiệm tối đa (MAP). Vì MAP ràng buộc các giá trị tham số, bạn có thể coi nó là một phiên bản được chính quy hóa của MLE.

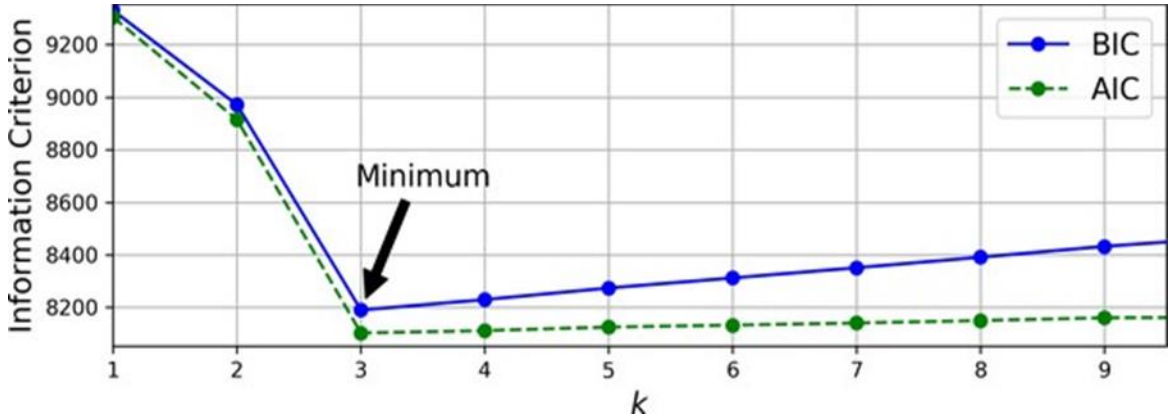
Lưu ý rằng việc tối đa hóa hàm khả năng xảy ra tương đương với việc tối đa hóa logarit của nó (được biểu diễn trong biểu đồ dưới cùng bên phải trong Hình 9-19). Thật vậy, logarit là một hàm tăng nghiêm ngặt, vì vậy nếu $theta$ tối đa hóa log khả năng xảy ra, nó cũng tối đa hóa khả năng xảy ra. Hóa ra thường dễ dàng hơn để tối đa hóa log khả năng xảy ra. Ví dụ, nếu bạn quan sát một số trường hợp độc lập $x^{(1)}$ đến $x^{(m)}$, bạn sẽ cần tìm giá trị của $theta$ tối đa hóa tích của các hàm khả năng xảy ra riêng lẻ. Nhưng tương đương, và đơn giản hơn nhiều, là tối đa hóa tổng (không phải tích) của các hàm log khả năng xảy ra, nhờ vào phép thuật của logarit chuyển đổi các tích thành tổng: $log(ab) = log(a) + log(b)$. Một khi bạn

đã ước tính $hattheta$, giá trị của $theta$ tối đa hóa hàm khả năng xảy ra, thì bạn đã sẵn sàng tính toán $hatL = mathcalL(hattheta, X)$, là giá trị được sử dụng để tính AIC và BIC; bạn có thể coi nó là một thước đo mức độ phù hợp của mô hình với dữ liệu.

Để tính BIC và AIC, hãy gọi các phương thức `bic()` và `aic()`:

```
>>> gm.bic(X)
8189.747000497186

>>> gm.aic(X)
8102.521720382148
```

Hình 9-20 cho thấy BIC cho các số lượng cụm k khác nhau. Như bạn có thể thấy, cả BIC và AIC đều thấp nhất khi $k = 3$, vì vậy đây rất có thể là lựa chọn tốt nhất.



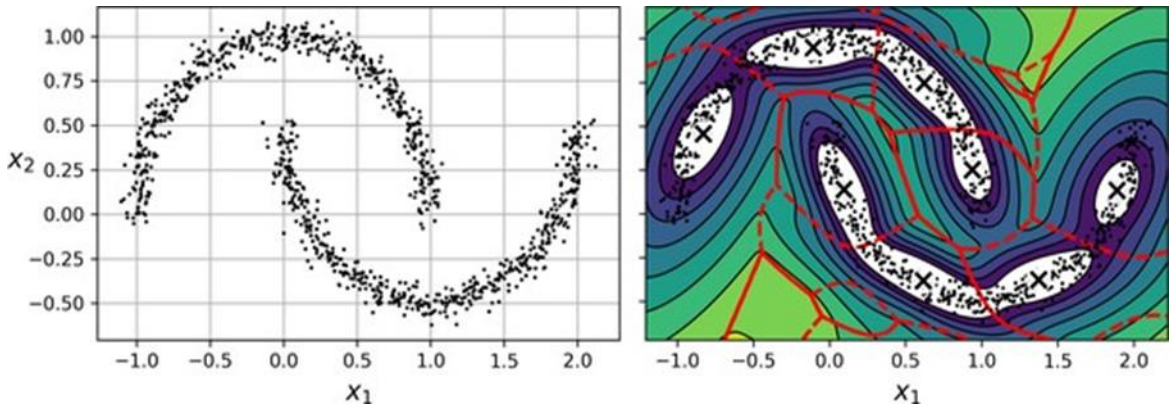
Hình 9-20. AIC và BIC cho các số lượng cụm k khác nhau

Mô hình hỗn hợp Gaussian Bayes (Bayesian Gaussian Mixture Models)

Thay vì tìm kiếm thủ công số lượng cụm tối ưu, bạn có thể sử dụng lớp `BayesianGaussianMixture`, lớp này có khả năng gán trọng số bằng (hoặc gần bằng) 0 cho các cụm không cần thiết. Đặt số lượng cụm `n_components` thành một giá trị mà bạn có lý do chính đáng để tin rằng lớn hơn số lượng cụm tối ưu (điều này giả định một số kiến thức tối thiểu về vấn đề đang xem xét), và thuật toán sẽ tự động loại bỏ các cụm không cần thiết. Ví dụ, hãy đặt số lượng cụm là 10 và xem điều gì xảy ra:

```
>>> from sklearn.mixture import BayesianGaussianMixture

>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10,
random_state=42) # Thêm random_state để có kết quả tái tạo được

>>> bgm.fit(X)

>>> bgm.weights_.round(2)
array([0.4 , 0.21, 0.4 , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ])
```

Hoàn hảo: thuật toán tự động phát hiện rằng chỉ cần ba cụm, và các cụm kết quả gần như giống hệt các cụm trong Hình 9-16. Một lưu ý cuối cùng về các mô hình hỗn hợp Gaussian: mặc dù chúng hoạt động rất tốt trên các cụm có hình elip, nhưng chúng không hoạt động tốt với các cụm có hình dạng rất khác nhau. Ví dụ, hãy xem điều gì xảy ra nếu chúng ta sử dụng mô hình hỗn hợp Gaussian Bayes để phân cụm tập dữ liệu moons (xem Hình 9-21).



Hình 9-21. Khớp hỗn hợp Gaussian với các cụm không hình elip

Ôi! Thuật toán đã tuyệt vọng tìm kiếm các hình elip, vì vậy nó đã tìm thấy tám cụm khác nhau thay vì hai. Việc ước tính mật độ không quá tệ, vì vậy mô hình này có lẽ có thể được sử dụng để phát hiện dị thường, nhưng nó đã thất bại trong việc xác định hai "mặt trăng". Để kết thúc chương này, hãy xem nhanh một vài thuật toán có khả năng xử lý các cụm có hình dạng tùy ý.

Các thuật toán khác để phát hiện dị thường và điểm mới

Scikit-Learn triển khai các thuật toán khác dành riêng cho phát hiện dị thường hoặc phát hiện điểm mới:

- **Fast-MCD (minimum covariance determinant)** Được triển khai bởi lớp `EllipticEnvelope`, thuật toán này hữu ích cho việc phát hiện ngoại lệ, đặc biệt để làm sạch tập dữ liệu. Nó giả định rằng các trường hợp bình thường (nội lệ) được tạo ra từ một phân phối Gaussian duy nhất (không phải là hỗn hợp). Nó cũng giả định rằng tập dữ liệu bị nhiễm các ngoại lệ không được tạo ra từ phân phối Gaussian này. Khi thuật toán ước tính các tham số của phân phối Gaussian (tức là hình dạng của bao elip xung quanh các nội lệ), nó cẩn thận bỏ qua các trường hợp có khả năng cao là ngoại lệ. Kỹ thuật này đưa ra ước tính tốt hơn về bao elip và do đó làm cho thuật toán tốt hơn trong việc xác định các ngoại lệ.
- **Rừng cách ly (Isolation forest)** Đây là một thuật toán hiệu quả để phát hiện ngoại lệ, đặc biệt trong các tập dữ liệu có chiều cao. Thuật toán xây dựng một rừng ngẫu nhiên trong đó mỗi cây quyết định được phát triển ngẫu nhiên: tại mỗi nút, nó chọn ngẫu nhiên một đặc trưng, sau đó nó chọn một giá trị ngưỡng ngẫu nhiên (giữa giá trị min và max) để chia tập dữ liệu thành hai. Tập dữ liệu dần dần bị chia thành các mảnh theo cách này, cho đến khi tất cả các trường hợp bị cô lập khỏi các trường

hợp khác. Các dị thường thường ở xa các trường hợp khác, vì vậy trung bình (trên tất cả các cây quyết định) chúng có xu hướng bị cô lập trong ít bước hơn so với các trường hợp bình thường.

- **Hệ số ngoại lệ cục bộ (Local outlier factor - LOF)** Thuật toán này cũng tốt cho việc phát hiện ngoại lệ. Nó so sánh mật độ của các trường hợp xung quanh một trường hợp đã cho với mật độ xung quanh các láng giềng của nó. Một dị thường thường bị cô lập hơn các k láng giềng gần nhất của nó.
- **One-class SVM** Thuật toán này phù hợp hơn để phát hiện điểm mới. Nhớ lại rằng một bộ phân loại SVM kernel hóa tách hai lớp bằng cách đầu tiên (ngầm định) ánh xạ tất cả các trường hợp vào một không gian có chiều cao, sau đó tách hai lớp bằng cách sử dụng bộ phân loại SVM tuyến tính trong không gian có chiều cao này (xem Chương 5). Vì chúng ta chỉ có một lớp trường hợp, thuật toán one-class SVM thay vào đó cố gắng tách các trường hợp trong không gian có chiều cao khỏi gốc. Trong không gian gốc, điều này sẽ tương ứng với việc tìm một vùng nhỏ bao trùm tất cả các trường hợp. Nếu một trường hợp mới không nằm trong vùng này, nó là một dị thường. Có một vài siêu tham số để điều chỉnh: các siêu tham số thông thường cho SVM kernel hóa, cộng với một siêu tham số biên tương ứng với xác suất một trường hợp mới bị coi là điểm mới một cách nhầm lẫn khi nó thực tế là bình thường. Nó hoạt động rất tốt, đặc biệt với các tập dữ liệu có chiều cao, nhưng giống như tất cả các SVM, nó không mở rộng được cho các tập dữ liệu lớn.
- **PCA và các kỹ thuật giảm chiều khác có phương thức `inverse_transform()`** Nếu bạn so sánh sai số tái tạo của một trường hợp bình thường với sai số tái tạo của một dị thường, cái sau thường sẽ lớn hơn nhiều. Đây là một cách tiếp cận phát hiện dị thường đơn giản và thường khá hiệu quả (xem các bài tập của chương này để biết ví dụ).

Bài tập

1. Bạn sẽ định nghĩa phân cụm như thế nào? Bạn có thể kể tên một vài thuật toán phân cụm không?
2. Một số ứng dụng chính của thuật toán phân cụm là gì?
3. Mô tả hai kỹ thuật để chọn số lượng cụm phù hợp khi sử dụng k-means.
4. Truyền nhãn là gì? Tại sao bạn lại triển khai nó, và bằng cách nào?
5. Bạn có thể kể tên hai thuật toán phân cụm có thể mở rộng cho các tập dữ liệu lớn không? Và hai thuật toán tìm kiếm các vùng mật độ cao?
6. Bạn có thể nghĩ ra một trường hợp sử dụng nào mà học tích cực sẽ hữu ích không? Bạn sẽ triển khai nó như thế nào?
7. Sự khác biệt giữa phát hiện dị thường và phát hiện điểm mới là gì?
8. Hỗn hợp Gaussian là gì? Bạn có thể sử dụng nó cho những tác vụ nào?
9. Bạn có thể kể tên hai kỹ thuật để tìm số lượng cụm phù hợp khi sử dụng mô hình hỗn hợp Gaussian không?
10. Tập dữ liệu khuôn mặt Olivetti cổ điển chứa 400 hình ảnh thang độ xám 64 × 64 pixel của khuôn mặt. Mỗi hình ảnh được làm phẳng thành một vector 1D kích thước

4.096. Bốn mươi người khác nhau đã được chụp ảnh (mỗi người 10 lần), và nhiệm vụ thông thường là huấn luyện một mô hình có thể dự đoán người nào được biểu diễn trong mỗi bức ảnh. Tải tập dữ liệu bằng cách sử dụng hàm `sklearn.datasets.fetch_olivetti_faces()`, sau đó chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra (lưu ý rằng tập dữ liệu đã được chuẩn hóa giữa 0 và 1). Vì tập dữ liệu khá nhỏ, bạn có thể sẽ muốn sử dụng lấy mẫu phân tầng để đảm bảo rằng có cùng số lượng hình ảnh cho mỗi người trong mỗi tập hợp. Tiếp theo, phân cụm các hình ảnh bằng k-means, và đảm bảo rằng bạn có một số lượng cụm tốt (sử dụng một trong các kỹ thuật đã thảo luận trong chương này). Trực quan hóa các cụm: bạn có thấy các khuôn mặt tương tự trong mỗi cụm không?

11. Tiếp tục với tập dữ liệu khuôn mặt Olivetti, huấn luyện một bộ phân loại để dự đoán người nào được biểu diễn trong mỗi bức ảnh, và đánh giá nó trên tập xác thực. Tiếp theo, sử dụng k-means như một công cụ giảm chiều, và huấn luyện một bộ phân loại trên tập đã giảm chiều. Tìm số lượng cụm cho phép bộ phân loại đạt hiệu suất tốt nhất: bạn có thể đạt được hiệu suất nào? Điều gì sẽ xảy ra nếu bạn nối các đặc trưng từ tập đã giảm chiều vào các đặc trưng gốc (một lần nữa, tìm kiếm số lượng cụm tốt nhất)?
12. Huấn luyện một mô hình hỗn hợp Gaussian trên tập dữ liệu khuôn mặt Olivetti. Để tăng tốc thuật toán, bạn có lẽ nên giảm chiều của tập dữ liệu (ví dụ: sử dụng PCA, bảo toàn 99% phương sai). Sử dụng mô hình để tạo ra một số khuôn mặt mới (sử dụng phương thức `sample()`), và trực quan hóa chúng (nếu bạn sử dụng PCA, bạn sẽ cần sử dụng phương thức `inverse_transform()` của nó). Thử sửa đổi một số hình ảnh (ví dụ: xoay, lật, làm tối) và xem liệu mô hình có thể phát hiện các dị thường không (tức là so sánh đầu ra của phương thức `score_samples()` cho hình ảnh bình thường và cho các dị thường).
13. Một số kỹ thuật giảm chiều cũng có thể được sử dụng để phát hiện dị thường. Ví dụ, lấy tập dữ liệu khuôn mặt Olivetti và giảm chiều nó bằng PCA, bảo toàn 99% phương sai. Sau đó tính sai số tái tạo cho mỗi hình ảnh. Tiếp theo, lấy một số hình ảnh đã sửa đổi mà bạn đã xây dựng trong bài tập trước và xem sai số tái tạo của chúng: lưu ý rằng nó lớn hơn nhiều. Nếu bạn vẽ một hình ảnh đã được tái tạo, bạn sẽ thấy lý do: nó cố gắng tái tạo một khuôn mặt bình thường. Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

PHẦN II. MẠNG NƠ-RON VÀ HỌC SÂU

CHƯƠNG 10. GIỚI THIỆU VỀ MẠNG NƠ-RON NHÂN TẠO VỚI KERAS

Loài chim đã truyền cảm hứng cho chúng ta bay, cây ngưu bàng truyền cảm hứng cho khóa dán Velcro, và thiên nhiên đã truyền cảm hứng cho vô số phát minh khác. Do đó, việc tìm kiếm nguồn cảm hứng từ kiến trúc của bộ não để xây dựng một cỗ máy thông minh dường như là điều hợp lý. Đây là logic đã châm ngòi cho các mạng nơ-ron nhân tạo (ANNs), các mô hình học máy được lấy cảm hứng từ mạng lưới các nơ-ron sinh học được tìm thấy trong bộ não của chúng ta. Tuy nhiên, mặc dù máy bay được lấy cảm hứng từ loài chim, chúng không cần phải vỗ cánh để bay. Tương tự như vậy, các ANN đã dần trở nên khá khác biệt so với các "họ hàng" sinh học của chúng. Một số nhà nghiên cứu thậm chí còn lập luận rằng chúng ta nên bỏ hẳn phép loại suy sinh học (ví dụ, bằng cách nói "đơn vị" thay vì "nơ-ron"), kẻo chúng ta hạn chế sự sáng tạo của mình vào các hệ thống khả thi về mặt sinh học.

ANNs là cốt lõi của học sâu. Chúng linh hoạt, mạnh mẽ và có khả năng mở rộng, làm cho chúng trở nên lý tưởng để giải quyết các tác vụ học máy lớn và phức tạp cao như phân loại hàng tỷ hình ảnh (ví dụ: Google Images), cung cấp dịch vụ nhận dạng giọng nói (ví dụ: Apple Siri), đề xuất các video tốt nhất để hàng trăm triệu người dùng xem mỗi ngày (ví dụ: YouTube), hoặc học cách đánh bại nhà vô địch thế giới trong trò chơi cờ vây (AlphaGo của DeepMind).

Phần đầu tiên của chương này giới thiệu các mạng nơ-ron nhân tạo, bắt đầu với một cái nhìn nhanh về các kiến trúc ANN đầu tiên và dẫn đến các mạng perceptron đa lớp, vốn được sử dụng rộng rãi ngày nay (các kiến trúc khác sẽ được khám phá trong các chương tiếp theo). Trong phần thứ hai, chúng ta sẽ xem xét cách triển khai mạng nơ-ron bằng API Keras của TensorFlow. Đây là một API cấp cao được thiết kế đẹp mắt và đơn giản để xây dựng, huấn luyện, đánh giá và chạy mạng nơ-ron. Nhưng đừng bị đánh lừa bởi sự đơn giản của nó: nó đủ biểu cảm và linh hoạt để cho phép bạn xây dựng nhiều loại kiến trúc mạng nơ-ron. Trên thực tế, nó có thể sẽ đủ cho hầu hết các trường hợp sử dụng của bạn. Và nếu bạn cần thêm sự linh hoạt, bạn luôn có thể viết các thành phần Keras tùy chỉnh bằng API cấp thấp hơn của nó, hoặc thậm chí sử dụng trực tiếp TensorFlow, như bạn sẽ thấy trong Chương 12. Nhưng trước tiên, hãy quay ngược thời gian để xem các mạng nơ-ron nhân tạo ra đời như thế nào!

Từ Nơ-ron Sinh học đến Nơ-ron Nhân tạo

Điều đáng ngạc nhiên là các ANN đã xuất hiện khá lâu: chúng được giới thiệu lần đầu tiên vào năm 1943 bởi nhà thần kinh học Warren McCulloch và nhà toán học Walter Pitts. Trong bài báo mang tính bước ngoặt của họ "A Logical Calculus of Ideas Immanent in Nervous Activity", McCulloch và Pitts đã trình bày một mô hình tính toán đơn giản về cách các nơ-ron sinh học có thể hoạt động cùng nhau trong não động vật để thực hiện các phép tính phức tạp bằng logic mệnh đề. Đây là kiến trúc mạng nơ-ron nhân tạo đầu tiên. Kể từ đó, nhiều kiến trúc khác đã được phát minh, như bạn sẽ thấy.

Những thành công ban đầu của ANN đã dẫn đến niềm tin rộng rãi rằng chúng ta sẽ sớm trò chuyện với những cỗ máy thực sự thông minh. Khi vào những năm 1960, rõ ràng là lời hứa này sẽ không được thực hiện (ít nhất là trong một thời gian khá dài), các khoản tài trợ đã chuyển hướng, và ANN bước vào một mùa đông dài. Đầu những năm 1980, các kiến trúc mới đã được phát minh và các kỹ thuật huấn luyện tốt hơn đã được phát triển, châm ngòi cho sự hồi sinh của mối quan tâm đến thuyết liên kết (connectionism) – nghiên cứu về mạng nơ-ron. Nhưng tiến độ chậm, và đến những năm 1990, các kỹ thuật học máy mạnh mẽ khác đã được phát minh, chẳng hạn như máy vector hỗ trợ (xem Chương 5). Các kỹ thuật này dường như mang lại kết quả tốt hơn và nền tảng lý thuyết vững chắc hơn so với ANN, vì vậy một lần nữa, nghiên cứu về mạng nơ-ron lại bị tạm dừng.

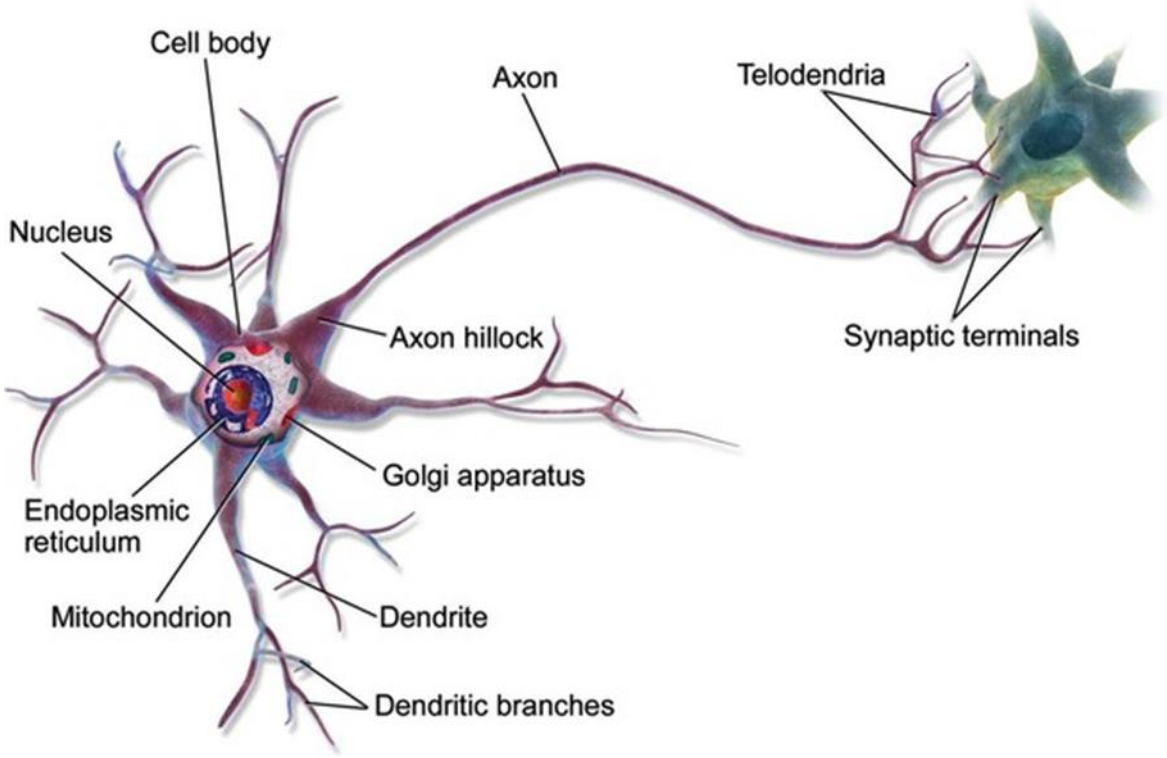
Hiện tại chúng ta đang chứng kiến một làn sóng quan tâm khác đến ANN. Liệu làn sóng này có biến mất như những làn sóng trước không? Chà, đây là một vài lý do tốt để tin rằng lần này sẽ khác và mối quan tâm mới đến ANN sẽ có tác động sâu sắc hơn nhiều đến cuộc sống của chúng ta:

- Hiện có một lượng lớn dữ liệu có sẵn để huấn luyện mạng nơ-ron, và ANN thường vượt trội hơn các kỹ thuật học máy khác trong các vấn đề rất lớn và phức tạp.
- Sự gia tăng đáng kể về sức mạnh tính toán kể từ những năm 1990 hiện cho phép huấn luyện các mạng nơ-ron lớn trong một khoảng thời gian hợp lý. Điều này một phần là do định luật Moore (số lượng linh kiện trong mạch tích hợp đã tăng gấp đôi khoảng 2 năm một lần trong 50 năm qua), nhưng cũng nhờ vào ngành công nghiệp trò chơi, đã thúc đẩy sản xuất hàng triệu thẻ GPU mạnh mẽ. Hơn nữa, các nền tảng đám mây đã giúp mọi người có thể tiếp cận sức mạnh này.
- Các thuật toán huấn luyện đã được cải thiện. Công bằng mà nói, chúng chỉ hơi khác so với những thuật toán được sử dụng vào những năm 1990, nhưng những thay đổi tương đối nhỏ này đã có tác động tích cực rất lớn.
- Một số hạn chế lý thuyết của ANN đã trở nên vô hại trong thực tế. Ví dụ, nhiều người nghĩ rằng các thuật toán huấn luyện ANN sẽ thất bại vì chúng có khả năng bị mắc kẹt trong các cực tiểu cục bộ, nhưng hóa ra đây không phải là vấn đề lớn trong thực tế, đặc biệt đối với các mạng nơ-ron lớn hơn: các cực tiểu cục bộ thường hoạt động tốt gần bằng cực đại toàn cục.
- ANNs dường như đã đi vào một vòng luẩn quẩn tích cực của việc tài trợ và tiến bộ. Các sản phẩm tuyệt vời dựa trên ANN thường xuyên xuất hiện trên các bản tin, thu hút ngày càng nhiều sự chú ý và tài trợ cho chúng, dẫn đến ngày càng nhiều tiến bộ và thậm chí nhiều sản phẩm tuyệt vời hơn.

Nơ-ron Sinh học

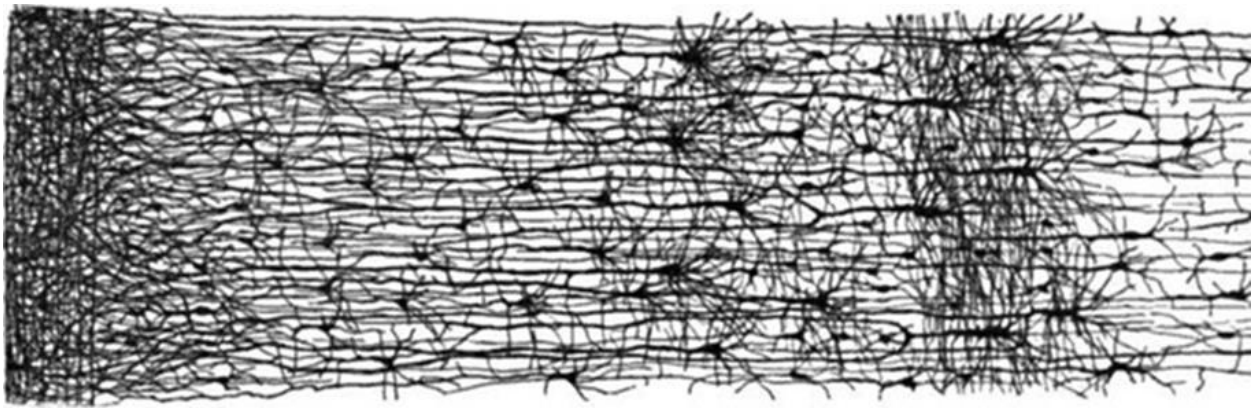
Trước khi chúng ta thảo luận về các nơ-ron nhân tạo, hãy xem nhanh một nơ-ron sinh học (được trình bày trong Hình 10-1). Nó là một tế bào có hình dạng bất thường chủ yếu được tìm thấy trong não động vật. Nó bao gồm một thân tế bào chứa nhân và hầu hết các thành phần phức tạp của tế bào, nhiều phần mở rộng phân nhánh được gọi là đuôi gai (dendrites), cộng với một phần mở rộng rất dài được gọi là sợi trục (axon). Chiều dài của sợi trục có thể chỉ dài hơn vài lần so với thân tế bào, hoặc lên đến hàng chục nghìn lần. Gần

cuối sợi trục, nó phân nhánh thành nhiều nhánh nhỏ hơn được gọi là telodendria, và ở đầu các nhánh này là các cấu trúc nhỏ xíu được gọi là đầu cuối khớp thần kinh (synaptic terminals) (hoặc đơn giản là khớp thần kinh), được kết nối với đuôi gai hoặc thân tế bào của các nơ-ron khác. Các nơ-ron sinh học tạo ra các xung điện ngắn gọi là điện thế hoạt động (APs, hoặc đơn giản là tín hiệu), di chuyển dọc theo các sợi trục và khiến các khớp thần kinh giải phóng các tín hiệu hóa học gọi là chất dẫn truyền thần kinh. Khi một nơ-ron nhận đủ lượng chất dẫn truyền thần kinh này trong vòng vài mili giây, nó sẽ phát ra các xung điện của riêng mình (thực tế, điều này phụ thuộc vào chất dẫn truyền thần kinh, vì một số trong số chúng ức chế nơ-ron phát xung).



Hình 10-1. Một nơ-ron sinh học

Do đó, các nơ-ron sinh học riêng lẻ dường như hoạt động theo một cách đơn giản, nhưng chúng được tổ chức trong một mạng lưới rộng lớn gồm hàng tỷ nơ-ron, với mỗi nơ-ron thường được kết nối với hàng nghìn nơ-ron khác. Các phép tính cực kỳ phức tạp có thể được thực hiện bởi một mạng lưới các nơ-ron khá đơn giản, giống như một tổ kiến phức tạp có thể xuất hiện từ những nỗ lực kết hợp của những con kiến đơn giản. Kiến trúc của mạng nơ-ron sinh học (BNN) là chủ đề nghiên cứu tích cực, nhưng một số phần của bộ não đã được lập bản đồ. Những nỗ lực này cho thấy rằng các nơ-ron thường được tổ chức thành các lớp liên tiếp, đặc biệt là ở vỏ não (lớp ngoài của não), như được trình bày trong Hình 10-2.

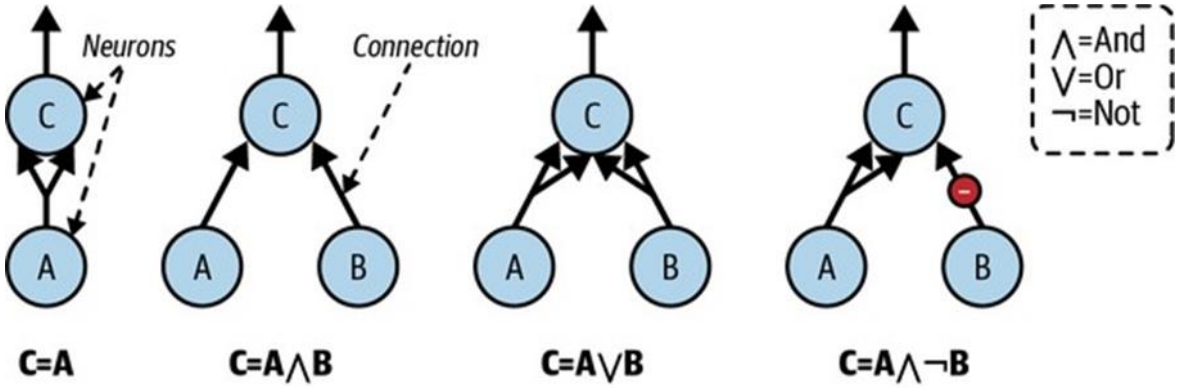


Hình 10-2. Nhiều lớp trong mạng nơ-ron sinh học (vỏ não người)

Các phép tính logic với nơ-ron

McCulloch và Pitts đã đề xuất một mô hình rất đơn giản của nơ-ron sinh học, sau này được gọi là nơ-ron nhân tạo: nó có một hoặc nhiều đầu vào nhị phân (bật/tắt) và một đầu ra nhị phân. Nơ-ron nhân tạo kích hoạt đầu ra của nó khi nhiều hơn một số lượng nhất định các đầu vào của nó đang hoạt động. Trong bài báo của họ, McCulloch và Pitts đã chỉ ra rằng ngay cả với một mô hình đơn giản như vậy, có thể xây dựng một mạng lưới các nơ-ron nhân tạo có thể tính toán bất kỳ mệnh đề logic nào bạn muốn.

Để xem mạng lưới như vậy hoạt động như thế nào, hãy xây dựng một vài ANN thực hiện các phép tính logic khác nhau (xem Hình 10-3), giả sử rằng một nơ-ron được kích hoạt khi ít nhất hai trong số các kết nối đầu vào của nó đang hoạt động.



Hình 10-3. ANN thực hiện các phép tính logic đơn giản

Hãy xem các mạng này làm gì:

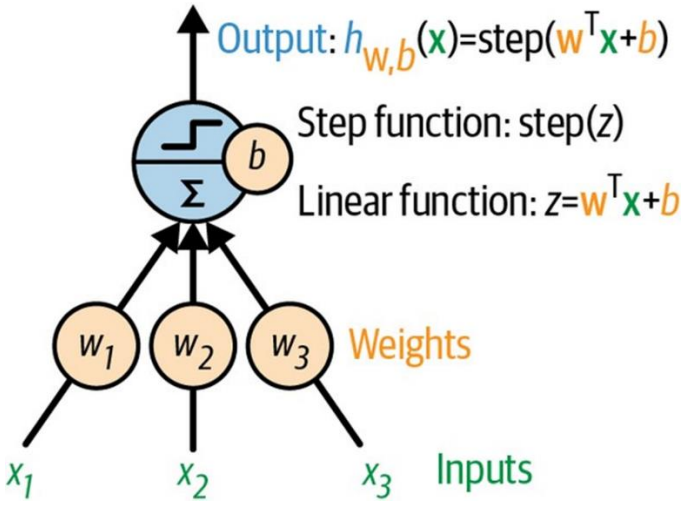
- Mạng đầu tiên ở bên trái là hàm đồng nhất (identity function): nếu nơ-ron A được kích hoạt, thì nơ-ron C cũng được kích hoạt (vì nó nhận hai tín hiệu đầu vào từ nơ-ron A); nhưng nếu nơ-ron A tắt, thì nơ-ron C cũng tắt.

- Mạng thứ hai thực hiện phép toán logic VÀ (AND): nơ-ron C chỉ được kích hoạt khi cả nơ-ron A và B đều được kích hoạt (một tín hiệu đầu vào duy nhất không đủ để kích hoạt nơ-ron C).
- Mạng thứ ba thực hiện phép toán logic HOẶC (OR): nơ-ron C được kích hoạt nếu nơ-ron A hoặc nơ-ron B được kích hoạt (hoặc cả hai).
- Cuối cùng, nếu chúng ta giả sử rằng một kết nối đầu vào có thể ức chế hoạt động của nơ-ron (điều này đúng với nơ-ron sinh học), thì mạng thứ tư tính toán một mệnh đề logic phức tạp hơn một chút: nơ-ron C chỉ được kích hoạt nếu nơ-ron A hoạt động và nơ-ron B tắt. Nếu nơ-ron A hoạt động mọi lúc, thì bạn nhận được một phép toán logic KHÔNG (NOT): nơ-ron C hoạt động khi nơ-ron B tắt và ngược lại.

Bạn có thể tưởng tượng cách các mạng này có thể được kết hợp để tính toán các biểu thức logic phức tạp (xem các bài tập ở cuối chương để biết ví dụ).

Perceptron

Perceptron là một trong những kiến trúc mạng nơ-ron nhân tạo (ANN) đơn giản nhất, được Frank Rosenblatt phát minh vào năm 1957. Nó dựa trên một nơ-ron nhân tạo hơi khác một chút (xem Hình 10-4) được gọi là đơn vị logic ngưỡng (TLU), hoặc đôi khi là đơn vị ngưỡng tuyến tính (LTU). Các đầu vào và đầu ra là các số (thay vì các giá trị bật/tắt nhị phân), và mỗi kết nối đầu vào được liên kết với một trọng số. TLU trước tiên tính toán một hàm tuyến tính của các đầu vào của nó: $z = w_1x_1 + w_2x_2 + +w_nx_n + b = \widehat{}\backslash + b$. Sau đó, nó áp dụng một hàm bước cho kết quả: $h_w(x) = textstep(z)$. Vì vậy, nó gần giống như hồi quy logistic, ngoại trừ việc nó sử dụng hàm bước thay vì hàm logistic (Chương 4). Giống như trong hồi quy logistic, các tham số của mô hình là trọng số đầu vào w và số hạng sai lệch b .



Hình 10-4. TLU: một nơ-ron nhân tạo tính tổng có trọng số của các đầu vào $\widehat{}\backslash$, cộng với số hạng sai lệch b , sau đó áp dụng hàm bước

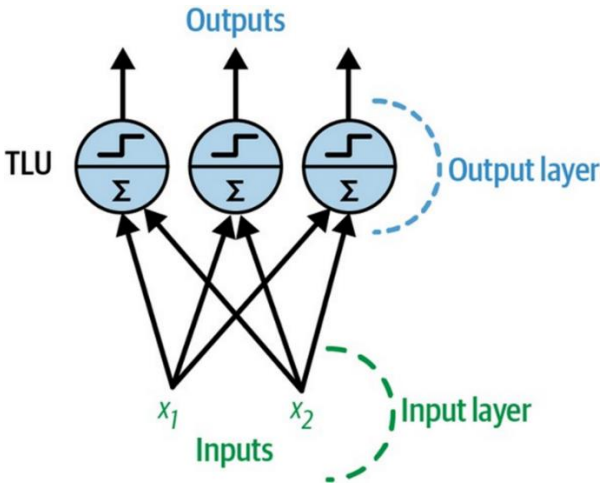
Hàm bước phổ biến nhất được sử dụng trong perceptron là hàm bước Heaviside (xem Công thức 10-1). Đôi khi hàm dấu được sử dụng thay thế.

Công thức 10-1. Các hàm bước phổ biến được sử dụng trong perceptron (giả sử ngưỡng = 0)

$$\text{heaviside}\,(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}\,(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

Một TLU đơn có thể được sử dụng cho phân loại nhị phân tuyến tính đơn giản. Nó tính toán một hàm tuyến tính của các đầu vào của nó, và nếu kết quả vượt quá ngưỡng, nó sẽ xuất ra lớp tích cực. Ngược lại, nó xuất ra lớp tiêu cực. Điều này có thể gợi cho bạn nhớ đến hồi quy logistic (Chương 4) hoặc phân loại SVM tuyến tính (Chương 5). Bạn có thể, ví dụ, sử dụng một TLU đơn để phân loại hoa diên vĩ dựa trên chiều dài và chiều rộng cánh hoa. Huấn luyện một TLU như vậy sẽ yêu cầu tìm các giá trị đúng cho w_1, w_2 , và b (thuật toán huấn luyện sẽ được thảo luận ngay sau đây).

Một perceptron được cấu tạo từ một hoặc nhiều TLU được tổ chức trong một lớp duy nhất, trong đó mỗi TLU được kết nối với mọi đầu vào. Một lớp như vậy được gọi là lớp kết nối đầy đủ, hoặc lớp dày đặc. Các đầu vào tạo thành lớp đầu vào. Và vì lớp TLU tạo ra các đầu ra cuối cùng, nó được gọi là lớp đầu ra. Ví dụ, một perceptron với hai đầu vào và ba đầu ra được biểu diễn trong Hình 10-5.



Hình 10-5. Kiến trúc của một perceptron với hai đầu vào và ba nơ-ron đầu ra

Perceptron này có thể phân loại các trường hợp đồng thời thành ba lớp nhị phân khác nhau, điều này làm cho nó trở thành một bộ phân loại đa nhãn. Nó cũng có thể được sử dụng cho phân loại đa lớp. Nhờ sự kỳ diệu của đại số tuyến tính, Công thức 10-2 có thể được sử dụng để tính toán hiệu quả các đầu ra của một lớp nơ-ron nhân tạo cho nhiều trường hợp cùng một lúc.

Công thức 10-2. Tính toán đầu ra của một lớp kết nối đầy đủ

$$h_W, b(X) = phi(XW + b)$$

Trong phương trình này:

- Như thường lệ, X biểu thị ma trận các đặc trưng đầu vào. Nó có một hàng cho mỗi trường hợp và một cột cho mỗi đặc trưng.
- Ma trận trọng số W chứa tất cả các trọng số kết nối. Nó có một hàng cho mỗi đầu vào và một cột cho mỗi nơ-ron.
- Vector độ lệch b chứa tất cả các số hạng độ lệch: một cho mỗi nơ-ron.
- Hàm phi được gọi là hàm kích hoạt: khi các nơ-ron nhân tạo là TLU, nó là một hàm bước (chúng ta sẽ thảo luận về các hàm kích hoạt khác sau).

Vậy, một perceptron được huấn luyện như thế nào? Thuật toán huấn luyện perceptron được Rosenblatt đề xuất phần lớn được lấy cảm hứng từ quy tắc Hebb. Trong cuốn sách năm 1949 của ông *The Organization of Behavior* (Wiley), Donald Hebb gợi ý rằng khi một nơ-ron sinh học kích hoạt một nơ-ron khác thường xuyên, kết nối giữa hai nơ-ron này sẽ mạnh hơn. Siegrid Löwel sau đó đã tóm tắt ý tưởng của Hebb bằng cụm từ hấp dẫn, “Các tế bào kích hoạt cùng nhau sẽ kết nối cùng nhau” (Cells that fire together, wire together); nghĩa là, trọng số kết nối giữa hai nơ-ron có xu hướng tăng lên khi chúng kích hoạt đồng thời. Quy tắc này sau đó được gọi là quy tắc Hebb (hoặc học Hebbian).

Perceptron được huấn luyện bằng cách sử dụng một biến thể của quy tắc này có tính đến lỗi mà mạng mắc phải khi đưa ra dự đoán; quy tắc học perceptron củng cố các kết nối giúp giảm lỗi. Cụ thể hơn, perceptron được đưa một trường hợp huấn luyện một lúc, và đối với mỗi trường hợp, nó đưa ra dự đoán của mình. Đối với mỗi nơ-ron đầu ra tạo ra dự đoán sai, nó sẽ củng cố trọng số kết nối từ các đầu vào lẽ ra đã góp phần vào dự đoán chính xác. Quy tắc được trình bày trong Công thức 10-3.

Công thức 10-3. Quy tắc học perceptron (cập nhật trọng số)

$$w_i, j^{(next\ step)} = w_{i,j} + eta\left(y_j - \hat{y}_j\right)x_i$$

Trong phương trình này:

- w_i, j là trọng số kết nối giữa đầu vào thứ i và nơ-ron thứ j .
- x_i là giá trị đầu vào thứ i của trường hợp huấn luyện hiện tại.
- $\hat{y}_j$ là đầu ra của nơ-ron đầu ra thứ j cho trường hợp huấn luyện hiện tại.
- y_j là đầu ra mục tiêu của nơ-ron đầu ra thứ j cho trường hợp huấn luyện hiện tại.
- eta là tốc độ học (xem Chương 4).

Ranh giới quyết định của mỗi nơ-ron đầu ra là tuyến tính, do đó perceptron không có khả năng học các mẫu phức tạp (giống như bộ phân loại hồi quy logistic). Tuy nhiên, nếu các trường hợp huấn luyện có thể phân tách tuyến tính, Rosenblatt đã chứng minh rằng thuật toán này sẽ hội tụ đến một giải pháp. Điều này được gọi là định lý hội tụ perceptron.

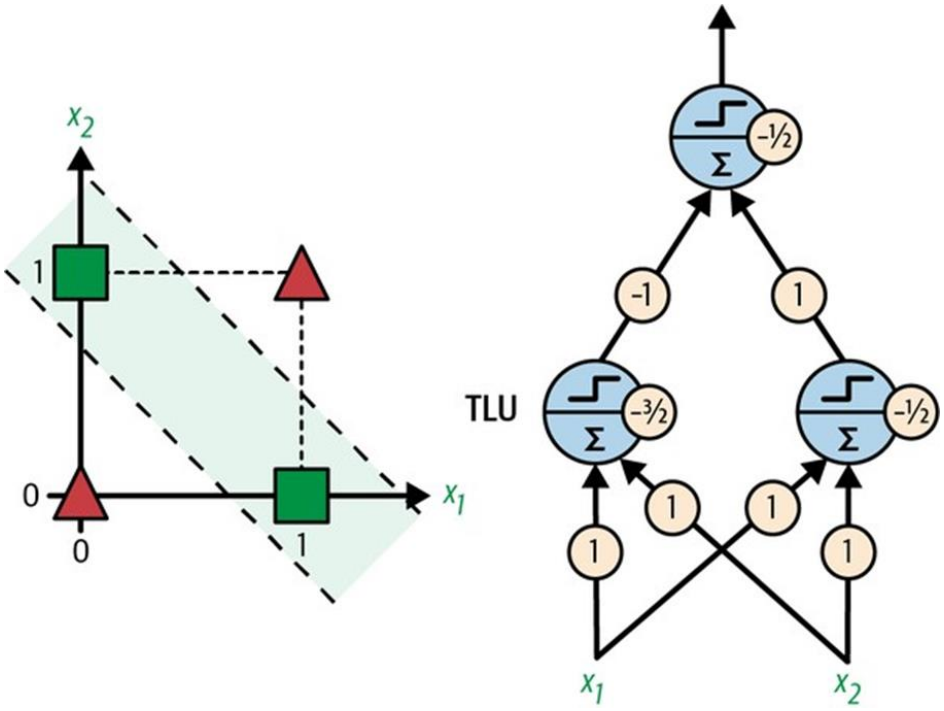
Scikit-Learn cung cấp lớp `Perceptron` có thể được sử dụng gần như theo cách bạn mong đợi - ví dụ, trên tập dữ liệu iris (được giới thiệu trong Chương 4):

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 0) # Iris setosa

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)

X_new = [[2, 0.5], [3, 1]]
y_pred = per_clf.predict(X_new) # predicts True and False for these 2 flowers
```

Bạn có thể nhận thấy rằng thuật toán học perceptron rất giống với thuật toán giảm độ dốc ngẫu nhiên (được giới thiệu trong Chương 4). Trên thực tế, lớp `Perceptron` của Scikit-Learn tương đương với việc sử dụng `SGDClassifier` với các siêu tham số sau: `loss="perceptron"`, `learning_rate="constant"`, `eta0=1` (tốc độ học), và `penalty=None` (không chính quy hóa).

Trong chuyên khảo *Perceptrons* năm 1969 của họ, Marvin Minsky và Seymour Papert đã nêu bật một số điểm yếu nghiêm trọng của perceptron - đặc biệt, thực tế là chúng không có khả năng giải quyết một số vấn đề tầm thường (ví dụ: vấn đề phân loại HOẶC LOẠI TRỪ (XOR); xem phía bên trái của Hình 10-6). Điều này đúng với bất kỳ mô hình phân loại tuyến tính nào khác (chẳng hạn như bộ phân loại hồi quy logistic), nhưng các nhà nghiên cứu đã kỳ vọng nhiều hơn từ perceptron, và một số người đã thất vọng đến mức họ từ bỏ hoàn toàn mạng nơ-ron để chuyển sang các vấn đề cấp cao hơn như logic, giải quyết vấn đề và tìm kiếm. Việc thiếu các ứng dụng thực tế cũng không giúp ích gì.

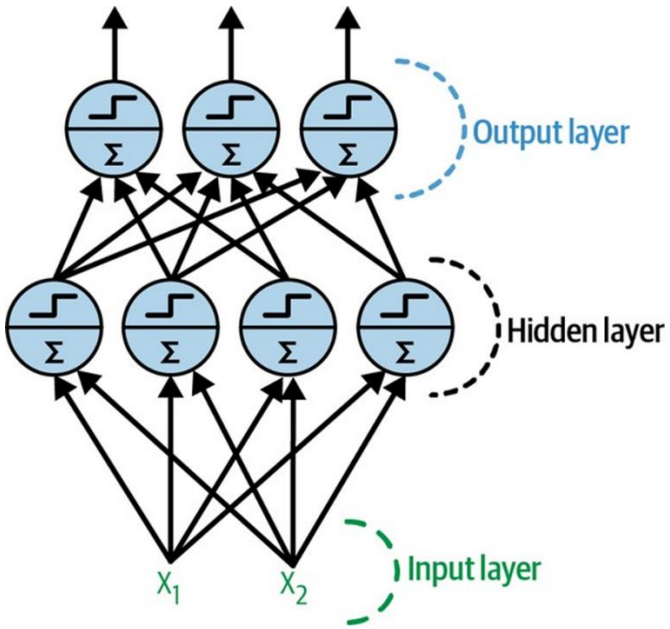
Hóa ra một số hạn chế của perceptron có thể được loại bỏ bằng cách xếp chồng nhiều perceptron. Mạng nơ-ron nhân tạo thu được được gọi là mạng perceptron đa lớp (MLP). Một MLP có thể giải quyết vấn đề XOR, như bạn có thể xác minh bằng cách tính toán đầu ra của MLP được biểu diễn ở phía bên phải của Hình 10-6: với đầu vào (0, 0) hoặc (1, 1), mạng xuất ra 0, và với đầu vào (0, 1) hoặc (1, 0) nó xuất ra 1. Hãy thử xác minh rằng mạng này thực sự giải quyết vấn đề XOR!



Hình 10-6. Vấn đề phân loại XOR và một MLP giải quyết nó

Mạng perceptron đa lớp và Backpropagation

Một MLP được cấu tạo từ một lớp đầu vào, một hoặc nhiều lớp TLU được gọi là lớp ẩn, và một lớp TLU cuối cùng được gọi là lớp đầu ra (xem Hình 10-7). Các lớp gần lớp đầu vào thường được gọi là lớp dưới, và các lớp gần đầu ra thường được gọi là lớp trên.



Hình 10-7. Kiến trúc của một mạng perceptron đa lớp với hai đầu vào, một lớp ẩn gồm bốn nơ-ron và ba nơ-ron đầu ra

Khi một ANN chứa một chồng sâu các lớp ẩn, nó được gọi là mạng nơ-ron sâu (DNN). Lĩnh vực học sâu nghiên cứu DNN, và tổng quát hơn là quan tâm đến các mô hình chứa các chồng tính toán sâu. Mặc dù vậy, nhiều người nói về học sâu bất cứ khi nào có liên quan đến mạng nơ-ron (ngay cả những mạng nông).

Trong nhiều năm, các nhà nghiên cứu đã cố gắng tìm cách huấn luyện MLP nhưng không thành công. Đầu những năm 1960, một số nhà nghiên cứu đã thảo luận về khả năng sử dụng gradient descent để huấn luyện mạng nơ-ron, nhưng như chúng ta đã thấy trong Chương 4, điều này đòi hỏi phải tính toán gradient của lỗi mô hình đối với các tham số mô hình; vào thời điểm đó, không rõ làm thế nào để thực hiện việc này một cách hiệu quả với một mô hình phức tạp như vậy chứa quá nhiều tham số, đặc biệt với các máy tính mà họ có khi đó.

Sau đó, vào năm 1970, một nhà nghiên cứu tên là Seppo Linnainmaa đã giới thiệu trong luận văn thạc sĩ của mình một kỹ thuật để tính toán tất cả các gradient một cách tự động và hiệu quả. Thuật toán này hiện được gọi là tự động đạo hàm ngược (reverse-mode automatic differentiation) (hoặc viết tắt là reverse-mode autodiff). Chỉ trong hai lượt qua mạng (một lượt tiến, một lượt lùi), nó có thể tính toán gradient của lỗi mạng nơ-ron đối với từng tham số mô hình. Nói cách khác, nó có thể tìm ra cách điều chỉnh từng trọng số kết nối và từng độ lệch để giảm lỗi của mạng nơ-ron. Các gradient này sau đó có thể được sử dụng để thực hiện một bước giảm độ dốc. Nếu bạn lặp lại quá trình tính toán gradient tự động và thực hiện một bước giảm độ dốc này, lỗi của mạng nơ-ron sẽ giảm dần cho đến khi cuối cùng đạt đến một cực tiểu. Sự kết hợp giữa tự động đạo hàm ngược và giảm độ dốc này hiện được gọi là backpropagation (hoặc viết tắt là backprop).

Backpropagation thực sự có thể được áp dụng cho tất cả các loại biểu đồ tính toán, không chỉ mạng nơ-ron : thực tế, luận văn thạc sĩ của Linnainmaa không phải về mạng nơ-ron, nó mang tính tổng quát hơn. Phải mất thêm vài năm nữa trước khi backprop bắt đầu được sử dụng để huấn luyện mạng nơ-ron, nhưng nó vẫn chưa phổ biến. Sau đó, vào năm 1985, David Rumelhart, Geoffrey Hinton và Ronald Williams đã công bố một bài báo đột phá phân tích cách backpropagation cho phép mạng nơ-ron học các biểu diễn nội bộ hữu ích. Kết quả của họ ấn tượng đến mức backpropagation nhanh chóng được phổ biến trong lĩnh vực này. Ngày nay, nó là kỹ thuật huấn luyện phổ biến nhất cho mạng nơ-ron.

Hãy cùng xem lại cách backpropagation hoạt động chi tiết hơn một chút:

- Nó xử lý từng mini-batch một (ví dụ, mỗi mini-batch chứa 32 trường hợp), và nó đi qua toàn bộ tập huấn luyện nhiều lần. Mỗi lượt được gọi là một epoch.
- Mỗi mini-batch đi vào mạng thông qua lớp đầu vào. Thuật toán sau đó tính toán đầu ra của tất cả các nơ-ron trong lớp ẩn đầu tiên, cho mọi trường hợp trong mini-batch. Kết quả được chuyển đến lớp tiếp theo, đầu ra của nó được tính toán và chuyển đến lớp tiếp theo, cứ thế cho đến khi chúng ta nhận được đầu ra của lớp cuối cùng, lớp đầu ra. Đây là lượt truyền tiến (forward pass): nó giống hệt như việc đưa ra dự đoán, ngoại trừ tất cả các kết quả trung gian được bảo toàn vì chúng cần thiết cho lượt truyền ngược.

- Tiếp theo, thuật toán đo lỗi đầu ra của mạng (nghĩa là, nó sử dụng một hàm mất mát so sánh đầu ra mong muốn và đầu ra thực tế của mạng, và trả về một số thước đo lỗi).
- Sau đó, nó tính toán mức độ mỗi độ lệch đầu ra và mỗi kết nối đến lớp đầu ra đã đóng góp vào lỗi. Điều này được thực hiện bằng cách phân tích áp dụng quy tắc chuỗi (có lẽ là quy tắc cơ bản nhất trong giải tích), điều này làm cho bước này nhanh chóng và chính xác.
- Thuật toán sau đó đo lường mức độ các đóng góp lỗi này đến từ mỗi kết nối trong lớp bên dưới, một lần nữa sử dụng quy tắc chuỗi, hoạt động ngược lại cho đến khi nó đạt đến lớp đầu vào. Như đã giải thích trước đó, lượt truyền ngược này đo lường hiệu quả gradient lỗi trên tất cả các trọng số kết nối và độ lệch trong mạng bằng cách truyền ngược gradient lỗi qua mạng (do đó có tên của thuật toán).
- Cuối cùng, thuật toán thực hiện một bước giảm độ dốc để điều chỉnh tất cả các trọng số kết nối trong mạng, sử dụng các gradient lỗi vừa được tính toán.

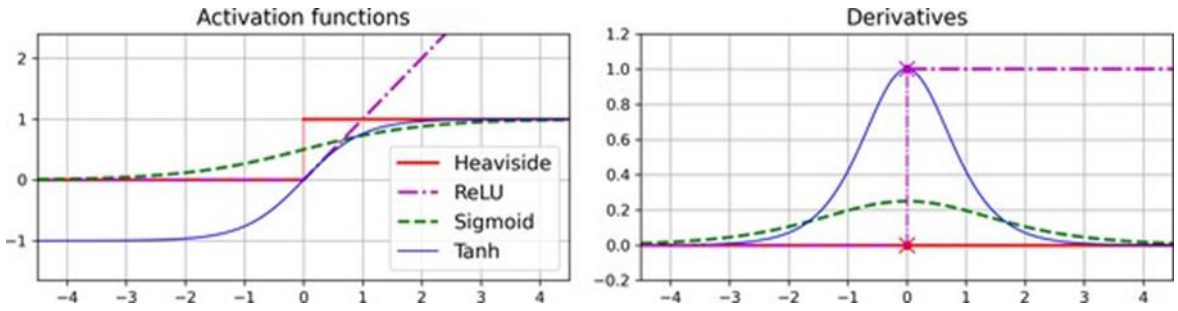
Tóm lại, backpropagation đưa ra dự đoán cho một mini-batch (lượt truyền tiến), đo lỗi, sau đó đi qua từng lớp theo chiều ngược lại để đo đóng góp lỗi từ mỗi tham số (lượt truyền ngược), và cuối cùng điều chỉnh trọng số kết nối và độ lệch để giảm lỗi (bước giảm độ dốc).

Để backprop hoạt động đúng cách, Rumelhart và các đồng nghiệp của ông đã thực hiện một thay đổi quan trọng đối với kiến trúc của MLP: họ thay thế hàm bước bằng hàm logistic, $sigma(z) = 1/(1 + exp(-z))$, còn được gọi là hàm sigmoid. Điều này rất cần thiết vì hàm bước chỉ chứa các đoạn phẳng, do đó không có gradient để làm việc (giảm độ dốc không thể di chuyển trên bề mặt phẳng), trong khi hàm sigmoid có đạo hàm khác 0 được xác định rõ ở khắp mọi nơi, cho phép giảm độ dốc đạt được một số tiến bộ ở mỗi bước. Trên thực tế, thuật toán backpropagation hoạt động tốt với nhiều hàm kích hoạt khác, không chỉ hàm sigmoid. Dưới đây là hai lựa chọn phổ biến khác:

- **Hàm hyperbolic tangent**: $texttanh(z) = 2sigma(2z) - 1$ Giống như hàm sigmoid, hàm kích hoạt này có hình chữ S, liên tục và khả vi, nhưng giá trị đầu ra của nó nằm trong khoảng từ -1 đến 1 (thay vì 0 đến 1 trong trường hợp hàm sigmoid). Khoảng giá trị đó có xu hướng làm cho đầu ra của mỗi lớp ít nhiều tập trung quanh 0 khi bắt đầu huấn luyện, điều này thường giúp tăng tốc độ hội tụ.
- **Hàm rectified linear unit (ReLU)**: $textReLU(z) = max(0, z)$ Hàm ReLU liên tục nhưng không may không khả vi tại $z = 0$ (độ dốc thay đổi đột ngột, điều này có thể khiến gradient descent bị dao động), và đạo hàm của nó bằng 0 với $z \< 0$. Tuy nhiên, trong thực tế, nó hoạt động rất tốt và có ưu điểm là tính toán nhanh, vì vậy nó đã trở thành mặc định. Quan trọng là, việc nó không có giá trị đầu ra tối đa giúp giảm một số vấn đề trong quá trình giảm độ dốc (chúng ta sẽ quay lại vấn đề này trong Chương 11).

Các hàm kích hoạt phổ biến này và đạo hàm của chúng được biểu diễn trong Hình 10-8. Nhưng khoan! Tại sao chúng ta cần các hàm kích hoạt ngay từ đầu? Chà, nếu bạn nối nhiều phép biến đổi tuyến tính, tất cả những gì bạn nhận được là một phép biến đổi tuyến tính. Ví

dụ, nếu $f(x) = 2x + 3$ và $g(x) = 5x - 1$, thì việc nối hai hàm tuyến tính này sẽ cho bạn một hàm tuyến tính khác: $f(g(x)) = 2(5x - 1) + 3 = 10x + 1$. Vì vậy, nếu bạn không có một số phi tuyến tính giữa các lớp, thì ngay cả một chồng sâu các lớp cũng tương đương với một lớp duy nhất, và bạn không thể giải quyết các vấn đề rất phức tạp với điều đó. Ngược lại, một DNN đủ lớn với các kích hoạt phi tuyến tính về mặt lý thuyết có thể xấp xỉ bất kỳ hàm liên tục nào.



Hình 10-8. Các hàm kích hoạt (trái) và đạo hàm của chúng (phải)

OK! Bạn đã biết mạng nơ-ron từ đâu mà có, kiến trúc của chúng là gì và cách tính toán đầu ra của chúng. Bạn cũng đã học về thuật toán backpropagation. Nhưng chính xác thì bạn có thể làm gì với mạng nơ-ron?

MLP hồi quy

Đầu tiên, MLP có thể được sử dụng cho các tác vụ hồi quy. Nếu bạn muốn dự đoán một giá trị duy nhất (ví dụ: giá của một căn nhà, với nhiều đặc trưng của nó), thì bạn chỉ cần một nơ-ron đầu ra duy nhất: đầu ra của nó là giá trị dự đoán. Đối với hồi quy đa biến (nghĩa là, để dự đoán nhiều giá trị cùng một lúc), bạn cần một nơ-ron đầu ra cho mỗi chiều đầu ra. Ví dụ, để định vị trung tâm của một đối tượng trong hình ảnh, bạn cần dự đoán tọa độ 2D, vì vậy bạn cần hai nơ-ron đầu ra. Nếu bạn cũng muốn đặt một hộp giới hạn xung quanh đối tượng, thì bạn cần thêm hai số nữa: chiều rộng và chiều cao của đối tượng. Vì vậy, bạn sẽ có bốn nơ-ron đầu ra.

Scikit-Learn bao gồm một lớp `MLPRegressor`, vì vậy hãy sử dụng nó để xây dựng một MLP với ba lớp ẩn gồm 50 nơ-ron mỗi lớp, và huấn luyện nó trên tập dữ liệu California housing. Để đơn giản, chúng ta sẽ sử dụng hàm `fetch_california_housing()` của Scikit-Learn để tải dữ liệu. Tập dữ liệu này đơn giản hơn tập dữ liệu chúng ta đã sử dụng trong Chương 2, vì nó chỉ chứa các đặc trưng số (không có đặc trưng `ocean_proximity`), và không có giá trị bị thiếu. Đoạn mã sau bắt đầu bằng cách lấy và chia tập dữ liệu, sau đó nó tạo một pipeline để chuẩn hóa các đặc trưng đầu vào trước khi gửi chúng đến `MLPRegressor`. Điều này rất quan trọng đối với mạng nơ-ron vì chúng được huấn luyện bằng cách sử dụng giảm độ dốc, và như chúng ta đã thấy trong Chương 4, giảm độ dốc không hội tụ tốt khi các đặc trưng có các thang đo rất khác nhau. Cuối cùng, mã này huấn luyện mô hình và đánh giá lỗi xác thực của nó. Mô hình sử dụng hàm kích hoạt ReLU trong các lớp ẩn, và nó sử dụng một biến thể của giảm độ dốc gọi là Adam (xem Chương 11) để giảm thiểu lỗi bình phương trung bình, với một chút chính quy hóa l_2 (mà bạn có thể kiểm soát thông qua siêu tham số `alpha`):

```
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(housing.data,
housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(X_train_full,
y_train_full, random_state=42)

mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], random_state=42)
pipeline = make_pipeline(StandardScaler(), mlp_reg)
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_valid)

rmse = mean_squared_error(y_valid, y_pred, squared=False) # about 0.505
```

Chúng ta nhận được RMSE xác thực khoảng 0.505, có thể so sánh với những gì bạn sẽ nhận được với một bộ phân loại rừng ngẫu nhiên. Không tệ cho lần thử đầu tiên!

Lưu ý rằng MLP này không sử dụng bất kỳ hàm kích hoạt nào cho lớp đầu ra, vì vậy nó được tự do xuất ra bất kỳ giá trị nào nó muốn. Điều này thường ổn, nhưng nếu bạn muốn đảm bảo rằng đầu ra sẽ luôn dương, thì bạn nên sử dụng hàm kích hoạt ReLU trong lớp đầu ra, hoặc hàm kích hoạt softplus, là một biến thể mượt mà của ReLU: $textsoftplus(z) = log\big(1 + exp(z)\big)$. Softplus gần bằng 0 khi z âm và gần bằng z khi z dương. Cuối cùng, nếu bạn muốn đảm bảo rằng các dự đoán sẽ luôn nằm trong một phạm vi giá trị nhất định, thì bạn nên sử dụng hàm sigmoid hoặc hàm hyperbolic tangent, và điều chỉnh các mục tiêu về phạm vi thích hợp: 0 đến 1 cho sigmoid và -1 đến 1 cho tanh. Đáng buồn thay, lớp `MLPRegressor` không hỗ trợ các hàm kích hoạt trong lớp đầu ra.

Lớp `MLPRegressor` sử dụng lỗi bình phương trung bình, đây thường là điều bạn muốn cho hồi quy, nhưng nếu bạn có nhiều giá trị ngoại lai trong tập huấn luyện, bạn có thể muốn sử dụng lỗi tuyệt đối trung bình thay thế. Ngoài ra, bạn có thể muốn sử dụng Huber loss, là sự kết hợp của cả hai. Nó là hàm bậc hai khi lỗi nhỏ hơn một ngưỡng $delta$ (thường là 1) nhưng là hàm tuyến tính khi lỗi lớn hơn $delta$. Phần tuyến tính làm cho nó ít nhạy cảm với các giá trị ngoại lai hơn lỗi bình phương trung bình, và phần bậc hai cho phép nó hội tụ nhanh hơn và chính xác hơn lỗi tuyệt đối trung bình. Tuy nhiên, `MLPRegressor` chỉ hỗ trợ MSE.

Bảng 10-1 tóm tắt kiến trúc điển hình của một MLP hồi quy.

Bảng 10-1. Kiến trúc MLP hồi quy điển hình

Siêu tham số	Giá trị điển hình
# lớp ẩn	Tùy thuộc vào vấn đề, nhưng thường từ 1 đến 5

Siêu tham số	Giá trị điển hình
# nơ-ron đầu ra	1 cho mỗi chiều dự đoán
Kích hoạt đầu ra	Không, hoặc ReLU/softplus (nếu đầu ra dương) hoặc sigmoid/tanh (nếu đầu ra có giới hạn)

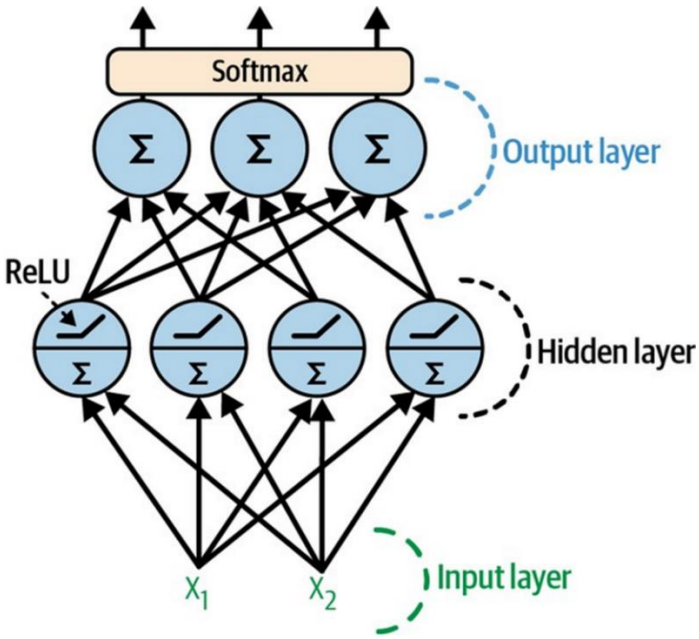
MLP phân loại

MLP cũng có thể được sử dụng cho các tác vụ phân loại. Đối với bài toán phân loại nhị phân, bạn chỉ cần một nơ-ron đầu ra duy nhất sử dụng hàm kích hoạt sigmoid: đầu ra sẽ là một số giữa 0 và 1, mà bạn có thể giải thích là xác suất ước tính của lớp dương. Xác suất ước tính của lớp âm bằng một trừ đi số đó.

MLP cũng có thể dễ dàng xử lý các tác vụ phân loại nhị phân đa nhãn (xem Chương 3). Ví dụ, bạn có thể có một hệ thống phân loại email dự đoán liệu mỗi email đến là thư hợp lệ hay thư rác, và đồng thời dự đoán liệu đó là email khẩn cấp hay không khẩn cấp. Trong trường hợp này, bạn sẽ cần hai nơ-ron đầu ra, cả hai đều sử dụng hàm kích hoạt sigmoid: nơ-ron đầu tiên sẽ xuất ra xác suất email là thư rác, và nơ-ron thứ hai sẽ xuất ra xác suất email là khẩn cấp. Tổng quát hơn, bạn sẽ dành một nơ-ron đầu ra cho mỗi lớp dương. Lưu ý rằng xác suất đầu ra không nhất thiết phải cộng lại bằng 1. Điều này cho phép mô hình xuất ra bất kỳ sự kết hợp nào của các nhãn: bạn có thể có thư hợp lệ không khẩn cấp, thư hợp lệ khẩn cấp, thư rác không khẩn cấp, và thậm chí có thể là thư rác khẩn cấp (mặc dù điều đó có lẽ sẽ là một lỗi).

Nếu mỗi trường hợp chỉ có thể thuộc về một lớp duy nhất, trong số ba hoặc nhiều lớp có thể (ví dụ: các lớp từ 0 đến 9 để phân loại hình ảnh chữ số), thì bạn cần có một nơ-ron đầu ra cho mỗi lớp, và bạn nên sử dụng hàm kích hoạt softmax cho toàn bộ lớp đầu ra (xem Hình 10-9). Hàm softmax (được giới thiệu trong Chương 4) sẽ đảm bảo rằng tất cả các xác suất ước tính nằm giữa 0 và 1 và chúng cộng lại bằng 1, vì các lớp là độc quyền. Như bạn đã thấy trong Chương 3, đây được gọi là phân loại đa lớp.

Đối với hàm mất mát, vì chúng ta đang dự đoán phân phối xác suất, mất mát entropy chéo (hoặc entropy X hoặc mất mát logarit, xem Chương 4) thường là một lựa chọn tốt.



Hình 10-9. Một MLP hiện đại (bao gồm ReLU và softmax) để phân loại

Scikit-Learn có một lớp `MLPClassifier` trong gói `sklearn.neural_network`. Nó gần như giống hệt với lớp `MLPRegressor`, ngoại trừ việc nó giảm thiểu entropy chéo thay vì MSE. Hãy thử nó ngay bây giờ, ví dụ trên tập dữ liệu iris. Đây gần như là một tác vụ tuyến tính, vì vậy một lớp duy nhất với 5 đến 10 nơ-ron sẽ đủ (đảm bảo chuẩn hóa các đặc trưng).

Bảng 10-2 tóm tắt kiến trúc điển hình của một MLP phân loại.

Bảng 10-2. Kiến trúc MLP phân loại điển hình

Siêu tham số	Phân loại nhị phân	Phân loại nhị phân đa nhãn	Phân loại đa lớp
# lớp ẩn	Thường từ 1 đến 5 lớp, tùy thuộc vào tác vụ		
# nơ-ron đầu ra	1	1 cho mỗi nhãn nhị phân	1 cho mỗi lớp
Kích hoạt lớp đầu ra	Sigmoid	Sigmoid	Softmax
Hàm mất mát	Entropy X	Entropy X	Entropy X

Bây giờ bạn đã có tất cả các khái niệm cần thiết để bắt đầu triển khai MLP với Keras!

Triển khai MLP với Keras

Keras là API học sâu cấp cao của TensorFlow: nó cho phép bạn xây dựng, huấn luyện, đánh giá và thực thi tất cả các loại mạng nơ-ron. Thư viện Keras ban đầu được François Chollet phát triển như một phần của dự án nghiên cứu và được phát hành dưới dạng một dự án mã

nguồn mở độc lập vào tháng 3 năm 2015. Nó nhanh chóng trở nên phổ biến nhờ tính dễ sử dụng, linh hoạt và thiết kế đẹp mắt.

Bây giờ hãy sử dụng Keras! Chúng ta sẽ bắt đầu bằng cách xây dựng một MLP để phân loại hình ảnh.

Xây dựng bộ phân loại hình ảnh bằng API tuần tự

Đầu tiên, chúng ta cần tải một tập dữ liệu. Chúng ta sẽ sử dụng Fashion MNIST, một tập dữ liệu thay thế trực tiếp cho MNIST (được giới thiệu trong Chương 3). Nó có định dạng chính xác giống như MNIST (70.000 hình ảnh thang độ xám 28 × 28 pixel mỗi hình, với 10 lớp), nhưng các hình ảnh này đại diện cho các mặt hàng thời trang chứ không phải chữ số viết tay, do đó mỗi lớp đa dạng hơn, và vấn đề này hóa ra thách thức hơn đáng kể so với MNIST. Ví dụ, một mô hình tuyến tính đơn giản đạt độ chính xác khoảng 92% trên MNIST, nhưng chỉ khoảng 83% trên Fashion MNIST.

Sử dụng Keras để tải tập dữ liệu

Keras cung cấp một số hàm tiện ích để lấy và tải các tập dữ liệu phổ biến, bao gồm MNIST, Fashion MNIST, và một vài tập dữ liệu khác. Hãy tải Fashion MNIST. Nó đã được xáo trộn và chia thành một tập huấn luyện (60.000 hình ảnh) và một tập kiểm tra (10.000 hình ảnh), nhưng chúng ta sẽ giữ lại 5.000 hình ảnh cuối cùng từ tập huấn luyện để xác thực:

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

Khi tải MNIST hoặc Fashion MNIST bằng Keras thay vì Scikit-Learn, một điểm khác biệt quan trọng là mỗi hình ảnh được biểu diễn dưới dạng một mảng 28 × 28 thay vì một mảng 1D có kích thước 784. Hơn nữa, cường độ pixel được biểu diễn dưới dạng số nguyên (từ 0 đến 255) thay vì số thập phân (từ 0.0 đến 255.0). Hãy xem hình dạng và kiểu dữ liệu của tập huấn luyện:

```
>>> X_train.shape
(55000, 28, 28)
>>> X_train.dtype
dtype('uint8')
```

Để đơn giản, chúng ta sẽ chia cường độ pixel cho 255.0 để đưa chúng về phạm vi 0-1 (điều này cũng chuyển chúng thành số thập phân):

```
X_train, X_valid, X_test = X_train / 255., X_valid / 255., X_test / 255.
```

Với MNIST, khi nhãn bằng 5, điều đó có nghĩa là hình ảnh đại diện cho chữ số viết tay 5. Dễ dàng. Tuy nhiên, đối với Fashion MNIST, chúng ta cần danh sách các tên lớp để biết chúng ta đang xử lý cái gì:

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
"Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

Ví dụ, hình ảnh đầu tiên trong tập huấn luyện đại diện cho một chiếc ủng cổ chân:

```
>>> class_names[y_train[0]]
'Ankle boot'
```

Hình 10-10 cho thấy một số mẫu từ tập dữ liệu Fashion MNIST.

Hình 10-10. Các mẫu từ Fashion MNIST

Tạo mô hình bằng API tuần tự

Bây giờ hãy xây dựng mạng nơ-ron! Đây là một MLP phân loại với hai lớp ẩn:

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Hãy đi qua đoạn mã này từng dòng:

- Đầu tiên, đặt hạt giống ngẫu nhiên của TensorFlow để làm cho kết quả có thể tái tạo: các trọng số ngẫu nhiên của các lớp ẩn và lớp đầu ra sẽ giống nhau mỗi khi bạn chạy notebook. Bạn cũng có thể chọn sử dụng hàm `tf.keras.utils.set_random_seed()`, chức năng này tiện lợi đặt các hạt giống ngẫu nhiên cho TensorFlow, Python (`random.seed()`), và NumPy (`np.random.seed()`).
- Dòng tiếp theo tạo một mô hình Sequential. Đây là loại mô hình Keras đơn giản nhất cho các mạng nơ-ron chỉ bao gồm một chồng các lớp được kết nối tuần tự. Đây được gọi là API tuần tự (sequential API).
- Tiếp theo, chúng ta xây dựng lớp đầu tiên (lớp Input) và thêm nó vào mô hình. Chúng ta chỉ định hình dạng đầu vào, không bao gồm kích thước batch, chỉ hình dạng của

các thể hiện. Keras cần biết hình dạng của các đầu vào để nó có thể xác định hình dạng của ma trận trọng số kết nối của lớp ẩn đầu tiên.

- Sau đó, chúng ta thêm một lớp Flatten. Vai trò của nó là chuyển đổi mỗi hình ảnh đầu vào thành một mảng 1D: ví dụ, nếu nó nhận được một batch có hình dạng [32, 28, 28], nó sẽ định hình lại thành [32, 784]. Nói cách khác, nếu nó nhận được dữ liệu đầu vào `X`, nó tính toán `X.reshape(-1, 784)`. Lớp này không có bất kỳ tham số nào; nó chỉ ở đó để thực hiện một số tiền xử lý đơn giản.
- Tiếp theo, chúng ta thêm một lớp ẩn Dense với 300 nơ-ron. Nó sẽ sử dụng hàm kích hoạt ReLU. Mỗi lớp Dense quản lý ma trận trọng số riêng của nó, chứa tất cả các trọng số kết nối giữa các nơ-ron và đầu vào của chúng. Nó cũng quản lý một vector các số hạng độ lệch (một cho mỗi nơ-ron). Khi nó nhận được một số dữ liệu đầu vào, nó tính toán theo Công thức 10-2.
- Sau đó, chúng ta thêm một lớp ẩn Dense thứ hai với 100 nơ-ron, cũng sử dụng hàm kích hoạt ReLU.
- Cuối cùng, chúng ta thêm một lớp đầu ra Dense với 10 nơ-ron (một cho mỗi lớp), sử dụng hàm kích hoạt softmax vì các lớp là độc quyền.

Thay vì thêm các lớp từng cái một như chúng ta vừa làm, thường thuận tiện hơn khi truyền một danh sách các lớp khi tạo mô hình Sequential. Bạn cũng có thể bỏ lớp Input và thay vào đó chỉ định `input_shape` trong lớp đầu tiên:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Phương thức `summary()` của mô hình hiển thị tất cả các lớp của mô hình, bao gồm tên của mỗi lớp (được tạo tự động trừ khi bạn đặt nó khi tạo lớp), hình dạng đầu ra của nó (`None` có nghĩa là kích thước batch có thể là bất kỳ thứ gì), và số lượng tham số của nó. Phần tóm tắt kết thúc với tổng số tham số, bao gồm các tham số có thể huấn luyện và không thể huấn luyện. Ở đây chúng ta chỉ có các tham số có thể huấn luyện (bạn sẽ thấy một số tham số không thể huấn luyện sau trong chương này):

```
>>> model.summary()
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
dense (Dense)                (None, 300)               235500
dense_1 (Dense)              (None, 100)               30100
dense_2 (Dense)              (None, 10)                1010
=================================================================
Total params: 266,610
Trainable params: 266,610
```

```
Non-trainable params: 0
_________________________________________________________________
```

Lưu ý rằng các lớp Dense thường có rất nhiều tham số. Ví dụ, lớp ẩn đầu tiên có 784 × 300 trọng số kết nối, cộng với 300 số hạng độ lệch, tổng cộng là 235.500 tham số! Điều này mang lại cho mô hình khá nhiều sự linh hoạt để phù hợp với dữ liệu huấn luyện, nhưng nó cũng có nghĩa là mô hình có nguy cơ bị overfitting, đặc biệt là khi bạn không có nhiều dữ liệu huấn luyện. Chúng ta sẽ quay lại vấn đề này sau.

Mỗi lớp trong một mô hình phải có một tên duy nhất (ví dụ: “dense_2”). Bạn có thể đặt tên lớp một cách rõ ràng bằng cách sử dụng đối số `name` của hàm khởi tạo, nhưng nhìn chung, việc để Keras tự động đặt tên cho các lớp sẽ đơn giản hơn, như chúng ta vừa làm. Keras lấy tên lớp và chuyển đổi nó thành snake case (ví dụ: một lớp từ lớp `MyCoolLayer` được đặt tên là “my_cool_layer” theo mặc định). Keras cũng đảm bảo rằng tên là duy nhất trên toàn cầu, ngay cả giữa các mô hình, bằng cách thêm một chỉ số nếu cần, như trong “dense_2”. Nhưng tại sao nó lại bận tâm đến việc tạo các tên duy nhất giữa các mô hình? Chà, điều này giúp hợp nhất các mô hình dễ dàng mà không gặp xung đột tên.

Bạn có thể dễ dàng lấy danh sách các lớp của một mô hình bằng thuộc tính `layers`, hoặc sử dụng phương thức `get_layer()` để truy cập một lớp theo tên:

```
>>> model.layers
[<keras.layers.core.flatten.Flatten at 0x7fa1dea02250>,
<keras.layers.core.dense.Dense at 0x7fa1c8f42520>,
<keras.layers.core.dense.Dense at 0x7fa188be7ac0>,
<keras.layers.core.dense.dense.Dense at 0x7fa188be7fa0>]

>>> hidden1 = model.layers[1]

>>> hidden1.name
'dense'

>>> model.get_layer('dense') is hidden1
True
```

Tất cả các tham số của một lớp có thể được truy cập bằng các phương thức `get_weights()` và `set_weights()` của nó. Đối với một lớp Dense, điều này bao gồm cả trọng số kết nối và số hạng độ lệch:

```
>>> weights, biases = hidden1.get_weights()

>>> weights
array([[ 0.02448617, -0.00877795, -0.02189048, ...,  0.03859074,
-0.06889391],
[ 0.00476504, -0.03105379, -0.0586676 , ..., -0.02763776,
-0.04165364],

...,
```

```
[ 0.07061854, -0.06960931,  0.07038955, ..., 0.00034875,
0.02878492],
[-0.06022581,  0.01577859, -0.02585464, ..., 0.00272203,
-0.06793761]],
dtype=float32)

>>> weights.shape
(784, 300)

>>> biases
array([0., 0., 0., 0., 0., 0., 0., 0., 0., ...,  0., 0., 0.],
dtype=float32)

>>> biases.shape
(300,)
```

Lưu ý rằng lớp Dense đã khởi tạo trọng số kết nối một cách ngẫu nhiên (điều này cần thiết để phá vỡ tính đối xứng, như đã thảo luận trước đó), và các độ lệch được khởi tạo bằng 0, điều này là ổn. Nếu bạn muốn sử dụng một phương pháp khởi tạo khác, bạn có thể đặt `kernel_initializer` (`kernel` là một tên khác cho ma trận trọng số kết nối) hoặc `bias_initializer` khi tạo lớp. Chúng ta sẽ thảo luận chi tiết hơn về các trình khởi tạo trong Chương 11, và danh sách đầy đủ có tại https://keras.io/api/layers/initializers .

Biên dịch mô hình

Sau khi tạo một mô hình, bạn phải gọi phương thức `compile()` của nó để chỉ định hàm mất mát và bộ tối ưu hóa cần sử dụng. Tùy chọn, bạn có thể chỉ định một danh sách các số liệu bổ sung để tính toán trong quá trình huấn luyện và đánh giá:

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd",
metrics=["accuracy"])
```

Đoạn mã này cần được giải thích. Chúng ta sử dụng mất mát “sparse_categorical_crossentropy” vì chúng ta có các nhãn thưa thớt (tức là, đối với mỗi trường hợp, chỉ có một chỉ số lớp mục tiêu, từ 0 đến 9 trong trường hợp này), và các lớp là độc quyền. Nếu thay vào đó chúng ta có một xác suất mục tiêu cho mỗi lớp cho mỗi trường hợp (chẳng hạn như các vector one-hot, ví dụ: `[0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]` để đại diện cho lớp 3), thì chúng ta sẽ cần sử dụng mất mát “categorical_crossentropy” thay thế. Nếu chúng ta đang thực hiện phân loại nhị phân hoặc phân loại nhị phân đa nhãn, thì chúng ta sẽ sử dụng hàm kích hoạt “sigmoid” trong lớp đầu ra thay vì hàm kích hoạt “softmax”, và chúng ta sẽ sử dụng mất mát “binary_crossentropy”.

Về bộ tối ưu hóa, “sgd” có nghĩa là chúng ta sẽ huấn luyện mô hình bằng cách sử dụng giảm độ dốc ngẫu nhiên (stochastic gradient descent). Nói cách khác, Keras sẽ thực hiện thuật toán backpropagation được mô tả trước đó (tức là, tự động đạo hàm ngược cộng với giảm độ dốc). Chúng ta sẽ thảo luận về các bộ tối ưu hóa hiệu quả hơn trong Chương 11. Chúng cải thiện giảm độ dốc, không phải tự động đạo hàm.

Cuối cùng, vì đây là một bộ phân loại, việc đo lường độ chính xác của nó trong quá trình huấn luyện và đánh giá là hữu ích, đó là lý do tại sao chúng ta đặt `metrics=["accuracy"]`.

Huấn luyện và đánh giá mô hình

Bây giờ mô hình đã sẵn sàng để được huấn luyện. Để làm điều này, chúng ta chỉ cần gọi phương thức `fit()` của nó:

```
>>> history = model.fit(X_train, y_train, epochs=30,
...                                 validation_data=(X_valid, y_valid))
...
Epoch 1/30
1719/1719 [==============================] - 2s 989us/step
-  loss: 0.7220 - sparse_categorical_accuracy: 0.7649
-  val_loss: 0.4959 - val_sparse_categorical_accuracy: 0.8332

Epoch 2/30
1719/1719 [==============================] - 2s 964us/step
-  loss: 0.4825 - sparse_categorical_accuracy: 0.8332
    -  val_loss: 0.4567 - val_sparse_categorical_accuracy: 0.8384
[...]
Epoch 30/30
1719/1719 [==============================] - 2s 963us/step
-  loss: 0.2235 - sparse_categorical_accuracy: 0.9200
-  val_loss: 0.3056 - val_sparse_categorical_accuracy: 0.8894
```

Chúng ta truyền cho nó các đặc trưng đầu vào (`X_train`) và các lớp mục tiêu (`y_train`), cũng như số epoch để huấn luyện (nếu không, nó sẽ mặc định chỉ là 1, điều này chắc chắn sẽ không đủ để hội tụ đến một giải pháp tốt). Chúng ta cũng truyền một tập xác thực (tùy chọn). Keras sẽ đo mất mát và các số liệu bổ sung trên tập này vào cuối mỗi epoch, điều này rất hữu ích để xem mô hình thực sự hoạt động tốt như thế nào. Nếu hiệu suất trên tập huấn luyện tốt hơn nhiều so với trên tập xác thực, mô hình của bạn có thể đang bị overfitting trên tập huấn luyện, hoặc có một lỗi, chẳng hạn như dữ liệu không khớp giữa tập huấn luyện và tập xác thực.

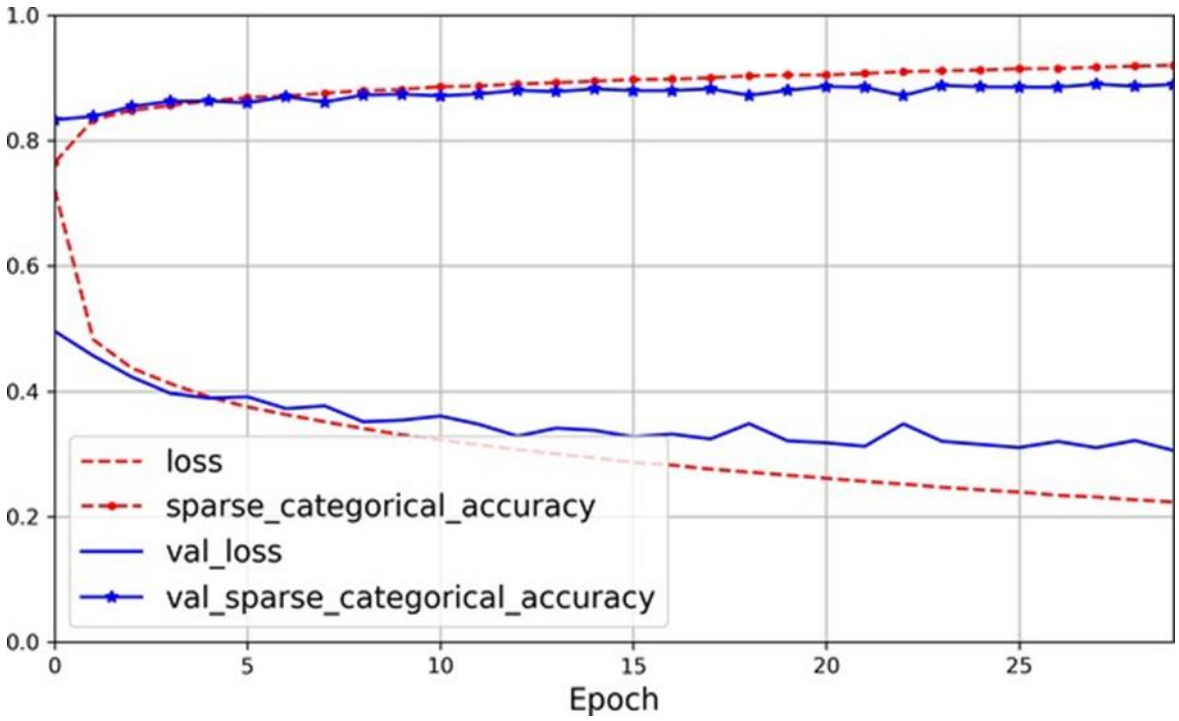
Và đó là tất cả! Mạng nơ-ron đã được huấn luyện. Ở mỗi epoch trong quá trình huấn luyện, Keras hiển thị số lượng mini-batch đã được xử lý ở phía bên trái của thanh tiến trình. Kích thước batch mặc định là 32, và vì tập huấn luyện có 55.000 hình ảnh, mô hình đi qua 1.719 batch mỗi epoch: 1.718 batch có kích thước 32 và 1 batch có kích thước 24. Sau thanh tiến trình, bạn có thể thấy thời gian huấn luyện trung bình cho mỗi mẫu, và mất mát và độ chính xác (hoặc bất kỳ số liệu bổ sung nào khác mà bạn yêu cầu) trên cả tập huấn luyện và tập xác thực. Lưu ý rằng mất mát huấn luyện đã giảm xuống, đây là một dấu hiệu tốt, và độ chính xác xác thực đạt 88,94% sau 30 epoch. Con số này hơi thấp hơn độ chính xác huấn luyện, vì vậy có một chút overfitting xảy ra, nhưng không đáng kể.

Nếu tập huấn luyện bị lệch nhiều, với một số lớp được biểu diễn quá mức và những lớp khác bị biểu diễn dưới mức, sẽ hữu ích nếu đặt đối số `class_weight` khi gọi phương thức `fit()`, để gán trọng số lớn hơn cho các lớp bị biểu diễn dưới mức và trọng số thấp hơn cho

các lớp được biểu diễn quá mức. Các trọng số này sẽ được Keras sử dụng khi tính toán mất mát. Nếu bạn cần trọng số cho mỗi trường hợp, hãy đặt đối số `sample_weight`. Nếu cả `class_weight` và `sample_weight` đều được cung cấp, thì Keras sẽ nhân chúng. Trọng số cho mỗi trường hợp có thể hữu ích, ví dụ, nếu một số trường hợp được dán nhãn bởi các chuyên gia trong khi những trường hợp khác được dán nhãn bằng nền tảng crowdsourcing: bạn có thể muốn gán trọng số lớn hơn cho những trường hợp trước. Bạn cũng có thể cung cấp trọng số mẫu (nhưng không phải trọng số lớp) cho tập xác thực bằng cách thêm chúng làm mục thứ ba trong tuple `validation_data`.

Phương thức `fit()` trả về một đối tượng `History` chứa các tham số huấn luyện (`history.params`), danh sách các epoch nó đã trải qua (`history.epoch`), và quan trọng nhất là một từ điển (`history.history`) chứa mất mát và các số liệu bổ sung mà nó đã đo được vào cuối mỗi epoch trên tập huấn luyện và trên tập xác thực (nếu có). Nếu bạn sử dụng từ điển này để tạo một Pandas DataFrame và gọi phương thức `plot()` của nó, bạn sẽ nhận được các đường cong học tập được hiển thị trong Hình 10-11:

```
import pandas as pd
import matplotlib.pyplot as plt

pd.DataFrame(history.history).plot(
    figsize=(8, 5), xlim=[0, 29], ylim=[0, 1], grid=True, xlabel="Epoch",
    style=["r--", "r--.", "b-", "b-*"])
plt.show()
```



Hình 10-11. Các đường cong học tập: mất mát huấn luyện trung bình và độ chính xác được đo qua mỗi epoch, và mất mát xác thực trung bình và độ chính xác được đo vào cuối mỗi epoch

Bạn có thể thấy rằng cả độ chính xác huấn luyện và độ chính xác xác thực đều tăng đều đặn trong quá trình huấn luyện, trong khi mất mát huấn luyện và mất mát xác thực giảm. Đây là một dấu hiệu tốt. Các đường cong xác thực ban đầu tương đối gần nhau, nhưng sau đó chúng ngày càng cách xa nhau, điều này cho thấy có một chút overfitting. Trong trường hợp cụ thể này, mô hình trông có vẻ hoạt động tốt hơn trên tập xác thực so với trên tập huấn luyện khi bắt đầu huấn luyện, nhưng điều đó thực sự không đúng. Lỗi xác thực được tính vào cuối mỗi epoch, trong khi lỗi huấn luyện được tính bằng cách sử dụng trung bình di chuyển trong mỗi epoch, vì vậy đường cong huấn luyện nên được dịch sang trái nửa epoch. Nếu bạn làm điều đó, bạn sẽ thấy rằng các đường cong huấn luyện và xác thực chồng lên nhau gần như hoàn hảo khi bắt đầu huấn luyện.

Hiệu suất của tập huấn luyện cuối cùng vượt qua hiệu suất xác thực, như thường lệ khi bạn huấn luyện đủ lâu. Bạn có thể thấy rằng mô hình chưa hội tụ hoàn toàn, vì mất mát xác thực vẫn đang giảm, vì vậy bạn có lẽ nên tiếp tục huấn luyện. Điều này đơn giản như việc gọi lại phương thức `fit()`, vì Keras chỉ tiếp tục huấn luyện từ nơi nó đã dừng lại: bạn sẽ có thể đạt được độ chính xác xác thực khoảng 89,8%, trong khi độ chính xác huấn luyện sẽ tiếp tục tăng lên đến 100% (điều này không phải lúc nào cũng xảy ra).

Nếu bạn không hài lòng với hiệu suất của mô hình, bạn nên quay lại và tinh chỉnh các siêu tham số. Điều đầu tiên cần kiểm tra là tốc độ học. Nếu điều đó không giúp ích, hãy thử một bộ tối ưu hóa khác (và luôn tinh chỉnh lại tốc độ học sau khi thay đổi bất kỳ siêu tham số nào). Nếu hiệu suất vẫn không tốt, thì hãy thử tinh chỉnh các siêu tham số mô hình như số lớp, số nơ-ron trên mỗi lớp và loại hàm kích hoạt để sử dụng cho mỗi lớp ẩn. Bạn cũng có thể thử tinh chỉnh các siêu tham số khác, chẳng hạn như kích thước batch (nó có thể được đặt trong phương thức `fit()` bằng cách sử dụng đối số `batch_size`, mặc định là 32). Chúng ta sẽ quay lại việc tinh chỉnh siêu tham số vào cuối chương này. Khi bạn hài lòng với độ chính xác xác thực của mô hình, bạn nên đánh giá nó trên tập kiểm tra để ước tính lỗi tổng quát hóa trước khi bạn triển khai mô hình vào sản xuất. Bạn có thể dễ dàng làm điều này bằng cách sử dụng phương thức `evaluate()` (nó cũng hỗ trợ một số đối số khác, chẳng hạn như `batch_size` và `sample_weight`; vui lòng kiểm tra tài liệu để biết thêm chi tiết):

```
>>> model.evaluate(X_test, y_test)
313/313 [==============================] - 0s 626us/step
- loss: 0.3243 - sparse_categorical_accuracy: 0.8864
[0.32431697845458984, 0.8863999843597412]
```

Như bạn đã thấy trong Chương 2, việc đạt được hiệu suất hơi thấp hơn trên tập kiểm tra so với trên tập xác thực là điều phổ biến, bởi vì các siêu tham số được tinh chỉnh trên tập xác thực, không phải tập kiểm tra (tuy nhiên, trong ví dụ này, chúng ta không thực hiện bất kỳ tinh chỉnh siêu tham số nào, vì vậy độ chính xác thấp hơn chỉ là xui xẻo). Hãy nhớ chống lại sự cám dỗ điều chỉnh các siêu tham số trên tập kiểm tra, nếu không ước tính lỗi tổng quát hóa của bạn sẽ quá lạc quan.

Sử dụng mô hình để đưa ra dự đoán

Bây giờ hãy sử dụng phương thức `predict()` của mô hình để đưa ra dự đoán trên các thể hiện mới. Vì chúng ta không có các thể hiện mới thực tế, chúng ta sẽ chỉ sử dụng ba thể hiện đầu tiên của tập kiểm tra:

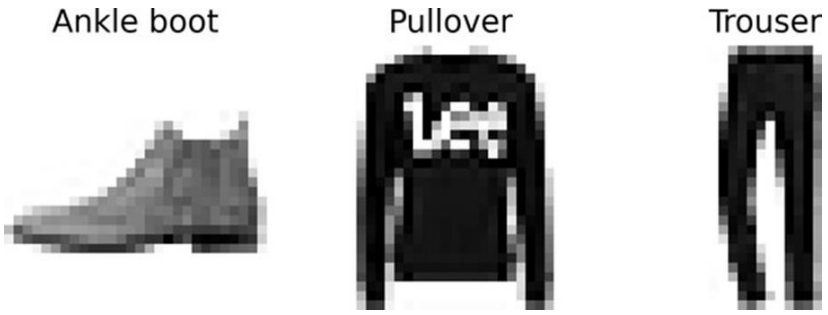
```
>>> X_new = X_test[:3]

>>> y_proba = model.predict(X_new)

>>> y_proba.round(2)
array([[0. , 0. , 0. , 0. , 0. , 0.01, 0. , 0.02, 0. , 0.97],

  [0. , 0. , 0.99, 0. , 0.01, 0. , 0. , 0. , 0. , 0. ],

  [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]],
  dtype=float32)
```

Đối với mỗi thể hiện, mô hình ước tính một xác suất cho mỗi lớp, từ lớp 0 đến lớp 9. Điều này tương tự như đầu ra của phương thức `predict_proba()` trong các bộ phân loại Scikit-Learn. Ví dụ, đối với hình ảnh đầu tiên, nó ước tính rằng xác suất của lớp 9 (ủng cổ chân) là 96%, xác suất của lớp 7 (giày thể thao) là 2%, xác suất của lớp 5 (dép) là 1%, và xác suất của các lớp khác là không đáng kể. Nói cách khác, nó rất tự tin rằng hình ảnh đầu tiên là giày dép, nhiều khả năng là ủng cổ chân nhưng cũng có thể là giày thể thao hoặc dép. Nếu bạn chỉ quan tâm đến lớp có xác suất ước tính cao nhất (ngay cả khi xác suất đó khá thấp), thì bạn có thể sử dụng phương thức `argmax()` để lấy chỉ số lớp có xác suất cao nhất cho mỗi thể hiện:

```
>>> import numpy as np

>>> y_pred = y_proba.argmax(axis=-1)

>>> y_pred
array([9, 2, 1])

>>> np.array(class_names)[y_pred]
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')
```

Ở đây, bộ phân loại thực sự đã phân loại đúng cả ba hình ảnh (các hình ảnh này được hiển thị trong Hình 10-12):

```
>>> y_new = y_test[:3]

>>> y_new
array([9, 2, 1], dtype=uint8)
```



Hình 10-12. Các hình ảnh Fashion MNIST được phân loại đúng

Bây giờ bạn đã biết cách sử dụng API tuần tự để xây dựng, huấn luyện và đánh giá một MLP phân loại. Nhưng còn hồi quy thì sao?

Xây dựng một MLP hồi quy bằng API tuần tự

Hãy quay lại vấn đề nhà ở California và giải quyết nó bằng cách sử dụng cùng một MLP như trước, với 3 lớp ẩn gồm 50 nơ-ron mỗi lớp, nhưng lần này xây dựng nó bằng Keras.

Sử dụng API tuần tự để xây dựng, huấn luyện, đánh giá và sử dụng một MLP hồi quy khá giống với những gì chúng ta đã làm để phân loại. Sự khác biệt chính trong ví dụ mã sau đây là việc lớp đầu ra chỉ có một nơ-ron (vì chúng ta chỉ muốn dự đoán một giá trị duy nhất) và nó không sử dụng hàm kích hoạt, hàm mất mát là lỗi bình phương trung bình, số liệu là RMSE, và chúng ta đang sử dụng bộ tối ưu hóa Adam giống như `MLPRegressor` của Scikit-Learn đã làm. Hơn nữa, trong ví dụ này, chúng ta không cần một lớp Flatten, và thay vào đó chúng ta đang sử dụng một lớp Normalization làm lớp đầu tiên: nó thực hiện cùng một việc như `StandardScaler` của Scikit-Learn, nhưng nó phải được điều chỉnh cho dữ liệu huấn luyện bằng phương thức `adapt()` của nó trước khi bạn gọi phương thức `fit()` của mô hình. (Keras có các lớp tiền xử lý khác, sẽ được đề cập trong Chương 13).

Hãy cùng xem:

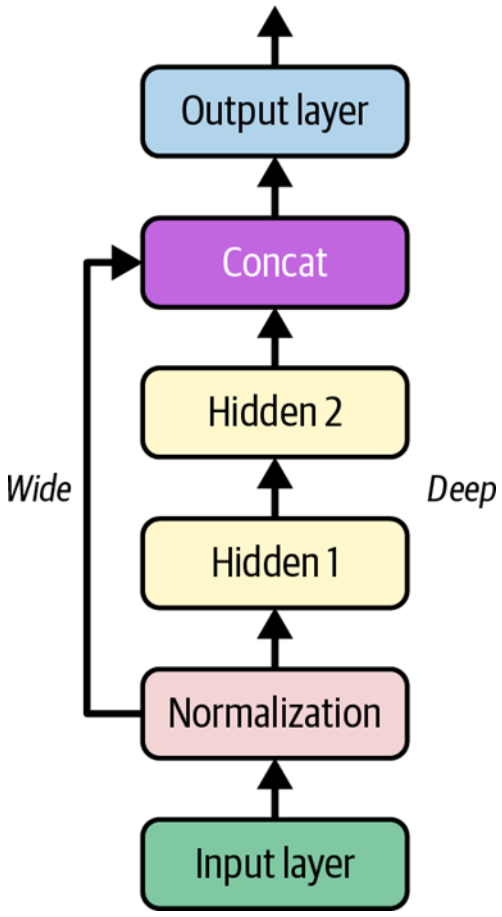
```
tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization(input_shape=X_train.shape[1:])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer,
metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
history = model.fit(X_train, y_train, epochs=20,
    validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)
```

```
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

Như bạn có thể thấy, API tuần tự khá sạch sẽ và đơn giản. Tuy nhiên, mặc dù các mô hình Sequential cực kỳ phổ biến, đôi khi việc xây dựng mạng nơ-ron với cấu trúc phức tạp hơn, hoặc với nhiều đầu vào hoặc đầu ra, là hữu ích. Với mục đích này, Keras cung cấp API chức năng.

Xây dựng mô hình phức tạp bằng API chức năng

Một ví dụ về mạng nơ-ron không tuần tự là mạng nơ-ron Rộng & Sâu (Wide & Deep). Kiến trúc mạng nơ-ron này được giới thiệu trong một bài báo năm 2016 của Heng-Tze Cheng và cộng sự. Nó kết nối tất cả hoặc một phần các đầu vào trực tiếp với lớp đầu ra, như thể hiện trong Hình 10-13. Kiến trúc này cho phép mạng nơ-ron học cả các mẫu sâu (sử dụng đường dẫn sâu) và các quy tắc đơn giản (thông qua đường dẫn ngắn). Ngược lại, một MLP thông thường buộc tất cả dữ liệu phải chảy qua toàn bộ chồng lớp; do đó, các mẫu đơn giản trong dữ liệu có thể bị biến dạng bởi chuỗi các phép biến đổi này.



Hình 10-13. Mạng nơ-ron Rộng & Sâu

Hãy xây dựng một mạng nơ-ron như vậy để giải quyết bài toán nhà ở California:

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

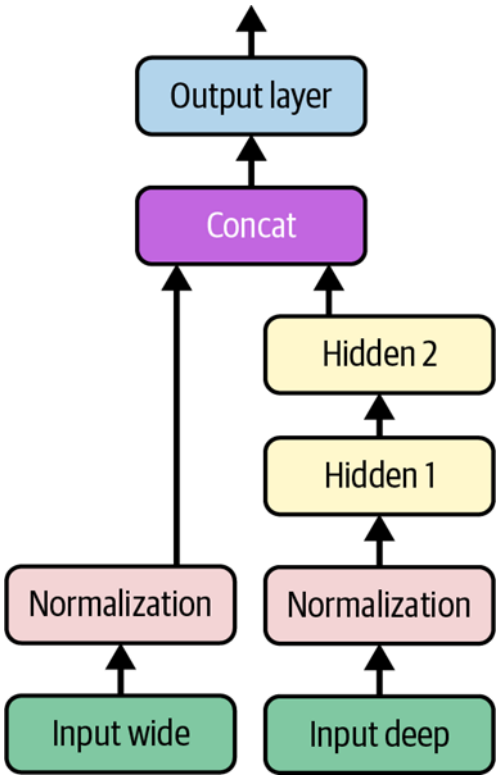
Về cơ bản, năm dòng đầu tiên tạo tất cả các lớp chúng ta cần để xây dựng mô hình, sáu dòng tiếp theo sử dụng các lớp này giống như các hàm để đi từ đầu vào đến đầu ra, và dòng cuối cùng tạo một đối tượng Keras `Model` bằng cách trỏ đến đầu vào và đầu ra. Hãy xem chi tiết hơn đoạn mã này:

- Đầu tiên, chúng ta tạo năm lớp: một lớp `Normalization` để chuẩn hóa đầu vào, hai lớp `Dense` với 30 nơ-ron mỗi lớp, sử dụng hàm kích hoạt ReLU, một lớp `Concatenate`, và một lớp `Dense` nữa với một nơ-ron duy nhất cho lớp đầu ra, không có bất kỳ hàm kích hoạt nào.
- Tiếp theo, chúng ta tạo một đối tượng `Input` (tên biến `input_` được sử dụng để tránh che khuất hàm `input()` tích hợp sẵn của Python). Đây là một đặc tả về loại đầu vào mà mô hình sẽ nhận được, bao gồm hình dạng của nó và tùy chọn kiểu dữ liệu của nó, mặc định là số thập phân 32 bit. Một mô hình thực sự có thể có nhiều đầu vào, như bạn sẽ thấy ngay sau đây.
- Sau đó, chúng ta sử dụng lớp `Normalization` giống như một hàm, truyền đối tượng `Input` cho nó. Đây là lý do tại sao nó được gọi là API chức năng. Lưu ý rằng chúng ta chỉ đang nói với Keras cách nó nên kết nối các lớp với nhau; chưa có dữ liệu thực tế nào được xử lý, vì đối tượng `Input` chỉ là một đặc tả dữ liệu. Nói cách khác, đó là một đầu vào mang tính biểu tượng. Đầu ra của lệnh gọi này cũng mang tính biểu tượng: `normalized` không lưu trữ bất kỳ dữ liệu thực tế nào, nó chỉ được sử dụng để xây dựng mô hình.
- Tương tự, chúng ta sau đó truyền `normalized` cho `hidden_layer1`, cái này xuất ra `hidden1`, và chúng ta truyền `hidden1` cho `hidden_layer2`, cái này xuất ra `hidden2`.
- Cho đến nay, chúng ta đã kết nối các lớp một cách tuần tự, nhưng sau đó chúng ta sử dụng `concat_layer` để nối đầu vào và đầu ra của lớp ẩn thứ hai. Một lần nữa, chưa có dữ liệu thực tế nào được nối: tất cả đều mang tính biểu tượng, để xây dựng mô hình.
- Sau đó, chúng ta truyền `concat` cho `output_layer`, cái này cho chúng ta đầu ra cuối cùng.
- Cuối cùng, chúng ta tạo một Keras `Model`, chỉ định đầu vào và đầu ra cần sử dụng.

Khi bạn đã xây dựng mô hình Keras này, mọi thứ đều giống hệt như trước, vì vậy không cần lặp lại ở đây: bạn biên dịch mô hình, điều chỉnh lớp `Normalization`, huấn luyện mô hình, đánh giá nó và sử dụng nó để đưa ra dự đoán.

Nhưng điều gì sẽ xảy ra nếu bạn muốn gửi một tập hợp con các đặc trưng qua đường dẫn rộng và một tập hợp con khác (có thể chồng chéo) qua đường dẫn sâu, như minh họa trong Hình 10-14? Trong trường hợp này, một giải pháp là sử dụng nhiều đầu vào. Ví dụ, giả sử chúng ta muốn gửi năm đặc trưng qua đường dẫn rộng (đặc trưng 0 đến 4), và sáu đặc trưng qua đường dẫn sâu (đặc trưng 2 đến 7). Chúng ta có thể làm điều này như sau:

```
input_wide = tf.keras.layers.Input(shape=[5]) # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6]) # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()

norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])

output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])
```



Hình 10-14. Xử lý nhiều đầu vào

Có một vài điều cần lưu ý trong ví dụ này, so với ví dụ trước:

- Mỗi lớp `Dense` được tạo và gọi trên cùng một dòng. Đây là một thực hành phổ biến, vì nó làm cho mã ngắn gọn hơn mà không mất đi sự rõ ràng. Tuy nhiên, chúng ta không thể làm điều này với lớp `Normalization` vì chúng ta cần một tham chiếu đến lớp để có thể gọi phương thức `adapt()` của nó trước khi huấn luyện mô hình.
- Chúng ta đã sử dụng `tf.keras.layers.concatenate()`, cái này tạo một lớp `Concatenate` và gọi nó với các đầu vào đã cho.
- Chúng ta đã chỉ định `inputs=[input_wide, input_deep]` khi tạo mô hình, vì có hai đầu vào.

Bây giờ chúng ta có thể biên dịch mô hình như bình thường, nhưng khi chúng ta gọi phương thức `fit()`, thay vì truyền một ma trận đầu vào `X_train` duy nhất, chúng ta phải truyền một cặp ma trận (`X_train_wide`, `X_train_deep`), một cho mỗi đầu vào. Điều tương tự cũng đúng với `X_valid`, và cả `X_test` và `X_new` khi bạn gọi `evaluate()` hoặc `predict()`:

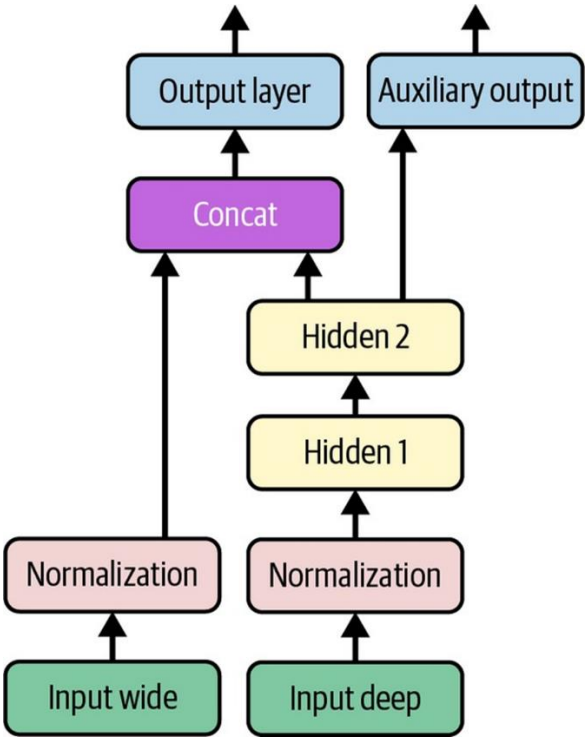
```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer,
metrics=["RootMeanSquaredError"])

X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit(
    (X_train_wide, X_train_deep),
    y_train,
    epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), y_valid)
)
mse_test = model.evaluate((X_test_wide, X_test_deep), y_test)
y_pred = model.predict((X_new_wide, X_new_deep))
```

Cũng có nhiều trường hợp sử dụng mà bạn có thể muốn có nhiều đầu ra:

- Tác vụ có thể yêu cầu điều đó. Ví dụ, bạn có thể muốn định vị và phân loại đối tượng chính trong một bức tranh. Đây vừa là tác vụ hồi quy vừa là tác vụ phân loại.
- Tương tự, bạn có thể có nhiều tác vụ độc lập dựa trên cùng một dữ liệu. Chắc chắn, bạn có thể huấn luyện một mạng nơ-ron cho mỗi tác vụ, nhưng trong nhiều trường hợp, bạn sẽ nhận được kết quả tốt hơn trên tất cả các tác vụ bằng cách huấn luyện một mạng nơ-ron duy nhất với một đầu ra cho mỗi tác vụ. Điều này là do mạng nơ-ron có thể học các đặc trưng trong dữ liệu hữu ích trên các tác vụ. Ví dụ, bạn có thể thực hiện phân loại đa tác vụ trên các bức ảnh khuôn mặt, sử dụng một đầu ra để

phân loại biểu cảm khuôn mặt của người đó (cười, ngạc nhiên, v.v.) và một đầu ra khác để xác định xem họ có đeo kính hay không.

- Một trường hợp sử dụng khác là một kỹ thuật chính quy hóa (nghĩa là, một ràng buộc huấn luyện với mục tiêu là giảm overfitting và do đó cải thiện khả năng tổng quát hóa của mô hình). Ví dụ, bạn có thể muốn thêm một đầu ra phụ trong kiến trúc mạng nơ-ron (xem Hình 10-15) để đảm bảo rằng phần cơ bản của mạng học được điều gì đó hữu ích một cách độc lập, mà không phụ thuộc vào phần còn lại của mạng.



Hình 10-15. Xử lý nhiều đầu ra, trong ví dụ này để thêm một đầu ra phụ để chính quy hóa

Thêm một đầu ra bổ sung khá dễ dàng: chúng ta chỉ cần kết nối nó với lớp thích hợp và thêm nó vào danh sách các đầu ra của mô hình. Ví dụ, đoạn mã sau xây dựng mạng được biểu diễn trong Hình 10-15:

```
# ... (Same as above, up to the main output layer)
output = tf.keras.layers.Dense(1)(concat)
aux_output = tf.keras.layers.Dense(1)(hidden2)

model = tf.keras.Model(inputs=[input_wide, input_deep],
                       outputs=[output, aux_output])
```

Mỗi đầu ra sẽ cần hàm mất mát riêng của nó. Do đó, khi chúng ta biên dịch mô hình, chúng ta nên truyền một danh sách các hàm mất mát. Nếu chúng ta truyền một hàm mất mát duy nhất, Keras sẽ giả định rằng cùng một hàm mất mát phải được sử dụng cho tất cả các đầu

ra. Theo mặc định, Keras sẽ tính toán tất cả các hàm mất mát và đơn giản là cộng chúng lại để có được tổng mất mát cuối cùng được sử dụng để huấn luyện. Vì chúng ta quan tâm nhiều hơn đến đầu ra chính hơn là đầu ra phụ (vì nó chỉ được sử dụng để chính quy hóa), chúng ta muốn gán trọng số lớn hơn nhiều cho mất mát của đầu ra chính. May mắn thay, có thể đặt tất cả các trọng số mất mát khi biên dịch mô hình:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss=("mse", "mse"), loss_weights=(0.9, 0.1),
optimizer=optimizer,
              metrics=["RootMeanSquaredError"])
```

Bây giờ khi chúng ta huấn luyện mô hình, chúng ta cần cung cấp nhãn cho mỗi đầu ra. Trong ví dụ này, đầu ra chính và đầu ra phụ nên cố gắng dự đoán cùng một thứ, vì vậy chúng nên sử dụng cùng một nhãn. Vì vậy, thay vì truyền `y_train`, chúng ta cần truyền (`y_train`, `y_train`), hoặc một từ điển `{"output": y_train, "aux_output": y_train}` nếu các đầu ra được đặt tên là “output” và “aux_output”. Điều tương tự cũng áp dụng cho `y_valid` và `y_test`:

```
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit(
    (X_train_wide, X_train_deep),
    (y_train, y_train),
    epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid))
)
```

Khi chúng ta đánh giá mô hình, Keras trả về tổng trọng số của các hàm mất mát, cũng như tất cả các hàm mất mát và số liệu riêng lẻ:

```
eval_results = model.evaluate((X_test_wide, X_test_deep), (y_test, y_test))
weighted_sum_of_losses, main_loss, aux_loss, main_rmse, aux_rmse =
eval_results
```

Tương tự, phương thức `predict()` sẽ trả về dự đoán cho mỗi đầu ra:

```
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Phương thức `predict()` trả về một tuple, và nó không có đối số `return_dict` để thay vào đó lấy một từ điển. Tuy nhiên, bạn có thể tạo một từ điển bằng cách sử dụng `model.output_names`:

```
y_pred_tuple = model.predict((X_new_wide, X_new_deep))
y_pred = dict(zip(model.output_names, y_pred_tuple))
```

Như bạn có thể thấy, bạn có thể xây dựng tất cả các loại kiến trúc với API chức năng. Tiếp theo, chúng ta sẽ xem xét một cách cuối cùng để bạn có thể xây dựng các mô hình Keras.

Sử dụng Subclassing API để xây dựng mô hình động

Cả API tuần tự và API chức năng đều mang tính khai báo: bạn bắt đầu bằng cách khai báo các lớp bạn muốn sử dụng và cách chúng nên được kết nối, và sau đó bạn mới có thể bắt đầu cấp dữ liệu cho mô hình để huấn luyện hoặc suy luận. Điều này có nhiều lợi thế: mô hình có thể dễ dàng được lưu, sao chép và chia sẻ; cấu trúc của nó có thể được hiển thị và phân tích; framework có thể suy ra hình dạng và kiểm tra kiểu, do đó các lỗi có thể được phát hiện sớm (tức là trước khi bất kỳ dữ liệu nào đi qua mô hình). Việc gỡ lỗi cũng khá đơn giản, vì toàn bộ mô hình là một biểu đồ tĩnh của các lớp. Nhưng mặt trái là: nó tĩnh. Một số mô hình liên quan đến các vòng lặp, hình dạng thay đổi, phân nhánh có điều kiện và các hành vi động khác. Đối với những trường hợp như vậy, hoặc đơn giản nếu bạn thích một phong cách lập trình mệnh lệnh hơn, API subclassing là dành cho bạn.

Với cách tiếp cận này, bạn kế thừa từ lớp `Model`, tạo các lớp bạn cần trong hàm tạo, và sử dụng chúng để thực hiện các phép tính bạn muốn trong phương thức `call()`. Ví dụ, tạo một thể hiện của lớp `WideAndDeepModel` sau đây cho chúng ta một mô hình tương đương với mô hình chúng ta vừa xây dựng bằng API chức năng:

```
class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs) # needed to support naming the model
        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)
        self.main_output = tf.keras.layers.Dense(1)
        self.aux_output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])
        output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return output, aux_output

model = WideAndDeepModel(30, activation="relu", name="my_cool_model")
```

Ví dụ này trông giống như ví dụ trước, ngoại trừ việc chúng ta tách biệt việc tạo các lớp trong hàm tạo khỏi việc sử dụng chúng trong phương thức `call()`. Và chúng ta không cần tạo các đối tượng `Input`: chúng ta có thể sử dụng đối số `input` cho phương thức `call()`.

Bây giờ chúng ta đã có một thể hiện mô hình, chúng ta có thể biên dịch nó, điều chỉnh các lớp chuẩn hóa của nó (ví dụ: sử dụng `model.norm_layer_wide.adapt(...)` và

`model.norm_layer_deep.adapt(...)`), huấn luyện nó, đánh giá nó và sử dụng nó để đưa ra dự đoán, chính xác như chúng ta đã làm với API chức năng.

Sự khác biệt lớn với API này là bạn có thể đưa vào phương thức `call()` bất cứ thứ gì bạn muốn: các vòng lặp `for`, các câu lệnh `if`, các phép toán TensorFlow cấp thấp – trí tưởng tượng của bạn là giới hạn (xem Chương 12)! Điều này làm cho nó trở thành một API tuyệt vời khi thử nghiệm các ý tưởng mới, đặc biệt đối với các nhà nghiên cứu. Tuy nhiên, sự linh hoạt bổ sung này đi kèm với một cái giá: kiến trúc mô hình của bạn bị ẩn trong phương thức `call()`, vì vậy Keras không thể dễ dàng kiểm tra nó; mô hình không thể được sao chép bằng `tf.keras.models.clone_model()`; và khi bạn gọi phương thức `summary()`, bạn chỉ nhận được danh sách các lớp, không có bất kỳ thông tin nào về cách chúng được kết nối với nhau. Hơn nữa, Keras không thể kiểm tra kiểu và hình dạng trước, và dễ mắc lỗi hơn. Vì vậy, trừ khi bạn thực sự cần sự linh hoạt bổ sung đó, bạn nên gắn bó với API tuần tự hoặc API chức năng.

Bây giờ bạn đã biết cách xây dựng và huấn luyện mạng nơ-ron bằng Keras, bạn sẽ muốn lưu chúng!

Lưu và khôi phục mô hình

Lưu một mô hình Keras đã được huấn luyện đơn giản nhất có thể:

```
model.save("my_keras_model", save_format="tf")
```

Khi bạn đặt `save_format="tf"`, Keras lưu mô hình bằng định dạng SavedModel của TensorFlow: đây là một thư mục (với tên đã cho) chứa nhiều tệp và thư mục con. Cụ thể, tệp `saved_model.pb` chứa kiến trúc và logic của mô hình dưới dạng biểu đồ tính toán được tuần tự hóa, vì vậy bạn không cần triển khai mã nguồn của mô hình để sử dụng nó trong sản xuất; SavedModel là đủ (bạn sẽ thấy cách hoạt động này trong Chương 12). Tệp `keras_metadata.pb` chứa thông tin bổ sung cần thiết của Keras. Thư mục con `variables` chứa tất cả các giá trị tham số (bao gồm trọng số kết nối, độ lệch, thống kê chuẩn hóa và tham số của bộ tối ưu hóa), có thể được chia thành nhiều tệp nếu mô hình rất lớn. Cuối cùng, thư mục `assets` có thể chứa các tệp bổ sung, chẳng hạn như mẫu dữ liệu, tên đặc trưng, tên lớp, v.v. Theo mặc định, thư mục `assets` trống. Vì bộ tối ưu hóa cũng được lưu, bao gồm các siêu tham số của nó và bất kỳ trạng thái nào nó có thể có, sau khi tải mô hình, bạn có thể tiếp tục huấn luyện nếu muốn.

Bạn thường sẽ có một tập lệnh huấn luyện mô hình và lưu nó, và một hoặc nhiều tập lệnh (hoặc dịch vụ web) tải mô hình và sử dụng nó để đánh giá hoặc để đưa ra dự đoán. Tải mô hình cũng dễ dàng như lưu nó:

```
model = tf.keras.models.load_model("my_keras_model")
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Bạn cũng có thể sử dụng `save_weights()` và `load_weights()` để chỉ lưu và tải các giá trị tham số. Điều này bao gồm trọng số kết nối, độ lệch, thống kê tiền xử lý, trạng thái bộ tối ưu hóa, v.v. Các giá trị tham số được lưu trong một hoặc nhiều tệp như `my_weights.data-`

00004-of-00052, cộng với một tệp chỉ mục như my_weights.index. Lưu chỉ các trọng số nhanh hơn và sử dụng ít không gian đĩa hơn so với lưu toàn bộ mô hình, vì vậy nó hoàn hảo để lưu các điểm kiểm tra nhanh trong quá trình huấn luyện. Nếu bạn đang huấn luyện một mô hình lớn, và nó mất hàng giờ hoặc hàng ngày, thì bạn phải lưu các điểm kiểm tra thường xuyên trong trường hợp máy tính bị lỗi. Nhưng làm thế nào bạn có thể bảo phương thức fit() lưu các điểm kiểm tra? Sử dụng callback.

Sử dụng Callbacks

Phương thức fit() chấp nhận một đối số callbacks cho phép bạn chỉ định một danh sách các đối tượng mà Keras sẽ gọi trước và sau khi huấn luyện, trước và sau mỗi epoch, và thậm chí trước và sau khi xử lý mỗi batch. Ví dụ, callback ModelCheckpoint lưu các điểm kiểm tra của mô hình của bạn theo các khoảng thời gian đều đặn trong quá trình huấn luyện, theo mặc định vào cuối mỗi epoch:

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_checkpoints",
save_weights_only=True)
history = model.fit([...], callbacks=[checkpoint_cb])
```

Hơn nữa, nếu bạn sử dụng tập xác thực trong quá trình huấn luyện, bạn có thể đặt save_best_only=True khi tạo ModelCheckpoint. Trong trường hợp này, nó sẽ chỉ lưu mô hình của bạn khi hiệu suất của nó trên tập xác thực là tốt nhất cho đến nay. Bằng cách này, bạn không cần phải lo lắng về việc huấn luyện quá lâu và overfitting tập huấn luyện: chỉ cần khôi phục mô hình được lưu cuối cùng sau khi huấn luyện, và đây sẽ là mô hình tốt nhất trên tập xác thực. Đây là một cách để triển khai dừng sớm (được giới thiệu trong Chương 4), nhưng nó sẽ không thực sự dừng huấn luyện.

Một cách khác là sử dụng callback EarlyStopping. Nó sẽ làm gián đoạn quá trình huấn luyện khi nó không đo được tiến bộ nào trên tập xác thực trong một số epoch (được xác định bởi đối số patience), và nếu bạn đặt restore_best_weights=True, nó sẽ khôi phục lại mô hình tốt nhất vào cuối quá trình huấn luyện. Bạn có thể kết hợp cả hai callback để lưu các điểm kiểm tra của mô hình trong trường hợp máy tính của bạn bị lỗi, và làm gián đoạn quá trình huấn luyện sớm khi không còn tiến bộ nào nữa, để tránh lãng phí thời gian và tài nguyên và để giảm overfitting:

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,
restore_best_weights=True)
history = model.fit([...], callbacks=[checkpoint_cb, early_stopping_cb])
```

Số lượng epoch có thể được đặt thành một giá trị lớn vì quá trình huấn luyện sẽ tự động dừng khi không còn tiến bộ (chỉ cần đảm bảo tốc độ học không quá nhỏ, nếu không nó có thể tiếp tục tiến bộ chậm cho đến cuối). Callback EarlyStopping sẽ lưu trữ trọng số của mô hình tốt nhất trong RAM, và nó sẽ khôi phục chúng cho bạn vào cuối quá trình huấn luyện.

Nếu bạn cần kiểm soát thêm, bạn có thể dễ dàng viết các callback tùy chỉnh của riêng mình. Ví dụ, callback tùy chỉnh sau đây sẽ hiển thị tỷ lệ giữa mất mát xác thực và mất mát huấn luyện trong quá trình huấn luyện (ví dụ: để phát hiện overfitting):

```
class PrintValTrainRatioCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs):
        ratio = logs["val_loss"] / logs["loss"]
        print(f"Epoch={epoch}, val/train={ratio:.2f}")
```

Như bạn có thể mong đợi, bạn có thể triển khai `on_train_begin()`, `on_train_end()`, `on_epoch_begin()`, `on_epoch_end()`, `on_batch_begin()`, và `on_batch_end()`. Các callback cũng có thể được sử dụng trong quá trình đánh giá và dự đoán, nếu bạn cần chúng (ví dụ: để gỡ lỗi). Để đánh giá, bạn nên triển khai `on_test_begin()`, `on_test_end()`, `on_test_batch_begin()`, hoặc `on_test_batch_end()`, được gọi bởi `evaluate()`. Để dự đoán, bạn nên triển khai `on_predict_begin()`, `on_predict_end()`, `on_predict_batch_begin()`, hoặc `on_predict_batch_end()`, được gọi bởi `predict()`.

Bây giờ chúng ta hãy xem một công cụ nữa mà bạn chắc chắn nên có trong bộ công cụ của mình khi sử dụng Keras: TensorBoard.

Sử dụng TensorBoard để trực quan hóa

TensorBoard là một công cụ trực quan hóa tương tác tuyệt vời mà bạn có thể sử dụng để xem các đường cong học tập trong quá trình huấn luyện, so sánh các đường cong và số liệu giữa nhiều lần chạy, trực quan hóa biểu đồ tính toán, phân tích thống kê huấn luyện, xem hình ảnh được tạo bởi mô hình của bạn, trực quan hóa dữ liệu đa chiều phức tạp được chiếu xuống 3D và tự động gom cụm cho bạn, phân tích hiệu suất mạng của bạn (tức là đo tốc độ để xác định các nút thắt cổ chai), và hơn thế nữa!

TensorBoard được cài đặt tự động khi bạn cài đặt TensorFlow. Tuy nhiên, bạn sẽ cần một plugin TensorBoard để trực quan hóa dữ liệu hồ sơ. Nếu bạn đã làm theo hướng dẫn cài đặt tại

https://homl.info/install để chạy mọi thứ cục bộ, thì bạn đã cài đặt plugin, nhưng nếu bạn đang sử dụng Colab, thì bạn phải chạy lệnh sau:

```
%pip install -q -U tensorboard-plugin-profile
```

Để sử dụng TensorBoard, bạn phải sửa đổi chương trình của mình để nó xuất dữ liệu bạn muốn trực quan hóa sang các tệp nhật ký nhị phân đặc biệt được gọi là tệp sự kiện. Mỗi bản ghi dữ liệu nhị phân được gọi là một bản tóm tắt. Máy chủ TensorBoard sẽ giám sát thư mục nhật ký, và nó sẽ tự động nhận các thay đổi và cập nhật các trực quan hóa: điều này cho phép bạn trực quan hóa dữ liệu trực tiếp (với một độ trễ ngắn), chẳng hạn như các đường cong học tập trong quá trình huấn luyện. Nói chung, bạn muốn trỏ máy chủ TensorBoard đến một thư mục nhật ký gốc và cấu hình chương trình của bạn để nó ghi vào một thư mục con khác mỗi khi nó chạy. Bằng cách này, cùng một thể hiện máy chủ TensorBoard sẽ cho phép bạn trực quan hóa và so sánh dữ liệu từ nhiều lần chạy chương trình của bạn, mà không làm mọi thứ bị lẫn lộn.

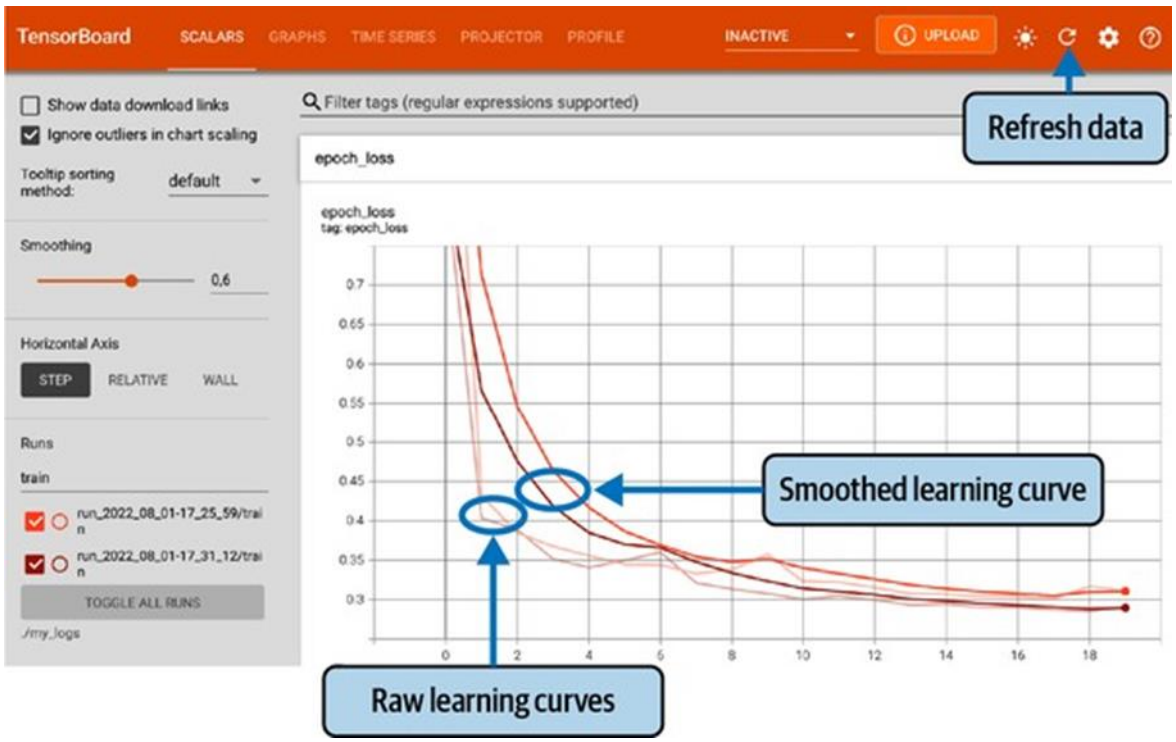
Hãy đặt tên thư mục nhật ký gốc là

`my_logs`, và hãy định nghĩa một hàm nhỏ tạo đường dẫn của thư mục con nhật ký dựa trên ngày và giờ hiện tại, để nó khác nhau ở mỗi lần chạy:

```
from pathlib import Path
from time import strftime

def get_run_logdir(root_logdir="my_logs"):
    return Path(root_logdir) / strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir() # e.g., my_logs/run_2022_08_01_17_25_59
```
``` [cite: 1]

Tin tốt là Keras cung cấp một callback `TensorBoard()` tiện lợi sẽ đảm nhiệm việc tạo thư mục nhật ký cho bạn (cùng với các thư mục cha của nó nếu cần), và nó sẽ tạo các tệp sự kiện và ghi các bản tóm tắt vào chúng trong quá trình huấn luyện. Nó sẽ đo mất mát và các số liệu huấn luyện và xác thực của mô hình của bạn (trong trường hợp này là MSE và RMSE), và nó cũng sẽ lập hồ sơ mạng nơ-ron của bạn. Nó rất dễ sử dụng: [cite: 1]

```python
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir,
profile_batch=(100, 200))
history = model.fit([...], callbacks=[tensorboard_cb])
```
``` [cite: 1]

Đó là tất cả! Trong ví dụ này, nó sẽ lập hồ sơ mạng giữa các batch 100 và 200 trong epoch đầu tiên. Tại sao lại là 100 và 200? Chà, thường mất một vài batch để mạng nơ-ron "khởi động", vì vậy bạn không muốn lập hồ sơ quá sớm, và việc lập hồ sơ sử dụng tài nguyên, vì vậy tốt nhất là không nên làm điều đó cho mọi batch. [cite: 1]

Tiếp theo, hãy thử thay đổi tốc độ học từ 0.001 thành 0.002, và chạy lại mã, với một thư mục con nhật ký mới. Bạn sẽ có cấu trúc thư mục tương tự như sau: [cite: 1]

my_logs ├── run_2022_08_01_17_25_59 │ ├── train │ │ ├── events.out.tfevents.1659331561.my_host_name.42042.0.v2 │ │ ├── events.out.tfevents.1659331562.my_host_name.profile-empty │ │ └── plugins │ │ └── profile │ │ └── 2022_08_01_17_26_02 │ │ ├── my_host_name.input_pipeline.pb │ │ └── [...] │ └── validation │ └── events.out.tfevents.1659331562.my_host_name.42042.1.v2 └── run_2022_08_01_17_31_12 └── [...] ```

Có một thư mục cho mỗi lần chạy, mỗi thư mục chứa một thư mục con cho nhật ký huấn luyện và một cho nhật ký xác thực. Cả hai đều chứa các tệp sự kiện, và nhật ký huấn luyện cũng bao gồm các dấu vết lập hồ sơ.

Bây giờ bạn đã có các tệp sự kiện sẵn sàng, đã đến lúc khởi động máy chủ TensorBoard. Điều này có thể được thực hiện trực tiếp trong Jupyter hoặc Colab bằng cách sử dụng tiện ích mở rộng Jupyter cho TensorBoard, được cài đặt cùng với thư viện TensorBoard. Tiện ích mở rộng này được cài đặt sẵn trong Colab. Đoạn mã sau tải tiện ích mở rộng Jupyter cho TensorBoard, và dòng thứ hai khởi động một máy chủ TensorBoard cho thư mục `my_logs`, kết nối với máy chủ này và hiển thị giao diện người dùng trực tiếp trong Jupyter. Máy chủ lắng nghe trên cổng TCP khả dụng đầu tiên lớn hơn hoặc bằng 6006 (hoặc bạn có thể đặt cổng bạn muốn bằng cách sử dụng tùy chọn

`--port`).

```
%load_ext tensorboard

%tensorboard --logdir=./my_logs
```

Bây giờ bạn sẽ thấy giao diện người dùng của TensorBoard. Nhấp vào tab SCALARS để xem các đường cong học tập (xem Hình 10-16). Ở phía dưới bên trái, chọn các nhật ký bạn muốn trực quan hóa (ví dụ: nhật ký huấn luyện từ lần chạy đầu tiên và thứ hai), và nhấp vào đại lượng vô hướng `epoch_loss`. Lưu ý rằng mất mát huấn luyện đã giảm đáng kể trong cả hai lần chạy, nhưng trong lần chạy thứ hai, nó giảm nhanh hơn một chút nhờ tốc độ học cao hơn.



Hình 10-16. Trực quan hóa các đường cong học tập với TensorBoard

Bạn cũng có thể trực quan hóa toàn bộ biểu đồ tính toán trong tab GRAPHS, trọng số đã học được chiếu xuống 3D trong tab PROJECTOR và các dấu vết lập hồ sơ trong tab PROFILE. Callback `TensorBoard()` có các tùy chọn để ghi thêm dữ liệu (xem tài liệu để biết

thêm chi tiết). Bạn có thể nhấp vào nút làm mới (🗘) ở phía trên bên phải để làm cho TensorBoard làm mới dữ liệu, và bạn có thể nhấp vào nút cài đặt (⚙) để kích hoạt tự động làm mới và chỉ định khoảng thời gian làm mới.

Ngoài ra, TensorFlow cung cấp một API cấp thấp hơn trong gói `tf.summary`. Đoạn mã sau đây tạo một `SummaryWriter` bằng cách sử dụng hàm `create_file_writer()`, và nó sử dụng writer này làm ngữ cảnh Python để ghi nhật ký các đại lượng vô hướng, biểu đồ, hình ảnh, âm thanh và văn bản, tất cả đều có thể được trực quan hóa bằng TensorBoard:

```
test_logdir = get_run_logdir()

writer = tf.summary.create_file_writer(str(test_logdir))
with writer.as_default():
    for step in range(1, 1000 + 1):
        tf.summary.scalar("my_scalar", np.sin(step / 10), step=step)
        data = (np.random.randn(100) + 2) * step / 100  # gets larger
        tf.summary.histogram("my_hist", data, buckets=50, step=step)

        images = np.random.rand(2, 32, 32, 3) * step / 1000 # gets brighter
        tf.summary.image("my_images", images, step=step)

        texts = ["The step is " + str(step), "Its square is " + str(step **
2)]
        tf.summary.text("my_text", texts, step=step)

        sine_wave = tf.math.sin(tf.range(12000) / 48000 * 2 * np.pi * 1)
        audio = tf.reshape(tf.cast(sine_wave, tf.float32), [1, -1, 1])
        tf.summary.audio("my_audio", audio, sample_rate=48000, step=step)
``` [cite: 1]

Nếu bạn chạy đoạn mã này và nhấp vào nút làm mới trong TensorBoard, bạn sẽ thấy một số tab xuất hiện: IMAGES, AUDIO, DISTRIBUTIONS, HISTOGRAMS và TEXT. Hãy thử nhấp vào tab IMAGES, và sử dụng thanh trượt phía trên mỗi hình ảnh để xem hình ảnh ở các bước thời gian khác nhau. Tương tự, hãy chuyển đến tab AUDIO và thử nghe âm thanh ở các bước thời gian khác nhau. Như bạn có thể thấy, TensorBoard là một công cụ hữu ích ngay cả ngoài TensorFlow hoặc học sâu. [cite: 1]

Tóm lại những gì bạn đã học được cho đến nay trong chương này: bạn đã biết mạng nơ-ron đến từ đâu, MLP là gì và cách bạn có thể sử dụng nó để phân loại và hồi quy, cách sử dụng API tuần tự của Keras để xây dựng MLP, và cách sử dụng API chức năng hoặc API subclassing để xây dựng các kiến trúc mô hình phức tạp hơn (bao gồm mô hình Rộng & Sâu, cũng như các mô hình có nhiều đầu vào và đầu ra). Bạn cũng đã học cách lưu và khôi phục mô hình và cách sử dụng các callback để lưu điểm kiểm tra, dừng sớm và hơn thế nữa. Cuối cùng, bạn đã học cách sử dụng TensorBoard để trực quan hóa. Bạn đã có thể tiếp tục và sử dụng mạng nơ-ron để giải quyết nhiều vấn đề! Tuy nhiên, bạn có thể tự hỏi làm

thế nào để chọn số lớp ẩn, số nơ-ron trong mạng và tất cả các siêu tham số khác. Hãy xem xét điều này bây giờ. [cite: 1]

Tinh chỉnh các siêu tham số của mạng nơ-ron

Tính linh hoạt của mạng nơ-ron cũng là một trong những hạn chế chính của chúng: có rất nhiều siêu tham số cần điều chỉnh. Bạn không chỉ có thể sử dụng bất kỳ kiến trúc mạng nào có thể tưởng tượng được, mà ngay cả trong một MLP cơ bản, bạn có thể thay đổi số lớp, số nơ-ron và loại hàm kích hoạt sẽ sử dụng trong mỗi lớp, logic khởi tạo trọng số, loại bộ tối ưu hóa sẽ sử dụng, tốc độ học của nó, kích thước batch và hơn thế nữa. Làm thế nào để bạn biết sự kết hợp siêu tham số nào là tốt nhất cho tác vụ của bạn?

Một lựa chọn là chuyển đổi mô hình Keras của bạn thành một bộ ước lượng Scikit-Learn, và sau đó sử dụng `GridSearchCV` hoặc `RandomizedSearchCV` để tinh chỉnh các siêu tham số, như bạn đã làm trong Chương 2. Đối với điều này, bạn có thể sử dụng các lớp bao bọc `KerasRegressor` và `KerasClassifier` từ thư viện SciKeras (xem https://github.com/adriangb/scikeras để biết thêm chi tiết). Tuy nhiên, có một cách tốt hơn: bạn có thể sử dụng thư viện Keras Tuner, một thư viện tinh chỉnh siêu tham số cho các mô hình Keras. Nó cung cấp một số chiến lược tinh chỉnh, có khả năng tùy chỉnh cao và tích hợp tuyệt vời với TensorBoard. Hãy xem cách sử dụng nó.

Nếu bạn đã làm theo hướng dẫn cài đặt tại https://homl.info/install để chạy mọi thứ cục bộ, thì bạn đã cài đặt Keras Tuner, nhưng nếu bạn đang sử dụng Colab, bạn sẽ cần chạy `%pip install -q -U keras-tuner`. Tiếp theo, nhập `keras_tuner`, thường là dưới dạng `kt`, sau đó viết một hàm xây dựng, biên dịch và trả về một mô hình Keras. Hàm phải nhận một đối tượng `kt.HyperParameters` làm đối số, đối tượng này có thể được sử dụng để định nghĩa các siêu tham số (số nguyên, số thập phân, chuỗi, v.v.) cùng với phạm vi giá trị có thể có của chúng, và các siêu tham số này có thể được sử dụng để xây dựng và biên dịch mô hình. Ví dụ, hàm sau xây dựng và biên dịch một MLP để phân loại hình ảnh Fashion MNIST, sử dụng các siêu tham số như số lớp ẩn (`n_hidden`), số nơ-ron trên mỗi lớp (`n_neurons`), tốc độ học (`learning_rate`), và loại bộ tối ưu hóa sẽ sử dụng (`optimizer`):

```
import tensorflow as tf
import keras_tuner as kt

def build_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    n_neurons = hp.Int("n_neurons", min_value=16, max_value=256)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,
sampling="log")
    optimizer = hp.Choice("optimizer", values=["sgd", "adam"])

    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    else:
        optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
```

```
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Flatten())
    for _ in range(n_hidden):
        model.add(tf.keras.layers.Dense(n_neurons, activation="relu"))
    model.add(tf.keras.layers.Dense(10, activation="softmax"))
    model.compile(loss="sparse_categorical_crossentropy",
optimizer=optimizer, metrics=["accuracy"])
    return model
```

Phần đầu tiên của hàm định nghĩa các siêu tham số. Ví dụ, `hp.Int("n_hidden", min_value=0, max_value=8, default=2)` kiểm tra xem một siêu tham số có tên "n_hidden" đã có trong đối tượng `HyperParameters hp` hay chưa, và nếu có thì nó trả về giá trị của nó. Nếu không, thì nó đăng ký một siêu tham số số nguyên mới có tên "n_hidden", có các giá trị có thể có từ 0 đến 8 (bao gồm), và nó trả về giá trị mặc định, trong trường hợp này là 2 (khi `default` không được đặt, thì `min_value` được trả về). Siêu tham số "n_neurons" được đăng ký theo cách tương tự. Siêu tham số "learning_rate" được đăng ký dưới dạng số thập phân nằm trong khoảng từ 10^-4 đến 10^-2, và vì `sampling="log"`, tốc độ học của tất cả các thang đo sẽ được lấy mẫu đều. Cuối cùng, siêu tham số `optimizer` được đăng ký với hai giá trị có thể có: "sgd" hoặc "adam" (giá trị mặc định là giá trị đầu tiên, trong trường hợp này là "sgd"). Tùy thuộc vào giá trị của `optimizer`, chúng ta tạo một bộ tối ưu hóa SGD hoặc một bộ tối ưu hóa Adam với tốc độ học đã cho.

Phần thứ hai của hàm chỉ xây dựng mô hình sử dụng các giá trị siêu tham số. Nó tạo một mô hình Sequential bắt đầu bằng một lớp Flatten, tiếp theo là số lượng lớp ẩn được yêu cầu (như được xác định bởi siêu tham số `n_hidden`) sử dụng hàm kích hoạt ReLU, và một lớp đầu ra với 10 nơ-ron (một cho mỗi lớp) sử dụng hàm kích hoạt softmax. Cuối cùng, hàm biên dịch mô hình và trả về nó.

Bây giờ nếu bạn muốn thực hiện tìm kiếm ngẫu nhiên cơ bản, bạn có thể tạo một tuner `kt.RandomSearch`, truyền hàm `build_model` vào hàm tạo, và gọi phương thức `search()` của tuner:

```
random_search_tuner = kt.RandomSearch(
    build_model, objective="val_accuracy", max_trials=5, overwrite=True,
    directory="my_fashion_mnist", project_name="my_rnd_search", seed=42)
random_search_tuner.search(X_train, y_train, epochs=10,
                           validation_data=(X_valid, y_valid))
```

Tuner `RandomSearch` trước tiên gọi `build_model()` một lần với một đối tượng `Hyperparameters` trống, chỉ để thu thập tất cả các đặc tả siêu tham số. Sau đó, trong ví dụ này, nó chạy 5 lần thử; đối với mỗi lần thử, nó xây dựng một mô hình sử dụng các siêu tham số được lấy mẫu ngẫu nhiên trong phạm vi tương ứng của chúng, sau đó nó huấn luyện mô hình đó trong 10 epoch và lưu nó vào một thư mục con của thư mục `my_fashion_mnist/my_rnd_search`. Vì `overwrite=True`, thư mục `my_rnd_search` bị xóa trước khi quá trình huấn luyện bắt đầu. Nếu bạn chạy mã này lần thứ hai nhưng với `overwrite=False` và `max_trials=10`, tuner sẽ tiếp tục tinh chỉnh từ nơi nó đã dừng lại, chạy thêm 5 lần thử: điều này có nghĩa là bạn không phải chạy tất cả các lần thử trong một lần.

Cuối cùng, vì objective được đặt thành “val_accuracy”, tuner ưu tiên các mô hình có độ chính xác xác thực cao hơn, vì vậy khi tuner đã hoàn thành tìm kiếm, bạn có thể nhận được các mô hình tốt nhất như sau:

```
top3_models = random_search_tuner.get_best_models(num_models=3)
best_model = top3_models[0]
```

Bạn cũng có thể gọi get_best_hyperparameters() để nhận kt.HyperParameters của các mô hình tốt nhất:

```
>>> top3_params = random_search_tuner.get_best_hyperparameters(num_trials=3)

>>> top3_params[0].values # best hyperparameter values
{'n_hidden': 5,
'n_neurons': 70,
'learning_rate': 0.00041268008323824807,
'optimizer': 'adam'}
```

Mỗi tuner được hướng dẫn bởi một cái gọi là oracle: trước mỗi lần thử, tuner yêu cầu oracle cho biết lần thử tiếp theo nên là gì. Tuner RandomSearch sử dụng RandomSearchOracle, khá cơ bản: nó chỉ chọn lần thử tiếp theo một cách ngẫu nhiên, như chúng ta đã thấy trước đó. Vì oracle theo dõi tất cả các lần thử, bạn có thể yêu cầu nó cung cấp cho bạn cái tốt nhất, và bạn có thể hiển thị một bản tóm tắt của lần thử đó:

```
>>> best_trial = random_search_tuner.oracle.get_best_trials(num_trials=1)[0]

>>> best_trial.summary()
Trial summary Hyperparameters:
n_hidden: 5
n_neurons: 70
learning_rate: 0.00041268008323824807
optimizer: adam
Score: 0.8736000061035156
```

Điều này cho thấy các siêu tham số tốt nhất (như trước đó), cũng như độ chính xác xác thực. Bạn cũng có thể truy cập tất cả các số liệu trực tiếp:

```
>>> best_trial.metrics.get_last_value("val_accuracy")
0.8736000061035156
```

Nếu bạn hài lòng với hiệu suất của mô hình tốt nhất, bạn có thể tiếp tục huấn luyện nó trong vài epoch trên tập huấn luyện đầy đủ (X_train_full và y_train_full), sau đó đánh giá nó trên tập kiểm tra, và triển khai nó vào sản xuất (xem Chương 19):

```
best_model.fit(X_train_full, y_train_full, epochs=10)
test_loss, test_accuracy = best_model.evaluate(X_test, y_test)
```

Trong một số trường hợp, bạn có thể muốn tinh chỉnh các siêu tham số tiền xử lý dữ liệu, hoặc các đối số của model.fit(), chẳng hạn như kích thước batch. Đối với điều này, bạn phải sử dụng một kỹ thuật hơi khác: thay vì viết một hàm build_model(), bạn phải kế thừa

từ lớp kt.HyperModel và định nghĩa hai phương thức, build() và fit(). Phương thức build() thực hiện chính xác những gì hàm build_model() làm. Phương thức fit() nhận một đối tượng HyperParameters và một mô hình đã biên dịch làm đối số, cũng như tất cả các đối số của model.fit(), và huấn luyện mô hình và trả về đối tượng History. Quan trọng nhất, phương thức fit() có thể sử dụng các siêu tham số để quyết định cách tiền xử lý dữ liệu, điều chỉnh kích thước batch và hơn thế nữa. Ví dụ, lớp sau xây dựng cùng một mô hình như trước, với cùng các siêu tham số, nhưng nó cũng sử dụng một siêu tham số Boolean "normalize" để kiểm soát xem có chuẩn hóa dữ liệu huấn luyện trước khi huấn luyện mô hình hay không:

```
class MyClassificationHyperModel(kt.HyperModel):
    def build(self, hp):
        return build_model(hp)

    def fit(self, hp, model, X, y, **kwargs):
        if hp.Boolean("normalize"):
            norm_layer = tf.keras.layers.Normalization()
            X = norm_layer(X)
        return model.fit(X, y, **kwargs)
```

Sau đó, bạn có thể truyền một thể hiện của lớp này cho tuner bạn chọn, thay vì truyền hàm build_model. Ví dụ, hãy xây dựng một tuner kt.Hyperband dựa trên một thể hiện MyClassificationHyperModel:

```
hyperband_tuner = kt.Hyperband(
    MyClassificationHyperModel(), objective="val_accuracy", seed=42,
max_epochs=10, factor=3, hyperband_iterations=2,
    overwrite=True, directory="my_fashion_mnist", project_name="hyperband")
```

Tuner này tương tự như lớp HalvingRandomSearchCV mà chúng ta đã thảo luận trong Chương 2: nó bắt đầu bằng cách huấn luyện nhiều mô hình khác nhau trong vài epoch, sau đó nó loại bỏ các mô hình tệ nhất và chỉ giữ lại các mô hình top 1/factor (tức là, một phần ba trên cùng trong trường hợp này), lặp lại quá trình lựa chọn này cho đến khi chỉ còn một mô hình duy nhất. Đối số max_epochs kiểm soát số epoch tối đa mà mô hình tốt nhất sẽ được huấn luyện. Toàn bộ quá trình được lặp lại hai lần trong trường hợp này (hyperband_iterations=2). Tổng số epoch huấn luyện trên tất cả các mô hình cho mỗi lần lặp hyperband là khoảng max_epochs * (log(max_epochs) / log(factor)) ** 2, vì vậy nó khoảng 44 epoch trong ví dụ này. Các đối số khác giống như đối với kt.RandomSearch.

Hãy chạy tuner Hyperband ngay bây giờ. Chúng ta sẽ sử dụng callback TensorBoard, lần này trỏ đến thư mục nhật ký gốc (tuner sẽ đảm nhiệm việc sử dụng một thư mục con khác cho mỗi lần thử), cũng như một callback EarlyStopping:

```
root_logdir = Path(hyperband_tuner.project_dir) / "tensorboard"
tensorboard_cb = tf.keras.callbacks.TensorBoard(root_logdir)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=2)
hyperband_tuner.search(X_train, y_train, epochs=10,
```

```
                      validation_data=(X_valid, y_valid),
                      callbacks=[early_stopping_cb, tensorboard_cb])
```

Bây giờ nếu bạn mở TensorBoard, trỏ `--logdir` đến thư mục `my_fashion_mnist/hyperband/tensorboard`, bạn sẽ thấy tất cả kết quả thử nghiệm khi chúng diễn ra. Đảm bảo truy cập tab HPARAMS: nó chứa bản tóm tắt tất cả các kết hợp siêu tham số đã được thử, cùng với các số liệu tương ứng. Lưu ý rằng có ba tab bên trong tab HPARAMS: chế độ xem bảng, chế độ xem tọa độ song song và chế độ xem ma trận biểu đồ phân tán. Ở phần dưới bên trái của bảng điều khiển, bỏ chọn tất cả các số liệu ngoại trừ `validation.epoch_accuracy`: điều này sẽ làm cho các biểu đồ rõ ràng hơn. Trong chế độ xem tọa độ song song, hãy thử chọn một phạm vi giá trị cao trong cột `validation.epoch_accuracy`: điều này sẽ lọc chỉ các kết hợp siêu tham số đạt hiệu suất tốt. Nhấp vào một trong các kết hợp siêu tham số, và các đường cong học tập tương ứng sẽ xuất hiện ở cuối trang. Dành một chút thời gian để xem xét từng tab; điều này sẽ giúp bạn hiểu tác động của từng siêu tham số đến hiệu suất, cũng như các tương tác giữa các siêu tham số.

Hyperband thông minh hơn tìm kiếm ngẫu nhiên thuần túy trong cách nó phân bổ tài nguyên, nhưng về cốt lõi, nó vẫn khám phá không gian siêu tham số một cách ngẫu nhiên; nó nhanh, nhưng thô. Tuy nhiên, Keras Tuner cũng bao gồm một tuner `kt.BayesianOptimization`: thuật toán này dần dần học được những vùng nào của không gian siêu tham số là hứa hẹn nhất bằng cách điều chỉnh một mô hình xác suất được gọi là quá trình Gaussian. Điều này cho phép nó dần dần phóng to vào các siêu tham số tốt nhất. Nhược điểm là thuật toán có các siêu tham số riêng: `alpha` đại diện cho mức độ nhiễu bạn mong đợi trong các phép đo hiệu suất trên các lần thử (mặc định là 10^-4), và `beta` chỉ định mức độ bạn muốn thuật toán khám phá, thay vì chỉ đơn giản là khai thác các vùng tốt đã biết của không gian siêu tham số (mặc định là 2.6). Ngoài ra, tuner này có thể được sử dụng giống như các tuner trước:

```
bayesian_opt_tuner = kt.BayesianOptimization(
    MyClassificationHyperModel(), objective="val_accuracy", seed=42,
max_trials=10, alpha=1e-4, beta=2.6,
    overwrite=True, directory="my_fashion_mnist",
project_name="bayesian_opt")
bayesian_opt_tuner.search([...])
```

Tinh chỉnh siêu tham số vẫn là một lĩnh vực nghiên cứu tích cực, và nhiều cách tiếp cận khác đang được khám phá. Ví dụ, hãy xem bài báo xuất sắc năm 2017 của DeepMind, nơi các tác giả đã sử dụng một thuật toán tiến hóa để cùng tối ưu hóa một tập hợp các mô hình và siêu tham số của chúng. Google cũng đã sử dụng một cách tiếp cận tiến hóa, không chỉ để tìm kiếm siêu tham số mà còn để khám phá tất cả các loại kiến trúc mô hình: nó hỗ trợ dịch vụ AutoML của họ trên Google Vertex AI (xem Chương 19). Thuật ngữ AutoML đề cập đến bất kỳ hệ thống nào đảm nhiệm một phần lớn quy trình làm việc ML.

Các thuật toán tiến hóa thậm chí đã được sử dụng thành công để huấn luyện các mạng nơ-ron riêng lẻ, thay thế giảm độ dốc phổ biến! Để biết ví dụ, xem bài đăng năm 2017 của Uber nơi các tác giả giới thiệu kỹ thuật Deep Neuroevolution của họ.

Nhưng bất chấp tất cả tiến bộ thú vị này và tất cả các công cụ và dịch vụ này, vẫn hữu ích khi có ý tưởng về những giá trị hợp lý cho mỗi siêu tham số để bạn có thể xây dựng một nguyên mẫu nhanh chóng và hạn chế không gian tìm kiếm. Các phần sau đây cung cấp hướng dẫn để chọn số lớp ẩn và nơ-ron trong một MLP và để chọn các giá trị tốt cho một số siêu tham số chính.

Số lượng lớp ẩn

Đối với nhiều vấn đề, bạn có thể bắt đầu với một lớp ẩn duy nhất và nhận được kết quả hợp lý. Một MLP chỉ với một lớp ẩn về mặt lý thuyết có thể mô hình hóa ngay cả các hàm phức tạp nhất, miễn là nó có đủ nơ-ron. Nhưng đối với các vấn đề phức tạp, mạng sâu có hiệu quả tham số cao hơn nhiều so với mạng nông: chúng có thể mô hình hóa các hàm phức tạp bằng cách sử dụng số nơ-ron ít hơn theo cấp số mũ so với mạng nông, cho phép chúng đạt được hiệu suất tốt hơn nhiều với cùng một lượng dữ liệu huấn luyện.

Để hiểu tại sao, giả sử bạn được yêu cầu vẽ một khu rừng bằng một phần mềm vẽ nào đó, nhưng bạn bị cấm sao chép và dán bất cứ thứ gì. Sẽ mất một lượng thời gian khổng lồ: bạn sẽ phải vẽ từng cây một, từng nhánh một, từng lá một. Nếu bạn có thể vẽ một chiếc lá, sao chép và dán nó để vẽ một nhánh, sau đó sao chép và dán nhánh đó để tạo ra một cái cây, và cuối cùng sao chép và dán cái cây này để tạo ra một khu rừng, bạn sẽ hoàn thành ngay lập tức.

Dữ liệu thực tế thường được cấu trúc theo cách phân cấp như vậy, và mạng nơ-ron sâu tự động tận dụng thực tế này: các lớp ẩn thấp hơn mô hình hóa các cấu trúc cấp thấp (ví dụ: các đoạn thẳng có hình dạng và hướng khác nhau), các lớp ẩn trung gian kết hợp các cấu trúc cấp thấp này để mô hình hóa các cấu trúc cấp trung gian (ví dụ: hình vuông, hình tròn), và các lớp ẩn cao nhất và lớp đầu ra kết hợp các cấu trúc trung gian này để mô hình hóa các cấu trúc cấp cao (ví dụ: khuôn mặt).

Kiến trúc phân cấp này không chỉ giúp DNN hội tụ nhanh hơn đến một giải pháp tốt, mà nó còn cải thiện khả năng tổng quát hóa của chúng đối với các tập dữ liệu mới. Ví dụ, nếu bạn đã huấn luyện một mô hình để nhận dạng khuôn mặt trong ảnh và bây giờ bạn muốn huấn luyện một mạng nơ-ron mới để nhận dạng kiểu tóc, bạn có thể khởi động quá trình huấn luyện bằng cách tái sử dụng các lớp thấp hơn của mạng đầu tiên. Thay vì khởi tạo ngẫu nhiên các trọng số và độ lệch của một vài lớp đầu tiên của mạng nơ-ron mới, bạn có thể khởi tạo chúng bằng các giá trị của trọng số và độ lệch của các lớp thấp hơn của mạng đầu tiên. Bằng cách này, mạng sẽ không phải học từ đầu tất cả các cấu trúc cấp thấp xảy ra trong hầu hết các hình ảnh; nó sẽ chỉ phải học các cấu trúc cấp cao hơn (ví dụ: kiểu tóc). Đây được gọi là học chuyển giao.

Tóm lại, đối với nhiều vấn đề, bạn có thể bắt đầu chỉ với một hoặc hai lớp ẩn và mạng nơ-ron sẽ hoạt động tốt. Chẳng hạn, bạn có thể dễ dàng đạt độ chính xác trên 97% trên tập dữ

liệu MNIST chỉ với một lớp ẩn với vài trăm nơ-ron, và trên 98% độ chính xác với hai lớp ẩn với cùng tổng số nơ-ron, trong khoảng cùng một lượng thời gian huấn luyện. Đối với các vấn đề phức tạp hơn, bạn có thể tăng số lượng lớp ẩn cho đến khi bạn bắt đầu overfitting tập huấn luyện. Các tác vụ rất phức tạp, chẳng hạn như phân loại hình ảnh lớn hoặc nhận dạng giọng nói, thường yêu cầu mạng có hàng chục lớp (hoặc thậm chí hàng trăm, nhưng không phải các lớp kết nối đầy đủ, như bạn sẽ thấy trong Chương 14), và chúng cần một lượng lớn dữ liệu huấn luyện. Bạn sẽ hiếm khi phải huấn luyện các mạng như vậy từ đầu: việc tái sử dụng các phần của một mạng hiện đại đã được huấn luyện trước thực hiện một tác vụ tương tự phổ biến hơn nhiều. Quá trình huấn luyện sau đó sẽ nhanh hơn nhiều và yêu cầu ít dữ liệu hơn nhiều (chúng ta sẽ thảo luận về điều này trong Chương 11).

Số lượng nơ-ron trên mỗi lớp ẩn

Số lượng nơ-ron trong các lớp đầu vào và đầu ra được xác định bởi loại đầu vào và đầu ra mà tác vụ của bạn yêu cầu. Ví dụ, tác vụ MNIST yêu cầu 28 × 28 = 784 đầu vào và 10 nơ-ron đầu ra.

Đối với các lớp ẩn, trước đây thường có kích thước chúng để tạo thành một kim tự tháp, với số lượng nơ-ron ngày càng ít hơn ở mỗi lớp - lý do là nhiều đặc trưng cấp thấp có thể hợp nhất thành ít đặc trưng cấp cao hơn. Một mạng nơ-ron điển hình cho MNIST có thể có 3 lớp ẩn, lớp đầu tiên với 300 nơ-ron, lớp thứ hai với 200, và lớp thứ ba với 100. Tuy nhiên, thực hành này đã bị loại bỏ phần lớn vì dường như việc sử dụng cùng một số lượng nơ-ron trong tất cả các lớp ẩn hoạt động tốt trong hầu hết các trường hợp, hoặc thậm chí tốt hơn; hơn nữa, chỉ có một siêu tham số để điều chỉnh, thay vì một cho mỗi lớp. Điều đó nói lên rằng, tùy thuộc vào tập dữ liệu, đôi khi có thể hữu ích khi làm cho lớp ẩn đầu tiên lớn hơn các lớp khác.

Giống như số lượng lớp, bạn có thể thử tăng số lượng nơ-ron dần dần cho đến khi mạng bắt đầu overfitting. Ngoài ra, bạn có thể thử xây dựng một mô hình với số lượng lớp và nơ-ron nhiều hơn một chút so với thực tế bạn cần, sau đó sử dụng dừng sớm và các kỹ thuật chính quy hóa khác để ngăn nó overfitting quá nhiều.

Vincent Vanhoucke, một nhà khoa học tại Google, đã gọi đây là cách tiếp cận "quần co giãn": thay vì lãng phí thời gian tìm kiếm chiếc quần vừa vặn hoàn hảo với kích thước của bạn, chỉ cần sử dụng quần co giãn lớn sẽ co lại đến kích thước phù hợp. Với cách tiếp cận này, bạn tránh được các lớp nút thắt cổ chai có thể làm hỏng mô hình của bạn. Thật vậy, nếu một lớp có quá ít nơ-ron, nó sẽ không có đủ sức mạnh biểu diễn để bảo toàn tất cả thông tin hữu ích từ các đầu vào (ví dụ: một lớp có hai nơ-ron chỉ có thể xuất ra dữ liệu 2D, vì vậy nếu nó nhận dữ liệu 3D làm đầu vào, một số thông tin sẽ bị mất). Bất kể phần còn lại của mạng lớn và mạnh đến đâu, thông tin đó sẽ không bao giờ được phục hồi.

Tốc độ học, kích thước Batch và các siêu tham số khác

Số lượng lớp ẩn và nơ-ron không phải là các siêu tham số duy nhất bạn có thể điều chỉnh trong một MLP. Dưới đây là một số siêu tham số quan trọng nhất, cũng như các mẹo về cách đặt chúng:

Tốc độ học Tốc độ học được cho là siêu tham số quan trọng nhất. Nói chung, tốc độ học tối ưu bằng khoảng một nửa tốc độ học tối đa (tức là tốc độ học mà thuật toán huấn luyện phân kỳ, như chúng ta đã thấy trong Chương 4). Một cách để tìm tốc độ học tốt là huấn luyện mô hình trong vài trăm lần lặp, bắt đầu với tốc độ học rất thấp (ví dụ: 10^{-5}) và tăng dần nó lên một giá trị rất lớn (ví dụ: 10). Điều này được thực hiện bằng cách nhân tốc độ học với một hệ số không đổi ở mỗi lần lặp (ví dụ: bằng $(10/10^{-5})^{1/500}$ để đi từ 10^{-5} đến 10 trong 500 lần lặp). Nếu bạn vẽ mất mát theo hàm của tốc độ học (sử dụng thang logarit cho tốc độ học), bạn sẽ thấy nó giảm ban đầu. Nhưng sau một thời gian, tốc độ học sẽ quá lớn, do đó mất mát sẽ tăng trở lại: tốc độ học tối ưu sẽ thấp hơn một chút so với điểm mà mất mát bắt đầu tăng (thường thấp hơn khoảng 10 lần so với điểm ngoặt). Sau đó, bạn có thể khởi tạo lại mô hình của mình và huấn luyện nó bình thường bằng tốc độ học tốt này. Chúng ta sẽ xem xét nhiều kỹ thuật tối ưu hóa tốc độ học hơn trong Chương 11.

Bộ tối ưu hóa Chọn một bộ tối ưu hóa tốt hơn giảm độ dốc mini-batch đơn giản (và điều chỉnh các siêu tham số của nó) cũng khá quan trọng. Chúng ta sẽ xem xét một số bộ tối ưu hóa nâng cao trong Chương 11.

Kích thước Batch Kích thước batch có thể có tác động đáng kể đến hiệu suất và thời gian huấn luyện của mô hình của bạn. Lợi ích chính của việc sử dụng kích thước batch lớn là các bộ tăng tốc phần cứng như GPU có thể xử lý chúng hiệu quả, do đó thuật toán huấn luyện sẽ thấy nhiều thể hiện hơn mỗi giây. Do đó, nhiều nhà nghiên cứu và thực hành khuyên dùng kích thước batch lớn nhất có thể vừa với RAM GPU. Tuy nhiên, có một nhược điểm: trong thực tế, kích thước batch lớn thường dẫn đến mất ổn định huấn luyện, đặc biệt là khi bắt đầu huấn luyện, và mô hình kết quả có thể không tổng quát hóa tốt bằng mô hình được huấn luyện với kích thước batch nhỏ. Vào tháng 4 năm 2018, Yann LeCun thậm chí đã tweet “Bạn bè không để bạn bè sử dụng mini-batch lớn hơn 32”, trích dẫn một bài báo năm 2018 của Dominic Masters và Carlo Luschi kết luận rằng việc sử dụng các batch nhỏ (từ 2 đến 32) được ưu tiên hơn vì các batch nhỏ dẫn đến các mô hình tốt hơn trong thời gian huấn luyện ít hơn. Tuy nhiên, các nghiên cứu khác lại chỉ ra hướng ngược lại. Ví dụ, vào năm 2017, các bài báo của Elad Hoffer et al. và Priya Goyal et al. cho thấy có thể sử dụng kích thước batch rất lớn (lên đến 8.192) cùng với các kỹ thuật khác nhau như làm nóng tốc độ học (tức là bắt đầu huấn luyện với tốc độ học nhỏ, sau đó tăng dần lên, như đã thảo luận trong Chương 11) và để có được thời gian huấn luyện rất ngắn, mà không có bất kỳ khoảng cách tổng quát hóa nào. Vì vậy, một chiến lược là cố gắng sử dụng kích thước batch lớn, với làm nóng tốc độ học, và nếu quá trình huấn luyện không ổn định hoặc hiệu suất cuối cùng đáng thất vọng, thì hãy thử sử dụng kích thước batch nhỏ thay thế.

Hàm kích hoạt Chúng ta đã thảo luận cách chọn hàm kích hoạt ở đầu chương này: nói chung, hàm kích hoạt ReLU sẽ là một mặc định tốt cho tất cả các lớp ẩn, nhưng đối với lớp đầu ra thì nó thực sự phụ thuộc vào tác vụ của bạn.

Số lần lặp Trong hầu hết các trường hợp, số lần lặp huấn luyện thực sự không cần phải điều chỉnh: chỉ cần sử dụng dừng sớm thay thế.

Để biết thêm các thực hành tốt nhất liên quan đến việc điều chỉnh siêu tham số mạng nơ-ron, hãy xem bài báo xuất sắc năm 2018 của Leslie Smith.

Điều này kết thúc phần giới thiệu của chúng ta về mạng nơ-ron nhân tạo và việc triển khai chúng với Keras. Trong vài chương tiếp theo, chúng ta sẽ thảo luận về các kỹ thuật để huấn luyện các mạng rất sâu. Chúng ta cũng sẽ khám phá cách tùy chỉnh mô hình bằng API cấp thấp hơn của TensorFlow và cách tải và tiền xử lý dữ liệu hiệu quả bằng API `tf.data`. Và chúng ta sẽ đi sâu vào các kiến trúc mạng nơ-ron phổ biến khác: mạng nơ-ron tích chập để xử lý hình ảnh, mạng nơ-ron đệ quy và transformers cho dữ liệu tuần tự và văn bản, bộ tự mã hóa để học biểu diễn và mạng đối kháng tạo sinh để mô hình hóa và tạo dữ liệu.

Bài tập

1. Sân chơi TensorFlow là một công cụ mô phỏng mạng nơ-ron tiện dụng được xây dựng bởi nhóm TensorFlow. Trong bài tập này, bạn sẽ huấn luyện một số bộ phân loại nhị phân chỉ bằng vài cú nhấp chuột, và điều chỉnh kiến trúc mô hình và các siêu tham số của nó để có được một số trực giác về cách mạng nơ-ron hoạt động và các siêu tham số của chúng làm gì. Dành một chút thời gian để khám phá những điều sau: a. Các mẫu được học bởi một mạng nơ-ron. Thử huấn luyện mạng nơ-ron mặc định bằng cách nhấp vào nút Chạy (trên cùng bên trái). Lưu ý cách nó nhanh chóng tìm thấy một giải pháp tốt cho tác vụ phân loại. Các nơ-ron trong lớp ẩn đầu tiên đã học các mẫu đơn giản, trong khi các nơ-ron trong lớp ẩn thứ hai đã học cách kết hợp các mẫu đơn giản của lớp ẩn đầu tiên thành các mẫu phức tạp hơn. Nói chung, càng có nhiều lớp, các mẫu càng có thể phức tạp hơn. b. Các hàm kích hoạt. Thử thay thế hàm kích hoạt tanh bằng hàm kích hoạt ReLU, và huấn luyện lại mạng. Lưu ý rằng nó tìm thấy một giải pháp thậm chí còn nhanh hơn, nhưng lần này các ranh giới là tuyến tính. Điều này là do hình dạng của hàm ReLU. c. Nguy cơ cực tiểu cục bộ. Sửa đổi kiến trúc mạng để chỉ có một lớp ẩn với ba nơ-ron. Huấn luyện nó nhiều lần (để đặt lại trọng số mạng, nhấp vào nút Đặt lại bên cạnh nút Phát). Lưu ý rằng thời gian huấn luyện thay đổi rất nhiều, và đôi khi nó thậm chí còn bị mắc kẹt trong một cực tiểu cục bộ. d. Điều gì xảy ra khi mạng nơ-ron quá nhỏ. Loại bỏ một nơ-ron để chỉ giữ lại hai nơ-ron. Lưu ý rằng mạng nơ-ron hiện không có khả năng tìm thấy một giải pháp tốt, ngay cả khi bạn thử nhiều lần. Mô hình có quá ít tham số và hệ thống hóa underfits tập huấn luyện. e. Điều gì xảy ra khi mạng nơ-ron đủ lớn. Đặt số lượng nơ-ron thành tám, và huấn luyện mạng nhiều lần. Lưu ý rằng bây giờ nó luôn nhanh chóng và không bao giờ bị mắc kẹt. Điều này làm nổi bật một phát hiện quan trọng trong lý thuyết mạng nơ-ron: các mạng nơ-ron lớn hiếm khi bị mắc kẹt trong các cực tiểu cục bộ, và ngay cả khi chúng bị mắc kẹt, các cực tiểu cục bộ này thường gần như tốt bằng cực đại toàn cục. Tuy nhiên, chúng vẫn có thể bị mắc kẹt trên các mặt phẳng dài trong một thời gian dài. f. Nguy cơ của các gradient biến mất trong mạng sâu. Chọn tập dữ liệu xoắn ốc (tập dữ liệu dưới cùng bên phải dưới "DATA"), và thay đổi kiến trúc mạng để có bốn lớp ẩn với tám nơ-ron mỗi lớp. Lưu ý rằng quá trình huấn luyện mất nhiều thời gian hơn và thường bị mắc kẹt trên các mặt phẳng trong thời gian dài. Cũng lưu ý rằng các nơ-ron trong các lớp cao nhất (ở bên phải) có xu hướng tiến hóa nhanh hơn các nơ-ron trong các lớp thấp nhất (ở bên trái). Vấn đề này, được gọi là vấn đề gradient biến mất, có thể được giảm nhẹ bằng cách khởi tạo trọng số tốt hơn và các kỹ thuật khác, các bộ tối ưu hóa tốt hơn (chẳng hạn như AdaGrad hoặc Adam), hoặc chuẩn hóa theo batch (được thảo luận trong

Chương 11). g. Đi xa hơn. Dành một giờ hoặc lâu hơn để chơi với các tham số khác và cảm nhận những gì chúng làm, để xây dựng một sự hiểu biết trực quan về mạng nơ-ron.

2. Vẽ một ANN sử dụng các nơ-ron nhân tạo ban đầu (giống như những nơ-ron trong Hình 10-3) tính toán A ⊕ B (trong đó ⊕ đại diện cho phép toán XOR). Gợi ý: A ⊕ B = (A ∧ ¬B) ∨ (¬A ∧ B).
3. Tại sao nói chung nên ưu tiên sử dụng bộ phân loại hồi quy logistic hơn là perceptron cổ điển (tức là một lớp duy nhất các đơn vị logic ngưỡng được huấn luyện bằng thuật toán huấn luyện perceptron)? Làm thế nào bạn có thể điều chỉnh một perceptron để làm cho nó tương đương với một bộ phân loại hồi quy logistic?
4. Tại sao hàm kích hoạt sigmoid là một thành phần quan trọng trong việc huấn luyện các MLP đầu tiên?
5. Kể tên ba hàm kích hoạt phổ biến. Bạn có thể vẽ chúng không?
6. Giả sử bạn có một MLP bao gồm một lớp đầu vào với 10 nơ-ron truyền qua, tiếp theo là một lớp ẩn với 50 nơ-ron nhân tạo, và cuối cùng là một lớp đầu ra với 3 nơ-ron nhân tạo. Tất cả các nơ-ron nhân tạo đều sử dụng hàm kích hoạt ReLU. a. Hình dạng của ma trận đầu vào X là gì? b. Hình dạng của ma trận trọng số W_h và vector độ lệch b_h của lớp ẩn là gì? c. Hình dạng của ma trận trọng số W_o và vector độ lệch b_o của lớp đầu ra là gì? d. Hình dạng của ma trận đầu ra Y của mạng là gì? e. Viết phương trình tính ma trận đầu ra Y của mạng dưới dạng hàm của X, W_h , b_h , W_o , và b_o .
7. Bạn cần bao nhiêu nơ-ron trong lớp đầu ra nếu bạn muốn phân loại email thành thư rác hoặc thư hợp lệ? Bạn nên sử dụng hàm kích hoạt nào trong lớp đầu ra? Nếu thay vào đó bạn muốn giải quyết MNIST, bạn cần bao nhiêu nơ-ron trong lớp đầu ra, và bạn nên sử dụng hàm kích hoạt nào? Còn đối với việc để mạng của bạn dự đoán giá nhà, như trong Chương 2 thì sao?
8. Backpropagation là gì và nó hoạt động như thế nào? Sự khác biệt giữa backpropagation và tự động đạo hàm ngược là gì?
9. Bạn có thể liệt kê tất cả các siêu tham số bạn có thể điều chỉnh trong một MLP cơ bản không? Nếu MLP overfitting dữ liệu huấn luyện, làm thế nào bạn có thể điều chỉnh các siêu tham số này để cố gắng giải quyết vấn đề?
10. Huấn luyện một MLP sâu trên tập dữ liệu MNIST (bạn có thể tải nó bằng cách sử dụng `tf.keras.datasets.mnist.load_data()`). Xem liệu bạn có thể đạt được độ chính xác trên 98% bằng cách điều chỉnh thủ công các siêu tham số. Thử tìm kiếm tốc độ học tối ưu bằng cách sử dụng cách tiếp cận được trình bày trong chương này (tức là, bằng cách tăng tốc độ học theo cấp số mũ, vẽ biểu đồ mất mát và tìm điểm mà mất mát tăng vọt). Tiếp theo, thử điều chỉnh các siêu tham số bằng Keras Tuner với tất cả các tính năng - lưu điểm kiểm tra, sử dụng dừng sớm và vẽ các đường cong học tập bằng TensorBoard.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 11. HUẤN LUYỆN MẠNG NƠ-RON SÂU

Trong Chương 10, bạn đã xây dựng, huấn luyện và tinh chỉnh mạng nơ-ron nhân tạo đầu tiên của mình. Nhưng đó là những mạng nông, chỉ với một vài lớp ẩn. Điều gì sẽ xảy ra nếu bạn cần giải quyết một vấn đề phức tạp, chẳng hạn như phát hiện hàng trăm loại đối tượng trong hình ảnh độ phân giải cao? Bạn có thể cần huấn luyện một ANN sâu hơn nhiều, có lẽ với 10 lớp hoặc nhiều hơn nữa, mỗi lớp chứa hàng trăm nơ-ron, được liên kết bởi hàng trăm nghìn kết nối. Huấn luyện một mạng nơ-ron sâu không hề dễ dàng. Dưới đây là một số vấn đề bạn có thể gặp phải:

- Bạn có thể phải đối mặt với vấn đề gradient ngày càng nhỏ hoặc lớn hơn, khi truyền ngược qua DNN trong quá trình huấn luyện. Cả hai vấn đề này đều làm cho các lớp thấp hơn rất khó huấn luyện.
- Bạn có thể không có đủ dữ liệu huấn luyện cho một mạng lớn như vậy, hoặc việc gắn nhãn có thể quá tốn kém.
- Quá trình huấn luyện có thể cực kỳ chậm.
- Một mô hình với hàng triệu tham số sẽ có nguy cơ cao bị overfitting tập huấn luyện, đặc biệt nếu không có đủ thể hiện huấn luyện hoặc nếu chúng quá nhiễu.

Trong chương này, chúng ta sẽ xem xét từng vấn đề này và trình bày các kỹ thuật để giải quyết chúng. Chúng ta sẽ bắt đầu bằng cách khám phá các vấn đề gradient biến mất và bùng nổ và một số giải pháp phổ biến nhất của chúng. Tiếp theo, chúng ta sẽ xem xét học chuyển giao và tiền huấn luyện không giám sát, có thể giúp bạn giải quyết các tác vụ phức tạp ngay cả khi bạn có ít dữ liệu được gắn nhãn. Sau đó, chúng ta sẽ thảo luận về các bộ tối ưu hóa khác nhau có thể tăng tốc độ huấn luyện các mô hình lớn đáng kể. Cuối cùng, chúng ta sẽ đề cập đến một vài kỹ thuật chính quy hóa phổ biến cho các mạng nơ-ron lớn. Với những công cụ này, bạn sẽ có thể huấn luyện các mạng rất sâu. Chào mừng đến với học sâu!

Vấn đề Gradient biến mất/bùng nổ

Như đã thảo luận trong Chương 10, giai đoạn thứ hai của thuật toán backpropagation hoạt động bằng cách đi từ lớp đầu ra đến lớp đầu vào, truyền gradient lỗi trên đường đi. Khi thuật toán đã tính toán gradient của hàm chi phí đối với từng tham số trong mạng, nó sử dụng các gradient này để cập nhật từng tham số bằng một bước giảm độ dốc.

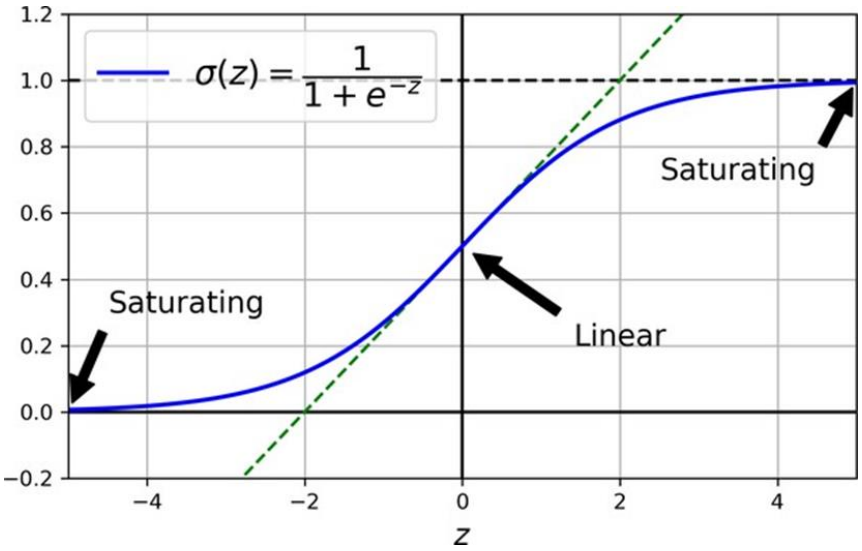
Thật không may, gradient thường trở nên nhỏ hơn và nhỏ hơn khi thuật toán tiến xuống các lớp thấp hơn. Do đó, bản cập nhật giảm độ dốc làm cho trọng số kết nối của các lớp thấp hơn hầu như không thay đổi, và quá trình huấn luyện không bao giờ hội tụ đến một giải pháp tốt. Đây được gọi là **vấn đề gradient biến mất**.

Trong một số trường hợp, điều ngược lại có thể xảy ra: gradient có thể ngày càng lớn hơn cho đến khi các lớp nhận được các bản cập nhật trọng số lớn bất thường và thuật toán phân kỳ. Đây là **vấn đề gradient bùng nổ**, thường xuất hiện nhất trong các mạng nơ-ron hồi

quy (xem Chương 15). Tổng quát hơn, các mạng nơ-ron sâu gặp phải các gradient không ổn định; các lớp khác nhau có thể học ở tốc độ rất khác nhau.

Hành vi không may này đã được quan sát theo kinh nghiệm từ lâu, và đó là một trong những lý do khiến mạng nơ-ron sâu phần lớn bị bỏ rơi vào đầu những năm 2000. Không rõ điều gì gây ra các gradient không ổn định như vậy khi huấn luyện DNN, nhưng một số ánh sáng đã được làm sáng tỏ trong một bài báo năm 2010 của Xavier Glorot và Yoshua Bengio. Các tác giả đã tìm thấy một vài nghi phạm, bao gồm sự kết hợp của hàm kích hoạt sigmoid (logistic) phổ biến và kỹ thuật khởi tạo trọng số phổ biến nhất vào thời điểm đó (tức là phân phối chuẩn với trung bình 0 và độ lệch chuẩn 1). Tóm lại, họ chỉ ra rằng với hàm kích hoạt này và sơ đồ khởi tạo này, phương sai của đầu ra của mỗi lớp lớn hơn nhiều so với phương sai của đầu vào của nó. Đi tiếp trong mạng, phương sai tiếp tục tăng sau mỗi lớp cho đến khi hàm kích hoạt bão hòa ở các lớp trên cùng. Sự bão hòa này thực sự trở nên tồi tệ hơn bởi thực tế là hàm sigmoid có trung bình 0.5, không phải 0 (hàm hyperbolic tangent có trung bình 0 và hoạt động tốt hơn một chút so với hàm sigmoid trong các mạng sâu).

Nhìn vào hàm kích hoạt sigmoid (xem Hình 11-1), bạn có thể thấy rằng khi đầu vào trở nên lớn (âm hoặc dương), hàm bão hòa ở 0 hoặc 1, với đạo hàm cực kỳ gần 0 (tức là đường cong phẳng ở cả hai thái cực). Do đó, khi backpropagation bắt đầu, nó hầu như không có gradient nào để truyền ngược qua mạng, và gradient ít ỏi tồn tại tiếp tục bị pha loãng khi backpropagation tiến xuống các lớp trên cùng, vì vậy thực sự không còn gì cho các lớp thấp hơn.



Hình 11-1. Sự bão hòa của hàm kích hoạt Sigmoid

Khởi tạo Glorot và He

Trong bài báo của họ, Glorot và Bengio đề xuất một cách để giảm đáng kể vấn đề gradient không ổn định. Họ chỉ ra rằng chúng ta cần tín hiệu truyền đúng cách theo cả hai hướng: theo hướng tiến khi đưa ra dự đoán và theo hướng ngược lại khi backpropagating gradient. Chúng ta không muốn tín hiệu chết đi, cũng không muốn nó bùng nổ và bão hòa. Để tín

hiệu truyền đúng cách, các tác giả lập luận rằng chúng ta cần phương sai của đầu ra của mỗi lớp bằng phương sai của đầu vào của nó, và chúng ta cần các gradient có phương sai bằng nhau trước và sau khi truyền qua một lớp theo hướng ngược lại (vui lòng kiểm tra bài báo nếu bạn quan tâm đến các chi tiết toán học). Thực tế không thể đảm bảo cả hai trừ khi lớp có số lượng đầu vào và đầu ra bằng nhau (các số này được gọi là *fan-in* và *fan-out* của lớp), nhưng Glorot và Bengio đã đề xuất một sự thỏa hiệp tốt đã được chứng minh là hoạt động rất tốt trong thực tế: trọng số kết nối của mỗi lớp phải được khởi tạo ngẫu nhiên như được mô tả trong Công thức 11-1, trong đó $fan_{avg} = (fan_{in} + fan_{out})/2$. Chiến lược khởi tạo này được gọi là **khởi tạo Xavier** hoặc **khởi tạo Glorot**, theo tên tác giả đầu tiên của bài báo.

Công thức 11-1. Khởi tạo Glorot (khi sử dụng hàm kích hoạt sigmoid)

Phân phối chuẩn với trung bình 0 và phương sai $\sigma^2 = \frac{1}{fan_{avg}}$

Hoặc phân phối đều giữa $-r$ và $+r$, với $r = \sqrt{\frac{3}{fan_{avg}}}$

Nếu bạn thay thế fan_avg bằng fan_in trong Công thức 11-1, bạn sẽ nhận được một chiến lược khởi tạo mà Yann LeCun đã đề xuất vào những năm 1990. Ông gọi đó là **khởi tạo LeCun**. Genevieve Orr và Klaus-Robert Müller thậm chí còn khuyến nghị nó trong cuốn sách *Neural Networks: Tricks of the Trade* (Springer) năm 1998 của họ. Khởi tạo LeCun tương đương với khởi tạo Glorot khi $r = \sqrt{3\sigma^2}$. Phải mất hơn một thập kỷ để các nhà nghiên cứu nhận ra mẹo này quan trọng đến mức nào. Sử dụng khởi tạo Glorot có thể tăng tốc đáng kể quá trình huấn luyện, và nó là một trong những thực hành đã dẫn đến thành công của học sâu.

Một số bài báo đã cung cấp các chiến lược tương tự cho các hàm kích hoạt khác nhau. Các chiến lược này chỉ khác nhau về thang đo phương sai và liệu chúng có sử dụng fan_avg hay fan_in hay không, như thể hiện trong Bảng 11-1 (đối với phân phối đều, chỉ cần sử dụng $r = sqrt3sigma^2$). Chiến lược khởi tạo được đề xuất cho hàm kích hoạt ReLU và các biến thể của nó được gọi là **khởi tạo He** hoặc **khởi tạo Kaiming**, theo tên tác giả đầu tiên của bài báo. Đối với SELU, hãy sử dụng phương pháp khởi tạo của Yann LeCun, tốt nhất là với phân phối chuẩn. Chúng ta sẽ đề cập đến tất cả các hàm kích hoạt này ngay sau đây.

Bảng 11-1. Các tham số khởi tạo cho từng loại hàm kích hoạt

Khởi tạo	Hàm kích hoạt	σ^2 (Chuẩn)
Glorot	None, tanh, sigmoid, softmax	$1/fan_{avg}$
He	ReLU, Leaky ReLU, ELU, GELU, Swish, Mish	$2/fan_{in}$
LeCun	SELU	$1/fan_{in}$

Theo mặc định, Keras sử dụng khởi tạo Glorot với phân phối đều. Khi bạn tạo một lớp, bạn có thể chuyển sang khởi tạo He bằng cách đặt `kernel_initializer="he_uniform"` hoặc `kernel_initializer="he_normal"` như sau:

```
import tensorflow as tf

dense = tf.keras.layers.Dense(50, activation="relu",
kernel_initializer="he_normal")
```

Ngoài ra, bạn có thể nhận được bất kỳ khởi tạo nào được liệt kê trong Bảng 11-1 và hơn thế nữa bằng cách sử dụng trình khởi tạo VarianceScaling. Ví dụ, nếu bạn muốn khởi tạo He với phân phối đều và dựa trên fan_avg (thay vì fan_in), bạn có thể sử dụng đoạn mã sau:

```
he_avg_init = tf.keras.initializers.VarianceScaling(scale=2., mode="fan_avg",
distribution="uniform")
dense = tf.keras.layers.Dense(50, activation="sigmoid",
kernel_initializer=he_avg_init)
```

Các hàm kích hoạt tốt hơn

Một trong những nhận thức trong bài báo năm 2010 của Glorot và Bengio là các vấn đề về gradient không ổn định một phần là do lựa chọn hàm kích hoạt kém. Cho đến lúc đó, hầu hết mọi người đều cho rằng nếu Mẹ Thiên nhiên đã chọn sử dụng các hàm kích hoạt gần như sigmoid trong các nơ-ron sinh học, thì chúng phải là một lựa chọn tuyệt vời. Nhưng hóa ra các hàm kích hoạt khác hoạt động tốt hơn nhiều trong mạng nơ-ron sâu — đặc biệt là hàm kích hoạt ReLU, chủ yếu vì nó không bão hòa đối với các giá trị dương, và cũng vì nó rất nhanh để tính toán.

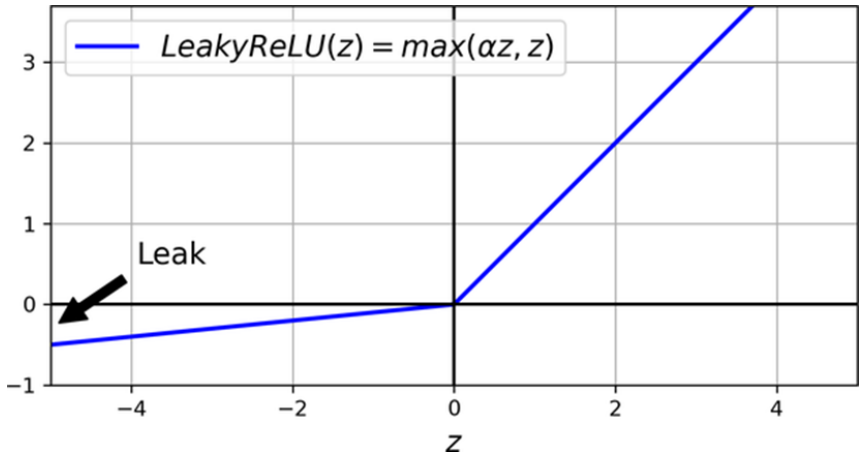
Thật không may, hàm kích hoạt ReLU không hoàn hảo. Nó gặp phải một vấn đề được gọi là **ReLU chết (dying ReLUs)**: trong quá trình huấn luyện, một số nơ-ron thực sự "chết", có nghĩa là chúng ngừng xuất ra bất cứ thứ gì khác ngoài 0. Trong một số trường hợp, bạn có thể thấy rằng một nửa số nơ-ron của mạng đã chết, đặc biệt nếu bạn sử dụng tốc độ học lớn. Một nơ-ron chết khi trọng số của nó được điều chỉnh theo cách mà đầu vào của hàm ReLU (tức là tổng có trọng số của đầu vào của nơ-ron cộng với số hạng độ lệch của nó) là âm đối với tất cả các thể hiện trong tập huấn luyện. Khi điều này xảy ra, nó chỉ tiếp tục xuất ra các số 0, và giảm độ dốc không còn ảnh hưởng đến nó nữa vì gradient của hàm ReLU bằng 0 khi đầu vào của nó là âm.

Để giải quyết vấn đề này, bạn có thể muốn sử dụng một biến thể của hàm ReLU, chẳng hạn như leaky ReLU.

Leaky ReLU

Hàm kích hoạt leaky ReLU được định nghĩa là $textLeakyReLU_alpha(z) = max(alphaz, z)$ (xem Hình 11-2). Siêu tham số $alpha$ định nghĩa mức độ hàm "rò rỉ": đó là độ dốc của hàm đối với $z \< 0$. Có một độ dốc đối với $z \< 0$ đảm bảo rằng leaky ReLU không bao giờ chết; chúng có thể rơi vào tình trạng hôn mê dài, nhưng chúng có cơ hội cuối cùng thức dậy. Một bài báo năm 2015 của Bing Xu và cộng sự đã so sánh một số biến thể của hàm kích hoạt ReLU, và một trong những kết luận của nó là các biến thể rò rỉ luôn hoạt động tốt hơn hàm kích hoạt ReLU nghiêm ngặt. Trên thực tế, đặt $alpha = 0.2$ (một rò rỉ lớn) dường như mang lại hiệu suất tốt hơn so với $alpha = 0.01$ (một rò rỉ nhỏ). Bài báo cũng đánh giá

randomized leaky ReLU (RReLU), trong đó $alpha$ được chọn ngẫu nhiên trong một phạm vi nhất định trong quá trình huấn luyện và được cố định thành một giá trị trung bình trong quá trình kiểm tra. RReLU cũng hoạt động khá tốt và dường như hoạt động như một bộ chính quy hóa, giảm nguy cơ overfitting tập huấn luyện. Cuối cùng, bài báo đã đánh giá parametric leaky ReLU (PReLU), trong đó $alpha$ được phép học trong quá trình huấn luyện: thay vì là một siêu tham số, nó trở thành một tham số có thể được sửa đổi bằng backpropagation như bất kỳ tham số nào khác. PReLU được báo cáo là hoạt động tốt hơn đáng kể so với ReLU trên các tập dữ liệu hình ảnh lớn, nhưng trên các tập dữ liệu nhỏ hơn, nó có nguy cơ overfitting tập huấn luyện.



Hình 11-2. Leaky ReLU: giống như ReLU, nhưng có độ dốc nhỏ cho các giá trị âm

Keras bao gồm các lớp `LeakyReLU` và `PReLU` trong gói `tf.keras.layers`. Giống như các biến thể ReLU khác, bạn nên sử dụng khởi tạo He với chúng. Ví dụ:

```
leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.2) # defaults to alpha=0.3
dense = tf.keras.layers.Dense(50, activation=leaky_relu,
kernel_initializer="he_normal")
```

Nếu bạn thích, bạn cũng có thể sử dụng `LeakyReLU` làm một lớp riêng biệt trong mô hình của mình; điều này không tạo ra sự khác biệt nào cho quá trình huấn luyện và dự đoán:

```
model = tf.keras.models.Sequential([
    # ... more layers
    tf.keras.layers.Dense(50, kernel_initializer="he_normal"), # no
activation
    tf.keras.layers.LeakyReLU(alpha=0.2), # activation as a separate layer
    # ... more layers
])
```

Đối với PReLU, hãy thay thế `LeakyReLU` bằng `PReLU`. Hiện tại không có triển khai chính thức của RReLU trong Keras, nhưng bạn có thể khá dễ dàng triển khai của riêng mình (để tìm hiểu cách làm điều đó, xem các bài tập ở cuối Chương 12).

ReLU, leaky ReLU và PReLU đều gặp phải vấn đề là chúng không phải là các hàm trơn: đạo hàm của chúng thay đổi đột ngột (tại $z = 0$). Như chúng ta đã thấy trong Chương 4 khi thảo luận về lasso, loại gián đoạn này có thể khiến giảm độ dốc dao động quanh cực tiểu và làm chậm quá trình hội tụ. Vì vậy, bây giờ chúng ta sẽ xem xét một số biến thể trơn của hàm kích hoạt ReLU, bắt đầu với ELU và SELU.

ELU và SELU

Vào năm 2015, một bài báo của Djork-Arné Clevert và cộng sự đã đề xuất một hàm kích hoạt mới, được gọi là **đơn vị tuyến tính mũ (ELU)**, đã hoạt động tốt hơn tất cả các biến thể ReLU trong các thí nghiệm của các tác giả: thời gian huấn luyện được giảm, và mạng nơ-ron hoạt động tốt hơn trên tập kiểm tra. Công thức 11-2 cho thấy định nghĩa của hàm kích hoạt này.

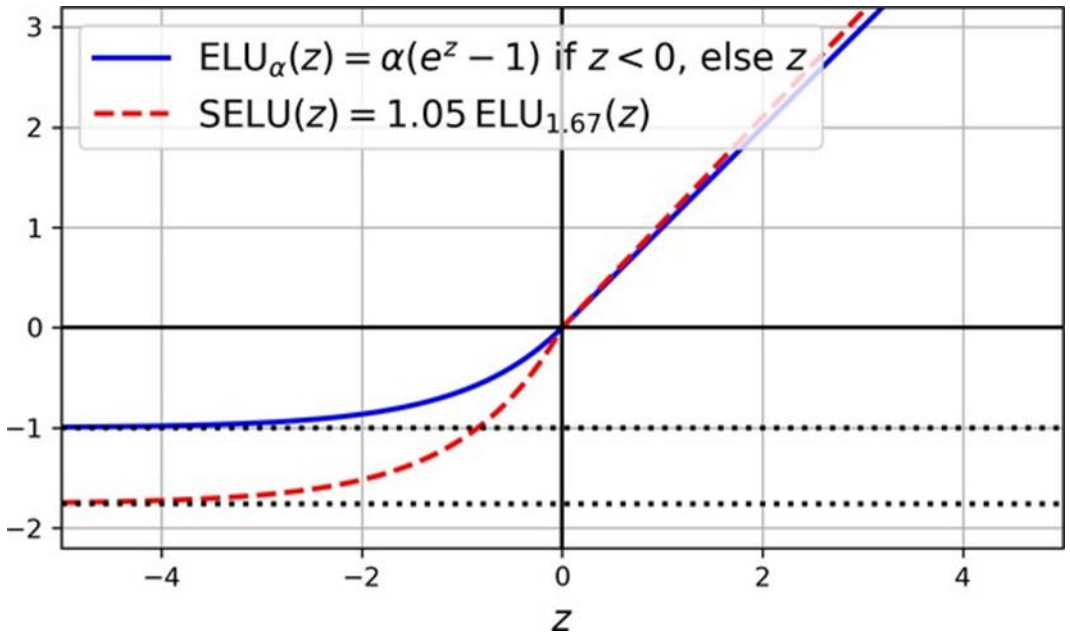
Công thức 11-2. Hàm kích hoạt ELU

$$\mathrm{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

Hàm kích hoạt ELU trông rất giống hàm ReLU (xem Hình 11-3), với một vài khác biệt lớn:

- Nó nhận các giá trị âm khi $z < 0$, điều này cho phép đơn vị có đầu ra trung bình gần 0 hơn và giúp giảm nhẹ vấn đề gradient biến mất. Siêu tham số $alpha$ định nghĩa ngược lại giá trị mà hàm ELU tiến tới khi z là một số âm lớn. Nó thường được đặt là 1, nhưng bạn có thể điều chỉnh nó như bất kỳ siêu tham số nào khác.
- Nó có một gradient khác 0 cho $z < 0$, điều này tránh được vấn đề nơ-ron chết.
- Nếu $alpha$ bằng 1 thì hàm trơn ở khắp mọi nơi, bao gồm cả quanh $z = 0$, điều này giúp tăng tốc giảm độ dốc vì nó không dao động nhiều sang trái và phải của $z = 0$.

Sử dụng ELU với Keras dễ dàng như đặt `activation="elu"`, và giống như các biến thể ReLU khác, bạn nên sử dụng khởi tạo He. Nhược điểm chính của hàm kích hoạt ELU là nó chậm hơn khi tính toán so với hàm ReLU và các biến thể của nó (do sử dụng hàm mũ). Tốc độ hội tụ nhanh hơn trong quá trình huấn luyện có thể bù đ đắp cho việc tính toán chậm đó, nhưng dù sao, ở thời điểm kiểm tra, một mạng ELU sẽ chậm hơn một chút so với một mạng ReLU.



Hình 11-3. Các hàm kích hoạt ELU và SELU

Không lâu sau đó, một bài báo năm 2017 của Günter Klambauer và cộng sự đã giới thiệu hàm kích hoạt **SELU (scaled ELU)**: đúng như tên gọi của nó, đây là một biến thể được điều chỉnh theo tỷ lệ của hàm kích hoạt ELU (khoảng 1.05 lần ELU, sử dụng $alphaapprox1.67$). Các tác giả đã chỉ ra rằng nếu bạn xây dựng một mạng nơ-ron chỉ bao gồm một chồng các lớp dày đặc (tức là một MLP), và nếu tất cả các lớp ẩn đều sử dụng hàm kích hoạt SELU, thì mạng sẽ tự chuẩn hóa: đầu ra của mỗi lớp sẽ có xu hướng giữ trung bình 0 và độ lệch chuẩn 1 trong quá trình huấn luyện, điều này giải quyết vấn đề gradient biến mất/bùng nổ. Kết quả là, hàm kích hoạt SELU có thể hoạt động tốt hơn các hàm kích hoạt khác cho MLP, đặc biệt là các mạng sâu. Để sử dụng nó với Keras, chỉ cần đặt `activation="selu"`. Tuy nhiên, có một vài điều kiện để tự chuẩn hóa xảy ra (xem bài báo để biết lý do toán học):

- Các đặc trưng đầu vào phải được chuẩn hóa: trung bình 0 và độ lệch chuẩn 1.
- Trọng số của mỗi lớp ẩn phải được khởi tạo bằng khởi tạo chuẩn LeCun. Trong Keras, điều này có nghĩa là đặt `kernel_initializer="lecun_normal"`.
- Thuộc tính tự chuẩn hóa chỉ được đảm bảo với các MLP đơn giản. Nếu bạn cố gắng sử dụng SELU trong các kiến trúc khác, như mạng hồi quy (xem Chương 15) hoặc mạng có kết nối bỏ qua (tức là các kết nối bỏ qua các lớp, chẳng hạn như trong mạng Wide & Deep), nó có lẽ sẽ không hoạt động tốt hơn ELU.
- Bạn không thể sử dụng các kỹ thuật chính quy hóa như chính quy hóa l_1 hoặc l_2 , max-norm, batch-norm hoặc dropout thông thường (những điều này sẽ được thảo luận sau trong chương này).

Đây là những ràng buộc đáng kể, vì vậy mặc dù có những hứa hẹn, SELU đã không được chấp nhận rộng rãi. Hơn nữa, ba hàm kích hoạt khác dường như hoạt động tốt hơn nó một cách khá nhất quán trên hầu hết các tác vụ: GELU, Swish và Mish.

GELU, Swish và Mish

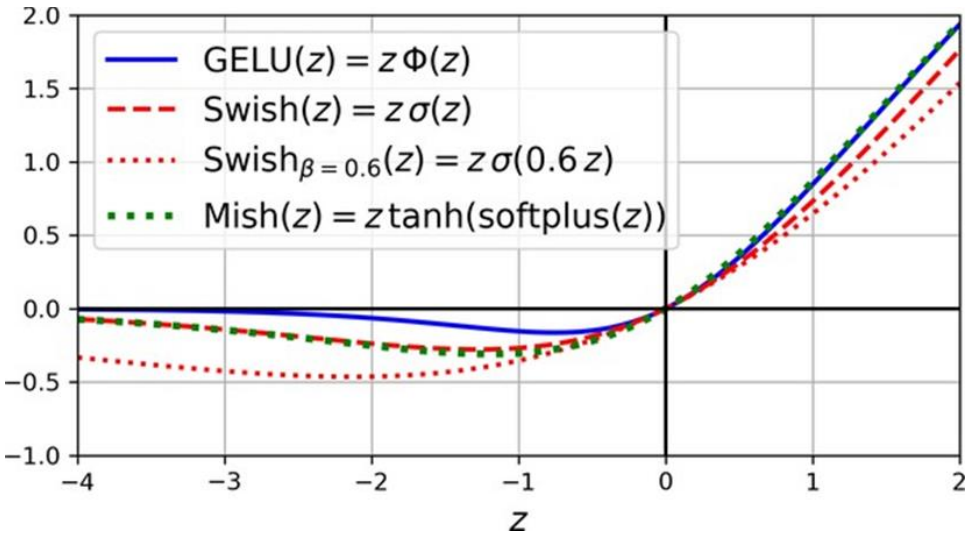
GELU được giới thiệu trong một bài báo năm 2016 của Dan Hendrycks và Kevin Gimpel. Một lần nữa, bạn có thể coi nó là một biến thể trơn của hàm kích hoạt ReLU. Định nghĩa của nó được đưa ra trong Công thức 11-3, trong đó Phi là hàm phân phối tích lũy Gaussian chuẩn (CDF): $Phi(z)$ tương ứng với xác suất mà một giá trị được lấy mẫu ngẫu nhiên từ phân phối chuẩn có trung bình 0 và phương sai 1 thấp hơn z .

Công thức 11-3. Hàm kích hoạt GELU

$$GELU(z) = zPhi(z)$$

Như bạn có thể thấy trong Hình 11-4, GELU giống như ReLU: nó tiến về 0 khi đầu vào z rất âm, và nó tiến về z khi z rất dương. Tuy nhiên, trong khi tất cả các hàm kích hoạt chúng ta đã thảo luận cho đến nay đều vừa lồi vừa đơn điệu, hàm kích hoạt GELU thì không: từ trái sang phải, nó bắt đầu đi thẳng, sau đó nó uốn lượn xuống, đạt đến điểm thấp nhất khoảng -0.17 (gần $z \approx 0.75$), và cuối cùng bật lên và kết thúc đi thẳng lên phía trên bên phải. Hình dạng khá phức tạp này và thực tế là nó có độ cong ở mọi điểm có thể giải thích tại sao nó hoạt động rất tốt, đặc biệt đối với các tác vụ phức tạp: giảm độ dốc có thể thấy dễ dàng hơn để phù hợp với các mẫu phức tạp. Trong thực tế, nó thường hoạt động tốt hơn mọi hàm kích hoạt khác đã được thảo luận cho đến nay.

Tuy nhiên, nó tốn nhiều tính toán hơn một chút, và sự tăng cường hiệu suất mà nó cung cấp không phải lúc nào cũng đủ để biện minh cho chi phí bổ sung. Điều đó nói lên rằng, có thể chứng minh rằng nó xấp xỉ bằng $z\sigma(1.702z)$, trong đó σ là hàm sigmoid: sử dụng xấp xỉ này cũng hoạt động rất tốt, và nó có ưu điểm là tính toán nhanh hơn nhiều.



Hình 11-4. Các hàm kích hoạt GELU, Swish, Swish tham số hóa và Mish

Bài báo GELU cũng giới thiệu hàm kích hoạt đơn vị tuyến tính sigmoid (SiLU), bằng $z\sigma(z)$, nhưng nó đã bị GELU vượt trội trong các thử nghiệm của các tác giả.

Điều thú vị là, một bài báo năm 2017 của Prajit Ramachandran và cộng sự đã khám phá lại hàm SiLU bằng cách tự động tìm kiếm các hàm kích hoạt tốt. Các tác giả đã đặt tên nó là Swish, và cái tên này đã được chấp nhận. Trong bài báo của họ, Swish đã vượt trội hơn mọi hàm khác, bao gồm cả GELU. Ramachandran và cộng sự sau đó đã tổng quát hóa Swish bằng cách thêm một siêu tham số $beta$ để điều chỉnh đầu vào của hàm sigmoid. Hàm Swish tổng quát là $Swish_\beta(z) = z\sigma(\beta z)$, vì vậy GELU xấp xỉ bằng hàm Swish tổng quát sử dụng $\beta = 1.702$. Bạn có thể điều chỉnh β như bất kỳ siêu tham số nào khác. Ngoài ra, cũng có thể làm cho $beta$ có thể huấn luyện và để giảm độ dốc tối ưu hóa nó: giống như PReLU, điều này có thể làm cho mô hình của bạn mạnh mẽ hơn, nhưng nó cũng có nguy cơ overfitting dữ liệu.

Một hàm kích hoạt khá tương tự khác là Mish, được giới thiệu trong một bài báo năm 2019 của Diganta Misra. Nó được định nghĩa là $mish(z) = ztanh\big(softplus(z)\big)$, trong đó $softplus(z) = log\big(1 + exp(z)\big)$. Giống như GELU và Swish, nó là một biến thể trơn, không lồi và không đơn điệu của ReLU, và một lần nữa tác giả đã thực hiện nhiều thí nghiệm và thấy rằng Mish nói chung hoạt động tốt hơn các hàm kích hoạt khác - thậm chí cả Swish và GELU, với một biên độ nhỏ. Hình 11-4 hiển thị GELU, Swish (cả với $\beta = 1$ mặc định và với $\beta = 0.6$), và cuối cùng là Mish. Như bạn có thể thấy, Mish chồng lên gần như hoàn hảo với Swish khi z âm, và gần như hoàn hảo với GELU khi z dương.

Keras hỗ trợ GELU và Swish ngay lập tức; chỉ cần sử dụng `activation="gelu"` hoặc `activation="swish"`. Tuy nhiên, nó chưa hỗ trợ Mish hoặc hàm kích hoạt Swish tổng quát (nhưng xem Chương 12 để biết cách triển khai các hàm kích hoạt và lớp của riêng bạn).

Đó là tất cả về các hàm kích hoạt! Bây giờ, hãy xem xét một cách hoàn toàn khác để giải quyết vấn đề gradient không ổn định: chuẩn hóa theo batch (batch normalization).

Chuẩn hóa theo Batch

Mặc dù sử dụng khởi tạo He cùng với ReLU (hoặc bất kỳ biến thể nào của nó) có thể giảm đáng kể nguy cơ của các vấn đề gradient biến mất/bùng nổ khi bắt đầu huấn luyện, nhưng nó không đảm bảo rằng chúng sẽ không quay trở lại trong quá trình huấn luyện.

Trong một bài báo năm 2015, Sergey Ioffe và Christian Szegedy đã đề xuất một kỹ thuật được gọi là **chuẩn hóa theo batch (BN)** giải quyết các vấn đề này. Kỹ thuật này bao gồm việc thêm một thao tác vào mô hình ngay trước hoặc sau hàm kích hoạt của mỗi lớp ẩn. Thao tác này chỉ đơn giản là đưa về tâm 0 và chuẩn hóa từng đầu vào, sau đó điều chỉnh tỷ lệ và dịch chuyển kết quả bằng cách sử dụng hai vector tham số mới cho mỗi lớp: một cho điều chỉnh tỷ lệ, một cho dịch chuyển. Nói cách khác, thao tác này cho phép mô hình học được tỷ lệ và trung bình tối ưu của từng đầu vào của lớp. Trong nhiều trường hợp, nếu bạn thêm một lớp BN làm lớp đầu tiên của mạng nơ-ron của bạn, bạn không cần phải chuẩn hóa tập huấn luyện của mình. Tức là, không cần `StandardScaler` hoặc `Normalization`; lớp BN sẽ làm điều đó cho bạn (tất nhiên, gần đúng, vì nó chỉ xem xét từng batch một, và nó cũng có thể điều chỉnh tỷ lệ và dịch chuyển từng đặc trưng đầu vào).

Để đưa về tâm 0 và chuẩn hóa các đầu vào, thuật toán cần ước tính trung bình và độ lệch chuẩn của từng đầu vào. Nó làm như vậy bằng cách đánh giá trung bình và độ lệch chuẩn của đầu vào trên mini-batch hiện tại (do đó có tên "chuẩn hóa theo batch"). Toàn bộ thao tác được tóm tắt từng bước trong Công thức 11-4.

Công thức 11-4. Thuật toán chuẩn hóa theo batch

$$
\begin{aligned}
1.\quad & \boldsymbol{\mu}_B = \frac{1}{m_B}\sum_{i=1}^{m_B}\mathbf{x}^{(i)} \\
2.\quad & {\boldsymbol{\sigma}_B}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B}\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_B\right)^2 \\
3.\quad & \widehat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \boldsymbol{\mu}_B}{\sqrt{{\boldsymbol{\sigma}_B}^2 + \varepsilon}} \\
4.\quad & \mathbf{z}^{(i)} = \boldsymbol{\gamma} \otimes \widehat{\mathbf{x}}^{(i)} + \boldsymbol{\beta}
\end{aligned}
$$

Trong thuật toán này:

- μ_B là **vector trung bình đầu vào**, được tính trên toàn bộ mini-batch B (nó chứa một giá trị trung bình cho mỗi đầu vào).
- m_B là **số lượng trường hợp** trong mini-batch.
- σ_B là **vector độ lệch chuẩn đầu vào**, cũng được tính trên toàn bộ mini-batch (nó chứa một độ lệch chuẩn cho mỗi đầu vào).
- $x^{(i)}$ là **vector đầu vào đã được chuẩn hóa về 0** và được chuẩn hóa cho trường hợp i .
- ϵ là một số rất nhỏ để **tránh phép chia cho 0** và đảm bảo gradient không trở nên quá lớn (thường là 10^{-5}). Số này được gọi là **số hạng làm mịn** (smoothing term).
- γ là **vector tham số tỷ lệ đầu ra** cho lớp (nó chứa một tham số tỷ lệ cho mỗi đầu vào).

Vì vậy, trong quá trình huấn luyện, BN chuẩn hóa các đầu vào của nó, sau đó điều chỉnh tỷ lệ và bù đắp chúng. Tốt! Còn ở thời điểm kiểm tra thì sao? Chà, không đơn giản như vậy. Thật vậy, chúng ta có thể cần đưa ra dự đoán cho các thể hiện riêng lẻ thay vì cho các batch của thể hiện: trong trường hợp này, chúng ta sẽ không có cách nào để tính toán trung bình và độ lệch chuẩn của mỗi đầu vào. Hơn nữa, ngay cả khi chúng ta có một batch các thể hiện, nó có thể quá nhỏ, hoặc các thể hiện có thể không độc lập và phân phối giống nhau, vì vậy việc tính toán thống kê trên các thể hiện batch sẽ không đáng tin cậy. Một giải pháp có thể là đợi đến cuối quá trình huấn luyện, sau đó chạy toàn bộ tập huấn luyện qua mạng nơ-ron và tính toán trung bình và độ lệch chuẩn của mỗi đầu vào của lớp BN. Các trung bình và độ lệch chuẩn đầu vào "cuối cùng" này sau đó có thể được sử dụng thay cho trung bình và độ lệch chuẩn đầu vào batch khi đưa ra dự đoán. Tuy nhiên, hầu hết các triển khai chuẩn hóa theo batch ước tính các thống kê cuối cùng này trong quá trình huấn luyện bằng cách sử dụng trung bình động của trung bình đầu vào và độ lệch chuẩn của lớp. Đây là điều Keras tự động làm khi bạn sử dụng lớp `BatchNormalization`. Tóm lại, bốn vector tham số được học trong mỗi lớp chuẩn hóa theo batch: $gamma$ (vector tỷ lệ đầu ra) và $beta$

(vector bù đắp đầu ra) được học thông qua backpropagation thông thường, và mu (vector trung bình đầu vào cuối cùng) và $sigma$ (vector độ lệch chuẩn đầu vào cuối cùng) được ước tính bằng cách sử dụng trung bình động mũ. Lưu ý rằng mu và $sigma$ được ước tính trong quá trình huấn luyện, nhưng chúng chỉ được sử dụng sau khi huấn luyện (để thay thế trung bình đầu vào batch và độ lệch chuẩn trong Công thức 11-4).

Ioffe và Szegedy đã chứng minh rằng chuẩn hóa theo batch cải thiện đáng kể tất cả các mạng nơ-ron sâu mà họ đã thử nghiệm, dẫn đến một cải thiện lớn trong tác vụ phân loại ImageNet (ImageNet là một cơ sở dữ liệu lớn các hình ảnh được phân loại thành nhiều lớp, thường được sử dụng để đánh giá các hệ thống thị giác máy tính). Vấn đề gradient biến mất đã được giảm mạnh, đến mức họ có thể sử dụng các hàm kích hoạt bão hòa như tanh và thậm chí cả hàm kích hoạt sigmoid. Các mạng cũng ít nhạy cảm hơn nhiều với khởi tạo trọng số. Các tác giả đã có thể sử dụng tốc độ học lớn hơn nhiều, tăng tốc đáng kể quá trình học. Cụ thể, họ lưu ý rằng: Áp dụng cho một mô hình phân loại hình ảnh hiện đại, chuẩn hóa theo batch đạt được cùng độ chính xác với số bước huấn luyện ít hơn 14 lần, và vượt trội mô hình gốc với một biên độ đáng kể. [...] Sử dụng một tập hợp các mạng được chuẩn hóa theo batch, chúng tôi cải thiện kết quả tốt nhất đã được công bố về phân loại ImageNet: đạt 4.9% lỗi xác thực top-5 (và 4.8% lỗi kiểm tra), vượt quá độ chính xác của người đánh giá.

Cuối cùng, như một món quà tiếp tục mang lại, chuẩn hóa theo batch hoạt động như một bộ chính quy hóa, giảm nhu cầu về các kỹ thuật chính quy hóa khác (chẳng hạn như dropout, được mô tả sau trong chương này).

May mắn thay, thường có thể **hợp nhất lớp BN** với lớp trước đó sau khi huấn luyện, từ đó tránh được chi phí khi chạy. Việc này được thực hiện bằng cách cập nhật các trọng số và độ lệch của lớp trước đó sao cho nó trực tiếp tạo ra đầu ra với tỷ lệ và độ lệch phù hợp. Ví dụ, nếu lớp trước đó tính $XW + b$, thì lớp BN sẽ tính $\gamma \otimes (XW + b - \mu)/\sigma + \beta$ (bỏ qua số hạng làm mịn ϵ ở mẫu số). Nếu chúng ta định nghĩa $W' = \gamma \otimes W/\sigma$ và $b' = \gamma \otimes (b - \mu)/\sigma + \beta$, thì phương trình sẽ được đơn giản hóa thành $XW' + b'$. Vì vậy, nếu chúng ta thay thế các trọng số và độ lệch của lớp trước đó (W và b) bằng các trọng số và độ lệch đã cập nhật (W' và b'), chúng ta có thể loại bỏ lớp BN. (Bộ chuyển đổi của TFLite thực hiện điều này một cách tự động).

Triển khai chuẩn hóa theo batch với Keras

Với hầu hết mọi thứ với Keras, việc triển khai chuẩn hóa theo batch rất đơn giản và trực quan. Chỉ cần thêm một lớp `BatchNormalization` trước hoặc sau hàm kích hoạt của mỗi lớp ẩn. Bạn cũng có thể thêm một lớp BN làm lớp đầu tiên trong mô hình của mình, nhưng một lớp `Normalization` đơn giản thường hoạt động tốt như vậy ở vị trí này (nhược điểm duy nhất của nó là bạn phải gọi phương thức `adapt()` của nó trước). Ví dụ, mô hình này áp dụng BN sau mỗi lớp ẩn và làm lớp đầu tiên trong mô hình (sau khi làm phẳng hình ảnh đầu vào):

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.BatchNormalization(),
```

```
    tf.keras.layers.Dense(300, activation="relu",
kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100, activation="relu",
kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Vậy là xong! Trong ví dụ nhỏ này chỉ với hai lớp ẩn, chuẩn hóa theo batch có thể không có tác động lớn, nhưng đối với các mạng sâu hơn, nó có thể tạo ra sự khác biệt lớn.

Hãy hiển thị tóm tắt mô hình:

```
>>> model.summary()
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param#
=================================================================
flatten (Flatten)            (None, 784)               0
batch_normalization (BatchNo (None, 784)               3136
dense (Dense)                (None, 300)               235500
batch_normalization_1 (Batch (None, 300)               1200
dense_1 (Dense)              (None, 100)               30100
batch_normalization_2 (Batch (None, 100)               400
dense_2 (Dense)              (None, 10)                1010
=================================================================
Total params: 271,346
Trainable params: 268,978
Non-trainable params: 2,368
_________________________________________________________________
```

Như bạn có thể thấy, mỗi lớp BN thêm bốn tham số cho mỗi đầu vào: $gamma$, $beta$, mu và $sigma$ (ví dụ: lớp BN đầu tiên thêm 3.136 tham số, tức là 4 × 784). Hai tham số cuối cùng, mu và $sigma$, là các trung bình di chuyển; chúng không bị ảnh hưởng bởi backpropagation, vì vậy Keras gọi chúng là “không thể huấn luyện” (nếu bạn đếm tổng số tham số BN, 3.136 + 1.200 + 400, và chia cho 2, bạn sẽ nhận được 2.368, là tổng số tham số không thể huấn luyện trong mô hình này).

Hãy xem các tham số của lớp BN đầu tiên. Hai trong số đó có thể huấn luyện (bằng backpropagation), và hai không:

```
>>> [(var.name, var.trainable) for var in model.layers[1].variables]
[('batch_normalization/gamma:0', True),
('batch_normalization/beta:0', True),
('batch_normalization/moving_mean:0', False),
('batch_normalization/moving_variance:0', False)]
```

Các tác giả của bài báo BN đã ủng hộ việc thêm các lớp BN trước các hàm kích hoạt, thay vì sau (như chúng ta vừa làm). Có một số tranh cãi về điều này, vì điều nào được ưu tiên

dường như phụ thuộc vào tác vụ — bạn cũng có thể thử nghiệm điều này để xem tùy chọn nào hoạt động tốt nhất trên tập dữ liệu của mình. Để thêm các lớp BN trước hàm kích hoạt, bạn phải loại bỏ các hàm kích hoạt khỏi các lớp ẩn và thêm chúng làm các lớp riêng biệt sau các lớp BN. Hơn nữa, vì một lớp chuẩn hóa theo batch bao gồm một tham số offset cho mỗi đầu vào, bạn có thể loại bỏ số hạng bias khỏi lớp trước đó bằng cách truyền `use_bias=False` khi tạo nó. Cuối cùng, bạn thường có thể bỏ qua lớp BN đầu tiên để tránh kẹp lớp ẩn đầu tiên giữa hai lớp BN. Mã đã cập nhật trông như thế này:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, kernel_initializer="he_normal",
use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(100, kernel_initializer="he_normal",
use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Lớp `BatchNormalization` có khá nhiều siêu tham số bạn có thể điều chỉnh. Các giá trị mặc định thường sẽ ổn, nhưng đôi khi bạn có thể cần điều chỉnh `momentum`. Siêu tham số này được lớp `BatchNormalization` sử dụng khi nó cập nhật các trung bình di chuyển mũ; với một giá trị mới v (tức là một vector mới của trung bình đầu vào hoặc độ lệch chuẩn được tính trên batch hiện tại), lớp cập nhật trung bình chạy $hatv$ bằng phương trình sau:

$$\widehat{\mathbf{v}} \leftarrow \widehat{\mathbf{v}} \times \text{momentum} + \mathbf{v} \times (1 - \text{momentum})$$

Giá trị `momentum` tốt thường gần 1; ví dụ, 0.9, 0.99, hoặc 0.999. Bạn muốn nhiều số 9 hơn cho các tập dữ liệu lớn hơn và cho các mini-batch nhỏ hơn.

Một siêu tham số quan trọng khác là `axis`: nó xác định trục nào nên được chuẩn hóa. Nó mặc định là -1, có nghĩa là theo mặc định nó sẽ chuẩn hóa trục cuối cùng (sử dụng trung bình và độ lệch chuẩn được tính trên các trục khác). Khi batch đầu vào là 2D (tức là hình dạng batch là [kích thước batch, đặc trưng]), điều này có nghĩa là mỗi đặc trưng đầu vào sẽ được chuẩn hóa dựa trên trung bình và độ lệch chuẩn được tính trên tất cả các thể hiện trong batch. Ví dụ, lớp BN đầu tiên trong ví dụ mã trước đó sẽ độc lập chuẩn hóa (và điều chỉnh tỷ lệ và dịch chuyển) mỗi trong số 784 đặc trưng đầu vào. Nếu chúng ta di chuyển lớp BN đầu tiên trước lớp `Flatten`, thì các batch đầu vào sẽ là 3D, với hình dạng [kích thước batch, chiều cao, chiều rộng]; do đó, lớp BN sẽ tính toán 28 trung bình và 28 độ lệch chuẩn (1 cho mỗi cột pixel, được tính trên tất cả các thể hiện trong batch và trên tất cả các hàng trong cột), và nó sẽ chuẩn hóa tất cả các pixel trong một cột nhất định bằng cách sử dụng cùng trung bình và độ lệch chuẩn. Cũng sẽ chỉ có 28 tham số tỷ lệ và 28 tham số dịch chuyển. Nếu thay vào đó bạn vẫn muốn xử lý từng trong số 784 pixel một cách độc lập, thì bạn nên đặt `axis=[1, 2]`.

Chuẩn hóa theo batch đã trở thành một trong những lớp được sử dụng nhiều nhất trong các mạng nơ-ron sâu, đặc biệt là các mạng nơ-ron tích chập sâu được thảo luận trong (Chương 14), đến mức nó thường bị bỏ qua trong các sơ đồ kiến trúc: người ta giả định rằng BN được thêm vào sau mỗi lớp. Bây giờ chúng ta hãy xem một kỹ thuật cuối cùng để ổn định gradient trong quá trình huấn luyện: cắt gradient.

Cắt Gradient (Gradient Clipping)

Một kỹ thuật khác để giảm thiểu vấn đề gradient bùng nổ là cắt gradient trong quá trình backpropagation để chúng không bao giờ vượt quá một ngưỡng nhất định. Kỹ thuật này thường được sử dụng trong các mạng nơ-ron hồi quy, nơi việc sử dụng chuẩn hóa theo batch rất phức tạp (như bạn sẽ thấy trong Chương 15).

Trong Keras, việc triển khai cắt gradient chỉ đơn giản là đặt đối số `clipvalue` hoặc `clipnorm` khi tạo một bộ tối ưu hóa, như thế này:

```
optimizer = tf.keras.optimizers.SGD(clipvalue=1.0)
model.compile([...], optimizer=optimizer)
```

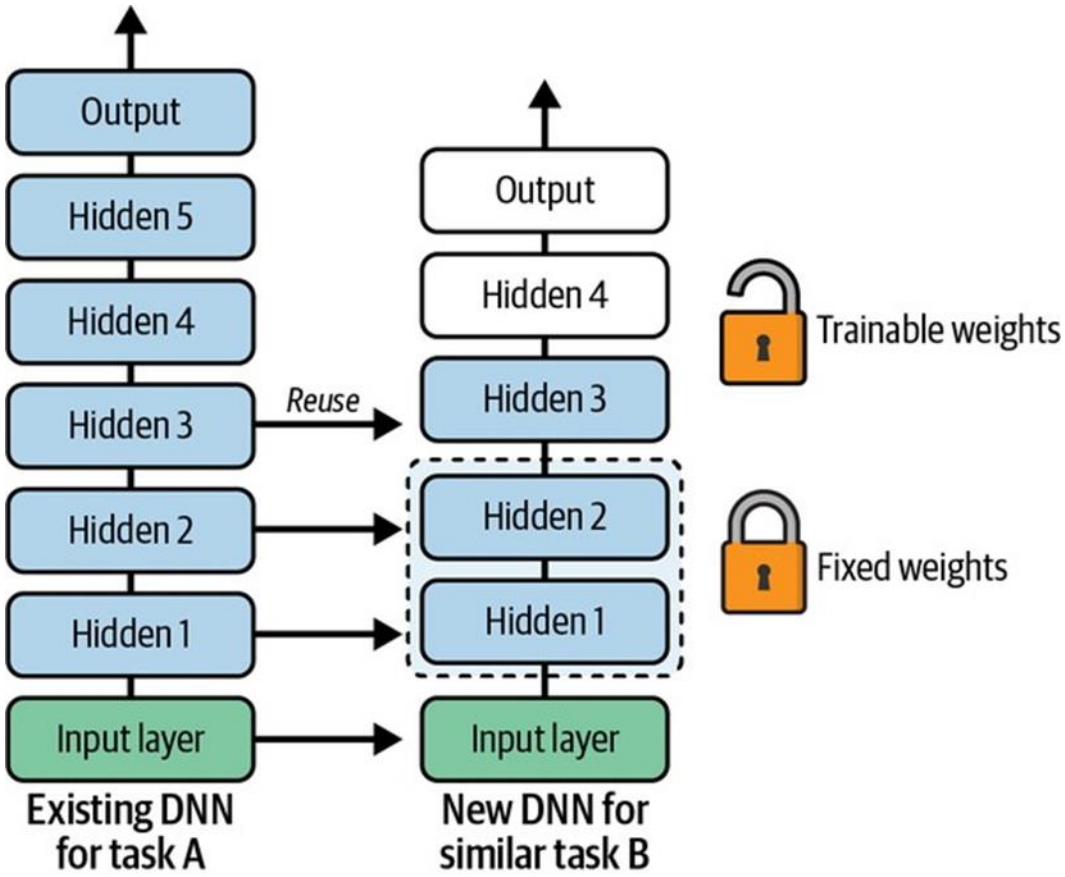
Bộ tối ưu hóa này sẽ cắt mọi thành phần của vector gradient xuống một giá trị giữa –1.0 và 1.0. Điều này có nghĩa là tất cả các đạo hàm riêng của hàm mất mát (đối với từng tham số có thể huấn luyện) sẽ được cắt giữa –1.0 và 1.0. Ngưỡng là một siêu tham số bạn có thể điều chỉnh. Lưu ý rằng nó có thể thay đổi hướng của vector gradient. Ví dụ, nếu vector gradient gốc là `[0.9, 100.0]`, nó chủ yếu chỉ theo hướng của trục thứ hai; nhưng một khi bạn cắt nó theo giá trị, bạn sẽ nhận được `[0.9, 1.0]`, cái này chỉ gần như theo đường chéo giữa hai trục. Trong thực tế, cách tiếp cận này hoạt động tốt. Nếu bạn muốn đảm bảo rằng cắt gradient không làm thay đổi hướng của vector gradient, bạn nên cắt theo chuẩn bằng cách đặt `clipnorm` thay vì `clipvalue`. Điều này sẽ cắt toàn bộ gradient nếu chuẩn `l2` của nó lớn hơn ngưỡng bạn đã chọn. Ví dụ, nếu bạn đặt `clipnorm=1.0`, thì vector `[0.9, 100.0]` sẽ được cắt thành `[0.00899964, 0.9999595]`, giữ nguyên hướng nhưng gần như loại bỏ thành phần đầu tiên. Nếu bạn quan sát thấy các gradient bùng nổ trong quá trình huấn luyện (bạn có thể theo dõi kích thước của các gradient bằng TensorBoard), bạn có thể muốn thử cắt theo giá trị hoặc cắt theo chuẩn, với các ngưỡng khác nhau, và xem tùy chọn nào hoạt động tốt nhất trên tập xác thực.

Tái sử dụng các lớp đã được huấn luyện trước

Nói chung, không nên huấn luyện một DNN rất lớn từ đầu mà không thử tìm một mạng nơ-ron hiện có thực hiện một tác vụ tương tự với tác vụ bạn đang cố gắng giải quyết (tôi sẽ thảo luận cách tìm chúng trong Chương 14). Nếu bạn tìm thấy một mạng nơ-ron như vậy, thì bạn thường có thể tái sử dụng hầu hết các lớp của nó, ngoại trừ các lớp trên cùng. Kỹ thuật này được gọi là **học chuyển giao**. Nó sẽ không chỉ tăng tốc quá trình huấn luyện đáng kể, mà còn yêu cầu ít dữ liệu huấn luyện hơn đáng kể.

Giả sử bạn có quyền truy cập vào một DNN đã được huấn luyện để phân loại hình ảnh thành 100 danh mục khác nhau, bao gồm động vật, thực vật, phương tiện và các vật thể hàng ngày, và bây giờ bạn muốn huấn luyện một DNN để phân loại các loại phương tiện cụ

thể. Các tác vụ này rất giống nhau, thậm chí một phần chồng chéo, vì vậy bạn nên thử tái sử dụng các phần của mạng đầu tiên (xem Hình 11-5).



Hình 11-5. Tái sử dụng các lớp đã được huấn luyện trước

Lớp đầu ra của mô hình gốc thường nên được thay thế vì nó rất có thể không hữu ích chút nào cho tác vụ mới, và có lẽ sẽ không có số lượng đầu ra phù hợp.

Tương tự, các lớp ẩn trên của mô hình gốc ít có khả năng hữu ích như các lớp thấp hơn, vì các đặc trưng cấp cao hữu ích nhất cho tác vụ mới có thể khác biệt đáng kể so với những đặc trưng hữu ích nhất cho tác vụ gốc. Bạn muốn tìm số lượng lớp phù hợp để tái sử dụng.

Hãy thử đóng băng tất cả các lớp đã được tái sử dụng trước (tức là làm cho trọng số của chúng không thể huấn luyện để giảm độ dốc sẽ không sửa đổi chúng và chúng sẽ vẫn cố định), sau đó huấn luyện mô hình của bạn và xem nó hoạt động như thế nào. Sau đó, thử bỏ đóng băng một hoặc hai lớp ẩn trên cùng để cho phép backpropagation điều chỉnh chúng và xem liệu hiệu suất có cải thiện không. Bạn càng có nhiều dữ liệu huấn luyện, bạn càng có thể bỏ đóng băng nhiều lớp. Cũng hữu ích khi giảm tốc độ học khi bạn bỏ đóng băng các lớp đã tái sử dụng: điều này sẽ tránh làm hỏng các trọng số đã được tinh chỉnh của chúng.

Nếu bạn vẫn không thể có được hiệu suất tốt, và bạn có ít dữ liệu huấn luyện, hãy thử bỏ lớp ẩn trên cùng và đóng băng tất cả các lớp ẩn còn lại một lần nữa. Bạn có thể lặp lại cho

đến khi tìm thấy số lượng lớp phù hợp để tái sử dụng. Nếu bạn có nhiều dữ liệu huấn luyện, bạn có thể thử thay thế các lớp ẩn trên cùng thay vì bỏ chúng, và thậm chí thêm nhiều lớp ẩn hơn.

Học chuyển giao với Keras

Hãy xem một ví dụ. Giả sử tập dữ liệu Fashion MNIST chỉ chứa tám lớp — ví dụ, tất cả các lớp ngoại trừ sandal và shirt. Ai đó đã xây dựng và huấn luyện một mô hình Keras trên tập dữ hợp này và đạt được hiệu suất khá tốt (độ chính xác >90%). Gọi mô hình này là mô hình A. Bây giờ bạn muốn giải quyết một tác vụ khác: bạn có hình ảnh áo thun và áo chui đầu, và bạn muốn huấn luyện một bộ phân loại nhị phân: tích cực cho áo thun (và áo phông), tiêu cực cho sandal.

Tập dữ liệu của bạn khá nhỏ; bạn chỉ có 200 hình ảnh được gắn nhãn. Khi bạn huấn luyện một mô hình mới cho tác vụ này (gọi là mô hình B) với cùng kiến trúc như mô hình A, bạn đạt độ chính xác kiểm tra 91.85%. Trong khi uống cà phê buổi sáng, bạn nhận ra rằng tác vụ của mình khá giống với tác vụ A, vì vậy có lẽ học chuyển giao có thể giúp ích?

Hãy cùng tìm hiểu!

Đầu tiên, bạn cần tải mô hình A và tạo một mô hình mới dựa trên các lớp của mô hình đó. Bạn quyết định tái sử dụng tất cả các lớp ngoại trừ lớp đầu ra:

```
# ... Giả sử mô hình A đã được huấn luyện và lưu vào "my_model_A"
model_A = tf.keras.models.load_model("my_model_A")
model_B_on_A = tf.keras.Sequential(model_A.layers[:-1])
model_B_on_A.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

Lưu ý rằng `model_A` và `model_B_on_A` hiện chia sẻ một số lớp. Khi bạn huấn luyện `model_B_on_A`, nó cũng sẽ ảnh hưởng đến `model_A`. Nếu bạn muốn tránh điều đó, bạn cần sao chép `model_A` trước khi bạn tái sử dụng các lớp của nó. Để làm điều này, bạn sao chép kiến trúc của mô hình A bằng `clone_model()`, sau đó sao chép trọng số của nó:

```
model_A_clone = tf.keras.models.clone_model(model_A)
model_A_clone.set_weights(model_A.get_weights())
```

Bây giờ bạn có thể huấn luyện `model_B_on_A` cho tác vụ B, nhưng vì lớp đầu ra mới được khởi tạo ngẫu nhiên nên nó sẽ tạo ra các lỗi lớn (ít nhất là trong vài epoch đầu tiên), do đó sẽ có các gradient lỗi lớn có thể làm hỏng các trọng số đã được tái sử dụng. Để tránh điều này, một cách tiếp cận là đóng băng các lớp đã tái sử dụng trong vài epoch đầu tiên, cho phép lớp mới có thời gian để học các trọng số hợp lý. Để làm điều này, đặt thuộc tính `trainable` của mỗi lớp thành `False` và biên dịch mô hình:

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
```

Bây giờ bạn có thể huấn luyện mô hình trong vài epoch, sau đó bỏ đóng băng các lớp đã tái sử dụng (điều này yêu cầu biên dịch lại mô hình) và tiếp tục huấn luyện để tinh chỉnh các lớp đã tái sử dụng cho tác vụ B. Sau khi bỏ đóng băng các lớp đã tái sử dụng, thường là một ý kiến hay để giảm tốc độ học, một lần nữa để tránh làm hỏng các trọng số đã tái sử dụng.

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                           validation_data=(X_valid_B, y_valid_B))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                           validation_data=(X_valid_B, y_valid_B))
```

Vậy, phán quyết cuối cùng là gì? Chà, độ chính xác kiểm tra của mô hình này là 93.85%, tăng chính xác hai phần trăm từ 91.85%! Điều này có nghĩa là học chuyển giao đã giảm tỷ lệ lỗi gần 25%:

```
>>> model_B_on_A.evaluate(X_test_B, y_test_B)
[0.2546142041683197, 0.9384999871253967]
```

Bạn có bị thuyết phục không? Bạn không nên: tôi đã gian lận! Tôi đã thử nhiều cấu hình cho đến khi tôi tìm thấy một cấu hình thể hiện sự cải thiện mạnh mẽ. Nếu bạn thử thay đổi các lớp hoặc hạt giống ngẫu nhiên, bạn sẽ thấy rằng sự cải thiện nói chung giảm, hoặc thậm chí biến mất hoặc đảo ngược. Điều tôi đã làm được gọi là “tra tấn dữ liệu cho đến khi nó thú nhận”. Khi một bài báo trông quá tích cực, bạn nên nghi ngờ: có lẽ kỹ thuật mới lạ không thực sự giúp ích nhiều (trên thực tế, nó thậm chí có thể làm giảm hiệu suất), nhưng các tác giả đã thử nhiều biến thể và chỉ báo cáo kết quả tốt nhất (có thể do may mắn thuần túy), mà không đề cập đến số lần thất bại họ gặp phải trên đường đi. Hầu hết thời gian, điều này hoàn toàn không có ác ý, nhưng nó là một phần lý do tại sao rất nhiều kết quả trong khoa học không bao giờ có thể được tái tạo.

Tại sao tôi lại gian lận? Hóa ra học chuyển giao không hoạt động tốt lắm với các mạng dày đặc nhỏ, có lẽ vì các mạng nhỏ học ít mẫu, và các mạng dày đặc học các mẫu rất cụ thể, không có khả năng hữu ích trong các tác vụ khác. Học chuyển giao hoạt động tốt nhất với các mạng nơ-ron tích chập sâu, có xu hướng học các bộ phát hiện đặc trưng tổng quát hơn nhiều (đặc biệt là ở các lớp thấp hơn). Chúng ta sẽ xem xét lại học chuyển giao trong Chương 14, sử dụng các kỹ thuật chúng ta vừa thảo luận (và lần này sẽ không có gian lận, tôi hứa!).

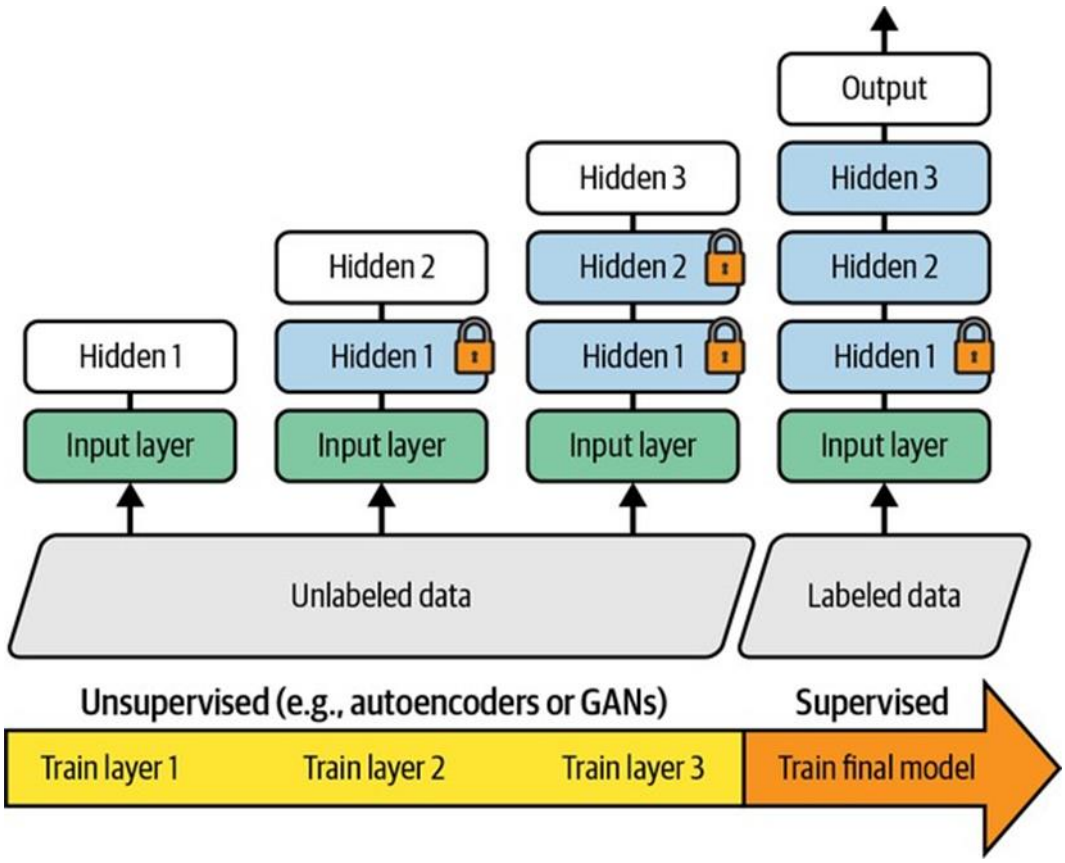
Tiền huấn luyện không giám sát

Giả sử bạn muốn giải quyết một tác vụ phức tạp mà bạn không có nhiều dữ liệu huấn luyện được gắn nhãn, nhưng thật không may, bạn không thể tìm thấy một mô hình đã được huấn luyện trên một tác vụ tương tự. Đừng mất hy vọng! Đầu tiên, bạn nên thử thu thập thêm dữ

liệu huấn luyện được gắn nhãn, nhưng nếu không thể, bạn vẫn có thể thực hiện **tiền huấn luyện không giám sát** (xem Hình 11-6). Thật vậy, việc thu thập các ví dụ huấn luyện không gắn nhãn thường rẻ, nhưng việc gắn nhãn chúng thì đắt. Nếu bạn có thể thu thập nhiều dữ liệu huấn luyện không gắn nhãn, bạn có thể thử sử dụng nó để huấn luyện một mô hình không giám sát, chẳng hạn như một bộ tự mã hóa (autoencoder) hoặc một mạng đối kháng tạo sinh (GAN; xem Chương 17). Sau đó, bạn có thể tái sử dụng các lớp thấp hơn của bộ tự mã hóa hoặc các lớp thấp hơn của bộ phân biệt của GAN, thêm lớp đầu ra cho tác vụ của bạn lên trên, và tinh chỉnh mạng cuối cùng bằng cách sử dụng học có giám sát (tức là với các ví dụ huấn luyện được gắn nhãn).

Chính kỹ thuật này đã được Geoffrey Hinton và nhóm của ông sử dụng vào năm 2006, và điều này đã dẫn đến sự hồi sinh của mạng nơ-ron và sự thành công của học sâu. Cho đến năm 2010, tiền huấn luyện không giám sát — thường là với các máy Boltzmann hạn chế (RBMs; xem sổ tay tại https://homl.info/extra-anns) — là tiêu chuẩn cho các mạng sâu, và chỉ sau khi vấn đề gradient biến mất được giảm nhẹ thì việc huấn luyện DNN hoàn toàn bằng học có giám sát mới trở nên phổ biến hơn nhiều. Tiền huấn luyện không giám sát (ngày nay thường sử dụng bộ tự mã hóa hoặc GANs thay vì RBMs) vẫn là một lựa chọn tốt khi bạn có một tác vụ phức tạp cần giải quyết, không có mô hình tương tự nào bạn có thể tái sử dụng, và ít dữ liệu huấn luyện được gắn nhãn nhưng có nhiều dữ liệu huấn luyện không gắn nhãn.

Lưu ý rằng trong những ngày đầu của học sâu, việc huấn luyện các mô hình sâu rất khó khăn, vì vậy mọi người sẽ sử dụng một kỹ thuật gọi là **tiền huấn luyện từng lớp tham lam** (greedy layer-wise pretraining) (được mô tả trong Hình 11-6). Họ sẽ huấn luyện một mô hình không giám sát với một lớp duy nhất, thường là một RBM, sau đó họ sẽ đóng băng lớp đó và thêm một lớp khác lên trên nó, sau đó huấn luyện lại mô hình (chỉ huấn luyện lớp mới một cách hiệu quả), sau đó đóng băng lớp mới và thêm một lớp khác lên trên nó, huấn luyện lại mô hình, v.v. Ngày nay, mọi thứ đơn giản hơn nhiều: mọi người thường huấn luyện mô hình không giám sát đầy đủ trong một lần và sử dụng bộ tự mã hóa hoặc GANs thay vì RBMs.



Hình 11-6. Trong huấn luyện không giám sát, một mô hình được huấn luyện trên tất cả dữ liệu, bao gồm dữ liệu không gắn nhãn, sử dụng kỹ thuật học không giám sát, sau đó nó được tinh chỉnh cho tác vụ cuối cùng chỉ trên dữ liệu được gắn nhãn bằng kỹ thuật học có giám sát; phần không giám sát có thể huấn luyện từng lớp một như thể hiện ở đây, hoặc nó có thể huấn luyện toàn bộ mô hình trực tiếp

Tiền huấn luyện (Pretraining) trên một Tác vụ Phụ trợ

Nếu bạn không có nhiều dữ liệu huấn luyện có nhãn, một lựa chọn cuối cùng là **huấn luyện một mạng nơ-ron đầu tiên trên một tác vụ phụ trợ** mà bạn có thể dễ dàng thu thập hoặc tạo ra dữ liệu huấn luyện có nhãn, sau đó **tái sử dụng các tầng dưới** của mạng đó cho tác vụ thực tế của bạn. Các tầng dưới của mạng nơ-ron đầu tiên sẽ học được các bộ phát hiện đặc trưng (feature detectors) mà có khả năng tái sử dụng được bởi mạng nơ-ron thứ hai.

Ví dụ, nếu bạn muốn xây dựng một hệ thống để nhận dạng khuôn mặt, bạn có thể chỉ có một vài bức ảnh của mỗi cá nhân—rõ ràng là không đủ để huấn luyện một bộ phân loại tốt. Việc thu thập hàng trăm bức ảnh của mỗi người sẽ không thực tế. Tuy nhiên, bạn có thể thu thập rất nhiều ảnh của những người ngẫu nhiên trên web và huấn luyện một mạng nơ-ron đầu tiên để phát hiện xem hai bức ảnh khác nhau có phải là cùng một người hay không. Một mạng như vậy sẽ học được các bộ phát hiện đặc trưng tốt cho khuôn mặt, vì vậy việc tái sử

dụng các tầng dưới của nó sẽ cho phép bạn huấn luyện một bộ phân loại khuôn mặt tốt chỉ với ít dữ liệu huấn luyện.

Đối với các ứng dụng xử lý ngôn ngữ tự nhiên (NLP), bạn có thể tải xuống một kho văn bản (corpus) gồm hàng triệu tài liệu và tự động tạo dữ liệu có nhãn từ đó. Ví dụ, bạn có thể che ngẫu nhiên một số từ và huấn luyện một mô hình để dự đoán các từ còn thiếu là gì (ví dụ, nó nên dự đoán rằng từ còn thiếu trong câu "What _____ you saying?" có lẽ là "are" hoặc "were"). Nếu bạn có thể huấn luyện một mô hình để đạt được hiệu suất tốt trong tác vụ này, thì nó sẽ biết khá nhiều về ngôn ngữ, và bạn chắc chắn có thể tái sử dụng nó cho tác vụ thực tế của mình và tinh chỉnh (fine-tune) nó trên dữ liệu có nhãn của bạn (chúng ta sẽ thảo luận thêm về các tác vụ tiền huấn luyện trong Chương 15).

Các Trình tối ưu hóa Nhanh hơn

Huấn luyện một mạng nơ-ron sâu rất lớn có thể chậm một cách đau đớn. Cho đến nay, chúng ta đã thấy bốn cách để tăng tốc độ huấn luyện (và đạt được một giải pháp tốt hơn): áp dụng một chiến lược **khởi tạo** tốt cho các trọng số kết nối, sử dụng một **hàm kích hoạt** tốt, sử dụng **chuẩn hóa theo lô** (batch normalization), và **tái sử dụng các phần của một mạng đã được tiền huấn luyện** (có thể được xây dựng cho một tác vụ phụ trợ hoặc sử dụng học không giám sát). Một sự tăng tốc lớn khác đến từ việc sử dụng một trình tối ưu hóa nhanh hơn so với trình tối ưu hóa hạ gradient thông thường. Trong phần này, chúng ta sẽ trình bày các thuật toán tối ưu hóa phổ biến nhất: **momentum**, **Nesterov accelerated gradient**, **AdaGrad**, **RMSProp**, và cuối cùng là **Adam** và các biến thể của nó.

Momentum (Động lượng)

Hãy tưởng tượng một quả bóng bowling lăn xuống một con dốc thoai thoải trên một bề mặt nhẵn: nó sẽ bắt đầu chậm, nhưng sẽ nhanh chóng tăng tốc cho đến khi đạt đến vận tốc cuối (terminal velocity) (nếu có một số ma sát hoặc sức cản không khí). Đây là ý tưởng cốt lõi đằng sau **tối ưu hóa động lượng**, được đề xuất bởi Boris Polyak vào năm 1964. Ngược lại, hạ gradient thông thường sẽ thực hiện các bước nhỏ khi dốc thoai thoải và các bước lớn khi dốc đứng, nhưng nó sẽ không bao giờ tăng tốc. Do đó, hạ gradient thông thường thường chậm hơn nhiều trong việc đạt đến điểm cực tiểu so với tối ưu hóa động lượng.

Hãy nhớ lại rằng hạ gradient cập nhật trọng số θ bằng cách trực tiếp trừ đi gradient của hàm chi phí $J(\theta)$ đối với trọng số, được nhân với tốc độ học η . Phương trình là $\theta \leftarrow \theta - \eta\nabla_\theta J(\theta)$. Nó không quan tâm đến các gradient trước đó. Nếu gradient cục bộ nhỏ, nó sẽ di chuyển rất chậm.

Hạ gradient động lượng quan tâm rất nhiều đến các gradient trước đó: ở mỗi lần lặp, nó trừ gradient cục bộ khỏi vector động lượng m (đã được nhân với tốc độ học η), và cập nhật trọng số bằng cách thêm vector động lượng này. Nói cách khác, gradient được sử dụng như một gia tốc, chứ không phải là một tốc độ. Để mô phỏng một cơ chế ma sát nào đó và ngăn

động lượng phát triển quá lớn, thuật toán giới thiệu một siêu tham số mới β , được gọi là **động lượng**. Giá trị của β phải nằm giữa 0 (không có ma sát) và 1. Một giá trị động lượng điển hình là 0,9.

Công thức 11-5: Thuật toán động lượng

- $m \leftarrow \beta m - \eta \nabla_\theta J(\theta)$
- $\theta \leftarrow \theta + m$

Bạn có thể xác minh rằng nếu gradient không đổi, vận tốc cuối cùng (tức là kích thước cập nhật trọng số tối đa) bằng với gradient đó nhân với tốc độ học nhân với $1/(1-\beta)$ (bỏ qua dấu). Ví dụ: nếu $\beta = 0.9$, thì vận tốc cuối cùng bằng 10 lần gradient nhân với tốc độ học, vì vậy hạ gradient động lượng cuối cùng đi nhanh gấp 10 lần so với hạ gradient thông thường. Điều này cho phép hạ gradient động lượng thoát ra khỏi các vùng bình nguyên (plateaus) nhanh hơn nhiều so với hạ gradient thông thường. Chúng ta đã thấy trong Chương 4 rằng khi các đầu vào có các tỷ lệ khác nhau, hàm chi phí sẽ có hình dạng giống như một cái bát thuôn dài. Hạ gradient thông thường đi xuống dốc khá nhanh, nhưng sau đó phải mất một thời gian rất dài để đi xuống thung lũng. Ngược lại, hạ gradient động lượng sẽ lăn xuống thung lũng nhanh hơn và dễ dàng hơn cho đến khi nó đến đáy (cực tiểu). Trong các mạng nơ-ron sâu nơi các đầu vào không có cùng tỷ lệ, các lớp trên sẽ thường kết thúc với các đầu vào có các tỷ lệ rất khác nhau, vì vậy việc sử dụng hạ gradient động lượng giúp rất nhiều. Nó cũng có thể giúp vượt qua các cực tiểu cục bộ.

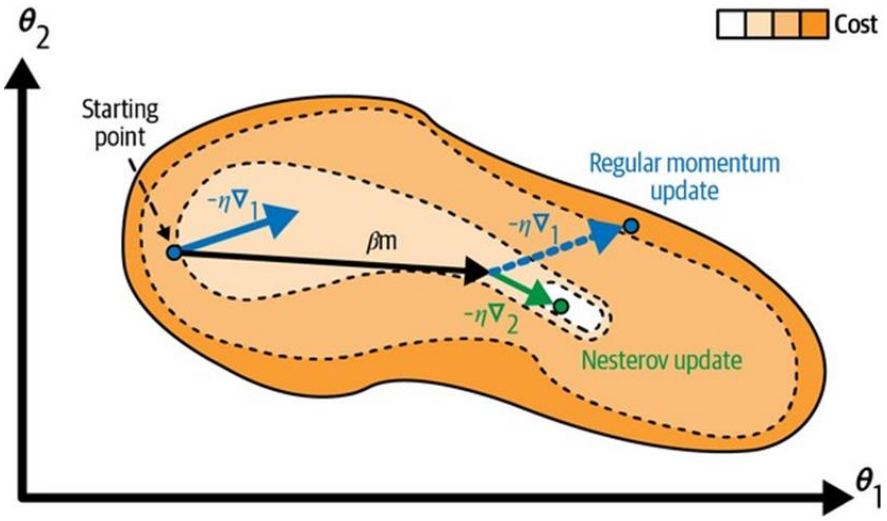
Nesterov Accelerated Gradient (NAG)

Một biến thể nhỏ của tối ưu hóa động lượng, do Yurii Nesterov đề xuất năm 1983, gần như luôn nhanh hơn tối ưu hóa động lượng thông thường. Phương pháp **Nesterov accelerated gradient (NAG)**, còn được gọi là tối ưu hóa động lượng Nesterov, đo gradient của hàm chi phí không phải tại vị trí cục bộ **θ** mà là **hơi lệch về phía trước theo hướng của động lượng**, tại **θ + βm** (xem Phương trình 11-6).

Phương trình 11-6. Thuật toán Nesterov Accelerated Gradient

- $\mathbf{m} \leftarrow \beta\mathbf{m} - \eta\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta} + \beta\mathbf{m})$
- $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{m}$

Sự điều chỉnh nhỏ này hoạt động vì nói chung vector động lượng sẽ trỏ đúng hướng (tức là về phía điểm tối ưu), vì vậy sẽ chính xác hơn một chút khi sử dụng gradient được đo ở vị trí xa hơn một chút theo hướng đó thay vì gradient tại vị trí ban đầu.



Hình 11-7. So sánh tối ưu hóa bằng động lượng thông thường và động lượng Nesterov: phương pháp đầu tiên áp dụng gradient được tính trước bước động lượng, trong khi phương pháp thứ hai áp dụng gradient được tính sau.

Như bạn có thể thấy, cập nhật Nesterov kết thúc gần điểm tối ưu hơn. Sau một thời gian, những cải tiến nhỏ này cộng lại và NAG cuối cùng nhanh hơn đáng kể so với tối ưu hóa động lượng thông thường. Hơn nữa, lưu ý rằng khi động lượng đẩy các trọng số qua một thung lũng, ∇1 tiếp tục đẩy xa hơn qua thung lũng, trong khi ∇2 đẩy ngược lại về phía đáy thung lũng. Điều này giúp giảm dao động và do đó NAG hội tụ nhanh hơn.

Để sử dụng NAG, chỉ cần đặt `nesterov=True` khi tạo trình tối ưu hóa `SGD`:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9,
nesterov=True)
```

AdaGrad

Hãy xem xét lại vấn đề "cái bát dài": hạ gradient bắt đầu bằng cách đi xuống nhanh chóng theo con dốc dốc nhất, vốn không chỉ thẳng đến điểm tối ưu toàn cục, sau đó nó đi xuống đáy thung lũng rất chậm. Sẽ thật tuyệt nếu thuật toán có thể **chỉnh hướng sớm hơn** để chỉ thẳng hơn về phía điểm tối ưu toàn cục. Thuật toán **AdaGrad** đạt được sự điều chỉnh này bằng cách **giảm tỷ lệ vector gradient dọc theo các chiều dốc nhất** (xem Phương trình 11-7).

Công thức 11-7: Thuật toán AdaGrad

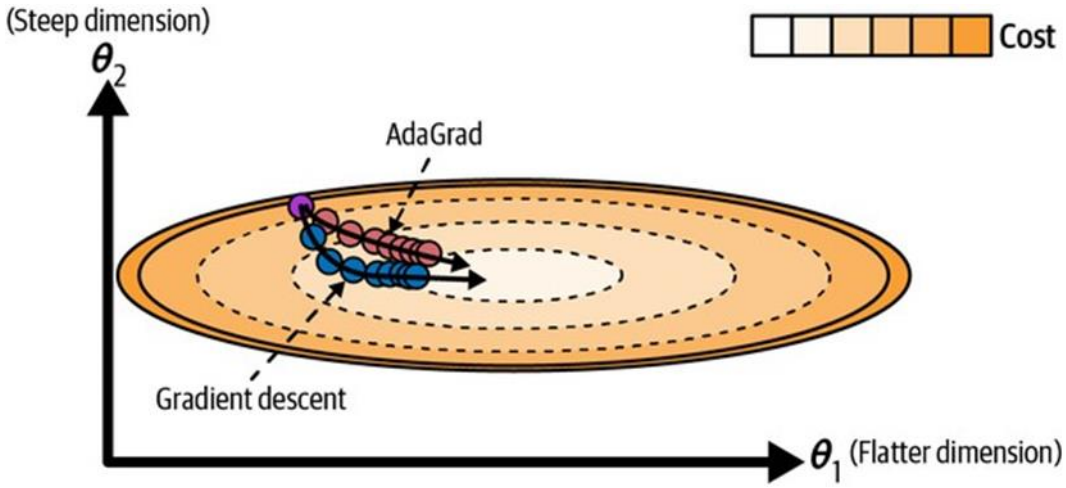
- $s \leftarrow s + \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$
- $\theta \leftarrow \theta - \eta \nabla_\theta J(\theta) \oslash \sqrt{s + \epsilon}$

Bước đầu tiên tích lũy bình phương của các gradient vào vector s (ký hiệu $\otimes$ đại diện cho phép nhân từng phần tử). Dạng vector này tương đương với việc tính toán $s_i \leftarrow s_i +$

$(\partial J(\theta)/\partial\theta_i)^2$ cho mỗi phần tử s_i của vector s . Nếu hàm chi phí dốc dọc theo chiều thứ i , s_i sẽ trở nên lớn hơn sau mỗi lần lặp.

Bước thứ hai gần như giống hệt hạ gradient, nhưng có một khác biệt lớn: vector gradient được chia tỷ lệ bởi một hệ số $\sqrt{s+\epsilon}$ ($\oslash$ là ký hiệu cho phép chia từng phần tử, và ϵ là một số hạng làm mịn để tránh chia cho 0, thường được đặt là 10^{-10}). Dạng vector này tương đương với việc tính toán đồng thời $\theta_i \leftarrow \theta_i - \eta\,\partial J(\theta)/\partial\theta_i/\sqrt{s_i+\epsilon}$ cho tất cả các tham số.

Nói tóm lại, thuật toán này làm giảm tốc độ học, nhưng nó làm như vậy nhanh hơn đối với các chiều dốc hơn so với các chiều có độ dốc nhẹ nhàng hơn. Điều này được gọi là **tốc độ học thích ứng**. Nó giúp các bản cập nhật hướng trực tiếp hơn đến cực tiểu toàn cục. Một lợi ích bổ sung là nó đòi hỏi ít điều chỉnh hơn đối với siêu tham số tốc độ học η .



Hình 11-8. So sánh AdaGrad và gradient descent: phương pháp đầu tiên (AdaGrad) có thể điều chỉnh hướng đi sớm hơn để hướng về điểm tối ưu.

AdaGrad thường hoạt động tốt cho các bài toán bậc hai đơn giản, nhưng nó thường dừng lại quá sớm khi huấn luyện mạng nơ-ron. Vì vậy, mặc dù Keras có trình tối ưu hóa `Adagrad`, bạn không nên sử dụng nó để huấn luyện các mạng nơ-ron sâu.

RMSProp

Như chúng ta đã thấy, AdaGrad có nguy cơ làm chậm tốc độ học quá nhanh và không bao giờ hội tụ về cực tiểu toàn cục. Thuật toán **RMSProp** khắc phục điều này bằng cách chỉ tích lũy gradient từ các lần lặp gần đây nhất, thay vì tất cả các gradient kể từ khi bắt đầu huấn luyện. Nó thực hiện điều này bằng cách sử dụng phép giảm theo cấp số nhân trong bước đầu tiên.

Công thức 11-8: Thuật toán RMSProp

- $s \leftarrow \rho s + (1-\rho)\nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$
- $\theta \leftarrow \theta - \eta \nabla_\theta J(\theta) \oslash \sqrt{s+\epsilon}$

Tỷ lệ giảm ρ thường được đặt là 0.9. Vâng, nó lại là một siêu tham số mới, nhưng giá trị mặc định này thường hoạt động tốt, vì vậy bạn có thể không cần phải điều chỉnh nó.

Như bạn có thể mong đợi, Keras có một bộ tối ưu hóa RMSProp.

```
optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

Ngoại trừ các bài toán rất đơn giản, trình tối ưu hóa này gần như luôn hoạt động tốt hơn nhiều so với AdaGrad.

Adam

Adam, viết tắt của **adaptive moment estimation**, kết hợp các ý tưởng của tối ưu hóa động lượng và RMSProp: giống như tối ưu hóa động lượng, nó theo dõi trung bình suy giảm theo cấp số nhân của các gradient trong quá khứ; và giống như RMSProp, nó theo dõi trung bình suy giảm theo cấp số nhân của các bình phương gradient trong quá khứ.

Công thức 11-9: Thuật toán Adam

- $m \leftarrow \beta_1 m - (1-\beta_1)\nabla_\theta J(\theta)$
- $s \leftarrow \beta_2 s + (1-\beta_2)\nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$
- $m^{\leftarrow} \frac{m}{1-\beta_{1t}}$
- $s^{\leftarrow} \frac{s}{1-\beta_{2t}}$
- $\theta \leftarrow \theta + \eta m^{\oslash} \sqrt{s^{+}\epsilon}$

Trong phương trình này, t là số lần lặp (bắt đầu từ 1).

Nếu bạn chỉ nhìn vào các bước 1, 2 và 5, bạn sẽ thấy sự tương đồng gần gũi của Adam với cả hạ gradient động lượng (momentum optimization) và RMSProp. β_1 tương ứng với động lượng, và β_2 tương ứng với ρ trong RMSProp. Điểm khác biệt duy nhất là bước 1 tính trung bình giảm theo cấp số nhân thay vì tổng giảm theo cấp số nhân, nhưng chúng thực sự tương đương nhau ngoại trừ một hằng số.

Các bước 3 và 4 là một chi tiết kỹ thuật: vì m và s được khởi tạo bằng 0 khi bắt đầu huấn luyện, nên chúng sẽ bị lệch về 0. Do đó, hai bước này sẽ giúp tăng cường m và s khi bắt đầu huấn luyện.

Siêu tham số giảm động lượng β_1 thường được khởi tạo là 0.9, trong khi siêu tham số giảm tỷ lệ β_2 thường được khởi tạo là 0.999. Tương tự, số hạng làm mịn ϵ thường được khởi tạo là một số rất nhỏ như 10^{-7} . Đây là các giá trị mặc định cho lớp Adam.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9,
beta_2=0.999)
```

Vì Adam là một thuật toán tốc độ học thích ứng, nó đòi hỏi ít tinh chỉnh hơn cho siêu tham số tốc độ học η.

Cuối cùng, ba biến thể của Adam đáng được đề cập: **AdaMax**, **Nadam**, và **AdamW**.

- **AdaMax**: Thay thế chuẩn ℓ_2 bằng chuẩn $\ell\infty$. Điều này có thể làm cho AdaMax ổn định hơn Adam, nhưng nói chung Adam hoạt động tốt hơn.
- **Nadam**: Là sự kết hợp giữa tối ưu hóa Adam và kỹ thuật Nesterov, vì vậy nó thường hội tụ nhanh hơn một chút so với Adam.
- **AdamW**: Là một biến thể của Adam tích hợp một kỹ thuật điều chuẩn (regularization) gọi là **suy giảm trọng số (weight decay)**. AdamW khắc phục vấn đề khi kết hợp Adam với điều chuẩn ℓ_2, thường dẫn đến các mô hình tổng quát hóa không tốt bằng SGD.

Để sử dụng Nadam, AdaMax hoặc AdamW trong Keras, hãy thay thế `tf.keras.optimizers.Adam` bằng `tf.keras.optimizers.Nadam`, `tf.keras.optimizers.Adamax`, hoặc `tf.keras.optimizers.experimental.AdamW`.

Tất cả các kỹ thuật tối ưu hóa được thảo luận cho đến nay chỉ dựa vào đạo hàm riêng bậc nhất. Các thuật toán dựa trên đạo hàm riêng bậc hai (ma trận Hessian) rất khó áp dụng cho các mạng nơ-ron sâu vì chi phí tính toán và bộ nhớ rất lớn.

Bảng 11-2. So sánh các trình tối ưu hóa (* là kém, ** là trung bình, và *** là tốt).

Lớp	Tốc độ hội tụ	Chất lượng hội tụ
SGD	*	***
SGD(momentum=...)	**	***
SGD(momentum=..., nesterov=True)	**	***
Adagrad	***	* (dừng quá sớm)
RMSprop	***	** hoặc ***
Adam	***	** hoặc ***
AdaMax	***	** hoặc ***
Nadam	***	** hoặc ***
AdamW	***	** hoặc ***

Lập lịch tốc độ học (Learning Rate Scheduling)

Việc tìm một tốc độ học tốt là rất quan trọng. Nếu bạn đặt nó quá cao, quá trình huấn luyện có thể phân kỳ (như đã thảo luận trong phần "Gradient Descent"). Nếu bạn đặt nó quá thấp, quá trình huấn luyện cuối cùng sẽ hội tụ đến cực tối ưu, nhưng sẽ mất rất nhiều thời

gian. Nếu bạn đặt nó hơi quá cao, nó sẽ tiến bộ rất nhanh lúc đầu, nhưng nó sẽ kết thúc việc nhảy múa quanh cực tối ưu và không bao giờ thực sự ổn định. Nếu bạn có ngân sách tính toán hạn chế, bạn có thể phải gián đoạn huấn luyện trước khi nó hội tụ đúng cách, dẫn đến một giải pháp không tối ưu (xem Hình 11-9).

Hình 11-9. Các đường cong học tập cho các tốc độ học khác nhau eta

Như đã thảo luận trong Chương 10, bạn có thể tìm thấy một tốc độ học tốt bằng cách huấn luyện mô hình trong vài trăm lần lặp, tăng tốc độ học theo cấp số mũ từ một giá trị rất nhỏ đến một giá trị rất lớn, và sau đó nhìn vào đường cong học tập và chọn một tốc độ học thấp hơn một chút so với điểm mà đường cong học tập bắt đầu tăng vọt trở lại. Sau đó, bạn có thể khởi tạo lại mô hình của mình và huấn luyện nó với tốc độ học đó.

Nhưng bạn có thể làm tốt hơn một tốc độ học cố định: nếu bạn bắt đầu với một tốc độ học lớn và sau đó giảm nó khi quá trình huấn luyện ngừng tiến triển nhanh, bạn có thể đạt được một giải pháp tốt nhanh hơn so với tốc độ học cố định tối ưu. Có nhiều chiến lược khác nhau để giảm tốc độ học trong quá trình huấn luyện. Cũng có thể có lợi khi bắt đầu với tốc độ học thấp, tăng nó, sau đó giảm nó trở lại. Các chiến lược này được gọi là **lịch trình học tập** (tôi đã giới thiệu ngắn gọn khái niệm này trong Chương 4).

Đây là những lịch trình học tập được sử dụng phổ biến nhất:

- **Lập lịch theo hàm lũy thừa (Power scheduling)**: Đặt tốc độ học là một hàm của số lần lặp t : $eta(t) = eta_0/(1 + t/s)^c$. Tốc độ học ban đầu eta_0 , số mũ c (thường được đặt là 1), và các bước s là các siêu tham số. Tốc độ học giảm ở mỗi bước. Sau s bước, tốc độ học giảm xuống còn $eta_0/2$. Sau s bước nữa, nó giảm xuống còn $eta_0/3$, sau đó nó giảm xuống còn $eta_0/4$, sau đó $eta_0/5$, v.v. Như bạn có thể thấy, lịch trình này ban đầu giảm nhanh, sau đó ngày càng chậm hơn. Tất nhiên, lập lịch theo hàm lũy thừa yêu cầu điều chỉnh eta_0 và s (và có thể cả c).

- **Lập lịch theo hàm mũ (Exponential scheduling)**: Đặt tốc độ học là $eta(t) = eta_0 cdot 0.1^{t/s}$. Tốc độ học sẽ dần dần giảm đi một hệ số 10 sau mỗi s bước. Trong khi lập lịch theo hàm lũy thừa làm giảm tốc độ học ngày càng chậm hơn, lập lịch theo hàm mũ vẫn tiếp tục cắt giảm nó theo hệ số 10 sau mỗi s bước.
- **Lập lịch hằng số từng phần (Piecewise constant scheduling)**: Sử dụng tốc độ học không đổi trong một số epoch (ví dụ: $eta_0 = 0.1$ trong 5 epoch), sau đó là tốc độ học nhỏ hơn cho một số epoch khác (ví dụ: $eta_1 = 0.001$ trong 50 epoch), và cứ thế. Mặc dù giải pháp này có thể hoạt động rất tốt, nhưng nó đòi hỏi phải điều chỉnh để tìm ra chuỗi tốc độ học phù hợp và thời gian sử dụng mỗi tốc độ.
- **Lập lịch theo hiệu suất (Performance scheduling)**: Đo lỗi xác thực sau mỗi N bước (giống như dừng sớm), và giảm tốc độ học bằng một hệ số $lambda$ khi lỗi ngừng giảm.
- **Lập lịch 1 chu kỳ (1cycle scheduling)**: 1cycle được giới thiệu trong một bài báo năm 2018 của Leslie Smith. Trái ngược với các cách tiếp cận khác, nó bắt đầu bằng cách tăng tốc độ học ban đầu eta_0 , tăng tuyến tính lên đến eta_1 ở giữa quá trình huấn luyện. Sau đó, nó giảm tốc độ học tuyến tính xuống eta_0 một lần nữa trong nửa sau của quá trình huấn luyện, kết thúc vài epoch cuối cùng bằng cách giảm tốc độ xuống vài bậc độ lớn (vẫn tuyến tính). Tốc độ học tối đa eta_1 được chọn bằng cách sử dụng cùng một cách tiếp cận mà chúng ta đã sử dụng để tìm tốc độ học tối ưu, và tốc độ học ban đầu eta_0 thường thấp hơn 10 lần. Khi sử dụng đà, chúng ta bắt đầu với đà cao trước (ví dụ: 0.95), sau đó giảm nó xuống đà thấp hơn trong nửa đầu của quá trình huấn luyện (ví dụ: xuống 0.85, tuyến tính), và sau đó đưa nó trở lại giá trị tối đa (ví dụ: 0.95) trong nửa sau của quá trình huấn luyện, kết thúc vài epoch cuối cùng với giá trị tối đa đó. Smith đã thực hiện nhiều thí nghiệm cho thấy cách tiếp cận này thường có thể tăng tốc đáng kể quá trình huấn luyện và đạt được hiệu suất tốt hơn. Ví dụ, trên tập dữ liệu hình ảnh CIFAR10 phổ biến, cách tiếp cận này đạt độ chính xác xác thực 91.9% chỉ trong 100 epoch, so với độ chính xác 90.3% trong 800 epoch thông qua một cách tiếp cận tiêu chuẩn (với cùng kiến trúc mạng nơ-ron). Thành tựu này được mệnh danh là siêu hội tụ (super-convergence).

Một bài báo năm 2013 của Andrew Senior và cộng sự đã so sánh hiệu suất của một số lịch trình học tập phổ biến nhất khi sử dụng tối ưu hóa đà để huấn luyện mạng nơ-ron sâu cho nhận dạng giọng nói. Các tác giả kết luận rằng, trong cài đặt này, cả lập lịch theo hiệu suất và lập lịch theo hàm mũ đều hoạt động tốt. Họ ủng hộ lập lịch theo hàm mũ vì nó dễ điều chỉnh và nó hội tụ nhanh hơn một chút đến giải pháp tối ưu. Họ cũng đề cập rằng nó dễ thực hiện hơn lập lịch theo hiệu suất, nhưng trong Keras cả hai tùy chọn đều dễ dàng. Điều đó nói lên rằng, cách tiếp cận 1cycle dường như hoạt động tốt hơn nữa.

Triển khai lập lịch theo hàm lũy thừa trong Keras là lựa chọn dễ nhất - chỉ cần đặt siêu tham số `decay` khi tạo một bộ tối ưu hóa:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, decay=1e-4)
```

`decay` là nghịch đảo của s (số bước để chia tốc độ học thêm một đơn vị), và Keras giả định rằng c bằng 1.

Lập lịch theo hàm mũ và lập lịch từng phần cũng khá đơn giản. Bạn cần định nghĩa một hàm nhận epoch hiện tại và trả về tốc độ học. Ví dụ, hãy triển khai lập lịch theo hàm mũ:

```
def exponential_decay_fn(epoch):
    return 0.01 * 0.1 ** (epoch / 20)
```

Nếu bạn không muốn mã hóa cứng eta_0 và s , bạn có thể tạo một hàm trả về một hàm đã được cấu hình:

```
def exponential_decay(lr0, s):
    def exponential_decay_fn(epoch):
        return lr0 * 0.1 ** (epoch / s)
    return exponential_decay_fn

exponential_decay_fn = exponential_decay(lr0=0.01, s=20)
```

Tiếp theo, tạo một callback `LearningRateScheduler`, truyền cho nó hàm lịch trình, và truyền callback này vào phương thức `fit()`:

```
lr_scheduler = tf.keras.callbacks.LearningRateScheduler(exponential_decay_fn)
history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])
```

`LearningRateScheduler` sẽ cập nhật thuộc tính `learning_rate` của bộ tối ưu hóa vào đầu mỗi epoch. Cập nhật tốc độ học một lần mỗi epoch thường là đủ, nhưng nếu bạn muốn nó được cập nhật thường xuyên hơn, ví dụ như ở mỗi bước, bạn luôn có thể viết callback của riêng mình (xem phần “Lập lịch theo hàm mũ” của sổ tay chương này để biết ví dụ). Cập nhật tốc độ học ở mỗi bước có thể hữu ích nếu có nhiều bước mỗi epoch. Ngoài ra, bạn có thể sử dụng cách tiếp cận `tf.keras.optimizers.schedules`, được mô tả ngay sau đây.

Hàm lịch trình tùy chọn có thể nhận tốc độ học hiện tại làm đối số thứ hai. Ví dụ, hàm lịch trình sau đây nhân tốc độ học trước đó với $0.1^{1/20}$, điều này dẫn đến cùng một sự suy giảm theo hàm mũ (ngoại trừ sự suy giảm bây giờ bắt đầu vào đầu epoch 0 thay vì 1):

```
def exponential_decay_fn(epoch, lr):
    return lr * 0.1 ** (1 / 20)
```

Việc triển khai này dựa vào tốc độ học ban đầu của bộ tối ưu hóa (trái ngược với triển khai trước), vì vậy hãy đảm bảo đặt nó một cách thích hợp.

Khi bạn lưu một mô hình, bộ tối ưu hóa và tốc độ học của nó cũng được lưu cùng với nó. Điều này có nghĩa là với hàm lịch trình mới này, bạn có thể chỉ cần tải một mô hình đã huấn luyện và tiếp tục huấn luyện từ nơi nó đã dừng lại, không có vấn đề gì. Tuy nhiên, mọi thứ không đơn giản như vậy nếu hàm lịch trình của bạn sử dụng đối số `epoch`: `epoch` không được lưu, và nó được đặt lại về 0 mỗi khi bạn gọi phương thức `fit()`. Nếu bạn tiếp tục huấn luyện một mô hình từ nơi nó đã dừng lại, điều này có thể dẫn đến tốc độ học rất lớn, điều này có thể làm hỏng trọng số của mô hình của bạn. Một giải pháp là đặt thủ công đối số `initial_epoch` của phương thức `fit()` để epoch bắt đầu ở giá trị đúng.

Đối với lập lịch hằng số từng phần, bạn có thể sử dụng một hàm lịch trình như sau (như trước, bạn có thể định nghĩa một hàm tổng quát hơn nếu muốn; xem phần "Lập lịch hằng số từng phần" của sổ tay để biết ví dụ), sau đó tạo một callback `LearningRateScheduler` với hàm này và truyền nó vào phương thức `fit()`, giống như lập lịch theo hàm mũ:

```
def piecewise_constant_fn(epoch):
    if epoch < 5:
        return 0.01
    elif epoch < 15:
        return 0.005
    else:
        return 0.001
```

Để lập lịch theo hiệu suất, hãy sử dụng callback `ReduceLROnPlateau`. Ví dụ, nếu bạn truyền callback sau đây vào phương thức `fit()`, nó sẽ nhân tốc độ học với 0.5 bất cứ khi nào mất mát xác thực tốt nhất không cải thiện trong năm epoch liên tiếp (các tùy chọn khác có sẵn; vui lòng kiểm tra tài liệu để biết thêm chi tiết):

```
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5)
history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])
```

Cuối cùng, Keras cung cấp một cách thay thế để triển khai lập lịch tốc độ học: bạn có thể định nghĩa một tốc độ học theo lịch trình bằng cách sử dụng một trong các lớp có sẵn trong `tf.keras.optimizers.schedules`, sau đó truyền nó cho bất kỳ bộ tối ưu hóa nào. Cách tiếp cận này cập nhật tốc độ học ở mỗi bước thay vì ở mỗi epoch. Ví dụ, đây là cách triển khai cùng một lịch trình theo hàm mũ như hàm `exponential_decay_fn()` chúng ta đã định nghĩa trước đó:

```
import math

batch_size = 32
n_epochs = 25
n_steps = n_epochs * math.ceil(len(X_train) / batch_size)
scheduled_learning_rate = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.01, decay_steps=n_steps, decay_rate=0.1
)
optimizer = tf.keras.optimizers.SGD(learning_rate=scheduled_learning_rate)
```

Điều này tốt và đơn giản, hơn nữa khi bạn lưu mô hình, tốc độ học và lịch trình của nó (bao gồm trạng thái của nó) cũng được lưu.

Đối với 1cycle, Keras không hỗ trợ nó, nhưng có thể triển khai nó trong chưa đầy 30 dòng mã bằng cách tạo một callback tùy chỉnh sửa đổi tốc độ học ở mỗi lần lặp. Để cập nhật tốc độ học của bộ tối ưu hóa từ bên trong phương thức `on_batch_begin()` của callback, bạn cần gọi `tf.keras.backend.set_value(self.model.optimizer.learning_rate, new_learning_rate)`. Xem phần "Lập lịch 1 chu kỳ" của sổ tay để biết ví dụ.

Tóm lại, sự suy giảm theo hàm mũ, lập lịch theo hiệu suất và 1cycle có thể tăng tốc đáng kể sự hội tụ, vì vậy hãy thử chúng!

Tránh Overfitting thông qua chính quy hóa

“Với bốn tham số, tôi có thể lắp vừa một con voi và với năm tham số, tôi có thể làm cho nó vẫy vòi của mình.” —John von Neumann, được Enrico Fermi trích dẫn trong Nature 427

Với hàng nghìn tham số, bạn có thể lắp vừa cả vườn bách thú. Mạng nơ-ron sâu thường có hàng chục nghìn tham số, đôi khi thậm chí hàng triệu. Điều này mang lại cho chúng một lượng tự do đáng kinh ngạc và có nghĩa là chúng có thể phù hợp với rất nhiều tập dữ liệu phức tạp khác nhau. Nhưng sự linh hoạt tuyệt vời này cũng khiến mạng dễ bị overfitting tập huấn luyện. Chính quy hóa thường cần thiết để ngăn chặn điều này.

Chúng ta đã triển khai một trong những kỹ thuật chính quy hóa tốt nhất trong Chương 10: dừng sớm. Hơn nữa, mặc dù chuẩn hóa theo batch được thiết kế để giải quyết các vấn đề gradient không ổn định, nó cũng hoạt động như một bộ chính quy hóa khá tốt. Trong phần này, chúng ta sẽ xem xét các kỹ thuật chính quy hóa phổ biến khác cho mạng nơ-ron: chính quy hóa l_1 và l_2 , dropout, và chính quy hóa max-norm.

Chính quy hóa l_1 và l_2

Giống như bạn đã làm trong Chương 4 đối với các mô hình tuyến tính đơn giản, bạn có thể sử dụng chính quy hóa l_2 để ràng buộc các trọng số kết nối của mạng nơ-ron, và/hoặc chính quy hóa l_1 nếu bạn muốn một mô hình thưa thớt (với nhiều trọng số bằng 0). Dưới đây là cách áp dụng chính quy hóa l_2 cho các trọng số kết nối của một lớp Keras, sử dụng hệ số chính quy hóa 0.01:

```
layer = tf.keras.layers.Dense(100, activation="relu",
                              kernel_initializer="he_normal",

kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

Hàm `l2()` trả về một bộ chính quy hóa sẽ được gọi ở mỗi bước trong quá trình huấn luyện để tính toán mất mát chính quy hóa. Sau đó, mất mát này được thêm vào tổng mất mát cuối cùng. Như bạn có thể mong đợi, bạn có thể chỉ cần sử dụng `tf.keras.regularizers.l1()` nếu bạn muốn chính quy hóa l_1 ; nếu bạn muốn cả chính quy hóa l_1 và l_2 , hãy sử dụng `tf.keras.regularizers.l1_l2()` (chỉ định cả hai hệ số chính quy hóa).

Vì bạn thường sẽ muốn áp dụng cùng một bộ chính quy hóa cho tất cả các lớp trong mạng của mình, cũng như sử dụng cùng một hàm kích hoạt và cùng một chiến lược khởi tạo trong tất cả các lớp ẩn, bạn có thể thấy mình lặp lại cùng một đối số. Điều này làm cho mã trở nên xấu và dễ bị lỗi. Để tránh điều này, bạn có thể thử tái cấu trúc mã của mình để sử dụng các vòng lặp. Một lựa chọn khác là sử dụng hàm `functools.partial()` của Python, cho phép bạn tạo một trình bao bọc mỏng cho bất kỳ đối tượng có thể gọi nào, với một số giá trị đối số mặc định:

```
from functools import partial

RegularizedDense = partial(tf.keras.layers.Dense,
```

```
                            activation="relu",
                            kernel_initializer="he_normal",
                            kernel_regularizer=tf.keras.regularizers.l2(0.01))

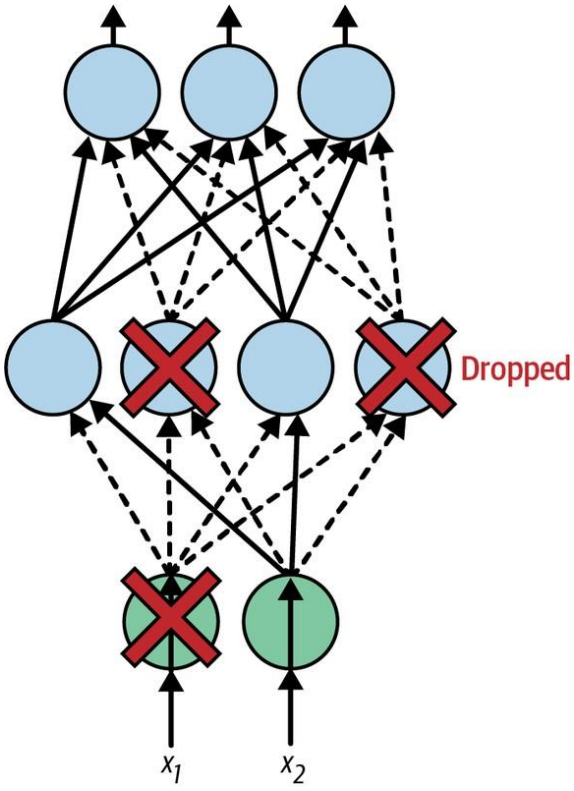
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    RegularizedDense(100),
    RegularizedDense(100),
    RegularizedDense(10, activation="softmax")
])
```

Dropout

Dropout là một trong những kỹ thuật chính quy hóa phổ biến nhất cho mạng nơ-ron sâu. Nó được đề xuất trong một bài báo của Geoffrey Hinton và cộng sự vào năm 2012 và được mô tả chi tiết hơn trong một bài báo năm 2014 của Nitish Srivastava và cộng sự, và nó đã được chứng minh là rất thành công: nhiều mạng nơ-ron hiện đại sử dụng dropout, vì nó giúp chúng tăng độ chính xác từ 1%–2%. Điều này nghe có vẻ không nhiều, nhưng khi một mô hình đã có độ chính xác 95%, việc tăng độ chính xác 2% có nghĩa là giảm tỷ lệ lỗi gần 40% (từ 5% lỗi xuống khoảng 3%).

Đây là một thuật toán khá đơn giản: ở mỗi bước huấn luyện, mỗi nơ-ron (bao gồm các nơ-ron đầu vào, nhưng luôn trừ các nơ-ron đầu ra) có xác suất p bị "bỏ qua" tạm thời, nghĩa là nó sẽ bị bỏ qua hoàn toàn trong bước huấn luyện này, nhưng nó có thể hoạt động trong bước tiếp theo (xem Hình 11-10). Siêu tham số p được gọi là **tỷ lệ dropout**, và nó thường được đặt giữa 10% và 50%: gần 20%–30% trong mạng nơ-ron hồi quy (xem Chương 15), và gần 40%–50% trong mạng nơ-ron tích chập (xem Chương 14). Sau khi huấn luyện, các nơ-ron không bị bỏ qua nữa. Và đó là tất cả (ngoại trừ một chi tiết kỹ thuật chúng ta sẽ thảo luận ngay sau đây).

Ban đầu thật đáng ngạc nhiên khi kỹ thuật phá hoại này lại hoạt động. Liệu một công ty có hoạt động tốt hơn nếu nhân viên của mình được yêu cầu tung đồng xu mỗi sáng để quyết định có đi làm hay không? Chà, ai biết được; có lẽ là có! Công ty sẽ buộc phải điều chỉnh tổ chức của mình; nó không thể dựa vào bất kỳ một người nào để vận hành máy pha cà phê hoặc thực hiện bất kỳ nhiệm vụ quan trọng nào khác, vì vậy chuyên môn này sẽ phải được phân tán cho nhiều người. Nhân viên sẽ phải học cách hợp tác với nhiều đồng nghiệp của mình, không chỉ một số ít. Công ty sẽ trở nên linh hoạt hơn nhiều. Nếu một người bỏ việc, nó sẽ không tạo ra nhiều khác biệt. Không rõ liệu ý tưởng này có thực sự hoạt động đối với các công ty hay không, nhưng nó chắc chắn hoạt động đối với mạng nơ-ron. Các nơ-ron được huấn luyện với dropout không thể cùng thích nghi với các nơ-ron lân cận của chúng; chúng phải hữu ích nhất có thể một mình. Chúng cũng không thể phụ thuộc quá mức vào chỉ một vài nơ-ron đầu vào; chúng phải chú ý đến từng nơ-ron đầu vào của chúng. Cuối cùng, bạn sẽ có một mạng mạnh mẽ hơn và tổng quát hóa tốt hơn.



Hình 11-10. Với chính quy hóa dropout, ở mỗi lần lặp huấn luyện, một tập hợp con ngẫu nhiên của tất cả các nơ-ron trong một hoặc nhiều lớp — ngoại trừ lớp đầu ra — bị "bỏ qua"; các nơ-ron này xuất ra 0 ở lần lặp này (được biểu diễn bằng các mũi tên đứt nét)

Một cách khác để hiểu sức mạnh của dropout là nhận ra rằng một mạng nơ-ron duy nhất được tạo ra ở mỗi bước huấn luyện. Vì mỗi nơ-ron có thể có mặt hoặc không có mặt, có tổng cộng 2^N mạng có thể có (trong đó N là tổng số nơ-ron có thể bị bỏ qua). Đây là một con số khổng lồ đến mức thực tế không thể có cùng một mạng nơ-ron được lấy mẫu hai lần. Một khi bạn đã chạy 10.000 bước huấn luyện, bạn đã huấn luyện 10.000 mạng nơ-ron khác nhau, mỗi mạng chỉ với một thể hiện huấn luyện. Các mạng nơ-ron này rõ ràng không độc lập vì chúng chia sẻ nhiều trọng số của chúng, nhưng dù sao chúng vẫn khác nhau. Mạng nơ-ron kết quả có thể được xem như một tập hợp trung bình của tất cả các mạng nơ-ron nhỏ hơn này.

Có một chi tiết kỹ thuật nhỏ nhưng quan trọng. Giả sử $p = 75$: trung bình chỉ 25% tổng số nơ-ron hoạt động ở mỗi bước trong quá trình huấn luyện. Điều này có nghĩa là sau khi huấn luyện, một nơ-ron sẽ được kết nối với số nơ-ron đầu vào gấp bốn lần so với trong quá trình huấn luyện. Để bù đắp cho thực tế này, chúng ta cần nhân trọng số kết nối đầu vào của mỗi nơ-ron với bốn trong quá trình huấn luyện. Nếu chúng ta không làm vậy, mạng nơ-ron sẽ không hoạt động tốt vì nó sẽ thấy dữ liệu khác nhau trong và sau quá trình huấn luyện. Tổng quát hơn, chúng ta cần chia trọng số kết nối cho **xác suất giữ lại** $(1-p)$ trong quá trình huấn luyện.

Để triển khai dropout bằng Keras, bạn có thể sử dụng lớp `tf.keras.layers.Dropout`. Trong quá trình huấn luyện, nó ngẫu nhiên bỏ qua một số đầu vào (đặt chúng bằng 0) và chia các đầu vào còn lại cho xác suất giữ lại. Sau khi huấn luyện, nó không làm gì cả; nó chỉ chuyển các đầu vào đến lớp tiếp theo. Đoạn mã sau áp dụng chính quy hóa dropout trước mỗi lớp dày đặc, sử dụng tỷ lệ dropout 0.2:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(10, activation="softmax")
])
# ... biên dịch và huấn luyện mô hình
```

Nếu bạn quan sát thấy mô hình bị overfitting, bạn có thể tăng tỷ lệ dropout. Ngược lại, bạn nên thử giảm tỷ lệ dropout nếu mô hình underfits tập huấn luyện. Cũng có thể hữu ích khi tăng tỷ lệ dropout cho các lớp lớn, và giảm nó cho các lớp nhỏ. Hơn nữa, nhiều kiến trúc hiện đại chỉ sử dụng dropout sau lớp ẩn cuối cùng, vì vậy bạn có thể thử điều này nếu dropout toàn diện quá mạnh.

Dropout có xu hướng làm chậm đáng kể sự hội tụ, nhưng nó thường dẫn đến một mô hình tốt hơn khi được điều chỉnh đúng cách. Vì vậy, nó thường rất đáng để dành thêm thời gian và công sức, đặc biệt đối với các mô hình lớn.

Monte Carlo (MC) Dropout

Năm 2016, một bài báo của Yarin Gal và Zoubin Ghahramani đã bổ sung thêm vài lý do chính đáng để sử dụng dropout:

- Thứ nhất, bài báo đã thiết lập một mối liên hệ sâu sắc giữa các mạng dropout (tức là các mạng nơ-ron chứa các lớp Dropout) và suy luận Bayes xấp xỉ , mang lại cho dropout một lý do toán học vững chắc.
- Thứ hai, các tác giả đã giới thiệu một kỹ thuật mạnh mẽ gọi là MC dropout, có thể tăng cường hiệu suất của bất kỳ mô hình dropout đã được huấn luyện nào mà không cần huấn luyện lại hay thậm chí sửa đổi nó một chút nào. Nó cũng cung cấp một thước đo tốt hơn nhiều về sự không chắc chắn của mô hình, và nó có thể được triển khai chỉ trong vài dòng mã.

Nếu tất cả điều này nghe có vẻ giống như một mánh khóe “một mẹo lạ” trên các trang giật tít, thì hãy xem đoạn mã sau. Đó là toàn bộ việc triển khai MC dropout, tăng cường mô hình dropout mà chúng ta đã huấn luyện trước đó mà không cần huấn luyện lại nó:

```
import numpy as np

y_probas = np.stack([model(X_test, training=True)
                     for sample in range(100)])
y_proba = y_probas.mean(axis=0)
```

Lưu ý rằng `model(X)` tương tự như `model.predict(X)` ngoại trừ việc nó trả về một tensor thay vì một mảng NumPy, và nó hỗ trợ đối số `training`. Trong ví dụ mã này, việc đặt `training=True` đảm bảo rằng lớp Dropout vẫn hoạt động, vì vậy tất cả các dự đoán sẽ hơi khác nhau. Chúng ta chỉ thực hiện 100 dự đoán trên tập kiểm tra, và chúng ta tính trung bình của chúng. Cụ thể hơn, mỗi lần gọi mô hình trả về một ma trận với một hàng cho mỗi thể hiện và một cột cho mỗi lớp. Vì có 10.000 thể hiện trong tập kiểm tra và 10 lớp, đây là một ma trận có hình dạng `[10000, 10]`. Chúng ta xếp chồng 100 ma trận như vậy, vì vậy `y_probas` là một mảng 3D có hình dạng `[100, 10000, 10]`. Khi chúng ta tính trung bình trên chiều đầu tiên (`axis=0`), chúng ta nhận được `y_proba`, một mảng có hình dạng `[10000, 10]`, giống như chúng ta sẽ nhận được với một dự đoán duy nhất. Thế thôi!

Tính trung bình trên nhiều dự đoán khi bật dropout mang lại cho chúng ta một ước lượng Monte Carlo thường đáng tin cậy hơn kết quả của một dự đoán duy nhất khi tắt dropout. Ví dụ, hãy xem dự đoán của mô hình cho thể hiện đầu tiên trong tập kiểm tra Fashion MNIST, khi tắt dropout:

```
>>> model.predict(X_test[:1]).round(3)
array([[0.   , 0.   , 0.   , 0.   , 0.   , 0.024, 0.   , 0.132, 0.   ,
        0.844]], dtype=float32)
```

Mô hình khá tự tin (84.4%) rằng hình ảnh này thuộc về lớp 9 (ủng cổ chân). So sánh điều này với dự đoán MC dropout:

```
>>> y_proba[0].round(3)
array([0.   , 0.   , 0.   , 0.   , 0.   , 0.067, 0.   , 0.209, 0.001,
        0.723], dtype=float32)
```

Mô hình vẫn dường như ưu tiên lớp 9, nhưng độ tin cậy của nó đã giảm xuống 72.3%, và xác suất ước tính cho các lớp 5 (sandal) và 7 (giày thể thao) đã tăng lên, điều này có ý nghĩa vì chúng cũng là giày dép.

MC dropout có xu hướng cải thiện độ tin cậy của các ước lượng xác suất của mô hình. Điều này có nghĩa là nó ít có khả năng tự tin nhưng sai, điều này có thể nguy hiểm: chỉ cần tưởng tượng một chiếc xe tự lái tự tin bỏ qua biển báo dừng. Việc biết chính xác những lớp nào khác có nhiều khả năng nhất cũng hữu ích.

Ngoài ra, bạn có thể xem độ lệch chuẩn của các ước lượng xác suất:

```
>>> y_std = y_probas.std(axis=0)

>>> y_std[0].round(3)
array([0.   , 0.   , 0.   , 0.001, 0.   , 0.096, 0.   , 0.162, 0.001,
        0.183], dtype=float32)
```

Rõ ràng có khá nhiều phương sai trong các ước lượng xác suất cho lớp 9: độ lệch chuẩn là 0.183, nên được so sánh với xác suất ước tính 0.723: nếu bạn đang xây dựng một hệ thống nhạy cảm với rủi ro (ví dụ: hệ thống y tế hoặc tài chính), bạn có lẽ sẽ xử lý một dự đoán không chắc chắn như vậy với sự thận trọng cực kỳ. Bạn chắc chắn sẽ không coi nó như một dự đoán tự tin 84.4%.

Độ chính xác của mô hình cũng nhận được một sự tăng nhẹ (rất nhỏ) từ 87.0% lên 87.2%:

```
>>> y_pred = y_proba.argmax(axis=1)

>>> accuracy = (y_pred == y_test).sum() / len(y_test)

>>> accuracy
0.8717
```

Nếu mô hình của bạn chứa các lớp khác hoạt động theo cách đặc biệt trong quá trình huấn luyện (chẳng hạn như các lớp `BatchNormalization`), thì bạn không nên ép buộc chế độ huấn luyện như chúng ta vừa làm. Thay vào đó, bạn nên thay thế các lớp `Dropout` bằng lớp `MCDropout` sau:

```
class MCDropout(tf.keras.layers.Dropout):
    def call(self, inputs, training=False):
        return super().call(inputs, training=True)
```

Ở đây, chúng ta chỉ kế thừa từ lớp `Dropout` và ghi đè phương thức `call()` để buộc đối số `training` của nó thành `True` (xem Chương 12). Tương tự, bạn có thể định nghĩa một lớp `MCAlphaDropout` bằng cách kế thừa từ `AlphaDropout` thay thế. Nếu bạn đang tạo một mô hình từ đầu, đó chỉ là vấn đề sử dụng `MCDropout` thay vì `Dropout`. Nhưng nếu bạn có một mô hình đã được huấn luyện bằng `Dropout`, bạn cần tạo một mô hình mới giống hệt mô hình hiện có nhưng với `MCDropout` thay vì `Dropout`, sau đó sao chép trọng số của mô hình hiện có vào mô hình mới của bạn.

Tóm lại, MC dropout là một kỹ thuật tuyệt vời giúp tăng cường các mô hình dropout và cung cấp các ước lượng không chắc chắn tốt hơn. Và tất nhiên, vì nó chỉ là dropout thông thường trong quá trình huấn luyện, nó cũng hoạt động như một bộ chính quy hóa.

Chính quy hóa Max-Norm

Một kỹ thuật chính quy hóa phổ biến khác cho mạng nơ-ron được gọi là **chính quy hóa max-norm**: đối với mỗi nơ-ron, nó ràng buộc trọng số w của các kết nối đến sao cho $\| w \|_2 \leq r$, trong đó r là siêu tham số max-norm và $\| cdot \|_2$ là chuẩn l_2.

Chính quy hóa max-norm không thêm một số hạng mất mát chính quy hóa vào hàm mất mát tổng thể. Thay vào đó, nó thường được triển khai bằng cách tính toán $\| w \|_2$ sau mỗi bước huấn luyện và điều chỉnh tỷ lệ w nếu cần ($w \leftarrow w\, r/\| w \|_2$). Việc giảm r làm tăng lượng chính quy hóa và giúp giảm overfitting. Chính quy hóa max-norm cũng có thể giúp giảm nhẹ các vấn đề gradient không ổn định (nếu bạn không sử dụng chuẩn hóa theo batch).

Để triển khai chính quy hóa max-norm trong Keras, hãy đặt đối số `kernel_constraint` của mỗi lớp ẩn thành một ràng buộc `max_norm()` với giá trị tối đa thích hợp, như thế này:

```
dense = tf.keras.layers.Dense(
    100, activation="relu", kernel_initializer="he_normal",
    kernel_constraint=tf.keras.constraints.max_norm(1.))
```

Sau mỗi lần lặp huấn luyện, phương thức `fit()` của mô hình sẽ gọi đối tượng được trả về bởi `max_norm()`, truyền cho nó trọng số của lớp và nhận lại trọng số đã được điều chỉnh tỷ lệ, sau đó thay thế trọng số của lớp. Như bạn sẽ thấy trong Chương 12, bạn có thể định nghĩa hàm ràng buộc tùy chỉnh của riêng mình nếu cần và sử dụng nó làm `kernel_constraint`. Bạn cũng có thể ràng buộc các số hạng bias bằng cách đặt đối số `bias_constraint`.

Hàm `max_norm()` có đối số `axis` mặc định là 0. Một lớp `Dense` thường có trọng số có hình dạng `[số đầu vào, số nơ-ron]`, vì vậy việc sử dụng `axis=0` có nghĩa là ràng buộc max-norm sẽ được áp dụng độc lập cho vector trọng số của mỗi nơ-ron. Nếu bạn muốn sử dụng max-norm với các lớp tích chập (xem Chương 14), hãy đảm bảo đặt đối số `axis` của ràng buộc `max_norm()` một cách thích hợp (thường là `axis=[0, 1, 2]`).

Tóm tắt và hướng dẫn thực hành

Trong chương này, chúng ta đã đề cập đến một loạt các kỹ thuật, và bạn có thể tự hỏi mình nên sử dụng kỹ thuật nào. Điều này phụ thuộc vào tác vụ, và chưa có sự đồng thuận rõ ràng, nhưng tôi đã thấy cấu hình trong Bảng 11-3 hoạt động tốt trong hầu hết các trường hợp, mà không yêu cầu nhiều tinh chỉnh siêu tham số. Điều đó nói lên rằng, xin đừng coi những mặc định này là những quy tắc cứng nhắc!

Bảng 11-3. Cấu hình DNN mặc định

Lựa chọn	Giá trị
Khởi tạo trọng số	He initialization
Hàm kích hoạt	ReLU hoặc biến thể (Leaky ReLU, ELU, GELU, Swish, Mish)
Chuẩn hóa	Batch Normalization
Bộ tối ưu hóa	Adam (hoặc Nadam, AdamW)
Lịch trình tốc độ học	Exponential decay hoặc 1cycle
Chính quy hóa	Early stopping, Batch Normalization, Dropout

Nếu mạng là một chồng đơn giản các lớp dày đặc, thì nó có thể tự chuẩn hóa, và bạn nên sử dụng cấu hình trong Bảng 11-4 thay thế.

Bảng 11-4. Cấu hình DNN cho mạng tự chuẩn hóa

Lựa chọn	Giá trị
Khởi tạo trọng số	LeCun normal initialization

Lựa chọn	Giá trị
Hàm kích hoạt	SELU
Chuẩn hóa	(Không cần Batch Normalization)
Bộ tối ưu hóa	Nesterov accelerated gradients
Lịch trình tốc độ học	(Có thể thử 1cycle nếu không dùng Batch Normalization)
Chính quy hóa	Early stopping (tránh l_1 , l_2 , max-norm, Batch Normalization, Dropout thông thường)

Đừng quên chuẩn hóa các đặc trưng đầu vào! Bạn cũng nên cố gắng tái sử dụng các phần của một mạng nơ-ron đã được huấn luyện trước nếu bạn có thể tìm thấy một mạng giải quyết một vấn đề tương tự, hoặc sử dụng tiền huấn luyện không giám sát nếu bạn có nhiều dữ liệu không gắn nhãn, hoặc sử dụng tiền huấn luyện trên một tác vụ phụ trợ nếu bạn có nhiều dữ liệu được gắn nhãn cho một tác vụ tương tự.

Mặc dù các hướng dẫn trước đó sẽ bao gồm hầu hết các trường hợp, nhưng đây là một số ngoại lệ:

- Nếu bạn cần một mô hình thưa thớt, bạn có thể sử dụng chính quy hóa l_1 (và tùy chọn đặt các trọng số rất nhỏ bằng 0 sau khi huấn luyện). Nếu bạn cần một mô hình thưa thớt hơn nữa, bạn có thể sử dụng TensorFlow Model Optimization Toolkit. Điều này sẽ phá vỡ khả năng tự chuẩn hóa, vì vậy bạn nên sử dụng cấu hình mặc định trong trường hợp này.
- Nếu bạn cần một mô hình có độ trễ thấp (một mô hình thực hiện dự đoán nhanh như chớp), bạn có thể cần sử dụng ít lớp hơn, sử dụng hàm kích hoạt nhanh như ReLU hoặc leaky ReLU, và gộp các lớp chuẩn hóa theo batch vào các lớp trước đó sau khi huấn luyện. Có một mô hình thưa thớt cũng sẽ giúp ích. Cuối cùng, bạn có thể muốn giảm độ chính xác dấu phẩy động từ 32 bit xuống 16 hoặc thậm chí 8 bit (xem “Triển khai mô hình sang thiết bị di động hoặc nhúng”). Một lần nữa, hãy kiểm tra TF-MOT.
- Nếu bạn đang xây dựng một ứng dụng nhạy cảm với rủi ro, hoặc độ trễ suy luận không quá quan trọng trong ứng dụng của bạn, bạn có thể sử dụng MC dropout để tăng hiệu suất và nhận được các ước lượng xác suất đáng tin cậy hơn, cùng với các ước lượng không chắc chắn.

Với các hướng dẫn này, bây giờ bạn đã sẵn sàng để huấn luyện các mạng sâu! Tôi hy vọng bạn đã bị thuyết phục rằng bạn có thể đi rất xa chỉ bằng cách sử dụng API Keras tiện lợi. Tuy nhiên, có thể đến lúc bạn cần kiểm soát nhiều hơn nữa; ví dụ, để viết một hàm mất mát tùy chỉnh hoặc để điều chỉnh thuật toán huấn luyện. Đối với những trường hợp như vậy, bạn sẽ cần sử dụng API cấp thấp hơn của TensorFlow, như bạn sẽ thấy trong chương tiếp theo.

Bài tập

1. Vấn đề mà khởi tạo Glorot và khởi tạo He nhằm mục đích khắc phục là gì?

2. Có ổn không khi khởi tạo tất cả các trọng số với cùng một giá trị miễn là giá trị đó được chọn ngẫu nhiên bằng cách sử dụng khởi tạo He?
3. Có ổn không khi khởi tạo các số hạng bias về 0?
4. Trong những trường hợp nào bạn sẽ muốn sử dụng từng hàm kích hoạt chúng ta đã thảo luận trong chương này?
5. Điều gì có thể xảy ra nếu bạn đặt siêu tham số momentum quá gần 1 (ví dụ: 0.99999) khi sử dụng bộ tối ưu hóa SGD?
6. Kể tên ba cách bạn có thể tạo một mô hình thưa thớt.
7. Dropout có làm chậm quá trình huấn luyện không? Nó có làm chậm quá trình suy luận (tức là đưa ra dự đoán trên các thể hiện mới) không? Còn MC dropout thì sao?
8. Thực hành huấn luyện một mạng nơ-ron sâu trên tập dữ liệu hình ảnh CIFAR10: a. Xây dựng một DNN với 20 lớp ẩn, mỗi lớp 100 nơ-ron (quá nhiều, nhưng đó là mục đích của bài tập này). Sử dụng khởi tạo He và hàm kích hoạt Swish. b. Sử dụng tối ưu hóa Nadam và dừng sớm, huấn luyện mạng trên tập dữ liệu CIFAR10. Bạn có thể tải nó bằng `tf.keras.datasets.cifar10.load_data()`. Tập dữ liệu bao gồm 60.000 hình ảnh màu 32 × 32 pixel (50.000 để huấn luyện, 10.000 để kiểm tra) với 10 lớp, vì vậy bạn sẽ cần một lớp đầu ra softmax với 10 nơ-ron. Hãy nhớ tìm kiếm tốc độ học phù hợp mỗi khi bạn thay đổi kiến trúc hoặc siêu tham số của mô hình. c. Bây giờ hãy thử thêm chuẩn hóa theo batch và so sánh các đường cong học tập: nó có hội tụ nhanh hơn trước không? Nó có tạo ra một mô hình tốt hơn không? Nó ảnh hưởng đến tốc độ huấn luyện như thế nào? d. Thử thay thế chuẩn hóa theo batch bằng SELU, và thực hiện các điều chỉnh cần thiết để đảm bảo mạng tự chuẩn hóa (tức là chuẩn hóa các đặc trưng đầu vào, sử dụng khởi tạo chuẩn LeCun, đảm bảo DNN chỉ chứa một chuỗi các lớp dày đặc, v.v.). e. Thử chính quy hóa mô hình bằng alpha dropout. Sau đó, không huấn luyện lại mô hình của bạn, xem liệu bạn có thể đạt được độ chính xác tốt hơn bằng cách sử dụng MC dropout. f. Huấn luyện lại mô hình của bạn bằng cách sử dụng lập lịch 1 chu kỳ và xem liệu nó có cải thiện tốc độ huấn luyện và độ chính xác của mô hình không.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 12. MÔ HÌNH TÙY CHỈNH VÀ HUẤN LUYỆN VỚI TENSORFLOW

Cho đến nay, chúng ta chỉ sử dụng API cấp cao của TensorFlow, Keras, nhưng nó đã giúp chúng ta đi được khá xa: chúng ta đã xây dựng nhiều kiến trúc mạng nơ-ron khác nhau, bao gồm mạng hồi quy và phân loại, mạng Rộng & Sâu, và mạng tự chuẩn hóa, sử dụng đủ loại kỹ thuật, chẳng hạn như chuẩn hóa theo batch, dropout và lịch trình tốc độ học. Thực tế, 95% các trường hợp sử dụng bạn sẽ gặp phải sẽ không yêu cầu bất cứ điều gì ngoài Keras (và `tf.data`; xem Chương 13). Nhưng bây giờ là lúc để đi sâu hơn vào TensorFlow và xem xét API Python cấp thấp hơn của nó. Điều này sẽ hữu ích khi bạn cần kiểm soát thêm để viết các hàm mất mát tùy chỉnh, số liệu tùy chỉnh, lớp, mô hình, trình khởi tạo, trình chính quy hóa, ràng buộc trọng số và hơn thế nữa. Bạn thậm chí có thể cần kiểm soát hoàn toàn vòng lặp huấn luyện; ví dụ, để áp dụng các phép biến đổi hoặc ràng buộc đặc biệt cho các gradient (ngoài việc chỉ cắt chúng) hoặc để sử dụng nhiều bộ tối ưu hóa cho các phần khác nhau của mạng. Chúng ta sẽ đề cập đến tất cả các trường hợp này trong chương này, và chúng ta cũng sẽ xem xét cách bạn có thể tăng cường các mô hình tùy chỉnh và thuật toán huấn luyện của mình bằng tính năng tạo biểu đồ tự động của TensorFlow. Nhưng trước tiên, hãy cùng khám phá nhanh TensorFlow.

Giới thiệu nhanh về TensorFlow

Như bạn đã biết, TensorFlow là một thư viện mạnh mẽ cho tính toán số, đặc biệt phù hợp và được tinh chỉnh cho học máy quy mô lớn (nhưng bạn có thể sử dụng nó cho bất cứ thứ gì khác yêu cầu tính toán nặng). Nó được phát triển bởi nhóm Google Brain và nó cung cấp năng lượng cho nhiều dịch vụ quy mô lớn của Google, chẳng hạn như Google Cloud Speech, Google Photos và Google Search. Nó được mã nguồn mở vào tháng 11 năm 2015, và hiện là thư viện học sâu được sử dụng rộng rãi nhất trong ngành. Vô số dự án sử dụng TensorFlow cho tất cả các loại tác vụ học máy, chẳng hạn như phân loại hình ảnh, xử lý ngôn ngữ tự nhiên, hệ thống đề xuất và dự báo chuỗi thời gian.

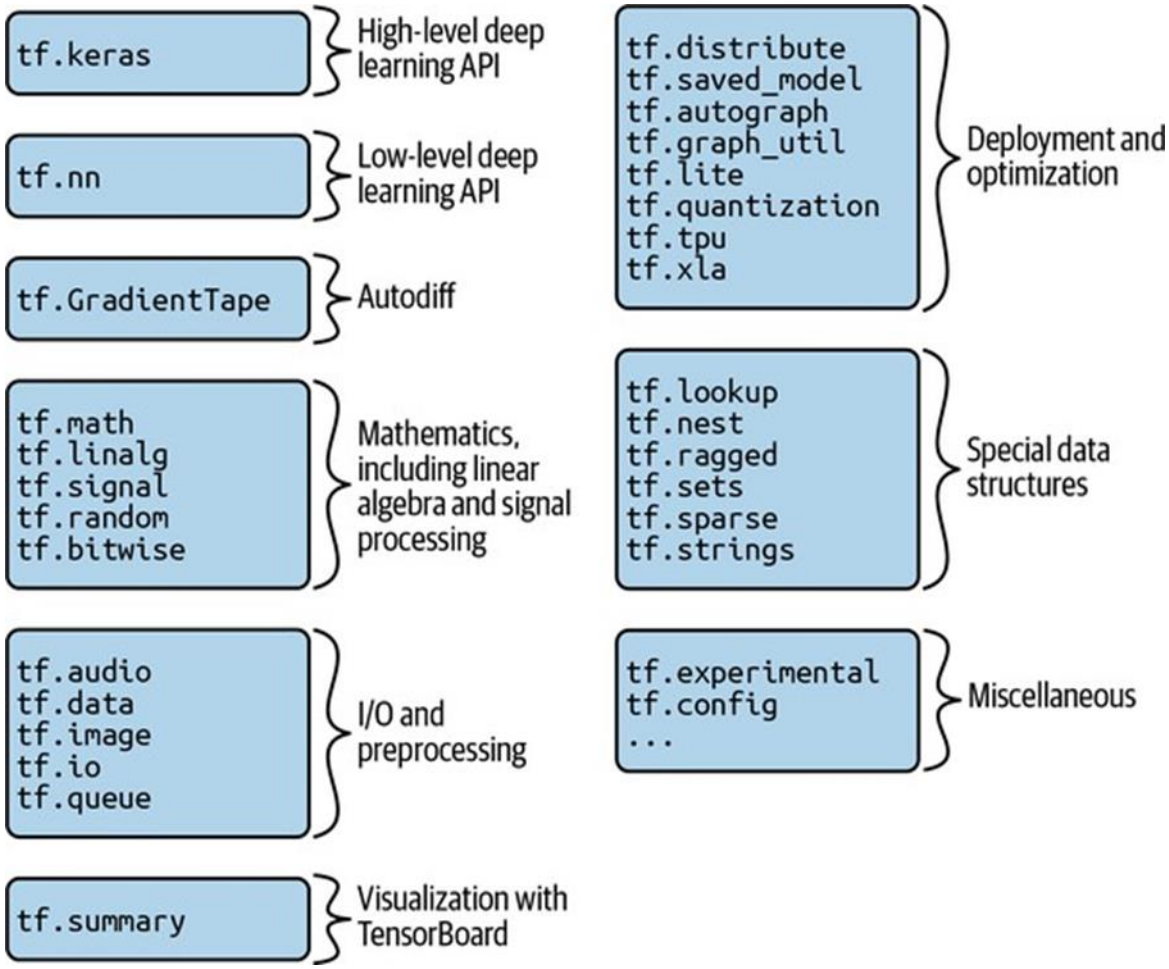
Vậy TensorFlow cung cấp những gì? Dưới đây là tóm tắt:

- Cốt lõi của nó rất giống NumPy, nhưng có hỗ trợ GPU.
- Nó hỗ trợ tính toán phân tán (trên nhiều thiết bị và máy chủ).
- Nó bao gồm một loại trình biên dịch just-in-time (JIT) cho phép nó tối ưu hóa các phép tính về tốc độ và sử dụng bộ nhớ. Nó hoạt động bằng cách trích xuất biểu đồ tính toán từ một hàm Python, tối ưu hóa nó (ví dụ: bằng cách cắt các nút không sử dụng), và chạy nó một cách hiệu quả (ví dụ: bằng cách tự động chạy các hoạt động độc lập song song).
- Biểu đồ tính toán có thể được xuất sang một định dạng di động, vì vậy bạn có thể huấn luyện một mô hình TensorFlow trong một môi trường (ví dụ: sử dụng Python trên Linux) và chạy nó trong một môi trường khác (ví dụ: sử dụng Java trên thiết bị Android).

- Nó triển khai tự động đạo hàm ngược (xem Chương 10 và Phụ lục B) và cung cấp một số bộ tối ưu hóa xuất sắc, chẳng hạn như RMSProp và Nadam (xem Chương 11), vì vậy bạn có thể dễ dàng giảm thiểu tất cả các loại hàm mất mát.

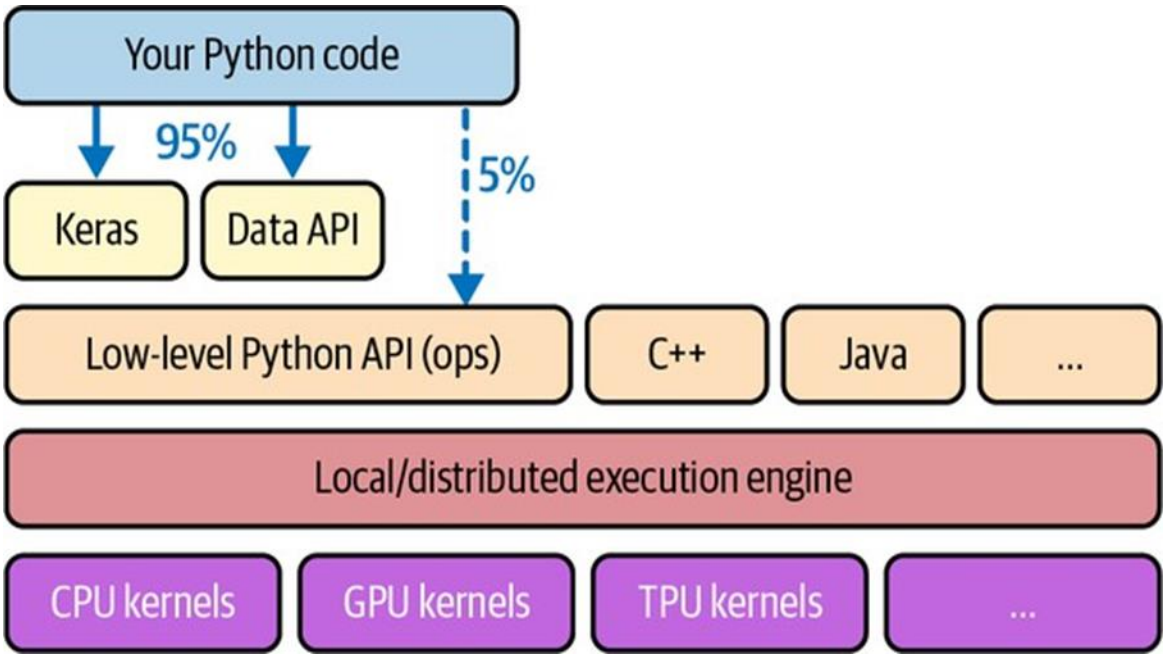
TensorFlow cung cấp nhiều tính năng hơn được xây dựng trên các tính năng cốt lõi này: quan trọng nhất tất nhiên là Keras, nhưng nó cũng có các phép toán tải và tiền xử lý dữ liệu (`tf.data`, `tf.io`, v.v.), các phép toán xử lý hình ảnh (`tf.image`), các phép toán xử lý tín hiệu (`tf.signal`), và hơn thế nữa (xem Hình 12-1 để biết tổng quan về API Python của TensorFlow).

Ở cấp độ thấp nhất, mỗi phép toán TensorFlow (op) được triển khai bằng mã C++ hiệu quả cao. Nhiều phép toán có nhiều triển khai được gọi là `kernels`: mỗi `kernel` dành riêng cho một loại thiết bị cụ thể, chẳng hạn như CPU, GPU hoặc thậm chí TPU (đơn vị xử lý tensor). Như bạn có thể biết, GPU có thể tăng tốc đáng kể các phép tính bằng cách chia chúng thành nhiều phần nhỏ hơn và chạy chúng song song trên nhiều luồng GPU. TPU thậm chí còn nhanh hơn: chúng là chip ASIC tùy chỉnh được xây dựng đặc biệt cho các phép toán học sâu (chúng ta sẽ thảo luận cách sử dụng TensorFlow với GPU hoặc TPU trong Chương 19).



Hình 12-1. API Python của TensorFlow

Kiến trúc của TensorFlow được thể hiện trong Hình 12-2. Hầu hết thời gian mã của bạn sẽ sử dụng các API cấp cao (đặc biệt là Keras và `tf.data`), nhưng khi bạn cần linh hoạt hơn, bạn sẽ sử dụng API Python cấp thấp hơn, xử lý trực tiếp các tensor. Trong mọi trường hợp, công cụ thực thi của TensorFlow sẽ đảm nhiệm việc chạy các hoạt động một cách hiệu quả, ngay cả trên nhiều thiết bị và máy nếu bạn yêu cầu. TensorFlow chạy không chỉ trên Windows, Linux và macOS, mà còn trên thiết bị di động (sử dụng TensorFlow Lite), bao gồm cả iOS và Android (xem Chương 19). Lưu ý rằng API cho các ngôn ngữ khác cũng có sẵn, nếu bạn không muốn sử dụng API Python: có API C++, Java và Swift. Thậm chí còn có một triển khai JavaScript được gọi là TensorFlow.js cho phép chạy các mô hình của bạn trực tiếp trong trình duyệt.



Hình 12-2. Kiến trúc của TensorFlow

Có nhiều hơn nữa về TensorFlow ngoài thư viện. TensorFlow là trung tâm của một hệ sinh thái thư viện rộng lớn. Đầu tiên, có TensorBoard để trực quan hóa (xem Chương 10). Tiếp theo, có TensorFlow Extended (TFX), là một tập hợp các thư viện được Google xây dựng để đưa các dự án TensorFlow vào sản xuất: nó bao gồm các công cụ để xác thực dữ liệu, tiền xử lý, phân tích mô hình và phục vụ (với TF Serving; xem Chương 19). TensorFlow Hub của Google cung cấp một cách để dễ dàng tải xuống và tái sử dụng các mạng nơ-ron đã được huấn luyện trước. Bạn cũng có thể nhận được nhiều kiến trúc mạng nơ-ron, một số trong đó đã được huấn luyện trước, trong kho mô hình của TensorFlow. Kiểm tra Tài nguyên TensorFlow và https://github.com/jtoy/awesome-tensorflow để biết thêm các dự án dựa trên TensorFlow. Bạn sẽ tìm thấy hàng trăm dự án TensorFlow trên GitHub, vì vậy việc tìm mã hiện có cho bất cứ điều gì bạn đang cố gắng làm thường rất dễ dàng.

Cuối cùng nhưng không kém phần quan trọng, TensorFlow có một nhóm các nhà phát triển nhiệt tình và hữu ích tận tâm, cũng như một cộng đồng lớn đóng góp vào việc cải thiện nó. Để hỏi các câu hỏi kỹ thuật, bạn nên sử dụng https://stackoverflow.com và gắn thẻ câu hỏi

của bạn với tensorflow và python. Bạn có thể gửi lỗi và yêu cầu tính năng thông qua GitHub. Đối với các cuộc thảo luận chung, hãy tham gia Diễn đàn TensorFlow. OK, đã đến lúc bắt đầu viết mã!

Sử dụng TensorFlow như NumPy

API của TensorFlow xoay quanh các tensor, chúng chảy từ phép toán này sang phép toán khác – do đó có tên TensorFlow. Một tensor rất giống với ndarray của NumPy: nó thường là một mảng đa chiều, nhưng nó cũng có thể chứa một đại lượng vô hướng (một giá trị đơn giản, chẳng hạn như 42). Các tensor này sẽ rất quan trọng khi chúng ta tạo các hàm chi phí tùy chỉnh, các số liệu tùy chỉnh, các lớp tùy chỉnh và hơn thế nữa, vì vậy hãy xem cách tạo và thao tác với chúng.

Tensor và các phép toán

Bạn có thể tạo một tensor với tf.constant(). Ví dụ, đây là một tensor đại diện cho một ma trận với hai hàng và ba cột số thập phân:

```
>>> import tensorflow as tf

>>> t = tf.constant([[1., 2., 3.], [4., 5., 6.]]) # matrix

>>> t
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
```

Giống như một ndarray, một tf.Tensor có hình dạng (shape) và kiểu dữ liệu (dtype):

```
>>> t.shape
TensorShape([2, 3])

>>> t.dtype
tf.float32
```

Đánh chỉ mục hoạt động rất giống trong NumPy:

```
>>> t[:, 1:]
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[2., 3.],
       [5., 6.]], dtype=float32)>

>>> t[..., 1, tf.newaxis]
<tf.Tensor: shape=(2, 1), dtype=float32, numpy=
array([[2.],
       [5.]], dtype=float32)>
```

Quan trọng nhất, tất cả các loại phép toán tensor đều có sẵn:

```
>>> t + 10
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[11., 12., 13.],
       [14., 15., 16.]], dtype=float32)>

>>> tf.square(t)
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)>

>>> t @ tf.transpose(t)
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[14., 32.],
       [32., 77.]], dtype=float32)>
```

Lưu ý rằng viết `t + 10` tương đương với gọi `tf.add(t, 10)` (thực tế, Python gọi phương thức magic `t.__add__(10)`, cái này chỉ gọi `tf.add(t, 10)`). Các toán tử khác, như - và *, cũng được hỗ trợ. Toán tử @ đã được thêm vào Python 3.5, cho phép nhân ma trận: nó tương đương với việc gọi hàm `tf.matmul()`.

Một tensor cũng có thể chứa một giá trị vô hướng. Trong trường hợp này, hình dạng là trống:

```
>>> tf.constant(42)
<tf.Tensor: shape=(), dtype=int32, numpy=42>
```

Bạn sẽ tìm thấy tất cả các phép toán cơ bản bạn cần (`tf.add()`, `tf.multiply()`, `tf.square()`, `tf.exp()`, `tf.sqrt()`, v.v.) và hầu hết các phép toán mà bạn có thể tìm thấy trong NumPy (ví dụ: `tf.reshape()`, `tf.squeeze()`, `tf.tile()`). Một số hàm có tên khác với trong NumPy; ví dụ, `tf.reduce_mean()`, `tf.reduce_sum()`, `tf.reduce_max()`, và `tf.math.log()` tương đương với `np.mean()`, `np.sum()`, `np.max()`, và `np.log()`. Khi tên khác nhau, thường có một lý do chính đáng. Ví dụ, trong TensorFlow bạn phải viết `tf.transpose(t)`; bạn không thể chỉ viết `t.T` như trong NumPy. Lý do là hàm `tf.transpose()` không làm chính xác những gì thuộc tính `T` của NumPy làm: trong TensorFlow, một tensor mới được tạo với bản sao riêng của dữ liệu được chuyển vị, trong khi trong NumPy, `t.T` chỉ là một chế độ xem chuyển vị trên cùng dữ liệu. Tương tự, phép toán `tf.reduce_sum()` được đặt tên như vậy vì kernel GPU của nó (tức là triển khai GPU) sử dụng thuật toán giảm không đảm bảo thứ tự các phần tử được thêm vào: vì số thập phân 32 bit có độ chính xác hạn chế, kết quả có thể thay đổi rất nhỏ mỗi khi bạn gọi phép toán này. Điều tương tự cũng đúng với `tf.reduce_mean()` (nhưng tất nhiên `tf.reduce_max()` là xác định).

Tensor và NumPy

Các tensor tương thích tốt với NumPy: bạn có thể tạo một tensor từ một mảng NumPy và ngược lại. Bạn thậm chí có thể áp dụng các phép toán TensorFlow cho mảng NumPy và các phép toán NumPy cho tensor:

```
>>> import numpy as np

>>> a = np.array([2., 4., 5.])

>>> tf.constant(a)
<tf.Tensor: id=111, shape=(3,), dtype=float64, numpy=array([2., 4., 5.])>

>>> t.numpy() # hoặc np.array(t)
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)

>>> tf.square(a)

<tf.Tensor: id=116, shape=(3,), dtype=float64, numpy=array([4., 16., 25.])>

>>> np.square(t)
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)
```

Chuyển đổi kiểu dữ liệu

Chuyển đổi kiểu dữ liệu có thể làm giảm hiệu suất đáng kể và chúng có thể dễ dàng bị bỏ qua khi được thực hiện tự động. Để tránh điều này, TensorFlow không tự động thực hiện bất kỳ chuyển đổi kiểu nào: nó chỉ đưa ra một ngoại lệ nếu bạn cố gắng thực hiện một phép toán trên các tensor có kiểu không tương thích. Ví dụ, bạn không thể cộng một tensor float và một tensor integer, và bạn thậm chí không thể cộng một float 32-bit và một float 64-bit:

```
>>> tf.constant(2.) + tf.constant(40)
[...]
InvalidArgumentError: [...] expected to be a float tensor [...]

>>> tf.constant(2.) + tf.constant(40., dtype=tf.float64)
[...]
InvalidArgumentError: [...] expected to be a float tensor [...]
```

Điều này có thể hơi khó chịu lúc đầu, nhưng hãy nhớ rằng đó là vì một lý do chính đáng! Và tất nhiên bạn có thể sử dụng

`tf.cast()` khi bạn thực sự cần chuyển đổi kiểu:

```
>>> t2 = tf.constant(40., dtype=tf.float64)

>>> tf.constant(2.0) + tf.cast(t2, tf.float32)

<tf.Tensor: id=136, shape=(), dtype=float32, numpy=42.0>
```

Biến (Variables)

Các giá trị `tf.Tensor` mà chúng ta đã thấy cho đến nay là bất biến: chúng ta không thể sửa đổi chúng. Điều này có nghĩa là chúng ta không thể sử dụng các tensor thông thường để triển khai trọng số trong mạng nơ-ron, vì chúng cần được điều chỉnh bởi backpropagation. Thêm vào đó, các tham số khác cũng có thể cần thay đổi theo thời gian (ví dụ: một bộ tối ưu hóa đà theo dõi các gradient trong quá khứ).

Những gì chúng ta cần là một `tf.Variable`:

```
>>> v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])

>>> v
<tf.Variable 'Variable:0' shape=(2, 3) dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
```

Một `tf.Variable` hoạt động rất giống một `tf.Tensor`: bạn có thể thực hiện các phép toán tương tự với nó, nó cũng tương thích tốt với NumPy, và nó cũng kén chọn về kiểu dữ liệu. Nhưng nó cũng có thể được sửa đổi tại chỗ bằng cách sử dụng phương thức `assign()` (hoặc `assign_add()` hoặc `assign_sub()`, cái này tăng hoặc giảm biến theo giá trị đã cho). Bạn cũng có thể sửa đổi từng ô (hoặc lát cắt), bằng cách sử dụng phương thức `assign()` của ô (hoặc lát cắt) hoặc bằng cách sử dụng các phương thức `scatter_update()` hoặc `scatter_nd_update()`:

```
v.assign(2 * v)             # v bây giờ bằng [[2., 4., 6.], [8., 10., 12.]]
v[0, 1].assign(42)          # v bây giờ bằng [[2., 42., 6.], [8., 10., 12.]]
v[:, 2].assign([0., 1.])    # v bây giờ bằng [[2., 42., 0.], [8., 10., 1.]]
v.scatter_nd_update(         # v bây giờ bằng [[100., 42., 0.], [8., 10.,
200.]]
    indices=[[0, 0], [1, 2]], updates=[100., 200.])
```

Gán trực tiếp sẽ không hoạt động:

```
>>> v[1] = [7., 8., 9.]
[...]
TypeError: 'ResourceVariable' object does not support item assignment
```

Các cấu trúc dữ liệu khác

TensorFlow hỗ trợ một số cấu trúc dữ liệu khác, bao gồm những điều sau (xem phần “Các cấu trúc dữ liệu khác” trong sổ tay chương này hoặc Phụ lục C để biết thêm chi tiết):

- **Tensor thưa thớt (`tf.SparseTensor`)**: Biểu diễn hiệu quả các tensor chứa chủ yếu là số 0. Gói `tf.sparse` chứa các phép toán cho tensor thưa thớt.
- **Mảng tensor (`tf.TensorArray`)**: Là danh sách các tensor. Chúng có độ dài cố định theo mặc định nhưng có thể tùy chọn được mở rộng. Tất cả các tensor mà chúng chứa phải có cùng hình dạng và kiểu dữ liệu.

- **Tensor không đều (`tf.RaggedTensor`)**: Biểu diễn danh sách các tensor, tất cả đều cùng bậc và kiểu dữ liệu, nhưng có kích thước thay đổi. Các chiều mà kích thước tensor thay đổi được gọi là các chiều không đều. Gói `tf.ragged` chứa các phép toán cho tensor không đều.
- **Tensor chuỗi**: Là các tensor thông thường có kiểu `tf.string`. Chúng đại diện cho các chuỗi byte, không phải chuỗi Unicode, vì vậy nếu bạn tạo một tensor chuỗi bằng cách sử dụng một chuỗi Unicode (ví dụ: một chuỗi Python 3 thông thường như "café"), thì nó sẽ được mã hóa sang UTF-8 tự động (ví dụ: `b"caf\xc3\xa9"`). Ngoài ra, bạn có thể biểu diễn chuỗi Unicode bằng cách sử dụng các tensor kiểu `tf.int32`, trong đó mỗi mục đại diện cho một điểm mã Unicode (ví dụ: `[99, 97, 102, 233]`). Gói `tf.strings` (có `s`) chứa các phép toán cho chuỗi byte và chuỗi Unicode (và để chuyển đổi cái này sang cái kia). Điều quan trọng cần lưu ý là một `tf.string` là nguyên tử, nghĩa là độ dài của nó không xuất hiện trong hình dạng của tensor. Một khi bạn chuyển đổi nó thành một tensor Unicode (tức là một tensor kiểu `tf.int32` chứa các điểm mã Unicode), độ dài sẽ xuất hiện trong hình dạng.
- **Tập hợp**: Được biểu diễn dưới dạng các tensor thông thường (hoặc tensor thưa thớt). Ví dụ, `tf.constant([[1,2], [3, 4]])` đại diện cho hai tập hợp {1, 2} và {3, 4}. Tổng quát hơn, mỗi tập hợp được biểu diễn bởi một vector trong trục cuối cùng của tensor. Bạn có thể thao tác với các tập hợp bằng cách sử dụng các phép toán từ gói `tf.sets`.
- **Hàng đợi**: Lưu trữ các tensor qua nhiều bước. TensorFlow cung cấp nhiều loại hàng đợi khác nhau: hàng đợi FIFO cơ bản (`FIFOQueue`), cộng với các hàng đợi có thể ưu tiên một số mục (`PriorityQueue`), xáo trộn các mục của chúng (`RandomShuffleQueue`), và nhóm các mục có hình dạng khác nhau bằng cách đệm (`PaddingFIFOQueue`). Các lớp này đều nằm trong gói `tf.queue`.

Với các tensor, phép toán, biến và các cấu trúc dữ liệu khác nhau theo ý của bạn, bây giờ bạn đã sẵn sàng tùy chỉnh các mô hình và thuật toán huấn luyện của mình!

Tùy chỉnh mô hình và thuật toán huấn luyện

Bạn sẽ bắt đầu bằng cách tạo một hàm mất mát tùy chỉnh, đây là một trường hợp sử dụng đơn giản và phổ biến.

Các hàm mất mát tùy chỉnh

Giả sử bạn muốn huấn luyện một mô hình hồi quy, nhưng tập huấn luyện của bạn hơi nhiễu. Tất nhiên, bạn bắt đầu bằng cách cố gắng làm sạch tập dữ liệu của mình bằng cách loại bỏ hoặc sửa các giá trị ngoại lai, nhưng điều đó hóa ra không đủ; tập dữ liệu vẫn bị nhiễu. Bạn nên sử dụng hàm mất mát nào? Sai số bình phương trung bình có thể phạt các lỗi lớn quá nhiều và khiến mô hình của bạn không chính xác. Sai số tuyệt đối trung bình sẽ không phạt các giá trị ngoại lai nhiều, nhưng quá trình huấn luyện có thể mất một thời gian để hội tụ, và mô hình đã huấn luyện có thể không chính xác lắm. Đây có lẽ là thời điểm tốt để sử dụng mất mát Huber (được giới thiệu trong Chương 10) thay vì MSE cũ kỹ. Mất mát Huber có sẵn trong Keras (chỉ cần sử dụng một thể hiện của lớp

`tf.keras.losses.Huber`), nhưng hãy giả vờ nó không có ở đó. Để triển khai nó, chỉ cần tạo một hàm nhận các nhãn và dự đoán của mô hình làm đối số, và sử dụng các phép toán TensorFlow để tính toán một tensor chứa tất cả các mất mát (một cho mỗi mẫu):

```
def huber_fn(y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < 1
    squared_loss = tf.square(error) / 2
    linear_loss = tf.abs(error) - 0.5
    return tf.where(is_small_error, squared_loss, linear_loss)
```

Cũng có thể trả về mất mát trung bình thay vì các mất mát mẫu riêng lẻ, nhưng điều này không được khuyến khích vì nó khiến không thể sử dụng trọng số lớp hoặc trọng số mẫu khi bạn cần chúng (xem Chương 10).

Bây giờ bạn có thể sử dụng hàm mất mát Huber này khi bạn biên dịch mô hình Keras, sau đó huấn luyện mô hình của bạn như bình thường:

```
model.compile(loss=huber_fn, optimizer="nadam")
model.fit(X_train, y_train, [...])
```

Và thế là xong! Đối với mỗi batch trong quá trình huấn luyện, Keras sẽ gọi hàm

`huber_fn()` để tính toán mất mát, sau đó nó sẽ sử dụng tự động đạo hàm ngược để tính toán gradient của mất mát đối với tất cả các tham số mô hình, và cuối cùng nó sẽ thực hiện một bước giảm độ dốc (trong ví dụ này sử dụng bộ tối ưu hóa Nadam). Hơn nữa, nó sẽ theo dõi tổng mất mát kể từ đầu epoch, và nó sẽ hiển thị mất mát trung bình.

Nhưng điều gì xảy ra với mất mát tùy chỉnh này khi bạn lưu mô hình?

Lưu và tải mô hình chứa các thành phần tùy chỉnh

Lưu một mô hình chứa hàm mất mát tùy chỉnh hoạt động tốt, nhưng khi bạn tải nó, bạn sẽ cần cung cấp một từ điển ánh xạ tên hàm tới hàm thực tế. Tổng quát hơn, khi bạn tải một mô hình chứa các đối tượng tùy chỉnh, bạn cần ánh xạ các tên tới các đối tượng:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss",
                                   custom_objects={"huber_fn": huber_fn})
```

Với việc triển khai hiện tại, bất kỳ lỗi nào giữa –1 và 1 đều được coi là “nhỏ”. Nhưng nếu bạn muốn một ngưỡng khác thì sao? Một giải pháp là tạo một hàm tạo một hàm mất mát đã được cấu hình:

```
def create_huber(threshold=1.0):
    def huber_fn(y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < threshold
        squared_loss = tf.square(error) / 2
        linear_loss = threshold * tf.abs(error) - threshold ** 2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)
```

```
    return huber_fn

model.compile(loss=create_huber(2.0), optimizer="nadam")
```

Thật không may, khi bạn lưu mô hình, ngưỡng sẽ không được lưu. Điều này có nghĩa là bạn sẽ phải chỉ định giá trị ngưỡng khi tải mô hình (lưu ý rằng tên cần sử dụng là “huber_fn”, là tên của hàm bạn đã cung cấp cho Keras, không phải tên của hàm đã tạo ra nó):

```
model = tf.keras.models.load_model(
    "my_model_with_a_custom_loss_threshold_2",
    custom_objects={"huber_fn": create_huber(2.0)}
)
```

Bạn có thể giải quyết vấn đề này bằng cách tạo một lớp con của lớp

`tf.keras.losses.Loss`, và sau đó triển khai phương thức `get_config()` của nó:

```
class HuberLoss(tf.keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)

    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold ** 2 /
2
        return tf.where(is_small_error, squared_loss, linear_loss)

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

Hãy cùng xem đoạn mã này:

- Hàm tạo chấp nhận `**kwargs` và truyền chúng cho hàm tạo cha, cái này xử lý các siêu tham số tiêu chuẩn: tên của mất mát và thuật toán giảm thiểu được sử dụng để tổng hợp các mất mát thể hiện riêng lẻ. Theo mặc định, đây là “AUTO”, tương đương với “SUM_OVER_BATCH_SIZE”: mất mát sẽ là tổng các mất mát thể hiện, được trọng số hóa bằng trọng số mẫu (nếu có), và chia cho kích thước batch (không phải tổng trọng số, vì vậy đây không phải là trung bình có trọng số). Các giá trị có thể có khác là “SUM” và “NONE”.
- Phương thức `call()` nhận các nhãn và dự đoán, tính toán tất cả các mất mát thể hiện và trả về chúng.
- Phương thức `get_config()` trả về một từ điển ánh xạ tên siêu tham số tới giá trị của nó. Nó trước tiên gọi phương thức `get_config()` của lớp cha, sau đó thêm các siêu tham số mới vào từ điển này.

Bạn có thể sử dụng bất kỳ thể hiện nào của lớp này khi bạn biên dịch mô hình:

```
model.compile(loss=HuberLoss(2.), optimizer="nadam")
```

Khi bạn lưu mô hình, ngưỡng sẽ được lưu cùng với nó; và khi bạn tải mô hình, bạn chỉ cần ánh xạ tên lớp tới chính lớp đó:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss_class",
                                   custom_objects={"HuberLoss": HuberLoss})
```

Khi bạn lưu một mô hình, Keras gọi phương thức

`get_config()` của thể hiện mất mát và lưu cấu hình ở định dạng SavedModel. Khi bạn tải mô hình, nó gọi phương thức lớp

`from_config()` trên lớp `HuberLoss`: phương thức này được triển khai bởi lớp cơ sở (`Loss`) và tạo một thể hiện của lớp, truyền `**config` vào hàm tạo.

Đó là tất cả về mất mát! Như bạn sẽ thấy bây giờ, các hàm kích hoạt, trình khởi tạo, trình chính quy hóa và ràng buộc tùy chỉnh không khác nhiều.

Các hàm kích hoạt, trình khởi tạo, trình chính quy hóa và ràng buộc tùy chỉnh

Hầu hết các chức năng của Keras, chẳng hạn như hàm mất mát, trình chính quy hóa, ràng buộc, trình khởi tạo, số liệu, hàm kích hoạt, lớp, và thậm chí cả mô hình đầy đủ, có thể được tùy chỉnh theo cách tương tự. Hầu hết thời gian, bạn chỉ cần viết một hàm đơn giản với các đầu vào và đầu ra thích hợp. Dưới đây là các ví dụ về một hàm kích hoạt tùy chỉnh (tương đương với `tf.keras.activations.softplus()` hoặc `tf.nn.softplus()`), một trình khởi tạo Glorot tùy chỉnh (tương đương với `tf.keras.initializers.glorot_normal()`), một trình chính quy hóa l_1 tùy chỉnh (tương đương với `tf.keras.regularizers.l1(0.01)`), và một ràng buộc tùy chỉnh đảm bảo tất cả các trọng số đều dương (tương đương với `tf.keras.constraints.nonneg()` hoặc `tf.nn.relu()`):

```
def my_softplus(z):
    return tf.math.log(1.0 + tf.exp(z))

def my_glorot_initializer(shape, dtype=tf.float32):
    stddev = tf.sqrt(2. / (shape[0] + shape[1]))
    return tf.random.normal(shape, stddev=stddev, dtype=dtype)

def my_l1_regularizer(weights):
    return tf.reduce_sum(tf.abs(0.01 * weights))

def my_positive_weights(weights): # giá trị trả về chỉ là tf.nn.relu(weights)
    return tf.where(weights < 0., tf.zeros_like(weights), weights)
```

Như bạn có thể thấy, các đối số phụ thuộc vào loại hàm tùy chỉnh. Các hàm tùy chỉnh này sau đó có thể được sử dụng bình thường, như được thể hiện ở đây:

```
layer = tf.keras.layers.Dense(1, activation=my_softplus,
                              kernel_initializer=my_glorot_initializer,
                              kernel_regularizer=my_l1_regularizer,
                              kernel_constraint=my_positive_weights)
```

Hàm kích hoạt sẽ được áp dụng cho đầu ra của lớp `Dense` này, và kết quả của nó sẽ được chuyển đến lớp tiếp theo. Trọng số của lớp sẽ được khởi tạo bằng giá trị trả về của trình khởi tạo. Ở mỗi bước huấn luyện, trọng số sẽ được truyền cho hàm chính quy hóa để tính toán mất mát chính quy hóa, mất mát này sẽ được thêm vào mất mát chính để có được mất mát cuối cùng được sử dụng cho quá trình huấn luyện. Cuối cùng, hàm ràng buộc sẽ được gọi sau mỗi bước huấn luyện, và trọng số của lớp sẽ được thay thế bằng trọng số bị ràng buộc.

Nếu một hàm có các siêu tham số cần được lưu cùng với mô hình, thì bạn sẽ muốn tạo lớp con từ lớp thích hợp, chẳng hạn như `tf.keras.regularizers.Regularizer`, `tf.keras.constraints.Constraint`, `tf.keras.initializers.Initializer`, hoặc `tf.keras.layers.Layer` (đối với bất kỳ lớp nào, bao gồm cả hàm kích hoạt). Tương tự như cách bạn đã làm với hàm mất mát tùy chỉnh, đây là một lớp đơn giản cho chính quy hóa l_1 lưu siêu tham số `factor` của nó (lần này bạn không cần gọi hàm tạo cha hoặc phương thức `get_config()`, vì chúng không được định nghĩa bởi lớp cha):

```
class MyL1Regularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, weights):
        return tf.reduce_sum(tf.abs(self.factor * weights))

    def get_config(self):
        return {"factor": self.factor}
```

Lưu ý rằng bạn phải triển khai phương thức `call()` cho các hàm mất mát, lớp (bao gồm cả hàm kích hoạt), và mô hình, hoặc phương thức `__call__()` cho các trình chính quy hóa, trình khởi tạo và ràng buộc. Đối với các số liệu, mọi thứ hơi khác một chút, như bạn sẽ thấy bây giờ.

Các số liệu tùy chỉnh

Các hàm mất mát và số liệu về mặt khái niệm không giống nhau: các hàm mất mát (ví dụ: entropy chéo) được giảm độ dốc sử dụng để huấn luyện một mô hình, vì vậy chúng phải khả vi (ít nhất là tại các điểm chúng được đánh giá), và gradient của chúng không nên bằng 0 ở khắp mọi nơi. Hơn nữa, chúng không cần phải dễ hiểu đối với con người. Ngược lại, các số liệu (ví dụ: độ chính xác) được sử dụng để đánh giá một mô hình: chúng phải dễ hiểu hơn, và chúng có thể không khả vi hoặc có gradient bằng 0 ở khắp mọi nơi.

Điều đó nói lên rằng, trong hầu hết các trường hợp, việc định nghĩa một hàm số liệu tùy chỉnh hoàn toàn giống như định nghĩa một hàm mất mát tùy chỉnh. Thực tế, chúng ta thậm chí có thể sử dụng hàm mất mát Huber mà chúng ta đã tạo trước đó làm một số liệu; nó sẽ

hoạt động tốt (và tính bền vững cũng sẽ hoạt động theo cùng một cách, trong trường hợp này chỉ lưu tên của hàm, “huber_fn”, không phải ngưỡng):

```
model.compile(loss="mse", optimizer="nadam", metrics=[create_huber(2.0)])
```

Đối với mỗi batch trong quá trình huấn luyện, Keras sẽ tính toán số liệu này và theo dõi trung bình của nó kể từ đầu epoch. Hầu hết thời gian, đây chính xác là những gì bạn muốn. Nhưng không phải lúc nào cũng vậy! Hãy xem xét độ chính xác của một bộ phân loại nhị phân, chẳng hạn. Như bạn đã thấy trong Chương 3, độ chính xác là số lượng dương tính đúng chia cho số lượng dự đoán dương tính (bao gồm cả dương tính đúng và dương tính sai). Giả sử mô hình đã thực hiện năm dự đoán dương tính trong batch đầu tiên, bốn trong số đó là đúng: đó là độ chính xác 80%. Sau đó, giả sử mô hình đã thực hiện ba dự đoán dương tính trong batch thứ hai, nhưng tất cả đều sai: đó là độ chính xác 0% cho batch thứ hai. Nếu bạn chỉ tính trung bình của hai độ chính xác này, bạn sẽ nhận được 40%. Nhưng khoan đã — đó không phải là độ chính xác của mô hình trên hai batch này! Thật vậy, có tổng cộng bốn dương tính đúng (4 + 0) trong số tám dự đoán dương tính (5 + 3), vì vậy độ chính xác tổng thể là 50%, không phải 40%. Những gì chúng ta cần là một đối tượng có thể theo dõi số lượng dương tính đúng và số lượng dương tính sai và có thể tính toán độ chính xác dựa trên các số này khi được yêu cầu. Đây chính xác là những gì lớp `tf.keras.metrics.Precision` thực hiện:

```
>>> precision = tf.keras.metrics.Precision()

>>> precision([0, 1, 1, 1, 0, 1, 0, 1], [1, 1, 0, 1, 0, 1, 0, 1])
<tf.Tensor: shape=(), dtype=float32, numpy=0.8>

>>> precision([0, 1, 0, 0, 1, 0, 1, 1], [1, 0, 1, 1, 0, 0, 0, 0])
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>
```

Trong ví dụ này, chúng ta đã tạo một đối tượng `Precision`, sau đó chúng ta đã sử dụng nó như một hàm, truyền cho nó các nhãn và dự đoán cho batch đầu tiên, sau đó cho batch thứ hai (bạn có thể tùy chọn truyền trọng số mẫu nếu muốn). Chúng ta đã sử dụng cùng số lượng dương tính đúng và dương tính sai như trong ví dụ chúng ta vừa thảo luận. Sau batch đầu tiên, nó trả về độ chính xác 80%; sau đó sau batch thứ hai, nó trả về 50% (là độ chính xác tổng thể cho đến nay, không phải độ chính xác của batch thứ hai). Đây được gọi là **số liệu theo luồng** (hoặc số liệu trạng thái), vì nó được cập nhật dần dần, từng batch một.

Tại bất kỳ thời điểm nào, chúng ta có thể gọi phương thức `result()` để lấy giá trị hiện tại của số liệu. Chúng ta cũng có thể xem các biến của nó (theo dõi số lượng dương tính đúng và dương tính sai) bằng cách sử dụng thuộc tính `variables`, và chúng ta có thể đặt lại các biến này bằng phương thức `reset_states()`:

```
>>> precision.result()
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>

>>> precision.variables
[<tf.Variable 'true_positives:0' [...], numpy=array([4.], dtype=float32)>,
```

```
 <tf.Variable 'false_positives:0' [...], numpy=array([4.], dtype=float32)>]

>>> precision.reset_states() # cả hai biến đều được đặt lại về 0.0
```

Nếu bạn cần định nghĩa số liệu theo luồng tùy chỉnh của riêng mình, hãy tạo một lớp con của lớp `tf.keras.metrics.Metric`. Dưới đây là một ví dụ cơ bản theo dõi tổng mất mát Huber và số lượng thể hiện đã thấy cho đến nay. Khi được yêu cầu kết quả, nó trả về tỷ lệ, đó chính là trung bình mất mát Huber:

```
class HuberMetric(tf.keras.metrics.Metric):
    def __init__(self, threshold=1.0, **kwargs):
        super().__init__(**kwargs) # xử lý các đối số cơ sở (ví dụ: dtype)
        self.threshold = threshold
        self.huber_fn = create_huber(threshold)
        self.total = self.add_weight("total", initializer="zeros")
        self.count = self.add_weight("count", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        sample_metrics = self.huber_fn(y_true, y_pred)
        self.total.assign_add(tf.reduce_sum(sample_metrics))
        self.count.assign_add(tf.cast(tf.size(y_true), tf.float32))

    def result(self):
        return self.total / self.count

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

Hãy cùng xem đoạn mã này:

- Hàm tạo sử dụng phương thức `add_weight()` để tạo các biến cần thiết để theo dõi trạng thái của số liệu qua nhiều batch — trong trường hợp này, tổng của tất cả các mất mát Huber (`total`) và số lượng thể hiện đã thấy cho đến nay (`count`). Bạn có thể chỉ cần tạo các biến thủ công nếu bạn thích. Keras theo dõi bất kỳ `tf.Variable` nào được đặt làm thuộc tính (và tổng quát hơn, bất kỳ đối tượng "có thể theo dõi" nào, chẳng hạn như lớp hoặc mô hình).
- Phương thức `update_state()` được gọi khi bạn sử dụng một thể hiện của lớp này làm hàm (như chúng ta đã làm với đối tượng `Precision`). Nó cập nhật các biến, đưa ra các nhãn và dự đoán cho một batch (và trọng số mẫu, nhưng trong trường hợp này chúng ta bỏ qua chúng).
- Phương thức `result()` tính toán và trả về kết quả cuối cùng, trong trường hợp này là số liệu Huber trung bình trên tất cả các thể hiện. Khi bạn sử dụng số liệu làm hàm, phương thức `update_state()` được gọi trước, sau đó phương thức `result()` được gọi, và đầu ra của nó được trả về.
- Chúng ta cũng triển khai phương thức `get_config()` để đảm bảo ngưỡng được lưu cùng với mô hình.

- Việc triển khai mặc định của phương thức `reset_states()` đặt lại tất cả các biến về 0.0 (nhưng bạn có thể ghi đè nó nếu cần).

Khi bạn định nghĩa một số liệu bằng một hàm đơn giản, Keras tự động gọi nó cho mỗi batch, và nó theo dõi trung bình trong mỗi epoch, giống như chúng ta đã làm thủ công. Vì vậy, lợi ích duy nhất của lớp `HuberMetric` của chúng ta là ngưỡng sẽ được lưu. Nhưng tất nhiên, một số số liệu, như độ chính xác, không thể đơn giản là tính trung bình trên các batch: trong những trường hợp đó, không có lựa chọn nào khác ngoài việc triển khai một số liệu theo luồng.

Bây giờ bạn đã xây dựng một số liệu theo luồng, việc xây dựng một lớp tùy chỉnh sẽ trở nên dễ dàng!

Các lớp tùy chỉnh

Bạn có thể thỉnh thoảng muốn xây dựng một kiến trúc chứa một lớp kỳ lạ mà TensorFlow không cung cấp triển khai mặc định. Hoặc bạn có thể đơn giản là muốn xây dựng một kiến trúc rất lặp đi lặp lại, trong đó một khối lớp cụ thể được lặp lại nhiều lần, và sẽ thuận tiện khi coi mỗi khối là một lớp duy nhất. Đối với những trường hợp như vậy, bạn sẽ muốn xây dựng một lớp tùy chỉnh.

Có một số lớp không có trọng số, chẳng hạn như `tf.keras.layers.Flatten` hoặc `tf.keras.layers.ReLU`. Nếu bạn muốn tạo một lớp tùy chỉnh không có bất kỳ trọng số nào, lựa chọn đơn giản nhất là viết một hàm và bọc nó trong một lớp `tf.keras.layers.Lambda`. Ví dụ, lớp sau đây sẽ áp dụng hàm mũ cho các đầu vào của nó:

```
exponential_layer = tf.keras.layers.Lambda(lambda x: tf.exp(x))
```

Lớp tùy chỉnh này sau đó có thể được sử dụng như bất kỳ lớp nào khác, sử dụng API tuần tự, API chức năng hoặc API subclassing. Bạn cũng có thể sử dụng nó làm hàm kích hoạt, hoặc bạn có thể sử dụng `activation=tf.exp`. Lớp mũ đôi khi được sử dụng trong lớp đầu ra của một mô hình hồi quy khi các giá trị cần dự đoán có các thang đo rất khác nhau (ví dụ: 0.001, 10., 1.000.). Thực tế, hàm mũ là một trong những hàm kích hoạt tiêu chuẩn trong Keras, vì vậy bạn có thể chỉ cần sử dụng `activation="exponential"`.

Như bạn có thể đoán, để xây dựng một lớp trạng thái tùy chỉnh (tức là một lớp có trọng số), bạn cần tạo một lớp con của lớp `tf.keras.layers.Layer`. Ví dụ, lớp sau đây triển khai một phiên bản đơn giản hóa của lớp `Dense`:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
```

```
            initializer="glorot_normal")
        self.bias = self.add_weight(
            name="bias", shape=[self.units], initializer="zeros")

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "units": self.units, "activation":
                tf.keras.activations.serialize(self.activation)}
```

Hãy cùng xem đoạn mã này:

- Hàm tạo nhận tất cả các siêu tham số làm đối số (trong ví dụ này, `units` và `activation`), và quan trọng là nó cũng nhận một đối số `**kwargs`. Nó gọi hàm tạo cha, truyền các `kwargs` cho nó: điều này xử lý các đối số tiêu chuẩn như `input_shape`, `trainable`, và `name`. Sau đó, nó lưu các siêu tham số dưới dạng thuộc tính, chuyển đổi đối số `activation` thành hàm kích hoạt thích hợp bằng cách sử dụng hàm `tf.keras.activations.get()` (nó chấp nhận các hàm, các chuỗi tiêu chuẩn như "relu" hoặc "swish", hoặc đơn giản là `None`).
- Vai trò của phương thức `build()` là tạo các biến của lớp bằng cách gọi phương thức `add_weight()` cho mỗi trọng số. Phương thức `build()` được gọi lần đầu tiên khi lớp được sử dụng. Tại thời điểm đó, Keras sẽ biết hình dạng của đầu vào của lớp này, và nó sẽ truyền hình dạng đó cho phương thức `build()`, điều này thường cần thiết để tạo ra một số trọng số. Ví dụ, chúng ta cần biết số lượng nơ-ron trong lớp trước đó để tạo ma trận trọng số kết nối (tức là "kernel"): điều này tương ứng với kích thước của chiều cuối cùng của đầu vào. Ở cuối phương thức `build()` (và chỉ ở cuối), bạn phải gọi phương thức `build()` của lớp cha: điều này cho Keras biết rằng lớp đã được xây dựng (nó chỉ đặt `self.built = True`).
- Phương thức `call()` thực hiện các phép toán mong muốn. Trong trường hợp này, chúng ta tính toán phép nhân ma trận giữa các đầu vào `X` và kernel của lớp, chúng ta cộng vector bias, và chúng ta áp dụng hàm kích hoạt cho kết quả, và điều này cho chúng ta đầu ra của lớp.
- Phương thức `get_config()` giống như trong các lớp tùy chỉnh trước đó. Lưu ý rằng chúng ta lưu cấu hình đầy đủ của hàm kích hoạt bằng cách gọi `tf.keras.activations.serialize()`.

Bạn có thể sử dụng lớp `MyDense` giống như bất kỳ lớp nào khác!

Để tạo một lớp có nhiều đầu vào (ví dụ: `Concatenate`), đối số cho phương thức `call()` phải là một tuple chứa tất cả các đầu vào. Để tạo một lớp có nhiều đầu ra, phương thức `call()` nên trả về danh sách các đầu ra. Ví dụ, lớp đồ chơi sau đây nhận hai đầu vào và trả về ba đầu ra:

```
class MyMultiLayer(tf.keras.layers.Layer):
    def call(self, X):
        X1, X2 = X
        return X1 + X2, X1 * X2, X1 / X2
```

Lớp này bây giờ có thể được sử dụng như bất kỳ lớp nào khác, nhưng tất nhiên chỉ sử dụng API chức năng và API subclassing, không phải API tuần tự (chỉ chấp nhận các lớp có một đầu vào và một đầu ra).

Nếu lớp của bạn cần có hành vi khác nhau trong quá trình huấn luyện và trong quá trình kiểm tra (ví dụ: nếu nó sử dụng các lớp `Dropout` hoặc `BatchNormalization`), thì bạn phải thêm đối số `training` vào phương thức `call()` và sử dụng đối số này để quyết định phải làm gì. Ví dụ, hãy tạo một lớp thêm nhiễu Gaussian trong quá trình huấn luyện (để chính quy hóa) nhưng không làm gì trong quá trình kiểm tra (Keras có một lớp làm điều tương tự, `tf.keras.layers.GaussianNoise`):

```
class MyGaussianNoise(tf.keras.layers.Layer):
    def __init__(self, stddev, **kwargs):
        super().__init__(**kwargs)
        self.stddev = stddev

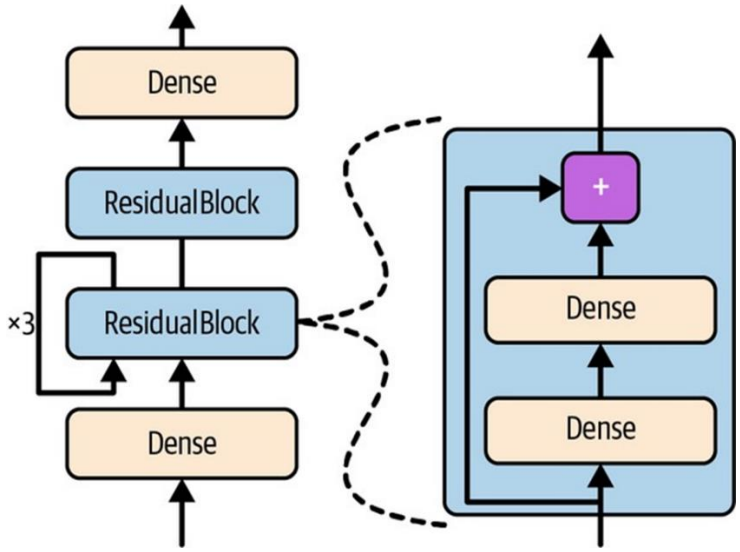
    def call(self, X, training=False):
        if training:
            noise = tf.random.normal(tf.shape(X), stddev=self.stddev)
            return X + noise
        else:
            return X
```

Với điều đó, bây giờ bạn có thể xây dựng bất kỳ lớp tùy chỉnh nào bạn cần! Bây giờ chúng ta hãy xem cách tạo các mô hình tùy chỉnh.

Mô hình tùy chỉnh

Chúng ta đã xem xét việc tạo các lớp mô hình tùy chỉnh trong Chương 10, khi chúng ta thảo luận về API subclassing. Thật đơn giản: kế thừa từ lớp

`tf.keras.Model`, tạo các lớp và biến trong hàm tạo, và triển khai phương thức `call()` để làm bất cứ điều gì bạn muốn mô hình thực hiện. Ví dụ, giả sử chúng ta muốn xây dựng mô hình được thể hiện trong Hình 12-3.



Hình 12-3. Ví dụ mô hình tùy chỉnh: một mô hình tùy ý với lớp ResidualBlock tùy chỉnh chứa kết nối bỏ qua

Các đầu vào đi qua một lớp dày đặc đầu tiên, sau đó qua một khối residual bao gồm hai lớp dày đặc và một phép toán cộng (như bạn sẽ thấy trong Chương 14, một khối residual cộng đầu vào của nó vào đầu ra của nó), sau đó qua cùng khối residual này thêm ba lần nữa, sau đó qua một khối residual thứ hai, và kết quả cuối cùng đi qua một lớp đầu ra dày đặc. Đừng lo lắng nếu mô hình này không có nhiều ý nghĩa; đó chỉ là một ví dụ để minh họa thực tế rằng bạn có thể dễ dàng xây dựng bất kỳ loại mô hình nào bạn muốn, ngay cả một mô hình chứa các vòng lặp và kết nối bỏ qua. Để triển khai mô hình này, tốt nhất là tạo một lớp `ResidualBlock` trước, vì chúng ta sẽ tạo một vài khối giống hệt nhau (và chúng ta có thể muốn tái sử dụng nó trong một mô hình khác):

```
class ResidualBlock(tf.keras.layers.Layer):
    def __init__(self, n_layers, n_neurons, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(n_neurons, activation="relu",
                                             kernel_initializer="he_normal")
                       for _ in range(n_layers)]

    def call(self, inputs):
        Z = inputs
        for layer in self.hidden:
            Z = layer(Z)
        return inputs + Z
```

Lớp này hơi đặc biệt vì nó chứa các lớp khác. Điều này được Keras xử lý một cách minh bạch: nó tự động phát hiện rằng thuộc tính `hidden` chứa các đối tượng có thể theo dõi (các lớp trong trường hợp này), vì vậy các biến của chúng tự động được thêm vào danh sách các biến của lớp này. Phần còn lại của lớp này tự giải thích. Tiếp theo, hãy sử dụng API subclassing để định nghĩa mô hình đó:

```
class ResidualRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = tf.keras.layers.Dense(30, activation="relu",
                                             kernel_initializer="he_normal")
        self.block1 = ResidualBlock(2, 30)
        self.block2 = ResidualBlock(2, 30)
        self.out = tf.keras.layers.Dense(output_dim)

    def call(self, inputs):
        Z = self.hidden1(inputs)
        for _ in range(1 + 3):
            Z = self.block1(Z)
        Z = self.block2(Z)
        return self.out(Z)
```

Chúng ta tạo các lớp trong hàm tạo và sử dụng chúng trong phương thức `call()`. Mô hình này sau đó có thể được sử dụng như bất kỳ mô hình nào khác (biên dịch nó, huấn luyện nó, đánh giá nó và sử dụng nó để đưa ra dự đoán). Nếu bạn cũng muốn có thể lưu mô hình bằng phương thức `save()` và tải nó bằng hàm `tf.keras.models.load_model()`, bạn phải triển khai phương thức `get_config()` (như chúng ta đã làm trước đó) trong cả lớp `ResidualBlock` và lớp `ResidualRegressor`. Ngoài ra, bạn có thể lưu và tải trọng số bằng các phương thức `save_weights()` và `load_weights()`.

Lớp `Model` là một lớp con của lớp `Layer`, vì vậy các mô hình có thể được định nghĩa và sử dụng chính xác như các lớp. Nhưng một mô hình có một số chức năng bổ sung, bao gồm tất nhiên các phương thức `compile()`, `fit()`, `evaluate()`, và `predict()` (và một vài biến thể), cộng với phương thức `get_layer()` (có thể trả về bất kỳ lớp nào của mô hình theo tên hoặc theo chỉ số) và phương thức `save()` (và hỗ trợ cho `tf.keras.models.load_model()` và `tf.keras.models.clone_model()`).

Với điều đó, bạn có thể tự nhiên và ngắn gọn xây dựng gần như bất kỳ mô hình nào bạn tìm thấy trong một bài báo, sử dụng API tuần tự, API chức năng, API subclassing, hoặc thậm chí là sự kết hợp của chúng. “Gần như” bất kỳ mô hình nào?

Vâng, vẫn còn một vài điều chúng ta cần xem xét: thứ nhất, làm thế nào để định nghĩa các hàm mất mát hoặc số liệu dựa trên các chi tiết nội bộ của mô hình, và thứ hai, làm thế nào để xây dựng một vòng lặp huấn luyện tùy chỉnh.

Các hàm mất mát và số liệu dựa trên các chi tiết nội bộ của mô hình

Các hàm mất mát và số liệu tùy chỉnh mà chúng ta đã định nghĩa trước đó đều dựa trên các nhãn và dự đoán (và tùy chọn là trọng số mẫu).

Sẽ có những lúc bạn muốn định nghĩa các hàm mất mát dựa trên các phần khác của mô hình của bạn, chẳng hạn như trọng số hoặc kích hoạt của các lớp ẩn của nó. Điều này có thể hữu ích cho mục đích chính quy hóa hoặc để giám sát một số khía cạnh nội bộ của mô hình của bạn.

Để định nghĩa một hàm mất mát tùy chỉnh dựa trên các chi tiết nội bộ của mô hình, hãy tính toán nó dựa trên bất kỳ phần nào của mô hình bạn muốn, sau đó truyền kết quả cho phương thức `add_loss()`. Ví dụ, hãy xây dựng một mô hình MLP hồi quy tùy chỉnh bao gồm một chồng năm lớp ẩn cộng với một lớp đầu ra. Mô hình tùy chỉnh này cũng sẽ có một đầu ra phụ trợ trên lớp ẩn trên cùng. Mất mát liên quan đến đầu ra phụ trợ này sẽ được gọi là **mất mát tái tạo** (xem Chương 17): đó là sự khác biệt bình phương trung bình giữa tái tạo và đầu vào. Bằng cách thêm mất mát tái tạo này vào mất mát chính, chúng ta sẽ khuyến khích mô hình bảo toàn càng nhiều thông tin càng tốt thông qua các lớp ẩn — ngay cả thông tin không trực tiếp hữu ích cho chính tác vụ hồi quy. Trong thực tế, mất mát này đôi khi cải thiện khả năng tổng quát hóa (nó là một mất mát chính quy hóa). Cũng có thể thêm một số liệu tùy chỉnh bằng cách sử dụng phương thức `add_metric()` của mô hình. Dưới đây là mã cho mô hình tùy chỉnh này với một mất mát tái tạo tùy chỉnh và một số liệu tương ứng:

```
class ReconstructingRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(30, activation="relu",
                                             kernel_initializer="he_normal")
                       for _ in range(5)]
        self.out = tf.keras.layers.Dense(output_dim)
        self.reconstruction_mean = tf.keras.metrics.Mean(
            name="reconstruction_error")

    def build(self, batch_input_shape):
        n_inputs = batch_input_shape[-1]
        self.reconstruct = tf.keras.layers.Dense(n_inputs)

    def call(self, inputs, training=False):
        Z = inputs
        for layer in self.hidden:
            Z = layer(Z)
        reconstruction = self.reconstruct(Z)
        recon_loss = tf.reduce_mean(tf.square(reconstruction - inputs))

        self.add_loss(0.05 * recon_loss)
        if training:
            result = self.reconstruction_mean(recon_loss)
            self.add_metric(result)

        return self.out(Z)
```

Hãy cùng xem đoạn mã này:

- Hàm tạo tạo DNN với năm lớp ẩn dày đặc và một lớp đầu ra dày đặc. Chúng ta cũng tạo một số liệu theo luồng `Mean` để theo dõi lỗi tái tạo trong quá trình huấn luyện.

- Phương thức `build()` tạo một lớp dày đặc bổ sung sẽ được sử dụng để tái tạo đầu vào của mô hình. Nó phải được tạo ở đây vì số đơn vị của nó phải bằng số đầu vào, và số này không xác định trước khi phương thức `build()` được gọi.
- Phương thức `call()` xử lý đầu vào thông qua tất cả năm lớp ẩn, sau đó truyền kết quả qua lớp tái tạo, cái này tạo ra tái tạo.
- Sau đó, phương thức `call()` tính toán mất mát tái tạo (sự khác biệt bình phương trung bình giữa tái tạo và đầu vào), và thêm nó vào danh sách các mất mát của mô hình bằng phương thức `add_loss()`. Lưu ý rằng chúng ta giảm tỷ lệ mất mát tái tạo bằng cách nhân nó với 0.05 (đây là một siêu tham số bạn có thể điều chỉnh). Điều này đảm bảo rằng mất mát tái tạo không chi phối mất mát chính.
- Tiếp theo, chỉ trong quá trình huấn luyện, phương thức `call()` cập nhật số liệu tái tạo và thêm nó vào mô hình để nó có thể được hiển thị. Ví dụ mã này thực sự có thể được đơn giản hóa bằng cách gọi `self.add_metric(recon_loss)` thay thế: Keras sẽ tự động theo dõi trung bình cho bạn.
- Cuối cùng, phương thức `call()` truyền đầu ra của các lớp ẩn đến lớp đầu ra và trả về đầu ra của nó.

Cả tổng mất mát và mất mát tái tạo sẽ giảm trong quá trình huấn luyện:

```
Epoch 1/5
363/363 [========] - 1s 820us/step - loss: 0.7640 - reconstruction_error:
1.2728
Epoch 2/5
363/363 [========] - 0s 809us/step - loss: 0.4584 - reconstruction_error:
0.6340
[...]
```

Trong hầu hết các trường hợp, mọi thứ chúng ta đã thảo luận cho đến nay sẽ đủ để triển khai bất kỳ mô hình nào bạn muốn xây dựng, ngay cả với kiến trúc, mất mát và số liệu phức tạp. Tuy nhiên, đối với một số kiến trúc, chẳng hạn như GANs (xem Chương 17), bạn sẽ phải tùy chỉnh vòng lặp huấn luyện. Trước khi đến đó, chúng ta phải xem cách tính toán gradient tự động trong TensorFlow.

Tính toán Gradient bằng Autodiff

Để hiểu cách sử dụng autodiff (xem Chương 10 và Phụ lục B) để tính toán gradient tự động, hãy xem xét một hàm đồ chơi đơn giản:

```
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2
```

Nếu bạn biết giải tích, bạn có thể phân tích thấy rằng đạo hàm riêng của hàm này đối với w_1 là $6w_1 + 2w_2$. Bạn cũng có thể thấy rằng đạo hàm riêng của nó đối với w_2 là $2w_1$. Ví dụ, tại điểm $(w_1, w_2) = (5,3)$, các đạo hàm riêng này lần lượt bằng 36 và 10, vì vậy vector gradient tại điểm này là $(36,10)$. Nhưng nếu đây là một mạng nơ-ron, hàm sẽ phức tạp hơn nhiều, thường có hàng chục nghìn tham số, và việc tìm các đạo hàm riêng bằng tay một cách phân tích sẽ là một nhiệm vụ gần như không thể. Một giải pháp có thể là tính toán một xấp xỉ của

mỗi đạo hàm riêng bằng cách đo lường mức độ thay đổi của đầu ra hàm khi bạn điều chỉnh tham số tương ứng một lượng rất nhỏ:

```
>>> w1, w2 = 5, 3

>>> eps = 1e-6

>>> (f(w1 + eps, w2) - f(w1, w2)) / eps
36.000003007075065

>>> (f(w1, w2 + eps) - f(w1, w2)) / eps
10.000000003174137
```

Trông khá đúng! Cách này hoạt động khá tốt và dễ thực hiện, nhưng nó chỉ là một xấp xỉ, và quan trọng là bạn cần gọi `f()` ít nhất một lần cho mỗi tham số (không phải hai lần, vì chúng ta có thể tính `f(w1, w2)` chỉ một lần). Việc phải gọi `f()` ít nhất một lần cho mỗi tham số khiến cách tiếp cận này không khả thi đối với các mạng nơ-ron lớn. Vì vậy, thay vào đó, chúng ta nên sử dụng tự động đạo hàm ngược (reverse-mode autodiff). TensorFlow làm cho điều này khá đơn giản:

```
w1, w2 = tf.Variable(5.), tf.Variable(3.)
with tf.GradientTape() as tape:
    z = f(w1, w2)

gradients = tape.gradient(z, [w1, w2])
```

Đầu tiên chúng ta định nghĩa hai biến `w1` và `w2`, sau đó chúng ta tạo một ngữ cảnh `tf.GradientTape` sẽ tự động ghi lại mọi phép toán liên quan đến một biến, và cuối cùng chúng ta yêu cầu tape này tính toán gradient của kết quả `z` đối với cả hai biến `[w1, w2]`. Hãy xem các gradient mà TensorFlow đã tính toán:

```
>>> gradients
[<tf.Tensor: shape=(), dtype=float32, numpy=36.0>,
 <tf.Tensor: shape=(), dtype=float32, numpy=10.0>]
```

Hoàn hảo! Kết quả không chỉ chính xác (độ chính xác chỉ bị giới hạn bởi các lỗi dấu phẩy động), mà phương thức `gradient()` chỉ đi qua các phép tính đã ghi một lần (theo thứ tự ngược lại), bất kể có bao nhiêu biến, vì vậy nó cực kỳ hiệu quả. Thật kỳ diệu!

Tape tự động bị xóa ngay sau khi bạn gọi phương thức `gradient()` của nó, vì vậy bạn sẽ gặp ngoại lệ nếu bạn cố gắng gọi `gradient()` hai lần:

```
with tf.GradientTape() as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1) # trả về tensor 36.0
dz_dw2 = tape.gradient(z, w2) # raises a RuntimeError!
```

Nếu bạn cần gọi `gradient()` nhiều hơn một lần, bạn phải làm cho tape bền bỉ và xóa nó mỗi khi bạn dùng xong để giải phóng tài nguyên:

```
with tf.GradientTape(persistent=True) as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1) # trả về tensor 36.0
dz_dw2 = tape.gradient(z, w2) # trả về tensor 10.0, hoạt động tốt!

del tape
```

Theo mặc định, tape sẽ chỉ theo dõi các phép toán liên quan đến biến, vì vậy nếu bạn cố gắng tính toán gradient của z đối với bất cứ thứ gì không phải là biến, kết quả sẽ là `None`:

```
c1, c2 = tf.constant(5.), tf.constant(3.)
with tf.GradientTape() as tape:
    z = f(c1, c2)

gradients = tape.gradient(z, [c1, c2]) # trả về [None, None]
```

Tuy nhiên, bạn có thể buộc tape theo dõi bất kỳ tensor nào bạn thích, để ghi lại mọi phép toán liên quan đến chúng. Sau đó, bạn có thể tính toán gradient đối với các tensor này, như thể chúng là biến:

```
with tf.GradientTape() as tape:
    tape.watch(c1)
    tape.watch(c2)
    z = f(c1, c2)

gradients = tape.gradient(z, [c1, c2]) # trả về [tensor 36., tensor 10.]
```

Điều này có thể hữu ích trong một số trường hợp, chẳng hạn như nếu bạn muốn triển khai một mất mát chính quy hóa phạt các kích hoạt thay đổi nhiều khi đầu vào thay đổi ít: mất mát sẽ dựa trên gradient của các kích hoạt đối với đầu vào. Vì đầu vào không phải là biến, bạn sẽ cần bảo tape theo dõi chúng.

Hầu hết thời gian, một tape gradient được sử dụng để tính toán gradient của một giá trị duy nhất (thường là mất mát) đối với một tập hợp các giá trị (thường là các tham số mô hình). Đây là nơi tự động đạo hàm ngược tỏa sáng, vì nó chỉ cần thực hiện một lượt tiến và một lượt ngược để nhận tất cả các gradient cùng một lúc. Nếu bạn cố gắng tính toán gradient của một vector, ví dụ một vector chứa nhiều mất mát, thì TensorFlow sẽ tính toán gradient của tổng vector đó. Vì vậy, nếu bạn cần lấy các gradient riêng lẻ (ví dụ: gradient của mỗi mất mát đối với các tham số mô hình), bạn phải gọi phương thức `jacobian()` của tape: nó sẽ thực hiện tự động đạo hàm ngược một lần cho mỗi mất mát trong vector (tất cả song song theo mặc định). Thậm chí có thể tính toán đạo hàm riêng cấp hai (Hessian, tức là đạo hàm riêng của đạo hàm riêng), nhưng điều này hiếm khi cần trong thực tế (xem phần “Tính toán Gradient bằng Autodiff” của sổ tay chương này để biết ví dụ).

Trong một số trường hợp, bạn có thể muốn dừng gradient backpropagating qua một phần nào đó của mạng nơ-ron của bạn. Để làm điều này, bạn phải sử dụng hàm `tf.stop_gradient()`. Hàm trả về đầu vào của nó trong lượt tiến (giống như `tf.identity()`), nhưng nó không cho phép gradient đi qua trong backpropagation (nó hoạt động như một hằng số):

```
def f(w1, w2):
    return 3 * w1 ** 2 + tf.stop_gradient(2 * w1 * w2)

with tf.GradientTape() as tape:
    z = f(w1, w2) # lượt tiến không bị ảnh hưởng bởi stop_gradient()

gradients = tape.gradient(z, [w1, w2]) # trả về [tensor 30., None]
```

Cuối cùng, đôi khi bạn có thể gặp phải một số vấn đề số học khi tính toán gradient. Ví dụ, nếu bạn tính toán gradient của hàm căn bậc hai tại $x = 10^{-50}$, kết quả sẽ là vô hạn. Trong thực tế, độ dốc tại điểm đó không phải là vô hạn, nhưng nó lớn hơn khả năng xử lý của các số thập phân 32 bit:

```
>>> x = tf.Variable(1e-50)

>>> with tf.GradientTape() as tape:
...     z = tf.sqrt(x)
...
>>> tape.gradient(z, [x])
[<tf.Tensor: shape=(), dtype=float32, numpy=inf>]
```

Để giải quyết vấn đề này, thường nên thêm một giá trị nhỏ vào x (chẳng hạn như 10^{-6}) khi tính căn bậc hai của nó.

Hàm mũ cũng là một nguồn gây đau đầu thường xuyên, vì nó tăng cực kỳ nhanh. Ví dụ, cách hàm `my_softplus()` được định nghĩa trước đó không ổn định về mặt số học. Nếu bạn tính `my_softplus(100.0)`, bạn sẽ nhận được vô hạn thay vì kết quả chính xác (khoảng 100). Nhưng có thể viết lại hàm để làm cho nó ổn định về mặt số học: hàm softplus được định nghĩa là $log\left(1 + exp(z)\right)$, cái này cũng bằng $log\left(1 + exp(-|\ z\ |)\right) + max(z, 0)$ (xem sổ tay để biết chứng minh toán học) và lợi thế của dạng thứ hai này là số hạng mũ không thể bùng nổ. Vì vậy, đây là một triển khai tốt hơn của hàm `my_softplus()`:

```
def my_softplus(z):
    return tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0., z)
```

Trong một số trường hợp hiếm hoi, một hàm ổn định về mặt số học vẫn có thể có các gradient không ổn định về mặt số học. Trong những trường hợp như vậy, bạn sẽ phải cho TensorFlow biết phương trình nào sẽ sử dụng cho các gradient, thay vì để nó sử dụng autodiff. Để làm điều này, bạn phải sử dụng bộ trang trí `@tf.custom_gradient` khi định nghĩa hàm, và trả về cả kết quả thông thường của hàm và một hàm tính toán gradient. Ví dụ, hãy cập nhật hàm `my_softplus()` để cũng trả về một hàm gradient ổn định về mặt số học:

```
@tf.custom_gradient
def my_softplus(z):
    def my_softplus_gradients(grads): # grads = được backprop từ các lớp trên
        return grads * (1 - 1 / (1 + tf.exp(z))) # gradient ổn định của
softplus
    result = tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0., z)
    return result, my_softplus_gradients
```

Nếu bạn biết giải tích vi phân (xem sổ tay hướng dẫn về chủ đề này), bạn có thể tìm thấy rằng đạo hàm của $log\big(1 + exp(z)\big)$ là $exp(z)/\big(1 + exp(z)\big)$. Nhưng dạng này không ổn định: đối với các giá trị lớn của z , nó kết thúc việc tính toán vô hạn chia cho vô hạn, trả về `NaN`. Tuy nhiên, với một chút biến đổi đại số, bạn có thể chứng minh rằng nó cũng bằng $1 - 1/\big(1 + exp(z)\big)$, cái này ổn định.

Hàm `my_softplus_gradients()` sử dụng phương trình này để tính toán gradient. Lưu ý rằng hàm này sẽ nhận làm đầu vào các gradient đã được backpropagated cho đến hàm `my_softplus()`, và theo quy tắc chuỗi chúng ta phải nhân chúng với gradient của hàm này.

Bây giờ khi chúng ta tính toán gradient của hàm `my_softplus()`, chúng ta nhận được kết quả phù hợp, ngay cả đối với các giá trị đầu vào lớn.

Chúc mừng! Bây giờ bạn có thể tính toán gradient của bất kỳ hàm nào (miễn là nó khả vi tại điểm bạn tính toán), thậm chí chặn backpropagation khi cần, và viết các hàm gradient của riêng bạn! Điều này có lẽ linh hoạt hơn bạn cần, ngay cả khi bạn xây dựng các vòng lặp huấn luyện tùy chỉnh của riêng bạn. Bạn sẽ thấy cách làm điều đó tiếp theo.

Vòng lặp huấn luyện tùy chỉnh

Trong một số trường hợp, phương thức `fit()` có thể không đủ linh hoạt cho những gì bạn cần làm. Ví dụ, bài báo Wide & Deep mà chúng ta đã thảo luận trong Chương 10 sử dụng hai bộ tối ưu hóa khác nhau: một cho đường dẫn rộng và một cho đường dẫn sâu. Vì phương thức `fit()` chỉ sử dụng một bộ tối ưu hóa (bộ tối ưu hóa mà chúng ta chỉ định khi biên dịch mô hình), việc triển khai bài báo này đòi hỏi phải viết vòng lặp tùy chỉnh của riêng bạn.

Bạn cũng có thể muốn viết các vòng lặp huấn luyện tùy chỉnh chỉ đơn giản là để cảm thấy tự tin hơn rằng chúng làm chính xác những gì bạn muốn (có lẽ bạn không chắc chắn về một số chi tiết của phương thức `fit()`). Đôi khi cảm thấy an toàn hơn khi làm mọi thứ rõ ràng. Tuy nhiên, hãy nhớ rằng việc viết một vòng lặp huấn luyện tùy chỉnh sẽ làm cho mã của bạn dài hơn, dễ bị lỗi hơn và khó bảo trì hơn.

Đầu tiên, hãy xây dựng một mô hình đơn giản. Không cần biên dịch nó, vì chúng ta sẽ xử lý vòng lặp huấn luyện thủ công:

```
import tensorflow as tf
import numpy as np

l2_reg = tf.keras.regularizers.l2(0.05)
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(30, activation="relu",
kernel_initializer="he_normal",
                          kernel_regularizer=l2_reg),
    tf.keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

Tiếp theo, hãy tạo một hàm nhỏ sẽ lấy ngẫu nhiên một batch các thể hiện từ tập huấn luyện (trong Chương 13 chúng ta sẽ thảo luận về API `tf.data`, cái này cung cấp một lựa chọn thay thế tốt hơn nhiều):

```
def random_batch(X, y, batch_size=32):
    idx = np.random.randint(len(X), size=batch_size)
    return X[idx], y[idx]
```

Hãy định nghĩa thêm một hàm sẽ hiển thị trạng thái huấn luyện, bao gồm số bước, tổng số bước, mất mát trung bình kể từ khi bắt đầu epoch (chúng ta sẽ sử dụng số liệu `Mean` để tính toán nó), và các số liệu khác:

```
def print_status_bar(step, total, loss, metrics=None):
    metrics = " - ".join([f"{m.name}: {m.result():.4f}"
                          for m in [loss] + (metrics or [])])
    end = "" if step < total else "\n"
    print(f"\r{step}/{total} - " + metrics, end=end)
```

Đoạn mã này tự giải thích, trừ khi bạn không quen với định dạng chuỗi Python: `{m.result():.4f}` sẽ định dạng kết quả của số liệu dưới dạng số thập phân với bốn chữ số sau dấu thập phân, và việc sử dụng `\r` (dấu quay về đầu dòng) cùng với `end=""` đảm bảo rằng thanh trạng thái luôn được in trên cùng một dòng.

Với điều đó, hãy bắt tay vào việc! Đầu tiên, chúng ta cần định nghĩa một số siêu tham số và chọn bộ tối ưu hóa, hàm mất mát và các số liệu (chỉ MAE trong ví dụ này):

```
# Giả định X_train_scaled và y_train đã được định nghĩa
# Ví dụ:
# X_train_scaled = np.random.rand(1000, 10).astype(np.float32)
# y_train = np.random.rand(1000, 1).astype(np.float32)

n_epochs = 5
batch_size = 32
n_steps = len(X_train_scaled) // batch_size
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
mean_loss = tf.keras.metrics.Mean(name="mean_loss")
metrics = [tf.keras.metrics.MeanAbsoluteError()]
```

Và bây giờ chúng ta đã sẵn sàng xây dựng vòng lặp tùy chỉnh!

```
for epoch in range(1, n_epochs + 1):
    print("Epoch {}/{}".format(epoch, n_epochs))
```

```
    for step in range(1, n_steps + 1):
        X_batch, y_batch = random_batch(X_train_scaled, y_train)
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)

        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))
        mean_loss(loss)

        for metric in metrics:
            metric(y_batch, y_pred)

        print_status_bar(step, n_steps, mean_loss, metrics)

    for metric in [mean_loss] + metrics:
        metric.reset_states()
```

Có rất nhiều điều đang diễn ra trong đoạn mã này, vì vậy hãy cùng xem xét chi tiết:

- Chúng ta tạo hai vòng lặp lồng nhau: một cho các epoch, một cho các batch trong một epoch.
- Sau đó, chúng ta lấy mẫu một batch ngẫu nhiên từ tập huấn luyện.
- Bên trong khối `tf.GradientTape()`, chúng ta đưa ra dự đoán cho một batch, sử dụng mô hình như một hàm, và chúng ta tính toán mất mát: nó bằng tổng mất mát chính cộng với các mất mát khác (trong mô hình này, có một mất mát chính quy hóa cho mỗi lớp). Vì hàm `mean_squared_error()` trả về một mất mát cho mỗi thể hiện, chúng ta tính trung bình trên batch bằng `tf.reduce_mean()` (nếu bạn muốn áp dụng các trọng số khác nhau cho mỗi thể hiện, đây là nơi bạn sẽ làm điều đó). Các mất mát chính quy hóa đã được giảm xuống thành một đại lượng vô hướng mỗi cái, vì vậy chúng ta chỉ cần cộng chúng lại (sử dụng `tf.add_n()`, cái này cộng nhiều tensor có cùng hình dạng và kiểu dữ liệu).
- Tiếp theo, chúng ta yêu cầu tape tính toán gradient của mất mát đối với từng biến có thể huấn luyện — không phải tất cả các biến! — và chúng ta áp dụng chúng cho bộ tối ưu hóa để thực hiện một bước giảm độ dốc.
- Sau đó, chúng ta cập nhật mất mát trung bình và các số liệu (trên epoch hiện tại), và chúng ta hiển thị thanh trạng thái.
- Cuối mỗi epoch, chúng ta đặt lại trạng thái của mất mát trung bình và các số liệu.

Nếu bạn muốn áp dụng cắt gradient (xem Chương 11), hãy đặt siêu tham số `clipnorm` hoặc `clipvalue` của bộ tối ưu hóa. Nếu bạn muốn áp dụng bất kỳ phép biến đổi nào khác cho các gradient, chỉ cần làm như vậy trước khi gọi phương thức `apply_gradients()`. Và nếu bạn muốn thêm ràng buộc trọng số vào mô hình của mình (ví dụ: bằng cách đặt `kernel_constraint` hoặc `bias_constraint` khi tạo một lớp), bạn nên cập nhật vòng lặp huấn luyện để áp dụng các ràng buộc này ngay sau `apply_gradients()`, như sau:

```
for variable in model.variables:
    if variable.constraint is not None:
        variable.assign(variable.constraint(variable))
```

Như bạn có thể thấy, có khá nhiều điều bạn cần làm đúng, và rất dễ mắc lỗi. Nhưng mặt tươi sáng là bạn có toàn quyền kiểm soát.

Bây giờ bạn đã biết cách tùy chỉnh bất kỳ phần nào của mô hình và thuật toán huấn luyện của mình, hãy xem cách bạn có thể sử dụng tính năng tạo biểu đồ tự động của TensorFlow: nó có thể tăng tốc đáng kể mã tùy chỉnh của bạn, và nó cũng sẽ làm cho nó có thể di động sang bất kỳ nền tảng nào được TensorFlow hỗ trợ (xem Chương 19).

Hàm và biểu đồ TensorFlow

Trở lại TensorFlow 1, biểu đồ là không thể tránh khỏi (cũng như sự phức tạp đi kèm với chúng) vì chúng là một phần trung tâm của API của TensorFlow. Kể từ TensorFlow 2 (phát hành năm 2019), biểu đồ vẫn ở đó, nhưng không còn là trung tâm, và chúng đơn giản hơn nhiều (rất nhiều!) để sử dụng. Để cho thấy sự đơn giản như thế nào, hãy bắt đầu với một hàm tầm thường tính toán lập phương của đầu vào của nó:

```
def cube(x):
    return x**3
```

Chúng ta rõ ràng có thể gọi hàm này với một giá trị Python, chẳng hạn như một số nguyên hoặc một số thập phân, hoặc chúng ta có thể gọi nó với một tensor:

```
>>> cube(2)
8

>>> cube(tf.constant(2.0))
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

Bây giờ, hãy sử dụng `tf.function()` để chuyển đổi hàm Python này thành một hàm TensorFlow:

```
>>> tf_cube = tf.function(cube)

>>> tf_cube
<tensorflow.python.eager.def_function.Function at 0x7fbfe0c54d50>
```

Hàm TF này sau đó có thể được sử dụng chính xác như hàm Python gốc, và nó sẽ trả về cùng một kết quả (nhưng luôn dưới dạng tensor):

```
>>> tf_cube(2)
<tf.Tensor: shape=(), dtype=int32, numpy=8>

>>> tf_cube(tf.constant(2.0))
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

Dưới nắp, `tf.function()` đã phân tích các phép tính được thực hiện bởi hàm `cube()` và tạo ra một biểu đồ tính toán tương đương! Như bạn có thể thấy, nó khá dễ dàng (chúng ta sẽ xem xét cách hoạt động này ngay sau đây).

Ngoài ra, chúng ta có thể đã sử dụng `tf.function` làm một decorator; điều này thực ra phổ biến hơn:

```
@tf.function
def tf_cube(x):
    return x**3
```

Hàm Python gốc vẫn có sẵn thông qua thuộc tính `python_function` của hàm TF, trong trường hợp bạn cần nó:

```
>>> tf_cube.python_function(2)
8
```

TensorFlow tối ưu hóa biểu đồ tính toán, cắt bỏ các nút không sử dụng, đơn giản hóa các biểu thức (ví dụ: `1 + 2` sẽ được thay thế bằng `3`), v.v. Khi biểu đồ được tối ưu hóa đã sẵn sàng, hàm TF thực thi hiệu quả các phép toán trong biểu đồ, theo đúng thứ tự (và song song khi có thể). Kết quả là, một hàm TF thường sẽ chạy nhanh hơn nhiều so với hàm Python gốc, đặc biệt nếu nó thực hiện các phép tính phức tạp. Hầu hết thời gian bạn sẽ không thực sự cần biết nhiều hơn thế: khi bạn muốn tăng cường một hàm Python, chỉ cần biến nó thành một hàm TF. Thế thôi!

Hơn nữa, nếu bạn đặt `jit_compile=True` khi gọi `tf.function()`, thì TensorFlow sẽ sử dụng đại số tuyến tính tăng tốc (XLA) để biên dịch các kernel chuyên dụng cho biểu đồ của bạn, thường hợp nhất nhiều phép toán. Ví dụ, nếu hàm TF của bạn gọi `tf.reduce_sum(a * b + c)`, thì không có XLA, hàm trước tiên sẽ cần tính `a * b` và lưu kết quả vào một biến tạm thời, sau đó thêm `c` vào biến đó, và cuối cùng gọi `tf.reduce_sum()` trên kết quả. Với XLA, toàn bộ phép tính được biên dịch thành một kernel duy nhất, cái này sẽ tính `tf.reduce_sum(a * b + c)` trong một lần, mà không sử dụng bất kỳ biến tạm thời lớn nào. Điều này không chỉ nhanh hơn nhiều, mà còn sử dụng ít RAM hơn đáng kể.

Khi bạn viết một hàm mất mát tùy chỉnh, một số liệu tùy chỉnh, một lớp tùy chỉnh hoặc bất kỳ hàm tùy chỉnh nào khác và bạn sử dụng nó trong một mô hình Keras (như chúng ta đã làm trong suốt chương này), Keras tự động chuyển đổi hàm của bạn thành một hàm TF — không cần sử dụng `tf.function()`. Vì vậy, hầu hết thời gian, điều kỳ diệu là hoàn toàn minh bạch. Và nếu bạn muốn Keras sử dụng XLA, bạn chỉ cần đặt `jit_compile=True` khi gọi phương thức `compile()`. Dễ dàng!

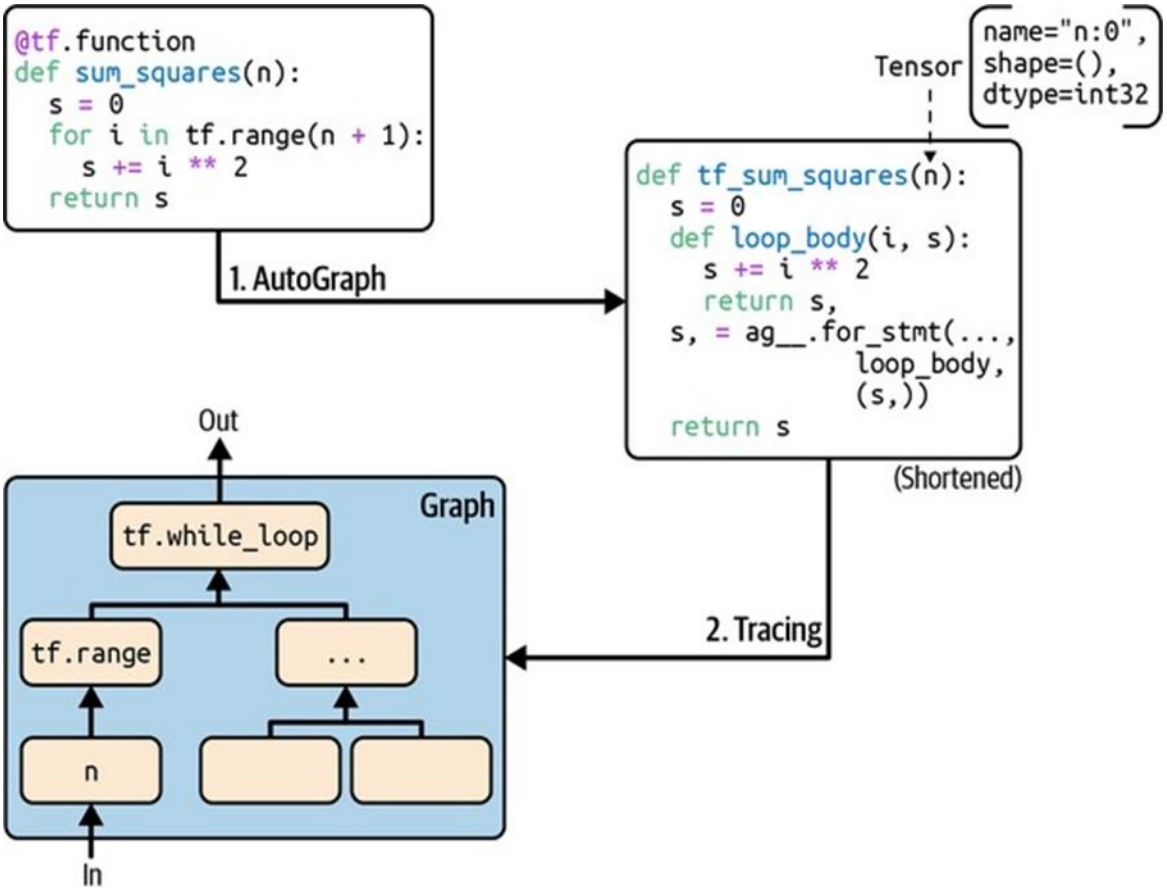
Theo mặc định, một hàm TF tạo một biểu đồ mới cho mỗi tập hợp hình dạng và kiểu dữ liệu đầu vào duy nhất và lưu trữ nó để gọi lại sau này. Ví dụ, nếu bạn gọi `tf_cube(tf.constant(10))`, một biểu đồ sẽ được tạo cho các tensor `int32` có hình dạng `[]`. Sau đó, nếu bạn gọi `tf_cube(tf.constant(20))`, cùng một biểu đồ sẽ được tái sử dụng. Nhưng nếu bạn sau đó gọi `tf_cube(tf.constant([10, 20]))`, một biểu đồ mới sẽ

được tạo cho các tensor `int32` có hình dạng `[2]`. Đây là cách các hàm TF xử lý tính đa hình (tức là các kiểu và hình dạng đối số thay đổi).

Tuy nhiên, điều này chỉ đúng đối với các đối số tensor: nếu bạn truyền các giá trị số Python cho một hàm TF, một biểu đồ mới sẽ được tạo cho mỗi giá trị riêng biệt: ví dụ, gọi `tf_cube(10)` và `tf_cube(20)` sẽ tạo ra hai biểu đồ.

AutoGraph và Tracing

Vậy TensorFlow tạo biểu đồ như thế nào? Nó bắt đầu bằng cách phân tích mã nguồn của hàm Python để nắm bắt tất cả các câu lệnh điều khiển luồng, chẳng hạn như vòng lặp `for`, vòng lặp `while`, và câu lệnh `if`, cũng như các câu lệnh `break`, `continue`, và `return`. Bước đầu tiên này được gọi là **AutoGraph**. Lý do TensorFlow phải phân tích mã nguồn là vì Python không cung cấp bất kỳ cách nào khác để nắm bắt các câu lệnh điều khiển luồng: nó cung cấp các phương thức "magic" như `__add__()` và `__mul__()` để nắm bắt các toán tử như + và *, nhưng không có phương thức magic `__while__()` hoặc `__if__()`. Sau khi phân tích mã của hàm, AutoGraph xuất ra một phiên bản nâng cấp của hàm đó, trong đó tất cả các câu lệnh điều khiển luồng được thay thế bằng các phép toán TensorFlow thích hợp, chẳng hạn như `tf.while_loop()` cho vòng lặp `for` và `tf.cond()` cho câu lệnh `if`. Ví dụ, trong Hình 12-4, AutoGraph phân tích mã nguồn của hàm Python `sum_squares()`, và nó tạo ra hàm `tf_sum_squares()`. Trong hàm này, vòng lặp `for` được thay thế bằng định nghĩa của hàm `loop_body()` (chứa thân của vòng lặp `for` gốc), sau đó là một lệnh gọi đến hàm `for_stmt()`. Lệnh gọi này sẽ xây dựng phép toán `tf.while_loop()` thích hợp trong biểu đồ tính toán.



Hình 12-4. Cách TensorFlow tạo biểu đồ bằng AutoGraph và tracing

Tiếp theo, TensorFlow gọi hàm "nâng cấp" này, nhưng thay vì truyền đối số, nó truyền một **tensor biểu tượng** — một tensor không có bất kỳ giá trị thực nào, chỉ có tên, kiểu dữ liệu và hình dạng. Ví dụ, nếu bạn gọi `sum_squares(tf.constant(10))`, thì hàm `tf_sum_squares()` sẽ được gọi với một tensor biểu tượng kiểu `int32` và hình dạng `[]`. Hàm sẽ chạy ở **chế độ biểu đồ (graph mode)**, nghĩa là mỗi phép toán TensorFlow sẽ thêm một nút vào biểu đồ để đại diện cho chính nó và các tensor đầu ra của nó (trái ngược với chế độ thông thường, được gọi là **thực thi tức thì (eager execution)**, hoặc **chế độ tức thì (eager mode)**). Trong chế độ biểu đồ, các phép toán TF không thực hiện bất kỳ phép tính nào. Chế độ biểu đồ là chế độ mặc định trong TensorFlow 1. Trong Hình 12-4, bạn có thể thấy hàm `tf_sum_squares()` được gọi với một tensor biểu tượng làm đối số (trong trường hợp này, một tensor `int32` có hình dạng `[]`) và biểu đồ cuối cùng được tạo ra trong quá trình tracing. Các nút đại diện cho các phép toán, và các mũi tên đại diện cho các tensor (cả hàm được tạo và biểu đồ đều được đơn giản hóa).

Quy tắc hàm TF

Hầu hết thời gian, việc chuyển đổi một hàm Python thực hiện các phép toán TensorFlow thành một hàm TF rất đơn giản: trang trí nó bằng `@tf.function` hoặc để Keras xử lý cho bạn. Tuy nhiên, có một vài quy tắc cần tuân thủ:

- Nếu bạn gọi bất kỳ thư viện bên ngoài nào, bao gồm NumPy hoặc thậm chí thư viện chuẩn, lệnh gọi này sẽ chỉ chạy trong quá trình tracing; nó sẽ không phải là một phần của biểu đồ. Thật vậy, một biểu đồ TensorFlow chỉ có thể bao gồm các cấu trúc TensorFlow (tensor, phép toán, biến, tập dữ liệu, v.v.). Vì vậy, hãy đảm bảo bạn sử dụng `tf.reduce_sum()` thay vì `np.sum()`, `tf.sort()` thay vì hàm `sorted()` tích hợp sẵn, v.v. (trừ khi bạn thực sự muốn mã chỉ chạy trong quá trình tracing). Điều này có một vài ý nghĩa bổ sung:
    - Nếu bạn định nghĩa một hàm TF `f(x)` chỉ trả về `np.random.rand()`, một số ngẫu nhiên sẽ chỉ được tạo khi hàm được traced, vì vậy `f(tf.constant(2.))` và `f(tf.constant(3.))` sẽ trả về cùng một số ngẫu nhiên, nhưng `f(tf.constant([2., 3.]))` sẽ trả về một số khác. Nếu bạn thay thế `np.random.rand()` bằng `tf.random.uniform([])`, thì một số ngẫu nhiên mới sẽ được tạo ra sau mỗi lần gọi, vì phép toán sẽ là một phần của biểu đồ.
    - Nếu mã không phải TensorFlow của bạn có các tác dụng phụ (chẳng hạn như ghi nhật ký một cái gì đó hoặc cập nhật bộ đếm Python), thì bạn không nên mong đợi những tác dụng phụ đó xảy ra mỗi khi bạn gọi hàm TF, vì chúng sẽ chỉ xảy ra khi hàm được traced.
    - Bạn có thể bọc mã Python tùy ý trong một phép toán `tf.py_function()`, nhưng làm như vậy sẽ cản trở hiệu suất, vì TensorFlow sẽ không thể thực hiện bất kỳ tối ưu hóa biểu đồ nào trên mã này. Nó cũng sẽ làm giảm tính di động, vì biểu đồ sẽ chỉ chạy trên các nền tảng nơi Python có sẵn (và nơi các thư viện phù hợp được cài đặt).
- Bạn có thể gọi các hàm Python hoặc hàm TF khác, nhưng chúng phải tuân theo các quy tắc tương tự, vì TensorFlow sẽ nắm bắt các phép toán của chúng trong biểu đồ tính toán. Lưu ý rằng các hàm khác này không cần phải được trang trí bằng `@tf.function`.
- Nếu hàm tạo một biến TensorFlow (hoặc bất kỳ đối tượng TensorFlow có trạng thái nào khác, chẳng hạn như tập dữ liệu hoặc hàng đợi), nó phải làm như vậy ngay từ lần gọi đầu tiên, và chỉ sau đó, nếu không bạn sẽ gặp ngoại lệ. Thường thì nên tạo biến bên ngoài hàm TF (ví dụ: trong phương thức `build()` của một lớp tùy chỉnh). Nếu bạn muốn gán một giá trị mới cho biến, hãy đảm bảo bạn gọi phương thức `assign()` của nó thay vì sử dụng toán tử `=`.
- Mã nguồn của hàm Python của bạn phải có sẵn cho TensorFlow. Nếu mã nguồn không có sẵn (ví dụ: nếu bạn định nghĩa hàm của mình trong shell Python, cái này không cấp quyền truy cập vào mã nguồn, hoặc nếu bạn chỉ triển khai các tệp Python `.pyc` đã biên dịch vào sản xuất), thì quá trình tạo biểu đồ sẽ thất bại hoặc có chức năng hạn chế.
- TensorFlow sẽ chỉ nắm bắt các vòng lặp `for` lặp lại trên một tensor hoặc một `tf.data.Dataset` (xem Chương 13). Do đó, hãy đảm bảo bạn sử dụng `for i in tf.range(x)` thay vì `for i in range(x)`, nếu không vòng lặp sẽ không được nắm bắt trong biểu đồ. Thay vào đó, nó sẽ chạy trong quá trình tracing. (Đây có thể là điều bạn muốn nếu vòng lặp `for` có ý nghĩa xây dựng biểu đồ; ví dụ: để tạo từng lớp trong mạng nơ-ron.)

- Như mọi khi, vì lý do hiệu suất, bạn nên ưu tiên triển khai vector hóa bất cứ khi nào có thể, thay vì sử dụng vòng lặp.

Đã đến lúc tóm tắt! Trong chương này, chúng ta bắt đầu với tổng quan ngắn gọn về TensorFlow, sau đó chúng ta xem xét API cấp thấp của TensorFlow, bao gồm tensor, phép toán, biến và các cấu trúc dữ liệu đặc biệt. Sau đó, chúng ta sử dụng các công cụ này để tùy chỉnh hầu hết mọi thành phần trong API Keras. Cuối cùng, chúng ta xem xét cách các hàm TF có thể tăng hiệu suất, cách các biểu đồ được tạo bằng AutoGraph và tracing, và các quy tắc cần tuân theo khi bạn viết các hàm TF (nếu bạn muốn mở hộp đen thêm một chút và khám phá các biểu đồ được tạo, bạn sẽ tìm thấy các chi tiết kỹ thuật trong Phụ lục D).

Trong chương tiếp theo, chúng ta sẽ xem xét cách tải và tiền xử lý dữ liệu hiệu quả với TensorFlow.

Bài tập

1. Bạn sẽ mô tả TensorFlow trong một câu ngắn gọn như thế nào? Các tính năng chính của nó là gì? Bạn có thể kể tên các thư viện học sâu phổ biến khác không?
2. TensorFlow có phải là một sự thay thế trực tiếp cho NumPy không? Sự khác biệt chính giữa hai cái là gì?
3. Bạn có nhận được kết quả tương tự với `tf.range(10)` và `tf.constant(np.arange(10))` không?
4. Bạn có thể kể tên sáu cấu trúc dữ liệu khác có sẵn trong TensorFlow, ngoài các tensor thông thường không?
5. Bạn có thể định nghĩa một hàm mất mát tùy chỉnh bằng cách viết một hàm hoặc bằng cách kế thừa từ lớp `tf.keras.losses.Loss`. Bạn sẽ sử dụng mỗi tùy chọn khi nào?
6. Tương tự, bạn có thể định nghĩa một số liệu tùy chỉnh trong một hàm hoặc là một lớp con của `tf.keras.metrics.Metric`. Bạn sẽ sử dụng mỗi tùy chọn khi nào?
7. Khi nào bạn nên tạo một lớp tùy chỉnh so với một mô hình tùy chỉnh?
8. Một số trường hợp sử dụng nào yêu cầu viết vòng lặp huấn luyện tùy chỉnh của riêng bạn?
9. Các thành phần Keras tùy chỉnh có thể chứa mã Python tùy ý, hay chúng phải có thể chuyển đổi sang các hàm TF?
10. Các quy tắc chính cần tuân thủ nếu bạn muốn một hàm có thể chuyển đổi thành hàm TF là gì?
11. Khi nào bạn cần tạo một mô hình Keras động? Bạn làm điều đó như thế nào? Tại sao không làm cho tất cả các mô hình của bạn động?
12. Triển khai một lớp tùy chỉnh thực hiện chuẩn hóa lớp (chúng ta sẽ sử dụng loại lớp này trong Chương 15): a. Phương thức `build()` nên định nghĩa hai trọng số có thể huấn luyện α và β , cả hai đều có hình dạng `input_shape[-1:]` và kiểu dữ liệu `tf.float32`. α nên được khởi tạo bằng 1, và β bằng 0. b. Phương thức `call()` nên tính toán trung bình μ và độ lệch chuẩn σ của các đặc trưng của mỗi thể hiện. Đối

với điều này, bạn có thể sử dụng `tf.nn.moments(inputs, axes=-1, keepdims=True)`, cái này trả về trung bình μ và phương sai σ^2 của tất cả các thể hiện (tính căn bậc hai của phương sai để lấy độ lệch chuẩn). Sau đó, hàm nên tính toán và trả về $\alpha \otimes (X - \mu)/(\sigma + \epsilon) + \beta$, trong đó $\otimes$ đại diện cho phép nhân từng phần tử (*) và ϵ là một số hạng làm mượt (một hằng số nhỏ để tránh chia cho 0, ví dụ: 0.001). c. Đảm bảo rằng lớp tùy chỉnh của bạn tạo ra cùng một đầu ra (hoặc rất gần) như lớp `tf.keras.layers.LayerNormalization`.

13. Huấn luyện một mô hình bằng cách sử dụng vòng lặp huấn luyện tùy chỉnh để giải quyết tập dữ liệu Fashion MNIST (xem Chương 10): a. Hiển thị epoch, lần lặp, mất mát huấn luyện trung bình, và độ chính xác trung bình trên mỗi epoch (được cập nhật ở mỗi lần lặp), cũng như mất mát xác thực và độ chính xác ở cuối mỗi epoch. b. Thử sử dụng một bộ tối ưu hóa khác với tốc độ học khác nhau cho các lớp trên và các lớp dưới.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 13. TẢI VÀ TIỀN XỬ LÝ DỮ LIỆU VỚI TENSORFLOW

Trong Chương 2, bạn đã thấy rằng tải và tiền xử lý dữ liệu là một phần quan trọng của bất kỳ dự án học máy nào. Bạn đã sử dụng Pandas để tải và khám phá tập dữ liệu nhà ở California (đã sửa đổi) — được lưu trữ trong tệp CSV — và bạn đã áp dụng các bộ biến đổi của Scikit-Learn để tiền xử lý. Các công cụ này khá tiện lợi, và bạn có thể sẽ thường xuyên sử dụng chúng, đặc biệt là khi khám phá và thử nghiệm dữ liệu.

Tuy nhiên, khi huấn luyện các mô hình TensorFlow trên các tập dữ liệu lớn, bạn có thể thích sử dụng API tải và tiền xử lý dữ liệu của riêng TensorFlow, được gọi là `tf.data`. Nó có khả năng tải và tiền xử lý dữ liệu cực kỳ hiệu quả, đọc từ nhiều tệp song song bằng đa luồng và hàng đợi, xáo trộn và nhóm mẫu, v.v. Hơn nữa, nó có thể làm tất cả những điều này ngay lập tức — nó tải và tiền xử lý batch dữ liệu tiếp theo trên nhiều lõi CPU, trong khi GPU hoặc TPU của bạn đang bận huấn luyện batch dữ liệu hiện tại.

API `tf.data` cho phép bạn xử lý các tập dữ liệu không vừa trong bộ nhớ, và nó cho phép bạn tận dụng tối đa tài nguyên phần cứng của mình, từ đó tăng tốc quá trình huấn luyện. Ngoài ra, API `tf.data` có thể đọc từ các tệp văn bản (chẳng hạn như tệp CSV), các tệp nhị phân có bản ghi kích thước cố định, và các tệp nhị phân sử dụng định dạng TFRecord của TensorFlow, định dạng này hỗ trợ các bản ghi có kích thước thay đổi. TFRecord là một định dạng nhị phân linh hoạt và hiệu quả thường chứa các bộ đệm giao thức (protocol buffers - một định dạng nhị phân mã nguồn mở). API `tf.data` cũng có hỗ trợ đọc từ các cơ sở dữ liệu SQL. Hơn nữa, nhiều tiện ích mở rộng mã nguồn mở có sẵn để đọc từ tất cả các loại nguồn dữ liệu, chẳng hạn như dịch vụ BigQuery của Google (xem https://tensorflow.org/io).

Keras cũng đi kèm với các lớp tiền xử lý mạnh mẽ nhưng dễ sử dụng có thể được nhúng vào các mô hình của bạn: bằng cách này, khi bạn triển khai một mô hình vào sản xuất, nó sẽ có thể nhận dữ liệu thô trực tiếp, mà không cần bạn phải thêm bất kỳ mã tiền xử lý bổ sung nào. Điều này loại bỏ nguy cơ không khớp giữa mã tiền xử lý được sử dụng trong quá trình huấn luyện và mã tiền xử lý được sử dụng trong sản xuất, điều này có thể gây ra sai lệch huấn luyện/phục vụ. Và nếu bạn triển khai mô hình của mình trong nhiều ứng dụng được mã hóa bằng các ngôn ngữ lập trình khác nhau, bạn sẽ không phải triển khai lại cùng một mã tiền xử lý nhiều lần, điều này cũng giảm nguy cơ không khớp.

Như bạn sẽ thấy, cả hai API có thể được sử dụng cùng nhau — ví dụ, để hưởng lợi từ việc tải dữ liệu hiệu quả do `tf.data` cung cấp và sự tiện lợi của các lớp tiền xử lý Keras.

Trong chương này, chúng ta sẽ đề cập đến API `tf.data` và định dạng TFRecord trước. Sau đó, chúng ta sẽ khám phá các lớp tiền xử lý Keras và cách sử dụng chúng với API `tf.data`. Cuối cùng, chúng ta sẽ xem xét nhanh một vài thư viện liên quan mà bạn có thể thấy hữu ích để tải và tiền xử lý dữ liệu, chẳng hạn như TensorFlow Datasets và TensorFlow Hub. Vậy, hãy bắt đầu!

API `tf.data`

Toàn bộ API `tf.data` xoay quanh khái niệm về một `tf.data.Dataset`: điều này đại diện cho một chuỗi các mục dữ liệu. Thông thường bạn sẽ sử dụng các tập dữ liệu đọc dần dữ liệu từ đĩa, nhưng để đơn giản, hãy tạo một tập dữ liệu từ một tensor dữ liệu đơn giản bằng cách sử dụng `tf.data.Dataset.from_tensor_slices()`:

```
>>> import tensorflow as tf

>>> X = tf.range(10) # bất kỳ tensor dữ liệu nào

>>> dataset = tf.data.Dataset.from_tensor_slices(X)

>>> dataset
<TensorSliceDataset shapes:(), types: tf.int32>
```

Hàm `from_tensor_slices()` nhận một tensor và tạo một `tf.data.Dataset` mà các phần tử của nó là tất cả các lát cắt của `X` dọc theo chiều đầu tiên, vì vậy tập dữ liệu này chứa 10 mục: các tensor 0, 1, 2, ..., 9. Trong trường hợp này, chúng ta sẽ thu được cùng tập dữ liệu nếu chúng ta đã sử dụng `tf.data.Dataset.range(10)` (ngoại trừ các phần tử sẽ là số nguyên 64-bit thay vì số nguyên 32-bit).

Bạn có thể đơn giản lặp qua các mục của một tập dữ liệu như thế này:

```
>>> for item in dataset:
...     print(item)
...
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
[...]
tf.Tensor(9, shape=(), dtype=int32)
```

Một tập dữ liệu cũng có thể chứa các bộ (tuple) tensor, hoặc các từ điển cặp tên/tensor, hoặc thậm chí các bộ và từ điển lồng nhau của các tensor. Khi lát cắt một bộ, một từ điển, hoặc một cấu trúc lồng nhau, tập dữ liệu sẽ chỉ lát cắt các tensor mà nó chứa, trong khi vẫn giữ nguyên cấu trúc bộ/từ điển. Ví dụ:

```
>>> X_nested = {"a": ([1, 2, 3], [4, 5, 6]), "b": [7, 8, 9]}

>>> dataset = tf.data.Dataset.from_tensor_slices(X_nested)

>>> for item in dataset:
...     print(item)
...
{'a': (<tf.Tensor: [...]=1>, <tf.Tensor: [...]=4>), 'b':
<tf.Tensor: [...]=7>}
{'a': (<tf.Tensor: [...]=2>, <tf.Tensor: [...]=5>), 'b':
<tf.Tensor: [...]=8>}
```

```
{'a': (<tf.Tensor: [...]=3>, <tf.Tensor: [...]=6>), 'b':
<tf.Tensor: [...]=9>}
```

Xếp chuỗi các phép biến đổi

Khi bạn có một tập dữ liệu, bạn có thể áp dụng tất cả các loại phép biến đổi cho nó bằng cách gọi các phương thức biến đổi của nó. Mỗi phương thức trả về một tập dữ liệu mới, vì vậy bạn có thể xếp chuỗi các phép biến đổi như thế này (chuỗi này được minh họa trong Hình 13-1):

```
>>> dataset = tf.data.Dataset.from_tensor_slices(tf.range(10))

>>> dataset = dataset.repeat(3).batch(7)

>>> for item in dataset:
...     print(item)
...

tf.Tensor([0 1 2 3 4 5 6], shape=(7,), dtype=int32)

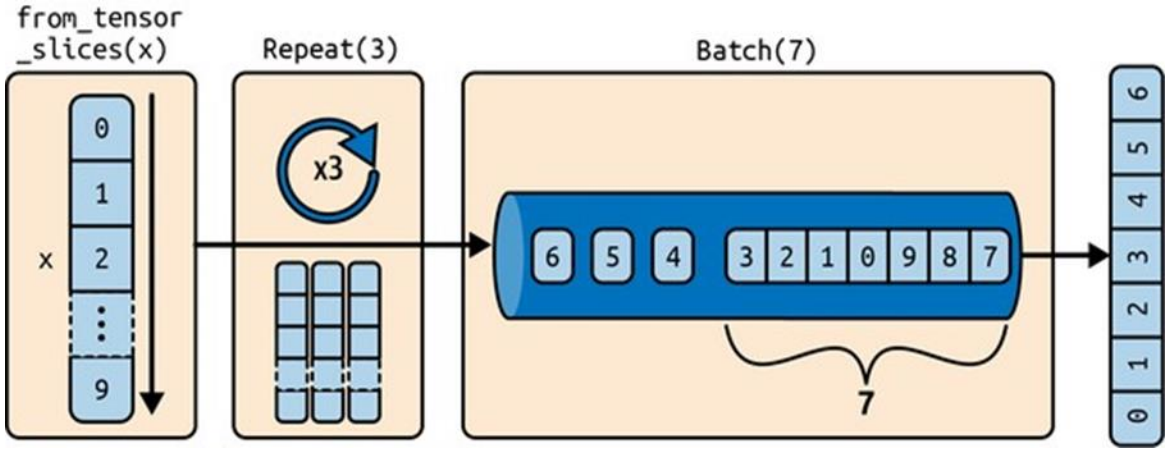
tf.Tensor([7 8 9 0 1 2 3], shape=(7,), dtype=int32)

tf.Tensor([4 5 6 7 8 9 0], shape=(7,), dtype=int32)

tf.Tensor([1 2 3 4 5 6 7], shape=(7,), dtype=int32)
tf.Tensor([8 9], shape=(2,), dtype=int32)
```

Trong ví dụ này, chúng ta đầu tiên gọi phương thức `repeat()` trên tập dữ liệu gốc, và nó trả về một tập dữ liệu mới lặp lại các mục của tập dữ liệu gốc ba lần. Tất nhiên, điều này sẽ không sao chép tất cả dữ liệu vào bộ nhớ ba lần! Nếu bạn gọi phương thức này mà không có đối số, tập dữ liệu mới sẽ lặp lại tập dữ liệu nguồn mãi mãi, vì vậy mã lặp qua tập dữ liệu sẽ phải quyết định khi nào dừng lại.

Sau đó, chúng ta gọi phương thức `batch()` trên tập dữ liệu mới này, và một lần nữa điều này tạo ra một tập dữ liệu mới. Cái này sẽ nhóm các mục của tập dữ liệu trước đó thành các batch gồm bảy mục.



Hình 13-1. Xếp chuỗi các phép biến đổi tập dữ liệu

Cuối cùng, chúng ta lặp qua các mục của tập dữ liệu cuối cùng này. Phương thức `batch()` phải xuất ra một batch cuối cùng có kích thước hai thay vì bảy, nhưng bạn có thể gọi `batch()` với `drop_remainder=True` nếu bạn muốn nó bỏ batch cuối cùng này, sao cho tất cả các batch có cùng kích thước chính xác.

Bạn cũng có thể biến đổi các mục bằng cách gọi phương thức `map()`. Ví dụ, đoạn mã này tạo một tập dữ liệu mới với tất cả các batch được nhân với hai:

```
>>> dataset = dataset.map(lambda x: x * 2) # x là một batch

>>> for item in dataset:
...     print(item)
...
tf.Tensor([ 0  2  4  6  8 10 12], shape=(7,), dtype=int32)
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
[...]
```

Phương thức `map()` này là phương thức bạn sẽ gọi để áp dụng bất kỳ tiền xử lý nào cho dữ liệu của mình. Đôi khi điều này sẽ bao gồm các phép tính có thể khá chuyên sâu, chẳng hạn như định hình lại hoặc xoay hình ảnh, vì vậy bạn thường sẽ muốn tạo nhiều luồng để tăng tốc mọi thứ. Điều này có thể được thực hiện bằng cách đặt đối số `num_parallel_calls` thành số luồng để chạy, hoặc thành `tf.data.AUTOTUNE`. Lưu ý rằng hàm bạn truyền cho phương thức `map()` phải có thể chuyển đổi thành hàm TF (xem Chương 12).

Cũng có thể chỉ đơn giản là lọc tập dữ liệu bằng cách sử dụng phương thức `filter()`. Ví dụ, đoạn mã này tạo một tập dữ liệu chỉ chứa các batch có tổng lớn hơn 50:

```
>>> dataset = dataset.filter(lambda x: tf.reduce_sum(x) > 50)

>>> for item in dataset:
...     print(item)
...
```

```
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)

tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)

tf.Tensor([ 2  4  6  8 10 12 14], shape=(7,), dtype=int32)
```

Bạn thường sẽ muốn xem xét chỉ một vài mục từ một tập dữ liệu. Bạn có thể sử dụng phương thức `take()` cho điều đó:

```
>>> for item in dataset.take(2):
...     print(item)
...
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)
```

Xáo trộn dữ liệu (Shuffling the Data)

Như chúng ta đã thảo luận trong Chương 4, giảm độ dốc hoạt động tốt nhất khi các thể hiện trong tập huấn luyện độc lập và phân phối giống hệt nhau (IID). Một cách đơn giản để đảm bảo điều này là xáo trộn các thể hiện, bằng cách sử dụng phương thức `shuffle()`. Nó sẽ tạo ra một tập dữ liệu mới bắt đầu bằng cách điền vào một bộ đệm với các mục đầu tiên của tập dữ liệu nguồn. Sau đó, bất cứ khi nào nó được yêu cầu một mục, nó sẽ kéo ngẫu nhiên một mục ra khỏi bộ đệm và thay thế nó bằng một mục mới từ tập dữ liệu nguồn, cho đến khi nó đã lặp hoàn toàn qua tập dữ liệu nguồn. Tại thời điểm này, nó sẽ tiếp tục kéo các mục ngẫu nhiên ra khỏi bộ đệm cho đến khi bộ đệm trống. Bạn phải chỉ định kích thước bộ đệm, và điều quan trọng là phải làm cho nó đủ lớn, nếu không việc xáo trộn sẽ không hiệu quả lắm. Chỉ cần không vượt quá lượng RAM bạn có, mặc dù ngay cả khi bạn có nhiều RAM, không cần vượt quá kích thước của tập dữ liệu. Bạn có thể cung cấp một hạt giống ngẫu nhiên nếu bạn muốn cùng một thứ tự ngẫu nhiên mỗi khi bạn chạy chương trình của mình. Ví dụ, đoạn mã sau tạo và hiển thị một tập dữ liệu chứa các số nguyên từ 0 đến 9, lặp lại hai lần, được xáo trộn bằng cách sử dụng bộ đệm có kích thước 4 và hạt giống ngẫu nhiên 42, và được nhóm với kích thước batch là 7:

```
>>> dataset = tf.data.Dataset.range(10).repeat(2)

>>> dataset = dataset.shuffle(buffer_size=4, seed=42).batch(7)

>>> for item in dataset:

...     print(item)

...

tf.Tensor([3 0 1 6 2 5 7], shape=(7,), dtype=int64)

tf.Tensor([8 4 1 9 4 2 3], shape=(7,), dtype=int64)
```

```
tf.Tensor([7 5 0 8 9 6], shape=(6,), dtype=int64)
```

Đối với một tập dữ liệu lớn không vừa trong bộ nhớ, cách tiếp cận bộ đệm xáo trộn đơn giản này có thể không đủ, vì bộ đệm sẽ nhỏ so với tập dữ liệu. Một giải pháp là xáo trộn dữ liệu nguồn đó (ví dụ, trên Linux bạn có thể xáo trộn các tệp văn bản bằng lệnh `shuf`). Điều này chắc chắn sẽ cải thiện việc xáo trộn rất nhiều! Ngay cả khi dữ liệu nguồn được xáo trộn, bạn thường sẽ muốn xáo trộn nó nhiều hơn, nếu không thứ tự tương tự sẽ được lặp lại ở mỗi epoch, và mô hình có thể bị sai lệch (ví dụ: do một số mẫu giả có mặt ngẫu nhiên trong thứ tự dữ liệu nguồn). Để xáo trộn các thể hiện nhiều hơn, một cách tiếp cận phổ biến là chia dữ liệu nguồn thành nhiều tệp, sau đó đọc chúng theo thứ tự ngẫu nhiên trong quá trình huấn luyện. Tuy nhiên, các thể hiện nằm trong cùng một tệp vẫn sẽ kết thúc gần nhau. Để tránh điều này, bạn có thể chọn nhiều tệp ngẫu nhiên và đọc chúng đồng thời, xen kẽ các bản ghi của chúng. Sau đó, trên hết bạn có thể thêm một bộ đệm xáo trộn bằng phương thức

`shuffle()`. Nếu điều này nghe có vẻ là rất nhiều công việc, đừng lo lắng: API

`tf.data` làm cho tất cả điều này có thể chỉ trong vài dòng mã. Hãy cùng xem cách bạn có thể làm điều này.

Xen kẽ các dòng từ nhiều tệp (Interleaving Lines from Multiple Files)

Đầu tiên, giả sử bạn đã tải tập dữ liệu nhà ở California, xáo trộn nó (trừ khi nó đã được xáo trộn), và chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra. Sau đó, bạn chia mỗi tập thành nhiều tệp CSV, mỗi tệp trông như thế này (mỗi hàng chứa tám đặc trưng đầu vào cộng với giá trị nhà trung bình mục tiêu):

```
MedInc,HouseAge,AveRooms,AveBedrms,Popul…,AveOccup,Lat…,Long…,Media
nHouseValue
3.5214,15.0,3.050,1.107,1447.0,1.606,37.63,-122.43,1.442
5.3275,5.0,6.490,0.991,3464.0,3.443,33.69,-117.39,1.687
3.1,29.0,7.542,1.592,1328.0,2.251,38.44,-122.98,1.621
[...]
```

Giả sử `train_filepaths` chứa danh sách các đường dẫn tệp huấn luyện (và bạn cũng có `valid_filepaths` và `test_filepaths`):

```
>>> train_filepaths
['datasets/housing/my_train_00.csv','datasets/housing/my_train_01.csv', ...]
```

Ngoài ra, bạn có thể sử dụng các mẫu tệp; ví dụ,

`train_filepaths = "datasets/housing/my_train_*.csv"`. Bây giờ hãy tạo một tập dữ liệu chỉ chứa các đường dẫn tệp này:

```
filepath_dataset = tf.data.Dataset.list_files(train_filepaths,seed=42)
```

Theo mặc định, hàm

`list_files()` trả về một tập dữ liệu xáo trộn các đường dẫn tệp. Nói chung đây là một điều tốt, nhưng bạn có thể đặt

`shuffle=False` nếu bạn không muốn điều đó vì một lý do nào đó.

Tiếp theo, bạn có thể gọi phương thức

`interleave()` để đọc từ năm tệp cùng một lúc và xen kẽ các dòng của chúng. Bạn cũng có thể bỏ qua dòng đầu tiên của mỗi tệp — đó là hàng tiêu đề — bằng phương thức

`skip()`:

```
n_readers = 5
dataset = filepath_dataset.interleave(
    lambda filepath:tf.data.TextLineDataset(filepath).skip(1),
cycle_length=n_readers)
```

Phương thức

`interleave()` sẽ tạo một tập dữ liệu sẽ lấy năm đường dẫn tệp từ `filepath_dataset`, và đối với mỗi đường dẫn, nó sẽ gọi hàm bạn đã cung cấp cho nó (một lambda trong ví dụ này) để tạo một tập dữ liệu mới (trong trường hợp này là `TextLineDataset`). Để rõ ràng, ở giai đoạn này sẽ có tổng cộng bảy tập dữ liệu: tập dữ liệu đường dẫn tệp, tập dữ liệu xen kẽ và năm `TextLineDataset` được tạo nội bộ bởi tập dữ liệu xen kẽ. Khi bạn lặp qua tập dữ liệu xen kẽ, nó sẽ luân phiên qua năm `TextLineDataset` này, đọc từng dòng một từ mỗi tập dữ liệu cho đến khi tất cả các tập dữ liệu hết mục. Sau đó, nó sẽ lấy năm đường dẫn tệp tiếp theo từ `filepath_dataset` và xen kẽ chúng theo cùng một cách, v.v. cho đến khi hết đường dẫn tệp. Để việc xen kẽ hoạt động tốt nhất, nên có các tệp có độ dài giống hệt nhau; nếu không, phần cuối của tệp dài nhất sẽ không được xen kẽ.

Theo mặc định, `interleave()` không sử dụng song song hóa; nó chỉ đọc từng dòng một từ mỗi tệp, tuần tự. Nếu bạn muốn nó thực sự đọc tệp song song, bạn có thể đặt đối số `num_parallel_calls` của phương thức `interleave()` thành số luồng bạn muốn (nhớ rằng phương thức `map()` cũng có đối số này). Bạn thậm chí có thể đặt nó thành `tf.data.AUTOTUNE` để TensorFlow chọn số luồng phù hợp một cách linh hoạt dựa trên CPU có sẵn. Hãy xem tập dữ liệu hiện chứa gì:

```
>>> for line in dataset.take(5):

...     print(line)

...
tf.Tensor(b'4.5909,16.0,[...],33.63,-117.71,2.418', shape=(),
dtype=string)
tf.Tensor(b'2.4792,24.0,[...],34.18,-118.38,2.0', shape=(),
dtype=string)
tf.Tensor(b'4.2708,45.0,[...],37.48,-122.19,2.67', shape=(),
dtype=string)
tf.Tensor(b'2.1856,41.0,[...],32.76,-117.12,1.205', shape=(),
```

```
dtype=string)
tf.Tensor(b'4.1812,52.0,[...],33.73,-118.31,3.215', shape=(),
dtype=string)
```

Đây là năm hàng đầu tiên (bỏ qua hàng tiêu đề) của năm tệp CSV, được chọn ngẫu nhiên. Trông tốt!

Tiền xử lý dữ liệu (Preprocessing the Data)

Bây giờ chúng ta có một tập dữ liệu nhà ở trả về mỗi thể hiện dưới dạng một tensor chứa một chuỗi byte, chúng ta cần thực hiện một chút tiền xử lý, bao gồm phân tích cú pháp các chuỗi và chia tỷ lệ dữ liệu.

Hãy triển khai một vài hàm tùy chỉnh sẽ thực hiện tiền xử lý này:

```
# Giả định X_mean và X_std đã được định nghĩa là tensor hoặc mảng NumPy
# Ví dụ:
# X_mean = tf.constant([1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0],
dtype=tf.float32)
# X_std = tf.constant([0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2],
dtype=tf.float32)

X_mean, X_std = [...] # trung bình và tỷ lệ của mỗi đặc trưng trong tập huấn
luyện
n_inputs = 8 [cite: 1]

def parse_csv_line(line):
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    fields = tf.io.decode_csv(line, record_defaults=defs)
    return tf.stack(fields[:-1]), tf.stack(fields[-1:])

def preprocess(line):
    x, y = parse_csv_line(line)
    return (x - X_mean) / X_std, y
```

Hãy cùng xem đoạn mã này:

- Đầu tiên, mã giả định rằng chúng ta đã tính toán trước trung bình và độ lệch chuẩn của mỗi đặc trưng trong tập huấn luyện. `X_mean` và `X_std` chỉ là các tensor 1D (hoặc mảng NumPy) chứa tám số thập phân, một cho mỗi đặc trưng đầu vào. Điều này có thể được thực hiện bằng cách sử dụng `StandardScaler` của Scikit-Learn trên một mẫu ngẫu nhiên đủ lớn của tập dữ liệu. Sau này trong chương này, chúng ta sẽ sử dụng một lớp tiền xử lý Keras thay thế.
- Hàm `parse_csv_line()` nhận một dòng CSV và phân tích cú pháp nó. Để giúp việc đó, nó sử dụng hàm `tf.io.decode_csv()`, hàm này nhận hai đối số: đối số đầu tiên là dòng cần phân tích cú pháp, và đối số thứ hai là một mảng chứa giá trị mặc định cho mỗi cột trong tệp CSV. Mảng này (`defs`) không chỉ cho TensorFlow biết giá trị mặc định cho mỗi cột, mà còn cả số lượng cột và kiểu của chúng. Trong ví dụ này,

chúng ta bảo nó rằng tất cả các cột đặc trưng đều là số thập phân và các giá trị bị thiếu nên mặc định là 0, nhưng chúng ta cung cấp một mảng trống kiểu `tf.float32` làm giá trị mặc định cho cột cuối cùng (mục tiêu): mảng này cho TensorFlow biết rằng cột này chứa số thập phân, nhưng không có giá trị mặc định, vì vậy nó sẽ đưa ra một ngoại lệ nếu nó gặp một giá trị bị thiếu.

- Hàm `tf.io.decode_csv()` trả về một danh sách các tensor vô hướng (một cho mỗi cột), nhưng chúng ta cần trả về một mảng tensor 1D. Vì vậy, chúng ta gọi `tf.stack()` trên tất cả các tensor ngoại trừ tensor cuối cùng (mục tiêu): điều này sẽ xếp chồng các tensor này thành một mảng 1D. Sau đó, chúng ta làm tương tự cho giá trị mục tiêu: điều này làm cho nó trở thành một mảng tensor 1D với một giá trị duy nhất, thay vì một tensor vô hướng. Hàm `tf.io.decode_csv()` đã hoàn thành, vì vậy nó trả về các đặc trưng đầu vào và mục tiêu.
- Cuối cùng, hàm `preprocess()` tùy chỉnh chỉ gọi hàm `parse_csv_line()`, chia tỷ lệ các đặc trưng đầu vào bằng cách trừ trung bình đặc trưng và sau đó chia cho độ lệch chuẩn đặc trưng, và trả về một bộ (tuple) chứa các đặc trưng đã chia tỷ lệ và mục tiêu.

Hãy kiểm tra hàm tiền xử lý này:

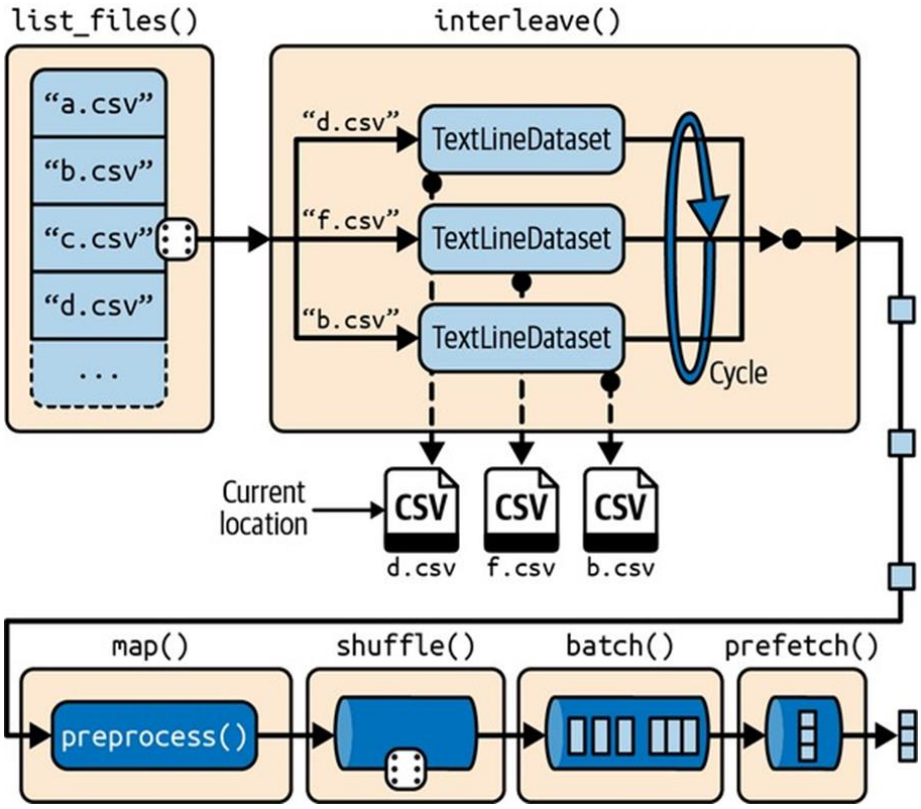
```
>>> preprocess(b'4.2083,44.0,5.3232,0.9171,846.0,2.3370,37.47,-122.2,2.782')
(<tf.Tensor: shape=(8,), dtype=float32, numpy=
array([ 0.16579159,  1.216324  , -0.05204564, -0.39215982,
       -0.5277444 , -0.2633488 ,  0.8543046 , -1.3072058 ], dtype=float32)>,
 <tf.Tensor: shape=(1,), dtype=float32, numpy=array([2.782], dtype=float32)>)
```

Trông tốt! Hàm `preprocess()` có thể chuyển đổi một thể hiện từ một chuỗi byte thành một tensor được chia tỷ lệ đẹp mắt, với nhãn tương ứng của nó. Bây giờ chúng ta có thể sử dụng phương thức `map()` của tập dữ liệu để áp dụng hàm `preprocess()` cho mỗi mẫu trong tập dữ liệu.

Để mã dễ tái sử dụng hơn, hãy gom tất cả những gì chúng ta đã thảo luận cho đến nay vào một hàm trợ giúp khác; nó sẽ tạo và trả về một tập dữ liệu sẽ tải hiệu quả dữ liệu nhà ở California từ nhiều tệp CSV, tiền xử lý, xáo trộn và nhóm nó (xem Hình 13-2):

```
def csv_reader_dataset(filepaths, n_readers=5, n_read_threads=None,
                       n_parse_threads=5, shuffle_buffer_size=10_000,
seed=42,
                       batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths, seed=seed)
    dataset = dataset.interleave(
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers, num_parallel_calls=n_read_threads)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
    dataset = dataset.shuffle(shuffle_buffer_size, seed=seed)
    return dataset.batch(batch_size).prefetch(1)
```

Lưu ý rằng chúng ta sử dụng phương thức `prefetch()` ở dòng cuối cùng. Điều này quan trọng đối với hiệu suất, như bạn sẽ thấy ngay bây giờ.

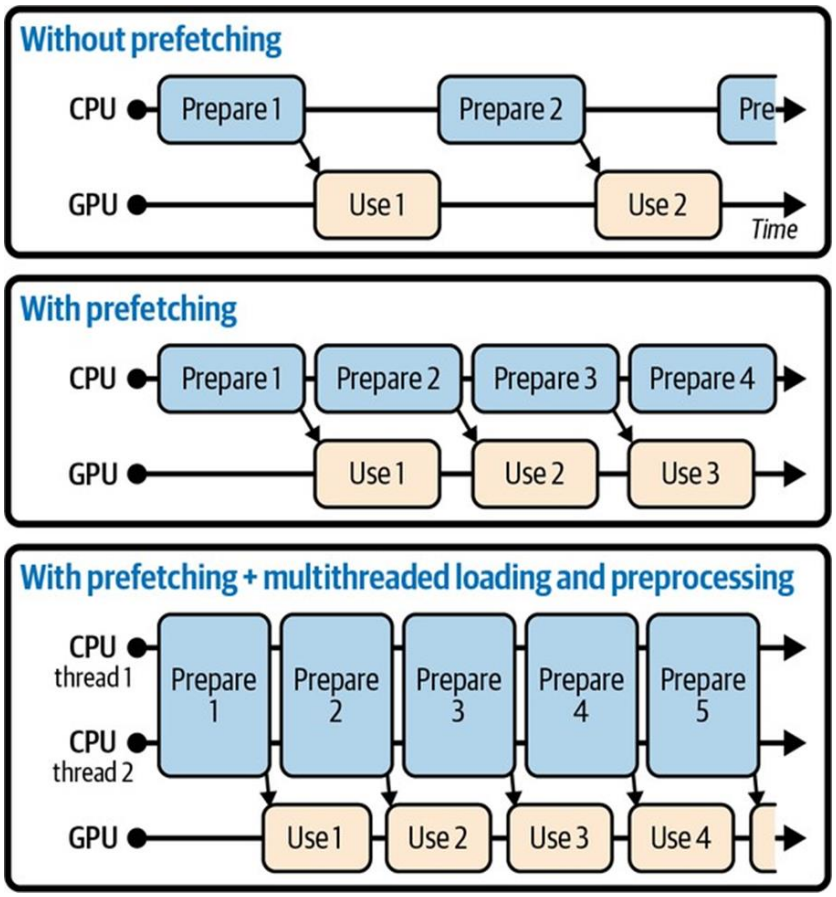


Hình 13-2. Tải và tiền xử lý dữ liệu từ nhiều tệp CSV

Prefetching

Bằng cách gọi `prefetch(1)` ở cuối hàm `csv_reader_dataset()` tùy chỉnh, chúng ta đang tạo một tập dữ liệu sẽ cố gắng hết sức để luôn đi trước một batch. Nói cách khác, trong khi thuật toán huấn luyện của chúng ta đang làm việc trên một batch, tập dữ liệu sẽ đã làm việc song song để chuẩn bị batch tiếp theo (ví dụ: đọc dữ liệu từ đĩa và tiền xử lý nó). Điều này có thể cải thiện hiệu suất đáng kể, như được minh họa trong Hình 13-3.

Nếu chúng ta cũng đảm bảo rằng việc tải và tiền xử lý được đa luồng (bằng cách đặt `num_parallel_calls` khi gọi `interleave()` và `map()`), chúng ta có thể khai thác nhiều lõi CPU và hy vọng làm cho việc chuẩn bị một batch dữ liệu ngắn hơn so với chạy một bước huấn luyện trên GPU: bằng cách này, GPU sẽ được sử dụng gần như 100% (ngoại trừ thời gian truyền dữ liệu từ CPU sang GPU), và quá trình huấn luyện sẽ chạy nhanh hơn nhiều.



Hình 13-3. Với prefetching, CPU và GPU hoạt động song song: khi GPU hoạt động trên một batch, CPU hoạt động trên batch tiếp theo

Nếu tập dữ liệu đủ nhỏ để vừa trong bộ nhớ, bạn có thể tăng tốc đáng kể quá trình huấn luyện bằng cách sử dụng phương thức `cache()` của tập dữ liệu để lưu nội dung của nó vào RAM. Bạn nên thực hiện điều này sau khi tải và tiền xử lý dữ liệu, nhưng trước khi xáo trộn, lặp lại, nhóm batch và prefetching. Bằng cách này, mỗi thể hiện sẽ chỉ được đọc và tiền xử lý một lần (thay vì một lần mỗi epoch), nhưng dữ liệu vẫn sẽ được xáo trộn khác nhau ở mỗi epoch, và batch tiếp theo vẫn sẽ được chuẩn bị trước.

Bây giờ bạn đã học cách xây dựng các pipeline đầu vào hiệu quả để tải và tiền xử lý dữ liệu từ nhiều tệp văn bản. Chúng ta đã thảo luận về các phương thức tập dữ liệu phổ biến nhất, nhưng còn một vài phương thức khác mà bạn có thể muốn xem xét, chẳng hạn như `concatenate()`, `zip()`, `window()`, `reduce()`, `shard()`, `flat_map()`, `apply()`, `unbatch()`, và `padded_batch()`. Cũng có một vài phương thức lớp khác, chẳng hạn như `from_generator()` và `from_tensors()`, cái này tạo một tập dữ liệu mới từ một trình tạo Python hoặc một danh sách các tensor, tương ứng. Vui lòng kiểm tra tài liệu API để biết thêm chi tiết. Cũng lưu ý rằng có các tính năng thử nghiệm có sẵn trong `tf.data.experimental`, nhiều tính năng trong số đó có thể sẽ được đưa vào API cốt lõi trong các bản phát hành trong tương lai (ví dụ, hãy xem lớp `CsvDataset`, cũng như phương thức `make_csv_dataset()`, cái này đảm nhiệm việc suy luận kiểu của mỗi cột).

Sử dụng Dataset với Keras

Bây giờ chúng ta có thể sử dụng hàm `csv_reader_dataset()` đã viết trước đó để tạo một tập dữ liệu cho tập huấn luyện, tập kiểm định (validation) và tập kiểm tra (test). Tập huấn luyện sẽ được xáo trộn ở mỗi epoch (lưu ý rằng tập kiểm định và tập kiểm tra cũng sẽ được xáo trộn, mặc dù chúng ta không thực sự cần điều đó):

```
train_set = csv_reader_dataset(train_filepaths)
valid_set = csv_reader_dataset(valid_filepaths)
test_set = csv_reader_dataset(test_filepaths)
```

Bây giờ bạn có thể chỉ cần xây dựng và huấn luyện một mô hình Keras bằng các tập dữ liệu này. Khi bạn gọi phương thức `fit()` của mô hình, bạn truyền `train_set` thay vì `X_train`, `y_train`, và truyền `validation_data=valid_set` thay vì `validation_data=(X_valid, y_valid)`. Phương thức `fit()` sẽ tự động lặp lại tập dữ liệu huấn luyện một lần mỗi epoch, sử dụng một thứ tự ngẫu nhiên khác nhau ở mỗi epoch:

```
model = tf.keras.Sequential([...])
model.compile(loss="mse", optimizer="sgd")
model.fit(train_set, validation_data=valid_set, epochs=5)
```

Tương tự, bạn có thể truyền một tập dữ liệu cho các phương thức `evaluate()` và `predict()`:

```
test_mse = model.evaluate(test_set)
new_set = test_set.take(3) # giả sử chúng ta có 3 mẫu mới
y_pred = model.predict(new_set) # hoặc bạn có thể chỉ cần truyền một mảng
NumPy
```

Không giống như các tập khác, `new_set` thường sẽ không chứa nhãn. Nếu có, như trong trường hợp này, Keras sẽ bỏ qua chúng. Lưu ý rằng trong tất cả các trường hợp này, bạn vẫn có thể sử dụng mảng NumPy thay vì tập dữ liệu nếu bạn thích (nhưng tất nhiên chúng cần được tải và tiền xử lý trước).

Nếu bạn muốn xây dựng vòng lặp huấn luyện tùy chỉnh của riêng mình (như đã thảo luận trong Chương 12), bạn có thể chỉ cần lặp qua tập huấn luyện một cách rất tự nhiên:

```
n_epochs = 5
for epoch in range(n_epochs):
    for X_batch, y_batch in train_set:
        [...] # thực hiện một bước hạ gradient
```

Thực tế, thậm chí có thể tạo một hàm TF (xem Chương 12) để huấn luyện mô hình trong cả một epoch. Điều này thực sự có thể tăng tốc độ huấn luyện:

```
@tf.function
def train_one_epoch(model, optimizer, loss_fn, train_set):
    for X_batch, y_batch in train_set:
        with tf.GradientTape() as tape:
            y_pred = model(X_batch)
```

```
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
for epoch in range(n_epochs):
    print("\rEpoch {}/{}".format(epoch + 1, n_epochs), end="")
    train_one_epoch(model, optimizer, loss_fn, train_set)
```

Trong Keras, đối số `steps_per_execution` của phương thức `compile()` cho phép bạn xác định số lượng lô mà phương thức `fit()` sẽ xử lý trong mỗi lần gọi đến hàm `tf.function` mà nó sử dụng để huấn luyện. Giá trị mặc định chỉ là 1, vì vậy nếu bạn đặt nó thành 50, bạn sẽ thường thấy một sự cải thiện hiệu suất đáng kể. Tuy nhiên, các phương thức `on_batch_*()` của các callback Keras sẽ chỉ được gọi sau mỗi 50 lô.

Xin chúc mừng, bây giờ bạn đã biết cách xây dựng các đường ống đầu vào mạnh mẽ bằng cách sử dụng API `tf.data`! Tuy nhiên, cho đến nay chúng ta đã sử dụng các tệp CSV, vốn phổ biến, đơn giản và tiện lợi nhưng không thực sự hiệu quả, và không hỗ trợ tốt các cấu trúc dữ liệu lớn hoặc phức tạp (như hình ảnh hoặc âm thanh). Vì vậy, hãy xem cách sử dụng TFRecords thay thế.

Định dạng TFRecord

Định dạng **TFRecord** là định dạng ưa thích của TensorFlow để lưu trữ lượng lớn dữ liệu và đọc nó một cách hiệu quả. Đó là một định dạng nhị phân rất đơn giản chỉ chứa một chuỗi các bản ghi nhị phân có kích thước thay đổi (mỗi bản ghi bao gồm độ dài, tổng kiểm CRC để kiểm tra xem độ dài có bị hỏng không, sau đó là dữ liệu thực tế và cuối cùng là tổng kiểm CRC cho dữ liệu). Bạn có thể dễ dàng tạo một tệp TFRecord bằng cách sử dụng lớp `tf.io.TFRecordWriter`:

```
with tf.io.TFRecordWriter("my_data.tfrecord") as f:
    f.write(b"This is the first record")
    f.write(b"And this is the second record")
```

Và sau đó bạn có thể sử dụng `tf.data.TFRecordDataset` để đọc một hoặc nhiều tệp TFRecord:

```
filepaths = ["my_data.tfrecord"]
dataset = tf.data.TFRecordDataset(filepaths)
for item in dataset:
    print(item)
```

Điều này sẽ xuất ra:

```
tf.Tensor(b'This is the first record', shape=(), dtype=string)
tf.Tensor(b'And this is the second record', shape=(), dtype=string)
```

Tệp TFRecord được nén

Đôi khi có thể hữu ích khi nén các tệp TFRecord của bạn, đặc biệt nếu chúng cần được tải qua kết nối mạng. Bạn có thể tạo một tệp TFRecord được nén bằng cách đặt đối số `options`:

```
options = tf.io.TFRecordOptions(compression_type="GZIP")
with tf.io.TFRecordWriter("my_compressed.tfrecord", options) as f:
    f.write(b"Compress, compress, compress!")
```

Khi đọc một tệp TFRecord được nén, bạn cần chỉ định loại nén:

```
dataset = tf.data.TFRecordDataset(["my_compressed.tfrecord"],
                                  compression_type="GZIP")
```

Giới thiệu ngắn gọn về Protocol Buffers

Mặc dù mỗi bản ghi có thể sử dụng bất kỳ định dạng nhị phân nào bạn muốn, các tệp TFRecord thường chứa các **protocol buffers** (còn gọi là **protobufs**) được tuần tự hóa. Đây là một định dạng nhị phân di động, có thể mở rộng và hiệu quả được phát triển tại Google vào năm 2001 và được mã nguồn mở vào năm 2008. Protobufs hiện được sử dụng rộng rãi, đặc biệt là trong gRPC, hệ thống gọi thủ tục từ xa của Google. Chúng được định nghĩa bằng một ngôn ngữ đơn giản trông như thế này:

```
syntax = "proto3";

message Person {
  string name = 1;
  int32 id = 2;
  repeated string email = 3;
}
```

Định nghĩa protobuf này cho biết chúng ta đang sử dụng phiên bản 3 của định dạng protobuf và nó chỉ định rằng mỗi đối tượng `Person` có thể (tùy chọn) có một `name` kiểu `string`, một `id` kiểu `int32` và không hoặc nhiều trường `email`, mỗi trường có kiểu `string`. Các số 1, 2 và 3 là các mã định danh trường: chúng sẽ được sử dụng trong biểu diễn nhị phân của mỗi bản ghi. Khi bạn có một định nghĩa trong tệp `.proto`, bạn có thể biên dịch nó. Điều này yêu cầu `protoc`, trình biên dịch protobuf, để tạo các lớp truy cập bằng Python (hoặc một ngôn ngữ khác).

Để minh họa những điều cơ bản, hãy xem một ví dụ đơn giản sử dụng các lớp truy cập được tạo cho protobuf `Person`:

```
>>> from person_pb2 import Person # nhập lớp truy cập đã tạo

>>> person = Person(name="Al", id=123, email=["a@b.com"]) # tạo một Person
>>> print(person) # hiển thị Person
```

```
name: "Al"
id: 123
email: "a@b.com"

>>> person.name # đọc một trường
'Al'
>>> person.name = "Alice" # sửa đổi một trường
>>> person.email[0] # các trường lặp lại có thể được truy cập như mảng
'a@b.com'
>>> person.email.append("c@d.com") # thêm một địa chỉ email

>>> serialized = person.SerializeToString() # tuần tự hóa person thành chuỗi
byte
>>> serialized
b'\n\x05Alice\x10{\x1a\x07a@b.com\x1a\x07c@d.com'

>>> person2 = Person() # tạo một Person mới
>>> person2.ParseFromString(serialized) # phân tích chuỗi byte (dài 27 byte)
27
>>> person == person2 # bây giờ chúng bằng nhau
True
```

Tóm lại, chúng ta nhập lớp `Person` được tạo bởi `protoc`, tạo một thể hiện và thao tác với nó, sau đó tuần tự hóa nó bằng phương thức `SerializeToString()`. Đây là dữ liệu nhị phân sẵn sàng để được lưu hoặc truyền qua mạng. Khi đọc hoặc nhận dữ liệu nhị phân này, chúng ta có thể phân tích cú pháp nó bằng phương thức `ParseFromString()` và nhận lại một bản sao của đối tượng đã được tuần tự hóa.

Các Protobuf của TensorFlow

Protobuf chính thường được sử dụng trong tệp TFRecord là **Example protobuf**, đại diện cho một mẫu trong tập dữ liệu. Nó chứa một danh sách các đặc trưng được đặt tên, trong đó mỗi đặc trưng có thể là một danh sách các chuỗi byte, một danh sách các số thực (float), hoặc một danh sách các số nguyên (integer). Đây là định nghĩa protobuf (từ mã nguồn của TensorFlow):

```
syntax = "proto3";

message BytesList { repeated bytes value = 1; }
message FloatList { repeated float value = 1 [packed = true]; }
message Int64List { repeated int64 value = 1 [packed = true]; }

message Feature {
  oneof kind {
    BytesList bytes_list = 1;
    FloatList float_list = 2;
    Int64List int64_list = 3;
```

```
  }
};

message Features { map<string, Feature> feature = 1; };

message Example { Features features = 1; };
```

Đây là cách bạn có thể tạo một `tf.train.Example` đại diện cho cùng một người như trước đó:

```
from tensorflow.train import BytesList, FloatList, Int64List
from tensorflow.train import Feature, Features, Example

person_example = Example(
    features=Features(
        feature={
            "name": Feature(bytes_list=BytesList(value=[b"Alice"])),
            "id": Feature(int64_list=Int64List(value=[123])),
            "emails": Feature(bytes_list=BytesList(value=[b"a@b.com",
b"c@d.com"]))
        }))
```

Mã này hơi dài dòng, nhưng bạn có thể dễ dàng gói nó vào một hàm trợ giúp nhỏ. Bây giờ chúng ta có một `Example` protobuf, chúng ta có thể tuần tự hóa nó bằng cách gọi phương thức `SerializeToString()` của nó, sau đó ghi dữ liệu kết quả vào một tệp TFRecord.

Tải và Phân tích cú pháp các Example

Để tải các `Example` protobuf đã được tuần tự hóa, chúng ta sẽ lại sử dụng `tf.data.TFRecordDataset` và chúng ta sẽ phân tích cú pháp mỗi `Example` bằng `tf.io.parse_single_example()`. Nó yêu cầu ít nhất hai đối số: một tensor vô hướng chuỗi chứa dữ liệu đã tuần tự hóa và một mô tả về mỗi đặc trưng. Mô tả là một từ điển ánh xạ mỗi tên đặc trưng tới một `tf.io.FixedLenFeature` (cho biết hình dạng, kiểu và giá trị mặc định của đặc trưng) hoặc một `tf.io.VarLenFeature` (chỉ cho biết kiểu nếu độ dài của danh sách đặc trưng có thể thay đổi).

```
feature_description = {
    "name": tf.io.FixedLenFeature([], tf.string, default_value=""),
    "id": tf.io.FixedLenFeature([], tf.int64, default_value=0),
    "emails": tf.io.VarLenFeature(tf.string),
}

def parse(serialized_example):
    return tf.io.parse_single_example(serialized_example,
feature_description)

dataset = tf.data.TFRecordDataset(["my_contacts.tfrecord"]).map(parse)
for parsed_example in dataset:
    print(parsed_example)
```

Các đặc trưng có độ dài cố định được phân tích cú pháp thành các tensor thông thường, nhưng các đặc trưng có độ dài thay đổi được phân tích cú pháp thành các **sparse tensors**. Bạn có thể chuyển đổi một sparse tensor thành một dense tensor bằng `tf.sparse.to_dense()`.

Thay vì phân tích cú pháp các ví dụ từng cái một, bạn có thể muốn phân tích cú pháp chúng theo lô bằng `tf.io.parse_example()`.

Cuối cùng, một `BytesList` có thể chứa bất kỳ dữ liệu nhị phân nào bạn muốn, bao gồm bất kỳ đối tượng được tuần tự hóa nào. Ví dụ, bạn có thể sử dụng `tf.io.encode_jpeg()` để mã hóa một hình ảnh và đặt dữ liệu nhị phân này vào một `BytesList`.

Xử lý Danh sách của các Danh sách bằng SequenceExample Protobuf

`SequenceExample` của TensorFlow được thiết kế cho các trường hợp sử dụng như phân loại tài liệu văn bản, nơi mỗi tài liệu có thể được biểu diễn dưới dạng danh sách các câu, và mỗi câu là một danh sách các từ. Một `SequenceExample` chứa một đối tượng `Features` cho dữ liệu ngữ cảnh và một đối tượng `FeatureLists` chứa một hoặc nhiều `FeatureList` được đặt tên. Việc xây dựng, tuần tự hóa và phân tích cú pháp một `SequenceExample` tương tự như một `Example`, nhưng bạn phải sử dụng `tf.io.parse_single_sequence_example()` hoặc `tf.io.parse_sequence_example()`.

Bây giờ bạn đã biết cách lưu trữ, tải, phân tích cú pháp và tiền xử lý dữ liệu một cách hiệu quả, đã đến lúc chuyển sự chú ý của chúng ta sang các lớp tiền xử lý của Keras.

Các Lớp Tiền xử lý của Keras

Việc chuẩn bị dữ liệu của bạn cho một mạng nơ-ron thường đòi hỏi việc chuẩn hóa các đặc trưng số, mã hóa các đặc trưng phân loại và văn bản, cắt và thay đổi kích thước hình ảnh, v.v. Có một số tùy chọn cho việc này:

- **Tiền xử lý trước**: Thực hiện khi chuẩn bị các tệp dữ liệu huấn luyện của bạn, sử dụng bất kỳ công cụ nào bạn thích.
- **Tiền xử lý tức thời với `tf.data`**: Áp dụng một hàm tiền xử lý cho mọi phần tử của một tập dữ liệu bằng phương thức `map()` của nó.
- **Lớp tiền xử lý trong mô hình**: Bao gồm các lớp tiền xử lý trực tiếp bên trong mô hình của bạn.

Phần còn lại của chương này sẽ xem xét phương pháp cuối cùng. Keras cung cấp nhiều lớp tiền xử lý mà bạn có thể đưa vào mô hình của mình.

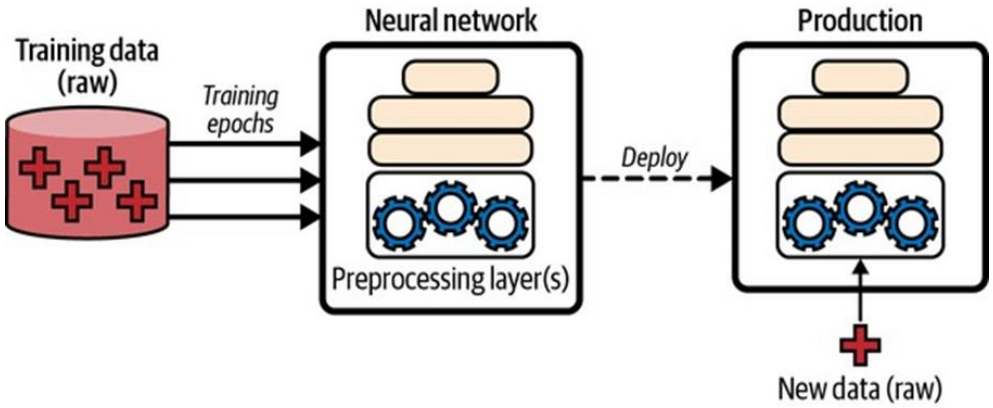
Lớp Normalization

Như chúng ta đã thấy, Keras cung cấp một lớp `Normalization` mà chúng ta có thể sử dụng để chuẩn hóa các đặc trưng đầu vào. Bạn có thể chỉ định `mean` và `variance` khi tạo lớp

hoặc—đơn giản hơn—truyền tập huấn luyện vào phương thức `adapt()` của lớp để nó tự đo lường chúng:

```
norm_layer = tf.keras.layers.Normalization()
model = tf.keras.models.Sequential([
    norm_layer,
    tf.keras.layers.Dense(1)
])
model.compile(...)
norm_layer.adapt(X_train) # tính toán giá trị trung bình và phương sai
model.fit(X_train, y_train, ...)
```

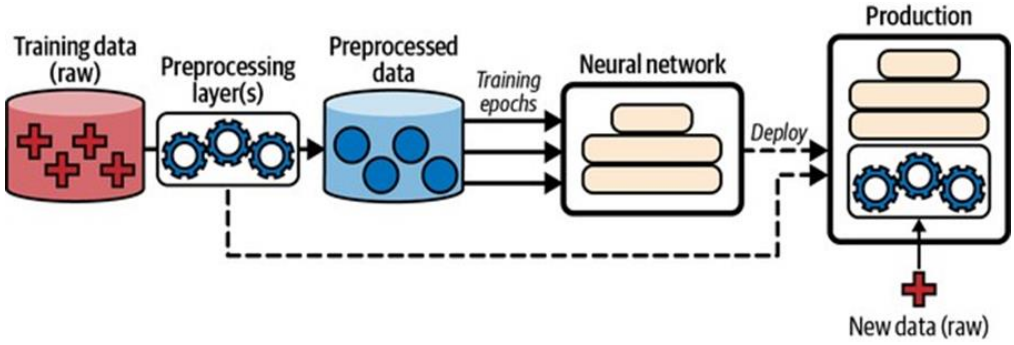
Vì chúng ta đã bao gồm lớp `Normalization` bên trong mô hình, chúng ta có thể triển khai mô hình này vào sản xuất mà không cần lo lắng về việc chuẩn hóa nữa: mô hình sẽ tự xử lý nó.



Hình 13-4. Bao gồm các lớp tiền xử lý bên trong một mô hình

Cách tiếp cận này giúp loại bỏ hoàn toàn nguy cơ **sai lệch tiền xử lý**.

Tuy nhiên, việc đưa lớp tiền xử lý trực tiếp vào mô hình sẽ làm chậm quá trình huấn luyện một chút. Để khắc phục, chúng ta có thể chuẩn hóa toàn bộ tập huấn luyện chỉ một lần trước khi huấn luyện. Sau đó, để triển khai, chúng ta tạo một mô hình mới bao bọc cả lớp `Normalization` đã được adapt và mô hình chúng ta vừa huấn luyện.



Hình 13-5. Tiền xử lý dữ liệu một lần trước khi huấn luyện, sau đó triển khai các lớp đó bên trong mô hình cuối cùng

Bây giờ chúng ta có được điều tốt nhất của cả hai thế giới: huấn luyện nhanh và mô hình cuối cùng có thể tự tiền xử lý đầu vào của nó.

Lớp Discretization

Lớp `Discretization` biến một đặc trưng số thành một đặc trưng phân loại bằng cách ánh xạ các phạm vi giá trị (gọi là **bins** - thùng) vào các danh mục. Ví dụ, mã sau đây ánh xạ một đặc trưng tuổi số thành ba loại: dưới 18, từ 18 đến 50 (không bao gồm), và 50 trở lên:

```
>>> age = tf.constant([[10.], [93.], [57.], [18.], [37.], [5.]])
>>> discretize_layer = tf.keras.layers.Discretization(bin_boundaries=[18.,
50.])
>>> age_categories = discretize_layer(age)
>>> age_categories
<tf.Tensor: shape=(6, 1), dtype=int64, numpy=array([[0],[2],[2],
[1],[1],[0]])>
```

Các mã định danh danh mục như thế này thường nên được mã hóa, ví dụ như sử dụng mã hóa one-hot.

Lớp CategoryEncoding

Khi chỉ có một vài danh mục, **mã hóa one-hot** thường là một lựa chọn tốt. Để làm điều này, Keras cung cấp lớp `CategoryEncoding`.

```
>>> onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3)
>>> onehot_layer(age_categories)
<tf.Tensor: shape=(6, 3), dtype=float32, numpy=
 array([[0., 1., 0.],
        [0., 0., 1.],
        [0., 0., 1.],
        [0., 1., 0.],
        [0., 0., 1.],
        [1., 0., 0.]], dtype=float32)>
```

Lớp StringLookup

Đối với các đặc trưng văn bản phân loại, bạn có thể sử dụng lớp `StringLookup`. Lớp này sẽ tìm tất cả các chuỗi duy nhất (từ vựng) và ánh xạ mỗi chuỗi đến một ID số nguyên. Các danh mục không xác định được ánh xạ tới 0 theo mặc định.

```
>>> cities = ["Auckland", "Paris", "Paris", "San Francisco"]
>>> str_lookup_layer = tf.keras.layers.StringLookup()
>>> str_lookup_layer.adapt(cities)
>>> str_lookup_layer([["Paris"], ["Auckland"], ["Auckland"], ["Montreal"]])
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=array([[1], [3], [3], [0]])>
```

Nếu bạn đặt `output_mode="one_hot"`, nó sẽ xuất ra một vector one-hot cho mỗi danh mục.

Để xử lý các danh mục hiếm hoặc không xác định, bạn có thể đặt `num_oov_indices` (số lượng thùng cho từ vựng ngoài - OOV). Mỗi danh mục không xác định sẽ được ánh xạ ngẫu nhiên vào một trong các thùng OOV.

Lớp Hashing

Đối với mỗi danh mục, lớp `Hashing` của Keras tính toán một giá trị băm, modulo số lượng thùng. Lợi ích của lớp này là nó không cần phải được `adapt`, điều này đôi khi có thể hữu ích. Tuy nhiên, có thể xảy ra **xung đột băm** (hashing collision), nơi các danh mục khác nhau được ánh xạ đến cùng một ID. Vì vậy, thường thì việc sử dụng lớp `StringLookup` sẽ tốt hơn.

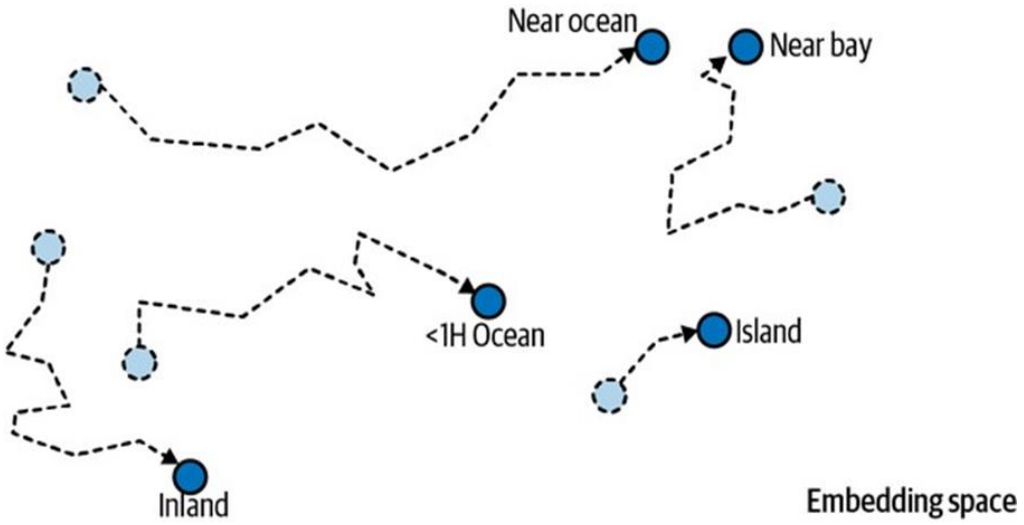
Bây giờ hãy xem một cách khác để mã hóa các danh mục: các nhúng có thể huấn luyện.

Mã hóa đặc trưng phân loại bằng Embedding

Một **embedding** là một biểu diễn dày đặc của một số dữ liệu có chiều cao hơn, chẳng hạn như một danh mục, hoặc một từ trong một từ vựng. Nếu có 50.000 danh mục có thể, thì mã hóa one-hot sẽ tạo ra một vector thưa thớt 50.000 chiều (tức là, chứa chủ yếu là số 0). Ngược lại, một embedding sẽ là một vector dày đặc tương đối nhỏ; ví dụ, chỉ với 100 chiều.

Trong học sâu, các embedding thường được khởi tạo ngẫu nhiên, và sau đó chúng được huấn luyện bằng giảm độ dốc, cùng với các tham số mô hình khác. Ví dụ, danh mục “NEAR BAY” trong tập dữ liệu nhà ở California ban đầu có thể được biểu diễn bằng một vector ngẫu nhiên như `[0.131, 0.890]`, trong khi danh mục “NEAR OCEAN” có thể được biểu diễn bằng một vector ngẫu nhiên khác như `[0.631, 0.791]`. Trong ví dụ này, chúng ta sử dụng các embedding 2D, nhưng số chiều là một siêu tham số bạn có thể điều chỉnh.

Vì các embedding này có thể huấn luyện được, chúng sẽ dần dần cải thiện trong quá trình huấn luyện; và vì chúng đại diện cho các danh mục khá giống nhau trong trường hợp này, giảm độ dốc chắc chắn sẽ đẩy chúng gần nhau hơn, trong khi nó sẽ có xu hướng di chuyển chúng ra xa embedding của danh mục “INLAND” (xem Hình 13-6). Thật vậy, biểu diễn càng tốt, mạng nơ-ron càng dễ đưa ra dự đoán chính xác, vì vậy quá trình huấn luyện có xu hướng làm cho các embedding trở thành biểu diễn hữu ích của các danh mục. Đây được gọi là **học biểu diễn (representation learning)** (bạn sẽ thấy các loại học biểu diễn khác trong Chương 17).



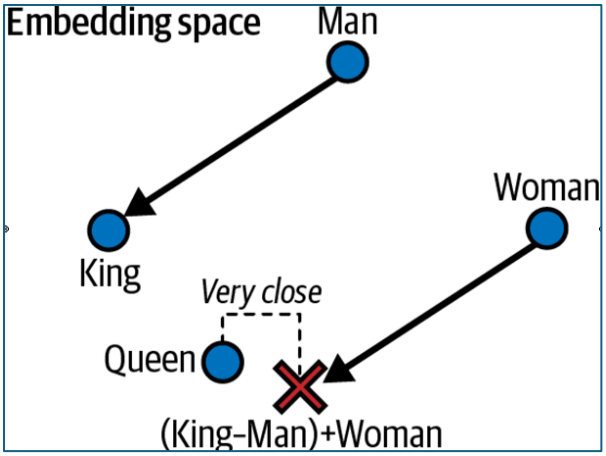
Hình 13-6. Các embedding sẽ dần dần cải thiện trong quá trình huấn luyện

WORD EMBEDDINGS (Embedding từ)

Các embedding không chỉ nói chung sẽ là biểu diễn hữu ích cho tác vụ hiện tại, mà khá thường xuyên các embedding tương tự này có thể được tái sử dụng thành công cho các tác vụ khác. Ví dụ phổ biến nhất về điều này là **embedding từ (word embeddings)** (tức là, embedding của các từ riêng lẻ): khi bạn đang làm việc với một tác vụ xử lý ngôn ngữ tự nhiên, bạn thường sẽ làm tốt hơn khi tái sử dụng các embedding từ đã được huấn luyện trước hơn là huấn luyện của riêng bạn.

Ý tưởng sử dụng vector để biểu diễn từ có từ những năm 1960, và nhiều kỹ thuật tinh vi đã được sử dụng để tạo ra các vector hữu ích, bao gồm cả việc sử dụng mạng nơ-ron. Nhưng mọi thứ thực sự bùng nổ vào năm 2013, khi Tomáš Mikolov và các nhà nghiên cứu Google khác công bố một bài báo mô tả một kỹ thuật hiệu quả để học embedding từ bằng mạng nơ-ron, vượt trội đáng kể so với các nỗ lực trước đó. Điều này cho phép họ học embedding trên một tập hợp văn bản rất lớn: họ đã huấn luyện một mạng nơ-ron để dự đoán các từ gần bất kỳ từ nào đã cho và thu được các embedding từ đáng kinh ngạc. Ví dụ, các từ đồng nghĩa có các embedding rất gần nhau, và các từ có liên quan về mặt ngữ nghĩa như France, Spain, và Italy cuối cùng được nhóm lại với nhau.

Tuy nhiên, nó không chỉ là về sự gần gũi: embedding từ cũng được tổ chức dọc theo các trục có ý nghĩa trong không gian embedding. Đây là một ví dụ nổi tiếng: nếu bạn tính toán King - Man + Woman (cộng và trừ các vector embedding của các từ này), thì kết quả sẽ rất gần với embedding của từ Queen (xem Hình 13-7). Nói cách khác, embedding từ mã hóa khái niệm giới tính! Tương tự, bạn có thể tính Madrid – Spain + France, và kết quả gần với Paris, điều này dường như cho thấy khái niệm thủ đô cũng được mã hóa trong embedding.



Hình 13-7. Embedding từ của các từ tương tự có xu hướng gần nhau, và một số trục dường như mã hóa các khái niệm có ý nghĩa

Thật không may, embedding từ đôi khi nắm bắt những thành kiến tồi tệ nhất của chúng ta. Ví dụ, mặc dù chúng học đúng rằng Man tương ứng với King như Woman tương ứng với Queen, nhưng chúng cũng dường như học rằng Man tương ứng với Doctor như Woman tương ứng với Nurse: một thành kiến phân biệt giới tính! Công bằng mà nói, ví dụ cụ thể này có lẽ đã bị phóng đại, như đã được chỉ ra trong một bài báo năm 2019 của Malvina Nissim et al. Tuy nhiên, đảm bảo công bằng trong các thuật toán học sâu là một chủ đề nghiên cứu quan trọng và tích cực.

Keras cung cấp một lớp `Embedding`, lớp này bao bọc một ma trận embedding: ma trận này có một hàng cho mỗi danh mục và một cột cho mỗi chiều embedding. Theo mặc định, nó được khởi tạo ngẫu nhiên. Để chuyển đổi một ID danh mục thành một embedding, lớp `Embedding` chỉ cần tra cứu và trả về hàng tương ứng với danh mục đó. Thế thôi! Ví dụ, hãy khởi tạo một lớp `Embedding` với năm hàng và các embedding 2D, và sử dụng nó để mã hóa một số danh mục:

```
>>> tf.random.set_seed(42)

>>> embedding_layer = tf.keras.layers.Embedding(input_dim=5, output_dim=2)

>>> embedding_layer(np.array([2, 4, 2]))
<tf.Tensor:
shape=(3, 2), dtype=float32, numpy= array([[-0.04663396, 0.01846724],
       [-0.02736737, -0.02768031],
       [-0.04663396, 0.01846724]], dtype=float32)>
```

Như bạn có thể thấy, danh mục 2 được mã hóa (hai lần) dưới dạng vector 2D `[-0.04663396, 0.01846724]`, trong khi danh mục 4 được mã hóa là `[-0.02736737, -0.02768031]`. Vì lớp chưa được huấn luyện, các mã hóa này chỉ là ngẫu nhiên.

Nếu bạn muốn nhúng một thuộc tính văn bản phân loại, bạn có thể đơn giản nối chuỗi một lớp `StringLookup` và một lớp `Embedding`, như thế này:

```
>>> tf.random.set_seed(42)

>>> ocean_prox = ["<1H OCEAN", "INLAND","NEAR OCEAN", "NEAR BAY", "ISLAND"]

>>> str_lookup_layer = tf.keras.layers.StringLookup()

>>> str_lookup_layer.adapt(ocean_prox)

>>> lookup_and_embed = tf.keras.Sequential([

...     str_lookup_layer,

...
tf.keras.layers.Embedding(input_dim=str_lookup_layer.vocabulary_size(),

...                                   output_dim=2)

... ])

...

>>> lookup_and_embed(np.array([["<1H OCEAN"], ["ISLAND"], ["<1H OCEAN"]]))
<tf.Tensor:
shape=(3, 2), dtype=float32, numpy= array([[-0.01896119, 0.02223358],
       [ 0.02401174, 0.03724445],
       [-0.01896119, 0.02223358]], dtype=float32)>
```

Lưu ý rằng số hàng trong ma trận embedding cần phải bằng kích thước từ vựng (`vocabulary_size`): đó là tổng số danh mục, bao gồm các danh mục đã biết cộng với các thùng OOV (mặc định chỉ có một). Phương thức `vocabulary_size()` của lớp `StringLookup` tiện lợi trả về số này.

Kết hợp mọi thứ lại với nhau, bây giờ chúng ta có thể tạo một mô hình Keras có thể xử lý một đặc trưng văn bản phân loại cùng với các đặc trưng số thông thường và học một embedding cho mỗi danh mục (cũng như cho mỗi thùng OOV):

```
# Giả định X_train_num, X_train_cat, y_train, X_valid_num, X_valid_cat,
y_valid đã được tải
# Ví dụ:
# X_train_num = np.random.rand(100, 8).astype(np.float32)
# X_train_cat = np.array([["<1H OCEAN"], ["INLAND"], ["NEAR OCEAN"], ["NEAR
BAY"], ["ISLAND"]] * 20)
# y_train = np.random.rand(100, 1).astype(np.float32)
# X_valid_num = np.random.rand(20, 8).astype(np.float32)
# X_valid_cat = np.array([["<1H OCEAN"], ["INLAND"]] * 10)
# y_valid = np.random.rand(20, 1).astype(np.float32)

num_input = tf.keras.layers.Input(shape=[8], name="num")
```

```
cat_input = tf.keras.layers.Input(shape=[], dtype=tf.string, name="cat")
cat_embeddings = lookup_and_embed(cat_input) # sử dụng lookup_and_embed đã
tạo ở trên
encoded_inputs = tf.keras.layers.concatenate([num_input, cat_embeddings])
outputs = tf.keras.layers.Dense(1)(encoded_inputs)
model = tf.keras.models.Model(inputs=[num_input, cat_input],
outputs=[outputs])
model.compile(loss="mse", optimizer="sgd")
history = model.fit((X_train_num, X_train_cat), y_train, epochs=5,
                    validation_data=((X_valid_num, X_valid_cat),
                                     y_valid))
```

Mô hình này nhận hai đầu vào: `num_input`, chứa tám đặc trưng số cho mỗi thể hiện, cộng với `cat_input`, chứa một đầu vào văn bản phân loại duy nhất cho mỗi thể hiện. Mô hình sử dụng mô hình `lookup_and_embed` chúng ta đã tạo trước đó để mã hóa mỗi danh mục khoảng cách đại dương (`ocean-proximity`) thành embedding có thể huấn luyện tương ứng. Tiếp theo, nó nối các đầu vào số và các embedding bằng hàm `concatenate()` để tạo ra các đầu vào đã được mã hóa hoàn chỉnh, sẵn sàng được đưa vào mạng nơ-ron. Chúng ta có thể thêm bất kỳ loại mạng nơ-ron nào tại thời điểm này, nhưng để đơn giản, chúng ta chỉ thêm một lớp đầu ra dày đặc duy nhất, và sau đó chúng ta tạo mô hình Keras với các đầu vào và đầu ra chúng ta vừa định nghĩa. Tiếp theo, chúng ta biên dịch mô hình và huấn luyện nó, truyền cả đầu vào số và phân loại.

Như bạn đã thấy trong Chương 10, vì các lớp `Input` được đặt tên là “num” và “cat”, chúng ta cũng có thể đã truyền dữ liệu huấn luyện cho phương thức `fit()` bằng cách sử dụng một từ điển thay vì một tuple: `{"num": X_train_num, "cat": X_train_cat}`. Ngoài ra, chúng ta có thể đã truyền một `tf.data.Dataset` chứa các batch, mỗi batch được biểu diễn dưới dạng `((X_batch_num, X_batch_cat), y_batch)` hoặc dưới dạng `({"num": X_batch_num, "cat": X_batch_cat}, y_batch)`. Và tất nhiên điều tương tự cũng áp dụng cho dữ liệu xác thực.

OK, bây giờ bạn đã học cách mã hóa các đặc trưng phân loại, đã đến lúc chúng ta chuyển sự chú ý sang tiền xử lý văn bản.

Tiền xử lý văn bản

Keras cung cấp một lớp `TextVectorization` để tiền xử lý văn bản cơ bản. Giống như lớp `StringLookup`, bạn phải truyền cho nó một từ vựng khi tạo, hoặc để nó học từ vựng từ một số dữ liệu huấn luyện bằng phương thức `adapt()`. Hãy xem một ví dụ:

```
>>> train_data = ["To be", "!(to be)", "That's the question","Be, be, be."]

>>> text_vec_layer = tf.keras.layers.TextVectorization()

>>> text_vec_layer.adapt(train_data)

>>> text_vec_layer(["Be good!", "Question: be or be?"])
<tf.Tensor:
```

```
shape=(2, 4), dtype=int64, numpy= array([[2, 1, 0, 0],
       [6, 2, 1, 2]])>
```

Hai câu “Be good!” và “Question: be or be?” được mã hóa lần lượt là `[2, 1, 0, 0]` và `[6, 2, 1, 2]`. Từ vựng được học từ bốn câu trong dữ liệu huấn luyện: “be” = 2, “to” = 3, v.v. Để xây dựng từ vựng, phương thức `adapt()` trước tiên chuyển các câu huấn luyện sang chữ thường và loại bỏ dấu câu, đó là lý do tại sao “Be”, “be”, và “be?” đều được mã hóa là “be” = 2. Tiếp theo, các câu được tách trên khoảng trắng, và các từ kết quả được sắp xếp theo tần suất giảm dần, tạo ra từ vựng cuối cùng. Khi mã hóa câu, các từ không xác định được mã hóa là 1. Cuối cùng, vì câu đầu tiên ngắn hơn câu thứ hai, nó được đệm bằng 0.

Các ID từ phải được mã hóa, thường sử dụng lớp `Embedding`: chúng ta sẽ làm điều này trong Chương 16.

Ngoài ra, bạn có thể đặt đối số `output_mode` của lớp `TextVectorization` thành “multi_hot” hoặc “count” để nhận được các mã hóa tương ứng. Tuy nhiên, việc chỉ đếm từ thường không lý tưởng: các từ như “to” và “the” quá thường xuyên đến mức chúng hầu như không quan trọng chút nào, trong khi các từ hiếm hơn như “basketball” lại mang nhiều thông tin hơn. Vì vậy, thay vì đặt `output_mode` thành “multi_hot” hoặc “count”, thường nên đặt nó thành “tf_idf”, viết tắt của “term-frequency × inverse-document-frequency” (TF-IDF). Điều này tương tự như mã hóa đếm, nhưng các từ xuất hiện thường xuyên trong dữ liệu huấn luyện bị giảm trọng số, và ngược lại, các từ hiếm bị tăng trọng số. Ví dụ:

```
>>> text_vec_layer = tf.keras.layers.TextVectorization(output_mode="tf_idf")

>>> text_vec_layer.adapt(train_data)

>>> text_vec_layer(["Be good!", "Question: be or be?"])
<tf.Tensor:
shape=(2, 6), dtype=float32, numpy= array([[0.96725637, 0.6931472 , 0.
, 0.        ,
        0.        , 0.        ],
       [0.96725637, 1.3862944 , 0.        , 0.        , 0.        ,
        1.0986123 ]], dtype=float32)>
```

Có nhiều biến thể TF-IDF, nhưng cách lớp `TextVectorization` triển khai nó là bằng cách nhân mỗi số đếm từ với một trọng số bằng $log\big(1 + d/(f + 1)\big)$, trong đó d là tổng số câu (còn gọi là tài liệu) trong dữ liệu huấn luyện và f đếm số câu huấn luyện chứa từ đã cho. Ví dụ, trong trường hợp này có $d = 4$ câu trong dữ liệu huấn luyện, và từ “be” xuất hiện trong $f = 3$ trong số đó. Vì từ “be” xuất hiện hai lần trong câu “Question: be or be?”, nó được mã hóa là $2timeslog\big(1 + 4/(1 + 3)\big)approx1.3862944$. Từ “question” chỉ xuất hiện một lần, nhưng vì nó là một từ ít phổ biến hơn, mã hóa của nó gần như cao: $1timeslog\big(1 + 4/(1 + 1)\big)approx1.0986123$. Lưu ý rằng trọng số trung bình được sử dụng cho các từ không xác định.

Cách tiếp cận mã hóa văn bản này dễ sử dụng và nó có thể mang lại kết quả khá tốt cho các tác vụ xử lý ngôn ngữ tự nhiên cơ bản, nhưng nó có một số hạn chế quan trọng: nó chỉ

hoạt động với các ngôn ngữ tách từ bằng khoảng trắng, nó không phân biệt giữa các từ đồng âm (ví dụ: “to bear” so với “teddy bear”), nó không gợi ý cho mô hình của bạn rằng các từ như “evolution” và “evolutionary” có liên quan, v.v. Và nếu bạn sử dụng mã hóa multi-hot, count hoặc TF-IDF, thì thứ tự của các từ bị mất. Vậy các lựa chọn khác là gì?

Một lựa chọn là sử dụng thư viện TensorFlow Text, cái này cung cấp các tính năng tiền xử lý văn bản nâng cao hơn lớp `TextVectorization`. Ví dụ, nó bao gồm một số bộ tách từ subword có khả năng chia văn bản thành các token nhỏ hơn từ, điều này giúp mô hình dễ dàng phát hiện rằng “evolution” và “evolutionary” có điểm chung (thêm về tokenization subword trong Chương 16).

Một lựa chọn khác là sử dụng các thành phần mô hình ngôn ngữ đã được huấn luyện trước. Hãy xem xét điều này ngay bây giờ.

Sử dụng các thành phần mô hình ngôn ngữ được huấn luyện trước

Thư viện TensorFlow Hub giúp dễ dàng tái sử dụng các thành phần mô hình đã được huấn luyện trước trong các mô hình của riêng bạn, cho văn bản, hình ảnh, âm thanh và hơn thế nữa. Các thành phần mô hình này được gọi là **module**. Chỉ cần duyệt kho lưu trữ TF Hub, tìm cái bạn cần và sao chép ví dụ mã vào dự án của bạn, và module sẽ tự động được tải xuống và đóng gói thành một lớp Keras mà bạn có thể trực tiếp đưa vào mô hình của mình. Các module thường chứa cả mã tiền xử lý và trọng số đã được huấn luyện trước, và chúng thường không yêu cầu huấn luyện thêm (nhưng tất nhiên, phần còn lại của mô hình của bạn chắc chắn sẽ yêu cầu huấn luyện).

Ví dụ, một số mô hình ngôn ngữ mạnh mẽ đã được huấn luyện trước có sẵn. Những mô hình mạnh mẽ nhất khá lớn (vài gigabyte), vì vậy để có một ví dụ nhanh, hãy sử dụng module `nnlm-en-dim50`, phiên bản 2, đây là một module khá cơ bản nhận văn bản thô làm đầu vào và xuất ra các embedding câu 50 chiều. Chúng ta sẽ nhập TensorFlow Hub và sử dụng nó để tải module, sau đó sử dụng module đó để mã hóa hai câu thành vector:

```
>>> import tensorflow_hub as hub

>>> hub_layer = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim50/2")

>>> sentence_embeddings = hub_layer(tf.constant(["To be", "Not to be"]))

>>> sentence_embeddings.numpy().round(2)
array([[-0.25, 0.28, 0.01, 0.1 , ..., 0.05, 0.31],
       [-0.2 , 0.2 , -0.08, 0.02, ..., -0.04, 0.15]],
      dtype=float32)
```

Lớp `hub.KerasLayer` tải module từ URL đã cho. Module cụ thể này là một bộ mã hóa câu: nó nhận các chuỗi làm đầu vào và mã hóa mỗi chuỗi thành một vector duy nhất (trong trường hợp này, một vector 50 chiều). Bên trong, nó phân tích cú pháp chuỗi (tách từ trên khoảng trắng) và nhúng từng từ bằng cách sử dụng một ma trận embedding đã được huấn

luyện trước trên một tập hợp lớn: tập hợp Google News 7B (dài bảy tỷ từ!). Sau đó, nó tính toán trung bình của tất cả các embedding từ, và kết quả là embedding câu.

Bạn chỉ cần đưa `hub_layer` này vào mô hình của mình, và bạn đã sẵn sàng. Lưu ý rằng mô hình ngôn ngữ cụ thể này được huấn luyện trên ngôn ngữ tiếng Anh, nhưng nhiều ngôn ngữ khác cũng có sẵn, cũng như các mô hình đa ngôn ngữ.

Cuối cùng nhưng không kém phần quan trọng, thư viện Transformers mã nguồn mở tuyệt vời của Hugging Face cũng giúp dễ dàng đưa các thành phần mô hình ngôn ngữ mạnh mẽ vào các mô hình của riêng bạn. Bạn có thể duyệt Hugging Face Hub, chọn mô hình bạn muốn và sử dụng các ví dụ mã được cung cấp để bắt đầu. Nó từng chỉ chứa các mô hình ngôn ngữ, nhưng hiện đã mở rộng để bao gồm các mô hình hình ảnh và hơn thế nữa.

Chúng ta sẽ quay lại xử lý ngôn ngữ tự nhiên sâu hơn trong Chương 16. Bây giờ hãy xem các lớp tiền xử lý hình ảnh của Keras.

Các lớp tiền xử lý hình ảnh

API tiền xử lý của Keras bao gồm ba lớp tiền xử lý hình ảnh:

- `tf.keras.layers.Resizing` thay đổi kích thước hình ảnh đầu vào theo kích thước mong muốn. Ví dụ, `Resizing(height=100, width=200)` thay đổi kích thước mỗi hình ảnh thành 100 × 200, có thể làm biến dạng hình ảnh. Nếu bạn đặt `crop_to_aspect_ratio=True`, thì hình ảnh sẽ được cắt theo tỷ lệ hình ảnh mục tiêu, để tránh biến dạng.
- `tf.keras.layers.Rescaling` điều chỉnh lại các giá trị pixel. Ví dụ, `Rescaling(scale=2/255, offset=-1)` điều chỉnh lại các giá trị từ 0 → 255 thành –1 → 1.
- `tf.keras.layers.CenterCrop` cắt hình ảnh, chỉ giữ lại một phần trung tâm có chiều cao và chiều rộng mong muốn.

Ví dụ, hãy tải một vài hình ảnh mẫu và cắt chúng ở giữa. Để làm điều này, chúng ta sẽ sử dụng hàm `load_sample_images()` của Scikit-Learn; hàm này tải hai hình ảnh màu, một là đền thờ Trung Quốc và một là hoa (điều này yêu cầu thư viện Pillow, cái này đã được cài đặt nếu bạn đang sử dụng Colab hoặc nếu bạn đã làm theo hướng dẫn cài đặt):

```
from sklearn.datasets import load_sample_images

images = load_sample_images()["images"]
crop_image_layer = tf.keras.layers.CenterCrop(height=100, width=100)
cropped_images = crop_image_layer(images)
```

Keras cũng bao gồm một số lớp để tăng cường dữ liệu, chẳng hạn như `RandomCrop`, `RandomFlip`, `RandomTranslation`, `RandomRotation`, `RandomZoom`, `RandomHeight`, `RandomWidth`, và `RandomContrast`. Các lớp này chỉ hoạt động trong quá trình huấn luyện, và chúng ngẫu nhiên áp dụng một số phép biến đổi cho hình ảnh đầu vào (tên của chúng tự giải thích). Tăng cường dữ liệu sẽ làm tăng kích thước của tập huấn luyện một cách nhân

tạo, điều này thường dẫn đến cải thiện hiệu suất, miễn là các hình ảnh đã biến đổi trông giống như hình ảnh thực tế (không tăng cường). Chúng ta sẽ xem xét kỹ hơn quá trình xử lý hình ảnh trong chương tiếp theo.

Bây giờ hãy xem một cách khác để tải dữ liệu dễ dàng và hiệu quả trong TensorFlow.

Dự án TensorFlow Datasets

Dự án TensorFlow Datasets (TFDS) giúp rất dễ dàng tải các tập dữ liệu phổ biến, từ nhỏ như MNIST hoặc Fashion MNIST đến các tập dữ liệu lớn như ImageNet (bạn sẽ cần khá nhiều không gian đĩa!). Danh sách bao gồm các tập dữ liệu hình ảnh, tập dữ liệu văn bản (bao gồm các tập dữ liệu dịch thuật), tập dữ liệu âm thanh và video, chuỗi thời gian, và nhiều hơn nữa. Bạn có thể truy cập https://homl.info/tfds để xem danh sách đầy đủ, cùng với mô tả của mỗi tập dữ liệu. Bạn cũng có thể kiểm tra KnowYourData, đây là một công cụ để khám phá và hiểu nhiều tập dữ liệu được TFDS cung cấp.

TFDS không được gói cùng với TensorFlow, nhưng nếu bạn đang chạy trên Colab hoặc nếu bạn đã làm theo hướng dẫn cài đặt tại https://homl.info/install , thì nó đã được cài đặt. Sau đó, bạn có thể nhập `tensorflow_datasets`, thường là dưới dạng `tfds`, sau đó gọi hàm `tfds.load()`, hàm này sẽ tải dữ liệu bạn muốn (trừ khi nó đã được tải trước đó) và trả về dữ liệu dưới dạng một từ điển các tập dữ liệu (thường là một cho huấn luyện và một cho kiểm tra, nhưng điều này tùy thuộc vào tập dữ liệu bạn chọn). Ví dụ, hãy tải MNIST:

```
import tensorflow_datasets as tfds

datasets = tfds.load(name="mnist")
mnist_train, mnist_test = datasets["train"], datasets["test"]
```

Sau đó, bạn có thể áp dụng bất kỳ phép biến đổi nào bạn muốn (thường là xáo trộn, nhóm batch và prefetching), và bạn đã sẵn sàng để huấn luyện mô hình của mình.

Dưới đây là một ví dụ đơn giản:

```
for batch in mnist_train.shuffle(10_000, seed=42).batch(32).prefetch(1):
    images = batch["image"]
    labels = batch["label"]
    # ... làm điều gì đó với hình ảnh và nhãn
```

Lưu ý rằng mỗi mục trong tập dữ liệu là một từ điển chứa cả đặc trưng và nhãn. Nhưng Keras mong đợi mỗi mục là một tuple chứa hai phần tử (một lần nữa, các đặc trưng và nhãn). Bạn có thể biến đổi tập dữ liệu bằng phương thức `map()`, như thế này:

```
mnist_train = mnist_train.shuffle(buffer_size=10_000, seed=42).batch(32)

mnist_train = mnist_train.map(lambda items:
                              (items["image"], items["label"]))
mnist_train = mnist_train.prefetch(1)
```

Nhưng đơn giản hơn là yêu cầu hàm `load()` làm điều này cho bạn bằng cách đặt `as_supervised=True` (rõ ràng điều này chỉ hoạt động đối với các tập dữ liệu được gắn nhãn).

Cuối cùng, TFDS cung cấp một cách tiện lợi để chia dữ liệu bằng đối số `split`. Ví dụ, nếu bạn muốn sử dụng 90% đầu tiên của tập huấn luyện để huấn luyện, 10% còn lại để xác thực, và toàn bộ tập kiểm tra để kiểm tra, thì bạn có thể đặt `split=["train[:90%]", "train[90%:]", "test"]`. Hàm `load()` sẽ trả về cả ba tập hợp. Dưới đây là một ví dụ hoàn chỉnh, tải và chia tập dữ liệu MNIST bằng TFDS, sau đó sử dụng các tập hợp này để huấn luyện và đánh giá một mô hình Keras đơn giản:

```
train_set, valid_set, test_set = tfds.load(
    name="mnist",
    split=["train[:90%]", "train[90%:]", "test"],
    as_supervised=True
)
train_set = train_set.shuffle(buffer_size=10_000,
                              seed=42).batch(32).prefetch(1)
valid_set = valid_set.batch(32).cache()
test_set = test_set.batch(32).cache()
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=5)
test_loss, test_accuracy = model.evaluate(test_set)
```

Chúc mừng, bạn đã đến cuối chương khá kỹ thuật này! Bạn có thể cảm thấy rằng nó hơi xa vời so với vẻ đẹp trừu tượng của mạng nơ-ron, nhưng thực tế là học sâu thường liên quan đến lượng lớn dữ liệu, và biết cách tải, phân tích cú pháp và tiền xử lý nó một cách hiệu quả là một kỹ năng rất quan trọng. Trong chương tiếp theo, chúng ta sẽ xem xét mạng nơ-ron tích chập, đây là một trong những kiến trúc mạng nơ-ron thành công nhất cho xử lý hình ảnh và nhiều ứng dụng khác.

Bài tập

1. Tại sao bạn muốn sử dụng API `tf.data`?
2. Lợi ích của việc chia một tập dữ liệu lớn thành nhiều tệp là gì?
3. Trong quá trình huấn luyện, làm thế nào bạn có thể biết rằng pipeline đầu vào của bạn là nút thắt cổ chai? Bạn có thể làm gì để khắc phục nó?
4. Bạn có thể lưu bất kỳ dữ liệu nhị phân nào vào tệp TFRecord, hay chỉ các bộ đệm giao thức đã được tuần tự hóa?
5. Tại sao bạn lại phải mất công chuyển đổi tất cả dữ liệu của mình sang định dạng protobuf `Example`? Tại sao không sử dụng định nghĩa protobuf của riêng bạn?

6. Khi sử dụng TFRecord, khi nào bạn muốn kích hoạt nén? Tại sao không làm điều đó một cách hệ thống?
7. Dữ liệu có thể được tiền xử lý trực tiếp khi ghi các tệp dữ liệu, hoặc trong pipeline `tf.data`, hoặc trong các lớp tiền xử lý trong mô hình của bạn. Bạn có thể liệt kê một vài ưu và nhược điểm của mỗi tùy chọn không?
8. Kể tên một vài cách phổ biến bạn có thể mã hóa các đặc trưng số nguyên phân loại. Còn văn bản thì sao?
9. Tải tập dữ liệu Fashion MNIST (được giới thiệu trong Chương 10); chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra; xáo trộn tập huấn luyện; và lưu mỗi tập dữ liệu vào nhiều tệp TFRecord. Mỗi bản ghi nên là một protobuf `Example` đã được tuần tự hóa với hai đặc trưng: hình ảnh đã được tuần tự hóa (sử dụng `tf.io.serialize_tensor()` để tuần tự hóa mỗi hình ảnh), và nhãn. Sau đó, sử dụng `tf.data` để tạo một tập dữ liệu hiệu quả cho mỗi tập hợp. Cuối cùng, sử dụng một mô hình Keras để huấn luyện các tập dữ liệu này, bao gồm một lớp tiền xử lý để chuẩn hóa mỗi đặc trưng đầu vào. Cố gắng làm cho pipeline đầu vào hiệu quả nhất có thể, sử dụng TensorBoard để trực quan hóa dữ liệu lập hồ sơ.
10. Trong bài tập này, bạn sẽ tải xuống một tập dữ liệu, chia nó, tạo một `tf.data.Dataset` để tải và tiền xử lý nó một cách hiệu quả, sau đó xây dựng và huấn luyện một mô hình phân loại nhị phân chứa một lớp `Embedding`: a. Tải xuống Large Movie Review Dataset, cái này chứa 50.000 bài đánh giá phim từ Internet Movie Database (IMDb). Dữ liệu được tổ chức trong hai thư mục, `train` và `test`, mỗi thư mục chứa một thư mục con `pos` với 12.500 bài đánh giá tích cực và một thư mục con `neg` với 12.500 bài đánh giá tiêu cực. Mỗi bài đánh giá được lưu trữ trong một tệp văn bản riêng biệt. Có các tệp và thư mục khác (bao gồm các phiên bản bag-of-words đã được tiền xử lý), nhưng chúng ta sẽ bỏ qua chúng trong bài tập này. b. Chia tập kiểm tra thành một tập xác thực (15.000) và một tập kiểm tra (10.000). c. Sử dụng `tf.data` để tạo một tập dữ liệu hiệu quả cho mỗi tập hợp. d. Tạo một mô hình phân loại nhị phân, sử dụng lớp `TextVectorization` để tiền xử lý mỗi bài đánh giá. e. Thêm một lớp `Embedding` và tính toán embedding trung bình cho mỗi bài đánh giá, nhân với căn bậc hai của số từ (xem Chương 16). Embedding trung bình đã được điều chỉnh tỷ lệ này sau đó có thể được truyền cho phần còn lại của mô hình của bạn. f. Huấn luyện mô hình và xem độ chính xác bạn đạt được. Cố gắng tối ưu hóa các pipeline của bạn để quá trình huấn luyện nhanh nhất có thể. g. Sử dụng TFDS để tải cùng tập dữ liệu dễ dàng hơn: `tfds.load("imdb_reviews")`.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 14. THỊ GIÁC MÁY TÍNH CHUYÊN SÂU SỬ DỤNG MẠNG NƠ-RON TÍCH CHẬP

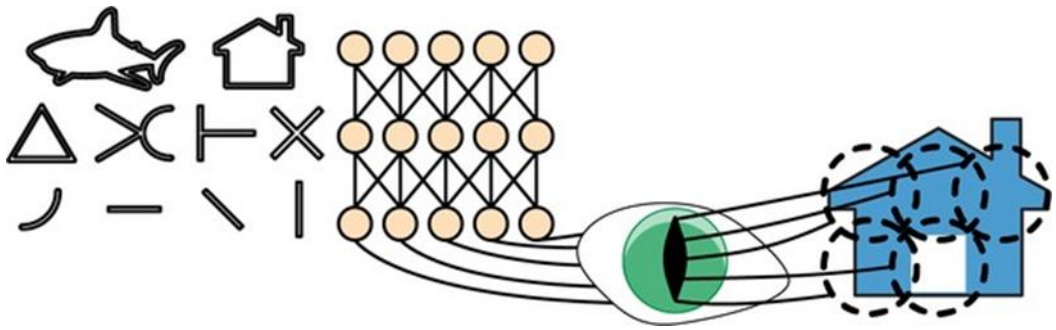
Mặc dù siêu máy tính Deep Blue của IBM đã đánh bại nhà vô địch cờ vua thế giới Garry Kasparov vào năm 1996, nhưng phải đến gần đây máy tính mới có thể thực hiện một cách đáng tin cậy các tác vụ tưởng chừng như đơn giản như phát hiện một chú chó con trong ảnh hoặc nhận dạng lời nói. Tại sao những tác vụ này lại dễ dàng đối với con người chúng ta đến vậy? Câu trả lời nằm ở chỗ nhận thức phần lớn diễn ra bên ngoài phạm vi ý thức của chúng ta, trong các mô-đun thị giác, thính giác và các giác quan khác chuyên biệt trong não bộ. Khi thông tin giác quan đến ý thức của chúng ta, nó đã được trang bị các tính năng cấp cao; ví dụ, khi bạn nhìn vào một bức ảnh về một chú chó con đáng yêu, bạn không thể không nhìn thấy chú chó con, không nhận thấy sự đáng yêu của nó. Bạn cũng không thể giải thích làm thế nào bạn nhận ra một chú chó con đáng yêu; điều đó chỉ đơn giản là hiển nhiên đối với bạn. Do đó, chúng ta không thể tin vào trải nghiệm chủ quan của mình: nhận thức không hề đơn giản chút nào, và để hiểu nó, chúng ta phải xem xét cách các mô-đun giác quan của chúng ta hoạt động.

Mạng nơ-ron tích chập (CNN) ra đời từ nghiên cứu về vỏ não thị giác của não bộ, và chúng đã được sử dụng trong nhận dạng hình ảnh máy tính từ những năm 1980. Trong 10 năm qua, nhờ sự gia tăng sức mạnh tính toán, lượng dữ liệu huấn luyện có sẵn và các thủ thuật được trình bày trong Chương 11 để huấn luyện các mạng sâu, CNN đã đạt được hiệu suất siêu phàm trong một số tác vụ thị giác phức tạp. Chúng cung cấp sức mạnh cho các dịch vụ tìm kiếm hình ảnh, xe tự lái, hệ thống phân loại video tự động và nhiều hơn nữa. Hơn nữa, CNN không chỉ giới hạn trong nhận thức thị giác: chúng còn thành công trong nhiều tác vụ khác, chẳng hạn như nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên. Tuy nhiên, chúng ta sẽ tập trung vào các ứng dụng thị giác trong chương này.

Trong chương này, chúng ta sẽ khám phá nguồn gốc của CNN, các khối xây dựng của chúng trông như thế nào và cách triển khai chúng bằng Keras. Sau đó, chúng ta sẽ thảo luận về một số kiến trúc CNN tốt nhất, cũng như các tác vụ thị giác khác, bao gồm phát hiện đối tượng (phân loại nhiều đối tượng trong một hình ảnh và đặt các hộp bao quanh chúng) và phân đoạn ngữ nghĩa (phân loại từng pixel theo lớp của đối tượng mà nó thuộc về).

Kiến trúc của vỏ não thị giác

David H. Hubel và Torsten Wiesel đã thực hiện một loạt thí nghiệm trên mèo vào năm 1958 và 1959 (và vài năm sau đó trên khỉ), đưa ra những hiểu biết quan trọng về cấu trúc của vỏ não thị giác (các tác giả đã nhận giải Nobel Sinh lý học hoặc Y học năm 1981 cho công trình của họ). Đặc biệt, họ đã chỉ ra rằng nhiều nơ-ron trong vỏ não thị giác có một trường tiếp nhận cục bộ nhỏ, có nghĩa là chúng chỉ phản ứng với các kích thích thị giác nằm trong một vùng giới hạn của trường thị giác (xem Hình 14-1, trong đó các trường tiếp nhận cục bộ của năm nơ-ron được biểu thị bằng các vòng tròn đứt nét). Các trường tiếp nhận của các nơ-ron khác nhau có thể chồng chéo lên nhau, và cùng nhau chúng bao phủ toàn bộ trường thị giác.



Hình 14-1. Các nơ-ron sinh học trong vỏ não thị giác phản ứng với các mẫu cụ thể trong các vùng nhỏ của trường thị giác được gọi là trường tiếp nhận; khi tín hiệu thị giác đi qua các mô-đun não liên tiếp, các nơ-ron phản ứng với các mẫu phức tạp hơn trong các trường tiếp nhận lớn hơn.

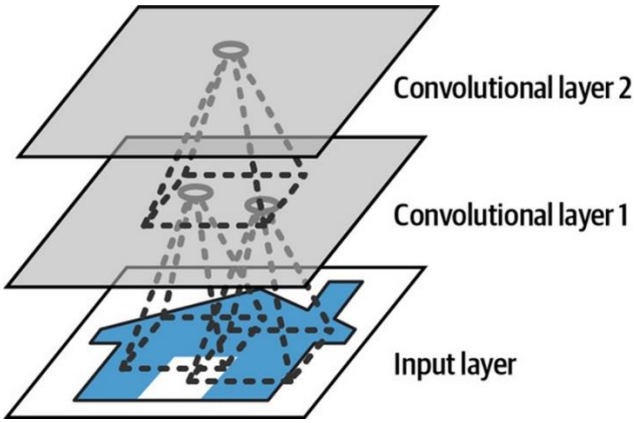
Hơn nữa, các tác giả đã chỉ ra rằng một số nơ-ron chỉ phản ứng với hình ảnh các đường ngang, trong khi những nơ-ron khác chỉ phản ứng với các đường có hướng khác nhau (hai nơ-ron có thể có cùng trường tiếp nhận nhưng phản ứng với các hướng đường khác nhau). Họ cũng nhận thấy rằng một số nơ-ron có trường tiếp nhận lớn hơn, và chúng phản ứng với các mẫu phức tạp hơn là sự kết hợp của các mẫu cấp thấp hơn. Những quan sát này đã dẫn đến ý tưởng rằng các nơ-ron cấp cao hơn dựa trên đầu ra của các nơ-ron cấp thấp hơn lân cận (trong Hình 14-1, lưu ý rằng mỗi nơ-ron chỉ kết nối với các nơ-ron gần đó từ lớp trước). Kiến trúc mạnh mẽ này có khả năng phát hiện tất cả các loại mẫu phức tạp ở bất kỳ khu vực nào của trường thị giác.

Những nghiên cứu về vỏ não thị giác này đã truyền cảm hứng cho neocognitron, được giới thiệu vào năm 1980, sau đó dần phát triển thành cái mà chúng ta hiện gọi là mạng nơ-ron tích chập. Một cột mốc quan trọng là bài báo năm 1998 của Yann LeCun et al. đã giới thiệu kiến trúc LeNet-5 nổi tiếng, được các ngân hàng sử dụng rộng rãi để nhận dạng chữ số viết tay trên séc. Kiến trúc này có một số khối xây dựng mà bạn đã biết, chẳng hạn như các lớp kết nối đầy đủ và các hàm kích hoạt sigmoid, nhưng nó cũng giới thiệu hai khối xây dựng mới: các lớp tích chập và các lớp gộp. Bây giờ chúng ta hãy xem xét chúng.

Các lớp tích chập

Khối xây dựng quan trọng nhất của CNN là lớp tích chập: các nơ-ron trong lớp tích chập đầu tiên không được kết nối với mọi pixel trong hình ảnh đầu vào (như chúng đã ở trong các lớp được thảo luận trong các chương trước), mà chỉ kết nối với các pixel trong trường tiếp nhận của chúng (xem Hình 14-2).

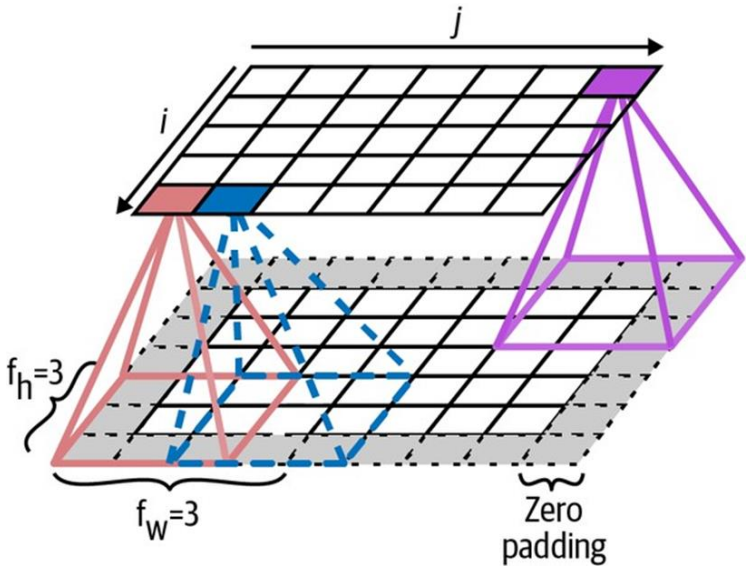
Đến lượt mình, mỗi nơ-ron trong lớp tích chập thứ hai chỉ được kết nối với các nơ-ron nằm trong một hình chữ nhật nhỏ ở lớp đầu tiên. Kiến trúc này cho phép mạng tập trung vào các đặc điểm cấp thấp nhỏ ở lớp ẩn đầu tiên, sau đó tập hợp chúng thành các đặc điểm cấp cao lớn hơn ở lớp ẩn tiếp theo, v.v. Cấu trúc phân cấp này phổ biến trong các hình ảnh thực tế, đây là một trong những lý do tại sao CNN hoạt động rất tốt cho nhận dạng hình ảnh.



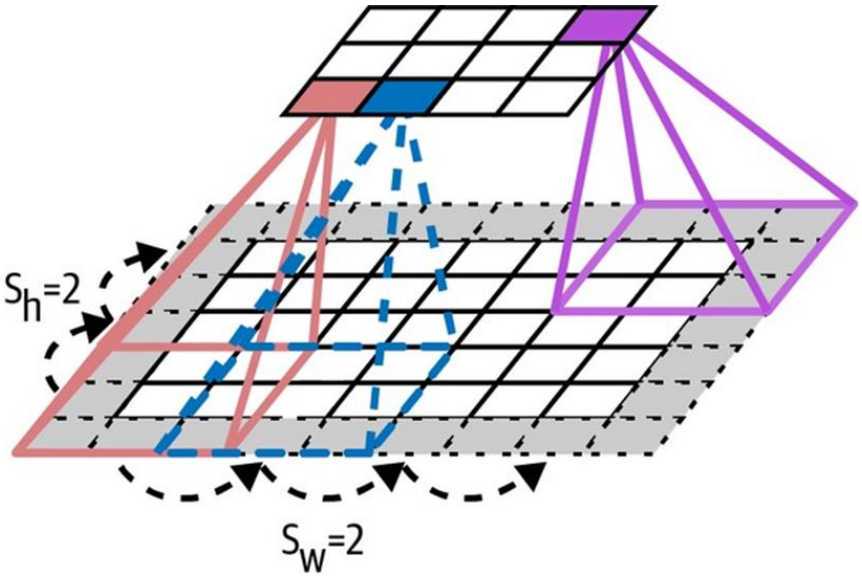
Hình 14-2. Các lớp CNN với trường tiếp nhận cục bộ hình chữ nhật.

Một nơ-ron nằm ở hàng i, cột j của một lớp nhất định được kết nối với đầu ra của các nơ-ron trong lớp trước nằm ở hàng i đến i + fh – 1, cột j đến j + fw – 1, trong đó fh và fw là chiều cao và chiều rộng của trường tiếp nhận (xem Hình 14-3). Để một lớp có cùng chiều cao và chiều rộng với lớp trước, việc thêm các số 0 xung quanh đầu vào là phổ biến, như thể hiện trong sơ đồ. Điều này được gọi là zero padding.

Cũng có thể kết nối một lớp đầu vào lớn với một lớp nhỏ hơn nhiều bằng cách giãn cách các trường tiếp nhận, như thể hiện trong Hình 14-4. Điều này làm giảm đáng kể độ phức tạp tính toán của mô hình. Kích thước bước ngang hoặc dọc từ trường tiếp nhận này sang trường tiếp nhận tiếp theo được gọi là stride. Trong sơ đồ, một lớp đầu vào 5 × 7 (cộng với zero padding) được kết nối với một lớp 3 × 4, sử dụng các trường tiếp nhận 3 × 3 và một stride là 2 (trong ví dụ này, stride giống nhau ở cả hai hướng, nhưng không nhất thiết phải như vậy). Một nơ-ron nằm ở hàng i, cột j trong lớp trên được kết nối với đầu ra của các nơ-ron trong lớp trước nằm ở hàng i × sh đến i × sh + fh – 1, cột j × sw đến j × sw + fw – 1, trong đó sh và sw là các strides dọc và ngang.



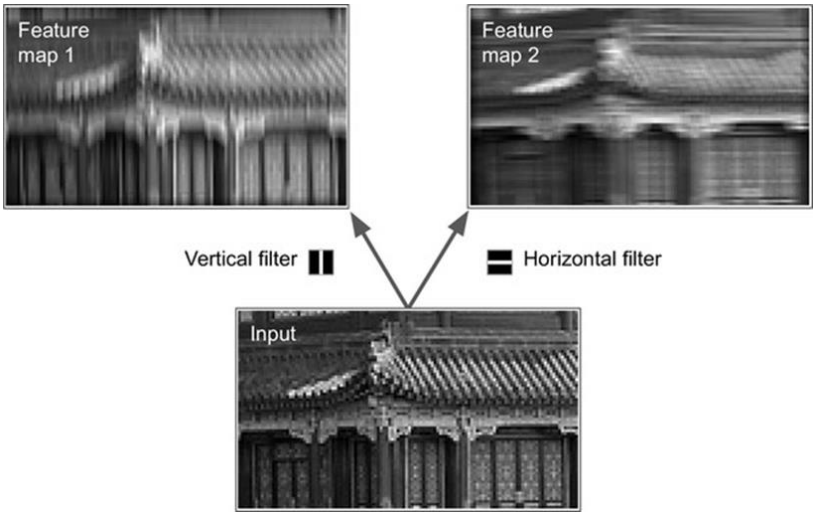
Hình 14-3. Các kết nối giữa các lớp và zero padding.



Hình 14-4. Giảm chiều bằng cách sử dụng stride là 2.

Bộ lọc

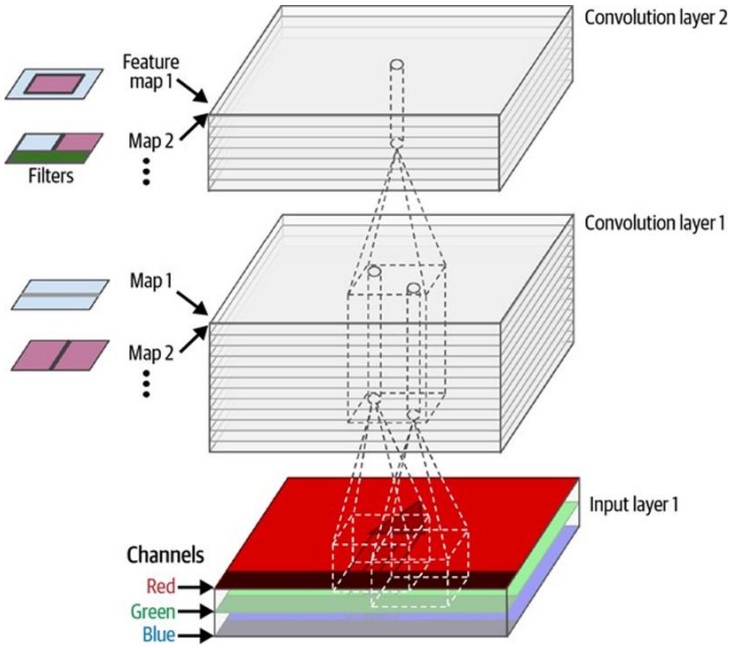
Các trọng số của một nơ-ron có thể được biểu diễn dưới dạng một hình ảnh nhỏ có kích thước bằng trường tiếp nhận. Ví dụ, Hình 14-5 cho thấy hai tập hợp trọng số có thể có, được gọi là bộ lọc (hoặc kernel tích chập, hoặc chỉ là kernel). Bộ lọc đầu tiên được biểu diễn dưới dạng một hình vuông màu đen với một đường thẳng đứng màu trắng ở giữa (đó là một ma trận 7 × 7 chứa toàn số 0 ngoại trừ cột trung tâm, chứa toàn số 1); các nơ-ron sử dụng các trọng số này sẽ bỏ qua mọi thứ trong trường tiếp nhận của chúng ngoại trừ đường thẳng đứng trung tâm (vì tất cả các đầu vào sẽ được nhân với 0, ngoại trừ những đầu vào nằm trên đường thẳng đứng trung tâm). Bộ lọc thứ hai là một hình vuông màu đen với một đường thẳng ngang màu trắng ở giữa. Các nơ-ron sử dụng các trọng số này sẽ bỏ qua mọi thứ trong trường tiếp nhận của chúng ngoại trừ đường thẳng ngang trung tâm.



Hình 14-5. Áp dụng hai bộ lọc khác nhau để có được hai bản đồ đặc trưng.

Bây giờ, nếu tất cả các nơ-ron trong một lớp sử dụng cùng một bộ lọc đường thẳng đứng (và cùng một thuật ngữ bias), và bạn cung cấp cho mạng hình ảnh đầu vào được hiển thị trong Hình 14-5 (hình ảnh dưới cùng), lớp đó sẽ xuất ra hình ảnh trên cùng bên trái. Lưu ý rằng các đường thẳng đứng màu trắng được tăng cường trong khi phần còn lại bị làm mờ. Tương tự, hình ảnh trên cùng bên phải là những gì bạn nhận được nếu tất cả các nơ-ron sử dụng cùng một bộ lọc đường thẳng ngang; lưu ý rằng các đường thẳng ngang màu trắng được tăng cường trong khi phần còn lại bị làm mờ. Do đó, một lớp đầy các nơ-ron sử dụng cùng một bộ lọc sẽ xuất ra một bản đồ đặc trưng, làm nổi bật các vùng trong một hình ảnh kích hoạt bộ lọc đó nhiều nhất. Nhưng đừng lo lắng, bạn sẽ không phải xác định các bộ lọc theo cách thủ công: thay vào đó, trong quá trình huấn luyện, lớp tích chập sẽ tự động học các bộ lọc hữu ích nhất cho tác vụ của nó, và các lớp phía trên sẽ học cách kết hợp chúng thành các mẫu phức tạp hơn.

Để thuận tiện, tôi đã trình bày đầu ra của mỗi lớp tích chập dưới dạng lớp 2D, nhưng trên thực tế, một lớp tích chập có nhiều bộ lọc (bạn quyết định bao nhiêu) và xuất ra một bản đồ đặc trưng cho mỗi bộ lọc, vì vậy nó được biểu diễn chính xác hơn ở dạng 3D (xem Hình 14-6). Nó có một nơ-ron trên mỗi pixel trong mỗi bản đồ đặc trưng, và tất cả các nơ-ron trong một bản đồ đặc trưng nhất định chia sẻ cùng các tham số (tức là cùng một kernel và thuật ngữ bias). Các nơ-ron trong các bản đồ đặc trưng khác nhau sử dụng các tham số khác nhau. Trường tiếp nhận của một nơ-ron giống như đã mô tả trước đó, nhưng nó mở rộng trên tất cả các bản đồ đặc trưng của lớp trước. Tóm lại, một lớp tích chập đồng thời áp dụng nhiều bộ lọc có thể huấn luyện cho đầu vào của nó, giúp nó có khả năng phát hiện nhiều đặc trưng ở bất cứ đâu trong đầu vào của nó.



Hình 14-6. Hai lớp tích chập với nhiều bộ lọc (kernels) mỗi lớp, xử lý một hình ảnh màu với ba kênh màu; mỗi lớp tích chập xuất ra một bản đồ đặc trưng cho mỗi bộ lọc.

Hình ảnh đầu vào cũng bao gồm nhiều lớp con: một lớp cho mỗi kênh màu. Như đã đề cập trong Chương 9, thường có ba kênh: đỏ, xanh lá cây và xanh lam (RGB). Hình ảnh thang độ xám chỉ có một kênh, nhưng một số hình ảnh có thể có nhiều kênh hơn—ví dụ, hình ảnh vệ tinh chụp các tần số ánh sáng bổ sung (chẳng hạn như hồng ngoại).

Cụ thể, một nơ-ron nằm ở hàng i, cột j của bản đồ đặc trưng k trong một lớp tích chập l nhất định được kết nối với đầu ra của các nơ-ron trong lớp trước l – 1, nằm ở hàng i × sh đến i × sh + fh – 1 và cột j × sw đến j × sw + fw – 1, trên tất cả các bản đồ đặc trưng (trong lớp l – 1). Lưu ý rằng, trong một lớp, tất cả các nơ-ron nằm ở cùng một hàng i và cột j nhưng trong các bản đồ đặc trưng khác nhau được kết nối với đầu ra của chính xác cùng các nơ-ron trong lớp trước.

Phương trình 14-1 tóm tắt các giải thích trên trong một phương trình toán học lớn: nó cho thấy cách tính đầu ra của một nơ-ron nhất định trong một lớp tích chập. Nó hơi khó nhìn do tất cả các chỉ số khác nhau, nhưng tất cả những gì nó làm là tính tổng trọng số của tất cả các đầu vào, cộng với thuật ngữ bias.

Phương trình 14-1. Tính toán đầu ra của một nơ-ron trong một lớp tích chập

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_{n'}-1} x_{i',j',k'} \times w_{u,v,k',k} \quad \text{with} \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

Trong phương trình này:

- z_i, j, k là đầu ra của nơ-ron nằm ở hàng i, cột j trong bản đồ đặc trưng k của lớp tích chập (lớp l).
- Như đã giải thích trước đó, s_h và s_w là các stride dọc và ngang, f_h và f_w là chiều cao và chiều rộng của trường tiếp nhận, và f_n' là số lượng bản đồ đặc trưng trong lớp trước (lớp l – 1).
- x_i', j', k' là đầu ra của nơ-ron nằm ở lớp l – 1, hàng i', cột j', bản đồ đặc trưng k' (hoặc kênh k' nếu lớp trước là lớp đầu vào).
- b_k là thuật ngữ bias cho bản đồ đặc trưng k (trong lớp l). Bạn có thể nghĩ nó như một nút điều chỉnh độ sáng tổng thể của bản đồ đặc trưng k.
- w_u, v, k', k là trọng số kết nối giữa bất kỳ nơ-ron nào trong bản đồ đặc trưng k của lớp l và đầu vào của nó nằm ở hàng u, cột v (so với trường tiếp nhận của nơ-ron), và bản đồ đặc trưng k'.

Hãy xem cách tạo và sử dụng một lớp tích chập bằng Keras.

Triển khai các lớp tích chập với Keras

Đầu tiên, hãy tải và tiền xử lý một vài hình ảnh mẫu, sử dụng hàm `load_sample_image()` của Scikit-Learn và các lớp `CenterCrop` và `Rescaling` của Keras (tất cả đều đã được giới thiệu trong Chương 13):

```
images = load_sample_images()["images"]
images = tf.keras.layers.CenterCrop(height=70, width=120)(images)
images = tf.keras.layers.Rescaling(scale=1 / 255)(images)
```

Hãy xem hình dạng của tensor `images`:

```
>>> images.shape
TensorShape([2, 70, 120, 3])
```

Ối, đó là một tensor 4D; chúng ta chưa từng thấy điều này trước đây! Tất cả các chiều này có nghĩa là gì? Vâng, có hai hình ảnh mẫu, điều này giải thích chiều đầu tiên. Sau đó, mỗi hình ảnh là 70 × 120, vì đó là kích thước chúng ta đã chỉ định khi tạo lớp `CenterCrop` (các hình ảnh gốc là 427 × 640). Điều này giải thích chiều thứ hai và thứ ba. Và cuối cùng, mỗi pixel chứa một giá trị trên mỗi kênh màu, và có ba kênh—đỏ, xanh lá cây và xanh lam—điều này giải thích chiều cuối cùng.

Bây giờ hãy tạo một lớp tích chập 2D và đưa các hình ảnh này vào để xem kết quả. Đối với điều này, Keras cung cấp một lớp `Convolution2D`, bí danh `Conv2D`. Bên dưới, lớp này dựa vào thao tác `tf.nn.conv2d()` của TensorFlow. Hãy tạo một lớp tích chập với 32 bộ lọc, mỗi bộ lọc có kích thước 7 × 7 (sử dụng `kernel_size=7`, tương đương với việc sử dụng `kernel_size=(7, 7)`), và áp dụng lớp này cho lô nhỏ gồm hai hình ảnh của chúng ta:

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7)
fmaps = conv_layer(images)
```

Bây giờ hãy xem hình dạng của đầu ra:

```
>>> fmaps.shape
TensorShape([2, 64, 114, 32])
```

Hình dạng đầu ra tương tự như hình dạng đầu vào, với hai khác biệt chính. Đầu tiên, có 32 kênh thay vì 3. Điều này là do chúng ta đã đặt `filters=32`, vì vậy chúng ta nhận được 32 bản đồ đặc trưng đầu ra: thay vì cường độ của màu đỏ, xanh lá cây và xanh lam tại mỗi vị trí, bây giờ chúng ta có cường độ của mỗi đặc trưng tại mỗi vị trí.

Thứ hai, chiều cao và chiều rộng đều giảm đi 6 pixel. Điều này là do lớp `Conv2D` không sử dụng bất kỳ zero-padding nào theo mặc định, điều đó có nghĩa là chúng ta mất một vài pixel ở các cạnh của bản đồ đặc trưng đầu ra, tùy thuộc vào kích thước của các bộ lọc. Trong trường hợp này, vì kích thước kernel là 7, chúng ta mất 6 pixel theo chiều ngang và 6 pixel theo chiều dọc (tức là 3 pixel ở mỗi bên).

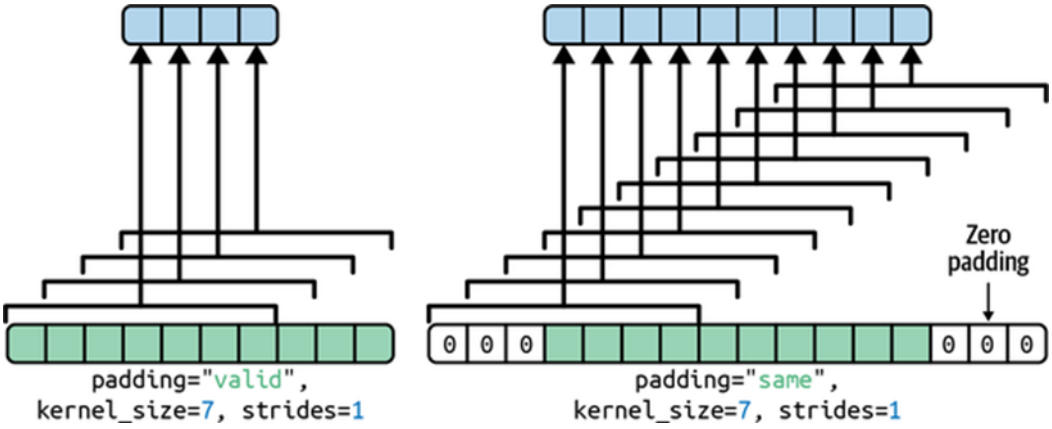
Nếu thay vào đó chúng ta đặt `padding="same"`, thì đầu vào sẽ được đệm bằng đủ số 0 ở tất cả các phía để đảm bảo rằng các bản đồ đặc trưng đầu ra có cùng kích thước với đầu vào (do đó có tên của tùy chọn này):

```
>>> conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7,
...                                     padding="same")
...
>>> fmaps = conv_layer(images)
```

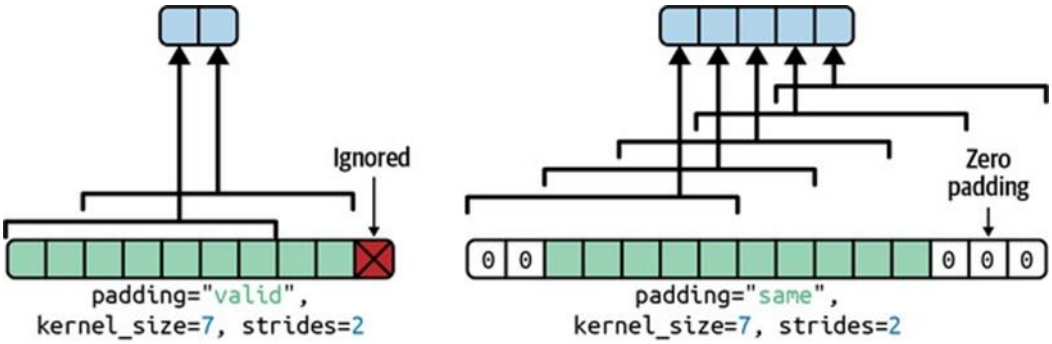
```
>>> fmaps.shape
TensorShape([2, 70, 120, 32])
```

Hai tùy chọn padding này được minh họa trong Hình 14-7. Để đơn giản, chỉ có chiều ngang được hiển thị ở đây, nhưng tất nhiên logic tương tự cũng áp dụng cho chiều dọc.

Nếu stride lớn hơn 1 (theo bất kỳ hướng nào), thì kích thước đầu ra sẽ không bằng kích thước đầu vào, ngay cả khi `padding="same"`. Ví dụ, nếu bạn đặt `strides=2` (hoặc tương đương `strides=(2, 2)`), thì các bản đồ đặc trưng đầu ra sẽ là 35 × 60: giảm một nửa cả theo chiều dọc và chiều ngang. Hình 14-8 cho thấy điều gì xảy ra khi `strides=2`, với cả hai tùy chọn padding.



Hình 14-7. Hai tùy chọn padding, khi strides=1.



Hình 14-8. Với strides lớn hơn 1, đầu ra nhỏ hơn nhiều ngay cả khi sử dụng padding “same” (và padding “valid” có thể bỏ qua một số đầu vào).

Nếu bạn tò mò, đây là cách tính kích thước đầu ra:

- Với `padding="valid"`, nếu chiều rộng của đầu vào là `ih`, thì chiều rộng đầu ra bằng `(ih - fh + sh) / sh`, làm tròn xuống. Nhắc lại rằng `fh` là chiều rộng kernel, và `sh` là stride ngang. Bất kỳ phần dư nào trong phép chia đều tương ứng với các cột bị bỏ qua ở phía bên phải của hình ảnh đầu vào. Logic tương tự có thể được sử dụng để tính chiều cao đầu ra và bất kỳ hàng bị bỏ qua nào ở cuối hình ảnh.

- Với `padding="same"`, chiều rộng đầu ra bằng `ih / sh`, làm tròn lên. Để điều này có thể thực hiện được, số cột 0 thích hợp được đệm vào bên trái và bên phải của hình ảnh đầu vào (số lượng bằng nhau nếu có thể, hoặc chỉ nhiều hơn một cột ở phía bên phải). Giả sử chiều rộng đầu ra là `ow`, thì số cột 0 được đệm là `(ow – 1) × sh + fh – ih`. Một lần nữa, logic tương tự có thể được sử dụng để tính chiều cao đầu ra và số hàng được đệm.

Bây giờ hãy xem xét các trọng số của lớp (được ký hiệu là w_u, v, k', k và b_k trong Phương trình 14-1). Giống như một lớp `Dense`, một lớp `Conv2D` chứa tất cả các trọng số của lớp, bao gồm các kernel và bias. Các kernel được khởi tạo ngẫu nhiên, trong khi các bias được khởi tạo bằng 0. Các trọng số này có thể truy cập được dưới dạng các biến TF thông qua thuộc tính `weights`, hoặc dưới dạng các mảng NumPy thông qua phương thức `get_weights()`:

```
>>> kernels, biases = conv_layer.get_weights()
>>> kernels.shape
(7, 7, 3, 32)
>>> biases.shape
(32,)
```

Mảng `kernels` là 4D, và hình dạng của nó là `[kernel_height, kernel_width, input_channels, output_channels]`. Mảng `biases` là 1D, với hình dạng `[output_channels]`. Số lượng kênh đầu ra bằng số lượng bản đồ đặc trưng đầu ra, cũng bằng số lượng bộ lọc.

Quan trọng nhất, lưu ý rằng chiều cao và chiều rộng của hình ảnh đầu vào không xuất hiện trong hình dạng của kernel: điều này là do tất cả các nơ-ron trong các bản đồ đặc trưng đầu ra chia sẻ cùng một trọng số, như đã giải thích trước đó. Điều này có nghĩa là bạn có thể đưa hình ảnh có bất kỳ kích thước nào vào lớp này, miễn là chúng ít nhất lớn bằng các kernel, và nếu chúng có đúng số lượng kênh (ba trong trường hợp này).

Cuối cùng, bạn thường muốn chỉ định một hàm kích hoạt (chẳng hạn như ReLU) khi tạo một lớp `Conv2D`, và cũng chỉ định trình khởi tạo kernel tương ứng (chẳng hạn như He initialization). Điều này cũng vì lý do tương tự như đối với các lớp `Dense`: một lớp tích chập thực hiện một phép toán tuyến tính, vì vậy nếu bạn xếp chồng nhiều lớp tích chập mà không có bất kỳ hàm kích hoạt nào, tất cả chúng sẽ tương đương với một lớp tích chập duy nhất, và chúng sẽ không thể học được bất cứ điều gì thực sự phức tạp.

Như bạn có thể thấy, các lớp tích chập có khá nhiều siêu tham số: `filters`, `kernel_size`, `padding`, `strides`, `activation`, `kernel_initializer`, v.v. Như mọi khi, bạn có thể sử dụng kiểm định chéo để tìm ra các giá trị siêu tham số phù hợp, nhưng điều này rất tốn thời gian. Chúng ta sẽ thảo luận về các kiến trúc CNN phổ biến sau này trong chương này, để cung cấp cho bạn một số ý tưởng về giá trị siêu tham số nào hoạt động tốt nhất trong thực tế.

Yêu cầu về bộ nhớ

Một thách thức khác với CNN là các lớp tích chập yêu cầu lượng RAM khổng lồ. Điều này đặc biệt đúng trong quá trình huấn luyện, vì lượt ngược của lan truyền ngược yêu cầu tất cả các giá trị trung gian được tính toán trong lượt thuận.

Ví dụ, hãy xem xét một lớp tích chập với 200 bộ lọc 5 × 5, với stride 1 và padding “same”. Nếu đầu vào là hình ảnh RGB 150 × 100 (ba kênh), thì số lượng tham số là (5 × 5 × 3 + 1) × 200 = 15.200 (cái + 1 tương ứng với các thuật ngữ bias), khá nhỏ so với một lớp kết nối đầy đủ. Tuy nhiên, mỗi trong số 200 bản đồ đặc trưng chứa 150 × 100 nơ-ron, và mỗi nơ-ron này cần tính tổng trọng số của 5 × 5 × 3 = 75 đầu vào của nó: tổng cộng là 225 triệu phép nhân dấu phẩy động. Không tệ bằng một lớp kết nối đầy đủ, nhưng vẫn khá tốn kém về mặt tính toán.

Hơn nữa, nếu các bản đồ đặc trưng được biểu diễn bằng số dấu phẩy động 32 bit, thì đầu ra của lớp tích chập sẽ chiếm 200 × 150 × 100 × 32 = 96 triệu bit (12 MB) RAM. Và đó chỉ là cho một trường hợp—nếu một lô huấn luyện chứa 100 trường hợp, thì lớp này sẽ sử dụng tới 1,2 GB RAM!

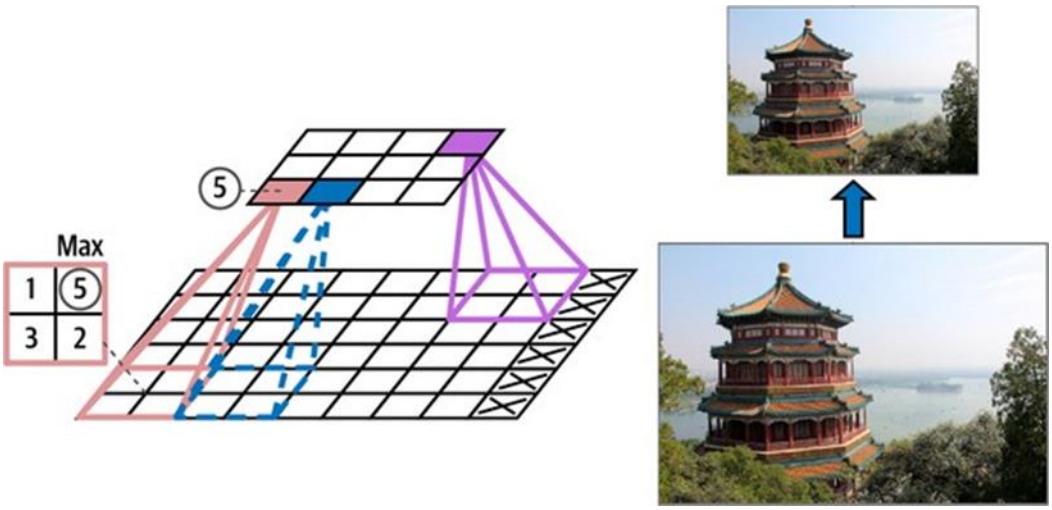
Trong quá trình suy luận (tức là khi đưa ra dự đoán cho một trường hợp mới), RAM bị chiếm bởi một lớp có thể được giải phóng ngay sau khi lớp tiếp theo đã được tính toán, vì vậy bạn chỉ cần lượng RAM cần thiết cho hai lớp liên tiếp. Nhưng trong quá trình huấn luyện, mọi thứ được tính toán trong lượt thuận cần phải được bảo toàn cho lượt ngược, vì vậy lượng RAM cần thiết là (ít nhất) tổng lượng RAM cần thiết cho tất cả các lớp.

Bây giờ chúng ta hãy xem khối xây dựng phổ biến thứ hai của CNN: lớp gộp (pooling layer).

Các lớp gộp

Khi bạn hiểu cách các lớp tích chập hoạt động, các lớp gộp khá dễ nắm bắt. Mục tiêu của chúng là lấy mẫu con (tức là thu nhỏ) hình ảnh đầu vào để giảm tải tính toán, sử dụng bộ nhớ và số lượng tham số (do đó hạn chế rủi ro quá khớp).

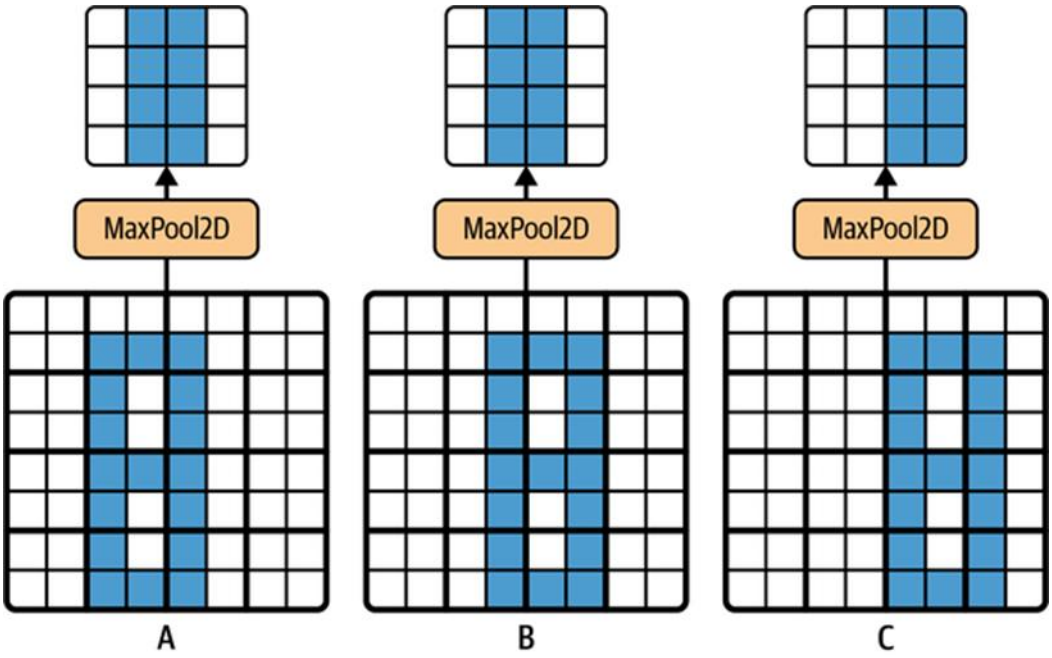
Giống như trong các lớp tích chập, mỗi nơ-ron trong một lớp gộp được kết nối với đầu ra của một số lượng hạn chế các nơ-ron trong lớp trước, nằm trong một trường tiếp nhận hình chữ nhật nhỏ. Bạn phải xác định kích thước của nó, stride và loại padding, giống như trước đây. Tuy nhiên, một nơ-ron gộp không có trọng số; tất cả những gì nó làm là tổng hợp các đầu vào bằng cách sử dụng một hàm tổng hợp như max hoặc mean. Hình 14-9 cho thấy một lớp gộp max (max pooling layer), là loại lớp gộp phổ biến nhất. Trong ví dụ này, chúng ta sử dụng một kernel gộp 2 × 2, với stride là 2 và không padding. Chỉ giá trị đầu vào max trong mỗi trường tiếp nhận mới được đưa đến lớp tiếp theo, trong khi các đầu vào khác bị loại bỏ. Ví dụ, trong trường tiếp nhận phía dưới bên trái trong Hình 14-9, các giá trị đầu vào là 1, 5, 3, 2, vì vậy chỉ giá trị max là 5 được truyền đến lớp tiếp theo. Do stride là 2, hình ảnh đầu ra có chiều cao bằng một nửa và chiều rộng bằng một nửa hình ảnh đầu vào (làm tròn xuống vì chúng ta không sử dụng padding).



Hình 14-9. Lớp gộp max (kernel gộp 2 × 2, stride 2, không padding).

Ngoài việc giảm tính toán, sử dụng bộ nhớ và số lượng tham số, một lớp gộp max còn giới thiệu một mức độ bất biến đối với các phép dịch chuyển nhỏ, như thể hiện trong Hình 14-10. Ở đây chúng ta giả định rằng các pixel sáng có giá trị thấp hơn các pixel tối, và chúng ta xem xét ba hình ảnh (A, B, C) đi qua một lớp gộp max với kernel 2 × 2 và stride 2. Hình ảnh B và C giống như hình ảnh A, nhưng dịch chuyển một và hai pixel sang phải. Như bạn có thể thấy, đầu ra của lớp gộp max cho hình ảnh A và B là giống hệt nhau. Đây là ý nghĩa của bất biến dịch chuyển. Đối với hình ảnh C, đầu ra khác: nó được dịch chuyển một pixel sang phải (nhưng vẫn có 50% bất biến). Bằng cách chèn một lớp gộp max sau mỗi vài lớp trong một CNN, có thể đạt được một mức độ bất biến dịch chuyển ở quy mô lớn hơn. Hơn nữa, gộp max cung cấp một lượng nhỏ bất biến xoay và một chút bất biến tỷ lệ. Bất biến như vậy (ngay cả khi nó bị hạn chế) có thể hữu ích trong các trường hợp mà dự đoán không nên phụ thuộc vào các chi tiết này, chẳng hạn như trong các tác vụ phân loại.

Tuy nhiên, gộp max cũng có một số nhược điểm. Nó rõ ràng là rất phá hủy: ngay cả với một kernel 2 × 2 nhỏ và stride 2, đầu ra sẽ nhỏ hơn hai lần theo cả hai hướng (vì vậy diện tích của nó sẽ nhỏ hơn bốn lần), đơn giản là loại bỏ 75% giá trị đầu vào. Và trong một số ứng dụng, bất biến không mong muốn. Lấy phân đoạn ngữ nghĩa (nhiệm vụ phân loại từng pixel trong một hình ảnh theo đối tượng mà pixel đó thuộc về, chúng ta sẽ khám phá sau trong chương này): rõ ràng, nếu hình ảnh đầu vào được dịch chuyển một pixel sang phải, đầu ra cũng phải được dịch chuyển một pixel sang phải. Mục tiêu trong trường hợp này là đồng biến, không phải bất biến: một thay đổi nhỏ đối với đầu vào sẽ dẫn đến một thay đổi nhỏ tương ứng trong đầu ra.



Hình 14-10. Bất biến đối với các phép dịch chuyển nhỏ.

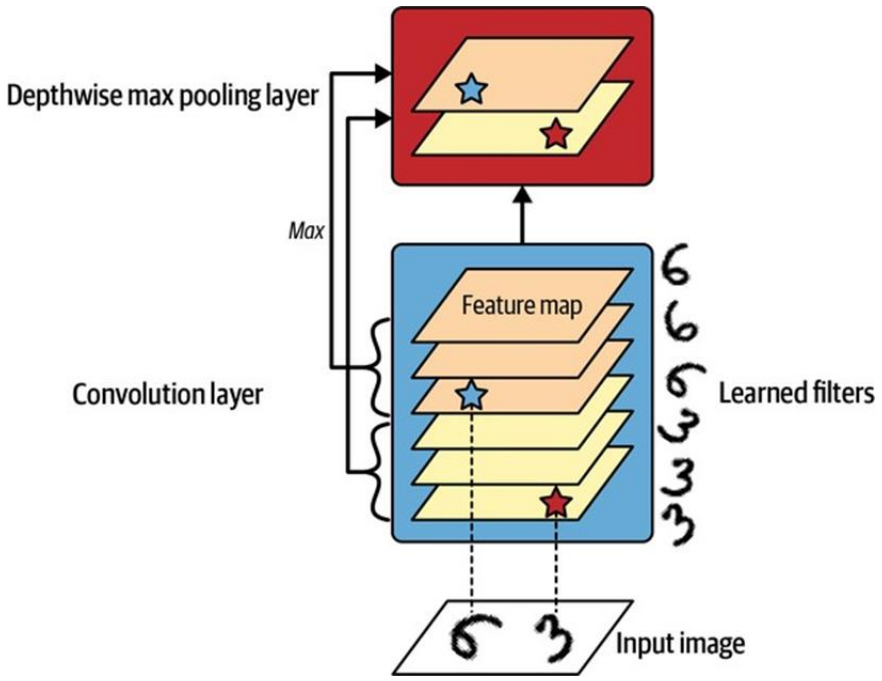
Triển khai các lớp gộp với Keras

Mã sau đây tạo một lớp `MaxPooling2D`, bí danh `MaxPool2D`, sử dụng một kernel 2 × 2. Các stride mặc định là kích thước kernel, vì vậy lớp này sử dụng stride là 2 (ngang và dọc). Theo mặc định, nó sử dụng padding "valid" (tức là không padding gì cả):

```
max_pool = tf.keras.layers.MaxPool2D(pool_size=2)
```

Để tạo một lớp gộp trung bình, chỉ cần sử dụng `AveragePooling2D`, bí danh `AvgPool2D`, thay vì `MaxPool2D`. Như bạn có thể mong đợi, nó hoạt động chính xác như một lớp gộp max, ngoại trừ nó tính toán giá trị trung bình chứ không phải giá trị max. Các lớp gộp trung bình từng rất phổ biến, nhưng bây giờ mọi người chủ yếu sử dụng các lớp gộp max, vì chúng thường hoạt động tốt hơn. Điều này có vẻ đáng ngạc nhiên, vì tính toán giá trị trung bình thường mất ít thông tin hơn so với tính toán giá trị max. Nhưng mặt khác, gộp max chỉ bảo tồn các đặc trưng mạnh nhất, loại bỏ tất cả các đặc trưng vô nghĩa, vì vậy các lớp tiếp theo có một tín hiệu sạch hơn để làm việc. Hơn nữa, gộp max cung cấp bất biến dịch chuyển mạnh hơn so với gộp trung bình, và nó yêu cầu ít tính toán hơn một chút.

Lưu ý rằng gộp max và gộp trung bình có thể được thực hiện dọc theo chiều sâu thay vì các chiều không gian, mặc dù nó không phổ biến bằng. Điều này có thể cho phép CNN học cách bất biến đối với các đặc trưng khác nhau. Ví dụ, nó có thể học nhiều bộ lọc, mỗi bộ lọc phát hiện một phép xoay khác nhau của cùng một mẫu (chẳng hạn như chữ số viết tay; xem Hình 14-11), và lớp gộp max theo chiều sâu sẽ đảm bảo rằng đầu ra giống nhau bất kể phép xoay. CNN cũng có thể học cách bất biến đối với bất cứ điều gì: độ dày, độ sáng, độ xiên, màu sắc, v.v.



Hình 14-11. Gộp max theo chiều sâu có thể giúp CNN học cách bất biến (trong trường hợp này là xoay).

Keras không bao gồm một lớp gộp max theo chiều sâu, nhưng không quá khó để triển khai một lớp tùy chỉnh cho việc đó:

```
class DepthPool(tf.keras.layers.Layer):
    def __init__(self, pool_size=2, **kwargs):
        super().__init__(**kwargs)
        self.pool_size = pool_size

    def call(self, inputs):
        shape = tf.shape(inputs)  # shape[-1] is the number of channels
        groups = shape[-1] // self.pool_size  # number of channel groups
        new_shape = tf.concat([shape[:-1], [groups, self.pool_size]], axis=0)
        return tf.reduce_max(tf.reshape(inputs, new_shape), axis=-1)
```

Lớp này định hình lại đầu vào của nó để chia các kênh thành các nhóm có kích thước mong muốn (`pool_size`), sau đó nó sử dụng `tf.reduce_max()` để tính toán giá trị max của mỗi nhóm. Việc triển khai này giả định rằng stride bằng kích thước pool, đây thường là điều bạn muốn. Ngoài ra, bạn có thể sử dụng thao tác `tf.nn.max_pool()` của TensorFlow và gói trong một lớp Lambda để sử dụng nó bên trong một mô hình Keras, nhưng đáng buồn là thao tác này không triển khai gộp theo chiều sâu cho GPU, chỉ cho CPU.

Một loại lớp gộp cuối cùng mà bạn thường thấy trong các kiến trúc hiện đại là lớp gộp trung bình toàn cục (global average pooling layer). Nó hoạt động rất khác: tất cả những gì nó làm là tính toán giá trị trung bình của mỗi toàn bộ bản đồ đặc trưng (nó giống như một lớp gộp trung bình sử dụng kernel gộp có cùng kích thước không gian với đầu vào). Điều này có nghĩa là nó chỉ xuất ra một số duy nhất cho mỗi bản đồ đặc trưng và mỗi trường hợp.

Mặc dù điều này rõ ràng là cực kỳ phá hủy (phần lớn thông tin trong bản đồ đặc trưng bị mất), nhưng nó có thể hữu ích ngay trước lớp đầu ra, như bạn sẽ thấy sau trong chương này. Để tạo một lớp như vậy, chỉ cần sử dụng lớp `GlobalAveragePooling2D`, bí danh `GlobalAvgPool2D`:

```
global_avg_pool = tf.keras.layers.GlobalAvgPool2D()
```

Nó tương đương với lớp `Lambda` sau đây, lớp này tính toán giá trị trung bình trên các chiều không gian (chiều cao và chiều rộng):

```
global_avg_pool = tf.keras.layers.Lambda(
    lambda X: tf.reduce_mean(X, axis=[1, 2])
)
```

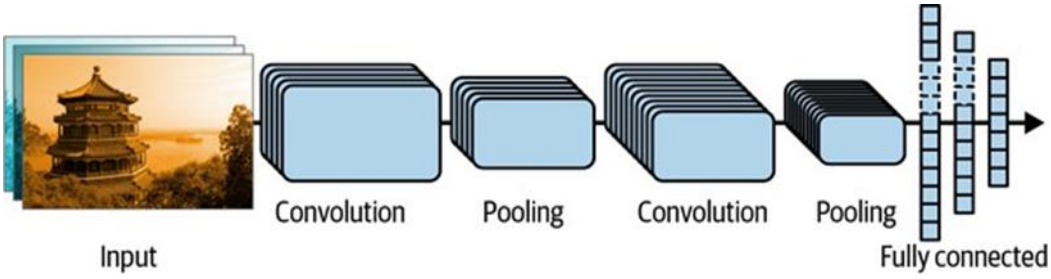
Ví dụ, nếu chúng ta áp dụng lớp này cho hình ảnh đầu vào, chúng ta sẽ nhận được cường độ trung bình của màu đỏ, xanh lá cây và xanh lam cho mỗi hình ảnh:

```
>>> global_avg_pool(images)
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[0.64338624, 0.5971759 , 0.5824972 ],
       [0.76306933, 0.26011038, 0.10849128]], dtype=float32)>
```

Bây giờ bạn đã biết tất cả các khối xây dựng để tạo mạng nơ-ron tích chập. Hãy xem cách lắp ráp chúng.

Kiến trúc CNN

Các kiến trúc CNN điển hình xếp chồng một vài lớp tích chập (mỗi lớp thường theo sau bởi một lớp ReLU), sau đó là một lớp gộp, sau đó lại vài lớp tích chập khác (+ReLU), sau đó lại một lớp gộp, v.v. Hình ảnh ngày càng nhỏ đi khi nó đi qua mạng, nhưng nó cũng thường trở nên sâu hơn (tức là có nhiều bản đồ đặc trưng hơn), nhờ các lớp tích chập (xem Hình 14-12). Ở trên cùng của chồng, một mạng nơ-ron truyền thẳng thông thường được thêm vào, bao gồm một vài lớp kết nối đầy đủ (+ReLU), và lớp cuối cùng xuất ra dự đoán (ví dụ: một lớp softmax xuất ra xác suất lớp ước tính).



Hình 14-12. Kiến trúc CNN điển hình.

Dưới đây là cách bạn có thể triển khai một CNN cơ bản để giải quyết tập dữ liệu Fashion MNIST (được giới thiệu trong Chương 10):

```
from functools import partial

DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3,
padding="same",
                        activation="relu", kernel_initializer="he_normal")

model = tf.keras.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(units=128, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=64, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=10, activation="softmax")
])
```

Hãy cùng xem qua đoạn mã này:

- Chúng ta sử dụng hàm `functools.partial()` (được giới thiệu trong Chương 11) để định nghĩa `DefaultConv2D`, hoạt động giống như `Conv2D` nhưng với các đối số mặc định khác: kích thước kernel nhỏ là 3, padding “same”, hàm kích hoạt ReLU và trình khởi tạo He tương ứng.
- Tiếp theo, chúng ta tạo mô hình `Sequential`. Lớp đầu tiên của nó là `DefaultConv2D` với 64 bộ lọc khá lớn (7 × 7). Nó sử dụng stride mặc định là 1 vì hình ảnh đầu vào không quá lớn. Nó cũng đặt `input_shape=[28, 28, 1]`, vì hình ảnh có kích thước 28 × 28 pixel, với một kênh màu duy nhất (tức là thang độ xám). Khi bạn tải tập dữ liệu Fashion MNIST, hãy đảm bảo mỗi hình ảnh có hình dạng này: bạn có thể cần sử dụng `np.reshape()` hoặc `np.expanddims()` để thêm chiều kênh. Hoặc, bạn có thể sử dụng lớp `Reshape` làm lớp đầu tiên trong mô hình.
- Sau đó, chúng ta thêm một lớp gộp max sử dụng kích thước pool mặc định là 2, vì vậy nó chia mỗi chiều không gian cho hệ số 2.
- Sau đó, chúng ta lặp lại cấu trúc tương tự hai lần: hai lớp tích chập theo sau bởi một lớp gộp max. Đối với hình ảnh lớn hơn, chúng ta có thể lặp lại cấu trúc này nhiều lần hơn. Số lần lặp lại là một siêu tham số mà bạn có thể điều chỉnh.
- Lưu ý rằng số lượng bộ lọc tăng gấp đôi khi chúng ta đi lên CNN về phía lớp đầu ra (ban đầu là 64, sau đó là 128, sau đó là 256): điều này hợp lý vì số lượng đặc trưng cấp thấp thường khá thấp (ví dụ: các vòng tròn nhỏ, đường ngang), nhưng có nhiều

cách khác nhau để kết hợp chúng thành các đặc trưng cấp cao hơn. Một thực hành phổ biến là tăng gấp đôi số lượng bộ lọc sau mỗi lớp gộp: vì một lớp gộp chia mỗi chiều không gian cho hệ số 2, chúng ta có thể tăng gấp đôi số lượng bản đồ đặc trưng trong lớp tiếp theo mà không sợ làm bùng nổ số lượng tham số, mức sử dụng bộ nhớ hoặc tải tính toán.

- Tiếp theo là mạng kết nối đầy đủ, bao gồm hai lớp ẩn dày đặc và một lớp đầu ra dày đặc. Vì đây là một tác vụ phân loại với 10 lớp, lớp đầu ra có 10 đơn vị và nó sử dụng hàm kích hoạt softmax. Lưu ý rằng chúng ta phải làm phẳng đầu vào ngay trước lớp dày đặc đầu tiên, vì nó mong đợi một mảng tính năng 1D cho mỗi phiên bản. Chúng ta cũng thêm hai lớp dropout, với tỷ lệ dropout 50% mỗi lớp, để giảm overfitting.

Nếu bạn biên dịch mô hình này bằng cách sử dụng hàm mất mát “sparse_categorical_crossentropy” và bạn huấn luyện mô hình trên tập huấn luyện Fashion MNIST, nó sẽ đạt được độ chính xác trên 92% trên tập kiểm tra. Nó không phải là công nghệ tiên tiến nhất, nhưng nó khá tốt và rõ ràng là tốt hơn nhiều so với những gì chúng ta đã đạt được với các mạng dày đặc trong Chương 10.

Trong những năm qua, các biến thể của kiến trúc cơ bản này đã được phát triển, dẫn đến những tiến bộ đáng kinh ngạc trong lĩnh vực này. Một thước đo tốt về sự tiến bộ này là tỷ lệ lỗi trong các cuộc thi như thử thách ILSVRC ImageNet. Trong cuộc thi này, tỷ lệ lỗi top-5 cho phân loại hình ảnh—tức là số lượng hình ảnh thử nghiệm mà hệ thống không bao gồm câu trả lời đúng trong năm dự đoán hàng đầu của nó—đã giảm từ hơn 26% xuống dưới 2,3% chỉ trong sáu năm. Các hình ảnh khá lớn (ví dụ: cao 256 pixel) và có 1.000 lớp, một số trong số đó thực sự rất khó phân biệt (thử phân biệt 120 giống chó). Nhìn vào sự tiến hóa của các mục chiến thắng là một cách tốt để hiểu cách CNN hoạt động và cách nghiên cứu trong học sâu tiến bộ.

Chúng ta sẽ xem xét kiến trúc LeNet-5 cổ điển (1998) trước, sau đó là một số người chiến thắng thử thách ILSVRC: AlexNet (2012), GoogLeNet (2014), ResNet (2015) và SENet (2017). Ngoài ra, chúng ta cũng sẽ xem xét một vài kiến trúc khác, bao gồm Xception, ResNeXt, DenseNet, MobileNet, CSPNet và EfficientNet.

LeNet-5

Kiến trúc LeNet-5 có lẽ là kiến trúc CNN được biết đến rộng rãi nhất. Như đã đề cập trước đó, nó được tạo ra bởi Yann LeCun vào năm 1998 và đã được sử dụng rộng rãi để nhận dạng chữ số viết tay (MNIST). Nó bao gồm các lớp được thể hiện trong Bảng 14-1.

Bảng 14-1. Kiến trúc LeNet-5

Lớp	Loại	Bản đồ	Kích thước	Kích thước Kernel	Stride
Out	Fully connected	–	10	–	–
F6	Fully connected	–	84	–	–
C5	Convolution	120	1 × 1	5 × 5	1
S4	Avg pooling	16	5 × 5	2 × 2	2

Lớp	Loại	Bản đồ	Kích thước	Kích thước Kernel	Stride
C3	Convolution	16	10 × 10	5 × 5	1
S2	Avg pooling	6	14 × 14	2 × 2	2
C1	Convolution	6	28 × 28	5 × 5	1
In	Input	1	32 × 32	–	–

Như bạn có thể thấy, điều này trông khá giống với mô hình Fashion MNIST của chúng ta: một chồng các lớp tích chập và lớp gộp, sau đó là một mạng dày đặc. Có lẽ sự khác biệt chính với các CNN phân loại hiện đại hơn là các hàm kích hoạt: ngày nay, chúng ta sẽ sử dụng ReLU thay vì tanh và softmax thay vì RBF. Có một số khác biệt nhỏ khác không thực sự quan trọng lắm, nhưng trong trường hợp bạn quan tâm, chúng được liệt kê trong sổ tay của chương này tại https://homl.info/colab3 . Trang web của Yann LeCun cũng có các bản demo tuyệt vời về LeNet-5 phân loại chữ số.

AlexNet

Kiến trúc AlexNet CNN đã giành chiến thắng thử thách ILSVRC 2012 với tỷ lệ lớn: nó đạt tỷ lệ lỗi top-5 là 17%, trong khi đối thủ tốt thứ hai chỉ đạt 26%! AlexNet được phát triển bởi Alex Krizhevsky (do đó có tên), Ilya Sutskever và Geoffrey Hinton. Nó tương tự như LeNet-5, chỉ lớn hơn và sâu hơn nhiều, và nó là kiến trúc đầu tiên xếp chồng trực tiếp các lớp tích chập lên nhau, thay vì xếp chồng một lớp gộp lên trên mỗi lớp tích chập. Bảng 14-2 trình bày kiến trúc này.

Bảng 14-2. Kiến trúc AlexNet

Lớp	Loại	Bản đồ	Kích thước	Kích thước Kernel	Stride
Out	Fully connected	–	1,000	–	–
F10	Fully connected	–	4,096	–	–
F9	Fully connected	–	4,096	–	–
S8	Max pooling	256	6 × 6	3 × 3	2
C7	Convolution	256	13 × 13	3 × 3	1
C6	Convolution	384	13 × 13	3 × 3	1
C5	Convolution	384	13 × 13	3 × 3	1
S4	Max pooling	256	13 × 13	3 × 3	2
C3	Convolution	256	27 × 27	5 × 5	1
S2	Max pooling	96	27 × 27	3 × 3	2
C1	Convolution	96	55 × 55	11 × 11	4
In	Input	3 (RGB)	227 × 227	–	–

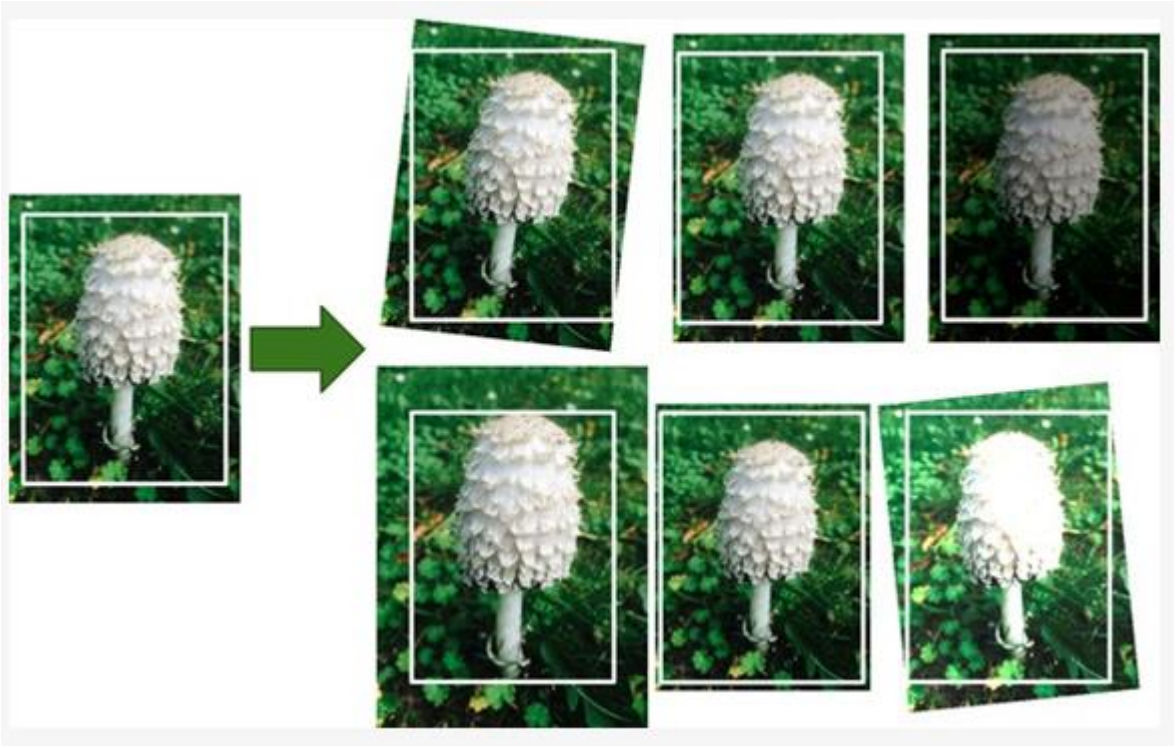
Để giảm overfitting, các tác giả đã sử dụng hai kỹ thuật điều hòa. Đầu tiên, họ áp dụng dropout (được giới thiệu trong Chương 11) với tỷ lệ dropout 50% trong quá trình huấn luyện

cho đầu ra của các lớp F9 và F10. Thứ hai, họ thực hiện tăng cường dữ liệu bằng cách dịch chuyển ngẫu nhiên các hình ảnh huấn luyện bằng các offset khác nhau, lật chúng theo chiều ngang và thay đổi điều kiện ánh sáng.

Tăng cường dữ liệu

Tăng cường dữ liệu làm tăng kích thước của tập huấn luyện một cách nhân tạo bằng cách tạo ra nhiều biến thể thực tế của mỗi phiên bản huấn luyện. Điều này làm giảm overfitting, khiến nó trở thành một kỹ thuật điều hòa. Các phiên bản được tạo ra phải càng thực tế càng tốt: lý tưởng nhất, với một hình ảnh từ tập huấn luyện được tăng cường, một con người không thể phân biệt được nó có được tăng cường hay không. Đơn giản là thêm nhiễu trắng sẽ không giúp ích; các sửa đổi phải có thể học được (nhiễu trắng thì không).

Ví dụ, bạn có thể dịch chuyển, xoay và thay đổi kích thước nhẹ nhàng mọi bức ảnh trong tập huấn luyện bằng các lượng khác nhau và thêm các bức ảnh thu được vào tập huấn luyện (xem Hình 14-13). Để làm điều này, bạn có thể sử dụng các lớp tăng cường dữ liệu của Keras, được giới thiệu trong Chương 13 (ví dụ: `RandomCrop`, `RandomRotation`, v.v.). Điều này buộc mô hình phải chịu đựng tốt hơn các biến thể về vị trí, hướng và kích thước của các đối tượng trong ảnh. Để tạo ra một mô hình chịu đựng tốt hơn các điều kiện ánh sáng khác nhau, bạn cũng có thể tạo ra nhiều hình ảnh với các độ tương phản khác nhau. Nói chung, bạn cũng có thể lật hình ảnh theo chiều ngang (trừ văn bản và các đối tượng không đối xứng khác). Bằng cách kết hợp các phép biến đổi này, bạn có thể tăng đáng kể kích thước tập huấn luyện của mình.



Hình 14-13. Tạo các phiên bản huấn luyện mới từ các phiên bản hiện có.

Tăng cường dữ liệu cũng hữu ích khi bạn có một tập dữ liệu không cân bằng: bạn có thể sử dụng nó để tạo thêm các mẫu của các lớp ít thường xuyên hơn.

AlexNet cũng sử dụng bước chuẩn hóa cạnh tranh ngay sau bước ReLU của các lớp C1 và C3, được gọi là chuẩn hóa phản hồi cục bộ (LRN): các nơ-ron được kích hoạt mạnh nhất ức chế các nơ-ron khác nằm ở cùng vị trí trong các bản đồ đặc trưng lân cận. Kích hoạt cạnh tranh như vậy đã được quan sát thấy trong các nơ-ron sinh học. Điều này khuyến khích các bản đồ đặc trưng khác nhau chuyên biệt hóa, đẩy chúng ra xa nhau và buộc chúng phải khám phá một phạm vi đặc trưng rộng hơn, cuối cùng cải thiện khả năng khái quát hóa. Phương trình 14-2 cho thấy cách áp dụng LRN.

Phương trình 14-2. Chuẩn hóa phản hồi cục bộ (LRN)

$$b_i = a_i\left(k + \alpha \sum_{j=j_{\text{low}}}^{j_{\text{high}}} {a_j}^2\right)^{-\beta} \quad \text{with} \begin{cases} j_{\text{high}} = \min\left(i + \frac{r}{2}, f_n - 1\right) \\ j_{\text{low}} = \max\left(0, i - \frac{r}{2}\right) \end{cases}$$

Trong phương trình này:

- b_i là đầu ra được chuẩn hóa của nơ-ron nằm trong bản đồ đặc trưng i, tại một hàng u và cột v nào đó (lưu ý rằng trong phương trình này chúng ta chỉ xem xét các nơ-ron nằm ở hàng và cột này, nên u và v không được hiển thị).
- a_i là sự kích hoạt của nơ-ron đó sau bước ReLU, nhưng trước khi chuẩn hóa.
- k, α, β và r là các siêu tham số. k được gọi là bias, và r được gọi là bán kính chiều sâu.
- f_n là số lượng bản đồ đặc trưng.

Ví dụ, nếu r = 2 và một nơ-ron có sự kích hoạt mạnh, nó sẽ ức chế sự kích hoạt của các nơ-ron nằm trong các bản đồ đặc trưng ngay phía trên và phía dưới của nó.

Trong AlexNet, các siêu tham số được đặt là: r = 5, α = 0.0001, β = 0.75 và k = 2. Bạn có thể triển khai bước này bằng cách sử dụng hàm `tf.nn.local_response_normalization()` (mà bạn có thể gói trong một lớp Lambda nếu muốn sử dụng nó trong một mô hình Keras).

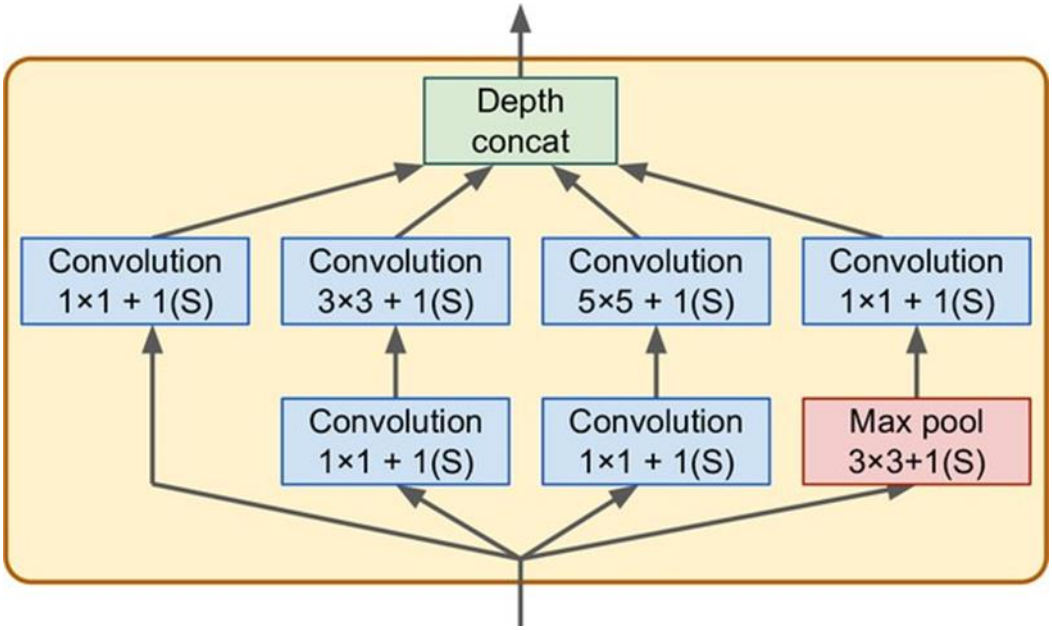
Một biến thể của AlexNet có tên ZF Net được phát triển bởi Matthew Zeiler và Rob Fergus và đã giành chiến thắng thử thách ILSVRC 2013. Về cơ bản, nó là AlexNet với một vài siêu tham số được điều chỉnh (số lượng bản đồ đặc trưng, kích thước kernel, stride, v.v.).

GoogLeNet

Kiến trúc GoogLeNet được phát triển bởi Christian Szegedy et al. từ Google Research, và nó đã giành chiến thắng thử thách ILSVRC 2014 bằng cách đẩy tỷ lệ lỗi top-5 xuống dưới 7%. Hiệu suất tuyệt vời này phần lớn đến từ việc mạng sâu hơn nhiều so với các CNN trước đó (như bạn sẽ thấy trong Hình 14-15). Điều này được thực hiện nhờ các mạng con được gọi là mô-đun inception, cho phép GoogLeNet sử dụng các tham số hiệu quả hơn nhiều so với

các kiến trúc trước đây: GoogLeNet thực sự có số lượng tham số ít hơn 10 lần so với AlexNet (khoảng 6 triệu thay vì 60 triệu).

Hình 14-14 cho thấy kiến trúc của một mô-đun inception. Ký hiệu "3 × 3 + 1(S)" có nghĩa là lớp sử dụng kernel 3 × 3, stride 1 và padding "same". Tín hiệu đầu vào được đưa vào bốn lớp khác nhau song song. Tất cả các lớp tích chập đều sử dụng hàm kích hoạt ReLU. Lưu ý rằng các lớp tích chập trên cùng sử dụng các kích thước kernel khác nhau (1 × 1, 3 × 3 và 5 × 5), cho phép chúng nắm bắt các mẫu ở các tỷ lệ khác nhau. Cũng lưu ý rằng mỗi lớp đều sử dụng stride là 1 và padding "same" (ngay cả lớp gộp max), vì vậy đầu ra của chúng đều có cùng chiều cao và chiều rộng với đầu vào của chúng. Điều này giúp có thể nối tất cả các đầu ra dọc theo chiều sâu trong lớp nối chiều sâu cuối cùng (tức là xếp chồng các bản đồ đặc trưng từ tất cả bốn lớp tích chập trên cùng). Nó có thể được triển khai bằng cách sử dụng lớp `Concatenate` của Keras, sử dụng `axis=-1` mặc định.



Hình 14-14. Mô-đun Inception.

Bạn có thể tự hỏi tại sao các mô-đun inception lại có các lớp tích chập với kernel 1 × 1. Chắc chắn các lớp này không thể nắm bắt bất kỳ đặc trưng nào vì chúng chỉ nhìn vào một pixel tại một thời điểm, phải không? Trên thực tế, các lớp này phục vụ ba mục đích:

- Mặc dù chúng không thể nắm bắt các mẫu không gian, nhưng chúng có thể nắm bắt các mẫu dọc theo chiều sâu (tức là trên các kênh).
- Chúng được cấu hình để xuất ra ít bản đồ đặc trưng hơn đầu vào của chúng, vì vậy chúng đóng vai trò là các lớp thắt cổ chai (bottleneck layers), có nghĩa là chúng giảm chiều. Điều này cắt giảm chi phí tính toán và số lượng tham số, tăng tốc huấn luyện và cải thiện khả năng khái quát hóa.
- Mỗi cặp lớp tích chập ([1 × 1, 3 × 3] và [1 × 1, 5 × 5]) hoạt động như một lớp tích chập mạnh mẽ duy nhất, có khả năng nắm bắt các mẫu phức tạp hơn. Một lớp tích chập

tương đương với việc quét một lớp dày đặc trên hình ảnh (tại mỗi vị trí, nó chỉ nhìn vào một trường tiếp nhận nhỏ), và các cặp lớp tích chập này tương đương với việc quét các mạng nơ-ron hai lớp trên hình ảnh.

Tóm lại, bạn có thể nghĩ toàn bộ mô-đun inception như một lớp tích chập được tăng cường, có khả năng xuất ra các bản đồ đặc trưng nắm bắt các mẫu phức tạp ở các tỷ lệ khác nhau.

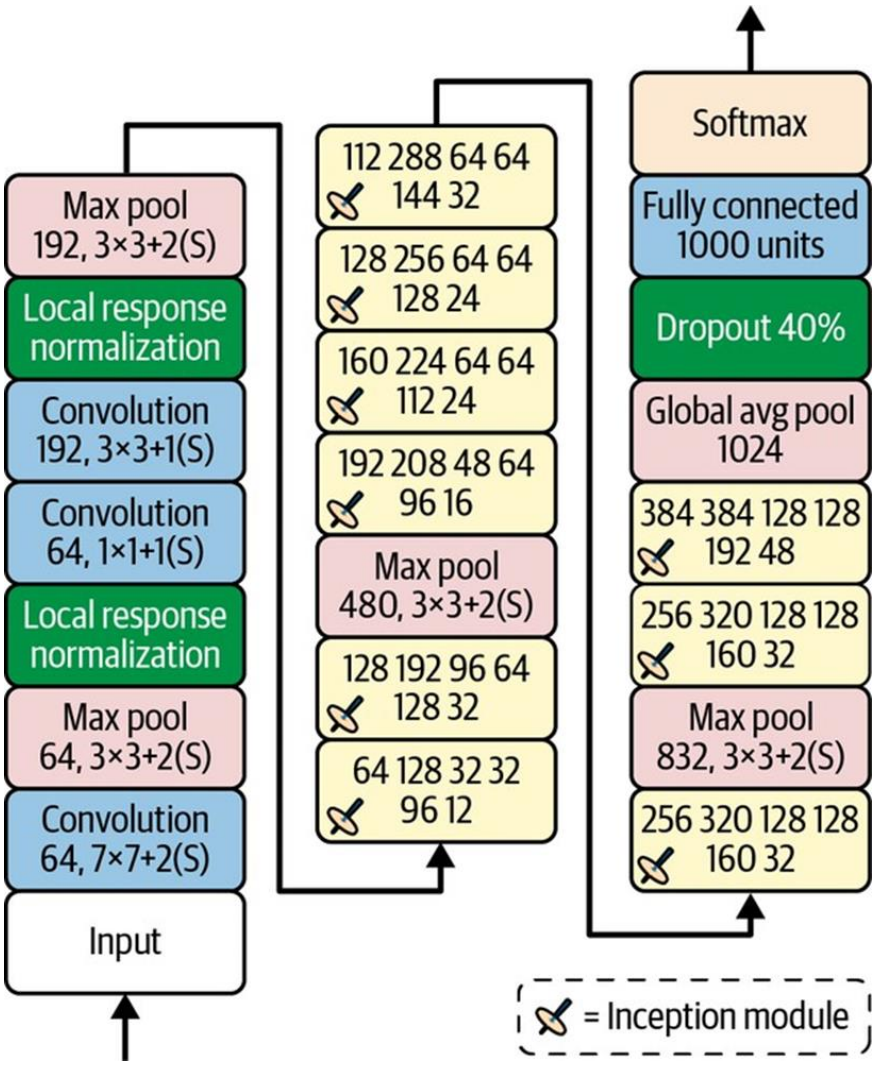
Bây giờ hãy xem kiến trúc của CNN GoogLeNet (xem Hình 14-15).

Số lượng bản đồ đặc trưng được xuất bởi mỗi lớp tích chập và mỗi lớp gộp được hiển thị trước kích thước kernel. Kiến trúc quá sâu đến mức phải được biểu diễn trong ba cột, nhưng GoogLeNet thực sự là một chồng cao, bao gồm chín mô-đun inception (các hộp có đỉnh xoay). Sáu số trong các mô-đun inception đại diện cho số lượng bản đồ đặc trưng được xuất bởi mỗi lớp tích chập trong mô-đun (theo cùng thứ tự như trong Hình 14-14).

Lưu ý rằng tất cả các lớp tích chập đều sử dụng hàm kích hoạt ReLU.

Hãy cùng xem qua mạng này:

- Hai lớp đầu tiên chia chiều cao và chiều rộng của hình ảnh cho 4 (vì vậy diện tích của nó được chia cho 16), để giảm tải tính toán. Lớp đầu tiên sử dụng kích thước kernel lớn, 7 × 7, để phần lớn thông tin được bảo toàn.
- Sau đó, lớp chuẩn hóa phản hồi cục bộ đảm bảo rằng các lớp trước học được nhiều loại đặc trưng khác nhau (như đã thảo luận trước đó).
- Hai lớp tích chập tiếp theo, trong đó lớp đầu tiên hoạt động như một lớp thắt cổ chai. Như đã đề cập, bạn có thể coi cặp này như một lớp tích chập thông minh hơn duy nhất.
- Một lần nữa, một lớp chuẩn hóa phản hồi cục bộ đảm bảo rằng các lớp trước nắm bắt được nhiều loại mẫu khác nhau.
- Tiếp theo, một lớp gộp max giảm chiều cao và chiều rộng của hình ảnh đi 2, một lần nữa để tăng tốc tính toán.
- Sau đó là xương sống của CNN: một chồng cao gồm chín mô-đun inception, xen kẽ với một vài lớp gộp max để giảm chiều và tăng tốc mạng.
- Tiếp theo, lớp gộp trung bình toàn cục xuất ra giá trị trung bình của mỗi bản đồ đặc trưng: điều này loại bỏ bất kỳ thông tin không gian nào còn lại, điều này không sao vì không còn nhiều thông tin không gian tại thời điểm đó. Thật vậy, hình ảnh đầu vào GoogLeNet thường được dự kiến là 224 × 224 pixel, vì vậy sau 5 lớp gộp max, mỗi lớp chia chiều cao và chiều rộng cho 2, các bản đồ đặc trưng giảm xuống còn 7 × 7. Hơn nữa, đây là một tác vụ phân loại, không phải định vị, vì vậy không quan trọng đối tượng ở đâu. Nhờ việc giảm chiều do lớp này mang lại, không cần phải có nhiều lớp kết nối đầy đủ ở trên cùng của CNN (như trong AlexNet), và điều này làm giảm đáng kể số lượng tham số trong mạng và hạn chế rủi ro quá khớp.
- Các lớp cuối cùng tự giải thích: dropout để điều hòa, sau đó là một lớp kết nối đầy đủ với 1.000 đơn vị (vì có 1.000 lớp) và hàm kích hoạt softmax để xuất ra xác suất lớp ước tính.



Hình 14-15. Kiến trúc GoogLeNet.

Kiến trúc GoogLeNet gốc bao gồm hai bộ phân loại phụ được cắm trên đỉnh của mô-đun inception thứ ba và thứ sáu. Cả hai đều bao gồm một lớp gộp trung bình, một lớp tích chập, hai lớp kết nối đầy đủ và một lớp kích hoạt softmax. Trong quá trình huấn luyện, mất mát của chúng (giảm 70%) được thêm vào mất mát tổng thể. Mục tiêu là chống lại vấn đề vanishing gradients và điều hòa mạng, nhưng sau đó đã chỉ ra rằng hiệu quả của chúng tương đối nhỏ.

Một số biến thể của kiến trúc GoogLeNet sau đó đã được các nhà nghiên cứu của Google đề xuất, bao gồm Inception-v3 và Inception-v4, sử dụng các mô-đun inception hơi khác nhau để đạt được hiệu suất tốt hơn nữa.

VGGNet

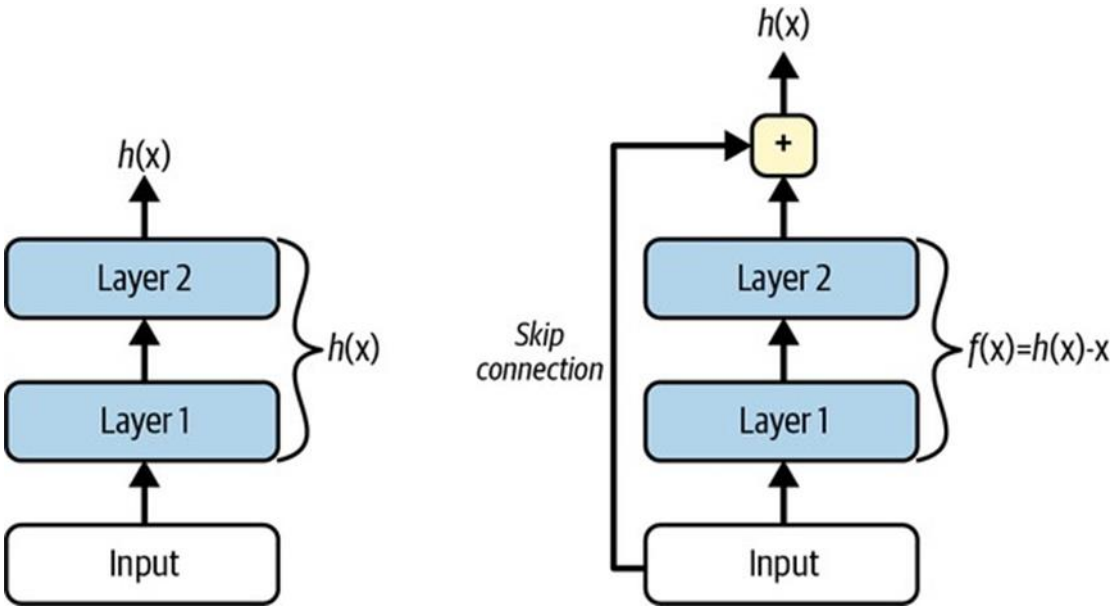
Á quân trong thử thách ILSVRC 2014 là VGGNet, được Karen Simonyan và Andrew Zisserman, từ phòng nghiên cứu Visual Geometry Group (VGG) tại Đại học Oxford, phát triển một kiến trúc rất đơn giản và cổ điển; nó có 2 hoặc 3 lớp tích chập và một lớp gộp, sau

đó lại 2 hoặc 3 lớp tích chập và một lớp gộp, v.v. (đạt tổng cộng 16 hoặc 19 lớp tích chập, tùy thuộc vào biến thể VGG), cộng với một mạng dày đặc cuối cùng với 2 lớp ẩn và lớp đầu ra. Nó sử dụng các bộ lọc 3 × 3 nhỏ, nhưng có rất nhiều bộ lọc.

ResNet

Kaiming He et al. đã giành chiến thắng thử thách ILSVRC 2015 bằng cách sử dụng Mạng lưới dư thừa (ResNet) với tỷ lệ lỗi top-5 đáng kinh ngạc dưới 3,6%. Biến thể chiến thắng sử dụng một CNN cực kỳ sâu bao gồm 152 lớp (các biến thể khác có 34, 50 và 101 lớp). Nó khẳng định xu hướng chung: các mô hình thị giác máy tính ngày càng sâu hơn, với số lượng tham số ngày càng ít hơn. Chìa khóa để có thể huấn luyện một mạng sâu như vậy là sử dụng các kết nối bỏ qua (skip connections, còn gọi là shortcut connections): tín hiệu đưa vào một lớp cũng được thêm vào đầu ra của một lớp nằm cao hơn trong chồng. Hãy xem tại sao điều này lại hữu ích.

Khi huấn luyện một mạng nơ-ron, mục tiêu là làm cho nó mô hình một hàm mục tiêu h(x). Nếu bạn thêm đầu vào x vào đầu ra của mạng (tức là bạn thêm một kết nối bỏ qua), thì mạng sẽ buộc phải mô hình f(x) = h(x) – x thay vì h(x). Đây được gọi là học dư thừa (residual learning) (xem Hình 14-16).



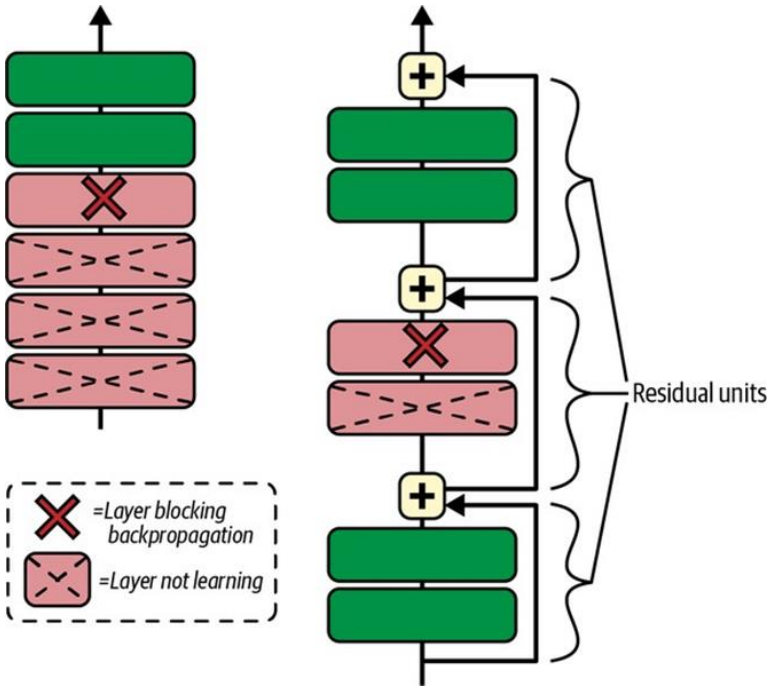
Hình 14-16. Học dư thừa.

Khi bạn khởi tạo một mạng nơ-ron thông thường, các trọng số của nó gần bằng 0, vì vậy mạng chỉ xuất ra các giá trị gần bằng 0. Nếu bạn thêm một kết nối bỏ qua, mạng kết quả chỉ xuất ra một bản sao của đầu vào của nó; nói cách khác, ban đầu nó mô hình hàm đồng nhất. Nếu hàm mục tiêu khá gần với hàm đồng nhất (thường là trường hợp này), điều này sẽ tăng tốc đáng kể quá trình huấn luyện.

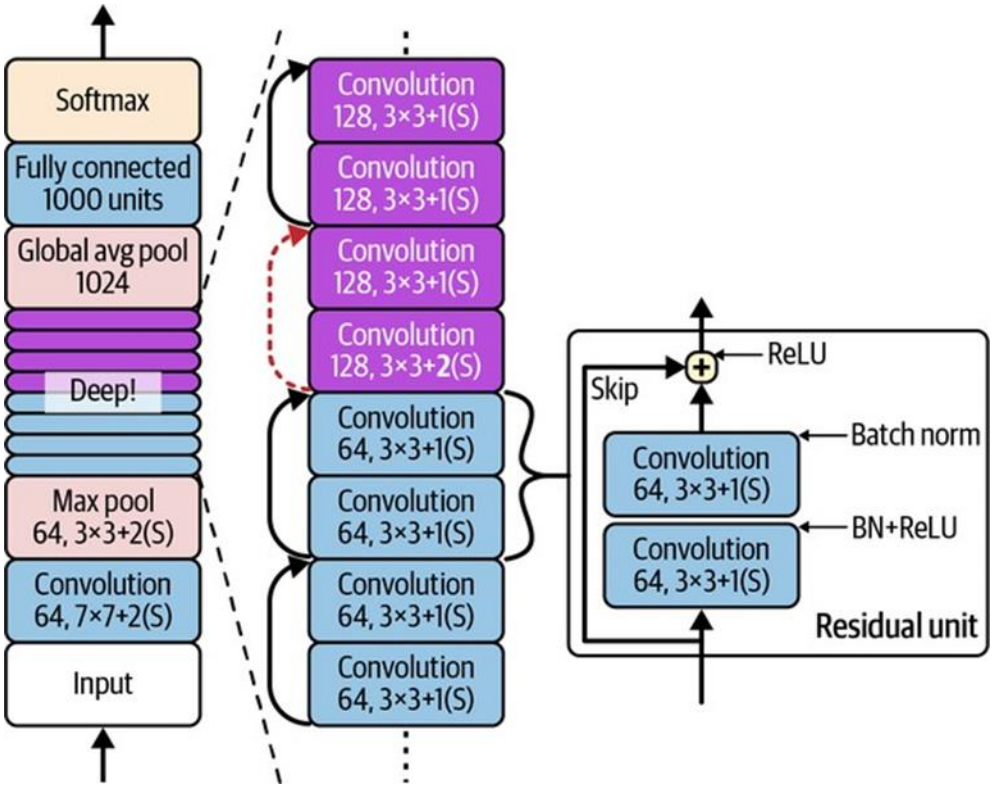
Hơn nữa, nếu bạn thêm nhiều kết nối bỏ qua, mạng có thể bắt đầu tạo ra tiến bộ ngay cả khi một số lớp chưa bắt đầu học (xem Hình 14-17). Nhờ các kết nối bỏ qua, tín hiệu có thể

dễ dàng đi qua toàn bộ mạng. Mạng dư thừa sâu có thể được coi là một chồng các đơn vị dư thừa (RUs), trong đó mỗi đơn vị dư thừa là một mạng nơ-ron nhỏ có kết nối bỏ qua.

Bây giờ hãy xem kiến trúc của ResNet (xem Hình 14-18). Nó đơn giản đáng ngạc nhiên. Nó bắt đầu và kết thúc chính xác như GoogLeNet (ngoại trừ không có lớp dropout), và ở giữa chỉ là một chồng rất sâu các đơn vị dư thừa. Mỗi đơn vị dư thừa bao gồm hai lớp tích chập (và không có lớp gộp!), với chuẩn hóa theo lô (BN) và kích hoạt ReLU, sử dụng kernel 3 × 3 và bảo toàn các chiều không gian (stride 1, padding “same”).

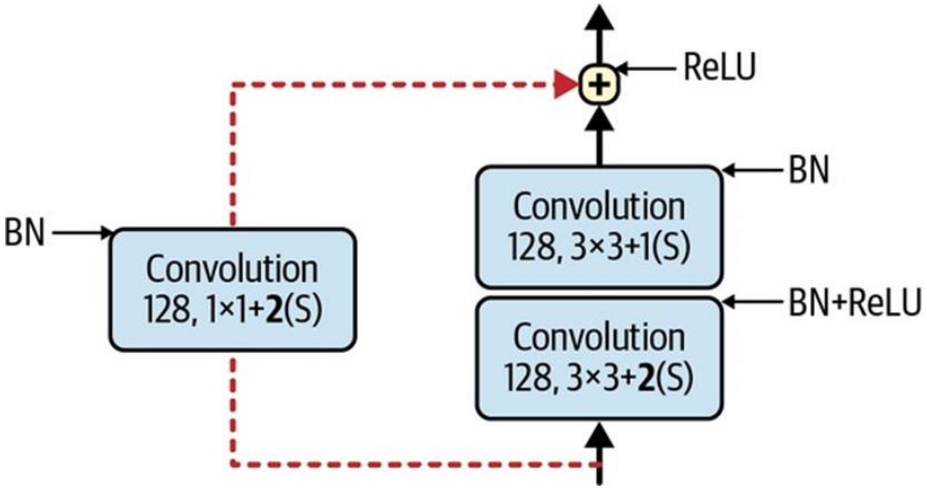


Hình 14-17. Mạng nơ-ron sâu thông thường (trái) và mạng nơ-ron dư thừa sâu (phải).



Hình 14-18. Kiến trúc ResNet.

Lưu ý rằng số lượng bản đồ đặc trưng được tăng gấp đôi sau mỗi vài đơn vị dư thừa, đồng thời chiều cao và chiều rộng của chúng được giảm một nửa (sử dụng một lớp tích chập với stride 2). Khi điều này xảy ra, đầu vào không thể được thêm trực tiếp vào đầu ra của đơn vị dư thừa vì chúng không có cùng hình dạng (ví dụ, vấn đề này ảnh hưởng đến kết nối bỏ qua được biểu thị bằng mũi tên nét đứt trong Hình 14-18). Để giải quyết vấn đề này, đầu vào được truyền qua một lớp tích chập 1 × 1 với stride 2 và số lượng bản đồ đặc trưng đầu ra phù hợp (xem Hình 14-19).



Hình 14-19. Kết nối bỏ qua khi thay đổi kích thước và chiều sâu bản đồ đặc trưng.

Các biến thể khác nhau của kiến trúc tồn tại, với số lượng lớp khác nhau. ResNet-34 là một ResNet với 34 lớp (chỉ tính các lớp tích chập và lớp kết nối đầy đủ) chứa 3 RU xuất ra 64 bản đồ đặc trưng, 4 RU với 128 bản đồ, 6 RU với 256 bản đồ và 3 RU với 512 bản đồ. Chúng ta sẽ triển khai kiến trúc này sau trong chương này.

Các ResNet sâu hơn, chẳng hạn như ResNet-152, sử dụng các đơn vị dư thừa hơi khác. Thay vì hai lớp tích chập 3 × 3 với, chẳng hạn, 256 bản đồ đặc trưng, chúng sử dụng ba lớp tích chập: đầu tiên là một lớp tích chập 1 × 1 với chỉ 64 bản đồ đặc trưng (ít hơn 4 lần), hoạt động như một lớp thắt cổ chai (như đã thảo luận), sau đó là một lớp 3 × 3 với 64 bản đồ đặc trưng, và cuối cùng là một lớp tích chập 1 × 1 khác với 256 bản đồ đặc trưng (4 lần 64) để khôi phục chiều sâu ban đầu. ResNet-152 chứa 3 RU như vậy xuất ra 256 bản đồ, sau đó 8 RU với 512 bản đồ, 36 RU khổng lồ với 1.024 bản đồ, và cuối cùng là 3 RU với 2.048 bản đồ.

Xception

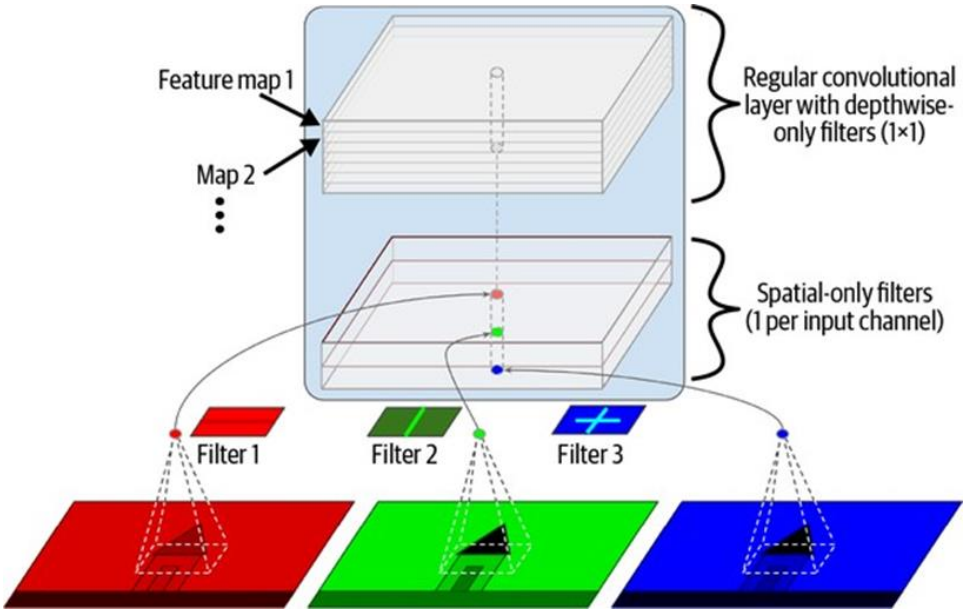
Một biến thể khác của kiến trúc GoogLeNet đáng chú ý: Xception (viết tắt của Extreme Inception) được đề xuất vào năm 2016 bởi François Chollet (tác giả của Keras), và nó đã vượt trội đáng kể so với Inception-v3 trong một tác vụ thị giác khổng lồ (350 triệu hình ảnh và 17.000 lớp). Giống như Inception-v4, nó hợp nhất các ý tưởng của GoogLeNet và ResNet, nhưng nó thay thế các mô-đun inception bằng một loại lớp đặc biệt gọi là lớp tích chập tách biệt chiều sâu (depthwise separable convolution layer, hoặc separable convolution layer viết tắt).

Các lớp này đã được sử dụng trước đây trong một số kiến trúc CNN, nhưng chúng không đóng vai trò trung tâm như trong kiến trúc Xception. Trong khi một lớp tích chập thông thường sử dụng các bộ lọc cố gắng đồng thời nắm bắt các mẫu không gian (ví dụ: hình bầu dục) và các mẫu chéo kênh (ví dụ: miệng + mũi + mắt = khuôn mặt), một lớp tích chập tách biệt đưa ra giả định mạnh mẽ rằng các mẫu không gian và các mẫu chéo kênh có thể được mô hình hóa riêng biệt (xem Hình 14-20). Do đó, nó bao gồm hai phần: phần đầu tiên áp dụng một bộ lọc không gian duy nhất cho mỗi bản đồ đặc trưng đầu vào, sau đó phần thứ hai chỉ tìm kiếm các mẫu chéo kênh—nó chỉ là một lớp tích chập thông thường với các bộ lọc 1 × 1.

Vì các lớp tích chập tách biệt chỉ có một bộ lọc không gian trên mỗi kênh đầu vào, bạn nên tránh sử dụng chúng sau các lớp có quá ít kênh, chẳng hạn như lớp đầu vào (thừa nhận rằng đó là những gì Hình 14-20 đại diện, nhưng nó chỉ nhằm mục đích minh họa). Vì lý do này, kiến trúc Xception bắt đầu với 2 lớp tích chập thông thường, nhưng sau đó phần còn lại của kiến trúc chỉ sử dụng các tích chập tách biệt (tổng cộng 34), cộng với một vài lớp gộp max và các lớp cuối cùng thông thường (một lớp gộp trung bình toàn cục và một lớp đầu ra dày đặc).

Bạn có thể tự hỏi tại sao Xception lại được coi là một biến thể của GoogLeNet, vì nó không chứa bất kỳ mô-đun inception nào cả. Vâng, như đã thảo luận trước đó, một mô-đun inception chứa các lớp tích chập với bộ lọc 1 × 1: những lớp này chỉ tìm kiếm các mẫu chéo kênh. Tuy nhiên, các lớp tích chập nằm trên chúng là các lớp tích chập thông thường tìm kiếm cả các mẫu không gian và các mẫu chéo kênh. Vì vậy, bạn có thể nghĩ một mô-đun

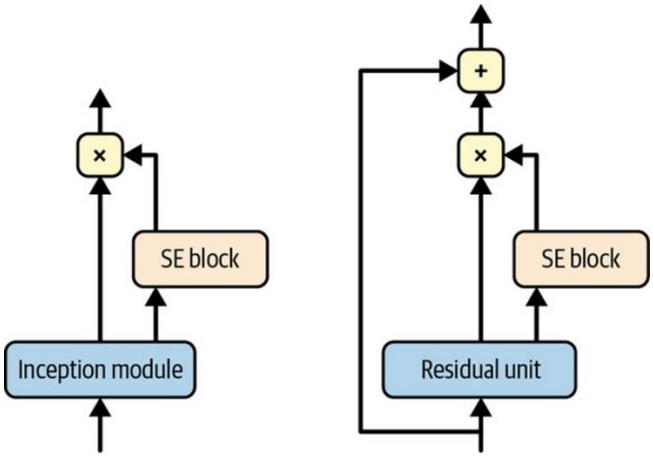
inception là một trung gian giữa một lớp tích chập thông thường (xem xét các mẫu không gian và các mẫu chéo kênh cùng nhau) và một lớp tích chập tách biệt (xem xét chúng riêng biệt). Trong thực tế, có vẻ như các lớp tích chập tách biệt thường hoạt động tốt hơn.



Hình 14-20. Lớp tích chập tách biệt chiều sâu.

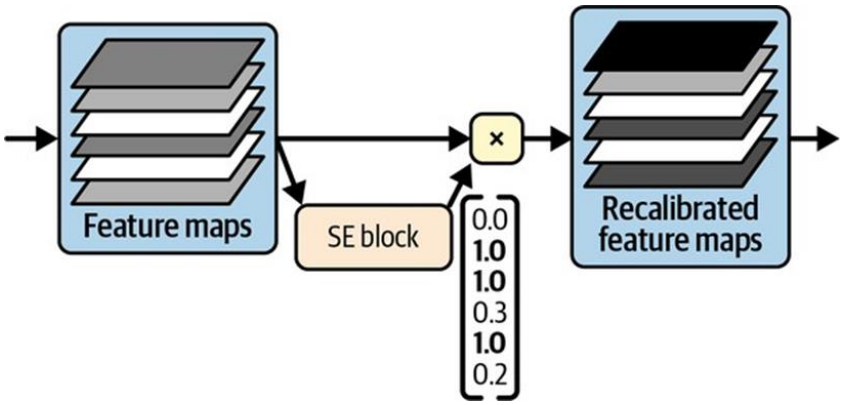
SENet

Kiến trúc chiến thắng trong thử thách ILSVRC 2017 là Mạng nén và kích thích (Squeeze-and-Excitation Network - SENet). Kiến trúc này mở rộng các kiến trúc hiện có như mạng inception và ResNet, và tăng cường hiệu suất của chúng. Điều này cho phép SENet giành chiến thắng trong cuộc thi với tỷ lệ lỗi top-5 đáng kinh ngạc 2,25%! Các phiên bản mở rộng của mạng inception và ResNet được gọi lần lượt là SE-Inception và SE-ResNet. Sự tăng cường đến từ việc SENet thêm một mạng nơ-ron nhỏ, gọi là khối SE, vào mỗi mô-đun inception hoặc đơn vị dư thừa trong kiến trúc gốc, như thể hiện trong Hình 14-21.



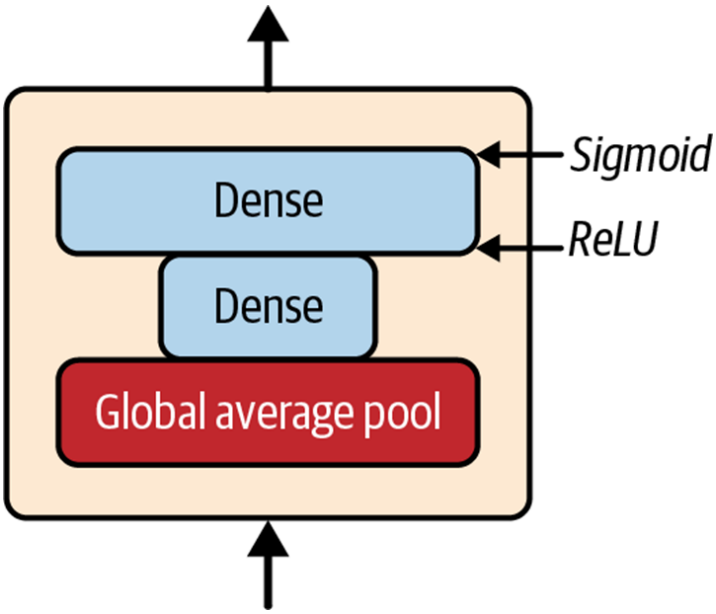
Hình 14-21. Mô-đun SE-Inception (trái) và đơn vị SE-ResNet (phải).

Một khối SE phân tích đầu ra của đơn vị mà nó được gắn vào, chỉ tập trung vào chiều sâu (nó không tìm kiếm bất kỳ mẫu không gian nào), và nó học được các đặc trưng nào thường hoạt động cùng nhau nhất. Sau đó, nó sử dụng thông tin này để hiệu chỉnh lại các bản đồ đặc trưng, như thể hiện trong Hình 14-22. Ví dụ, một khối SE có thể học được rằng miệng, mũi và mắt thường xuất hiện cùng nhau trong các bức ảnh: nếu bạn nhìn thấy miệng và mũi, bạn nên mong đợi cũng thấy mắt. Vì vậy, nếu khối nhìn thấy sự kích hoạt mạnh mẽ trong các bản đồ đặc trưng miệng và mũi, nhưng chỉ kích hoạt nhẹ trong bản đồ đặc trưng mắt, nó sẽ tăng cường bản đồ đặc trưng mắt (chính xác hơn, nó sẽ giảm các bản đồ đặc trưng không liên quan). Nếu mắt bị nhầm lẫn với thứ khác, việc hiệu chỉnh lại bản đồ đặc trưng này sẽ giúp giải quyết sự mơ hồ.



Hình 14-22. Một khối SE thực hiện hiệu chỉnh lại bản đồ đặc trưng.

Một khối SE bao gồm chỉ ba lớp: một lớp gộp trung bình toàn cục, một lớp ẩn dày đặc sử dụng hàm kích hoạt ReLU, và một lớp đầu ra dày đặc sử dụng hàm kích hoạt sigmoid (xem Hình 14-23).



Hình 14-23. Kiến trúc khối SE.

Như trước đây, lớp gộp trung bình toàn cục tính toán sự kích hoạt trung bình cho mỗi bản đồ đặc trưng: ví dụ, nếu đầu vào của nó chứa 256 bản đồ đặc trưng, nó sẽ xuất ra 256 số đại diện cho mức độ phản hồi tổng thể cho mỗi bộ lọc.

Lớp tiếp theo là nơi xảy ra "nén" (squeeze): lớp này có số lượng nơ-ron ít hơn đáng kể so với 256—thường ít hơn 16 lần so với số lượng bản đồ đặc trưng (ví dụ: 16 nơ-ron)—vì vậy 256 số được nén thành một vectơ nhỏ (ví dụ: 16 chiều). Đây là một biểu diễn vectơ chiều thấp (tức là một embedding) của phân bố phản hồi đặc trưng. Bước thắt cổ chai này buộc khối SE phải học một biểu diễn tổng quát của các kết hợp đặc trưng (chúng ta sẽ thấy nguyên tắc này hoạt động trở lại khi thảo luận về autoencoder trong Chương 17). Cuối cùng, lớp đầu ra lấy embedding và xuất ra một vectơ hiệu chỉnh lại chứa một số trên mỗi bản đồ đặc trưng (ví dụ: 256), mỗi số nằm trong khoảng từ 0 đến 1. Các bản đồ đặc trưng sau đó được nhân với vectơ hiệu chỉnh lại này, vì vậy các đặc trưng không liên quan (với điểm hiệu chỉnh lại thấp) sẽ được giảm tỷ lệ trong khi các đặc trưng liên quan (với điểm hiệu chỉnh lại gần 1) được giữ nguyên.

Các kiến trúc đáng chú ý khác

Có nhiều kiến trúc CNN khác để khám phá. Dưới đây là tổng quan ngắn gọn về một số kiến trúc đáng chú ý nhất:

ResNeXt

ResNeXt cải thiện các đơn vị dư thừa trong ResNet. Trong khi các đơn vị dư thừa trong các mô hình ResNet tốt nhất chỉ chứa 3 lớp tích chập mỗi đơn vị, các đơn vị dư thừa ResNeXt bao gồm nhiều chồng song song (ví dụ: 32 chồng), với 3 lớp tích chập mỗi chồng. Tuy nhiên, hai lớp đầu tiên trong mỗi chồng chỉ sử dụng một vài bộ lọc (ví dụ: chỉ bốn), vì vậy tổng số tham số vẫn giữ nguyên như trong ResNet. Sau đó, đầu ra của tất cả các chồng được cộng lại với nhau, và kết quả được truyền đến đơn vị dư thừa tiếp theo (cùng với kết nối bỏ qua).

DenseNet

Một DenseNet bao gồm một số khối dày đặc, mỗi khối được tạo thành từ một vài lớp tích chập được kết nối dày đặc. Kiến trúc này đạt được độ chính xác tuyệt vời trong khi sử dụng tương đối ít tham số. "Kết nối dày đặc" có nghĩa là gì? Đầu ra của mỗi lớp được đưa vào làm đầu vào cho mọi lớp sau nó trong cùng một khối. Ví dụ, lớp 4 trong một khối lấy làm đầu vào sự nối chiều sâu của đầu ra của các lớp 1, 2 và 3 trong khối đó. Các khối dày đặc được phân tách bởi một vài lớp chuyển tiếp.

MobileNet

MobileNet là các mô hình được sắp xếp hợp lý được thiết kế để nhẹ và nhanh, khiến chúng phổ biến trong các ứng dụng di động và web. Chúng dựa trên các lớp tích chập tách biệt chiều sâu, giống như Xception. Các tác giả đã đề xuất một số biến thể, đánh đổi một chút độ chính xác để có các mô hình nhanh hơn và nhỏ hơn.

CSPNet

Mạng bán phần xuyên giai đoạn (CrossStage Partial Network - CSPNet) tương tự như DenseNet, nhưng một phần đầu vào của mỗi khối dày đặc được nối trực tiếp vào đầu ra của khối đó, mà không đi qua khối.

EfficientNet

EfficientNet có thể nói là mô hình quan trọng nhất trong danh sách này. Các tác giả đã đề xuất một phương pháp để mở rộng bất kỳ CNN nào một cách hiệu quả, bằng cách đồng thời tăng chiều sâu (số lượng lớp), chiều rộng (số lượng bộ lọc trên mỗi lớp) và độ phân giải (kích thước hình ảnh đầu vào) một cách có nguyên tắc. Điều này được gọi là mở rộng tổng hợp (compound scaling). Họ đã sử dụng tìm kiếm kiến trúc nơ-ron để tìm một kiến trúc tốt cho phiên bản ImageNet được thu nhỏ (với ít hình ảnh hơn và nhỏ hơn), và sau đó sử dụng mở rộng tổng hợp để tạo ra các phiên bản lớn hơn và lớn hơn của kiến trúc này. Khi các mô hình EfficientNet ra đời, chúng đã vượt trội đáng kể so với tất cả các mô hình hiện có, trên tất cả các ngân sách tính toán, và chúng vẫn nằm trong số các mô hình tốt nhất hiện nay.

Việc hiểu phương pháp mở rộng tổng hợp của EfficientNet rất hữu ích để có được hiểu biết sâu sắc hơn về CNN, đặc biệt nếu bạn cần mở rộng một kiến trúc CNN. Nó dựa trên một phép đo logarit của ngân sách tính toán, được ký hiệu là ϕ: nếu ngân sách tính toán của bạn tăng gấp đôi, thì ϕ tăng thêm 1. Nói cách khác, số lượng phép toán dấu phẩy động có sẵn để huấn luyện tỷ lệ thuận với 2^{φ} . Chiều sâu, chiều rộng và độ phân giải của kiến trúc CNN của bạn phải tỷ lệ theo α^{φ} , β^{φ} và γ^{φ} tương ứng. Các hệ số α, β và γ phải lớn hơn 1, và $\alpha + \beta^2 + \gamma^2$ phải gần bằng 2. Các giá trị tối ưu cho các hệ số này phụ thuộc vào kiến trúc của CNN. Để tìm các giá trị tối ưu cho kiến trúc EfficientNet, các tác giả bắt đầu với một mô hình cơ sở nhỏ (EfficientNetB0), cố định ϕ = 1, và đơn giản là chạy tìm kiếm lưới: họ tìm thấy α = 1.2, β = 1.1 và γ = 1.1. Sau đó, họ sử dụng các hệ số này để tạo ra một số kiến trúc lớn hơn, được đặt tên là EfficientNetB1 đến EfficientNetB7, cho các giá trị ϕ tăng dần.

Chọn kiến trúc CNN phù hợp

Với rất nhiều kiến trúc CNN, làm thế nào để bạn chọn kiến trúc tốt nhất cho dự án của mình? Vâng, điều đó phụ thuộc vào điều gì quan trọng nhất đối với bạn: Độ chính xác? Kích thước mô hình (ví dụ: để triển khai trên thiết bị di động)? Tốc độ suy luận trên CPU? Trên GPU?

Bảng 14-3 liệt kê các mô hình đã được huấn luyện sẵn tốt nhất hiện có trong Keras (bạn sẽ thấy cách sử dụng chúng sau trong chương này), được sắp xếp theo kích thước mô hình. Bạn có thể tìm thấy danh sách đầy đủ tại https://keras.io/api/applications . Đối với mỗi mô hình, bảng hiển thị tên lớp Keras để sử dụng (trong gói tf.keras.applications), kích thước của mô hình tính bằng MB, độ chính xác xác thực top-1 và top-5 trên tập dữ liệu ImageNet, số lượng tham số (triệu), và thời gian suy luận trên CPU và GPU tính bằng ms, sử dụng các lô 32 hình ảnh trên phần cứng đủ mạnh. Đối với mỗi cột, giá trị tốt nhất được làm nổi bật. Như bạn có thể thấy, các mô hình lớn hơn thường chính xác hơn, nhưng không phải lúc nào cũng vậy; ví dụ, EfficientNetB2 vượt trội hơn InceptionV3 cả về kích thước và độ chính xác.

Tôi chỉ giữ InceptionV3 trong danh sách vì nó nhanh hơn EfficientNetB2 gần gấp đôi trên CPU. Tương tự, InceptionResNetV2 nhanh trên CPU, và ResNet50V2 và ResNet101V2 cực kỳ nhanh trên GPU.

Bảng 14-3. Các mô hình đã được huấn luyện sẵn có trong Keras

Tên lớp Keras	Kích thước (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters (M)	CPU Inference (ms)	GPU Inference (ms)
...	...	...	...	...	...	...
EfficientNetB7	256	84.3%	97.0%	66.7M	1578.9	...
...	...	...	...	...	...	...

Tôi hy vọng bạn thích chuyến đi sâu vào các kiến trúc CNN chính này! Bây giờ hãy xem cách triển khai một trong số chúng bằng Keras.

Triển khai một CNN ResNet-34 bằng Keras

Hầu hết các kiến trúc CNN được mô tả cho đến nay đều có thể được triển khai khá tự nhiên bằng Keras (mặc dù nói chung bạn sẽ tải một mạng đã được huấn luyện sẵn thay vì triển khai thủ công, như bạn sẽ thấy). Để minh họa quá trình này, hãy triển khai một ResNet-34 từ đầu bằng Keras. Đầu tiên, chúng ta sẽ tạo một lớp `ResidualUnit`:

```
DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, strides=1,
                        padding="same", kernel_initializer="he_normal",
use_bias=False)

class ResidualUnit(tf.keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = tf.keras.activations.get(activation)
        self.main_layers = [
            DefaultConv2D(filters, strides=strides),
            tf.keras.layers.BatchNormalization(),
            self.activation,
            DefaultConv2D(filters),
            tf.keras.layers.BatchNormalization()
        ]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                DefaultConv2D(filters, kernel_size=1, strides=strides),
                tf.keras.layers.BatchNormalization()
            ]

    def call(self, inputs):
```

```
        Z = inputs
        for layer in self.main_layers:
            Z = layer(Z)
        skip_Z = inputs
        for layer in self.skip_layers:
            skip_Z = layer(skip_Z)
        return self.activation(Z + skip_Z)
```

Như bạn có thể thấy, mã này khá khớp với Hình 14-19. Trong hàm khởi tạo, chúng ta tạo tất cả các lớp mà chúng ta sẽ cần: các lớp chính là các lớp ở phía bên phải của sơ đồ, và các lớp bỏ qua là các lớp ở phía bên trái (chỉ cần thiết nếu stride lớn hơn 1). Sau đó, trong phương thức `call()`, chúng ta cho đầu vào đi qua các lớp chính và các lớp bỏ qua (nếu có), và chúng ta cộng cả hai đầu ra và áp dụng hàm kích hoạt.

Bây giờ chúng ta có thể xây dựng một ResNet-34 bằng cách sử dụng mô hình `Sequential`, vì nó thực sự chỉ là một chuỗi dài các lớp—chúng ta có thể coi mỗi đơn vị dư thừa là một lớp duy nhất bây giờ khi chúng ta có lớp `ResidualUnit`. Mã này khớp chặt chẽ với Hình 14-18:

```
model = tf.keras.Sequential([
    DefaultConv2D(64, kernel_size=7, strides=2, input_shape=[224, 224, 3]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.MaxPool2D(pool_size=3, strides=2, padding="same"),
])
prev_filters = 64

for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3:
    strides = 1 if filters == prev_filters else 2
    model.add(ResidualUnit(filters, strides=strides))
    prev_filters = filters

model.add(tf.keras.layers.GlobalAvgPool2D())
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Phần khó nhất trong mã này là vòng lặp thêm các lớp `ResidualUnit` vào mô hình: như đã giải thích trước đó, 3 RU đầu tiên có 64 bộ lọc, sau đó 4 RU tiếp theo có 128 bộ lọc, v.v. Ở mỗi lần lặp, chúng ta phải đặt stride thành 1 khi số lượng bộ lọc giống như trong RU trước, hoặc chúng ta đặt nó thành 2; sau đó chúng ta thêm `ResidualUnit`, và cuối cùng chúng ta cập nhật `prev_filters`.

Thật tuyệt vời khi chỉ trong khoảng 40 dòng mã, chúng ta có thể xây dựng mô hình đã giành chiến thắng thử thách ILSVRC 2015! Điều này chứng tỏ cả sự tinh tế của mô hình ResNet và tính biểu cảm của API Keras. Việc triển khai các kiến trúc CNN khác dài hơn một chút, nhưng không khó hơn nhiều. Tuy nhiên, Keras đi kèm với một số kiến trúc này được tích hợp sẵn, vậy tại sao không sử dụng chúng thay thế?

Sử dụng các mô hình đã được huấn luyện sẵn từ Keras

Nói chung, bạn sẽ không phải triển khai các mô hình tiêu chuẩn như GoogLeNet hoặc ResNet theo cách thủ công, vì các mạng đã được huấn luyện sẵn có sẵn chỉ với một dòng mã trong gói `tf.keras.applications`.

Ví dụ, bạn có thể tải mô hình ResNet-50, đã được huấn luyện sẵn trên ImageNet, với dòng mã sau:

```
model = tf.keras.applications.ResNet50(weights="imagenet")
```

Vậy thôi! Điều này sẽ tạo một mô hình ResNet-50 và tải xuống các trọng số đã được huấn luyện sẵn trên tập dữ liệu ImageNet. Để sử dụng nó, trước tiên bạn cần đảm bảo rằng các hình ảnh có kích thước phù hợp. Một mô hình ResNet-50 mong đợi các hình ảnh 224 × 224 pixel (các mô hình khác có thể mong đợi các kích thước khác, chẳng hạn như 299 × 299), vì vậy hãy sử dụng lớp `Resizing` của Keras (được giới thiệu trong Chương 13) để thay đổi kích thước hai hình ảnh mẫu (sau khi cắt chúng theo tỷ lệ khung hình mục tiêu):

```
images = load_sample_images()["images"]
images_resized = tf.keras.layers.Resizing(height=224, width=224,
                                          crop_to_aspect_ratio=True)(images)
```

Các mô hình đã được huấn luyện sẵn giả định rằng các hình ảnh được tiền xử lý theo một cách cụ thể. Trong một số trường hợp, chúng có thể mong đợi đầu vào được chia tỷ lệ từ 0 đến 1, hoặc từ –1 đến 1, v.v. Mỗi mô hình cung cấp một hàm `preprocess_input()` mà bạn có thể sử dụng để tiền xử lý hình ảnh của mình. Các hàm này giả định rằng các giá trị pixel gốc nằm trong khoảng từ 0 đến 255, đây là trường hợp ở đây:

```
inputs = tf.keras.applications.resnet50.preprocess_input(images_resized)
```

Bây giờ chúng ta có thể sử dụng mô hình đã được huấn luyện sẵn để đưa ra dự đoán:

```
>>> Y_proba = model.predict(inputs)
>>> Y_proba.shape
(2, 1000)
```

Như thường lệ, đầu ra `Y_proba` là một ma trận với một hàng trên mỗi hình ảnh và một cột trên mỗi lớp (trong trường hợp này, có 1.000 lớp). Nếu bạn muốn hiển thị K dự đoán hàng đầu, bao gồm tên lớp và xác suất ước tính của mỗi lớp được dự đoán, hãy sử dụng hàm `decode_predictions()`. Đối với mỗi hình ảnh, nó trả về một mảng chứa K dự đoán hàng đầu, trong đó mỗi dự đoán được biểu diễn dưới dạng một mảng chứa định danh lớp, tên của nó và điểm tin cậy tương ứng:

```
top_K = tf.keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print(f"Image #{image_index}")
    for class_id, name, y_proba in top_K[image_index]:
        print(f" {class_id} - {name:12s} {y_proba:.2%}")
```

Đầu ra trông như thế này:

```
Image #0
 n03877845 - palace       54.69%
 n03781244 - monastery    24.72%
 n02825657 - bell_cote    18.55%
Image #1
 n04522168 - vase         32.66%
 n11939491 - daisy        17.81%
 n03530642 - honeycomb    12.06%
```

Các lớp chính xác là palace và dahlia, vì vậy mô hình đúng cho hình ảnh đầu tiên nhưng sai cho hình ảnh thứ hai. Tuy nhiên, đó là do dahlia không phải là một trong 1.000 lớp ImageNet. Với điều đó, vase là một phỏng đoán hợp lý (có lẽ bông hoa nằm trong một chiếc bình?), và daisy cũng không phải là một lựa chọn tồi, vì dahlia và daisy đều thuộc cùng họ Compositae.

Như bạn có thể thấy, rất dễ dàng để tạo một bộ phân loại hình ảnh khá tốt bằng cách sử dụng một mô hình đã được huấn luyện sẵn. Như bạn đã thấy trong Bảng 14-3, nhiều mô hình thị giác khác có sẵn trong `tf.keras.applications`, từ các mô hình nhẹ và nhanh đến các mô hình lớn và chính xác. Nhưng điều gì sẽ xảy ra nếu bạn muốn sử dụng bộ phân loại hình ảnh cho các lớp hình ảnh không phải là một phần của ImageNet? Trong trường hợp đó, bạn vẫn có thể hưởng lợi từ các mô hình đã được huấn luyện sẵn bằng cách sử dụng chúng để thực hiện học chuyển đổi.

Học chuyển đổi với các mô hình đã được huấn luyện sẵn

Nếu bạn muốn xây dựng một bộ phân loại hình ảnh nhưng không có đủ dữ liệu để huấn luyện nó từ đầu, thì thường là một ý kiến hay khi tái sử dụng các lớp thấp hơn của một mô hình đã được huấn luyện sẵn, như chúng ta đã thảo luận trong Chương 11.

Ví dụ, hãy huấn luyện một mô hình để phân loại hình ảnh hoa, tái sử dụng một mô hình Xception đã được huấn luyện sẵn. Đầu tiên, chúng ta sẽ tải tập dữ liệu hoa bằng cách sử dụng TensorFlow Datasets (được giới thiệu trong Chương 13):

```
import tensorflow_datasets as tfds

dataset, info = tfds.load("tf_flowers", as_supervised=True, with_info=True)
dataset_size = info.splits["train"].num_examples  # 3670
class_names = info.features["label"].names   # ["dandelion", "daisy", ...]
n_classes = info.features["label"].num_classes   # 5
```

Lưu ý rằng bạn có thể nhận thông tin về tập dữ liệu bằng cách đặt `with_info=True`. Ở đây, chúng ta nhận được kích thước tập dữ liệu và tên của các lớp. Thật không may, chỉ có tập dữ liệu "train", không có tập kiểm tra hoặc tập xác thực, vì vậy chúng ta cần chia tập huấn luyện. Hãy gọi `tfds.load()` một lần nữa, nhưng lần này lấy 10% đầu tiên của tập dữ liệu để kiểm tra, 15% tiếp theo để xác thực và 75% còn lại để huấn luyện:

```
test_set_raw, valid_set_raw, train_set_raw = tfds.load(
    "tf_flowers",
    split=["train[:10%]", "train[10%:25%]", "train[25%:]"],
    as_supervised=True
)
```

Tất cả ba tập dữ liệu đều chứa các hình ảnh riêng lẻ. Chúng ta cần nhóm chúng lại (batch), nhưng trước tiên chúng ta cần đảm bảo tất cả chúng có cùng kích thước, nếu không việc nhóm sẽ thất bại. Chúng ta có thể sử dụng lớp `Resizing` cho việc này. Chúng ta cũng phải gọi hàm `tf.keras.applications.xception.preprocess_input()` để tiền xử lý hình ảnh một cách thích hợp cho mô hình Xception. Cuối cùng, chúng ta cũng sẽ xáo trộn tập huấn luyện và sử dụng prefetching:

```
batch_size = 32
preprocess = tf.keras.Sequential([
    tf.keras.layers.Resizing(height=224, width=224,
                             crop_to_aspect_ratio=True),
    tf.keras.layers.Lambda(tf.keras.applications.xception.preprocess_input)
])

train_set = train_set_raw.map(lambda X, y: (preprocess(X), y))
train_set = train_set.shuffle(1000, seed=42).batch(batch_size).prefetch(1)
valid_set = valid_set_raw.map(lambda X, y: (preprocess(X),
y)).batch(batch_size)
test_set = test_set_raw.map(lambda X, y: (preprocess(X),
y)).batch(batch_size)
```

Bây giờ mỗi lô chứa 32 hình ảnh, tất cả đều có kích thước 224 × 224 pixel, với các giá trị pixel trong khoảng từ –1 đến 1. Hoàn hảo!

Vì tập dữ liệu không quá lớn, một chút tăng cường dữ liệu chắc chắn sẽ hữu ích. Hãy tạo một mô hình tăng cường dữ liệu mà chúng ta sẽ nhúng vào mô hình cuối cùng của mình. Trong quá trình huấn luyện, nó sẽ ngẫu nhiên lật hình ảnh theo chiều ngang, xoay chúng một chút và điều chỉnh độ tương phản:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),
    tf.keras.layers.RandomRotation(factor=0.05, seed=42),
    tf.keras.layers.RandomContrast(factor=0.2, seed=42)
])
```

Tiếp theo, hãy tải một mô hình Xception, đã được huấn luyện sẵn trên ImageNet. Chúng ta loại trừ phần trên cùng của mạng bằng cách đặt `include_top=False`. Điều này loại trừ lớp gộp trung bình toàn cục và lớp đầu ra dày đặc. Sau đó, chúng ta thêm lớp gộp trung bình toàn cục của riêng mình (cấp cho nó đầu ra của mô hình cơ sở), theo sau bởi một lớp đầu ra dày đặc với một đơn vị trên mỗi lớp, sử dụng hàm kích hoạt softmax.

Cuối cùng, chúng ta gói tất cả điều này trong một `Keras Model`:

```
import tensorflow as tf

base_model = tf.keras.applications.xception.Xception(weights="imagenet",
                                                     include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
model = tf.keras.Model(inputs=base_model.input, outputs=output)
```

Như đã giải thích trong Chương 11, thường là một ý kiến hay khi đóng băng các trọng số của các lớp đã được huấn luyện sẵn, ít nhất là ở đầu quá trình huấn luyện:

```
for layer in base_model.layers:
    layer.trainable = False
```

Cuối cùng, chúng ta có thể biên dịch mô hình và bắt đầu huấn luyện:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=3)
```

Sau khi huấn luyện mô hình trong vài epoch, độ chính xác xác thực của nó sẽ đạt hơn 80% một chút và sau đó ngừng cải thiện. Điều này có nghĩa là các lớp trên cùng hiện đã được huấn luyện khá tốt, và chúng ta đã sẵn sàng bỏ đóng băng một số lớp trên cùng của mô hình cơ sở, sau đó tiếp tục huấn luyện. Ví dụ, hãy bỏ đóng băng các lớp 56 trở lên (đó là khởi đầu của đơn vị dư thừa 7 trên 14, như bạn có thể thấy nếu liệt kê tên các lớp):

```
for layer in base_model.layers[56:]:
    layer.trainable = True
```

Đừng quên biên dịch mô hình mỗi khi bạn đóng băng hoặc bỏ đóng băng các lớp. Ngoài ra, hãy đảm bảo sử dụng tốc độ học thấp hơn nhiều để tránh làm hỏng các trọng số đã được huấn luyện sẵn:

```
optimizer    =    tf.keras.optimizers.SGD(learning_rate=0.01,    momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy",    optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=10)
```

Mô hình này sẽ đạt khoảng 92% độ chính xác trên tập kiểm tra, chỉ trong vài phút huấn luyện (với GPU). Nếu bạn điều chỉnh các siêu tham số, giảm tốc độ học và huấn luyện lâu hơn một chút, bạn sẽ có thể đạt 95% đến 97%. Với điều đó, bạn có thể bắt đầu huấn luyện các bộ phân loại hình ảnh tuyệt vời trên hình ảnh và các lớp của riêng mình! Nhưng thị giác máy tính còn nhiều điều hơn là chỉ phân loại. Ví dụ, điều gì sẽ xảy ra nếu bạn cũng muốn biết bông hoa nằm ở đâu trong một bức ảnh? Hãy xem xét điều này ngay bây giờ.

Phân loại và định vị

Định vị một đối tượng trong một bức ảnh có thể được biểu thị dưới dạng một tác vụ hồi quy, như đã thảo luận trong Chương 10: để dự đoán một hộp bao quanh đối tượng, một cách

tiếp cận phổ biến là dự đoán tọa độ ngang và dọc của tâm đối tượng, cũng như chiều cao và chiều rộng của nó. Điều này có nghĩa là chúng ta có bốn số cần dự đoán. Nó không yêu cầu nhiều thay đổi đối với mô hình; chúng ta chỉ cần thêm một lớp đầu ra dày đặc thứ hai với bốn đơn vị (thường nằm trên lớp gộp trung bình toàn cục), và nó có thể được huấn luyện bằng hàm mất mát MSE:

```
base_model = tf.keras.applications.xception.Xception(weights="imagenet",
                                                     include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
class_output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
loc_output = tf.keras.layers.Dense(4)(avg)  # 4 units for x, y, width, height
model = tf.keras.Model(inputs=base_model.input,
                       outputs=[class_output, loc_output])
model.compile(loss=["sparse_categorical_crossentropy", "mse"],
              loss_weights=[0.8, 0.2],  # depends on what you care most about
              optimizer=optimizer, metrics=["accuracy"])
```

Nhưng bây giờ chúng ta có một vấn đề: tập dữ liệu hoa không có hộp bao quanh các bông hoa. Vì vậy, chúng ta cần tự thêm chúng vào. Đây thường là một trong những phần khó khăn và tốn kém nhất của một dự án học máy: lấy nhãn. Nên dành thời gian tìm kiếm các công cụ phù hợp. Để chú thích hình ảnh với hộp bao quanh, bạn có thể muốn sử dụng một công cụ gắn nhãn hình ảnh mã nguồn mở như VGG Image Annotator, LabelImg, OpenLabeler, hoặc ImgLab, hoặc có lẽ một công cụ thương mại như LabelBox hoặc Supervisely. Bạn cũng có thể muốn xem xét các nền tảng crowdsourcing như Amazon Mechanical Turk nếu bạn có số lượng hình ảnh rất lớn cần chú thích. Tuy nhiên, việc thiết lập một nền tảng crowdsourcing, chuẩn bị biểu mẫu để gửi cho người lao động, giám sát họ và đảm bảo chất lượng của các hộp bao quanh mà họ tạo ra là tốt, là khá nhiều công việc, vì vậy hãy đảm bảo rằng nó đáng công sức.

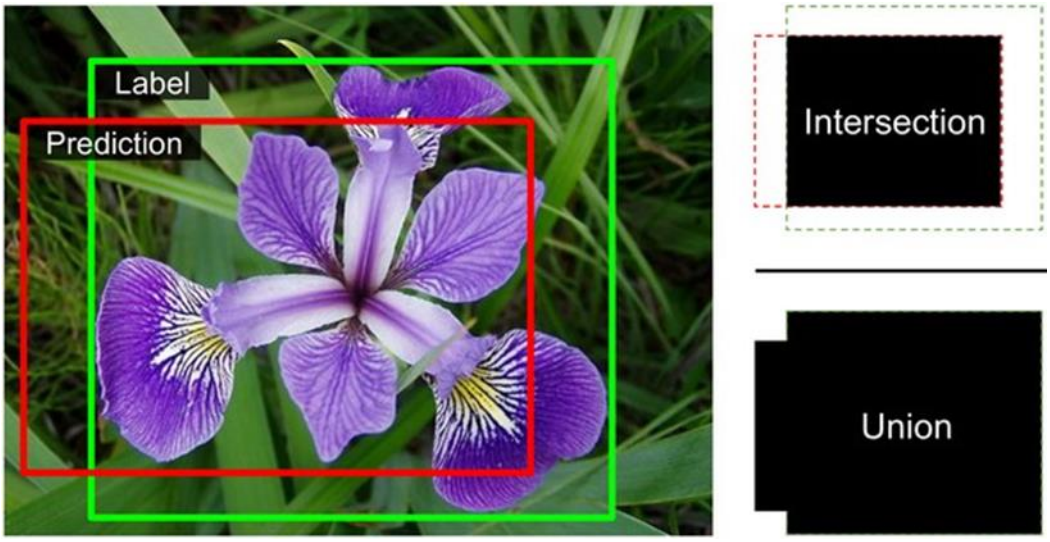
Adriana Kovashka và cộng sự đã viết một bài báo rất thực tế về crowdsourcing trong thị giác máy tính. Tôi khuyên bạn nên xem nó, ngay cả khi bạn không có kế hoạch sử dụng crowdsourcing. Nếu chỉ có vài trăm hoặc thậm chí vài nghìn hình ảnh cần gắn nhãn, và bạn không có kế hoạch làm điều này thường xuyên, có thể tốt hơn là tự làm: với các công cụ phù hợp, nó sẽ chỉ mất vài ngày, và bạn cũng sẽ hiểu rõ hơn về tập dữ liệu và nhiệm vụ của mình.

Bây giờ, giả sử bạn đã có được các hộp bao quanh cho mỗi hình ảnh trong tập dữ liệu hoa (hiện tại chúng ta sẽ giả định có một hộp bao quanh duy nhất trên mỗi hình ảnh). Sau đó, bạn cần tạo một tập dữ liệu có các mục sẽ là các lô hình ảnh đã được tiền xử lý cùng với nhãn lớp và hộp bao quanh của chúng. Mỗi mục phải là một tuple có dạng `(images, (class_labels, bounding_boxes))`. Sau đó, bạn đã sẵn sàng để huấn luyện mô hình của mình!

MSE thường hoạt động khá tốt như một hàm chi phí để huấn luyện mô hình, nhưng nó không phải là một thước đo tuyệt vời để đánh giá mức độ mô hình có thể dự đoán hộp bao quanh. Thước đo phổ biến nhất cho điều này là Intersection over Union (IoU): diện tích chồng chéo giữa hộp bao quanh được dự đoán và hộp bao quanh mục tiêu, chia cho diện

tích hợp của chúng (xem Hình 14-24). Trong Keras, nó được triển khai bởi lớp `tf.keras.metrics.MeanIoU`.

Phân loại và định vị một đối tượng duy nhất là tốt, nhưng điều gì sẽ xảy ra nếu hình ảnh chứa nhiều đối tượng (như thường xảy ra trong tập dữ liệu hoa)?

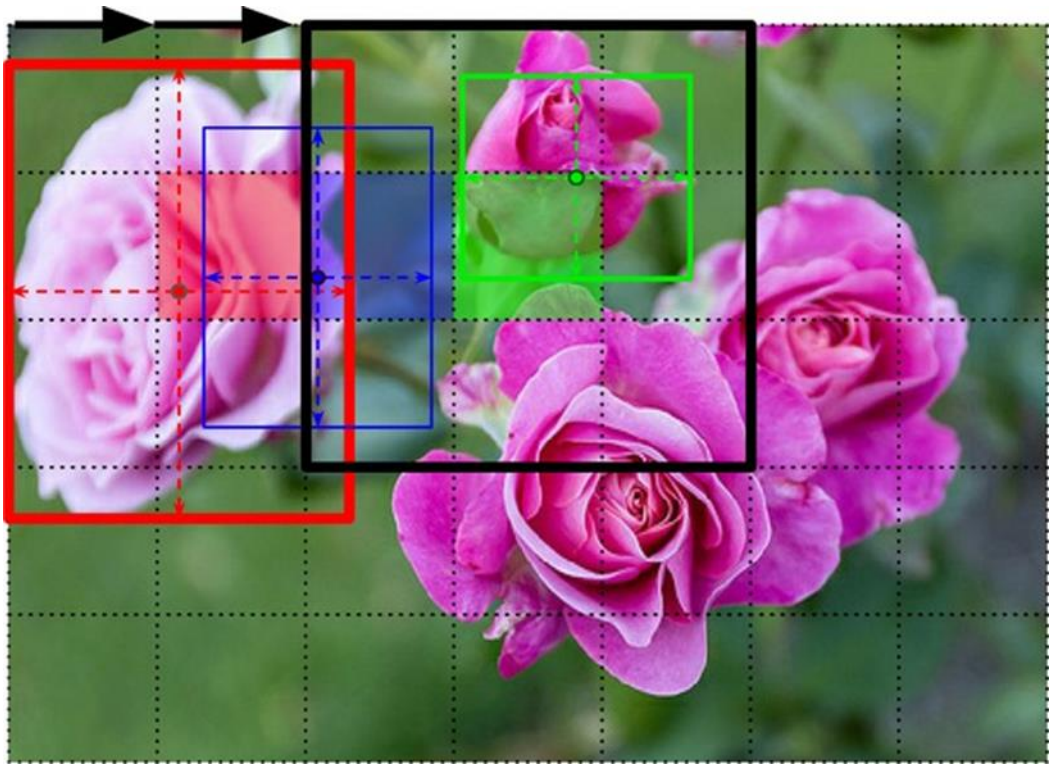


Hình 14-24. Thước đo IoU cho hộp bao quanh.

Phát hiện đối tượng

Nhiệm vụ phân loại và định vị nhiều đối tượng trong một hình ảnh được gọi là phát hiện đối tượng. Cho đến vài năm trước, một cách tiếp cận phổ biến là lấy một CNN đã được huấn luyện để phân loại và định vị một đối tượng duy nhất nằm gần giữa hình ảnh, sau đó trượt CNN này trên hình ảnh và đưa ra dự đoán ở mỗi bước. CNN thường được huấn luyện để dự đoán không chỉ xác suất lớp và một hộp bao quanh, mà còn cả điểm đối tượng (objectness score): đây là xác suất ước tính rằng hình ảnh thực sự chứa một đối tượng nằm gần giữa. Đây là một đầu ra phân loại nhị phân; nó có thể được tạo ra bởi một lớp đầu ra dày đặc với một đơn vị duy nhất, sử dụng hàm kích hoạt sigmoid và được huấn luyện bằng hàm mất mát cross-entropy nhị phân.

Cách tiếp cận CNN trượt này được minh họa trong Hình 14-25. Trong ví dụ này, hình ảnh được chia thành một lưới 5 × 7, và chúng ta thấy một CNN—hình chữ nhật đen dày—trượt qua tất cả các vùng 3 × 3 và đưa ra dự đoán ở mỗi bước.



Hình 14-25. Phát hiện nhiều đối tượng bằng cách trượt CNN trên hình ảnh.

Trong hình này, CNN đã đưa ra dự đoán cho ba trong số các vùng 3 × 3 này:

- Khi nhìn vào vùng 3 × 3 phía trên bên trái (tâm ở ô lưới màu đỏ nằm ở hàng thứ hai và cột thứ hai), nó đã phát hiện ra bông hồng ngoài cùng bên trái. Lưu ý rằng hộp bao quanh được dự đoán vượt quá ranh giới của vùng 3 × 3 này. Điều đó hoàn toàn bình thường: mặc dù CNN không thể nhìn thấy phần dưới của bông hồng, nhưng nó vẫn có thể đưa ra một phỏng đoán hợp lý về vị trí của nó. Nó cũng dự đoán xác suất lớp, cho xác suất cao cho lớp “rose”. Cuối cùng, nó dự đoán điểm đối tượng khá cao, vì tâm của hộp bao quanh nằm trong ô lưới trung tâm (trong hình này, điểm đối tượng được biểu thị bằng độ dày của hộp bao quanh).
- Khi nhìn vào vùng 3 × 3 tiếp theo, một ô lưới sang phải (tâm ở ô vuông màu xanh lam được tô bóng), nó không phát hiện bất kỳ bông hoa nào có tâm trong vùng đó, vì vậy nó dự đoán điểm đối tượng rất thấp; do đó, hộp bao quanh và xác suất lớp được dự đoán có thể bỏ qua một cách an toàn. Bạn có thể thấy rằng hộp bao quanh được dự đoán không tốt chút nào.
- Cuối cùng, khi nhìn vào vùng 3 × 3 tiếp theo, một lần nữa một ô lưới sang phải (tâm ở ô màu xanh lá cây được tô bóng), nó đã phát hiện bông hồng ở phía trên, mặc dù không hoàn hảo: bông hồng này không được căn giữa tốt trong vùng này, vì vậy điểm đối tượng được dự đoán không cao lắm.

Bạn có thể hình dung việc trượt CNN trên toàn bộ hình ảnh sẽ cho bạn tổng cộng 15 hộp bao quanh được dự đoán, được sắp xếp trong một lưới 3 × 5, với mỗi hộp bao quanh kèm theo xác suất lớp ước tính và điểm đối tượng của nó. Vì các đối tượng có thể có kích thước

khác nhau, bạn có thể muốn trượt lại CNN trên các vùng 4 × 4 lớn hơn, để có được nhiều hộp bao quanh hơn.

Kỹ thuật này khá đơn giản, nhưng như bạn có thể thấy nó sẽ thường phát hiện cùng một đối tượng nhiều lần, ở các vị trí hơi khác nhau. Cần có một số xử lý hậu kỳ để loại bỏ tất cả các hộp bao quanh không cần thiết. Một cách tiếp cận phổ biến cho việc này được gọi là **non-max suppression**. Đây là cách nó hoạt động:

11. Đầu tiên, loại bỏ tất cả các hộp bao quanh mà điểm đối tượng thấp hơn một ngưỡng nào đó: vì CNN tin rằng không có đối tượng ở vị trí đó, hộp bao quanh là vô dụng.
12. Tìm hộp bao quanh còn lại có điểm đối tượng cao nhất, và loại bỏ tất cả các hộp bao quanh còn lại khác chồng lấn nhiều với nó (ví dụ: với IoU lớn hơn 60%). Ví dụ, trong Hình 14-25, hộp bao quanh có điểm đối tượng max là hộp bao quanh dày trên bông hồng ngoài cùng bên trái. Hộp bao quanh khác chạm vào cùng bông hồng này chồng lấn nhiều với hộp bao quanh max, vì vậy chúng ta sẽ loại bỏ nó (mặc dù trong ví dụ này nó đã được loại bỏ ở bước trước).
13. Lặp lại bước 2 cho đến khi không còn hộp bao quanh nào để loại bỏ.

Cách tiếp cận đơn giản này để phát hiện đối tượng hoạt động khá tốt, nhưng nó yêu cầu chạy CNN nhiều lần (15 lần trong ví dụ này), vì vậy nó khá chậm. May mắn thay, có một cách nhanh hơn nhiều để trượt CNN trên một hình ảnh: sử dụng **mạng hoàn toàn tích chập (FCN)**.

Mạng hoàn toàn tích chập

Ý tưởng về FCN lần đầu tiên được giới thiệu trong một bài báo năm 2015 của Jonathan Long et al., cho phân đoạn ngữ nghĩa (nhiệm vụ phân loại mọi pixel trong một hình ảnh theo lớp của đối tượng mà pixel đó thuộc về). Các tác giả đã chỉ ra rằng bạn có thể thay thế các lớp dày đặc ở trên cùng của một CNN bằng các lớp tích chập. Để hiểu điều này, hãy xem một ví dụ: giả sử một lớp dày đặc với 200 nơ-ron nằm trên một lớp tích chập xuất ra 100 bản đồ đặc trưng, mỗi bản đồ có kích thước 7 × 7 (đây là kích thước bản đồ đặc trưng, không phải kích thước kernel). Mỗi nơ-ron sẽ tính tổng trọng số của tất cả 100 × 7 × 7 kích hoạt từ lớp tích chập (cộng với một thuật ngữ bias). Bây giờ hãy xem điều gì xảy ra nếu chúng ta thay thế lớp dày đặc bằng một lớp tích chập sử dụng 200 bộ lọc, mỗi bộ lọc có kích thước 7 × 7, và với padding “valid”. Lớp này sẽ xuất ra 200 bản đồ đặc trưng, mỗi bản đồ có kích thước 1 × 1 (vì kernel có kích thước chính xác bằng bản đồ đặc trưng đầu vào và chúng ta đang sử dụng padding “valid”). Nói cách khác, nó sẽ xuất ra 200 số, giống như lớp dày đặc đã làm; và nếu bạn nhìn kỹ vào các phép tính được thực hiện bởi một lớp tích chập, bạn sẽ nhận thấy rằng các số này sẽ chính xác giống như những gì lớp dày đặc đã tạo ra. Điểm khác biệt duy nhất là đầu ra của lớp dày đặc là một tensor có hình dạng `[kích thước lô, 200]`, trong khi lớp tích chập sẽ xuất ra một tensor có hình dạng `[kích thước lô, 1, 1, 200]`.

Tại sao điều này lại quan trọng? Vâng, trong khi một lớp dày đặc mong đợi một kích thước đầu vào cụ thể (vì nó có một trọng số trên mỗi tính năng đầu vào), một lớp tích chập sẽ vui vẻ xử lý hình ảnh ở bất kỳ kích thước nào (tuy nhiên, nó mong đợi đầu vào của nó có một số lượng kênh cụ thể, vì mỗi kernel chứa một tập hợp trọng số khác nhau cho mỗi kênh đầu

vào). Vì FCN chỉ chứa các lớp tích chập (và các lớp gộp, có cùng thuộc tính), nó có thể được huấn luyện và thực thi trên hình ảnh ở bất kỳ kích thước nào!

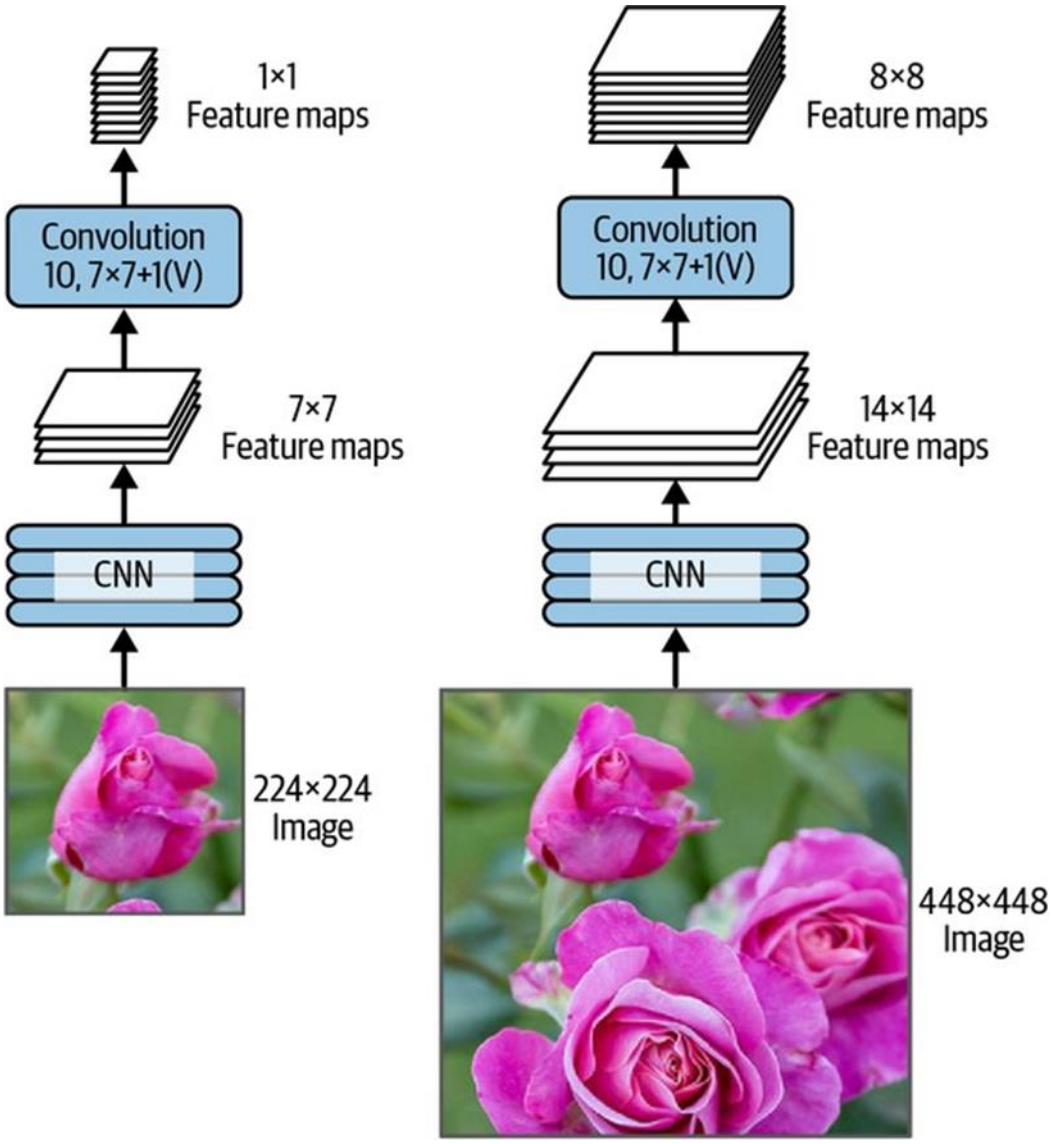
Ví dụ, giả sử chúng ta đã huấn luyện một CNN để phân loại và định vị hoa. Nó được huấn luyện trên hình ảnh 224 × 224, và nó xuất ra 10 số:

- Đầu ra 0 đến 4 được gửi qua hàm kích hoạt softmax, và điều này cho các xác suất lớp (một trên mỗi lớp).
- Đầu ra 5 được gửi qua hàm kích hoạt sigmoid, và điều này cho điểm đối tượng.
- Đầu ra 6 và 7 đại diện cho tọa độ tâm của hộp bao quanh; chúng cũng đi qua hàm kích hoạt sigmoid để đảm bảo chúng nằm trong khoảng từ 0 đến 1.
- Cuối cùng, đầu ra 8 và 9 đại diện cho chiều cao và chiều rộng của hộp bao quanh; chúng không đi qua bất kỳ hàm kích hoạt nào để cho phép các hộp bao quanh mở rộng vượt ra ngoài ranh giới của hình ảnh.

Bây giờ chúng ta có thể chuyển đổi các lớp dày đặc của CNN thành các lớp tích chập. Trên thực tế, chúng ta thậm chí không cần huấn luyện lại nó; chúng ta chỉ cần sao chép các trọng số từ các lớp dày đặc sang các lớp tích chập! Hoặc, chúng ta có thể đã chuyển đổi CNN thành FCN trước khi huấn luyện.

Bây giờ giả sử lớp tích chập cuối cùng trước lớp đầu ra (còn gọi là lớp thắt cổ chai) xuất ra các bản đồ đặc trưng 7 × 7 khi mạng được cấp một hình ảnh 224 × 224 (xem phía bên trái của Hình 14-26). Nếu chúng ta cấp FCN một hình ảnh 448 × 448 (xem phía bên phải của Hình 14-26), lớp thắt cổ chai bây giờ sẽ xuất ra các bản đồ đặc trưng 14 × 14. Vì lớp đầu ra dày đặc đã được thay thế bằng một lớp tích chập sử dụng 10 bộ lọc có kích thước 7 × 7, với padding “valid” và stride 1, đầu ra sẽ bao gồm 10 bản đồ tính năng, mỗi bản đồ có kích thước 8 × 8 (vì 14 – 7 + 1 = 8). Nói cách khác, FCN sẽ xử lý toàn bộ hình ảnh chỉ một lần, và nó sẽ xuất ra một lưới 8 × 8 trong đó mỗi ô chứa 10 số (5 xác suất lớp, 1 điểm đối tượng, và 4 tọa độ hộp bao quanh). Nó giống hệt như lấy CNN gốc và trượt nó trên hình ảnh bằng cách sử dụng 8 bước trên mỗi hàng và 8 bước trên mỗi cột. Để hình dung điều này, hãy tưởng tượng cắt hình ảnh gốc thành một lưới 14 × 14, sau đó trượt một cửa sổ 7 × 7 trên lưới này; sẽ có 8 × 8 = 64 vị trí có thể cho cửa sổ, do đó có 8 × 8 dự đoán.

Tuy nhiên, cách tiếp cận FCN hiệu quả hơn nhiều, vì mạng chỉ nhìn vào hình ảnh một lần. Trên thực tế, You Only Look Once (YOLO) là tên của một kiến trúc phát hiện đối tượng rất phổ biến, chúng ta sẽ xem xét tiếp theo.



Hình 14-26. Cùng một mạng hoàn toàn tích chập xử lý một hình ảnh nhỏ (trái) và một hình ảnh lớn (phải).

You Only Look Once

YOLO là một kiến trúc phát hiện đối tượng nhanh và chính xác được Joseph Redmon và cộng sự đề xuất trong một bài báo năm 2015. Nó nhanh đến mức có thể chạy trong thời gian thực trên một video, như đã thấy trong bản demo của Redmon. Kiến trúc của YOLO khá giống với kiến trúc chúng ta vừa thảo luận, nhưng có một vài khác biệt quan trọng:

- Đối với mỗi ô lưới, YOLO chỉ xem xét các đối tượng có tâm hộp bao quanh nằm trong ô đó. Tọa độ hộp bao quanh tương đối so với ô đó, trong đó (0, 0) có nghĩa là góc trên bên trái của ô và (1, 1) có nghĩa là góc dưới bên phải. Tuy nhiên, chiều cao và chiều rộng của hộp bao quanh có thể mở rộng vượt ra ngoài ô.

- Nó xuất ra hai hộp bao quanh cho mỗi ô lưới (thay vì chỉ một), cho phép mô hình xử lý các trường hợp hai đối tượng quá gần nhau đến mức tâm hộp bao quanh của chúng nằm trong cùng một ô. Mỗi hộp bao quanh cũng đi kèm với điểm đối tượng riêng của nó.
- YOLO cũng xuất ra phân phối xác suất lớp cho mỗi ô lưới, dự đoán 20 xác suất lớp trên mỗi ô lưới vì YOLO được huấn luyện trên tập dữ liệu PASCAL VOC, chứa 20 lớp. Điều này tạo ra một bản đồ xác suất lớp thô. Lưu ý rằng mô hình dự đoán một phân phối xác suất lớp trên mỗi ô lưới, không phải trên mỗi hộp bao quanh. Tuy nhiên, có thể ước tính xác suất lớp cho mỗi hộp bao quanh trong quá trình xử lý hậu kỳ, bằng cách đo mức độ phù hợp của mỗi hộp bao quanh với mỗi lớp trong bản đồ xác suất lớp. Ví dụ, hãy tưởng tượng một bức ảnh một người đứng trước một chiếc ô tô. Sẽ có hai hộp bao quanh: một hộp lớn hình ngang cho chiếc ô tô, và một hộp nhỏ hơn hình dọc cho người. Các hộp bao quanh này có thể có tâm trong cùng một ô lưới. Vậy làm thế nào chúng ta có thể biết lớp nào nên được gán cho mỗi hộp bao quanh? Vâng, bản đồ xác suất lớp sẽ chứa một vùng lớn nơi lớp “car” chiếm ưu thế, và bên trong nó sẽ có một vùng nhỏ hơn nơi lớp “person” chiếm ưu thế. Hy vọng rằng, hộp bao quanh của chiếc ô tô sẽ khớp khoảng với vùng “car”, trong khi hộp bao quanh của người sẽ khớp khoảng với vùng “person”: điều này sẽ cho phép gán đúng lớp cho mỗi hộp bao quanh.

YOLO ban đầu được phát triển bằng Darknet, một framework học sâu mã nguồn mở ban đầu được Joseph Redmon phát triển bằng C, nhưng nó nhanh chóng được chuyển sang TensorFlow, Keras, PyTorch và nhiều hơn nữa. Nó liên tục được cải thiện trong những năm qua, với YOLOv2, YOLOv3 và YOLO9000 (cũng bởi Joseph Redmon và cộng sự), YOLOv4 (bởi Alexey Bochkovskiy và cộng sự), YOLOv5 (bởi Glenn Jocher) và PP-YOLO (bởi Xiang Long và cộng sự).

Mỗi phiên bản mang lại một số cải tiến ấn tượng về tốc độ và độ chính xác, sử dụng nhiều kỹ thuật khác nhau; ví dụ, YOLOv3 đã tăng cường độ chính xác một phần nhờ các neo tiên nghiệm (anchor priors), khai thác thực tế rằng một số hình dạng hộp bao quanh có nhiều khả năng hơn các hình dạng khác, tùy thuộc vào lớp (ví dụ: người có xu hướng có hộp bao quanh hình dọc, trong khi ô tô thường không). Họ cũng tăng số lượng hộp bao quanh trên mỗi ô lưới, họ huấn luyện trên các tập dữ liệu khác nhau với nhiều lớp hơn (lên đến 9.000 lớp được tổ chức theo hệ thống phân cấp trong trường hợp YOLO9000), họ thêm các kết nối bỏ qua để khôi phục một số độ phân giải không gian bị mất trong CNN (chúng ta sẽ thảo luận ngắn gọn về điều này, khi chúng ta xem xét phân đoạn ngữ nghĩa), và nhiều hơn nữa. Cũng có nhiều biến thể của các mô hình này, chẳng hạn như YOLOv4-tiny, được tối ưu hóa để được huấn luyện trên các máy kém mạnh hơn và có thể chạy cực nhanh (hơn 1.000 khung hình mỗi giây!), nhưng với độ chính xác trung bình (mAP) hơi thấp hơn.

Độ chính xác trung bình

Một thước đo rất phổ biến được sử dụng trong các tác vụ phát hiện đối tượng là độ chính xác trung bình (mean average precision). “Độ chính xác trung bình” nghe có vẻ hơi thừa, phải không? Để hiểu thước đo này, hãy quay lại hai thước đo phân loại chúng ta đã thảo

luận trong Chương 3: độ chính xác và độ thu hồi. Hãy nhớ sự đánh đổi: độ thu hồi càng cao, độ chính xác càng thấp. Bạn có thể hình dung điều này trong một đường cong độ chính xác/độ thu hồi (xem Hình 3-6). Để tóm tắt đường cong này thành một số duy nhất, chúng ta có thể tính diện tích dưới đường cong (AUC) của nó. Nhưng lưu ý rằng đường cong độ chính xác/độ thu hồi có thể chứa một vài phần mà độ chính xác thực sự tăng lên khi độ thu hồi tăng, đặc biệt ở các giá trị độ thu hồi thấp (bạn có thể thấy điều này ở phía trên bên trái của Hình 3-6). Đây là một trong những động lực cho thước đo mAP.

Giả sử bộ phân loại có độ chính xác 90% ở độ thu hồi 10%, nhưng độ chính xác 96% ở độ thu hồi 20%. Thực sự không có sự đánh đổi nào ở đây: đơn giản là có lý hơn khi sử dụng bộ phân loại ở độ thu hồi 20% hơn là ở độ thu hồi 10%, vì bạn sẽ nhận được cả độ thu hồi cao hơn và độ chính xác cao hơn. Vì vậy, thay vì nhìn vào độ chính xác ở độ thu hồi 10%, chúng ta nên thực sự nhìn vào độ chính xác tối đa mà bộ phân loại có thể cung cấp với ít nhất 10% độ thu hồi. Nó sẽ là 96%, không phải 90%.

Do đó, một cách để có được một ý tưởng công bằng về hiệu suất của mô hình là tính toán độ chính xác tối đa bạn có thể nhận được với ít nhất 0% độ thu hồi, sau đó 10% độ thu hồi, 20%, v.v. lên đến 100%, và sau đó tính trung bình các độ chính xác tối đa này. Điều này được gọi là thước đo độ chính xác trung bình (AP). Bây giờ khi có nhiều hơn hai lớp, chúng ta có thể tính AP cho mỗi lớp, và sau đó tính AP trung bình (mAP). Thế thôi!

Trong một hệ thống phát hiện đối tượng, có một mức độ phức tạp bổ sung: điều gì sẽ xảy ra nếu hệ thống phát hiện đúng lớp, nhưng ở sai vị trí (tức là hộp bao quanh hoàn toàn sai)? Chắc chắn chúng ta không nên coi đây là một dự đoán tích cực. Một cách tiếp cận là định nghĩa một ngưỡng IoU: ví dụ, chúng ta có thể coi một dự đoán là đúng chỉ khi IoU lớn hơn, chẳng hạn, 0,5, và lớp được dự đoán là đúng. mAP tương ứng thường được ký hiệu là mAP@0.5 (hoặc mAP@50%, hoặc đôi khi chỉ AP50). Trong một số cuộc thi (chẳng hạn như thử thách PASCAL VOC), đây là điều được thực hiện. Trong các cuộc thi khác (chẳng hạn như cuộc thi COCO), mAP được tính toán cho các ngưỡng IoU khác nhau (0.50, 0.55, 0.60, ..., 0.95), và thước đo cuối cùng là trung bình của tất cả các mAP này (được ký hiệu là mAP@[.50:.95] hoặc mAP@[.50:0.05:.95]). Vâng, đó là một trung bình của trung bình.

Nhiều mô hình phát hiện đối tượng có sẵn trên TensorFlow Hub, thường với các trọng số đã được huấn luyện sẵn, chẳng hạn như YOLOv5, SSD, Faster R-CNN và EfficientDet. SSD và EfficientDet là các mô hình phát hiện “nhìn một lần”, tương tự như YOLO. EfficientDet dựa trên kiến trúc tích chập EfficientNet. Faster R-CNN phức tạp hơn: hình ảnh đầu tiên đi qua một CNN, sau đó đầu ra được truyền đến một mạng đề xuất vùng (RPN) đề xuất các hộp bao quanh có nhiều khả năng chứa một đối tượng; một bộ phân loại sau đó được chạy cho mỗi hộp bao quanh, dựa trên đầu ra đã cắt của CNN. Nơi tốt nhất để bắt đầu sử dụng các mô hình này là hướng dẫn phát hiện đối tượng tuyệt vời của TensorFlow Hub.

Cho đến nay, chúng ta chỉ xem xét việc phát hiện đối tượng trong các hình ảnh đơn lẻ. Nhưng còn video thì sao? Các đối tượng không chỉ phải được phát hiện trong mỗi khung hình, mà chúng còn phải được theo dõi theo thời gian. Hãy cùng xem nhanh về theo dõi đối tượng ngay bây giờ.

Theo dõi đối tượng

Theo dõi đối tượng là một nhiệm vụ đầy thách thức: các đối tượng di chuyển, chúng có thể lớn lên hoặc co lại khi đến gần hoặc di chuyển ra xa camera, hình dáng của chúng có thể thay đổi khi chúng quay hoặc di chuyển đến các điều kiện ánh sáng hoặc nền khác nhau, chúng có thể tạm thời bị che khuất bởi các đối tượng khác, v.v.

Một trong những hệ thống theo dõi đối tượng phổ biến nhất là DeepSORT. Nó dựa trên sự kết hợp giữa các thuật toán cổ điển và học sâu:

- Nó sử dụng các bộ lọc Kalman để ước tính vị trí hiện tại có khả năng nhất của một đối tượng dựa trên các phát hiện trước đó, và giả định rằng các đối tượng có xu hướng di chuyển với tốc độ không đổi.
- Nó sử dụng một mô hình học sâu để đo lường sự tương đồng giữa các phát hiện mới và các đối tượng đã được theo dõi.
- Cuối cùng, nó sử dụng thuật toán Hungary để ánh xạ các phát hiện mới vào các đối tượng đã được theo dõi (hoặc vào các đối tượng được theo dõi mới): thuật toán này tìm kiếm hiệu quả sự kết hợp các ánh xạ giúp giảm thiểu khoảng cách giữa các phát hiện và vị trí dự đoán của các đối tượng được theo dõi, đồng thời giảm thiểu sự khác biệt về hình dáng.

Ví dụ, hãy tưởng tượng một quả bóng đỏ vừa bật ra khỏi một quả bóng xanh lam đang di chuyển theo hướng ngược lại. Dựa trên các vị trí trước đó của các quả bóng, bộ lọc Kalman sẽ dự đoán rằng các quả bóng sẽ đi xuyên qua nhau: thực tế, nó giả định rằng các đối tượng di chuyển với tốc độ không đổi, vì vậy nó sẽ không mong đợi cú bật. Nếu thuật toán Hungary chỉ xem xét các vị trí, thì nó sẽ vui vẻ ánh xạ các phát hiện mới vào các quả bóng sai, như thể chúng vừa đi xuyên qua nhau và đổi màu. Nhưng nhờ vào thước đo sự tương đồng, thuật toán Hungary sẽ nhận thấy vấn đề. Giả sử các quả bóng không quá giống nhau, thuật toán sẽ ánh xạ các phát hiện mới vào các quả bóng đúng.

Cho đến nay, chúng ta đã định vị các đối tượng bằng cách sử dụng các hộp bao quanh. Điều này thường là đủ, nhưng đôi khi bạn cần định vị các đối tượng với độ chính xác cao hơn nhiều—ví dụ, để loại bỏ nền phía sau một người trong cuộc gọi hội nghị truyền hình. Hãy xem cách đi xuống cấp độ pixel.

Phân đoạn ngữ nghĩa

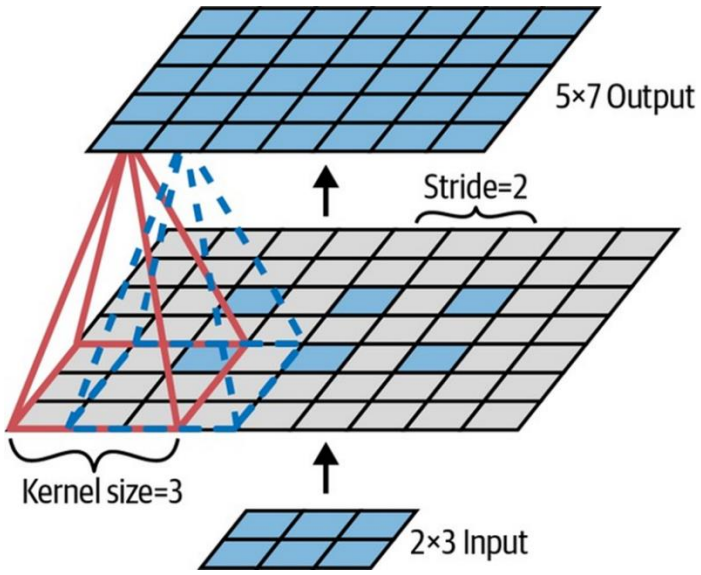
Trong phân đoạn ngữ nghĩa, mỗi pixel được phân loại theo lớp của đối tượng mà nó thuộc về (ví dụ: đường, ô tô, người đi bộ, tòa nhà, v.v.), như thể hiện trong Hình 14-27. Lưu ý rằng các đối tượng khác nhau cùng lớp không được phân biệt. Ví dụ, tất cả các chiếc xe đạp ở phía bên phải của hình ảnh đã phân đoạn cuối cùng trở thành một khối pixel lớn. Khó khăn chính trong nhiệm vụ này là khi hình ảnh đi qua một CNN thông thường, chúng dần dần mất độ phân giải không gian (do các lớp có stride lớn hơn 1); do đó, một CNN thông thường có thể biết rằng có một người ở đâu đó ở phía dưới bên trái của hình ảnh, nhưng nó sẽ không chính xác hơn thế.

Hình 14-27. Phân đoạn ngữ nghĩa.

Giống như đối với phát hiện đối tượng, có nhiều cách tiếp cận khác nhau để giải quyết vấn đề này, một số khá phức tạp. Tuy nhiên, một giải pháp khá đơn giản đã được đề xuất trong bài báo năm 2015 của Jonathan Long et al. mà tôi đã đề cập trước đó, về mạng hoàn toàn tích chập. Các tác giả bắt đầu bằng cách lấy một CNN đã được huấn luyện sẵn và biến nó thành một FCN. CNN áp dụng một stride tổng thể là 32 cho hình ảnh đầu vào (tức là, nếu bạn cộng tất cả các stride lớn hơn 1), có nghĩa là lớp cuối cùng xuất ra các bản đồ đặc trưng nhỏ hơn 32 lần so với hình ảnh đầu vào. Điều này rõ ràng là quá thô, vì vậy họ đã thêm một lớp lấy mẫu lên (upsampling layer) nhân độ phân giải lên 32.

Có một số giải pháp có sẵn để lấy mẫu lên (tăng kích thước của hình ảnh), chẳng hạn như nội suy song tuyến, nhưng điều đó chỉ hoạt động khá tốt lên đến ×4 hoặc ×8. Thay vào đó, họ sử dụng một lớp tích chập chuyển vị (transposed convolutional layer): điều này tương đương với việc đầu tiên kéo giãn hình ảnh bằng cách chèn các hàng và cột trống (đầy số 0), sau đó thực hiện một tích chập thông thường (xem Hình 14-28). Hoặc, một số người thích nghĩ nó như một lớp tích chập thông thường sử dụng các stride phân số (ví dụ: stride là 1/2 trong Hình 14-28).

Lớp tích chập chuyển vị có thể được khởi tạo để thực hiện một cái gì đó gần với nội suy tuyến tính, nhưng vì nó là một lớp có thể huấn luyện được, nó sẽ học cách làm tốt hơn trong quá trình huấn luyện. Trong Keras, bạn có thể sử dụng lớp `Conv2DTranspose`.



Hình 14-28. Lấy mẫu lên bằng cách sử dụng một lớp tích chập chuyển vị.

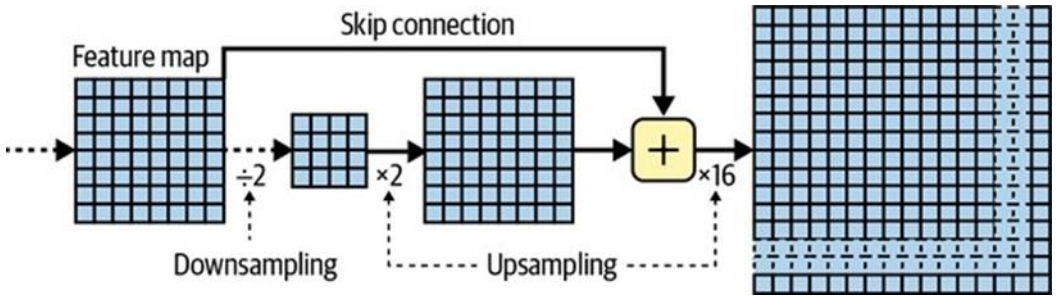
Các lớp tích chập Keras khác

Keras cũng cung cấp một vài loại lớp tích chập khác:

- `tf.keras.layers.Conv1D`: Một lớp tích chập cho đầu vào 1D, chẳng hạn như chuỗi thời gian hoặc văn bản (chuỗi chữ cái hoặc từ), như bạn sẽ thấy trong Chương 15.
- `tf.keras.layers.Conv3D`: Một lớp tích chập cho đầu vào 3D, chẳng hạn như quét PET 3D.
- `dilation_rate`: Đặt siêu tham số `dilation_rate` của bất kỳ lớp tích chập nào thành giá trị từ 2 trở lên sẽ tạo ra một lớp tích chập à-trous (“à trous” trong tiếng Pháp có nghĩa là “có lỗ”). Điều này tương đương với việc sử dụng một lớp tích chập thông thường với một bộ lọc được giãn ra bằng cách chèn các hàng và cột số 0 (tức là các lỗ). Ví dụ, một bộ lọc 1 × 3 bằng [[1,2,3]] có thể được giãn ra với tốc độ giãn nở là 4, dẫn đến một bộ lọc giãn nở [[1, 0, 0, 0, 2, 0, 0, 0, 3]]. Điều này cho phép lớp tích chập có trường tiếp nhận lớn hơn mà không tốn chi phí tính toán và không sử dụng thêm tham số.

Sử dụng các lớp tích chập chuyển vị để lấy mẫu lên là được, nhưng vẫn quá thiếu chính xác. Để làm tốt hơn, Long et al. đã thêm các kết nối bỏ qua từ các lớp thấp hơn: ví dụ, họ lấy mẫu lên hình ảnh đầu ra theo hệ số 2 (thay vì 32), và họ thêm đầu ra của một lớp thấp hơn có độ phân giải gấp đôi này. Sau đó, họ lấy mẫu lên kết quả theo hệ số 16, dẫn đến tổng hệ số lấy mẫu lên là 32 (xem Hình 14-29). Điều này đã khôi phục một số độ phân giải không gian bị mất trong các lớp gộp trước đó. Trong kiến trúc tốt nhất của họ, họ đã sử dụng một kết nối bỏ qua tương tự thứ hai để khôi phục các chi tiết thậm chí còn tốt hơn từ một lớp thậm chí còn thấp hơn. Tóm lại, đầu ra của CNN gốc đi qua các bước bổ sung sau: lấy mẫu lên ×2, thêm đầu ra của một lớp thấp hơn (ở tỷ lệ thích hợp), lấy mẫu lên ×2, thêm đầu ra của một lớp thậm chí còn thấp hơn, và cuối cùng lấy mẫu lên ×8. Thậm chí có thể

mở rộng vượt quá kích thước của hình ảnh gốc: điều này có thể được sử dụng để tăng độ phân giải của một hình ảnh, đây là một kỹ thuật gọi là siêu phân giải (super-resolution).



Hình 14-29. Các lớp bỏ qua khôi phục một số độ phân giải không gian từ các lớp thấp hơn.

Phân đoạn thể hiện (Instance segmentation) tương tự như phân đoạn ngữ nghĩa, nhưng thay vì hợp nhất tất cả các đối tượng cùng lớp thành một khối lớn, mỗi đối tượng được phân biệt với các đối tượng khác (ví dụ: nó xác định từng chiếc xe đạp riêng lẻ). Ví dụ, kiến trúc Mask R-CNN, được đề xuất trong một bài báo năm 2017 bởi Kaiming He et al., mở rộng mô hình Faster R-CNN bằng cách tạo thêm một mặt nạ pixel cho mỗi hộp bao quanh. Vì vậy, bạn không chỉ nhận được một hộp bao quanh xung quanh mỗi đối tượng, với một tập hợp các xác suất lớp ước tính, mà bạn còn nhận được một mặt nạ pixel định vị các pixel trong hộp bao quanh thuộc về đối tượng. Mô hình này có sẵn trên TensorFlow Hub, đã được huấn luyện sẵn trên tập dữ liệu COCO 2017. Lĩnh vực này đang phát triển nhanh chóng, vì vậy nếu bạn muốn thử các mô hình mới nhất và tốt nhất, vui lòng kiểm tra phần state-of-the-art của https://paperswithcode.com .

Như bạn có thể thấy, lĩnh vực thị giác máy tính sâu rộng lớn và phát triển nhanh chóng, với tất cả các loại kiến trúc xuất hiện hàng năm. Hầu hết trong số đó đều dựa trên mạng nơ-ron tích chập, nhưng kể từ năm 2020, một kiến trúc mạng nơ-ron khác đã gia nhập không gian thị giác máy tính: transformers (mà chúng ta sẽ thảo luận trong Chương 16). Sự tiến bộ đạt được trong thập kỷ qua là đáng kinh ngạc, và các nhà nghiên cứu hiện đang tập trung vào các vấn đề ngày càng khó hơn, chẳng hạn như học đối nghịch (cố gắng làm cho mạng có khả năng chống lại các hình ảnh được thiết kế để đánh lừa nó), khả năng giải thích (hiểu tại sao mạng đưa ra một phân loại cụ thể), tạo hình ảnh thực tế (chúng ta sẽ quay lại vấn đề này trong Chương 17), học một lần (một hệ thống có thể nhận dạng một đối tượng sau khi nó chỉ nhìn thấy nó một lần), dự đoán các khung hình tiếp theo trong một video, kết hợp các tác vụ văn bản và hình ảnh, và nhiều hơn nữa.

Bây giờ sang chương tiếp theo, chúng ta sẽ xem xét cách xử lý dữ liệu tuần tự như chuỗi thời gian bằng cách sử dụng mạng nơ-ron hồi quy và mạng nơ-ron tích chập.

Bài tập

1. Ưu điểm của CNN so với DNN kết nối đầy đủ cho phân loại hình ảnh là gì?
2. Xem xét một CNN gồm ba lớp tích chập, mỗi lớp có kernel 3 × 3, stride 2 và padding “same”. Lớp thấp nhất xuất ra 100 bản đồ đặc trưng, lớp giữa xuất ra 200, và lớp trên cùng xuất ra 400. Các hình ảnh đầu vào là hình ảnh RGB có kích thước 200 ×

300 pixel: a. Tổng số tham số trong CNN là bao nhiêu? b. Nếu chúng ta sử dụng số dấu phẩy động 32 bit, mạng này sẽ yêu cầu ít nhất bao nhiêu RAM khi đưa ra dự đoán cho một trường hợp duy nhất? c. Còn khi huấn luyện trên một mini-batch gồm 50 hình ảnh thì sao?

3. Nếu GPU của bạn hết bộ nhớ khi huấn luyện CNN, bạn có thể thử năm điều gì để giải quyết vấn đề?
4. Tại sao bạn muốn thêm một lớp gộp max thay vì một lớp tích chập có cùng stride?
5. Khi nào bạn muốn thêm một lớp chuẩn hóa phản hồi cục bộ?
6. Bạn có thể nêu tên những đổi mới chính trong AlexNet, so với LeNet-5 không? Còn những đổi mới chính trong GoogLeNet, ResNet, SENet, Xception và EfficientNet thì sao?
7. Mạng hoàn toàn tích chập là gì? Làm thế nào bạn có thể chuyển đổi một lớp dày đặc thành một lớp tích chập?
8. Khó khăn kỹ thuật chính của phân đoạn ngữ nghĩa là gì?
9. Xây dựng CNN của riêng bạn từ đầu và cố gắng đạt được độ chính xác cao nhất có thể trên MNIST.
10. Sử dụng học chuyển đổi cho phân loại hình ảnh lớn, trải qua các bước sau: a. Tạo một tập huấn luyện chứa ít nhất 100 hình ảnh trên mỗi lớp. Ví dụ, bạn có thể phân loại hình ảnh của riêng mình dựa trên vị trí (bãi biển, núi, thành phố, v.v.), hoặc thay vào đó bạn có thể sử dụng một tập dữ liệu hiện có (ví dụ: từ TensorFlow Datasets). b. Chia nó thành tập huấn luyện, tập xác thực và tập kiểm tra. c. Xây dựng đường ống đầu vào, áp dụng các thao tác tiền xử lý thích hợp và tùy chọn thêm tăng cường dữ liệu. d. Tinh chỉnh một mô hình đã được huấn luyện sẵn trên tập dữ liệu này.
11. Đọc qua hướng dẫn Chuyển đổi phong cách của TensorFlow. Đây là một cách thú vị để tạo ra nghệ thuật bằng cách sử dụng học sâu. Giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

CHƯƠNG 15. XỬ LÝ CHUỖI BẰNG RNN VÀ CNN

Dự đoán tương lai là điều bạn làm mọi lúc, cho dù bạn đang kết thúc câu nói của một người bạn hay dự đoán mùi cà phê vào bữa sáng. Trong chương này, chúng ta sẽ thảo luận về mạng thần kinh hồi quy (RNNs) - một loại mạng có thể dự đoán tương lai (tất nhiên, ở một mức độ nhất định). RNNs có thể phân tích dữ liệu chuỗi thời gian, chẳng hạn như số lượng người dùng hoạt động hàng ngày trên trang web của bạn, nhiệt độ hàng giờ trong thành phố của bạn, mức tiêu thụ điện hàng ngày của nhà bạn, quỹ đạo của các xe ô gần đó, và nhiều hơn nữa. Khi một RNN học được các mẫu trong dữ liệu quá khứ, nó có thể sử dụng kiến thức của mình để dự báo tương lai, tất nhiên là với giả định rằng các mẫu quá khứ vẫn giữ nguyên trong tương lai.

Tổng quát hơn, RNNs có thể hoạt động trên các chuỗi có độ dài tùy ý, thay vì các đầu vào có kích thước cố định. Ví dụ, chúng có thể nhận câu, tài liệu hoặc mẫu âm thanh làm đầu vào, làm cho chúng cực kỳ hữu ích cho các ứng dụng xử lý ngôn ngữ tự nhiên như dịch tự động hoặc chuyển giọng nói thành văn bản.

Trong chương này, chúng ta sẽ bắt đầu với các khái niệm cơ bản về RNN và cách huấn luyện chúng bằng cách lan truyền ngược qua thời gian (backpropagation through time). Sau đó, chúng ta sẽ sử dụng chúng để dự báo một chuỗi thời gian.

Trong quá trình này, chúng ta sẽ xem xét các mô hình phổ biến thuộc họ ARMA, thường được sử dụng để dự báo chuỗi thời gian và sử dụng chúng làm đường cơ sở để so sánh với RNN của chúng ta. Sau đó, chúng ta sẽ khám phá hai khó khăn chính mà RNN phải đối mặt:

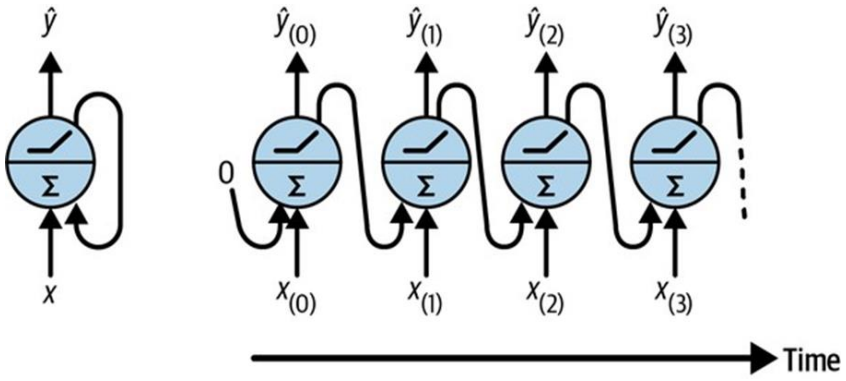
- **Đạo hàm không ổn định** (thảo luận trong Chương 11), có thể được giảm nhẹ bằng các kỹ thuật khác nhau, bao gồm dropout hồi quy và chuẩn hóa lớp hồi quy.
- **Bộ nhớ ngắn hạn (rất) hạn chế**, có thể được mở rộng bằng cách sử dụng các ô LSTM và GRU.

RNN không phải là loại mạng thần kinh duy nhất có khả năng xử lý dữ liệu tuần tự. Đối với các chuỗi nhỏ, một mạng dày đặc thông thường có thể thực hiện được, và đối với các chuỗi rất dài, chẳng hạn như mẫu âm thanh hoặc văn bản, mạng thần kinh tích chập (CNN) cũng có thể hoạt động khá tốt. Chúng ta sẽ thảo luận cả hai khả năng này, và chúng ta sẽ kết thúc chương này bằng cách triển khai WaveNet - một kiến trúc CNN có khả năng xử lý các chuỗi có hàng chục nghìn bước thời gian. Hãy bắt đầu thôi!

15.1 Các nơ-ron và lớp hồi quy

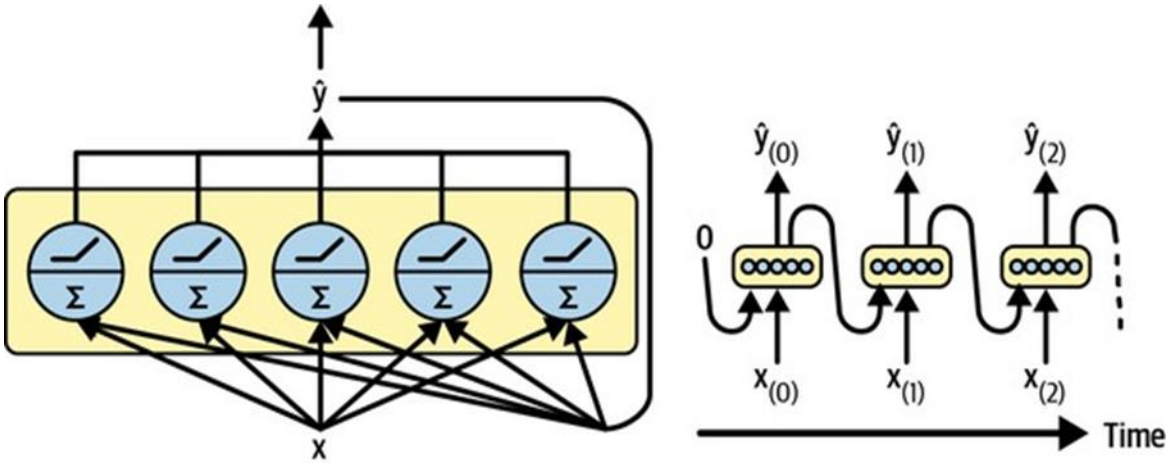
Cho đến nay, chúng ta đã tập trung vào các mạng thần kinh truyền thẳng, nơi các kích hoạt chỉ chảy theo một hướng, từ lớp đầu vào đến lớp đầu ra. Một mạng thần kinh hồi quy trông rất giống một mạng thần kinh truyền thẳng, ngoại trừ việc nó cũng có các kết nối chỉ ngược lại.

Hãy xem xét RNN đơn giản nhất có thể, bao gồm một nơ-ron nhận đầu vào, tạo ra một đầu ra và gửi đầu ra đó trở lại chính nó, như được thể hiện trong Hình 15-1 (trái). Tại mỗi bước thời gian t (còn gọi là một khung), nơ-ron hồi quy này nhận các đầu vào x(t) cũng như đầu ra của chính nó từ bước thời gian trước đó, ŷ(t–1). Vì không có đầu ra trước đó ở bước thời gian đầu tiên, nó thường được đặt thành 0. Chúng ta có thể biểu diễn mạng nhỏ này trên trục thời gian, như được thể hiện trong Hình 15-1 (phải). Đây được gọi là mở rộng mạng qua thời gian (nó là cùng một nơ-ron hồi quy được biểu diễn một lần cho mỗi bước thời gian).



Hình 15-1. Một nơ-ron hồi quy (trái) được mở rộng qua thời gian (phải)

Bạn có thể dễ dàng tạo một lớp các nơ-ron hồi quy. Tại mỗi bước thời gian t, mỗi nơ-ron nhận cả vector đầu vào x(t) và vector đầu ra từ bước thời gian trước đó ŷ(t–1), như được thể hiện trong Hình 15-2. Lưu ý rằng cả đầu vào và đầu ra hiện là các vector (khi chỉ có một nơ-ron duy nhất, đầu ra là một đại lượng vô hướng).



Hình 15-2. Một lớp các nơ-ron hồi quy (trái) được mở rộng qua thời gian (phải)

Mỗi nơ-ron hồi quy có hai tập hợp trọng số: một cho các đầu vào x(t) và một cho các đầu ra của bước thời gian trước đó, ŷ(t–1). Hãy gọi các vector trọng số này là w_x và wŷ. Nếu chúng ta xem xét toàn bộ lớp hồi quy thay vì chỉ một nơ-ron hồi quy, chúng ta có thể đặt tất cả các vector trọng số vào hai ma trận trọng số: W_x và Wŷ.

Vector đầu ra của toàn bộ lớp hồi quy sau đó có thể được tính toán như bạn mong đợi, như được thể hiện trong Phương trình 15-1, trong đó b là vector độ lệch và ϕ(·) là hàm kích hoạt (ví dụ: ReLU).

Phương trình 15-1. Đầu ra của một lớp hồi quy cho một thể hiện đơn lẻ

$$y^{(t)} = \varphi\left(W_{xT}x^{(t)} + W_{yT}y^{(t-1)} + b\right)$$

Cũng như với các mạng thần kinh truyền thẳng, chúng ta có thể tính toán đầu ra của một lớp hồi quy trong một lần cho toàn bộ một mini-batch bằng cách đặt tất cả các đầu vào tại bước thời gian t vào một ma trận đầu vào X(t) (xem Phương trình 15-2).

Phương trình 15-2. Đầu ra của một lớp nơ-ron hồi quy cho tất cả các thể hiện trong một lượt:

$$\begin{aligned}\hat{\mathbf{Y}}_{(t)} &= \phi\left(\mathbf{X}_{(t)}\mathbf{W}_x + \hat{\mathbf{Y}}_{(t-1)}\mathbf{W}_{\hat{y}} + \mathbf{b}\right) \\ &= \phi\left(\begin{bmatrix}\mathbf{X}_{(t)} & \hat{\mathbf{Y}}_{(t-1)}\end{bmatrix}\mathbf{W} + \mathbf{b}\right) \text{ with } \mathbf{W} = \begin{bmatrix}\mathbf{W}_x \\ \mathbf{W}_{\hat{y}}\end{bmatrix}\end{aligned}$$

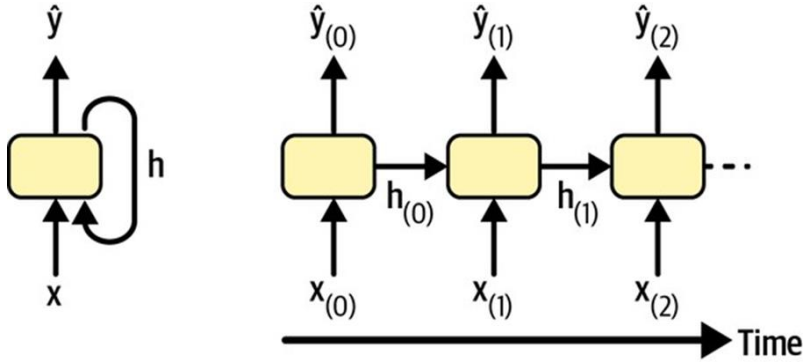
Trong phương trình này:

- $\hat{Y}_{(t)}$ là ma trận $m \times n_{neurons}$ chứa đầu ra của lớp tại thời điểm t cho mỗi trường hợp trong mini-batch (m là số trường hợp trong mini-batch và $n_{neurons}$ là số nơ-ron).
- $X_{(t)}$ là ma trận $m \times n_{inputs}$ chứa các đầu vào cho tất cả các trường hợp (n_{inputs} là số đặc trưng đầu vào).
- W_x là ma trận $n_{inputs} \times n_{neurons}$ chứa các trọng số kết nối cho các đầu vào của thời điểm hiện tại.
- $W_{\hat{y}}$ là ma trận $n_{neurons} \times n_{neurons}$ chứa các trọng số kết nối cho các đầu ra của thời điểm trước đó.
- b là một vector có kích thước $n_{neurons}$ chứa số hạng độ lệch của mỗi nơ-ron.
- Các ma trận trọng số W_x và W_y thường được nối theo chiều dọc thành một ma trận trọng số duy nhất W có kích thước $\left(n_{inputs} + n_{neurons}\right) \times n_{neurons}$.
- Ký hiệu $\left[X_{(t)}Y^{(t-1)}\right]$ đại diện cho việc nối theo chiều ngang của các ma trận $X_{(t)}$ và $Y^{(t-1)}$.
- Lưu ý rằng $Y^{(t)}$ là một hàm của $X_{(t)}$ và $Y^{(t-1)}$, mà $Y^{(t-1)}$ lại là một hàm của $X_{(t-1)}$ và $Y^{(t-2)}$, và cứ tiếp tục như vậy. Điều này làm cho $Y^{(t)}$ trở thành một hàm của tất cả các đầu vào kể từ thời điểm $t = 0$. Tại thời điểm đầu tiên, $t = 0$, không có đầu ra trước đó, vì vậy chúng thường được giả định là tất cả đều bằng không.

Các ô bộ nhớ

Vì đầu ra của một nơ-ron hồi quy tại bước thời gian t là một hàm của tất cả các đầu vào từ các bước thời gian trước đó, bạn có thể nói nó có một dạng bộ nhớ. Một phần của mạng

thần kinh giữ lại một số trạng thái qua các bước thời gian được gọi là một ô bộ nhớ (hoặc đơn giản là một ô). Một nơ-ron hồi quy đơn lẻ, hoặc một lớp các nơ-ron hồi quy, là một ô rất cơ bản, chỉ có khả năng học các mẫu ngắn (thường dài khoảng 10 bước, nhưng điều này thay đổi tùy thuộc vào tác vụ). Sau này trong chương này, chúng ta sẽ xem xét một số loại ô phức tạp và mạnh mẽ hơn có khả năng học các mẫu dài hơn (khoảng 10 lần dài hơn, nhưng một lần nữa, điều này phụ thuộc vào tác vụ). Trạng thái của một ô tại bước thời gian t, được ký hiệu là $h^{(t)}$ ("h" là viết tắt của "ẩn"), là một hàm của một số đầu vào tại bước thời gian đó và trạng thái của nó tại bước thời gian trước đó: $h^{(t)} = f\left(x^{(t)}, h^{(t-1)}\right)$. Đầu ra của nó tại bước thời gian t, được ký hiệu là $\hat{y}_{(t)}$, cũng là một hàm của trạng thái trước đó và các đầu vào hiện tại. Trong trường hợp các ô cơ bản chúng ta đã thảo luận cho đến nay, đầu ra chỉ bằng trạng thái, nhưng trong các ô phức tạp hơn, điều này không phải lúc nào cũng đúng, như được thể hiện trong Hình 15-3.



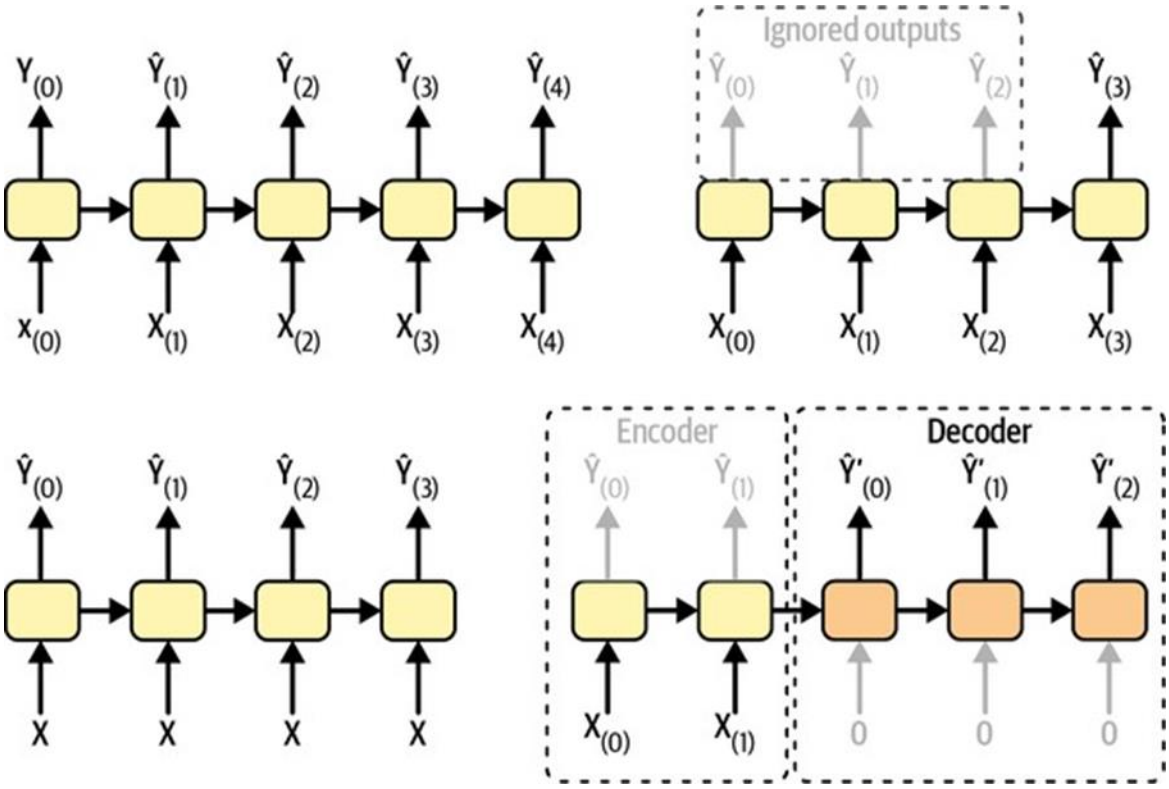
Hình 15-3. Trạng thái ẩn và đầu ra của một ô có thể khác nhau

Các chuỗi đầu vào và đầu ra

Một RNN có thể đồng thời nhận một chuỗi đầu vào và tạo ra một chuỗi đầu ra (xem mạng phía trên bên trái trong Hình 15-4). Loại mạng chuỗi-sang-chuỗi này hữu ích để dự báo chuỗi thời gian, chẳng hạn như mức tiêu thụ điện hàng ngày của nhà bạn: bạn cung cấp dữ liệu trong N ngày qua, và bạn huấn luyện nó để xuất ra mức tiêu thụ điện được dịch chuyển một ngày vào tương lai (tức là, từ N-1 ngày trước đến ngày mai). Ngoài ra, bạn có thể cung cấp cho mạng một chuỗi đầu vào và bỏ qua tất cả các đầu ra trừ đầu ra cuối cùng (xem mạng phía trên bên phải trong Hình 15-4). Đây là một mạng chuỗi-sang-vector. Ví dụ, bạn có thể cung cấp cho mạng một chuỗi các từ tương ứng với một bài đánh giá phim, và mạng sẽ xuất ra một điểm tình cảm (ví dụ: từ 0 [ghét] đến 1 [yêu]). Ngược lại, bạn có thể cung cấp cho mạng cùng một vector đầu vào lặp đi lặp lại tại mỗi bước thời gian và để nó xuất ra một chuỗi (xem mạng phía dưới bên trái của Hình 15-4). Đây là một mạng vector-sang-chuỗi. Ví dụ, đầu vào có thể là một hình ảnh (hoặc đầu ra của một CNN), và đầu ra có thể là một chú thích cho hình ảnh đó.

Cuối cùng, bạn có thể có một mạng chuỗi-sang-vector, được gọi là bộ mã hóa (encoder), tiếp theo là một mạng vector-sang-chuỗi, được gọi là bộ giải mã (decoder) (xem mạng phía dưới bên phải của Hình 15-4). Ví dụ, điều này có thể được sử dụng để dịch một câu từ ngôn ngữ này sang ngôn ngữ khác. Bạn sẽ cung cấp cho mạng một câu bằng một ngôn ngữ, bộ

mã hóa sẽ chuyển đổi câu này thành một biểu diễn vector duy nhất, và sau đó bộ giải mã sẽ giải mã vector này thành một câu bằng một ngôn ngữ khác. Mô hình hai bước này, được gọi là bộ mã hóa-bộ giải mã, hoạt động tốt hơn nhiều so với việc cố gắng dịch ngay lập tức bằng một RNN chuỗi-sang-chuỗi đơn lẻ (như cái được biểu diễn ở phía trên bên trái): những từ cuối cùng của một câu có thể ảnh hưởng đến những từ đầu tiên của bản dịch, vì vậy bạn cần đợi cho đến khi bạn đã thấy toàn bộ câu trước khi dịch nó. Chúng ta sẽ tìm hiểu việc triển khai một bộ mã hóa-bộ giải mã trong Chương 16 (như bạn sẽ thấy, nó phức tạp hơn một chút so với những gì Hình 15-4 gợi ý).

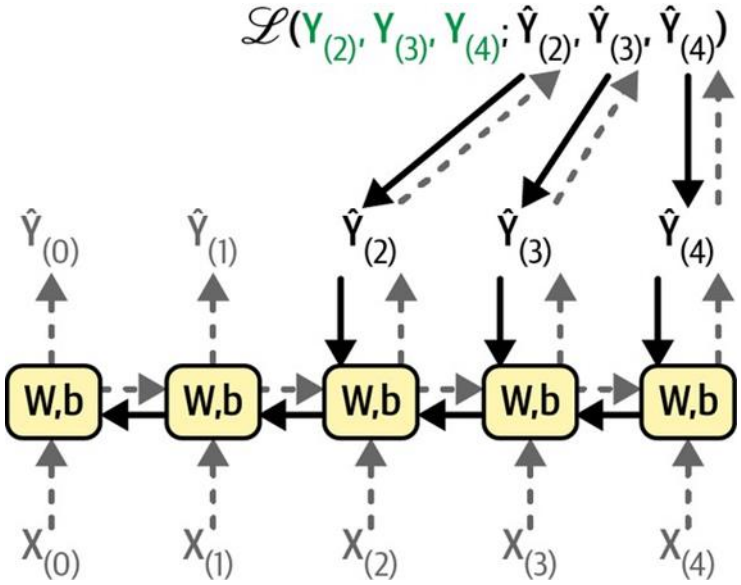


Hình 15-4. Các mạng chuỗi-sang-chuỗi (trên trái), chuỗi-sang-vector (trên phải), vector-sang-chuỗi (dưới trái), và bộ mã hóa-bộ giải mã (dưới phải) Tính linh hoạt này nghe có vẻ hứa hẹn, nhưng làm thế nào để bạn huấn luyện một mạng thần kinh hồi quy?

15.2 Huấn luyện RNN

Để huấn luyện một RNN, cách làm là mở rộng nó theo thời gian (như chúng ta vừa làm) và sau đó sử dụng phương pháp lan truyền ngược thông thường (xem Hình 15-5). Chiến lược này được gọi là **lan truyền ngược qua thời gian (BPTT)**. Giống như trong lan truyền ngược thông thường, có một lượt truyền thẳng đầu tiên qua mạng được mở rộng (được biểu diễn bằng các mũi tên nét đứt). Sau đó, chuỗi đầu ra được đánh giá bằng một hàm mất mát $mathcalL\left(Y^{(0)}, Y^{(1)}, .., Y^{(T)}; \hat{y}^{(0)}, \hat{Y}^{(1)}, .., \hat{Y}^{(T)}\right)$ (trong đó $Y^{(i)}$ là mục tiêu thứ i, $\hat{Y}^{(i)}$ là dự đoán thứ i, và T là bước thời gian tối đa). Lưu ý rằng hàm mất mát này có thể bỏ qua một số đầu ra. Ví dụ, trong một RNN chuỗi-sang-vector, tất cả các đầu ra đều bị bỏ qua trừ đầu ra cuối cùng. Trong Hình 15-5, hàm mất mát được tính toán chỉ dựa trên ba đầu ra cuối cùng.

Các đạo hàm của hàm mất mát đó sau đó được lan truyền ngược qua mạng được mở rộng (được biểu diễn bằng các mũi tên liền nét). Trong ví dụ này, vì các đầu ra $hatY^{(0)}$ và $hatY^{(1)}$ không được sử dụng để tính toán mất mát, các đạo hàm không chảy ngược qua chúng; chúng chỉ chảy qua $\hat{Y}^{(2)}$, $\hat{Y}^{(3)}$ và $\hat{Y}^{(4)}$. Hơn nữa, vì cùng các tham số W và b được sử dụng ở mỗi bước thời gian, các đạo hàm của chúng sẽ được điều chỉnh nhiều lần trong quá trình lan truyền ngược. Sau khi giai đoạn lan truyền ngược hoàn tất và tất cả các đạo hàm đã được tính toán, BPTT có thể thực hiện một bước giảm độ dốc để cập nhật các tham số (điều này không khác gì lan truyền ngược thông thường).



Hình 15-5. Lan truyền ngược qua thời gian

May mắn thay, Keras sẽ lo tất cả sự phức tạp này cho bạn, như bạn sẽ thấy. Nhưng trước khi chúng ta đến đó, hãy tải một chuỗi thời gian và bắt đầu phân tích nó bằng các công cụ cổ điển để hiểu rõ hơn những gì chúng ta đang xử lý và để có được một số số liệu cơ bản.

15.3 Dự báo chuỗi thời gian

Được rồi! Giả sử bạn vừa được thuê làm nhà khoa học dữ liệu bởi Cơ quan Vận tải Chicago. Nhiệm vụ đầu tiên của bạn là xây dựng một mô hình có khả năng dự báo số lượng hành khách sẽ đi xe buýt và đường sắt vào ngày hôm sau. Bạn có quyền truy cập vào dữ liệu lượng hành khách hàng ngày kể từ năm 2001. Hãy cùng nhau tìm hiểu cách bạn sẽ xử lý việc này. Chúng ta sẽ bắt đầu bằng cách tải và làm sạch dữ liệu:

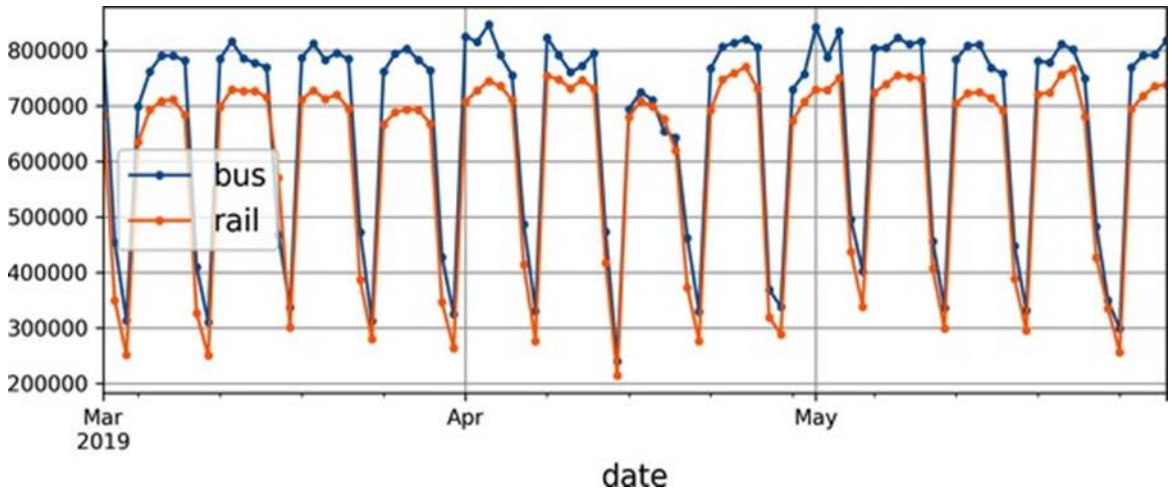
```
path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
df = pd.read_csv(path, parse_dates=["service_date"])
df.columns = ["date", "day_type", "bus", "rail", "total"] # shorter names
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1) # no need for total, it's just bus + rail
df = df.drop_duplicates() # remove duplicated months (2011-10 and 2014-07)
```

Chúng ta tải tệp CSV, đặt tên cột ngắn gọn, sắp xếp các hàng theo ngày, loại bỏ cột “total” thừa và loại bỏ các hàng trùng lặp. Bây giờ hãy kiểm tra vài hàng đầu tiên trông như thế nào:

```
>>> df.head()
            day_type     bus    rail
date
2001-01-01        U  297192  126455
2001-01-02        W  780827  501952
2001-01-03        W  824923  536432
2001-01-04        W  870021  550011
2001-01-05        W  890426  557917
```

Vào ngày 1 tháng 1 năm 2001, 297.192 người đã đi xe buýt ở Chicago và 126.455 người đã đi tàu. Cột `day_type` chứa W cho Ngày trong tuần, A cho Thứ Bảy và U cho Chủ Nhật hoặc ngày lễ. Bây giờ chúng ta hãy vẽ biểu đồ số lượng hành khách đi xe buýt và đường sắt trong vài tháng vào năm 2019, để xem nó trông như thế nào (xem Hình 15-6):

```
df["2019-03":"2019-05"].plot(grid=True, marker=".", figsize=(8, 3.5))
plt.show()
```



Hình 15-6. Lượng hành khách hàng ngày ở Chicago

Lưu ý rằng Pandas bao gồm cả tháng bắt đầu và tháng kết thúc trong phạm vi, vì vậy biểu đồ này vẽ dữ liệu từ ngày 1 tháng 3 cho đến ngày 31 tháng 5. Đây là một **chuỗi thời gian**: dữ liệu có các giá trị tại các bước thời gian khác nhau, thường là ở các khoảng thời gian đều đặn. Cụ thể hơn, vì có nhiều giá trị trên mỗi bước thời gian, đây được gọi là **chuỗi thời gian đa biến**. Nếu chúng ta chỉ nhìn vào cột “bus”, nó sẽ là một chuỗi thời gian đơn biến, với một giá trị duy nhất trên mỗi bước thời gian. Dự đoán các giá trị tương lai (tức là **dự báo**) là nhiệm vụ điển hình nhất khi xử lý chuỗi thời gian, và đây là điều chúng ta sẽ tập trung vào trong chương này. Các nhiệm vụ khác bao gồm **điền dữ liệu** (điền các giá trị quá khứ bị thiếu), **phân loại**, **phát hiện bất thường**, v.v. Nhìn vào Hình 15-6, chúng ta có thể thấy rằng một mô hình tương tự được lặp lại rõ ràng mỗi tuần. Đây được gọi là **tính thời vụ hàng tuần**. Trên thực tế, trong trường hợp này nó rất mạnh đến nỗi việc dự báo lượng hành khách ngày mai bằng cách chỉ sao chép các giá trị từ một tuần trước sẽ cho kết quả khá tốt. Đây

được gọi là **dự báo ngây thơ**: chỉ đơn giản là sao chép một giá trị trong quá khứ để thực hiện dự báo của chúng ta. Dự báo ngây thơ thường là một đường cơ sở tuyệt vời, và đôi khi thậm chí khó bị đánh bại.

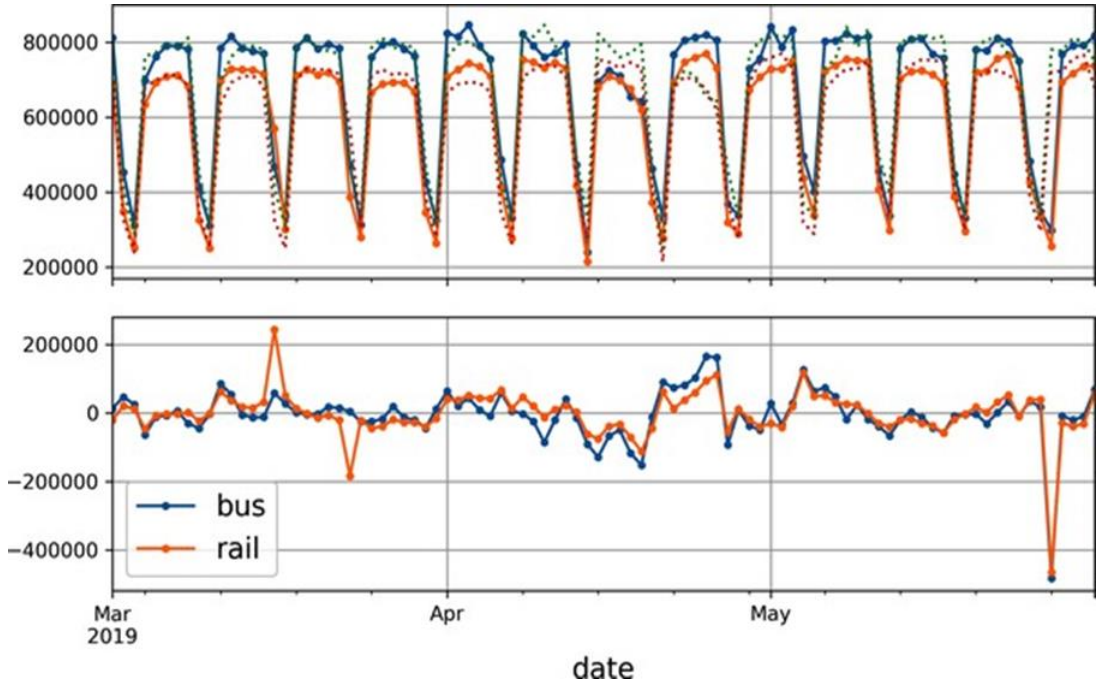
Để hình dung các dự báo ngây thơ này, chúng ta hãy phủ lên hai chuỗi thời gian (đối với xe buýt và đường sắt) cũng như cùng một chuỗi thời gian bị trễ một tuần (tức là dịch chuyển sang phải) bằng các đường chấm chấm. Chúng ta cũng sẽ vẽ biểu đồ sự khác biệt giữa hai chuỗi (tức là giá trị tại thời điểm t trừ đi giá trị tại thời điểm t – 7); đây được gọi là **sai phân** (xem Hình 15-7):

```
diff_7 = df[["bus", "rail"]].diff(7)["2019-03":"2019-05"]

fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8, 5))
df.plot(ax=axs[0], legend=False, marker=".") # original time series
df.shift(7).plot(ax=axs[0], grid=True, legend=False, linestyle=":") # lagged
diff_7.plot(ax=axs[1], grid=True, marker=".") # 7-day difference time series
plt.show()
```

Không tệ lắm! Lưu ý rằng chuỗi thời gian bị trễ theo dõi chặt chẽ chuỗi thời gian thực tế như thế nào. Khi một chuỗi thời gian tương quan với một phiên bản trễ của chính nó, chúng ta nói rằng chuỗi thời gian đó có **tự tương quan**. Như bạn có thể thấy, hầu hết các khác biệt đều khá nhỏ, ngoại trừ vào cuối tháng Năm. Có lẽ có một kỳ nghỉ vào thời điểm đó? Hãy kiểm tra cột day_type:

```
>>> list(df.loc["2019-05-25":"2019-05-27"]["day_type"])
['A', 'U', 'U']
```



Hình 15-7. Chuỗi thời gian được phủ với chuỗi thời gian bị trễ 7 ngày (trên), và sự khác biệt giữa t và t – 7 (dưới)

Thật vậy, lúc đó có một kỳ nghỉ cuối tuần dài: Thứ Hai là ngày lễ Tưởng niệm. Chúng ta có thể sử dụng cột này để cải thiện dự báo của mình, nhưng bây giờ chúng ta hãy chỉ đo lỗi tuyệt đối trung bình trong khoảng thời gian ba tháng mà chúng ta đang tập trung vào—tháng 3, tháng 4 và tháng 5 năm 2019—để có một ý tưởng sơ bộ:

```
>>> diff_7.abs().mean()
bus      43915.608696
rail     42143.271739
dtype: float64
```

Các dự báo ngây thơ của chúng ta có MAE khoảng 43.916 hành khách xe buýt và khoảng 42.143 hành khách đường sắt. Rất khó để nói ngay lập tức điều này tốt hay xấu, vì vậy hãy đặt các lỗi dự báo vào góc nhìn bằng cách chia chúng cho các giá trị mục tiêu:

```
targets = df[["bus", "rail"]]["2019-03":"2019-05"]
(diff_7 / targets).abs().mean()

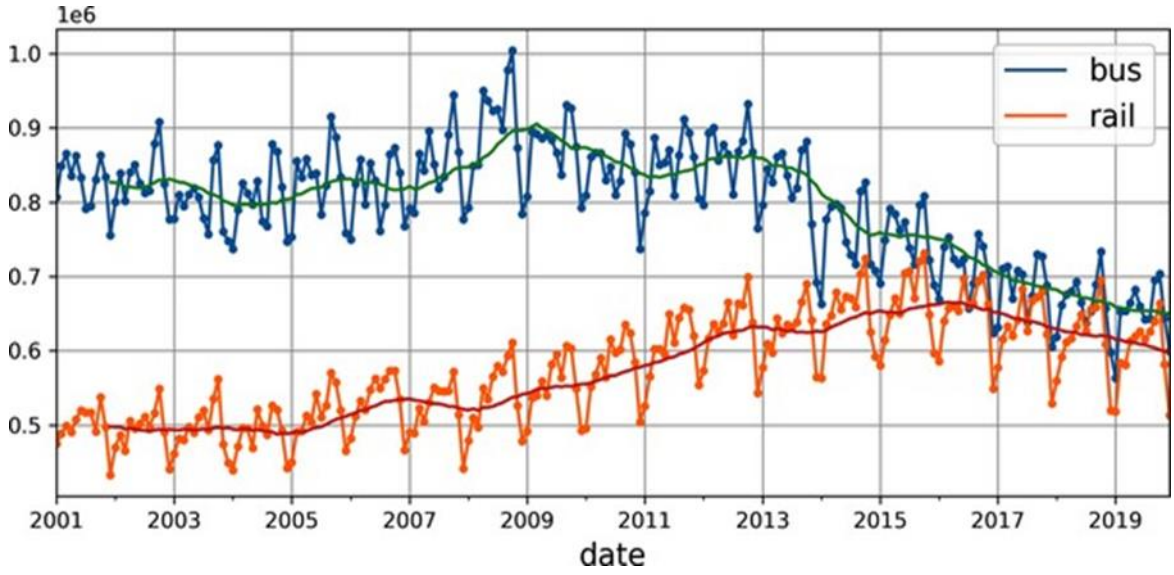
bus     0.082938
rail    0.089948
dtype: float64
```

Những gì chúng ta vừa tính toán được gọi là **lỗi phần trăm tuyệt đối trung bình (MAPE)**: có vẻ như các dự báo ngây thơ của chúng ta cho chúng ta MAPE khoảng 8,3% đối với xe buýt và 9,0% đối với đường sắt. Điều thú vị là MAE cho các dự báo đường sắt trông hơi tốt hơn MAE cho các dự báo xe buýt, trong khi điều ngược lại đúng với MAPE. Điều đó là do lượng hành khách xe buýt lớn hơn lượng hành khách đường sắt, vì vậy tự nhiên lỗi dự báo cũng lớn hơn, nhưng khi chúng ta đặt các lỗi vào góc nhìn, hóa ra các dự báo xe buýt thực sự hơi tốt hơn các dự báo đường sắt.

Nhìn vào chuỗi thời gian, dường như không có bất kỳ tính thời vụ hàng tháng đáng kể nào, nhưng hãy kiểm tra xem có bất kỳ tính thời vụ hàng năm nào không. Chúng ta sẽ xem xét dữ liệu từ năm 2001 đến năm 2019. Để giảm nguy cơ dò dữ liệu, chúng ta sẽ bỏ qua dữ liệu gần đây hơn. Chúng ta cũng sẽ vẽ một đường trung bình động 12 tháng cho mỗi chuỗi để hình dung các xu hướng dài hạn (xem Hình 15-8):

```
period = slice("2001", "2019")
df_monthly = df.resample('M').mean() # compute the mean for each month
rolling_average_12_months = df_monthly[period].rolling(window=12).mean()

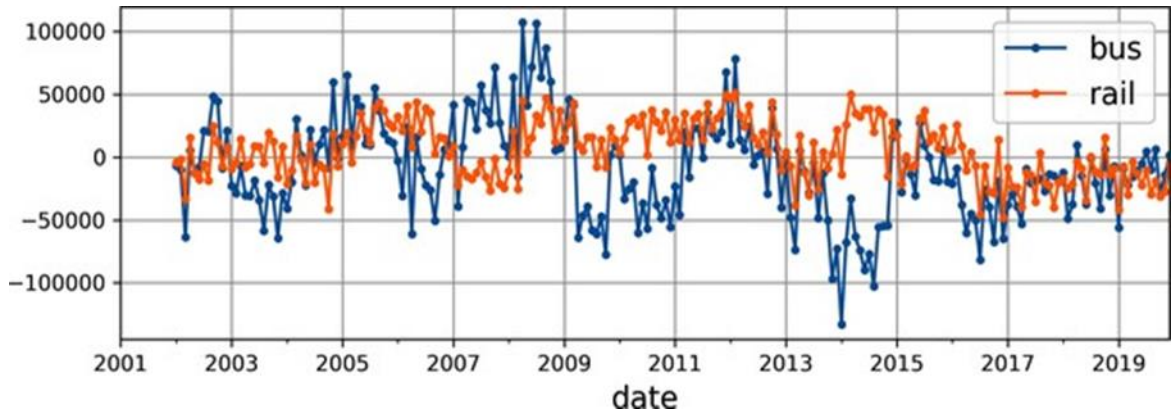
fig, ax = plt.subplots(figsize=(8, 4))
df_monthly[period].plot(ax=ax, marker=".")
rolling_average_12_months.plot(ax=ax, grid=True, legend=False)
plt.show()
```



Hình 15-8. Tính thời vụ hàng năm và các xu hướng dài hạn

Đúng vậy! Chắc chắn có một số **tính thời vụ hàng năm** nữa, mặc dù nó nhiễu hơn tính thời vụ hàng tuần và rõ ràng hơn đối với chuỗi đường sắt so với chuỗi xe buýt: chúng ta thấy các đỉnh và đáy vào cùng những ngày gần như nhau mỗi năm. Hãy kiểm tra xem chúng ta nhận được gì nếu chúng ta vẽ biểu đồ sự khác biệt 12 tháng (xem Hình 15-9):

```
df_monthly.diff(12)[period].plot(grid=True, marker=".", figsize=(8, 3))
plt.show()
```



Hình 15-9. Sự khác biệt 12 tháng

Lưu ý rằng việc **sai phân** không chỉ loại bỏ tính thời vụ hàng năm, mà còn loại bỏ các xu hướng dài hạn. Ví dụ, xu hướng giảm tuyến tính hiện diện trong chuỗi thời gian từ năm 2016 đến 2019 đã trở thành một giá trị âm gần như không đổi trong chuỗi thời gian đã được sai phân. Trên thực tế, sai phân là một kỹ thuật phổ biến được sử dụng để loại bỏ xu hướng và tính thời vụ khỏi một chuỗi thời gian: việc nghiên cứu một **chuỗi thời gian tĩnh** dễ dàng hơn, nghĩa là một chuỗi có các thuộc tính thống kê không đổi theo thời gian, không có bất kỳ tính thời vụ hoặc xu hướng nào. Khi bạn có thể đưa ra các dự báo chính xác trên chuỗi thời gian đã được sai phân, việc biến chúng thành các dự báo cho chuỗi thời gian thực tế

rất dễ dàng bằng cách chỉ cần cộng lại các giá trị trong quá khứ đã bị trừ trước đó. Bạn có thể nghĩ rằng chúng ta chỉ đang cố gắng dự đoán lượng hành khách vào ngày mai, vì vậy các mẫu dài hạn ít quan trọng hơn nhiều so với các mẫu ngắn hạn. Bạn đúng, nhưng dù sao đi nữa, chúng ta vẫn có thể cải thiện hiệu suất một chút bằng cách tính đến các mẫu dài hạn. Ví dụ, lượng hành khách xe buýt hàng ngày đã giảm khoảng 2.500 vào tháng 10 năm 2017, đại diện cho khoảng 570 hành khách ít hơn mỗi tuần, vì vậy nếu chúng ta đang ở cuối tháng 10 năm 2017, việc dự báo lượng hành khách vào ngày mai bằng cách sao chép giá trị từ tuần trước, trừ đi 570 sẽ có ý nghĩa. Việc tính đến xu hướng sẽ làm cho dự báo của bạn chính xác hơn một chút so với mức trung bình. Bây giờ bạn đã quen thuộc với chuỗi thời gian lượng hành khách, cũng như một số khái niệm quan trọng nhất trong phân tích chuỗi thời gian, bao gồm tính thời vụ, xu hướng, sai phân và trung bình động, hãy cùng xem nhanh một họ các mô hình thống kê rất phổ biến thường được sử dụng để phân tích chuỗi thời gian.

Các mô hình họ ARMA

Chúng ta sẽ bắt đầu với mô hình trung bình động tự hồi quy (ARMA), được Herman Wold phát triển vào những năm 1930: nó tính toán các dự báo của mình bằng cách sử dụng tổng trọng số đơn giản của các giá trị bị trễ và hiệu chỉnh các dự báo này bằng cách thêm một trung bình động, rất giống như chúng ta vừa thảo luận. Cụ thể, thành phần trung bình động được tính toán bằng cách sử dụng tổng trọng số của một vài lỗi dự báo gần nhất. Phương trình 15-3 cho thấy cách mô hình đưa ra các dự báo của nó.

Phương trình 15-3. Dự báo bằng mô hình ARMA

$$y^{(t)} = \sum_{i=1}^{p} \alpha_i \, y^{(t-i)} + \sum_{i=1}^{q} \theta_i \, \epsilon^{(t-i)}$$

với $epsilon^{(t)} = y^{(t)} - haty^{(t)}$ Trong phương trình này:

- $haty^{(t)}$ là dự báo của mô hình cho bước thời gian t .
- $y^{(t)}$ là giá trị của chuỗi thời gian tại bước thời gian t .
- Tổng đầu tiên là tổng trọng số của p giá trị quá khứ của chuỗi thời gian, sử dụng các trọng số đã học $alpha_i$. Số p là một siêu tham số, và nó xác định mô hình nên nhìn lại quá khứ bao xa. Tổng này là thành phần tự hồi quy của mô hình: nó thực hiện hồi quy dựa trên các giá trị trong quá khứ.
- Tổng thứ hai là tổng trọng số trên q lỗi dự báo quá khứ $epsilon^{(t)}$, sử dụng các trọng số đã học $theta_i$. Số q là một siêu tham số. Tổng này là thành phần trung bình động của mô hình.

Quan trọng là, mô hình này giả định rằng chuỗi thời gian là **tĩnh (stationary)**. Nếu không, thì việc sai phân có thể giúp ích. Sử dụng sai phân trên một bước thời gian duy nhất sẽ tạo ra một xấp xỉ của đạo hàm chuỗi thời gian: thực sự, nó sẽ cho độ dốc của chuỗi tại mỗi bước thời gian. Điều này có nghĩa là nó sẽ loại bỏ bất kỳ xu hướng tuyến tính nào, biến nó thành một giá trị không đổi. Ví dụ, nếu bạn áp dụng sai phân một bước cho chuỗi [3, 5, 7, 9, 11],

bạn sẽ nhận được chuỗi sai phân [2, 2, 2, 2]. Nếu chuỗi thời gian gốc có xu hướng bậc hai thay vì xu hướng tuyến tính, thì một lần sai phân sẽ không đủ. Ví dụ, chuỗi [1, 4, 9, 16, 25, 36] trở thành [3, 5, 7, 9, 11] sau một lần sai phân, nhưng nếu bạn chạy sai phân lần thứ hai, thì bạn sẽ nhận được [2, 2, 2, 2]. Vì vậy, chạy hai lần sai phân liên tiếp sẽ loại bỏ các xu hướng bậc hai. Tổng quát hơn, chạy d lần sai phân liên tiếp tính toán một xấp xỉ của đạo hàm bậc d của chuỗi thời gian, vì vậy nó sẽ loại bỏ các xu hướng đa thức bậc lên đến d . Siêu tham số d này được gọi là **bậc tích hợp**.

Sai phân là đóng góp trung tâm của mô hình **trung bình động tích hợp tự hồi quy (ARIMA)**, được giới thiệu vào năm 1970 bởi George Box và Gwilym Jenkins trong cuốn sách *Time Series Analysis* của họ (Wiley): mô hình này chạy d lần sai phân để làm cho chuỗi thời gian trở nên tĩnh hơn, sau đó nó áp dụng một mô hình ARMA thông thường. Khi đưa ra dự báo, nó sử dụng mô hình ARMA này, sau đó nó cộng lại các thuật ngữ đã bị trừ đi bởi sai phân. Một thành viên cuối cùng của họ ARMA là mô hình **ARIMA mùa vụ (SARIMA)**: nó mô hình chuỗi thời gian theo cùng một cách như ARIMA, nhưng nó bổ sung thêm mô hình một thành phần mùa vụ cho một tần số nhất định (ví dụ: hàng tuần), sử dụng cùng một phương pháp ARIMA chính xác. Nó có tổng cộng bảy siêu tham số: các siêu tham số p, d , và q giống như ARIMA, cộng với các siêu tham số P, D , và Q bổ sung để mô hình mẫu mùa vụ, và cuối cùng là chu kỳ của mẫu mùa vụ, được ký hiệu là s . Các siêu tham số P, D , và Q giống như p, d , và q , nhưng chúng được sử dụng để mô hình chuỗi thời gian tại $t-s, t-2s, t-3s$, v.v.

Hãy xem cách điều chỉnh mô hình SARIMA cho chuỗi thời gian đường sắt và sử dụng nó để đưa ra dự báo về lượng hành khách vào ngày mai. Chúng ta sẽ giả vờ hôm nay là ngày cuối cùng của tháng 5 năm 2019, và chúng ta muốn dự báo lượng hành khách đường sắt cho "ngày mai", ngày 1 tháng 6 năm 2019. Để làm điều này, chúng ta có thể sử dụng thư viện `statsmodels`, chứa nhiều mô hình thống kê khác nhau, bao gồm mô hình ARMA và các biến thể của nó, được triển khai bởi lớp `ARIMA`:

```
from statsmodels.tsa.arima.model import ARIMA

origin, today = "2019-01-01", "2019-05-31"
rail_series = df.loc[origin:today]["rail"].asfreq("D")
model = ARIMA(rail_series,
              order=(1, 0, 0),
              seasonal_order=(0, 1, 1, 7))
model = model.fit()
y_pred = model.forecast() # returns 427,758.6
```

Trong ví dụ mã này:

- Chúng ta bắt đầu bằng cách nhập lớp `ARIMA`, sau đó chúng ta lấy dữ liệu lượng hành khách đường sắt từ đầu năm 2019 cho đến "ngày hôm nay", và chúng ta sử dụng `asfreq("D")` để đặt tần suất của chuỗi thời gian thành hàng ngày: điều này không thay đổi dữ liệu trong trường hợp này, vì nó đã là hàng ngày, nhưng nếu không có điều này, lớp `ARIMA` sẽ phải đoán tần suất và nó sẽ hiển thị cảnh báo.

- Tiếp theo, chúng ta tạo một thể hiện `ARIMA`, truyền cho nó tất cả dữ liệu cho đến "ngày hôm nay", và chúng ta đặt các siêu tham số mô hình: `order=(1, 0, 0)` có nghĩa là $p = 1, d = 0, q = 0$, và `seasonal_order=(0, 1, 1, 7)` có nghĩa là $P = 0, D = 1, Q = 1$, và $s = 7$. Lưu ý rằng API của `statsmodels` hơi khác so với API của Scikit-Learn, vì chúng ta truyền dữ liệu cho mô hình tại thời điểm khởi tạo, thay vì truyền nó cho phương thức `fit()`.
- Tiếp theo, chúng ta điều chỉnh mô hình và sử dụng nó để đưa ra dự báo cho "ngày mai", ngày 1 tháng 6 năm 2019.

Dự báo là 427.759 hành khách, trong khi thực tế có 379.044. Chà, chúng ta đã sai lệch 12,9%—khá tệ. Nó thực sự tệ hơn một chút so với dự báo ngây thơ, dự báo 426.932, sai lệch 12,6%. Nhưng có lẽ chúng ta chỉ không may mắn vào ngày hôm đó? Để kiểm tra điều này, chúng ta có thể chạy cùng một mã trong một vòng lặp để đưa ra dự báo cho mỗi ngày trong tháng 3, tháng 4 và tháng 5, và tính MAE trong khoảng thời gian đó:

```
origin, start_date, end_date = "2019-01-01", "2019-03-01", "2019-05-31"
time_period = pd.date_range(start_date, end_date)
rail_series = df.loc[origin:end_date]["rail"].asfreq("D")
y_preds= []

for today in time_period.shift(-1):
    model = ARIMA(rail_series[origin:today], # train on data up to "today"
                  order=(1, 0, 0),
                  seasonal_order=(0, 1, 1, 7))
    model = model.fit() # note that we retrain the model every day!
    y_pred = model.forecast()[0]
    y_preds.append(y_pred)

y_preds = pd.Series(y_preds, index=time_period)
mae = (y_preds - rail_series[time_period]).abs().mean() # returns 32,040.7
```

À, tốt hơn nhiều! MAE khoảng 32.041, thấp hơn đáng kể so với MAE chúng ta nhận được với dự báo ngây thơ (42.143). Vì vậy, mặc dù mô hình không hoàn hảo, nhưng nó vẫn đánh bại dự báo ngây thơ với một biên độ lớn, trung bình. Tại thời điểm này, bạn có thể tự hỏi làm thế nào để chọn các siêu tham số tốt cho mô hình SARIMA. Có một số phương pháp, nhưng phương pháp đơn giản nhất để hiểu và bắt đầu là phương pháp vét cạn: chỉ cần chạy tìm kiếm lưới (grid search). Đối với mỗi mô hình bạn muốn đánh giá (tức là mỗi sự kết hợp siêu tham số), bạn có thể chạy ví dụ mã trước đó, chỉ thay đổi các giá trị siêu tham số. Các giá trị p, q, P , và Q tốt thường khá nhỏ (thường từ 0 đến 2, đôi khi lên đến 5 hoặc 6), và d và D thường là 0 hoặc 1, đôi khi là 2. Đối với s , nó chỉ là chu kỳ của mẫu mùa vụ chính: trong trường hợp của chúng ta là 7 vì có tính thời vụ hàng tuần mạnh mẽ. Mô hình có MAE thấp nhất sẽ thắng. Tất nhiên, bạn có thể thay thế MAE bằng một chỉ số khác nếu nó phù hợp hơn với mục tiêu kinh doanh của bạn. Và thế là xong!

Chuẩn bị dữ liệu cho các mô hình học máy

Bây giờ chúng ta có hai đường cơ sở, dự báo ngây thơ và SARIMA, hãy thử sử dụng các mô hình học máy mà chúng ta đã tìm hiểu cho đến nay để dự báo chuỗi thời gian này, bắt đầu với một mô hình tuyến tính cơ bản. Mục tiêu của chúng ta sẽ là dự báo lượng hành khách vào ngày mai dựa trên dữ liệu lượng hành khách của 8 tuần qua (56 ngày). Do đó, đầu vào của mô hình của chúng ta sẽ là các chuỗi (thường là một chuỗi duy nhất mỗi ngày khi mô hình đi vào sản xuất), mỗi chuỗi chứa 56 giá trị từ bước thời gian $t - 55$ đến t . Đối với mỗi chuỗi đầu vào, mô hình sẽ xuất ra một giá trị duy nhất: dự báo cho bước thời gian $t + 1$. Nhưng chúng ta sẽ sử dụng gì làm dữ liệu huấn luyện? À, đó là mẹo: chúng ta sẽ sử dụng mọi cửa sổ 56 ngày từ quá khứ làm dữ liệu huấn luyện, và mục tiêu cho mỗi cửa sổ sẽ là giá trị ngay sau đó. Keras thực sự có một hàm tiện ích hay tên là `tf.keras.utils.timeseries_dataset_from_array()` để giúp chúng ta chuẩn bị tập huấn luyện. Nó nhận một chuỗi thời gian làm đầu vào, và nó xây dựng một `tf.data.Dataset` (được giới thiệu trong Chương 13) chứa tất cả các cửa sổ có độ dài mong muốn, cũng như các mục tiêu tương ứng của chúng. Đây là một ví dụ lấy một chuỗi thời gian chứa các số từ 0 đến 5 và tạo một tập dữ liệu chứa tất cả các cửa sổ có độ dài 3, với các mục tiêu tương ứng của chúng, được nhóm thành các batch có kích thước 2:

```
import tensorflow as tf

my_series = [0, 1, 2, 3, 4, 5]
my_dataset = tf.keras.utils.timeseries_dataset_from_array(
    my_series,
    targets=my_series[3:], # the targets are 3 steps into the future
    sequence_length=3,
    batch_size=2
)
```

Hãy kiểm tra nội dung của tập dữ liệu này:

```
>>> list(my_dataset)
[(<tf.Tensor: shape=(2, 3), dtype=int32, numpy=
    array([[0, 1, 2],
           [1, 2, 3]], dtype=int32)>,
  <tf.Tensor: shape=(2,), dtype=int32, numpy=array([3, 4], dtype=int32)>),
 (<tf.Tensor: shape=(1, 3), dtype=int32, numpy=array([[2, 3, 4]],
dtype=int32)>,
  <tf.Tensor: shape=(1,), dtype=int32, numpy=array([5], dtype=int32)>)]
```

Mỗi mẫu trong tập dữ liệu là một cửa sổ có độ dài 3, cùng với mục tiêu tương ứng của nó (tức là giá trị ngay sau cửa sổ). Các cửa sổ là [0, 1, 2], [1, 2, 3] và [2, 3, 4], và các mục tiêu tương ứng của chúng là 3, 4 và 5. Vì có tổng cộng ba cửa sổ, không phải là bội số của kích thước batch, nên batch cuối cùng chỉ chứa một cửa sổ thay vì hai. Một cách khác để có được kết quả tương tự là sử dụng phương thức `window()` của lớp `tf.data.Dataset`. Nó phức tạp hơn, nhưng nó cho bạn toàn quyền kiểm soát, điều này sẽ hữu ích sau này trong

chương này, vì vậy hãy xem nó hoạt động như thế nào. Phương thức `window()` trả về một tập dữ liệu của các tập dữ liệu cửa sổ:

```
>>> for window_dataset in tf.data.Dataset.range(6).window(4, shift=1):
...     for element in window_dataset:
...         print(f"{element}", end=" ")
...     print()
...
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5
4 5
5
```

Trong ví dụ này, tập dữ liệu chứa sáu cửa sổ, mỗi cửa sổ được dịch chuyển một bước so với cửa sổ trước đó, và ba cửa sổ cuối cùng nhỏ hơn vì chúng đã đạt đến cuối chuỗi. Nói chung, bạn sẽ muốn loại bỏ các cửa sổ nhỏ hơn này bằng cách truyền `drop_remainder=True` cho phương thức `window()`.

Phương thức `window()` trả về một tập dữ liệu lồng nhau, tương tự như một danh sách các danh sách. Điều này hữu ích khi bạn muốn biến đổi mỗi cửa sổ bằng cách gọi các phương thức tập dữ liệu của nó (ví dụ: để xáo trộn hoặc nhóm chúng). Tuy nhiên, chúng ta không thể sử dụng trực tiếp tập dữ liệu lồng nhau để huấn luyện, vì mô hình của chúng ta sẽ mong đợi các tensor làm đầu vào, không phải tập dữ liệu. Do đó, chúng ta phải gọi phương thức `flat_map()`: nó chuyển đổi một tập dữ liệu lồng nhau thành một tập dữ liệu phẳng (một tập dữ liệu chứa các tensor, không phải tập dữ liệu). Ví dụ, giả sử `{1, 2, 3}` đại diện cho một tập dữ liệu chứa chuỗi các tensor 1, 2 và 3. Nếu bạn làm phẳng tập dữ liệu lồng nhau `{{1, 2}, {3, 4, 5, 6}}`, bạn sẽ nhận được tập dữ liệu phẳng `{1, 2, 3, 4, 5, 6}`. Hơn nữa, phương thức `flat_map()` nhận một hàm làm đối số, cho phép bạn biến đổi mỗi tập dữ liệu trong tập dữ liệu lồng nhau trước khi làm phẳng. Ví dụ, nếu bạn truyền hàm `lambda ds: ds.batch(2)` cho `flat_map()`, thì nó sẽ biến đổi tập dữ liệu lồng nhau `{{1, 2}, {3, 4, 5, 6}}` thành tập dữ liệu phẳng `{[1, 2], [3, 4], [5, 6]}`: đó là một tập dữ liệu chứa 3 tensor, mỗi tensor có kích thước 2. Với những điều đó, chúng ta đã sẵn sàng làm phẳng tập dữ liệu của mình:

```
>>> dataset = tf.data.Dataset.range(6).window(4, shift=1,
drop_remainder=True)
>>> dataset = dataset.flat_map(lambda window_dataset:
window_dataset.batch(4))
>>> for window_tensor in dataset:
...     print(f"{window_tensor}")

[0 1 2 3]
[1 2 3 4]
[2 3 4 5]
```

Vì mỗi tập dữ liệu cửa sổ chứa chính xác bốn mục, việc gọi `batch(4)` trên một cửa sổ tạo ra một tensor duy nhất có kích thước 4. Tuyệt vời! Bây giờ chúng ta có một tập dữ liệu chứa các cửa sổ liên tiếp được biểu diễn dưới dạng tensor. Hãy tạo một hàm trợ giúp nhỏ để dễ dàng trích xuất các cửa sổ từ một tập dữ liệu hơn:

```
def to_windows(dataset, length):
    dataset = dataset.window(length, shift=1, drop_remainder=True)
    return dataset.flat_map(lambda window_ds:
                            window_ds.batch(length))
```

Bước cuối cùng là chia mỗi cửa sổ thành đầu vào và mục tiêu, sử dụng phương thức `map()`. Chúng ta cũng có thể nhóm các cửa sổ kết quả thành các batch có kích thước 2:

```
>>> dataset = to_windows(tf.data.Dataset.range(6), 4) # 3 inputs + 1 target =
4
>>> dataset = dataset.map(lambda window: (window[:-1], window[-1]))
>>> list(dataset.batch(2))
[(<tf.Tensor: shape=(2, 3), dtype=int64, numpy=
    array([[0, 1, 2],
           [1, 2, 3]])>,
  <tf.Tensor: shape=(2,), dtype=int64, numpy=array([3, 4])>),
 (<tf.Tensor: shape=(1, 3), dtype=int64, numpy=array([[2, 3, 4]])>,
  <tf.Tensor: shape=(1,), dtype=int64, numpy=array([5])>)]
```

Như bạn có thể thấy, bây giờ chúng ta có cùng đầu ra như chúng ta đã nhận được trước đó với hàm `timeseries_dataset_from_array()` (với một chút nỗ lực hơn, nhưng sẽ sớm có giá trị). Bây giờ, trước khi chúng ta bắt đầu huấn luyện, chúng ta cần chia dữ liệu của mình thành một giai đoạn huấn luyện, một giai đoạn xác thực và một giai đoạn kiểm tra. Chúng ta sẽ tập trung vào lượng hành khách đường sắt trước. Chúng ta cũng sẽ chia tỷ lệ nó xuống một yếu tố một triệu, để đảm bảo các giá trị gần với phạm vi 0-1; điều này hoạt động tốt với khởi tạo trọng số và tốc độ học mặc định:

```
import pandas as pd
# Giả định df đã được tải và xử lý như trong phần trước
# (Nếu bạn chưa chạy phần đó, bạn cần chạy lại để có df)
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-
_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

rail_train = df["rail"]["2016-01":"2018-12"] / 1e6
rail_valid = df["rail"]["2019-01":"2019-05"] / 1e6
rail_test = df["rail"]["2019-06":] / 1e6
```

Tiếp theo, hãy sử dụng `timeseries_dataset_from_array()` để tạo tập dữ liệu cho huấn luyện và xác thực. Vì gradient descent mong đợi các thể hiện trong tập huấn luyện là độc lập và phân bố giống hệt nhau (IID), như chúng ta đã thấy trong Chương 4, chúng ta phải đặt đối số `shuffle=True` để xáo trộn các cửa sổ huấn luyện (nhưng không phải nội dung của chúng):

```
seq_length = 56
train_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_train.to_numpy(),
    targets=rail_train[seq_length:],
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_valid.to_numpy(),
    targets=rail_valid[seq_length:],
    sequence_length=seq_length,
    batch_size=32
)
```

Và bây giờ chúng ta đã sẵn sàng xây dựng và huấn luyện bất kỳ mô hình hồi quy nào chúng ta muốn!

Dự báo bằng mô hình tuyến tính

Hãy thử một mô hình tuyến tính cơ bản trước. Chúng ta sẽ sử dụng hàm mất mát Huber, thường hoạt động tốt hơn là trực tiếp giảm thiểu MAE, như đã thảo luận trong Chương 10. Chúng ta cũng sẽ sử dụng early stopping:

```
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[seq_length])
])

early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history = model.fit(train_ds, validation_data=valid_ds, epochs=500,
callbacks=[early_stopping_cb])
```

Mô hình này đạt MAE xác thực khoảng 37.866 (kết quả của bạn có thể khác). Điều này tốt hơn dự báo ngây thơ, nhưng tệ hơn mô hình SARIMA.

Chúng ta có thể làm tốt hơn với một RNN không? Hãy xem!

Dự báo bằng RNN đơn giản

Hãy thử RNN cơ bản nhất, chứa một lớp hồi quy đơn với chỉ một nơ-ron hồi quy, như chúng ta đã thấy trong Hình 15-1:

```
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(1, input_shape=[None, 1])
])
```

Tất cả các lớp hồi quy trong Keras mong đợi đầu vào 3D có hình dạng `[batch size, time steps, dimensionality]`, trong đó `dimensionality` là 1 đối với chuỗi thời gian đơn biến và lớn hơn đối với chuỗi thời gian đa biến. Nhớ lại rằng đối số `input_shape` bỏ qua chiều đầu tiên (tức là kích thước batch), và vì các lớp hồi quy có thể chấp nhận các chuỗi đầu vào có độ dài bất kỳ, chúng ta có thể đặt chiều thứ hai thành `None`, có nghĩa là "kích thước bất kỳ". Cuối cùng, vì chúng ta đang xử lý một chuỗi thời gian đơn biến, chúng ta cần kích thước của chiều cuối cùng là 1. Đây là lý do tại sao chúng ta chỉ định hình dạng đầu vào `[None, 1]`: nó có nghĩa là "các chuỗi đơn biến có độ dài bất kỳ". Lưu ý rằng các tập dữ liệu thực sự chứa đầu vào có hình dạng `[batch size, time steps]`, vì vậy chúng ta đang thiếu chiều cuối cùng, có kích thước 1, nhưng Keras đủ tốt bụng để thêm nó cho chúng ta trong trường hợp này.

Mô hình này hoạt động chính xác như chúng ta đã thấy trước đó: trạng thái ban đầu $h^{(init)}$ được đặt thành 0, và nó được truyền đến một nơ-ron hồi quy duy nhất, cùng với giá trị của bước thời gian đầu tiên, $x^{(0)}$. Nơ-ron tính toán tổng trọng số của các giá trị này cộng với số hạng độ lệch, và nó áp dụng hàm kích hoạt cho kết quả, sử dụng hàm hyperbolic tangent theo mặc định. Kết quả là đầu ra đầu tiên, $y^{(0)}$. Trong một RNN đơn giản, đầu ra này cũng là trạng thái mới $h^{(0)}$. Trạng thái mới này được truyền đến cùng một nơ-ron hồi quy cùng với giá trị đầu vào tiếp theo, $x^{(1)}$, và quá trình lặp lại cho đến bước thời gian cuối cùng. Cuối cùng, lớp chỉ xuất ra giá trị cuối cùng: trong trường hợp của chúng ta, các chuỗi dài 56 bước, vì vậy giá trị cuối cùng là $y^{(55)}$. Tất cả điều này được thực hiện đồng thời cho mỗi chuỗi trong batch, trong trường hợp này có 32 chuỗi.

Đó là mô hình hồi quy đầu tiên của chúng ta! Đó là một mô hình chuỗi-sang-vector. Vì chỉ có một nơ-ron đầu ra, vector đầu ra có kích thước là 1.

Bây giờ, nếu bạn biên dịch, huấn luyện và đánh giá mô hình này giống như mô hình trước, bạn sẽ thấy rằng nó không tốt chút nào: MAE xác thực của nó lớn hơn 100.000! Ôi. Điều đó đã được dự đoán trước, vì hai lý do:

12. Mô hình chỉ có một nơ-ron hồi quy duy nhất, vì vậy dữ liệu duy nhất nó có thể sử dụng để đưa ra dự đoán ở mỗi bước thời gian là giá trị đầu vào ở bước thời gian hiện tại và giá trị đầu ra từ bước thời gian trước đó. Điều đó không đủ để dựa vào! Nói cách khác, bộ nhớ của RNN cực kỳ hạn chế: nó chỉ là một số duy nhất, đầu ra trước đó của nó. Và hãy đếm xem mô hình này có bao nhiêu tham số: vì chỉ có một nơ-ron hồi quy với chỉ hai giá trị đầu vào, toàn bộ mô hình chỉ có ba tham số (hai trọng số cộng với một số hạng độ lệch). Điều đó còn xa mới đủ cho chuỗi thời gian này.

Ngược lại, mô hình trước của chúng ta có thể xem xét tất cả 56 giá trị trước đó cùng một lúc, và nó có tổng cộng 57 tham số.

13. Chuỗi thời gian chứa các giá trị từ 0 đến khoảng 1.4, nhưng vì hàm kích hoạt mặc định là `tanh`, lớp hồi quy chỉ có thể xuất ra các giá trị từ –1 đến +1. Không có cách nào nó có thể dự đoán các giá trị từ 1.0 đến 1.4.

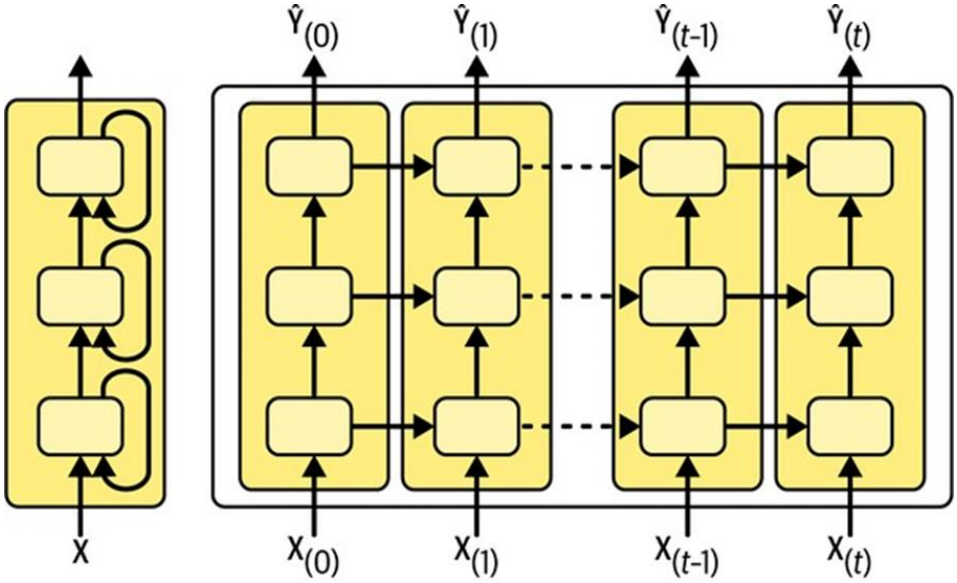
Hãy khắc phục cả hai vấn đề này: chúng ta sẽ tạo một mô hình với một lớp hồi quy lớn hơn, chứa 32 nơ-ron hồi quy, và chúng ta sẽ thêm một lớp đầu ra dày đặc lên trên nó với một nơ-ron đầu ra duy nhất và không có hàm kích hoạt. Lớp hồi quy sẽ có thể mang nhiều thông tin hơn từ bước thời gian này sang bước thời gian tiếp theo, và lớp đầu ra dày đặc sẽ chiếu đầu ra cuối cùng từ 32 chiều xuống 1, không có bất kỳ ràng buộc phạm vi giá trị nào:

```
univar_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 1]),
    tf.keras.layers.Dense(1) # no activation function by default
])
```

Bây giờ nếu bạn biên dịch, điều chỉnh và đánh giá mô hình này giống như mô hình trước, bạn sẽ thấy rằng MAE xác thực của nó đạt 27.703. Đó là mô hình tốt nhất chúng ta đã huấn luyện cho đến nay, và nó thậm chí còn đánh bại mô hình SARIMA: chúng ta đang làm khá tốt!

Dự báo bằng RNN sâu

Thông thường, người ta thường chồng nhiều lớp ô lên nhau, như thể hiện trong Hình 15-10. Điều này tạo thành một **RNN sâu**.



Hình 15-10. Một RNN sâu (trái) được mở rộng theo thời gian (phải)

Việc triển khai một RNN sâu với Keras rất đơn giản: chỉ cần chồng các lớp hồi quy lên nhau. Trong ví dụ sau, chúng ta sử dụng ba lớp `SimpleRNN` (nhưng chúng ta có thể sử dụng bất kỳ

loại lớp hồi quy nào khác thay thế, chẳng hạn như lớp LSTM hoặc lớp GRU, mà chúng ta sẽ thảo luận ngay sau đây). Hai lớp đầu tiên là các lớp chuỗi-sang-chuỗi, và lớp cuối cùng là một lớp chuỗi-sang-vector. Cuối cùng, lớp Dense tạo ra dự báo của mô hình (bạn có thể coi nó là một lớp vector-sang-vector). Vì vậy, mô hình này giống như mô hình được biểu diễn trong Hình 15-10, ngoại trừ các đầu ra $hatY^{(0)}$ đến $hatY^{(t-1)}$ bị bỏ qua, và có một lớp Dense nằm trên $hatY^{(t)}$, xuất ra dự báo thực tế:

```
import tensorflow as tf
import pandas as pd
import numpy as np

# Giả định df đã được tải và xử lý như trong các phần trước
# (Nếu bạn chưa chạy phần đó, bạn cần chạy lại để có df)
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-
_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

# Chia dữ liệu và chuẩn hóa như đã làm trước đó
rail_train = df["rail"]["2016-01":"2018-12"] / 1e6
rail_valid = df["rail"]["2019-01":"2019-05"] / 1e6
rail_test = df["rail"]["2019-06":] / 1e6

seq_length = 56
train_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_train.to_numpy(),
    targets=rail_train[seq_length:],
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_valid.to_numpy(),
    targets=rail_valid[seq_length:],
    sequence_length=seq_length,
    batch_size=32
)

deep_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True,
                              input_shape=[None, 1]),
    tf.keras.layers.SimpleRNN(32, return_sequences=True),
    tf.keras.layers.SimpleRNN(32),
```

```
    tf.keras.layers.Dense(1)
])

# Biên dịch và huấn luyện mô hình (sử dụng cùng các đối số từ mô hình tuyến
tính trước)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
deep_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])
history = deep_model.fit(train_ds, validation_data=valid_ds, epochs=500,
callbacks=[early_stopping_cb])
```

Nếu bạn huấn luyện và đánh giá mô hình này, bạn sẽ thấy rằng nó đạt MAE khoảng 31.211. Con số này tốt hơn cả hai đường cơ sở, nhưng nó không đánh bại RNN "nông" hơn của chúng ta. Có vẻ như RNN này hơi quá lớn so với nhiệm vụ của chúng ta.

Dự báo chuỗi thời gian đa biến

Một ưu điểm lớn của mạng thần kinh là tính linh hoạt của chúng: đặc biệt, chúng có thể xử lý các chuỗi thời gian đa biến mà hầu như không cần thay đổi kiến trúc. Ví dụ, hãy thử dự báo chuỗi thời gian đường sắt bằng cách sử dụng cả dữ liệu xe buýt và đường sắt làm đầu vào. Trên thực tế, hãy thêm cả loại ngày nữa! Vì chúng ta luôn có thể biết trước liệu ngày mai sẽ là ngày trong tuần, cuối tuần hay ngày lễ, chúng ta có thể dịch chuỗi loại ngày một ngày về phía trước, để mô hình được cung cấp loại ngày của ngày mai làm đầu vào. Để đơn giản, chúng ta sẽ thực hiện việc xử lý này bằng cách sử dụng Pandas:

```
# Giả định df đã được tải và xử lý như trong các phần trước
df_mulvar = df[["bus", "rail"]] / 1e6 # sử dụng cả chuỗi xe buýt & đường sắt
làm đầu vào
df_mulvar["next_day_type"] = df["day_type"].shift(-1) # chúng ta biết loại
ngày của ngày mai
df_mulvar = pd.get_dummies(df_mulvar) # mã hóa one-hot loại ngày
```

Bây giờ `df_mulvar` là một DataFrame với năm cột: dữ liệu xe buýt và đường sắt, cộng với ba cột chứa mã hóa one-hot của loại ngày tiếp theo (nhớ lại rằng có ba loại ngày có thể có, W, A và U). Tiếp theo, chúng ta có thể tiến hành như chúng ta đã làm trước đó. Đầu tiên chúng ta chia dữ liệu thành ba giai đoạn, để huấn luyện, xác thực và kiểm tra:

```
mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]
```

Sau đó, chúng ta tạo các tập dữ liệu:

```
seq_length = 56 # Vẫn sử dụng độ dài chuỗi 56
train_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(), # sử dụng tất cả 5 cột làm đầu vào
    targets=mulvar_train["rail"][seq_length:], # chỉ dự báo chuỗi đường sắt
    sequence_length=seq_length,
```

```
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_valid.to_numpy(),
    targets=mulvar_valid["rail"][seq_length:],
    sequence_length=seq_length,
    batch_size=32
)
```

Và cuối cùng chúng ta tạo RNN:

```
mulvar_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
    tf.keras.layers.Dense(1)
])

# Biên dịch và huấn luyện mô hình
mulvar_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])
history_mulvar = mulvar_model.fit(train_mulvar_ds,
validation_data=valid_mulvar_ds, epochs=500, callbacks=[early_stopping_cb])
```

Lưu ý rằng sự khác biệt duy nhất so với RNN `univar_model` chúng ta đã xây dựng trước đó là hình dạng đầu vào: ở mỗi bước thời gian, mô hình hiện nhận năm đầu vào thay vì một. Mô hình này thực sự đạt MAE xác thực là 22.062. Bây giờ chúng ta đang đạt được tiến bộ lớn!

Trên thực tế, không quá khó để làm cho RNN dự báo cả lượng hành khách xe buýt và đường sắt. Bạn chỉ cần thay đổi các mục tiêu khi tạo tập dữ liệu, đặt chúng thành `mulvar_train[["bus", "rail"]][seq_length:]` cho tập huấn luyện, và `mulvar_valid[["bus", "rail"]][seq_length:]` cho tập xác thực. Bạn cũng phải thêm một nơ-ron bổ sung vào lớp `Dense` đầu ra, vì giờ đây nó phải đưa ra hai dự báo: một cho lượng hành khách xe buýt vào ngày mai, và một cho đường sắt. Chỉ vậy thôi! Như chúng ta đã thảo luận trong Chương 10, việc sử dụng một mô hình duy nhất cho nhiều nhiệm vụ liên quan thường mang lại hiệu suất tốt hơn so với việc sử dụng một mô hình riêng biệt cho mỗi nhiệm vụ, vì các đặc trưng được học cho một nhiệm vụ có thể hữu ích cho các nhiệm vụ khác, và cũng vì việc phải thực hiện tốt trên nhiều nhiệm vụ ngăn mô hình bị quá khớp (đó là một dạng điều hòa). Tuy nhiên, điều đó phụ thuộc vào nhiệm vụ, và trong trường hợp cụ thể này, RNN đa nhiệm vụ dự báo cả lượng hành khách xe buýt và đường sắt không hoạt động tốt bằng các mô hình chuyên dụng dự báo một trong hai (sử dụng cả năm cột làm đầu vào). Tuy nhiên, nó đạt MAE xác thực là 25.330 cho đường sắt và 26.369 cho xe buýt, khá tốt.

Dự báo nhiều bước thời gian phía trước

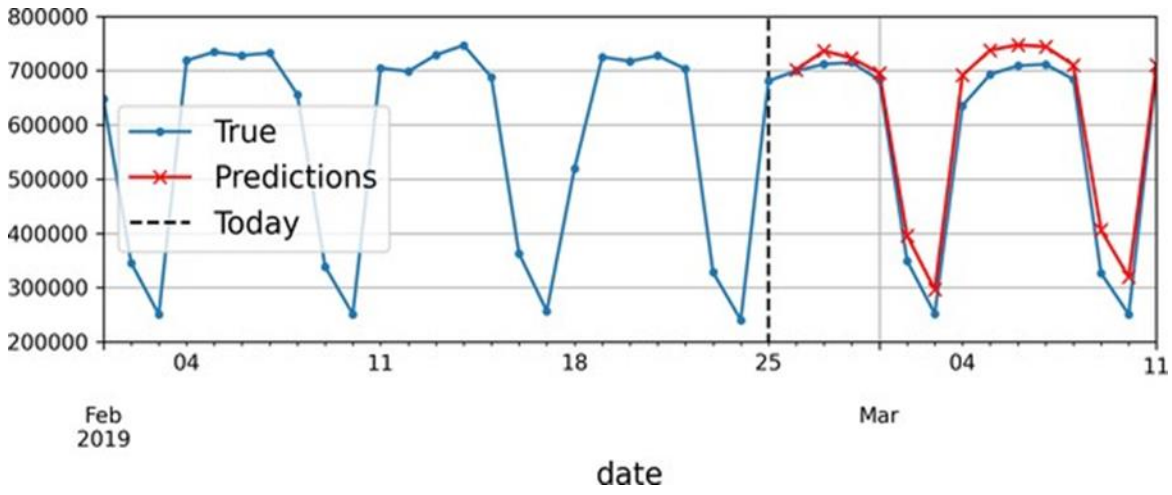
Cho đến nay, chúng ta chỉ dự đoán giá trị ở bước thời gian tiếp theo, nhưng chúng ta cũng có thể dễ dàng dự đoán giá trị nhiều bước phía trước bằng cách thay đổi các mục tiêu một

cách thích hợp (ví dụ: để dự đoán lượng hành khách 2 tuần kể từ bây giờ, chúng ta có thể chỉ cần thay đổi các mục tiêu là giá trị 14 ngày tới thay vì 1 ngày tới). Nhưng điều gì sẽ xảy ra nếu chúng ta muốn dự đoán 14 giá trị tiếp theo? Tùy chọn đầu tiên là lấy RNN `univar_model` mà chúng ta đã huấn luyện trước đó cho chuỗi thời gian đường sắt, làm cho nó dự đoán giá trị tiếp theo, và thêm giá trị đó vào đầu vào, hành động như thể giá trị dự đoán đã thực sự xảy ra; sau đó chúng ta sẽ sử dụng lại mô hình để dự đoán giá trị tiếp theo, và cứ thế tiếp tục, như trong đoạn mã sau:

```
# Đảm bảo univar_model đã được huấn luyện
# tf.random.set_seed(42)
# univar_model = tf.keras.Sequential([
# tf.keras.layers.SimpleRNN(32, input_shape=[None,1]),
# tf.keras.layers.Dense(1)
# ])
# early_stopping_cb và opt đã được định nghĩa
# univar_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=
["mae"])
# history_univar = univar_model.fit(train_ds, validation_data=valid_ds,
epochs=500, callbacks=[early_stopping_cb])

X = rail_valid.to_numpy()[np.newaxis, :seq_length, np.newaxis]
for step_ahead in range(14):
    y_pred_one = univar_model.predict(X)
    X = np.concatenate([X, y_pred_one.reshape(1, 1, 1)], axis=1)
```

Trong mã này, chúng ta lấy lượng hành khách đường sắt của 56 ngày đầu tiên của giai đoạn xác thực, và chúng ta chuyển đổi dữ liệu thành một mảng NumPy có hình dạng `[1, 56, 1]` (nhớ lại rằng các lớp hồi quy mong đợi đầu vào 3D). Sau đó, chúng ta liên tục sử dụng mô hình để dự báo giá trị tiếp theo, và chúng ta nối mỗi dự báo vào chuỗi đầu vào, dọc theo trục thời gian (`axis=1`). Các dự báo kết quả được vẽ trong Hình 15-11.



Hình 15-11. Dự báo 14 bước phía trước, mỗi lần 1 bước

Tùy chọn thứ hai là huấn luyện một RNN để dự đoán 14 giá trị tiếp theo trong một lần. Chúng ta vẫn có thể sử dụng một mô hình chuỗi-sang-vector, nhưng nó sẽ xuất ra 14 giá trị thay vì 1. Tuy nhiên, trước tiên chúng ta cần thay đổi các mục tiêu thành các vector chứa 14 giá trị tiếp theo. Để làm điều này, chúng ta có thể sử dụng lại `timeseries_dataset_from_array()`, nhưng lần này yêu cầu nó tạo tập dữ liệu không có mục tiêu (`targets=None`) và với các chuỗi dài hơn, có độ dài `seq_length + 14`. Sau đó, chúng ta có thể sử dụng phương thức `map()` của tập dữ liệu để áp dụng một hàm tùy chỉnh cho mỗi batch của chuỗi, chia chúng thành đầu vào và mục tiêu. Trong ví dụ này, chúng ta sử dụng chuỗi thời gian đa biến làm đầu vào (sử dụng cả năm cột), và chúng ta dự báo lượng hành khách đường sắt trong 14 ngày tới:

```
def split_inputs_and_targets(mulvar_series, ahead=14, target_col=1):
    return mulvar_series[:, :-ahead], mulvar_series[:, -ahead:, target_col]

ahead_train_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(),
    targets=None,
    sequence_length=seq_length + 14,
    batch_size=32,
    shuffle=True,
    seed=42
).map(split_inputs_and_targets)

ahead_valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_valid.to_numpy(),
    targets=None,
    sequence_length=seq_length + 14,
    batch_size=32
).map(split_inputs_and_targets)
```

Bây giờ chúng ta chỉ cần lớp đầu ra có 14 đơn vị thay vì 1:

```
ahead_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

# Biên dịch và huấn luyện mô hình
ahead_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])
history_ahead = ahead_model.fit(ahead_train_ds,
validation_data=ahead_valid_ds, epochs=500, callbacks=[early_stopping_cb])
```

Sau khi huấn luyện mô hình này, bạn có thể dự đoán 14 giá trị tiếp theo cùng một lúc như sau:

```
X = mulvar_valid.to_numpy()[np.newaxis, :seq_length] # shape [1, 56, 5]
Y_pred = ahead_model.predict(X) # shape [1, 14]
```

Cách tiếp cận này hoạt động khá tốt. Các dự báo của nó cho ngày tiếp theo rõ ràng tốt hơn các dự báo của nó cho 14 ngày trong tương lai, nhưng nó không tích lũy lỗi như cách tiếp cận trước đó. Tuy nhiên, chúng ta vẫn có thể làm tốt hơn, sử dụng mô hình chuỗi-sang-chuỗi (hoặc seq2seq).

Dự báo bằng mô hình chuỗi-sang-chuỗi Thay vì huấn luyện mô hình để dự báo 14 giá trị tiếp theo chỉ ở bước thời gian cuối cùng, chúng ta có thể huấn luyện nó dự báo 14 giá trị tiếp theo ở mỗi và mọi bước thời gian. Nói cách khác, chúng ta có thể biến RNN chuỗi-sang-vector này thành một **RNN chuỗi-sang-chuỗi**. Ưu điểm của kỹ thuật này là hàm mất mát sẽ chứa một số hạng cho đầu ra của RNN ở mỗi và mọi bước thời gian, không chỉ cho đầu ra ở bước thời gian cuối cùng. Điều này có nghĩa là sẽ có nhiều gradient lỗi hơn chảy qua mô hình, và chúng sẽ không phải chảy qua thời gian nhiều vì chúng sẽ đến từ đầu ra của mỗi bước thời gian, không chỉ bước cuối cùng. Điều này sẽ vừa ổn định vừa tăng tốc độ huấn luyện. Để rõ ràng, tại bước thời gian 0, mô hình sẽ xuất ra một vector chứa các dự báo cho các bước thời gian 1 đến 14, sau đó tại bước thời gian 1, mô hình sẽ dự báo các bước thời gian 2 đến 15, v.v. Nói cách khác, các mục tiêu là các chuỗi các cửa sổ liên tiếp, được dịch chuyển một bước thời gian ở mỗi bước thời gian. Mục tiêu không còn là một vector nữa, mà là một chuỗi có cùng độ dài với đầu vào, chứa một vector 14 chiều ở mỗi bước. Việc chuẩn bị tập dữ liệu không hề đơn giản, vì mỗi thể hiện có một cửa sổ làm đầu vào và một chuỗi các cửa sổ làm đầu ra. Một cách để làm điều này là sử dụng hàm tiện ích `to_windows()` mà chúng ta đã tạo trước đó, hai lần liên tiếp, để có được các cửa sổ của các cửa sổ liên tiếp. Ví dụ, hãy biến chuỗi các số từ 0 đến 6 thành một tập dữ liệu chứa các chuỗi gồm 4 cửa sổ liên tiếp, mỗi cửa sổ có độ dài 3:

```
import tensorflow as tf
import numpy as np

# Định nghĩa lại hàm to_windows nếu chưa có
def to_windows(dataset, length):
    dataset = dataset.window(length, shift=1, drop_remainder=True)
    return dataset.flat_map(lambda window_ds:
                            window_ds.batch(length))

my_series = tf.data.Dataset.range(7)
dataset = to_windows(to_windows(my_series, 3), 4)

for item in dataset:
    print(item)
```

```
<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4],
       [3, 4, 5]])>
<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
array([[1, 2, 3],
       [2, 3, 4],
```

```
       [3, 4, 5],
       [4, 5, 6]])>
```

Bây giờ chúng ta có thể sử dụng phương thức `map()` để chia các cửa sổ của các cửa sổ này thành đầu vào và mục tiêu:

```
dataset = dataset.map(lambda S: (S[:, 0], S[:, 1:]))

for item in dataset:
    print(item)
```

```
(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([0, 1, 2, 3])>, <tf.Tensor:
shape=(4, 2), dtype=int64, numpy=
array([[1, 2],
       [2, 3],
       [3, 4],
       [4, 5]])>)
(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([1, 2, 3, 4])>, <tf.Tensor:
shape=(4, 2), dtype=int64, numpy=
array([[2, 3],
       [3, 4],
       [4, 5],
       [5, 6]])>)
```

Bây giờ tập dữ liệu chứa các chuỗi có độ dài 4 làm đầu vào, và các mục tiêu là các chuỗi chứa hai bước tiếp theo, cho mỗi bước thời gian. Ví dụ, chuỗi đầu vào đầu tiên là [0, 1, 2, 3], và các mục tiêu tương ứng của nó là [[1, 2], [2, 3], [3, 4], [4, 5]], đó là hai giá trị tiếp theo cho mỗi bước thời gian. Nếu bạn giống tôi, bạn có thể sẽ cần vài phút để hiểu rõ điều này. Cứ từ từ!

Hãy tạo một hàm tiện ích nhỏ khác để chuẩn bị tập dữ liệu cho mô hình chuỗi-sang-chuỗi của chúng ta. Nó cũng sẽ lo việc xáo trộn (tùy chọn) và nhóm batch:

```
import pandas as pd # Cần thiết cho mulvar_train, mulvar_valid

# Giả định df đã được tải và xử lý như trong các phần trước
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-
_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

# Tạo df_mulvar và chia dữ liệu như đã làm trước đó
df_mulvar = df[["bus", "rail"]] / 1e6
df_mulvar["next_day_type"] = df["day_type"].shift(-1)
df_mulvar = pd.get_dummies(df_mulvar)
```

```
mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]

def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                       batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series),
                    ahead + 1)
    ds = to_windows(ds, seq_length).map(lambda S: (S[:, 0], S[:, 1:,
target_col])) # Sử dụng target_col để lấy cột mục tiêu

    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)
```

Bây giờ chúng ta có thể sử dụng hàm này để tạo tập dữ liệu:

```
seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)
```

Và cuối cùng, chúng ta có thể xây dựng mô hình chuỗi-sang-chuỗi:

```
seq2seq_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True,
                              input_shape=[None, 5]),
    tf.keras.layers.Dense(14) # output 14 values
])

# Biên dịch và huấn luyện mô hình (sử dụng các đối số tương tự)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
seq2seq_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])
history_seq2seq = seq2seq_model.fit(seq2seq_train,
validation_data=seq2seq_valid, epochs=500, callbacks=[early_stopping_cb])
```

Nó gần như giống hệt mô hình trước của chúng ta: điểm khác biệt duy nhất là chúng ta đặt `return_sequences=True` trong lớp `SimpleRNN`. Bằng cách này, nó sẽ xuất ra một chuỗi các vector (mỗi vector có kích thước 32), thay vì xuất ra một vector duy nhất ở bước thời gian cuối cùng. Lớp `Dense` đủ thông minh để xử lý các chuỗi làm đầu vào: nó sẽ được áp dụng ở mỗi bước thời gian, lấy một vector 32 chiều làm đầu vào và xuất ra một vector 14 chiều. Trên thực tế, một cách khác để có được kết quả chính xác tương tự là sử dụng một lớp `Conv1D` với kích thước kernel là 1: `Conv1D(14, kernel_size=1)`.

Mã huấn luyện vẫn như thường lệ. Trong quá trình huấn luyện, tất cả các đầu ra của mô hình đều được sử dụng, nhưng sau khi huấn luyện, chỉ có đầu ra của bước thời gian cuối

cùng là quan trọng, và phần còn lại có thể bị bỏ qua. Ví dụ, chúng ta có thể dự báo lượng hành khách đường sắt trong 14 ngày tới như sau:

```
seq_length = 56 # Đảm bảo seq_length được định nghĩa
X = mulvar_valid.to_numpy()[np.newaxis, :seq_length]
y_pred_14 = seq2seq_model.predict(X)[0, -1] # chỉ lấy đầu ra của bước thời
gian cuối cùng
```

Nếu bạn đánh giá dự báo của mô hình này cho $t + 1$, bạn sẽ thấy MAE xác thực là 25.519. Đối với $t + 2$ là 26.274, và hiệu suất tiếp tục giảm dần khi mô hình cố gắng dự báo xa hơn trong tương lai. Tại $t + 14$, MAE là 34.322.

Các RNN đơn giản có thể khá tốt trong việc dự báo chuỗi thời gian hoặc xử lý các loại chuỗi khác, nhưng chúng không hoạt động tốt trên các chuỗi hoặc chuỗi thời gian dài. Hãy thảo luận lý do và xem chúng ta có thể làm gì để khắc phục.

Dự báo bằng mô hình Sequence-to-Sequence

Thay vì huấn luyện mô hình chỉ dự báo 14 giá trị tiếp theo tại bước thời gian cuối cùng, chúng ta có thể huấn luyện nó dự báo 14 giá trị tiếp theo tại **mỗi và mọi bước thời gian**. Nói cách khác, chúng ta có thể biến RNN sequence-to-vector này thành **RNN sequence-to-sequence**. Ưu điểm của kỹ thuật này là hàm mất mát sẽ chứa một thành phần cho đầu ra của RNN tại mỗi và mọi bước thời gian, không chỉ cho đầu ra ở bước thời gian cuối cùng. Điều này có nghĩa là sẽ có nhiều gradient lỗi hơn truyền qua mô hình, và chúng sẽ không phải truyền qua thời gian nhiều vì chúng đến từ đầu ra của mỗi bước thời gian, không chỉ từ bước cuối cùng. Điều này sẽ giúp **ổn định và tăng tốc độ huấn luyện**.

Để rõ ràng hơn, tại bước thời gian 0, mô hình sẽ xuất ra một vector chứa các dự báo cho các bước thời gian từ 1 đến 14, sau đó tại bước thời gian 1, mô hình sẽ dự báo cho các bước thời gian từ 2 đến 15, v.v. Nói cách khác, các mục tiêu là các chuỗi cửa sổ liên tiếp, được dịch chuyển đi một bước thời gian tại mỗi bước thời gian. Mục tiêu không còn là một vector nữa, mà là một chuỗi có cùng độ dài với đầu vào, chứa một vector 14 chiều ở mỗi bước.

Việc chuẩn bị các tập dữ liệu không hề đơn giản, vì mỗi mẫu có một cửa sổ làm đầu vào và một chuỗi các cửa sổ làm đầu ra. Một cách để làm điều này là sử dụng hàm tiện ích `to_windows()` mà chúng ta đã tạo trước đó, hai lần liên tiếp, để nhận được các cửa sổ của các cửa sổ liên tiếp. Ví dụ, hãy biến chuỗi số từ 0 đến 6 thành một tập dữ liệu chứa các chuỗi gồm 4 cửa sổ liên tiếp, mỗi cửa sổ có độ dài 3:

```
>>> my_series = tf.data.Dataset.range(7)
>>> dataset = to_windows(to_windows(my_series, 3), 4)
>>> list(dataset)
[<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
  array([[0, 1, 2],
         [1, 2, 3],
         [2, 3, 4],
         [3, 4, 5]])>,
```

```
 <tf.Tensor: shape=(4, 3), dtype=int64, numpy=
  array([[1, 2, 3],
         [2, 3, 4],
         [3, 4, 5],
         [4, 5, 6]])>]
```

Bây giờ chúng ta có thể sử dụng phương thức `map()` để tách các cửa sổ của cửa sổ này thành đầu vào và mục tiêu:

```
>>> dataset = dataset.map(lambda S: (S[:, 0], S[:, 1:]))
>>> list(dataset)
[(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([0, 1, 2, 3])>,
  <tf.Tensor: shape=(4, 2), dtype=int64, numpy=
   array([[1, 2],
          [2, 3],
          [3, 4],
          [4, 5]])>),
 (<tf.Tensor: shape=(4,), dtype=int64, numpy=array([1, 2, 3, 4])>,
  <tf.Tensor: shape=(4, 2), dtype=int64, numpy=
   array([[2, 3],
          [3, 4],
          [4, 5],
          [5, 6]])>)]
```

Bây giờ tập dữ liệu chứa các chuỗi có độ dài 4 làm đầu vào, và các mục tiêu là các chuỗi chứa hai bước tiếp theo, cho mỗi bước thời gian. Ví dụ, chuỗi đầu vào đầu tiên là `[0, 1, 2, 3]`, và các mục tiêu tương ứng của nó là `[[1, 2], [2, 3], [3, 4], [4, 5]]`, là hai giá trị tiếp theo cho mỗi bước thời gian. Nếu bạn giống tôi, bạn có thể sẽ cần vài phút để hiểu rõ điều này. Cứ từ từ nhé!

Hãy tạo một hàm tiện ích nhỏ khác để chuẩn bị các tập dữ liệu cho mô hình sequence-to-sequence của chúng ta. Nó cũng sẽ xử lý việc xáo trộn (tùy chọn) và tạo lô:

```
def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                       batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series), ahead + 1)
    ds = to_windows(ds, seq_length).map(lambda S: (S[:, 0], S[:, 1:, 1]))
    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)
```

Bây giờ chúng ta có thể sử dụng hàm này để tạo các tập dữ liệu:

```
seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)
```

Và cuối cùng, chúng ta có thể xây dựng mô hình sequence-to-sequence:

```
seq2seq_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True, input_shape=[None,
5]),
```

```
    tf.keras.layers.Dense(14)
])
```

Nó gần như giống hệt với mô hình trước của chúng ta: điểm khác biệt duy nhất là chúng ta đặt `return_sequences=True` trong lớp `SimpleRNN`. Bằng cách này, nó sẽ xuất ra một chuỗi các vector (mỗi vector có kích thước 32), thay vì xuất ra một vector duy nhất ở bước thời gian cuối cùng. Lớp `Dense` đủ thông minh để xử lý các chuỗi làm đầu vào: nó sẽ được áp dụng tại mỗi bước thời gian, nhận một vector 32 chiều làm đầu vào và xuất ra một vector 14 chiều. Trên thực tế, một cách khác để có được kết quả hoàn toàn giống hệt là sử dụng một lớp `Conv1D` với kích thước kernel là 1: `Conv1D(14, kernel_size=1)`.

Mã huấn luyện vẫn như thường lệ. Trong quá trình huấn luyện, tất cả các đầu ra của mô hình đều được sử dụng, nhưng sau khi huấn luyện, chỉ có đầu ra của bước thời gian cuối cùng mới quan trọng, và phần còn lại có thể được bỏ qua. Ví dụ, chúng ta có thể dự báo lượng hành khách đi tàu cho 14 ngày tiếp theo như sau:

```
X = mulvar_valid.to_numpy()[np.newaxis, :seq_length]
y_pred_14 = seq2seq_model.predict(X)[0, -1] # chỉ lấy đầu ra của bước thời
gian cuối cùng
```

Nếu bạn đánh giá các dự báo của mô hình này cho t+1, bạn sẽ tìm thấy một MAE (Sai số tuyệt đối trung bình) trên tập kiểm định là 25,519. Đối với t+2, nó là 26,274, và hiệu suất tiếp tục giảm dần khi mô hình cố gắng dự báo xa hơn trong tương lai. Tại t+14, MAE là 34,322.

Các RNN đơn giản có thể khá tốt trong việc dự báo chuỗi thời gian hoặc xử lý các loại chuỗi khác, nhưng chúng không hoạt động tốt trên các chuỗi thời gian hoặc chuỗi dài. Hãy thảo luận tại sao và xem chúng ta có thể làm gì về nó.

Xử lý các chuỗi dài

Để huấn luyện một RNN trên các chuỗi dài, chúng ta phải chạy nó qua nhiều bước thời gian, làm cho RNN được mở rộng trở thành một mạng rất sâu. Giống như bất kỳ mạng thần kinh sâu nào, nó có thể gặp phải vấn đề gradient không ổn định, được thảo luận trong Chương 11: nó có thể mất mãi mãi để huấn luyện, hoặc quá trình huấn luyện có thể không ổn định. Hơn nữa, khi một RNN xử lý một chuỗi dài, nó sẽ dần dần quên đi các đầu vào đầu tiên trong chuỗi. Hãy xem xét cả hai vấn đề này, bắt đầu với vấn đề gradient không ổn định.

Giải quyết vấn đề gradient không ổn định

Nhiều thủ thuật chúng ta đã sử dụng trong các mạng sâu để giảm bớt vấn đề gradient không ổn định cũng có thể được sử dụng cho RNN: khởi tạo tham số tốt, bộ tối ưu hóa nhanh hơn, dropout, v.v. Tuy nhiên, các hàm kích hoạt không bão hòa (ví dụ: ReLU) có thể không giúp ích nhiều ở đây. Trên thực tế, chúng thực sự có thể khiến RNN không ổn định hơn trong quá trình huấn luyện. Tại sao? Chà, giả sử gradient descent cập nhật các trọng số theo cách làm tăng nhẹ các đầu ra ở bước thời gian đầu tiên. Vì cùng các trọng số được sử dụng ở mọi bước thời gian, các đầu ra ở bước thời gian thứ hai cũng có thể tăng nhẹ, và những thứ ở bước thứ ba, v.v. cho đến khi các đầu ra bùng nổ—và một hàm kích hoạt không bão hòa không ngăn được điều đó. Bạn có thể giảm thiểu rủi ro này bằng cách sử

dụng tốc độ học nhỏ hơn, hoặc bạn có thể sử dụng một hàm kích hoạt bão hòa như hàm hyperbolic tangent (điều này giải thích tại sao nó là mặc định). Cũng theo cách tương tự, các gradient có thể bùng nổ. Nếu bạn nhận thấy quá trình huấn luyện không ổn định, bạn có thể muốn theo dõi kích thước của các gradient (ví dụ: sử dụng TensorBoard) và có thể sử dụng **cắt gradient (gradient clipping)**. Hơn nữa, **chuẩn hóa hàng loạt (batch normalization)** không thể được sử dụng hiệu quả với RNN như với các mạng truyền thẳng sâu. Trên thực tế, bạn không thể sử dụng nó giữa các bước thời gian, chỉ giữa các lớp hồi quy. Để chính xác hơn, về mặt kỹ thuật, có thể thêm một lớp BN vào một ô bộ nhớ (như bạn sẽ thấy ngay sau đây) để nó sẽ được áp dụng ở mỗi bước thời gian (cả trên đầu vào cho bước thời gian đó và trên trạng thái ẩn từ bước trước). Tuy nhiên, cùng một lớp BN sẽ được sử dụng ở mỗi bước thời gian, với cùng các tham số, bất kể thang đo và độ lệch thực tế của đầu vào và trạng thái ẩn. Trong thực tế, điều này không mang lại kết quả tốt, như đã được César Laurent et al. chứng minh trong một bài báo năm 2015: các tác giả nhận thấy rằng BN chỉ có lợi một chút khi nó được áp dụng cho đầu vào của lớp, không phải cho trạng thái ẩn. Nói cách khác, nó tốt hơn một chút so với không có gì khi được áp dụng giữa các lớp hồi quy (tức là theo chiều dọc trong Hình 15-10), nhưng không phải trong các lớp hồi quy (tức là theo chiều ngang). Trong Keras, bạn có thể áp dụng BN giữa các lớp đơn giản bằng cách thêm một lớp `BatchNormalization` trước mỗi lớp hồi quy, nhưng nó sẽ làm chậm quá trình huấn luyện và có thể không giúp ích nhiều.

Một dạng chuẩn hóa khác thường hoạt động tốt hơn với RNN: **chuẩn hóa lớp (layer normalization)**. Ý tưởng này được Jimmy Lei Ba et al. giới thiệu trong một bài báo năm 2016: nó rất giống với chuẩn hóa hàng loạt, nhưng thay vì chuẩn hóa trên chiều batch, chuẩn hóa lớp chuẩn hóa trên chiều đặc trưng. Một ưu điểm là nó có thể tính toán các thống kê cần thiết một cách nhanh chóng, ở mỗi bước thời gian, độc lập cho mỗi thể hiện. Điều này cũng có nghĩa là nó hoạt động theo cùng một cách trong quá trình huấn luyện và kiểm tra (ngược lại với BN), và nó không cần sử dụng trung bình động hàm mũ để ước tính các thống kê đặc trưng trên tất cả các thể hiện trong tập huấn luyện, như BN làm. Giống như BN, chuẩn hóa lớp học một tham số tỷ lệ và một tham số bù cho mỗi đầu vào. Trong một RNN, nó thường được sử dụng ngay sau khi kết hợp tuyến tính các đầu vào và các trạng thái ẩn. Hãy sử dụng Keras để triển khai chuẩn hóa lớp trong một ô bộ nhớ đơn giản. Để làm điều này, chúng ta cần định nghĩa một ô bộ nhớ tùy chỉnh, giống như một lớp thông thường, ngoại trừ phương thức `call()` của nó nhận hai đối số: các đầu vào ở bước thời gian hiện tại và các trạng thái ẩn từ bước thời gian trước đó. Lưu ý rằng đối số `states` là một danh sách chứa một hoặc nhiều tensor. Trong trường hợp một ô RNN đơn giản, nó chứa một tensor duy nhất bằng với đầu ra của bước thời gian trước đó, nhưng các ô khác có thể có nhiều tensor trạng thái (ví dụ: một `LSTMCell` có trạng thái dài hạn và trạng thái ngắn hạn, như bạn sẽ thấy ngay sau đây). Một ô cũng phải có thuộc tính `state_size` và thuộc tính `output_size`. Trong một RNN đơn giản, cả hai đều đơn giản bằng số đơn vị. Mã sau đây triển khai một ô bộ nhớ tùy chỉnh sẽ hoạt động như một `SimpleRNNCell`, ngoại trừ nó cũng sẽ áp dụng chuẩn hóa lớp ở mỗi bước thời gian:

```
class LNSimpleRNNCell(tf.keras.layers.Layer):
    def __init__(self, units, activation="tanh", **kwargs):
        super().__init__(**kwargs)
```

```
        self.state_size = units
        self.output_size = units
        self.simple_rnn_cell = tf.keras.layers.SimpleRNNCell(units,

activation=None)
        self.layer_norm = tf.keras.layers.LayerNormalization()
        self.activation = tf.keras.activations.get(activation)

    def call(self, inputs, states):
        outputs, new_states = self.simple_rnn_cell(inputs, states)
        norm_outputs = self.activation(self.layer_norm(outputs))
        return norm_outputs, [norm_outputs]
```

Hãy đi qua mã này:

- Lớp `LNSimpleRNNCell` của chúng ta kế thừa từ lớp `tf.keras.layers.Layer`, giống như bất kỳ lớp tùy chỉnh nào khác.
- Hàm tạo lấy số đơn vị và hàm kích hoạt mong muốn và đặt các thuộc tính `state_size` và `output_size`, sau đó tạo một `SimpleRNNCell` không có hàm kích hoạt (vì chúng ta muốn thực hiện chuẩn hóa lớp sau phép toán tuyến tính nhưng trước hàm kích hoạt). Sau đó, hàm tạo tạo lớp `LayerNormalization`, và cuối cùng nó tìm nạp hàm kích hoạt mong muốn.
- Phương thức `call()` bắt đầu bằng cách áp dụng `simple_rnn_cell`, tính toán sự kết hợp tuyến tính của các đầu vào hiện tại và các trạng thái ẩn trước đó, và nó trả về kết quả hai lần (thực ra, trong một `SimpleRNNCell`, các đầu ra chỉ bằng với các trạng thái ẩn: nói cách khác, `new_states[0]` bằng với `outputs`, vì vậy chúng ta có thể bỏ qua `new_states` một cách an toàn trong phần còn lại của phương thức `call()`).
- Tiếp theo, phương thức `call()` áp dụng chuẩn hóa lớp, sau đó là hàm kích hoạt. Cuối cùng, nó trả về các đầu ra hai lần: một lần dưới dạng đầu ra, và một lần dưới dạng các trạng thái ẩn mới. Để sử dụng ô tùy chỉnh này, tất cả những gì chúng ta cần làm là tạo một lớp `tf.keras.layers.RNN`, truyền cho nó một thể hiện ô:

```
custom_ln_model = tf.keras.Sequential([
    tf.keras.layers.RNN(LNSimpleRNNCell(32), return_sequences=True,
                                input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

# Biên dịch và huấn luyện mô hình (sử dụng các đối số tương tự)
custom_ln_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])
history_custom_ln = custom_ln_model.fit(seq2seq_train,
validation_data=seq2seq_valid, epochs=500, callbacks=[early_stopping_cb])
```

Tương tự, bạn có thể tạo một ô tùy chỉnh để áp dụng dropout giữa mỗi bước thời gian. Nhưng có một cách đơn giản hơn: hầu hết các lớp và ô hồi quy được cung cấp bởi Keras có các siêu tham số `dropout` và `recurrent_dropout`: cái trước định nghĩa tỷ lệ dropout để áp dụng cho đầu vào, và cái sau định nghĩa tỷ lệ dropout cho các trạng thái ẩn, giữa các bước

thời gian. Vì vậy, không cần thiết phải tạo một ô tùy chỉnh để áp dụng dropout ở mỗi bước thời gian trong một RNN. Với các kỹ thuật này, bạn có thể giảm thiểu vấn đề gradient không ổn định và huấn luyện một RNN hiệu quả hơn nhiều. Bây giờ chúng ta hãy xem cách giải quyết vấn đề bộ nhớ ngắn hạn.

Nghĩ lại vấn đề bộ nhớ ngắn hạn

Do các biến đổi mà dữ liệu trải qua khi đi qua một RNN, một số thông tin bị mất ở mỗi bước thời gian. Sau một thời gian, trạng thái của RNN hầu như không còn dấu vết nào của các đầu vào đầu tiên. Điều này có thể là một trở ngại lớn. Hãy tưởng tượng Dory con cá đang cố gắng dịch một câu dài; đến khi cô ấy đọc xong, cô ấy không biết câu đó bắt đầu như thế nào. Để giải quyết vấn đề này, nhiều loại ô có bộ nhớ dài hạn đã được giới thiệu. Chúng đã chứng minh thành công đến mức các ô cơ bản không còn được sử dụng nhiều nữa. Đầu tiên hãy cùng xem xét ô bộ nhớ dài hạn phổ biến nhất trong số này: ô LSTM.

Các ô LSTM Ô bộ nhớ dài hạn ngắn hạn (LSTM) được Sepp Hochreiter và Jürgen Schmidhuber đề xuất vào năm 1997 và dần được cải thiện qua nhiều năm bởi một số nhà nghiên cứu, chẳng hạn như Alex Graves, Haşim Sak, và Wojciech Zaremba. Nếu bạn coi ô LSTM như một hộp đen, nó có thể được sử dụng rất giống một ô cơ bản, ngoại trừ nó sẽ hoạt động tốt hơn nhiều; quá trình huấn luyện sẽ hội tụ nhanh hơn và nó sẽ phát hiện các mẫu dài hạn hơn trong dữ liệu. Trong Keras, bạn có thể đơn giản sử dụng lớp `LSTM` thay vì lớp `SimpleRNN`:

```
import tensorflow as tf
import pandas as pd
import numpy as np

# Giả định df đã được tải và xử lý như trong các phần trước
# (Nếu bạn chưa chạy phần đó, bạn cần chạy lại để có df)
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-
_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

# Tạo df_mulvar và chia dữ liệu như đã làm trước đó
df_mulvar = df[["bus", "rail"]] / 1e6
df_mulvar["next_day_type"] = df["day_type"].shift(-1)
df_mulvar = pd.get_dummies(df_mulvar)

mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]
```

```
seq_length = 56 # Vẫn sử dụng độ dài chuỗi 56
ahead = 14 # Số bước thời gian dự báo

def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                       batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series),
                    ahead + 1)
    ds = to_windows(ds, seq_length).map(lambda S: (S[:, 0], S[:, 1:,
target_col]))

    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)

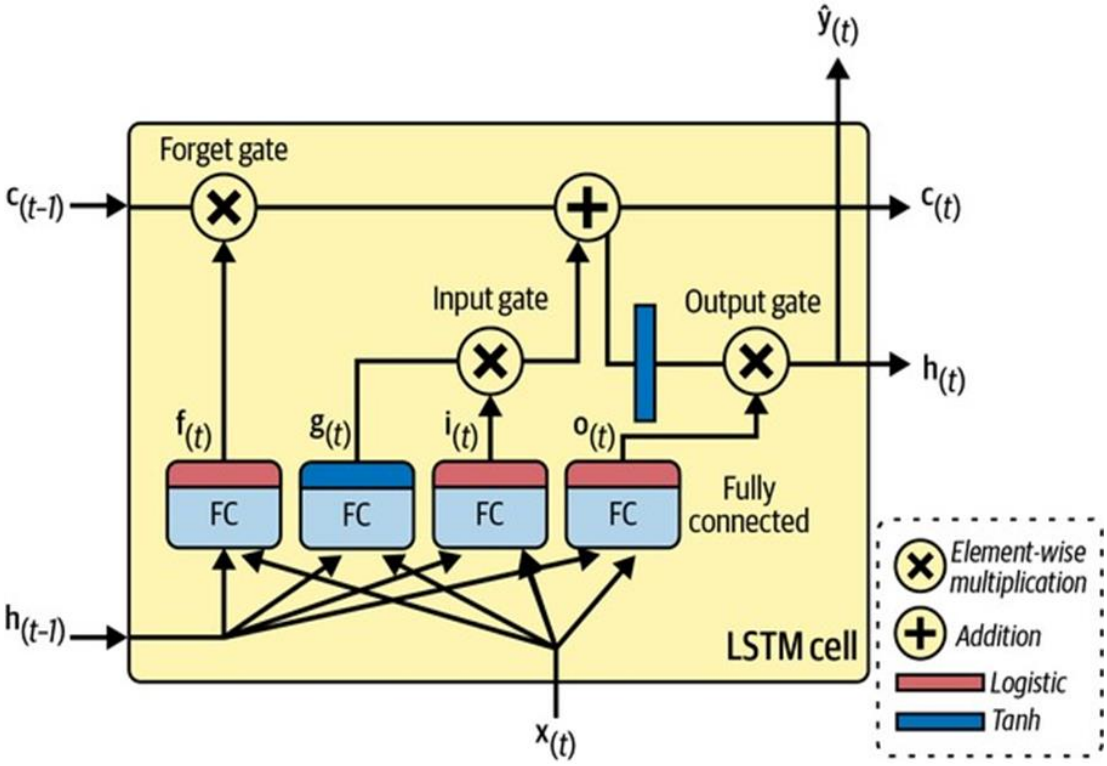
seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)

model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

# Biên dịch và huấn luyện mô hình (sử dụng các đối số tương tự)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history_lstm = model.fit(seq2seq_train, validation_data=seq2seq_valid,
epochs=500, callbacks=[early_stopping_cb])
```

Ngoài ra, bạn có thể sử dụng lớp `tf.keras.layers.RNN` đa năng, truyền cho nó một `LSTMCell` làm đối số. Tuy nhiên, lớp `LSTM` sử dụng một triển khai được tối ưu hóa khi chạy trên GPU (xem Chương 19), vì vậy nói chung nên ưu tiên sử dụng nó (lớp `RNN` chủ yếu hữu ích khi bạn định nghĩa các ô tùy chỉnh, như chúng ta đã làm trước đó).

Vậy một ô LSTM hoạt động như thế nào? Kiến trúc của nó được thể hiện trong Hình 15-12. Nếu bạn không nhìn vào bên trong hộp, ô LSTM trông giống hệt một ô thông thường, ngoại trừ trạng thái của nó được chia thành hai vector: $h^{(t)}$ và $c^{(t)}$ ("c" là viết tắt của "cell"). Bạn có thể coi $h^{(t)}$ là trạng thái ngắn hạn và $c^{(t)}$ là trạng thái dài hạn.



Hình 15-12. Một ô LSTM

Bây giờ hãy mở hộp ra! Ý tưởng chính là mạng có thể học cách lưu trữ những gì vào trạng thái dài hạn, những gì cần loại bỏ và những gì cần đọc từ đó. Khi trạng thái dài hạn $c^{(t-1)}$ đi qua mạng từ trái sang phải, bạn có thể thấy rằng nó đi qua một **cổng quên (forget gate)**, loại bỏ một số ký ức, và sau đó nó thêm một số ký ức mới thông qua phép toán cộng (thêm các ký ức đã được chọn bởi một **cổng đầu vào (input gate)**). Kết quả $c^{(t)}$ được gửi thẳng ra ngoài, không có bất kỳ biến đổi nào nữa. Vì vậy, ở mỗi bước thời gian, một số ký ức bị loại bỏ và một số ký ức được thêm vào. Hơn nữa, sau phép toán cộng, trạng thái dài hạn được sao chép và truyền qua hàm tanh, và sau đó kết quả được lọc bởi **cổng đầu ra (output gate)**. Điều này tạo ra trạng thái ngắn hạn $h^{(t)}$ (bằng với đầu ra của ô cho bước thời gian này, $y^{(t)}$). Bây giờ hãy xem các ký ức mới đến từ đâu và các cổng hoạt động như thế nào.

Đầu tiên, vector đầu vào hiện tại $x^{(t)}$ và trạng thái ngắn hạn trước đó $h^{(t-1)}$ được đưa vào bốn lớp kết nối đầy đủ khác nhau. Tất cả chúng đều phục vụ một mục đích khác nhau:

- **Lớp chính** là lớp xuất ra $g^{(t)}$. Nó có vai trò thông thường là phân tích các đầu vào hiện tại $x^{(t)}$ và trạng thái (ngắn hạn) trước đó $h^{(t-1)}$. Trong một ô cơ bản, không có gì khác ngoài lớp này, và đầu ra của nó đi thẳng đến $y^{(t)}$ và $h^{(t)}$. Nhưng trong một ô LSTM, đầu ra của lớp này không đi thẳng ra ngoài; thay vào đó, các phần quan trọng nhất của nó được lưu trữ trong trạng thái dài hạn (và phần còn lại bị loại bỏ).
- **Ba lớp khác** là các **bộ điều khiển cổng (gate controllers)**. Vì chúng sử dụng hàm kích hoạt logistic, các đầu ra nằm trong khoảng từ 0 đến 1. Như bạn có thể thấy, các

đầu ra của bộ điều khiển cổng được đưa vào các phép nhân từng phần tử: nếu chúng xuất ra 0, chúng đóng cổng, và nếu chúng xuất ra 1, chúng mở cổng. Cụ thể:

- **Cổng quên** (được điều khiển bởi $f^{(t)}$) kiểm soát phần nào của trạng thái dài hạn nên bị xóa.
- **Cổng đầu vào** (được điều khiển bởi $i^{(t)}$) kiểm soát phần nào của $g^{(t)}$ nên được thêm vào trạng thái dài hạn.
- Cuối cùng, **cổng đầu ra** (được điều khiển bởi $o^{(t)}$) kiểm soát phần nào của trạng thái dài hạn nên được đọc và xuất ra ở bước thời gian này, cả đến $h^{(t)}$ và đến $y^{(t)}$.

Tóm lại, một ô LSTM có thể học cách nhận dạng một đầu vào quan trọng (đó là vai trò của cổng đầu vào), lưu trữ nó trong trạng thái dài hạn, bảo tồn nó miễn là nó cần thiết (đó là vai trò của cổng quên), và trích xuất nó bất cứ khi nào cần. Điều này giải thích tại sao các ô này đã thành công đáng kinh ngạc trong việc nắm bắt các mẫu dài hạn trong chuỗi thời gian, văn bản dài, bản ghi âm và hơn thế nữa.

Phương trình 15-4 tóm tắt cách tính trạng thái dài hạn của ô, trạng thái ngắn hạn của nó và đầu ra của nó ở mỗi bước thời gian cho một thể hiện duy nhất (các phương trình cho toàn bộ một mini-batch rất giống nhau).

Phương trình 15-4. Các phép tính của LSTM

Công thức 15-4: Tính toán LSTM

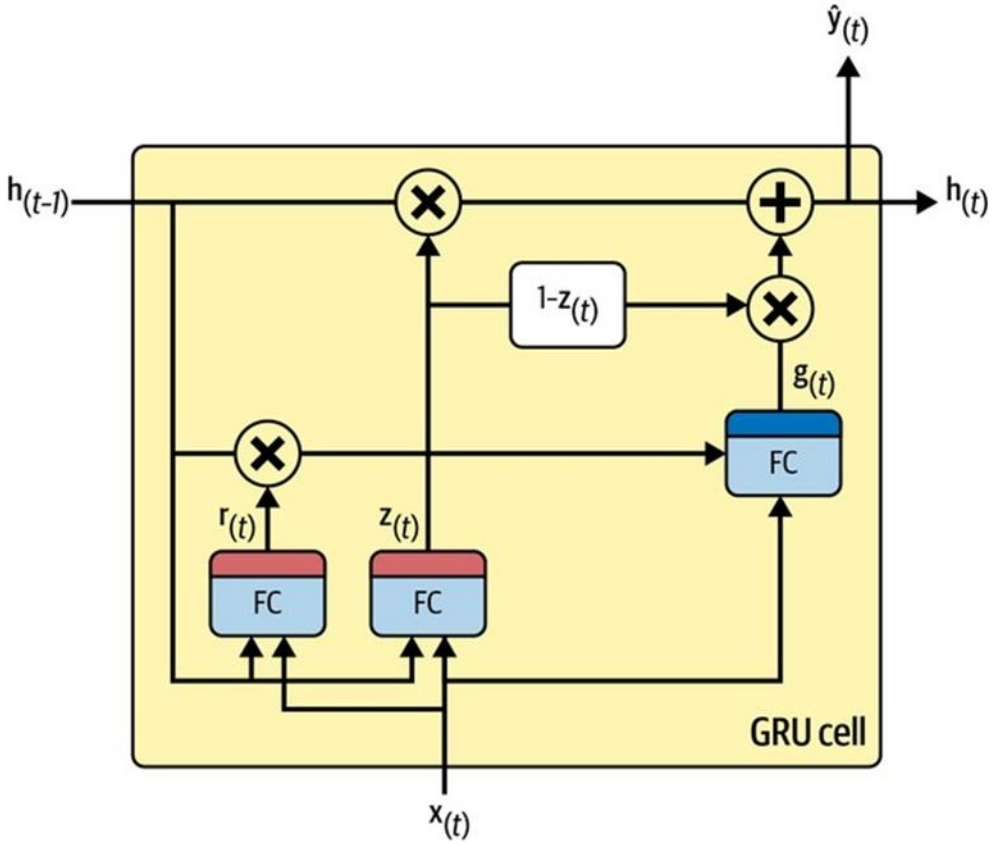
- $i_{(t)} = \sigma\left(W_{xi}x_{(t)} + W_{hi}h_{(t-1)} + b_i\right)$
- $f_{(t)} = \sigma\left(W_{xf}x_{(t)} + W_{hf}h_{(t-1)} + b_f\right)$
- $o_{(t)} = \sigma\left(W_{xo}x_{(t)} + W_{ho}h_{(t-1)} + b_o\right)$
- $g_{(t)} = \tanh\left(W_{xg}x_{(t)} + W_{hg}h_{(t-1)} + b_g\right)$
- $c_{(t)} = f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)}$
- $y_{(t)} = h_{(t)} = o_{(t)} \otimes \tanh\left(c_{(t)}\right)$

Trong các phương trình này:

- W_{xi} , W_{xf} , W_{xo} và W_{xg} là các ma trận trọng số của bốn lớp kết nối đến vector đầu vào $x_{(t)}$.
- W_{hi} , W_{hf} , W_{ho} và W_{hg} là các ma trận trọng số của bốn lớp kết nối đến trạng thái ngắn hạn trước đó $h_{(t-1)}$.
- b_i , b_f , b_o và b_g là các số hạng độ lệch cho mỗi trong bốn lớp. TensorFlow khởi tạo b_f là một vector toàn giá trị 1 thay vì 0, điều này giúp ngăn việc quên đi tất cả mọi thứ ngay từ đầu quá trình huấn luyện.

Có một vài biến thể của tế bào LSTM. Một biến thể đặc biệt phổ biến là tế bào GRU, chúng ta sẽ xem xét nó ngay bây giờ.

Các ô GRU Ô đơn vị hồi quy cổng (GRU) (xem Hình 15-13) được Kyunghyun Cho et al. đề xuất trong một bài báo năm 2014 cũng giới thiệu mạng mã hóa-giải mã mà chúng ta đã thảo luận trước đó.



Hình 15-13. Ô GRU

Ô GRU là một phiên bản đơn giản hóa của ô LSTM, và nó dường như hoạt động tốt như nhau (điều này giải thích sự phổ biến ngày càng tăng của nó). Đây là những điểm đơn giản hóa chính:

- Cả hai vector trạng thái được hợp nhất thành một vector duy nhất $h^{(t)}$.
- Một bộ điều khiển cổng duy nhất $z^{(t)}$ kiểm soát cả cổng quên và cổng đầu vào. Nếu bộ điều khiển cổng xuất ra 1, cổng quên mở (= 1) và cổng đầu vào đóng (1 – 1 = 0). Nếu nó xuất ra 0, điều ngược lại xảy ra. Nói cách khác, bất cứ khi nào một bộ nhớ phải được lưu trữ, vị trí mà nó sẽ được lưu trữ sẽ bị xóa trước. Đây thực sự là một biến thể thường xuyên của ô LSTM.
- Không có cổng đầu ra; toàn bộ vector trạng thái được xuất ra ở mỗi bước thời gian. Tuy nhiên, có một bộ điều khiển cổng mới $r^{(t)}$ kiểm soát phần nào của trạng thái trước đó sẽ được hiển thị cho lớp chính ($g^{(t)}$).

Phương trình 15-5 tóm tắt cách tính trạng thái của ô ở mỗi bước thời gian cho một thể hiện duy nhất.

Công thức 15-5: Tính toán GRU

- $z_{(t)} = \sigma\left(W_{xz}x_{(t)} + W_{hz}h_{(t-1)} + b_z\right)$
- $r_{(t)} = \sigma\left(W_{xr}x_{(t)} + W_{hr}h_{(t-1)} + b_r\right)$
- $g_{(t)} = \tanh\left(W_{xg}x_{(t)} + W_{hg}\left(r_{(t)} \otimes h_{(t-1)}\right) + b_g\right)$
- $h_{(t)} = z_{(t)} \otimes h_{(t-1)} + \left(1 - z_{(t)}\right) \otimes g_{(t)}$

Keras cung cấp một lớp `tf.keras.layers.GRU`; việc sử dụng nó chỉ đơn giản là thay thế `SimpleRNN` hoặc `LSTM` bằng `GRU`. Nó cũng cung cấp một `tf.keras.layers.GRUCell` trong trường hợp bạn muốn tạo một tế bào tùy chỉnh dựa trên tế bào GRU.

Các tế bào LSTM và GRU là một trong những lý do chính đằng sau sự thành công của các mạng nơ-ron hồi quy (RNN). Tuy nhiên, mặc dù chúng có thể xử lý các chuỗi dài hơn nhiều so với các RNN đơn giản, chúng vẫn có bộ nhớ ngắn hạn khá hạn chế và gặp khó khăn trong việc học các mẫu dài trong chuỗi hơn 100 bước thời gian, chẳng hạn như mẫu âm thanh, chuỗi thời gian dài hoặc câu dài. Một cách để giải quyết vấn đề này là rút ngắn chuỗi đầu vào; ví dụ: sử dụng các lớp tích chập 1D.

Sử dụng các lớp tích chập 1D để xử lý chuỗi

Trong Chương 14, chúng ta đã thấy rằng một lớp tích chập 2D hoạt động bằng cách trượt một số hạt nhân (hoặc bộ lọc) khá nhỏ qua một hình ảnh, tạo ra nhiều bản đồ đặc trưng 2D (một cho mỗi hạt nhân). Tương tự, một **lớp tích chập 1D** trượt một số hạt nhân qua một chuỗi, tạo ra một bản đồ đặc trưng 1D cho mỗi hạt nhân. Mỗi hạt nhân sẽ học cách phát hiện một mẫu tuần tự rất ngắn (không dài hơn kích thước hạt nhân). Nếu bạn sử dụng 10 hạt nhân, thì đầu ra của lớp sẽ bao gồm 10 chuỗi 1D (tất cả có cùng độ dài), hoặc tương đương bạn có thể xem đầu ra này là một chuỗi 10D duy nhất. Điều này có nghĩa là bạn có thể xây dựng một mạng thần kinh bao gồm một sự pha trộn giữa các lớp hồi quy và các lớp tích chập 1D (hoặc thậm chí các lớp gộp 1D). Nếu bạn sử dụng một lớp tích chập 1D với bước nhảy là 1 và padding “same”, thì chuỗi đầu ra sẽ có cùng độ dài với chuỗi đầu vào. Nhưng nếu bạn sử dụng padding “valid” hoặc bước nhảy lớn hơn 1, thì chuỗi đầu ra sẽ ngắn hơn chuỗi đầu vào, vì vậy hãy đảm bảo bạn điều chỉnh các mục tiêu cho phù hợp.

Ví dụ, mô hình sau đây giống như trước đó, ngoại trừ nó bắt đầu bằng một lớp tích chập 1D làm giảm mẫu chuỗi đầu vào theo hệ số 2, sử dụng bước nhảy là 2. Kích thước hạt nhân lớn hơn bước nhảy, vì vậy tất cả các đầu vào sẽ được sử dụng để tính toán đầu ra của lớp, và do đó mô hình có thể học cách bảo toàn thông tin hữu ích, chỉ loại bỏ các chi tiết không quan trọng. Bằng cách rút ngắn các chuỗi, lớp tích chập có thể giúp các lớp GRU phát hiện các mẫu dài hơn, vì vậy chúng ta có thể tăng gấp đôi độ dài chuỗi đầu vào lên 112 ngày. Lưu ý rằng chúng ta cũng phải cắt bỏ ba bước thời gian đầu tiên trong các mục tiêu: thực tế, kích thước hạt nhân là 4, vì vậy đầu ra đầu tiên của lớp tích chập sẽ dựa trên các bước thời gian đầu vào 0 đến 3, và các dự báo đầu tiên sẽ cho các bước thời gian 4 đến 17 (thay vì các bước thời gian 1 đến 14). Hơn nữa, chúng ta phải giảm mẫu các mục tiêu theo hệ số 2, do bước nhảy:

```
conv_rnn_model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=32, kernel_size=4, strides=2,
                           activation="relu", input_shape=[None, 5]),
    tf.keras.layers.GRU(32, return_sequences=True),
    tf.keras.layers.Dense(14)
])

# Các hàm và dữ liệu cần thiết đã được định nghĩa ở trên (mulvar_train,
to_seq2seq_dataset)
longer_train = to_seq2seq_dataset(mulvar_train, seq_length=112,
                                  shuffle=True, seed=42)
longer_valid = to_seq2seq_dataset(mulvar_valid, seq_length=112)

# Điều chỉnh mục tiêu do Conv1D
# Các giá trị mục tiêu Y đã được tạo với ahead=14.
# Y[:, 3::2] là cần thiết vì:
# 1. kernel_size=4, strides=2 nghĩa là đầu ra đầu tiên của Conv1D tương ứng
với input [0,1,2,3].
#    Vậy, dự đoán cho Y_t+1, Y_t+2, ..., Y_t+14 sẽ cần bắt đầu từ
input_time_step 0.
# 2. Sau Conv1D, mỗi output của Conv1D tương ứng với 2 time steps trong input
gốc.
#    Output của Dense layer sau GRU sẽ dự đoán 14 bước.
#    Y_seq2seq_ds có dạng (X_window, Y_window_sequence).
#    Y_window_sequence có shape (batch_size, seq_length, ahead).
#    Khi seq_length=112, và ahead=14, Y_window_sequence có shape (batch_size,
112, 14).
#    Stride=2 nghĩa là ta chỉ cần 112/2 = 56 outputs từ GRU layer.
#    Mỗi output của GRU layer dự đoán 14 bước.
#    Điều chỉnh Y[:, 3::2] là để căn chỉnh mục tiêu với các dự đoán sau khi
Conv1D giảm mẫu.
downsampled_train = longer_train.map(lambda X, Y: (X, Y[:, 3::2]))
downsampled_valid = longer_valid.map(lambda X, Y: (X, Y[:, 3::2]))

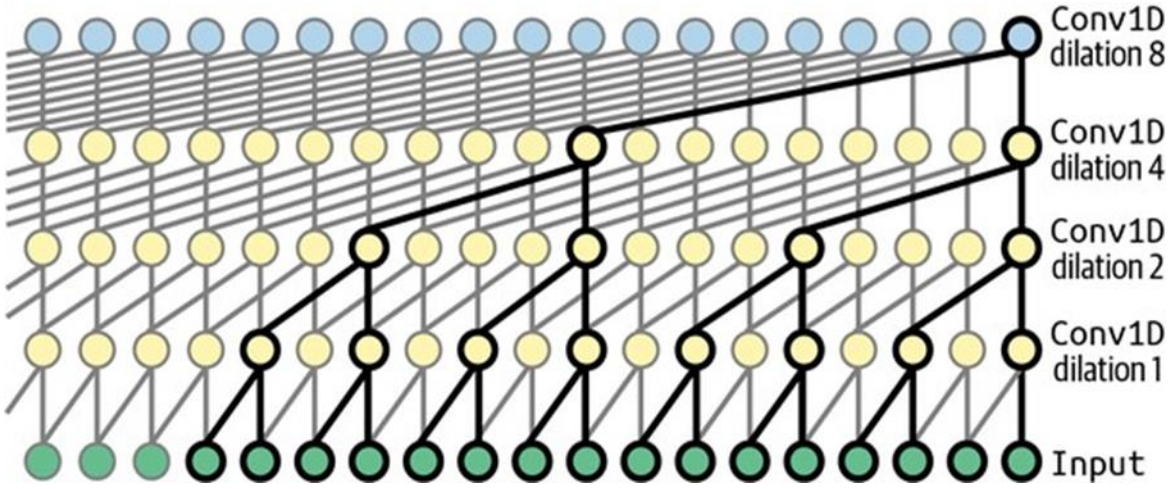
# Biên dịch và huấn luyện mô hình
conv_rnn_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])
history_conv_rnn = conv_rnn_model.fit(downsampled_train,
validation_data=downsampled_valid, epochs=500, callbacks=[early_stopping_cb])
```

Nếu bạn huấn luyện và đánh giá mô hình này, bạn sẽ thấy rằng nó vượt trội hơn mô hình trước đó (với một biên độ nhỏ). Trên thực tế, có thể chỉ sử dụng các lớp tích chập 1D và loại bỏ hoàn toàn các lớp hồi quy!

WaveNet

Trong một bài báo năm 2016, Aaron van den Oord và các nhà nghiên cứu DeepMind khác đã giới thiệu một kiến trúc mới lạ có tên **WaveNet**. Họ chồng các lớp tích chập 1D lên nhau, tăng gấp đôi tỷ lệ giãn nở (khoảng cách giữa các đầu vào của mỗi nơ-ron) ở mỗi lớp:

lớp tích chập đầu tiên chỉ nhìn thấy hai bước thời gian tại một thời điểm, trong khi lớp tiếp theo nhìn thấy bốn bước thời gian (trường thụ cảm của nó dài bốn bước thời gian), lớp tiếp theo nhìn thấy tám bước thời gian, v.v. (xem Hình 15-14). Bằng cách này, các lớp thấp hơn học các mẫu ngắn hạn, trong khi các lớp cao hơn học các mẫu dài hạn. Nhờ tỷ lệ giãn nở tăng gấp đôi, mạng có thể xử lý các chuỗi cực kỳ lớn rất hiệu quả.



Hình 15-14. Kiến trúc WaveNet

Các tác giả của bài báo thực sự đã chồng 10 lớp tích chập với tỷ lệ giãn nở là 1, 2, 4, 8, ..., 256, 512, sau đó họ chồng một nhóm 10 lớp giống hệt khác (cũng với tỷ lệ giãn nở 1, 2, 4, 8, ..., 256, 512), sau đó lại một nhóm 10 lớp giống hệt khác. Họ giải thích kiến trúc này bằng cách chỉ ra rằng một chồng 10 lớp tích chập với các tỷ lệ giãn nở này sẽ hoạt động như một lớp tích chập siêu hiệu quả với kích thước hạt nhân 1.024 (ngoại trừ nhanh hơn nhiều, mạnh hơn và sử dụng ít tham số hơn đáng kể). Họ cũng đệm đầu vào các chuỗi bằng một số lượng số không bằng với tỷ lệ giãn nở trước mỗi lớp, để giữ cùng độ dài chuỗi trong toàn bộ mạng.

Đây là cách triển khai một WaveNet đơn giản hóa để giải quyết cùng các chuỗi như trước đây: Với WaveNet, chúng ta sẽ cần xây dựng một mô hình phức tạp hơn một chút bằng cách sử dụng API chức năng của Keras (hoặc kế thừa từ `tf.keras.Model`). Cấu trúc cơ bản sẽ bao gồm các lớp `Conv1D` xếp chồng lên nhau với tỷ lệ giãn nở tăng dần. Để giữ cùng độ dài chuỗi, chúng ta sẽ sử dụng `padding="causal"` (đảm bảo đầu ra tại thời điểm t chỉ phụ thuộc vào đầu vào tại thời điểm t và các thời điểm trước đó, phù hợp với chuỗi thời gian).

Dưới đây là cách triển khai một WaveNet đơn giản hóa trong Keras để giải quyết nhiệm vụ dự báo chuỗi:

```
import tensorflow as tf
import pandas as pd
import numpy as np

# Giả định df, df_mulvar, mulvar_train, mulvar_valid, to_seq2seq_dataset,
# seq_length, ahead, opt, early_stopping_cb đã được định nghĩa ở các bước
```

```
trước.

# Chuẩn bị dữ liệu cho WaveNet
# WaveNet thường yêu cầu một receptive field lớn, nên seq_length cần đủ lớn.
# Ví dụ, nếu chúng ta dùng 10 lớp với dilation_rate từ 1 đến 512, receptive
field là 1023
# (2^10 - 1). Chúng ta sẽ dùng seq_length = 2 * (2**num_dilation_layers - 1)
để đảm bảo
# đủ dữ liệu cho kernel_size = 2.
num_dilation_layers = 10 # Số lớp giãn nở
wavenet_seq_length = 2 * (2**num_dilation_layers - 1) + 1 # +1 cho target đầu
tiên, đảm bảo padding
wavenet_ahead = ahead # Vẫn dự báo 14 bước
wavenet_target_col = 1 # Cột 'rail'

wavenet_train_ds = to_seq2seq_dataset(mulvar_train.to_numpy(),
seq_length=wavenet_seq_length,
                                      ahead=wavenet_ahead,
target_col=wavenet_target_col,
                                      shuffle=True, seed=42)
wavenet_valid_ds = to_seq2seq_dataset(mulvar_valid.to_numpy(),
seq_length=wavenet_seq_length,
                                      ahead=wavenet_ahead,
target_col=wavenet_target_col)

# Xây dựng mô hình WaveNet
inputs = tf.keras.layers.Input(shape=[None, 5]) # Input shape [batch,
time_steps, features]
x = inputs

# Tương tự như trong bài báo gốc WaveNet, sử dụng gated activations (sigmoid
* tanh)
# Tuy nhiên, để đơn giản, chúng ta sẽ sử dụng ReLU và Dense cho các kết nối
còn lại.
# Mô hình dưới đây là một phiên bản đơn giản hóa, tập trung vào Dilated
Convolutions.

# Lớp Conv1D đầu tiên
x = tf.keras.layers.Conv1D(filters=32, kernel_size=2, padding="causal",
activation="relu")(x)

# Các lớp tích chập giãn nở
skip_connections = []
for i in range(num_dilation_layers):
    dilation_rate = 2**i
    conv_layer = tf.keras.layers.Conv1D(filters=32, kernel_size=2,
                                        padding="causal",
                                        dilation_rate=dilation_rate,
                                        activation="relu")
```

```
    x = conv_layer(x)
    skip_connections.append(x) # Thu thập skip connections

# Kết hợp các skip connections và thêm Dense layer
x = tf.keras.layers.Add()(skip_connections) # Tổng hợp các đầu ra từ các lớp
giãn nở
x = tf.keras.layers.Conv1D(filters=32, kernel_size=1, activation="relu")(x) #
Conv1D 1x1
outputs = tf.keras.layers.Dense(wavenet_ahead)(x[:, -1, :]) # Chỉ lấy đầu ra
cuối cùng cho dự báo

wavenet_model = tf.keras.Model(inputs=inputs, outputs=outputs)

# Biên dịch và huấn luyện mô hình
wavenet_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
metrics=["mae"])

# Chú ý: Việc huấn luyện WaveNet có thể tốn thời gian và tài nguyên hơn.
# Bạn có thể điều chỉnh số lượng epoch và patience của early_stopping_cb.
history_wavenet = wavenet_model.fit(wavenet_train_ds,
validation_data=wavenet_valid_ds, epochs=500, callbacks=[early_stopping_cb])
```

Trong triển khai này:

- **`tf.keras.layers.Input`**: Định nghĩa đầu vào của mô hình.
- **`Conv1D` với `padding="causal"`**: Đảm bảo rằng mỗi đầu ra tại bước thời gian `t` chỉ phụ thuộc vào các đầu vào tại hoặc trước `t`. Điều này rất quan trọng đối với các chuỗi thời gian để tránh "nhìn thấy" dữ liệu trong tương lai.
- **`dilation_rate`**: Tăng gấp đôi ở mỗi lớp (`2**i`) để mở rộng trường thụ cảm một cách hiệu quả, cho phép mô hình nắm bắt các phụ thuộc dài hạn với ít lớp hơn.
- **`skip_connections`**: Các đầu ra từ mỗi lớp giãn nở được thu thập và cộng lại (sử dụng `tf.keras.layers.Add()`). Điều này giúp cải thiện luồng gradient và cho phép mô hình truy cập trực tiếp các đặc trưng ở các cấp độ trừu tượng khác nhau.
- **`Conv1D` với `kernel_size=1`**: Được sử dụng sau các kết nối bỏ qua để tổng hợp các thông tin trước khi đưa vào lớp đầu ra cuối cùng.
- **`Dense(wavenet_ahead)`**: Lớp Dense cuối cùng để tạo ra dự báo cho `wavenet_ahead` (14) bước thời gian tiếp theo. `x[:, -1, :]` được sử dụng để chỉ lấy đầu ra của bước thời gian cuối cùng từ chuỗi đầu ra của Conv1D trước đó, biến nó thành mô hình chuỗi-sang-vector dự báo nhiều bước.

Mô hình này là một phiên bản đơn giản hóa của WaveNet gốc (ví dụ, nó không sử dụng "gated activations" sigmoid-tanh phức tạp hoặc các khối dư), nhưng nó thể hiện ý tưởng cốt lõi của việc sử dụng dilated convolutions để xử lý các chuỗi dài.

wavenet_model = tf.keras.Sequential()
wavenet_model.add(tf.keras.layers.Input(shape=[None, 5]))

for rate in (1, 2, 4, 8) * 2: wavenet_model.add(tf.keras.layers.Conv1D(filters=32, kernel_size=2, padding="causal", activation="relu", dilation_rate=rate)) wavenet_model.add(tf.keras.layers.Conv1D(filters=14, kernel_size=1))

Mô hình `Sequential` này bắt đầu với một lớp đầu vào được chỉ định rõ ràng—điều này đơn giản hơn là cố gắng chỉ đặt `input_shape` trên lớp đầu tiên. Sau đó, nó tiếp tục với một lớp tích chập 1D sử dụng padding "causal", giống như padding "same" ngoại trừ việc các số 0 chỉ được thêm vào đầu chuỗi đầu vào, thay vì ở cả hai bên. Điều này đảm bảo rằng lớp tích chập không "nhìn trộm" vào tương lai khi đưa ra dự đoán. Sau đó, chúng ta thêm các cặp lớp tương tự sử dụng tỷ lệ giãn nở tăng dần: 1, 2, 4, 8, và lặp lại 1, 2, 4, 8. Cuối cùng, chúng ta thêm lớp đầu ra: một lớp tích chập với 14 bộ lọc kích thước 1 và không có bất kỳ hàm kích hoạt nào. Như chúng ta đã thấy trước đó, một lớp tích chập như vậy tương đương với một lớp `Dense` với 14 đơn vị. Nhờ có padding causal, mỗi lớp tích chập xuất ra một chuỗi có cùng độ dài với chuỗi đầu vào của nó, vì vậy các mục tiêu chúng ta sử dụng trong quá trình huấn luyện có thể là toàn bộ các chuỗi dài 112 ngày: không cần cắt hoặc giảm mẫu chúng.

Các mô hình chúng ta đã thảo luận trong phần này mang lại hiệu suất tương tự cho nhiệm vụ dự báo lượng hành khách, nhưng chúng có thể thay đổi đáng kể tùy thuộc vào nhiệm vụ và lượng dữ liệu có sẵn. Trong bài báo WaveNet, các tác giả đã đạt được hiệu suất tiên tiến trên nhiều nhiệm vụ âm thanh khác nhau (do đó có tên kiến trúc), bao gồm các nhiệm vụ chuyển văn bản thành giọng nói, tạo ra giọng nói cực kỳ chân thực trên nhiều ngôn ngữ. Họ cũng sử dụng mô hình để tạo nhạc, từng mẫu âm thanh một. Thành tựu này càng ấn tượng hơn khi bạn nhận ra rằng một giây âm thanh có thể chứa hàng chục nghìn bước thời gian—ngay cả LSTM và GRU cũng không thể xử lý các chuỗi dài như vậy.

Với những điều đó, bây giờ bạn có thể giải quyết tất cả các loại chuỗi thời gian! Trong Chương 16, chúng ta sẽ tiếp tục khám phá RNN, và chúng ta sẽ xem chúng có thể giải quyết nhiều nhiệm vụ NLP khác nhau như thế nào.

Bài tập

1. Bạn có thể nghĩ ra một vài ứng dụng cho RNN chuỗi-sang-chuỗi không? Thế còn RNN chuỗi-sang-vector, và RNN vector-sang-chuỗi thì sao?
2. Đầu vào của một lớp RNN phải có bao nhiêu chiều? Mỗi chiều đại diện cho điều gì? Còn đầu ra của nó thì sao?
3. Nếu bạn muốn xây dựng một RNN sâu chuỗi-sang-chuỗi, những lớp RNN nào nên có `return_sequences=True`? Thế còn RNN chuỗi-sang-vector thì sao?
4. Giả sử bạn có một chuỗi thời gian đơn biến hàng ngày, và bạn muốn dự báo bảy ngày tiếp theo. Bạn nên sử dụng kiến trúc RNN nào?
5. Những khó khăn chính khi huấn luyện RNN là gì? Bạn có thể xử lý chúng như thế nào?
6. Bạn có thể phác họa kiến trúc của ô LSTM không?
7. Tại sao bạn lại muốn sử dụng các lớp tích chập 1D trong một RNN?
8. Bạn có thể sử dụng kiến trúc mạng thần kinh nào để phân loại video?

9. Huấn luyện một mô hình phân loại cho tập dữ liệu SketchRNN, có sẵn trong TensorFlow Datasets.
10. Tải tập dữ liệu Bach chorales và giải nén nó. Nó bao gồm 382 hợp xướng do Johann Sebastian Bach sáng tác. Mỗi hợp xướng dài từ 100 đến 640 bước thời gian, và mỗi bước thời gian chứa 4 số nguyên, trong đó mỗi số nguyên tương ứng với chỉ số của một nốt nhạc trên đàn piano (ngoại trừ giá trị 0, có nghĩa là không có nốt nhạc nào được chơi). Huấn luyện một mô hình—hồi quy, tích chập hoặc cả hai—có thể dự đoán bước thời gian tiếp theo (bốn nốt nhạc), dựa trên một chuỗi các bước thời gian từ một hợp xướng. Sau đó sử dụng mô hình này để tạo nhạc giống Bach, từng nốt một: bạn có thể làm điều này bằng cách cung cấp cho mô hình phần đầu của một hợp xướng và yêu cầu nó dự đoán bước thời gian tiếp theo, sau đó nối các bước thời gian này vào chuỗi đầu vào và yêu cầu mô hình nốt tiếp theo, v.v. Cũng hãy nhớ xem mô hình Coconet của Google, đã được sử dụng cho một hình vẽ nguệch ngoạc Google đẹp mắt về Bach.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay chương này, tại https://homl.info/colab3 .

1 Lưu ý rằng nhiều nhà nghiên cứu thích sử dụng hàm kích hoạt hyperbolic tangent (tanh) trong RNN hơn là hàm kích hoạt ReLU. Ví dụ, xem bài báo năm 2013 của Vu Pham et al. “Dropout Improves Recurrent Neural Networks for Handwriting Recognition”. RNN dựa trên ReLU cũng có thể thực hiện được, như được thể hiện trong bài báo năm 2015 của Quoc V. Le et al. “A Simple Way to Initialize Recurrent Networks of Rectified Linear Units”. 2 Nal Kalchbrenner và Phil Blunsom, “Recurrent Continuous Translation Models”, Kỷ yếu Hội nghị về Phương pháp thực nghiệm trong Xử lý ngôn ngữ tự nhiên năm 2013 (2013): 1700–1709. 3 Dữ liệu mới nhất từ Cơ quan Giao thông Chicago có sẵn tại Cổng dữ liệu Chicago. 4 Có những cách tiếp cận nguyên tắc hơn để chọn các siêu tham số tốt, dựa trên phân tích hàm tự tương quan (ACF) và hàm tự tương quan riêng phần (PACF), hoặc giảm thiểu các chỉ số AIC hoặc BIC (được giới thiệu trong Chương 9) để phạt các mô hình sử dụng quá nhiều tham số và giảm nguy cơ quá khớp dữ liệu, nhưng tìm kiếm lưới là một nơi tốt để bắt đầu. Để biết thêm chi tiết về cách tiếp cận ACF-PACF, hãy xem bài đăng rất hay này của Jason Brownlee. 5 Lưu ý rằng giai đoạn xác thực bắt đầu vào ngày 1 tháng 1 năm 2019, vì vậy dự đoán đầu tiên là cho ngày 26 tháng 2 năm 2019, tám tuần sau đó. Khi chúng ta đánh giá các mô hình đường cơ sở, chúng ta đã sử dụng các dự đoán bắt đầu vào ngày 1 tháng 3, nhưng điều này đủ gần. 6 Cứ thoải mái thử nghiệm với mô hình này. Ví dụ, bạn có thể thử dự báo cả lượng hành khách xe buýt và đường sắt trong 14 ngày tiếp theo. Bạn sẽ cần điều chỉnh các mục tiêu để bao gồm cả hai, và làm cho mô hình của bạn xuất ra 28 dự báo thay vì 14. 7 César Laurent et al., “Batch Normalized Recurrent Neural Networks”, Kỷ yếu Hội nghị quốc tế IEEE về Âm học, Lời nói và Xử lý tín hiệu (2016): 2657–2661. 8 Jimmy Lei Ba et al., “LayerNormalization”, arXiv preprint arXiv:1607.06450 (2016).

9 Sẽ đơn giản hơn nếu kế thừa từ `SimpleRNNCell` thay vì vậy để chúng ta không phải tạo một `SimpleRNNCell` nội bộ hoặc xử lý các thuộc tính `state_size` và `output_size`, nhưng mục tiêu ở đây là để cho thấy cách tạo một ô tùy chỉnh từ đầu. 10 Một nhân vật trong các bộ phim hoạt hình *Tìm kiếm Nemo* và *Tìm kiếm Dory* bị mất trí nhớ ngắn hạn. 11 Sepp

Hochreiter và Jürgen Schmidhuber, “Long Short-Term Memory”, *Neural Computation* 9, số 8 (1997): 1735–1780. 12 Haşim Sak et al., “Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition”, arXiv preprint arXiv:1402.1128 (2014). 13 Wojciech Zaremba et al., “Recurrent Neural Network Regularization”, arXiv preprint arXiv:1409.2329 (2014). 14 Kyunghyun Cho et al., “Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation”, Kỷ yếu Hội nghị về Phương pháp thực nghiệm trong Xử lý ngôn ngữ tự nhiên năm 2014 (2014): 1724–1734. 15 Xem Klaus Greff et al., “LSTM: A Search Space Odyssey”, *IEEE Transactions on Neural Networks and Learning Systems* 28, số 10 (2017): 2222–2232. Bài báo này dường như cho thấy tất cả các biến thể LSTM đều có hiệu suất gần như nhau. 16 Aaron van den Oord et al., “WaveNet: A Generative Model for Raw Audio”, arXiv preprint arXiv:1609.03499 (2016). 17 WaveNet hoàn chỉnh sử dụng thêm một vài thủ thuật, chẳng hạn như skip connections giống như trong ResNet, và các đơn vị kích hoạt có cổng tương tự như những gì được tìm thấy trong một ô GRU. Xem sổ tay chương này để biết thêm chi tiết.

CHƯƠNG 16. XỬ LÝ NGÔN NGỮ TỰ NHIÊN VỚI RNN VÀ CƠ CHẾ CHÚ Ý

Khi Alan Turing hình dung bài kiểm tra Turing nổi tiếng của mình vào năm 1950, ông đã đề xuất một cách để đánh giá khả năng của một cỗ máy trong việc sánh ngang với trí thông minh của con người. Ông có thể đã kiểm tra nhiều thứ, chẳng hạn như khả năng nhận diện mèo trong hình ảnh, chơi cờ vua, sáng tác nhạc hoặc thoát khỏi mê cung, nhưng điều thú vị là ông đã chọn một nhiệm vụ ngôn ngữ.

Cụ thể hơn, ông đã nghĩ ra một chatbot có khả năng đánh lừa người đối thoại nghĩ rằng đó là con người. Bài kiểm tra này có những điểm yếu của nó: một tập hợp các quy tắc được mã hóa cứng có thể đánh lừa những người cả tin hoặc ngây thơ (ví dụ: cỗ máy có thể đưa ra các câu trả lời chung chung đã được định nghĩa trước để phản hồi một số từ khóa, nó có thể giả vờ đang nói đùa hoặc say xỉn để vượt qua những câu trả lời kỳ lạ nhất của nó, hoặc nó có thể tránh những câu hỏi khó bằng cách trả lời chúng bằng chính câu hỏi của mình), và nhiều khía cạnh của trí thông minh con người hoàn toàn bị bỏ qua (ví dụ: khả năng diễn giải giao tiếp phi ngôn ngữ như biểu cảm khuôn mặt, hoặc học một công việc thủ công). Nhưng bài kiểm tra này đã làm nổi bật một thực tế rằng việc nắm vững ngôn ngữ được cho là khả năng nhận thức vĩ đại nhất của loài Homo sapiens.

Liệu chúng ta có thể xây dựng một cỗ máy có thể nắm vững ngôn ngữ viết và nói? Đây là mục tiêu cuối cùng của nghiên cứu NLP, nhưng nó hơi quá rộng, vì vậy trên thực tế, các nhà nghiên cứu tập trung vào các nhiệm vụ cụ thể hơn, chẳng hạn như phân loại văn bản, dịch thuật, tóm tắt, trả lời câu hỏi và nhiều hơn nữa. Một cách tiếp cận phổ biến cho các nhiệm vụ ngôn ngữ tự nhiên là sử dụng mạng nơ-ron hồi quy. Do đó, chúng ta sẽ tiếp tục khám phá RNN (đã được giới thiệu trong Chương 15), bắt đầu với một RNN ký tự, hay char-RNN, được huấn luyện để dự đoán ký tự tiếp theo trong một câu. Điều này sẽ cho phép chúng ta tạo ra một số văn bản gốc. Chúng ta sẽ sử dụng một RNN không trạng thái (học trên các phần văn bản ngẫu nhiên ở mỗi lần lặp, không có bất kỳ thông tin nào về phần còn lại của văn bản), sau đó chúng ta sẽ xây dựng một RNN có trạng thái (giữ lại trạng thái ẩn giữa các lần lặp huấn luyện và tiếp tục đọc từ nơi nó dừng lại, cho phép nó học các mẫu dài hơn). Tiếp theo, chúng ta sẽ xây dựng một RNN để thực hiện phân tích cảm xúc (ví dụ: đọc các bài đánh giá phim và trích xuất cảm nhận của người đánh giá về bộ phim), lần này xử lý các câu như chuỗi các từ, thay vì các ký tự. Sau đó, chúng ta sẽ trình bày cách RNN có thể được sử dụng để xây dựng kiến trúc mã hóa-giải mã có khả năng thực hiện dịch máy thần kinh (NMT), dịch tiếng Anh sang tiếng Tây Ban Nha.

Trong phần thứ hai của chương này, chúng ta sẽ khám phá các cơ chế chú ý. Đúng như tên gọi của chúng, đây là các thành phần mạng nơ-ron học cách chọn phần đầu vào mà phần còn lại của mô hình nên tập trung vào ở mỗi bước thời gian. Đầu tiên, chúng ta sẽ tăng hiệu suất của kiến trúc mã hóa-giải mã dựa trên RNN bằng cách sử dụng cơ chế chú ý. Tiếp theo, chúng ta sẽ bỏ qua RNN hoàn toàn và sử dụng một kiến trúc chỉ có cơ chế chú ý rất thành công, được gọi là transformer, để xây dựng một mô hình dịch thuật. Sau đó, chúng ta sẽ thảo luận về một số tiến bộ quan trọng nhất trong NLP trong vài năm gần đây, bao gồm các mô hình ngôn ngữ cực kỳ mạnh mẽ như GPT và BERT, cả hai đều dựa trên transformer.

Cuối cùng, tôi sẽ chỉ cho bạn cách bắt đầu với thư viện Transformers xuất sắc của Hugging Face.

Hãy bắt đầu với một mô hình đơn giản và thú vị có thể viết giống như Shakespeare (đại loại thế).

Tạo văn bản kiểu Shakespeare bằng cách sử dụng RNN ký tự

Trong một bài đăng blog nổi tiếng năm 2015 có tựa đề "Hiệu quả bất hợp lý của mạng nơ-ron hồi quy", Andrej Karpathy đã chỉ ra cách huấn luyện một RNN để dự đoán ký tự tiếp theo trong một câu. Char-RNN này sau đó có thể được sử dụng để tạo văn bản mới, từng ký tự một. Đây là một mẫu nhỏ văn bản được tạo bởi một mô hình char-RNN sau khi nó được huấn luyện trên tất cả các tác phẩm của Shakespeare:

PANDARUS: Ôi, tôi nghĩ anh ta sẽ đến gần và ngày đó Khi cơn mưa nhỏ sẽ được ban cho để không bao giờ được nuôi dưỡng, Và ai là một chuỗi và đối tượng cái chết của anh ta, Tôi sẽ không ngủ.

Không hẳn là một kiệt tác, nhưng điều ấn tượng là mô hình đã có thể học được từ ngữ, ngữ pháp, dấu câu đúng và hơn thế nữa, chỉ bằng cách học cách dự đoán ký tự tiếp theo trong một câu. Đây là ví dụ đầu tiên của chúng ta về một mô hình ngôn ngữ; các mô hình ngôn ngữ tương tự (nhưng mạnh mẽ hơn nhiều), được thảo luận sau trong chương này, là cốt lõi của NLP hiện đại. Trong phần còn lại của phần này, chúng ta sẽ xây dựng một char-RNN từng bước, bắt đầu bằng việc tạo tập dữ liệu.

Tạo tập dữ liệu huấn luyện

Đầu tiên, sử dụng hàm tiện ích `tf.keras.utils.get_file()` của Keras, hãy tải xuống tất cả các tác phẩm của Shakespeare. Dữ liệu được tải từ dự án char-rnn của Andrej Karpathy:

```
import tensorflow as tf

shakespeare_url = "https://homl.info/shakespeare" # URL viết tắt
filepath = tf.keras.utils.get_file("shakespeare.txt", shakespeare_url)
with open(filepath) as f:
    shakespeare_text = f.read()
```

Hãy in vài dòng đầu tiên:

```
>>> print(shakespeare_text[:80])
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.
```

Trông giống Shakespeare rồi!

Tiếp theo, chúng ta sẽ sử dụng một lớp `tf.keras.layers.TextVectorization` (đã được giới thiệu trong Chương 13) để mã hóa văn bản này. Chúng ta đặt `split="character"` để có mã hóa cấp ký tự thay vì mã hóa cấp từ mặc định, và chúng ta sử dụng `standardize="lower"` để chuyển đổi văn bản sang chữ thường (điều này sẽ đơn giản hóa nhiệm vụ):

```
text_vec_layer = tf.keras.layers.TextVectorization(split="character",
                                                   standardize="lower")
text_vec_layer.adapt([shakespeare_text])
encoded = text_vec_layer([shakespeare_text])[0]
```

Mỗi ký tự hiện được ánh xạ tới một số nguyên, bắt đầu từ 2. Lớp `TextVectorization` đã dành giá trị 0 cho các token đệm, và nó dành 1 cho các ký tự không xác định. Hiện tại chúng ta sẽ không cần cả hai token này, vì vậy hãy trừ 2 từ các ID ký tự và tính toán số lượng ký tự riêng biệt và tổng số ký tự:

```
encoded -= 2 # bỏ các token 0 (đệm) và 1 (không xác định), mà chúng ta sẽ
không sử dụng
n_tokens = text_vec_layer.vocabulary_size() - 2 # số ký tự riêng biệt = 39
dataset_size = len(encoded) # tổng số ký tự = 1,115,394
```

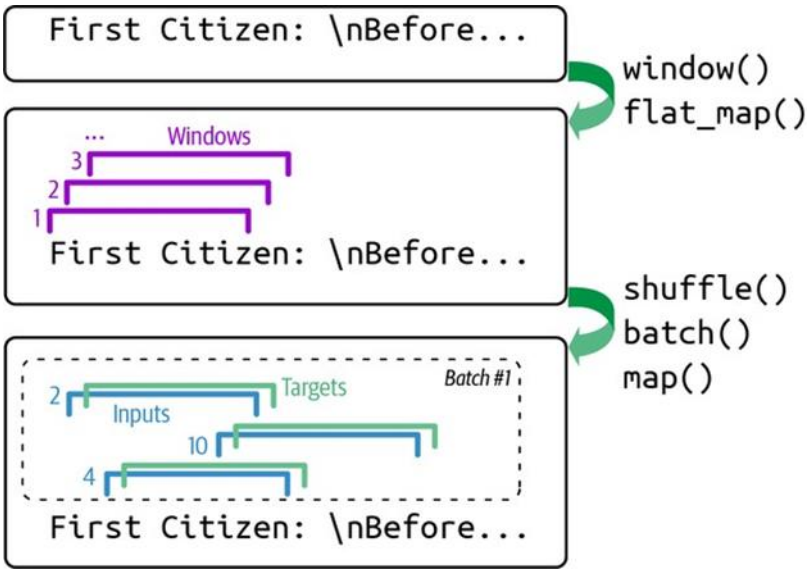
Tiếp theo, giống như chúng ta đã làm trong Chương 15, chúng ta có thể biến chuỗi rất dài này thành một tập dữ liệu các cửa sổ mà sau đó chúng ta có thể sử dụng để huấn luyện một RNN tuần tự. Các mục tiêu sẽ tương tự như đầu vào, nhưng được dịch chuyển một bước thời gian sang "tương lai". Ví dụ, một mẫu trong tập dữ liệu có thể là một chuỗi các ID ký tự đại diện cho văn bản "to be or not to b" (không có chữ "e" cuối cùng), và mục tiêu tương ứng — một chuỗi các ID ký tự đại diện cho văn bản "o be or not to be" (với chữ "e" cuối cùng, nhưng không có chữ "t" đầu). Hãy viết một hàm tiện ích nhỏ để chuyển đổi một chuỗi dài các ID ký tự thành một tập dữ liệu các cặp cửa sổ đầu vào/mục tiêu:

```
def to_dataset(sequence, length, shuffle=False, seed=None, batch_size=32):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=1, drop_remainder=True)
    ds = ds.flat_map(lambda window_ds: window_ds.batch(length + 1))
    if shuffle:
        ds = ds.shuffle(buffer_size=100_000, seed=seed)
    ds = ds.batch(batch_size)
    return ds.map(lambda window: (window[:, :-1], window[:, 1:])).prefetch(1)
```

Hàm này bắt đầu khá giống với hàm tiện ích tùy chỉnh `to_windows()` mà chúng ta đã tạo trong Chương 15:

- Nó nhận một chuỗi làm đầu vào (tức là văn bản đã được mã hóa), và tạo ra một tập dữ liệu chứa tất cả các cửa sổ có độ dài mong muốn.
- Nó tăng độ dài lên một, vì chúng ta cần ký tự tiếp theo cho mục tiêu.
- Sau đó, nó xáo trộn các cửa sổ (tùy chọn), gộp chúng lại, chia chúng thành các cặp đầu vào/đầu ra, và kích hoạt prefetching.

Hình 16-1 tóm tắt các bước chuẩn bị tập dữ liệu: nó hiển thị các cửa sổ có độ dài 11, và kích thước batch là 3. Chỉ số bắt đầu của mỗi cửa sổ được chỉ ra bên cạnh nó.



Hình 16-1. Chuẩn bị tập dữ liệu các cửa sổ đã xáo trộn

Bây giờ chúng ta đã sẵn sàng tạo tập huấn luyện, tập xác thực và tập kiểm tra. Chúng ta sẽ sử dụng khoảng 90% văn bản để huấn luyện, 5% để xác thực và 5% để kiểm tra:

```
length = 100
tf.random.set_seed(42)

train_set = to_dataset(encoded[:1_000_000], length=length, shuffle=True,
                       seed=42)
valid_set = to_dataset(encoded[1_000_000:1_060_000], length=length)
test_set = to_dataset(encoded[1_060_000:], length=length)
```

Thế là xong! Chuẩn bị tập dữ liệu là phần khó nhất. Bây giờ hãy tạo mô hình.

Xây dựng và huấn luyện mô hình Char-RNN

Vì tập dữ liệu của chúng ta khá lớn, và việc mô hình hóa ngôn ngữ là một nhiệm vụ khá khó khăn, chúng ta cần nhiều hơn một RNN đơn giản với vài nơ-ron hồi quy. Hãy xây dựng và huấn luyện một mô hình với một lớp GRU bao gồm 128 đơn vị (bạn có thể thử điều chỉnh số lượng lớp và đơn vị sau, nếu cần):

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16),
    tf.keras.layers.GRU(128, return_sequences=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])
model_ckpt = tf.keras.callbacks.ModelCheckpoint(
```

```
    "my_shakespeare_model", monitor="val_accuracy", save_best_only=True)
history = model.fit(train_set, validation_data=valid_set, epochs=10,
                    callbacks=[model_ckpt])
```

Hãy xem qua đoạn code này:

- Chúng ta sử dụng một lớp `Embedding` làm lớp đầu tiên, để mã hóa các ID ký tự (embedding đã được giới thiệu trong Chương 13). Số chiều đầu vào của lớp `Embedding` là số ID ký tự riêng biệt, và số chiều đầu ra là một siêu tham số bạn có thể điều chỉnh — chúng ta sẽ đặt nó là 16. Trong khi đầu vào của lớp `Embedding` sẽ là các tensor 2D có hình dạng `[batch size, window length]`, đầu ra của lớp `Embedding` sẽ là một tensor 3D có hình dạng `[batch size, window length, embedding size]`.
- Chúng ta sử dụng một lớp `Dense` cho lớp đầu ra: nó phải có 39 đơn vị (`n_tokens`) vì có 39 ký tự riêng biệt trong văn bản, và chúng ta muốn xuất ra một xác suất cho mỗi ký tự có thể có (ở mỗi bước thời gian). 39 xác suất đầu ra nên tổng cộng là 1 ở mỗi bước thời gian, vì vậy chúng ta áp dụng hàm kích hoạt softmax cho đầu ra của lớp `Dense`.
- Cuối cùng, chúng ta biên dịch mô hình này, sử dụng hàm mất mát `"sparse_categorical_crossentropy"` và bộ tối ưu hóa Nadam, và chúng ta huấn luyện mô hình trong vài epoch, sử dụng callback `ModelCheckpoint` để lưu mô hình tốt nhất (về độ chính xác xác thực) khi quá trình huấn luyện diễn ra.

Mô hình này không xử lý tiền xử lý văn bản, vì vậy hãy bọc nó trong một mô hình cuối cùng chứa lớp `tf.keras.layers.TextVectorization` làm lớp đầu tiên, cộng với một lớp `tf.keras.layers.Lambda` để trừ 2 từ các ID ký tự vì hiện tại chúng ta không sử dụng các token đệm và không xác định:

```
shakespeare_model = tf.keras.Sequential([
    text_vec_layer,
    tf.keras.layers.Lambda(lambda X: X - 2), # không có token <PAD> hoặc
<UNK>
    model
])
```

Và bây giờ hãy sử dụng nó để dự đoán ký tự tiếp theo trong một câu:

```
>>> y_proba = shakespeare_model.predict(["To be or not to b"])[0, -1]
>>> y_pred = tf.argmax(y_proba) # chọn ID ký tự có khả năng nhất
>>> text_vec_layer.get_vocabulary()[y_pred + 2]
'e'
```

Tuyệt vời, mô hình đã dự đoán đúng ký tự tiếp theo. Bây giờ hãy sử dụng mô hình này để giả vờ chúng ta là Shakespeare!

Tạo văn bản Shakespeare giả

Để tạo văn bản mới bằng cách sử dụng mô hình char-RNN, chúng ta có thể cung cấp cho nó một số văn bản, để mô hình dự đoán ký tự tiếp theo có khả năng nhất, thêm nó vào cuối văn bản, sau đó cung cấp văn bản mở rộng cho mô hình để đoán ký tự tiếp theo, và cứ thế. Điều này được gọi là giải mã tham lam (greedy decoding). Nhưng trong thực tế, điều này thường dẫn đến việc các từ giống nhau bị lặp đi lặp lại. Thay vào đó, chúng ta có thể lấy mẫu ký tự tiếp theo một cách ngẫu nhiên, với xác suất bằng xác suất ước tính, bằng cách sử dụng hàm `tf.random.categorical()` của TensorFlow. Điều này sẽ tạo ra văn bản đa dạng và thú vị hơn. Hàm `categorical()` lấy mẫu các chỉ số lớp ngẫu nhiên, cho các log xác suất lớp (logits). Ví dụ:

```
>>> log_probas = tf.math.log([[0.5, 0.4, 0.1]]) # xác suất = 50%, 40%, và 10%
>>> tf.random.set_seed(42)
>>> tf.random.categorical(log_probas, num_samples=8) # lấy 8 mẫu
<tf.Tensor: shape=(1, 8), dtype=int64, numpy=array([[0, 1, 0, 2, 1, 0, 0,
1]])>
```

Để kiểm soát tốt hơn sự đa dạng của văn bản được tạo ra, chúng ta có thể chia các logits cho một số được gọi là nhiệt độ (temperature), mà chúng ta có thể điều chỉnh tùy ý. Nhiệt độ gần bằng 0 sẽ ưu tiên các ký tự có xác suất cao, trong khi nhiệt độ cao sẽ cho tất cả các ký tự một xác suất bằng nhau. Nhiệt độ thấp thường được ưu tiên khi tạo văn bản khá cứng nhắc và chính xác, chẳng hạn như các phương trình toán học, trong khi nhiệt độ cao hơn được ưu tiên khi tạo văn bản đa dạng và sáng tạo hơn.

Hàm trợ giúp tùy chỉnh `next_char()` tiếp theo sử dụng phương pháp này để chọn ký tự tiếp theo để thêm vào văn bản đầu vào:

```
def next_char(text, temperature=1):
    y_proba = shakespeare_model.predict([text])[0, -1:]
    rescaled_logits = tf.math.log(y_proba) / temperature
    char_id = tf.random.categorical(rescaled_logits, num_samples=1)[0, 0]
    return text_vec_layer.get_vocabulary()[char_id + 2]
```

Tiếp theo, chúng ta có thể viết một hàm trợ giúp nhỏ khác sẽ liên tục gọi `next_char()` để lấy ký tự tiếp theo và thêm nó vào văn bản đã cho:

```
def extend_text(text, n_chars=50, temperature=1):
    for _ in range(n_chars):
        text += next_char(text, temperature)
    return text
```

Bây giờ chúng ta đã sẵn sàng tạo văn bản! Hãy thử với các giá trị nhiệt độ khác nhau:

```
>>> tf.random.set_seed(42)

>>> print(extend_text("To be or not to be", temperature=0.01))
To be or not to be the duke
as it is a proper
```

```
strange death, and the

>>> print(extend_text("To
be or not to be", temperature=1)) To be or
not to behold?

second push:
gremio, lord all, a sistermen,

>>> print(extend_text("To be or not to be", temperature=100))
To be or not to bef
,mt'&o3fpadm!$ wh!nse?bws3est--vgerdjw?c-y-ewznq
```

Shakespeare dường như đang chịu đựng một đợt nắng nóng. Để tạo văn bản thuyết phục hơn, một kỹ thuật phổ biến là chỉ lấy mẫu từ k ký tự hàng đầu, hoặc chỉ từ tập hợp nhỏ nhất các ký tự hàng đầu có tổng xác suất vượt quá một ngưỡng nhất định (điều này được gọi là lấy mẫu hạt nhân). Ngoài ra, bạn có thể thử sử dụng tìm kiếm chùm (beam search), mà chúng ta sẽ thảo luận sau trong chương này, hoặc sử dụng nhiều lớp GRU hơn và nhiều nơ-ron hơn cho mỗi lớp, huấn luyện lâu hơn và thêm một số chuẩn hóa nếu cần. Cũng lưu ý rằng mô hình hiện không thể học các mẫu dài hơn độ dài `length`, tức là chỉ 100 ký tự. Bạn có thể thử làm cho cửa sổ này lớn hơn, nhưng nó cũng sẽ làm cho việc huấn luyện khó hơn, và ngay cả các ô LSTM và GRU cũng không thể xử lý các chuỗi rất dài. Một cách tiếp cận thay thế là sử dụng một RNN có trạng thái.

RNN có trạng thái

Cho đến nay, chúng ta chỉ sử dụng các RNN không trạng thái: ở mỗi lần lặp huấn luyện, mô hình bắt đầu với một trạng thái ẩn đầy các số 0, sau đó nó cập nhật trạng thái này ở mỗi bước thời gian, và sau bước thời gian cuối cùng, nó loại bỏ trạng thái đó vì không cần nữa. Điều gì sẽ xảy ra nếu chúng ta hướng dẫn RNN giữ lại trạng thái cuối cùng này sau khi xử lý một batch huấn luyện và sử dụng nó làm trạng thái ban đầu cho batch huấn luyện tiếp theo? Bằng cách này, mô hình có thể học các mẫu dài hạn mặc dù chỉ lan truyền ngược qua các chuỗi ngắn. Đây được gọi là RNN có trạng thái. Hãy xem cách xây dựng một mô hình như vậy.

Đầu tiên, lưu ý rằng một RNN có trạng thái chỉ có ý nghĩa nếu mỗi chuỗi đầu vào trong một batch bắt đầu chính xác từ nơi chuỗi tương ứng trong batch trước đó dừng lại. Vì vậy, điều đầu tiên chúng ta cần làm để xây dựng một RNN có trạng thái là sử dụng các chuỗi đầu vào tuần tự và không chồng chéo (thay vì các chuỗi bị xáo trộn và chồng chéo mà chúng ta đã sử dụng để huấn luyện các RNN không trạng thái). Khi tạo `tf.data.Dataset`, chúng ta phải sử dụng `shift=length` (thay vì `shift=1`) khi gọi phương thức `window()`. Hơn nữa, chúng ta không được gọi phương thức `shuffle()`.

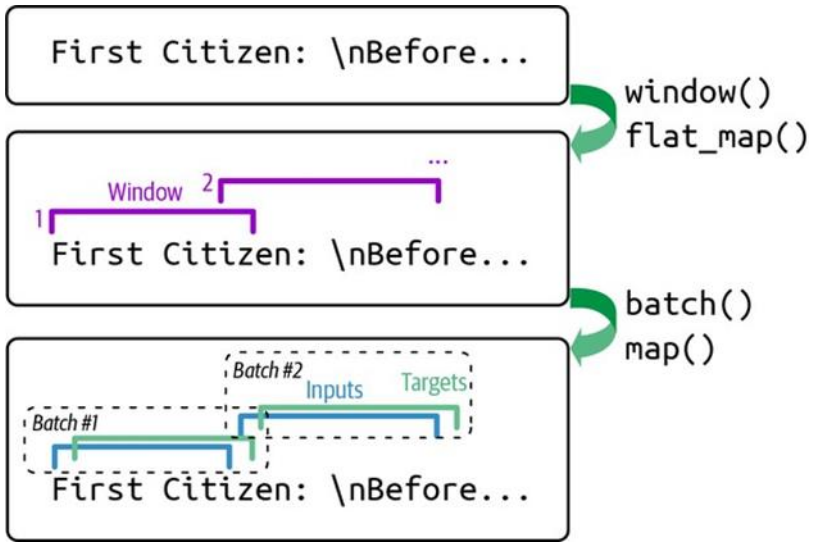
Rất tiếc, việc gộp batch khó hơn nhiều khi chuẩn bị tập dữ liệu cho một RNN có trạng thái so với RNN không trạng thái. Thật vậy, nếu chúng ta gọi `batch(32)`, thì 32 cửa sổ liên tiếp sẽ được đặt trong cùng một batch, và batch tiếp theo sẽ không tiếp tục mỗi cửa sổ này từ nơi nó dừng lại. Batch đầu tiên sẽ chứa các cửa sổ từ 1 đến 32 và batch thứ hai sẽ chứa các

cửa sổ từ 33 đến 64, vì vậy nếu bạn xem xét, ví dụ, cửa sổ đầu tiên của mỗi batch (tức là cửa sổ 1 và 33), bạn có thể thấy rằng chúng không liên tiếp. Giải pháp đơn giản nhất cho vấn đề này là chỉ sử dụng kích thước batch là 1. Hàm tiện ích tùy chỉnh `to_dataset_for_stateful_rnn()` sau đây sử dụng chiến lược này để chuẩn bị tập dữ liệu cho một RNN có trạng thái:

```
def to_dataset_for_stateful_rnn(sequence, length):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=length, drop_remainder=True)
    ds = ds.flat_map(lambda window: window.batch(length + 1)).batch(1)
    return ds.map(lambda window: (window[:, :-1], window[:, 1:])).prefetch(1)

stateful_train_set = to_dataset_for_stateful_rnn(encoded[:1_000_000], length)
stateful_valid_set =
to_dataset_for_stateful_rnn(encoded[1_000_000:1_060_000],
                                                    length)
stateful_test_set = to_dataset_for_stateful_rnn(encoded[1_060_000:], length)
```

Hình 16-2 tóm tắt các bước chính của hàm này.



Hình 16-2. Chuẩn bị tập dữ liệu các đoạn chuỗi liên tiếp cho một RNN có trạng thái

Việc gộp batch khó hơn, nhưng không phải là không thể. Ví dụ, chúng ta có thể chia văn bản của Shakespeare thành 32 văn bản có độ dài bằng nhau, tạo một tập dữ liệu các chuỗi đầu vào liên tiếp cho mỗi văn bản, và cuối cùng sử dụng `tf.data.Dataset.zip(datasets).map(lambda *windows: tf.stack(windows))` để tạo các batch liên tiếp phù hợp, trong đó chuỗi đầu vào thứ n trong một batch bắt đầu chính xác từ nơi chuỗi đầu vào thứ n kết thúc trong batch trước đó (xem sổ tay để biết toàn bộ mã).

Bây giờ, hãy tạo RNN có trạng thái. Chúng ta cần đặt đối số `stateful` thành `True` khi tạo mỗi lớp hồi quy, và vì RNN có trạng thái cần biết kích thước batch (vì nó sẽ giữ một trạng

thái cho mỗi chuỗi đầu vào trong batch). Do đó, chúng ta phải đặt đối số `batch_input_shape` trong lớp đầu tiên. Lưu ý rằng chúng ta có thể để chiều thứ hai không xác định, vì các chuỗi đầu vào có thể có bất kỳ độ dài nào:

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16,
                              batch_input_shape=[1, None]),
    tf.keras.layers.GRU(128, return_sequences=True, stateful=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])
```

Vào cuối mỗi epoch, chúng ta cần đặt lại các trạng thái trước khi quay lại đầu văn bản. Để làm điều này, chúng ta có thể sử dụng một callback Keras tùy chỉnh nhỏ:

```
class ResetStatesCallback(tf.keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs):
        self.model.reset_states()
```

Và bây giờ chúng ta có thể biên dịch mô hình và huấn luyện nó bằng cách sử dụng callback của chúng ta:

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])
history = model.fit(stateful_train_set, validation_data=stateful_valid_set,
                    epochs=10, callbacks=[ResetStatesCallback(), model_ckpt])
```

Điều thú vị là, mặc dù mô hình char-RNN chỉ được huấn luyện để dự đoán ký tự tiếp theo, nhiệm vụ tưởng chừng đơn giản này thực sự yêu cầu nó học một số nhiệm vụ cấp cao hơn. Ví dụ, để tìm ký tự tiếp theo sau "Great movie, I really", sẽ rất hữu ích nếu hiểu rằng câu đó là tích cực, vì vậy điều tiếp theo có nhiều khả năng là chữ "l" (cho "loved") hơn là "h" (cho "hated"). Trên thực tế, một bài báo năm 2017 của Alec Radford và các nhà nghiên cứu OpenAI khác mô tả cách các tác giả đã huấn luyện một mô hình giống char-RNN lớn trên một tập dữ liệu lớn, và thấy rằng một trong các nơ-ron hoạt động như một bộ phân loại phân tích cảm xúc xuất sắc: mặc dù mô hình được huấn luyện mà không có bất kỳ nhãn nào, nơ-ron cảm xúc — như họ gọi nó — đã đạt được hiệu suất tiên tiến trên các tiêu chuẩn phân tích cảm xúc. Điều này đã báo trước và thúc đẩy việc tiền huấn luyện không giám sát trong NLP.

Nhưng trước khi chúng ta khám phá tiền huấn luyện không giám sát, hãy chuyển sự chú ý của chúng ta sang các mô hình cấp từ và cách sử dụng chúng theo cách giám sát để phân tích cảm xúc. Trong quá trình này, bạn sẽ học cách xử lý các chuỗi có độ dài thay đổi bằng cách sử dụng mặt nạ (masking).

Phân tích cảm xúc

Tạo văn bản có thể thú vị và mang tính hướng dẫn, nhưng trong các dự án thực tế, một trong những ứng dụng phổ biến nhất của NLP là phân loại văn bản — đặc biệt là phân tích cảm xúc. Nếu phân loại hình ảnh trên tập dữ liệu MNIST là "Hello world!" của thị giác máy

tính, thì phân tích cảm xúc trên tập dữ liệu đánh giá IMDb là “Hello world!” của xử lý ngôn ngữ tự nhiên. Tập dữ liệu IMDb bao gồm 50.000 đánh giá phim bằng tiếng Anh (25.000 để huấn luyện, 25.000 để kiểm tra) được trích xuất từ Internet Movie Database nổi tiếng, cùng với một mục tiêu nhị phân đơn giản cho mỗi đánh giá cho biết nó là tiêu cực (0) hay tích cực (1). Giống như MNIST, tập dữ liệu đánh giá IMDb phổ biến vì những lý do chính đáng: nó đủ đơn giản để được giải quyết trên một máy tính xách tay trong một khoảng thời gian hợp lý, nhưng đủ thách thức để trở nên thú vị và bổ ích.

Hãy tải tập dữ liệu IMDb bằng thư viện TensorFlow Datasets (đã được giới thiệu trong Chương 13). Chúng ta sẽ sử dụng 90% đầu tiên của tập huấn luyện để huấn luyện, và 10% còn lại để xác thực:

```
import tensorflow_datasets as tfds

raw_train_set, raw_valid_set, raw_test_set = tfds.load(
    name="imdb_reviews",
    split=["train[:90%]", "train[90%:]", "test"],
    as_supervised=True
)
tf.random.set_seed(42)
train_set = raw_train_set.shuffle(5000, seed=42).batch(32).prefetch(1)
valid_set = raw_valid_set.batch(32).prefetch(1)
test_set = raw_test_set.batch(32).prefetch(1)
```

Hãy kiểm tra một vài đánh giá:

```
>>> for review, label in raw_train_set.take(4):
...         print(review.numpy().decode("utf-8"))
...         print("Label:", label.numpy())
...
This was an absolutely
terrible movie. Don't be lured in by Christopher [...]
Label: 0
I have been known to
fall asleep during films, but this is usually due to [...]
Label: 0
Mann photographs the
Alberta Rocky Mountains in a superb fashion, and [...]
Label: 0
This is the kind of
film for a snowy Sunday afternoon when the rest of the [...]
Label: 1
```

Một số đánh giá dễ phân loại. Ví dụ, đánh giá đầu tiên bao gồm các từ “terrible movie” ngay trong câu đầu tiên. Nhưng trong nhiều trường hợp, mọi thứ không đơn giản như vậy. Ví dụ, đánh giá thứ ba bắt đầu một cách tích cực, mặc dù cuối cùng nó là một đánh giá tiêu cực (nhãn 0).

Để xây dựng một mô hình cho nhiệm vụ này, chúng ta cần tiền xử lý văn bản, nhưng lần này chúng ta sẽ cắt nó thành các từ thay vì các ký tự. Để làm điều này, chúng ta có thể sử dụng lại lớp `tf.keras.layers.TextVectorization`. Lưu ý rằng nó sử dụng dấu cách để xác định ranh giới từ, điều này sẽ không hoạt động tốt trong một số ngôn ngữ. Ví dụ, chữ viết tiếng Trung không sử dụng dấu cách giữa các từ, tiếng Việt sử dụng dấu cách ngay cả trong các từ, và tiếng Đức thường gắn nhiều từ lại với nhau, không có dấu cách. Ngay cả trong tiếng Anh, dấu cách không phải lúc nào cũng là cách tốt nhất để phân tách văn bản: hãy nghĩ đến "San Francisco" hoặc "#ILoveDeepLearning".

May mắn thay, có những giải pháp để giải quyết các vấn đề này. Trong một bài báo năm 2016, Rico Sennrich et al. từ Đại học Edinburgh đã khám phá một số phương pháp để phân tách và ghép lại văn bản ở cấp độ subword. Bằng cách này, ngay cả khi mô hình của bạn gặp một từ hiếm mà nó chưa từng thấy trước đây, nó vẫn có thể đoán được ý nghĩa của nó. Ví dụ, ngay cả khi mô hình chưa bao giờ thấy từ "smartest" trong quá trình huấn luyện, nếu nó học được từ "smart" và nó cũng học được rằng hậu tố "est" có nghĩa là "nhất", nó có thể suy ra ý nghĩa của "smartest". Một trong những kỹ thuật mà các tác giả đánh giá là mã hóa cặp byte (BPE). BPE hoạt động bằng cách chia toàn bộ tập huấn luyện thành các ký tự riêng lẻ (bao gồm cả dấu cách), sau đó liên tục hợp nhất các cặp liền kề thường xuyên nhất cho đến khi từ vựng đạt đến kích thước mong muốn.

Một bài báo năm 2018 của Taku Kudo tại Google đã cải thiện hơn nữa việc mã hóa subword, thường loại bỏ nhu cầu tiền xử lý cụ thể theo ngôn ngữ trước khi mã hóa. Hơn nữa, bài báo đã đề xuất một kỹ thuật chuẩn hóa mới được gọi là chuẩn hóa subword, giúp cải thiện độ chính xác và tính mạnh mẽ bằng cách đưa một số ngẫu nhiên vào việc mã hóa trong quá trình huấn luyện: ví dụ, "New England" có thể được mã hóa thành "New" + "England", hoặc "New" + "Eng" + "land", hoặc đơn giản là "New England" (chỉ một token). Dự án SentencePiece của Google cung cấp một triển khai mã nguồn mở, được mô tả trong một bài báo của Taku Kudo và John Richardson.

Thư viện TensorFlow Text cũng triển khai các chiến lược mã hóa khác nhau, bao gồm WordPiece (một biến thể của BPE), và cuối cùng nhưng không kém phần quan trọng, thư viện Tokenizers của Hugging Face triển khai một loạt các bộ mã hóa cực kỳ nhanh.

Tuy nhiên, đối với nhiệm vụ IMDb trong tiếng Anh, việc sử dụng dấu cách cho ranh giới từ sẽ là đủ tốt. Vì vậy, hãy tiếp tục tạo một lớp `TextVectorization` và điều chỉnh nó với tập huấn luyện. Chúng ta sẽ giới hạn từ vựng ở 1.000 token, bao gồm 998 từ thường xuyên nhất cộng với một token đệm và một token cho các từ không xác định, vì rất khó có khả năng các từ rất hiếm sẽ quan trọng cho nhiệm vụ này, và việc giới hạn kích thước từ vựng sẽ giảm số lượng tham số mà mô hình cần học:

```
vocab_size = 1000

text_vec_layer = tf.keras.layers.TextVectorization(max_tokens=vocab_size)
text_vec_layer.adapt(train_set.map(lambda reviews, labels: reviews))
```

Cuối cùng, chúng ta có thể tạo mô hình và huấn luyện nó:

```
embed_size = 128
tf.random.set_seed(42)
model = tf.keras.Sequential([
    text_vec_layer,
    tf.keras.layers.Embedding(vocab_size, embed_size),
    tf.keras.layers.GRU(128),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(loss="binary_crossentropy",
              optimizer="nadam", metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=2)
```

Lớp đầu tiên là lớp `TextVectorization` mà chúng ta vừa chuẩn bị, tiếp theo là một lớp `Embedding` sẽ chuyển đổi các ID từ thành các embedding. Ma trận embedding cần có một hàng cho mỗi token trong từ vựng (`vocab_size`) và một cột cho mỗi chiều embedding (ví dụ này sử dụng 128 chiều, nhưng đây là một siêu tham số bạn có thể điều chỉnh). Tiếp theo, chúng ta sử dụng một lớp GRU và một lớp `Dense` với một nơ-ron duy nhất và hàm kích hoạt sigmoid, vì đây là một nhiệm vụ phân loại nhị phân: đầu ra của mô hình sẽ là xác suất ước tính rằng đánh giá thể hiện một cảm xúc tích cực về bộ phim. Sau đó, chúng ta biên dịch mô hình, và chúng ta fit nó trên tập dữ liệu đã chuẩn bị trước đó trong vài epoch (hoặc bạn có thể huấn luyện lâu hơn để có kết quả tốt hơn).

Đáng buồn thay, nếu bạn chạy đoạn mã này, bạn thường sẽ thấy rằng mô hình không học được gì cả: độ chính xác vẫn gần 50%, không tốt hơn cơ hội ngẫu nhiên. Tại sao lại như vậy? Các đánh giá có độ dài khác nhau, vì vậy khi lớp `TextVectorization` chuyển đổi chúng thành chuỗi các ID token, nó sẽ đệm các chuỗi ngắn hơn bằng cách sử dụng token đệm (với ID 0) để làm cho chúng dài bằng chuỗi dài nhất trong batch. Kết quả là, hầu hết các chuỗi kết thúc bằng nhiều token đệm — thường là hàng chục hoặc thậm chí hàng trăm. Mặc dù chúng ta đang sử dụng một lớp GRU, tốt hơn nhiều so với lớp SimpleRNN, nhưng bộ nhớ ngắn hạn của nó vẫn không tốt, vì vậy khi nó đi qua nhiều token đệm, nó cuối cùng sẽ quên đánh giá đó là về cái gì! Một giải pháp là cung cấp cho mô hình các batch câu có độ dài bằng nhau (điều này cũng tăng tốc độ huấn luyện). Một giải pháp khác là làm cho RNN bỏ qua các token đệm. Điều này có thể được thực hiện bằng cách sử dụng mặt nạ (masking).

Mặt nạ (Masking)

Việc làm cho mô hình bỏ qua các token đệm là điều dễ dàng khi sử dụng Keras: chỉ cần thêm `mask_zero=True` khi tạo lớp `Embedding`. Điều này có nghĩa là các token đệm (có ID là 0) sẽ bị bỏ qua bởi tất cả các lớp phía sau. Chỉ vậy thôi! Nếu bạn huấn luyện lại mô hình trước đó trong vài epoch, bạn sẽ thấy rằng độ chính xác xác thực nhanh chóng đạt trên 80%.

Cách hoạt động của nó là lớp `Embedding` tạo một tensor mặt nạ bằng `tf.math.not_equal(inputs, 0)`: đó là một tensor Boolean có hình dạng giống như đầu vào, và nó bằng `False` ở bất cứ đâu các ID token là 0, hoặc `True` ở những nơi khác. Tensor mặt nạ này sau đó được mô hình tự động lan truyền đến lớp tiếp theo. Nếu phương thức

`call()` của lớp đó có một đối số `mask`, thì nó sẽ tự động nhận mặt nạ. Điều này cho phép lớp bỏ qua các bước thời gian thích hợp. Mỗi lớp có thể xử lý mặt nạ khác nhau, nhưng nói chung chúng chỉ đơn giản bỏ qua các bước thời gian được che (tức là các bước thời gian mà mặt nạ là `False`).

Ví dụ, khi một lớp hồi quy gặp một bước thời gian được che, nó chỉ đơn giản sao chép đầu ra từ bước thời gian trước đó.

Tiếp theo, nếu thuộc tính `supports_masking` của lớp là `True`, thì mặt nạ sẽ tự động lan truyền đến lớp tiếp theo. Nó tiếp tục lan truyền theo cách này miễn là các lớp có `supports_masking=True`.

Ví dụ, thuộc tính `supports_masking` của lớp hồi quy là `True` khi `return_sequences=True`, nhưng nó là `False` khi `return_sequences=False` vì không cần mặt nạ nữa trong trường hợp này. Vì vậy, nếu bạn có một mô hình với một số lớp hồi quy với `return_sequences=True`, theo sau là một lớp hồi quy với `return_sequences=False`, thì mặt nạ sẽ tự động lan truyền đến lớp hồi quy cuối cùng: lớp đó sẽ sử dụng mặt nạ để bỏ qua các bước được che, nhưng nó sẽ không lan truyền mặt nạ đi xa hơn.

Tương tự, nếu bạn đặt `mask_zero=True` khi tạo lớp `Embedding` trong mô hình phân tích cảm xúc mà chúng ta vừa xây dựng, thì lớp GRU sẽ tự động nhận và sử dụng mặt nạ, nhưng nó sẽ không lan truyền mặt nạ đi xa hơn, vì `return_sequences` không được đặt thành `True`.

Nhiều lớp Keras hỗ trợ mặt nạ: `SimpleRNN`, `GRU`, `LSTM`, `Bidirectional`, `Dense`, `TimeDistributed`, `Add`, và một vài lớp khác (tất cả trong gói `tf.keras.layers`). Tuy nhiên, các lớp tích chập (bao gồm `Conv1D`) không hỗ trợ mặt nạ — dù sao thì không rõ chúng sẽ làm điều đó như thế nào.

Nếu mặt nạ lan truyền đến tận đầu ra, thì nó cũng được áp dụng cho các hàm mất mát, vì vậy các bước thời gian được che sẽ không đóng góp vào mất mát (mất mát của chúng sẽ là 0). Điều này giả định rằng mô hình xuất ra các chuỗi, điều này không đúng trong mô hình phân tích cảm xúc của chúng ta.

Nếu bạn muốn triển khai lớp tùy chỉnh của riêng mình với hỗ trợ mặt nạ, bạn nên thêm một đối số `mask` vào phương thức `call()`, và rõ ràng làm cho phương thức sử dụng mặt nạ. Ngoài ra, nếu mặt nạ phải được lan truyền đến các lớp tiếp theo, thì bạn nên đặt `self.supports_masking=True` trong hàm tạo. Nếu mặt nạ phải được cập nhật trước khi nó được lan truyền, thì bạn phải triển khai phương thức `compute_mask()`.

Nếu mô hình của bạn không bắt đầu bằng một lớp `Embedding`, bạn có thể sử dụng lớp `tf.keras.layers.Masking` thay thế: theo mặc định, nó đặt mặt nạ thành `tf.math.reduce_any(tf.math.not_equal(X, 0), axis=-1)`, nghĩa là các bước thời gian mà chiều cuối cùng chứa đầy các số 0 sẽ bị che trong các lớp tiếp theo.

Sử dụng các lớp mặt nạ và lan truyền mặt nạ tự động hoạt động tốt nhất cho các mô hình đơn giản. Nó sẽ không phải lúc nào cũng hoạt động cho các mô hình phức tạp hơn, chẳng hạn như khi bạn cần trộn các lớp `Conv1D` với các lớp hồi quy. Trong những trường hợp như

vậy, bạn sẽ cần tính toán mặt nạ một cách rõ ràng và truyền nó đến các lớp thích hợp, sử dụng API chức năng hoặc API phân lớp. Ví dụ, mô hình sau đây tương đương với mô hình trước đó, ngoại trừ nó được xây dựng bằng API chức năng và xử lý mặt nạ thủ công. Nó cũng thêm một chút dropout vì mô hình trước đó hơi bị overfitting:

```
inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
token_ids = text_vec_layer(inputs)
mask = tf.math.not_equal(token_ids, 0)
Z = tf.keras.layers.Embedding(vocab_size, embed_size)(token_ids)
Z = tf.keras.layers.GRU(128, dropout=0.2)(Z, mask=mask)
outputs = tf.keras.layers.Dense(1, activation="sigmoid")(Z)
model = tf.keras.Model(inputs=[inputs], outputs=[outputs])
```

Một cách tiếp cận cuối cùng để che mặt là cung cấp cho mô hình các tensor lởm chởm (ragged tensors). Trên thực tế, tất cả những gì bạn cần làm là đặt `ragged=True` khi tạo lớp `TextVectorization`, để các chuỗi đầu vào được biểu diễn dưới dạng các tensor lởm chởm:

```
>>> text_vec_layer_ragged = tf.keras.layers.TextVectorization(
...        max_tokens=vocab_size, ragged=True)
...
>>> text_vec_layer_ragged.adapt(train_set.map(lambda reviews, labels:
reviews))
>>> text_vec_layer_ragged(["Great movie!", "This is DiCaprio's best role."])
<tf.RaggedTensor [[86, 18], [11, 7, 1, 116, 217]]>
```

So sánh biểu diễn tensor lởm chởm này với biểu diễn tensor thông thường, sử dụng các token đệm:

```
>>> text_vec_layer(["Great movie!", "This is DiCaprio's best role."])
<tf.Tensor:
shape=(2, 5), dtype=int64, numpy= array([[ 86,18,         0,  0,  0],
[ 11,  7,  1, 116, 217]])>
```

Các lớp hồi quy của Keras có hỗ trợ tích hợp cho các tensor lởm chởm, vì vậy bạn không cần làm gì khác: chỉ cần sử dụng lớp `TextVectorization` này trong mô hình của bạn. Không cần truyền `mask_zero=True` hoặc xử lý mặt nạ một cách rõ ràng — tất cả đều được triển khai cho bạn. Điều đó thật tiện lợi! Tuy nhiên, tính đến đầu năm 2022, hỗ trợ cho các tensor lởm chởm trong Keras vẫn còn khá mới, vì vậy vẫn còn một vài khuyết điểm. Ví dụ, hiện tại không thể sử dụng các tensor lởm chởm làm mục tiêu khi chạy trên GPU (nhưng điều này có thể được giải quyết vào thời điểm bạn đọc những dòng này).

Bất kể phương pháp che mặt nào bạn thích, sau khi huấn luyện mô hình này trong vài epoch, nó sẽ trở nên khá tốt trong việc đánh giá một đánh giá là tích cực hay không. Nếu bạn sử dụng callback `tf.keras.callbacks.TensorBoard()`, bạn có thể trực quan hóa các embedding trong TensorBoard khi chúng đang được học: thật thú vị khi thấy các từ như “awesome” và “amazing” dần dần tập hợp ở một phía của không gian embedding, trong khi các từ như “awful” và “terrible” tập hợp ở phía bên kia. Một số từ không tích cực như bạn

mong đợi (ít nhất là với mô hình này), chẳng hạn như từ “good”, có lẽ vì nhiều đánh giá tiêu cực chứa cụm từ “not good”.

Tái sử dụng Embedding và Mô hình ngôn ngữ đã được huấn luyện trước

Thật ấn tượng khi mô hình có thể học được các embedding từ có ích chỉ dựa trên 25.000 đánh giá phim. Hãy tưởng tượng các embedding sẽ tốt đến mức nào nếu chúng ta có hàng tỷ đánh giá để huấn luyện!

Thật không may, chúng ta không có, nhưng có lẽ chúng ta có thể tái sử dụng các embedding từ đã được huấn luyện trên một kho văn bản (rất) lớn khác (ví dụ: đánh giá của Amazon, có sẵn trên TensorFlow Datasets), ngay cả khi nó không bao gồm các đánh giá phim? Dù sao đi nữa, từ “amazing” thường có cùng ý nghĩa dù bạn sử dụng nó để nói về phim hay bất cứ điều gì khác.

Hơn nữa, có lẽ các embedding sẽ hữu ích cho phân tích cảm xúc ngay cả khi chúng được huấn luyện cho một nhiệm vụ khác: vì các từ như “awesome” và “amazing” có ý nghĩa tương tự, chúng có khả năng tập hợp trong không gian embedding ngay cả đối với các nhiệm vụ như dự đoán từ tiếp theo trong câu. Nếu tất cả các từ tích cực và tất cả các từ tiêu cực tạo thành các cụm, thì điều này sẽ hữu ích cho phân tích cảm xúc. Vì vậy, thay vì huấn luyện các embedding từ, chúng ta có thể tải xuống và sử dụng các embedding đã được huấn luyện trước, chẳng hạn như embedding Word2vec của Google, embedding GloVe của Stanford hoặc embedding FastText của Facebook.

Sử dụng các embedding từ đã được huấn luyện trước đã phổ biến trong vài năm, nhưng cách tiếp cận này có những giới hạn. Đặc biệt, một từ có một biểu diễn duy nhất, bất kể ngữ cảnh. Ví dụ, từ “right” được mã hóa giống nhau trong “left and right” và “right and wrong”, mặc dù nó có nghĩa là hai điều rất khác nhau. Để giải quyết hạn chế này, một bài báo năm 2018 của Matthew Peters đã giới thiệu Embedding from Language Models (ELMo): đây là các embedding từ theo ngữ cảnh được học từ các trạng thái bên trong của một mô hình ngôn ngữ hai chiều sâu. Thay vì chỉ sử dụng các embedding đã được huấn luyện trước trong mô hình của bạn, bạn tái sử dụng một phần của mô hình ngôn ngữ đã được huấn luyện trước.

Khoảng cùng thời gian, bài báo Universal Language Model Fine-Tuning (ULMFiT) của Jeremy Howard và Sebastian Ruder đã chứng minh hiệu quả của việc tiền huấn luyện không giám sát cho các nhiệm vụ NLP: các tác giả đã huấn luyện một mô hình ngôn ngữ LSTM trên một kho văn bản khổng lồ bằng cách sử dụng học tự giám sát (tức là tự động tạo nhãn từ dữ liệu), sau đó họ tinh chỉnh nó trên các nhiệm vụ khác nhau. Mô hình của họ vượt trội so với các mô hình tiên tiến nhất trên sáu nhiệm vụ phân loại văn bản với biên độ lớn (giảm tỷ lệ lỗi 18–24% trong hầu hết các trường hợp). Hơn nữa, các tác giả đã chỉ ra rằng một mô hình được huấn luyện trước được tinh chỉnh chỉ với 100 ví dụ được gán nhãn có thể đạt được hiệu suất tương tự như một mô hình được huấn luyện từ đầu trên 10.000 ví dụ. Trước bài báo ULMFiT, việc sử dụng các mô hình được huấn luyện trước chỉ là chuẩn mực trong thị giác máy tính; trong bối cảnh NLP, việc tiền huấn luyện chỉ giới hạn ở các embedding từ.

Bài báo này đã đánh dấu sự khởi đầu của một kỷ nguyên mới trong NLP: ngày nay, việc tái sử dụng các mô hình ngôn ngữ được huấn luyện trước là chuẩn mực.

Ví dụ, hãy xây dựng một bộ phân loại dựa trên Universal Sentence Encoder, một kiến trúc mô hình được giới thiệu trong một bài báo năm 2018 của một nhóm các nhà nghiên cứu Google. Mô hình này dựa trên kiến trúc transformer, mà chúng ta sẽ xem xét sau trong chương này. Tiện lợi thay, mô hình này có sẵn trên TensorFlow Hub:

```
import tensorflow_hub as hub
import os

os.environ["TFHUB_CACHE_DIR"] = "my_tfhub_cache"
model = tf.keras.Sequential([
    hub.KerasLayer("https://tfhub.dev/google/universal-sentence-encoder/4",
                   trainable=True, dtype=tf.string, input_shape=[]),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(loss="binary_crossentropy",
              optimizer="nadam", metrics=["accuracy"])
model.fit(train_set, validation_data=valid_set, epochs=10)
```

Lưu ý rằng phần cuối cùng của URL mô-đun TensorFlow Hub chỉ định rằng chúng ta muốn phiên bản 4 của mô hình. Việc đánh phiên bản này đảm bảo rằng nếu một phiên bản mô-đun mới được phát hành trên TF Hub, nó sẽ không phá vỡ mô hình của chúng ta. Tiện lợi thay, nếu bạn chỉ cần nhập URL này vào trình duyệt web, bạn sẽ nhận được tài liệu cho mô-đun này.

Cũng lưu ý rằng chúng ta đặt `trainable=True` khi tạo `hub.KerasLayer`. Bằng cách này, Universal Sentence Encoder đã được huấn luyện trước được tinh chỉnh trong quá trình huấn luyện: một số trọng số của nó được điều chỉnh thông qua lan truyền ngược. Không phải tất cả các mô-đun TensorFlow Hub đều có thể tinh chỉnh, vì vậy hãy đảm bảo kiểm tra tài liệu cho từng mô-đun đã được huấn luyện trước mà bạn quan tâm.

Sau khi huấn luyện, mô hình này sẽ đạt độ chính xác xác thực trên 90%. Điều đó thực sự rất tốt: nếu bạn cố gắng tự thực hiện nhiệm vụ, bạn có thể chỉ làm tốt hơn một chút vì nhiều đánh giá chứa cả bình luận tích cực và tiêu cực. Phân loại các đánh giá mơ hồ này giống như tung đồng xu.

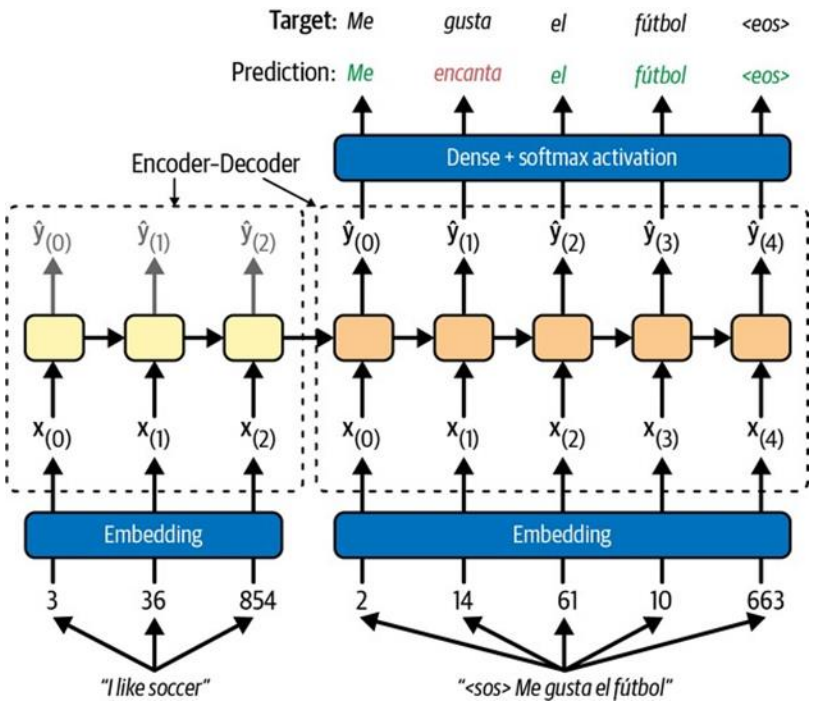
Cho đến nay, chúng ta đã xem xét việc tạo văn bản bằng char-RNN và phân tích cảm xúc bằng các mô hình RNN cấp từ (dựa trên các embedding có thể huấn luyện) và sử dụng một mô hình ngôn ngữ mạnh mẽ đã được huấn luyện trước từ TensorFlow Hub. Trong phần tiếp theo, chúng ta sẽ khám phá một nhiệm vụ NLP quan trọng khác: dịch máy thần kinh (NMT).

Mạng mã hóa-giải mã cho dịch máy thần kinh

Hãy bắt đầu với một mô hình NMT đơn giản sẽ dịch các câu tiếng Anh sang tiếng Tây Ban Nha (xem Hình 16-3). Tóm lại, kiến trúc như sau: các câu tiếng Anh được đưa vào làm đầu

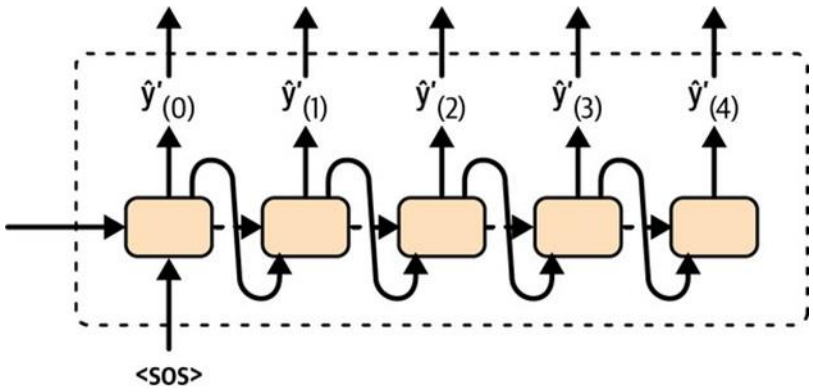
vào cho bộ mã hóa, và bộ giải mã xuất ra các bản dịch tiếng Tây Ban Nha. Lưu ý rằng các bản dịch tiếng Tây Ban Nha cũng được sử dụng làm đầu vào cho bộ giải mã trong quá trình huấn luyện, nhưng được dịch chuyển lùi một bước. Nói cách khác, trong quá trình huấn luyện, bộ giải mã được cung cấp làm đầu vào từ mà nó đáng lẽ phải xuất ra ở bước trước đó, bất kể nó thực sự xuất ra gì. Đây được gọi là teacher forcing — một kỹ thuật giúp tăng tốc đáng kể việc huấn luyện và cải thiện hiệu suất của mô hình. Đối với từ đầu tiên, bộ giải mã được cung cấp token bắt đầu chuỗi (SOS), và bộ giải mã được kỳ vọng kết thúc câu bằng token kết thúc chuỗi (EOS). Mỗi từ ban đầu được biểu diễn bằng ID của nó (ví dụ: 854 cho từ “soccer”). Tiếp theo, một lớp `Embedding` trả về embedding của từ. Các embedding từ này sau đó được đưa vào bộ mã hóa và bộ giải mã.

Ở mỗi bước, bộ giải mã xuất ra một điểm cho mỗi từ trong từ vựng đầu ra (tức là tiếng Tây Ban Nha), sau đó hàm kích hoạt softmax biến các điểm này thành xác suất. Ví dụ, ở bước đầu tiên, từ “Me” có thể có xác suất 7%, “Yo” có thể có xác suất 1%, v.v. Từ có xác suất cao nhất được xuất ra. Điều này rất giống với một nhiệm vụ phân loại thông thường, và thực sự bạn có thể huấn luyện mô hình bằng cách sử dụng hàm mất mát `"sparse_categorical_crossentropy"`, giống như chúng ta đã làm trong mô hình char-RNN.



Hình 16-3. Một mô hình dịch máy đơn giản

Lưu ý rằng tại thời điểm suy luận (sau khi huấn luyện), bạn sẽ không có câu mục tiêu để đưa vào bộ giải mã. Thay vào đó, bạn cần đưa vào từ mà nó vừa xuất ra ở bước trước đó, như trong Hình 16-4 (điều này sẽ yêu cầu tra cứu embedding không được hiển thị trong sơ đồ).



Hình 16-4. Tại thời điểm suy luận, bộ giải mã được cung cấp làm đầu vào từ mà nó vừa xuất ra ở bước thời gian trước đó

Hãy xây dựng và huấn luyện mô hình này! Đầu tiên, chúng ta cần tải xuống một tập dữ liệu các cặp câu tiếng Anh/Tây Ban Nha:

```
import tensorflow as tf
from pathlib import Path
import numpy as np

url = "https://storage.googleapis.com/download.tensorflow.org/data/spa-
eng.zip"
path = tf.keras.utils.get_file("spa-eng.zip", origin=url,
cache_dir="datasets",
                               extract=True)
text = (Path(path).with_name("spa-eng") / "spa.txt").read_text()
```

Mỗi dòng chứa một câu tiếng Anh và bản dịch tiếng Tây Ban Nha tương ứng, được phân tách bằng một tab. Chúng ta sẽ bắt đầu bằng cách loại bỏ các ký tự tiếng Tây Ban Nha “¡” và “¿”, mà lớp `TextVectorization` không xử lý, sau đó chúng ta sẽ phân tích cú pháp các cặp câu và xáo trộn chúng. Cuối cùng, chúng ta sẽ chia chúng thành hai danh sách riêng biệt, mỗi danh sách một ngôn ngữ:

```
text = text.replace("¡", "").replace("¿", "")

pairs = [line.split("\t") for line in text.splitlines()]
np.random.shuffle(pairs)
sentences_en, sentences_es = zip(*pairs) # tách các cặp thành 2 danh sách
```

Hãy xem xét ba cặp câu đầu tiên:

```
>>> for i in range(3):
...     print(sentences_en[i], "=>", sentences_es[i])
...
How boring! => Qué aburrimiento!
I love sports. => Adoro el deporte.
Would you like to swap jobs? => Te gustaría que intercambiemos los trabajos?
```

Tiếp theo, hãy tạo hai lớp `TextVectorization` — một cho mỗi ngôn ngữ — và điều chỉnh chúng vào văn bản:

```
vocab_size = 1000
max_length = 50
text_vec_layer_en = tf.keras.layers.TextVectorization(
    vocab_size, output_sequence_length=max_length)
text_vec_layer_es = tf.keras.layers.TextVectorization(
    vocab_size, output_sequence_length=max_length)
text_vec_layer_en.adapt(sentences_en)

text_vec_layer_es.adapt([f"startofseq {s} endofseq" for s in sentences_es])
```

Có một vài điều cần lưu ý ở đây:

- Chúng ta giới hạn kích thước từ vựng ở 1.000, khá nhỏ. Điều này là do tập huấn luyện không quá lớn, và vì sử dụng một giá trị nhỏ sẽ tăng tốc độ huấn luyện. Các mô hình dịch thuật tiên tiến thường sử dụng từ vựng lớn hơn nhiều (ví dụ: 30.000), tập huấn luyện lớn hơn nhiều (gigabyte), và mô hình lớn hơn nhiều (hàng trăm hoặc thậm chí hàng nghìn megabyte). Ví dụ, hãy xem các mô hình Opus-MT của Đại học Helsinki, hoặc mô hình M2M-100 của Facebook.
- Vì tất cả các câu trong tập dữ liệu có tối đa 50 từ, chúng ta đặt `output_sequence_length` thành 50: bằng cách này, các chuỗi đầu vào sẽ tự động được đệm bằng các số 0 cho đến khi chúng đều dài 50 token. Nếu có bất kỳ câu nào dài hơn 50 token trong tập huấn luyện, nó sẽ được cắt thành 50 token.
- Đối với văn bản tiếng Tây Ban Nha, chúng ta thêm “startofseq” và “endofseq” vào mỗi câu khi điều chỉnh lớp `TextVectorization`: chúng ta sẽ sử dụng các từ này làm token SOS và EOS. Bạn có thể sử dụng bất kỳ từ nào khác, miễn là chúng không phải là các từ tiếng Tây Ban Nha thực tế.

Hãy kiểm tra 10 token đầu tiên trong cả hai từ vựng. Chúng bắt đầu bằng token đệm, token không xác định, token SOS và EOS (chỉ trong từ vựng tiếng Tây Ban Nha), sau đó là các từ thực tế, được sắp xếp theo tần suất giảm dần:

```
>>> text_vec_layer_en.get_vocabulary()[:10]
['', '[UNK]', 'the', 'i', 'to', 'you', 'tom', 'a', 'is', 'he']

>>> text_vec_layer_es.get_vocabulary()[:10]
['', '[UNK]', 'startofseq', 'endofseq', 'de', 'que', 'a', 'no', 'tom', 'la']
```

Tiếp theo, hãy tạo tập huấn luyện và tập xác thực (bạn cũng có thể tạo tập kiểm tra nếu bạn cần). Chúng ta sẽ sử dụng 100.000 cặp câu đầu tiên để huấn luyện, và phần còn lại để xác thực. Đầu vào của bộ giải mã là các câu tiếng Tây Ban Nha cộng với tiền tố token SOS. Các mục tiêu là các câu tiếng Tây Ban Nha cộng với hậu tố EOS:

```
X_train = tf.constant(sentences_en[:100_000])
X_valid = tf.constant(sentences_en[100_000:])
X_train_dec = tf.constant([f"startofseq {s}" for s in
```

```
sentences_es[:100_000]])
X_valid_dec = tf.constant([f"startofseq {s}" for s in
sentences_es[100_000:]])
Y_train = text_vec_layer_es([f"{s} endofseq" for s in
sentences_es[:100_000]])
Y_valid = text_vec_layer_es([f"{s} endofseq" for s in
sentences_es[100_000:]])
```

OK, bây giờ chúng ta đã sẵn sàng xây dựng mô hình dịch thuật của mình. Chúng ta sẽ sử dụng API chức năng cho việc đó vì mô hình không phải là tuần tự. Nó yêu cầu hai đầu vào văn bản — một cho bộ mã hóa và một cho bộ giải mã — vì vậy hãy bắt đầu với điều đó:

```
encoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
decoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
```

Tiếp theo, chúng ta cần mã hóa các câu này bằng cách sử dụng các lớp `TextVectorization` mà chúng ta đã chuẩn bị trước đó, tiếp theo là một lớp `Embedding` cho mỗi ngôn ngữ, với `mask_zero=True` để đảm bảo che mặt được xử lý tự động. Kích thước embedding là một siêu tham số bạn có thể điều chỉnh, như mọi khi:

```
embed_size = 128
encoder_input_ids = text_vec_layer_en(encoder_inputs)
decoder_input_ids = text_vec_layer_es(decoder_inputs)
encoder_embedding_layer = tf.keras.layers.Embedding(vocab_size, embed_size,
                                                    mask_zero=True)
decoder_embedding_layer = tf.keras.layers.Embedding(vocab_size, embed_size,
                                                    mask_zero=True)
encoder_embeddings = encoder_embedding_layer(encoder_input_ids)
decoder_embeddings = decoder_embedding_layer(decoder_input_ids)
```

Bây giờ hãy tạo bộ mã hóa và truyền các đầu vào đã được nhúng vào nó:

```
encoder = tf.keras.layers.LSTM(512, return_state=True)
encoder_outputs, *encoder_state = encoder(encoder_embeddings)
```

Để giữ mọi thứ đơn giản, chúng ta chỉ sử dụng một lớp LSTM duy nhất, nhưng bạn có thể xếp chồng nhiều lớp. Chúng ta cũng đặt `return_state=True` để có một tham chiếu đến trạng thái cuối cùng của lớp. Vì chúng ta đang sử dụng lớp LSTM, thực sự có hai trạng thái: trạng thái ngắn hạn và trạng thái dài hạn. Lớp trả về các trạng thái này riêng biệt, đó là lý do tại sao chúng ta phải viết `*encoder_state` để nhóm cả hai trạng thái vào một danh sách. Bây giờ chúng ta có thể sử dụng trạng thái (kép) này làm trạng thái ban đầu của bộ giải mã:

```
decoder = tf.keras.layers.LSTM(512, return_sequences=True)
decoder_outputs = decoder(decoder_embeddings, initial_state=encoder_state)
```

Tiếp theo, chúng ta có thể truyền đầu ra của bộ giải mã qua một lớp `Dense` với hàm kích hoạt softmax để lấy xác suất từ cho mỗi bước:

```
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")
Y_proba = output_layer(decoder_outputs)
```

Và thế là xong! Chúng ta chỉ cần tạo Keras Model, biên dịch nó và huấn luyện nó:

```
model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                       outputs=[Y_proba])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])

model.fit((X_train, X_train_dec), Y_train, epochs=10,
          validation_data=((X_valid, X_valid_dec), Y_valid))
```

Sau khi huấn luyện, chúng ta có thể sử dụng mô hình để dịch các câu tiếng Anh mới sang tiếng Tây Ban Nha. Nhưng nó không đơn giản như gọi `model.predict()`, bởi vì bộ giải mã mong đợi đầu vào là từ đã được dự đoán ở bước thời gian trước đó. Một cách để làm điều này là viết một ô nhớ tùy chỉnh theo dõi đầu ra trước đó và truyền nó đến bộ mã hóa ở bước thời gian tiếp theo. Tuy nhiên, để giữ mọi thứ đơn giản, chúng ta có thể chỉ cần gọi mô hình nhiều lần, dự đoán thêm một từ ở mỗi vòng. Hãy viết một hàm tiện ích nhỏ cho việc đó:

```
def translate(sentence_en):
    translation = ""
    for word_idx in range(max_length):
        X = np.array([sentence_en]) # đầu vào bộ mã hóa
        X_dec = np.array(["startofseq " + translation]) # đầu vào bộ giải mã
        y_proba = model.predict((X, X_dec))[0, word_idx] # xác suất của token
cuối cùng
        predicted_word_id = np.argmax(y_proba)

        predicted_word =
text_vec_layer_es.get_vocabulary()[predicted_word_id]
        if predicted_word == "endofseq":
            break
        translation += " " + predicted_word
    return translation.strip()
```

Hàm này chỉ đơn giản là tiếp tục dự đoán từng từ một, dần dần hoàn thành bản dịch, và nó dừng lại khi đạt đến token EOS. Hãy thử xem!

```
>>> translate("I
like soccer") 'me gusta el fútbol'
```

Hoan hô, nó hoạt động! Chà, ít nhất là với những câu rất ngắn. Nếu bạn thử chơi với mô hình này một lúc, bạn sẽ thấy rằng nó chưa song ngữ, và đặc biệt là nó thực sự gặp khó khăn với những câu dài hơn. Ví dụ:

```
>>> translate("I
like soccer and also going to the beach") 'me gusta el fútbol y a
veces mismo al bus'
```

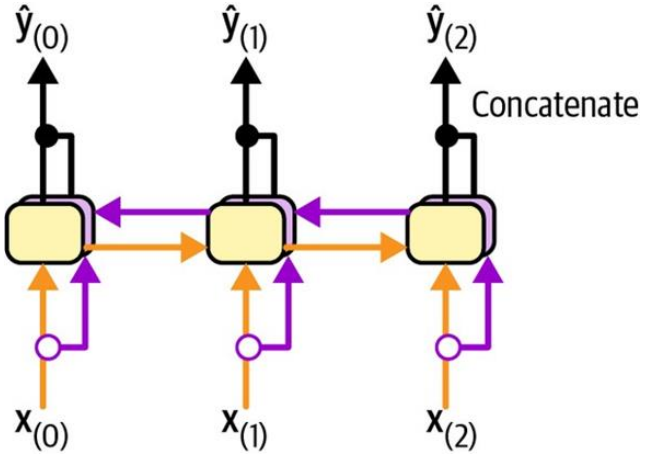
Bản dịch nói “Tôi thích bóng đá và đôi khi cả xe buýt”. Vậy làm thế nào bạn có thể cải thiện nó? Một cách là tăng kích thước tập huấn luyện và thêm nhiều lớp LSTM hơn trong cả bộ

mã hóa và bộ giải mã. Nhưng điều này sẽ chỉ giúp bạn đến một mức độ nào đó, vì vậy hãy xem xét các kỹ thuật phức tạp hơn, bắt đầu với các lớp hồi quy hai chiều.

RNN hai chiều

Ở mỗi bước thời gian, một lớp hồi quy thông thường chỉ xem xét các đầu vào trong quá khứ và hiện tại trước khi tạo ra đầu ra của nó. Nói cách khác, nó mang tính nhân quả, nghĩa là nó không thể nhìn thấy tương lai. Loại RNN này có ý nghĩa khi dự báo chuỗi thời gian, hoặc trong bộ giải mã của một mô hình tuần tự (seq2seq). Nhưng đối với các nhiệm vụ như phân loại văn bản, hoặc trong bộ mã hóa của một mô hình seq2seq, thường thì nên nhìn trước các từ tiếp theo trước khi mã hóa một từ đã cho.

Ví dụ, hãy xem xét các cụm từ “the right arm” (cánh tay phải), “the right person” (người phù hợp), và “the right to criticize” (quyền chỉ trích): để mã hóa đúng từ “right”, bạn cần nhìn về phía trước. Một giải pháp là chạy hai lớp hồi quy trên cùng một đầu vào, một lớp đọc các từ từ trái sang phải và lớp còn lại đọc chúng từ phải sang trái, sau đó kết hợp các đầu ra của chúng ở mỗi bước thời gian, thường là bằng cách nối chúng lại. Đây là những gì một lớp hồi quy hai chiều thực hiện (xem Hình 16-5).



Hình 16-5. Một lớp hồi quy hai chiều

Để triển khai một lớp hồi quy hai chiều trong Keras, chỉ cần gói một lớp hồi quy trong một lớp `tf.keras.layers.Bidirectional`. Ví dụ, lớp `Bidirectional` sau đây có thể được sử dụng làm bộ mã hóa trong mô hình dịch thuật của chúng ta:

```
encoder = tf.keras.layers.Bidirectional(
    tf.keras.layers.LSTM(256, return_sequences=True, return_state=True))
```

Chỉ có một vấn đề. Lớp này bây giờ sẽ trả về bốn trạng thái thay vì hai: các trạng thái ngắn hạn và dài hạn cuối cùng của lớp LSTM chuyển tiếp, và các trạng thái ngắn hạn và dài hạn cuối cùng của lớp LSTM ngược. Chúng ta không thể sử dụng trạng thái bốn này trực tiếp làm trạng thái ban đầu của lớp LSTM của bộ giải mã, vì nó chỉ mong đợi hai trạng thái (ngắn hạn và dài hạn). Chúng ta không thể làm cho bộ giải mã hai chiều, vì nó phải giữ tính nhân quả: nếu không nó sẽ gian lận trong quá trình huấn luyện và nó sẽ không hoạt động. Thay

vào đó, chúng ta có thể nối hai trạng thái ngắn hạn lại với nhau, và cũng nối hai trạng thái dài hạn lại với nhau:

```
encoder_outputs, *encoder_state = encoder(encoder_embeddings)
encoder_state = [tf.concat(encoder_state[::2], axis=-1), # ngắn hạn (0 & 2)
                 tf.concat(encoder_state[1::2], axis=-1)] # dài hạn (1 & 3)
```

Bây giờ chúng ta hãy xem một kỹ thuật phổ biến khác có thể cải thiện đáng kể hiệu suất của mô hình dịch thuật tại thời điểm suy luận: tìm kiếm chùm (beam search).

Tìm kiếm chùm (Beam Search)

Giả sử bạn đã huấn luyện một mô hình mã hóa-giải mã, và bạn sử dụng nó để dịch câu “I like soccer” sang tiếng Tây Ban Nha. Bạn đang hy vọng rằng nó sẽ xuất ra bản dịch chính xác “me gusta el fútbol”, nhưng thật không may nó lại xuất ra “me gustan los jugadores”, có nghĩa là “Tôi thích các cầu thủ”. Nhìn vào tập huấn luyện, bạn nhận thấy nhiều câu như “I like cars”, được dịch là “me gustan los autos”, vì vậy không phải là vô lý khi mô hình xuất ra “me gustan los” sau khi thấy “I like”. Thật không may, trong trường hợp này đó là một lỗi vì “soccer” là số ít. Mô hình không thể quay lại và sửa lỗi, vì vậy nó cố gắng hoàn thành câu tốt nhất có thể, trong trường hợp này sử dụng từ “jugadores”. Làm thế nào chúng ta có thể cho mô hình cơ hội quay lại và sửa lỗi mà nó đã mắc phải trước đó? Một trong những giải pháp phổ biến nhất là tìm kiếm chùm: nó theo dõi một danh sách ngắn gồm k câu hứa hẹn nhất (ví dụ, ba câu hàng đầu), và ở mỗi bước giải mã, nó cố gắng mở rộng chúng thêm một từ, chỉ giữ lại k câu có khả năng nhất. Tham số k được gọi là chiều rộng chùm (beam width).

Ví dụ, giả sử bạn sử dụng mô hình để dịch câu “I like soccer” bằng cách sử dụng tìm kiếm chùm với chiều rộng chùm là 3 (xem Hình 16-6). Ở bước giải mã đầu tiên, mô hình sẽ xuất ra xác suất ước tính cho mỗi từ đầu tiên có thể có trong câu được dịch. Giả sử ba từ hàng đầu là “me” (xác suất ước tính 75%), “a” (3%), và “como” (1%). Đó là danh sách ngắn của chúng ta cho đến nay. Tiếp theo, chúng ta sử dụng mô hình để tìm từ tiếp theo cho mỗi câu. Đối với câu đầu tiên (“me”), có lẽ mô hình xuất ra xác suất 36% cho từ “gustan”, 32% cho từ “gusta”, 16% cho từ “encanta”, v.v. Lưu ý rằng đây thực sự là các xác suất có điều kiện, với điều kiện câu bắt đầu bằng “me”. Đối với câu thứ hai (“a”), mô hình có thể xuất ra xác suất có điều kiện 50% cho từ “mi”, v.v. Giả sử từ vựng có 1.000 từ, chúng ta sẽ có 1.000 xác suất cho mỗi câu.

Tiếp theo, chúng ta tính xác suất của mỗi trong số 3.000 câu hai từ mà chúng ta đã xem xét (3 × 1.000). Chúng ta làm điều này bằng cách nhân xác suất có điều kiện ước tính của mỗi từ với xác suất ước tính của câu mà nó hoàn thành. Ví dụ, xác suất ước tính của câu “me” là 75%, trong khi xác suất có điều kiện ước tính của từ “gustan” (với điều kiện từ đầu tiên là “me”) là 36%, vì vậy xác suất ước tính của câu “me gustan” là 75% × 36% = 27%. Sau khi tính toán xác suất của tất cả 3.000 câu hai từ, chúng ta chỉ giữ lại 3 câu hàng đầu. Trong ví dụ này, tất cả chúng đều bắt đầu bằng từ “me”: “me gustan” (27%), “me gusta” (24%), và “me encanta” (12%).

Hiện tại, câu “me gustan” đang thắng, nhưng “me gusta” chưa bị loại bỏ.

Hình 16-6. Tìm kiếm chùm, với chiều rộng chùm là 3

Sau đó, chúng ta lặp lại cùng một quá trình: chúng ta sử dụng mô hình để dự đoán từ tiếp theo trong mỗi ba câu này, và chúng ta tính toán xác suất của tất cả 3.000 câu ba từ mà chúng ta đã xem xét. Có lẽ ba câu hàng đầu hiện nay là “me gustan los” (10%), “me gusta el” (8%), và “me gusta mucho” (2%). Ở bước tiếp theo, chúng ta có thể nhận được “me gusta el fútbol” (6%), “me gusta mucho el” (1%), và “me gusta el deporte” (0,2%). Lưu ý rằng “me gustan” đã bị loại bỏ, và bản dịch đúng hiện đang dẫn trước. Chúng ta đã tăng hiệu suất của mô hình mã hóa-giải mã của mình mà không cần huấn luyện thêm, chỉ đơn giản bằng cách sử dụng nó một cách khôn ngoan hơn.

Với tất cả những điều này, bạn có thể nhận được các bản dịch khá tốt cho các câu tương đối ngắn. Thật không may, mô hình này sẽ rất tệ trong việc dịch các câu dài. Một lần nữa, vấn đề đến từ bộ nhớ ngắn hạn hạn chế của RNN. Cơ chế chú ý là sự đổi mới thay đổi cuộc chơi đã giải quyết vấn đề này.

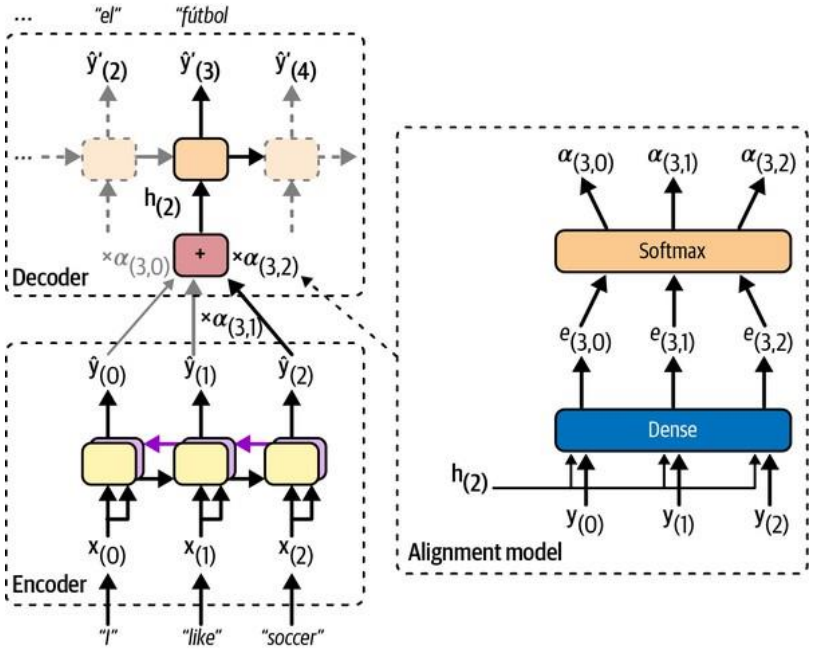
Cơ chế chú ý (Attention Mechanisms)

Hãy xem xét đường dẫn từ từ “soccer” đến bản dịch “fútbol” trong Hình 16-3: nó khá dài! Điều này có nghĩa là một biểu diễn của từ này (cùng với tất cả các từ khác) cần phải được truyền qua nhiều bước trước khi nó thực sự được sử dụng. Chúng ta không thể làm cho đường dẫn này ngắn hơn sao?

Đây là ý tưởng cốt lõi trong một bài báo mang tính bước ngoặt năm 2014 của Dzmitry Bahdanau et al., nơi các tác giả đã giới thiệu một kỹ thuật cho phép bộ giải mã tập trung vào các từ thích hợp (được mã hóa bởi bộ mã hóa) ở mỗi bước thời gian. Ví dụ, tại bước thời gian mà bộ giải mã cần xuất ra từ “fútbol”, nó sẽ tập trung sự chú ý của mình vào từ “soccer”. Điều này có nghĩa là đường dẫn từ một từ đầu vào đến bản dịch của nó giờ đây ngắn hơn nhiều, vì vậy các hạn chế bộ nhớ ngắn hạn của RNN có ít tác động hơn. Cơ chế chú ý đã cách mạng hóa dịch máy thần kinh (và học sâu nói chung), cho phép cải thiện đáng kể trạng thái nghệ thuật, đặc biệt đối với các câu dài (ví dụ: hơn 30 từ).

Hình 16-7 hiển thị mô hình mã hóa-giải mã của chúng ta với cơ chế chú ý được thêm vào. Bên trái, bạn có bộ mã hóa và bộ giải mã. Thay vì chỉ gửi trạng thái ẩn cuối cùng của bộ mã

hóa đến bộ giải mã, cũng như từ mục tiêu trước đó ở mỗi bước (vẫn được thực hiện, mặc dù không được hiển thị trong hình), bây giờ chúng ta cũng gửi tất cả các đầu ra của bộ mã hóa đến bộ giải mã. Vì bộ giải mã không thể xử lý tất cả các đầu ra bộ mã hóa này cùng một lúc, chúng cần được tổng hợp: ở mỗi bước thời gian, ô nhớ của bộ giải mã tính toán tổng trọng số của tất cả các đầu ra bộ mã hóa. Điều này xác định từ nào nó sẽ tập trung vào ở bước này. Trọng số α(t,i) là trọng số của đầu ra bộ mã hóa thứ i tại bước thời gian giải mã thứ t. Ví dụ, nếu trọng số α(3,2) lớn hơn nhiều so với trọng số α(3,0) và α(3,1), thì bộ giải mã sẽ chú ý nhiều hơn đến đầu ra của bộ mã hóa cho từ #2 (“soccer”) hơn là hai đầu ra khác, ít nhất là ở bước thời gian này. Phần còn lại của bộ giải mã hoạt động giống như trước: ở mỗi bước thời gian, ô nhớ nhận các đầu vào chúng ta vừa thảo luận, cộng với trạng thái ẩn từ bước thời gian trước đó, và cuối cùng (mặc dù không được biểu diễn trong sơ đồ) nó nhận từ mục tiêu từ bước thời gian trước đó (hoặc tại thời điểm suy luận, đầu ra từ bước thời gian trước đó).



Hình 16-7. Dịch máy thần kinh sử dụng mạng mã hóa-giải mã với mô hình chú ý

Nhưng các trọng số α(t,i) này đến từ đâu? Chà, chúng được tạo ra bởi một mạng nơ-ron nhỏ gọi là mô hình căn chỉnh (hoặc một lớp chú ý), được huấn luyện cùng với phần còn lại của mô hình mã hóa-giải mã. Mô hình căn chỉnh này được minh họa ở phía bên phải của Hình 16-7. Nó bắt đầu bằng một lớp `Dense` bao gồm một nơ-ron duy nhất xử lý từng đầu ra của bộ mã hóa, cùng với trạng thái ẩn trước đó của bộ giải mã (ví dụ: h(2)). Lớp này xuất ra một điểm (hoặc năng lượng) cho mỗi đầu ra bộ mã hóa (ví dụ: e(3, 2)): điểm này đo lường mức độ phù hợp của mỗi đầu ra với trạng thái ẩn trước đó của bộ giải mã. Ví dụ, trong Hình 16-7, mô hình đã xuất ra “me gusta el” (nghĩa là “Tôi thích”), vì vậy bây giờ nó đang mong đợi một danh từ: từ “soccer” là từ phù hợp nhất với trạng thái hiện tại, vì vậy nó nhận được điểm cao. Cuối cùng, tất cả các điểm đi qua một lớp softmax để nhận được trọng số cuối cùng cho mỗi đầu ra bộ mã hóa (ví dụ: α(3,2)).

Tất cả các trọng số cho một bước thời gian giải mã nhất định cộng lại bằng 1. Cơ chế chú ý đặc biệt này được gọi là Bahdanau attention (đặt theo tên tác giả đầu tiên của bài báo năm 2014). Vì nó nối đầu ra bộ mã hóa với trạng thái ẩn trước đó của bộ giải mã, đôi khi nó được gọi là concatenative attention (hoặc additive attention).

Một cơ chế chú ý phổ biến khác, được gọi là Luong attention hoặc multiplicative attention, được đề xuất ngay sau đó, vào năm 2015, bởi Minh-Thang Luong et al. Vì mục tiêu của mô hình căn chỉnh là đo lường sự tương đồng giữa một trong các đầu ra của bộ mã hóa và trạng thái ẩn trước đó của bộ giải mã, các tác giả đã đề xuất chỉ đơn giản tính tích vô hướng (xem Chương 4) của hai vectơ này, vì đây thường là một thước đo tương đồng khá tốt, và phần cứng hiện đại có thể tính toán nó rất hiệu quả. Để điều này có thể thực hiện được, cả hai vectơ phải có cùng số chiều. Tích vô hướng cho ra một điểm, và tất cả các điểm (tại một bước thời gian giải mã nhất định) đi qua một lớp softmax để cho ra các trọng số cuối cùng, giống như trong Bahdanau attention. Một sự đơn giản hóa khác mà Luong et al. đề xuất là sử dụng trạng thái ẩn của bộ giải mã ở bước thời gian hiện tại thay vì ở bước thời gian trước đó (tức là h(t) thay vì h(t–1)), sau đó sử dụng đầu ra của cơ chế chú ý (ký hiệu $\tilde{h}(t)$) trực tiếp để tính toán các dự đoán của bộ giải mã, thay vì sử dụng nó để tính toán trạng thái ẩn hiện tại của bộ giải mã. Các nhà nghiên cứu cũng đề xuất một biến thể của cơ chế tích vô hướng trong đó các đầu ra bộ mã hóa trước tiên đi qua một lớp kết nối đầy đủ (không có số hạng bias) trước khi các tích vô hướng được tính toán. Điều này được gọi là cách tiếp cận tích vô hướng "chung". Các nhà nghiên cứu đã so sánh cả hai cách tiếp cận tích vô hướng với cơ chế chú ý nối (thêm một vectơ tham số chia tỷ lệ v), và họ quan sát thấy rằng các biến thể tích vô hướng hoạt động tốt hơn chú ý nối. Vì lý do này, chú ý nối ít được sử dụng hơn bây giờ. Các phương trình cho ba cơ chế chú ý này được tóm tắt trong Phương trình 16-1.

Phương trình 16-1. Cơ chế chú ý

$$\tilde{\mathbf{h}}_{(t)} = \sum_i \alpha_{(t,i)} \mathbf{y}_{(i)} \text{ with } \alpha_{(t,i)} = \frac{\exp\left(e_{(t,i)}\right)}{\sum_{i'} \exp\left(e_{(t,i')}\right)} \text{ and } e_{(t,i)} = \begin{cases} \mathbf{h}_{(t)}{}^\top \mathbf{y}_{(i)} & do \\ \mathbf{h}_{(t)}{}^\top \mathbf{W}\, \mathbf{y}_{(i)} & ge \\ \mathbf{v}^\top \tanh\left(\mathbf{W}\left[\mathbf{h}_{(t)}; \mathbf{y}_{(i)}\right]\right) & co \end{cases}$$

Keras cung cấp một lớp `tf.keras.layers.Attention` cho Luong attention, và một lớp `AdditiveAttention` cho Bahdanau attention. Hãy thêm Luong attention vào mô hình mã hóa-giải mã của chúng ta. Vì chúng ta sẽ cần truyền tất cả các đầu ra của bộ mã hóa đến lớp `Attention`, trước tiên chúng ta cần đặt `return_sequences=True` khi tạo bộ mã hóa:

```
encoder = tf.keras.layers.Bidirectional(
    tf.keras.layers.LSTM(256, return_sequences=True, return_state=True))
```

Tiếp theo, chúng ta cần tạo lớp chú ý và truyền trạng thái của bộ giải mã và đầu ra của bộ mã hóa cho nó. Tuy nhiên, để truy cập trạng thái của bộ giải mã ở mỗi bước, chúng ta sẽ cần viết một ô nhớ tùy chỉnh. Để đơn giản, hãy sử dụng đầu ra của bộ giải mã thay vì trạng thái của nó: trên thực tế, điều này cũng hoạt động tốt, và dễ mã hóa hơn nhiều. Sau đó, chúng ta chỉ cần truyền trực tiếp đầu ra của lớp chú ý đến lớp đầu ra, như đề xuất trong bài báo Luong attention:

```
attention_layer = tf.keras.layers.Attention()
attention_outputs = attention_layer([decoder_outputs, encoder_outputs])
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")
Y_proba = output_layer(attention_outputs)
```

Và thế là xong! Nếu bạn huấn luyện mô hình này, bạn sẽ thấy rằng nó hiện xử lý các câu dài hơn nhiều. Ví dụ:

```
>>> translate("I
like soccer and also going to the beach") 'me gusta el fútbol y
también ir a la playa'
```

Tóm lại, lớp chú ý cung cấp một cách để tập trung sự chú ý của mô hình vào một phần của đầu vào.

Nhưng có một cách khác để nghĩ về lớp này: nó hoạt động như một cơ chế truy xuất bộ nhớ có thể phân biệt được.

Ví dụ, giả sử bộ mã hóa đã phân tích câu đầu vào “I like soccer”, và nó đã hiểu rằng từ “I” là chủ ngữ và từ “like” là động từ, vì vậy nó đã mã hóa thông tin này trong đầu ra của nó cho các từ này. Bây giờ giả sử bộ giải mã đã dịch chủ ngữ, và nó nghĩ rằng nó nên dịch động từ tiếp theo. Để làm điều này, nó cần tìm động từ từ câu đầu vào. Điều này tương tự như tra cứu từ điển: giống như bộ mã hóa đã tạo một từ điển `{"subject": "They", "verb": "played", …}` và bộ giải mã muốn tra cứu giá trị tương ứng với khóa “verb”.

Tuy nhiên, mô hình không có các token rời rạc để biểu diễn các khóa (như “subject” hoặc “verb”); thay vào đó, nó có các biểu diễn vector hóa của các khái niệm này mà nó đã học được trong quá trình huấn luyện, vì vậy truy vấn mà nó sẽ sử dụng để tra cứu sẽ không khớp hoàn hảo với bất kỳ khóa nào trong từ điển. Giải pháp là tính toán một thước đo tương đồng giữa truy vấn và mỗi khóa trong từ điển, sau đó sử dụng hàm softmax để chuyển đổi các điểm tương đồng này thành các trọng số có tổng bằng 1. Như chúng ta đã thấy trước đó, đó chính xác là những gì lớp chú ý thực hiện. Nếu khóa biểu diễn động từ là tương tự nhất với truy vấn, thì trọng số của khóa đó sẽ gần bằng 1.

Tiếp theo, lớp chú ý tính toán tổng trọng số của các giá trị tương ứng: nếu trọng số của khóa “verb” gần bằng 1, thì tổng trọng số sẽ rất gần với biểu diễn của từ “played”. Đây là lý do tại sao các lớp `Keras Attention` và `AdditiveAttention` đều mong đợi một danh sách làm đầu vào, chứa hai hoặc ba mục: các truy vấn, các khóa, và tùy chọn các giá trị. Nếu bạn không truyền bất kỳ giá trị nào, thì chúng sẽ tự động bằng các khóa. Vì vậy, nhìn lại ví dụ mã trước đó, đầu ra của bộ giải mã là các truy vấn, và đầu ra của bộ mã hóa là cả các khóa và các giá trị. Đối với mỗi đầu ra bộ giải mã (tức là mỗi truy vấn), lớp chú ý trả về tổng trọng số của các đầu ra bộ mã hóa (tức là các khóa/giá trị) tương tự nhất với đầu ra của bộ giải mã.

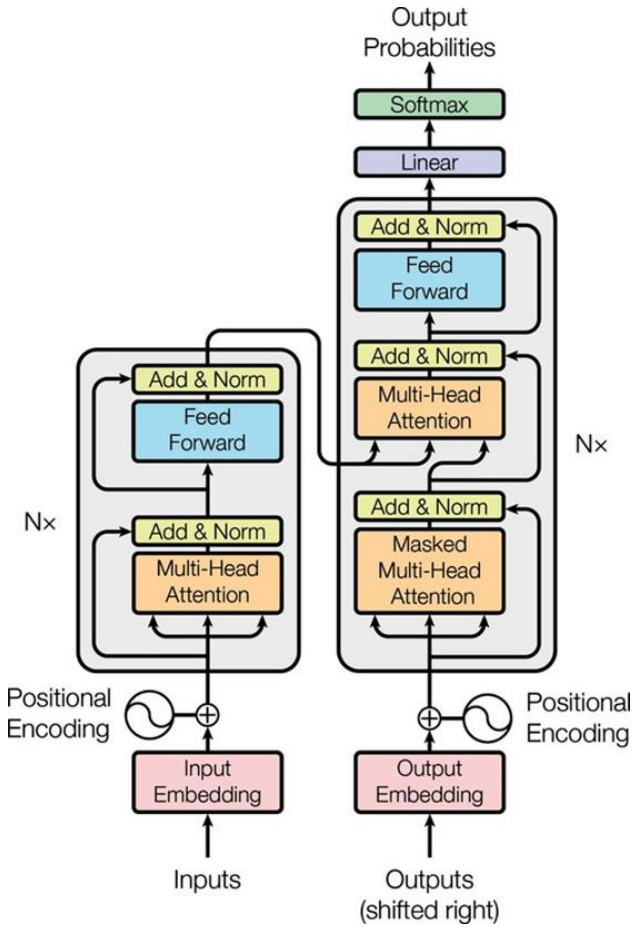
Điểm mấu chốt là cơ chế chú ý là một hệ thống truy xuất bộ nhớ có thể huấn luyện. Nó mạnh đến mức bạn thực sự có thể xây dựng các mô hình tiên tiến nhất chỉ bằng cách sử dụng cơ chế chú ý. Đi vào kiến trúc transformer.

Chủ đề “Attention Is All You Need: Kiến trúc Transformer gốc” nói về việc sử dụng cơ chế chú ý để cải thiện việc dịch máy thần kinh (NMT) mà không cần lớp hồi quy hoặc tích chập. Dưới đây là phần dịch của bạn:

Attention Is All You Need: Kiến trúc Transformer gốc

Trong một bài báo đột phá năm 2017, một nhóm các nhà nghiên cứu của Google đã gợi ý rằng “Attention Is All You Need”. Họ đã tạo ra một kiến trúc gọi là transformer, giúp cải thiện đáng kể trạng thái nghệ thuật trong NMT mà không sử dụng bất kỳ lớp hồi quy hoặc tích chập nào, chỉ sử dụng cơ chế chú ý (cộng với các lớp embedding, lớp dense, lớp chuẩn hóa và một vài thành phần khác). Vì mô hình không hồi quy, nó ít bị các vấn đề về đạo hàm biến mất hoặc bùng nổ như RNN, nó có thể được huấn luyện trong ít bước hơn, dễ dàng song song hóa trên nhiều GPU hơn và nó có thể nắm bắt các mẫu dài hạn tốt hơn RNN. Kiến trúc transformer gốc năm 2017 được thể hiện trong Hình 16-8.

Tóm lại, phần bên trái của Hình 16-8 là bộ mã hóa, và phần bên phải là bộ giải mã. Mỗi lớp embedding xuất ra một tensor 3D có hình dạng `[kích thước batch, độ dài chuỗi, kích thước embedding]`. Sau đó, các tensor dần dần được biến đổi khi chúng chảy qua transformer, nhưng hình dạng của chúng vẫn giữ nguyên.



Hình 16-8. Kiến trúc transformer gốc năm 2017

Nếu bạn sử dụng transformer cho NMT, thì trong quá trình huấn luyện, bạn phải cấp các câu tiếng Anh cho bộ mã hóa và các bản dịch tiếng Tây Ban Nha tương ứng cho bộ giải mã, với một token SOS bổ sung được chèn vào đầu mỗi câu. Tại thời điểm suy luận, bạn phải gọi transformer nhiều lần, tạo ra các bản dịch từng từ một và cấp các bản dịch một phần cho bộ giải mã ở mỗi vòng, giống như chúng ta đã làm trước đó trong hàm `translate()`.

Vai trò của bộ mã hóa là dần dần biến đổi các đầu vào — biểu diễn từ của câu tiếng Anh — cho đến khi biểu diễn của mỗi từ nắm bắt hoàn hảo ý nghĩa của từ đó, trong ngữ cảnh của câu. Ví dụ, nếu bạn cấp cho bộ mã hóa câu "I like soccer", thì từ "like" sẽ bắt đầu với một biểu diễn khá mơ hồ, vì từ này có thể có nghĩa khác nhau trong các ngữ cảnh khác nhau: hãy nghĩ đến "I like soccer" so với "It's like that". Nhưng sau khi đi qua bộ mã hóa, biểu diễn của từ phải nắm bắt được ý nghĩa chính xác của "like" trong câu đã cho (tức là yêu thích), cũng như bất kỳ thông tin nào khác có thể cần thiết cho việc dịch (ví dụ: nó là một động từ).

Vai trò của bộ giải mã là dần dần biến đổi mỗi biểu diễn từ trong câu đã dịch thành một biểu diễn từ của từ tiếp theo trong bản dịch. Ví dụ, nếu câu cần dịch là "I like soccer", và câu đầu vào của bộ giải mã là "<SOS> me gusta el fútbol", thì sau khi đi qua bộ giải mã, biểu diễn từ của từ "el" sẽ được biến đổi thành một biểu diễn của từ "fútbol". Tương tự, biểu diễn của từ "fútbol" sẽ được biến đổi thành một biểu diễn của token EOS.

Sau khi đi qua bộ giải mã, mỗi biểu diễn từ đi qua một lớp `Dense` cuối cùng với hàm kích hoạt softmax, hy vọng sẽ xuất ra xác suất cao cho từ tiếp theo đúng và xác suất thấp cho tất cả các từ khác. Câu được dự đoán phải là "me gusta el fútbol <EOS>".

Đó là bức tranh tổng thể; bây giờ chúng ta hãy đi sâu vào Hình 16-8 một cách chi tiết hơn:

- Đầu tiên, lưu ý rằng cả bộ mã hóa và bộ giải mã đều chứa các mô-đun được xếp chồng N lần. Trong bài báo, N = 6. Các đầu ra cuối cùng của toàn bộ chồng bộ mã hóa được cấp cho bộ giải mã ở mỗi trong số N cấp độ này.
- Phóng to hơn, bạn có thể thấy rằng bạn đã quen thuộc với hầu hết các thành phần: có hai lớp embedding; một số kết nối bỏ qua (skip connection), mỗi kết nối theo sau bởi một lớp chuẩn hóa lớp; một số mô-đun truyền thẳng (feedforward) bao gồm hai lớp dense mỗi mô-đun (lớp đầu tiên sử dụng hàm kích hoạt ReLU, lớp thứ hai không có hàm kích hoạt); và cuối cùng lớp đầu ra là một lớp dense sử dụng hàm kích hoạt softmax. Bạn cũng có thể thêm một chút dropout sau các lớp chú ý và các mô-đun truyền thẳng, nếu cần. Vì tất cả các lớp này là phân phối thời gian (time-distributed), mỗi từ được xử lý độc lập với tất cả các từ khác. Nhưng làm thế nào chúng ta có thể dịch một câu bằng cách nhìn vào các từ hoàn toàn riêng biệt? Chà, chúng ta không thể, đó là lúc các thành phần mới xuất hiện:
    - Lớp chú ý đa đầu (multi-head attention) của bộ mã hóa cập nhật mỗi biểu diễn từ bằng cách chú ý đến (tức là tập trung vào) tất cả các từ khác trong cùng câu. Đó là nơi biểu diễn mơ hồ của từ "like" trở thành một biểu diễn phong phú hơn và chính xác hơn, nắm bắt ý nghĩa chính xác của nó trong câu đã cho. Chúng ta sẽ thảo luận chính xác cách hoạt động này ngay sau đây.

- Lớp chú ý đa đầu được che (masked multi-head attention) của bộ giải mã làm điều tương tự, nhưng khi nó xử lý một từ, nó không chú ý đến các từ nằm sau nó: đó là một lớp nhân quả (causal layer). Ví dụ, khi nó xử lý từ “gusta”, nó chỉ chú ý đến các từ “<SOS> me gusta”, và nó bỏ qua các từ “el fútbol” (nếu không đó sẽ là gian lận).
- Lớp chú ý đa đầu phía trên của bộ giải mã là nơi bộ giải mã chú ý đến các từ trong câu tiếng Anh. Đây được gọi là cross-attention, không phải self-attention trong trường hợp này. Ví dụ, bộ giải mã có thể sẽ chú ý kỹ đến từ “soccer” khi nó xử lý từ “el” và biến đổi biểu diễn của nó thành một biểu diễn của từ “fútbol”.
- Các mã hóa vị trí (positional encodings) là các vector dày đặc (rất giống với embedding từ) đại diện cho vị trí của mỗi từ trong câu. Mã hóa vị trí thứ n được thêm vào embedding từ của từ thứ n trong mỗi câu. Điều này là cần thiết vì tất cả các lớp trong kiến trúc transformer bỏ qua vị trí từ: nếu không có mã hóa vị trí, bạn có thể xáo trộn các chuỗi đầu vào, và nó sẽ chỉ xáo trộn các chuỗi đầu ra theo cùng một cách. Rõ ràng, thứ tự các từ rất quan trọng, đó là lý do tại sao chúng ta cần cung cấp thông tin vị trí cho transformer bằng cách nào đó: thêm mã hóa vị trí vào biểu diễn từ là một cách tốt để đạt được điều này.

Hãy đi sâu vào các thành phần mới của kiến trúc transformer một cách chi tiết hơn, bắt đầu với các mã hóa vị trí.

Mã hóa vị trí

Mã hóa vị trí là một vector dày đặc mã hóa vị trí của một từ trong một câu: mã hóa vị trí thứ i được thêm vào embedding từ của từ thứ i trong câu. Cách dễ nhất để triển khai điều này là sử dụng lớp `Embedding` và làm cho nó mã hóa tất cả các vị trí từ 0 đến độ dài chuỗi tối đa trong batch, sau đó thêm kết quả vào các embedding từ. Các quy tắc quảng bá (broadcasting) sẽ đảm bảo rằng các mã hóa vị trí được áp dụng cho mọi chuỗi đầu vào. Ví dụ, đây là cách thêm mã hóa vị trí vào đầu vào của bộ mã hóa và bộ giải mã:

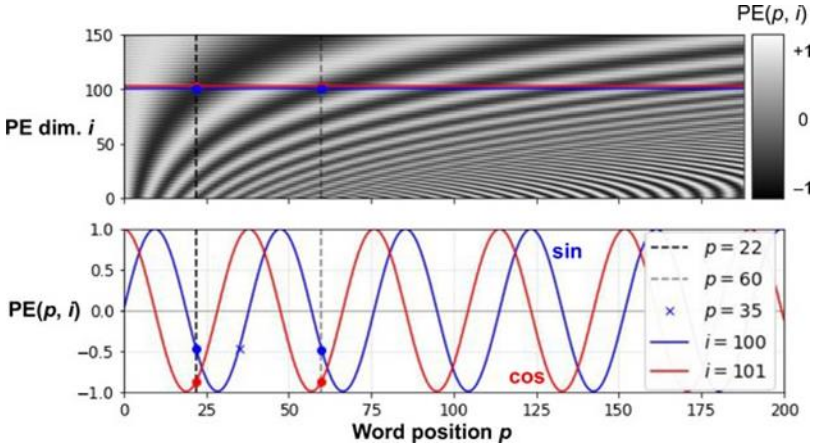
```
max_length = 50 # độ dài tối đa trong toàn bộ tập huấn luyện
embed_size = 128
pos_embed_layer = tf.keras.layers.Embedding(max_length, embed_size)
batch_max_len_enc = tf.shape(encoder_embeddings)[1]
encoder_in = encoder_embeddings +
pos_embed_layer(tf.range(batch_max_len_enc))
batch_max_len_dec = tf.shape(decoder_embeddings)[1]
decoder_in = decoder_embeddings +
pos_embed_layer(tf.range(batch_max_len_dec))
```

Lưu ý rằng việc triển khai này giả định rằng các embedding được biểu diễn dưới dạng các tensor thông thường, không phải các tensor lởm chởm. Bộ mã hóa và bộ giải mã chia sẻ cùng một lớp `Embedding` cho các mã hóa vị trí, vì chúng có cùng kích thước embedding (đây thường là trường hợp).

Thay vì sử dụng các mã hóa vị trí có thể huấn luyện, các tác giả của bài báo transformer đã chọn sử dụng các mã hóa vị trí cố định, dựa trên các hàm sin và cosin ở các tần số khác nhau. Ma trận mã hóa vị trí P được định nghĩa trong Phương trình 16-2 và được biểu diễn ở trên cùng của Hình 16-9 (chuyển vị), trong đó Pp,i là thành phần thứ i của mã hóa cho từ nằm ở vị trí thứ p trong câu.

Phương trình 16-2. Mã hóa vị trí sin/cosin

$$P_{p,i} = \begin{cases} \sin\left(p/10000^{i/d}\right) & \text{if } i \text{ is even} \\ \cos\left(p/10000^{(i-1)/d}\right) & \text{if } i \text{ is odd} \end{cases}$$



Hình 16-9. Ma trận mã hóa vị trí sin/cosin (chuyển vị, trên cùng) với tiêu điểm vào hai giá trị của i (dưới cùng)

Giải pháp này có thể cho cùng hiệu suất như các mã hóa vị trí có thể huấn luyện, và nó có thể mở rộng đến các câu dài tùy ý mà không cần thêm bất kỳ tham số nào vào mô hình (tuy nhiên, khi có một lượng lớn dữ liệu tiền huấn luyện, các mã hóa vị trí có thể huấn luyện thường được ưu tiên). Sau khi các mã hóa vị trí này được thêm vào các embedding từ, phần còn lại của mô hình có quyền truy cập vào vị trí tuyệt đối của mỗi từ trong câu vì có một mã hóa vị trí duy nhất cho mỗi vị trí (ví dụ: mã hóa vị trí cho từ nằm ở vị trí thứ 22 trong một câu được biểu diễn bằng đường nét đứt dọc ở phía trên bên trái của Hình 16-9, và bạn có thể thấy rằng nó là duy nhất đối với vị trí đó). Hơn nữa, việc lựa chọn các hàm dao động (sin và cosin) giúp mô hình có thể học cả các vị trí tương đối. Ví dụ, các từ cách nhau 38 từ (ví dụ: ở vị trí p = 22 và p = 60) luôn có cùng giá trị mã hóa vị trí trong các chiều mã hóa i = 100 và i = 101, như bạn có thể thấy trong Hình 16-9. Điều này giải thích tại sao chúng ta cần cả sin và cosin cho mỗi tần số: nếu chúng ta chỉ sử dụng sin (sóng xanh ở i = 100), mô hình sẽ không thể phân biệt các vị trí p = 22 và p = 35 (được đánh dấu bằng dấu thập).

Không có lớp `PositionalEncoding` trong TensorFlow, nhưng không quá khó để tạo một lớp. Vì lý do hiệu quả, chúng ta tính toán trước ma trận mã hóa vị trí trong hàm tạo. Phương thức `call()` chỉ cắt ma trận mã hóa này theo độ dài tối đa của chuỗi đầu vào, và nó thêm chúng vào các đầu vào. Chúng ta cũng đặt `supports_masking=True` để lan truyền mặt nạ tự động của đầu vào đến lớp tiếp theo:

```
import numpy as np

class PositionalEncoding(tf.keras.layers.Layer):
    def __init__(self, max_length, embed_size, dtype=tf.float32, **kwargs):
        super().__init__(dtype=dtype, **kwargs)
        assert embed_size % 2 == 0, "embed_size must be even"
        p, i = np.meshgrid(np.arange(max_length),
                           2 * np.arange(embed_size // 2))
        pos_emb = np.empty((1, max_length, embed_size))
        pos_emb[0, :, ::2] = np.sin(p / 10_000 ** (i / embed_size)).T
        pos_emb[0, :, 1::2] = np.cos(p / 10_000 ** (i / embed_size)).T
        self.pos_encodings = tf.constant(pos_emb.astype(self.dtype))
        self.supports_masking = True

    def call(self, inputs):
        batch_max_length = tf.shape(inputs)[1]
        return inputs + self.pos_encodings[:, :batch_max_length]
```

Hãy sử dụng lớp này để thêm mã hóa vị trí vào đầu vào của bộ mã hóa:

```
pos_embed_layer = PositionalEncoding(max_length, embed_size)
encoder_in = pos_embed_layer(encoder_embeddings)
decoder_in = pos_embed_layer(decoder_embeddings)
```

Bây giờ chúng ta hãy xem sâu hơn vào trái tim của mô hình transformer, tại lớp chú ý đa đầu.

Chú ý đa đầu (Multi-head attention)

Để hiểu cách hoạt động của lớp chú ý đa đầu, trước tiên chúng ta phải hiểu lớp chú ý tích vô hướng có tỷ lệ (scaled dot-product attention), mà nó dựa vào. Phương trình của nó được thể hiện trong Phương trình 16-3, dưới dạng vector hóa. Nó giống như Luong attention, ngoại trừ một hệ số tỷ lệ.

Phương trình 16-3. Chú ý tích vô hướng có tỷ lệ

$$\text{Attention}\left(\mathbf{Q}, \mathbf{K}, \mathbf{V}\right) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\intercal}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

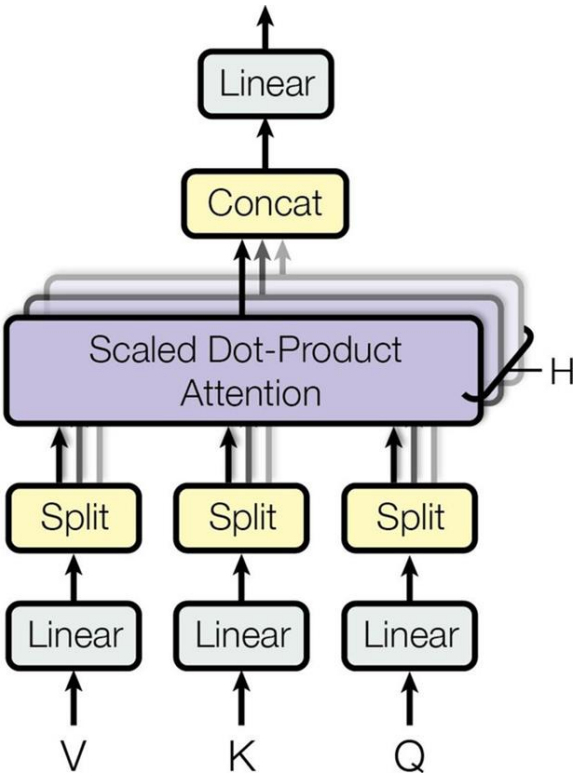
Trong phương trình này:

- Q là ma trận chứa một hàng cho mỗi truy vấn. Hình dạng của nó là `[nqueries, dkeys]`, trong đó `nqueries` là số lượng truy vấn và `dkeys` là số chiều của mỗi truy vấn và mỗi khóa.
- K là ma trận chứa một hàng cho mỗi khóa. Hình dạng của nó là `[nkeys, dkeys]`, trong đó `nkeys` là số lượng khóa và giá trị.
- V là ma trận chứa một hàng cho mỗi giá trị. Hình dạng của nó là `[nkeys, dvalues]`, trong đó `dvalues` là số chiều của mỗi giá trị.

- Hình dạng của $QK^\$ là `[nqueries, nkeys]`: nó chứa một điểm tương đồng cho mỗi cặp truy vấn/khóa. Để ngăn ma trận này trở nên quá lớn, chuỗi đầu vào không được quá dài (chúng ta sẽ thảo luận cách khắc phục hạn chế này sau trong chương này). Đầu ra của hàm softmax có cùng hình dạng, nhưng tất cả các hàng tổng cộng bằng 1.
- Đầu ra cuối cùng có hình dạng `[nqueries, dvalues]`: có một hàng cho mỗi truy vấn, trong đó mỗi hàng biểu diễn kết quả truy vấn (tổng trọng số của các giá trị).
- Hệ số tỷ lệ $1/\left(\sqrt{d_{keys}}\right)$ giảm tỷ lệ các điểm tương đồng để tránh làm bão hòa hàm softmax, điều này sẽ dẫn đến các gradient rất nhỏ.
- Có thể che một số cặp khóa/giá trị bằng cách thêm một giá trị âm rất lớn vào các điểm tương đồng tương ứng, ngay trước khi tính toán softmax. Điều này hữu ích trong lớp chú ý đa đầu được che.
- Nếu bạn đặt `use_scale=True` khi tạo một lớp `tf.keras.layers.Attention`, thì nó sẽ tạo ra một tham số bổ sung cho phép lớp học cách giảm tỷ lệ các điểm tương đồng một cách hợp lý. Chú ý tích vô hướng có tỷ lệ được sử dụng trong mô hình transformer gần như giống hệt, ngoại trừ việc nó luôn giảm tỷ lệ các điểm tương đồng bằng cùng một hệ số, $1/\left(\sqrt{d_{keys}}\right)$.

Lưu ý rằng đầu vào của lớp `Attention` giống như Q, K và V, ngoại trừ một chiều batch bổ sung (chiều đầu tiên). Bên trong, lớp tính toán tất cả các điểm chú ý cho tất cả các câu trong batch chỉ bằng một lần gọi `tf.matmul(queries, keys)`: điều này làm cho nó cực kỳ hiệu quả. Thật vậy, trong TensorFlow, nếu A và B là các tensor có nhiều hơn hai chiều — ví dụ, có hình dạng `[2, 3, 4, 5]` và `[2, 3, 5, 6]` tương ứng — thì `tf.matmul(A, B)` sẽ xử lý các tensor này như các mảng 2 × 3 trong đó mỗi ô chứa một ma trận, và nó sẽ nhân các ma trận tương ứng: ma trận ở hàng thứ i và cột thứ j trong A sẽ được nhân với ma trận ở hàng thứ i và cột thứ j trong B. Vì tích của một ma trận 4 × 5 với một ma trận 5 × 6 là một ma trận 4 × 6, `tf.matmul(A, B)` sẽ trả về một mảng có hình dạng `[2, 3, 4, 6]`.

Bây giờ chúng ta đã sẵn sàng xem xét lớp chú ý đa đầu. Kiến trúc của nó được thể hiện trong Hình 16-10.



Hình 16-10. Kiến trúc lớp chú ý đa đầu

Như bạn có thể thấy, nó chỉ là một tập hợp các lớp chú ý tích vô hướng có tỷ lệ, mỗi lớp được đặt trước bởi một phép biến đổi tuyến tính của các giá trị, khóa và truy vấn (tức là một lớp dense phân phối thời gian không có hàm kích hoạt). Tất cả các đầu ra chỉ đơn giản được nối lại, và chúng đi qua một phép biến đổi tuyến tính cuối cùng (một lần nữa, phân phối thời gian).

Nhưng tại sao? Trực giác đằng sau kiến trúc này là gì? Chà, hãy xem xét lại từ “like” trong câu “I like soccer”. Bộ mã hóa đủ thông minh để mã hóa thực tế rằng nó là một động từ. Nhưng biểu diễn từ cũng bao gồm vị trí của nó trong văn bản, nhờ các mã hóa vị trí, và nó có lẽ bao gồm nhiều đặc điểm khác hữu ích cho bản dịch của nó, chẳng hạn như thực tế là nó ở thì hiện tại đơn. Tóm lại, biểu diễn từ mã hóa nhiều đặc điểm khác nhau của từ. Nếu chúng ta chỉ sử dụng một lớp chú ý tích vô hướng có tỷ lệ duy nhất, chúng ta sẽ chỉ có thể truy vấn tất cả các đặc điểm này trong một lần.

Đây là lý do tại sao lớp chú ý đa đầu áp dụng nhiều phép biến đổi tuyến tính khác nhau của các giá trị, khóa và truy vấn: điều này cho phép mô hình áp dụng nhiều phép chiếu khác nhau của biểu diễn từ vào các không gian con khác nhau, mỗi không gian con tập trung vào một tập hợp con các đặc điểm của từ. Có lẽ một trong các lớp tuyến tính sẽ chiếu biểu diễn từ vào một không gian con nơi tất cả những gì còn lại là thông tin từ đó là một động từ, một lớp tuyến tính khác sẽ chỉ trích xuất thực tế rằng nó ở thì hiện tại đơn, v.v. Sau đó, các lớp chú ý tích vô hướng có tỷ lệ triển khai giai đoạn tra cứu, và cuối cùng chúng ta nối tất cả các kết quả và chiếu chúng trở lại không gian ban đầu.

Keras bao gồm một lớp `tf.keras.layers.MultiHeadAttention`, vì vậy bây giờ chúng ta có mọi thứ cần thiết để xây dựng phần còn lại của transformer. Hãy bắt đầu với bộ mã hóa hoàn chỉnh, giống hệt như trong Hình 16-8, ngoại trừ chúng ta sử dụng một chồng hai khối (N = 2) thay vì sáu, vì chúng ta không có một tập huấn luyện lớn, và chúng ta cũng thêm một chút dropout:

```
N = 2 # thay vì 6
num_heads = 8
dropout_rate = 0.1
n_units = 128 # cho lớp dense đầu tiên trong mỗi khối feedforward
encoder_pad_mask = tf.math.not_equal(encoder_input_ids, 0)[:, tf.newaxis]
Z = encoder_in

for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=encoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))
    skip = Z
    Z = tf.keras.layers.Dense(n_units, activation="relu")(Z)
    Z = tf.keras.layers.Dense(embed_size)(Z)
    Z = tf.keras.layers.Dropout(dropout_rate)(Z)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))
```

Đoạn mã này hầu hết là dễ hiểu, ngoại trừ một điều: che mặt (masking). Tại thời điểm viết bài, lớp `MultiHeadAttention` không hỗ trợ che mặt tự động, vì vậy chúng ta phải xử lý thủ công. Làm thế nào chúng ta có thể làm điều đó?

Lớp `MultiHeadAttention` chấp nhận một đối số `attention_mask`, là một tensor Boolean có hình dạng `[kích thước batch, độ dài truy vấn tối đa, độ dài giá trị tối đa]`: đối với mỗi token trong mỗi chuỗi truy vấn, mặt nạ này cho biết token nào trong chuỗi giá trị tương ứng nên được chú ý. Chúng ta muốn nói với lớp `MultiHeadAttention` bỏ qua tất cả các token đệm trong các giá trị. Vì vậy, trước tiên, chúng ta tính toán mặt nạ đệm bằng cách sử dụng `tf.math.not_equal(encoder_input_ids, 0)`. Điều này trả về một tensor Boolean có hình dạng `[kích thước batch, độ dài chuỗi tối đa]`. Sau đó, chúng ta chèn một trục thứ hai bằng cách sử dụng `[:, tf.newaxis]`, để nhận được một mặt nạ có hình dạng `[kích thước batch, 1, độ dài chuỗi tối đa]`. Điều này cho phép chúng ta sử dụng mặt nạ này làm `attention_mask` khi gọi lớp `MultiHeadAttention`: nhờ tính năng broadcasting, cùng một mặt nạ sẽ được sử dụng cho tất cả các token trong mỗi truy vấn. Bằng cách này, các token đệm trong các giá trị sẽ được bỏ qua một cách chính xác.

Tuy nhiên, lớp sẽ tính toán đầu ra cho mọi token truy vấn, bao gồm cả các token đệm. Chúng ta cần che đầu ra tương ứng với các token đệm này. Nhớ lại rằng chúng ta đã sử dụng `mask_zero` trong các lớp `Embedding`, và chúng ta đã đặt `supports_masking` thành `True` trong lớp `PositionalEncoding`, vì vậy mặt nạ tự động đã được lan truyền đến đầu vào của

lớp `MultiHeadAttention` (`encoder_in`). Chúng ta có thể tận dụng điều này trong kết nối bỏ qua: thực tế, lớp `Add` hỗ trợ che mặt tự động, vì vậy khi chúng ta thêm `Z` và `skip` (ban đầu bằng `encoder_in`), đầu ra sẽ tự động được che chính xác. Chà! Che mặt đòi hỏi nhiều giải thích hơn là mã.

Bây giờ đến bộ giải mã! Một lần nữa, che mặt sẽ là phần khó khăn duy nhất, vì vậy hãy bắt đầu với điều đó. Lớp chú ý đa đầu đầu tiên là một lớp tự chú ý (self-attention layer), giống như trong bộ mã hóa, nhưng nó là một lớp chú ý đa đầu được che, nghĩa là nó có tính nhân quả: nó phải bỏ qua tất cả các token trong tương lai. Vì vậy, chúng ta cần hai mặt nạ: một mặt nạ đệm và một mặt nạ nhân quả. Hãy tạo chúng:

```
decoder_pad_mask = tf.math.not_equal(decoder_input_ids, 0)[:, tf.newaxis]
causal_mask = tf.linalg.band_part( # tạo một ma trận tam giác dưới
    tf.ones((batch_max_len_dec, batch_max_len_dec), tf.bool), -1, 0)
```

Mặt nạ đệm chính xác giống như mặt nạ chúng ta đã tạo cho bộ mã hóa, ngoại trừ nó dựa trên đầu vào của bộ giải mã thay vì của bộ mã hóa. Mặt nạ nhân quả được tạo bằng hàm `tf.linalg.band_part()`, hàm này nhận một tensor và trả về một bản sao với tất cả các giá trị bên ngoài một dải chéo được đặt thành 0. Với các đối số này, chúng ta nhận được một ma trận vuông có kích thước `batch_max_len_dec` (độ dài tối đa của chuỗi đầu vào trong batch), với các số 1 ở tam giác dưới bên trái và các số 0 ở phía trên bên phải. Nếu chúng ta sử dụng mặt nạ này làm mặt nạ chú ý, chúng ta sẽ nhận được chính xác những gì chúng ta muốn: token truy vấn đầu tiên sẽ chỉ chú ý đến token giá trị đầu tiên, token thứ hai sẽ chỉ chú ý đến hai token đầu tiên, token thứ ba sẽ chỉ chú ý đến ba token đầu tiên, v.v. Nói cách khác, các token truy vấn không thể chú ý đến bất kỳ token giá trị nào trong tương lai.

Bây giờ hãy xây dựng bộ giải mã:

```
encoder_outputs = Z # hãy lưu các đầu ra cuối cùng của bộ mã hóa
Z = decoder_in # bộ giải mã bắt đầu với đầu vào riêng của nó

for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=causal_mask & decoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=encoder_outputs, attention_mask=encoder_pad_mask)
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))
    skip = Z
    Z = tf.keras.layers.Dense(n_units, activation="relu")(Z)
    Z = tf.keras.layers.Dense(embed_size)(Z)
```

```
    Z = tf.keras.layers.LayerNormalization()(tf.keras.layers.Add()([Z,
skip]))
```

Đối với lớp chú ý đầu tiên, chúng ta sử dụng `causal_mask & decoder_pad_mask` để che cả token đệm và token tương lai. Mặt nạ nhân quả chỉ có hai chiều: nó thiếu chiều batch, nhưng không sao vì broadcasting đảm bảo rằng nó được sao chép trên tất cả các instance trong batch.

Đối với lớp chú ý thứ hai, không có gì đặc biệt. Điều duy nhất cần lưu ý là chúng ta đang sử dụng `encoder_pad_mask`, không phải `decoder_pad_mask`, vì lớp chú ý này sử dụng đầu ra cuối cùng của bộ mã hóa làm giá trị của nó.

Chúng ta gần xong rồi. Chúng ta chỉ cần thêm lớp đầu ra cuối cùng, tạo mô hình, biên dịch nó và huấn luyện nó:

```
Y_proba = tf.keras.layers.Dense(vocab_size, activation="softmax")(Z)
model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                       outputs=[Y_proba])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
model.fit((X_train, X_train_dec), Y_train, epochs=10,
          validation_data=((X_valid, X_valid_dec), Y_valid))
```

Chúc mừng! Bạn đã xây dựng một transformer hoàn chỉnh từ đầu, và huấn luyện nó để dịch tự động. Điều này đang trở nên khá nâng cao!

Nhưng lĩnh vực này không dừng lại ở đó. Bây giờ chúng ta hãy khám phá một số tiến bộ gần đây.

Tuyết lở các mô hình Transformer

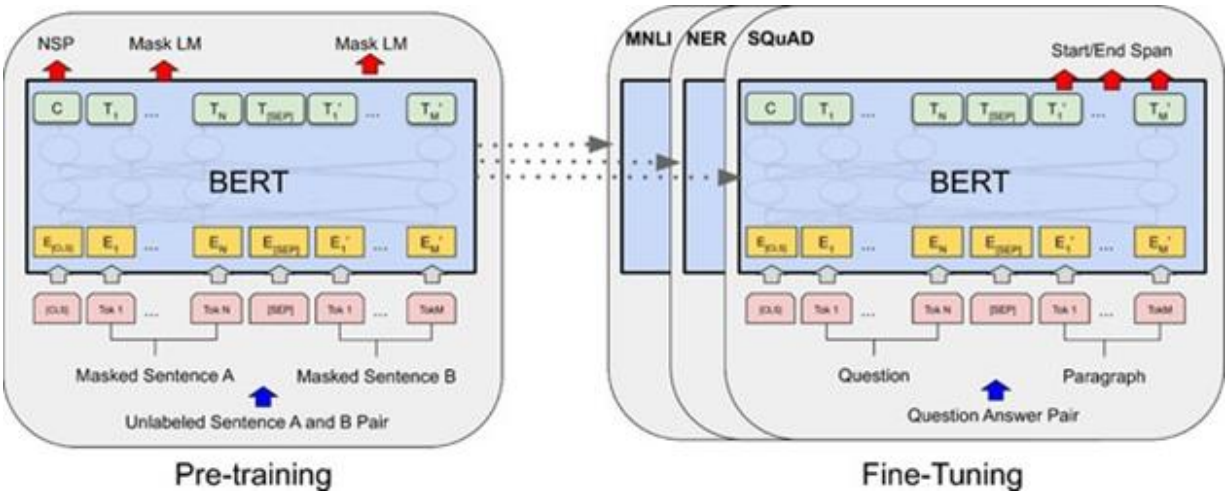
Năm 2018 được gọi là “thời điểm ImageNet cho NLP”. Kể từ đó, sự tiến bộ thật đáng kinh ngạc, với các kiến trúc dựa trên transformer ngày càng lớn hơn được huấn luyện trên các tập dữ liệu khổng lồ.

Đầu tiên, bài báo GPT của Alec Radford và các nhà nghiên cứu khác của OpenAI một lần nữa chứng minh hiệu quả của việc tiền huấn luyện không giám sát, giống như các bài báo ELMo và ULMFiT trước đó, nhưng lần này sử dụng kiến trúc giống transformer. Các tác giả đã tiền huấn luyện một kiến trúc lớn nhưng khá đơn giản bao gồm một chồng 12 mô-đun transformer chỉ sử dụng các lớp chú ý đa đầu được che, giống như trong bộ giải mã transformer gốc. Họ đã huấn luyện nó trên một tập dữ liệu rất lớn, sử dụng cùng kỹ thuật tự hồi quy mà chúng ta đã sử dụng cho char-RNN kiểu Shakespeare của mình: chỉ cần dự đoán token tiếp theo. Đây là một dạng học tự giám sát. Sau đó, họ đã tinh chỉnh nó trên các tác vụ ngôn ngữ khác nhau, chỉ sử dụng các điều chỉnh nhỏ cho mỗi tác vụ. Các tác vụ khá đa dạng: chúng bao gồm phân loại văn bản, suy luận (liệu câu A có áp đặt, liên quan hoặc ngụ ý câu B như một hệ quả tất yếu hay không), tương đồng (ví dụ: “Nice weather today” rất giống với “It is sunny”), và trả lời câu hỏi (cho một vài đoạn văn bản cung cấp ngữ cảnh, mô hình phải trả lời một số câu hỏi trắc nghiệm).

Sau đó, bài báo BERT của Google đã ra đời: nó cũng chứng minh hiệu quả của việc tiền huấn luyện tự giám sát trên một corpus lớn, sử dụng một kiến trúc tương tự GPT nhưng chỉ với các lớp chú ý đa đầu không che, giống như trong bộ mã hóa transformer gốc. Điều này có nghĩa là mô hình tự nhiên là hai chiều; do đó chữ B trong BERT (Bidirectional Encoder Representations from Transformers - Biểu diễn mã hóa hai chiều từ Transformers). Quan trọng nhất, các tác giả đã đề xuất hai tác vụ tiền huấn luyện giải thích hầu hết sức mạnh của mô hình:

- **Mô hình ngôn ngữ được che (MLM)** Mỗi từ trong một câu có 15% xác suất bị che, và mô hình được huấn luyện để dự đoán các từ bị che. Ví dụ, nếu câu gốc là "She had fun at the birthday party", thì mô hình có thể được cung cấp câu "She `<mask>` fun at the `<mask>` party" và nó phải dự đoán các từ "had" và "birthday" (các đầu ra khác sẽ bị bỏ qua). Để chính xác hơn, mỗi từ được chọn có 80% cơ hội bị che, 10% cơ hội được thay thế bằng một từ ngẫu nhiên (để giảm sự khác biệt giữa tiền huấn luyện và tinh chỉnh, vì mô hình sẽ không thấy các token `<mask>` trong quá trình tinh chỉnh), và 10% cơ hội được giữ nguyên (để làm lệch mô hình về câu trả lời đúng).
- **Dự đoán câu tiếp theo (NSP)** Mô hình được huấn luyện để dự đoán liệu hai câu có liên tiếp nhau hay không. Ví dụ, nó nên dự đoán rằng "The dog sleeps" và "It snores loudly" là các câu liên tiếp, trong khi "The dog sleeps" và "The Earth orbits the Sun" không liên tiếp. Nghiên cứu sau này cho thấy rằng NSP không quan trọng như ban đầu nghĩ, vì vậy nó đã bị loại bỏ trong hầu hết các kiến trúc sau này.

Mô hình được huấn luyện trên hai tác vụ này đồng thời (xem Hình 16-11). Đối với tác vụ NSP, các tác giả đã chèn một token lớp (`<CLS>`) vào đầu mỗi đầu vào, và token đầu ra tương ứng đại diện cho dự đoán của mô hình: câu B theo sau câu A, hoặc không. Hai câu đầu vào được nối lại, chỉ được phân tách bằng một token phân tách đặc biệt (`<SEP>`), và chúng được cấp làm đầu vào cho mô hình. Để giúp mô hình biết mỗi token đầu vào thuộc về câu nào, một embedding phân đoạn được thêm vào trên mỗi embedding vị trí của token: chỉ có hai embedding phân đoạn có thể có, một cho câu A và một cho câu B. Đối với tác vụ MLM, một số từ đầu vào bị che (như chúng ta vừa thấy) và mô hình cố gắng dự đoán những từ đó là gì. Hàm mất mát chỉ được tính toán trên dự đoán NSP và các token bị che, không phải trên các token không che.



Hình 16-11. Quy trình huấn luyện và tinh chỉnh BERT

Sau giai đoạn tiền huấn luyện không giám sát này trên một corpus văn bản rất lớn, mô hình sau đó được tinh chỉnh trên nhiều tác vụ khác nhau, thay đổi rất ít cho mỗi tác vụ. Ví dụ, đối với phân loại văn bản như phân tích cảm xúc, tất cả các token đầu ra đều bị bỏ qua ngoại trừ token đầu tiên, tương ứng với token lớp, và một lớp đầu ra mới thay thế lớp trước đó, vốn chỉ là một lớp phân loại nhị phân cho NSP.

Vào tháng 2 năm 2019, chỉ vài tháng sau khi BERT được xuất bản, Alec Radford, Jeffrey Wu và các nhà nghiên cứu khác của OpenAI đã xuất bản bài báo GPT-2, đề xuất một kiến trúc rất giống GPT, nhưng lớn hơn nữa (với hơn 1,5 tỷ tham số!). Các nhà nghiên cứu đã chỉ ra rằng mô hình GPT mới và cải tiến có thể thực hiện học zero-shot (ZSL), nghĩa là nó có thể đạt được hiệu suất tốt trên nhiều tác vụ mà không cần tinh chỉnh. Đây chỉ là sự khởi đầu của một cuộc đua hướng tới các mô hình ngày càng lớn hơn: Google's Switch Transformers (được giới thiệu vào tháng 1 năm 2021) đã sử dụng 1 nghìn tỷ tham số, và ngay sau đó các mô hình lớn hơn nhiều đã ra đời, chẳng hạn như mô hình Wu Dao 2.0 của Học viện Trí tuệ Nhân tạo Bắc Kinh (BAII), được công bố vào tháng 6 năm 2021.

Một hệ quả đáng tiếc của xu hướng hướng tới các mô hình khổng lồ này là chỉ các tổ chức được tài trợ tốt mới có đủ khả năng huấn luyện các mô hình như vậy: nó có thể dễ dàng tốn hàng trăm nghìn đô la hoặc hơn. Và năng lượng cần thiết để huấn luyện một mô hình duy nhất tương ứng với mức tiêu thụ điện của một hộ gia đình Mỹ trong vài năm; nó hoàn toàn không thân thiện với môi trường. Nhiều mô hình trong số này quá lớn đến mức không thể sử dụng trên phần cứng thông thường: chúng sẽ không vừa RAM, và chúng sẽ chậm khủng khiếp. Cuối cùng, một số quá tốn kém đến mức chúng không được phát hành công khai.

May mắn thay, các nhà nghiên cứu tài tình đang tìm ra những cách mới để giảm kích thước transformer và làm cho chúng hiệu quả hơn về dữ liệu. Ví dụ, mô hình DistilBERT, được giới thiệu vào tháng 10 năm 2019 bởi Victor Sanh et al. từ Hugging Face, là một mô hình transformer nhỏ và nhanh dựa trên BERT. Nó có sẵn trên trung tâm mô hình xuất sắc của Hugging Face, cùng với hàng nghìn mô hình khác — bạn sẽ thấy một ví dụ sau trong chương này. DistilBERT được huấn luyện bằng cách sử dụng chưng cất (do đó có tên): điều này có

nghĩa là truyền kiến thức từ mô hình giáo viên sang mô hình học sinh, thường nhỏ hơn nhiều so với mô hình giáo viên. Điều này thường được thực hiện bằng cách sử dụng các xác suất dự đoán của giáo viên cho mỗi instance huấn luyện làm mục tiêu cho học sinh. Đáng ngạc nhiên, chưng cất thường hoạt động tốt hơn là huấn luyện học sinh từ đầu trên cùng một tập dữ liệu với giáo viên! Thật vậy, học sinh được hưởng lợi từ các nhãn sắc thái hơn của giáo viên.

Nhiều kiến trúc transformer khác đã ra đời sau BERT, gần như hàng tháng, thường cải thiện trạng thái nghệ thuật trên tất cả các tác vụ NLP: XLNet (tháng 6 năm 2019), RoBERTa (tháng 7 năm 2019), StructBERT (tháng 8 năm 2019), ALBERT (tháng 9 năm 2019), T5 (tháng 10 năm 2019), ELECTRA (tháng 3 năm 2020), GPT-3 (tháng 5 năm 2020), DeBERTa (tháng 6 năm 2020), Switch Transformers (tháng 1 năm 2021), Wu Dao 2.0 (tháng 6 năm 2021), Gopher (tháng 12 năm 2021), GPT-NeoX-20B (tháng 2 năm 2022), Chinchilla (tháng 3 năm 2022), OPT (tháng 5 năm 2022), và danh sách cứ tiếp tục dài. Mỗi mô hình này đều mang đến những ý tưởng và kỹ thuật mới, nhưng tôi đặc biệt thích bài báo T5 của các nhà nghiên cứu Google: nó định hình tất cả các tác vụ NLP dưới dạng văn bản-thành-văn bản, sử dụng một transformer mã hóa-giải mã. Ví dụ, để dịch “I like soccer” sang tiếng Tây Ban Nha, bạn chỉ cần gọi mô hình với câu đầu vào “translate English to Spanish: I like soccer” và nó xuất ra “me gusta el fútbol”. Để tóm tắt một đoạn văn, bạn chỉ cần nhập “summarize:” theo sau đoạn văn, và nó xuất ra bản tóm tắt. Để phân loại, bạn chỉ cần thay đổi tiền tố thành “classify:” và mô hình xuất ra tên lớp, dưới dạng văn bản. Điều này đơn giản hóa việc sử dụng mô hình, và nó cũng giúp có thể tiền huấn luyện nó trên nhiều tác vụ hơn.

Cuối cùng nhưng không kém phần quan trọng, vào tháng 4 năm 2022, các nhà nghiên cứu Google đã sử dụng một nền tảng huấn luyện quy mô lớn mới có tên Pathways (chúng ta sẽ thảo luận ngắn gọn trong Chương 19) để huấn luyện một mô hình ngôn ngữ khổng lồ có tên Pathways Language Model (PaLM), với 540 tỷ tham số, sử dụng hơn 6.000 TPU.

Ngoài kích thước đáng kinh ngạc của nó, mô hình này là một transformer tiêu chuẩn, chỉ sử dụng bộ giải mã (tức là với các lớp chú ý đa đầu được che), với một vài điều chỉnh nhỏ (xem bài báo để biết chi tiết). Mô hình này đã đạt được hiệu suất đáng kinh ngạc trên tất cả các loại tác vụ NLP, đặc biệt là trong hiểu ngôn ngữ tự nhiên (NLU). Nó có khả năng thực hiện các kỳ tích ấn tượng, chẳng hạn như giải thích truyện cười, đưa ra các câu trả lời chi tiết từng bước cho các câu hỏi, và thậm chí là lập trình. Điều này một phần là do kích thước của mô hình, nhưng cũng nhờ vào một kỹ thuật được gọi là Chain of thought prompting, được giới thiệu vài tháng trước đó bởi một nhóm các nhà nghiên cứu khác của Google.

Trong các tác vụ trả lời câu hỏi, nhắc nhở thông thường thường bao gồm một vài ví dụ về câu hỏi và câu trả lời, chẳng hạn như: “Q: Roger có 5 quả bóng tennis. Anh ấy mua thêm 2 hộp bóng tennis. Mỗi hộp có 3 quả bóng tennis. Bây giờ anh ấy có bao nhiêu quả bóng tennis? A: 11.” Lời nhắc sau đó tiếp tục với câu hỏi thực tế, chẳng hạn như “Q: John chăm sóc 10 con chó. Mỗi con chó mất 0,5 giờ mỗi ngày để đi dạo và giải quyết công việc của chúng. Anh ấy dành bao nhiêu giờ một tuần để chăm sóc chó? A:”, và công việc của mô hình là thêm câu trả lời: trong trường hợp này, “35.”

Nhưng với chain of thought prompting, các câu trả lời ví dụ bao gồm tất cả các bước lập luận dẫn đến kết luận. Ví dụ, thay vì “A: 11”, lời nhắc chứa “A: Roger bắt đầu với 5 quả bóng. 2 hộp, mỗi hộp 3 quả bóng tennis là 6 quả bóng tennis. 5 + 6 = 11.” Điều này khuyến khích mô hình đưa ra câu trả lời chi tiết cho câu hỏi thực tế, chẳng hạn như “John chăm sóc 10 con chó. Mỗi con chó mất 0,5 giờ mỗi ngày để đi dạo và giải quyết công việc của chúng. Vì vậy, đó là 10 × 0,5 = 5 giờ một ngày. 5 giờ một ngày × 7 ngày một tuần = 35 giờ một tuần. Câu trả lời là 35 giờ một tuần.” Đây là một ví dụ thực tế từ bài báo!

Mô hình không chỉ đưa ra câu trả lời đúng thường xuyên hơn nhiều so với việc sử dụng nhắc nhở thông thường — chúng ta đang khuyến khích mô hình suy nghĩ kỹ lưỡng — mà nó còn cung cấp tất cả các bước lập luận, điều này có thể hữu ích để hiểu rõ hơn lý do đằng sau câu trả lời của mô hình.

Transformers đã chiếm lĩnh NLP, nhưng chúng không dừng lại ở đó: chúng sớm mở rộng sang thị giác máy tính.

Vision Transformers

Một trong những ứng dụng đầu tiên của cơ chế chú ý ngoài NMT là tạo chú thích hình ảnh bằng cách sử dụng chú ý thị giác: một mạng nơ-ron tích chập đầu tiên xử lý hình ảnh và xuất ra một số bản đồ tính năng, sau đó một bộ giải mã RNN được trang bị cơ chế chú ý tạo ra chú thích, từng từ một.

Ở mỗi bước thời gian của bộ giải mã (tức là mỗi từ), bộ giải mã sử dụng mô hình chú ý để tập trung vào đúng phần của hình ảnh. Ví dụ, trong Hình 16-12, mô hình đã tạo chú thích “A woman is throwing a frisbee in a park”, và bạn có thể thấy phần nào của hình ảnh đầu vào mà bộ giải mã đã tập trung chú ý khi nó sắp xuất ra từ “frisbee”: rõ ràng, hầu hết sự chú ý của nó được tập trung vào frisbee.

Hình 16-12. Chú ý thị giác: một hình ảnh đầu vào (trái) và tiêu điểm của mô hình trước khi tạo ra từ “frisbee” (phải)

Khi transformer ra đời vào năm 2017 và mọi người bắt đầu thử nghiệm chúng ngoài NLP, chúng lần đầu tiên được sử dụng cùng với CNN, mà không thay thế chúng. Thay vào đó, transformer thường được sử dụng để thay thế RNN, ví dụ, trong các mô hình chú thích hình ảnh. Transformer trở nên trực quan hơn một chút trong một bài báo năm 2020 của các nhà nghiên cứu Facebook, đề xuất một kiến trúc CNN–transformer lai để phát hiện đối tượng. Một lần nữa, CNN đầu tiên xử lý các hình ảnh đầu vào và xuất ra một tập hợp các bản đồ tính năng, sau đó các bản đồ tính năng này được chuyển đổi thành chuỗi và cấp cho một transformer, xuất ra các dự đoán hộp giới hạn. Nhưng một lần nữa, hầu hết công việc thị giác vẫn được thực hiện bởi CNN.

Sau đó, vào tháng 10 năm 2020, một nhóm các nhà nghiên cứu Google đã phát hành một bài báo giới thiệu một mô hình thị giác hoàn toàn dựa trên transformer, được gọi là vision transformer (ViT). Ý tưởng này đơn giản đến ngạc nhiên: chỉ cần cắt hình ảnh thành các ô vuông nhỏ 16 × 16, và coi chuỗi các ô vuông như thể nó là một chuỗi các biểu diễn từ. Để chính xác hơn, các ô vuông trước tiên được làm phẳng thành các vector 16 × 16 × 3 = 768 chiều — số 3 là dành cho các kênh màu RGB — sau đó các vector này đi qua một lớp tuyến tính biến đổi chúng nhưng giữ lại số chiều của chúng. Chuỗi các vector kết quả sau đó có thể được xử lý giống như một chuỗi các embedding từ: điều này có nghĩa là thêm các embedding vị trí, và truyền kết quả đến transformer. Thế là xong! Mô hình này đã đánh bại trạng thái nghệ thuật về phân loại hình ảnh ImageNet, nhưng công bằng mà nói, các tác giả đã phải sử dụng hơn 300 triệu hình ảnh bổ sung để huấn luyện. Điều này có ý nghĩa vì transformer không có nhiều bias cảm ứng như mạng nơ-ron tích chập, vì vậy chúng cần thêm dữ liệu chỉ để học những điều mà CNN ngầm định giả định.

Chỉ hai tháng sau, một nhóm các nhà nghiên cứu Facebook đã phát hành một bài báo giới thiệu các transformer hình ảnh hiệu quả dữ liệu (DeiT). Mô hình của họ đã đạt được kết quả cạnh tranh trên ImageNet mà không yêu cầu bất kỳ dữ liệu bổ sung nào để huấn luyện. Kiến trúc của mô hình hầu như giống với ViT gốc, nhưng các tác giả đã sử dụng một kỹ thuật chưng cất để truyền kiến thức từ các mô hình CNN tiên tiến nhất sang mô hình của họ.

Sau đó, vào tháng 3 năm 2021, DeepMind đã phát hành một bài báo quan trọng giới thiệu kiến trúc Perceiver. Nó là một transformer đa phương thức, nghĩa là bạn có thể cấp cho nó văn bản, hình ảnh, âm thanh hoặc hầu như bất kỳ phương thức nào khác. Cho đến lúc đó, transformer đã bị hạn chế ở các chuỗi khá ngắn do hiệu suất và nút cổ chai RAM trong các lớp chú ý. Điều này loại trừ các phương thức như âm thanh hoặc video, và nó buộc các nhà nghiên cứu phải xử lý hình ảnh dưới dạng chuỗi các mảng con, thay vì chuỗi các pixel. Nút cổ chai là do tự chú ý (self-attention), nơi mỗi token phải chú ý đến mọi token khác: nếu chuỗi đầu vào có M token, thì lớp chú ý phải tính toán một ma trận M × M, có thể rất lớn nếu M rất lớn. Perceiver giải quyết vấn đề này bằng cách dần dần cải thiện một biểu diễn tiềm ẩn khá ngắn của các đầu vào, bao gồm N token — thường chỉ vài trăm. (Từ “tiềm ẩn” có nghĩa là ẩn, hoặc bên trong.) Mô hình chỉ sử dụng các lớp chú ý chéo (cross-attention layers), cấp cho chúng biểu diễn tiềm ẩn làm truy vấn, và các đầu vào (có thể lớn) làm giá trị. Điều này chỉ yêu cầu tính toán một ma trận M × N, vì vậy độ phức tạp tính toán là tuyến tính đối với M, thay vì bậc hai. Sau khi đi qua một số lớp chú ý chéo, nếu mọi thứ diễn ra tốt đẹp, biểu diễn tiềm ẩn cuối cùng sẽ nắm bắt được mọi thứ quan trọng trong các đầu vào.

Các tác giả cũng gợi ý chia sẻ trọng số giữa các lớp chú ý chéo liên tiếp: nếu bạn làm điều đó, thì Perceiver thực sự trở thành một RNN. Thật vậy, các lớp chú ý chéo được chia sẻ có thể được xem là cùng một ô nhớ ở các bước thời gian khác nhau, và biểu diễn tiềm ẩn tương ứng với vector ngữ cảnh của ô. Cùng một đầu vào được cấp liên tục cho ô nhớ ở mỗi bước thời gian. Có vẻ như RNN chưa chết!

Chỉ một tháng sau, Mathilde Caron et al. đã giới thiệu DINO, một transformer hình ảnh ấn tượng được huấn luyện hoàn toàn không nhãn, sử dụng tự giám sát, và có khả năng phân đoạn ngữ nghĩa độ chính xác cao. Mô hình được nhân đôi trong quá trình huấn luyện, với một mạng hoạt động như một giáo viên và mạng kia hoạt động như một học sinh. Gradient descent chỉ ảnh hưởng đến học sinh, trong khi trọng số của giáo viên chỉ là trung bình động theo cấp số nhân của trọng số của học sinh. Học sinh được huấn luyện để khớp với các dự đoán của giáo viên: vì chúng gần như là cùng một mô hình, điều này được gọi là tự chưng cất. Ở mỗi bước huấn luyện, các hình ảnh đầu vào được tăng cường theo những cách khác nhau cho giáo viên và học sinh, vì vậy chúng không nhìn thấy cùng một hình ảnh chính xác, nhưng các dự đoán của chúng phải khớp. Điều này buộc chúng phải đưa ra các biểu diễn cấp cao. Để ngăn chặn sự sụp đổ chế độ (mode collapse), nơi cả học sinh và giáo viên sẽ luôn xuất ra cùng một thứ, hoàn toàn bỏ qua đầu vào, DINO theo dõi trung bình động của đầu ra của giáo viên, và nó điều chỉnh các dự đoán của giáo viên để đảm bảo rằng chúng vẫn tập trung vào số 0, trung bình. DINO cũng buộc giáo viên phải có độ tin cậy cao trong các dự đoán của mình: điều này được gọi là làm sắc nét (sharpening). Cùng nhau, các kỹ thuật này duy trì sự đa dạng trong đầu ra của giáo viên.

Trong một bài báo năm 2021, các nhà nghiên cứu Google đã chỉ ra cách thay đổi kích thước ViT lên hoặc xuống, tùy thuộc vào lượng dữ liệu. Họ đã tạo ra một mô hình khổng lồ 2 tỷ tham số đạt độ chính xác top-1 trên 90,4% trên ImageNet. Ngược lại, họ cũng đã huấn luyện một mô hình giảm kích thước đạt độ chính xác top-1 trên 84,8% trên ImageNet, chỉ sử dụng 10.000 hình ảnh: đó chỉ là 10 hình ảnh cho mỗi lớp!

Và sự tiến bộ trong các transformer thị giác vẫn tiếp tục ổn định cho đến ngày nay. Ví dụ, vào tháng 3 năm 2022, một bài báo của Mitchell Wortsman et al. đã chứng minh rằng có thể huấn luyện nhiều transformer trước, sau đó tính trung bình trọng số của chúng để tạo ra một mô hình mới và cải tiến. Điều này tương tự như một ensemble (xem Chương 7), ngoại trừ cuối cùng chỉ có một mô hình, nghĩa là không có hình phạt thời gian suy luận.

Xu hướng mới nhất trong transformer bao gồm việc xây dựng các mô hình đa phương thức lớn, thường có khả năng học zero-shot hoặc few-shot. Ví dụ, bài báo CLIP năm 2021 của OpenAI đã đề xuất một mô hình transformer lớn được tiền huấn luyện để khớp chú thích với hình ảnh: tác vụ này cho phép nó học các biểu diễn hình ảnh xuất sắc, và mô hình sau đó có thể được sử dụng trực tiếp cho các tác vụ như phân loại hình ảnh bằng các lời nhắc văn bản đơn giản như “a photo of a cat”. Ngay sau đó, OpenAI đã công bố DALL·E, có khả năng tạo ra những hình ảnh tuyệt vời dựa trên các lời nhắc văn bản. DALL·E 2, tạo ra những hình ảnh chất lượng cao hơn nữa bằng cách sử dụng mô hình khuếch tán (xem Chương 17).

Vào tháng 4 năm 2022, các nhà nghiên cứu Google đã phát hành bài báo Flamingo, giới thiệu một họ mô hình được tiền huấn luyện trên nhiều tác vụ khác nhau trên nhiều phương

thức, bao gồm văn bản, hình ảnh và video. Một mô hình duy nhất có thể được sử dụng trên các tác vụ rất khác nhau, chẳng hạn như trả lời câu hỏi, tạo chú thích hình ảnh, v.v. Ngay sau đó, vào tháng 5 năm 2022, DeepMind đã giới thiệu GATO, một mô hình đa phương thức có thể được sử dụng làm chính sách cho một tác nhân học tăng cường (RL sẽ được giới thiệu trong Chương 18). Cùng một transformer có thể trò chuyện với bạn, chú thích hình ảnh, chơi trò chơi Atari, điều khiển cánh tay robot (mô phỏng), v.v., tất cả chỉ với "chỉ" 1,2 tỷ tham số. Và cuộc phiêu lưu vẫn tiếp tục!

Như bạn có thể thấy, transformer có mặt ở khắp mọi nơi! Và tin tốt là bạn thường sẽ không phải tự triển khai transformer vì nhiều mô hình tiền huấn luyện xuất sắc đã có sẵn để tải xuống thông qua TensorFlow Hub hoặc trung tâm mô hình của Hugging Face. Bạn đã thấy cách sử dụng một mô hình từ TF Hub, vì vậy hãy kết thúc chương này bằng cách xem nhanh hệ sinh thái của Hugging Face.

Thư viện Transformers của Hugging Face

Không thể nói về transformer ngày nay mà không nhắc đến Hugging Face, một công ty AI đã xây dựng toàn bộ hệ sinh thái các công cụ mã nguồn mở dễ sử dụng cho NLP, thị giác, và hơn thế nữa. Thành phần trung tâm của hệ sinh thái của họ là thư viện Transformers, cho phép bạn dễ dàng tải xuống một mô hình tiền huấn luyện, bao gồm bộ mã hóa tương ứng của nó, và sau đó tinh chỉnh nó trên tập dữ liệu của riêng bạn, nếu cần. Thêm vào đó, thư viện hỗ trợ TensorFlow, PyTorch và JAX (với thư viện Flax).

Cách đơn giản nhất để sử dụng thư viện Transformers là sử dụng hàm `transformers.pipeline()`: bạn chỉ cần chỉ định tác vụ bạn muốn, chẳng hạn như phân tích cảm xúc, và nó sẽ tải xuống một mô hình tiền huấn luyện mặc định, sẵn sàng để sử dụng — thực sự không thể đơn giản hơn:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis") # nhiều tác vụ khác có sẵn
result = classifier("The actors were very convincing.")
```

Kết quả là một danh sách Python chứa một từ điển cho mỗi văn bản đầu vào:

```
>>> result
[{'label': 'POSITIVE', 'score': 0.9998071789741516}]
```

Trong ví dụ này, mô hình đã tìm thấy chính xác rằng câu là tích cực, với độ tin cậy khoảng 99,98%. Tất nhiên, bạn cũng có thể cấp một batch các câu cho mô hình:

```
>>> classifier(["I am from India.", "I am from Iraq."])
[{'label': 'POSITIVE', 'score': 0.9896161556243896},
{'label': 'NEGATIVE', 'score': 0.9811071157455444}]
```

Thiên vị và Công bằng

Như đầu ra gợi ý, bộ phân loại cụ thể này yêu thích người Ấn Độ, nhưng lại bị thiên vị nghiêm trọng đối với người Iraq. Bạn có thể thử đoạn mã này với quốc gia hoặc thành phố

của riêng bạn. Sự thiên vị không mong muốn như vậy thường phần lớn đến từ chính dữ liệu huấn luyện: trong trường hợp này, có rất nhiều câu tiêu cực liên quan đến các cuộc chiến tranh ở Iraq trong dữ liệu huấn luyện. Sự thiên vị này sau đó đã được khuếch đại trong quá trình tinh chỉnh vì mô hình buộc phải chọn giữa hai lớp: tích cực hoặc tiêu cực. Nếu bạn thêm một lớp trung tính khi tinh chỉnh, thì sự thiên vị quốc gia hầu hết sẽ biến mất. Nhưng dữ liệu huấn luyện không phải là nguồn gốc duy nhất của sự thiên vị: kiến trúc của mô hình, loại hàm mất mát hoặc chuẩn hóa được sử dụng để huấn luyện, bộ tối ưu hóa; tất cả những điều này đều có thể ảnh hưởng đến những gì mô hình học được. Ngay cả một mô hình hầu hết không thiên vị cũng có thể được sử dụng theo cách thiên vị, giống như các câu hỏi khảo sát có thể bị thiên vị.

Hiểu về sự thiên vị trong AI và giảm thiểu các tác động tiêu cực của nó vẫn là một lĩnh vực nghiên cứu tích cực, nhưng một điều chắc chắn: bạn nên dừng lại và suy nghĩ trước khi vội vàng triển khai một mô hình vào sản xuất. Tự hỏi bản thân xem mô hình có thể gây hại như thế nào, ngay cả gián tiếp. Ví dụ, nếu các dự đoán của mô hình được sử dụng để quyết định có cho ai đó vay tiền hay không, quy trình phải công bằng. Vì vậy, hãy đảm bảo rằng bạn đánh giá hiệu suất của mô hình không chỉ trung bình trên toàn bộ tập kiểm tra, mà còn trên các tập con khác nhau: ví dụ, bạn có thể thấy rằng mặc dù mô hình hoạt động rất tốt trung bình, nhưng hiệu suất của nó rất tệ đối với một số nhóm người. Bạn cũng có thể muốn chạy các thử nghiệm phản thực tế: ví dụ, bạn có thể muốn kiểm tra xem các dự đoán của mô hình có thay đổi hay không khi bạn chỉ cần thay đổi giới tính của ai đó.

Nếu mô hình hoạt động tốt trung bình, thật hấp dẫn để đẩy nó vào sản xuất và chuyển sang việc khác, đặc biệt nếu nó chỉ là một thành phần của một hệ thống lớn hơn nhiều. Nhưng nói chung, nếu bạn không khắc phục những vấn đề như vậy, sẽ không ai khác làm, và mô hình của bạn có thể gây hại nhiều hơn lợi. Giải pháp phụ thuộc vào vấn đề: nó có thể yêu cầu cân bằng lại tập dữ liệu, tinh chỉnh trên một tập dữ liệu khác, chuyển sang một mô hình tiền huấn luyện khác, điều chỉnh kiến trúc hoặc siêu tham số của mô hình, v.v.

Hàm `pipeline()` sử dụng mô hình mặc định cho tác vụ đã cho. Ví dụ, đối với các tác vụ phân loại văn bản như phân tích cảm xúc, tại thời điểm viết bài, nó mặc định là `distilbert-base-uncased-finetuned-sst-2-english` — một mô hình DistilBERT với bộ mã hóa không phân biệt chữ hoa chữ thường, được huấn luyện trên Wikipedia tiếng Anh và một corpus các sách tiếng Anh, và được tinh chỉnh trên tác vụ Stanford Sentiment Treebank v2 (SST-2). Cũng có thể chỉ định thủ công một mô hình khác. Ví dụ, bạn có thể sử dụng một mô hình DistilBERT được tinh chỉnh trên tác vụ Multi-Genre Natural Language Inference (MultiNLI), phân loại hai câu thành ba lớp: mâu thuẫn, trung tính hoặc suy luận. Đây là cách thực hiện:

```
model_name = "huggingface/distilbert-base-uncased-finetuned-mnli"
classifier_mnli = pipeline("text-classification", model=model_name)
classifier_mnli("She loves me. [SEP] She loves me not.")
# Output: [{'label': 'contradiction', 'score': 0.9790192246437073}]
```

API pipeline rất đơn giản và tiện lợi, nhưng đôi khi bạn sẽ cần kiểm soát nhiều hơn. Đối với những trường hợp như vậy, thư viện Transformers cung cấp nhiều lớp, bao gồm tất cả các

loại bộ mã hóa, mô hình, cấu hình, callbacks, và nhiều hơn nữa. Ví dụ, hãy tải cùng mô hình DistilBERT, cùng với bộ mã hóa tương ứng của nó, bằng cách sử dụng các lớp `TFAutoModelForSequenceClassification` và `AutoTokenizer`:

```
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
```

Tiếp theo, hãy mã hóa một vài cặp câu. Trong đoạn mã này, chúng ta kích hoạt padding và chỉ định rằng chúng ta muốn các tensor TensorFlow thay vì các danh sách Python:

```
token_ids = tokenizer(["I like soccer. [SEP] We all love soccer!",
                       "Joe lived for a very long time. [SEP] Joe is old."],
                      padding=True, return_tensors="tf")
```

Đầu ra là một instance giống từ điển của lớp `BatchEncoding`, chứa các chuỗi ID token, cũng như một mặt nạ chứa các số 0 cho các token đệm:

```
>>> token_ids
{'input_ids': <tf.Tensor: shape=(2,15), dtype=int32, numpy=
array([[ 101, 1045, 2066, 4715, 1012, 102, 2057, 2035, 2293, 4715, 999,
102,    0,    0,    0],
[ 101, 3533, 2973, 2005, 1037, 2200, 2146, 2051, 1012, 102, 3533,
2003, 2214, 1012, 102]], dtype=int32)>,
    'attention_mask':
<tf.Tensor: shape=(2, 15), dtype=int32, numpy= array([[1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 0, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]], dtype=int32)>}
```

Nếu bạn đặt `return_token_type_ids=True` khi gọi tokenizer, bạn cũng sẽ nhận được một tensor bổ sung cho biết mỗi token thuộc về câu nào. Điều này cần thiết cho một số mô hình, nhưng không phải DistilBERT.

Tiếp theo, chúng ta có thể trực tiếp truyền đối tượng `BatchEncoding` này cho mô hình; nó trả về một đối tượng `TFSequenceClassifierOutput` chứa các logits lớp được dự đoán của nó:

```
>>> outputs = model(token_ids)
>>> outputs
TFSequenceClassifierOutput(loss=None,
logits=[<tf.Tensor: [...] numpy= array([[-2.1123817 , 1.1786783 , 1.4101017
],
[-0.01478387, 1.0962474 , -0.9919954 ]], dtype=float32)>], [...])
```

Cuối cùng, chúng ta có thể áp dụng hàm kích hoạt softmax để chuyển đổi các logits này thành xác suất lớp, và sử dụng hàm `argmax()` để dự đoán lớp có xác suất cao nhất cho mỗi cặp câu đầu vào:

```
>>> Y_probas = tf.keras.activations.softmax(outputs.logits)
>>> Y_probas
<tf.Tensor:
shape=(2, 3), dtype=float32, numpy= array([[0.01619702, 0.43523544, 0.5485676
],
[0.08672056, 0.85204804, 0.06123142]], dtype=float32)>

>>> Y_pred = tf.argmax(Y_probas, axis=1)
>>> Y_pred # 0 = contradiction, 1 = entailment, 2 = neutral
<tf.Tensor:shape=(2,), dtype=int64, numpy=array([2, 1])>
```

Trong ví dụ này, mô hình phân loại đúng cặp câu đầu tiên là trung tính (việc tôi thích bóng đá không có nghĩa là mọi người khác cũng thích) và cặp thứ hai là suy luận (Joe thực sự phải khá già).

Nếu bạn muốn tinh chỉnh mô hình này trên tập dữ liệu của riêng bạn, bạn có thể huấn luyện mô hình như bình thường với Keras vì nó chỉ là một mô hình Keras thông thường với một vài phương thức bổ sung. Tuy nhiên, vì mô hình xuất ra logits thay vì xác suất, bạn phải sử dụng hàm mất mát `tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)` thay vì hàm mất mát thông thường `"sparse_categorical_crossentropy"`. Hơn nữa, mô hình không hỗ trợ đầu vào `BatchEncoding` trong quá trình huấn luyện, vì vậy bạn phải sử dụng thuộc tính `data` của nó để lấy một từ điển thông thường thay thế:

```
sentences = [("Sky is blue", "Sky is red"), ("I love her", "She loves me")]
X_train = tokenizer(sentences, padding=True, return_tensors="tf").data
y_train = tf.constant([0, 2]) # contradiction, neutral

loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(loss=loss, optimizer="nadam", metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=2)
```

Hugging Face cũng đã xây dựng một thư viện Datasets mà bạn có thể sử dụng để dễ dàng tải xuống một tập dữ liệu tiêu chuẩn (như IMDb) hoặc một tập dữ liệu tùy chỉnh, và sử dụng nó để tinh chỉnh mô hình của bạn. Nó tương tự như TensorFlow Datasets, nhưng nó cũng cung cấp các công cụ để thực hiện các tác vụ tiền xử lý phổ biến một cách nhanh chóng, chẳng hạn như che mặt. Danh sách các tập dữ liệu có sẵn tại https://huggingface.co/datasets .

Điều này sẽ giúp bạn bắt đầu với hệ sinh thái của Hugging Face. Để tìm hiểu thêm, bạn có thể truy cập https://huggingface.co/docs để xem tài liệu, bao gồm nhiều sổ tay hướng dẫn, video, API đầy đủ, v.v. Tôi cũng khuyên bạn nên xem cuốn sách của O'Reilly "Natural Language Processing with Transformers: Building Language Applications with Hugging Face" của Lewis Tunstall, Leandro von Werra, và Thomas Wolf — tất cả đều từ nhóm Hugging Face.

Trong chương tiếp theo, chúng ta sẽ thảo luận về cách học các biểu diễn sâu một cách không giám sát bằng cách sử dụng autoencoder, và chúng ta sẽ sử dụng mạng đối nghịch tạo sinh để tạo ra hình ảnh và nhiều hơn nữa!

Bài tập

1. Ưu và nhược điểm của việc sử dụng RNN có trạng thái so với RNN không trạng thái là gì?
2. Tại sao người ta sử dụng RNN mã hóa-giải mã thay vì RNN tuần tự thông thường để dịch tự động?
3. Làm thế nào bạn có thể xử lý các chuỗi đầu vào có độ dài thay đổi? Còn các chuỗi đầu ra có độ dài thay đổi thì sao?
4. Beam search là gì, và tại sao bạn lại sử dụng nó? Bạn có thể sử dụng công cụ nào để triển khai nó?
5. Cơ chế chú ý là gì? Nó giúp ích như thế nào?
6. Lớp quan trọng nhất trong kiến trúc transformer là gì? Mục đích của nó là gì?
7. Khi nào bạn cần sử dụng softmax lấy mẫu?
8. Ngữ pháp Reber nhúng được Hochreiter và Schmidhuber sử dụng trong bài báo của họ về LSTM. Chúng là các ngữ pháp nhân tạo tạo ra các chuỗi như "BPBTSXXVPSEPE". Hãy xem phần giới thiệu hay của Jenny Orr về chủ đề này, sau đó chọn một ngữ pháp Reber nhúng cụ thể (chẳng hạn như ngữ pháp được biểu diễn trên trang của Orr), sau đó huấn luyện một RNN để xác định xem một chuỗi có tuân thủ ngữ pháp đó hay không. Bạn sẽ cần viết một hàm có khả năng tạo một batch huấn luyện chứa khoảng 50% chuỗi tuân thủ ngữ pháp và 50% chuỗi không tuân thủ.
9. Huấn luyện một mô hình mã hóa-giải mã có thể chuyển đổi chuỗi ngày từ định dạng này sang định dạng khác (ví dụ: từ "April 22, 2019" sang "2019-04-22").
10. Tham khảo ví dụ trên trang web Keras về "Tìm kiếm hình ảnh ngôn ngữ tự nhiên với Bộ mã hóa kép". Bạn sẽ học cách xây dựng một mô hình có khả năng biểu diễn cả hình ảnh và văn bản trong cùng một không gian embedding. Điều này giúp có thể tìm kiếm hình ảnh bằng lời nhắc văn bản, giống như trong mô hình CLIP của OpenAI.
11. Sử dụng thư viện Hugging Face Transformers để tải xuống một mô hình ngôn ngữ tiền huấn luyện có khả năng tạo văn bản (ví dụ: GPT), và thử tạo văn bản kiểu Shakespeare thuyết phục hơn. Bạn sẽ cần sử dụng phương thức `generate()` của mô hình — xem tài liệu của Hugging Face để biết thêm chi tiết.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay chương này, tại https://homl.info/colab3 .

Tài liệu tham khảo

- Alan Turing, "Computing Machinery and Intelligence", Mind 49 (1950): 433–460.
- Tất nhiên, từ chatbot xuất hiện muộn hơn nhiều. Turing gọi bài kiểm tra của mình là trò chơi bắt chước: máy A và người B trò chuyện với người thẩm vấn C qua tin nhắn văn bản; người thẩm vấn hỏi các câu hỏi để tìm ra cái nào là máy (A hay B). Máy vượt qua bài kiểm tra nếu nó có thể đánh lừa người thẩm vấn, trong khi người B phải cố gắng giúp người thẩm vấn.

- Vì các cửa sổ đầu vào chồng lên nhau, khái niệm epoch không rõ ràng trong trường hợp này: trong mỗi epoch (theo cách triển khai của Keras), mô hình sẽ thực sự nhìn thấy cùng một ký tự nhiều lần.
- Alec Radford et al., “Learning to Generate Reviews and Discovering Sentiment”, arXiv preprint arXiv:1704.01444 (2017).
- Rico Sennrich et al., “Neural Machine Translation of Rare Words with Subword Units”, Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics 1 (2016): 1715–1725.
- Taku Kudo, “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates”, arXiv preprint arXiv:1804.10959 (2018).
- Taku Kudo và John Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing”, arXiv preprint arXiv:1808.06226 (2018).
- Yonghui Wu et al., “Google’s Neural Machine Translation System: Bridging the Gap Between Human and Machine Translation”, arXiv preprint arXiv:1609.08144 (2016).
- Ragged tensors được giới thiệu trong Chương 12, và chúng được chi tiết trong Phụ lục C.
- Matthew Peters et al., “Deep Contextualized Word Representations”, Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1 (2018): 2227–2237.
- Jeremy Howard và Sebastian Ruder, “Universal Language Model Fine-Tuning for Text Classification”, Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics 1 (2018): 328–339.
- Daniel Cer et al., “Universal Sentence Encoder”, arXiv preprint arXiv:1803.11175 (2018).
- Ilya Sutskever et al., “Sequence to Sequence Learning with Neural Networks”, arXiv preprint (2014).
- Samy Bengio et al., “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks”, arXiv preprint arXiv:1506.03099 (2015).
- Tập dữ liệu này bao gồm các cặp câu được tạo bởi những người đóng góp của dự án Tatoeba. Khoảng 120.000 cặp câu đã được chọn bởi các tác giả của trang web https://manythings.org/anki . Tập dữ liệu này được phát hành theo giấy phép Creative Commons Attribution 2.0 France. Các cặp ngôn ngữ khác cũng có sẵn.
- Trong Python, nếu bạn chạy `a, *b = [1, 2, 3, 4]`, thì a bằng 1 và b bằng `[2, 3, 4]`.
- Sébastien Jean et al., “On Using Very Large Target Vocabulary for Neural Machine Translation”, Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing 1 (2015): 1–10.

- Dzmitry Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate”, arXiv preprint arXiv:1409.0473 (2014).
- Minh-Thang Luong et al., “Effective Approaches to Attention-Based Neural Machine Translation”, Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (2015): 1412–1421.
- Ashish Vaswani et al., “Attention Is All You Need”, Proceedings of the 31st International Conference on Neural Information Processing Systems (2017): 6000–6010.
- Vì transformer sử dụng các lớp dense phân phối thời gian, bạn có thể cho rằng nó sử dụng các lớp tích chập 1D với kích thước kernel là 1.
- Đây là hình 1 từ bài báo “Attention Is All You Need”, được tái bản với sự cho phép của các tác giả.
- Có thể sử dụng ragged tensors thay thế, nếu bạn đang sử dụng phiên bản TensorFlow mới nhất.
- Đây là phần bên phải của hình 2 từ “Attention Is All You Need”, được tái bản với sự cho phép của các tác giả.
- Điều này rất có thể sẽ thay đổi vào thời điểm bạn đọc bài này; hãy xem Keras issue #16248 để biết thêm chi tiết. Khi điều này xảy ra, sẽ không cần đặt đối số `attention_mask`, và do đó không cần tạo `encoder_pad_mask`.
- Hiện tại `Z + skip` không hỗ trợ che mặt tự động, đó là lý do tại sao chúng ta phải viết `tf.keras.layers.Add()([Z, skip])` thay thế. Một lần nữa, điều này có thể thay đổi vào thời điểm bạn đọc bài này.
- Alec Radford et al., “Improving Language Understanding by Generative Pre-Training” (2018).
- Ví dụ, câu “Jane had a lot of fun at her friend’s birthday party” suy luận “Jane enjoyed the party”, nhưng nó bị mâu thuẫn bởi “Everyone hated the party” và nó không liên quan đến “The Earth is flat”.
- Jacob Devlin et al., “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding”, Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1 (2019).
- Đây là hình 1 từ bài báo, được tái bản với sự cho phép của các tác giả.
- Alec Radford et al., “Language Models Are Unsupervised Multitask Learners” (2019).
- William Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity” (2021).
- Victor Sanh et al., “DistilBERT, A Distilled Version of Bert: Smaller, Faster, Cheaper and Lighter”, arXiv preprint arXiv:1910.01108 (2019).
- Mariya Yao đã tóm tắt nhiều mô hình này trong bài đăng này: https://homl.info/yaopost .

- Colin Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”, arXiv preprint arXiv:1910.10683 (2019).
- Aakanksha Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways”, arXiv preprint arXiv:2204.02311 (2022).
- Jason Wei et al., “Chain of Thought Prompting Elicits Reasoning in Large Language Models”, arXiv preprint arXiv:2201.11903 (2022).
- Kelvin Xu et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”, Proceedings of the 32nd International Conference on Machine Learning (2015): 2048–2057.
- Đây là một phần của hình 3 từ bài báo. Nó được tái bản với sự cho phép của các tác giả.
- Marco Tulio Ribeiro et al., “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier”, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2016): 1135–1144.
- Nicolas Carion et al., “End-to-End Object Detection with Transformers”, arXiv preprint arxiv:2005.12872 (2020).
- Alexey Dosovitskiy et al., “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale”, arXiv preprint arxiv:2010.11929 (2020).
- Hugo Touvron et al., “Training Data-Efficient Image Transformers & Distillation Through Attention”, arXiv preprint arxiv:2012.12877 (2020).
- Andrew Jaegle et al., “Perceiver: General Perception with Iterative Attention”, arXiv preprint arxiv:2103.03206 (2021).
- Mathilde Caron et al., “Emerging Properties in Self-Supervised Vision Transformers”, arXiv preprint arxiv:2104.14294 (2021).
- Xiaohua Zhai et al., “Scaling Vision Transformers”, arXiv preprint arxiv:2106.04560v1 (2021).
- Mitchell Wortsman et al., “Model Soups: Averaging Weights of Multiple Fine-tuned Models Improves Accuracy Without Increasing Inference Time”, arXiv preprint arxiv:2203.05482v1 (2022).
- Alec Radford et al., “Learning Transferable Visual Models From Natural Language Supervision”, arXiv preprint arxiv:2103.00020 (2021).
- Aditya Ramesh et al., “Zero-Shot Text-to-Image Generation”, arXiv preprint arxiv:2102.12092 (2021).
- Aditya Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv preprint arxiv:2204.06125 (2022).
- Jean-Baptiste Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning”, arXiv preprint arxiv:2204.14198 (2022).
- Scott Reed et al., “A Generalist Agent”, arXiv preprint arxiv:2205.06175 (2022).

CHƯƠNG 17. AUTOENCODER, MẠNG ĐỐI KHÁNG SINH (GAN) VÀ MÔ HÌNH KHUẾCH TÁN

Autoencoder là các mạng thần kinh nhân tạo có khả năng học các biểu diễn dày đặc của dữ liệu đầu vào, được gọi là biểu diễn tiềm ẩn (latent representations) hoặc mã hóa (codings), mà không cần bất kỳ sự giám sát nào (tức là tập dữ liệu huấn luyện không được gán nhãn). Các mã hóa này thường có chiều thấp hơn nhiều so với dữ liệu đầu vào, làm cho autoencoder hữu ích cho việc giảm chiều (xem Chương 8), đặc biệt cho mục đích trực quan hóa. Autoencoder cũng hoạt động như bộ phát hiện đặc trưng, và chúng có thể được sử dụng để tiền huấn luyện không giám sát các mạng thần kinh sâu (như chúng ta đã thảo luận trong Chương 11). Cuối cùng, một số autoencoder là mô hình sinh (generative models): chúng có khả năng tạo ngẫu nhiên dữ liệu mới trông rất giống với dữ liệu huấn luyện. Ví dụ, bạn có thể huấn luyện một autoencoder trên ảnh khuôn mặt, và sau đó nó có thể tạo ra các khuôn mặt mới.

Mạng đối kháng sinh (GAN) cũng là mạng thần kinh có khả năng tạo dữ liệu. Trên thực tế, chúng có thể tạo ra những bức ảnh khuôn mặt thuyết phục đến mức khó tin rằng những người mà chúng đại diện không hề tồn tại. Bạn có thể tự mình đánh giá điều này bằng cách truy cập https://thispersondoesnotexist.com , một trang web hiển thị các khuôn mặt được tạo bởi kiến trúc GAN có tên là StyleGAN. Bạn cũng có thể xem https://thisrentaldoesnotexist.com để xem một số danh sách Airbnb được tạo. GAN hiện đang được sử dụng rộng rãi cho siêu phân giải (tăng độ phân giải của hình ảnh), tô màu, chỉnh sửa hình ảnh mạnh mẽ (ví dụ: thay thế những người làm hỏng ảnh bằng nền thực tế), biến các bản phác thảo đơn giản thành hình ảnh chân thực, dự đoán các khung hình tiếp theo trong video, tăng cường tập dữ liệu (để huấn luyện các mô hình khác), tạo các loại dữ liệu khác (như văn bản, âm thanh và chuỗi thời gian), xác định các điểm yếu trong các mô hình khác để tăng cường chúng, và nhiều hơn nữa.

Một bổ sung gần đây hơn vào nhóm học sinh là các mô hình khuếch tán (diffusion models). Năm 2021, chúng đã tạo ra những hình ảnh đa dạng và chất lượng cao hơn GAN, đồng thời cũng dễ huấn luyện hơn nhiều. Tuy nhiên, các mô hình khuếch tán chạy chậm hơn nhiều.

Autoencoder, GAN và mô hình khuếch tán đều không giám sát, chúng đều học các biểu diễn tiềm ẩn, chúng đều có thể được sử dụng làm mô hình sinh, và chúng có nhiều ứng dụng tương tự. Tuy nhiên, chúng hoạt động rất khác nhau:

Autoencoder đơn giản học cách sao chép đầu vào của chúng sang đầu ra. Điều này nghe có vẻ là một nhiệm vụ tầm thường, nhưng như bạn sẽ thấy, việc giới hạn mạng theo nhiều cách khác nhau có thể làm cho nó khá khó khăn. Ví dụ, bạn có thể giới hạn kích thước của các biểu diễn tiềm ẩn, hoặc bạn có thể thêm nhiễu vào đầu vào và huấn luyện mạng để khôi phục đầu vào gốc. Những ràng buộc này ngăn autoencoder sao chép trực tiếp đầu vào sang đầu ra một cách tầm thường, điều này buộc nó phải học các cách hiệu quả để biểu diễn dữ liệu. Tóm lại, các mã hóa là sản phẩm phụ của autoencoder học hàm đồng nhất dưới một số ràng buộc.

GAN bao gồm hai mạng thần kinh: một bộ tạo (generator) cố gắng tạo ra dữ liệu trông giống với dữ liệu huấn luyện, và một bộ phân biệt (discriminator) cố gắng phân biệt dữ liệu thật với dữ liệu giả. Kiến trúc này rất độc đáo trong học sâu ở chỗ bộ tạo và bộ phân biệt cạnh tranh với nhau trong quá trình huấn luyện: bộ tạo thường được so sánh với một tội phạm cố gắng tạo ra tiền giả giống thật, trong khi bộ phân biệt giống như điều tra viên cảnh sát cố gắng phân biệt tiền thật với tiền giả. Huấn luyện đối kháng (huấn luyện các mạng thần kinh cạnh tranh) được coi là một trong những đổi mới quan trọng nhất của những năm 2010. Năm 2016, Yann LeCun thậm chí đã nói rằng đó là "ý tưởng thú vị nhất trong 10 năm qua trong học máy".

Một mô hình xác suất khuếch tán khử nhiễu (Denoising Diffusion Probabilistic Model - DDPM) được huấn luyện để loại bỏ một chút nhiễu khỏi một hình ảnh. Nếu bạn sau đó lấy một hình ảnh hoàn toàn đầy nhiễu Gaussian và lặp đi lặp lại chạy mô hình khuếch tán trên hình ảnh đó, một hình ảnh chất lượng cao sẽ dần dần xuất hiện, tương tự như các hình ảnh huấn luyện (nhưng không giống hệt).

Trong chương này, chúng ta sẽ bắt đầu bằng cách khám phá sâu hơn cách autoencoder hoạt động và cách sử dụng chúng để giảm chiều, trích xuất đặc trưng, tiền huấn luyện không giám sát, hoặc làm mô hình sinh. Điều này sẽ tự nhiên dẫn chúng ta đến GAN. Chúng ta sẽ xây dựng một GAN đơn giản để tạo ra hình ảnh giả, nhưng chúng ta sẽ thấy rằng việc huấn luyện thường khá khó khăn. Chúng ta sẽ thảo luận về những khó khăn chính mà bạn sẽ gặp phải khi huấn luyện đối kháng, cũng như một số kỹ thuật chính để khắc phục những khó khăn này. Và cuối cùng, chúng ta sẽ xây dựng và huấn luyện một DDPM và sử dụng nó để tạo ra hình ảnh. Hãy bắt đầu với autoencoder!

Biểu diễn dữ liệu hiệu quả

Trong các dãy số sau đây, bạn thấy dãy nào dễ nhớ nhất?

40, 27, 25, 36, 81, 57, 10, 73, 19, 68

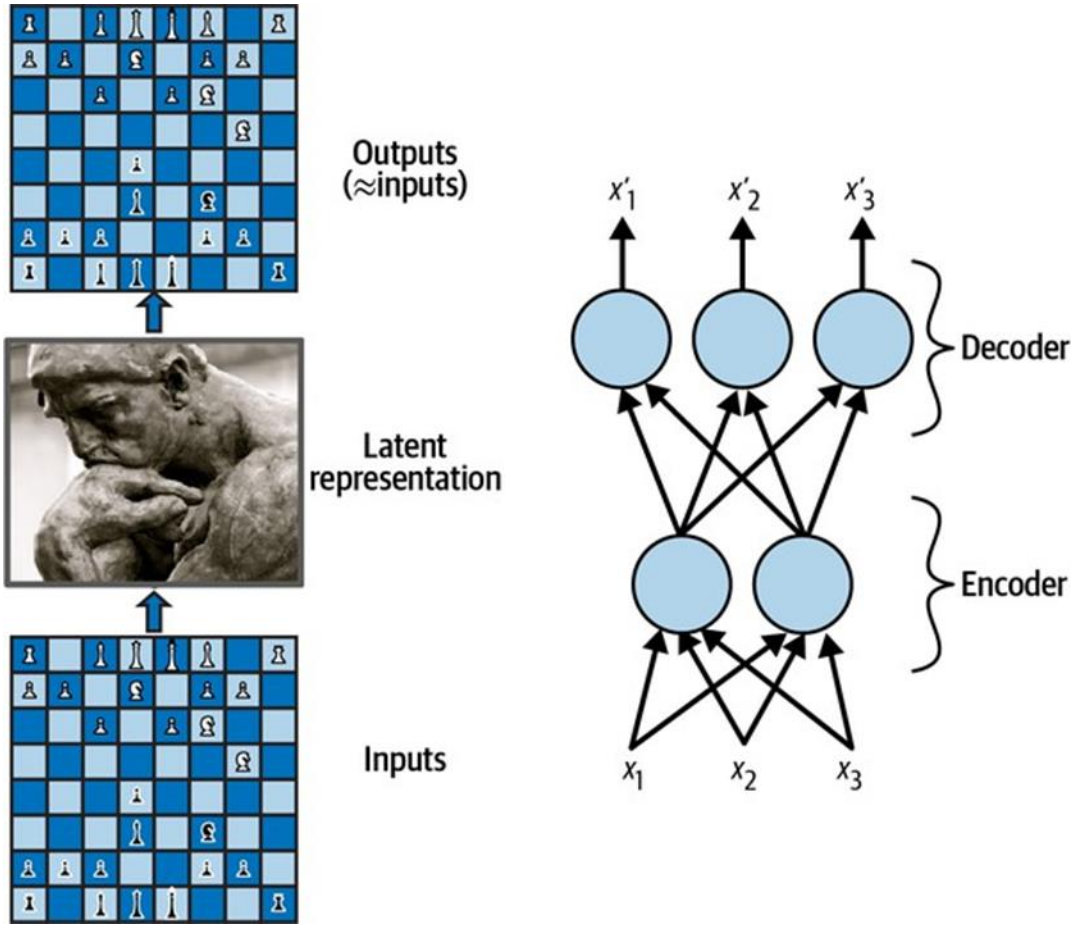
50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14

Thoạt nhìn, có vẻ như dãy đầu tiên sẽ dễ hơn, vì nó ngắn hơn nhiều. Tuy nhiên, nếu bạn nhìn kỹ dãy thứ hai, bạn sẽ nhận thấy rằng đó chỉ là danh sách các số chẵn từ 50 xuống 14. Một khi bạn nhận ra quy luật này, dãy thứ hai trở nên dễ nhớ hơn nhiều so với dãy đầu tiên vì bạn chỉ cần nhớ quy luật (tức là các số chẵn giảm dần) và các số bắt đầu và kết thúc (tức là 50 và 14). Lưu ý rằng nếu bạn có thể nhanh chóng và dễ dàng ghi nhớ các dãy rất dài, bạn sẽ không quan tâm nhiều đến sự tồn tại của một quy luật trong dãy thứ hai. Bạn sẽ chỉ học thuộc lòng từng số, và thế là xong. Thực tế là khó ghi nhớ các dãy dài khiến việc nhận ra các quy luật trở nên hữu ích, và hy vọng điều này làm rõ lý do tại sao việc ràng buộc một autoencoder trong quá trình huấn luyện thúc đẩy nó khám phá và khai thác các quy luật trong dữ liệu.

Mối quan hệ giữa trí nhớ, nhận thức và khớp mẫu đã được William Chase và Herbert Simon nổi tiếng nghiên cứu vào đầu những năm 1970. Họ quan sát thấy rằng các kỳ thủ cờ vua chuyên nghiệp có thể ghi nhớ vị trí của tất cả các quân cờ trong một ván đấu chỉ bằng cách

nhìn vào bàn cờ trong năm giây, một nhiệm vụ mà hầu hết mọi người sẽ thấy không thể. Tuy nhiên, điều này chỉ đúng khi các quân cờ được đặt ở vị trí thực tế (từ các ván đấu thực tế), chứ không phải khi các quân cờ được đặt ngẫu nhiên. Các chuyên gia cờ vua không có trí nhớ tốt hơn nhiều so với bạn và tôi; họ chỉ nhìn thấy các mẫu cờ vua dễ dàng hơn, nhờ kinh nghiệm của họ với trò chơi. Việc nhận thấy các mẫu giúp họ lưu trữ thông tin hiệu quả.

Giống như các kỳ thủ cờ vua trong thí nghiệm trí nhớ này, một autoencoder nhìn vào đầu vào, chuyển đổi chúng thành một biểu diễn tiềm ẩn hiệu quả, và sau đó cho ra thứ gì đó (hy vọng) trông rất giống với đầu vào. Một autoencoder luôn bao gồm hai phần: một bộ mã hóa (encoder) (hoặc mạng nhận dạng) chuyển đổi đầu vào thành một biểu diễn tiềm ẩn, tiếp theo là một bộ giải mã (decoder) (hoặc mạng sinh) chuyển đổi biểu diễn nội bộ thành đầu ra (xem Hình 17-1).



Hình 17-1. Thí nghiệm trí nhớ cờ vua (trái) và một autoencoder đơn giản (phải)

Như bạn có thể thấy, một autoencoder thường có kiến trúc giống như một perceptron đa lớp (MLP; xem Chương 10), ngoại trừ số lượng neuron trong lớp đầu ra phải bằng số lượng đầu vào. Trong ví dụ này, chỉ có một lớp ẩn bao gồm hai neuron (bộ mã hóa), và một lớp đầu ra bao gồm ba neuron (bộ giải mã). Các đầu ra thường được gọi là các bản tái tạo (reconstructions) vì autoencoder cố gắng tái tạo lại đầu vào. Hàm chi phí chứa một mất tái tạo (reconstruction loss) để phạt mô hình khi các bản tái tạo khác với đầu vào.

Vì biểu diễn nội bộ có chiều thấp hơn dữ liệu đầu vào (nó là 2D thay vì 3D), autoencoder được gọi là thiếu hoàn chỉnh (undercomplete). Một autoencoder thiếu hoàn chỉnh không thể sao chép trực tiếp đầu vào của nó sang các mã hóa, nhưng nó phải tìm cách xuất ra một bản sao của đầu vào của nó. Nó buộc phải học các đặc trưng quan trọng nhất trong dữ liệu đầu vào (và bỏ qua những đặc trưng không quan trọng).

Hãy xem cách triển khai một autoencoder thiếu hoàn chỉnh rất đơn giản để giảm chiều.

Thực hiện PCA bằng Autoencoder tuyến tính thiếu hoàn chỉnh

Nếu autoencoder chỉ sử dụng các kích hoạt tuyến tính và hàm chi phí là sai số bình phương trung bình (MSE), thì nó sẽ thực hiện phân tích thành phần chính (PCA; xem Chương 8).

Đoạn mã sau đây xây dựng một autoencoder tuyến tính đơn giản để thực hiện PCA trên tập dữ liệu 3D, chiếu nó thành 2D:

```
import tensorflow as tf

encoder = tf.keras.Sequential([tf.keras.layers.Dense(2)])
decoder = tf.keras.Sequential([tf.keras.layers.Dense(3)])
autoencoder = tf.keras.Sequential([encoder, decoder])

optimizer = tf.keras.optimizers.SGD(learning_rate=0.5)
autoencoder.compile(loss="mse", optimizer=optimizer)
```

Mã này thực sự không khác nhiều so với tất cả các MLP mà chúng ta đã xây dựng trong các chương trước, nhưng có một vài điều cần lưu ý:

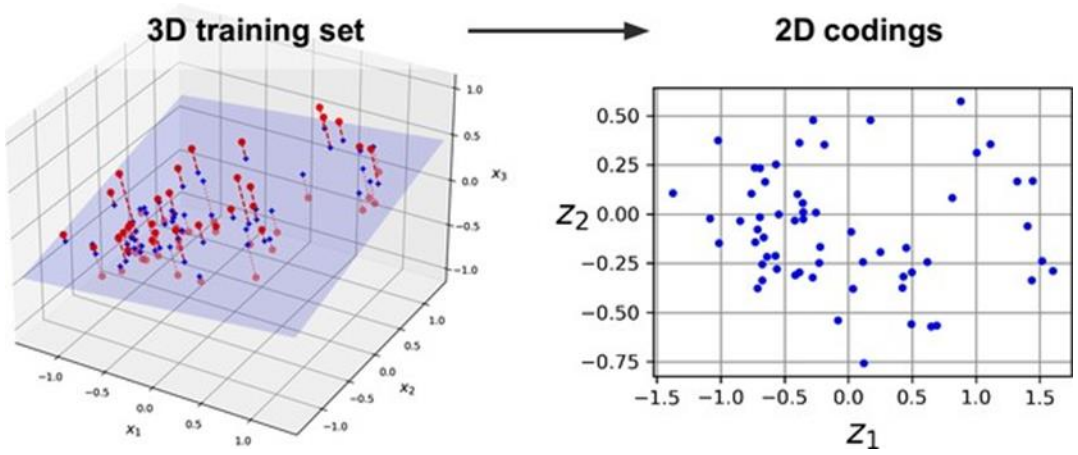
- Chúng ta đã tổ chức autoencoder thành hai thành phần con: bộ mã hóa (encoder) và bộ giải mã (decoder). Cả hai đều là các mô hình `Sequential` thông thường với một lớp `Dense` duy nhất, và autoencoder là một mô hình `Sequential` chứa bộ mã hóa theo sau là bộ giải mã (hãy nhớ rằng một mô hình có thể được sử dụng làm một lớp trong một mô hình khác).
- Số lượng đầu ra của autoencoder bằng số lượng đầu vào (tức là 3).
- Để thực hiện PCA, chúng ta không sử dụng bất kỳ hàm kích hoạt nào (tức là tất cả các neuron đều tuyến tính), và hàm chi phí là MSE. Điều này là do PCA là một phép biến đổi tuyến tính. Chúng ta sẽ sớm thấy các autoencoder phức tạp hơn và phi tuyến tính.

Bây giờ, hãy huấn luyện mô hình trên cùng một tập dữ liệu 3D đơn giản được tạo mà chúng ta đã sử dụng trong Chương 8 và sử dụng nó để mã hóa tập dữ liệu đó (tức là chiếu nó thành 2D):

```
X_train = [...] #generate a 3D dataset, like in Chapter 8

history = autoencoder.fit(X_train, X_train, epochs=500, verbose=False)
codings = encoder.predict(X_train)
```

Lưu ý rằng `X_train` được sử dụng làm cả đầu vào và mục tiêu. Hình 17-2 cho thấy tập dữ liệu 3D gốc (bên trái) và đầu ra của lớp ẩn của autoencoder (tức là lớp mã hóa, bên phải). Như bạn có thể thấy, autoencoder đã tìm thấy mặt phẳng 2D tốt nhất để chiếu dữ liệu lên đó, bảo toàn nhiều phương sai trong dữ liệu nhất có thể (giống như PCA).

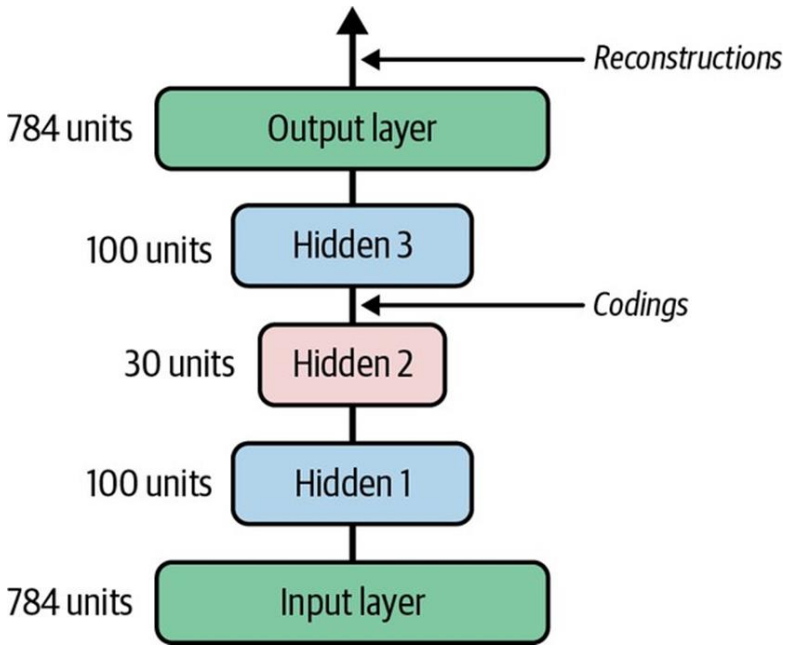


Hình 17-2. PCA gần đúng được thực hiện bởi một autoencoder tuyến tính thiếu hoàn chỉnh

Autoencoder xếp chồng (Stacked Autoencoders)

Giống như các mạng thần kinh khác mà chúng ta đã thảo luận, autoencoder có thể có nhiều lớp ẩn. Trong trường hợp này, chúng được gọi là autoencoder xếp chồng (stacked autoencoders) (hoặc autoencoder sâu). Việc thêm nhiều lớp giúp autoencoder học được các mã hóa phức tạp hơn. Điều đó nói lên rằng, người ta phải cẩn thận không làm cho autoencoder quá mạnh mẽ. Hãy tưởng tượng một bộ mã hóa mạnh đến mức nó chỉ học cách ánh xạ từng đầu vào tới một số tùy ý duy nhất (và bộ giải mã học ánh xạ ngược lại). Rõ ràng một autoencoder như vậy sẽ tái tạo dữ liệu huấn luyện một cách hoàn hảo, nhưng nó sẽ không học được bất kỳ biểu diễn dữ liệu hữu ích nào trong quá trình đó, và nó không có khả năng khái quát hóa tốt cho các trường hợp mới.

Kiến trúc của một autoencoder xếp chồng thường đối xứng so với lớp ẩn trung tâm (lớp mã hóa). Nói một cách đơn giản, nó trông giống như một chiếc bánh sandwich. Ví dụ, một autoencoder cho Fashion MNIST (được giới thiệu trong Chương 10) có thể có 784 đầu vào, tiếp theo là một lớp ẩn với 100 neuron, sau đó là một lớp ẩn trung tâm gồm 30 neuron, sau đó là một lớp ẩn khác với 100 neuron, và một lớp đầu ra với 784 neuron. Autoencoder xếp chồng này được biểu diễn trong Hình 17-3.



Hình 17-3. Autoencoder xếp chồng

Triển khai Autoencoder xếp chồng bằng Keras

Bạn có thể triển khai một autoencoder xếp chồng rất giống với một MLP sâu thông thường:

```
stacked_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
stacked_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
stacked_ae = tf.keras.Sequential([stacked_encoder, stacked_decoder])

stacked_ae.compile(loss="mse", optimizer="nadam")
history = stacked_ae.fit(X_train, X_train, epochs=20,
                         validation_data=(X_valid, X_valid))
```

Hãy xem qua đoạn mã này:

- Cũng như trước đây, chúng ta chia mô hình autoencoder thành hai mô hình con: bộ mã hóa (encoder) và bộ giải mã (decoder).
- Bộ mã hóa nhận các hình ảnh thang độ xám 28 × 28 pixel, làm phẳng chúng để mỗi hình ảnh được biểu diễn dưới dạng một vector có kích thước 784, sau đó xử lý các vector này thông qua hai lớp `Dense` với kích thước giảm dần (100 đơn vị sau đó 30

đơn vị), cả hai đều sử dụng hàm kích hoạt ReLU. Đối với mỗi hình ảnh đầu vào, bộ mã hóa xuất ra một vector có kích thước 30.

- Bộ giải mã nhận các mã hóa có kích thước 30 (được xuất bởi bộ mã hóa) và xử lý chúng thông qua hai lớp `Dense` với kích thước tăng dần (100 đơn vị sau đó 784 đơn vị), và nó định hình lại các vector cuối cùng thành các mảng 28 × 28 để đầu ra của bộ giải mã có cùng hình dạng với đầu vào của bộ mã hóa.
- Khi biên dịch autoencoder xếp chồng, chúng ta sử dụng hàm mất MSE và tối ưu hóa Nadam.
- Cuối cùng, chúng ta huấn luyện mô hình bằng cách sử dụng `X_train` làm cả đầu vào và mục tiêu. Tương tự, chúng ta sử dụng `X_valid` làm cả đầu vào và mục tiêu xác thực.

Để dịch phần còn lại của nội dung sang tiếng Việt, tôi sẽ tiếp tục dịch từng phần nhỏ để đảm bảo tính chính xác và mạch lạc. Tôi cũng sẽ chú ý đến các công thức toán học và cấu trúc mã nguồn để giữ nguyên định dạng.

Trực quan hóa các bản tái tạo

Một cách để đảm bảo rằng autoencoder được huấn luyện đúng cách là so sánh đầu vào và đầu ra: sự khác biệt không nên quá đáng kể. Hãy vẽ một vài hình ảnh từ tập xác thực, cũng như các bản tái tạo của chúng:

```
import numpy as np
import matplotlib.pyplot as plt

def plot_reconstructions(model, images=X_valid, n_images=5):
    # Đảm bảo đầu ra nằm trong khoảng [0, 1] để hiển thị ảnh
    reconstructions = np.clip(model.predict(images[:n_images]), 0, 1)

    fig = plt.figure(figsize=(n_images * 1.5, 3))
    for image_index in range(n_images):
        plt.subplot(2, n_images, 1 + image_index)
        plt.imshow(images[image_index], cmap="binary")
        plt.axis("off")
        plt.subplot(2, n_images, 1 + n_images + image_index)
        plt.imshow(reconstructions[image_index], cmap="binary")
        plt.axis("off")

# Đoạn mã này cần một môi trường Python với TensorFlow, NumPy và Matplotlib
# cùng với các biến X_valid và stacked_ae được định nghĩa trước đó để chạy.
# plot_reconstructions(stacked_ae)
# plt.show()
```

Hình 17-4 cho thấy các hình ảnh kết quả.

Hình 17-4. Hình ảnh gốc (trên cùng) và các bản tái tạo của chúng (dưới cùng)

Các bản tái tạo có thể nhận ra được, nhưng hơi mất mát quá nhiều. Chúng ta có thể cần huấn luyện mô hình lâu hơn, hoặc làm cho bộ mã hóa và bộ giải mã sâu hơn, hoặc làm cho các mã hóa lớn hơn. Nhưng nếu chúng ta làm cho mạng quá mạnh, nó sẽ quản lý để tạo ra các bản tái tạo hoàn hảo mà không học được bất kỳ mẫu hữu ích nào trong dữ liệu. Hiện tại, hãy tiếp tục với mô hình này.

Trực quan hóa tập dữ liệu Fashion MNIST

Bây giờ chúng ta đã huấn luyện một autoencoder xếp chồng, chúng ta có thể sử dụng nó để giảm chiều của tập dữ liệu. Đối với việc trực quan hóa, điều này không mang lại kết quả tuyệt vời so với các thuật toán giảm chiều khác (chẳng hạn như những gì chúng ta đã thảo luận trong Chương 8), nhưng một lợi thế lớn của autoencoder là chúng có thể xử lý các tập dữ liệu lớn với nhiều trường hợp và nhiều đặc trưng. Vì vậy, một chiến lược là sử dụng autoencoder để giảm chiều xuống một mức hợp lý, sau đó sử dụng một thuật toán giảm chiều khác để trực quan hóa. Hãy sử dụng chiến lược này để trực quan hóa Fashion MNIST. Đầu tiên chúng ta sẽ sử dụng bộ mã hóa từ autoencoder xếp chồng của chúng ta để giảm chiều xuống 30, sau đó chúng ta sẽ sử dụng triển khai thuật toán t-SNE của Scikit-Learn để giảm chiều xuống 2 để trực quan hóa:

```
from sklearn.manifold import TSNE
# import numpy as np # Đảm bảo import numpy nếu chưa có
# import matplotlib.pyplot as plt # Đảm bảo import matplotlib.pyplot nếu chưa
có

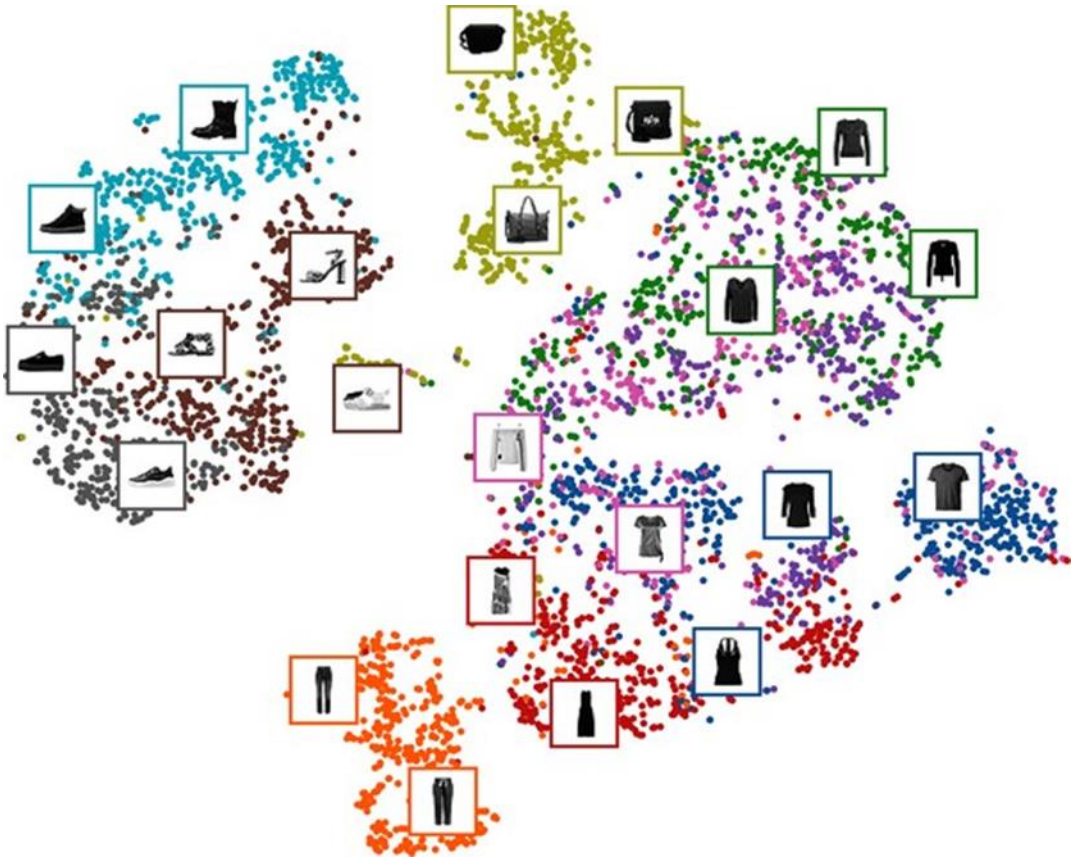
# Giả định X_valid và y_valid đã được định nghĩa và có sẵn từ ngữ cảnh trước
đó
# Giả định stacked_encoder đã được huấn luyện và có sẵn từ ngữ cảnh trước đó

# X_valid_compressed = stacked_encoder.predict(X_valid)
# tsne = TSNE(init="pca", learning_rate="auto", random_state=42)
# X_valid_2D = tsne.fit_transform(X_valid_compressed)
```

Bây giờ chúng ta có thể vẽ biểu đồ tập dữ liệu:

```
#   plt.scatter(X_valid_2D[:,   0],   X_valid_2D[:,   1],   c=y_valid,   s=10,
cmap="tab10")
# plt.show()
```

Hình 17-5 cho thấy biểu đồ phân tán kết quả, được làm đẹp một chút bằng cách hiển thị một số hình ảnh. Thuật toán t-SNE đã xác định một số cụm khớp với các lớp khá tốt (mỗi lớp được biểu thị bằng một màu khác nhau).



Hình 17-5. Trực quan hóa Fashion MNIST bằng cách sử dụng autoencoder sau đó là t-SNE

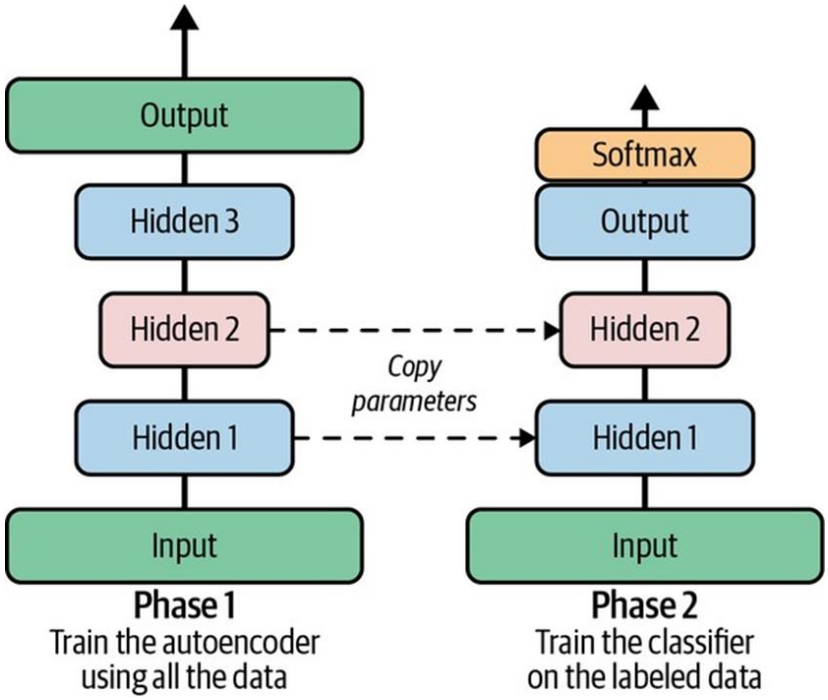
Vì vậy, autoencoder có thể được sử dụng để giảm chiều. Một ứng dụng khác là để tiền huấn luyện không giám sát.

Tiền huấn luyện không giám sát bằng Autoencoder xếp chồng

Như chúng ta đã thảo luận trong Chương 11, nếu bạn đang giải quyết một tác vụ giám sát phức tạp nhưng bạn không có nhiều dữ liệu huấn luyện được gán nhãn, một giải pháp là tìm một mạng thần kinh thực hiện một tác vụ tương tự và sử dụng lại các lớp thấp hơn của nó. Điều này giúp có thể huấn luyện một mô hình hiệu suất cao bằng cách sử dụng ít dữ liệu huấn luyện vì mạng thần kinh của bạn sẽ không phải học tất cả các đặc trưng cấp thấp; nó sẽ chỉ sử dụng lại các bộ phát hiện đặc trưng được học bởi mạng hiện có.

Tương tự, nếu bạn có một tập dữ liệu lớn nhưng hầu hết không được gán nhãn, bạn có thể huấn luyện trước một autoencoder xếp chồng bằng cách sử dụng tất cả dữ liệu, sau đó sử

dụng lại các lớp thấp hơn để tạo một mạng thần kinh cho tác vụ thực tế của bạn và huấn luyện nó bằng cách sử dụng dữ liệu được gán nhãn. Ví dụ, Hình 17-6 cho thấy cách sử dụng một autoencoder xếp chồng để thực hiện tiền huấn luyện không giám sát cho một mạng thần kinh phân loại. Khi huấn luyện bộ phân loại, nếu bạn thực sự không có nhiều dữ liệu huấn luyện được gán nhãn, bạn có thể muốn đóng băng các lớp được tiền huấn luyện (ít nhất là các lớp thấp hơn).



Hình 17-6. Tiền huấn luyện không giám sát bằng cách sử dụng autoencoder

Không có gì đặc biệt về việc triển khai: chỉ cần huấn luyện một autoencoder bằng cách sử dụng tất cả dữ liệu huấn luyện (có gán nhãn và không gán nhãn), sau đó sử dụng lại các lớp mã hóa của nó để tạo một mạng thần kinh mới (xem các bài tập ở cuối chương này để biết ví dụ).

Tiếp theo, hãy xem xét một vài kỹ thuật để huấn luyện autoencoder xếp chồng.

Ràng buộc trọng số (Tying Weights)

Khi một autoencoder đối xứng gọn gàng, giống như cái chúng ta vừa xây dựng, một kỹ thuật phổ biến là ràng buộc trọng số của các lớp giải mã với trọng số của các lớp mã hóa. Điều này làm giảm một nửa số trọng số trong mô hình, tăng tốc độ huấn luyện và hạn chế rủi ro quá khớp. Cụ thể, nếu autoencoder có tổng số `N` lớp (không tính lớp đầu vào), và `W_L` biểu thị trọng số kết nối của lớp thứ `L` (ví dụ: lớp 1 là lớp ẩn đầu tiên, lớp `N/2` là lớp mã hóa, và lớp `N` là lớp đầu ra), thì trọng số lớp giải mã có thể được định nghĩa là $W_L = W_{N-L+1}^T$ (với `L = N/2 + 1, ..., N`).

Để ràng buộc trọng số giữa các lớp bằng Keras, hãy định nghĩa một lớp tùy chỉnh:

```
import tensorflow as tf

class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.biases = self.add_weight(name="bias",
                                      shape=self.dense.input_shape[-1],
                                      initializer="zeros")
        super().build(batch_input_shape)

    def call(self, inputs):
        Z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)
        return self.activation(Z + self.biases)
```

Lớp tùy chỉnh này hoạt động giống như một lớp `Dense` thông thường, nhưng nó sử dụng trọng số của một lớp `Dense` khác, được chuyển vị (đặt `transpose_b=True` tương đương với việc chuyển vị đối số thứ hai, nhưng hiệu quả hơn vì nó thực hiện phép chuyển vị ngay lập tức trong hoạt động `matmul()`). Tuy nhiên, nó sử dụng vector độ lệch riêng của nó. Bây giờ chúng ta có thể xây dựng một autoencoder xếp chồng mới, rất giống với cái trước nhưng với các lớp `Dense` của bộ giải mã được ràng buộc với các lớp `Dense` của bộ mã hóa:

```
import tensorflow as tf

dense_1 = tf.keras.layers.Dense(100, activation="relu")
dense_2 = tf.keras.layers.Dense(30, activation="relu")

tied_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    dense_1,
    dense_2
])

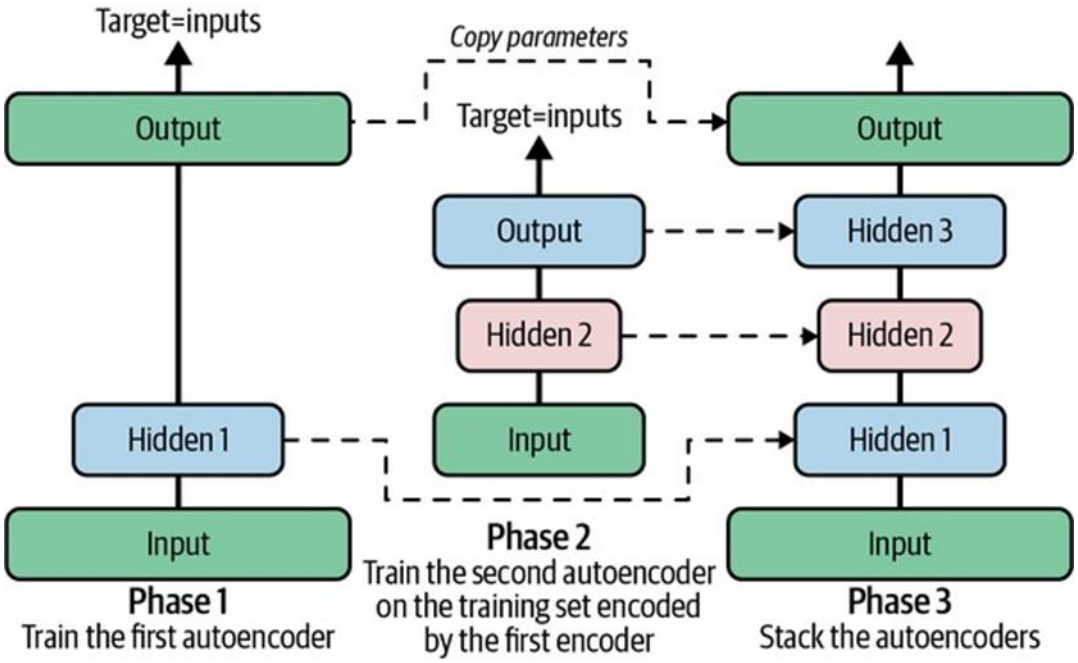
tied_decoder = tf.keras.Sequential([
    DenseTranspose(dense_2, activation="relu"),
    DenseTranspose(dense_1),
    tf.keras.layers.Reshape([28, 28])
])

tied_ae = tf.keras.Sequential([tied_encoder, tied_decoder])
```

Mô hình này đạt được sai số tái tạo xấp xỉ tương tự như mô hình trước, sử dụng gần một nửa số tham số.

Huấn luyện từng Autoencoder một

Thay vì huấn luyện toàn bộ autoencoder xếp chồng trong một lần như chúng ta vừa làm, có thể huấn luyện từng autoencoder nông một, sau đó xếp chồng tất cả chúng thành một autoencoder xếp chồng duy nhất (do đó có tên gọi này), như được hiển thị trong Hình 17-7. Kỹ thuật này ngày nay không được sử dụng nhiều, nhưng bạn vẫn có thể gặp các bài báo nói về "greedy layerwise training", vì vậy điều quan trọng là phải biết ý nghĩa của nó.



Hình 17-7. Huấn luyện từng autoencoder một

Trong giai đoạn huấn luyện đầu tiên, autoencoder đầu tiên học cách tái tạo đầu vào. Sau đó, chúng ta mã hóa toàn bộ tập huấn luyện bằng cách sử dụng autoencoder đầu tiên này, và điều này mang lại cho chúng ta một tập huấn luyện mới (đã nén). Chúng ta sau đó huấn luyện autoencoder thứ hai trên tập dữ liệu mới này. Đây là giai đoạn huấn luyện thứ hai. Cuối cùng, chúng ta xây dựng một "chiếc bánh sandwich" lớn bằng cách sử dụng tất cả các autoencoder này, như được hiển thị trong Hình 17-7 (tức là chúng ta đầu tiên xếp chồng các lớp ẩn của mỗi autoencoder, sau đó là các lớp đầu ra theo thứ tự ngược lại). Điều này mang lại cho chúng ta autoencoder xếp chồng cuối cùng (xem phần "Huấn luyện từng Autoencoder một" trong sổ tay của chương để biết ví dụ triển khai). Chúng ta có thể dễ dàng huấn luyện nhiều autoencoder hơn theo cách này, xây dựng một autoencoder xếp chồng rất sâu.

Như tôi đã đề cập trước đây, một trong những yếu tố kích hoạt "sóng thần học sâu" là việc phát hiện vào năm 2006 bởi Geoffrey Hinton và cộng sự rằng các mạng thần kinh sâu có thể được tiền huấn luyện một cách không giám sát, sử dụng phương pháp "greedy layerwise" này. Họ đã sử dụng máy Boltzmann hạn chế (RBM; xem https://homl.info/extra-anns) cho mục đích này, nhưng vào năm 2007 Yoshua Bengio và cộng sự đã chỉ ra rằng autoencoder cũng hoạt động tốt. Trong vài năm, đây là cách duy nhất hiệu quả để huấn

luyện các mạng sâu, cho đến khi nhiều kỹ thuật được giới thiệu trong Chương 11 giúp có thể huấn luyện một mạng sâu trong một lần.

Autoencoder không giới hạn ở các mạng dày đặc: bạn cũng có thể xây dựng autoencoder tích chập. Hãy xem xét chúng bây giờ.

Autoencoder tích chập (Convolutional Autoencoders)

Nếu bạn đang xử lý hình ảnh, thì các autoencoder chúng ta đã thấy cho đến nay sẽ không hoạt động tốt (trừ khi hình ảnh rất nhỏ): như bạn đã thấy trong Chương 14, mạng thần kinh tích chập phù hợp hơn nhiều so với mạng dày đặc để làm việc với hình ảnh. Vì vậy, nếu bạn muốn xây dựng một autoencoder cho hình ảnh (ví dụ: để tiền huấn luyện không giám sát hoặc giảm chiều), bạn sẽ cần xây dựng một autoencoder tích chập. Bộ mã hóa là một CNN thông thường bao gồm các lớp tích chập và lớp gộp. Nó thường làm giảm chiều không gian của đầu vào (tức là chiều cao và chiều rộng) trong khi tăng chiều sâu (tức là số lượng bản đồ đặc trưng). Bộ giải mã phải làm ngược lại (phóng to hình ảnh và giảm chiều sâu của nó trở lại kích thước gốc), và đối với điều này, bạn có thể sử dụng các lớp tích chập chuyển vị (transpose convolutional layers) (cách khác, bạn có thể kết hợp các lớp lấy mẫu lên (upsampling layers) với các lớp tích chập). Dưới đây là một autoencoder tích chập cơ bản cho Fashion MNIST:

```
import tensorflow as tf

conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),  # output: 14 × 14 x 16
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),  # output: 7 × 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),  # output: 3 × 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same", activation="relu"),
    tf.keras.layers.GlobalAvgPool2D()  # output: 30
])

conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16),
    tf.keras.layers.Reshape((3, 3, 16)),
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2, activation="relu"),
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same",
                                    activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2, padding="same"),
    tf.keras.layers.Reshape([28, 28])
])

conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder])
```

Cũng có thể tạo autoencoder với các loại kiến trúc khác, chẳng hạn như RNN (xem sổ tay để biết ví dụ).

Được rồi, hãy lùi lại một chút. Cho đến nay, chúng ta đã xem xét các loại autoencoder khác nhau (cơ bản, xếp chồng và tích chập), và cách huấn luyện chúng (hoặc trong một lần hoặc từng lớp một). Chúng ta cũng đã xem xét một vài ứng dụng: trực quan hóa dữ liệu và tiền huấn luyện không giám sát.

Cho đến bây giờ, để buộc autoencoder học các đặc trưng thú vị, chúng ta đã giới hạn kích thước của lớp mã hóa, làm cho nó thiếu hoàn chỉnh. Thực tế có nhiều loại ràng buộc khác có thể được sử dụng, bao gồm những ràng buộc cho phép lớp mã hóa lớn bằng đầu vào, hoặc thậm chí lớn hơn, dẫn đến một autoencoder thừa hoàn chỉnh (overcomplete). Sau đó, trong các phần sau, chúng ta sẽ xem xét thêm một vài loại autoencoder: autoencoder khử nhiễu (denoising autoencoders), autoencoder thưa (sparse autoencoders) và autoencoder biến phân (variational autoencoders).

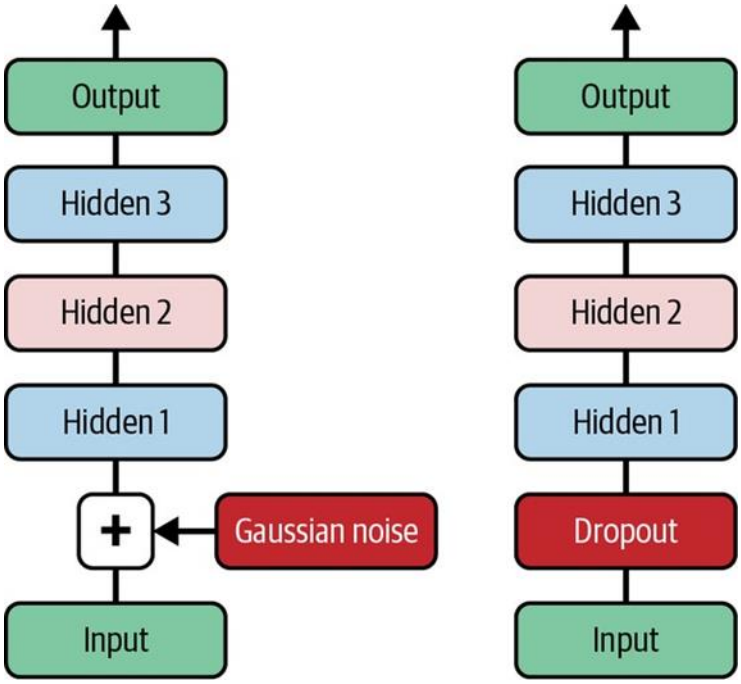
Autoencoder khử nhiễu (Denoising Autoencoders)

Một cách khác để buộc autoencoder học các đặc trưng hữu ích là thêm nhiễu vào đầu vào của nó, huấn luyện nó để khôi phục lại các đầu vào gốc, không nhiễu. Ý tưởng này đã tồn tại từ những năm 1980 (ví dụ, nó được đề cập trong luận văn thạc sĩ năm 1987 của Yann LeCun). Trong một bài báo năm 2008, Pascal Vincent và cộng sự đã chỉ ra rằng autoencoder cũng có thể được sử dụng để trích xuất đặc trưng. Trong một bài báo năm 2010, Vincent và cộng sự đã giới thiệu autoencoder khử nhiễu xếp chồng.

Nhiễu có thể là nhiễu Gaussian thuần túy được thêm vào đầu vào, hoặc nó có thể là các đầu vào bị tắt ngẫu nhiên, giống như trong dropout (được giới thiệu trong Chương 11). Hình 17-8 cho thấy cả hai tùy chọn. Việc triển khai rất đơn giản: đó là một autoencoder xếp chồng thông thường với một lớp `Dropout` bổ sung được áp dụng cho đầu vào của bộ mã hóa (hoặc bạn có thể sử dụng lớp `GaussianNoise` thay thế). <1>Hãy nhớ rằng lớp `Dropout` chỉ hoạt động trong quá trình huấn luyện (và lớp `GaussianNoise` cũng vậy):</1>

```
import tensorflow as tf

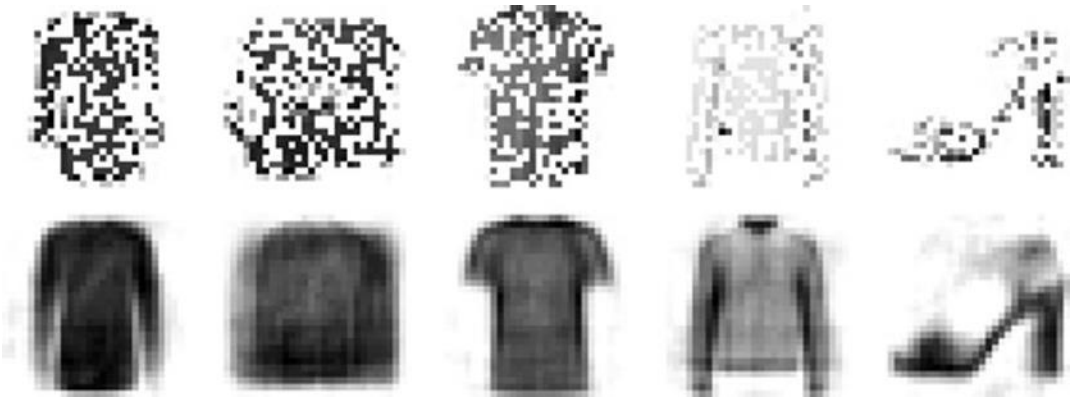
dropout_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5), # Tỷ lệ dropout 0.5
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu")
])
dropout_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
dropout_ae = tf.keras.Sequential([dropout_encoder, dropout_decoder])
```



Hình 17-8. Autoencoder khử nhiễu, với nhiễu Gaussian (trái) hoặc dropout (phải)

Hình 17-9 cho thấy một vài hình ảnh bị nhiễu (với một nửa số pixel bị tắt), và các hình ảnh được tái tạo bởi autoencoder khử nhiễu dựa trên dropout.

Lưu ý cách autoencoder đoán các chi tiết thực sự không có trong đầu vào, chẳng hạn như phần trên của chiếc áo sơ mi trắng (hàng dưới cùng, hình thứ tư). Như bạn có thể thấy, autoencoder khử nhiễu không chỉ có thể được sử dụng để trực quan hóa dữ liệu hoặc tiền huấn luyện không giám sát, giống như các autoencoder khác mà chúng ta đã thảo luận cho đến nay, mà chúng còn có thể được sử dụng khá đơn giản và hiệu quả để loại bỏ nhiễu khỏi hình ảnh.



Hình 17-9. Hình ảnh bị nhiễu (trên cùng) và các bản tái tạo của chúng (dưới cùng)

Autoencoder thưa (Sparse Autoencoders)

Một loại ràng buộc khác thường dẫn đến trích xuất đặc trưng tốt là tính thưa thớt: bằng cách thêm một thuật ngữ thích hợp vào hàm chi phí, autoencoder được đẩy để giảm số lượng neuron hoạt động trong lớp mã hóa. Ví dụ, nó có thể được đẩy để trung bình chỉ có 5% các neuron hoạt động đáng kể trong lớp mã hóa. Điều này buộc autoencoder phải biểu diễn mỗi đầu vào như một sự kết hợp của một số lượng nhỏ các kích hoạt. Kết quả là, mỗi neuron trong lớp mã hóa thường kết thúc biểu diễn một đặc trưng hữu ích (nếu bạn chỉ có thể nói vài từ mỗi tháng, bạn có lẽ sẽ cố gắng làm cho chúng đáng nghe).

Một cách tiếp cận đơn giản là sử dụng hàm kích hoạt sigmoid trong lớp mã hóa (để ràng buộc các mã hóa với các giá trị từ 0 đến 1), sử dụng một lớp mã hóa lớn (ví dụ, với 300 đơn vị), và thêm một số chuẩn hóa L1 vào các kích hoạt của lớp mã hóa. Bộ giải mã chỉ là một bộ giải mã thông thường:

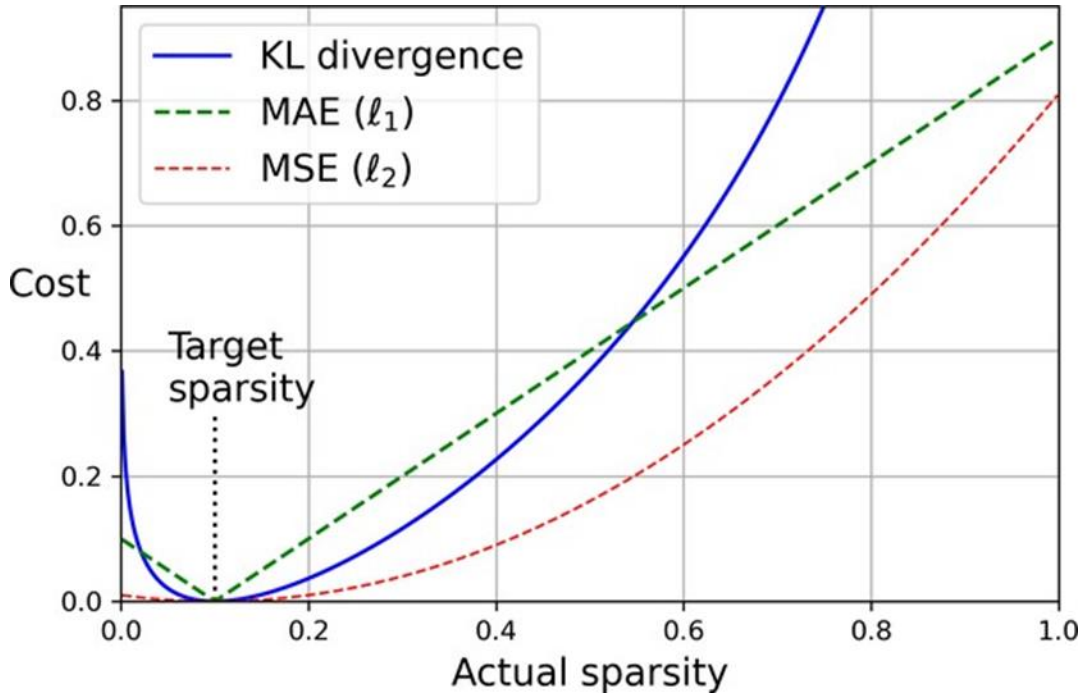
```
import tensorflow as tf

sparse_l1_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid"),
    tf.keras.layers.ActivityRegularization(l1=1e-4) # Chuẩn hóa L1
])
sparse_l1_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
sparse_l1_ae = tf.keras.Sequential([sparse_l1_encoder, sparse_l1_decoder])
```

Lớp `ActivityRegularization` này chỉ trả về đầu vào của nó, nhưng như một tác dụng phụ, nó thêm một mất huấn luyện bằng tổng các giá trị tuyệt đối của đầu vào của nó. Điều này chỉ ảnh hưởng đến quá trình huấn luyện. Tương đương, bạn có thể loại bỏ lớp `ActivityRegularization` và đặt `activity_regularizer=tf.keras.regularizers.l1(1e-4)` trong lớp trước đó. Hình phạt này sẽ khuyến khích mạng thần kinh tạo ra các mã hóa gần 0, nhưng vì nó cũng sẽ bị phạt nếu không tái tạo đúng đầu vào, nó sẽ phải xuất ít nhất một vài giá trị khác 0. Việc sử dụng chuẩn L1 thay vì chuẩn L2 sẽ đẩy mạng thần kinh bảo toàn các mã hóa quan trọng nhất trong khi loại bỏ những mã hóa không cần thiết cho hình ảnh đầu vào (thay vì chỉ giảm tất cả các mã hóa).

Một cách tiếp cận khác, thường mang lại kết quả tốt hơn, là đo lường tính thưa thớt thực tế của lớp mã hóa ở mỗi lần lặp huấn luyện và phạt mô hình khi tính thưa thớt đo được khác với tính thưa thớt mục tiêu. Chúng ta làm như vậy bằng cách tính kích hoạt trung bình của mỗi neuron trong lớp mã hóa, trên toàn bộ batch huấn luyện. Kích thước batch không được quá nhỏ, nếu không giá trị trung bình sẽ không chính xác.

Một khi chúng ta có kích hoạt trung bình trên mỗi neuron, chúng ta muốn phạt các neuron quá hoạt động, hoặc không đủ hoạt động, bằng cách thêm một mất thưa thớt vào hàm chi phí. Ví dụ, nếu chúng ta đo được rằng một neuron có kích hoạt trung bình là 0.3, nhưng tính thưa thớt mục tiêu là 0.1, nó phải bị phạt để kích hoạt ít hơn. Một cách tiếp cận có thể là đơn giản thêm sai số bình phương (0.3 – 0.1)² vào hàm chi phí, nhưng trong thực tế, một cách tiếp cận tốt hơn là sử dụng độ phân kỳ Kullback–Leibler (KL) (đã được thảo luận ngắn gọn trong Chương 4), có gradient mạnh hơn nhiều so với sai số bình phương trung bình, như bạn có thể thấy trong Hình 17-10.



Hình 17-10. Mất thưa thớt

Với hai phân phối xác suất rời rạc P và Q, độ phân kỳ KL giữa các phân phối này, ký hiệu D_{KL}(P ‖ Q), có thể được tính bằng Công thức 17-1.

Công thức 17-1. Độ phân kỳ Kullback–Leibler

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Trong trường hợp của chúng ta, chúng ta muốn đo độ phân kỳ giữa xác suất mục tiêu p mà một neuron trong lớp mã hóa sẽ kích hoạt và xác suất thực tế q, được ước tính bằng cách đo kích hoạt trung bình trên batch huấn luyện. Vì vậy, độ phân kỳ KL được đơn giản hóa thành Công thức 17-2.

Công thức 17-2. Độ phân kỳ KL giữa tính thưa thớt mục tiêu p và tính thưa thớt thực tế q

$$D_{\mathrm{KL}}\left(p \parallel q\right) = p\ \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}$$

Một khi chúng ta đã tính mất thưa thớt cho mỗi neuron trong lớp mã hóa, chúng ta tổng hợp các mất này và thêm kết quả vào hàm chi phí. Để kiểm soát tầm quan trọng tương đối của mất thưa thớt và mất tái tạo, chúng ta có thể nhân mất thưa thớt với một siêu tham số trọng số thưa thớt. Nếu trọng số này quá cao, mô hình sẽ bám sát tính thưa thớt mục tiêu, nhưng nó có thể không tái tạo đầu vào đúng cách, làm cho mô hình trở nên vô dụng. Ngược lại, nếu nó quá thấp, mô hình sẽ bỏ qua mục tiêu thưa thớt và sẽ không học được bất kỳ đặc trưng thú vị nào.

Bây giờ chúng ta có tất cả những gì cần thiết để triển khai một autoencoder thưa dựa trên độ phân kỳ KL. Đầu tiên, hãy tạo một bộ chuẩn hóa tùy chỉnh để áp dụng chuẩn hóa độ phân kỳ KL:

```
import tensorflow as tf

kl_divergence = tf.keras.losses.kullback_leibler_divergence

class KLDivergenceRegularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, weight, target):
        self.weight = weight
        self.target = target

    def __call__(self, inputs):
        mean_activities = tf.reduce_mean(inputs, axis=0)
        return self.weight * (
            kl_divergence(self.target, mean_activities) +
            kl_divergence(1. - self.target, 1. - mean_activities)
        )
```

Bây giờ chúng ta có thể xây dựng autoencoder thưa, sử dụng `KLDivergenceRegularizer` cho các kích hoạt của lớp mã hóa:

```
import tensorflow as tf

kld_reg = KLDivergenceRegularizer(weight=5e-3, target=0.1)
sparse_kl_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid",
                          activity_regularizer=kld_reg) # Áp dụng bộ chuẩn
hóa KL
])
sparse_kl_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
```

```
])
sparse_kl_ae = tf.keras.Sequential([sparse_kl_encoder, sparse_kl_decoder])
```

Sau khi huấn luyện autoencoder thưa này trên Fashion MNIST, lớp mã hóa sẽ có độ thưa thớt khoảng 10%.

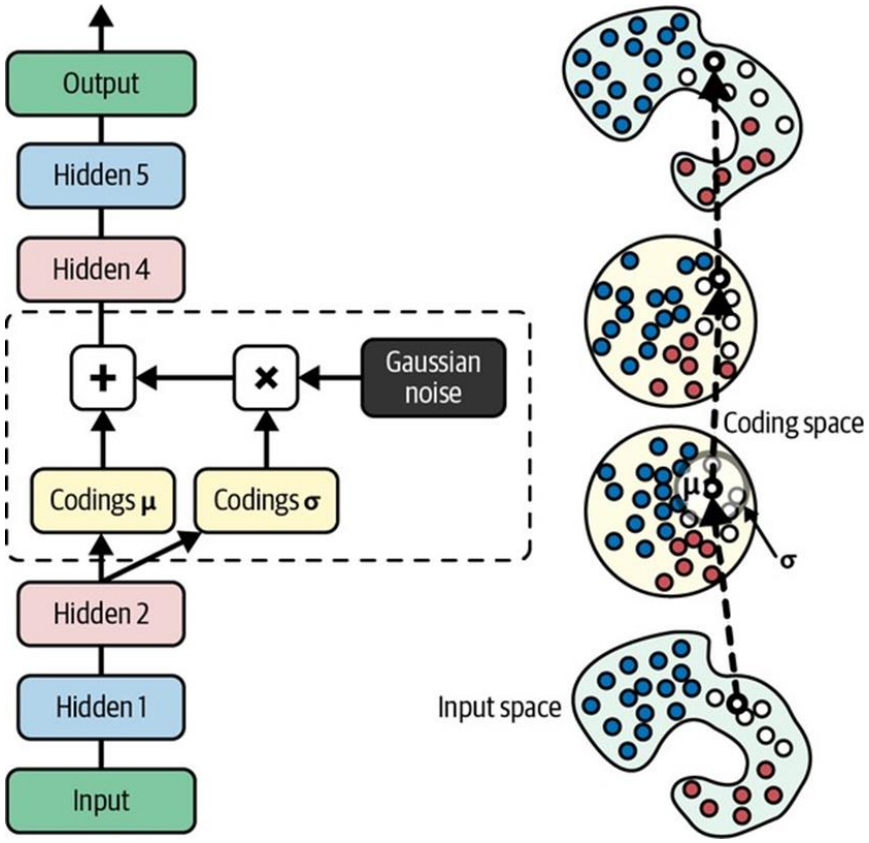
Autoencoder biến phân (Variational Autoencoders - VAEs)

Một loại autoencoder quan trọng đã được giới thiệu vào năm 2013 bởi Diederik Kingma và Max Welling và nhanh chóng trở thành một trong những biến thể phổ biến nhất: autoencoder biến phân (VAEs). VAEs khá khác biệt so với tất cả các autoencoder chúng ta đã thảo luận cho đến nay, theo những cách đặc biệt sau:

- Chúng là các autoencoder xác suất, nghĩa là đầu ra của chúng một phần được xác định bởi sự ngẫu nhiên, ngay cả sau khi huấn luyện (ngược lại với autoencoder khử nhiễu, chỉ sử dụng tính ngẫu nhiên trong quá trình huấn luyện).
- Quan trọng nhất, chúng là các autoencoder sinh, nghĩa là chúng có thể tạo ra các trường hợp mới trông giống như chúng được lấy mẫu từ tập huấn luyện.

Cả hai thuộc tính này làm cho VAEs khá giống với RBMs, nhưng chúng dễ huấn luyện hơn và quá trình lấy mẫu nhanh hơn nhiều (với RBMs bạn cần đợi mạng ổn định thành "cân bằng nhiệt" trước khi có thể lấy mẫu một trường hợp mới). Như tên gọi của chúng, autoencoder biến phân thực hiện suy luận Bayes biến phân, đây là một cách hiệu quả để thực hiện suy luận Bayes gần đúng. Nhớ lại rằng suy luận Bayes có nghĩa là cập nhật một phân phối xác suất dựa trên dữ liệu mới, sử dụng các phương trình được lấy từ định lý Bayes. Phân phối gốc được gọi là tiền định (prior), trong khi phân phối được cập nhật được gọi là hậu định (posterior). Trong trường hợp của chúng ta, chúng ta muốn tìm một xấp xỉ tốt của phân phối dữ liệu. Một khi chúng ta có điều đó, chúng ta có thể lấy mẫu từ nó.

Hãy cùng xem VAEs hoạt động như thế nào. Hình 17-11 (trái) cho thấy một autoencoder biến phân. Bạn có thể nhận ra cấu trúc cơ bản của tất cả các autoencoder, với một bộ mã hóa theo sau bởi một bộ giải mã (trong ví dụ này, cả hai đều có hai lớp ẩn), nhưng có một điểm khác biệt: thay vì trực tiếp tạo ra một mã hóa cho một đầu vào nhất định, bộ mã hóa tạo ra một mã hóa trung bình μ và một độ lệch chuẩn σ. Mã hóa thực tế sau đó được lấy mẫu ngẫu nhiên từ một phân phối Gaussian với trung bình μ và độ lệch chuẩn σ. Sau đó, bộ giải mã giải mã mã hóa đã lấy mẫu một cách bình thường. Phần bên phải của sơ đồ cho thấy một trường hợp huấn luyện đi qua autoencoder này. Đầu tiên, bộ mã hóa tạo ra μ và σ, sau đó một mã hóa được lấy mẫu ngẫu nhiên (lưu ý rằng nó không nằm chính xác tại μ), và cuối cùng mã hóa này được giải mã; đầu ra cuối cùng giống với trường hợp huấn luyện.



Hình 17-11. Một autoencoder biến phân (trái) và một trường hợp đi qua nó (phải)

Như bạn có thể thấy trong sơ đồ, mặc dù đầu vào có thể có một phân phối rất phức tạp, một autoencoder biến phân có xu hướng tạo ra các mã hóa trông như thể chúng được lấy mẫu từ một phân phối Gaussian đơn giản: trong quá trình huấn luyện, hàm chi phí (được thảo luận tiếp theo) đẩy các mã hóa dần dần di chuyển trong không gian mã hóa (còn gọi là không gian tiềm ẩn) để cuối cùng trông giống như một đám mây các điểm Gaussian. Một hệ quả tuyệt vời là sau khi huấn luyện một autoencoder biến phân, bạn có thể rất dễ dàng tạo ra một trường hợp mới: chỉ cần lấy mẫu một mã hóa ngẫu nhiên từ phân phối Gaussian, giải mã nó, và thế là xong!

Bây giờ, hãy xem xét hàm chi phí. Nó bao gồm hai phần. Phần đầu tiên là mất tái tạo thông thường đẩy autoencoder tái tạo lại đầu vào của nó. Chúng ta có thể sử dụng MSE cho việc này, như chúng ta đã làm trước đây. Phần thứ hai là mất tiềm ẩn đẩy autoencoder có các mã hóa trông như thể chúng được lấy mẫu từ một phân phối Gaussian đơn giản: đó là độ phân kỳ KL giữa phân phối mục tiêu (tức là phân phối Gaussian) và phân phối thực tế của các mã hóa. Toán học phức tạp hơn một chút so với autoencoder thưa, đặc biệt là do nhiễu Gaussian, giới hạn lượng thông tin có thể được truyền đến lớp mã hóa. Điều này thúc đẩy autoencoder học các đặc trưng hữu ích. May mắn thay, các phương trình được đơn giản hóa, vì vậy mất tiềm ẩn có thể được tính bằng Công thức 17-3.

Công thức 17-3. Mất tiềm ẩn của autoencoder biến phân

$$\mathcal{L} = -\frac{1}{2}\sum_{i=1}^{n}\left[1 + \log\left({\sigma_i}^2\right) - {\sigma_i}^2 - {\mu_i}^2\right]$$

Trong phương trình này, `L` là mất tiềm ẩn, `n` là chiều của các mã hóa, và `μ_i` và `σ_i` là trung bình và độ lệch chuẩn của thành phần thứ `i` của các mã hóa. Các vector μ và σ (chứa tất cả các `μ_i` và `σ_i`) được xuất bởi bộ mã hóa, như được hiển thị trong Hình 17-11 (trái).

Một điều chỉnh phổ biến cho kiến trúc của autoencoder biến phân là làm cho bộ mã hóa xuất `γ = log(σ^2)` thay vì σ. Mất tiềm ẩn sau đó có thể được tính như trong Công thức 17-4. Cách tiếp cận này ổn định hơn về mặt số học và tăng tốc độ huấn luyện.

Công thức 17-4. Mất tiềm ẩn của autoencoder biến phân, được viết lại bằng `γ = log(σ^2)`

$$\mathcal{L} = -\frac{1}{2}\sum_{i=1}^{n}\left[1 + \gamma_i - \exp\left(\gamma_i\right) - {\mu_i}^2\right]$$

Hãy bắt đầu xây dựng một autoencoder biến phân cho Fashion MNIST (như được hiển thị trong Hình 17-11, nhưng sử dụng điều chỉnh γ). Đầu tiên, chúng ta sẽ cần một lớp tùy chỉnh để lấy mẫu các mã hóa, với μ và γ:

```
import tensorflow as tf

class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        mean, log_var = inputs
        return tf.random.normal(tf.shape(log_var)) * tf.exp(log_var / 2) +
mean
```

Lớp `Sampling` này nhận hai đầu vào: `mean` (μ) và `log_var` (γ). Nó sử dụng hàm `tf.random.normal()` để lấy mẫu một vector ngẫu nhiên (có cùng hình dạng với γ) từ phân phối Gaussian, với trung bình 0 và độ lệch chuẩn 1. Sau đó, nó nhân với `exp(γ / 2)` (bằng σ, như bạn có thể kiểm tra bằng toán học), và cuối cùng nó cộng μ và trả về kết quả. Điều này lấy mẫu một vector mã hóa từ phân phối Gaussian với trung bình μ và độ lệch chuẩn σ.

Tiếp theo, chúng ta có thể tạo bộ mã hóa, sử dụng API hàm vì mô hình không hoàn toàn tuần tự:

```
import tensorflow as tf

codings_size = 10

inputs = tf.keras.layers.Input(shape=[28, 28])
Z = tf.keras.layers.Flatten()(inputs)
Z = tf.keras.layers.Dense(150, activation="relu")(Z)
Z = tf.keras.layers.Dense(100, activation="relu")(Z)
codings_mean = tf.keras.layers.Dense(codings_size)(Z) # μ
codings_log_var = tf.keras.layers.Dense(codings_size)(Z) # γ
codings = Sampling()([codings_mean, codings_log_var])
```

```
variational_encoder = tf.keras.Model(
    inputs=[inputs], outputs=[codings_mean, codings_log_var, codings])
```

Lưu ý rằng các lớp Dense xuất codings_mean (μ) và codings_log_var (γ) có cùng đầu vào (tức là đầu ra của lớp Dense thứ hai). Sau đó, chúng ta truyền cả codings_mean và codings_log_var cho lớp Sampling. Cuối cùng, mô hình variational_encoder có ba đầu ra. Chỉ cần các mã hóa, nhưng chúng ta cũng thêm codings_mean và codings_log_var, phòng khi chúng ta muốn kiểm tra giá trị của chúng. Bây giờ hãy xây dựng bộ giải mã:

```
import tensorflow as tf

decoder_inputs = tf.keras.layers.Input(shape=[codings_size])
x = tf.keras.layers.Dense(100, activation="relu")(decoder_inputs)
x = tf.keras.layers.Dense(150, activation="relu")(x)
x = tf.keras.layers.Dense(28 * 28)(x)
outputs = tf.keras.layers.Reshape([28, 28])(x)
variational_decoder = tf.keras.Model(inputs=[decoder_inputs],
outputs=[outputs])
```

Đối với bộ giải mã này, chúng ta có thể đã sử dụng API tuần tự thay vì API hàm, vì nó thực sự chỉ là một chồng lớp đơn giản, gần như giống hệt với nhiều bộ giải mã chúng ta đã xây dựng cho đến nay. Cuối cùng, hãy xây dựng mô hình autoencoder biến phân:

```
import tensorflow as tf

# Đảm bảo inputs đã được định nghĩa từ phần variational_encoder
# _, _, codings = variational_encoder(inputs) # Lấy codings từ encoder đã tạo
reconstructions = variational_decoder(codings)
variational_ae = tf.keras.Model(inputs=[inputs], outputs=[reconstructions])
```

Chúng ta bỏ qua hai đầu ra đầu tiên của bộ mã hóa (chúng ta chỉ muốn cấp các mã hóa cho bộ giải mã). Cuối cùng, chúng ta phải thêm mất tiềm ẩn và mất tái tạo:

```
import tensorflow as tf

latent_loss = -0.5 * tf.reduce_sum(
    1 + codings_log_var - tf.exp(codings_log_var) - tf.square(codings_mean),
    axis=-1)
variational_ae.add_loss(tf.reduce_mean(latent_loss) / 784.)
```

Chúng ta đầu tiên áp dụng Công thức 17-4 để tính mất tiềm ẩn cho mỗi trường hợp trong batch, tổng hợp trên trục cuối cùng. Sau đó, chúng ta tính mất trung bình trên tất cả các trường hợp trong batch, và chúng ta chia kết quả cho 784 để đảm bảo nó có tỷ lệ thích hợp so với mất tái tạo. Thật vậy, mất tái tạo của autoencoder biến phân được cho là tổng các lỗi tái tạo pixel, nhưng khi Keras tính mất "mse", nó tính giá trị trung bình trên tất cả 784 pixel, thay vì tổng. Vì vậy, mất tái tạo nhỏ hơn 784 lần so với mức chúng ta cần. Chúng ta có thể định nghĩa một hàm mất tùy chỉnh để tính tổng thay vì trung bình, nhưng đơn giản hơn là chia mất tiềm ẩn cho 784 (tổng mất cuối cùng sẽ nhỏ hơn 784 lần so với đáng lẽ, nhưng điều này chỉ có nghĩa là chúng ta nên sử dụng tốc độ học lớn hơn).

Và cuối cùng, chúng ta có thể biên dịch và huấn luyện autoencoder!

```
import tensorflow as tf

# Giả định X_train và X_valid đã được định nghĩa từ ngữ cảnh trước đó
variational_ae.compile(loss="mse", optimizer="nadam")
# history = variational_ae.fit(X_train, X_train, epochs=25, batch_size=128,
#                              validation_data=(X_valid, X_valid))
```

Tạo ảnh Fashion MNIST

Bây giờ chúng ta hãy sử dụng autoencoder biến phân này để tạo ra những hình ảnh trông giống các mặt hàng thời trang. Tất cả những gì chúng ta cần làm là lấy mẫu các mã hóa ngẫu nhiên từ một phân phối Gaussian và giải mã chúng:

```
import tensorflow as tf
import numpy as np # Đảm bảo import numpy nếu chưa có

# Giả định codings_size đã được định nghĩa là 10 từ phần Variational
Autoencoders trước đó
# Giả định variational_decoder đã được xây dựng và có sẵn từ ngữ cảnh trước
đó

# codings = tf.random.normal(shape=[3 * 7, codings_size])
# images = variational_decoder(codings).numpy()
```

Hình 17-12 cho thấy 12 hình ảnh được tạo ra.

Hình 17-12. Hình ảnh Fashion MNIST được tạo bởi autoencoder biến phân

Phần lớn các hình ảnh này trông khá thuyết phục, mặc dù hơi mờ. Phần còn lại thì không đẹp lắm, nhưng đừng quá khắc nghiệt với autoencoder—nó chỉ có vài phút để học!

Autoencoder biến phân giúp có thể thực hiện phép nội suy ngữ nghĩa: thay vì nội suy giữa hai hình ảnh ở cấp độ pixel, điều này sẽ trông như thể hai hình ảnh chỉ được phủ lên nhau, chúng ta có thể nội suy ở cấp độ mã hóa. Ví dụ, chúng ta hãy lấy một vài mã hóa dọc theo

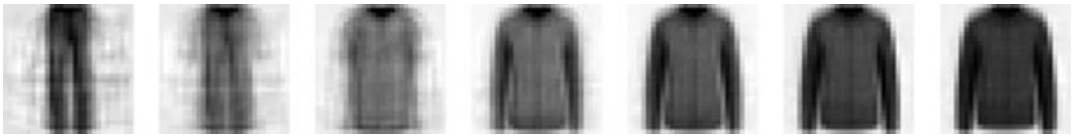
một đường tùy ý trong không gian tiềm ẩn và giải mã chúng. Chúng ta sẽ có một chuỗi hình ảnh dần dần chuyển từ quần sang áo len (xem Hình 17-13):

```
import numpy as np
import matplotlib.pyplot as plt # Đảm bảo import matplotlib.pyplot nếu chưa
có

# Giả định codings_size đã được định nghĩa là 10 từ phần Variational
Autoencoders trước đó
# Giả định variational_decoder đã được xây dựng và có sẵn từ ngữ cảnh trước
đó

# codings = np.zeros([7, codings_size])
# codings[:, 3] = np.linspace(-0.8, 0.8, 7) #trục 3 trông tốt nhất trong
trường hợp này
# images = variational_decoder(codings).numpy()

# Để hiển thị các hình ảnh nội suy, bạn sẽ cần một hàm hiển thị tương tự
plot_reconstructions
# hoặc vòng lặp để hiển thị từng hình ảnh.
# Ví dụ:
# fig = plt.figure(figsize=(7 * 1.5, 3))
# for i in range(7):
#    plt.subplot(1, 7, 1 + i)
#    plt.imshow(images[i], cmap="binary")
#    plt.axis("off")
# plt.show()
```



Hình 17-13. Nội suy ngữ nghĩa

Bây giờ chúng ta hãy chuyển sự chú ý sang GAN: chúng khó huấn luyện hơn, nhưng khi bạn xoay sở để làm cho chúng hoạt động, chúng tạo ra những hình ảnh khá tuyệt vời.

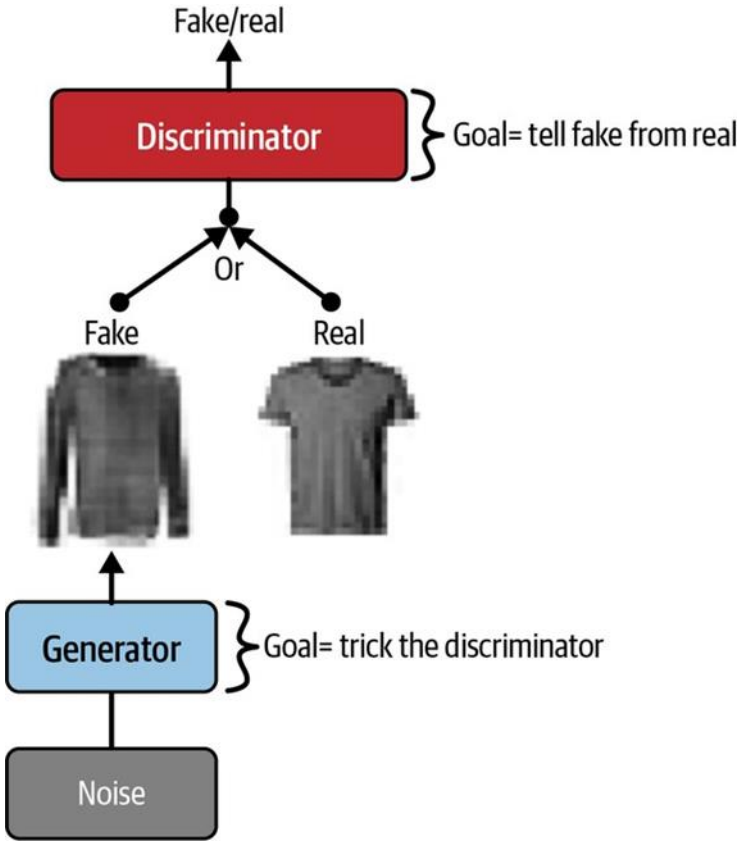
Mạng đối kháng sinh (Generative Adversarial Networks - GANs)

Mạng đối kháng sinh được đề xuất trong một bài báo năm 2014 bởi Ian Goodfellow và cộng sự, và mặc dù ý tưởng này đã khiến các nhà nghiên cứu hứng thú gần như ngay lập tức, phải mất vài năm để khắc phục một số khó khăn trong việc huấn luyện GAN. Giống như nhiều ý tưởng tuyệt vời, nó có vẻ đơn giản khi nhìn lại: làm cho các mạng thần kinh cạnh tranh với nhau với hy vọng rằng sự cạnh tranh này sẽ thúc đẩy chúng vượt trội. Như được hiển thị trong Hình 17-14, một GAN bao gồm hai mạng thần kinh:

- **Bộ tạo (Generator):** Lấy một phân phối ngẫu nhiên làm đầu vào (thường là Gaussian) và xuất ra một số dữ liệu — điển hình là một hình ảnh. Bạn có thể coi các

đầu vào ngẫu nhiên là các biểu diễn tiềm ẩn (tức là mã hóa) của hình ảnh sẽ được tạo ra. Vì vậy, như bạn có thể thấy, bộ tạo cung cấp chức năng tương tự như một bộ giải mã trong autoencoder biến phân, và nó có thể được sử dụng theo cùng một cách để tạo ra hình ảnh mới: chỉ cần cấp cho nó một số nhiễu Gaussian, và nó xuất ra một hình ảnh hoàn toàn mới. Tuy nhiên, nó được huấn luyện rất khác, như bạn sẽ sớm thấy.

- **Bộ phân biệt (Discriminator):** Lấy một hình ảnh giả từ bộ tạo hoặc một hình ảnh thật từ tập huấn luyện làm đầu vào, và phải đoán xem hình ảnh đầu vào là giả hay thật.



Hình 17-14. Một mạng đối kháng sinh

Trong quá trình huấn luyện, bộ tạo và bộ phân biệt có các mục tiêu đối lập: bộ phân biệt cố gắng phân biệt hình ảnh giả với hình ảnh thật, trong khi bộ tạo cố gắng tạo ra những hình ảnh trông đủ thật để đánh lừa bộ phân biệt.

Vì GAN bao gồm hai mạng với các mục tiêu khác nhau, nó không thể được huấn luyện như một mạng thần kinh thông thường. Mỗi lần lặp huấn luyện được chia thành hai giai đoạn:

- **Trong giai đoạn đầu tiên,** chúng ta huấn luyện bộ phân biệt. Một batch các hình ảnh thật được lấy mẫu từ tập huấn luyện và được bổ sung bằng một số lượng hình ảnh giả tương đương được tạo bởi bộ tạo. Các nhãn được đặt là 0 cho hình ảnh giả và 1 cho hình ảnh thật, và bộ phân biệt được huấn luyện trên batch được gán nhãn

này trong một bước, sử dụng hàm mất entropy chéo nhị phân. Quan trọng là, lan truyền ngược chỉ tối ưu hóa trọng số của bộ phân biệt trong giai đoạn này.

- **Trong giai đoạn thứ hai,** chúng ta huấn luyện bộ tạo. Đầu tiên chúng ta sử dụng nó để tạo ra một batch hình ảnh giả khác, và một lần nữa bộ phân biệt được sử dụng để phân biệt xem hình ảnh là giả hay thật. Lần này chúng ta không thêm hình ảnh thật vào batch, và tất cả các nhãn được đặt là 1 (thật): nói cách khác, chúng ta muốn bộ tạo tạo ra những hình ảnh mà bộ phân biệt sẽ (sai lầm) tin là thật! Quan trọng là, trọng số của bộ phân biệt bị đóng băng trong bước này, vì vậy lan truyền ngược chỉ ảnh hưởng đến trọng số của bộ tạo.

Hãy tiến hành xây dựng một GAN đơn giản cho Fashion MNIST.

Đầu tiên, chúng ta cần xây dựng bộ tạo và bộ phân biệt. Bộ tạo tương tự như bộ giải mã của autoencoder, và bộ phân biệt là một bộ phân loại nhị phân thông thường: nó nhận một hình ảnh làm đầu vào và kết thúc bằng một lớp Dense chứa một đơn vị duy nhất và sử dụng hàm kích hoạt sigmoid. Đối với giai đoạn thứ hai của mỗi lần lặp huấn luyện, chúng ta cũng cần mô hình GAN đầy đủ chứa bộ tạo theo sau bởi bộ phân biệt:

```
import tensorflow as tf

codings_size = 30

Dense = tf.keras.layers.Dense # Đặt một biến ngắn gọn cho lớp Dense
generator = tf.keras.Sequential([
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(28 * 28, activation="sigmoid"),
    tf.keras.layers.Reshape([28, 28])
])
discriminator = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(1, activation="sigmoid") # Đầu ra nhị phân (thật/giả)
])
gan = tf.keras.Sequential([generator, discriminator])
```

Tiếp theo, chúng ta cần biên dịch các mô hình này. Vì bộ phân biệt là một bộ phân loại nhị phân, chúng ta có thể tự nhiên sử dụng hàm mất entropy chéo nhị phân. Mô hình gan cũng là một bộ phân loại nhị phân, vì vậy nó cũng có thể sử dụng hàm mất entropy chéo nhị phân.

Tuy nhiên, bộ tạo sẽ chỉ được huấn luyện thông qua mô hình gan, vì vậy chúng ta không cần biên dịch nó. Quan trọng là, bộ phân biệt không nên được huấn luyện trong giai đoạn thứ hai, vì vậy chúng ta đặt nó không thể huấn luyện được trước khi biên dịch mô hình gan:

```
import tensorflow as tf
```

```
# Giả định discriminator và gan đã được định nghĩa từ mã trước đó
discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
discriminator.trainable = False # Đóng băng trọng số của discriminator
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

Vì vòng lặp huấn luyện bất thường, chúng ta không thể sử dụng phương thức `fit()` thông thường. Thay vào đó, chúng ta sẽ viết một vòng lặp huấn luyện tùy chỉnh. Đối với điều này, chúng ta cần tạo một `Dataset` để lặp qua các hình ảnh:

```
import tensorflow as tf
# Giả định X_train đã được định nghĩa và có sẵn từ ngữ cảnh trước đó

batch_size = 32
dataset =
tf.data.Dataset.from_tensor_slices(X_train).shuffle(buffer_size=1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

Bây giờ chúng ta đã sẵn sàng để viết vòng lặp huấn luyện. Hãy gói nó trong một hàm `train_gan()`:

```
import tensorflow as tf
import numpy as np # Đảm bảo import numpy nếu chưa có

def train_gan(gan, dataset, batch_size, codings_size, n_epochs):
    generator, discriminator = gan.layers

    for epoch in range(n_epochs):
        print(f"Epoch {epoch+1}/{n_epochs}")
        for X_batch in dataset:
            # Giai đoạn 1 - huấn luyện bộ phân biệt
            noise = tf.random.normal(shape=[batch_size, codings_size])
            generated_images = generator(noise)
            X_fake_and_real = tf.concat([generated_images, X_batch], axis=0)
            # Nhãn: 0 cho giả, 1 cho thật
            y1 = tf.constant([[0.]] * batch_size + [[1.]] * batch_size)
            discriminator.train_on_batch(X_fake_and_real, y1)

            # Giai đoạn 2 - huấn luyện bộ tạo
            noise = tf.random.normal(shape=[batch_size, codings_size])
            # Nhãn: tất cả là 1 (muốn bộ phân biệt nghĩ là thật)
            y2 = tf.constant([[1.]] * batch_size)
            gan.train_on_batch(noise, y2)

# Giả định gan, dataset, batch_size, codings_size đã được định nghĩa
# train_gan(gan, dataset, batch_size, codings_size, n_epochs=50)
```

Như đã thảo luận trước đó, bạn có thể thấy hai giai đoạn ở mỗi lần lặp:

- **Trong giai đoạn một,** chúng ta cấp nhiễu Gaussian cho bộ tạo để tạo ra hình ảnh giả, và chúng ta bổ sung batch này bằng cách nối một số lượng hình ảnh thật tương

đương. Các mục tiêu y1 được đặt là 0 cho hình ảnh giả và 1 cho hình ảnh thật. Sau đó, chúng ta huấn luyện bộ phân biệt trên batch này. Hãy nhớ rằng bộ phân biệt có thể huấn luyện được trong giai đoạn này, nhưng chúng ta không động đến bộ tạo.

- **Trong giai đoạn hai,** chúng ta cấp cho GAN một số nhiễu Gaussian. Bộ tạo của nó sẽ bắt đầu bằng cách tạo ra hình ảnh giả, sau đó bộ phân biệt sẽ cố gắng đoán xem những hình ảnh này là giả hay thật. Trong giai đoạn này, chúng ta đang cố gắng cải thiện bộ tạo, điều đó có nghĩa là chúng ta muốn bộ phân biệt thất bại: đây là lý do tại sao các mục tiêu y2 đều được đặt là 1, mặc dù hình ảnh là giả. Trong giai đoạn này, bộ phân biệt không thể huấn luyện được, vì vậy phần duy nhất của mô hình gan sẽ cải thiện là bộ tạo.

Vậy đó! Sau khi huấn luyện, bạn có thể lấy mẫu ngẫu nhiên một số mã hóa từ phân phối Gaussian, và cấp chúng cho bộ tạo để tạo ra hình ảnh mới:

```
import tensorflow as tf
# Giả định batch_size, codings_size đã được định nghĩa
# Giả định generator đã được huấn luyện và có sẵn

# codings = tf.random.normal(shape=[batch_size, codings_size])
# generated_images = generator.predict(codings)
```

Nếu bạn hiển thị các hình ảnh được tạo (xem Hình 17-15), bạn sẽ thấy rằng ở cuối epoch đầu tiên, chúng đã bắt đầu trông giống (rất nhiễu) hình ảnh Fashion MNIST.

Hình 17-15. Hình ảnh được tạo bởi GAN sau một epoch huấn luyện

Thật không may, các hình ảnh không bao giờ thực sự tốt hơn nhiều so với thế, và bạn thậm chí có thể thấy các epoch mà GAN dường như đang quên những gì nó đã học. Tại sao lại như vậy? Chà, hóa ra việc huấn luyện một GAN có thể rất khó khăn. Hãy xem tại sao.

Những khó khăn khi huấn luyện GAN

Trong quá trình huấn luyện, bộ tạo và bộ phân biệt liên tục cố gắng vượt trội lẫn nhau, trong một trò chơi tổng bằng không. Khi quá trình huấn luyện tiến triển, trò chơi có thể kết thúc ở trạng thái mà các nhà lý thuyết trò chơi gọi là cân bằng Nash, được đặt tên theo nhà toán học John Nash: đây là khi không người chơi nào tốt hơn nếu thay đổi chiến lược của riêng họ, giả sử những người chơi khác không thay đổi của họ. Ví dụ, cân bằng Nash đạt được khi mọi người lái xe bên trái đường: không người lái nào tốt hơn nếu là người duy nhất chuyển làn. Tất nhiên, có một cân bằng Nash thứ hai có thể xảy ra: khi mọi người lái xe bên phải đường. Các trạng thái và động lực ban đầu khác nhau có thể dẫn đến cân bằng này hay cân bằng khác. Trong ví dụ này, có một chiến lược tối ưu duy nhất một khi cân bằng được đạt đến (tức là lái xe cùng phía với mọi người khác), nhưng cân bằng Nash có thể liên quan đến nhiều chiến lược cạnh tranh (ví dụ: một kẻ săn mồi đuổi theo con mồi, con mồi cố gắng thoát thân, và cả hai đều không tốt hơn nếu thay đổi chiến lược của họ).

Vậy điều này áp dụng cho GAN như thế nào? Chà, các tác giả của bài báo GAN đã chứng minh rằng một GAN chỉ có thể đạt được một cân bằng Nash duy nhất: đó là khi bộ tạo tạo ra hình ảnh hoàn toàn chân thực, và bộ phân biệt bị buộc phải đoán (50% thật, 50% giả). Thực tế này rất đáng khích lệ: dường như bạn chỉ cần huấn luyện GAN đủ lâu, và nó cuối cùng sẽ đạt đến cân bằng này, mang lại cho bạn một bộ tạo hoàn hảo. Thật không may, điều đó không đơn giản: không có gì đảm bảo rằng cân bằng sẽ đạt được.

Khó khăn lớn nhất được gọi là **sập chế độ (mode collapse)**: đây là khi đầu ra của bộ tạo dần trở nên kém đa dạng hơn. Điều này có thể xảy ra như thế nào? Giả sử bộ tạo giỏi tạo ra những đôi giày thuyết phục hơn bất kỳ lớp nào khác. Nó sẽ đánh lừa bộ phân biệt nhiều hơn một chút với giày, và điều này sẽ khuyến khích nó tạo ra nhiều hình ảnh giày hơn nữa. Dần dần, nó sẽ quên cách tạo ra bất cứ thứ gì khác. Trong khi đó, những hình ảnh giả duy nhất mà bộ phân biệt sẽ thấy là giày, vì vậy nó cũng sẽ quên cách phân biệt hình ảnh giả của các lớp khác. Cuối cùng, khi bộ phân biệt quản lý phân biệt giày giả với giày thật, bộ tạo sẽ bị buộc phải chuyển sang một lớp khác. Sau đó, nó có thể trở nên giỏi về áo sơ mi, quên giày, và bộ phân biệt sẽ theo sau. GAN có thể dần dần luân chuyển qua một vài lớp, không bao giờ thực sự trở nên rất giỏi về bất kỳ lớp nào trong số đó.

Hơn nữa, vì bộ tạo và bộ phân biệt liên tục đẩy nhau, các tham số của chúng có thể kết thúc dao động và trở nên không ổn định. Quá trình huấn luyện có thể bắt đầu đúng cách, sau đó đột nhiên phân kỳ không rõ lý do, do những bất ổn này. Và vì nhiều yếu tố ảnh hưởng đến các động lực phức tạp này, GAN rất nhạy cảm với các siêu tham số: bạn có thể phải tốn nhiều công sức để tinh chỉnh chúng. Trên thực tế, đó là lý do tại sao tôi sử dụng RMSProp thay vì Nadam khi biên dịch các mô hình: khi sử dụng Nadam, tôi đã gặp phải sự sập chế độ nghiêm trọng.

Những vấn đề này đã khiến các nhà nghiên cứu rất bận rộn kể từ năm 2014: nhiều bài báo đã được xuất bản về chủ đề này, một số đề xuất các hàm chi phí mới (mặc dù một bài báo năm 2018 của các nhà nghiên cứu Google đặt câu hỏi về hiệu quả của chúng) hoặc các kỹ thuật để ổn định huấn luyện hoặc để tránh vấn đề sập chế độ. Ví dụ, một kỹ thuật phổ biến gọi là **replay kinh nghiệm (experience replay)** bao gồm việc lưu trữ các hình ảnh được tạo

bởi bộ tạo ở mỗi lần lặp trong một bộ đệm phát lại (dần dần loại bỏ các hình ảnh được tạo cũ hơn) và huấn luyện bộ phân biệt bằng cách sử dụng hình ảnh thật cộng với hình ảnh giả được lấy từ bộ đệm này (thay vì chỉ các hình ảnh giả được tạo bởi bộ tạo hiện tại). Điều này làm giảm khả năng bộ phân biệt sẽ quá khớp với đầu ra của bộ tạo mới nhất. Một kỹ thuật phổ biến khác được gọi là **phân biệt theo mini-batch (mini-batch discrimination)**: nó đo lường mức độ tương tự của hình ảnh trong toàn bộ batch và cung cấp thống kê này cho bộ phân biệt, để nó có thể dễ dàng từ chối toàn bộ batch hình ảnh giả thiếu đa dạng. Điều này khuyến khích bộ tạo tạo ra nhiều loại hình ảnh hơn, giảm khả năng sập chế độ. Các bài báo khác đơn giản đề xuất các kiến trúc cụ thể có hiệu suất tốt.

Tóm lại, đây vẫn là một lĩnh vực nghiên cứu rất năng động, và động lực của GAN vẫn chưa được hiểu hoàn hảo. Nhưng tin tốt là đã có những tiến bộ lớn, và một số kết quả thực sự đáng kinh ngạc! Vì vậy, chúng ta hãy xem xét một số kiến trúc thành công nhất, bắt đầu với GAN tích chập sâu, vốn là công nghệ tiên tiến chỉ vài năm trước. Sau đó, chúng ta sẽ xem xét hai kiến trúc gần đây hơn (và phức tạp hơn).

GAN tích chập sâu (Deep Convolutional GANs - DCGANs)

Các tác giả của bài báo GAN gốc đã thử nghiệm với các lớp tích chập, nhưng chỉ cố gắng tạo ra các hình ảnh nhỏ. Ngay sau đó, nhiều nhà nghiên cứu đã cố gắng xây dựng GAN dựa trên các mạng tích chập sâu hơn để tạo ra các hình ảnh lớn hơn.

Điều này tỏ ra khó khăn, vì quá trình huấn luyện rất không ổn định, nhưng Alec Radford và cộng sự cuối cùng đã thành công vào cuối năm 2015, sau khi thử nghiệm với nhiều kiến trúc và siêu tham số khác nhau. Họ gọi kiến trúc của mình là GAN tích chập sâu (DCGANs). Dưới đây là các hướng dẫn chính mà họ đề xuất để xây dựng các GAN tích chập ổn định:

- **Thay thế bất kỳ lớp gộp nào bằng tích chập bước (strided convolutions)** (trong bộ phân biệt) và **tích chập chuyển vị (transposed convolutions)** (trong bộ tạo).
- **Sử dụng chuẩn hóa batch (batch normalization)** trong cả bộ tạo và bộ phân biệt, ngoại trừ lớp đầu ra của bộ tạo và lớp đầu vào của bộ phân biệt.
- **Loại bỏ các lớp ẩn kết nối đầy đủ** cho các kiến trúc sâu hơn.
- **Sử dụng kích hoạt ReLU** trong bộ tạo cho tất cả các lớp ngoại trừ lớp đầu ra, lớp này nên sử dụng tanh.
- **Sử dụng kích hoạt Leaky ReLU** trong bộ phân biệt cho tất cả các lớp.

Những hướng dẫn này sẽ hoạt động trong nhiều trường hợp, nhưng không phải lúc nào cũng vậy, vì vậy bạn vẫn có thể cần thử nghiệm với các siêu tham số khác nhau. Trên thực tế, chỉ cần thay đổi hạt ngẫu nhiên và huấn luyện lại cùng một mô hình chính xác đôi khi sẽ có tác dụng. Dưới đây là một DCGAN nhỏ hoạt động khá tốt với Fashion MNIST:

```
import tensorflow as tf

codings_size = 100

generator = tf.keras.Sequential([
```

```
    tf.keras.layers.Dense(7 * 7 * 128),
    tf.keras.layers.Reshape([7, 7, 128]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(64, kernel_size=5, strides=2,
                                    padding="same", activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(1, kernel_size=5, strides=2,
                                    padding="same", activation="tanh"),
])
discriminator = tf.keras.Sequential([
    tf.keras.layers.Conv2D(64, kernel_size=5, strides=2, padding="same",
                           activation=tf.keras.layers.LeakyReLU(0.2)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Conv2D(128, kernel_size=5, strides=2, padding="same",
                           activation=tf.keras.layers.LeakyReLU(0.2)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
gan = tf.keras.Sequential([generator, discriminator])
```

Bộ tạo nhận các mã hóa có kích thước 100, chiếu chúng lên 6.272 chiều (7 * 7 * 128), và định hình lại kết quả để có một tensor 7 × 7 × 128. Tensor này được chuẩn hóa batch và cấp cho một lớp tích chập chuyển vị với bước nhảy 2, lớp này sẽ tăng mẫu từ 7 × 7 lên 14 × 14 và giảm chiều sâu từ 128 xuống 64. Kết quả được chuẩn hóa batch lại và cấp cho một lớp tích chập chuyển vị khác với bước nhảy 2, lớp này sẽ tăng mẫu từ 14 × 14 lên 28 × 28 và giảm chiều sâu từ 64 xuống 1. Lớp này sử dụng hàm kích hoạt tanh, vì vậy đầu ra sẽ nằm trong khoảng từ -1 đến 1. Vì lý do này, trước khi huấn luyện GAN, chúng ta cần thay đổi tỷ lệ tập huấn luyện về cùng một khoảng này. Chúng ta cũng cần định hình lại nó để thêm chiều kênh:

```
import numpy as np
# Giả định X_train đã được định nghĩa và có sẵn từ ngữ cảnh trước đó
# X_train_dcgan = X_train.reshape(-1, 28, 28, 1) * 2. - 1. # định hình lại và
thay đổi tỷ lệ
```

Bộ phân biệt trông rất giống một CNN thông thường để phân loại nhị phân, ngoại trừ thay vì sử dụng các lớp gộp cực đại để giảm mẫu hình ảnh, chúng ta sử dụng các tích chập bước (strides=2). Lưu ý rằng chúng ta sử dụng hàm kích hoạt Leaky ReLU. Nhìn chung, chúng ta đã tuân thủ các hướng dẫn của DCGAN, ngoại trừ việc chúng ta đã thay thế các lớp `BatchNormalization` trong bộ phân biệt bằng các lớp `Dropout`; nếu không, quá trình huấn luyện trong trường hợp này không ổn định. Hãy thoải mái điều chỉnh kiến trúc này: bạn sẽ thấy nó nhạy cảm như thế nào với các siêu tham số, đặc biệt là tốc độ học tương đối của hai mạng.

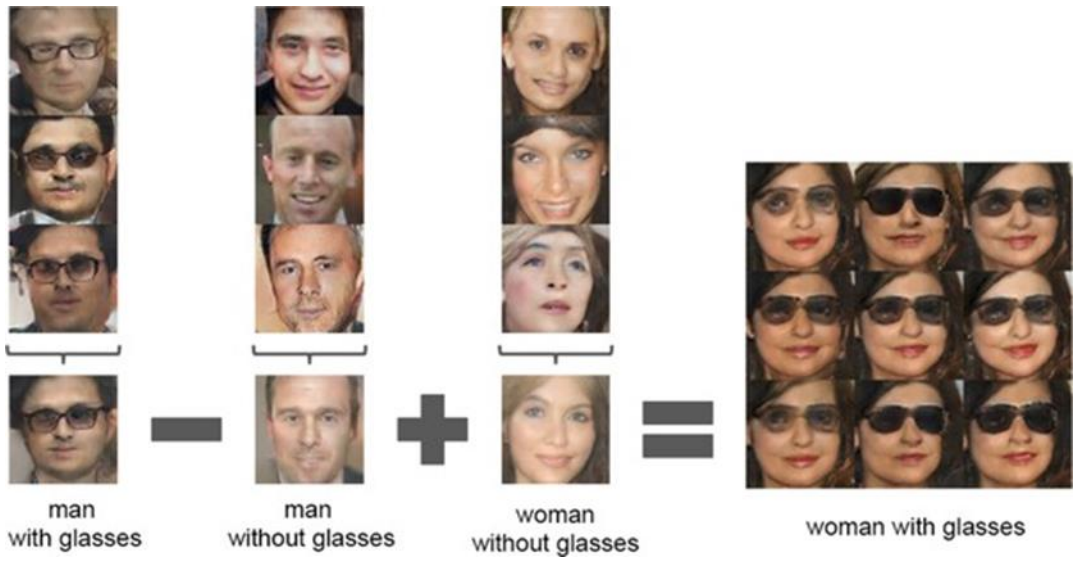
Cuối cùng, để xây dựng tập dữ liệu và sau đó biên dịch và huấn luyện mô hình này, chúng ta có thể sử dụng cùng một mã như trước đó. Sau 50 epoch huấn luyện, bộ tạo tạo ra các hình

ảnh như được hiển thị trong Hình 17-16. Nó vẫn chưa hoàn hảo, nhưng nhiều hình ảnh này khá thuyết phục.

Hình 17-16. Hình ảnh được tạo bởi DCGAN sau 50 epoch huấn luyện

Nếu bạn mở rộng kiến trúc này và huấn luyện nó trên một tập dữ liệu lớn về khuôn mặt, bạn có thể nhận được những hình ảnh khá chân thực. Trên thực tế, DCGANs có thể học các biểu diễn tiềm ẩn khá có ý nghĩa, như bạn có thể thấy trong Hình 17-17: nhiều hình ảnh đã được tạo ra, và chín trong số đó được chọn thủ công (góc trên bên trái), bao gồm ba hình đại diện cho đàn ông đeo kính, ba người đàn ông không đeo kính và ba phụ nữ không đeo kính. Đối với mỗi loại này, các mã hóa được sử dụng để tạo ra các hình ảnh đã được tính trung bình, và một hình ảnh được tạo ra dựa trên các mã hóa trung bình kết quả (phía dưới bên trái). Tóm lại, mỗi trong ba hình ảnh phía dưới bên trái đại diện cho giá trị trung bình của ba hình ảnh nằm phía trên nó. Nhưng đây không phải là một giá trị trung bình đơn giản được tính ở cấp độ pixel (điều này sẽ dẫn đến ba khuôn mặt chồng lên nhau), đó là một giá trị trung bình được tính trong không gian tiềm ẩn, vì vậy các hình ảnh vẫn trông giống như khuôn mặt bình thường. Thật đáng kinh ngạc, nếu bạn tính đàn ông đeo kính, trừ đàn ông không đeo kính, cộng với phụ nữ không đeo kính — trong đó mỗi thuật ngữ tương ứng với một trong các mã hóa trung bình — và bạn tạo ra hình ảnh tương ứng với mã hóa này, bạn sẽ nhận được hình ảnh ở trung tâm của lưới 3 × 3 khuôn mặt bên phải: một người phụ nữ đeo kính! Tám hình ảnh khác xung quanh nó được tạo ra dựa trên cùng một vector cộng với một chút nhiễu, để minh họa khả năng nội suy ngữ nghĩa của DCGANs. Có thể thực hiện các phép tính số học trên khuôn mặt cảm giác giống như khoa học viễn tưởng!

Tuy nhiên, DCGANs không hoàn hảo. Ví dụ, khi bạn cố gắng tạo ra các hình ảnh rất lớn bằng DCGANs, bạn thường kết thúc với các đặc trưng cục bộ thuyết phục nhưng tổng thể không nhất quán, chẳng hạn như áo sơ mi có một ống tay dài hơn nhiều so với ống kia, hoa tai khác nhau, hoặc mắt nhìn theo các hướng đối diện. Làm thế nào bạn có thể khắc phục điều này?

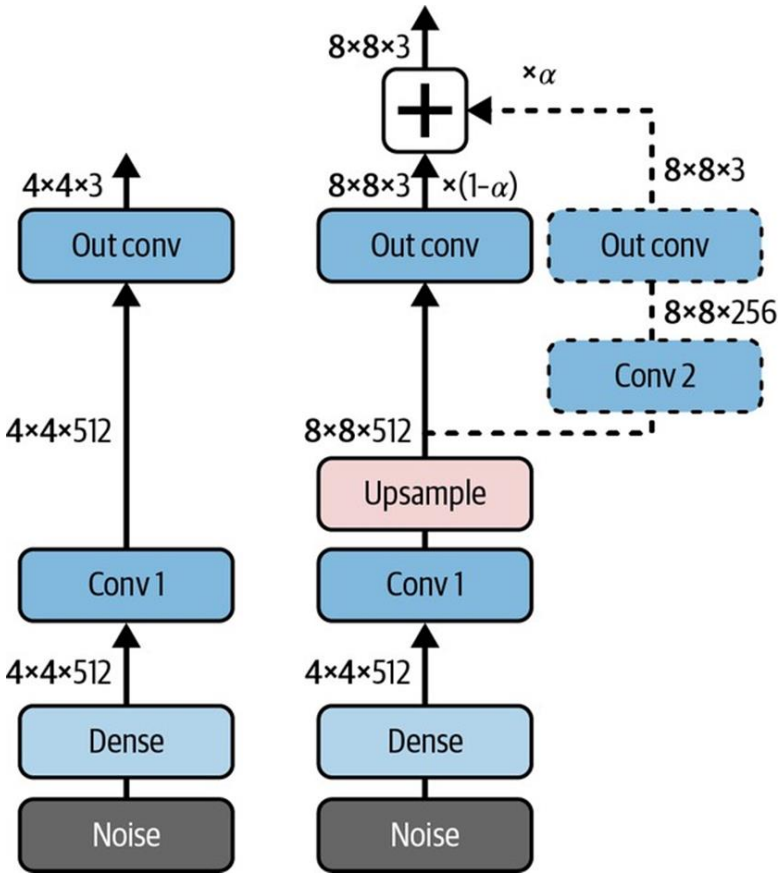


Hình 17-17. Phép toán vector cho các khái niệm hình ảnh (một phần của hình 7 từ bài báo DCGAN)

Sự phát triển tăng dần của GANs (Progressive Growing of GANs)

Trong một bài báo năm 2018, các nhà nghiên cứu của Nvidia Tero Karras và cộng sự đã đề xuất một kỹ thuật quan trọng: họ gợi ý tạo ra các hình ảnh nhỏ ở đầu quá trình huấn luyện, sau đó dần dần thêm các lớp tích chập vào cả bộ tạo và bộ phân biệt để tạo ra các hình ảnh ngày càng lớn hơn (4 × 4, 8 × 8, 16 × 16, ..., 512 × 512, 1.024 × 1.024). Cách tiếp cận này giống với việc huấn luyện từng lớp tham lam của các autoencoder xếp chồng. Các lớp bổ sung được thêm vào cuối bộ tạo và ở đầu bộ phân biệt, và các lớp đã được huấn luyện trước đó vẫn có thể huấn luyện được.

Ví dụ, khi tăng kích thước đầu ra của bộ tạo từ 4 × 4 lên 8 × 8 (xem Hình 17-18), một lớp tăng mẫu (sử dụng lọc lân cận gần nhất) được thêm vào lớp tích chập hiện có (“Conv 1”) để tạo ra các bản đồ đặc trưng 8 × 8. Những bản đồ này được cấp cho lớp tích chập mới (“Conv 2”), lớp này lần lượt cấp vào một lớp tích chập đầu ra mới. Để tránh làm hỏng các trọng số đã được huấn luyện của Conv 1, chúng ta dần dần làm mờ hai lớp tích chập mới (được biểu thị bằng đường đứt nét trong Hình 17-18) và làm mờ lớp đầu ra gốc. Đầu ra cuối cùng là tổng có trọng số của các đầu ra mới (với trọng số α) và các đầu ra gốc (với trọng số 1 – α), từ từ tăng α từ 0 đến 1. Một kỹ thuật làm mờ tương tự được sử dụng khi một lớp tích chập mới được thêm vào bộ phân biệt (theo sau là một lớp gộp trung bình để giảm mẫu). Lưu ý rằng tất cả các lớp tích chập đều sử dụng padding “same” và bước nhảy 1, vì vậy chúng bảo toàn chiều cao và chiều rộng của đầu vào. Điều này bao gồm lớp tích chập gốc, vì vậy giờ đây nó tạo ra đầu ra 8 × 8 (vì đầu vào của nó bây giờ là 8 × 8). Cuối cùng, các lớp đầu ra sử dụng kích thước kernel 1. Chúng chỉ chiếu đầu vào của chúng xuống số lượng kênh màu mong muốn (thường là 3).



Hình 17-18. Một GAN phát triển dần dần: bộ tạo GAN xuất ra hình ảnh màu 4 × 4 (trái); chúng tôi mở rộng nó để xuất ra hình ảnh 8 × 8 (phải)

Bài báo cũng giới thiệu một số kỹ thuật khác nhằm tăng tính đa dạng của đầu ra (để tránh sập chế độ) và làm cho quá trình huấn luyện ổn định hơn:

- **Lớp độ lệch chuẩn mini-batch (Mini-batch standard deviation layer):** Được thêm gần cuối bộ phân biệt. Đối với mỗi vị trí trong đầu vào, nó tính độ lệch chuẩn trên tất cả các kênh và tất cả các trường hợp trong batch (`S = tf.math.reduce_std(inputs, axis=[0, -1])`). Các độ lệch chuẩn này sau đó được tính trung bình trên tất cả các điểm để có được một giá trị duy nhất (`v = tf.reduce_mean(S)`). Cuối cùng, một bản đồ đặc trưng bổ sung được thêm vào mỗi trường hợp trong batch và được điền bằng giá trị đã tính (`tf.concat([inputs, tf.fill([batch_size, height, width, 1], v)], axis=-1)`). Điều này giúp ích như thế nào? Chà, nếu bộ tạo tạo ra hình ảnh ít đa dạng, thì sẽ có độ lệch chuẩn nhỏ trên các bản đồ đặc trưng trong bộ phân biệt. Nhờ lớp này, bộ phân biệt sẽ dễ dàng truy cập vào thống kê này, khiến nó ít bị đánh lừa bởi một bộ tạo tạo ra quá ít đa dạng. Điều này sẽ khuyến khích bộ tạo tạo ra các đầu ra đa dạng hơn, giảm nguy cơ sập chế độ.
- **Tốc độ học bằng nhau (Equalized learning rate):** Khởi tạo tất cả các trọng số bằng cách sử dụng phân phối Gaussian với trung bình 0 và độ lệch chuẩn 1 thay vì sử

dụng khởi tạo He. Tuy nhiên, các trọng số được thu nhỏ trong thời gian chạy (tức là mỗi khi lớp được thực thi) bởi cùng một hệ số như trong khởi tạo He: chúng được chia cho `sqrt(2 / n_inputs)`, trong đó `n_inputs` là số đầu vào của lớp. Bài báo đã chứng minh rằng kỹ thuật này cải thiện đáng kể hiệu suất của GAN khi sử dụng RMSProp, Adam hoặc các bộ tối ưu hóa gradient thích ứng khác. Thật vậy, các bộ tối ưu hóa này chuẩn hóa các cập nhật gradient bằng độ lệch chuẩn ước tính của chúng (xem Chương 11), vì vậy các tham số có dải động lớn hơn sẽ mất nhiều thời gian hơn để huấn luyện, trong khi các tham số có dải động nhỏ có thể được cập nhật quá nhanh, dẫn đến mất ổn định. Bằng cách thay đổi tỷ lệ trọng số như một phần của chính mô hình thay vì chỉ thay đổi tỷ lệ chúng khi khởi tạo, cách tiếp cận này đảm bảo rằng dải động là như nhau cho tất cả các tham số trong suốt quá trình huấn luyện, vì vậy tất cả chúng đều học với cùng tốc độ. Điều này vừa tăng tốc vừa ổn định quá trình huấn luyện.

- **Lớp chuẩn hóa theo pixel (Pixelwise normalization layer):** Được thêm vào sau mỗi lớp tích chập trong bộ tạo. Nó chuẩn hóa mỗi kích hoạt dựa trên tất cả các kích hoạt trong cùng một hình ảnh và ở cùng một vị trí, nhưng trên tất cả các kênh (chia cho căn bậc hai của kích hoạt bình phương trung bình). Trong mã TensorFlow, điều này là `inputs / tf.sqrt(tf.reduce_mean(tf.square(X), axis=-1, keepdims=True) + 1e-8)` (thuật ngữ làm mượt `1e-8` là cần thiết để tránh chia cho 0). Kỹ thuật này tránh sự bùng nổ trong các kích hoạt do sự cạnh tranh quá mức giữa bộ tạo và bộ phân biệt.

Sự kết hợp của tất cả các kỹ thuật này đã cho phép các tác giả tạo ra những hình ảnh khuôn mặt độ phân giải cao cực kỳ thuyết phục. Nhưng chính xác thì chúng ta gọi "thuyết phục" là gì? Đánh giá là một trong những thách thức lớn khi làm việc với GAN: mặc dù có thể tự động đánh giá sự đa dạng của các hình ảnh được tạo ra, nhưng việc đánh giá chất lượng của chúng là một nhiệm vụ khó khăn và chủ quan hơn nhiều. Một kỹ thuật là sử dụng những người đánh giá, nhưng điều này tốn kém và tốn thời gian. Vì vậy, các tác giả đã đề xuất đo lường sự tương đồng giữa cấu trúc hình ảnh cục bộ của các hình ảnh được tạo ra và các hình ảnh huấn luyện, xem xét mọi tỷ lệ. Ý tưởng này đã dẫn họ đến một đổi mới đột phá khác: StyleGANs.

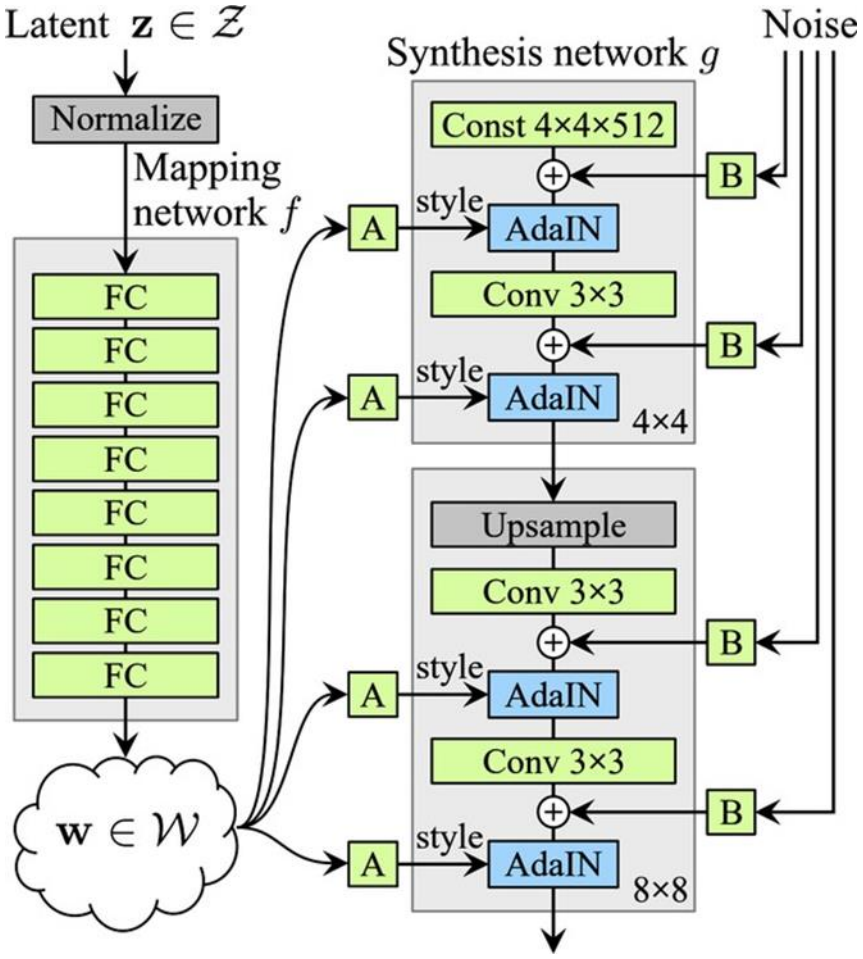
StyleGANs

Công nghệ tiên tiến trong việc tạo ra hình ảnh độ phân giải cao một lần nữa được đội ngũ Nvidia đó phát triển trong một bài báo năm 2018 đã giới thiệu kiến trúc StyleGAN phổ biến. Các tác giả đã sử dụng các kỹ thuật chuyển đổi phong cách trong bộ tạo để đảm bảo rằng các hình ảnh được tạo ra có cùng cấu trúc cục bộ với hình ảnh huấn luyện, ở mọi tỷ lệ, cải thiện đáng kể chất lượng của hình ảnh được tạo ra. Bộ phân biệt và hàm mất không được sửa đổi, chỉ có bộ tạo. Một bộ tạo StyleGAN bao gồm hai mạng (xem Hình 17-19):

- **Mạng ánh xạ (Mapping network):** Một MLP tám lớp ánh xạ các biểu diễn tiềm ẩn `z` (tức là các mã hóa) thành một vector `w`. Vector này sau đó được truyền qua nhiều phép biến đổi affine (tức là các lớp `Dense` không có hàm kích hoạt, được biểu thị bằng các hộp "A" trong Hình 17-19), tạo ra nhiều vector. Các vector này kiểm soát

phong cách của hình ảnh được tạo ở các cấp độ khác nhau, từ kết cấu chi tiết (ví dụ: màu tóc) đến các đặc trưng cấp cao (ví dụ: người lớn hay trẻ em). Tóm lại, mạng ánh xạ ánh xạ các mã hóa thành nhiều vector phong cách.

- **Mạng tổng hợp (Synthesis network):** Chịu trách nhiệm tạo ra các hình ảnh. Nó có một đầu vào đã học không đổi (nói rõ hơn, đầu vào này sẽ không đổi sau khi huấn luyện, nhưng trong quá trình huấn luyện nó vẫn được điều chỉnh bởi lan truyền ngược). Nó xử lý đầu vào này thông qua nhiều lớp tích chập và lớp tăng mẫu, như trước đây, nhưng có hai điểm khác biệt. Thứ nhất, một số nhiễu được thêm vào đầu vào và vào tất cả các đầu ra của các lớp tích chập (trước hàm kích hoạt). Thứ hai, mỗi lớp nhiễu được theo sau bởi một lớp chuẩn hóa thể hiện thích ứng (AdaIN): nó chuẩn hóa từng bản đồ đặc trưng một cách độc lập (bằng cách trừ đi giá trị trung bình của bản đồ đặc trưng và chia cho độ lệch chuẩn của nó), sau đó nó sử dụng vector phong cách để xác định tỷ lệ và độ lệch của mỗi bản đồ đặc trưng (vector phong cách chứa một tỷ lệ và một thuật ngữ thiên vị cho mỗi bản đồ đặc trưng).



Hình 17-19. Kiến trúc bộ tạo của StyleGAN (một phần của Hình 1 từ bài báo StyleGAN)

Ý tưởng thêm nhiễu độc lập với các mã hóa là rất quan trọng. Một số phần của hình ảnh khá ngẫu nhiên, chẳng hạn như vị trí chính xác của mỗi nốt ruồi hay sợi tóc. Trong các GAN

trước đây, sự ngẫu nhiên này phải đến từ các mã hóa hoặc là một số nhiễu giả ngẫu nhiên do chính bộ tạo tạo ra. Nếu nó đến từ các mã hóa, điều đó có nghĩa là bộ tạo phải dành một phần đáng kể sức mạnh biểu diễn của các mã hóa để lưu trữ nhiễu, điều này khá lãng phí. Hơn nữa, nhiễu phải có khả năng chảy qua mạng và đến các lớp cuối cùng của bộ tạo: điều này dường như là một ràng buộc không cần thiết có lẽ đã làm chậm quá trình huấn luyện. Và cuối cùng, một số hiện tượng hình ảnh có thể xuất hiện vì cùng một nhiễu được sử dụng ở các cấp độ khác nhau. Nếu thay vào đó, bộ tạo cố gắng tạo ra nhiễu giả ngẫu nhiên của riêng nó, nhiễu này có thể không trông rất thuyết phục, dẫn đến nhiều hiện tượng hình ảnh hơn. Thêm vào đó, một phần trọng số của bộ tạo sẽ được dành để tạo ra nhiễu giả ngẫu nhiên, điều này lại có vẻ lãng phí. Bằng cách thêm các đầu vào nhiễu bổ sung, tất cả các vấn đề này được tránh; GAN có thể sử dụng nhiễu được cung cấp để thêm lượng ngẫu nhiên thích hợp vào mỗi phần của hình ảnh.

Nhiễu được thêm vào khác nhau cho mỗi cấp độ. Mỗi đầu vào nhiễu bao gồm một bản đồ đặc trưng duy nhất đầy nhiễu Gaussian, được lan truyền đến tất cả các bản đồ đặc trưng (ở cấp độ đã cho) và được chia tỷ lệ bằng cách sử dụng các hệ số tỷ lệ đã học trên mỗi đặc trưng (điều này được biểu thị bằng các hộp “B” trong Hình 17-19) trước khi nó được thêm vào.

Cuối cùng, StyleGAN sử dụng một kỹ thuật gọi là **chuẩn hóa trộn (mixing regularization)** (hoặc trộn kiểu - style mixing), trong đó một tỷ lệ phần trăm của các hình ảnh được tạo ra bằng cách sử dụng hai mã hóa khác nhau. Cụ thể, các mã hóa `c1` và `c2` được gửi qua mạng ánh xạ, tạo ra hai vector kiểu `w1` và `w2`. Sau đó, mạng tổng hợp tạo ra một hình ảnh dựa trên các kiểu `w1` cho các cấp độ đầu tiên và các kiểu `w2` cho các cấp độ còn lại. Cấp độ cắt được chọn ngẫu nhiên.

Điều này ngăn mạng giả định rằng các kiểu ở các cấp độ liền kề có tương quan, điều này lần lượt khuyến khích tính cục bộ trong GAN, nghĩa là mỗi vector kiểu chỉ ảnh hưởng đến một số lượng giới hạn các đặc điểm trong hình ảnh được tạo ra.

Có rất nhiều loại GAN khác nhau đến nỗi sẽ cần cả một cuốn sách để bao quát tất cả. Hy vọng phần giới thiệu này đã cung cấp cho bạn những ý tưởng chính, và quan trọng nhất là mong muốn tìm hiểu thêm. Hãy tiếp tục và tự triển khai GAN của riêng bạn, và đừng nản lòng nếu lúc đầu nó gặp khó khăn trong việc học: thật không may, điều này là bình thường, và sẽ cần khá nhiều kiên nhẫn để làm cho nó hoạt động, nhưng kết quả sẽ xứng đáng. Nếu bạn đang gặp khó khăn với một chi tiết triển khai, có rất nhiều triển khai Keras hoặc TensorFlow mà bạn có thể tham khảo. Trên thực tế, nếu tất cả những gì bạn muốn là có được một số kết quả tuyệt vời một cách nhanh chóng, thì bạn chỉ cần sử dụng một mô hình đã được huấn luyện trước (ví dụ: có các mô hình StyleGAN đã được huấn luyện trước có sẵn cho Keras).

Bây giờ chúng ta đã kiểm tra autoencoder và GAN, hãy xem xét một loại kiến trúc cuối cùng: mô hình khuếch tán.

Mô hình khuếch tán (Diffusion Models)

Những ý tưởng đằng sau các mô hình khuếch tán đã tồn tại trong nhiều năm, nhưng chúng lần đầu tiên được chính thức hóa dưới dạng hiện đại trong một bài báo năm 2015 của Jascha Sohl-Dickstein và cộng sự từ Đại học Stanford và UC Berkeley. Các tác giả đã áp dụng các công cụ từ nhiệt động lực học để mô hình hóa một quá trình khuếch tán, tương tự như một giọt sữa khuếch tán trong một cốc trà. Ý tưởng cốt lõi là huấn luyện một mô hình để học quá trình ngược lại: bắt đầu từ trạng thái hoàn toàn hỗn hợp, và dần dần "tách" sữa ra khỏi trà. Sử dụng ý tưởng này, họ đã đạt được kết quả đầy hứa hẹn trong việc tạo hình ảnh, nhưng vì GANs tạo ra hình ảnh thuyết phục hơn vào thời điểm đó, các mô hình khuếch tán đã không nhận được nhiều sự chú ý.

Sau đó, vào năm 2020, Jonathan Ho và cộng sự, cũng từ UC Berkeley, đã xây dựng được một mô hình khuếch tán có khả năng tạo ra hình ảnh có độ chân thực cao, mà họ gọi là mô hình xác suất khuếch tán khử nhiễu (denoising diffusion probabilistic model - DDPM). Vài tháng sau, một bài báo năm 2021 của các nhà nghiên cứu OpenAI Alex Nichol và Prafulla Dhariwal đã phân tích kiến trúc DDPM và đề xuất một số cải tiến cho phép DDPMs cuối cùng đánh bại GANs: DDPMs không chỉ dễ huấn luyện hơn nhiều so với GANs, mà các hình ảnh được tạo ra còn đa dạng hơn và có chất lượng cao hơn nữa. Nhược điểm chính của DDPMs, như bạn sẽ thấy, là chúng mất rất nhiều thời gian để tạo ra hình ảnh, so với GANs hoặc VAEs.

Vậy chính xác thì DDPM hoạt động như thế nào? Chà, giả sử bạn bắt đầu với một bức ảnh con mèo (như cái bạn sẽ thấy trong Hình 17-20), ký hiệu `x0`, và ở mỗi bước thời gian `t` bạn thêm một chút nhiễu Gaussian vào hình ảnh, với trung bình 0 và phương sai `βt`. Nhiễu này độc lập cho mỗi pixel: chúng ta gọi nó là đẳng hướng. Bạn đầu tiên có được hình ảnh `x1`, sau đó `x2`, v.v., cho đến khi con mèo hoàn toàn bị nhiễu che khuất, không thể nhìn thấy. Bước thời gian cuối cùng được ký hiệu là `T`. Trong bài báo DDPM gốc, các tác giả đã sử dụng `T = 1.000`, và họ đã lên lịch phương sai `βt` theo cách mà tín hiệu mèo mờ dần tuyến tính giữa các bước thời gian 0 và `T`. Trong bài báo DDPM cải tiến, `T` được nâng lên 4.000, và lịch phương sai được điều chỉnh để thay đổi chậm hơn ở đầu và cuối. Tóm lại, chúng ta đang dần dần nhấn chìm con mèo trong nhiễu: đây được gọi là **quá trình tiến (forward process)**.

Khi chúng ta thêm ngày càng nhiều nhiễu Gaussian vào quá trình tiến, phân phối của các giá trị pixel ngày càng trở nên Gaussian hơn. Một chi tiết quan trọng tôi đã bỏ qua là các giá trị pixel được thay đổi tỷ lệ nhẹ ở mỗi bước, bằng một yếu tố `sqrt(1 – βt)`. Điều này đảm bảo rằng trung bình của các giá trị pixel dần dần tiến đến 0, vì yếu tố tỷ lệ nhỏ hơn 1 một chút (hãy tưởng tượng liên tục nhân một số với 0.99). Nó cũng đảm bảo rằng phương sai sẽ dần dần hội tụ về 1. Điều này là do độ lệch chuẩn của các giá trị pixel cũng được chia tỷ lệ bởi `sqrt(1 – βt)`, vì vậy phương sai được chia tỷ lệ bởi `1 – βt` (tức là bình phương của yếu tố tỷ lệ). Nhưng phương sai không thể giảm xuống 0 vì chúng ta đang thêm nhiễu Gaussian với phương sai `βt` ở mỗi bước. Và vì phương sai cộng lại khi bạn tổng hợp các phân phối Gaussian, bạn có thể thấy rằng phương sai chỉ có thể hội tụ về `1 – βt + βt = 1`.

Quá trình khuếch tán tiến được tóm tắt trong Công thức 17-5. Phương trình này sẽ không dạy bạn bất cứ điều gì mới về quá trình tiến, nhưng nó hữu ích để hiểu loại ký hiệu toán học này, vì nó thường được sử dụng trong các bài báo ML. Phương trình này định nghĩa phân phối xác suất q của `xt` cho trước `xt–1` là một phân phối Gaussian với trung bình `xt–1` nhân với yếu tố tỷ lệ, và với ma trận hiệp phương sai bằng `βtI`. Đây là ma trận đơn vị `I` nhân với `βt`, có nghĩa là nhiễu là đẳng hướng với phương sai `βt`.

Công thức 17-5. Phân phối xác suất q của quá trình khuếch tán tiến

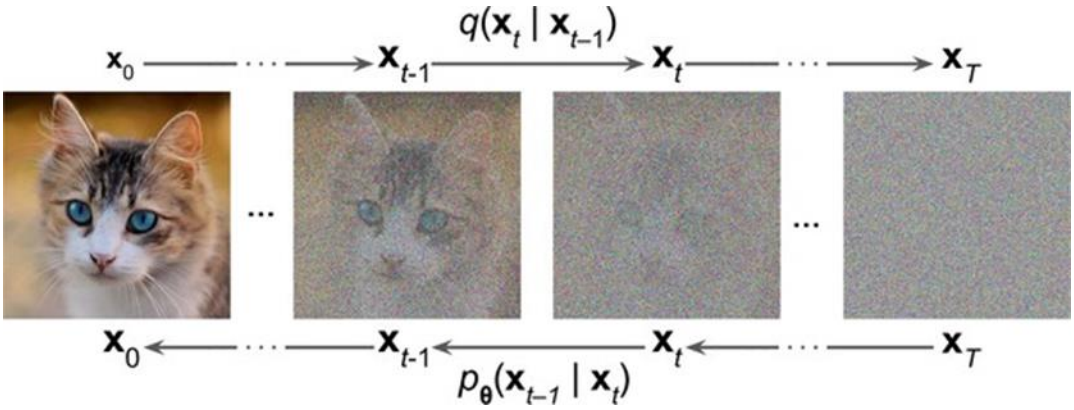
$$q\left(\mathbf{x}_t \middle| \mathbf{x}_{t-1}\right) = \mathcal{N}\left(\sqrt{1-\beta_t}\mathbf{x}_{t-1}\ ,\ \ \beta_t\mathbf{I}\right)$$

Điều thú vị là, có một lối tắt cho quá trình tiến: có thể lấy mẫu một hình ảnh `xt` cho trước `x0` mà không cần phải tính `x1`, `x2`, ..., `xt–1` trước. Thật vậy, vì tổng của nhiều phân phối Gaussian cũng là một phân phối Gaussian, tất cả nhiễu có thể được thêm vào chỉ trong một lần bằng cách sử dụng Công thức 17-6. Đây là phương trình chúng ta sẽ sử dụng, vì nó nhanh hơn nhiều.

Công thức 17-6. Lối tắt cho quá trình khuếch tán tiến

$$q\left(\mathbf{x}_t \middle| \mathbf{x}_0\right) = \mathcal{N}\left(\sqrt{\overline{\alpha}_t}\mathbf{x}_0\ ,\ \left(1-\overline{\alpha}_t\right)\mathbf{I}\right)$$

Mục tiêu của chúng ta, tất nhiên, không phải là nhấn chìm mèo trong nhiễu. Ngược lại, chúng ta muốn tạo ra nhiều con mèo mới! Chúng ta có thể làm như vậy bằng cách huấn luyện một mô hình có thể thực hiện quá trình ngược lại: đi từ `xt` đến `xt–1`. Sau đó, chúng ta có thể sử dụng nó để loại bỏ một chút nhiễu khỏi một hình ảnh, và lặp lại thao tác nhiều lần cho đến khi tất cả nhiễu biến mất. Nếu chúng ta huấn luyện mô hình trên một tập dữ liệu chứa nhiều hình ảnh mèo, thì chúng ta có thể cung cấp cho nó một bức ảnh hoàn toàn đầy nhiễu Gaussian, và mô hình sẽ dần dần tạo ra một con mèo hoàn toàn mới (xem Hình 17-20).



Hình 17-20. Quá trình tiến q và quá trình ngược p

Okay, bây giờ chúng ta sẽ bắt đầu viết code! Việc đầu tiên cần làm là lập trình quá trình chuyển tiếp (forward process). Để làm điều này, chúng ta cần triển khai lịch trình phương

sai. Làm thế nào chúng ta có thể kiểm soát tốc độ con mèo biến mất? Ban đầu, 100% phương sai đến từ hình ảnh con mèo gốc. Sau đó, tại mỗi bước thời gian t , phương sai được nhân với $1 - \beta_t$ và nhiễu được thêm vào. Vì vậy, phần phương sai đến từ phân phối ban đầu co lại bằng một hệ số $1 - \beta_t$ ở mỗi bước. Nếu chúng ta định nghĩa $a_t = 1 - \beta_t$, thì sau t bước thời gian, hình ảnh con mèo sẽ được nhân với một hệ số $\bar{\alpha}_t = a_1 a_2 \dots a_t = \prod_{i=1}^{t} a_i$. Đây là "hệ số tín hiệu con mèo" $\bar{\alpha}_t$ mà chúng ta muốn lên lịch trình để nó giảm dần từ 1 xuống 0 một cách từ từ giữa bước thời gian 0 và T .

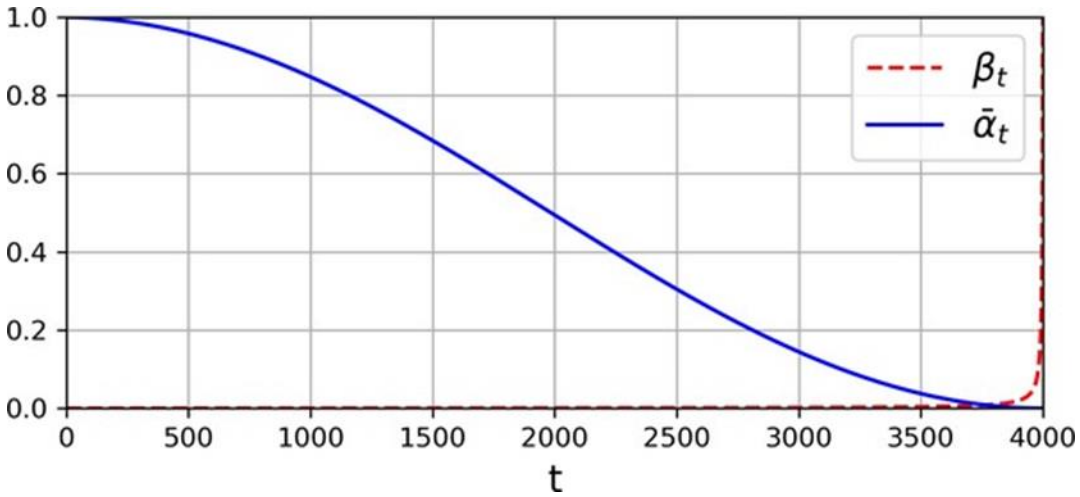
Trong bài báo DDPM được cải tiến, các tác giả đã lên lịch trình cho a_t theo **Công thức 17-7**. Lịch trình này được thể hiện trong Hình 17-21.

Công thức 17-7: Phương trình lịch trình phương sai cho quá trình khuếch tán chuyển tiếp

$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}, với \bar{\alpha}_t = \frac{f(t)}{f(0)} và f(t) = \left(\cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)\right)^2$$

Trong các phương trình này:

- s là một giá trị nhỏ ngăn β_t không quá nhỏ gần $t = 0$. Trong bài báo, các tác giả đã sử dụng $s = 0.008$.
- β_t được cắt để không lớn hơn 0.999, nhằm tránh sự mất ổn định gần $t = T$.



Hình 17-21. Lịch phương sai nhiễu βt, và phương sai tín hiệu còn lại α_bar_t

Hãy tạo một hàm nhỏ để tính αt, βt, và α_bar_t, và gọi nó với `T = 4.000`:

```
import numpy as np
import tensorflow as tf

def variance_schedule(T, s=0.008, max_beta=0.999):
    t = np.arange(T + 1)
    f = np.cos((t / T + s) / (1 + s) * np.pi / 2) ** 2
```

```
    alpha = np.clip(f[1:] / f[:-1], 1 - max_beta, 1)
    alpha = np.append(1, alpha).astype(np.float32) # add α₀ = 1
    beta = 1 - alpha
    alpha_cumprod = np.cumprod(alpha)

    return alpha, alpha_cumprod, beta # αₜ , ᾱₜ , βₜ for t = 0 to T

T = 4000
alpha, alpha_cumprod, beta = variance_schedule(T)
```

Để huấn luyện mô hình của chúng ta đảo ngược quá trình khuếch tán, chúng ta sẽ cần hình ảnh bị nhiễu từ các bước thời gian khác nhau của quá trình tiến. Để làm điều này, hãy tạo một hàm `prepare_batch()` sẽ lấy một batch hình ảnh sạch từ tập dữ liệu và chuẩn bị chúng:

```
import tensorflow as tf
import numpy as np

def prepare_batch(X):
    # Đảm bảo X là float32 và mở rộng từ [0, 1] sang [-1, 1]
    X = tf.cast(X[..., tf.newaxis], tf.float32) * 2 - 1
    X_shape = tf.shape(X)
    # Lấy một bước thời gian ngẫu nhiên cho mỗi hình ảnh trong batch
    t = tf.random.uniform([X_shape[0]], minval=1, maxval=T + 1,
dtype=tf.int32)
    # Lấy alpha_cumprod tương ứng với bước thời gian t
    alpha_cm = tf.gather(alpha_cumprod, t)
    # Định hình lại alpha_cm để có thể broadcast với X (ví dụ: [batch_size,
1, 1, 1])
    alpha_cm = tf.reshape(alpha_cm, [X_shape[0]] + [1] * (len(X_shape) - 1))
    noise = tf.random.normal(X_shape)

    # Áp dụng quá trình khuếch tán tiến (Công thức 17-6)
    X_noisy = alpha_cm ** 0.5 * X + (1 - alpha_cm) ** 0.5 * noise

    return {
        "X_noisy": X_noisy,
        "time": t,
    }, noise
```

Hãy xem qua đoạn mã này:

- Để đơn giản, chúng ta sẽ sử dụng Fashion MNIST, vì vậy hàm phải thêm một trục kênh trước. Nó cũng sẽ giúp thay đổi tỷ lệ các giá trị pixel từ –1 đến 1, để nó gần hơn với phân phối Gaussian cuối cùng với trung bình 0 và phương sai 1.
- Tiếp theo, hàm tạo `t`, một vector chứa một bước thời gian ngẫu nhiên cho mỗi hình ảnh trong batch, từ 1 đến `T`.

- Sau đó, nó sử dụng `tf.gather()` để lấy giá trị của `alpha_cumprod` cho mỗi bước thời gian trong vector `t`. Điều này mang lại cho chúng ta vector `alpha_cm`, chứa một giá trị `α_bar_t` cho mỗi hình ảnh.
- Dòng tiếp theo định hình lại `alpha_cm` từ `[kích thước batch]` thành `[kích thước batch, 1, 1, 1]`. Điều này là cần thiết để đảm bảo `alpha_cm` có thể được broadcast với batch `X`.
- Sau đó chúng ta tạo ra một số nhiễu Gaussian với trung bình 0 và phương sai 1.
- Cuối cùng, chúng ta sử dụng Công thức 17-6 để áp dụng quá trình khuếch tán vào hình ảnh. Lưu ý rằng `x ** 0.5` bằng căn bậc hai của `x`. Hàm trả về một tuple chứa đầu vào và mục tiêu. Các đầu vào được biểu diễn dưới dạng một dict Python chứa các hình ảnh bị nhiễu và các bước thời gian được sử dụng để tạo ra chúng. Các mục tiêu là nhiễu Gaussian được sử dụng để tạo ra mỗi hình ảnh.

Tiếp theo, chúng ta sẽ tạo một tập dữ liệu huấn luyện và một tập xác thực sẽ áp dụng hàm `prepare_batch()` cho mọi batch. Như trước đó, `X_train` và `X_valid` chứa các hình ảnh Fashion MNIST với các giá trị pixel từ 0 đến 1:

```
import tensorflow as tf

def prepare_dataset(X, batch_size=32, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices(X)

    if shuffle:
        ds = ds.shuffle(buffer_size=10_000)

    return ds.batch(batch_size).map(prepare_batch).prefetch(1)

# Giả định X_train và X_valid đã được định nghĩa
# train_set = prepare_dataset(X_train, batch_size=32, shuffle=True)
# valid_set = prepare_dataset(X_valid, batch_size=32)
```

Bây giờ chúng ta đã sẵn sàng để xây dựng mô hình khuếch tán thực tế. Nó có thể là bất kỳ mô hình nào bạn muốn, miễn là nó nhận các hình ảnh bị nhiễu và các bước thời gian làm đầu vào, và dự đoán nhiễu cần trừ khỏi hình ảnh đầu vào:

```
import tensorflow as tf

def build_diffusion_model():
    X_noisy = tf.keras.layers.Input(shape=[28, 28, 1], name="X_noisy")
    time_input = tf.keras.layers.Input(shape=[], dtype=tf.int32, name="time")

    # [...] # xây dựng mô hình dựa trên hình ảnh bị nhiễu và các bước thời
gian
    # outputs = [...] # dự đoán nhiễu (cùng hình dạng với hình ảnh đầu vào)
    # return tf.keras.Model(inputs=[X_noisy, time_input], outputs=[outputs])

    # Đây là một ví dụ đơn giản để minh họa cấu trúc, không phải kiến trúc U-
```

```
Net thực tế
    # Bạn sẽ thay thế phần này bằng kiến trúc U-Net được mô tả
    x = tf.keras.layers.Conv2D(32, 3, padding="same",
activation="relu")(X_noisy)
    x = tf.keras.layers.MaxPool2D(pool_size=2)(x)
    x = tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    # Để đơn giản, bỏ qua việc tích hợp time_input vào đây
    # Trong một DDPM thực tế, time_input sẽ được mã hóa và nhúng vào mạng

    # Giả định đầu ra có cùng hình dạng với đầu vào nhiễu
    outputs = tf.keras.layers.Conv2DTranspose(32, 3, strides=2,
padding="same", activation="relu")(x)
    outputs = tf.keras.layers.Conv2DTranspose(1, 3, padding="same",
activation="linear")(outputs) # Đầu ra là nhiễu, không phải ảnh, nên dùng
linear

    return tf.keras.Model(inputs=[X_noisy, time_input], outputs=[outputs])

# model = build_diffusion_model()
```

Các tác giả của DDPM đã sử dụng kiến trúc U-Net đã được sửa đổi, có nhiều điểm tương đồng với kiến trúc FCN mà chúng ta đã thảo luận trong Chương 14 về phân đoạn ngữ nghĩa: đó là một mạng thần kinh tích chập dần dần giảm mẫu hình ảnh đầu vào, sau đó dần dần tăng mẫu chúng trở lại, với các kết nối bỏ qua (skip connections) từ mỗi cấp độ của phần giảm mẫu sang cấp độ tương ứng trong phần tăng mẫu. Để tính đến các bước thời gian, họ đã mã hóa chúng bằng cách sử dụng cùng kỹ thuật như mã hóa vị trí trong kiến trúc transformer (xem Chương 16). Ở mỗi cấp độ trong kiến trúc U-Net, họ truyền các mã hóa thời gian này qua các lớp `Dense` và cấp chúng cho U-Net. Cuối cùng, họ cũng sử dụng các lớp chú ý đa đầu (multi-head attention) ở các cấp độ khác nhau. Xem sổ tay của chương này để biết một triển khai cơ bản, hoặc https://homl.info/ddpmcode để biết triển khai chính thức: nó dựa trên TF 1.x, đã lỗi thời, nhưng khá dễ đọc.

Chúng ta bây giờ có thể huấn luyện mô hình bình thường. Các tác giả lưu ý rằng việc sử dụng hàm mất MAE hoạt động tốt hơn MSE. Bạn cũng có thể sử dụng hàm mất Huber:

```
import tensorflow as tf

# Giả định model đã được xây dựng bằng build_diffusion_model()
# Giả định train_set và valid_set đã được chuẩn bị bằng prepare_dataset()

# model.compile(loss=tf.keras.losses.Huber(), optimizer="nadam")
# history = model.fit(train_set, validation_data=valid_set, epochs=100)
```

Sau khi mô hình được huấn luyện, bạn có thể sử dụng nó để tạo hình ảnh mới. Thật không may, không có lối tắt trong quá trình khuếch tán ngược, vì vậy bạn phải lấy mẫu $\mathbf{x}_T$ ngẫu nhiên từ một phân phối Gaussian với trung bình 0 và phương sai 1, sau đó truyền nó cho mô hình để dự đoán nhiễu; trừ nó khỏi hình ảnh bằng cách sử dụng Công thức 17-8, và bạn sẽ

có xT–1. Lặp lại quá trình thêm 3.999 lần nữa cho đến khi bạn có x0: nếu mọi thứ diễn ra tốt đẹp, nó sẽ trông giống như một hình ảnh Fashion MNIST thông thường!

Công thức 17-8. Đi ngược một bước trong quá trình khuếch tán

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\overline{\alpha}_t}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\mathbf{x}_t, t\right)\right) + \sqrt{\beta_t}\mathbf{z}$$

Trong phương trình này, ϵθ(xt, t) biểu thị nhiễu được dự đoán bởi mô hình cho trước hình ảnh đầu vào xt và bước thời gian t. θ biểu thị các tham số mô hình. Hơn nữa, z là nhiễu Gaussian với trung bình 0 và phương sai 1. Điều này làm cho quá trình ngược trở nên ngẫu nhiên: nếu bạn chạy nó nhiều lần, bạn sẽ nhận được các hình ảnh khác nhau.

Hãy viết một hàm triển khai quá trình ngược này, và gọi nó để tạo một vài hình ảnh:

```
import tensorflow as tf
import numpy as np

def generate(model, batch_size=32):
    # Lấy mẫu xT ngẫu nhiên từ phân phối Gaussian với trung bình 0 và phương
sai 1
    X = tf.random.normal([batch_size, 28, 28, 1])

    for t in range(T, 0, -1): # Lặp từ T xuống 1
        # Nhiễu z. Đối với bước cuối cùng (t=1), z là một tensor zero
        noise = (tf.random.normal if t > 1 else tf.zeros)(tf.shape(X))

        # Dự đoán nhiễu (epsilon_theta)
        X_noise = model({"X_noisy": X, "time": tf.constant([t] *
batch_size)})

        # Áp dụng Công thức 17-8
        # Các biến alpha, beta, alpha_cumprod phải có sẵn từ biến toàn cục
        X = (
            1 / alpha[t] ** 0.5
            * (X - beta[t] / (1 - alpha_cumprod[t]) ** 0.5 * X_noise)
            + (1 - alpha[t]) ** 0.5 * noise
        )

    return X

# X_gen = generate(model) # Các hình ảnh được tạo
```

Điều này có thể mất một hoặc hai phút. Đó là nhược điểm chính của các mô hình khuếch tán: tạo hình ảnh chậm vì mô hình cần được gọi nhiều lần. Có thể làm điều này nhanh hơn bằng cách sử dụng giá trị T nhỏ hơn, hoặc bằng cách sử dụng cùng một dự đoán mô hình cho một vài bước cùng một lúc, nhưng các hình ảnh kết quả có thể không đẹp bằng. Điều

đó nói rằng, bất chấp hạn chế về tốc độ này, các mô hình khuếch tán vẫn tạo ra hình ảnh chất lượng cao và đa dạng, như bạn có thể thấy trong Hình 17-22.

Hình 17-22. Hình ảnh được tạo bởi DDPM

Các mô hình khuếch tán đã có những tiến bộ vượt bậc gần đây. Đặc biệt, một bài báo được xuất bản vào tháng 12 năm 2021 bởi Robin Rombach, Andreas Blattmann, và cộng sự, đã giới thiệu **mô hình khuếch tán tiềm ẩn (latent diffusion models)**, trong đó quá trình khuếch tán diễn ra trong không gian tiềm ẩn, thay vì không gian pixel. Để đạt được điều này, một autoencoder mạnh mẽ được sử dụng để nén mỗi hình ảnh huấn luyện thành một không gian tiềm ẩn nhỏ hơn nhiều, nơi quá trình khuếch tán diễn ra, sau đó autoencoder được sử dụng để giải nén biểu diễn tiềm ẩn cuối cùng, tạo ra hình ảnh đầu ra. Điều này làm tăng tốc đáng kể việc tạo hình ảnh, và giảm thời gian và chi phí huấn luyện một cách đáng kể. Quan trọng là, chất lượng của các hình ảnh được tạo ra là xuất sắc.

Hơn nữa, các nhà nghiên cứu cũng đã điều chỉnh các kỹ thuật điều kiện hóa khác nhau để hướng dẫn quá trình khuếch tán bằng cách sử dụng các lời nhắc văn bản, hình ảnh hoặc bất kỳ đầu vào nào khác. Điều này giúp có thể nhanh chóng tạo ra một hình ảnh độ phân giải cao, đẹp mắt về một con kỳ nhông đang đọc sách, hoặc bất cứ thứ gì khác bạn có thể thích. Bạn cũng có thể điều kiện hóa quá trình tạo hình ảnh bằng cách sử dụng một hình ảnh đầu vào. Điều này cho phép nhiều ứng dụng, chẳng hạn như vẽ ngoài (outpainting) — trong đó một hình ảnh đầu vào được mở rộng ra ngoài biên giới của nó — hoặc vẽ trong (inpainting) — trong đó các lỗ trên một hình ảnh được lấp đầy.

Cuối cùng, một mô hình khuếch tán tiềm ẩn mạnh mẽ đã được huấn luyện trước có tên Stable Diffusion đã được mã nguồn mở vào tháng 8 năm 2022 bởi sự hợp tác giữa LMU Munich và một vài công ty, bao gồm StabilityAI và Runway, với sự hỗ trợ từ EleutherAI và LAION. Vào tháng 9 năm 2022, nó đã được chuyển sang TensorFlow và được bao gồm trong KerasCV, một thư viện thị giác máy tính được xây dựng bởi nhóm Keras. Bây giờ bất kỳ ai cũng có thể tạo ra những hình ảnh tuyệt vời trong vài giây, miễn phí, ngay cả trên một máy tính xách tay thông thường (xem bài tập cuối cùng trong chương này). Khả năng là vô tận!

Trong chương tiếp theo, chúng ta sẽ chuyển sang một nhánh hoàn toàn khác của học sâu: học tăng cường sâu (deep reinforcement learning).

Bài tập

- Các nhiệm vụ chính mà autoencoder được sử dụng là gì?
- Giả sử bạn muốn huấn luyện một bộ phân loại, và bạn có rất nhiều dữ liệu huấn luyện không được gán nhãn nhưng chỉ có vài nghìn trường hợp được gán nhãn. Autoencoder có thể giúp ích như thế nào? Bạn sẽ tiến hành như thế nào?
- Nếu một autoencoder tái tạo hoàn hảo đầu vào, nó có nhất thiết là một autoencoder tốt không? Làm thế nào bạn có thể đánh giá hiệu suất của một autoencoder?
- Autoencoder thiếu hoàn chỉnh (undercomplete) và thừa hoàn chỉnh (overcomplete) là gì? Rủi ro chính của một autoencoder thiếu hoàn chỉnh quá mức là gì? Còn rủi ro chính của một autoencoder thừa hoàn chỉnh thì sao?
- Bạn làm thế nào để ràng buộc trọng số trong một autoencoder xếp chồng? Mục đích của việc làm như vậy là gì?
- Mô hình sinh là gì? Bạn có thể kể tên một loại autoencoder sinh không?
- GAN là gì? Bạn có thể kể tên một vài nhiệm vụ mà GAN có thể phát huy tác dụng không?
- Những khó khăn chính khi huấn luyện GAN là gì?
- Mô hình khuếch tán giỏi điều gì? Hạn chế chính của chúng là gì?
- Hãy thử sử dụng autoencoder khử nhiễu để tiền huấn luyện một bộ phân loại hình ảnh. Bạn có thể sử dụng MNIST (tùy chọn đơn giản nhất), hoặc một tập dữ liệu hình ảnh phức tạp hơn như CIFAR10 nếu bạn muốn thử thách lớn hơn. Bất kể tập dữ liệu bạn đang sử dụng là gì, hãy làm theo các bước sau: a. Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra. Huấn luyện một autoencoder khử nhiễu sâu trên toàn bộ tập huấn luyện. b. Kiểm tra xem các hình ảnh có được tái tạo khá tốt không. Trực quan hóa các hình ảnh kích hoạt mạnh nhất mỗi neuron trong lớp mã hóa. c. Xây dựng một DNN phân loại, sử dụng lại các lớp thấp hơn của autoencoder. Huấn luyện nó chỉ với 500 hình ảnh từ tập huấn luyện. Nó hoạt động tốt hơn khi có hoặc không có tiền huấn luyện?
- Huấn luyện một autoencoder biến phân trên tập dữ liệu hình ảnh bạn chọn, và sử dụng nó để tạo hình ảnh. Ngoài ra, bạn có thể thử tìm một tập dữ liệu không được gán nhãn mà bạn quan tâm và xem liệu bạn có thể tạo ra các mẫu mới không.
- Huấn luyện một DCGAN để giải quyết tập dữ liệu hình ảnh bạn chọn, và sử dụng nó để tạo hình ảnh. Thêm replay kinh nghiệm và xem liệu điều này có giúp ích không. Biến nó thành một GAN có điều kiện nơi bạn có thể kiểm soát lớp được tạo.
- Hãy xem hướng dẫn Stable Diffusion tuyệt vời của KerasCV, và tạo một bản vẽ đẹp về một con kỳ nhông đang đọc sách. Nếu bạn đăng bản vẽ đẹp nhất của mình lên Twitter, vui lòng gắn thẻ tôi tại @aureliengeron. Tôi rất muốn xem các tác phẩm của bạn!

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

---

[1] William G. Chase và Herbert A. Simon, “Perception in Chess”, *Cognitive Psychology* 4, số 1 (1973): 55–81. [2] Yoshua Bengio và cộng sự, “Greedy Layer-Wise Training of Deep Networks”, *Proceedings of the 19th International Conference on Neural Information Processing Systems* (2006): 153–160. [3] Jonathan Masci và cộng sự, “Stacked Convolutional Auto-Encoders for Hierarchical Feature Extraction”, *Proceedings of the 21st International Conference on Artificial Neural Networks* 1 (2011): 52–59. [4] Pascal Vincent và cộng sự, “Extracting and Composing Robust Features with Denoising Autoencoders”, *Proceedings of the 25th International Conference on Machine Learning* (2008): 1096–1103. [5] Pascal Vincent và cộng sự, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion”, *Journal of Machine Learning Research* 11 (2010): 3371–3408. [6] Diederik Kingma và Max Welling, “Auto-Encoding Variational Bayes”, *arXiv preprint* arXiv:1312.6114 (2013). [7] Autoencoder biến phân thực tế tổng quát hơn; các mã hóa không giới hạn ở phân phối Gaussian. [8] Để biết thêm chi tiết toán học, hãy xem bài báo gốc về autoencoder biến phân, hoặc hướng dẫn tuyệt vời của Carl Doersch (2016). [9] Ian Goodfellow và cộng sự, “Generative Adversarial Nets”, *Proceedings of the 27th International Conference on Neural Information Processing Systems* 2 (2014): 2672–2680. [10] Để so sánh các hàm mất GAN chính, hãy xem dự án GitHub tuyệt vời này của Hwalsuk Lee. [11] Mario Lucic và cộng sự, “Are GANs Created Equal? A Large-Scale Study”, *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (2018): 698–707. [12] Alec Radford và cộng sự, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”, *arXiv preprint* arXiv:1511.06434 (2015). [13] Được tái bản với sự cho phép của các tác giả. [14] Mehdi Mirza và Simon Osindero, “Conditional Generative Adversarial Nets”, *arXiv preprint* arXiv:1411.1784 (2014). [15] Tero Karras và cộng sự, “Progressive Growing of GANs for Improved Quality, Stability, and Variation”, *Proceedings of the International Conference on Learning Representations* (2018). [16] Dải động của một biến là tỷ lệ giữa giá trị cao nhất và thấp nhất mà nó có thể nhận. [17] Tero Karras và cộng sự, “A Style-Based Generator Architecture for Generative Adversarial Networks”, *arXiv preprint* arXiv:1812.04948 (2018). [18] Được tái bản với sự cho phép của các tác giả. [19] Jascha Sohl-Dickstein và cộng sự, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”, *arXiv preprint* arXiv:1503.03585 (2015). [20] Jonathan Ho và cộng sự, “Denoising Diffusion Probabilistic Models” (2020). [21] Alex Nichol và Prafulla Dhariwal, “Improved Denoising Diffusion Probabilistic Models” (2021). [22] Olaf Ronneberger và cộng sự, “U-Net: Convolutional Networks for Biomedical Image Segmentation”, *arXiv preprint* arXiv:1505.04597 (2015). [23] Robin Rombach, Andreas Blattmann, và cộng sự, “High-Resolution Image Synthesis with Latent Diffusion Models”, *arXiv preprint* arXiv:2112.10752 (2021).

CHƯƠNG 18. HỌC TĂNG CƯỜNG

Học tăng cường (RL) là một trong những lĩnh vực thú vị nhất của học máy hiện nay, và cũng là một trong những lĩnh vực lâu đời nhất. Nó đã xuất hiện từ những năm 1950, tạo ra nhiều ứng dụng thú vị trong những năm qua, đặc biệt trong các trò chơi (ví dụ: TD-Gammon, một chương trình chơi Backgammon) và trong điều khiển máy móc, nhưng hiếm khi trở thành tin tức nóng hổi. Tuy nhiên, một cuộc cách mạng đã diễn ra vào năm 2013, khi các nhà nghiên cứu từ một công ty khởi nghiệp của Anh tên là DeepMind đã chứng minh một hệ thống có thể học chơi gần như bất kỳ trò chơi Atari nào từ đầu , cuối cùng vượt qua con người trong hầu hết các trò chơi đó, chỉ sử dụng các pixel thô làm đầu vào và không có bất kỳ kiến thức nào về luật chơi trước đó. Đây là thành tựu đầu tiên trong một loạt các thành tựu đáng kinh ngạc, đỉnh điểm là chiến thắng của hệ thống AlphaGo của họ trước Lee Sedol, một kỳ thủ cờ vây chuyên nghiệp huyền thoại, vào tháng 3 năm 2016 và trước Ke Jie, nhà vô địch thế giới, vào tháng 5 năm 2017. Chưa có chương trình nào từng đến gần việc đánh bại một bậc thầy của trò chơi này, chứ đừng nói đến nhà vô địch thế giới. Ngày nay, toàn bộ lĩnh vực RL đang sôi sục với những ý tưởng mới, với nhiều ứng dụng đa dạng.

Vậy DeepMind (được Google mua lại với giá hơn 500 triệu đô la vào năm 2014) đã đạt được tất cả những điều này như thế nào? Nhìn lại, có vẻ khá đơn giản: họ đã áp dụng sức mạnh của học sâu vào lĩnh vực học tăng cường, và nó đã hoạt động vượt xa những gì họ tưởng tượng. Trong chương này, tôi sẽ giải thích học tăng cường là gì và nó tốt ở điểm nào, sau đó trình bày hai trong số các kỹ thuật quan trọng nhất trong học tăng cường sâu: độ dốc chính sách (policy gradients) và mạng Q sâu (deep Q-networks), bao gồm thảo luận về các quá trình quyết định Markov. Hãy bắt đầu!

Học để tối ưu hóa phần thưởng

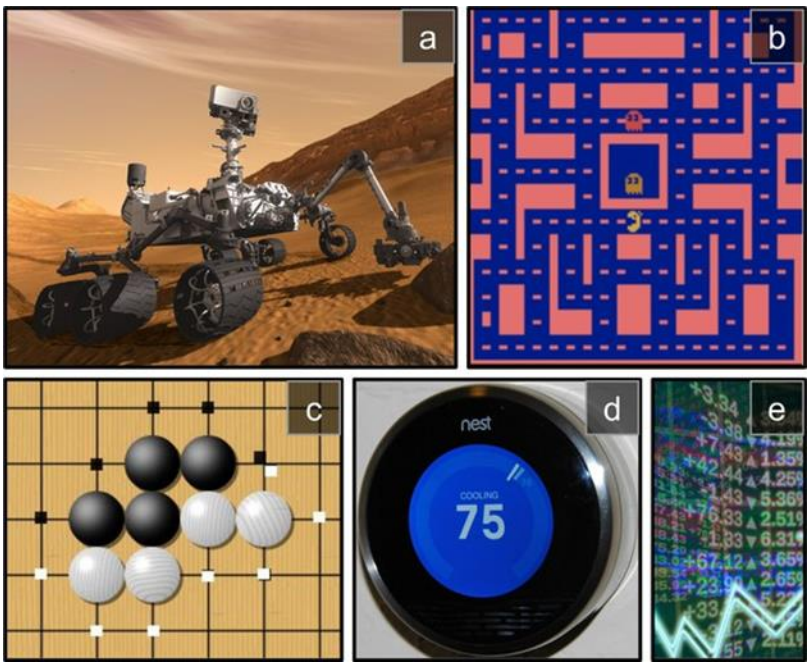
Trong học tăng cường, một tác nhân phần mềm thực hiện các quan sát và thực hiện các hành động trong một môi trường, và đổi lại nó nhận được phần thưởng từ môi trường. Mục tiêu của nó là học cách hành động theo cách sẽ tối đa hóa phần thưởng dự kiến của nó theo thời gian. Nếu bạn không ngại một chút nhân hóa, bạn có thể coi phần thưởng tích cực là khoái cảm và phần thưởng tiêu cực là nỗi đau (thuật ngữ “phần thưởng” hơi gây hiểu lầm trong trường hợp này). Tóm lại, tác nhân hành động trong môi trường và học bằng thử và sai để tối đa hóa khoái cảm và giảm thiểu nỗi đau.

Đây là một bối cảnh khá rộng, có thể áp dụng cho nhiều nhiệm vụ khác nhau. Dưới đây là một vài ví dụ (xem Hình 18-1):

- Tác nhân có thể là chương trình điều khiển robot. Trong trường hợp này, môi trường là thế giới thực, tác nhân quan sát môi trường thông qua một bộ cảm biến như camera và cảm biến chạm, và các hành động của nó bao gồm việc gửi tín hiệu để kích hoạt động cơ. Nó có thể được lập trình để nhận phần thưởng tích cực bất cứ khi nào nó tiếp cận đích đến, và phần thưởng tiêu cực bất cứ khi nào nó lãng phí thời gian hoặc đi sai hướng.

- Tác nhân có thể là chương trình điều khiển Ms. Pac-Man. Trong trường hợp này, môi trường là một mô phỏng của trò chơi Atari, các hành động là chín vị trí joystick có thể có (trên cùng bên trái, xuống, giữa, v.v.), các quan sát là ảnh chụp màn hình, và phần thưởng chỉ là điểm trò chơi.
- Tương tự, tác nhân có thể là chương trình chơi một trò chơi cờ bàn như Cờ vây. Nó chỉ nhận được phần thưởng nếu nó thắng.
- Tác nhân không nhất thiết phải điều khiển một vật thể di chuyển vật lý (hoặc ảo). Ví dụ, nó có thể là một bộ điều nhiệt thông minh, nhận phần thưởng tích cực bất cứ khi nào nó gần nhiệt độ mục tiêu và tiết kiệm năng lượng, và phần thưởng tiêu cực khi con người cần điều chỉnh nhiệt độ, do đó tác nhân phải học cách dự đoán nhu cầu của con người.
- Tác nhân có thể quan sát giá thị trường chứng khoán và quyết định mua hoặc bán bao nhiêu mỗi giây. Phần thưởng rõ ràng là lợi nhuận và thua lỗ tiền tệ.

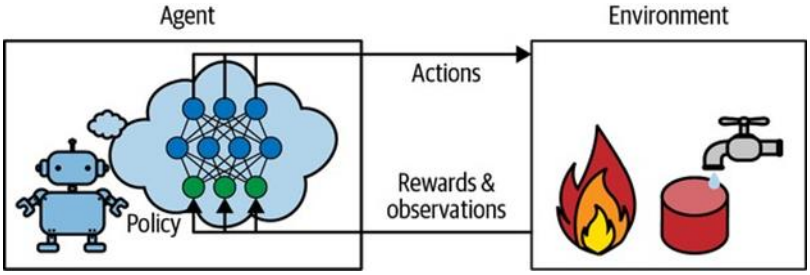
Lưu ý rằng có thể không có bất kỳ phần thưởng tích cực nào; ví dụ, tác nhân có thể di chuyển trong một mê cung, nhận phần thưởng tiêu cực ở mỗi bước thời gian, vì vậy tốt hơn là nó nên tìm lối ra càng nhanh càng tốt! Có nhiều ví dụ khác về các nhiệm vụ mà học tăng cường rất phù hợp, chẳng hạn như ô tô tự lái, hệ thống khuyến nghị, đặt quảng cáo trên trang web hoặc kiểm soát nơi hệ thống phân loại hình ảnh nên tập trung sự chú ý.



Hình 18-1. Các ví dụ về học tăng cường: (a) robot, (b) Ms. Pac-Man, (c) người chơi Cờ vây, (d) bộ điều nhiệt, (e) nhà giao dịch tự động

Tìm kiếm chính sách (Policy Search)

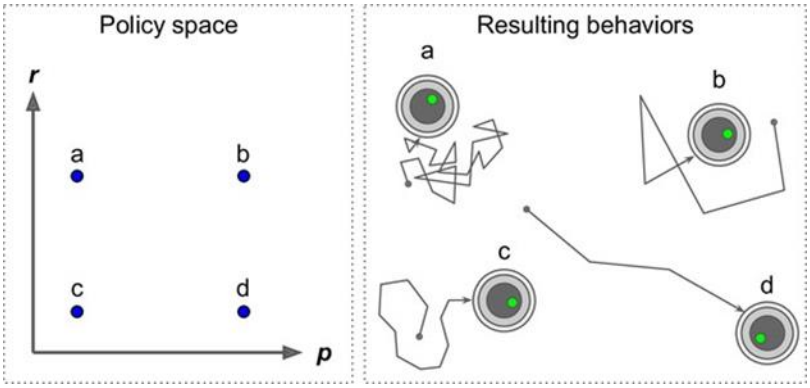
Thuật toán mà một tác nhân phần mềm sử dụng để xác định các hành động của nó được gọi là chính sách của nó. Chính sách có thể là một mạng nơ-ron nhận các quan sát làm đầu vào và xuất ra hành động cần thực hiện (xem Hình 18-2).



Hình 18-2. Học tăng cường sử dụng chính sách mạng nơ-ron

Chính sách có thể là bất kỳ thuật toán nào bạn có thể nghĩ ra, và nó không nhất thiết phải là xác định. Trên thực tế, trong một số trường hợp, nó thậm chí không cần phải quan sát môi trường! Ví dụ, hãy xem xét một robot hút bụi mà phần thưởng của nó là lượng bụi nó hút được trong 30 phút. Chính sách của nó có thể là di chuyển về phía trước với một xác suất p mỗi giây, hoặc xoay ngẫu nhiên sang trái hoặc phải với xác suất $1 - p$. Góc xoay sẽ là một góc ngẫu nhiên giữa $-r$ và $+r$. Vì chính sách này liên quan đến một số ngẫu nhiên, nó được gọi là chính sách ngẫu nhiên. Robot sẽ có một quỹ đạo thất thường, đảm bảo rằng nó cuối cùng sẽ đến bất kỳ nơi nào nó có thể tiếp cận và hút hết bụi. Câu hỏi là, nó sẽ hút được bao nhiêu bụi trong 30 phút?

Bạn sẽ huấn luyện một robot như vậy như thế nào? Chỉ có hai tham số chính sách mà bạn có thể điều chỉnh: xác suất p và phạm vi góc r . Một thuật toán học có thể là thử nhiều giá trị khác nhau cho các tham số này, và chọn sự kết hợp hoạt động tốt nhất (xem Hình 18-3). Đây là một ví dụ về tìm kiếm chính sách, trong trường hợp này sử dụng cách tiếp cận vét cạn. Khi không gian chính sách quá lớn (thường là như vậy), việc tìm một bộ tham số tốt theo cách này giống như tìm kim đáy bể. Một cách khác để khám phá không gian chính sách là sử dụng thuật toán di truyền. Ví dụ, bạn có thể tạo ngẫu nhiên thế hệ chính sách đầu tiên gồm 100 chính sách và thử chúng, sau đó “loại bỏ” 80 chính sách tệ nhất và để 20 chính sách sống sót tạo ra 4 con mỗi chính sách. Một con là một bản sao của bố mẹ nó cộng với một số biến thể ngẫu nhiên. Các chính sách sống sót cùng với con cái của chúng tạo thành thế hệ thứ hai. Bạn có thể tiếp tục lặp lại các thế hệ theo cách này cho đến khi bạn tìm thấy một chính sách tốt.



Hình 18-3. Bốn điểm trong không gian chính sách (trái) và hành vi tương ứng của tác nhân (phải)

Một cách tiếp cận khác là sử dụng các kỹ thuật tối ưu hóa, bằng cách đánh giá độ dốc của phần thưởng đối với các tham số chính sách, sau đó điều chỉnh các tham số này bằng cách đi theo độ dốc về phía phần thưởng cao hơn. Chúng ta sẽ thảo luận chi tiết hơn về cách tiếp cận này, được gọi là độ dốc chính sách (PG), sau trong chương này. Quay trở lại robot hút bụi, bạn có thể tăng nhẹ

p và đánh giá xem việc làm đó có làm tăng lượng bụi robot hút được trong 30 phút hay không; nếu có, thì tăng p thêm nữa, hoặc ngược lại giảm p . Chúng ta sẽ triển khai một thuật toán PG phổ biến bằng TensorFlow, nhưng trước khi làm vậy, chúng ta cần tạo một môi trường để tác nhân sống — vì vậy đã đến lúc giới thiệu OpenAI Gym.

Giới thiệu về OpenAI Gym

Một trong những thách thức của học tăng cường là để huấn luyện một tác nhân, trước tiên bạn cần có một môi trường hoạt động. Nếu bạn muốn lập trình một tác nhân sẽ học cách chơi một trò chơi Atari, bạn sẽ cần một trình mô phỏng trò chơi Atari. Nếu bạn muốn lập trình một robot đi bộ, thì môi trường là thế giới thực, và bạn có thể trực tiếp huấn luyện robot của mình trong môi trường đó. Tuy nhiên, điều này có những hạn chế: nếu robot rơi xuống vách đá, bạn không thể nhấp Hoàn tác. Bạn cũng không thể tăng tốc thời gian — thêm sức mạnh tính toán sẽ không làm robot di chuyển nhanh hơn — và nói chung quá tốn kém để huấn luyện 1.000 robot song song. Tóm lại, việc huấn luyện rất khó và chậm trong thế giới thực, vì vậy bạn thường cần một môi trường mô phỏng ít nhất để huấn luyện khởi động. Ví dụ, bạn có thể sử dụng một thư viện như PyBullet hoặc MuJoCo để mô phỏng vật lý 3D.

OpenAI Gym là một bộ công cụ cung cấp nhiều môi trường mô phỏng đa dạng (trò chơi Atari, trò chơi cờ bàn, mô phỏng vật lý 2D và 3D, v.v.) mà bạn có thể sử dụng để huấn luyện tác nhân, so sánh chúng hoặc phát triển các thuật toán RL mới.

OpenAI Gym được cài đặt sẵn trên Colab, nhưng đó là một phiên bản cũ hơn, vì vậy bạn sẽ cần thay thế nó bằng phiên bản mới nhất. Bạn cũng cần cài đặt một vài phần phụ thuộc của nó. Nếu bạn đang viết mã trên máy của riêng mình thay vì Colab và bạn đã làm theo hướng

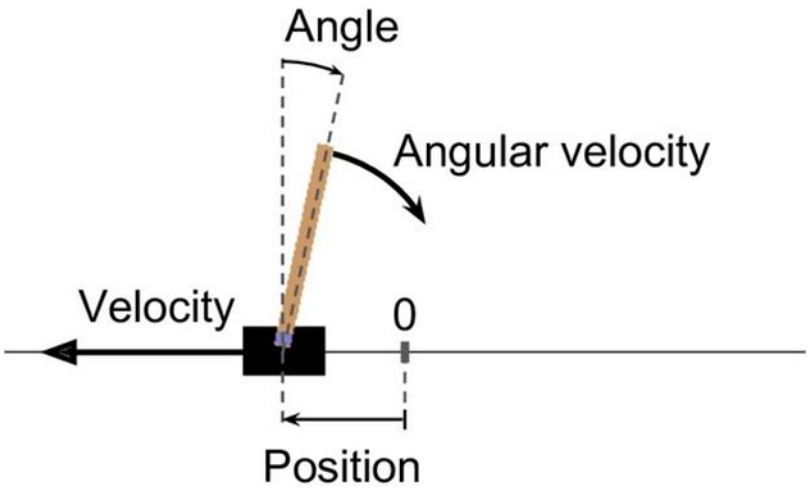
dẫn cài đặt tại https://homl.info/install , thì bạn có thể bỏ qua bước này; nếu không, hãy nhập các lệnh này:

```
#Chỉ chạy các lệnh này trên Colab hoặc Kaggle!
%pip install -q -U gym
%pip install -q -U gym[classic_control,box2d,atari,accept-rom-license]
```

Lệnh `%pip` đầu tiên nâng cấp Gym lên phiên bản mới nhất. Tùy chọn `-q` có nghĩa là "quiet": nó làm cho đầu ra ít dài dòng hơn. Tùy chọn `-U` có nghĩa là "upgrade". Lệnh `%pip` thứ hai cài đặt các thư viện cần thiết để chạy các loại môi trường khác nhau. Điều này bao gồm các môi trường cổ điển từ lý thuyết điều khiển — khoa học về điều khiển hệ thống động lực — chẳng hạn như giữ thăng bằng một cây cột trên một chiếc xe đẩy. Nó cũng bao gồm các môi trường dựa trên thư viện Box2D — một công cụ vật lý 2D cho trò chơi. Cuối cùng, nó bao gồm các môi trường dựa trên Arcade Learning Environment (ALE), là một trình giả lập cho các trò chơi Atari 2600. Một số ROM trò chơi Atari được tải xuống tự động và bằng cách chạy mã này, bạn đồng ý với giấy phép ROM của Atari. Với những điều đó, bạn đã sẵn sàng sử dụng OpenAI Gym. Hãy nhập nó và tạo một môi trường:

```
env = gym.make("CartPole-v1", render_mode="rgb_array")
```

Ở đây, chúng ta đã tạo một môi trường CartPole. Đây là một mô phỏng 2D trong đó một chiếc xe đẩy có thể được tăng tốc sang trái hoặc phải để giữ thăng bằng một cây cột đặt trên đó (xem Hình 18-4). Đây là một nhiệm vụ điều khiển cổ điển.



Hình 18-4. Môi trường CartPole

Sau khi môi trường được tạo, bạn phải khởi tạo nó bằng phương thức `reset()`, tùy chọn chỉ định một hạt giống ngẫu nhiên. Điều này trả về quan sát đầu tiên. Các quan sát phụ thuộc vào loại môi trường. Đối với môi trường CartPole, mỗi quan sát là một mảng NumPy 1D chứa bốn số float đại diện cho vị trí ngang của xe đẩy (0.0 = trung tâm), vận tốc của nó (dương nghĩa là sang phải), góc của cây cột (0.0 = thẳng đứng) và vận tốc góc của nó (dương nghĩa là theo chiều kim đồng hồ). Phương thức `reset()` cũng trả về một từ điển có thể chứa thông tin bổ sung cụ thể cho môi trường. Điều này có thể hữu ích cho việc gỡ lỗi

hoặc huấn luyện. Ví dụ, trong nhiều môi trường Atari, nó chứa số mạng còn lại. Tuy nhiên, trong môi trường CartPole, từ điển này trống.

```
>>> obs, info = env.reset(seed=42)
>>> obs
array([ 0.0273956 , -0.00611216, 0.03585979, 0.0197368 ], dtype=float32)
>>> info
{}
```

Hãy gọi phương thức `render()` để hiển thị môi trường này dưới dạng một hình ảnh. Vì chúng ta đã đặt `render_mode="rgb_array"` khi tạo môi trường, hình ảnh sẽ được trả về dưới dạng một mảng NumPy:

```
>>> img = env.render()
>>> img.shape # chiều cao, chiều rộng, kênh (3 = Đỏ, Xanh lá, Xanh dương)
(400, 600, 3)
```

Bạn có thể sử dụng hàm `imshow()` của Matplotlib để hiển thị hình ảnh này, như bình thường. Bây giờ hãy hỏi môi trường những hành động nào có thể thực hiện:

```
>>> env.action_space Discrete(2)
```

`Discrete(2)` có nghĩa là các hành động có thể là số nguyên 0 và 1 , đại diện cho việc tăng tốc sang trái hoặc phải. Các môi trường khác có thể có các hành động rời rạc bổ sung, hoặc các loại hành động khác (ví dụ: liên tục). Vì cây cột đang nghiêng về bên phải (`obs[2] > 0`), hãy tăng tốc xe đẩy về bên phải:

```
>>> action = 1 # tăng tốc sang phải
>>> obs, reward, done, truncated, info = env.step(action)
>>> obs
array([ 0.02727336, 0.18847767, 0.03625453, -0.26141977], dtype=float32)
>>> reward 1.0
>>> done False
>>> truncated False
>>> info
{}
```

Phương thức `step()` thực hiện hành động mong muốn và trả về năm giá trị: `obs` Đây là quan sát mới. Xe đẩy hiện đang di chuyển về bên phải (`obs[1] > 0`). Cây cột vẫn nghiêng về bên phải (`obs[2] > 0`), nhưng vận tốc góc của nó hiện là âm (`obs[3] < 0`), vì vậy nó có thể sẽ nghiêng về bên trái sau bước tiếp theo.

`reward` Trong môi trường này, bạn nhận được phần thưởng 1.0 ở mỗi bước, bất kể bạn làm gì, vì vậy mục tiêu là giữ cho tập phim chạy càng lâu càng tốt.

done Giá trị này sẽ là `True` khi tập phim kết thúc. Điều này sẽ xảy ra khi cây cột nghiêng quá nhiều, hoặc đi ra khỏi màn hình, hoặc sau 200 bước (trong trường hợp sau, bạn đã thắng). Sau đó, môi trường phải được đặt lại trước khi có thể sử dụng lại.

`truncated` Giá trị này sẽ là `True` khi một tập phim bị gián đoạn sớm, ví dụ bởi một trình bao môi trường áp đặt số bước tối đa cho mỗi tập phim (xem tài liệu của Gym để biết thêm chi tiết về trình bao môi trường). Một số thuật toán RL xử lý các tập phim bị cắt cụt khác với các tập phim kết thúc bình thường (tức là khi `done` là `True`), nhưng trong chương này chúng ta sẽ xử lý chúng giống hệt nhau.

`info` Từ điển cụ thể của môi trường này có thể cung cấp thông tin bổ sung, giống như cái được trả về bởi phương thức `reset()`.

Hãy mã hóa cứng một chính sách đơn giản là tăng tốc sang trái khi cây cột nghiêng về bên trái và tăng tốc sang phải khi cây cột nghiêng về bên phải. Chúng ta sẽ chạy chính sách này để xem phần thưởng trung bình nó nhận được trong 500 tập phim:

```
def basic_policy(obs):
    angle = obs[2]
    return 0 if angle < 0 else 1

totals = []
for episode in range(500):
    episode_rewards = 0
    obs, info = env.reset(seed=episode)
    for step in range(200):
        action = basic_policy(obs)
        obs, reward, done, truncated, info = env.step(action)
        episode_rewards += reward
        if done or truncated:
            break
    totals.append(episode_rewards)
```

Mã này tự giải thích. Hãy xem kết quả:

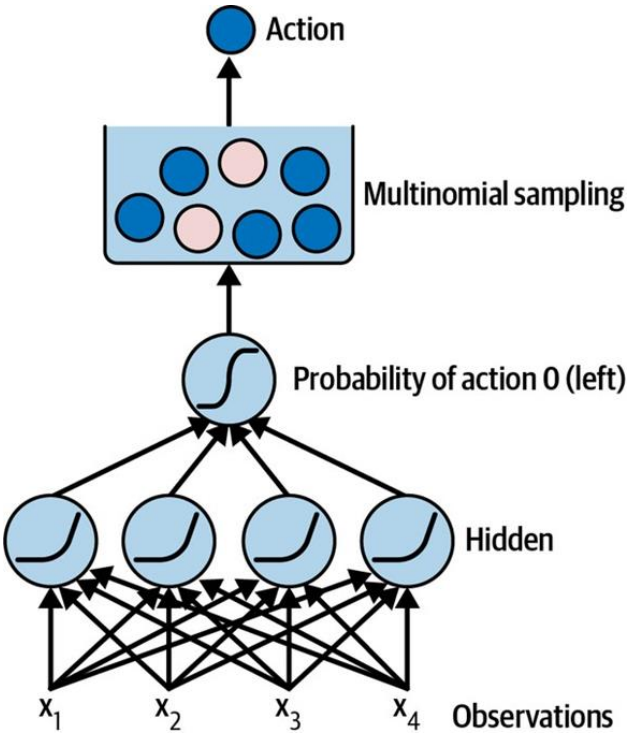
```
>>> import numpy as np
>>> np.mean(totals), np.std(totals), min(totals), max(totals)
(41.698, 8.389445512070509, 24.0, 63.0)
```

Ngay cả với 500 lần thử, chính sách này không bao giờ giữ cây cột thẳng đứng được quá 63 bước liên tiếp. Không tốt lắm. Nếu bạn nhìn vào mô phỏng trong sổ ghi chép của chương này, bạn sẽ thấy rằng xe đẩy dao động sang trái và phải ngày càng mạnh cho đến khi cây cột nghiêng quá nhiều. Hãy xem liệu một mạng nơ-ron có thể đưa ra một chính sách tốt hơn không.

Các chính sách mạng nơ-ron

Hãy tạo một chính sách mạng nơ-ron. Mạng nơ-ron này sẽ nhận một quan sát làm đầu vào, và nó sẽ xuất ra hành động cần thực hiện, giống như chính sách chúng ta đã mã hóa cứng trước đó. Chính xác hơn, nó sẽ ước tính một xác suất cho mỗi hành động, và sau đó chúng ta sẽ chọn một hành động ngẫu nhiên, theo các xác suất ước tính (xem Hình 18-5). Trong trường hợp môi trường CartPole, chỉ có hai hành động có thể (trái hoặc phải), vì vậy chúng ta chỉ cần một nơ-ron đầu ra. Nó sẽ xuất ra xác suất p của hành động 0 (trái), và tất nhiên

xác suất của hành động 1 (phải) sẽ là $1 - p$. Ví dụ, nếu nó xuất ra 0.7 , thì chúng ta sẽ chọn hành động 0 với 70 xác suất, hoặc hành động 1 với 30 xác suất.



Hình 18-5. Chính sách mạng nơ-ron

Bạn có thể tự hỏi tại sao chúng ta lại chọn một hành động ngẫu nhiên dựa trên các xác suất được đưa ra bởi mạng nơ-ron, thay vì chỉ chọn hành động có điểm cao nhất. Cách tiếp cận này cho phép tác nhân tìm được sự cân bằng phù hợp giữa việc khám phá các hành động mới và khai thác các hành động đã biết là hoạt động tốt. Đây là một ví dụ tương tự: giả sử bạn lần đầu tiên đến một nhà hàng, và tất cả các món ăn đều trông hấp dẫn như nhau, vì vậy bạn ngẫu nhiên chọn một món. Nếu nó hóa ra là ngon, bạn có thể tăng xác suất bạn sẽ gọi món đó lần sau, nhưng bạn không nên tăng xác suất đó lên đến 100 , nếu không bạn sẽ không bao giờ thử các món khác, một số trong đó có thể còn ngon hơn món bạn đã thử. Tình thế tiến thoái lưỡng nan khám phá/khai thác này là trung tâm trong học tăng cường. Cũng lưu ý rằng trong môi trường cụ thể này, các hành động và quan sát trong quá khứ có thể được bỏ qua một cách an toàn, vì mỗi quan sát chứa đầy đủ trạng thái của môi trường. Nếu có một trạng thái ẩn nào đó, thì bạn có thể cần phải xem xét cả các hành động và quan sát trong quá khứ. Ví dụ, nếu môi trường chỉ tiết lộ vị trí của xe đẩy mà không phải vận tốc của nó, bạn sẽ phải xem xét không chỉ quan sát hiện tại mà còn cả quan sát trước đó để ước tính vận tốc hiện tại. Một ví dụ khác là khi các quan sát bị nhiễu; trong trường hợp đó, bạn thường muốn sử dụng vài quan sát gần đây để ước tính trạng thái hiện tại có khả năng nhất. Vấn đề CartPole do đó đơn giản nhất có thể; các quan sát không bị nhiễu và chúng chứa đầy đủ trạng thái của môi trường. Đây là mã để xây dựng một chính sách mạng nơ-ron cơ bản bằng Keras:

```
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Dense(5, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
```

Chúng ta sử dụng một mô hình `Sequential` để định nghĩa mạng chính sách. Số lượng đầu vào là kích thước của không gian quan sát — trong trường hợp CartPole là 4 — và chúng ta chỉ có năm đơn vị ẩn vì đây là một nhiệm vụ khá đơn giản. Cuối cùng, chúng ta muốn xuất ra một xác suất duy nhất — xác suất đi sang trái — vì vậy chúng ta có một nơ-ron đầu ra duy nhất sử dụng hàm kích hoạt sigmoid. Nếu có nhiều hơn hai hành động có thể, sẽ có một nơ-ron đầu ra cho mỗi hành động, và chúng ta sẽ sử dụng hàm kích hoạt softmax thay thế. Được rồi, bây giờ chúng ta có một chính sách mạng nơ-ron sẽ nhận các quan sát và xuất ra xác suất hành động. Nhưng làm thế nào để chúng ta huấn luyện nó?

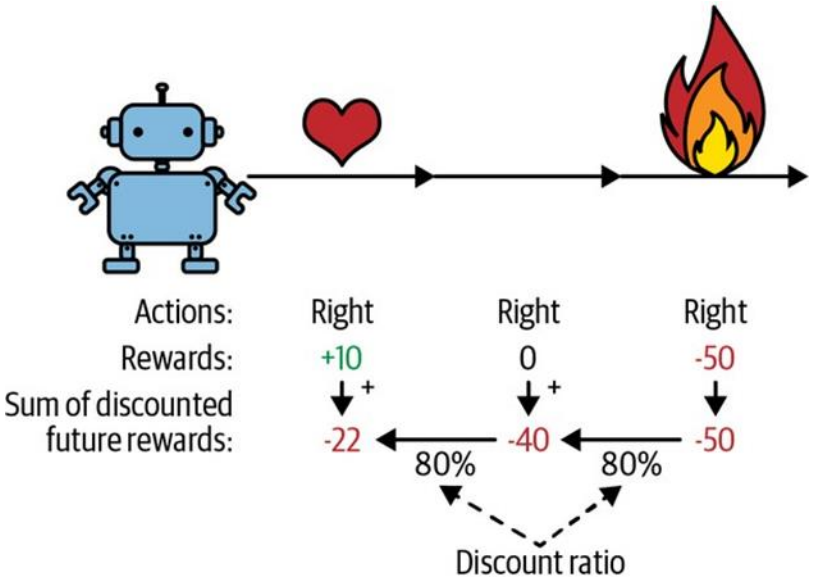
Đánh giá hành động: Bài toán phân bổ tín nhiệm

Nếu chúng ta biết hành động tốt nhất là gì ở mỗi bước, chúng ta có thể huấn luyện mạng nơ-ron như bình thường, bằng cách giảm thiểu entropy chéo giữa phân phối xác suất ước tính và phân phối xác suất mục tiêu. Đó sẽ chỉ là học có giám sát thông thường. Tuy nhiên, trong học tăng cường, hướng dẫn duy nhất mà tác nhân nhận được là thông qua phần thưởng, và phần thưởng thường thưa thớt và bị trì hoãn. Ví dụ, nếu tác nhân giữ thăng bằng được cây cột trong 100 bước, làm thế nào nó có thể biết hành động nào trong số 100 hành động nó đã thực hiện là tốt, và hành động nào là xấu? Tất cả những gì nó biết là cây cột đã rơi sau hành động cuối cùng, nhưng chắc chắn hành động cuối cùng này không hoàn toàn chịu trách nhiệm. Đây được gọi là **bài toán phân bổ tín nhiệm (credit assignment problem)**: khi tác nhân nhận được phần thưởng, rất khó để nó biết hành động nào nên được ghi công (hoặc đổ lỗi) cho phần thưởng đó. Hãy nghĩ về một con chó được thưởng sau nhiều giờ sau khi nó cư xử tốt; liệu nó có hiểu nó đang được thưởng vì điều gì không?

Để giải quyết vấn đề này, một chiến lược phổ biến là đánh giá một hành động dựa trên tổng của tất cả các phần thưởng đến sau nó, thường áp dụng một hệ số chiết khấu, $gamma$ (gamma), ở mỗi bước. Tổng của các phần thưởng được chiết khấu này được gọi là **lợi tức (return)** của hành động. Xem xét ví dụ trong Hình 18-6.

Nếu một tác nhân quyết định đi sang phải ba lần liên tiếp và nhận được phần thưởng $+10$ sau bước đầu tiên, 0 sau bước thứ hai, và cuối cùng -50 sau bước thứ ba, thì giả sử chúng ta sử dụng hệ số chiết khấu $gamma = 0.8$, hành động đầu tiên sẽ có lợi tức là $10 + gammatimes0 + gamma^2times(-50) = -22$. Nếu hệ số chiết khấu gần 0 , thì các phần thưởng trong tương lai sẽ không có nhiều giá trị so với các phần thưởng tức thời. Ngược lại, nếu hệ số chiết khấu gần 1 , thì các phần thưởng ở xa trong tương lai sẽ có giá trị gần như phần thưởng tức thời. Các hệ số chiết khấu điển hình thay đổi từ 0.9 đến 0.99 . Với hệ số chiết khấu 0.95 , phần thưởng 13 bước trong tương lai có giá trị bằng khoảng một nửa phần thưởng tức thời (vì $0.95^{13}approx0.5$), trong khi với hệ số chiết khấu 0.99 , phần thưởng 69 bước trong tương lai có giá trị bằng một nửa phần thưởng tức thời. Trong môi trường

CartPole, các hành động có tác động khá ngắn hạn, vì vậy việc chọn hệ số chiết khấu là 0.95 có vẻ hợp lý.



Hình 18-6. Tính toán lợi tức của một hành động: tổng các phần thưởng tương lai được chiết khấu

Tất nhiên, một hành động tốt có thể được theo sau bởi một số hành động xấu khiến cây cột rơi nhanh chóng, dẫn đến hành động tốt nhận được lợi tức thấp. Tương tự, một diễn viên giỏi đôi khi có thể đóng vai chính trong một bộ phim tồi tệ. Tuy nhiên, nếu chúng ta chơi trò chơi đủ lần, trung bình các hành động tốt sẽ nhận được lợi tức cao hơn các hành động xấu. Chúng ta muốn ước tính một hành động tốt hơn hoặc tệ hơn bao nhiêu, so với các hành động khả thi khác, trung bình. Đây được gọi là **lợi thế hành động (action advantage)**. Để làm được điều này, chúng ta phải chạy nhiều tập và chuẩn hóa tất cả các lợi tức hành động, bằng cách trừ đi giá trị trung bình và chia cho độ lệch chuẩn. Sau đó, chúng ta có thể hợp lý giả định rằng các hành động có lợi thế âm là xấu trong khi các hành động có lợi thế dương là tốt. Được rồi, bây giờ chúng ta đã có cách để đánh giá từng hành động, chúng ta đã sẵn sàng huấn luyện tác nhân đầu tiên của mình bằng cách sử dụng độ dốc chính sách. Hãy xem cách thực hiện.

Độ dốc chính sách (Policy Gradients)

Như đã thảo luận trước đó, các thuật toán PG tối ưu hóa các tham số của một chính sách bằng cách đi theo độ dốc về phía phần thưởng cao hơn. Một lớp thuật toán PG phổ biến, được gọi là thuật toán REINFORCE, được Ronald Williams giới thiệu vào năm 1992. Đây là một biến thể phổ biến:

- Đầu tiên, hãy để chính sách mạng nơ-ron chơi trò chơi nhiều lần, và ở mỗi bước, tính toán các độ dốc sẽ làm cho hành động đã chọn có khả năng xảy ra cao hơn — nhưng đừng áp dụng các độ dốc này ngay lập tức.

- Sau khi bạn đã chạy một số tập, hãy tính lợi thế của mỗi hành động, bằng cách sử dụng phương pháp được mô tả trong phần trước.
- Nếu lợi thế của một hành động là dương, điều đó có nghĩa là hành động đó có lẽ là tốt, và bạn muốn áp dụng các độ dốc đã tính toán trước đó để làm cho hành động đó có khả năng được chọn trong tương lai cao hơn. Tuy nhiên, nếu lợi thế của hành động là âm, điều đó có nghĩa là hành động đó có lẽ là xấu, và bạn muốn áp dụng các độ dốc ngược lại để làm cho hành động này ít khả năng xảy ra hơn trong tương lai. Giải pháp là nhân mỗi vector độ dốc với lợi thế tương ứng của hành động.
- Cuối cùng, tính giá trị trung bình của tất cả các vector độ dốc thu được, và sử dụng nó để thực hiện một bước giảm độ dốc.

Hãy sử dụng Keras để triển khai thuật toán này. Chúng ta sẽ huấn luyện chính sách mạng nơ-ron mà chúng ta đã xây dựng trước đó để nó học cách giữ thăng bằng cây cột trên xe đẩy. Đầu tiên, chúng ta cần một hàm sẽ chơi một bước. Hiện tại chúng ta sẽ giả vờ rằng bất kỳ hành động nào nó thực hiện đều là hành động đúng để chúng ta có thể tính toán tổn thất và độ dốc của nó. Các độ dốc này sẽ chỉ được lưu trong một thời gian, và chúng ta sẽ sửa đổi chúng sau này tùy thuộc vào hành động đó hóa ra tốt hay xấu:

```
def play_one_step(env, obs, model, loss_fn):
    with tf.GradientTape() as tape:
        left_proba = model(obs[np.newaxis])
        action = (tf.random.uniform([1, 1]) > left_proba)
        y_target = tf.constant([[1.]]) - tf.cast(action, tf.float32)
        loss = tf.reduce_mean(loss_fn(y_target, left_proba))

    grads = tape.gradient(loss, model.trainable_variables)
    obs, reward, done, truncated, info = env.step(int(action))
    return obs, reward, done, truncated, grads
```

Hãy cùng xem qua hàm này:

- Trong khối `GradientTape` (xem Chương 12), chúng ta bắt đầu bằng cách gọi mô hình, đưa cho nó một quan sát duy nhất. Chúng ta định hình lại quan sát để nó trở thành một lô chứa một thể hiện duy nhất, vì mô hình mong đợi một lô. Điều này xuất ra xác suất đi sang trái.
- Tiếp theo, chúng ta lấy mẫu một số float ngẫu nhiên giữa 0 và 1 , và chúng ta kiểm tra xem nó có lớn hơn `left_proba` hay không. Hành động sẽ là `False` với xác suất `left_proba`, hoặc `True` với xác suất $1 - textleft_proba$. Khi chúng ta chuyển đổi giá trị Boolean này thành một số nguyên, hành động sẽ là 0 (trái) hoặc 1 (phải) với các xác suất thích hợp.
- Bây giờ chúng ta định nghĩa xác suất mục tiêu của việc đi sang trái: đó là 1 trừ đi hành động (được chuyển đổi thành một số float). Nếu hành động là 0 (trái), thì xác suất mục tiêu của việc đi sang trái sẽ là 1 . Nếu hành động là 1 (phải), thì xác suất mục tiêu sẽ là 0 .

- Sau đó, chúng ta tính toán tổn thất bằng cách sử dụng hàm tổn thất đã cho, và chúng ta sử dụng băng để tính toán độ dốc của tổn thất đối với các biến có thể huấn luyện của mô hình. Một lần nữa, các độ dốc này sẽ được điều chỉnh sau này, trước khi chúng ta áp dụng chúng, tùy thuộc vào hành động đó hóa ra tốt hay xấu.
- Cuối cùng, chúng ta thực hiện hành động đã chọn, và chúng ta trả về quan sát mới, phần thưởng, liệu tập phim đã kết thúc hay chưa, liệu nó có bị cắt cụt hay không, và tất nhiên là các độ dốc mà chúng ta vừa tính toán.

Bây giờ hãy tạo một hàm khác sẽ dựa vào hàm `play_one_step()` để chơi nhiều tập, trả về tất cả các phần thưởng và độ dốc cho mỗi tập và mỗi bước:

```
def play_multiple_episodes(env, n_episodes, n_max_steps, model, loss_fn):
    all_rewards = []
    all_grads = []

    for episode in range(n_episodes):
        current_rewards = []
        current_grads = []
        obs, info = env.reset()

        for step in range(n_max_steps):
            obs, reward, done, truncated, grads = play_one_step(env, obs,
model, loss_fn)
            current_rewards.append(reward)
            current_grads.append(grads)

            if done or truncated:
                break

        all_rewards.append(current_rewards)
        all_grads.append(current_grads)

    return all_rewards, all_grads
```

Mã này trả về một danh sách các danh sách phần thưởng: một danh sách phần thưởng cho mỗi tập, chứa một phần thưởng cho mỗi bước. Nó cũng trả về một danh sách các danh sách độ dốc: một danh sách độ dốc cho mỗi tập, mỗi danh sách chứa một bộ độ dốc cho mỗi bước và mỗi bộ chứa một tensor độ dốc cho mỗi biến có thể huấn luyện. Thuật toán sẽ sử dụng hàm `play_multiple_episodes()` để chơi trò chơi nhiều lần (ví dụ: 10 lần), sau đó nó sẽ quay lại và xem tất cả các phần thưởng, chiết khấu chúng và chuẩn hóa chúng. Để làm điều đó, chúng ta cần thêm một vài hàm; hàm đầu tiên sẽ tính tổng các phần thưởng tương lai được chiết khấu ở mỗi bước, và hàm thứ hai sẽ chuẩn hóa tất cả các phần thưởng được chiết khấu này (tức là lợi tức) trên nhiều tập bằng cách trừ đi giá trị trung bình và chia cho độ lệch chuẩn:

```
def discount_rewards(rewards, discount_factor):
    discounted = np.array(rewards)
```

```
    for step in range(len(rewards) - 2, -1, -1):
        discounted[step] += discounted[step + 1] * discount_factor
    return discounted

def discount_and_normalize_rewards(all_rewards, discount_factor):
    all_discounted_rewards = [discount_rewards(rewards, discount_factor)
                              for rewards in all_rewards]
    flat_rewards = np.concatenate(all_discounted_rewards)
    reward_mean = flat_rewards.mean()
    reward_std = flat_rewards.std()

    return [(discounted_rewards - reward_mean) / reward_std for
discounted_rewards in all_discounted_rewards]
```

Hãy kiểm tra xem điều này có hoạt động không:

```
>>> discount_rewards([10, 0, -50], discount_factor=0.8)
array([-22, -40, -50])

>>> discount_and_normalize_rewards([[10, 0, -50], [10, 20]],
...                                discount_factor=0.8)
...
[array([-0.28435071, -0.86597718, -1.18910299]),
array([1.26665318, 1.0727777 ])]
```

Cuộc gọi tới `discount_rewards()` trả về chính xác những gì chúng ta mong đợi (xem Hình 18-6). Bạn có thể xác minh rằng hàm `discount_and_normalize_rewards()` thực sự trả về lợi thế hành động đã chuẩn hóa cho mỗi hành động trong cả hai tập. Lưu ý rằng tập đầu tiên tệ hơn nhiều so với tập thứ hai, vì vậy tất cả các lợi thế đã chuẩn hóa của nó đều là số âm; tất cả các hành động từ tập đầu tiên sẽ được coi là xấu, và ngược lại tất cả các hành động từ tập thứ hai sẽ được coi là tốt. Chúng ta gần như đã sẵn sàng để chạy thuật toán! Bây giờ hãy định nghĩa các siêu tham số. Chúng ta sẽ chạy 150 lần lặp huấn luyện, chơi 10 tập cho mỗi lần lặp, và mỗi tập sẽ kéo dài tối đa 200 bước. Chúng ta sẽ sử dụng hệ số chiết khấu là 0.95 :

```
n_iterations = 150
n_episodes_per_update = 10
n_max_steps = 200
discount_factor = 0.95
```

Chúng ta cũng cần một trình tối ưu hóa và hàm tổn thất. Một trình tối ưu hóa Nadam thông thường với tốc độ học 0.01 sẽ hoạt động tốt, và chúng ta sẽ sử dụng hàm tổn thất entropy chéo nhị phân vì chúng ta đang huấn luyện một bộ phân loại nhị phân (có hai hành động có thể — trái hoặc phải):

```
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.01)
loss_fn = tf.keras.losses.binary_crossentropy
```

Bây giờ chúng ta đã sẵn sàng để xây dựng và chạy vòng lặp huấn luyện!

```
for iteration in range(n_iterations):
    all_rewards, all_grads = play_multiple_episodes(
        env, n_episodes_per_update, n_max_steps, model, loss_fn)
    all_final_rewards = discount_and_normalize_rewards(all_rewards,
                                                       discount_factor)
    all_mean_grads = []

    for var_index in range(len(model.trainable_variables)):
        mean_grads = tf.reduce_mean(
            [final_reward * all_grads[episode_index][step][var_index]
             for episode_index, final_rewards in enumerate(all_final_rewards)
             for step, final_reward in enumerate(final_rewards)], axis=0)
        all_mean_grads.append(mean_grads)
    optimizer.apply_gradients(zip(all_mean_grads, model.trainable_variables))
```

Hãy cùng xem qua mã này:

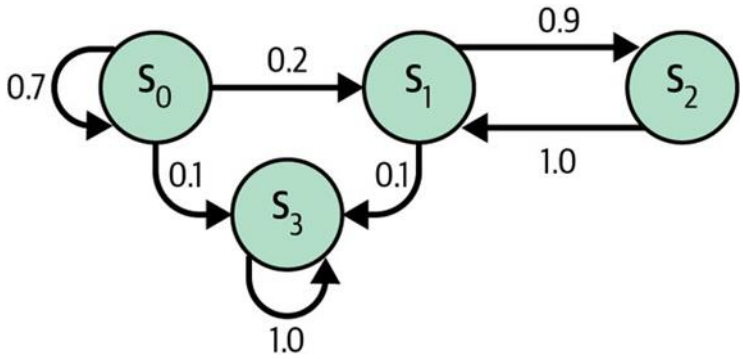
- Ở mỗi lần lặp huấn luyện, vòng lặp này gọi hàm `play_multiple_episodes()`, hàm này chơi 10 tập và trả về phần thưởng và độ dốc cho mỗi bước trong mỗi tập.
- Sau đó, chúng ta gọi hàm `discount_and_normalize_rewards()` để tính lợi thế chuẩn hóa của mỗi hành động, được gọi là `final_reward` trong mã này. Điều này cung cấp một thước đo về mức độ tốt hay xấu của mỗi hành động, nhìn lại.
- Tiếp theo, chúng ta xem xét từng biến có thể huấn luyện, và đối với mỗi biến đó, chúng ta tính toán giá trị trung bình có trọng số của độ dốc cho biến đó trên tất cả các tập và tất cả các bước, được trọng số bằng `final_reward`.
- Cuối cùng, chúng ta áp dụng các độ dốc trung bình này bằng cách sử dụng trình tối ưu hóa: các biến có thể huấn luyện của mô hình sẽ được điều chỉnh, và hy vọng chính sách sẽ tốt hơn một chút.

Và chúng ta đã hoàn tất! Mã này sẽ huấn luyện chính sách mạng nơ-ron, và nó sẽ học cách giữ thăng bằng cây cột trên xe đẩy một cách thành công. Phần thưởng trung bình mỗi tập sẽ đạt rất gần 200. Theo mặc định, đó là giá trị tối đa cho môi trường này. Thành công! Thuật toán độ dốc chính sách đơn giản mà chúng ta vừa huấn luyện đã giải quyết nhiệm vụ CartPole, nhưng nó sẽ không mở rộng tốt cho các nhiệm vụ lớn hơn và phức tạp hơn. Thật vậy, nó rất kém hiệu quả về mẫu, nghĩa là nó cần khám phá trò chơi trong một thời gian rất dài trước khi có thể đạt được tiến bộ đáng kể. Điều này là do nó phải chạy nhiều tập để ước tính lợi thế của mỗi hành động, như chúng ta đã thấy. Tuy nhiên, nó là nền tảng của các thuật toán mạnh mẽ hơn, chẳng hạn như thuật toán actor-critic (mà chúng ta sẽ thảo luận ngắn gọn ở cuối chương này).

Bây giờ chúng ta sẽ xem xét một nhóm thuật toán phổ biến khác. Trong khi các thuật toán PG trực tiếp cố gắng tối ưu hóa chính sách để tăng phần thưởng, các thuật toán chúng ta sẽ khám phá bây giờ ít trực tiếp hơn: tác nhân học cách ước tính lợi tức dự kiến cho mỗi trạng thái, hoặc cho mỗi hành động trong mỗi trạng thái, sau đó nó sử dụng kiến thức này để quyết định cách hành động. Để hiểu các thuật toán này, trước tiên chúng ta phải xem xét các quá trình quyết định Markov (MDPs).

Các quá trình quyết định Markov (Markov Decision Processes)

Vào đầu thế kỷ 20, nhà toán học Andrey Markov đã nghiên cứu các quá trình ngẫu nhiên không có bộ nhớ, được gọi là **chuỗi Markov (Markov chains)**. Một quá trình như vậy có một số trạng thái cố định, và nó ngẫu nhiên tiến hóa từ trạng thái này sang trạng thái khác ở mỗi bước. Xác suất để nó tiến hóa từ trạng thái s sang trạng thái s' là cố định, và nó chỉ phụ thuộc vào cặp (s, s'), không phụ thuộc vào các trạng thái trong quá khứ. Đây là lý do tại sao chúng ta nói rằng hệ thống không có bộ nhớ. Hình 18-7 cho thấy một ví dụ về một chuỗi Markov với bốn trạng thái.

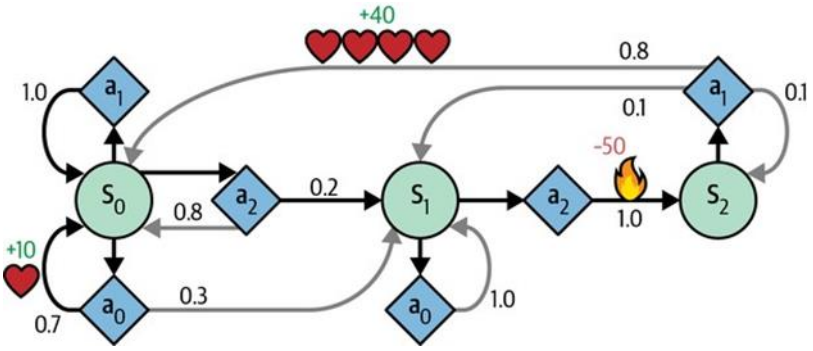


Hình 18-7. Ví dụ về một chuỗi Markov

Giả sử quá trình bắt đầu ở trạng thái s_0, và có 70 khả năng nó sẽ giữ nguyên trạng thái đó ở bước tiếp theo. Cuối cùng nó sẽ phải rời khỏi trạng thái đó và không bao giờ quay lại, bởi vì không có trạng thái nào khác chỉ ngược lại s_0. Nếu nó chuyển sang trạng thái s_1, nó sẽ rất có khả năng chuyển sang trạng thái s_2 (90 xác suất), sau đó ngay lập tức quay lại trạng thái s_1 (100 xác suất). Nó có thể luân phiên một số lần giữa hai trạng thái này, nhưng cuối cùng nó sẽ rơi vào trạng thái s_3 và ở đó mãi mãi, vì không có lối thoát: đây được gọi là **trạng thái kết thúc (terminal state)**. Các chuỗi Markov có thể có động lực học rất khác nhau, và chúng được sử dụng nhiều trong nhiệt động lực học, hóa học, thống kê, và nhiều hơn nữa.

Các quá trình quyết định Markov được Richard Bellman mô tả lần đầu tiên vào những năm 1950. Chúng giống với chuỗi Markov, nhưng có một sự khác biệt: ở mỗi bước, một tác nhân có thể chọn một trong số nhiều hành động khả thi, và xác suất chuyển đổi phụ thuộc vào hành động đã chọn. Hơn nữa, một số chuyển đổi trạng thái trả về một phần thưởng (tích cực hoặc tiêu cực), và mục tiêu của tác nhân là tìm một chính sách sẽ tối đa hóa phần thưởng theo thời gian.

Ví dụ, MDP được biểu diễn trong Hình 18-8 có ba trạng thái (được biểu thị bằng các vòng tròn) và tối đa ba hành động rời rạc có thể ở mỗi bước (được biểu thị bằng các hình thoi).



Hình 18-8. Ví dụ về một quá trình quyết định Markov

Nếu nó bắt đầu ở trạng thái s_0 , tác nhân có thể chọn giữa các hành động a_0, a_1 , hoặc a_2 . Nếu nó chọn hành động a_1 , nó chỉ giữ nguyên trạng thái s_0 một cách chắc chắn, và không có bất kỳ phần thưởng nào. Do đó nó có thể quyết định ở đó mãi mãi nếu muốn. Nhưng nếu nó chọn hành động a_0 , nó có 70 xác suất nhận được phần thưởng $+10$ và giữ nguyên trạng thái s_0 . Sau đó nó có thể thử đi thử lại để nhận được càng nhiều phần thưởng càng tốt, nhưng đến một lúc nào đó nó sẽ kết thúc ở trạng thái s_1 . Ở trạng thái s_1 nó chỉ có hai hành động có thể: a_0 hoặc a_2 . Nó có thể chọn ở lại bằng cách liên tục chọn hành động a_0 , hoặc nó có thể chọn chuyển sang trạng thái s_2 và nhận một phần thưởng âm -50 (ouch). Ở trạng thái s_2 nó không có lựa chọn nào khác ngoài việc thực hiện hành động a_1 , điều này rất có thể sẽ đưa nó trở lại trạng thái s_0 , nhận được phần thưởng $+40$ trên đường đi. Bạn hình dung được rồi chứ. Bằng cách nhìn vào MDP này, bạn có thể đoán chiến lược nào sẽ mang lại nhiều phần thưởng nhất theo thời gian không? Ở trạng thái s_0 rõ ràng hành động a_0 là lựa chọn tốt nhất, và ở trạng thái s_2 tác nhân không có lựa chọn nào khác ngoài việc thực hiện hành động a_1 , nhưng ở trạng thái s_1 không rõ liệu tác nhân nên ở lại (a_0) hay đi qua lửa (a_2).

Bellman đã tìm ra một cách để ước tính **giá trị trạng thái tối ưu (optimal state value)** của bất kỳ trạng thái s nào, được ký hiệu là $V^*(s)$, là tổng của tất cả các phần thưởng tương lai được chiết khấu mà tác nhân có thể mong đợi trung bình sau khi nó đạt đến trạng thái đó, giả sử nó hành động tối ưu. Ông đã chỉ ra rằng nếu tác nhân hành động tối ưu, thì **phương trình tối ưu Bellman (Bellman optimality equation)** sẽ áp dụng (xem Phương trình 18-1). Phương trình đệ quy này nói rằng nếu tác nhân hành động tối ưu, thì giá trị tối ưu của trạng thái hiện tại bằng phần thưởng nó sẽ nhận được trung bình sau khi thực hiện một hành động tối ưu, cộng với giá trị tối ưu dự kiến của tất cả các trạng thái tiếp theo có thể mà hành động này có thể dẫn đến. Phương trình 18-1. Phương trình tối ưu Bellman

$$V^*(s) = \max_a \sum_{s'} T\,(s,a,s')[R(s,a,s') + \gamma \cdot V^*(s')]$$

Trong phương trình này:

- $T(s,a,s')$ là xác suất chuyển đổi từ trạng thái s sang trạng thái s' , với điều kiện tác nhân đã chọn hành động a . Ví dụ, trong Hình 18-8, $T(s_2,a_1,s_0) = 0.8$.

- $R(s, a, s')$ là phần thưởng mà tác nhân nhận được khi nó đi từ trạng thái s sang trạng thái s' , với điều kiện tác nhân đã chọn hành động a . Ví dụ, trong Hình 18-8, $R(s_2, a_1, s_0) = +40$.
- $gamma$ là hệ số chiết khấu.

Phương trình này dẫn trực tiếp đến một thuật toán có thể ước tính chính xác giá trị trạng thái tối ưu của mọi trạng thái có thể: đầu tiên khởi tạo tất cả các ước tính giá trị trạng thái bằng không, và sau đó cập nhật chúng lặp đi lặp lại bằng thuật toán **lặp giá trị (value iteration algorithm)** (xem Phương trình 18-2). Một kết quả đáng chú ý là, nếu có đủ thời gian, các ước tính này được đảm bảo hội tụ về các giá trị trạng thái tối ưu, tương ứng với chính sách tối ưu. Phương trình 18-2. Thuật toán lặp giá trị

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T\,(s, a, s')[R(s, a, s') + \gamma \cdot V_k(s')]$$

Trong phương trình này, $V_k(s)$ là giá trị ước tính của trạng thái s tại lần lặp thứ k của thuật toán.

Biết các giá trị trạng thái tối ưu có thể hữu ích, đặc biệt để đánh giá một chính sách, nhưng nó không cung cấp cho chúng ta chính sách tối ưu cho tác nhân. May mắn thay, Bellman đã tìm ra một thuật toán rất tương tự để ước tính các **giá trị trạng thái-hành động tối ưu (optimal state-action values)**, thường được gọi là **giá trị Q (Q-values)** (giá trị chất lượng). Giá trị Q tối ưu của cặp trạng thái-hành động (s, a) , được ký hiệu là $Q^*(s, a)$, là tổng của các phần thưởng tương lai được chiết khấu mà tác nhân có thể mong đợi trung bình sau khi nó đạt đến trạng thái s và chọn hành động a , nhưng trước khi nó thấy kết quả của hành động này, giả sử nó hành động tối ưu sau hành động đó.

Hãy xem nó hoạt động như thế nào. Một lần nữa, bạn bắt đầu bằng cách khởi tạo tất cả các ước tính giá trị Q bằng không, sau đó bạn cập nhật chúng bằng thuật toán **lặp giá trị Q (Q-value iteration algorithm)** (xem Phương trình 18-3).

Phương trình 18-3. Thuật toán lặp giá trị Q

Công thức 18-3: Thuật toán lặp giá trị Q

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T\,(s, a, s') \left[R(s, a, s') + \gamma \cdot \max_{a'} Q_k(s', a')\right] \forall (s, a)$$

Một khi bạn có các giá trị Q tối ưu, việc xác định chính sách tối ưu, được ký hiệu là $\pi^*(s)$, là rất đơn giản: khi tác nhân ở trạng thái s , nó nên chọn hành động có giá trị Q cao nhất cho trạng thái đó:

$$\pi^*(s) = argmax_a Q^*(s, a)$$

Hãy áp dụng thuật toán này cho MDP được biểu diễn trong Hình 18-8. Đầu tiên, chúng ta cần định nghĩa MDP:

```
transition_probabilities = [  # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [  # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

Ví dụ, để biết xác suất chuyển đổi từ s_2 sang s_0 sau khi thực hiện hành động a_1 , chúng ta sẽ tra `transition_probabilities[2][1][0]` (là 0.8). Tương tự, để nhận được phần thưởng tương ứng, chúng ta sẽ tra `rewards[2][1][0]` (là $+40$). Và để nhận danh sách các hành động có thể ở s_2 , chúng ta sẽ tra `possible_actions[2]` (trong trường hợp này, chỉ có hành động a_1 là có thể). Tiếp theo, chúng ta phải khởi tạo tất cả các giá trị Q bằng không (trừ các hành động không thể, đối với chúng ta đặt giá trị Q là $-infty$):

```
Q_values = np.full((3, 3), -np.inf)  # -np.inf for impossible actions

for state, actions in enumerate(possible_actions):
    Q_values[state, actions] = 0.0  # for all possible actions
```

Bây giờ hãy chạy thuật toán lặp giá trị Q. Nó áp dụng Phương trình 18-3 lặp đi lặp lại, cho tất cả các giá trị Q, cho mọi trạng thái và mọi hành động có thể:

```
gamma = 0.90  # the discount factor

for iteration in range(50):
    Q_prev = Q_values.copy()
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                transition_probabilities[s][a][sp]
                * (rewards[s][a][sp] + gamma * Q_prev[sp].max())
                for sp in range(3)])
```

Vậy là xong! Các giá trị Q thu được trông như thế này:

```
>>> Q_values
array([[18.91891892, 17.02702702, 13.62162162],
       [ 0.        ,        -inf,  -4.87971488],
       [      -inf, 50.13365013,        -inf]])
```

Ví dụ, khi tác nhân ở trạng thái s_0 và nó chọn hành động a_1 , tổng dự kiến của các phần thưởng tương lai được chiết khấu xấp xỉ 17.0 . Đối với mỗi trạng thái, chúng ta có thể tìm hành động có giá trị Q cao nhất:

```
>>> Q_values.argmax(axis=1) # optimal action for each state
array([0, 0, 1])
```

Điều này cung cấp cho chúng ta chính sách tối ưu cho MDP này khi sử dụng hệ số chiết khấu là 0.90 : ở trạng thái s_0 chọn hành động a_0 , ở trạng thái s_1 chọn hành động a_0 (tức là giữ nguyên), và ở trạng thái s_2 chọn hành động a_1 (hành động duy nhất có thể). Thật thú vị, nếu chúng ta tăng hệ số chiết khấu lên 0.95 , chính sách tối ưu thay đổi: ở trạng thái s_1 hành động tốt nhất trở thành a_2 (đi qua lửa!). Điều này có ý nghĩa vì bạn càng coi trọng phần thưởng trong tương lai, bạn càng sẵn lòng chịu đựng một số nỗi đau hiện tại để đổi lấy lời hứa về hạnh phúc trong tương lai.

Học chênh lệch thời gian (Temporal Difference Learning)

Các bài toán học tăng cường với các hành động rời rạc thường có thể được mô hình hóa dưới dạng các quá trình quyết định Markov (MDPs), nhưng ban đầu tác nhân không biết xác suất chuyển đổi là gì (nó không biết

$T(s, a, s')$), và nó cũng không biết phần thưởng sẽ là gì ($R(s, a, s')$). Nó phải trải nghiệm mỗi trạng thái và mỗi lần chuyển đổi ít nhất một lần để biết phần thưởng, và nó phải trải nghiệm chúng nhiều lần nếu muốn có ước tính hợp lý về xác suất chuyển đổi.

Thuật toán học chênh lệch thời gian (TD) rất giống với thuật toán lặp giá trị Q, nhưng được điều chỉnh để tính đến việc tác nhân chỉ có kiến thức một phần về MDP. Nói chung, chúng ta giả định rằng tác nhân ban đầu chỉ biết các trạng thái và hành động có thể, và không có gì hơn. Tác nhân sử dụng một

chính sách khám phá (exploration policy) — ví dụ, một chính sách hoàn toàn ngẫu nhiên — để khám phá MDP, và khi nó tiến triển, thuật toán học TD cập nhật các ước tính của các giá trị trạng thái dựa trên các chuyển đổi và phần thưởng thực sự được quan sát (xem Phương trình 18-4).

Công thức 18-4: Thuật toán học TD $V_{k+1}(s)(1-)V_k(s) + \left(r + V_k(s')\right)$ hoặc tương đương:

$$V_{k+1}(s) \leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s')$$

Trong phương trình này:

- α là **tốc độ học** (ví dụ: 0.01).
- $r + \gamma \cdot V_k(s')$ được gọi là **mục tiêu TD** (TD target).
- $\delta_k(s, r, s')$ được gọi là **sai số TD** (TD error).

Một cách viết ngắn gọn hơn cho dạng đầu tiên của phương trình này là sử dụng ký hiệu $a \leftarrow_b c$, có nghĩa là $a_{k+1} \leftarrow (1 - b_k)a_k + b_k c$. Vì vậy, dòng đầu tiên của Công thức 18-4 có thể được viết lại như sau:

$$V(s) \leftarrow_\alpha r + \gamma \cdot V(s')$$

Đối với mỗi trạng thái s, thuật toán này theo dõi một giá trị trung bình chạy của các phần thưởng tức thời mà tác nhân nhận được khi rời khỏi trạng thái đó, cộng với các phần thưởng mà nó mong đợi nhận được sau này, giả sử nó hành động tối ưu.

Học Q (Q-Learning)

Tương tự, thuật toán **học Q (Q-learning)** là một sự điều chỉnh của thuật toán lặp giá trị Q cho tình huống mà xác suất chuyển đổi và phần thưởng ban đầu không xác định (xem Phương trình 18-5). Học Q hoạt động bằng cách quan sát một tác nhân chơi (ví dụ: ngẫu nhiên) và dần dần cải thiện các ước tính của các giá trị Q của nó. Khi nó có ước tính giá trị Q chính xác (hoặc đủ gần), thì chính sách tối ưu chỉ đơn giản là chọn hành động có giá trị Q cao nhất (tức là chính sách tham lam).

Phương trình 18-5. Thuật toán học Q

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left(r + \gamma \cdot \max_{a'} Q(s',a')\right)$$

Đối với mỗi cặp trạng thái-hành động

(s,a) , thuật toán này theo dõi một giá trị trung bình chạy của các phần thưởng r mà tác nhân nhận được khi rời trạng thái s với hành động a , cộng với tổng các phần thưởng tương lai được chiết khấu mà nó mong đợi nhận được. Để ước tính tổng này, chúng ta lấy giá trị tối đa của các ước tính giá trị Q cho trạng thái tiếp theo

s' , vì chúng ta giả định rằng chính sách mục tiêu sẽ hành động tối ưu từ đó trở đi.

Hãy triển khai thuật toán học Q. Đầu tiên, chúng ta sẽ cần làm cho một tác nhân khám phá môi trường. Để làm điều này, chúng ta cần một hàm

`step` để tác nhân có thể thực hiện một hành động và nhận được trạng thái và phần thưởng kết quả:

```
def step(state, action):
    probas = transition_probabilities[state][action]
    next_state = np.random.choice([0, 1, 2], p=probas)
    reward = rewards[state][action][next_state]
    return next_state, reward
```

Bây giờ hãy triển khai chính sách khám phá của tác nhân. Vì không gian trạng thái khá nhỏ, một chính sách ngẫu nhiên đơn giản sẽ đủ. Nếu chúng ta chạy thuật toán đủ lâu, tác nhân sẽ ghé thăm mọi trạng thái nhiều lần, và nó cũng sẽ thử mọi hành động có thể nhiều lần:

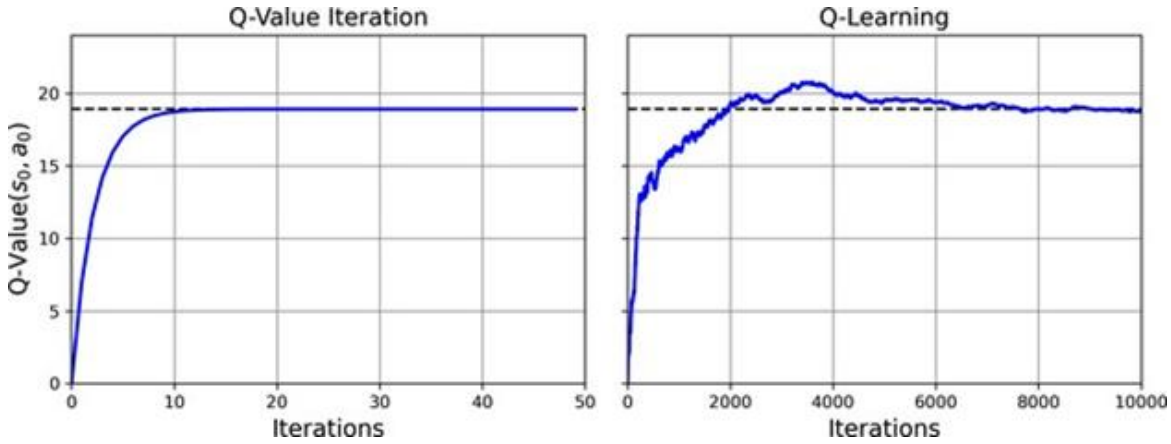
```
def exploration_policy(state):
    return np.random.choice(possible_actions[state])
```

Tiếp theo, sau khi chúng ta khởi tạo các giá trị Q giống như trước đó, chúng ta đã sẵn sàng chạy thuật toán học Q với suy giảm tốc độ học (sử dụng lập lịch lũy thừa, được giới thiệu trong Chương 11):

```
alpha0 = 0.05  # initial learning rate
decay = 0.005  # learning rate decay
gamma = 0.90  # discount factor
state = 0  # initial state

for iteration in range(10_000):
    action = exploration_policy(state)
    next_state, reward = step(state, action)
    next_value = Q_values[next_state].max()  # greedy policy at the next step
    alpha = alpha0 / (1 + iteration * decay)
    Q_values[state, action] *= 1 - alpha
    Q_values[state, action] += alpha * (reward + gamma * next_value)
    state = next_state
```

Thuật toán này sẽ hội tụ đến các giá trị Q tối ưu, nhưng nó sẽ mất nhiều lần lặp, và có thể khá nhiều lần điều chỉnh siêu tham số. Như bạn có thể thấy trong Hình 18-9, thuật toán lặp giá trị Q (trái) hội tụ rất nhanh, trong vòng chưa đầy 20 lần lặp, trong khi thuật toán học Q (phải) mất khoảng 8.000 lần lặp để hội tụ. Rõ ràng, việc không biết xác suất chuyển đổi hoặc phần thưởng làm cho việc tìm chính sách tối ưu khó hơn đáng kể!



Hình 18-9. Đường cong học của thuật toán lặp giá trị Q so với thuật toán học Q

Thuật toán học Q được gọi là

thuật toán off-policy (off-policy algorithm) vì chính sách đang được huấn luyện không nhất thiết là chính sách được sử dụng trong quá trình huấn luyện. Ví dụ, trong mã chúng ta vừa chạy, chính sách được thực thi (chính sách khám phá) hoàn toàn ngẫu nhiên, trong khi chính sách đang được huấn luyện không bao giờ được sử dụng. Sau khi huấn luyện, chính sách tối ưu tương ứng với việc chọn hành động có giá trị Q cao nhất một cách có hệ thống. Ngược lại, thuật toán độ dốc chính sách là một

thuật toán on-policy (on-policy algorithm): nó khám phá thế giới bằng cách sử dụng chính sách đang được huấn luyện. Điều hơi đáng ngạc nhiên là học Q có khả năng học chính sách tối ưu chỉ bằng cách quan sát một tác nhân hành động ngẫu nhiên. Hãy tưởng

tượng việc học chơi gôn khi giáo viên của bạn là một con khỉ bị bịt mắt. Chúng ta có thể làm tốt hơn không?

Các chính sách khám phá (Exploration Policies)

Tất nhiên, học Q chỉ có thể hoạt động nếu chính sách khám phá khám phá MDP đủ kỹ lưỡng. Mặc dù một chính sách hoàn toàn ngẫu nhiên được đảm bảo cuối cùng sẽ ghé thăm mọi trạng thái và mọi lần chuyển đổi nhiều lần, nhưng nó có thể mất một thời gian cực kỳ dài để làm được điều đó. Do đó, một lựa chọn tốt hơn là sử dụng

chính sách $epsilon$ **-tham lam (** $epsilon$ **-greedy policy)** ($epsilon$ là epsilon): ở mỗi bước, nó hành động ngẫu nhiên với xác suất $epsilon$, hoặc tham lam với xác suất $1-epsilon$ (tức là chọn hành động có giá trị Q cao nhất). Ưu điểm của chính sách

$epsilon$ -tham lam (so với một chính sách hoàn toàn ngẫu nhiên) là nó sẽ dành ngày càng nhiều thời gian để khám phá các phần thú vị của môi trường, khi các ước tính giá trị Q ngày càng tốt hơn, trong khi vẫn dành một chút thời gian để ghé thăm các vùng không xác định của MDP. Khá phổ biến khi bắt đầu với một giá trị

$epsilon$ cao (ví dụ: 1.0) và sau đó giảm dần nó (ví dụ: xuống 0.05).

Ngoài ra, thay vì chỉ dựa vào cơ hội để khám phá, một cách tiếp cận khác là khuyến khích chính sách khám phá thử các hành động mà nó chưa thử nhiều trước đây. Điều này có thể được triển khai dưới dạng một phần thưởng được thêm vào các ước tính giá trị Q, như thể hiện trong Phương trình 18-6.

Công thức 18-6: Q-learning sử dụng hàm thăm dò

$$Q(s,a) \leftarrow_{\alpha} r + \gamma \cdot \max_{a'} f\big(Q(s',a'), N(s',a')\big)$$

Trong phương trình này:

- $N(s',a')$ đếm số lần hành động a' đã được chọn trong trạng thái s' .
- $f(Q,N)$ là một **hàm thăm dò**, ví dụ như $f(Q,N) = Q + \frac{\kappa}{1+N}$, trong đó κ là một **siêu tham số tò mò** (curiosity hyperparameter) đo lường mức độ tác nhân bị thu hút bởi những điều chưa biết.

Học Q xấp xỉ và Học Q sâu (Approximate Q-Learning and Deep Q-Learning)

Vấn đề chính với học Q là nó không mở rộng tốt cho các MDP lớn (hoặc thậm chí trung bình) với nhiều trạng thái và hành động. Ví dụ, giả sử bạn muốn sử dụng học Q để huấn luyện một tác nhân chơi Ms. Pac-Man (xem Hình 18-1). Có khoảng 150 viên thức ăn mà Ms. Pac-Man có thể ăn, mỗi viên có thể có hoặc không có (tức là đã ăn). Vì vậy, số lượng trạng thái có thể lớn hơn

$2^{150} approx 10^{45}$. Và nếu bạn thêm tất cả các tổ hợp vị trí có thể cho tất cả các bóng ma và Ms. Pac-Man, số lượng trạng thái có thể trở nên lớn hơn số nguyên tử trên hành tinh của

chúng ta, vì vậy hoàn toàn không có cách nào bạn có thể theo dõi ước tính cho từng giá trị Q riêng lẻ.

Giải pháp là tìm một hàm

$Q_theta(s, a)$ xấp xỉ giá trị Q của bất kỳ cặp trạng thái-hành động (s, a) nào bằng cách sử dụng một số lượng tham số có thể quản lý được (được cho bởi vector tham số $theta$). Đây được gọi là

học Q xấp xỉ (approximate Q-learning). Trong nhiều năm, người ta khuyến nghị sử dụng các kết hợp tuyến tính của các đặc trưng được tạo thủ công được trích xuất từ trạng thái (ví dụ: khoảng cách của các bóng ma gần nhất, hướng của chúng, v.v.) để ước tính giá trị Q, nhưng vào năm 2013, DeepMind đã chỉ ra rằng việc sử dụng mạng nơ-ron sâu có thể hoạt động tốt hơn nhiều, đặc biệt đối với các vấn đề phức tạp, và nó không yêu cầu bất kỳ kỹ thuật đặc trưng nào. Một DNN được sử dụng để ước tính giá trị Q được gọi là

mạng Q sâu (deep Q-network - DQN), và việc sử dụng DQN cho học Q xấp xỉ được gọi là **học Q sâu (deep Q-learning)**.

Bây giờ, làm thế nào chúng ta có thể huấn luyện một DQN? Chà, hãy xem xét giá trị Q xấp xỉ được tính toán bởi DQN cho một cặp trạng thái-hành động

(s, a) nhất định. Nhờ Bellman, chúng ta biết rằng chúng ta muốn giá trị Q xấp xỉ này càng gần càng tốt với phần thưởng

r mà chúng ta thực sự quan sát được sau khi thực hiện hành động a ở trạng thái s , cộng với giá trị được chiết khấu của việc chơi tối ưu từ đó trở đi. Để ước tính tổng các phần thưởng tương lai được chiết khấu này, chúng ta có thể chỉ cần thực thi DQN trên trạng thái tiếp theo

s' , cho tất cả các hành động có thể a' . Chúng ta nhận được một giá trị Q tương lai xấp xỉ cho mỗi hành động có thể. Sau đó, chúng ta chọn giá trị cao nhất (vì chúng ta giả định rằng chúng ta sẽ chơi tối ưu) và chiết khấu nó, và điều này cung cấp cho chúng ta một ước tính tổng các phần thưởng tương lai được chiết khấu. Bằng cách cộng phần thưởng

r và ước tính giá trị được chiết khấu trong tương lai, chúng ta nhận được **giá trị Q mục tiêu (target Q-value)** $y(s, a)$ cho cặp trạng thái-hành động (s, a) , như thể hiện trong Phương trình 18-7.

Phương trình 18-7. Giá trị Q mục tiêu

$$y(s, a) = r + \gamma \cdot \max_{a'} Q_\theta(s', a')$$

Với giá trị Q mục tiêu này, chúng ta có thể chạy một bước huấn luyện bằng cách sử dụng bất kỳ thuật toán giảm độ dốc nào. Cụ thể, chúng ta thường cố gắng giảm thiểu sai số bình phương giữa giá trị Q ước tính

$Q_theta(s, a)$ và giá trị Q mục tiêu $y(s, a)$, hoặc hàm Huber để giảm độ nhạy của thuật toán đối với các sai số lớn. Và đó là thuật toán học Q sâu! Hãy xem cách triển khai nó để giải quyết môi trường CartPole.

Triển khai học Q sâu

Điều đầu tiên chúng ta cần là một mạng Q sâu. Về lý thuyết, chúng ta cần một mạng nơ-ron nhận một cặp trạng thái-hành động làm đầu vào, và xuất ra một giá trị Q xấp xỉ. Tuy nhiên, trên thực tế, hiệu quả hơn nhiều khi sử dụng một mạng nơ-ron chỉ nhận một trạng thái làm đầu vào, và xuất ra một giá trị Q xấp xỉ cho mỗi hành động có thể. Để giải quyết môi trường CartPole, chúng ta không cần một mạng nơ-ron quá phức tạp; một vài lớp ẩn là đủ:

```
input_shape = [4]  # == env.observation_space.shape
n_outputs = 2  # == env.action_space.n

model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="elu", input_shape=input_shape),
    tf.keras.layers.Dense(32, activation="elu"),
    tf.keras.layers.Dense(n_outputs)
])
```

Để chọn một hành động bằng cách sử dụng DQN này, chúng ta chọn hành động có giá trị Q dự đoán lớn nhất. Để đảm bảo rằng tác nhân khám phá môi trường, chúng ta sẽ sử dụng chính sách

$epsilon$ -tham lam (tức là, chúng ta sẽ chọn một hành động ngẫu nhiên với xác suất $epsilon$):

```
def epsilon_greedy_policy(state, epsilon=0):
    if np.random.rand() < epsilon:
        return np.random.randint(n_outputs)  # random action
    Q_values = model.predict(state[np.newaxis], verbose=0)[0]
    return Q_values.argmax()  # optimal action according to the DQN
```

Thay vì huấn luyện DQN chỉ dựa trên các kinh nghiệm gần đây nhất, chúng ta sẽ lưu trữ tất cả các kinh nghiệm trong một **bộ đệm phát lại (replay buffer)** (hoặc bộ nhớ phát lại), và chúng ta sẽ lấy mẫu một lô huấn luyện ngẫu nhiên từ đó ở mỗi lần lặp huấn luyện. Điều này giúp giảm các mối tương quan giữa các kinh nghiệm trong một lô huấn luyện, điều này hỗ trợ rất nhiều cho việc huấn luyện. Để làm điều này, chúng ta sẽ chỉ sử dụng một hàng đợi hai đầu (deque):

```
from collections import deque

replay_buffer = deque(maxlen=2000)
```

Mỗi kinh nghiệm sẽ bao gồm sáu yếu tố: một trạng thái

s , hành động a mà tác nhân đã thực hiện, phần thưởng r thu được, trạng thái tiếp theo s' mà nó đạt được, một Boolean chỉ ra liệu tập phim đã kết thúc tại thời điểm đó (done), và

cuối cùng là một Boolean khác chỉ ra liệu tập phim có bị cắt cụt tại thời điểm đó hay không. Chúng ta sẽ cần một hàm nhỏ để lấy mẫu một lô kinh nghiệm ngẫu nhiên từ bộ đệm phát lại. Nó sẽ trả về sáu mảng NumPy tương ứng với sáu yếu tố kinh nghiệm:

```
def sample_experiences(batch_size):
    indices = np.random.randint(len(replay_buffer), size=batch_size)
    batch = [replay_buffer[index] for index in indices]
    return [
        np.array([experience[field_index]
                  for experience in batch])
        for field_index in range(6)
    ]  # [states, actions, rewards, next_states, dones, truncateds]
```

Hãy tạo thêm một hàm sẽ chơi một bước duy nhất bằng cách sử dụng chính sách

$epsilon$ -tham lam, sau đó lưu trữ kinh nghiệm thu được vào bộ đệm phát lại:

```
def play_one_step(env, state, epsilon):
    action = epsilon_greedy_policy(state, epsilon)
    next_state, reward, done, truncated, info = env.step(action)
    replay_buffer.append((state, action, reward, next_state, done,
truncated))
    return next_state, reward, done, truncated, info
```

Cuối cùng, hãy tạo một hàm cuối cùng sẽ lấy mẫu một lô kinh nghiệm từ bộ đệm phát lại và huấn luyện DQN bằng cách thực hiện một bước giảm độ dốc duy nhất trên lô này:

```
batch_size = 32
discount_factor = 0.95

optimizer = tf.keras.optimizers.Nadam(learning_rate=1e-2)
loss_fn = tf.keras.losses.mean_squared_error

def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0)
    max_next_Q_values = next_Q_values.max(axis=1)
    runs = 1.0 - (dones | truncateds)  # episode is not done or truncated
    target_Q_values = rewards + runs * discount_factor * max_next_Q_values
    target_Q_values = target_Q_values.reshape(-1, 1)

    mask = tf.one_hot(actions, n_outputs)
    with tf.GradientTape() as tape:
        all_Q_values = model(states)
        Q_values = tf.reduce_sum(all_Q_values * mask, axis=1, keepdims=True)
        loss = tf.reduce_mean(loss_fn(target_Q_values, Q_values))

    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

Đây là những gì đang diễn ra trong mã này:

- Đầu tiên, chúng ta định nghĩa một số siêu tham số, và chúng ta tạo trình tối ưu hóa và hàm tổn thất.
- Sau đó, chúng ta tạo hàm `training_step()`. Nó bắt đầu bằng cách lấy mẫu một lô kinh nghiệm, sau đó nó sử dụng DQN để dự đoán giá trị Q cho mỗi hành động có thể trong trạng thái tiếp theo của mỗi kinh nghiệm. Vì chúng ta giả định rằng tác nhân sẽ chơi tối ưu, chúng ta chỉ giữ giá trị Q tối đa cho mỗi trạng thái tiếp theo. Tiếp theo, chúng ta sử dụng Phương trình 18-7 để tính toán giá trị Q mục tiêu cho cặp trạng thái-hành động của mỗi kinh nghiệm.
- Chúng ta muốn sử dụng DQN để tính toán giá trị Q cho mỗi cặp trạng thái-hành động đã trải nghiệm, nhưng DQN cũng sẽ xuất ra các giá trị Q cho các hành động khả thi khác, không chỉ cho hành động thực sự được tác nhân chọn. Vì vậy, chúng ta cần che đi tất cả các giá trị Q mà chúng ta không cần. Hàm `tf.one_hot()` giúp chuyển đổi một mảng các chỉ số hành động thành một mặt nạ như vậy. Ví dụ, nếu ba kinh nghiệm đầu tiên chứa các hành động $1,1,0$ tương ứng, thì mặt nạ sẽ bắt đầu bằng $\left[[0,1],[0,1],[1,0],\ldots\right]$. Sau đó chúng ta có thể nhân đầu ra của DQN với mặt nạ này, và điều này sẽ làm cho tất cả các giá trị Q mà chúng ta không muốn trở thành 0. Sau đó chúng ta tính tổng theo trục 1 để loại bỏ tất cả các số không, chỉ giữ lại các giá trị Q của các cặp trạng thái-hành động đã trải nghiệm. Điều này cung cấp cho chúng ta tensor `Q_values`, chứa một giá trị Q được dự đoán cho mỗi kinh nghiệm trong lô.
- Tiếp theo, chúng ta tính toán tổn thất: đó là sai số bình phương trung bình giữa các giá trị Q mục tiêu và dự đoán cho các cặp trạng thái-hành động đã trải nghiệm.
- Cuối cùng, chúng ta thực hiện một bước giảm độ dốc để giảm thiểu tổn thất đối với các biến có thể huấn luyện của mô hình.

Đây là phần khó nhất. Bây giờ việc huấn luyện mô hình rất đơn giản:

```
for episode in range(600):
    obs, info = env.reset()
    for step in range(200):
        epsilon = max(1 - episode / 500, 0.01)

        obs, reward, done, truncated, info = play_one_step(env, obs, epsilon)
        if done or truncated:
            break

    if episode > 50:
        training_step(batch_size)
```

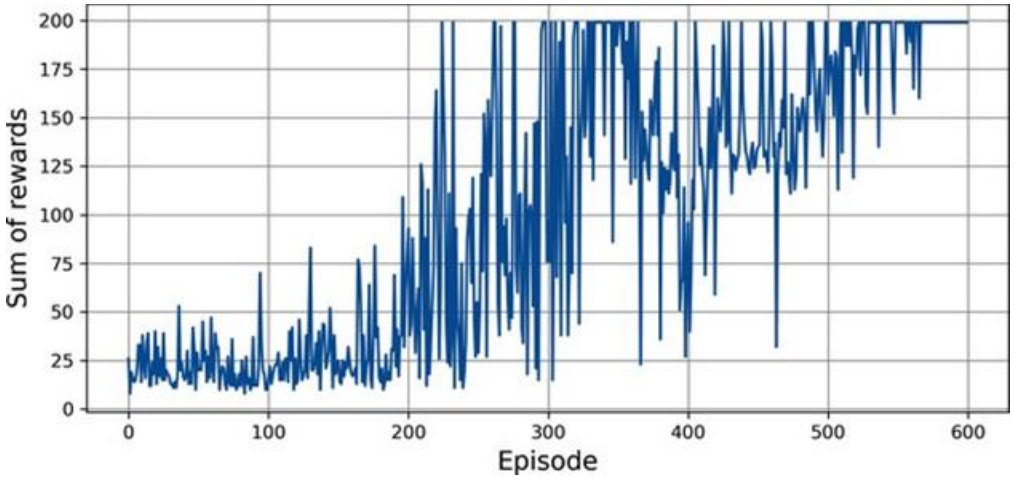
Chúng ta chạy 600 tập, mỗi tập tối đa 200 bước. Ở mỗi bước, chúng ta đầu tiên tính toán giá trị epsilon cho chính sách

$epsilon$ -tham lam: nó sẽ đi từ 1 xuống 0.01, tuyến tính, trong vòng chưa đầy 500 tập. Sau đó chúng ta gọi hàm

`play_one_step()`, hàm này sẽ sử dụng chính sách $epsilon$ -tham lam để chọn một hành động, sau đó thực hiện nó và ghi lại kinh nghiệm vào bộ đệm phát lại. Nếu tập phim đã hoàn thành hoặc bị cắt cụt, chúng ta thoát khỏi vòng lặp. Cuối cùng, nếu chúng ta đã qua tập 50, chúng ta gọi hàm

`training_step()` để huấn luyện mô hình trên một lô được lấy mẫu từ bộ đệm phát lại. Lý do chúng ta chơi nhiều tập mà không huấn luyện là để cho bộ đệm phát lại có thời gian để lấp đầy (nếu chúng ta không đợi đủ, sẽ không có đủ sự đa dạng trong bộ đệm phát lại). Và đó là nó: chúng ta vừa triển khai thuật toán học Q sâu!

Hình 18-10 cho thấy tổng phần thưởng mà tác nhân nhận được trong mỗi tập.



Hình 18-10. Đường cong học của thuật toán học Q sâu

Như bạn có thể thấy, thuật toán mất một thời gian để bắt đầu học bất cứ điều gì, một phần vì

$epsilon$ rất cao ở ban đầu. Sau đó, tiến độ của nó thất thường: nó lần đầu tiên đạt phần thưởng tối đa vào khoảng tập 220, nhưng ngay lập tức giảm, sau đó nảy lên nảy xuống một vài lần, và ngay sau đó có vẻ như nó đã ổn định gần phần thưởng tối đa, vào khoảng tập 320, điểm số của nó lại giảm mạnh. Điều này được gọi là **quên thảm khốc (catastrophic forgetting)**, và đó là một trong những vấn đề lớn mà hầu hết các thuật toán RL đều gặp phải: khi tác nhân khám phá môi trường, nó cập nhật chính sách của mình, nhưng những gì nó học được ở một phần của môi trường có thể làm hỏng những gì nó đã học trước đó ở các phần khác của môi trường. Các kinh nghiệm khá tương quan, và môi trường học tập liên tục thay đổi — điều này không lý tưởng cho giảm độ dốc! Nếu bạn tăng kích thước của bộ đệm phát lại, thuật toán sẽ ít bị vấn đề này hơn. Điều chỉnh tốc độ học cũng có thể giúp ích. Nhưng sự thật là, học tăng cường rất khó: việc huấn luyện thường không ổn định, và bạn có thể cần thử nhiều giá trị siêu tham số và hạt giống ngẫu nhiên trước khi tìm thấy một sự kết hợp hoạt động tốt. Ví dụ, nếu bạn thử thay đổi hàm kích hoạt từ “elu” sang “relu”, hiệu suất sẽ thấp hơn nhiều.

Bạn có thể tự hỏi tại sao chúng ta không vẽ đồ thị tổn thất. Hóa ra tổn thất là một chỉ số kém về hiệu suất của mô hình. Tổn thất có thể giảm, nhưng tác nhân có thể hoạt động tệ hơn (ví dụ: điều này có thể xảy ra khi tác nhân bị kẹt trong một vùng nhỏ của môi trường, và DQN bắt đầu overfitting vùng này). Ngược lại, tổn thất có thể tăng, nhưng tác nhân có thể hoạt động tốt hơn (ví dụ: nếu DQN đang đánh giá thấp các giá trị Q và nó bắt đầu tăng dự đoán của mình một cách chính xác, tác nhân có thể sẽ hoạt động tốt hơn, nhận được nhiều phần thưởng hơn, nhưng tổn thất có thể tăng vì DQN cũng đặt các mục tiêu, vốn cũng sẽ lớn hơn). Vì vậy, tốt hơn là vẽ đồ thị phần thưởng.

Thuật toán học Q sâu cơ bản mà chúng ta đã sử dụng cho đến nay sẽ quá không ổn định để học cách chơi các trò chơi Atari. Vậy DeepMind đã làm điều đó như thế nào? Chà, họ đã điều chỉnh thuật toán!

Các biến thể của học Q sâu (Deep Q-Learning Variants)

Hãy cùng xem xét một vài biến thể của thuật toán học Q sâu có thể ổn định và tăng tốc quá trình huấn luyện.

Mục tiêu giá trị Q cố định (Fixed Q-value Targets)

Trong thuật toán học Q sâu cơ bản, mô hình được sử dụng cả để đưa ra dự đoán và để thiết lập các mục tiêu của riêng nó. Điều này có thể dẫn đến một tình huống tương tự như một con chó đuổi theo cái đuôi của chính nó. Vòng lặp phản hồi này có thể làm cho mạng không ổn định: nó có thể phân kỳ, dao động, đóng băng, v.v. Để giải quyết vấn đề này, trong bài báo năm 2013 của họ, các nhà nghiên cứu của DeepMind đã sử dụng hai DQN thay vì một: một là **mô hình trực tuyến (online model)**, học ở mỗi bước và được sử dụng để di chuyển tác nhân, và một là **mô hình mục tiêu (target model)** chỉ được sử dụng để định nghĩa các mục tiêu. Mô hình mục tiêu chỉ là một bản sao của mô hình trực tuyến:

```
target = tf.keras.models.clone_model(model) # clone the model's architecture
target.set_weights(model.get_weights()) # copy the weights
```

Sau đó, trong hàm `training_step()`, chúng ta chỉ cần thay đổi một dòng để sử dụng mô hình mục tiêu thay vì mô hình trực tuyến khi tính toán các giá trị Q của các trạng thái tiếp theo:

```
next_Q_values = target.predict(next_states, verbose=0)
```

Cuối cùng, trong vòng lặp huấn luyện, chúng ta phải sao chép các trọng số của mô hình trực tuyến sang mô hình mục tiêu, theo các khoảng thời gian đều đặn (ví dụ: cứ sau 50 tập):

```
if episode % 50 == 0:
    target.set_weights(model.get_weights())
```

Vì mô hình mục tiêu được cập nhật ít thường xuyên hơn nhiều so với mô hình trực tuyến, các mục tiêu giá trị Q ổn định hơn, vòng lặp phản hồi mà chúng ta đã thảo luận trước đó được làm suy yếu, và các tác động của nó ít nghiêm trọng hơn. Cách tiếp cận này là một trong những đóng góp chính của các nhà nghiên cứu DeepMind trong bài báo năm 2013

của họ, cho phép các tác nhân học chơi các trò chơi Atari từ các pixel thô. Để ổn định quá trình huấn luyện, họ đã sử dụng tốc độ học rất nhỏ là

0.00025 , họ chỉ cập nhật mô hình mục tiêu sau mỗi 10.000 bước (thay vì 50), và họ đã sử dụng một bộ đệm phát lại rất lớn gồm 1 triệu kinh nghiệm. Họ giảm epsilon rất chậm, từ 1 xuống 0.1 trong 1 triệu bước, và họ để thuật toán chạy trong 50 triệu bước. Hơn nữa, DQN của họ là một mạng tích chập sâu. Bây giờ hãy cùng xem xét một biến thể DQN khác đã đánh bại hiệu suất tốt nhất một lần nữa.

DQN kép (Double DQN)

Trong một bài báo năm 2015, các nhà nghiên cứu của DeepMind đã điều chỉnh thuật toán DQN của họ, tăng hiệu suất và phần nào ổn định quá trình huấn luyện. Họ gọi biến thể này là **DQN kép (double DQN)**. Bản cập nhật này dựa trên quan sát rằng mạng mục tiêu có xu hướng đánh giá quá cao các giá trị Q. Thật vậy, giả sử tất cả các hành động đều tốt như nhau: các giá trị Q được ước tính bởi mô hình mục tiêu phải giống hệt nhau, nhưng vì chúng là các xấp xỉ, một số có thể lớn hơn một chút so với những cái khác, hoàn toàn do ngẫu nhiên. Mô hình mục tiêu sẽ luôn chọn giá trị Q lớn nhất, giá trị này sẽ lớn hơn một chút so với giá trị Q trung bình, rất có thể đánh giá quá cao giá trị Q thực (một chút giống như đếm chiều cao của con sóng ngẫu nhiên cao nhất khi đo độ sâu của một cái hồ bơi). Để khắc phục điều này, các nhà nghiên cứu đã đề xuất sử dụng mô hình trực tuyến thay vì mô hình mục tiêu khi chọn các hành động tốt nhất cho các trạng thái tiếp theo, và chỉ sử dụng mô hình mục tiêu để ước tính các giá trị Q cho các hành động tốt nhất này. Đây là hàm `training_step()` đã được cập nhật:

```
def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0)  # ≠
target.predict()
    best_next_actions = next_Q_values.argmax(axis=1)
    next_mask = tf.one_hot(best_next_actions, n_outputs).numpy()
    max_next_Q_values = (target.predict(next_states, verbose=0) *
next_mask).sum(axis=1)
    [...]  # the rest is the same as earlier
```

Chỉ vài tháng sau, một cải tiến khác cho thuật toán DQN đã được đề xuất; chúng ta sẽ xem xét nó tiếp theo.

Phát lại kinh nghiệm ưu tiên (Prioritized Experience Replay)

Thay vì lấy mẫu kinh nghiệm đồng đều từ bộ đệm phát lại, tại sao không lấy mẫu các kinh nghiệm quan trọng thường xuyên hơn? Ý tưởng này được gọi là **lấy mẫu quan trọng (importance sampling - IS)** hoặc **phát lại kinh nghiệm ưu tiên (prioritized experience replay - PER)**, và nó được giới thiệu trong một bài báo năm 2015 bởi các nhà nghiên cứu của DeepMind (một lần nữa!).

Cụ thể hơn, các kinh nghiệm được coi là "quan trọng" nếu chúng có khả năng dẫn đến tiến bộ học nhanh chóng. Nhưng làm thế nào chúng ta có thể ước tính điều này? Một cách tiếp cận hợp lý là đo độ lớn của sai số TD $delta = r + gammacdotV(s') - V(s)$. Sai số TD lớn cho thấy rằng một chuyển đổi (s, a, s') rất bất ngờ, và do đó có lẽ đáng học hỏi từ. Khi một kinh nghiệm được ghi vào bộ đệm phát lại, ưu tiên của nó được đặt thành một giá trị rất lớn, để đảm bảo rằng nó được lấy mẫu ít nhất một lần. Tuy nhiên, một khi nó được lấy mẫu (và mỗi khi nó được lấy mẫu), sai số TD $delta$ được tính toán, và ưu tiên của kinh nghiệm này được đặt thành $p =| delta |$ (cộng với một hằng số nhỏ để đảm bảo rằng mọi kinh nghiệm đều có xác suất được lấy mẫu khác không). Xác suất P lấy mẫu một kinh nghiệm với ưu tiên p tỷ lệ thuận với p^{zeta} , trong đó $zeta$ là một siêu tham số kiểm soát mức độ tham lam mà chúng ta muốn lấy mẫu quan trọng: khi $zeta = 0$, chúng ta chỉ nhận được lấy mẫu đồng đều, và khi $zeta = 1$, chúng ta nhận được lấy mẫu quan trọng hoàn chỉnh. Trong bài báo, các tác giả đã sử dụng $zeta = 0.6$, nhưng giá trị tối ưu sẽ phụ thuộc vào nhiệm vụ.

Tuy nhiên, có một điều đáng chú ý: vì các mẫu sẽ bị thiên vị theo hướng các kinh nghiệm quan trọng, chúng ta phải bù đắp cho sự thiên vị này trong quá trình huấn luyện bằng cách giảm trọng số các kinh nghiệm theo mức độ quan trọng của chúng, nếu không mô hình sẽ chỉ overfitting các kinh nghiệm quan trọng. Nói rõ hơn, chúng ta muốn các kinh nghiệm quan trọng được lấy mẫu thường xuyên hơn, nhưng điều này cũng có nghĩa là chúng ta phải cho chúng một trọng số thấp hơn trong quá trình huấn luyện. Để làm điều này, chúng ta định nghĩa trọng số huấn luyện của mỗi kinh nghiệm là $w = (nP)^{-beta}$, trong đó n là số lượng kinh nghiệm trong bộ đệm phát lại, và $beta$ là một siêu tham số kiểm soát mức độ chúng ta muốn bù đắp cho sự thiên vị lấy mẫu quan trọng (0 có nghĩa là không bù đắp gì cả, trong khi 1 có nghĩa là bù đắp hoàn toàn). Trong bài báo, các tác giả đã sử dụng

$beta = 0.4$ ở đầu quá trình huấn luyện và tăng tuyến tính nó lên $beta = 1$ vào cuối quá trình huấn luyện. Một lần nữa, giá trị tối ưu sẽ phụ thuộc vào nhiệm vụ, nhưng nếu bạn tăng một cái, bạn thường sẽ muốn tăng cái kia. Bây giờ hãy cùng xem xét một biến thể quan trọng cuối cùng của thuật toán DQN.

DQN đấu (Dueling DQN)

Thuật toán **DQN đấu (dueling DQN - DDQN)** (không nên nhầm lẫn với DQN kép, mặc dù cả hai kỹ thuật này có thể dễ dàng kết hợp) được giới thiệu trong một bài báo khác vào năm 2015 bởi các nhà nghiên cứu của DeepMind. Để hiểu cách nó hoạt động, trước tiên chúng ta phải lưu ý rằng giá trị Q của một cặp trạng thái-hành động (s, a) có thể được biểu thị là $Q(s, a) = V(s) + A(s, a)$, trong đó $V(s)$ là giá trị của trạng thái s và $A(s, a)$ là **lợi thế (advantage)** của việc thực hiện hành động a ở trạng thái s , so với tất cả các hành động có thể khác trong trạng thái đó. Hơn nữa, giá trị của một trạng thái bằng giá trị Q của hành động tốt nhất a^* cho trạng thái đó (vì chúng ta giả định chính sách tối ưu sẽ chọn hành động tốt nhất), do đó $V(s) = Q(s, a^*)$, điều này ngụ ý rằng $A(s, a^*) = 0$. Trong một DQN đấu, mô hình ước tính cả giá trị của trạng thái và lợi thế của mỗi hành động có thể. Vì hành động tốt nhất nên có lợi thế bằng 0 , mô hình trừ đi giá trị lợi thế dự đoán tối đa từ tất cả các lợi thế dự đoán. Đây là một mô hình DDQN đơn giản, được triển khai bằng API chức năng:

```
input_states = tf.keras.layers.Input(shape=[4])
hidden1 = tf.keras.layers.Dense(32, activation="elu")(input_states)
hidden2 = tf.keras.layers.Dense(32, activation="elu")(hidden1)
state_values = tf.keras.layers.Dense(1)(hidden2)
raw_advantages = tf.keras.layers.Dense(n_outputs)(hidden2)
advantages = raw_advantages - tf.reduce_max(raw_advantages, axis=1,
keepdims=True)
Q_values = state_values + advantages
model = tf.keras.Model(inputs=[input_states], outputs=[Q_values])
```

Phần còn lại của thuật toán giống như trước đây. Trên thực tế, bạn có thể xây dựng một DQN kép đấu và kết hợp nó với phát lại kinh nghiệm ưu tiên! Tổng quát hơn, nhiều kỹ thuật RL có thể được kết hợp, như DeepMind đã chứng minh trong một bài báo năm 2017: các tác giả của bài báo đã kết hợp sáu kỹ thuật khác nhau thành một tác nhân có tên là Rainbow, vượt trội hơn rất nhiều so với hiệu suất tốt nhất hiện có.

Như bạn có thể thấy, học tăng cường sâu là một lĩnh vực phát triển nhanh chóng và còn nhiều điều để khám phá!

Tổng quan về một số thuật toán RL phổ biến

Trước khi chúng ta kết thúc chương này, hãy xem xét nhanh một vài thuật toán phổ biến khác:

AlphaGo

AlphaGo sử dụng một biến thể của tìm kiếm cây Monte Carlo (MCTS) dựa trên mạng nơ-ron sâu để đánh bại các nhà vô địch cờ vây. MCTS được Nicholas Metropolis và Stanislaw Ulam phát minh vào năm 1949. Nó chọn nước đi tốt nhất sau khi chạy nhiều mô phỏng, liên tục khám phá cây tìm kiếm bắt đầu từ vị trí hiện tại và dành nhiều thời gian hơn cho các nhánh hứa hẹn nhất. Khi nó đạt đến một nút mà nó chưa từng truy cập trước đây, nó chơi ngẫu nhiên cho đến khi trò chơi kết thúc, và cập nhật các ước tính cho mỗi nút đã truy cập (trừ các nước đi ngẫu nhiên), tăng hoặc giảm mỗi ước tính tùy thuộc vào kết quả cuối cùng. AlphaGo dựa trên cùng một nguyên tắc, nhưng nó sử dụng mạng chính sách để chọn nước đi, thay vì chơi ngẫu nhiên. Mạng chính sách này được huấn luyện bằng cách sử dụng độ dốc chính sách. Thuật toán ban đầu bao gồm ba mạng nơ-ron nữa, và phức tạp hơn, nhưng nó đã được đơn giản hóa trong bài báo AlphaGo Zero, sử dụng một mạng nơ-ron duy nhất để cả chọn nước đi và đánh giá trạng thái trò chơi. Bài báo AlphaZero đã khái quát hóa thuật toán này, làm cho nó có khả năng giải quyết không chỉ trò chơi cờ vây, mà còn cờ vua và shogi (cờ tướng Nhật Bản). Cuối cùng, bài báo MuZero tiếp tục cải thiện thuật toán này, vượt trội hơn các lần lặp trước đó mặc dù tác nhân bắt đầu mà thậm chí không biết luật chơi!

Các thuật toán Actor-critic Actor-critics là một nhóm các thuật toán RL kết hợp độ dốc chính sách với mạng Q sâu. Một tác nhân actor-critic chứa hai mạng nơ-ron: một mạng chính sách và một DQN. DQN được huấn luyện bình thường, bằng cách học từ kinh nghiệm của tác nhân. Mạng chính sách học khác (và nhanh hơn nhiều) so với PG thông thường:

thay vì ước tính giá trị của mỗi hành động bằng cách trải qua nhiều tập, sau đó tính tổng các phần thưởng tương lai được chiết khấu cho mỗi hành động, và cuối cùng chuẩn hóa chúng, tác nhân (actor) dựa vào các giá trị hành động được ước tính bởi DQN (critic). Điều này hơi giống một vận động viên (tác nhân) học với sự giúp đỡ của một huấn luyện viên (DQN).

Actor-critic lợi thế không đồng bộ (Asynchronous advantage actor-critic - A3C)

Đây là một biến thể actor-critic quan trọng được các nhà nghiên cứu DeepMind giới thiệu vào năm 2016, trong đó nhiều tác nhân học song song, khám phá các bản sao khác nhau của môi trường. Theo các khoảng thời gian đều đặn, nhưng không đồng bộ (do đó có tên), mỗi tác nhân đẩy một số cập nhật trọng số lên mạng chính, sau đó nó kéo các trọng số mới nhất từ mạng đó. Mỗi tác nhân do đó đóng góp vào việc cải thiện mạng chính và hưởng lợi từ những gì các tác nhân khác đã học. Hơn nữa, thay vì ước tính các giá trị Q, DQN ước tính lợi thế của mỗi hành động (do đó có chữ A thứ hai trong tên), điều này ổn định quá trình huấn luyện.

Actor-critic lợi thế đồng bộ (Advantage actor-critic - A2C) A2C là một biến thể của thuật toán A3C loại bỏ tính không đồng bộ. Tất cả các cập nhật mô hình đều đồng bộ, vì vậy các cập nhật độ dốc được thực hiện trên các lô lớn hơn, cho phép mô hình tận dụng tốt hơn sức mạnh của GPU.

Actor-critic mềm (Soft actor-critic - SAC)

SAC là một biến thể actor-critic được Tuomas Haarnoja và các nhà nghiên cứu UC Berkeley khác đề xuất vào năm 2018. Nó học không chỉ phần thưởng, mà còn để tối đa hóa entropy của các hành động của nó. Nói cách khác, nó cố gắng không thể đoán trước càng nhiều càng tốt trong khi vẫn nhận được nhiều phần thưởng nhất có thể. Điều này khuyến khích tác nhân khám phá môi trường, điều này tăng tốc quá trình huấn luyện, và làm cho nó ít có khả năng lặp lại cùng một hành động khi DQN tạo ra các ước tính không hoàn hảo. Thuật toán này đã chứng minh hiệu quả mẫu đáng kinh ngạc (ngược lại với tất cả các thuật toán trước đó, học rất chậm).

Tối ưu hóa chính sách cận biên (Proximal policy optimization - PPO)

Thuật toán này của John Schulman và các nhà nghiên cứu OpenAI khác dựa trên A2C, nhưng nó cắt xén hàm tổn thất để tránh các cập nhật trọng số quá lớn (thường dẫn đến sự mất ổn định trong huấn luyện). PPO là một sự đơn giản hóa của thuật toán tối ưu hóa chính sách vùng tin cậy (TRPO) trước đó, cũng của OpenAI. OpenAI đã gây chú ý vào tháng 4 năm 2019 với AI của họ có tên OpenAI Five, dựa trên thuật toán PPO, đã đánh bại các nhà vô địch thế giới trong trò chơi nhiều người chơi Dota 2.

Khám phá dựa trên sự tò mò (Curiosity-based exploration)

Một vấn đề tái diễn trong RL là sự thưa thớt của phần thưởng, điều này làm cho việc học rất chậm và không hiệu quả. Deepak Pathak và các nhà nghiên cứu UC Berkeley khác đã đề xuất một cách thú vị để giải quyết vấn đề này: tại sao không bỏ qua phần thưởng, và chỉ làm

cho tác nhân cực kỳ tò mò để khám phá môi trường? Phần thưởng do đó trở thành nội tại đối với tác nhân, chứ không phải đến từ môi trường. Tương tự, việc kích thích sự tò mò ở một đứa trẻ có nhiều khả năng mang lại kết quả tốt hơn so với việc chỉ đơn thuần thưởng cho đứa trẻ vì đạt điểm cao. Điều này hoạt động như thế nào? Tác nhân liên tục cố gắng dự đoán kết quả của các hành động của nó, và nó tìm kiếm những tình huống mà kết quả không khớp với dự đoán của nó. Nói cách khác, nó muốn được ngạc nhiên. Nếu kết quả có thể dự đoán được (nhàm chán), nó sẽ đi nơi khác. Tuy nhiên, nếu kết quả không thể đoán trước nhưng tác nhân nhận thấy rằng nó không kiểm soát được nó, nó cũng sẽ chán sau một thời gian. Chỉ với sự tò mò, các tác giả đã thành công trong việc huấn luyện một tác nhân ở nhiều trò chơi điện tử: mặc dù tác nhân không bị phạt khi thua, trò chơi bắt đầu lại, điều này nhàm chán nên nó học cách tránh nó.

Học mở (Open-ended learning - OEL)

Mục tiêu của OEL là huấn luyện các tác nhân có khả năng học vô tận các nhiệm vụ mới và thú vị, thường được tạo ra một cách thủ tục. Chúng ta chưa đạt đến đó, nhưng đã có một số tiến bộ đáng kinh ngạc trong vài năm qua. Ví dụ, một bài báo năm 2019 của một nhóm các nhà nghiên cứu từ Uber AI đã giới thiệu thuật toán POET, tạo ra nhiều môi trường 2D mô phỏng với các va chạm và lỗ hổng và huấn luyện một tác nhân cho mỗi môi trường: mục tiêu của tác nhân là đi bộ nhanh nhất có thể trong khi tránh các chướng ngại vật. Thuật toán bắt đầu với các môi trường đơn giản, nhưng chúng dần dần trở nên khó hơn theo thời gian: đây được gọi là

học theo chương trình (curriculum learning). Hơn nữa, mặc dù mỗi tác nhân chỉ được huấn luyện trong một môi trường, nó phải thường xuyên cạnh tranh với các tác nhân khác, trên tất cả các môi trường. Trong mỗi môi trường, người chiến thắng được sao chép và thay thế tác nhân đã ở đó trước đó. Bằng cách này, kiến thức được thường xuyên chuyển giao giữa các môi trường, và các tác nhân dễ thích nghi nhất được chọn. Cuối cùng, các tác nhân là những người đi bộ tốt hơn nhiều so với các tác nhân được huấn luyện trên một nhiệm vụ duy nhất, và chúng có thể giải quyết các môi trường khó hơn nhiều. Tất nhiên, nguyên tắc này cũng có thể được áp dụng cho các môi trường và nhiệm vụ khác. Nếu bạn quan tâm đến OEL, hãy chắc chắn xem bài báo Enhanced POET , cũng như bài báo năm 2021 của DeepMind về chủ đề này.

Chúng ta đã đề cập đến nhiều chủ đề trong chương này: độ dốc chính sách, chuỗi Markov, quá trình quyết định Markov, học Q, học Q xấp xỉ, và học Q sâu và các biến thể chính của nó (mục tiêu giá trị Q cố định, DQN kép, DQN đấu, và phát lại kinh nghiệm ưu tiên), và cuối cùng chúng ta đã xem nhanh một vài thuật toán phổ biến khác. Học tăng cường là một lĩnh vực rộng lớn và thú vị, với những ý tưởng và thuật toán mới xuất hiện mỗi ngày, vì vậy tôi hy vọng chương này đã khơi gợi sự tò mò của bạn: có cả một thế giới để khám phá!

Bài tập

1. Bạn sẽ định nghĩa học tăng cường như thế nào? Nó khác gì so với học có giám sát hoặc không giám sát thông thường?

2. Bạn có thể nghĩ ra ba ứng dụng có thể của RL mà không được đề cập trong chương này không? Đối với mỗi ứng dụng, môi trường là gì? Tác nhân là gì? Một số hành động có thể là gì? Phần thưởng là gì?
3. Hệ số chiết khấu là gì? Chính sách tối ưu có thể thay đổi nếu bạn sửa đổi hệ số chiết khấu không?
4. Làm thế nào để bạn đo lường hiệu suất của một tác nhân học tăng cường?
5. Bài toán phân bổ tín nhiệm là gì? Khi nào nó xảy ra? Làm thế nào bạn có thể giảm thiểu nó?
6. Mục đích của việc sử dụng bộ đệm phát lại là gì?
7. Thuật toán RL off-policy là gì?
8. Sử dụng độ dốc chính sách để giải quyết môi trường LunarLander-v2 của OpenAI Gym.
9. Sử dụng DQN kép đấu để huấn luyện một tác nhân có thể đạt được cấp độ siêu nhân trong trò chơi Atari Breakout nổi tiếng (“ALE/Breakout-v5”). Các quan sát là hình ảnh. Để đơn giản hóa nhiệm vụ, bạn nên chuyển đổi chúng sang thang độ xám (tức là tính trung bình trên trục kênh) sau đó cắt và giảm kích thước chúng, để chúng chỉ đủ lớn để chơi, nhưng không hơn. Một hình ảnh riêng lẻ không cho bạn biết quả bóng và các mái chèo đang đi theo hướng nào, vì vậy bạn nên hợp nhất hai hoặc ba hình ảnh liên tiếp để tạo thành mỗi trạng thái. Cuối cùng, DQN nên được cấu tạo chủ yếu từ các lớp tích chập.
10. Nếu bạn có khoảng $100 để dành, bạn có thể mua một Raspberry Pi 3 cộng với một số thành phần robot giá rẻ, cài đặt TensorFlow trên Pi, và thỏa sức sáng tạo! Để có ví dụ, hãy xem bài đăng thú vị này của Lukas Biewald, hoặc xem GoPiGo hoặc BrickPi. Bắt đầu với các mục tiêu đơn giản, như làm cho robot quay xung quanh để tìm góc sáng nhất (nếu nó có cảm biến ánh sáng) hoặc vật thể gần nhất (nếu nó có cảm biến siêu âm), và di chuyển theo hướng đó. Sau đó, bạn có thể bắt đầu sử dụng học sâu: ví dụ, nếu robot có camera, bạn có thể thử triển khai thuật toán phát hiện đối tượng để nó phát hiện người và di chuyển về phía họ. Bạn cũng có thể thử sử dụng RL để làm cho tác nhân tự học cách sử dụng động cơ để đạt được mục tiêu đó. Chúc bạn vui vẻ! Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại https://homl.info/colab3 .

CHƯƠNG 19. HUẤN LUYỆN VÀ TRIỂN KHAI MÔ HÌNH TENSORFLOW QUY MÔ LỚN

Khi bạn đã có một mô hình tuyệt vời có khả năng dự đoán đáng kinh ngạc, bạn sẽ làm gì với nó? Chà, bạn cần đưa nó vào sản xuất! Việc này có thể đơn giản như chạy mô hình trên một lô dữ liệu, và có lẽ viết một tập lệnh để chạy mô hình này mỗi đêm. Tuy nhiên, nó thường phức tạp hơn nhiều. Các phần khác nhau trong cơ sở hạ tầng của bạn có thể cần sử dụng mô hình này trên dữ liệu trực tiếp, trong trường hợp đó bạn có thể sẽ muốn đóng gói mô hình của mình trong một dịch vụ web: theo cách này, bất kỳ phần nào trong cơ sở hạ tầng của bạn đều có thể truy vấn mô hình bất cứ lúc nào bằng cách sử dụng một API REST đơn giản (hoặc một giao thức khác), như chúng ta đã thảo luận trong Chương 2.

Nhưng theo thời gian, bạn sẽ cần thường xuyên huấn luyện lại mô hình của mình trên dữ liệu mới và đẩy phiên bản cập nhật vào sản xuất. Bạn phải xử lý việc quản lý phiên bản mô hình, chuyển đổi một cách linh hoạt từ mô hình này sang mô hình tiếp theo, có thể quay lại mô hình trước đó trong trường hợp có vấn đề, và có thể chạy nhiều mô hình khác nhau song song để thực hiện các thử nghiệm A/B. Nếu sản phẩm của bạn trở nên thành công, dịch vụ của bạn có thể bắt đầu nhận được một số lượng lớn các truy vấn mỗi giây (QPS), và nó phải mở rộng để hỗ trợ tải. Một giải pháp tuyệt vời để mở rộng dịch vụ của bạn, như bạn sẽ thấy trong chương này, là sử dụng TF Serving, trên cơ sở hạ tầng phần cứng của riêng bạn hoặc thông qua một dịch vụ đám mây như Google Vertex AI. Nó sẽ đảm nhiệm việc phục vụ mô hình của bạn một cách hiệu quả, xử lý các chuyển đổi mô hình linh hoạt, và nhiều hơn nữa. Nếu bạn sử dụng nền tảng đám mây, bạn cũng sẽ nhận được nhiều tính năng bổ sung, chẳng hạn như các công cụ giám sát mạnh mẽ.

Hơn nữa, nếu bạn có nhiều dữ liệu huấn luyện và các mô hình yêu cầu tính toán chuyên sâu, thì thời gian huấn luyện có thể kéo dài một cách không thể chấp nhận được. Nếu sản phẩm của bạn cần thích ứng nhanh chóng với các thay đổi, thì thời gian huấn luyện dài có thể là một trở ngại lớn (ví dụ, hãy nghĩ đến một hệ thống đề xuất tin tức quảng bá tin tức từ tuần trước). Có lẽ quan trọng hơn, thời gian huấn luyện dài sẽ ngăn cản bạn thử nghiệm những ý tưởng mới. Trong học máy (cũng như trong nhiều lĩnh vực khác), rất khó để biết trước ý tưởng nào sẽ hiệu quả, vì vậy bạn nên thử càng nhiều càng tốt, càng nhanh càng tốt. Một cách để tăng tốc độ huấn luyện là sử dụng các bộ tăng tốc phần cứng như GPU hoặc TPU. Để đi nhanh hơn nữa, bạn có thể huấn luyện một mô hình trên nhiều máy, mỗi máy được trang bị nhiều bộ tăng tốc phần cứng. API chiến lược phân phối đơn giản nhưng mạnh mẽ của TensorFlow giúp việc này trở nên dễ dàng, như bạn sẽ thấy.

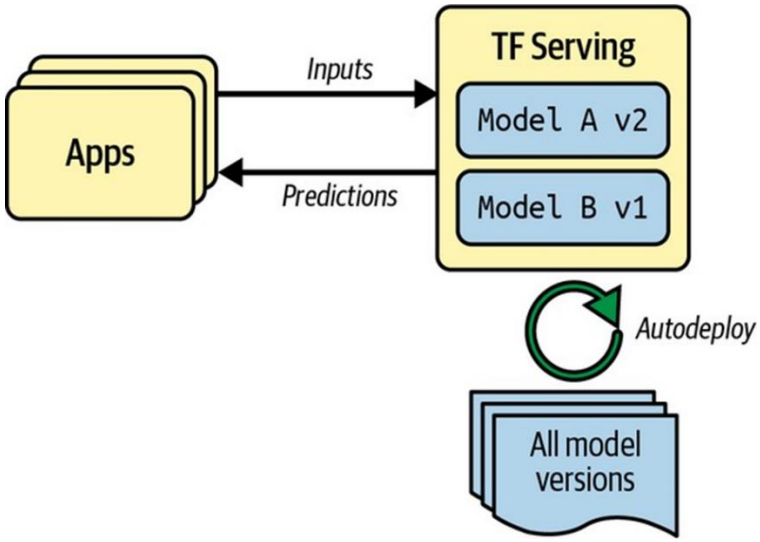
Trong chương này, chúng ta sẽ xem xét cách triển khai mô hình, đầu tiên sử dụng TF Serving, sau đó sử dụng Vertex AI. Chúng ta cũng sẽ xem nhanh về việc triển khai mô hình cho ứng dụng di động, thiết bị nhúng và ứng dụng web. Sau đó, chúng ta sẽ thảo luận về cách tăng tốc tính toán bằng GPU và cách huấn luyện mô hình trên nhiều thiết bị và máy chủ bằng API chiến lược phân phối. Cuối cùng, chúng ta sẽ khám phá cách huấn luyện mô hình và tinh chỉnh các siêu tham số của chúng ở quy mô lớn bằng Vertex AI. Có rất nhiều chủ đề để thảo luận, vậy hãy cùng tìm hiểu!

Phục vụ một mô hình TensorFlow

Khi bạn đã huấn luyện một mô hình TensorFlow, bạn có thể dễ dàng sử dụng nó trong bất kỳ mã Python nào: nếu đó là mô hình Keras, chỉ cần gọi phương thức `predict()` của nó! Nhưng khi cơ sở hạ tầng của bạn phát triển, sẽ đến lúc nên đóng gói mô hình của bạn trong một dịch vụ nhỏ mà vai trò duy nhất của nó là thực hiện các dự đoán và phần còn lại của cơ sở hạ tầng truy vấn nó (ví dụ: thông qua API REST hoặc gRPC). Điều này giúp tách mô hình của bạn khỏi phần còn lại của cơ sở hạ tầng, giúp dễ dàng chuyển đổi phiên bản mô hình hoặc mở rộng dịch vụ khi cần (độc lập với phần còn lại của cơ sở hạ tầng của bạn), thực hiện các thử nghiệm A/B và đảm bảo rằng tất cả các thành phần phần mềm của bạn đều dựa trên cùng một phiên bản mô hình. Nó cũng đơn giản hóa việc kiểm tra và phát triển, và nhiều hơn nữa. Bạn có thể tạo microservice của riêng mình bằng bất kỳ công nghệ nào bạn muốn (ví dụ: sử dụng thư viện Flask), nhưng tại sao phải phát minh lại bánh xe khi bạn có thể chỉ sử dụng TF Serving?

Sử dụng TensorFlow Serving

TF Serving là một máy chủ mô hình rất hiệu quả, đã được thử nghiệm kỹ lưỡng, được viết bằng C++. Nó có thể chịu được tải cao, phục vụ nhiều phiên bản mô hình của bạn và theo dõi kho lưu trữ mô hình để tự động triển khai các phiên bản mới nhất, v.v. (xem Hình 19-1).



Hình 19-1. TF Serving có thể phục vụ nhiều mô hình và tự động triển khai phiên bản mới nhất của mỗi mô hình.

Vậy giả sử bạn đã huấn luyện một mô hình MNIST bằng Keras và bạn muốn triển khai nó lên TF Serving. Điều đầu tiên bạn phải làm là xuất mô hình này sang định dạng SavedModel, đã được giới thiệu trong Chương 10.

Xuất SavedModels

Bạn đã biết cách lưu mô hình: chỉ cần gọi `model.save()`. Bây giờ để quản lý phiên bản mô hình, bạn chỉ cần tạo một thư mục con cho mỗi phiên bản mô hình. Dễ dàng!

```
from pathlib import Path
import tensorflow as tf

X_train, X_valid, X_test = [...] # tải và chia tập dữ liệu MNIST
model = [...] # xây dựng & huấn luyện một mô hình MNIST (cũng xử lý tiền xử
lý ảnh)

model_name = "my_mnist_model"
model_version = "0001"
model_path = Path(model_name) / model_version
model.save(model_path, save_format="tf")
```

Thường thì nên bao gồm tất cả các lớp tiền xử lý trong mô hình cuối cùng mà bạn xuất để nó có thể tiếp nhận dữ liệu ở dạng tự nhiên sau khi được triển khai vào sản xuất. Điều này tránh việc phải xử lý tiền xử lý riêng biệt trong ứng dụng sử dụng mô hình. Việc đóng gói các bước tiền xử lý trong mô hình cũng giúp việc cập nhật chúng sau này đơn giản hơn và hạn chế rủi ro không khớp giữa mô hình và các bước tiền xử lý mà nó yêu cầu.

TensorFlow đi kèm với một giao diện dòng lệnh nhỏ `saved_model_cli` để kiểm tra SavedModels. Hãy sử dụng nó để kiểm tra mô hình đã xuất của chúng ta:

```
$ saved_model_cli show --dir my_mnist_model/0001
```

Kết quả này có nghĩa là gì? Chà, một SavedModel chứa một hoặc nhiều metagraph. Metagraph là một đồ thị tính toán cộng với một số định nghĩa chữ ký hàm, bao gồm tên đầu vào và đầu ra, kiểu và hình dạng của chúng. Mỗi metagraph được xác định bằng một tập hợp các thẻ. Ví dụ, bạn có thể muốn có một metagraph chứa toàn bộ đồ thị tính toán, bao gồm các hoạt động huấn luyện: bạn thường sẽ gắn thẻ này là “train”. Và bạn có thể có một metagraph khác chứa một đồ thị tính toán đã được cắt tỉa chỉ với các hoạt động dự đoán, bao gồm một số hoạt động cụ thể của GPU: thẻ này có thể được gắn là “serve”, “gpu”. Bạn cũng có thể muốn có các metagraph khác. Điều này có thể được thực hiện bằng cách sử dụng API SavedModel cấp thấp của TensorFlow. Tuy nhiên, khi bạn lưu một mô hình Keras bằng phương thức `save()` của nó, nó sẽ lưu một metagraph duy nhất được gắn thẻ là “serve”.

Hãy kiểm tra tập thẻ “serve” này:

```
$ saved_model_cli show --dir 0001/my_mnist_model --tag_set serve

The given SavedModel MetaGraphDef contains SignatureDefs with these keys:
SignatureDef key: "saved_model_init_op"
SignatureDef key: "serving_default"
```

Metagraph này chứa hai định nghĩa chữ ký: một hàm khởi tạo có tên “saved_model_init_op”, mà bạn không cần phải lo lắng, và một hàm phục vụ mặc định có tên “serving_default”. Khi lưu một mô hình Keras, hàm phục vụ mặc định là phương thức `call()` của mô hình, thực hiện các dự đoán, như bạn đã biết. Hãy lấy thêm chi tiết về hàm phục vụ này:

```
$ saved_model_cli show --dir 0001/my_mnist_model --tag_set serve \
--signature_def serving_default

The given SavedModel SignatureDef contains the following input(s):
inputs['flatten_input'] tensor_info:
dtype: DT_UINT8
shape: (-1, 28, 28)
name: serving_default_flatten_input:0

The given SavedModel SignatureDef contains the following output(s):
outputs['dense_1'] tensor_info:
dtype: DT_FLOAT
shape: (-1, 10)
name: StatefulPartitionedCall:0
Method name is: tensorflow/serving/predict
```

Lưu ý rằng đầu vào của hàm được đặt tên là "flatten_input", và đầu ra được đặt tên là "dense_1". Những cái này tương ứng với tên lớp đầu vào và đầu ra của mô hình Keras. Bạn cũng có thể thấy kiểu và hình dạng của dữ liệu đầu vào và đầu ra. Trông rất tốt! Bây giờ bạn đã có một SavedModel, bước tiếp theo là cài đặt TF Serving.

Cài đặt và khởi động TensorFlow Serving

Có nhiều cách để cài đặt TF Serving: sử dụng trình quản lý gói của hệ thống, sử dụng ảnh Docker, cài đặt từ mã nguồn, và nhiều hơn nữa. Vì Colab chạy trên Ubuntu, chúng ta có thể sử dụng trình quản lý gói `apt` của Ubuntu như sau:

```
url = "https://storage.googleapis.com/tensorflow-serving-apt"
src = "stable tensorflow-model-server tensorflow-model-server-universal"
!echo 'deb {url} {src}' > /etc/apt/sources.list.d/tensorflow-serving.list
!curl '{url}/tensorflow-serving.release.pub.gpg' | apt-key add -
!apt update -q && apt-get install -y tensorflow-model-server

%pip install -q -U tensorflow-serving-api
```

Mã này bắt đầu bằng cách thêm kho lưu trữ gói của TensorFlow vào danh sách các nguồn gói của Ubuntu. Sau đó, nó tải xuống khóa GPG công khai của TensorFlow và thêm nó vào danh sách khóa của trình quản lý gói để nó có thể xác minh chữ ký gói của TensorFlow. Tiếp theo, nó sử dụng `apt` để cài đặt gói `tensorflow-model-server`. Cuối cùng, nó cài đặt thư viện `tensorflow-serving-api`, mà chúng ta sẽ cần để giao tiếp với máy chủ.

Bây giờ chúng ta muốn khởi động máy chủ. Lệnh này sẽ yêu cầu đường dẫn tuyệt đối của thư mục mô hình cơ sở (nghĩa là đường dẫn đến `my_mnist_model`, không phải `0001`), vì vậy hãy lưu nó vào biến môi trường `MODEL_DIR`:

```
import os

os.environ["MODEL_DIR"] = str(model_path.parent.absolute())
```

Sau đó, chúng ta có thể khởi động máy chủ:

```
%%bash --bg
tensorflow_model_server \
--port=8500 \
--rest_api_port=8501 \
--model_name=my_mnist_model \
--model_base_path="${MODEL_DIR}" >my_server.log 2>&1
```

Trong Jupyter hoặc Colab, lệnh ma thuật `%%bash --bg` thực thi ô dưới dạng tập lệnh bash, chạy nó trong nền. Phần `>my_server.log 2>&1` chuyển hướng đầu ra tiêu chuẩn và lỗi tiêu chuẩn đến tệp `my_server.log`. Và thế là xong! TF Serving hiện đang chạy trong nền, và nhật ký của nó được lưu vào `my_server.log`. Nó đã tải mô hình MNIST của chúng ta (phiên bản 1), và bây giờ nó đang chờ các yêu cầu gRPC và REST, tương ứng, trên các cổng 8500 và 8501.

CHẠY TF SERVING TRONG MỘT CONTAINER DOCKER Nếu bạn đang chạy sổ tay trên máy của mình và bạn đã cài đặt Docker, bạn có thể chạy `docker pull tensorflow/serving` trong một terminal để tải ảnh TF Serving. Đội ngũ TensorFlow rất khuyến nghị phương pháp cài đặt này vì nó đơn giản, sẽ không làm rối hệ thống của bạn và mang lại hiệu suất cao. Để khởi động máy chủ bên trong một container Docker, bạn có thể chạy lệnh sau trong một terminal:

```
$ docker run -it --rm -v "/path/to/my_mnist_model:/models/my_mnist_model"\
    -p 8500:8500 -p 8501:8501 -e MODEL_NAME=my_mnist_model tensorflow/serving
```

Đây là ý nghĩa của tất cả các tùy chọn dòng lệnh này:

- `-it`: Làm cho container tương tác (để bạn có thể nhấn Ctrl-C để dừng nó) và hiển thị đầu ra của máy chủ.
- `--rm`: Xóa container khi bạn dừng nó: không cần làm lộn xộn máy của bạn với các container bị gián đoạn. Tuy nhiên, nó không xóa ảnh.
- `-v "/path/to/my_mnist_model:/models/my_mnist_model"`: Làm cho thư mục `my_mnist_model` của máy chủ có sẵn cho container tại đường dẫn `/models/mnist_model`. Bạn phải thay thế `/path/to/my_mnist_model` bằng đường dẫn tuyệt đối của thư mục này. Trên Windows, hãy nhớ sử dụng `\` thay vì `/` trong đường dẫn máy chủ, nhưng không phải trong đường dẫn container (vì container chạy trên Linux).
- `-p 8500:8500`: Khiến Docker engine chuyển tiếp cổng TCP 8500 của máy chủ đến cổng TCP 8500 của container. Theo mặc định, TF Serving sử dụng cổng này để phục vụ API gRPC.
- `-p 8501:8501`: Chuyển tiếp cổng TCP 8501 của máy chủ đến cổng TCP 8501 của container. Ảnh Docker được cấu hình để sử dụng cổng này theo mặc định để phục vụ API REST.
- `-e MODEL_NAME=my_mnist_model`: Đặt biến môi trường `MODEL_NAME` của container, để TF Serving biết mô hình nào sẽ phục vụ. Theo mặc định, nó sẽ tìm kiếm các mô

hình trong thư mục `/models`, và nó sẽ tự động phục vụ phiên bản mới nhất mà nó tìm thấy.

- `tensorflow/serving`: Đây là tên của ảnh để chạy.

Bây giờ máy chủ đã hoạt động, hãy truy vấn nó, đầu tiên sử dụng API REST, sau đó là API gRPC.

Truy vấn TF Serving qua API REST

Hãy bắt đầu bằng cách tạo truy vấn. Nó phải chứa tên của chữ ký hàm bạn muốn gọi, và tất nhiên là dữ liệu đầu vào. Vì yêu cầu phải sử dụng định dạng JSON, chúng ta phải chuyển đổi các ảnh đầu vào từ một mảng NumPy sang một danh sách Python:

```
import json
import requests
# Giả định X_test đã được định nghĩa từ trước, ví dụ: X_test =
tf.keras.datasets.mnist.load_data()[1][0]
X_new = X_test[:3] # giả vờ chúng ta có 3 ảnh chữ số mới để phân loại
request_json = json.dumps({
    "signature_name": "serving_default",
    "instances": X_new.tolist(),
})
```

Lưu ý rằng định dạng JSON hoàn toàn dựa trên văn bản. Chuỗi yêu cầu trông như thế này:

```
>>> request_json
'{"signature_name": "serving_default", "instances": [[[0, 0, 0, 0, ... ]]]}'
```

Bây giờ hãy gửi yêu cầu này đến TF Serving qua yêu cầu HTTP POST. Việc này có thể được thực hiện bằng thư viện `requests` (nó không phải là một phần của thư viện chuẩn Python, nhưng nó được cài đặt sẵn trên Colab):

```
server_url = "http://localhost:8501/v1/models/my_mnist_model:predict"
response = requests.post(server_url, data=request_json)
response.raise_for_status() # đưa ra một ngoại lệ trong trường hợp lỗi
response = response.json()
```

Phán đoán và viết công thức toán:

Trong đoạn văn bản này, không có công thức toán học tường minh nào được viết ra. Tuy nhiên, các khái niệm được đề cập ngụ ý đến các phép toán và cấu trúc dữ liệu cơ bản trong học máy, đặc biệt là liên quan đến các mô hình mạng nơ-ron:

1. **Mô hình dự đoán (`model.predict()`):** Đây là một hàm thực hiện phép toán tiến (forward pass) của mô hình. Nếu giả sử mô hình Keras là một mạng nơ-ron truyền thẳng (Feedforward Neural Network) với các lớp dày đặc (Dense layers), quá trình dự đoán cho một đầu vào x có thể được biểu diễn tổng quát như sau: $$$$$y = f(x) $$ $$$$Trong đó:

- x là dữ liệu đầu vào (ví dụ: ảnh MNIST 28x28 pixel).
- f là hàm biến đổi phi tuyến tính tổng hợp của mô hình (chuỗi các lớp mạng nơ-ron).
- y là đầu ra dự đoán của mô hình (ví dụ: xác suất cho 10 chữ số trong MNIST). Đối với một lớp Dense trong mạng nơ-ron, phép toán cơ bản là biến đổi affine theo sau bởi một hàm kích hoạt: $$$$$$\^{(l)} = \^{(l)} \^{(l-1)} + \^{(l)} \\ \^{(l)} = g^{(l)}(\^{(l)}) $$ $$$$Trong đó:
- $a^{(l-1)}$ là đầu ra (kích hoạt) của lớp trước đó.
- $W^{(l)}$ là ma trận trọng số của lớp thứ l .
- $b^{(l)}$ là vector bias của lớp thứ l .
- $z^{(l)}$ là đầu vào tuyến tính cho hàm kích hoạt của lớp thứ l .
- $g^{(l)}$ là hàm kích hoạt của lớp thứ l (ví dụ: ReLU, Sigmoid, Softmax).
- $a^{(l)}$ là đầu ra (kích hoạt) của lớp thứ l .

2. **Dữ liệu đầu vào và đầu ra (Input/Output `tensor_info`):**
   - `inputs['flatten_input'] tensor_info: dtype: DT_UINT8 shape: (-1, 28, 28)`
     - Đây là Tensor đầu vào có kiểu dữ liệu `UINT8` (số nguyên không dấu 8 bit, thường dùng cho giá trị pixel 0-255).
     - Shape `(-1, 28, 28)` biểu thị:
       - `-1`: Kích thước batch động (số lượng mẫu trong một lô có thể thay đổi).
       - `28`: Chiều cao của ảnh (28 pixel).
       - `28`: Chiều rộng của ảnh (28 pixel).
     - Đối với ảnh MNIST, mỗi mẫu đầu vào là một ma trận 28x28.
   - `outputs['dense_1'] tensor_info: dtype: DT_FLOAT shape: (-1, 10)`
     - Đây là Tensor đầu ra có kiểu dữ liệu `FLOAT` (số thực).
     - Shape `(-1, 10)` biểu thị:
       - `-1`: Kích thước batch động.
       - `10`: Kích thước của vector đầu ra, thường là 10 lớp cho phân loại chữ số từ 0 đến 9. Nếu `dense_1` là lớp cuối cùng và dùng cho phân loại đa lớp, hàm kích hoạt thường là Softmax. Công thức của Softmax cho một vector đầu ra z có K phần tử là: $$$$$$P(y=k|\) = \ $$ $$$$Trong đó z_k là phần tử thứ k của vector đầu ra tuyến tính trước hàm Softmax.

Các công thức trên là những công thức toán học cơ bản mà một mô hình mạng nơ-ron, đặc biệt là mô hình Keras MNIST, sẽ sử dụng trong quá trình huấn luyện và dự đoán.

Truy vấn TF Serving qua API REST (Tiếp theo)

Nếu mọi việc suôn sẻ, phản hồi sẽ là một dictionary chứa một khóa “predictions” duy nhất. Giá trị tương ứng là danh sách các dự đoán. Danh sách này là một danh sách Python, vì vậy

hãy chuyển đổi nó thành một mảng NumPy và làm tròn các số thập phân mà nó chứa đến hai chữ số thập phân:

```
>>> import numpy as np

>>> y_proba = np.array(response["predictions"])

>>> y_proba.round(2)
array([[0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 1.  , 0.  , 0.  ],
       [0.  , 0.  , 0.99, 0.01, 0.  , 0.  , 0.  , 0.  , 0.  ],
       [0.  , 0.97, 0.01, 0.  , 0.  , 0.  , 0.01, 0.  , 0.  ]])
```

Hoan hô, chúng ta đã có các dự đoán! Mô hình tự tin gần 100% rằng ảnh đầu tiên là số 7, tự tin 99% rằng ảnh thứ hai là số 2, và tự tin 97% rằng ảnh thứ ba là số 1. Điều đó hoàn toàn chính xác.

API REST đẹp và đơn giản, và nó hoạt động tốt khi dữ liệu đầu vào và đầu ra không quá lớn. Hơn nữa, hầu hết mọi ứng dụng khách hàng đều có thể thực hiện các truy vấn REST mà không cần thêm phụ thuộc, trong khi các giao thức khác không phải lúc nào cũng sẵn có như vậy. Tuy nhiên, nó dựa trên JSON, là định dạng văn bản và khá dài dòng. Ví dụ, chúng ta phải chuyển đổi mảng NumPy sang danh sách Python, và mỗi số float cuối cùng được biểu diễn dưới dạng một chuỗi. Điều này rất không hiệu quả, cả về thời gian tuần tự hóa/giải tuần tự hóa — chúng ta phải chuyển đổi tất cả các số float sang chuỗi và ngược lại — và về kích thước tải trọng: nhiều số float cuối cùng được biểu diễn bằng hơn 15 ký tự, tương đương với hơn 120 bit cho số float 32 bit! Điều này sẽ dẫn đến độ trễ cao và sử dụng băng thông lớn khi truyền các mảng NumPy lớn. Vì vậy, hãy xem cách sử dụng gRPC thay thế.

Truy vấn TF Serving thông qua API gRPC

API gRPC mong đợi một protocol buffer `PredictRequest` đã được tuần tự hóa làm đầu vào, và nó xuất ra một protocol buffer `PredictResponse` đã được tuần tự hóa. Các protobuf này là một phần của thư viện `tensorflow-serving-api`, mà chúng ta đã cài đặt trước đó. Đầu tiên, hãy tạo yêu cầu:

```
from tensorflow_serving.apis.predict_pb2 import PredictRequest
import tensorflow as tf # Đảm bảo tensorflow đã được import
# Giả định model và X_new đã được định nghĩa từ trước
# Ví dụ: model = tf.keras.models.load_model(model_path)
# X_new = X_test[:3]

request = PredictRequest()
request.model_spec.name = "my_mnist_model" # Sử dụng model_name đã định nghĩa
trước đó
request.model_spec.signature_name = "serving_default"
input_name = "flatten_input" # == model.input_names[0]
request.inputs[input_name].CopyFrom(tf.make_tensor_proto(X_new))
```

Đoạn mã này tạo ra một protocol buffer `PredictRequest` và điền vào các trường bắt buộc, bao gồm tên mô hình (đã được định nghĩa trước đó), tên chữ ký của hàm chúng ta muốn gọi, và cuối cùng là dữ liệu đầu vào, dưới dạng một protocol buffer Tensor. Hàm `tf.make_tensor_proto()` tạo ra một protocol buffer Tensor dựa trên tensor hoặc mảng NumPy đã cho, trong trường hợp này là `X_new`.

Tiếp theo, chúng ta sẽ gửi yêu cầu đến máy chủ và nhận phản hồi. Để làm điều này, chúng ta sẽ cần thư viện `grpcio`, đã được cài đặt sẵn trong Colab:

```
import grpc
from tensorflow_serving.apis import prediction_service_pb2_grpc
# Giả định request đã được tạo ở trên

channel = grpc.insecure_channel('localhost:8500')
predict_service = prediction_service_pb2_grpc.PredictionServiceStub(channel)
response = predict_service.Predict(request, timeout=10.0)
```

Đoạn mã khá đơn giản: sau khi import, chúng ta tạo một kênh giao tiếp gRPC đến `localhost` trên cổng TCP 8500, sau đó chúng ta tạo một dịch vụ gRPC trên kênh này và sử dụng nó để gửi yêu cầu, với thời gian chờ là 10 giây. Lưu ý rằng cuộc gọi là đồng bộ: nó sẽ chặn cho đến khi nhận được phản hồi hoặc khi hết thời gian chờ. Trong ví dụ này, kênh không an toàn (không mã hóa, không xác thực), nhưng gRPC và TF Serving cũng hỗ trợ các kênh an toàn qua SSL/TLS.

Tiếp theo, hãy chuyển đổi protocol buffer `PredictResponse` thành một tensor:

```
# Giả định response đã nhận được từ gRPC call
output_name = "dense_1" # == model.output_names[0]
outputs_proto = response.outputs[output_name]
y_proba = tf.make_ndarray(outputs_proto)
```

Nếu bạn chạy đoạn mã này và in `y_proba.round(2)`, bạn sẽ nhận được chính xác các xác suất lớp ước tính giống như trước đó. Và thế là xong: chỉ với một vài dòng mã, bạn đã có thể truy cập mô hình TensorFlow của mình từ xa, sử dụng REST hoặc gRPC.

Triển khai phiên bản mô hình mới

Bây giờ hãy tạo một phiên bản mô hình mới và xuất một SavedModel, lần này vào thư mục `my_mnist_model/0002`:

```
# Giả định model_name đã được định nghĩa là "my_mnist_model"
# Giả định bạn đã có một mô hình MNIST mới được huấn luyện
# model = build_and_train_new_mnist_model() # ví dụ về hàm huấn luyện mô hình mới

model_version = "0002"
model_path = Path(model_name) / model_version
model.save(model_path, save_format="tf")
```

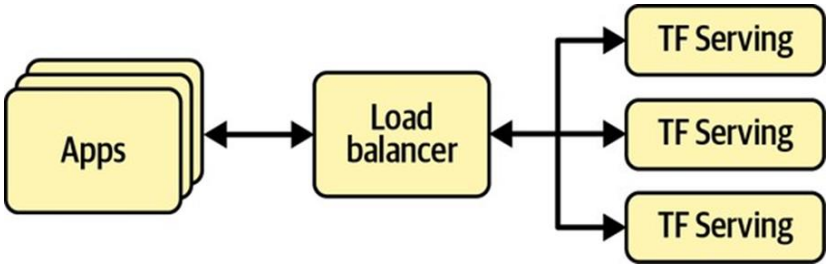
Theo các khoảng thời gian đều đặn (thời gian trễ có thể cấu hình được), TF Serving kiểm tra thư mục mô hình để tìm các phiên bản mô hình mới. Nếu tìm thấy một phiên bản, nó sẽ tự động xử lý quá trình chuyển đổi một cách linh hoạt: theo mặc định, nó trả lời các yêu cầu đang chờ xử lý (nếu có) bằng phiên bản mô hình trước đó, trong khi xử lý các yêu cầu mới bằng phiên bản mới. Ngay sau khi mọi yêu cầu đang chờ xử lý đã được trả lời, phiên bản mô hình trước đó sẽ được giải phóng. Bạn có thể thấy điều này hoạt động trong nhật ký TF Serving (trong `my_server.log`):

```
[...]
Reading SavedModel from: /models/my_mnist_model/0002
Reading meta graph with tags { serve }
[...]
Successfully loaded servable version {name: my_mnist_model version: 2}
Quiescing servable version {name: my_mnist_model version: 1}
Done quiescing servable version {name: my_mnist_model version: 1}
Unloading servable version {name: my_mnist_model version: 1}
```

Cách tiếp cận này mang lại sự chuyển đổi mượt mà, nhưng nó có thể sử dụng quá nhiều RAM — đặc biệt là RAM GPU, thường là bị hạn chế nhất. Trong trường hợp này, bạn có thể cấu hình TF Serving để nó xử lý tất cả các yêu cầu đang chờ xử lý bằng phiên bản mô hình trước đó và giải phóng nó trước khi tải và sử dụng phiên bản mô hình mới. Cấu hình này sẽ tránh việc có hai phiên bản mô hình được tải cùng lúc, nhưng dịch vụ sẽ không khả dụng trong một thời gian ngắn.

Như bạn có thể thấy, TF Serving giúp việc triển khai các mô hình mới trở nên đơn giản. Hơn nữa, nếu bạn phát hiện ra rằng phiên bản 2 không hoạt động tốt như bạn mong đợi, thì việc quay lại phiên bản 1 đơn giản như việc xóa thư mục `my_mnist_model/0002`.

Nếu bạn mong đợi nhận được nhiều truy vấn mỗi giây, bạn sẽ muốn triển khai TF Serving trên nhiều máy chủ và cân bằng tải các truy vấn (xem Hình 19-2). Điều này sẽ yêu cầu triển khai và quản lý nhiều container TF Serving trên các máy chủ này. Một cách để xử lý việc đó là sử dụng một công cụ như Kubernetes, là một hệ thống mã nguồn mở để đơn giản hóa việc điều phối container trên nhiều máy chủ. Nếu bạn không muốn mua, bảo trì và nâng cấp tất cả cơ sở hạ tầng phần cứng, bạn sẽ muốn sử dụng máy ảo trên nền tảng đám mây như Amazon AWS, Microsoft Azure, Google Cloud Platform, IBM Cloud, Alibaba Cloud, Oracle Cloud hoặc một số dịch vụ PaaS (Platform as a Service) khác. Việc quản lý tất cả các máy ảo, xử lý điều phối container (ngay cả với sự giúp đỡ của Kubernetes), chăm sóc cấu hình, điều chỉnh và giám sát TF Serving — tất cả những điều này có thể là một công việc toàn thời gian. May mắn thay, một số nhà cung cấp dịch vụ có thể lo tất cả những điều này cho bạn. Trong chương này, chúng ta sẽ sử dụng Vertex AI: đây là nền tảng duy nhất có TPU hiện nay; nó hỗ trợ TensorFlow 2, Scikit-Learn và XGBoost; và nó cung cấp một bộ dịch vụ AI tuyệt vời. Tuy nhiên, có một số nhà cung cấp khác trong lĩnh vực này cũng có khả năng phục vụ các mô hình TensorFlow, chẳng hạn như Amazon AWS SageMaker và Microsoft AI Platform, vì vậy hãy đảm bảo kiểm tra chúng.



Hình 19-2. Mở rộng TF Serving với cân bằng tải

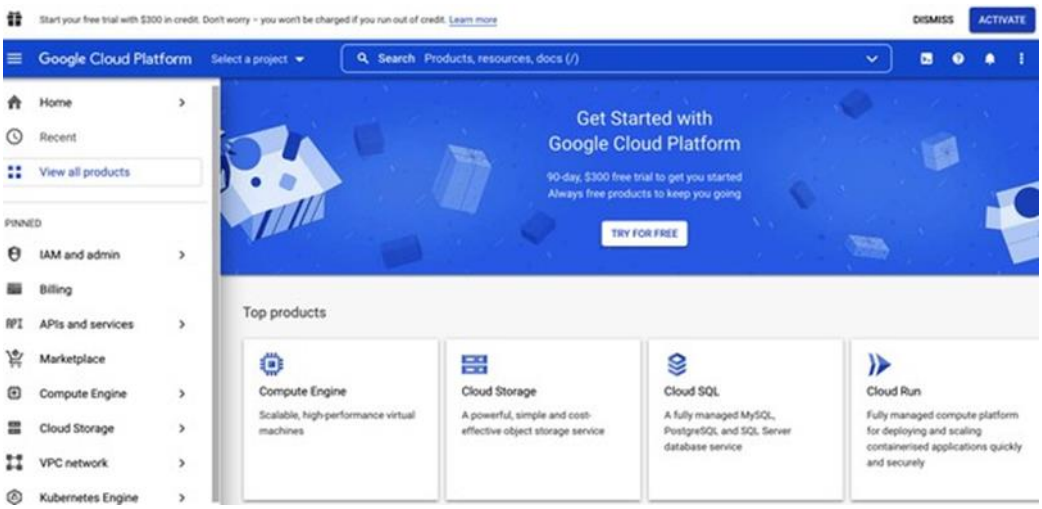
Bây giờ hãy xem cách phục vụ mô hình MNIST tuyệt vời của chúng ta trên đám mây!

Tạo dịch vụ dự đoán trên Vertex AI

Vertex AI là một nền tảng trong Google Cloud Platform (GCP) cung cấp một loạt các công cụ và dịch vụ liên quan đến AI. Bạn có thể tải lên tập dữ liệu, nhờ con người gán nhãn chúng, lưu trữ các tính năng thường được sử dụng trong một kho tính năng và sử dụng chúng để huấn luyện hoặc trong sản xuất, và huấn luyện mô hình trên nhiều máy chủ GPU hoặc TPU với điều chỉnh siêu tham số tự động hoặc tìm kiếm kiến trúc mô hình (AutoML). Bạn cũng có thể quản lý các mô hình đã huấn luyện của mình, sử dụng chúng để thực hiện dự đoán hàng loạt trên lượng lớn dữ liệu, lên lịch nhiều công việc cho quy trình dữ liệu của bạn, phục vụ mô hình của bạn qua REST hoặc gRPC ở quy mô lớn, và thử nghiệm với dữ liệu và mô hình của bạn trong môi trường Jupyter được lưu trữ gọi là Workbench. Thậm chí còn có dịch vụ Matching Engine cho phép bạn so sánh các vector rất hiệu quả (tức là hàng xóm gần nhất xấp xỉ). GCP cũng bao gồm các dịch vụ AI khác, chẳng hạn như API cho thị giác máy tính, dịch thuật, chuyển giọng nói thành văn bản, và nhiều hơn nữa.

Trước khi chúng ta bắt đầu, có một chút thiết lập cần thực hiện:

3. Đăng nhập vào tài khoản Google của bạn, và sau đó truy cập bảng điều khiển Google Cloud Platform (xem Hình 19-3). Nếu bạn chưa có tài khoản Google, bạn sẽ phải tạo một tài khoản.
4. Nếu đây là lần đầu tiên bạn sử dụng GCP, bạn sẽ phải đọc và chấp nhận các điều khoản và điều kiện. Người dùng mới được cung cấp bản dùng thử miễn phí, bao gồm 300 đô la tín dụng GCP mà bạn có thể sử dụng trong 90 ngày (tính đến tháng 5 năm 2022). Bạn sẽ chỉ cần một phần nhỏ trong số đó để thanh toán cho các dịch vụ bạn sẽ sử dụng trong chương này. Khi đăng ký bản dùng thử miễn phí, bạn vẫn cần tạo hồ sơ thanh toán và nhập số thẻ tín dụng: nó được sử dụng cho mục đích xác minh — có lẽ để tránh những người sử dụng bản dùng thử miễn phí nhiều lần — nhưng bạn sẽ không bị tính phí cho 300 đô la đầu tiên, và sau đó bạn sẽ chỉ bị tính phí nếu bạn chọn nâng cấp lên tài khoản trả phí.



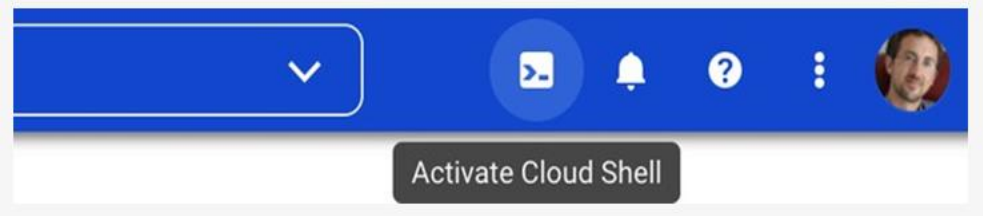
Hình 19-3. Bảng điều khiển Google Cloud Platform

3. Nếu bạn đã sử dụng GCP trước đây và bản dùng thử miễn phí của bạn đã hết hạn, thì các dịch vụ bạn sẽ sử dụng trong chương này sẽ tốn một ít tiền. Nó sẽ không quá nhiều, đặc biệt nếu bạn nhớ tắt các dịch vụ khi bạn không cần chúng nữa. Đảm bảo bạn hiểu và đồng ý với các điều kiện giá trước khi bạn chạy bất kỳ dịch vụ nào. Tôi từ chối mọi trách nhiệm nếu dịch vụ cuối cùng tốn kém hơn bạn mong đợi! Cũng đảm bảo tài khoản thanh toán của bạn đang hoạt động. Để kiểm tra, mở menu điều hướng ☰ ở trên cùng bên trái và nhấp vào “Billing”, sau đó đảm bảo bạn đã thiết lập phương thức thanh toán và tài khoản thanh toán đang hoạt động.
4. Mọi tài nguyên trong GCP đều thuộc về một dự án. Điều này bao gồm tất cả các máy ảo bạn có thể sử dụng, các tệp bạn lưu trữ và các công việc đào tạo bạn chạy. Khi bạn tạo tài khoản, GCP tự động tạo một dự án cho bạn, được gọi là “My First Project”. Nếu muốn, bạn có thể thay đổi tên hiển thị của nó bằng cách truy cập cài đặt dự án: trong menu điều hướng ☰, chọn “IAM and admin → Settings”, thay đổi tên hiển thị của dự án và nhấp vào SAVE. Lưu ý rằng dự án cũng có một ID và số duy nhất. Bạn có thể chọn ID dự án khi tạo dự án, nhưng bạn không thể thay đổi nó sau này. Số dự án được tự động tạo và không thể thay đổi. Nếu bạn muốn tạo một dự án mới, nhấp vào tên dự án ở đầu trang, sau đó nhấp vào NEW PROJECT và nhập tên dự án. Bạn cũng có thể nhấp vào EDIT để đặt ID dự án. Đảm bảo thanh toán đang hoạt động cho dự án mới này để phí dịch vụ có thể được thanh toán (vào tín dụng miễn phí của bạn, nếu có).
5. Bây giờ bạn đã có tài khoản GCP và một dự án, và thanh toán đã được kích hoạt, bạn phải kích hoạt các API bạn cần. Trong menu điều hướng ☰, chọn “APIs and services”, và đảm bảo Cloud Storage API được bật. Nếu cần, nhấp vào + ENABLE APIS AND SERVICES, tìm Cloud Storage và bật nó. Cũng bật Vertex AI API.

Bạn có thể tiếp tục thực hiện mọi thứ thông qua bảng điều khiển GCP, nhưng tôi khuyên bạn nên sử dụng Python thay thế: theo cách này, bạn có thể viết các tập lệnh để tự động hóa gần như mọi thứ bạn muốn với GCP, và nó thường tiện lợi hơn là nhấp qua các menu và biểu mẫu, đặc biệt đối với các tác vụ thông thường.

GOOGLE CLOUD CLI VÀ SHELL Giao diện dòng lệnh (CLI) của Google Cloud bao gồm lệnh `gcloud`, cho phép bạn kiểm soát hầu hết mọi thứ trong GCP, và `gsutil`, cho phép bạn tương tác với Google Cloud Storage. CLI này đã được cài đặt sẵn trong Colab: tất cả những gì bạn cần làm là xác thực bằng `google.auth.authenticate_user()`, và bạn đã sẵn sàng. Ví dụ, `!gcloud config list` sẽ hiển thị cấu hình.

GCP cũng cung cấp một môi trường shell được cấu hình sẵn gọi là Google Cloud Shell, mà bạn có thể sử dụng trực tiếp trong trình duyệt web của mình; nó chạy trên một máy ảo Linux (Debian) miễn phí với Google Cloud SDK đã được cài đặt sẵn và cấu hình cho bạn, vì vậy không cần xác thực. Cloud Shell có sẵn ở bất cứ đâu trong GCP: chỉ cần nhấp vào biểu tượng Activate Cloud Shell ở phía trên bên phải trang (xem Hình 19-4).



Hình 19-4. Kích hoạt Google Cloud Shell

Nếu bạn muốn cài đặt CLI trên máy của mình, thì sau khi cài đặt, bạn cần khởi tạo nó bằng cách chạy `gcloud init`: làm theo hướng dẫn để đăng nhập vào GCP và cấp quyền truy cập vào tài nguyên GCP của bạn, sau đó chọn dự án GCP mặc định bạn muốn sử dụng (nếu bạn có nhiều hơn một) và khu vực mặc định nơi bạn muốn các công việc của mình chạy.

Điều đầu tiên bạn cần làm trước khi có thể sử dụng bất kỳ dịch vụ GCP nào là xác thực. Giải pháp đơn giản nhất khi sử dụng Colab là thực thi đoạn mã sau:

```
from google.colab import auth

auth.authenticate_user()
```

Quá trình xác thực dựa trên OAuth 2.0: một cửa sổ bật lên sẽ yêu cầu bạn xác nhận rằng bạn muốn sổ tay Colab truy cập thông tin đăng nhập Google của bạn. Nếu bạn chấp nhận, bạn phải chọn cùng tài khoản Google bạn đã sử dụng cho GCP. Sau đó, bạn sẽ được yêu cầu xác nhận rằng bạn đồng ý cấp cho Colab toàn quyền truy cập vào tất cả dữ liệu của bạn trên Google Drive và trong GCP. Nếu bạn cho phép truy cập, chỉ sổ tay hiện tại mới có quyền truy cập, và chỉ cho đến khi thời gian chạy Colab hết hạn. Rõ ràng, bạn chỉ nên chấp nhận điều này nếu bạn tin tưởng mã trong sổ tay.

Phán đoán và công thức toán:

Trong phần này của văn bản, không có công thức toán học tường minh nào xuất hiện. Tuy nhiên, các khái niệm liên quan đến “hiệu suất” và “kích thước tải trọng” khi so sánh REST và gRPC vẫn có thể được liên hệ với các phép đo định lượng:

5. **Hiệu suất Serialization/Deserialization (REST vs gRPC):**
    - **REST (JSON):** Chuyển đổi số float thành chuỗi và ngược lại. Quá trình này có thể tốn thời gian và tài nguyên CPU. Ví dụ: số float `0.123456789` có thể được biểu diễn dưới dạng chuỗi "0.123456789", chiếm nhiều byte hơn so với biểu diễn nhị phân.
    - **gRPC (Protocol Buffers):** Sử dụng định dạng nhị phân nhỏ gọn hơn. Việc chuyển đổi từ dữ liệu gốc sang định dạng protobuf và ngược lại nhanh hơn và hiệu quả hơn về không gian.
6. **Kích thước tải trọng (Payload Size):**
    - **REST (JSON):** Văn bản dài dòng, mỗi số float có thể chiếm hơn 15 ký tự (cho float 32-bit). Một float 32-bit (single-precision floating-point number) trong máy tính chiếm 4 byte = 32 bit. Nếu biểu diễn chuỗi của nó chiếm 15 ký tự, và mỗi ký tự là 1 byte (ASCII/UTF-8 đơn giản), thì sẽ tốn 15 byte. Tức là tăng kích thước gấp $15/4 = 3.75$ lần. Nếu "hơn 120 bit" được nhắc đến, có thể là 15 ký tự * 8 bit/ký tự = 120 bit. Điều này dẫn đến công thức so sánh hiệu quả truyền tải: $$$$$$\ \\ \\ \\ $$ $$ \ \\ \\$$ $$$$Sự khác biệt về kích thước này ảnh hưởng trực tiếp đến băng thông sử dụng và độ trễ truyền dữ liệu.

Mặc dù không có công thức toán học phức tạp nào, nhưng những thảo luận trên ngụ ý rằng việc tối ưu hóa hiệu suất (thời gian tính toán và băng thông) là rất quan trọng khi triển khai mô hình học máy ở quy mô lớn.

Xác thực và Cấp quyền trên GCP

Nói chung, việc sử dụng xác thực OAuth 2.0 chỉ được khuyến nghị khi một ứng dụng cần truy cập dữ liệu cá nhân hoặc tài nguyên của người dùng từ một ứng dụng khác, thay mặt người dùng. Ví dụ, một số ứng dụng cho phép người dùng lưu dữ liệu vào Google Drive của họ, nhưng để làm được điều đó, ứng dụng trước tiên cần người dùng xác thực với Google và cho phép truy cập vào Google Drive. Nói chung, ứng dụng sẽ chỉ yêu cầu mức độ truy cập mà nó cần; nó sẽ không phải là quyền truy cập không giới hạn: ví dụ, ứng dụng sẽ chỉ yêu cầu truy cập vào Google Drive, không phải Gmail hay bất kỳ dịch vụ Google nào khác. Hơn nữa, quyền ủy quyền thường hết hạn sau một thời gian, và nó luôn có thể bị thu hồi.

Khi một ứng dụng cần truy cập một dịch vụ trên GCP thay mặt cho chính nó, không phải thay mặt người dùng, thì nó thường nên sử dụng tài khoản dịch vụ (service account). Ví dụ, nếu bạn xây dựng một trang web cần gửi yêu cầu dự đoán đến một điểm cuối của Vertex AI, thì trang web đó sẽ truy cập dịch vụ thay mặt cho chính nó. Không có dữ liệu hoặc tài nguyên nào mà nó cần truy cập trong tài khoản Google của người dùng. Trên thực tế, nhiều người dùng của trang web thậm chí sẽ không có tài khoản Google. Đối với kịch bản này, bạn trước tiên cần tạo một tài khoản dịch vụ. Chọn "IAM and admin → Service accounts" trong menu điều hướng ☰ của bảng điều khiển GCP (hoặc sử dụng hộp tìm kiếm), sau đó nhấp vào + CREATE SERVICE ACCOUNT, điền vào trang đầu tiên của biểu mẫu (tên tài khoản dịch vụ, ID, mô tả), và nhấp vào CREATE AND CONTINUE. Tiếp theo, bạn phải cấp cho tài khoản này một số quyền truy cập. Chọn vai trò "Vertex AI user": điều này sẽ cho phép tài khoản dịch vụ thực hiện dự đoán và sử dụng các dịch vụ Vertex AI khác, nhưng không có gì

khác. Nhấp vào CONTINUE. Bạn có thể tùy chọn cấp cho một số người dùng quyền truy cập vào tài khoản dịch vụ: điều này hữu ích khi tài khoản người dùng GCP của bạn là một phần của một tổ chức và bạn muốn ủy quyền cho những người dùng khác trong tổ chức triển khai các ứng dụng sẽ dựa trên tài khoản dịch vụ này, hoặc để quản lý tài khoản dịch vụ đó. Tiếp theo, nhấp vào DONE.

Khi bạn đã tạo một tài khoản dịch vụ, ứng dụng của bạn phải xác thực với tư cách là tài khoản dịch vụ đó. Có một số cách để làm điều đó. Nếu ứng dụng của bạn được lưu trữ trên GCP — ví dụ, nếu bạn đang viết mã một trang web được lưu trữ trên Google Compute Engine — thì giải pháp đơn giản nhất và an toàn nhất là đính kèm tài khoản dịch vụ vào tài nguyên GCP lưu trữ trang web của bạn, chẳng hạn như một phiên bản máy ảo (VM instance) hoặc dịch vụ Google App Engine. Điều này có thể được thực hiện khi tạo tài nguyên GCP, bằng cách chọn tài khoản dịch vụ trong phần “Identity and API access”. Một số tài nguyên, chẳng hạn như phiên bản máy ảo, cũng cho phép bạn đính kèm tài khoản dịch vụ sau khi phiên bản máy ảo được tạo: bạn phải dừng nó và chỉnh sửa cài đặt của nó. Trong mọi trường hợp, khi một tài khoản dịch vụ được đính kèm vào một phiên bản máy ảo, hoặc bất kỳ tài nguyên GCP nào khác đang chạy mã của bạn, các thư viện khách hàng của GCP (sẽ được thảo luận ngay sau đây) sẽ tự động xác thực với tư cách là tài khoản dịch vụ đã chọn, không cần thêm bước nào.

Nếu ứng dụng của bạn được lưu trữ bằng Kubernetes, thì bạn nên sử dụng dịch vụ Workload Identity của Google để ánh xạ tài khoản dịch vụ phù hợp với từng tài khoản dịch vụ Kubernetes. Nếu ứng dụng của bạn không được lưu trữ trên GCP — ví dụ, nếu bạn chỉ đang chạy sổ tay Jupyter trên máy của mình — thì bạn có thể sử dụng dịch vụ Workload Identity Federation (đây là tùy chọn an toàn nhất nhưng khó nhất), hoặc chỉ cần tạo khóa truy cập cho tài khoản dịch vụ của bạn, lưu nó vào một tệp JSON, và trỏ biến môi trường `GOOGLE_APPLICATION_CREDENTIALS` đến nó để ứng dụng khách của bạn có thể truy cập. Bạn có thể quản lý các khóa truy cập bằng cách nhấp vào tài khoản dịch vụ bạn vừa tạo, và sau đó mở tab KEYS. Đảm bảo giữ tệp khóa bí mật: nó giống như một mật khẩu cho tài khoản dịch vụ. Để biết thêm chi tiết về việc thiết lập xác thực và cấp quyền để ứng dụng của bạn có thể truy cập các dịch vụ GCP, hãy xem tài liệu.

Bây giờ hãy tạo một nhóm lưu trữ Google Cloud Storage (GCS bucket) để lưu trữ các SavedModels của chúng ta (một GCS bucket là một vùng chứa cho dữ liệu của bạn). Để làm điều này, chúng ta sẽ sử dụng thư viện `google-cloud-storage`, đã được cài đặt sẵn trong Colab. Chúng ta đầu tiên tạo một đối tượng `Client`, sẽ đóng vai trò là giao diện với GCS, sau đó chúng ta sử dụng nó để tạo nhóm:

```
from google.cloud import storage

project_id = "my_project" # thay đổi cái này thành ID dự án của bạn
bucket_name = "my_bucket" # thay đổi cái này thành một tên nhóm duy nhất
location = "us-central1"

storage_client = storage.Client(project=project_id)
bucket = storage_client.create_bucket(bucket_name, location=location)
```

GCS sử dụng một không gian tên duy nhất trên toàn thế giới cho các nhóm, vì vậy các tên đơn giản như “machine-learning” rất có thể sẽ không khả dụng. Đảm bảo tên nhóm tuân thủ các quy ước đặt tên DNS, vì nó có thể được sử dụng trong các bản ghi DNS. Hơn nữa, tên nhóm là công khai, vì vậy đừng đặt bất cứ điều gì riêng tư vào tên. Thường thì người ta sử dụng tên miền của bạn, tên công ty của bạn, hoặc ID dự án của bạn làm tiền tố để đảm bảo tính duy nhất, hoặc đơn giản là sử dụng một số ngẫu nhiên làm một phần của tên.

Bạn có thể thay đổi khu vực nếu muốn, nhưng hãy chắc chắn chọn một khu vực hỗ trợ GPU. Ngoài ra, bạn có thể muốn xem xét thực tế là giá cả thay đổi rất nhiều giữa các khu vực, một số khu vực sản xuất nhiều CO_2 hơn những khu vực khác, một số khu vực không hỗ trợ tất cả các dịch vụ, và việc sử dụng một nhóm một khu vực cải thiện hiệu suất. Xem danh sách các khu vực của Google Cloud và tài liệu của Vertex AI về các vị trí để biết thêm chi tiết. Nếu bạn không chắc chắn, tốt nhất có thể là gắn bó với “us-central1”.

Tiếp theo, hãy tải thư mục `my_mnist_model` lên nhóm mới. Các tệp trong GCS được gọi là blob (hoặc đối tượng), và về bản chất, tất cả chúng đều được đặt trong nhóm mà không có bất kỳ cấu trúc thư mục nào. Tên blob có thể là các chuỗi Unicode tùy ý, và chúng thậm chí có thể chứa dấu gạch chéo (/ forward slashes). Bảng điều khiển GCP và các công cụ khác sử dụng các dấu gạch chéo này để tạo ra ảo giác rằng có các thư mục. Vì vậy, khi chúng ta tải thư mục `my_mnist_model` lên, chúng ta chỉ quan tâm đến các tệp, không phải các thư mục:

```
from pathlib import Path
# Giả định bucket đã được tạo

def upload_directory(bucket, dirpath):
    dirpath = Path(dirpath)
    for filepath in dirpath.glob("**/*"):
        if filepath.is_file():
            # Tạo đường dẫn tương đối để blob_name không bao gồm thư mục gốc
của model
            blob_name = filepath.relative_to(dirpath.parent).as_posix()
            blob = bucket.blob(blob_name)
            blob.upload_from_filename(filepath)

# Thay thế "my_mnist_model" bằng đường dẫn thực tế đến thư mục mô hình của
bạn
upload_directory(bucket, "my_mnist_model")
```

Hàm này hoạt động tốt bây giờ, nhưng nó sẽ rất chậm nếu có nhiều tệp để tải lên. Không quá khó để tăng tốc nó lên rất nhiều bằng cách đa luồng hóa nó (xem sổ tay để biết triển khai). Ngoài ra, nếu bạn có Google Cloud CLI, thì bạn có thể sử dụng lệnh sau thay thế:

```
!gsutil -m cp -r my_mnist_model gs://{bucket_name}/
```

Tiếp theo, hãy thông báo cho Vertex AI về mô hình MNIST của chúng ta. Để giao tiếp với Vertex AI, chúng ta có thể sử dụng thư viện `google-cloud-aiplatform` (nó vẫn sử dụng tên AI Platform cũ thay vì Vertex AI). Nó không được cài đặt sẵn trong Colab, vì vậy chúng ta

cần cài đặt nó. Sau đó, chúng ta có thể import thư viện và khởi tạo nó — chỉ để chỉ định một số giá trị mặc định cho ID dự án và vị trí — sau đó chúng ta có thể tạo một mô hình Vertex AI mới: chúng ta chỉ định tên hiển thị, đường dẫn GCS đến mô hình của chúng ta (trong trường hợp này là phiên bản 0001), và URL của container Docker chúng ta muốn Vertex AI sử dụng để chạy mô hình này. Nếu bạn truy cập URL đó và điều hướng lên một cấp, bạn sẽ tìm thấy các container khác mà bạn có thể sử dụng. Cái này hỗ trợ TensorFlow 2.8 với GPU:

```
# Cài đặt thư viện nếu chưa có
# %pip install google-cloud-aiplatform

from google.cloud import aiplatform

server_image = "gcr.io/cloud-aiplatform/prediction/tf2-gpu.2-8:latest"

aiplatform.init(project=project_id, location=location) # Sử dụng project_id
và location đã định nghĩa ở trên
mnist_model = aiplatform.Model.upload(
    display_name="mnist",
    artifact_uri=f"gs://{bucket_name}/my_mnist_model/0001",
    serving_container_image_uri=server_image,
)
```

Bây giờ hãy triển khai mô hình này để chúng ta có thể truy vấn nó qua API gRPC hoặc REST để thực hiện dự đoán. Để làm điều này, trước tiên chúng ta cần tạo một điểm cuối (endpoint). Đây là nơi các ứng dụng khách hàng kết nối khi chúng muốn truy cập một dịch vụ. Sau đó, chúng ta cần triển khai mô hình của chúng ta đến điểm cuối này:

```
# Giả định mnist_model đã được tải lên
endpoint = aiplatform.Endpoint.create(display_name="mnist-endpoint")

endpoint.deploy(
    mnist_model,
    min_replica_count=1,
    max_replica_count=5,
    machine_type="n1-standard-4",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=1
)
```

Đoạn mã này có thể mất vài phút để chạy, vì Vertex AI cần thiết lập một máy ảo. Trong ví dụ này, chúng ta sử dụng một máy khá cơ bản loại `n1-standard-4` (xem https://homl.info/machinetypes để biết các loại khác). Chúng ta cũng sử dụng một GPU cơ bản loại `NVIDIA_TESLA_K80` (xem https://homl.info/accelerators để biết các loại khác). Nếu bạn chọn một khu vực khác ngoài “us-central1”, thì bạn có thể cần thay đổi loại máy hoặc loại bộ tăng tốc thành các giá trị được hỗ trợ trong khu vực đó (ví dụ, không phải tất cả các khu vực đều có GPU Nvidia Tesla K80).

Vertex AI ban đầu sẽ tạo ra số lượng nút tính toán tối thiểu (chỉ một trong trường hợp này), và bất cứ khi nào số lượng truy vấn mỗi giây trở nên quá cao, nó sẽ tạo ra thêm các nút (lên đến số lượng tối đa bạn đã định nghĩa, năm trong trường hợp này) và sẽ cân bằng tải các truy vấn giữa chúng. Nếu tốc độ QPS giảm trong một thời gian, Vertex AI sẽ tự động dừng các nút tính toán bổ sung.

Chi phí do đó liên quan trực tiếp đến tải, cũng như loại máy và bộ tăng tốc bạn đã chọn và lượng dữ liệu bạn lưu trữ trên GCS. Mô hình định giá này rất phù hợp cho người dùng không thường xuyên và cho các dịch vụ có mức sử dụng tăng đột biến quan trọng. Nó cũng lý tưởng cho các công ty khởi nghiệp: giá vẫn thấp cho đến khi công ty khởi nghiệp thực sự bắt đầu hoạt động.

Chúc mừng, bạn đã triển khai mô hình đầu tiên của mình lên đám mây! Bây giờ hãy truy vấn dịch vụ dự đoán này:

```
# Giả định endpoint đã được deploy và X_new đã được chuẩn bị
response = endpoint.predict(instances=X_new.tolist())
```

Chúng ta đầu tiên cần chuyển đổi các ảnh chúng ta muốn phân loại thành một danh sách Python, như chúng ta đã làm trước đó khi gửi yêu cầu đến TF Serving bằng API REST. Đối tượng `response` chứa các dự đoán, được biểu diễn dưới dạng một danh sách các danh sách số float trong Python. Hãy làm tròn chúng đến hai chữ số thập phân và chuyển đổi chúng thành một mảng NumPy:

```
>>> import numpy as np

>>> np.round(response.predictions, 2)
array([[0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 1.  , 0.  , 0.  ],
       [0.  , 0.  , 0.99, 0.01, 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ],
       [0.  , 0.97, 0.01, 0.  , 0.  , 0.  , 0.  , 0.01, 0.  , 0.  ]])
```

Vâng! Chúng ta nhận được chính xác các dự đoán giống như trước đó. Bây giờ chúng ta có một dịch vụ dự đoán tuyệt vời đang chạy trên đám mây mà chúng ta có thể truy vấn từ bất cứ đâu một cách an toàn, và nó có thể tự động mở rộng lên hoặc xuống tùy thuộc vào số lượng QPS. Khi bạn hoàn thành việc sử dụng điểm cuối, đừng quên xóa nó, để tránh phải trả tiền vô ích:

```
endpoint.undeploy_all() # gỡ bỏ tất cả các mô hình khỏi điểm cuối
endpoint.delete()
```

Bây giờ hãy xem cách chạy một công việc trên Vertex AI để thực hiện dự đoán trên một lô dữ liệu có khả năng rất lớn.

Chạy các Công việc Dự đoán Hàng loạt trên Vertex AI

Nếu chúng ta có một số lượng lớn các dự đoán cần thực hiện, thay vì gọi dịch vụ dự đoán của chúng ta nhiều lần, chúng ta có thể yêu cầu Vertex AI chạy một công việc dự đoán hàng loạt cho chúng ta. Việc này không yêu cầu điểm cuối (endpoint), chỉ cần một mô hình. Ví

dụ, hãy chạy một công việc dự đoán trên 100 ảnh đầu tiên của tập dữ liệu kiểm tra, sử dụng mô hình MNIST của chúng ta. Để làm điều này, trước tiên chúng ta cần chuẩn bị lô dữ liệu và tải nó lên GCS. Một cách để làm điều này là tạo một tệp chứa một instance mỗi dòng, mỗi instance được định dạng dưới dạng giá trị JSON — định dạng này được gọi là JSON Lines — sau đó truyền tệp này cho Vertex AI. Vì vậy, hãy tạo một tệp JSON Lines trong một thư mục mới, sau đó tải thư mục này lên GCS:

```
from pathlib import Path
import json
# Giả định X_test đã được định nghĩa, ví dụ: X_test =
tf.keras.datasets.mnist.load_data()[1][0]
# Giả định bucket đã được tạo và hàm upload_directory đã được định nghĩa

batch_path = Path("my_mnist_batch")
batch_path.mkdir(exist_ok=True)

with open(batch_path / "my_mnist_batch.jsonl", "w") as jsonl_file:
    for image in X_test[:100].tolist():
        jsonl_file.write(json.dumps(image))
        jsonl_file.write("\n")

upload_directory(bucket, batch_path)
```

Bây giờ chúng ta đã sẵn sàng khởi chạy công việc dự đoán, chỉ định tên công việc, loại và số lượng máy và bộ tăng tốc cần sử dụng, đường dẫn GCS đến tệp JSON Lines mà chúng ta vừa tạo, và đường dẫn đến thư mục GCS nơi Vertex AI sẽ lưu các dự đoán của mô hình:

```
# Giả định mnist_model đã được tải lên Vertex AI trước đó
# Giả định bucket_name và batch_path đã được định nghĩa

batch_prediction_job = mnist_model.batch_predict(
    job_display_name="my_batch_prediction_job",
    machine_type="n1-standard-4",
    starting_replica_count=1,
    max_replica_count=5,
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=1,

gcs_source=[f"gs://{bucket_name}/{batch_path.name}/my_mnist_batch.jsonl"],
    gcs_destination_prefix=f"gs://{bucket_name}/my_mnist_predictions/",
    sync=True # đặt thành False nếu bạn không muốn chờ hoàn thành
)
```

Việc này sẽ mất vài phút, chủ yếu là để tạo các nút tính toán trên Vertex AI. Khi lệnh này hoàn tất, các dự đoán sẽ có sẵn trong một tập hợp các tệp có tên giống như `prediction.results-00001-of-00002`.

Các tệp này sử dụng định dạng JSON Lines theo mặc định, và mỗi giá trị là một dictionary chứa một instance và dự đoán tương ứng của nó (tức là 10 xác suất). Các instance được

liệt kê theo cùng thứ tự với các đầu vào. Công việc cũng xuất ra các tệp prediction-errors*, có thể hữu ích cho việc gỡ lỗi nếu có gì đó không ổn. Chúng ta có thể lặp qua tất cả các tệp đầu ra này bằng cách sử dụng batch_prediction_job.iter_outputs(), vì vậy hãy đi qua tất cả các dự đoán và lưu trữ chúng trong một mảng y_probas:

```
import numpy as np
import json
# Giả định batch_prediction_job đã hoàn thành

y_probas = []

for blob in batch_prediction_job.iter_outputs():
    if "prediction.results" in blob.name:
        for line in blob.download_as_text().splitlines():
            y_proba = json.loads(line)["prediction"]
            y_probas.append(y_proba)
```

Bây giờ hãy xem các dự đoán này tốt đến mức nào:

```
# Giả định y_test đã được định nghĩa từ trước, ví dụ: y_test =
tf.keras.datasets.mnist.load_data()[1][1]
>>> y_pred = np.argmax(y_probas, axis=1)

>>> accuracy = np.sum(y_pred == y_test[:100]) / 100
0.98
```

Tuyệt vời, độ chính xác 98%!

Định dạng JSON Lines là mặc định, nhưng khi xử lý các instance lớn như ảnh, nó quá dài dòng. May mắn thay, phương thức batch_predict() chấp nhận một đối số instances_format cho phép bạn chọn một định dạng khác nếu muốn. Nó mặc định là "jsonl", nhưng bạn có thể thay đổi nó thành "csv", "tf-record", "tf-record-gzip", "bigquery", hoặc "file-list".

Nếu bạn đặt nó thành "file-list", thì đối số gcs_source sẽ trỏ đến một tệp văn bản chứa một đường dẫn tệp đầu vào mỗi dòng; ví dụ, trỏ đến các tệp ảnh PNG. Vertex AI sẽ đọc các tệp này dưới dạng nhị phân, mã hóa chúng bằng Base64, và truyền các chuỗi byte kết quả cho mô hình. Điều này có nghĩa là bạn phải thêm một lớp tiền xử lý trong mô hình của mình để phân tích cú pháp các chuỗi Base64, bằng cách sử dụng tf.io.decode_base64(). Nếu các tệp là ảnh, bạn phải phân tích cú pháp kết quả bằng một hàm như tf.io.decode_image() hoặc tf.io.decode_png(), như đã thảo luận trong Chương 13.

Khi bạn đã hoàn thành việc sử dụng mô hình, bạn có thể xóa nó nếu muốn, bằng cách chạy mnist_model.delete(). Bạn cũng có thể xóa các thư mục bạn đã tạo trong GCS bucket của mình, tùy chọn xóa chính bucket (nếu nó trống), và công việc dự đoán hàng loạt:

```
# Giả định bucket và batch_prediction_job đã được định nghĩa

for prefix in ["my_mnist_model/", "my_mnist_batch/",
```

```
"my_mnist_predictions/"]:
    blobs = bucket.list_blobs(prefix=prefix)
    for blob in blobs:
        blob.delete()

# Chỉ xóa bucket nếu nó trống
# bucket.delete()

batch_prediction_job.delete()
```

Bây giờ bạn đã biết cách triển khai một mô hình lên Vertex AI, tạo dịch vụ dự đoán và chạy các công việc dự đoán hàng loạt. Nhưng nếu bạn muốn triển khai mô hình của mình vào một ứng dụng di động thì sao? Hoặc vào một thiết bị nhúng, chẳng hạn như hệ thống điều khiển nhiệt độ, thiết bị theo dõi thể dục, hay một chiếc xe tự lái?

Triển khai Mô hình lên Thiết bị Di động hoặc Thiết bị Nhúng

Các mô hình học máy không chỉ giới hạn ở việc chạy trên các máy chủ tập trung lớn với nhiều GPU: chúng có thể chạy gần nguồn dữ liệu hơn (điều này được gọi là điện toán biên - edge computing), ví dụ trong thiết bị di động của người dùng hoặc trong một thiết bị nhúng. Có nhiều lợi ích khi phân tán các phép tính và di chuyển chúng đến biên: nó cho phép thiết bị trở nên thông minh ngay cả khi không được kết nối internet, nó giảm độ trễ bằng cách không phải gửi dữ liệu đến máy chủ từ xa và giảm tải trên các máy chủ, và nó có thể cải thiện quyền riêng tư, vì dữ liệu của người dùng có thể ở lại trên thiết bị.

Tuy nhiên, việc triển khai mô hình đến biên cũng có những nhược điểm. Tài nguyên tính toán của thiết bị thường rất nhỏ so với một máy chủ nhiều GPU mạnh mẽ. Một mô hình lớn có thể không vừa với thiết bị, nó có thể sử dụng quá nhiều RAM và CPU, và có thể mất quá nhiều thời gian để tải xuống. Kết quả là, ứng dụng có thể trở nên không phản hồi, và thiết bị có thể nóng lên và nhanh chóng hết pin. Để tránh tất cả những điều này, bạn cần tạo một mô hình nhẹ và hiệu quả, mà không phải hy sinh quá nhiều độ chính xác. Thư viện TFLite cung cấp một số công cụ để giúp bạn triển khai mô hình của mình đến biên, với ba mục tiêu chính:

- **Giảm kích thước mô hình**, để rút ngắn thời gian tải xuống và giảm sử dụng RAM.
- **Giảm lượng tính toán cần thiết cho mỗi dự đoán**, để giảm độ trễ, sử dụng pin và nhiệt độ.
- **Thích nghi mô hình với các ràng buộc cụ thể của thiết bị.**

Để giảm kích thước mô hình, bộ chuyển đổi mô hình của TFLite có thể lấy một SavedModel và nén nó thành một định dạng nhẹ hơn nhiều dựa trên FlatBuffers. Đây là một thư viện tuần tự hóa đa nền tảng hiệu quả (hơi giống protocol buffers) ban đầu được Google tạo ra cho trò chơi. Nó được thiết kế để bạn có thể tải FlatBuffers trực tiếp vào RAM mà không cần bất kỳ tiền xử lý nào: điều này giúp giảm thời gian tải và dấu vết bộ nhớ.

Khi mô hình được tải vào một thiết bị di động hoặc thiết bị nhúng, trình thông dịch TFLite sẽ thực thi nó để đưa ra các dự đoán. Đây là cách bạn có thể chuyển đổi một SavedModel thành FlatBuffer và lưu nó vào tệp `.tflite`:

```
import tensorflow as tf
# Giả định model_path đã được định nghĩa từ bước xuất SavedModel

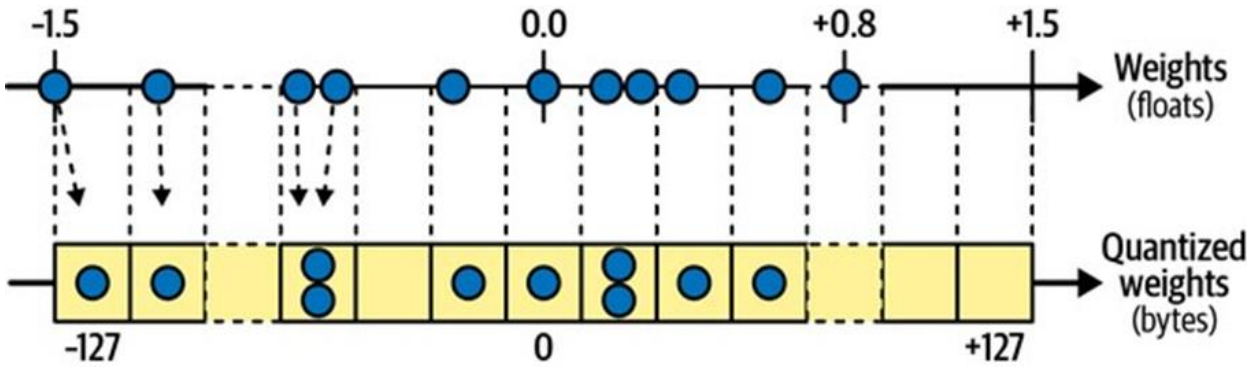
converter = tf.lite.TFLiteConverter.from_saved_model(str(model_path))
tflite_model = converter.convert()

with open("my_converted_savedmodel.tflite", "wb") as f:
    f.write(tflite_model)
```

Bộ chuyển đổi cũng tối ưu hóa mô hình, cả để thu nhỏ nó và để giảm độ trễ. Nó loại bỏ tất cả các hoạt động không cần thiết để đưa ra dự đoán (chẳng hạn như các hoạt động huấn luyện), và nó tối ưu hóa các phép tính bất cứ khi nào có thể; ví dụ, $3timesa + 4timesa + 5timesa$ sẽ được chuyển đổi thành $12timesa$. Ngoài ra, nó cố gắng hợp nhất các hoạt động bất cứ khi nào có thể. Ví dụ, nếu có thể, các lớp chuẩn hóa hàng loạt (batch normalization layers) cuối cùng được gộp vào các phép cộng và nhân của lớp trước đó. Để có ý tưởng hay về mức độ TFLite có thể tối ưu hóa một mô hình, hãy tải xuống một trong các mô hình TFLite đã được huấn luyện trước, chẳng hạn như Inception_V1_quant (nhấp vào `tflite&pb`), giải nén tệp lưu trữ, sau đó mở công cụ trực quan hóa đồ thị Netron tuyệt vời và tải tệp `.pb` lên để xem mô hình gốc. Đó là một đồ thị lớn, phức tạp, phải không? Tiếp theo, mở mô hình `optimized.tflite` và chiêm ngưỡng vẻ đẹp của nó!

Một cách khác bạn có thể giảm kích thước mô hình — ngoài việc đơn giản là sử dụng các kiến trúc mạng nơ-ron nhỏ hơn — là bằng cách sử dụng độ rộng bit nhỏ hơn: ví dụ, nếu bạn sử dụng half-floats (16 bit) thay vì regular floats (32 bit), kích thước mô hình sẽ giảm đi 2 lần, với chi phí là sự sụt giảm độ chính xác (thường là nhỏ). Hơn nữa, quá trình huấn luyện sẽ nhanh hơn, và bạn sẽ sử dụng khoảng một nửa lượng RAM GPU.

Bộ chuyển đổi của TFLite có thể đi xa hơn nữa, bằng cách lượng tử hóa các trọng số mô hình xuống các số nguyên 8-bit, dấu phẩy tĩnh! Điều này dẫn đến giảm kích thước gấp 4 lần so với việc sử dụng số float 32-bit. Cách tiếp cận đơn giản nhất được gọi là lượng tử hóa sau huấn luyện (post-training quantization): nó chỉ lượng tử hóa các trọng số sau khi huấn luyện, sử dụng một kỹ thuật lượng tử hóa đối xứng khá cơ bản nhưng hiệu quả. Nó tìm giá trị tuyệt đối tối đa của trọng số, m , sau đó nó ánh xạ dải số float từ $-m$ đến $+m$ sang dải số nguyên (fixed-point) từ -127 đến $+127$. Ví dụ, nếu các trọng số nằm trong khoảng từ -1.5 đến $+0.8$, thì các byte $-127,0,textva` + 127$ sẽ tương ứng với các số float $-1.5,0.0,textva` + 1.5$, tương ứng (xem Hình 19-5). Lưu ý rằng 0.0 luôn ánh xạ đến 0 khi sử dụng lượng tử hóa đối xứng. Cũng lưu ý rằng các giá trị byte từ $+68$ đến $+127$ sẽ không được sử dụng trong ví dụ này, vì chúng ánh xạ đến các số float lớn hơn $+0.8$.



Hình 19-5. Từ số float 32-bit sang số nguyên 8-bit, sử dụng lượng tử hóa đối xứng.

Để thực hiện lượng tử hóa sau huấn luyện này, bạn chỉ cần thêm `DEFAULT` vào danh sách các tối ưu hóa của bộ chuyển đổi trước khi gọi phương thức `convert()`:

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

Kỹ thuật này giảm đáng kể kích thước mô hình, giúp tải xuống nhanh hơn nhiều và sử dụng ít không gian lưu trữ hơn. Khi chạy, các trọng số đã được lượng tử hóa sẽ được chuyển đổi trở lại thành số float trước khi chúng được sử dụng. Các số float được khôi phục này không hoàn toàn giống với các số float gốc, nhưng chúng không quá khác biệt, vì vậy việc mất độ chính xác thường có thể chấp nhận được. Để tránh tính toán lại các giá trị float mọi lúc, điều này sẽ làm chậm mô hình nghiêm trọng, TFLite lưu trữ chúng: thật không may, điều này có nghĩa là kỹ thuật này không làm giảm việc sử dụng RAM, và nó cũng không làm tăng tốc mô hình. Nó chủ yếu hữu ích để giảm kích thước ứng dụng.

Cách hiệu quả nhất để giảm độ trễ và tiêu thụ điện năng là cũng lượng tử hóa các kích hoạt (activations) để các phép tính có thể được thực hiện hoàn toàn bằng số nguyên, mà không cần bất kỳ phép toán dấu phẩy động nào. Ngay cả khi sử dụng cùng độ rộng bit (ví dụ: số nguyên 32-bit thay vì số float 32-bit), các phép tính số nguyên sử dụng ít chu kỳ CPU hơn, tiêu thụ ít năng lượng hơn và tạo ra ít nhiệt hơn. Và nếu bạn cũng giảm độ rộng bit (ví dụ: xuống số nguyên 8-bit), bạn có thể nhận được tốc độ tăng đáng kể. Hơn nữa, một số thiết bị tăng tốc mạng nơ-ron — chẳng hạn như Edge TPU của Google — chỉ có thể xử lý số nguyên, vì vậy việc lượng tử hóa hoàn toàn cả trọng số và kích hoạt là bắt buộc. Điều này có thể được thực hiện sau huấn luyện; nó yêu cầu một bước hiệu chuẩn để tìm giá trị tuyệt đối tối đa của các kích hoạt, vì vậy bạn cần cung cấp một mẫu dữ liệu huấn luyện đại diện cho TFLite (nó không cần phải quá lớn), và nó sẽ xử lý dữ liệu thông qua mô hình và đo lường các thống kê kích hoạt cần thiết cho việc lượng tử hóa. Bước này thường nhanh.

Vấn đề chính với lượng tử hóa là nó làm mất một chút độ chính xác: nó tương tự như việc thêm nhiễu vào trọng số và kích hoạt. Nếu việc mất độ chính xác quá nghiêm trọng, thì bạn có thể cần sử dụng **lượng tử hóa có nhận thức huấn luyện (quantization-aware training)**. Điều này có nghĩa là thêm các phép toán lượng tử hóa giả vào mô hình để nó có thể học cách bỏ qua nhiễu lượng tử hóa trong quá trình huấn luyện; các trọng số cuối cùng sau đó sẽ mạnh mẽ hơn đối với lượng tử hóa. Hơn nữa, bước hiệu chuẩn có thể được tự động hóa trong quá trình huấn luyện, giúp đơn giản hóa toàn bộ quá trình.

Tôi đã giải thích các khái niệm cốt lõi của TFLite, nhưng việc đi sâu vào mã hóa một ứng dụng di động hoặc thiết bị nhúng sẽ đòi hỏi một cuốn sách chuyên biệt. May mắn thay, một số cuốn đã tồn tại: nếu bạn muốn tìm hiểu thêm về xây dựng ứng dụng TensorFlow cho thiết bị di động và thiết bị nhúng, hãy xem các cuốn sách của O'Reilly là *TinyML: Machine Learning with TensorFlow on Arduino and Ultra-Low Power Micro-Controllers* của Pete Warden (cựu trưởng nhóm TFLite) và Daniel Situnayake, và *AI and Machine Learning for On-Device Development* của Laurence Moroney.

Bây giờ nếu bạn muốn sử dụng mô hình của mình trong một trang web, chạy trực tiếp trong trình duyệt của người dùng thì sao?

Công thức Toán (Phán đoán):

7. **Tính độ chính xác (Accuracy):** Độ chính xác được tính bằng tỷ lệ số dự đoán đúng trên tổng số dự đoán. $$$$$$\ = \ = \ $$ $$$$Trong đoạn mã: `accuracy = np.sum(y_pred == y_test[:100]) / 100` Ở đây, $N = 100$ (100 ảnh đầu tiên), $y_predicted, i$ là dự đoán của mô hình cho ảnh thứ i , và y_true, i là nhãn thực của ảnh thứ i . Hàm $mathbbI(cdot)$ là hàm chỉ báo, trả về 1 nếu điều kiện đúng, và 0 nếu sai.
8. **Tối ưu hóa phép tính trong TFLite:** Khi TFLite tối ưu hóa các phép tính, ví dụ: $3timesa + 4timesa + 5timesa$ được chuyển đổi thành $12timesa$. Đây là một dạng rút gọn đại số cơ bản: $$$$$$3a + 4a + 5a = (3+4+5)a = 12a $$ $$$$Đây là một tối ưu hóa đơn giản nhưng hiệu quả, giảm số lượng phép nhân và cộng cần thiết.
9. **Lượng tử hóa đối xứng (Symmetrical Quantization):** Quá trình ánh xạ giá trị dấu phẩy động (x_float) sang số nguyên 8-bit (x_int8) trong khoảng từ -127 đến $+127$, dựa trên giá trị tuyệt đối tối đa m . Giả sử dải giá trị float là $[-m, +m]$. Để ánh xạ nó sang $[-127, +127]$, chúng ta có thể sử dụng công thức: $$$$$$x\{int8} = \\(\ \ \) $$ $$$$Và khi chuyển đổi ngược lại từ x_int8 sang x_float (để sử dụng tại runtime): $$$$$$x\{float}' = \ \m $$ $$$$Trong ví dụ: nếu trọng số từ -1.5 đến $+0.8$, thì m sẽ là 1.5 (giá trị tuyệt đối lớn nhất).
    - $x_float = -1.5impliesx_int8 = textroundleft(frac -1.51.5times127right) = textround(-127) = -127$
    - $x_float = 0.0impliesx_int8 = textroundleft(frac0.01.5times127right) = textround(0) = 0$
    - $x_float = +1.5impliesx_int8 = textroundleft(frac1.51.5times127right) = textround(127) = +127$ (Lưu ý: ví dụ gốc có +0.8, nhưng để ánh xạ tới +127 thì phải là +1.5 nếu m=1.5) Mục tiêu của lượng tử hóa là giảm kích thước mô hình (ví dụ: từ 32 bit float xuống 8 bit integer, giảm 4 lần kích thước) và tăng tốc độ tính toán (phép tính số nguyên nhanh hơn phép tính số thực), cũng như giảm tiêu thụ năng lượng.

Chạy Mô hình trong Trang Web

Chạy mô hình học máy của bạn ở phía máy khách, trong trình duyệt của người dùng, thay vì ở phía máy chủ có thể hữu ích trong nhiều tình huống, chẳng hạn như:

- Khi ứng dụng web của bạn thường được sử dụng trong các tình huống mà kết nối của người dùng bị gián đoạn hoặc chậm (ví dụ: một trang web dành cho người đi bộ đường dài), vì vậy chạy mô hình trực tiếp ở phía máy khách là cách duy nhất để làm cho trang web của bạn đáng tin cậy.
- Khi bạn cần phản hồi của mô hình nhanh nhất có thể (ví dụ: đối với một trò chơi trực tuyến). Việc loại bỏ nhu cầu truy vấn máy chủ để đưa ra dự đoán chắc chắn sẽ giảm độ trễ và làm cho trang web phản hồi nhanh hơn nhiều.
- Khi dịch vụ web của bạn đưa ra dự đoán dựa trên một số dữ liệu riêng tư của người dùng, và bạn muốn bảo vệ quyền riêng tư của người dùng bằng cách đưa ra dự đoán ở phía máy khách để dữ liệu riêng tư không bao giờ phải rời khỏi máy của người dùng.

Đối với tất cả các kịch bản này, bạn có thể sử dụng thư viện TensorFlow.js (TFJS) JavaScript. Thư viện này có thể tải một mô hình TFLite và đưa ra dự đoán trực tiếp trong trình duyệt của người dùng. Ví dụ, module JavaScript sau đây import thư viện TFJS, tải xuống một mô hình MobileNet đã được huấn luyện trước, và sử dụng mô hình này để phân loại một ảnh và ghi lại các dự đoán. Bạn có thể thử nghiệm với mã tại https://homl.info/tfjscode , sử dụng Glitch.com, một trang web cho phép bạn xây dựng ứng dụng web trong trình duyệt của mình miễn phí; nhấp vào nút PREVIEW ở góc dưới bên phải của trang để xem mã hoạt động:

```
import * as tf from "https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@latest";
import * as mobilenet from "https://cdn.jsdelivr.net/npm/@tensorflow-
models/mobilenet@1.0.0";

const image = document.getElementById("image");

mobilenet.load().then(model => {
    model.classify(image).then(predictions => {
        for (var i = 0; i < predictions.length; i++) {
            let className = predictions[i].className;
            let proba = (predictions[i].probability * 100).toFixed(1);
            console.log(className + " : " + proba + "%");
        }
    });
});
```

Thậm chí có thể biến trang web này thành một ứng dụng web tiến bộ (PWA): đây là một trang web tuân thủ một số tiêu chí cho phép nó được xem trong bất kỳ trình duyệt nào, và thậm chí được cài đặt như một ứng dụng độc lập trên thiết bị di động. Ví dụ, hãy thử truy cập https://homl.info/tfjswpa trên một thiết bị di động: hầu hết các trình duyệt hiện đại sẽ hỏi bạn có muốn thêm “TFJS Demo” vào màn hình chính của bạn hay không. Nếu bạn chấp

nhận, bạn sẽ thấy một biểu tượng mới trong danh sách ứng dụng của mình. Nhấp vào biểu tượng này sẽ tải trang web "TFJS Demo" bên trong cửa sổ riêng của nó, giống như một ứng dụng di động thông thường. Một PWA thậm chí có thể được cấu hình để hoạt động ngoại tuyến, bằng cách sử dụng một service worker: đây là một module JavaScript chạy trong luồng riêng biệt của nó trong trình duyệt và chặn các yêu cầu mạng, cho phép nó lưu trữ tài nguyên để PWA có thể chạy nhanh hơn, hoặc thậm chí hoàn toàn ngoại tuyến. Nó cũng có thể gửi tin nhắn đẩy, chạy các tác vụ trong nền, và nhiều hơn nữa. PWAs cho phép bạn quản lý một cơ sở mã duy nhất cho web và cho thiết bị di động. Chúng cũng giúp dễ dàng đảm bảo rằng tất cả người dùng chạy cùng một phiên bản của ứng dụng của bạn. Bạn có thể thử nghiệm với mã PWA của TFJS Demo này trên Glitch.com tại https://homl.info/wpacode .

TFJS cũng hỗ trợ huấn luyện mô hình trực tiếp trong trình duyệt web của bạn! Và nó thực sự khá nhanh. Nếu máy tính của bạn có card GPU, thì TFJS thường có thể sử dụng nó, ngay cả khi đó không phải là card Nvidia. Thật vậy, TFJS sẽ sử dụng WebGL khi nó khả dụng, và vì các trình duyệt web hiện đại thường hỗ trợ một loạt các card GPU, TFJS thực sự hỗ trợ nhiều loại card GPU hơn TensorFlow thông thường (chỉ hỗ trợ card Nvidia).

Huấn luyện một mô hình trong trình duyệt web của người dùng có thể đặc biệt hữu ích để đảm bảo rằng dữ liệu của người dùng này vẫn riêng tư. Một mô hình có thể được huấn luyện tập trung, và sau đó được tinh chỉnh cục bộ, trong trình duyệt, dựa trên dữ liệu của người dùng đó. Nếu bạn quan tâm đến chủ đề này, hãy tìm hiểu về học liên kết (federated learning). Một lần nữa, để làm rõ chủ đề này sẽ cần cả một cuốn sách. Nếu bạn muốn tìm hiểu thêm về TensorFlow.js, hãy xem các cuốn sách của O'Reilly là *Practical Deep Learning for Cloud, Mobile, and Edge*, của Anirudh Koul et al., hoặc *Learning TensorFlow.js*, của Gant Laborde.

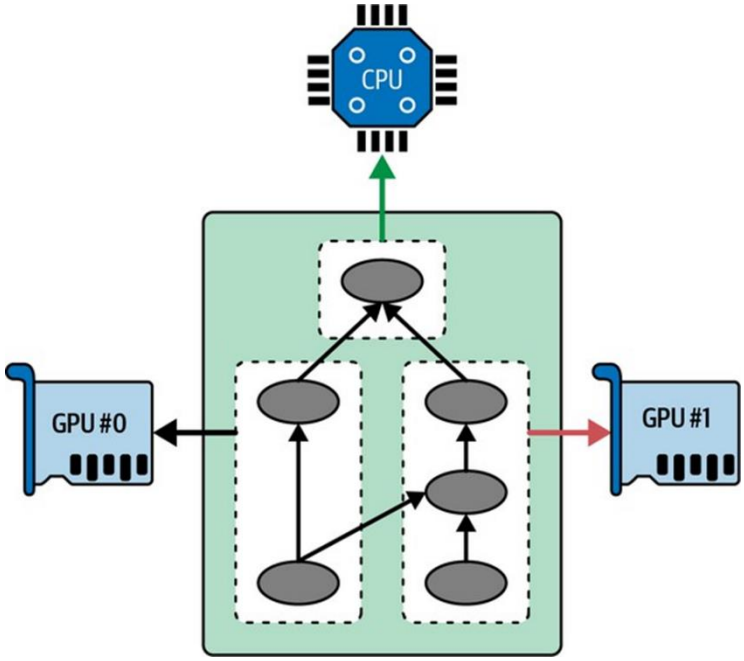
Bây giờ bạn đã thấy cách triển khai các mô hình TensorFlow lên TF Serving, hoặc lên đám mây với Vertex AI, hoặc lên các thiết bị di động và nhúng bằng TFLite, hoặc lên trình duyệt web bằng TFJS, hãy thảo luận về cách sử dụng GPU để tăng tốc tính toán.

Sử dụng GPU để Tăng tốc Tính toán

Trong Chương 11, chúng ta đã xem xét một số kỹ thuật có thể tăng tốc đáng kể quá trình huấn luyện: khởi tạo trọng số tốt hơn, các bộ tối ưu hóa tinh vi, v.v. Nhưng ngay cả với tất cả các kỹ thuật này, việc huấn luyện một mạng nơ-ron lớn trên một máy với một CPU duy nhất có thể mất hàng giờ, hàng ngày, hoặc thậm chí hàng tuần, tùy thuộc vào tác vụ. Nhờ GPU, thời gian huấn luyện này có thể giảm xuống còn vài phút hoặc vài giờ. Điều này không chỉ tiết kiệm một lượng thời gian khổng lồ, mà còn có nghĩa là bạn có thể thử nghiệm với các mô hình khác nhau dễ dàng hơn nhiều, và thường xuyên huấn luyện lại mô hình của bạn trên dữ liệu mới.

Trong các chương trước, chúng ta đã sử dụng các môi trường chạy có hỗ trợ GPU trên Google Colab. Tất cả những gì bạn phải làm để có được điều này là chọn "Change runtime type" từ menu Runtime, và chọn loại bộ tăng tốc GPU; TensorFlow tự động phát hiện GPU và sử dụng nó để tăng tốc tính toán, và mã hoàn toàn giống như khi không có GPU. Sau đó,

trong chương này, bạn đã thấy cách triển khai các mô hình của mình lên Vertex AI trên nhiều nút tính toán có hỗ trợ GPU: đó chỉ là vấn đề chọn đúng ảnh Docker có hỗ trợ GPU khi tạo mô hình Vertex AI, và chọn loại GPU mong muốn khi gọi `endpoint.deploy()`. Nhưng nếu bạn muốn mua GPU của riêng mình thì sao? Và nếu bạn muốn phân tán các phép tính trên CPU và nhiều thiết bị GPU trên một máy duy nhất (xem Hình 19-6)? Đây là điều chúng ta sẽ thảo luận bây giờ, sau đó trong chương này chúng ta sẽ thảo luận về cách phân tán các phép tính trên nhiều máy chủ.



Hình 19-6. Thực thi đồ thị TensorFlow trên nhiều thiết bị song song.

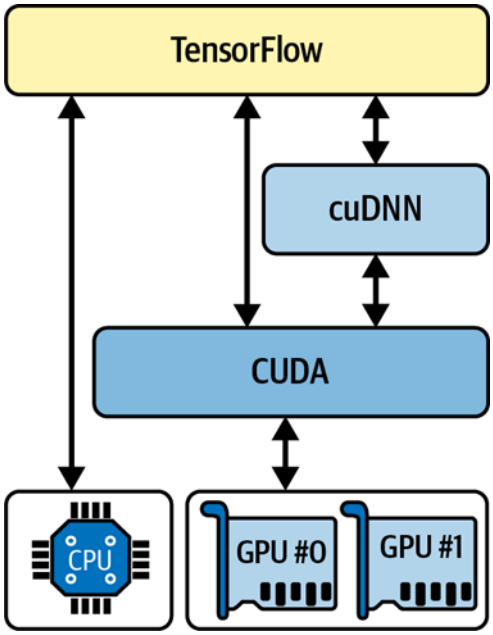
Có được GPU của riêng bạn

Nếu bạn biết rằng mình sẽ sử dụng GPU nhiều và trong thời gian dài, thì việc mua GPU của riêng bạn có thể có ý nghĩa tài chính. Bạn cũng có thể muốn huấn luyện các mô hình của mình cục bộ vì bạn không muốn tải dữ liệu của mình lên đám mây. Hoặc có lẽ bạn chỉ muốn mua một card GPU để chơi game, và bạn cũng muốn sử dụng nó cho học sâu.

Nếu bạn quyết định mua một card GPU, thì hãy dành thời gian để đưa ra lựa chọn đúng đắn. Bạn sẽ cần xem xét lượng RAM bạn sẽ cần cho các tác vụ của mình (ví dụ: thường ít nhất 10 GB cho xử lý ảnh hoặc NLP), băng thông (tức là bạn có thể gửi dữ liệu vào và ra GPU nhanh như thế nào), số lượng lõi, hệ thống làm mát, v.v. Tim Dettmers đã viết một bài đăng blog tuyệt vời để giúp bạn lựa chọn: tôi khuyến khích bạn đọc kỹ nó. Tại thời điểm viết bài này, TensorFlow chỉ hỗ trợ các card Nvidia với CUDA Compute Capability 3.5+ (cũng như TPUs của Google, tất nhiên), nhưng nó có thể mở rộng hỗ trợ cho các nhà sản xuất khác, vì vậy hãy đảm bảo kiểm tra tài liệu của TensorFlow để xem thiết bị nào được hỗ trợ hiện nay.

Nếu bạn chọn một card GPU Nvidia, bạn sẽ cần cài đặt các driver Nvidia và một số thư viện Nvidia phù hợp. Những thứ này bao gồm thư viện CUDA Toolkit (Compute Unified Device

Architecture), cho phép các nhà phát triển sử dụng GPU có hỗ trợ CUDA cho tất cả các loại tính toán (không chỉ tăng tốc đồ họa), và thư viện CUDA Deep Neural Network (cuDNN), một thư viện tăng tốc GPU cho các phép tính DNN phổ biến như các lớp kích hoạt, chuẩn hóa, tích chập xuôi và ngược, và gộp (pooling) (xem Chương 14). cuDNN là một phần của Nvidia's Deep Learning SDK. Lưu ý rằng bạn sẽ cần tạo một tài khoản nhà phát triển Nvidia để tải xuống nó. TensorFlow sử dụng CUDA và cuDNN để điều khiển các card GPU và tăng tốc tính toán (xem Hình 19-7).



Hình 19-7. TensorFlow sử dụng CUDA và cuDNN để điều khiển GPU và tăng cường DNN.

Khi bạn đã cài đặt (các) card GPU và tất cả các driver và thư viện cần thiết, bạn có thể sử dụng lệnh `nvidia-smi` để kiểm tra xem mọi thứ đã được cài đặt đúng cách chưa. Lệnh này liệt kê các card GPU khả dụng, cũng như tất cả các tiến trình đang chạy trên mỗi card. Trong ví dụ này, đó là một card GPU Nvidia Tesla T4 với khoảng 15 GB RAM khả dụng, và hiện không có tiến trình nào đang chạy trên đó:

```
$ nvidia-smi
Sun Apr 10 04:52:10 2022
+-----------------------------------------------------------------------------
---+
| NVIDIA-SMI 460.32.03   Driver Version: 460.32.03       CUDA Version: 11.2
|
|-------------------------------+----------------------+----------------------
--+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC
|
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M.
|
|                               |                      |               MIG M.
```

```
|
|===============================+======================+======================
==|
|   0  Tesla T4            Off  | 00000000:00:04.0 Off |
0 |
| N/A   34C    P8     9W / 70W  |    3MiB / 15109MiB   |       0%
Default |
|                               |                      |                  N/A
|
+-------------------------------+----------------------+----------------------
--+
| Processes:
|
|  GPU   GI   CI        PID   Type   Process name
GPU Memory |
|        ID   ID
Usage      |
|=============================================================================
===|
| No running processes found
|
+-----------------------------------------------------------------------------
---+
```

Để kiểm tra xem TensorFlow thực sự có thấy GPU của bạn hay không, hãy chạy các lệnh sau và đảm bảo kết quả không trống:

```
import tensorflow as tf
>>> physical_gpus = tf.config.list_physical_devices("GPU")

>>> physical_gpus
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Quản lý RAM GPU

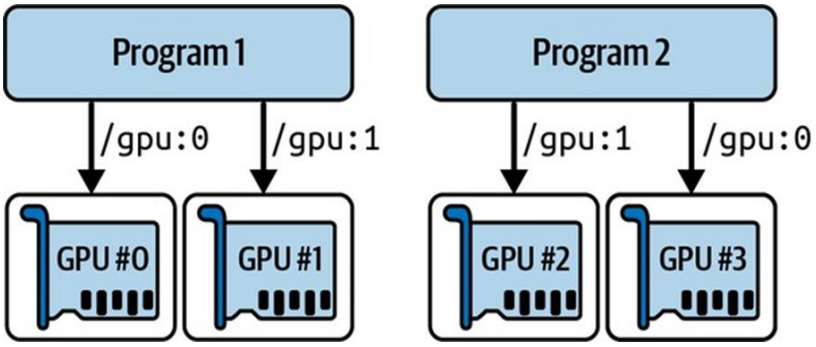
Theo mặc định, TensorFlow tự động chiếm gần như toàn bộ RAM trong tất cả các GPU khả dụng ngay lần đầu tiên bạn chạy một phép tính. Nó làm điều này để hạn chế sự phân mảnh RAM GPU. Điều này có nghĩa là nếu bạn cố gắng khởi động một chương trình TensorFlow thứ hai (hoặc bất kỳ chương trình nào yêu cầu GPU), nó sẽ nhanh chóng hết RAM. Điều này không xảy ra thường xuyên như bạn nghĩ, vì bạn sẽ thường chỉ có một chương trình TensorFlow chạy trên một máy: thường là một script huấn luyện, một nút TF Serving, hoặc một sổ tay Jupyter. Nếu bạn cần chạy nhiều chương trình vì một lý do nào đó (ví dụ: để huấn luyện hai mô hình khác nhau song song trên cùng một máy), thì bạn sẽ cần chia RAM GPU giữa các tiến trình này đồng đều hơn.

Nếu bạn có nhiều card GPU trên máy của mình, một giải pháp đơn giản là gán mỗi card cho một tiến trình duy nhất. Để làm điều này, bạn có thể đặt biến môi trường `CUDA_VISIBLE_DEVICES` sao cho mỗi tiến trình chỉ thấy (các) card GPU phù hợp. Cũng đặt biến môi trường `CUDA_DEVICE_ORDER` thành `PCI_BUS_ID` để đảm bảo rằng mỗi ID luôn tham

chiếu đến cùng một card GPU. Ví dụ, nếu bạn có bốn card GPU, bạn có thể khởi động hai chương trình, gán hai GPU cho mỗi chương trình, bằng cách thực thi các lệnh sau trong hai cửa sổ terminal riêng biệt:

```
$ CUDA_DEVICE_ORDER=PCI_BUS_ID CUDA_VISIBLE_DEVICES=0,1 python3 program_1.py
# và trong một terminal khác:
$ CUDA_DEVICE_ORDER=PCI_BUS_ID CUDA_VISIBLE_DEVICES=3,2 python3 program_2.py
```

Chương trình 1 sau đó sẽ chỉ thấy các card GPU 0 và 1, được đặt tên lần lượt là “/gpu:0” và “/gpu:1” trong TensorFlow, và chương trình 2 sẽ chỉ thấy các card GPU 2 và 3, được đặt tên lần lượt là “/gpu:1” và “/gpu:0” (lưu ý thứ tự). Mọi thứ sẽ hoạt động tốt (xem Hình 19-8). Tất nhiên, bạn cũng có thể định nghĩa các biến môi trường này trong Python bằng cách đặt `os.environ["CUDA_DEVICE_ORDER"]` và `os.environ["CUDA_VISIBLE_DEVICES"]`, miễn là bạn thực hiện trước khi sử dụng TensorFlow.



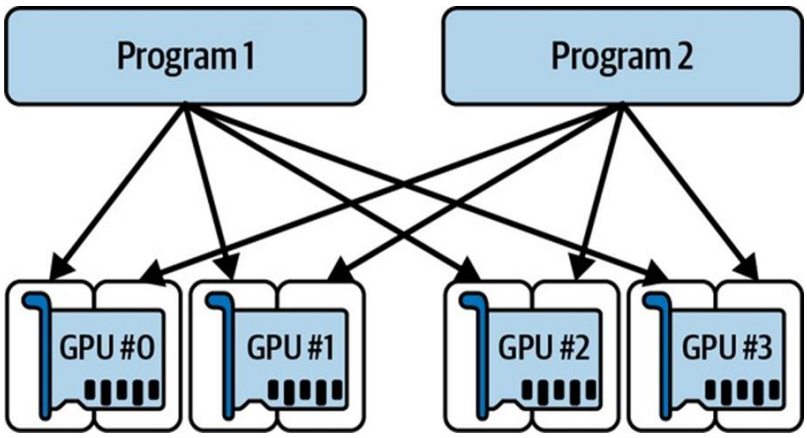
Hình 19-8. Mỗi chương trình nhận hai GPU.

Một tùy chọn khác là yêu cầu TensorFlow chỉ chiếm một lượng RAM GPU cụ thể. Điều này phải được thực hiện ngay sau khi import TensorFlow. Ví dụ, để yêu cầu TensorFlow chỉ chiếm 2 GiB RAM trên mỗi GPU, bạn phải tạo một thiết bị GPU logic (đôi khi được gọi là thiết bị GPU ảo) cho mỗi thiết bị GPU vật lý và đặt giới hạn bộ nhớ của nó là 2 GiB (tức là 2.048 MiB):

```
import tensorflow as tf
# Giả định physical_gpus đã được lấy bằng
tf.config.list_physical_devices("GPU")

for gpu in physical_gpus:
    tf.config.set_logical_device_configuration(
        gpu, [tf.config.LogicalDeviceConfiguration(memory_limit=2048)]
    )
```

Giả sử bạn có bốn GPU, mỗi GPU có ít nhất 4 GiB RAM: trong trường hợp này, hai chương trình như thế này có thể chạy song song, mỗi chương trình sử dụng cả bốn card GPU (xem Hình 19-9). Nếu bạn chạy lệnh `nvidia-smi` trong khi cả hai chương trình đang chạy, bạn sẽ thấy rằng mỗi tiến trình giữ 2 GiB RAM trên mỗi card.



Hình 19-9. Mỗi chương trình nhận tất cả bốn GPU, nhưng chỉ với 2 GiB RAM trên mỗi GPU.

Một tùy chọn khác nữa là yêu cầu TensorFlow chỉ chiếm bộ nhớ khi nó cần. Một lần nữa, điều này phải được thực hiện ngay sau khi import TensorFlow:

```
import tensorflow as tf
# Giả định physical_gpus đã được lấy

for gpu in physical_gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
```

Một cách khác để làm điều này là đặt biến môi trường `TF_FORCE_GPU_ALLOW_GROWTH` thành `true`. Với tùy chọn này, TensorFlow sẽ không bao giờ giải phóng bộ nhớ một khi nó đã chiếm (một lần nữa, để tránh phân mảnh bộ nhớ), ngoại trừ tất nhiên khi chương trình kết thúc. Có thể khó hơn để đảm bảo hành vi xác định khi sử dụng tùy chọn này (ví dụ: một chương trình có thể gặp sự cố vì mức sử dụng bộ nhớ của chương trình khác tăng vọt), vì vậy trong sản xuất, bạn có thể sẽ muốn gắn bó với một trong các tùy chọn trước đó. Tuy nhiên, có một số trường hợp nó rất hữu ích: ví dụ, khi bạn sử dụng một máy để chạy nhiều sổ tay Jupyter, một số trong số đó sử dụng TensorFlow. Biến môi trường `TF_FORCE_GPU_ALLOW_GROWTH` được đặt thành `true` trong các môi trường chạy Colab.

Cuối cùng, trong một số trường hợp, bạn có thể muốn chia một GPU thành hai hoặc nhiều thiết bị logic. Ví dụ, điều này hữu ích nếu bạn chỉ có một GPU vật lý — như trong môi trường chạy Colab — nhưng bạn muốn kiểm tra một thuật toán đa GPU. Đoạn mã sau chia GPU #0 thành hai thiết bị logic, mỗi thiết bị có 2 GiB RAM (một lần nữa, điều này phải được thực hiện ngay sau khi import TensorFlow):

```
import tensorflow as tf
# Giả định physical_gpus đã được lấy và physical_gpus[0] tồn tại

tf.config.set_logical_device_configuration(
    physical_gpus[0],
    [tf.config.LogicalDeviceConfiguration(memory_limit=2048),
```

```
    tf.config.LogicalDeviceConfiguration(memory_limit=2048)]
)
```

Hai thiết bị logic này được gọi là "/gpu:0" và "/gpu:1", và bạn có thể sử dụng chúng như thể chúng là hai GPU bình thường. Bạn có thể liệt kê tất cả các thiết bị logic như sau:

```
>>> logical_gpus = tf.config.list_logical_devices("GPU")

>>> logical_gpus
[LogicalDevice(name='/device:GPU:0', device_type='GPU'),
 LogicalDevice(name='/device:GPU:1', device_type='GPU')]
```

Bây giờ hãy xem TensorFlow quyết định thiết bị nào nó nên sử dụng để đặt biến và thực thi các phép toán.

Đặt các Phép toán và Biến trên các Thiết bị

Keras và `tf.data` thường làm rất tốt việc đặt các phép toán và biến vào đúng vị trí, nhưng bạn cũng có thể đặt chúng theo cách thủ công trên từng thiết bị nếu muốn kiểm soát nhiều hơn.

- Thông thường, bạn sẽ muốn đặt các **phép toán tiền xử lý dữ liệu trên CPU** và đặt các **phép toán của mạng nơ-ron trên GPU**.
- GPU thường có băng thông giao tiếp khá hạn chế, vì vậy điều quan trọng là phải **tránh các hoạt động truyền dữ liệu không cần thiết** vào và ra khỏi GPU.
- Việc thêm RAM CPU vào máy khá đơn giản và rẻ, nên thường có rất nhiều. Ngược lại, **RAM của GPU được tích hợp sẵn**: đây là một tài nguyên đắt đỏ và do đó bị hạn chế. Vì vậy, nếu một biến không cần thiết trong vài bước huấn luyện tiếp theo, nó có lẽ nên được đặt trên CPU (ví dụ: các tập dữ liệu thường thuộc về CPU).

Theo mặc định, tất cả các biến và phép toán sẽ được đặt trên **GPU đầu tiên** (có tên là `"/gpu:0"`), ngoại trừ các biến và phép toán không có kernel GPU; những thứ này được đặt trên CPU (luôn được đặt tên là `"/cpu:0"`).

Thuộc tính `.device` của một tensor hoặc biến cho bạn biết nó được đặt trên thiết bị nào:

```
>>> a = tf.Variable([1., 2., 3.]) # biến float32 sẽ được đưa lên GPU
>>> a.device
'/job:localhost/replica:0/task:0/device:GPU:0'

>>> b = tf.Variable([1, 2, 3]) # biến int32 sẽ được đưa lên CPU
>>> b.device
'/job:localhost/replica:0/task:0/device:CPU:0'
```

Bạn có thể tạm thời bỏ qua tiền tố `/job:localhost/replica:0/task:0`; chúng ta sẽ thảo luận về các `job`, `replica`, và `task` sau trong chương này. Như bạn thấy, biến đầu tiên được đặt trên GPU #0, là thiết bị mặc định. Tuy nhiên, biến thứ hai được đặt trên CPU: điều này là

do **không có kernel GPU cho các biến số nguyên**, hoặc cho các phép toán liên quan đến tensor số nguyên, vì vậy TensorFlow đã chuyển về sử dụng CPU.

Nếu bạn muốn đặt một phép toán trên một thiết bị khác với thiết bị mặc định, hãy sử dụng ngữ cảnh `tf.device()`:

```
>>> with tf.device("/cpu:0"):
...     c = tf.Variable([1., 2., 3.])
...
>>> c.device
'/job:localhost/replica:0/task:0/device:CPU:0'
```

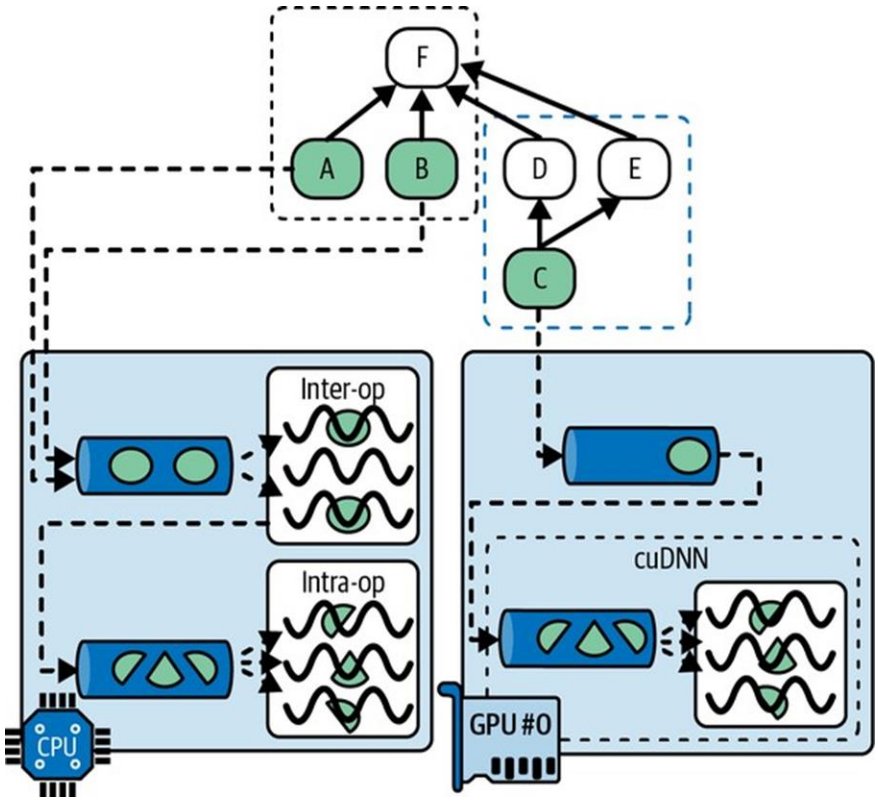
Nếu bạn cố gắng đặt một phép toán hoặc biến một cách tường minh trên một thiết bị không tồn tại hoặc không có kernel tương ứng, TensorFlow sẽ **âm thầm chuyển về thiết bị mà nó sẽ chọn theo mặc định**. Điều này hữu ích khi bạn muốn chạy cùng một đoạn mã trên các máy khác nhau không có cùng số lượng GPU.

Tuy nhiên, bạn có thể chạy `tf.config.set_soft_device_placement(False)` nếu bạn muốn nhận một ngoại lệ (exception) thay thế.

Bây giờ, chính xác thì TensorFlow thực thi các phép toán trên nhiều thiết bị như thế nào?

Thực thi song song trên nhiều thiết bị

Như chúng ta đã thấy trong Chương 12, một trong những lợi ích của việc sử dụng các hàm TF (TF functions) là khả năng song song hóa. Hãy xem xét kỹ hơn một chút. Khi TensorFlow chạy một hàm TF, nó bắt đầu bằng cách phân tích đồ thị của hàm để tìm danh sách các phép toán cần được đánh giá và đếm xem mỗi phép toán có bao nhiêu sự phụ thuộc. Sau đó, TensorFlow thêm mỗi phép toán không có sự phụ thuộc nào (tức là mỗi phép toán nguồn) vào **hàng đợi đánh giá** (evaluation queue) của thiết bị chứa phép toán đó (xem Hình 19-10). Khi một phép toán đã được đánh giá xong, bộ đếm phụ thuộc của mỗi phép toán phụ thuộc vào nó sẽ giảm đi một. Khi bộ đếm phụ thuộc của một phép toán về 0, nó sẽ được đẩy vào hàng đợi đánh giá của thiết bị của nó. Và một khi tất cả các đầu ra đã được tính toán, chúng sẽ được trả về.



Hình 19-10. Thực thi song song một đồ thị TensorFlow

Các phép toán trong hàng đợi đánh giá của CPU được điều phối đến một **nhóm luồng** (thread pool) gọi là **nhóm luồng liên toán tử** (inter-op thread pool). Nếu CPU có nhiều lõi, các phép toán này sẽ được đánh giá song song một cách hiệu quả. Một số phép toán có kernel CPU đa luồng: các kernel này chia nhiệm vụ của chúng thành nhiều phép toán con, được đặt trong một hàng đợi đánh giá khác và được điều phối đến một nhóm luồng thứ hai gọi là **nhóm luồng nội toán tử** (intra-op thread pool) (được chia sẻ bởi tất cả các kernel CPU đa luồng). Tóm lại, nhiều phép toán và phép toán con có thể được đánh giá song song trên các lõi CPU khác nhau.

Đối với GPU, mọi thứ đơn giản hơn một chút. Các phép toán trong hàng đợi đánh giá của GPU được **đánh giá tuần tự**. Tuy nhiên, hầu hết các phép toán đều có kernel GPU đa luồng, thường được triển khai bởi các thư viện mà TensorFlow phụ thuộc vào, chẳng hạn như CUDA và cuDNN. Các triển khai này có các nhóm luồng riêng, và chúng thường khai thác càng nhiều luồng GPU càng tốt (đó là lý do tại sao không cần có nhóm luồng liên toán tử trong GPU: mỗi phép toán đã chiếm gần hết các luồng GPU).

Ví dụ, trong Hình 19-10, các phép toán A, B và C là các phép toán nguồn, vì vậy chúng có thể được đánh giá ngay lập tức. Phép toán A và B được đặt trên CPU, vì vậy chúng được gửi đến hàng đợi đánh giá của CPU, sau đó được điều phối đến nhóm luồng liên toán tử và được đánh giá song song ngay lập tức. Phép toán A tình cờ có một kernel đa luồng; các tính toán của nó được chia thành ba phần, được thực thi song song bởi nhóm luồng nội toán tử. Phép toán C được đưa đến hàng đợi đánh giá của GPU #0, và trong ví dụ này, kernel GPU

của nó sử dụng cuDNN, vốn quản lý nhóm luồng nội toán tử của riêng nó và chạy phép toán trên nhiều luồng GPU song song. Giả sử C hoàn thành trước. Bộ đếm phụ thuộc của D và E được giảm đi và chúng đạt đến 0, vì vậy cả hai phép toán đều được đẩy vào hàng đợi đánh giá của GPU #0 và được thực thi tuần tự.

Lưu ý rằng **C chỉ được đánh giá một lần**, mặc dù cả D và E đều phụ thuộc vào nó. Giả sử B hoàn thành tiếp theo. Bộ đếm phụ thuộc của F giảm từ 4 xuống 3, và vì đó không phải là 0, nó chưa chạy. Khi A, D và E hoàn thành, bộ đếm phụ thuộc của F đạt đến 0, và nó được đẩy vào hàng đợi đánh giá của CPU và được đánh giá. Cuối cùng, TensorFlow trả về các đầu ra được yêu cầu.

Một chút "ma thuật" ✨ khác mà TensorFlow thực hiện là khi hàm TF sửa đổi một tài nguyên có trạng thái, chẳng hạn như một biến: nó đảm bảo rằng **thứ tự thực thi khớp với thứ tự trong mã**, ngay cả khi không có sự phụ thuộc rõ ràng giữa các câu lệnh. Ví dụ, nếu hàm TF của bạn chứa `v.assign_add(1)` theo sau là `v.assign(v * 2)`, TensorFlow sẽ đảm bảo rằng các phép toán này được thực thi theo thứ tự đó.

Với những kiến thức đó, bạn đã có đủ mọi thứ cần thiết để chạy bất kỳ phép toán nào trên bất kỳ thiết bị nào và khai thác sức mạnh của GPU! Dưới đây là một số việc bạn có thể làm:

- Bạn có thể **huấn luyện nhiều mô hình song song**, mỗi mô hình trên một GPU riêng: chỉ cần viết một kịch bản huấn luyện cho mỗi mô hình và chạy chúng song song, đặt `CUDA_DEVICE_ORDER` và `CUDA_VISIBLE_DEVICES` để mỗi kịch bản chỉ thấy một thiết bị GPU duy nhất. Điều này rất tuyệt vời cho việc tinh chỉnh siêu tham số, vì bạn có thể huấn luyện song song nhiều mô hình với các siêu tham số khác nhau.
- Bạn có thể huấn luyện một mô hình trên một GPU và thực hiện **toàn bộ quá trình tiền xử lý song song trên CPU**, sử dụng phương thức `prefetch()` của tập dữ liệu để chuẩn bị trước một vài lô dữ liệu sao cho chúng sẵn sàng khi GPU cần.
- Nếu mô hình của bạn nhận hai hình ảnh làm đầu vào và xử lý chúng bằng **hai CNN** trước khi kết hợp đầu ra của chúng, nó có thể sẽ chạy nhanh hơn nhiều nếu bạn đặt mỗi CNN trên một GPU khác nhau.
- Bạn có thể tạo một **ensemble hiệu quả**: chỉ cần đặt một mô hình đã được huấn luyện khác nhau trên mỗi GPU để bạn có thể nhận được tất cả các dự đoán nhanh hơn nhiều nhằm tạo ra dự đoán cuối cùng của ensemble.

Nhưng nếu bạn muốn tăng tốc độ huấn luyện bằng cách sử dụng nhiều GPU thì sao?

Huấn luyện mô hình trên nhiều thiết bị

Có hai phương pháp chính để huấn luyện một mô hình duy nhất trên nhiều thiết bị: **song song hóa mô hình** (model parallelism), nơi mô hình được chia nhỏ trên các thiết bị, và **song song hóa dữ liệu** (data parallelism), nơi mô hình được sao chép trên mỗi thiết bị, và mỗi bản sao được huấn luyện trên một tập hợp con dữ liệu khác nhau. Hãy cùng xem xét hai lựa chọn này.

Công thức Toán (Phán đoán):

Phần này không chứa các công thức toán học tường minh, mà tập trung vào các khái niệm kỹ thuật và cấu hình phần cứng/phần mềm. Tuy nhiên, có một vài điểm có thể được liên hệ với tính toán:

10. **Tính toán thời gian giảm tốc độ với GPU:** Nếu một tác vụ mất T_CPU thời gian trên CPU và T_GPU thời gian trên GPU, thì sự tăng tốc (speedup) được tính bằng: $$$$$\ = \ $$ $$$$Văn bản ngụ ý rằng T_GPU có thể nhỏ hơn T_CPU rất nhiều (từ hàng tuần/ngày xuống hàng giờ/phút).
11. **Phân chia RAM GPU:** Khái niệm chia RAM GPU liên quan đến việc cấp phát tài nguyên. Nếu một GPU có tổng RAM là R_total và bạn muốn chia nó cho N chương trình (hoặc thiết bị logic), mỗi chương trình (hoặc thiết bị logic) sẽ nhận được R_chunk RAM, thì: $$$$$R_{chunk} = \ $$ $$$$Ví dụ, nếu một GPU có 4 GiB RAM và bạn muốn chia nó thành hai thiết bị logic, mỗi thiết bị sẽ có $4textGiB/2 = 2textGiB$ RAM. Hoặc nếu bạn muốn giới hạn mỗi chương trình chỉ sử dụng 2 GiB RAM trên mỗi GPU, bạn đang đặt một giới hạn cứng cho việc cấp phát bộ nhớ.

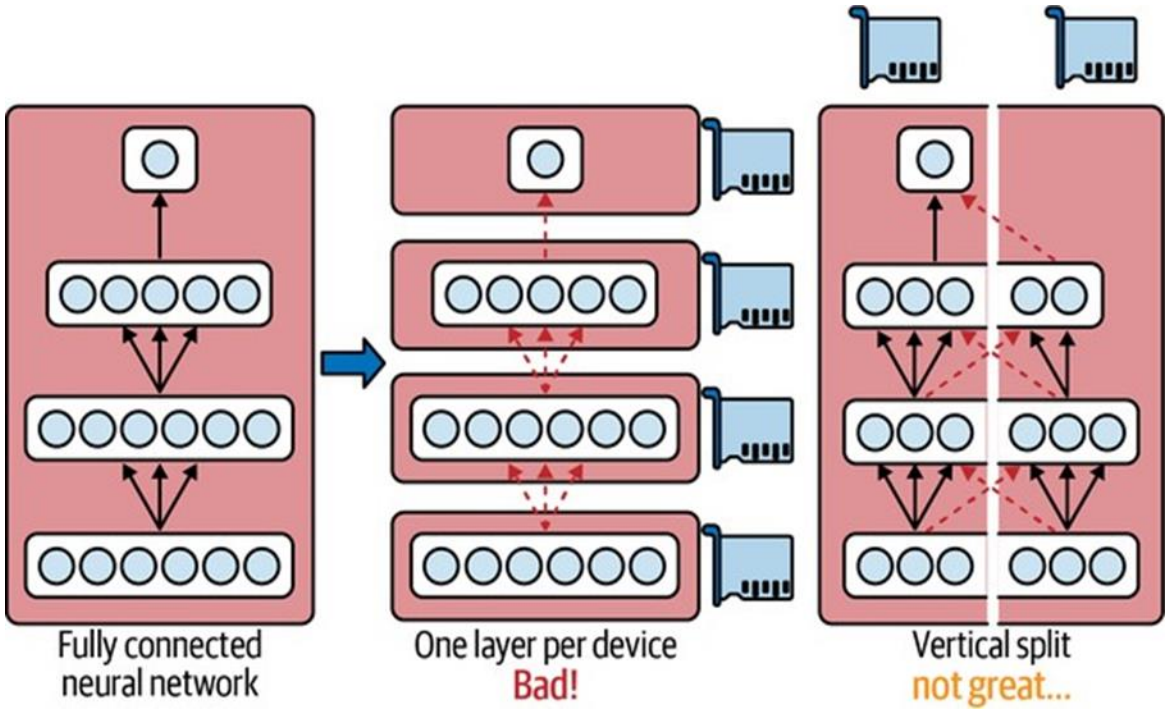
Các phần thảo luận về `tf.config.set_logical_device_configuration` và `tf.config.experimental.set_memory_growth` đều liên quan đến việc kiểm soát cấp phát bộ nhớ, một khía cạnh quan trọng của tối ưu hóa hiệu suất tính toán.

Mô hình song song (Model Parallelism)

Cho đến nay, chúng ta đã huấn luyện mỗi mạng nơ-ron trên một thiết bị duy nhất. Điều gì sẽ xảy ra nếu chúng ta muốn huấn luyện một mạng nơ-ron duy nhất trên nhiều thiết bị?

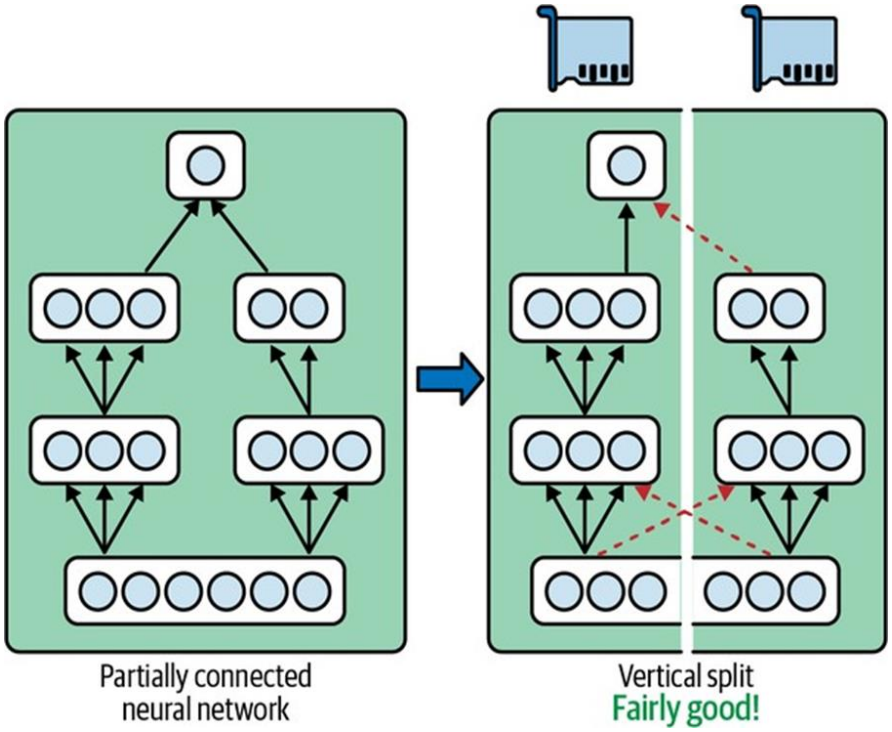
Điều này yêu cầu chia mô hình thành các phần riêng biệt và chạy mỗi phần trên một thiết bị khác nhau. Thật không may, mô hình song song như vậy hóa ra khá phức tạp, và hiệu quả của nó thực sự phụ thuộc vào kiến trúc của mạng nơ-ron của bạn. Đối với các mạng kết nối đầy đủ (fully connected networks), thường không có nhiều lợi ích từ cách tiếp cận này (xem Hình 19-11).

Một cách trực quan, có vẻ như một cách dễ dàng để chia mô hình là đặt mỗi lớp trên một thiết bị khác nhau, nhưng điều này không hoạt động vì mỗi lớp cần đợi đầu ra của lớp trước đó trước khi nó có thể làm bất cứ điều gì. Vậy có lẽ bạn có thể cắt nó theo chiều dọc — ví dụ, với nửa bên trái của mỗi lớp trên một thiết bị, và nửa bên phải trên một thiết bị khác? Điều này tốt hơn một chút, vì cả hai nửa của mỗi lớp thực sự có thể hoạt động song song, nhưng vấn đề là mỗi nửa của lớp tiếp theo yêu cầu đầu ra của cả hai nửa, vì vậy sẽ có rất nhiều giao tiếp giữa các thiết bị (được biểu thị bằng các mũi tên đứt nét). Điều này có khả năng loại bỏ hoàn toàn lợi ích của tính toán song song, vì giao tiếp giữa các thiết bị chậm (và thậm chí còn chậm hơn khi các thiết bị nằm trên các máy khác nhau).



Hình 19-11. Chia một mạng nơ-ron kết nối đầy đủ.

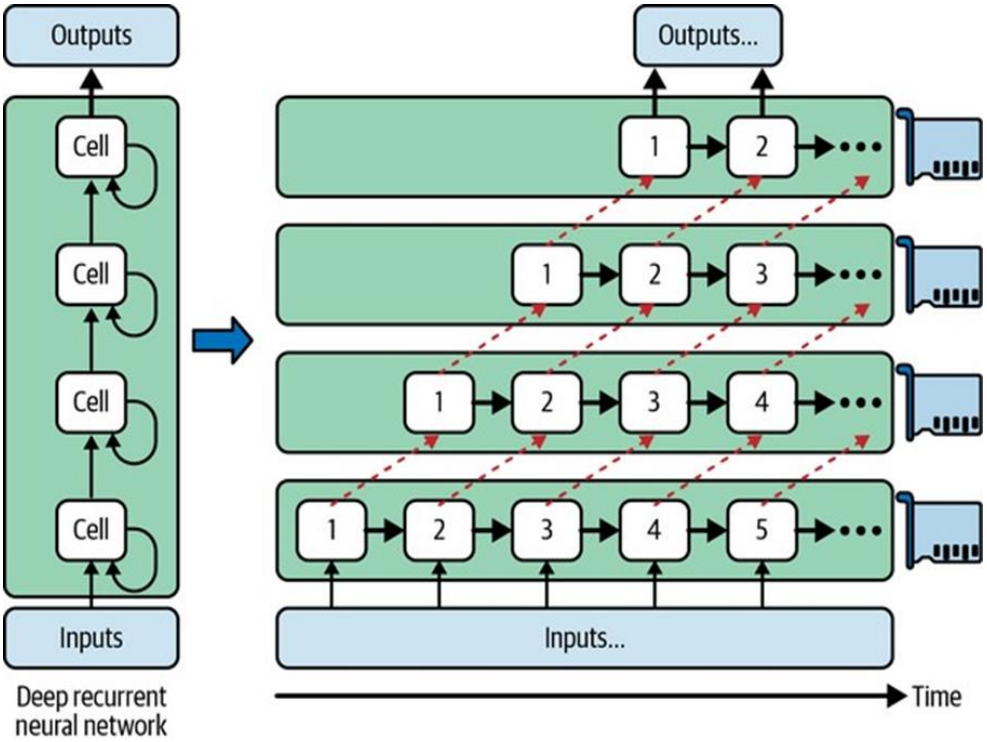
Một số kiến trúc mạng nơ-ron, chẳng hạn như mạng nơ-ron tích chập (convolutional neural networks) (xem Chương 14), chứa các lớp chỉ được kết nối một phần với các lớp thấp hơn, vì vậy việc phân phối các phần trên các thiết bị một cách hiệu quả sẽ dễ dàng hơn nhiều (Hình 19-12).



Hình 19-12. Chia một mạng nơ-ron kết nối một phần.

Các mạng nơ-ron hồi quy sâu (deep recurrent neural networks) (xem Chương 15) có thể được chia hiệu quả hơn một chút trên nhiều GPU. Nếu bạn chia mạng theo chiều ngang bằng cách đặt mỗi lớp trên một thiết bị khác nhau, và cấp dữ liệu đầu vào tuần tự cho mạng để xử lý, thì ở bước thời gian đầu tiên chỉ một thiết bị sẽ hoạt động (xử lý giá trị đầu tiên của chuỗi), ở bước thứ hai hai thiết bị sẽ hoạt động (lớp thứ hai sẽ xử lý đầu ra của lớp đầu tiên cho giá trị đầu tiên, trong khi lớp đầu tiên sẽ xử lý giá trị thứ hai), và đến khi tín hiệu lan truyền đến lớp đầu ra, tất cả các thiết bị sẽ hoạt động đồng thời (Hình 19-13).

Vẫn có rất nhiều giao tiếp giữa các thiết bị đang diễn ra, nhưng vì mỗi ô có thể khá phức tạp, lợi ích của việc chạy nhiều ô song song có thể (về lý thuyết) lớn hơn chi phí giao tiếp. Tuy nhiên, trên thực tế, một chồng các lớp LSTM thông thường chạy trên một GPU duy nhất thực sự chạy nhanh hơn nhiều.



Hình 19-13. Chia một mạng nơ-ron hồi quy sâu.

Tóm lại, mô hình song song có thể tăng tốc việc chạy hoặc huấn luyện một số loại mạng nơ-ron, nhưng không phải tất cả, và nó đòi hỏi sự chăm sóc và điều chỉnh đặc biệt, chẳng hạn như đảm bảo rằng các thiết bị cần giao tiếp nhiều nhất chạy trên cùng một máy. Tiếp theo, chúng ta sẽ xem xét một tùy chọn đơn giản hơn nhiều và thường hiệu quả hơn: song song dữ liệu (data parallelism).

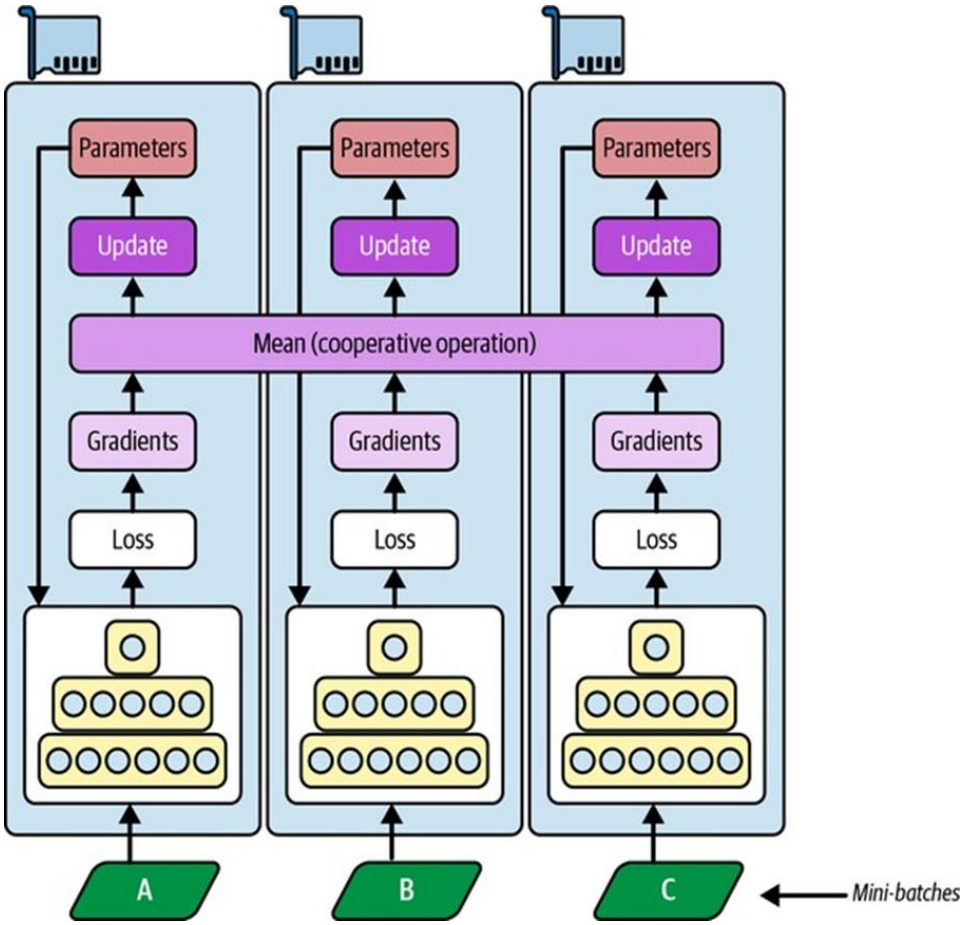
Song song Dữ liệu (Data Parallelism)

Một cách khác để song song hóa quá trình huấn luyện của một mạng nơ-ron là nhân bản nó trên mọi thiết bị và chạy mỗi bước huấn luyện đồng thời trên tất cả các bản sao, sử dụng

một mini-batch khác nhau cho mỗi bản sao. Các gradient được tính toán bởi mỗi bản sao sau đó được tính trung bình, và kết quả được sử dụng để cập nhật các tham số mô hình. Đây được gọi là **song song dữ liệu (data parallelism)**, hoặc đôi khi là **một chương trình, nhiều dữ liệu (single program, multiple data - SPMD)**. Có nhiều biến thể của ý tưởng này, vì vậy hãy xem xét những biến thể quan trọng nhất.

Song song dữ liệu sử dụng chiến lược nhân bản (mirrored strategy)

Cách tiếp cận đơn giản nhất có lẽ là nhân bản hoàn toàn tất cả các tham số mô hình trên tất cả các GPU và luôn áp dụng chính xác các bản cập nhật tham số giống nhau trên mọi GPU. Bằng cách này, tất cả các bản sao luôn giữ nguyên trạng hoàn hảo. Đây được gọi là **chiến lược nhân bản (mirrored strategy)**, và nó hóa ra khá hiệu quả, đặc biệt khi sử dụng một máy duy nhất (xem Hình 19-14).



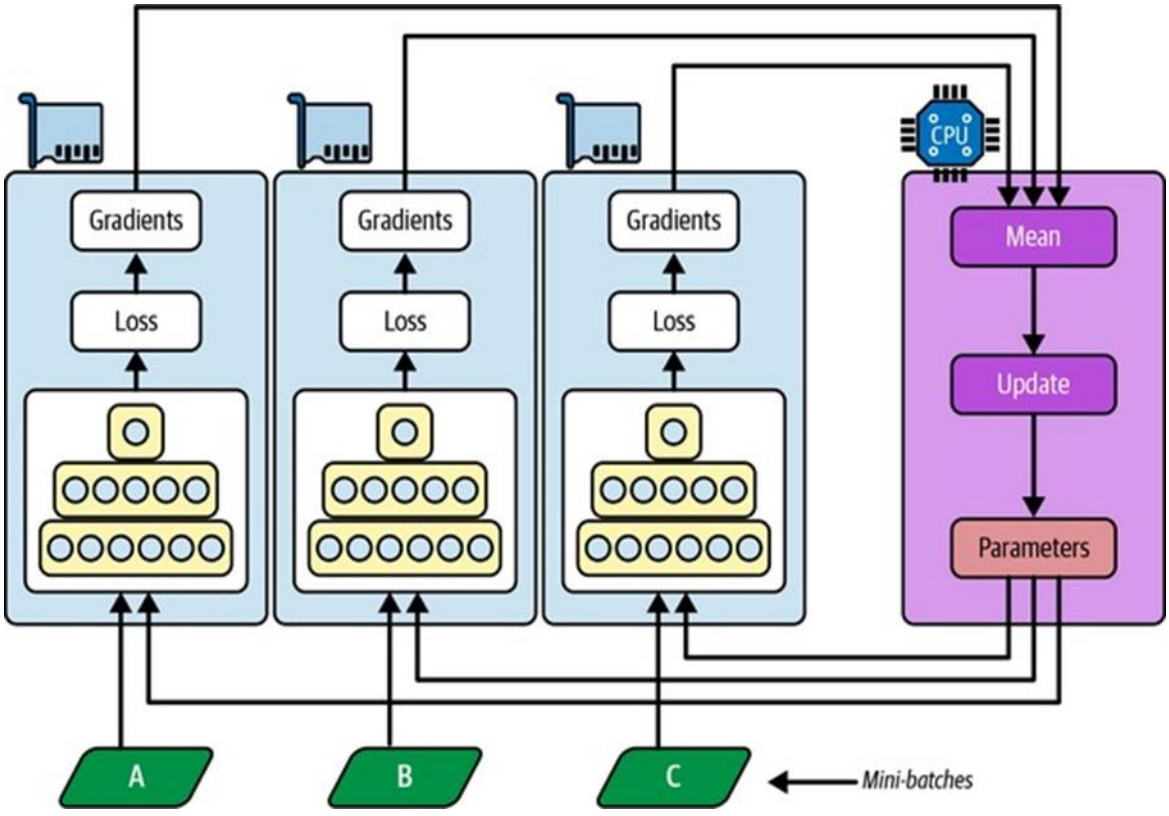
Hình 19-14. Song song dữ liệu sử dụng chiến lược nhân bản.

Phần phức tạp khi sử dụng cách tiếp cận này là tính toán hiệu quả giá trị trung bình của tất cả các gradient từ tất cả các GPU và phân phối kết quả trên tất cả các GPU. Điều này có thể được thực hiện bằng cách sử dụng thuật toán **AllReduce**, một loại thuật toán trong đó nhiều nút hợp tác để thực hiện hiệu quả một phép toán giảm (chẳng hạn như tính trung bình, tổng, và giá trị lớn nhất), đồng thời đảm bảo rằng tất cả các nút đều nhận được cùng

một kết quả cuối cùng. May mắn thay, có các triển khai sẵn có của các thuật toán như vậy, như bạn sẽ thấy.

Song song dữ liệu với tham số tập trung (centralized parameters)

Một cách tiếp cận khác là lưu trữ các tham số mô hình bên ngoài các thiết bị GPU thực hiện các phép tính (được gọi là workers); ví dụ, trên CPU (xem Hình 19-15). Trong một thiết lập phân tán, bạn có thể đặt tất cả các tham số trên một hoặc nhiều máy chủ chỉ có CPU được gọi là **máy chủ tham số (parameter servers)**, mà vai trò duy nhất của chúng là lưu trữ và cập nhật các tham số.



Hình 19-15. Song song dữ liệu với tham số tập trung.

Trong khi chiến lược nhân bản áp đặt cập nhật trọng số đồng bộ trên tất cả các GPU, cách tiếp cận tập trung này cho phép cả cập nhật đồng bộ hoặc bất đồng bộ. Hãy xem xét ưu và nhược điểm của cả hai tùy chọn.

Cập nhật đồng bộ (Synchronous updates)

Với cập nhật đồng bộ, bộ tổng hợp đợi cho đến khi tất cả các gradient có sẵn trước khi nó tính toán gradient trung bình và chuyển chúng cho bộ tối ưu hóa, bộ tối ưu hóa này sẽ cập nhật các tham số mô hình.

Khi một bản sao đã hoàn thành việc tính toán gradient của nó, nó phải đợi các tham số được cập nhật trước khi nó có thể chuyển sang mini-batch tiếp theo. Nhược điểm là một số

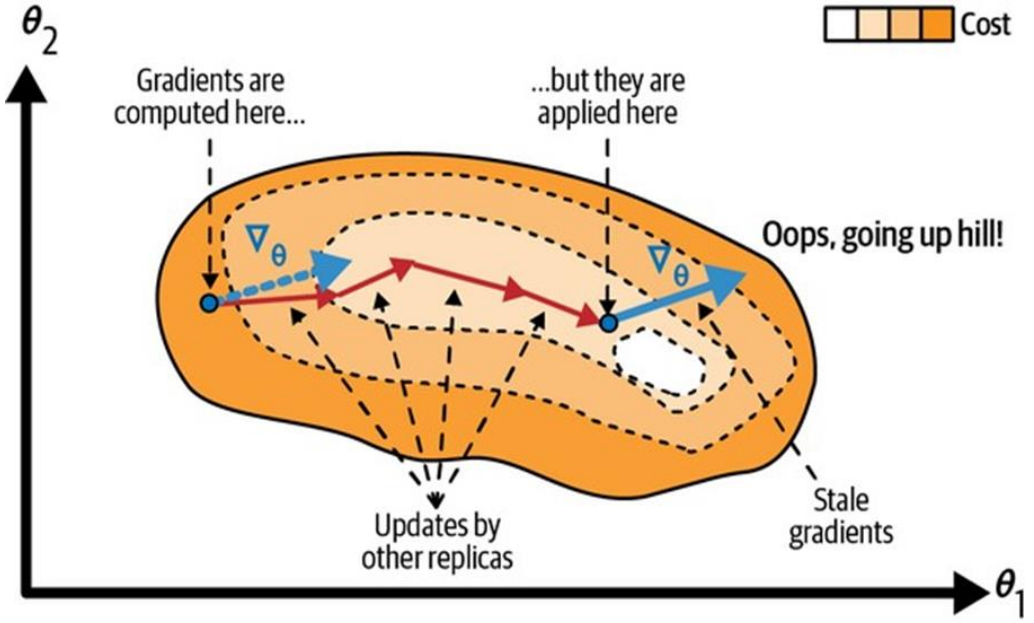
thiết bị có thể chậm hơn các thiết bị khác, vì vậy các thiết bị nhanh sẽ phải đợi các thiết bị chậm ở mỗi bước, làm cho toàn bộ quá trình chậm bằng thiết bị chậm nhất. Hơn nữa, các tham số sẽ được sao chép đến mọi thiết bị gần như cùng lúc (ngay sau khi các gradient được áp dụng), điều này có thể làm bão hòa băng thông của các máy chủ tham số.

Cập nhật bất đồng bộ (Asynchronous updates)

Với cập nhật bất đồng bộ, bất cứ khi nào một bản sao đã hoàn thành việc tính toán gradient, các gradient sẽ ngay lập tức được sử dụng để cập nhật các tham số mô hình. Không có tổng hợp (nó loại bỏ bước "mean" trong Hình 19-15) và không có đồng bộ hóa. Các bản sao hoạt động độc lập với các bản sao khác. Vì không có việc chờ đợi các bản sao khác, cách tiếp cận này chạy nhiều bước huấn luyện hơn mỗi phút. Hơn nữa, mặc dù các tham số vẫn cần được sao chép đến mọi thiết bị ở mỗi bước, điều này xảy ra vào các thời điểm khác nhau đối với mỗi bản sao, vì vậy rủi ro bão hòa băng thông giảm.

Song song dữ liệu với cập nhật bất đồng bộ là một lựa chọn hấp dẫn vì sự đơn giản của nó, không có độ trễ đồng bộ hóa và sử dụng băng thông tốt hơn. Tuy nhiên, mặc dù nó hoạt động khá tốt trên thực tế, nhưng gần như đáng ngạc nhiên khi nó hoạt động được! Thật vậy, vào thời điểm một bản sao đã hoàn thành việc tính toán gradient dựa trên một số giá trị tham số, các tham số này sẽ đã được cập nhật nhiều lần bởi các bản sao khác (trung bình $N - 1$ lần, nếu có N bản sao), và không có gì đảm bảo rằng các gradient đã tính toán vẫn sẽ trỏ đúng hướng (xem Hình 19-16).

Khi các gradient bị lỗi thời nghiêm trọng, chúng được gọi là **gradient lỗi thời (stale gradients)**: chúng có thể làm chậm quá trình hội tụ, gây ra nhiễu và hiệu ứng lung lay (đường cong học tập có thể chứa các dao động tạm thời), hoặc chúng thậm chí có thể làm cho thuật toán huấn luyện phân kỳ.



Hình 19-16. Gradient lỗi thời khi sử dụng cập nhật bất đồng bộ.

Có một vài cách bạn có thể giảm ảnh hưởng của gradient lỗi thời:

- Giảm tốc độ học (learning rate).
- Bỏ qua các gradient lỗi thời hoặc giảm tỷ lệ của chúng.
- Điều chỉnh kích thước mini-batch.
- Bắt đầu vài epoch đầu tiên chỉ sử dụng một bản sao (đây được gọi là **giai đoạn khởi động - warmup phase**). Gradient lỗi thời có xu hướng gây hại nhiều hơn vào đầu quá trình huấn luyện, khi gradient thường lớn và các tham số chưa ổn định vào một thung lũng của hàm chi phí, vì vậy các bản sao khác nhau có thể đẩy các tham số theo những hướng khá khác nhau.

Một bài báo được xuất bản bởi nhóm Google Brain vào năm 2016 đã so sánh các cách tiếp cận khác nhau và thấy rằng việc sử dụng cập nhật đồng bộ với một vài bản sao dự phòng hiệu quả hơn so với việc sử dụng cập nhật bất đồng bộ, không chỉ hội tụ nhanh hơn mà còn tạo ra một mô hình tốt hơn. Tuy nhiên, đây vẫn là một lĩnh vực nghiên cứu tích cực, vì vậy bạn không nên loại bỏ cập nhật bất đồng bộ ngay lập tức.

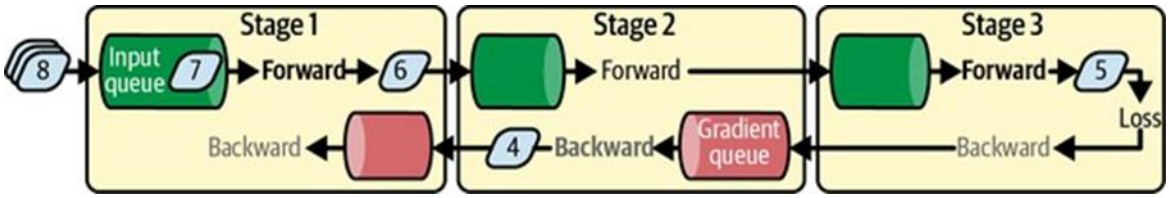
Bão hòa băng thông (Bandwidth saturation)

Cho dù bạn sử dụng cập nhật đồng bộ hay bất đồng bộ, song song dữ liệu với các tham số tập trung vẫn yêu cầu truyền tham số mô hình từ máy chủ tham số đến mọi bản sao ở đầu mỗi bước huấn luyện, và các gradient theo hướng ngược lại ở cuối mỗi bước huấn luyện. Tương tự, khi sử dụng chiến lược nhân bản, các gradient được tạo ra bởi mỗi GPU sẽ cần được chia sẻ với mọi GPU khác. Thật không may, thường có một điểm mà việc thêm một GPU bổ sung sẽ không cải thiện hiệu suất chút nào vì thời gian dành cho việc di chuyển dữ liệu vào và ra khỏi RAM GPU (và qua mạng trong một thiết lập phân tán) sẽ lớn hơn lợi ích tăng tốc đạt được bằng cách chia tải tính toán. Tại thời điểm đó, việc thêm nhiều GPU hơn sẽ chỉ làm trầm trọng thêm tình trạng bão hòa băng thông và thực sự làm chậm quá trình huấn luyện.

Bão hòa nghiêm trọng hơn đối với các mô hình dày đặc lớn (large dense models), vì chúng có nhiều tham số và gradient để truyền. Nó ít nghiêm trọng hơn đối với các mô hình nhỏ (nhưng lợi ích song song hóa bị hạn chế) và đối với các mô hình thưa lớn (large sparse models), nơi các gradient thường chủ yếu là số 0 và do đó có thể được truyền thông hiệu quả. Jeff Dean, người khởi xướng và lãnh đạo dự án Google Brain, đã báo cáo tốc độ tăng tốc điển hình là 25–40 lần khi phân tán các phép tính trên 50 GPU cho các mô hình dày đặc, và tốc độ tăng tốc 300 lần cho các mô hình thưa hơn được huấn luyện trên 500 GPU. Như bạn có thể thấy, các mô hình thưa thực sự mở rộng tốt hơn. Dưới đây là một vài ví dụ cụ thể:

- Dịch máy nơ-ron: Tăng tốc 6 lần trên 8 GPU
- Inception/ImageNet: Tăng tốc 32 lần trên 50 GPU
- RankBrain: Tăng tốc 300 lần trên 500 GPU

Có rất nhiều nghiên cứu đang diễn ra để giảm thiểu vấn đề bão hòa băng thông, với mục tiêu cho phép huấn luyện mở rộng tuyến tính với số lượng GPU có sẵn. Ví dụ, một bài báo năm 2018 của một nhóm các nhà nghiên cứu từ Đại học Carnegie Mellon, Đại học Stanford và Microsoft Research đã đề xuất một hệ thống có tên PipeDream đã giảm hơn 90% giao tiếp mạng, giúp có thể huấn luyện các mô hình lớn trên nhiều máy. Họ đã đạt được điều này bằng cách sử dụng một kỹ thuật mới được gọi là **song song đường ống (pipeline parallelism)**, kết hợp song song mô hình và song song dữ liệu: mô hình được chia thành các phần liên tiếp, được gọi là các **giai đoạn (stages)**, mỗi giai đoạn được huấn luyện trên một máy khác nhau. Điều này tạo ra một đường ống bất đồng bộ trong đó tất cả các máy hoạt động song song với rất ít thời gian nhàn rỗi. Trong quá trình huấn luyện, mỗi giai đoạn luân phiên một vòng truyền xuôi (forward propagation) và một vòng truyền ngược (backpropagation) (xem Hình 19-17): nó lấy một mini-batch từ hàng đợi đầu vào của nó, xử lý nó, và gửi đầu ra đến hàng đợi đầu vào của giai đoạn tiếp theo, sau đó nó lấy một mini-batch gradient từ hàng đợi gradient của nó, truyền ngược các gradient này và cập nhật các tham số mô hình của chính nó, và đẩy các gradient đã truyền ngược đến hàng đợi gradient của giai đoạn trước đó. Sau đó, nó lặp lại toàn bộ quá trình. Mỗi giai đoạn cũng có thể sử dụng song song dữ liệu thông thường (ví dụ: sử dụng chiến lược nhân bản), độc lập với các giai đoạn khác.



Hình 19-17. Song song đường ống của PipeDream.

Tuy nhiên, như được trình bày ở đây, PipeDream sẽ không hoạt động tốt. Để hiểu tại sao, hãy xem xét mini-batch #5 trong Hình 19-17: khi nó đi qua giai đoạn 1 trong quá trình truyền xuôi, các gradient từ mini-batch #4 vẫn chưa được truyền ngược qua giai đoạn đó, nhưng đến khi gradient của #5 chảy ngược về giai đoạn 1, gradient của #4 sẽ đã được sử dụng để cập nhật các tham số mô hình, vì vậy gradient của #5 sẽ hơi lỗi thời. Như chúng ta đã thấy, điều này có thể làm giảm tốc độ huấn luyện và độ chính xác, và thậm chí làm cho nó phân kỳ: càng có nhiều giai đoạn, vấn đề này càng trở nên tồi tệ hơn. Các tác giả của bài báo đã đề xuất các phương pháp để giảm thiểu vấn đề này, tuy nhiên: ví dụ, mỗi giai đoạn lưu trọng số trong quá trình truyền xuôi và khôi phục chúng trong quá trình truyền ngược, để đảm bảo rằng cùng một trọng số được sử dụng cho cả truyền xuôi và truyền ngược. Điều này được gọi là **cất giữ trọng số (weight stashing)**.

Nhờ điều này, PipeDream thể hiện khả năng mở rộng ấn tượng, vượt xa song song dữ liệu đơn giản.

Bước đột phá mới nhất trong lĩnh vực nghiên cứu này đã được xuất bản trong một bài báo năm 2022 bởi các nhà nghiên cứu của Google: họ đã phát triển một hệ thống có tên Pathways sử dụng song song mô hình tự động, lập lịch nhóm bất đồng bộ, và các kỹ thuật khác để đạt được gần 100% sử dụng phần cứng trên hàng ngàn TPU! **Lập lịch (scheduling)**

có nghĩa là tổ chức khi nào và ở đâu mỗi tác vụ phải chạy, và **lập lịch nhóm (gang scheduling)** có nghĩa là chạy các tác vụ liên quan cùng lúc song song và gần nhau để giảm thời gian các tác vụ phải đợi đầu ra của các tác vụ khác. Như chúng ta đã thấy trong Chương 16, hệ thống này đã được sử dụng để huấn luyện một mô hình ngôn ngữ khổng lồ trên hơn 6.000 TPU, với gần 100% sử dụng phần cứng: đó là một kỳ tích kỹ thuật đáng kinh ngạc.

Tại thời điểm viết bài này, Pathways chưa công khai, nhưng có khả năng trong tương lai gần bạn sẽ có thể huấn luyện các mô hình lớn trên Vertex AI bằng cách sử dụng Pathways hoặc một hệ thống tương tự. Trong thời gian chờ đợi, để giảm vấn đề bão hòa, bạn có thể sẽ muốn sử dụng một vài GPU mạnh mẽ hơn là nhiều GPU yếu, và nếu bạn cần huấn luyện một mô hình trên nhiều máy chủ, bạn nên nhóm các GPU của mình trên ít máy chủ và có kết nối rất tốt. Bạn cũng có thể thử giảm độ chính xác dấu phẩy động từ 32 bit (`tf.float32`) xuống 16 bit (`tf.bfloat16`). Điều này sẽ giảm một nửa lượng dữ liệu cần truyền, thường không ảnh hưởng nhiều đến tốc độ hội tụ hoặc hiệu suất của mô hình. Cuối cùng, nếu bạn đang sử dụng các tham số tập trung, bạn có thể **chia nhỏ (shard)** các tham số trên nhiều máy chủ tham số: việc thêm nhiều máy chủ tham số sẽ giảm tải mạng trên mỗi máy chủ và hạn chế rủi ro bão hòa băng thông.

Được rồi, bây giờ chúng ta đã đi qua tất cả lý thuyết, hãy thực sự huấn luyện một mô hình trên nhiều GPU!

Phán đoán và Công thức Toán:

Trong phần này, các công thức toán học không được trình bày rõ ràng nhưng các khái niệm về hiệu suất và tính toán được thảo luận sâu. Dưới đây là những điểm có thể liên hệ với toán học:

12. **Hiệu quả của Model Parallelism và Data Parallelism:** Cả hai phương pháp đều nhằm mục đích giảm thời gian huấn luyện. Nếu thời gian huấn luyện trên một thiết bị là T_1 , và trên N thiết bị là T_N . Mục tiêu là $T_N \ll T_1$.
    - **Speedup:** Một thước đo hiệu suất là "speedup", được định nghĩa là $S_N = T_1/T_N$. Mục tiêu là đạt được speedup càng gần N càng tốt (linear speedup).
    - **Hiệu quả (Efficiency):** $E_N = S_N/N$. Mục tiêu là $E_N \approx 1$. Văn bản cho thấy rằng trong thực tế, E_N thường nhỏ hơn 1 do chi phí giao tiếp và độ trễ.
13. **Chi phí Giao tiếp (Communication Overhead):** Chi phí giao tiếp giữa các thiết bị hoặc máy chủ là một yếu tố quan trọng làm giảm hiệu quả song song. Thời gian dành cho giao tiếp có thể được xem là T_{comm} .

    $$T_N = T_{comp} + T_{comm}$$

    Trong đó T_{comp} là thời gian tính toán thực tế. Khi N tăng, T_{comp} có thể giảm, nhưng T_{comm} thường tăng, dẫn đến điểm bão hòa (saturation point) nơi việc thêm GPU không còn cải thiện hiệu suất, thậm chí làm tệ hơn.

14. **Tác động của Stale Gradients:** Trong cập nhật bất đồng bộ, gradient từ một bản sao có thể bị "lỗi thời" khi áp dụng do các bản sao khác đã cập nhật tham số. Nếu có N bản sao, trung bình một gradient sẽ lỗi thời $N-1$ lần cập nhật của các bản sao khác. Điều này có thể được hình dung bằng một hàm lỗi (loss function) $L(\theta)$ và việc cập nhật tham số θ theo gradient $-\nabla L(\theta)$. Gradient lỗi thời có thể không còn chỉ hướng dốc nhất đến cực tiểu, dẫn đến đường đi cập nhật "lung lay" (oscillations) hoặc thậm chí phân kỳ.
$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_{t-\delta})$$
Trong đó δ là độ trễ (staleness) của gradient.
15. **Lợi ích của Half-Precision Floats (`tf.bfloat16`):** Việc chuyển từ `tf.float32` (32 bit) sang `tf.bfloat16` (16 bit) giảm kích thước dữ liệu cần truyền đi một nửa (factor of 2). Điều này trực tiếp giảm T_{comm} , giúp giảm bão hòa băng thông.

$$Kı´chthướcdữliệumới = \frac{16}{32} \times Kı´chthướcdữliệugo\hat{}c = 0.5 \times Kı´chthướcdữliệugo\hat{}c$$

16. **Sharding Parameters:** Khi chia nhỏ tham số (sharding), tải mạng trên mỗi máy chủ tham số được phân chia. Nếu tổng tải mạng là L_{total} và bạn chia nhỏ trên M máy chủ tham số, thì tải trung bình trên mỗi máy chủ là L_{total}/M . Điều này giúp phân tán tải và tránh tắc nghẽn.

Các con số về tốc độ tăng tốc (ví dụ: 25-40 lần trên 50 GPU, 300 lần trên 500 GPU) là các giá trị thực nghiệm, cho thấy hiệu quả của song song hóa trong các kịch bản thực tế.

Huấn luyện Quy mô lớn bằng API Chiến lược Phân phối (Distribution Strategies API)

May mắn thay, TensorFlow đi kèm với một API rất tuyệt vời, giúp xử lý tất cả sự phức tạp của việc phân phối mô hình của bạn trên nhiều thiết bị và máy: API chiến lược phân phối. Để huấn luyện một mô hình Keras trên tất cả các GPU khả dụng (trên một máy duy nhất, hiện tại) bằng cách sử dụng song song dữ liệu với chiến lược nhân bản (mirrored strategy), chỉ cần tạo một đối tượng `MirroredStrategy`, gọi phương thức `scope()` của nó để có được một ngữ cảnh phân phối, và gói việc tạo và biên dịch mô hình của bạn bên trong ngữ cảnh đó. Sau đó, gọi phương thức `fit()` của mô hình như bình thường:

```
import tensorflow as tf

# Giả định X_train, y_train, X_valid, y_valid đã được tải và chia
# Ví dụ: (X_train, y_train), (X_test, y_test) =
tf.keras.datasets.mnist.load_data()
# X_train = X_train.astype('float32') / 255.0
# X_valid = X_valid.astype('float32') / 255.0
# y_train = tf.keras.utils.to_categorical(y_train, num_classes=10)
# y_valid = tf.keras.utils.to_categorical(y_valid, num_classes=10)

strategy = tf.distribute.MirroredStrategy()
```

```
with strategy.scope():
    model = tf.keras.Sequential([ # tạo một mô hình Keras như bình thường
        tf.keras.layers.Flatten(input_shape=(28, 28)),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy']) # biên dịch mô hình như bình thường

batch_size = 100 # tốt nhất nên chia hết cho số lượng bản sao
model.fit(X_train, y_train, epochs=10, validation_data=(X_valid, y_valid),
          batch_size=batch_size)
```

Dưới lớp vỏ, Keras nhận thức được phân phối, vì vậy trong ngữ cảnh `MirroredStrategy` này, nó biết rằng nó phải nhân bản tất cả các biến và phép toán trên tất cả các thiết bị GPU khả dụng. Nếu bạn nhìn vào trọng số của mô hình, chúng có kiểu `MirroredVariable`:

```
>>> type(model.weights[0])
<class 'tensorflow.python.distribute.values.MirroredVariable'>
```

Lưu ý rằng phương thức `fit()` sẽ tự động chia mỗi lô huấn luyện trên tất cả các bản sao, vì vậy tốt hơn là đảm bảo rằng kích thước lô chia hết cho số lượng bản sao (tức là số lượng GPU khả dụng) để tất cả các bản sao nhận được các lô có cùng kích thước. Và thế là xong! Quá trình huấn luyện nói chung sẽ nhanh hơn đáng kể so với việc sử dụng một thiết bị, và thay đổi mã là rất nhỏ.

Khi bạn đã hoàn thành việc huấn luyện mô hình của mình, bạn có thể sử dụng nó để đưa ra dự đoán một cách hiệu quả: gọi phương thức `predict()`, và nó sẽ tự động chia lô trên tất cả các bản sao, thực hiện dự đoán song song. Một lần nữa, kích thước lô phải chia hết cho số lượng bản sao. Nếu bạn gọi phương thức `save()` của mô hình, nó sẽ được lưu dưới dạng một mô hình thông thường, không phải là một mô hình được nhân bản với nhiều bản sao. Vì vậy, khi bạn tải nó, nó sẽ chạy như một mô hình thông thường, trên một thiết bị duy nhất: theo mặc định trên GPU #0, hoặc trên CPU nếu không có GPU. Nếu bạn muốn tải một mô hình và chạy nó trên tất cả các thiết bị khả dụng, bạn phải gọi `tf.keras.models.load_model()` trong một ngữ cảnh phân phối:

```
# Giả định strategy đã được định nghĩa và "my_mirrored_model" là đường dẫn
đến mô hình đã lưu
with strategy.scope():
    model = tf.keras.models.load_model("my_mirrored_model")
```

Nếu bạn chỉ muốn sử dụng một tập hợp con của tất cả các thiết bị GPU khả dụng, bạn có thể truyền danh sách đó cho hàm tạo của `MirroredStrategy`:

```
strategy = tf.distribute.MirroredStrategy(devices=["/gpu:0", "/gpu:1"])
```

Theo mặc định, lớp `MirroredStrategy` sử dụng NVIDIA Collective Communications Library (NCCL) cho phép toán AllReduce trung bình, nhưng bạn có thể thay đổi nó bằng cách đặt

đối số `cross_device_ops` thành một instance của lớp `tf.distribute.HierarchicalCopyAllReduce`, hoặc một instance của lớp `tf.distribute.ReductionToOneDevice`. Tùy chọn NCCL mặc định dựa trên lớp `tf.distribute.NcclAllReduce`, thường nhanh hơn, nhưng điều này phụ thuộc vào số lượng và loại GPU, vì vậy bạn có thể muốn thử các lựa chọn thay thế.

Nếu bạn muốn thử sử dụng song song dữ liệu với các tham số tập trung, hãy thay thế `MirroredStrategy` bằng `CentralStorageStrategy`:

```
strategy = tf.distribute.experimental.CentralStorageStrategy()
```

Bạn có thể tùy chọn đặt đối số `compute_devices` để chỉ định danh sách các thiết bị bạn muốn sử dụng làm workers — theo mặc định nó sẽ sử dụng tất cả các GPU khả dụng — và bạn có thể tùy chọn đặt đối số `parameter_device` để chỉ định thiết bị bạn muốn lưu trữ các tham số. Theo mặc định nó sẽ sử dụng CPU, hoặc GPU nếu chỉ có một.

Bây giờ hãy xem cách huấn luyện một mô hình trên một cụm máy chủ TensorFlow!

Huấn luyện Mô hình trên Cụm TensorFlow

Một cụm TensorFlow là một nhóm các tiến trình TensorFlow chạy song song, thường trên các máy khác nhau, và giao tiếp với nhau để hoàn thành một công việc — ví dụ, huấn luyện hoặc thực thi một mô hình mạng nơ-ron. Mỗi tiến trình TF trong cụm được gọi là một **tác vụ (task)**, hoặc một **máy chủ TF (TF server)**. Nó có một địa chỉ IP, một cổng, và một loại (còn được gọi là vai trò hoặc công việc của nó). Loại này có thể là “worker”, “chief”, “ps” (parameter server), hoặc “evaluator”:

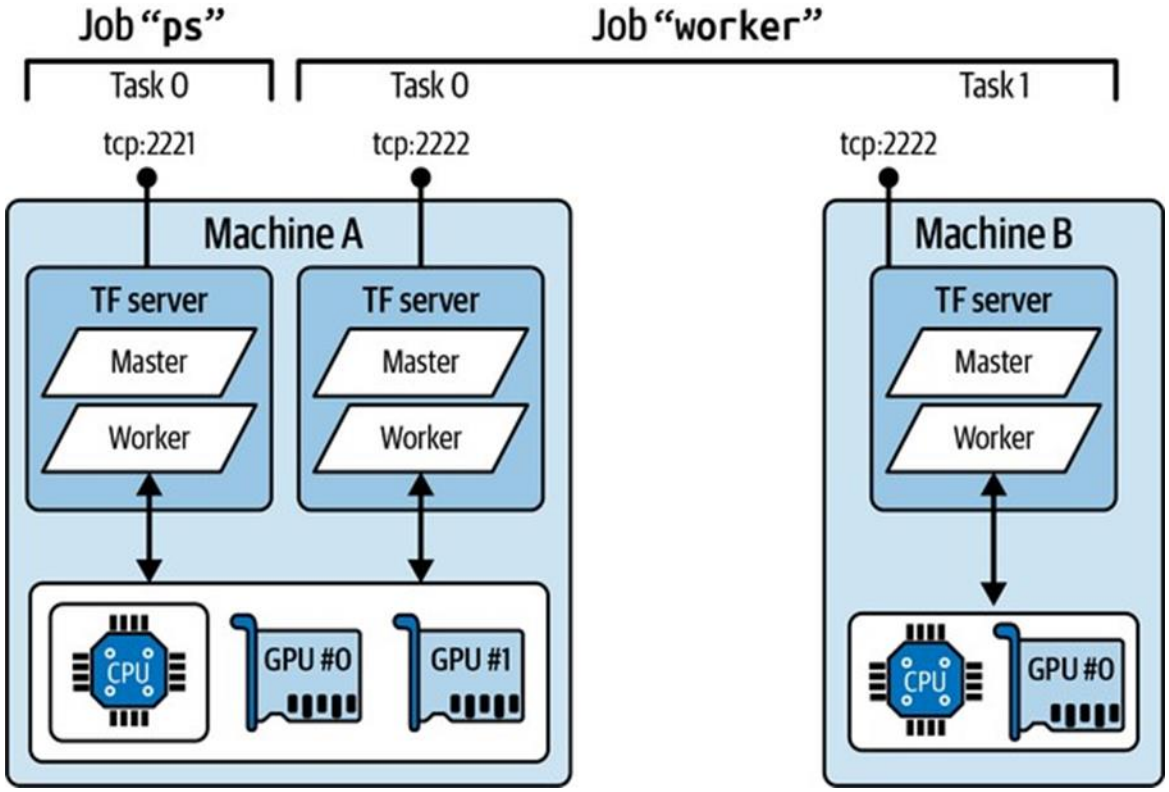
- Mỗi **worker** thực hiện các phép tính, thường trên một máy có một hoặc nhiều GPU.
- **Chief** cũng thực hiện các phép tính (nó là một worker), nhưng nó cũng xử lý các công việc bổ sung như ghi nhật ký TensorBoard hoặc lưu điểm kiểm tra (checkpoints). Có một chief duy nhất trong một cụm. Nếu không có chief nào được chỉ định rõ ràng, thì theo quy ước, worker đầu tiên là chief.
- Một **parameter server** chỉ theo dõi các giá trị biến, và nó thường nằm trên một máy chỉ có CPU. Loại tác vụ này chỉ được sử dụng với `ParameterServerStrategy`.
- Một **evaluator** rõ ràng là chịu trách nhiệm đánh giá. Loại này không được sử dụng thường xuyên, và khi được sử dụng, thường chỉ có một evaluator.

Để khởi động một cụm TensorFlow, bạn phải xác định đặc tả của nó. Điều này có nghĩa là xác định địa chỉ IP, cổng TCP và loại của mỗi tác vụ. Ví dụ, đặc tả cụm sau đây định nghĩa một cụm với ba tác vụ (hai worker và một parameter server; xem Hình 19-18). Đặc tả cụm là một dictionary với một khóa cho mỗi công việc, và các giá trị là danh sách các địa chỉ tác vụ (IP:cổng):

```
cluster_spec = {
    "worker": [
        "machine-a.example.com:2222",  # /job:worker/task:0
        "machine-b.example.com:2222"   # /job:worker/task:1
```

```
    ],
    "ps": ["machine-a.example.com:2221"] # /job:ps/task:0
}
```

Nói chung sẽ có một tác vụ duy nhất cho mỗi máy, nhưng như ví dụ này cho thấy, bạn có thể cấu hình nhiều tác vụ trên cùng một máy nếu muốn. Trong trường hợp này, nếu chúng chia sẻ cùng GPU, hãy đảm bảo RAM được chia thích hợp, như đã thảo luận trước đó.



Hình 19-18. Một ví dụ về cụm TensorFlow.

Khi bạn khởi động một tác vụ, bạn phải cung cấp cho nó đặc tả cụm, và bạn cũng phải cho nó biết loại và chỉ mục của nó (ví dụ: worker #0). Cách đơn giản nhất để chỉ định mọi thứ cùng một lúc (cả đặc tả cụm và loại và chỉ mục của tác vụ hiện tại) là đặt biến môi trường `TF_CONFIG` trước khi khởi động TensorFlow. Nó phải là một dictionary được mã hóa JSON chứa một đặc tả cụm (dưới khóa "cluster") và loại và chỉ mục của tác vụ hiện tại (dưới khóa "task"). Ví dụ, biến môi trường `TF_CONFIG` sau đây sử dụng cụm chúng ta vừa định nghĩa và chỉ định rằng tác vụ sẽ khởi động là worker #0:

```
import os
import json

os.environ["TF_CONFIG"] = json.dumps({
    "cluster": cluster_spec,
    "task": {"type": "worker", "index": 0}
})
```

Bây giờ hãy huấn luyện một mô hình trên một cụm! Chúng ta sẽ bắt đầu với chiến lược nhân bản (mirrored strategy). Đầu tiên, bạn cần đặt biến môi trường TF_CONFIG thích hợp cho mỗi tác vụ. Sẽ không có máy chủ tham số (xóa khóa “ps” trong đặc tả cụm), và nói chung bạn sẽ muốn một worker duy nhất cho mỗi máy. Đảm bảo đặt một chỉ mục tác vụ khác nhau cho mỗi tác vụ. Cuối cùng, chạy script sau trên mỗi worker:

```
import tempfile
import tensorflow as tf
# Giả định X_train, y_train, X_valid, y_valid đã được tải và chia
# Giả định os.environ["TF_CONFIG"] đã được thiết lập đúng cho từng tác vụ

strategy = tf.distribute.MultiWorkerMirroredStrategy() # ở phần đầu!
resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()
print(f"Starting task {resolver.task_type} #{resolver.task_id}")

# ... Tải và chia tập dữ liệu MNIST (các đoạn mã từ trước) ...
# Ví dụ đơn giản:
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train = X_train.astype('float32') / 255.0
X_valid = X_valid.astype('float32') / 255.0 # Bạn có thể cần tạo
X_valid/y_valid từ X_train/y_train
y_train = tf.keras.utils.to_categorical(y_train, num_classes=10)
y_valid = tf.keras.utils.to_categorical(y_valid, num_classes=10)

with strategy.scope():
    model = tf.keras.Sequential([ # xây dựng mô hình Keras
        tf.keras.layers.Flatten(input_shape=(28, 28)),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy']) # biên dịch mô hình

model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=10)

if resolver.task_id == 0: # chief lưu mô hình vào đúng vị trí
    model.save("my_mnist_multiworker_model", save_format="tf")
else:
    tmpdir = tempfile.mkdtemp() # các worker khác lưu vào thư mục tạm thời
    model.save(tmpdir, save_format="tf")
    tf.io.gfile.rmtree(tmpdir) # và chúng ta có thể xóa thư mục này ở cuối!
```

Đó gần như là cùng một mã bạn đã sử dụng trước đó, ngoại trừ lần này bạn đang sử dụng MultiWorkerMirroredStrategy. Khi bạn khởi động script này trên các worker đầu tiên, chúng sẽ bị chặn ở bước AllReduce, nhưng quá trình huấn luyện sẽ bắt đầu ngay khi worker cuối cùng khởi động, và bạn sẽ thấy tất cả chúng cùng tiến độ ở cùng một tốc độ chính xác vì chúng đồng bộ hóa ở mỗi bước.

Có hai triển khai AllReduce cho chiến lược phân phối này: một thuật toán AllReduce vòng dựa trên gRPC cho giao tiếp mạng, và triển khai của NCCL. Thuật toán tốt nhất để sử dụng phụ thuộc vào số lượng worker, số lượng và loại GPU, và mạng. Theo mặc định, TensorFlow sẽ áp dụng một số phương pháp heuristic để chọn thuật toán phù hợp cho bạn, nhưng bạn có thể buộc NCCL (hoặc RING) như sau:

```
strategy = tf.distribute.MultiWorkerMirroredStrategy(
    communication_options=tf.distribute.experimental.CommunicationOptions(

implementation=tf.distribute.experimental.CollectiveCommunication.NCCL
    )
)
```

Nếu bạn muốn triển khai song song dữ liệu bất đồng bộ với các máy chủ tham số, hãy thay đổi chiến lược thành `ParameterServerStrategy`, thêm một hoặc nhiều máy chủ tham số, và cấu hình `TF_CONFIG` thích hợp cho mỗi tác vụ. Lưu ý rằng mặc dù các worker sẽ hoạt động bất đồng bộ, các bản sao trên mỗi worker sẽ hoạt động đồng bộ.

Cuối cùng, nếu bạn có quyền truy cập vào TPU trên Google Cloud — ví dụ, nếu bạn sử dụng Colab và bạn đặt loại bộ tăng tốc thành TPU — thì bạn có thể tạo một `TPUStrategy` như sau:

```
import tensorflow as tf

# Điều này cần được chạy ngay sau khi import TensorFlow.
resolver = tf.distribute.cluster_resolver.TPUClusterResolver()
tf.tpu.experimental.initialize_tpu_system(resolver)
strategy = tf.distribute.experimental.TPUStrategy(resolver)
# Sau đó, bạn có thể sử dụng chiến lược này như bình thường.
```

Bây giờ bạn đã có thể huấn luyện các mô hình trên nhiều GPU và nhiều máy chủ: hãy tự thưởng cho mình! Tuy nhiên, nếu bạn muốn huấn luyện một mô hình rất lớn, bạn sẽ cần nhiều GPU, trên nhiều máy chủ, điều này sẽ yêu cầu hoặc mua rất nhiều phần cứng hoặc quản lý rất nhiều máy ảo đám mây. Trong nhiều trường hợp, việc sử dụng dịch vụ đám mây chuyên lo việc cấp phát và quản lý tất cả cơ sở hạ tầng này cho bạn, ngay khi bạn cần, sẽ ít rắc rối và ít tốn kém hơn. Hãy xem cách thực hiện điều đó bằng cách sử dụng Vertex AI.

Phán đoán và Công thức Toán:

Trong phần này, các công thức toán học không được trình bày rõ ràng, nhưng các khái niệm về huấn luyện phân tán và tối ưu hóa hiệu suất được thảo luận rất sâu sắc. Dưới đây là những điểm có thể liên hệ với toán học:

1. **Phân chia Batch Size (Batch Size Splitting):** Khi sử dụng `MirroredStrategy`, `fit()` tự động chia `batch_size` cho số lượng bản sao (replica), tức là số lượng GPU. Nếu có N_GPU GPU, và `batch_size` là B , thì mỗi GPU sẽ nhận một mini-batch có kích

thước B/N_GPU . $$$$$$\ = \ $$ $$$$Để đảm bảo tất cả các bản sao nhận được các lô có cùng kích thước, B phải chia hết cho N_GPU .

2. **Thuật toán AllReduce:** Đây là một lớp thuật toán phân tán được sử dụng để tổng hợp (ví dụ: tính trung bình, tổng) các giá trị (như gradient) từ nhiều thiết bị và sau đó phân phối kết quả trở lại tất cả các thiết bị. Mặc dù không có công thức cụ thể, cơ sở của AllReduce liên quan đến các phép toán tuyến tính (tổng, trung bình) trên các vector/tensor.
    - **Reduce (ví dụ: sum):** Nếu mỗi thiết bị i có một vector gradient g_i , thì phép toán tổng hợp là $sum_i = 1^{N_GPU} g_i$.
    - **Broadcast:** Sau khi tổng hợp, kết quả được gửi lại cho tất cả các thiết bị. Các triển khai như NCCL (NVIDIA Collective Communications Library) tối ưu hóa các phép toán này để tận dụng kiến trúc phần cứng đặc biệt của GPU, giảm thiểu thời gian giao tiếp.
3. **Chiến lược Đồng bộ (Synchronous Updates):** Trong `MultiWorkerMirroredStrategy`, tất cả các worker đều đồng bộ hóa ở bước AllReduce. Điều này có nghĩa là mỗi worker phải đợi cho đến khi tất cả các worker khác đã tính toán gradient của chúng và tham gia vào phép toán AllReduce. Nếu T_comp là thời gian tính toán gradient trên một mini-batch và T_comm là thời gian giao tiếp (AllReduce), thì tổng thời gian cho một bước là: $$$$$$T*{step}* = *(T*\{comp, i}) + T_{comm} $$ $$$$Ở đây, $max(T_comp, i)$ ngụ ý rằng bước này bị giới hạn bởi worker chậm nhất.
4. **Chiến lược Bất đồng bộ (Asynchronous Updates):** Trong `ParameterServerStrategy` với cập nhật bất đồng bộ, các worker hoạt động độc lập. Khi một worker tính toán xong gradient, nó gửi ngay lập tức đến parameter server để cập nhật. Không có sự chờ đợi giữa các worker. Điều này có thể dẫn đến gradient lỗi thời (`stale gradients`) vì tham số có thể đã được cập nhật bởi các worker khác.

Văn bản mô tả các phương pháp kỹ thuật để tối ưu hóa việc sử dụng tài nguyên (CPU, GPU, RAM, băng thông) nhằm đạt được hiệu suất cao nhất trong quá trình huấn luyện mô hình học máy quy mô lớn.

Chạy các Tác vụ Huấn luyện Lớn trên Vertex AI

Vertex AI cho phép bạn tạo các tác vụ huấn luyện tùy chỉnh với mã huấn luyện của riêng bạn. Trên thực tế, bạn có thể sử dụng gần như cùng một mã huấn luyện như bạn sẽ sử dụng trên cụm TF của riêng bạn. Điều chính bạn phải thay đổi là nơi chief nên lưu mô hình, các điểm kiểm tra (checkpoints) và nhật ký TensorBoard. Thay vì lưu mô hình vào một thư mục cục bộ, chief phải lưu nó vào GCS, sử dụng đường dẫn được Vertex AI cung cấp trong biến môi trường `AIP_MODEL_DIR`. Đối với các điểm kiểm tra mô hình và nhật ký TensorBoard, bạn nên sử dụng các đường dẫn trong các biến môi trường `AIP_CHECKPOINT_DIR` và `AIP_TENSORBOARD_LOG_DIR`, tương ứng. Tất nhiên, bạn cũng phải đảm bảo rằng dữ liệu huấn luyện có thể được truy cập từ các máy ảo, chẳng hạn như trên

GCS, hoặc một dịch vụ GCP khác như BigQuery, hoặc trực tiếp từ web. Cuối cùng, Vertex AI đặt loại tác vụ "chief" một cách rõ ràng, vì vậy bạn nên xác định chief bằng cách sử dụng `resolver.task_type == "chief"` thay vì `resolver.task_id == 0`:

```
import os
import tempfile
from pathlib import Path
import tensorflow as tf

# Giả định các imports khác, tạo MultiWorkerMirroredStrategy và resolver đã
được thực hiện
# Ví dụ:
# strategy = tf.distribute.MultiWorkerMirroredStrategy()
# resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()
# print(f"Starting task {resolver.task_type} #{resolver.task_id}")

if resolver.task_type == "chief":
    model_dir = os.getenv("AIP_MODEL_DIR") # đường dẫn được cung cấp bởi
Vertex AI
    tensorboard_log_dir = os.getenv("AIP_TENSORBOARD_LOG_DIR")
    checkpoint_dir = os.getenv("AIP_CHECKPOINT_DIR")
else:
    tmp_dir = Path(tempfile.mkdtemp()) # các worker khác sử dụng thư mục tạm
thời
    model_dir = tmp_dir / "model"
    tensorboard_log_dir = tmp_dir / "logs"
    checkpoint_dir = tmp_dir / "ckpt"

callbacks = [
    tf.keras.callbacks.TensorBoard(log_dir=tensorboard_log_dir),
    tf.keras.callbacks.ModelCheckpoint(filepath=checkpoint_dir)
]
# ... xây dựng và biên dịch bằng cách sử dụng phạm vi chiến lược, giống như
trước đó ...
# Ví dụ: (X_train, y_train), (X_valid, y_valid) đã được tải và tiền xử lý
# with strategy.scope():
#     model = tf.keras.Sequential([...])
#     model.compile([...])

model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=10,
          callbacks=callbacks)
model.save(model_dir, save_format="tf")
```

Bây giờ bạn có thể tạo một tác vụ huấn luyện tùy chỉnh trên Vertex AI, dựa trên script này. Bạn sẽ cần chỉ định tên tác vụ, đường dẫn đến script huấn luyện của bạn, ảnh Docker để sử dụng cho huấn luyện, ảnh Docker để sử dụng cho dự đoán (sau khi huấn luyện), bất kỳ thư viện Python bổ sung nào bạn có thể cần, và cuối cùng là bucket mà Vertex AI nên sử dụng làm thư mục dàn dựng (staging directory) để lưu trữ script huấn luyện. Theo mặc định, đó

cũng là nơi script huấn luyện sẽ lưu mô hình đã huấn luyện, cũng như nhật ký TensorBoard và điểm kiểm tra mô hình (nếu có). Hãy tạo tác vụ:

```
from google.cloud import aiplatform

# Giả định bucket_name và server_image (đã định nghĩa trước đó)

custom_training_job = aiplatform.CustomTrainingJob(
    display_name="my_custom_training_job",
    script_path="my_vertex_ai_training_task.py", # Đảm bảo tên tệp này khớp
với tệp script của bạn
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest", #
Hoặc phiên bản mới hơn
    model_serving_container_image_uri=server_image,
    requirements=["gcsfs==2022.3.0"], # không cần thiết, đây chỉ là một ví dụ
    staging_bucket=f"gs://{bucket_name}/staging"
)
```

Và bây giờ hãy chạy nó trên hai worker, mỗi worker với hai GPU:

```
# Giả định custom_training_job đã được tạo

mnist_model2 = custom_training_job.run(
    machine_type="n1-standard-4",
    replica_count=2, # Số lượng worker
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2, # Số lượng GPU cho mỗi worker
)
```

Và thế là xong: Vertex AI sẽ cấp phát các nút tính toán bạn yêu cầu (trong hạn ngạch của bạn), và nó sẽ chạy script huấn luyện của bạn trên chúng. Khi tác vụ hoàn tất, phương thức `run()` sẽ trả về một mô hình đã huấn luyện mà bạn có thể sử dụng chính xác như mô hình bạn đã tạo trước đó: bạn có thể triển khai nó đến một điểm cuối, hoặc sử dụng nó để thực hiện dự đoán hàng loạt. Nếu có bất kỳ điều gì không ổn trong quá trình huấn luyện, bạn có thể xem nhật ký trong bảng điều khiển GCP: trong menu điều hướng ☰, chọn Vertex AI → Training, nhấp vào tác vụ huấn luyện của bạn, và nhấp vào VIEW LOGS. Ngoài ra, bạn có thể nhấp vào tab CUSTOM JOBS và sao chép ID tác vụ (ví dụ: 1234), sau đó chọn Logging từ menu điều hướng ☰ và truy vấn `resource.labels.job_id=1234`.

Nếu bạn muốn thử một vài giá trị siêu tham số, một lựa chọn là chạy nhiều tác vụ. Bạn có thể truyền các giá trị siêu tham số vào script của bạn dưới dạng đối số dòng lệnh bằng cách đặt tham số `args` khi gọi phương thức `run()`, hoặc bạn có thể truyền chúng dưới dạng biến môi trường bằng cách sử dụng tham số `environment_variables`.

Tuy nhiên, nếu bạn muốn chạy một tác vụ điều chỉnh siêu tham số lớn trên đám mây, một lựa chọn tốt hơn nhiều là sử dụng dịch vụ điều chỉnh siêu tham số của Vertex AI. Hãy xem cách thực hiện.

Điều chỉnh Siêu Tham số trên Vertex AI

Dịch vụ điều chỉnh siêu tham số của Vertex AI dựa trên một thuật toán tối ưu hóa Bayes (Bayesian optimization algorithm), có khả năng nhanh chóng tìm ra các kết hợp tối ưu của siêu tham số. Để sử dụng nó, trước tiên bạn cần tạo một script huấn luyện chấp nhận các giá trị siêu tham số dưới dạng đối số dòng lệnh. Ví dụ, script của bạn có thể sử dụng thư viện chuẩn argparse như sau:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--n_hidden", type=int, default=2)
parser.add_argument("--n_neurons", type=int, default=256)
parser.add_argument("--learning_rate", type=float, default=1e-2)
parser.add_argument("--optimizer", default="adam")
args = parser.parse_args()
```

Dịch vụ điều chỉnh siêu tham số sẽ gọi script của bạn nhiều lần, mỗi lần với các giá trị siêu tham số khác nhau: mỗi lần chạy được gọi là một **thử nghiệm (trial)**, và tập hợp các thử nghiệm được gọi là một **nghiên cứu (study)**.

Script huấn luyện của bạn sau đó phải sử dụng các giá trị siêu tham số đã cho để xây dựng và biên dịch một mô hình. Bạn có thể sử dụng chiến lược phân phối nhân bản nếu muốn, trong trường hợp mỗi thử nghiệm chạy trên một máy đa GPU. Sau đó, script có thể tải tập dữ liệu và huấn luyện mô hình. Ví dụ:

```
import tensorflow as tf
# Giả định argparse đã được thiết lập và args đã có các giá trị siêu tham số

def build_model(args):
    with tf.distribute.MirroredStrategy().scope():
        model = tf.keras.Sequential()
        model.add(tf.keras.layers.Flatten(input_shape=[28, 28],
dtype=tf.uint8))
        for _ in range(args.n_hidden):
            model.add(tf.keras.layers.Dense(args.n_neurons,
activation="relu"))
        model.add(tf.keras.layers.Dense(10, activation="softmax"))
        opt = tf.keras.optimizers.get(args.optimizer)
        opt.learning_rate = args.learning_rate
        model.compile(loss="sparse_categorical_crossentropy",
                      optimizer=opt,
                      metrics=["accuracy"])
    return model

# ... tải tập dữ liệu (X_train, y_train, X_valid, y_valid) ...
# Ví dụ:
# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
# X_train = X_train.astype('float32') / 255.0
```

```
# X_valid = X_valid.astype('float32') / 255.0 # hoặc chia từ X_train
# y_train_sparse = y_train # Nếu sparse_categorical_crossentropy, y_train
không cần one-hot

model = build_model(args)
history = model.fit(X_train, y_train_sparse, validation_data=(X_valid,
y_valid_sparse), epochs=10)
```

Cuối cùng, script phải báo cáo hiệu suất của mô hình trở lại dịch vụ điều chỉnh siêu tham số của Vertex AI, để nó có thể quyết định các siêu tham số nào sẽ thử tiếp theo. Để làm điều này, bạn phải sử dụng thư viện hypertune, thư viện này được tự động cài đặt trên các máy ảo huấn luyện của Vertex AI:

```
import hypertune
# Giả định history đã có từ model.fit()

hypertune = hypertune.HyperTune()
hypertune.report_hyperparameter_tuning_metric(
    hyperparameter_metric_tag="accuracy", # tên của chỉ số được báo cáo
    metric_value=max(history.history["val_accuracy"]), # giá trị chỉ số
    global_step=model.optimizer.iterations.numpy(),
)
```

Bây giờ script huấn luyện của bạn đã sẵn sàng, bạn cần định nghĩa loại máy bạn muốn chạy nó. Để làm điều này, bạn phải định nghĩa một `CustomJob`, mà Vertex AI sẽ sử dụng làm mẫu cho mỗi thử nghiệm:

```
# Giả định bucket_name đã được định nghĩa
from google.cloud import aiplatform

trial_job = aiplatform.CustomJob.from_local_script(
    display_name="my_search_trial_job",
    script_path="my_vertex_ai_trial.py", # đường dẫn đến script huấn luyện
của bạn
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest", #
Hoặc phiên bản mới hơn
    staging_bucket=f"gs://{bucket_name}/staging",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2, # trong ví dụ này, mỗi thử nghiệm sẽ có 2 GPU
)
```

Cuối cùng, bạn đã sẵn sàng tạo và chạy tác vụ điều chỉnh siêu tham số:

```
from google.cloud import
aiplatform_v1.services.hyperparameter_tuning_job.types.hyperparameter_tuning_
job as hpt_pb2
from google.cloud import aiplatform

# Giả định trial_job đã được tạo
```

```
hp_job = aiplatform.HyperparameterTuningJob(
    display_name="my_hp_search_job",
    custom_job=trial_job,
    metric_spec={"accuracy": "maximize"}, # Tên chỉ số phải khớp với tên
trong hypertune.report_hyperparameter_tuning_metric
    parameter_spec={
        "learning_rate":
aiplatform.hyperparameter_tuning.DoubleParameterSpec(min=1e-3, max=10,
scale="log"),
        "n_neurons":
aiplatform.hyperparameter_tuning.IntegerParameterSpec(min=1, max=300,
scale="linear"),
        "n_hidden":
aiplatform.hyperparameter_tuning.IntegerParameterSpec(min=1, max=10,
scale="linear"),
        "optimizer":
aiplatform.hyperparameter_tuning.CategoricalParameterSpec(["sgd", "adam"]),
    },
    max_trial_count=100,
    parallel_trial_count=20,
)
hp_job.run()
```

Ở đây, chúng ta yêu cầu Vertex AI tối đa hóa chỉ số có tên “accuracy”: tên này phải khớp với tên chỉ số được báo cáo bởi script huấn luyện. Chúng ta cũng định nghĩa không gian tìm kiếm, sử dụng thang đo log cho tốc độ học và thang đo tuyến tính (tức là đồng nhất) cho các siêu tham số khác. Tên siêu tham số phải khớp với các đối số dòng lệnh của script huấn luyện. Sau đó, chúng ta đặt số lượng thử nghiệm tối đa là 100, và số lượng thử nghiệm chạy song song tối đa là 20. Nếu bạn tăng số lượng thử nghiệm song song lên (ví dụ) 60, tổng thời gian tìm kiếm sẽ giảm đáng kể, lên đến 3 lần. Nhưng 60 thử nghiệm đầu tiên sẽ được bắt đầu song song, vì vậy chúng sẽ không được hưởng lợi từ phản hồi của các thử nghiệm khác. Do đó, bạn nên tăng số lượng thử nghiệm tối đa để bù đ đắp — ví dụ, lên khoảng 140.

Điều này sẽ mất khá nhiều thời gian. Khi tác vụ hoàn tất, bạn có thể lấy kết quả thử nghiệm bằng cách sử dụng `hp_job.trials`. Mỗi kết quả thử nghiệm được biểu diễn dưới dạng một đối tượng protobuf, chứa các giá trị siêu tham số và các chỉ số kết quả. Hãy tìm thử nghiệm tốt nhất:

```
import numpy as np

def get_final_metric(trial, metric_id):
    for metric in trial.final_measurement.metrics:
        if metric.metric_id == metric_id:
            return metric.value
    return None # Trả về None nếu không tìm thấy chỉ số

trials = hp_job.trials
```

```
trial_accuracies = [get_final_metric(trial, "accuracy")
                    for trial in trials if get_final_metric(trial,
"accuracy") is not None] # Lọc các thử nghiệm không có chỉ số

best_trial = None
if trial_accuracies:
    best_trial_index = np.argmax(trial_accuracies)
    best_trial = trials[best_trial_index]
else:
    print("No trials with 'accuracy' metric found.")

if best_trial:
    print(f">>> max(trial_accuracies) {max(trial_accuracies)}")
    print(f">>> best_trial.id '{best_trial.id}'")
    print(f">>> best_trial.parameters")
    # In parameters
    for param in best_trial.parameters:
        print(f"parameter_id: \"{param.parameter_id}\" value {{ {param.value}
}}")
```

```
>>> max(trial_accuracies) 0.977400004863739

>>> best_trial.id '98'

>>> best_trial.parameters
[parameter_id: "learning_rate" value { number_value: 0.001 },
parameter_id: "n_hidden" value { number_value: 8.0 }, parameter_id:
"n_neurons" value { number_value: 216.0 }, parameter_id:
"optimizer" value { string_value: "adam" }
]
```

Vậy là xong! Bây giờ bạn có thể lấy SavedModel của thử nghiệm này, tùy chọn huấn luyện thêm một chút, và triển khai nó vào sản xuất.

Điều chỉnh Siêu Tham số bằng Keras Tuner trên Vertex AI

Thay vì sử dụng dịch vụ điều chỉnh siêu tham số của Vertex AI, bạn có thể sử dụng Keras Tuner (đã giới thiệu trong Chương 10) và chạy nó trên các máy ảo Vertex AI. Keras Tuner cung cấp một cách đơn giản để mở rộng tìm kiếm siêu tham số bằng cách phân phối nó trên nhiều máy: nó chỉ yêu cầu thiết lập ba biến môi trường trên mỗi máy, sau đó chạy mã Keras Tuner thông thường của bạn trên mỗi máy. Bạn có thể sử dụng cùng một script chính xác trên tất cả các máy. Một trong các máy hoạt động như chief (tức là oracle), và các máy khác hoạt động như workers. Mỗi worker hỏi chief xem nên thử những giá trị siêu tham số nào, sau đó worker huấn luyện mô hình bằng cách sử dụng các giá trị siêu tham số này, và cuối cùng nó báo cáo hiệu suất của mô hình trở lại chief, chief sau đó có thể quyết định những giá trị siêu tham số nào worker nên thử tiếp theo.

Ba biến môi trường bạn cần thiết lập trên mỗi máy là:

- **KERASTUNER_TUNER_ID**: Biến này bằng “chief” trên máy chief, hoặc một định danh duy nhất trên mỗi máy worker, chẳng hạn như “worker0”, “worker1”, v.v.
- **KERASTUNER_ORACLE_IP**: Đây là địa chỉ IP hoặc tên máy chủ của máy chief. Chief bản thân nó thường nên sử dụng “0.0.0.0” để lắng nghe trên mọi địa chỉ IP trên máy.
- **KERASTUNER_ORACLE_PORT**: Đây là cổng TCP mà chief sẽ lắng nghe.

Bây giờ bạn đã có tất cả các công cụ và kiến thức cần thiết để tạo các kiến trúc mạng nơ-ron hiện đại và huấn luyện chúng ở quy mô lớn bằng cách sử dụng các chiến lược phân phối khác nhau, trên cơ sở hạ tầng của riêng bạn hoặc trên đám mây, và sau đó triển khai chúng ở bất cứ đâu. Nói cách khác, bạn bây giờ có siêu năng lực: hãy sử dụng chúng thật tốt!

Bài tập

1. Một SavedModel chứa gì? Làm thế nào để bạn kiểm tra nội dung của nó?
2. Khi nào bạn nên sử dụng TF Serving? Các tính năng chính của nó là gì? Một số công cụ bạn có thể sử dụng để triển khai nó là gì?
3. Làm thế nào để bạn triển khai một mô hình trên nhiều instance TF Serving?
4. Khi nào bạn nên sử dụng API gRPC thay vì API REST để truy vấn một mô hình được phục vụ bởi TF Serving?
5. Các cách khác nhau mà TFLite giảm kích thước mô hình để nó chạy trên thiết bị di động hoặc thiết bị nhúng là gì?
6. Huấn luyện nhận thức lượng tử hóa (quantization-aware training) là gì, và tại sao bạn lại cần nó?
7. Song song mô hình (model parallelism) và song song dữ liệu (data parallelism) là gì? Tại sao cái sau thường được khuyến nghị?
8. Khi huấn luyện một mô hình trên nhiều máy chủ, bạn có thể sử dụng những chiến lược phân phối nào? Làm thế nào để bạn chọn chiến lược nào để sử dụng?
9. Huấn luyện một mô hình (bất kỳ mô hình nào bạn thích) và triển khai nó lên TF Serving hoặc Google Vertex AI. Viết mã máy khách để truy vấn nó bằng API REST hoặc API gRPC. Cập nhật mô hình và triển khai phiên bản mới. Mã máy khách của bạn bây giờ sẽ truy vấn phiên bản mới. Quay lại phiên bản đầu tiên.
10. Huấn luyện bất kỳ mô hình nào trên nhiều GPU trên cùng một máy bằng cách sử dụng `MirroredStrategy` (nếu bạn không có quyền truy cập vào GPU, bạn có thể sử dụng Google Colab với thời gian chạy GPU và tạo hai GPU logic). Huấn luyện lại mô hình bằng cách sử dụng `CentralStorageStrategy` và so sánh thời gian huấn luyện.
11. Tinh chỉnh một mô hình tùy chọn của bạn trên Vertex AI, sử dụng Keras Tuner hoặc dịch vụ điều chỉnh siêu tham số của Vertex AI. Lời giải cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại https://homl.info/colab3 .

Trước khi chúng ta khép lại chương cuối cùng của cuốn sách này, tôi muốn cảm ơn bạn đã đọc nó đến đoạn cuối cùng. Tôi thực sự hy vọng rằng bạn đã có nhiều niềm vui khi đọc cuốn sách này như tôi khi viết nó, và rằng nó sẽ hữu ích cho các dự án của bạn, dù lớn hay nhỏ.

Nếu bạn tìm thấy lỗi, vui lòng gửi phản hồi. Nói chung, tôi rất muốn biết suy nghĩ của bạn, vì vậy đừng ngần ngại liên hệ với tôi qua O'Reilly, qua dự án GitHub `ageron/handson-ml3`, hoặc trên Twitter tại `@aureliengeron`.

Trong tương lai, lời khuyên tốt nhất của tôi dành cho bạn là hãy luyện tập và luyện tập: hãy cố gắng hoàn thành tất cả các bài tập (nếu bạn chưa làm), thử nghiệm với các sổ tay, tham gia Kaggle hoặc một số cộng đồng ML khác, xem các khóa học ML, đọc các bài báo, tham dự các hội nghị và gặp gỡ các chuyên gia. Mọi thứ thay đổi nhanh chóng, vì vậy hãy cố gắng cập nhật. Một số kênh YouTube thường xuyên trình bày các bài báo học sâu một cách rất chi tiết, theo cách dễ tiếp cận. Tôi đặc biệt giới thiệu các kênh của Yannic Kilcher, Letitia Parcalabescu, và Xander Steenbrugge.

Để có những cuộc thảo luận ML hấp dẫn và những hiểu biết cấp cao hơn, hãy nhớ xem ML Street Talk và kênh của Lex Fridman. Việc có một dự án cụ thể để làm việc cũng giúp ích rất nhiều, dù là cho công việc hay để giải trí (lý tưởng là cả hai), vì vậy nếu có bất cứ điều gì bạn luôn mơ ước được xây dựng, hãy thử sức!

Hãy làm việc một cách tăng dần; đừng vội vàng đạt được những điều to lớn ngay lập tức, nhưng hãy tập trung vào dự án của bạn và xây dựng nó từng chút một. Nó sẽ đòi hỏi sự kiên nhẫn và kiên trì, nhưng khi bạn có một con robot đi bộ, hoặc một chatbot hoạt động, hoặc bất cứ thứ gì khác bạn muốn xây dựng, nó sẽ vô cùng bổ ích!

Hy vọng lớn nhất của tôi là cuốn sách này sẽ truyền cảm hứng cho bạn để xây dựng một ứng dụng ML tuyệt vời sẽ mang lại lợi ích cho tất cả chúng ta. Nó sẽ là gì? —Aurélien Géron

Công thức Toán (Phán đoán):

Trong phần này, không có công thức toán học tường minh nào được trình bày. Các khái niệm chủ yếu liên quan đến việc cấu hình môi trường huấn luyện phân tán trên Vertex AI và các nguyên tắc đằng sau việc điều chỉnh siêu tham số.

Tuy nhiên, có một số điểm có thể liên hệ ngầm với toán học hoặc các nguyên tắc kỹ thuật:

12. **Chức năng `hypertune.report_hyperparameter_tuning_metric`:**
    - `metric_value=max(history.history["val_accuracy"])`: Đây là việc lấy giá trị cao nhất của độ chính xác trên tập validation trong suốt quá trình huấn luyện của một thử nghiệm. Hàm `max()` là một phép toán tìm giá trị lớn nhất trong một tập hợp.
    - `global_step=model.optimizer.iterations.numpy()`: `global_step` thường là số bước huấn luyện hoặc số lần lặp mà mô hình đã trải qua. Nó đại diện cho "thời điểm" mà chỉ số được báo cáo.
13. **Định nghĩa Không gian Tìm kiếm Siêu Tham số:**
    - `hpt.DoubleParameterSpec(min=1e-3, max=10, scale="log")` cho `learning_rate`: Điều này ngụ ý rằng `learning_rate` sẽ được lấy mẫu từ một phân phối logarit trong khoảng $[10^{-3}, 10]$. Nếu x là một biến được lấy mẫu

tuyến tính từ một khoảng, thì 10^x (hoặc e^x) sẽ tạo ra một phân phối logarit. Cụ thể, nếu $usimU(0,1)$ là một biến ngẫu nhiên đồng nhất trong khoảng [0, 1], thì một giá trị siêu tham số hp trên thang log trong khoảng $[min_val, max_val]$ có thể được lấy mẫu bằng công thức: $$$$$$*{base}(hp)* = \\{base}(min*val)* + *u* \(\\{base}(max*val)* - \\{base}(min_val))$$ $$$$Trong đó $base$ thường là 10 hoặc e .

- `hpt.IntegerParameterSpec(min=1, max=300, scale="linear")` cho `n_neurons` và `n_hidden`: Điều này ngụ ý rằng các số nguyên sẽ được lấy mẫu đồng nhất trong khoảng đã cho.
- `hpt.CategoricalParameterSpec(["sgd", "adam"])` cho `optimizer`: Đây là việc chọn một giá trị từ một tập hợp rời rạc.

14. **Số lượng thử nghiệm song song (`parallel_trial_count`) và tổng thời gian tìm kiếm:** Văn bản đề cập rằng nếu tăng `parallel_trial_count` từ 20 lên 60, tổng thời gian tìm kiếm có thể giảm 3 lần (a factor of up to 3). Điều này ngụ ý một mối quan hệ nghịch đảo giữa số lượng thử nghiệm song song và thời gian hoàn thành (trong điều kiện lý tưởng): $$$$$$\ = \ $$ $$$$Tuy nhiên, đây chỉ là một ước tính lý tưởng vì hiệu quả thực tế bị ảnh hưởng bởi các yếu tố như phụ thuộc giữa các thử nghiệm, overhead của hệ thống, và hiệu quả của thuật toán tối ưu hóa Bayes. Các thử nghiệm song song không thể hưởng lợi từ phản hồi của nhau ngay lập tức, do đó cần tăng `max_trial_count` để bù đ đắp.

Các công thức trên là những suy luận toán học từ các nguyên tắc được mô tả trong văn bản, chứ không phải là công thức tường minh.

PHỤ LỤC A. DANH SÁCH KIỂM TRA DỰ ÁN HỌC MÁY

Danh sách kiểm tra này có thể hướng dẫn bạn thực hiện các dự án học máy của mình. Có tám bước chính:

1. Xác định vấn đề và nhìn vào bức tranh lớn.
2. Lấy dữ liệu.
3. Khám phá dữ liệu để hiểu rõ hơn.
4. Chuẩn bị dữ liệu để làm nổi bật các mẫu dữ liệu tiềm ẩn cho thuật toán học máy.
5. Khám phá nhiều mô hình khác nhau và lập danh sách rút gọn những mô hình tốt nhất.
6. Tinh chỉnh các mô hình của bạn và kết hợp chúng thành một giải pháp tuyệt vời.
7. Trình bày giải pháp của bạn.
8. Khởi chạy, giám sát và bảo trì hệ thống của bạn.

Rõ ràng, bạn nên thoải mái điều chỉnh danh sách kiểm tra này cho phù hợp với nhu cầu của mình.

Xác định Vấn đề và Nhìn vào Bức tranh lớn

- Xác định mục tiêu bằng các thuật ngữ kinh doanh.
- Giải pháp của bạn sẽ được sử dụng như thế nào?
- Các giải pháp/cách giải quyết hiện tại (nếu có) là gì?
- Bạn nên xác định vấn đề này như thế nào (có giám sát/không giám sát, trực tuyến/ngoại tuyến, v.v.)?
- Hiệu suất nên được đo lường như thế nào?
- Biện pháp hiệu suất có phù hợp với mục tiêu kinh doanh không?
- Hiệu suất tối thiểu cần thiết để đạt được mục tiêu kinh doanh là gì?
- Có những vấn đề nào tương tự? Bạn có thể tái sử dụng kinh nghiệm hoặc công cụ không?
- Có chuyên môn của con người không?
- Bạn sẽ giải quyết vấn đề bằng cách thủ công như thế nào?
- Liệt kê các giả định bạn (hoặc người khác) đã đưa ra cho đến nay.
- Xác minh các giả định nếu có thể.

Lấy Dữ liệu

Lưu ý: tự động hóa càng nhiều càng tốt để bạn có thể dễ dàng lấy dữ liệu mới.

- Liệt kê dữ liệu bạn cần và số lượng bạn cần.
- Tìm và ghi lại nơi bạn có thể lấy dữ liệu đó.
- Kiểm tra dung lượng nó sẽ chiếm.
- Kiểm tra các nghĩa vụ pháp lý và xin cấp phép nếu cần.

- Nhận quyền truy cập.
- Tạo không gian làm việc (với đủ dung lượng lưu trữ).
- Lấy dữ liệu.
- Chuyển đổi dữ liệu sang định dạng bạn có thể dễ dàng thao tác (mà không làm thay đổi dữ liệu).
- Đảm bảo thông tin nhạy cảm bị xóa hoặc bảo vệ (ví dụ: ẩn danh).
- Kiểm tra kích thước và loại dữ liệu (chuỗi thời gian, mẫu, địa lý, v.v.).
- Lấy mẫu một tập hợp thử nghiệm, đặt nó sang một bên và không bao giờ nhìn vào nó (không rình mò dữ liệu!).

Khám phá Dữ liệu

Lưu ý: cố gắng thu thập thông tin chuyên sâu từ một chuyên gia trong lĩnh vực này cho các bước này.

- Tạo một bản sao của dữ liệu để khám phá (lấy mẫu xuống kích thước dễ quản lý nếu cần).
- Tạo một sổ tay Jupyter để ghi lại quá trình khám phá dữ liệu của bạn.
- Nghiên cứu từng thuộc tính và các đặc điểm của nó:
    - Tên
    - Loại (phân loại, int/float, có giới hạn/không giới hạn, văn bản, có cấu trúc, v.v.)
    - % giá trị bị thiếu
    - Độ nhiễu và loại nhiễu (ngẫu nhiên, ngoại lệ, lỗi làm tròn, v.v.)
    - Mức độ hữu ích cho tác vụ
    - Loại phân phối (Gaussian, đồng nhất, logarit, v.v.)
- Đối với các tác vụ học có giám sát, xác định thuộc tính mục tiêu.
- Trực quan hóa dữ liệu.
- Nghiên cứu các mối tương quan giữa các thuộc tính.
- Nghiên cứu cách bạn sẽ giải quyết vấn đề bằng cách thủ công.
- Xác định các phép biến đổi tiềm năng mà bạn có thể muốn áp dụng.
- Xác định dữ liệu bổ sung sẽ hữu ích (quay lại phần “Lấy Dữ liệu”).
- Ghi lại những gì bạn đã học được.

Chuẩn bị Dữ liệu

Lưu ý:

- Làm việc trên các bản sao của dữ liệu (giữ nguyên tập dữ liệu gốc).
- Viết hàm cho tất cả các phép biến đổi dữ liệu bạn áp dụng, vì năm lý do:
    - Để bạn có thể dễ dàng chuẩn bị dữ liệu vào lần tới khi bạn nhận được một tập dữ liệu mới.

    - Để bạn có thể áp dụng các phép biến đổi này trong các dự án tương lai.
    - Để làm sạch và chuẩn bị tập kiểm tra.
    - Để làm sạch và chuẩn bị các instance dữ liệu mới sau khi giải pháp của bạn hoạt động trực tiếp.
    - Để dễ dàng coi các lựa chọn chuẩn bị của bạn là các siêu tham số.
- Làm sạch dữ liệu:
    - Sửa hoặc loại bỏ các giá trị ngoại lệ (tùy chọn).
    - Điền vào các giá trị bị thiếu (ví dụ: bằng 0, trung bình, trung vị...) hoặc loại bỏ các hàng (hoặc cột) của chúng.
- Thực hiện chọn tính năng (tùy chọn):
    - Loại bỏ các thuộc tính không cung cấp thông tin hữu ích cho tác vụ.
- Thực hiện kỹ thuật tính năng (feature engineering), khi thích hợp:
    - Rời rạc hóa các tính năng liên tục.
    - Phân tích các tính năng (ví dụ: phân loại, ngày/giờ, v.v.).
    - Thêm các phép biến đổi tiềm năng của tính năng (ví dụ: $\log(x)$, $\sqrt{x}$, x^2 , v.v.).
    - Tổng hợp các tính năng thành các tính năng mới đầy hứa hẹn.
- Thực hiện chia tỷ lệ tính năng (feature scaling):
    - Chuẩn hóa hoặc bình thường hóa các tính năng.

Lập danh sách rút gọn các Mô hình Tiềm năng

Lưu ý:

- Nếu dữ liệu rất lớn, bạn có thể muốn lấy mẫu các tập huấn luyện nhỏ hơn để bạn có thể huấn luyện nhiều mô hình khác nhau trong một thời gian hợp lý (lưu ý rằng điều này làm giảm hiệu suất của các mô hình phức tạp như mạng nơ-ron lớn hoặc rừng ngẫu nhiên).
- Một lần nữa, cố gắng tự động hóa các bước này càng nhiều càng tốt.
- Huấn luyện nhiều mô hình "nhanh chóng và sơ sài" từ các danh mục khác nhau (ví dụ: tuyến tính, naive Bayes, SVM, rừng ngẫu nhiên, mạng nơ-ron, v.v.) bằng cách sử dụng các tham số tiêu chuẩn.
- Đo lường và so sánh hiệu suất của chúng:
    - Đối với mỗi mô hình, sử dụng xác thực chéo N lần (N-fold cross-validation) và tính trung bình và độ lệch chuẩn của biện pháp hiệu suất trên N lần.
- Phân tích các biến quan trọng nhất cho mỗi thuật toán.
- Phân tích các loại lỗi mà các mô hình mắc phải:
    - Dữ liệu nào mà con người đã sử dụng để tránh những lỗi này?
- Thực hiện một vòng chọn tính năng và kỹ thuật tính năng nhanh chóng.
- Thực hiện thêm một hoặc hai lần lặp nhanh chóng các bước trước đó.
- Lập danh sách rút gọn ba đến năm mô hình tiềm năng nhất, ưu tiên các mô hình mắc các loại lỗi khác nhau.

Tinh chỉnh Hệ thống

Lưu ý:

- Bạn sẽ muốn sử dụng càng nhiều dữ liệu càng tốt cho bước này, đặc biệt là khi bạn tiến gần đến cuối quá trình tinh chỉnh.
- Như mọi khi, tự động hóa những gì bạn có thể.
- Tinh chỉnh các siêu tham số bằng cách sử dụng xác thực chéo:
    - Coi các lựa chọn biến đổi dữ liệu của bạn là các siêu tham số, đặc biệt khi bạn không chắc chắn về chúng (ví dụ: nếu bạn không chắc chắn nên thay thế các giá trị bị thiếu bằng số 0 hay bằng giá trị trung vị, hoặc chỉ đơn giản là loại bỏ các hàng).
    - Trừ khi có rất ít giá trị siêu tham số để khám phá, hãy ưu tiên tìm kiếm ngẫu nhiên (random search) hơn tìm kiếm lưới (grid search). Nếu quá trình huấn luyện rất dài, bạn có thể ưu tiên phương pháp tối ưu hóa Bayes (ví dụ: sử dụng Gaussian process priors, như được mô tả bởi Jasper Snoek et al.).
- Thử các phương pháp tập hợp (ensemble methods). Kết hợp các mô hình tốt nhất của bạn thường sẽ mang lại hiệu suất tốt hơn so với việc chạy chúng riêng lẻ.
- Khi bạn tin tưởng vào mô hình cuối cùng của mình, hãy đo lường hiệu suất của nó trên tập kiểm tra để ước tính lỗi khái quát hóa (generalization error).

Trình bày Giải pháp của bạn

- Ghi lại những gì bạn đã làm.
- Tạo một bài thuyết trình đẹp mắt:
    - Đảm bảo bạn làm nổi bật bức tranh lớn trước tiên.
- Giải thích tại sao giải pháp của bạn đạt được mục tiêu kinh doanh.
- Đừng quên trình bày những điểm thú vị bạn nhận thấy trong quá trình:
    - Mô tả những gì đã hoạt động và những gì không.
    - Liệt kê các giả định và các hạn chế của hệ thống của bạn.
- Đảm bảo các phát hiện chính của bạn được truyền đạt thông qua các hình ảnh trực quan đẹp mắt hoặc các câu dễ nhớ (ví dụ: “thu nhập trung bình là yếu tố dự đoán số một về giá nhà ở”).

Khởi chạy!

- Chuẩn bị giải pháp của bạn để sản xuất (cắm vào đầu vào dữ liệu sản xuất, viết kiểm thử đơn vị, v.v.).
- Viết mã giám sát để kiểm tra hiệu suất trực tiếp của hệ thống của bạn theo định kỳ và kích hoạt cảnh báo khi nó giảm:
    - Cẩn thận với sự xuống cấp chậm: các mô hình có xu hướng “thối rữa” khi dữ liệu phát triển.
    - Việc đo lường hiệu suất có thể yêu cầu một quy trình của con người (ví dụ: thông qua dịch vụ crowdsourcing).

- Cũng giám sát chất lượng đầu vào của bạn (ví dụ: một cảm biến trục trặc gửi các giá trị ngẫu nhiên, hoặc đầu ra của một nhóm khác trở nên lỗi thời). Điều này đặc biệt quan trọng đối với các hệ thống học trực tuyến (online learning systems).

- Huấn luyện lại các mô hình của bạn một cách thường xuyên trên dữ liệu mới (tự động hóa càng nhiều càng tốt).

Phán đoán và Công thức Toán:

Phụ lục này là một danh sách kiểm tra dự án học máy, tập trung vào quy trình và các bước thực hành hơn là các công thức toán học cụ thể. Do đó, không có công thức toán học tường minh nào trong phần này.

Tuy nhiên, các khái niệm được đề cập có liên quan đến các nguyên tắc toán học và thống kê cơ bản trong học máy:

- **Đo lường Hiệu suất (Performance Measurement):**
  - "How should performance be measured?" và "Is the performance measure aligned with the business objective?": Điều này liên quan đến việc chọn các **metrics** phù hợp (ví dụ: độ chính xác, độ chính xác, độ thu hồi, F1-score, RMSE, MAE, R-squared). Việc lựa chọn metrics là một vấn đề thống kê và tối ưu hóa, đảm bảo rằng việc tối ưu hóa mô hình cũng tối ưu hóa mục tiêu kinh doanh.
  - "For each model, use N-fold cross-validation and compute the mean and standard deviation of the performance measure on the N folds":
    - **Cross-validation (Xác thực chéo):** Một kỹ thuật thống kê để ước tính hiệu suất của mô hình.
    - **Mean (Giá trị trung bình):** $\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$
    - **Standard Deviation (Độ lệch chuẩn):** $\sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2}$ Việc tính toán trung bình và độ lệch chuẩn của các metrics trên các fold giúp đánh giá sự ổn định và độ tin cậy của hiệu suất mô hình.
- **Kỹ thuật Tính năng (Feature Engineering):**
  - "Add promising transformations of features (e.g., $\log(x)$, $\sqrt{x}$, x^2 , etc.)": Đây là các phép biến đổi toán học cơ bản áp dụng lên các đặc trưng để giúp thuật toán học máy học tốt hơn từ dữ liệu. Ví dụ, $\log(x)$ có thể được dùng để chuẩn hóa các phân phối bị lệch, x^2 tạo ra các đặc trưng bậc hai để mô hình có thể học các mối quan hệ phi tuyến.
- **Chia tỷ lệ Tính năng (Feature Scaling):**
  - "Standardize or normalize features":
    - **Standardization (Chuẩn hóa Z-score):** $x' = \frac{x-\mu}{\sigma}$ (trong đó μ là trung bình, σ là độ lệch chuẩn).

    - **Normalization (Min-Max Scaling):** $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$ (chuẩn hóa về khoảng $[0,1]$). Các phép biến đổi này là các công thức toán học giúp đưa các đặc trưng về cùng một phạm vi, cải thiện hiệu suất của nhiều thuật toán học máy.
- **Tối ưu hóa Siêu Tham số (Hyperparameter Tuning):**
    - "Prefer random search over grid search. If training is very long, you may prefer a Bayesian optimization approach...": Đây là các chiến lược tìm kiếm siêu tham số, với Bayesian optimization sử dụng các mô hình xác suất (ví dụ: Gaussian processes) để ước tính hàm hiệu suất của siêu tham số, từ đó đưa ra lựa chọn thông minh hơn về các siêu tham số để thử tiếp theo. Mặc dù không có công thức cụ thể, nó dựa trên lý thuyết xác suất và tối ưu hóa.
- **Lỗi Khái quát hóa (Generalization Error):**
    - "measure its performance on the test set to estimate the generalization error": Đây là ước tính hiệu suất của mô hình trên dữ liệu chưa từng thấy. Mục tiêu của học máy là tối thiểu hóa lỗi khái quát hóa, thường được đo bằng các metrics đã chọn.
- **Giám sát Hệ thống (Monitoring):**
    - "Beware of slow degradation: models tend to "rot" as data evolves." (Khái niệm drift dữ liệu/mô hình)
    - "Also monitor your inputs' quality...": Các khái niệm này liên quan đến việc giám sát các phân phối thống kê của dữ liệu đầu vào và đầu ra theo thời gian để phát hiện sự thay đổi, điều này thường yêu cầu các phép đo thống kê liên tục.

Tóm lại, mặc dù không có công thức toán học phức tạp, toàn bộ phụ lục này được xây dựng dựa trên các nguyên tắc toán học, thống kê và tối ưu hóa cốt lõi của học máy.

PHỤ LỤC B. TỰ ĐỘNG VI PHÂN (AUTODIFF)

Phụ lục này giải thích cách hoạt động của tính năng tự động vi phân (autodiff) của TensorFlow, và cách nó so sánh với các giải pháp khác.

Giả sử bạn định nghĩa một hàm $f(x,y) = x^2y + y + 2$, và bạn cần các đạo hàm riêng $partialf/partialx$ và $partialf/partialy$, thường là để thực hiện tối ưu hóa gradient descent (hoặc một thuật toán tối ưu hóa khác). Các lựa chọn chính của bạn là vi phân thủ công, xấp xỉ sai phân hữu hạn, autodiff chế độ xuôi (forward-mode autodiff), và autodiff chế độ ngược (reverse-mode autodiff). TensorFlow triển khai autodiff chế độ ngược, nhưng để hiểu nó, hữu ích khi xem xét các lựa chọn khác trước. Vì vậy, hãy đi qua từng lựa chọn, bắt đầu với vi phân thủ công.

Vi Phân Thủ Công (Manual Differentiation)

Cách tiếp cận đầu tiên để tính đạo hàm là cầm bút chì và một tờ giấy và sử dụng kiến thức giải tích của bạn để tìm ra phương trình thích hợp. Đối với hàm $f(x,y)$ vừa định nghĩa, không quá khó; bạn chỉ cần sử dụng năm quy tắc:

- Đạo hàm của một hằng số là 0.
- Đạo hàm của $lambdax$ là $lambda$ (trong đó $lambda$ là một hằng số).
- Đạo hàm của x^{lambda} là $lambdax^{lambda-1}$, vì vậy đạo hàm của x^2 là $2x$.
- Đạo hàm của tổng các hàm là tổng các đạo hàm của các hàm đó.
- Đạo hàm của $lambda$ nhân với một hàm là $lambda$ nhân với đạo hàm của nó.

Từ các quy tắc này, bạn có thể tìm ra Phương trình B-1.

Phương trình B-1. Các đạo hàm riêng của $f(x,y)$

$$\frac{\partial f}{\partial x} = \frac{\partial(x^2y)}{\partial x} + \frac{\partial y}{\partial x} + \frac{\partial 2}{\partial x} = y\frac{\partial x^2}{\partial x} + 0 + 0 = y(2x) = 2xy$$

$$\frac{\partial f}{\partial y} = \frac{\partial(x^2y)}{\partial y} + \frac{\partial y}{\partial y} + \frac{\partial 2}{\partial y} = x^2\frac{\partial y}{\partial y} + 1 + 0 = x^2(1) + 1 = x^2 + 1$$

Cách tiếp cận này có thể trở nên rất tẻ nhạt đối với các hàm phức tạp hơn, và bạn có nguy cơ mắc lỗi. May mắn thay, có những lựa chọn khác. Bây giờ chúng ta hãy xem xét xấp xỉ sai phân hữu hạn.

Xấp Xỉ Sai Phân Hữu Hạn (Finite Difference Approximation)

Nhắc lại rằng đạo hàm $h'(x_0)$ của một hàm $h(x)$ tại một điểm x_0 là độ dốc của hàm tại điểm đó. Chính xác hơn, đạo hàm được định nghĩa là giới hạn của độ dốc của một đường thẳng đi qua điểm x_0 này và một điểm x khác trên hàm, khi x tiến gần vô hạn đến x_0 (xem Phương trình B-2).

Phương trình B-2. Định nghĩa đạo hàm của một hàm $h(x)$ tại điểm x_0

$$h'(x_0) = \lim_{x \to x_0} \frac{h(x) - h(x_0)}{x - x_0} = \lim_{\epsilon \to 0} \frac{h(x_0 + \epsilon) - h(x_0)}{\epsilon}$$

Vì vậy, nếu chúng ta muốn tính đạo hàm riêng của $f(x, y)$ đối với x tại $x = 3$ và $y = 4$, chúng ta có thể tính $f(3 + epsilon, 4) - f(3,4)$ và chia kết quả cho $epsilon$, sử dụng một giá trị rất nhỏ cho $epsilon$. Loại xấp xỉ số học này của đạo hàm được gọi là **xấp xỉ sai phân hữu hạn (finite difference approximation)**, và phương trình cụ thể này được gọi là **thương số sai phân của Newton (Newton's difference quotient)**. Đó chính xác là những gì mã sau đây thực hiện:

```
def f(x, y):
    return x**2 * y + y + 2

def derivative(f, x, y, x_eps, y_eps):
    # Đây là một hàm helper để minh họa ý tưởng, không phải cách tính đạo hàm
chính xác
    # Nếu x_eps và y_eps đều khác 0, nó tính đạo hàm theo hướng chéo, không
phải đạo hàm riêng
    # Để tính đạo hàm riêng, một trong x_eps hoặc y_eps phải là 0
    return (f(x + x_eps, y + y_eps) - f(x, y)) / (x_eps + y_eps)

# Để tính đạo hàm riêng theo x:
# x_eps = một số rất nhỏ, y_eps = 0
df_dx = (f(3 + 0.00001, 4) - f(3, 4)) / 0.00001
# Để tính đạo hàm riêng theo y:
# x_eps = 0, y_eps = một số rất nhỏ
df_dy = (f(3, 4 + 0.00001) - f(3, 4)) / 0.00001
```

Thật không may, kết quả không chính xác (và nó trở nên tệ hơn đối với các hàm phức tạp hơn). Các kết quả chính xác lần lượt là 24 và 10, nhưng thay vào đó chúng ta nhận được:

```
>>> df_dx 24.000039999805264
>>> df_dy 10.000000000331966
```

Lưu ý rằng để tính cả hai đạo hàm riêng, chúng ta phải gọi hàm `f()` ít nhất ba lần (chúng ta đã gọi nó bốn lần trong mã trước, nhưng có thể tối ưu hóa). Nếu có 1.000 tham số, chúng ta sẽ cần gọi `f()` ít nhất 1.001 lần. Khi bạn đang xử lý các mạng nơ-ron lớn, điều này làm cho xấp xỉ sai phân hữu hạn trở nên quá không hiệu quả.

Tuy nhiên, phương pháp này rất đơn giản để triển khai nên nó là một công cụ tuyệt vời để kiểm tra xem các phương pháp khác có được triển khai đúng cách hay không. Ví dụ, nếu nó không khớp với hàm bạn đã đạo hàm thủ công, thì hàm của bạn có thể chứa lỗi.

Cho đến nay, chúng ta đã xem xét hai cách để tính gradient: sử dụng vi phân thủ công và sử dụng xấp xỉ sai phân hữu hạn. Thật không may, cả hai đều có những hạn chế nghiêm trọng khi huấn luyện một mạng nơ-ron quy mô lớn. Vì vậy, hãy chuyển sang autodiff, bắt đầu với chế độ xuôi.

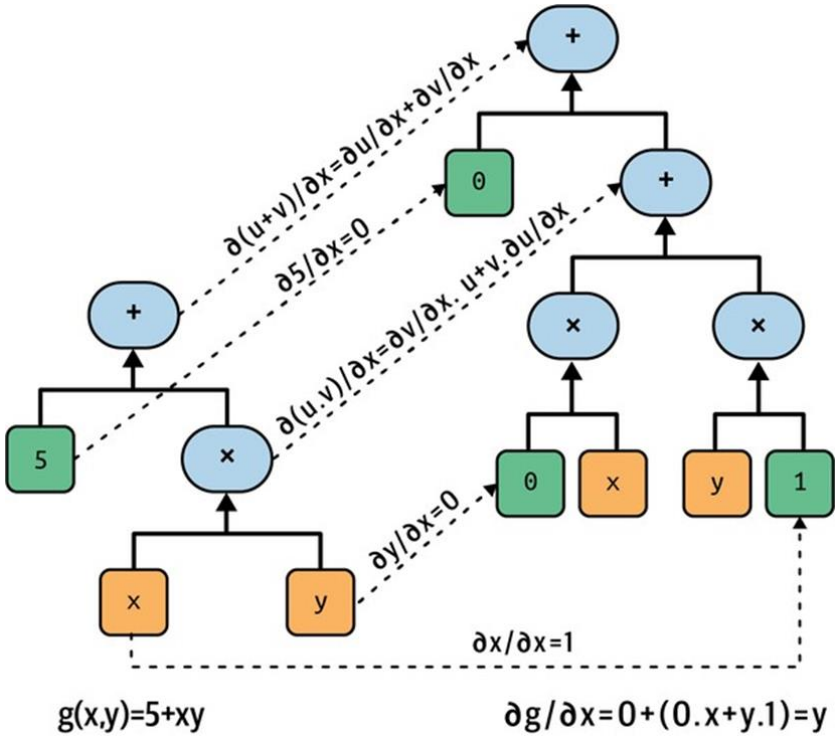
Autodiff Chế Độ Xuôi (Forward-Mode Autodiff)

Hình B-1 cho thấy cách hoạt động của autodiff chế độ xuôi trên một hàm thậm chí còn đơn giản hơn, $g(x, y) = 5 + xy$. Đồ thị cho hàm đó được biểu diễn ở bên trái. Sau autodiff chế độ xuôi, chúng ta có đồ thị ở bên phải, biểu diễn đạo hàm riêng $partialg/partialx = 0 + (0timesx + ytimes1) = y$ (chúng ta cũng có thể tìm đạo hàm riêng đối với y một cách tương tự).

Thuật toán sẽ đi qua đồ thị tính toán từ đầu vào đến đầu ra (do đó có tên "chế độ xuôi"). Nó bắt đầu bằng cách lấy các đạo hàm riêng của các nút lá. Nút hằng số (5) trả về hằng số 0, vì đạo hàm của một hằng số luôn là 0. Biến x trả về hằng số 1 vì $partialx/partialx = 1$, và biến y trả về hằng số 0 vì $partialy/partialx = 0$ (nếu chúng ta đang tìm đạo hàm riêng đối với y , nó sẽ ngược lại).

Bây giờ chúng ta có tất cả những gì cần thiết để di chuyển lên đồ thị đến nút nhân trong hàm g . Giải tích cho chúng ta biết rằng đạo hàm của tích của hai hàm u và v là $partial(utimesv)/partialx = partialv/partialxtimesu + vtimespartialu/partialx$. Do đó, chúng ta có thể xây dựng một phần lớn của đồ thị ở bên phải, biểu diễn $0timesx + ytimes1$.

Cuối cùng, chúng ta có thể đi lên nút cộng trong hàm g . Như đã đề cập, đạo hàm của tổng các hàm là tổng các đạo hàm của các hàm đó, vì vậy chúng ta chỉ cần tạo một nút cộng và kết nối nó với các phần của đồ thị mà chúng ta đã tính toán. Chúng ta nhận được đạo hàm riêng chính xác: $partialg/partialx = 0 + (0timesx + ytimes1)$.



Hình B-1. Autodiff chế độ xuôi.

Tuy nhiên, phương trình này có thể được đơn giản hóa (rất nhiều). Bằng cách áp dụng một vài bước cắt tỉa đồ thị tính toán để loại bỏ tất cả các phép toán không cần thiết, chúng ta có được một đồ thị nhỏ hơn nhiều chỉ với một nút: $partialg/partialx = y$. Trong trường hợp này, việc đơn giản hóa khá dễ dàng, nhưng đối với một hàm phức tạp hơn, autodiff chế độ xuôi có thể tạo ra một đồ thị khổng lồ mà có thể khó đơn giản hóa và dẫn đến hiệu suất không tối ưu.

Lưu ý rằng chúng ta bắt đầu với một đồ thị tính toán, và autodiff chế độ xuôi tạo ra một đồ thị tính toán khác. Đây được gọi là **vi phân ký hiệu (symbolic differentiation)**, và nó có hai tính năng hay: thứ nhất, một khi đồ thị tính toán của đạo hàm đã được tạo ra, chúng ta có thể sử dụng nó bao nhiêu lần tùy thích để tính đạo hàm của hàm đã cho với bất kỳ giá trị nào của x và y ; thứ hai, chúng ta có thể chạy lại autodiff chế độ xuôi trên đồ thị kết quả để nhận được đạo hàm bậc hai nếu chúng ta cần (tức là đạo hàm của đạo hàm). Chúng ta thậm chí có thể tính đạo hàm bậc ba, v.v.

Nhưng cũng có thể chạy autodiff chế độ xuôi mà không cần xây dựng đồ thị (tức là theo số học, không phải ký hiệu), chỉ bằng cách tính toán các kết quả trung gian ngay lập tức. Một cách để làm điều này là sử dụng **số kép (dual numbers)**, là những số kỳ lạ nhưng hấp dẫn có dạng $a + bepsilon$, trong đó a và b là số thực và $epsilon$ là một số vô cùng nhỏ sao cho $epsilon^2 = 0$ (nhưng $epsilonneq0$). Bạn có thể nghĩ số kép $42 + 24epsilon$ như một cái gì đó tương tự $42.0000dots000024$ với vô số số 0 (nhưng tất nhiên đây chỉ là đơn giản hóa để bạn có ý tưởng về số kép). Một số kép được biểu diễn trong bộ nhớ dưới dạng một cặp số float. Ví dụ, $42 + 24epsilon$ được biểu diễn bởi cặp $(42.0,24.0)$.

Số kép có thể được cộng, nhân, v.v., như được thể hiện trong Phương trình B-3.

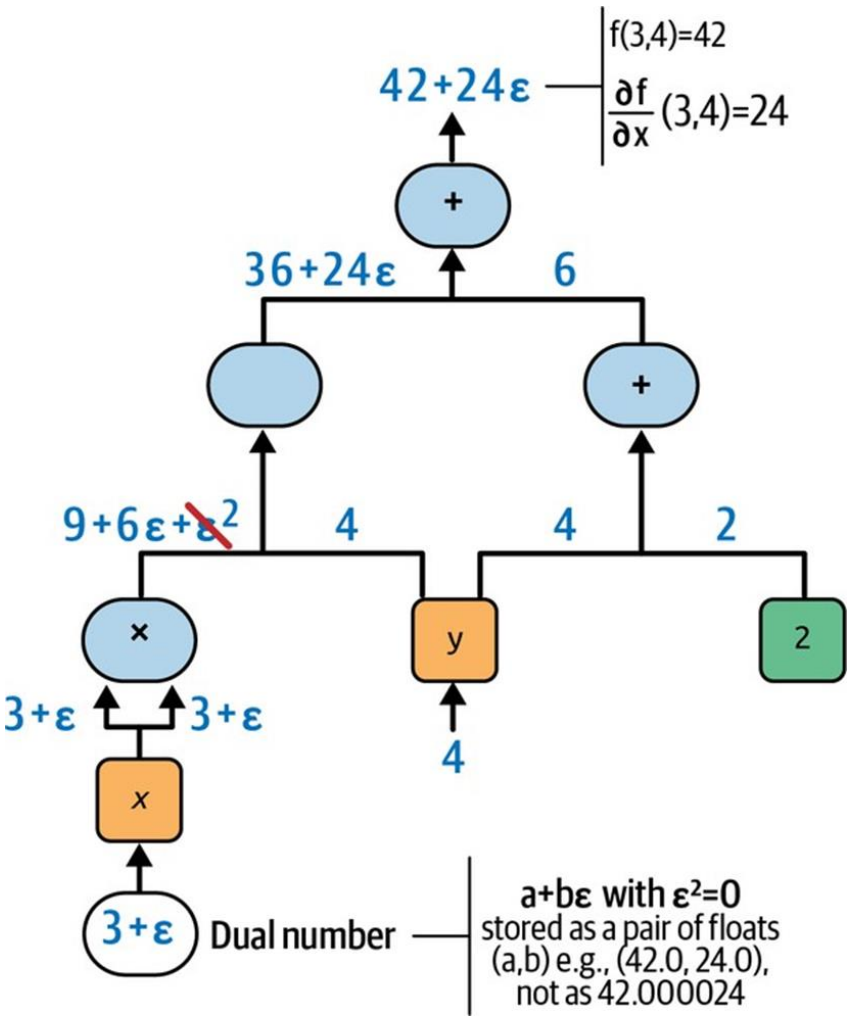
Phương trình B-3. Một vài phép toán với số kép

Công thức B-3: Một vài phép toán với số kép

- $\lambda(a + b\epsilon) = \lambda a + \lambda b\epsilon$
- $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$
- $(a + b\epsilon) \times (c + d\epsilon) = ac + (ad + bc)\epsilon + (bd)\epsilon^2 = ac + (ad + bc)\epsilon$

Quan trọng nhất, có thể chứng minh rằng $h(a + b\epsilon) = h(a) + bh'(a)\epsilon$. Do đó, việc tính $h(a + \epsilon)$ sẽ cho bạn cả $h(a)$ và đạo hàm $h'(a)$ chỉ trong một lần tính.

Điều này cho phép tính đạo hàm riêng của một hàm. Ví dụ, để tính đạo hàm riêng của $f(x, y)$ theo x tại $x = 3$ và $y = 4$ (ký hiệu là $\partial f / \partial x(3,4)$), chúng ta có thể sử dụng số kép. Tất cả những gì chúng ta cần làm là tính $f(3 + \epsilon, 4)$; kết quả sẽ là một số kép có thành phần đầu tiên bằng với $f(3,4)$ và thành phần thứ hai bằng với $\partial f / \partial x(3,4)$.



Hình B-2. Autodiff chế độ xuôi sử dụng số kép.

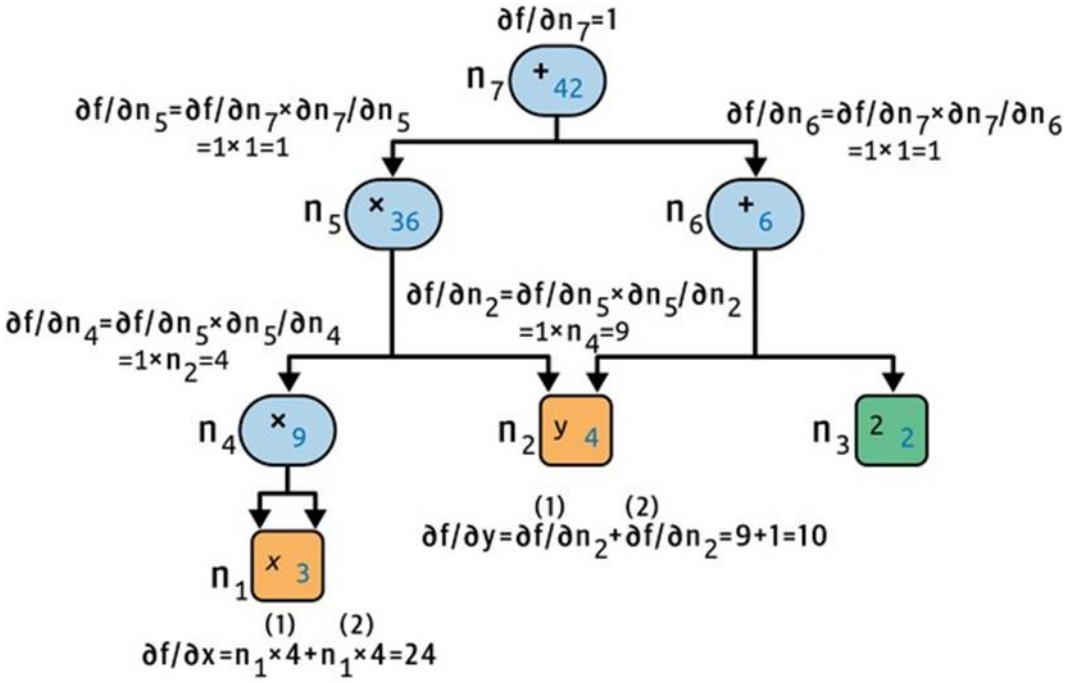
Để tính $partialf/partialy(3,4)$, chúng ta sẽ phải đi qua đồ thị một lần nữa, nhưng lần này với $x = 3$ và $y = 4 + epsilon$.

Vì vậy, autodiff chế độ xuôi chính xác hơn nhiều so với xấp xỉ sai phân hữu hạn, nhưng nó mắc phải cùng một lỗi lớn, ít nhất là khi có nhiều đầu vào và ít đầu ra (như trường hợp xử lý mạng nơ-ron): nếu có 1.000 tham số, nó sẽ yêu cầu 1.000 lần đi qua đồ thị để tính tất cả các đạo hàm riêng. Đây là nơi autodiff chế độ ngược phát huy tác dụng: nó có thể tính tất cả chúng chỉ trong hai lần đi qua đồ thị. Hãy xem cách thực hiện.

Autodiff Chế Độ Ngược (Reverse-Mode Autodiff)

Autodiff chế độ ngược là giải pháp được TensorFlow triển khai. Nó đầu tiên đi qua đồ thị theo hướng xuôi (tức là từ đầu vào đến đầu ra) để tính giá trị của mỗi nút. Sau đó, nó thực hiện một lần đi qua thứ hai, lần này theo hướng ngược lại (tức là từ đầu ra đến đầu vào), để tính tất cả các đạo hàm riêng. Tên “chế độ ngược” xuất phát từ lần đi qua thứ hai này qua đồ thị, nơi các gradient chảy theo hướng ngược lại. Hình B-3 biểu diễn lần đi qua thứ hai. Trong lần đi qua đầu tiên, tất cả các giá trị nút đã được tính toán, bắt đầu từ $x = 3$ và $y = 4$.

Bạn có thể thấy các giá trị đó ở phía dưới bên phải của mỗi nút (ví dụ: $xtimesx = 9$). Các nút được dán nhãn n_1 đến n_7 để rõ ràng. Nút đầu ra là $n_7 : f(3,4) = n_7 = 42$.



Hình B-3. Autodiff chế độ ngược.

Ý tưởng là dần dần đi xuống đồ thị, tính đạo hàm riêng của $f(x, y)$ đối với mỗi nút liên tiếp, cho đến khi chúng ta đạt đến các nút biến. Đối với điều này, autodiff chế độ ngược dựa rất nhiều vào **quy tắc chuỗi (chain rule)**, được thể hiện trong Phương trình B-4.

Công thức B-4: Quy tắc chuỗi

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_i} \times \frac{\partial n_i}{\partial x}$$

Vì n_7 là nút đầu ra, nên $f = n_7$, do đó $\partial f / \partial n_7 = 1$.

Hãy tiếp tục đi xuống đồ thị: f thay đổi như thế nào khi n_5 thay đổi? Câu trả lời là $\partial f / \partial n_5 = \partial f / \partial n_7 \times \partial n_7 / \partial n_5$. Chúng ta đã biết $\partial f / \partial n_7 = 1$. Vì n_7 chỉ đơn giản là thực hiện phép tổng $n_5 + n_6$, nên ta thấy $\partial n_7 / \partial n_5 = 1$. Do đó, $\partial f / \partial n_5 = 1 \times 1 = 1$.

Bây giờ chúng ta có thể tiếp tục đến nút $n_4 : f$ thay đổi bao nhiêu khi n_4 thay đổi? Câu trả lời là $\partial f / \partial n_4 = \partial f / \partial n_5 \times \partial n_5 / \partial n_4$. Vì $n_5 = n_4 \times n_2$, nên ta thấy $\partial n_5 / \partial n_4 = n_2$. Do đó, $\partial f / \partial n_4 = 1 \times n_2 = 4$.

Quá trình này tiếp tục cho đến khi chúng ta đi đến đáy của đồ thị. Tại thời điểm đó, chúng ta đã tính tất cả các đạo hàm riêng của $f(x, y)$ tại điểm $x = 3$ và $y = 4$. Trong ví dụ này, chúng ta tìm thấy $\partial f / \partial x = 24$ và $\partial f / \partial y = 10$. Nghe có vẻ đúng!

Quá trình này tiếp tục cho đến khi chúng ta đạt đến đáy của đồ thị. Tại thời điểm đó, chúng ta sẽ đã tính toán tất cả các đạo hàm riêng của $f(x, y)$ tại điểm $x = 3$ và $y = 4$. Trong ví dụ này, chúng ta tìm thấy $partialf/partialx = 24$ và $partialf/partialy = 10$. Nghe có vẻ đúng!

Autodiff chế độ ngược là một kỹ thuật rất mạnh mẽ và chính xác, đặc biệt khi có nhiều đầu vào và ít đầu ra, vì nó chỉ yêu cầu một lần đi qua xuôi cộng với một lần đi qua ngược cho mỗi đầu ra để tính tất cả các đạo hàm riêng cho tất cả các đầu ra đối với tất cả các đầu vào. Khi huấn luyện mạng nơ-ron, chúng ta thường muốn tối thiểu hóa hàm mất mát (loss), vì vậy chỉ có một đầu ra (hàm mất mát), và do đó chỉ cần hai lần đi qua đồ thị để tính gradient. Autodiff chế độ ngược cũng có thể xử lý các hàm không hoàn toàn khả vi, miễn là bạn yêu cầu nó tính đạo hàm riêng tại các điểm khả vi.

Trong Hình B-3, các kết quả số học được tính toán ngay lập tức, tại mỗi nút. Tuy nhiên, đó không phải chính xác những gì TensorFlow làm: thay vào đó, nó tạo ra một đồ thị tính toán mới. Nói cách khác, nó triển khai **autodiff chế độ ngược ký hiệu (symbolic reverse-mode autodiff)**. Bằng cách này, đồ thị tính toán để tính gradient của hàm mất mát đối với tất cả các tham số trong mạng nơ-ron chỉ cần được tạo một lần, và sau đó nó có thể được thực thi lặp đi lặp lại, bất cứ khi nào bộ tối ưu hóa cần tính gradient. Hơn nữa, điều này giúp có thể tính toán các đạo hàm bậc cao hơn nếu cần.

PHỤ LỤC C. CÁC CẤU TRÚC DỮ LIỆU ĐẶC BIỆT

Trong phụ lục này, chúng ta sẽ xem xét rất nhanh các cấu trúc dữ liệu được TensorFlow hỗ trợ, ngoài các tensor số thực hoặc số nguyên thông thường. Điều này bao gồm chuỗi, tensor răng cưa, tensor thưa, mảng tensor, tập hợp và hàng đợi.

Chuỗi (Strings)

Các tensor có thể chứa chuỗi byte, điều này đặc biệt hữu ích cho xử lý ngôn ngữ tự nhiên (xem Chương 16):

```
import tensorflow as tf
>>> tf.constant(b"hello world")
<tf.Tensor: shape=(), dtype=string, numpy=b'hello world'>
```

Nếu bạn cố gắng xây dựng một tensor với một chuỗi Unicode, TensorFlow sẽ tự động mã hóa nó sang UTF-8:

```
>>> tf.constant("café")
<tf.Tensor: shape=(), dtype=string, numpy=b'caf\xc3\xa9'>
```

Cũng có thể tạo các tensor biểu diễn chuỗi Unicode. Chỉ cần tạo một mảng các số nguyên 32-bit, mỗi số biểu diễn một điểm mã Unicode:

```
>>> u = tf.constant([ord(c) for c in "café"])

>>> u
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([ 99, 97, 102, 233],
dtype=int32)>
```

Gói `tf.strings` chứa một số hàm để thao tác với các tensor chuỗi, chẳng hạn như `length()` để đếm số byte trong một chuỗi byte (hoặc số điểm mã nếu bạn đặt `unit="UTF8_CHAR"`), `unicode_encode()` để chuyển đổi một tensor chuỗi Unicode (tức là tensor int32) thành một tensor chuỗi byte, và `unicode_decode()` để làm ngược lại:

```
>>> b = tf.strings.unicode_encode(u, "UTF-8")

>>> b
<tf.Tensor: shape=(), dtype=string, numpy=b'caf\xc3\xa9'>

>>> tf.strings.length(b, unit="UTF8_CHAR")
<tf.Tensor: shape=(), dtype=int32, numpy=4>

>>> tf.strings.unicode_decode(b, "UTF-8")
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([ 99, 97, 102, 233],
dtype=int32)>
```

Bạn cũng có thể thao tác với các tensor chứa nhiều chuỗi:

```
>>> p = tf.constant(["Café", "Coffee", "caffè", "咖啡"])

>>> tf.strings.length(p, unit="UTF8_CHAR")
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([4, 6, 5, 2], dtype=int32)>

>>> r = tf.strings.unicode_decode(p, "UTF8")

>>> r
<tf.RaggedTensor [[67, 97, 102, 233], [67, 111, 102, 102, 101, 101], [99, 97,
102, 102, 232], [21654, 21857]]>
```

Lưu ý rằng các chuỗi đã giải mã được lưu trữ trong một `RaggedTensor`. `RaggedTensor` là gì?

Tensor Răng Cưa (Ragged Tensors)

Một tensor răng cưa là một loại tensor đặc biệt biểu diễn một danh sách các mảng có kích thước khác nhau. Tổng quát hơn, nó là một tensor có một hoặc nhiều chiều răng cưa, nghĩa là các chiều mà các lát cắt của chúng có thể có độ dài khác nhau. Trong tensor răng cưa `r`, chiều thứ hai là một chiều răng cưa. Trong tất cả các tensor răng cưa, chiều đầu tiên luôn là một chiều thông thường (còn gọi là chiều đồng nhất).

Tất cả các phần tử của tensor răng cưa `r` đều là các tensor thông thường. Ví dụ, hãy xem phần tử thứ hai của tensor răng cưa:

```
>>> r[1]
<tf.Tensor: shape=(6,), dtype=int32, numpy=array([ 67, 111, 102, 102, 101,
101], dtype=int32)>
```

Gói `tf.ragged` chứa một số hàm để tạo và thao tác với các tensor răng cưa. Hãy tạo một tensor răng cưa thứ hai bằng cách sử dụng `tf.ragged.constant()` và nối nó với tensor răng cưa đầu tiên, dọc theo trục 0:

```
>>> r2 = tf.ragged.constant([[65, 66], [], [67]])

>>> tf.concat([r, r2], axis=0)
<tf.RaggedTensor [[67, 97, 102, 233], [67, 111, 102, 102, 101, 101], [99, 97,
102, 102, 232], [21654, 21857], [65, 66], [], [67]]>
```

Kết quả không quá ngạc nhiên: các tensor trong `r2` được nối vào sau các tensor trong `r` dọc theo trục 0. Nhưng điều gì sẽ xảy ra nếu chúng ta nối `r` và một tensor răng cưa khác dọc theo trục 1?

```
>>> r3 = tf.ragged.constant([[68, 69, 70], [71], [], [72, 73]])

>>> print(tf.concat([r, r3], axis=1))
<tf.RaggedTensor [[67, 97, 102, 233, 68, 69, 70], [67, 111, 102, 102, 101,
101, 71], [99, 97, 102, 102, 232], [21654, 21857, 72, 73]]>
```

Lần này, lưu ý rằng tensor thứ i trong `r` và tensor thứ i trong `r3` đã được nối với nhau. Điều này bất thường hơn, vì tất cả các tensor này có thể có độ dài khác nhau.

Nếu bạn gọi phương thức `to_tensor()`, tensor răng cưa sẽ được chuyển đổi thành một tensor thông thường, đệm các tensor ngắn hơn bằng số 0 để có được các tensor có độ dài bằng nhau (bạn có thể thay đổi giá trị mặc định bằng cách đặt đối số `default_value`):

```
>>> r.to_tensor()
<tf.Tensor: shape=(4, 6), dtype=int32, numpy=
array([[   67,    97,   102,   233,     0,     0],
       [   67,   111,   102,   102,   101,   101],
       [   99,    97,   102,   102,   232,     0],
       [21654, 21857,     0,     0,     0,     0]], dtype=int32)>
```

Nhiều phép toán TF hỗ trợ các tensor răng cưa. Để biết danh sách đầy đủ, xem tài liệu của lớp `tf.RaggedTensor`.

Tensor Thưa (Sparse Tensors)

TensorFlow cũng có thể biểu diễn hiệu quả các tensor thưa (tức là các tensor chứa chủ yếu là số không). Chỉ cần tạo một `tf.SparseTensor`, chỉ định các chỉ số và giá trị của các phần tử khác không và hình dạng của tensor. Các chỉ số phải được liệt kê theo "thứ tự đọc" (từ trái sang phải, và từ trên xuống dưới). Nếu bạn không chắc chắn, chỉ cần sử dụng `tf.sparse.reorder()`.

Bạn có thể chuyển đổi một tensor thưa thành một tensor dày đặc (tức là một tensor thông thường) bằng cách sử dụng `tf.sparse.to_dense()`:

```
>>> s = tf.SparseTensor(indices=[[0, 1], [1, 0], [2, 3]],
...                     values=[1., 2., 3.],
...                     dense_shape=[3, 4])
...
>>> tf.sparse.to_dense(s)
<tf.Tensor: shape=(3, 4), dtype=float32, numpy=
array([[0., 1., 0., 0.],
       [2., 0., 0., 0.],
       [0., 0., 0., 3.]], dtype=float32)>
```

Lưu ý rằng các tensor thưa không hỗ trợ nhiều phép toán như các tensor dày đặc. Ví dụ, bạn có thể nhân một tensor thưa với bất kỳ giá trị vô hướng nào, và bạn sẽ nhận được một tensor thưa mới, nhưng bạn không thể cộng một giá trị vô hướng vào một tensor thưa, vì điều này sẽ không trả về một tensor thưa:

```
>>> s * 42.0
<tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x...>

>>> s + 42.0
[...]
TypeError: unsupported operand type(s) for +: 'SparseTensor' and 'float'
```

Mảng Tensor (Tensor Arrays)

Một `tf.TensorArray` biểu diễn một danh sách các tensor. Điều này có thể tiện dụng trong các mô hình động chứa vòng lặp, để tích lũy kết quả và sau đó tính toán một số thống kê. Bạn có thể đọc hoặc ghi các tensor tại bất kỳ vị trí nào trong mảng:

```
array = tf.TensorArray(dtype=tf.float32, size=3)
array = array.write(0, tf.constant([1., 2.]))
array = array.write(1, tf.constant([3., 10.]))
array = array.write(2, tf.constant([5., 7.]))
tensor1 = array.read(1) #=> trả về (và xóa thành 0!) tf.constant([3., 10.])
print(tensor1)
```

Theo mặc định, việc đọc một mục cũng sẽ thay thế nó bằng một tensor có cùng hình dạng nhưng chứa đầy số không. Bạn có thể đặt `clear_after_read` thành `False` nếu bạn không muốn điều này.

Theo mặc định, một `TensorArray` có kích thước cố định được đặt khi tạo. Thay vào đó, bạn có thể đặt `size=0` và `dynamic_size=True` để cho phép mảng tự động mở rộng khi cần. Tuy nhiên, điều này sẽ cản trở hiệu suất, vì vậy nếu bạn biết kích thước trước, tốt hơn là sử dụng một mảng có kích thước cố định. Bạn cũng phải chỉ định `dtype`, và tất cả các phần tử phải có cùng hình dạng với phần tử đầu tiên được ghi vào mảng.

Bạn có thể xếp chồng tất cả các mục thành một tensor thông thường bằng cách gọi phương thức `stack()`:

```
>>> array.stack()
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[1., 2.],
       [0., 0.],
       [5., 7.]], dtype=float32)>
```

Tập hợp (Sets)

TensorFlow hỗ trợ các tập hợp số nguyên hoặc chuỗi (nhưng không phải số float). Nó biểu diễn các tập hợp bằng cách sử dụng các tensor thông thường. Ví dụ, tập hợp `{1, 5, 9}` chỉ được biểu diễn dưới dạng tensor `[[1, 5, 9]]`. Lưu ý rằng tensor phải có ít nhất hai chiều, và các tập hợp phải nằm ở chiều cuối cùng. Ví dụ, `[[1, 5, 9], [2, 5, 11]]` là một tensor chứa hai tập hợp độc lập: `{1, 5, 9}` và `{2, 5, 11}`.

Gói `tf.sets` chứa một số hàm để thao tác với các tập hợp. Ví dụ, hãy tạo hai tập hợp và tính hợp của chúng (kết quả là một tensor thưa, vì vậy chúng ta gọi `to_dense()` để hiển thị nó):

```
>>> a = tf.constant([[1, 5, 9]])

>>> b = tf.constant([[5, 6, 9, 11]])

>>> u = tf.sets.union(a, b)
```

```
>>> u
<tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x...>

>>> tf.sparse.to_dense(u)
<tf.Tensor: shape=(1, 5), dtype=int32, numpy=array([[ 1,  5,  6,  9, 11]],
dtype=int32)>
```

Bạn cũng có thể tính hợp của nhiều cặp tập hợp đồng thời. Nếu một số tập hợp ngắn hơn các tập hợp khác, bạn phải đệm chúng bằng một giá trị đệm, chẳng hạn như 0:

```
>>> a = tf.constant([[1, 5, 9], [10, 0, 0]])

>>> b = tf.constant([[5, 6, 9, 11], [13, 0, 0, 0]])

>>> u = tf.sets.union(a, b)

>>> tf.sparse.to_dense(u)
<tf.Tensor: shape=(2, 5), dtype=int32, numpy=
array([[ 1,  5,  6,  9, 11],
       [ 0, 10, 13,  0,  0]], dtype=int32)>
```

Nếu bạn muốn sử dụng một giá trị đệm khác, chẳng hạn như –1, thì bạn phải đặt `default_value=-1` (hoặc giá trị ưa thích của bạn) khi gọi `to_dense()`.

Các hàm khác có sẵn trong `tf.sets` bao gồm `difference()`, `intersection()`, và `size()`, những hàm này tự giải thích. Nếu bạn muốn kiểm tra xem một tập hợp có chứa một số giá trị đã cho hay không, bạn có thể tính giao của tập hợp đó và các giá trị. Nếu bạn muốn thêm một số giá trị vào một tập hợp, bạn có thể tính hợp của tập hợp và các giá trị.

Hàng đợi (Queues)

Một hàng đợi là một cấu trúc dữ liệu mà bạn có thể đẩy các bản ghi dữ liệu vào, và sau đó kéo chúng ra. TensorFlow triển khai một số loại hàng đợi trong gói `tf.queue`. Chúng từng rất quan trọng khi triển khai các pipeline tải và tiền xử lý dữ liệu hiệu quả, nhưng API `tf.data` về cơ bản đã làm cho chúng trở nên vô dụng (trừ một số trường hợp hiếm hoi) vì nó đơn giản hơn nhiều để sử dụng và cung cấp tất cả các công cụ bạn cần để xây dựng các pipeline hiệu quả. Tuy nhiên, vì mục đích hoàn chỉnh, chúng ta hãy xem xét nhanh chúng.

Loại hàng đợi đơn giản nhất là hàng đợi vào trước, ra trước (FIFO). Để xây dựng nó, bạn cần chỉ định số lượng bản ghi tối đa mà nó có thể chứa. Hơn nữa, mỗi bản ghi là một tuple các tensor, vì vậy bạn phải chỉ định kiểu của mỗi tensor, và tùy chọn hình dạng của chúng. Ví dụ, đoạn mã sau đây tạo một hàng đợi FIFO với tối đa ba bản ghi, mỗi bản ghi chứa một tuple với một số nguyên 32-bit và một chuỗi. Sau đó, nó đẩy hai bản ghi vào, xem kích thước (là 2 tại thời điểm này), và kéo một bản ghi ra:

```
>>> q = tf.queue.FIFOQueue(3, [tf.int32, tf.string], shapes=[(), ()])
```

```
>>> q.enqueue([10, b"windy"])

>>> q.enqueue([15, b"sunny"])

>>> q.size()
<tf.Tensor: shape=(), dtype=int32, numpy=2>

>>> q.dequeue()
[<tf.Tensor: shape=(), dtype=int32, numpy=10>,
 <tf.Tensor: shape=(), dtype=string, numpy=b'windy'>]
```

Cũng có thể đẩy và kéo nhiều bản ghi cùng lúc bằng cách sử dụng `enqueue_many()` và `dequeue_many()` (để sử dụng `dequeue_many()`, bạn phải chỉ định đối số `shapes` khi bạn tạo hàng đợi, như chúng ta đã làm trước đó):

```
>>> q.enqueue_many([[13, 16], [b'cloudy', b'rainy']])

>>> q.dequeue_many(3)
[<tf.Tensor: shape=(3,), dtype=int32, numpy=array([15, 13, 16],
dtype=int32)>,
 <tf.Tensor: shape=(3,), dtype=string, numpy=array([b'sunny', b'cloudy',
b'rainy'], dtype=object)>]
```

Các loại hàng đợi khác bao gồm:

- **PaddingFIFOQueue**: Giống như FIFOQueue, nhưng phương thức `dequeue_many()` của nó hỗ trợ kéo ra nhiều bản ghi có hình dạng khác nhau. Nó tự động đệm các bản ghi ngắn nhất để đảm bảo tất cả các bản ghi trong lô có cùng hình dạng.
- **PriorityQueue**: Một hàng đợi kéo các bản ghi theo thứ tự ưu tiên. Ưu tiên phải là một số nguyên 64-bit được bao gồm làm phần tử đầu tiên của mỗi bản ghi. Đáng ngạc nhiên, các bản ghi có ưu tiên thấp hơn sẽ được kéo ra trước. Các bản ghi có cùng ưu tiên sẽ được kéo ra theo thứ tự FIFO.
- **RandomShuffleQueue**: Một hàng đợi mà các bản ghi của nó được kéo ra theo thứ tự ngẫu nhiên. Điều này hữu ích để triển khai một bộ đệm xáo trộn trước khi `tf.data` tồn tại.

Nếu một hàng đợi đã đầy và bạn cố gắng đẩy một bản ghi khác vào, phương thức `enqueue*()` sẽ bị đóng băng cho đến khi một bản ghi được kéo ra bởi một luồng khác. Tương tự, nếu một hàng đợi trống và bạn cố gắng kéo một bản ghi ra, phương thức `dequeue*()` sẽ bị đóng băng cho đến khi các bản ghi được đẩy vào hàng đợi bởi một luồng khác.

Phán đoán và Công thức Toán:

Phụ lục này chủ yếu mô tả các cấu trúc dữ liệu đặc biệt trong TensorFlow và cách chúng được sử dụng, không đi sâu vào các công thức toán học. Tuy nhiên, có một số điểm có thể liên hệ với khái niệm toán học và cách chúng được biểu diễn hoặc thao tác:

- **Unicode Code Points:**
  - `tf.constant([ord(c) for c in "café"])`: Hàm `ord(c)` trả về giá trị số nguyên của điểm mã Unicode của ký tự `c`. Đây là một ánh xạ từ ký tự sang một số nguyên duy nhất, về bản chất là một phép biểu diễn số. Ví dụ:
    - `ord('c') = 99`
    - `ord('a') = 97`
    - `ord('f') = 102`
    - `ord('é') = 233`
  - Biểu diễn UTF-8: `b'caf\xc3\xa9'` cho "café". Đây là một cách mã hóa chuỗi Unicode thành chuỗi byte, trong đó một số ký tự (như é) yêu cầu nhiều byte.
- **Ragged Tensors:**
  - Đây là cấu trúc dữ liệu để biểu diễn các mảng có độ dài không đồng nhất. Mặc dù không có công thức toán học, khái niệm này liên quan đến việc quản lý bộ nhớ và truy cập hiệu quả các dữ liệu có kích thước thay đổi.
  - Khi chuyển đổi sang tensor thông thường bằng `to_tensor()`, việc đệm bằng số 0 (hoặc giá trị mặc định) có thể được coi là việc thêm các phần tử giả vào các hàng/cột ngắn hơn để chúng có cùng kích thước, làm cho tensor có hình dạng hình chữ nhật hoặc khối lập phương.
- **Sparse Tensors:**
  - Biểu diễn hiệu quả các tensor chứa chủ yếu là số không. Điều này là một kỹ thuật tối ưu hóa bộ nhớ cho các ma trận thưa (sparse matrices).
  - `tf.SparseTensor(indices=[[0, 1], [1, 0], [2, 3]], values=[1., 2., 3.], dense_shape=[3, 4])`: Cấu trúc này lưu trữ chỉ các phần tử khác không cùng với chỉ số của chúng và hình dạng tổng thể của tensor. Ví dụ, phần tử `1.` nằm ở `(0, 1)`, phần tử `2.` ở `(1, 0)`, v.v. Các vị trí không được liệt kê ngụ ý giá trị 0.
  - Phép nhân vô hướng `s * 42.0`: Chỉ các giá trị khác không cần được nhân với vô hướng.
    - Đối với mỗi (idx, val) trong `SparseTensor`, phép toán trở thành $(idx, valtimesscalar)$.
  - Phép cộng vô hướng `s + 42.0` không được hỗ trợ vì nó sẽ biến hầu hết các phần tử 0 thành 42, làm cho tensor không còn thưa nữa và yêu cầu chuyển đổi sang tensor dày đặc.
- **Tập hợp (Sets):**
  - Các phép toán tập hợp như hợp (`union`), giao (`intersection`), hiệu (`difference`), và kích thước (`size`) là các phép toán cơ bản trong lý thuyết tập hợp.

- `tf.sets.union(a, b)`: Tính hợp của hai tập hợp. Nếu A và B là hai tập hợp, hợp của chúng là $AcupB = xmidxinAtexthoặcxinB$.

- **Hàng đợi (Queues):**
  - Các hàng đợi (FIFO, Priority, RandomShuffle) là các cấu trúc dữ liệu trừu tượng với các quy tắc cụ thể cho việc thêm (enqueue) và xóa (dequeue) phần tử.
  - **FIFO (First-In, First-Out):** Phần tử được thêm vào đầu tiên sẽ được xóa ra đầu tiên. Giống như một hàng người.
  - **PriorityQueue:** Các phần tử được xóa ra dựa trên một giá trị ưu tiên. Mặc dù văn bản nói rằng "lower priority will be dequeued first" (ưu tiên thấp hơn sẽ được kéo ra trước), điều này có thể hiểu là "giá trị số ưu tiên thấp hơn" tương ứng với "mức độ ưu tiên cao hơn" trong một số ngữ cảnh.
  - **RandomShuffleQueue:** Các phần tử được xóa ra theo thứ tự ngẫu nhiên, ngụ ý một quá trình lấy mẫu ngẫu nhiên từ tập hợp các phần tử trong hàng đợi.

Tóm lại, phần phụ lục này trình bày các cấu trúc dữ liệu được thiết kế để xử lý hiệu quả các loại dữ liệu cụ thể (chuỗi, dữ liệu có kích thước không đồng nhất, dữ liệu thưa, dữ liệu động) trong bối cảnh TensorFlow, phản ánh sự giao thoa giữa khoa học máy tính và các nguyên tắc toán học cơ bản.

Phụ lục D. Đồ thị TensorFlow

Trong phụ lục này, chúng ta sẽ khám phá các đồ thị được tạo bởi các hàm TF (xem Chương 12).

Hàm TF và Hàm Cụ thể (Concrete Functions)

Các hàm TF là đa hình (polymorphic), nghĩa là chúng hỗ trợ đầu vào thuộc các kiểu (và hình dạng) khác nhau. Ví dụ, hãy xem xét hàm `tf_cube()` sau:

```
import tensorflow as tf

@tf.function
def tf_cube(x):
    return x**3
```

Mỗi khi bạn gọi một hàm TF với một sự kết hợp mới của các kiểu hoặc hình dạng đầu vào, nó sẽ tạo ra một **hàm cụ thể (concrete function)** mới, với đồ thị riêng của nó được chuyên biệt hóa cho sự kết hợp cụ thể này. Sự kết hợp các kiểu và hình dạng đối số như vậy được gọi là một **chữ ký đầu vào (input signature)**. Nếu bạn gọi hàm TF với một chữ ký đầu vào mà nó đã thấy trước đó, nó sẽ sử dụng lại hàm cụ thể mà nó đã tạo trước đó. Ví dụ, nếu bạn gọi `tf_cube(tf.constant(3.0))`, hàm TF sẽ sử dụng lại cùng hàm cụ thể mà nó đã sử dụng cho `tf_cube(tf.constant(2.0))` (đối với các tensor vô hướng float32). Nhưng nó sẽ tạo ra một hàm cụ thể mới nếu bạn gọi `tf_cube(tf.constant([2.0]))` hoặc `tf_cube(tf.constant([3.0]))` (đối với các tensor float32 có hình dạng [1]), và một hàm khác nữa cho `tf_cube(tf.constant([[1.0, 2.0], [3.0, 4.0]]))` (đối với các tensor float32 có hình dạng [2, 2]). Bạn có thể lấy hàm cụ thể cho một sự kết hợp đầu vào cụ thể bằng cách gọi phương thức `get_concrete_function()` của hàm TF. Sau đó, nó có thể được gọi như một hàm thông thường, nhưng nó sẽ chỉ hỗ trợ một chữ ký đầu vào (trong ví dụ này, các tensor vô hướng float32):

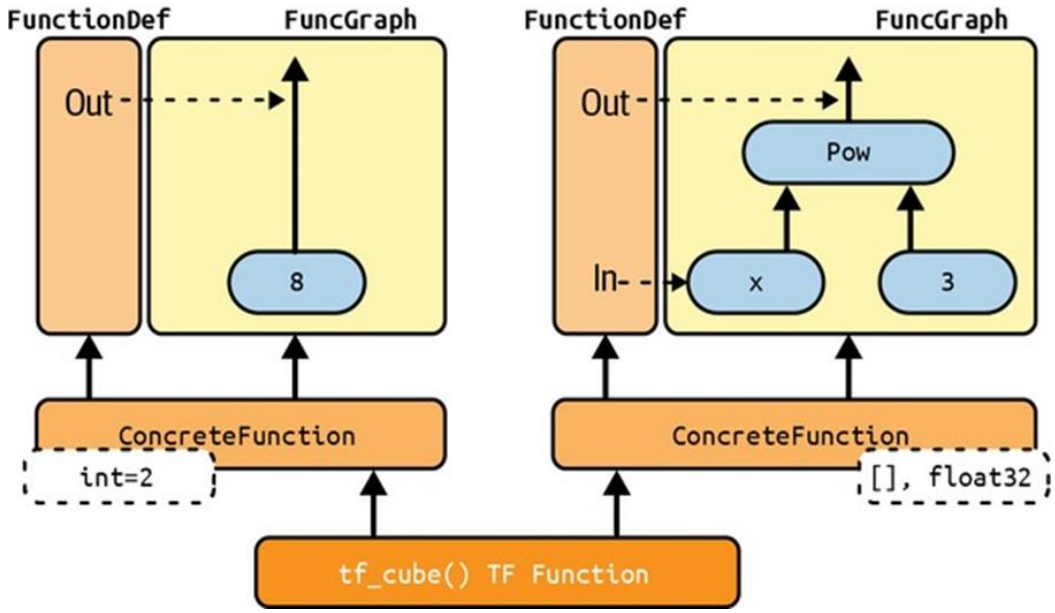
```
>>> concrete_function = tf_cube.get_concrete_function(tf.constant(2.0))

>>> concrete_function
<ConcreteFunction tf_cube(x) at 0x...> # Địa chỉ có thể khác

>>> concrete_function(tf.constant(2.0))
<tf.Tensor: shape=(), dtype=tf.float32, numpy=8.0>
```

Hình D-1 cho thấy hàm TF `tf_cube()`, sau khi chúng ta gọi `tf_cube(2)` và `tf_cube(tf.constant(2.0))`: hai hàm cụ thể đã được tạo, một cho mỗi chữ ký, mỗi hàm có đồ thị hàm tối ưu hóa riêng (`FuncGraph`) và định nghĩa hàm riêng (`FunctionDef`). Một định nghĩa hàm trỏ đến các phần của đồ thị tương ứng với đầu vào và đầu ra của hàm. Trong mỗi `FuncGraph`, các nút (hình bầu dục) biểu thị các phép toán (ví dụ: lũy thừa, hằng số, hoặc các placeholder cho các đối số như `x`), trong khi các cạnh (các mũi tên liền nét giữa các phép toán) biểu thị các tensor sẽ chảy qua đồ thị. Hàm cụ thể bên trái được chuyên biệt hóa cho $x = 2$, vì vậy TensorFlow đã quản lý để đơn giản hóa nó chỉ để xuất ra 8 mọi lúc

(lưu ý rằng định nghĩa hàm thậm chí không có đầu vào). Hàm cụ thể bên phải được chuyên biệt hóa cho các tensor vô hướng float32, và nó không thể được đơn giản hóa. Nếu chúng ta gọi `tf_cube(tf.constant(5.0))`, hàm cụ thể thứ hai sẽ được gọi, phép toán placeholder cho x sẽ xuất ra 5.0, sau đó phép toán lũy thừa sẽ tính 5.0^3 , vì vậy đầu ra sẽ là 125.0.



Hình D-1. Hàm TF `tf_cube()`, với các Hàm Cụ thể và `FuncGraph` của chúng.

Các tensor trong các đồ thị này là các **tensor ký hiệu (symbolic tensors)**, nghĩa là chúng không có giá trị thực tế, chỉ có kiểu dữ liệu, hình dạng và tên. Chúng biểu thị các tensor trong tương lai sẽ chảy qua đồ thị một khi một giá trị thực tế được đưa vào placeholder x và đồ thị được thực thi. Các tensor ký hiệu giúp có thể chỉ định trước cách kết nối các phép toán, và chúng cũng cho phép TensorFlow tự động suy luận kiểu dữ liệu và hình dạng của tất cả các tensor một cách đệ quy, dựa trên kiểu dữ liệu và hình dạng của đầu vào của chúng.

Bây giờ hãy tiếp tục tìm hiểu sâu hơn, và xem cách truy cập định nghĩa hàm và đồ thị hàm và cách khám phá các phép toán và tensor của đồ thị.

Khám phá Định nghĩa Hàm và Đồ thị

Bạn có thể truy cập đồ thị tính toán của một hàm cụ thể bằng thuộc tính `graph`, và lấy danh sách các phép toán của nó bằng cách gọi phương thức `get_operations()` của đồ thị:

```
>>> concrete_function.graph
<tensorflow.python.framework.func_graph.FuncGraph object at 0x...> # Địa chỉ
có thể khác

>>> ops = concrete_function.graph.get_operations()
```

```
>>> ops
[<tf.Operation 'x' type=Placeholder>,
<tf.Operation 'pow/y' type=Const>,
<tf.Operation 'pow' type=Pow>,
<tf.Operation 'Identity' type=Identity>]
```

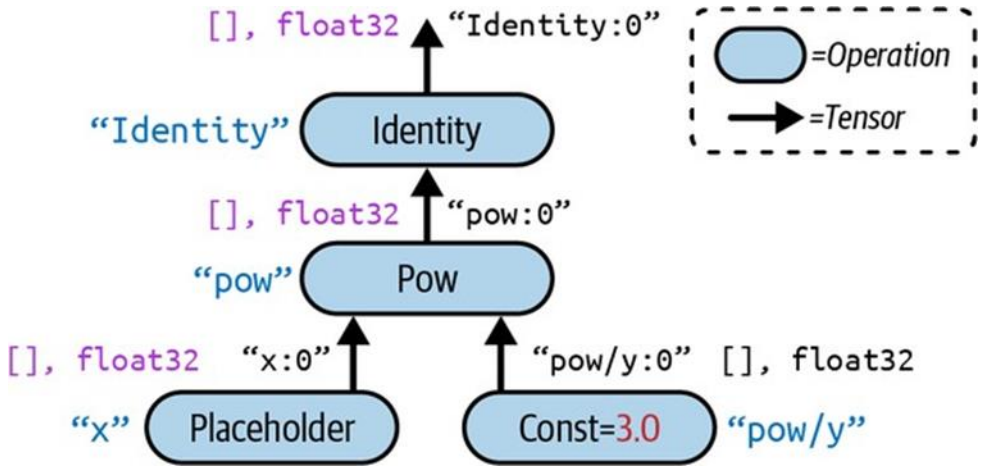
Trong ví dụ này, phép toán đầu tiên biểu thị đối số đầu vào x (nó được gọi là một **placeholder**), phép toán thứ hai biểu thị hằng số 3, phép toán thứ ba biểu thị phép toán lũy thừa (**), và phép toán cuối cùng biểu thị đầu ra của hàm này (nó là một phép toán đồng nhất (identity operation), nghĩa là nó sẽ không làm gì khác ngoài việc sao chép đầu ra của phép toán lũy thừa). Mỗi phép toán có một danh sách các tensor đầu vào và đầu ra mà bạn có thể dễ dàng truy cập bằng các thuộc tính `inputs` và `outputs` của phép toán. Ví dụ, hãy lấy danh sách các đầu vào và đầu ra của phép toán lũy thừa:

```
>>> pow_op = ops[2]

>>> list(pow_op.inputs)
[<tf.Tensor 'x:0' shape=() dtype=tf.float32>,
<tf.Tensor 'pow/y:0' shape=() dtype=tf.float32>]

>>> pow_op.outputs
[<tf.Tensor 'pow:0' shape=() dtype=tf.float32>]
```

Đồ thị tính toán này được biểu diễn trong Hình D-2.



Hình D-2. Ví dụ về đồ thị tính toán.

Lưu ý rằng mỗi phép toán có một tên. Nó mặc định là tên của phép toán (ví dụ: "pow"), nhưng bạn có thể định nghĩa thủ công khi gọi phép toán (ví dụ: `tf.pow(x, 3, name="other_name")`). Nếu một tên đã tồn tại, TensorFlow tự động thêm một chỉ mục duy nhất (ví dụ: "pow_1", "pow_2", v.v.).

Mỗi tensor cũng có một tên duy nhất: nó luôn là tên của phép toán xuất ra tensor này, cộng với `:0` nếu đó là đầu ra đầu tiên của phép toán, hoặc `:1` nếu đó là đầu ra thứ hai, v.v. Bạn có

thể lấy một phép toán hoặc một tensor bằng tên bằng cách sử dụng các phương thức `get_operation_by_name()` hoặc `get_tensor_by_name()` của đồ thị:

```
>>> concrete_function.graph.get_operation_by_name('x')
<tf.Operation 'x' type=Placeholder>

>>> concrete_function.graph.get_tensor_by_name('Identity:0')
<tf.Tensor 'Identity:0' shape=() dtype=tf.float32>
```

Hàm cụ thể cũng chứa định nghĩa hàm (được biểu diễn dưới dạng protocol buffer), bao gồm chữ ký của hàm. Chữ ký này cho phép hàm cụ thể biết các placeholder nào sẽ được cấp các giá trị đầu vào, và các tensor nào sẽ được trả về:

```
>>> concrete_function.function_def.signature
name: "inference_tf_cube_..." # Tên có thể khác nhau
input_arg {
  name: "x"
  type: DT_FLOAT
}
output_arg {
  name: "identity"
  type: DT_FLOAT
}
```

Bây giờ chúng ta hãy xem xét kỹ hơn về việc truy vết (tracing).

Xem xét kỹ hơn về Tracing

Hãy điều chỉnh hàm `tf_cube()` để in đầu vào của nó:

```
@tf.function
def tf_cube(x):
    print(f"x = {x}")
    return x ** 3
```

Bây giờ hãy gọi nó:

```
>>> result = tf_cube(tf.constant(2.0))
x = Tensor("x:0", shape=(), dtype=tf.float32)
llll
>>> result
<tf.Tensor: shape=(), dtype=tf.float32, numpy=8.0>
```

Kết quả trông tốt, nhưng hãy nhìn vào những gì đã được in: x là một tensor ký hiệu! Nó có hình dạng và kiểu dữ liệu, nhưng không có giá trị. Hơn nữa nó có một tên (“x:0”). Điều này là do hàm `print()` không phải là một phép toán TensorFlow, vì vậy nó sẽ chỉ chạy khi hàm Python được truy vết (traced), điều này xảy ra ở chế độ đồ thị, với các đối số được thay thế bằng các tensor ký hiệu (cùng kiểu và hình dạng, nhưng không có giá trị). Vì hàm `print()` không được ghi vào đồ thị, những lần tiếp theo chúng ta gọi `tf_cube()` với các tensor vô hướng float32, không có gì được in:

```
>>> result = tf_cube(tf.constant(3.0))

>>> result = tf_cube(tf.constant(4.0))
```

Nhưng nếu chúng ta gọi `tf_cube()` với một tensor có kiểu hoặc hình dạng khác, hoặc với một giá trị Python mới, hàm sẽ được truy vết lại, vì vậy hàm `print()` sẽ được gọi:

```
>>> result = tf_cube(2) # giá trị Python mới: truy vết!
x = 2

>>> result = tf_cube(3) # giá trị Python mới: truy vết!
x = 3

>>> result = tf_cube(tf.constant([[1., 2.]])) # hình dạng mới: truy vết!
x = Tensor("x:0", shape=(1, 2), dtype=tf.float32)

>>> result = tf_cube(tf.constant([[3., 4.], [5., 6.]])) # hình dạng mới: truy
vết!
x = Tensor("x:0", shape=(None, 2), dtype=tf.float32)

>>> result = tf_cube(tf.constant([[7., 8.], [9., 10.]])) # cùng hình dạng:
không truy vết
```

Trong một số trường hợp, bạn có thể muốn hạn chế một hàm TF vào một chữ ký đầu vào cụ thể. Ví dụ, giả sử bạn biết rằng bạn sẽ chỉ gọi một hàm TF với các lô ảnh 28 × 28 pixel, nhưng các lô sẽ có kích thước rất khác nhau. Bạn có thể không muốn TensorFlow tạo ra một hàm cụ thể khác cho mỗi kích thước lô, hoặc dựa vào nó để tự mình tìm ra khi nào nên sử dụng `None`. Trong trường hợp này, bạn có thể chỉ định chữ ký đầu vào như sau:

```
@tf.function(input_signature=[tf.TensorSpec([None, 28, 28], tf.float32)])
def shrink(images):
    return images[:, ::2, ::2] # loại bỏ một nửa số hàng và cột
```

Hàm TF này sẽ chấp nhận bất kỳ tensor float32 nào có hình dạng `[*, 28, 28]`, và nó sẽ sử dụng lại cùng hàm cụ thể mọi lúc:

```
img_batch_1 = tf.random.uniform(shape=[100, 28, 28])
img_batch_2 = tf.random.uniform(shape=[50, 28, 28])
preprocessed_images = shrink(img_batch_1) # hoạt động tốt, truy vết hàm
preprocessed_images = shrink(img_batch_2) # hoạt động tốt, cùng hàm cụ thể
```

Tuy nhiên, nếu bạn cố gắng gọi hàm TF này với một giá trị Python, hoặc một tensor có kiểu dữ liệu hoặc hình dạng không mong muốn, bạn sẽ nhận được một ngoại lệ:

```
img_batch_3 = tf.random.uniform(shape=[2, 2, 2])
# preprocessed_images = shrink(img_batch_3) # ValueError! Đầu vào không tương
thích (chạy dòng này sẽ gây lỗi)
```

Sử dụng AutoGraph để Ghi lại Luồng Điều khiển (Control Flow)

Nếu hàm của bạn chứa một vòng lặp `for` đơn giản, bạn mong đợi điều gì sẽ xảy ra? Ví dụ, hãy viết một hàm sẽ cộng 10 vào đầu vào của nó, bằng cách chỉ cộng 1 10 lần:

```
@tf.function
def add_10(x):
    for i in range(10):
        x += 1
    return x
```

Nó hoạt động tốt, nhưng khi chúng ta nhìn vào đồ thị của nó, chúng ta thấy rằng nó không chứa một vòng lặp: nó chỉ chứa 10 phép toán cộng!

```
>>> add_10(tf.constant(0))
<tf.Tensor: shape=(), dtype=tf.int32, numpy=10> # Kết quả là 10, không phải
15 như ví dụ gốc, do x ban đầu là 0

>>> add_10.get_concrete_function(tf.constant(0)).graph.get_operations()

# Output sẽ tương tự như sau (đã giản lược để dễ đọc):
# [<tf.Operation 'x' type=Placeholder>,
# <tf.Operation 'add' type=AddV2>,
# <tf.Operation 'add_1' type=AddV2>,
# ... (thêm 7 phép AddV2 nữa)
# <tf.Operation 'add_9' type=AddV2>,
# <tf.Operation 'Identity' type=Identity>]
```

Điều này thực sự có lý: khi hàm được truy vết, vòng lặp chạy 10 lần, vì vậy phép toán `x += 1` được chạy 10 lần, và vì nó ở chế độ đồ thị, nó đã ghi lại phép toán này 10 lần trong đồ thị. Bạn có thể nghĩ vòng lặp `for` này như một vòng lặp "tĩnh" được mở rộng (unrolled) khi đồ thị được tạo.

Nếu bạn muốn đồ thị chứa một vòng lặp "động" thay vào đó (tức là một vòng lặp chạy khi đồ thị được thực thi), bạn có thể tạo một cách thủ công bằng cách sử dụng phép toán `tf.while_loop()`, nhưng nó không trực quan lắm (xem phần "Using AutoGraph to Capture Control Flow" của sổ tay Chương 12 để biết ví dụ). Thay vào đó, đơn giản hơn nhiều là sử dụng tính năng **AutoGraph** của TensorFlow, đã thảo luận trong Chương 12.

AutoGraph thực sự được kích hoạt theo mặc định (nếu bạn cần tắt nó, bạn có thể truyền `autograph=False` cho `tf.function()`). Vậy nếu nó được bật, tại sao nó không ghi lại vòng lặp `for` trong hàm `add_10()`? Nó chỉ ghi lại các vòng lặp `for` lặp lại trên các tensor của đối tượng `tf.data.Dataset`, vì vậy bạn nên sử dụng `tf.range()`, không phải `range()`. Điều này là để cho bạn lựa chọn:

- Nếu bạn sử dụng `range()`, vòng lặp `for` sẽ là tĩnh, nghĩa là nó sẽ chỉ được thực thi khi hàm được truy vết. Vòng lặp sẽ được "mở rộng" thành một tập hợp các phép toán cho mỗi lần lặp, như chúng ta đã thấy.

- Nếu bạn sử dụng `tf.range()`, vòng lặp sẽ là động, nghĩa là nó sẽ được bao gồm trong chính đồ thị (nhưng nó sẽ không chạy trong quá trình truy vết).

Hãy xem đồ thị được tạo ra nếu chúng ta chỉ thay thế `range()` bằng `tf.range()` trong hàm `add_10()`:

```
@tf.function
def add_10_tf_range(x):
    for i in tf.range(10): # Thay đổi ở đây
        x += 1
    return x

>>>
add_10_tf_range.get_concrete_function(tf.constant(0)).graph.get_operations()
# Output sẽ tương tự như sau (đã giản lược):
# [<tf.Operation 'x' type=Placeholder>,
#  <tf.Operation 'while' type=StatelessWhile>, ...]
```

Như bạn có thể thấy, đồ thị bây giờ chứa một phép toán vòng lặp `While`, như thể chúng ta đã gọi hàm `tf.while_loop()`.

Xử lý Biến và các Tài nguyên Khác trong Hàm TF

Trong TensorFlow, các biến và các đối tượng có trạng thái khác, chẳng hạn như hàng đợi hoặc tập dữ liệu, được gọi là **tài nguyên (resources)**. Các hàm TF xử lý chúng một cách đặc biệt: bất kỳ phép toán nào đọc hoặc cập nhật một tài nguyên đều được coi là **có trạng thái (stateful)**, và các hàm TF đảm bảo rằng các phép toán có trạng thái được thực thi theo thứ tự chúng xuất hiện (ngược lại với các phép toán không trạng thái, có thể chạy song song, vì vậy thứ tự thực thi của chúng không được đảm bảo). Hơn nữa, khi bạn truyền một tài nguyên làm đối số cho một hàm TF, nó được truyền theo tham chiếu (by reference), vì vậy hàm có thể sửa đổi nó. Ví dụ:

```
counter = tf.Variable(0)

@tf.function
def increment(counter_var, c=1): # Đổi tên counter thành counter_var để tránh
nhầm lẫn với biến global
    return counter_var.assign_add(c)

increment(counter) # counter là 1
increment(counter) # counter là 2
print(counter)
```

Nếu bạn nhìn vào định nghĩa hàm, đối số đầu tiên được đánh dấu là một tài nguyên:

```
>>> function_def = increment.get_concrete_function(counter).function_def

>>> function_def.signature.input_arg[0]
```

```
name: "counter_var"
type: DT_RESOURCE
```

Cũng có thể sử dụng một `tf.Variable` được định nghĩa bên ngoài hàm, mà không cần truyền nó rõ ràng làm đối số:

```
global_counter = tf.Variable(0)

@tf.function
def increment_global(c=1):
    return global_counter.assign_add(c)

increment_global() # global_counter là 1
increment_global() # global_counter là 2
print(global_counter)
```

Hàm TF sẽ coi đây là một đối số đầu tiên ngầm định, vì vậy nó thực sự sẽ có cùng chữ ký (trừ tên đối số). Tuy nhiên, việc sử dụng biến toàn cục có thể nhanh chóng trở nên lộn xộn, vì vậy bạn thường nên gói các biến (và các tài nguyên khác) bên trong các lớp. Tin tốt là `@tf.function` cũng hoạt động tốt với các phương thức:

```
class MyCounter:
    def __init__(self):
        self.counter = tf.Variable(0)

    @tf.function
    def increment(self, c=1):
        return self.counter.assign_add(c)

my_obj_counter = MyCounter()
my_obj_counter.increment()
my_obj_counter.increment()
print(my_obj_counter.counter)
```

Một ví dụ điển hình của cách tiếp cận hướng đối tượng này là Keras. Hãy xem cách sử dụng các hàm TF với Keras.

Sử dụng Hàm TF với Keras (hoặc không)

Theo mặc định, bất kỳ hàm, lớp hoặc mô hình tùy chỉnh nào bạn sử dụng với Keras sẽ tự động được chuyển đổi thành hàm TF; bạn không cần làm gì cả! Tuy nhiên, trong một số trường hợp, bạn có thể muốn hủy kích hoạt tính năng chuyển đổi tự động này — ví dụ, nếu mã tùy chỉnh của bạn không thể biến thành hàm TF, hoặc nếu bạn chỉ muốn gỡ lỗi mã của mình (dễ hơn nhiều ở chế độ eager). Để làm điều này, bạn có thể đơn giản truyền `dynamic=True` khi tạo mô hình hoặc bất kỳ lớp nào của nó:

```
#               class               MyModel(tf.keras.Model):               ...
# model = MyModel(dynamic=True)
```

Nếu mô hình hoặc lớp tùy chỉnh của bạn sẽ luôn là động, bạn có thể thay vào đó gọi hàm tạo của lớp cơ sở với `dynamic=True`:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(dynamic=True, **kwargs)
        self.units = units
        # ... các khởi tạo khác ...

    def build(self, input_shape):
        self.w = self.add_weight(shape=(input_shape[-1], self.units),
                                 initializer="random_normal",
                                 trainable=True)
        self.b = self.add_weight(shape=(self.units,),
                                 initializer="zeros",
                                 trainable=True)
        super().build(input_shape)

    def call(self, inputs):
        return tf.matmul(inputs, self.w) + self.b
```

Ngoài ra, bạn có thể truyền `run_eagerly=True` khi gọi phương thức `compile()`:

```
#     model.compile(loss=my_mse,     optimizer="nadam",     metrics=[my_mae],
run_eagerly=True)
```

Bây giờ bạn đã biết cách các hàm TF xử lý tính đa hình (với nhiều hàm cụ thể), cách các đồ thị được tự động tạo bằng AutoGraph và truy vết, đồ thị trông như thế nào, cách khám phá các phép toán và tensor ký hiệu của chúng, cách xử lý các biến và tài nguyên, và cách sử dụng các hàm TF với Keras.

Phán đoán và Công thức Toán:

Phụ lục này tập trung vào cách TensorFlow xây dựng và quản lý các đồ thị tính toán thông qua cơ chế AutoGraph và Concrete Functions, cũng như cách xử lý các tài nguyên và luồng điều khiển. Mặc dù không có công thức toán học phức tạp được trình bày trực tiếp, các khái niệm liên quan đến đồ thị tính toán và biến đổi dữ liệu có thể được liên hệ với toán học.

- **Hàm `tf_cube(x) = x^3`:** Đây là một phép toán lũy thừa cơ bản.
    - Đầu vào x có thể là một tensor (số vô hướng, vector, ma trận, tensor đa chiều).
    - Phép toán là nâng lũy thừa bậc ba cho từng phần tử của tensor đầu vào.
    - Ví dụ:
        - Nếu $x = 2.0$ (vô hướng), thì $x^3 = 2.0^3 = 8.0$.
        - Nếu $x = [2.0]$ (vector), thì $x^3 = [2.0^3] = [8.0]$.

    - Nếu $x = [[1.0,2.0],[3.0,4.0]]$ (ma trận), thì $x^3 = [[1.0^3, 2.0^3],[3.0^3, 4.0^3]] = [[1.0,8.0],[27.0,64.0]]$.
- **Phép toán `images[:, ::2, ::2]` trong hàm `shrink`:** Đây là một phép cắt lát (slicing) tensor trong NumPy/TensorFlow.
    - `::` chọn tất cả các phần tử dọc theo chiều đó.
    - `::2`: chọn các phần tử với bước nhảy là 2, tức là chỉ lấy các phần tử ở vị trí 0, 2, 4, ...
    - Với `images` có hình dạng `[batch_size, height, width]`, phép toán này sẽ giảm chiều cao và chiều rộng đi một nửa. Nếu `height` ban đầu là H và `width` ban đầu là W, thì chiều cao mới sẽ là $lceilH/2rceil$ và chiều rộng mới là $lceilW/2rceil$.
      $$shape_{new} = (batch_size, \lceil height/2 \rceil, \lceil width/2 \rceil)$$
    - Ví dụ: Nếu `images` có shape `[100, 28, 28]`, `shrink(images)` sẽ trả về một tensor có shape `[100, 14, 14]`.
- **Vòng lặp `x += 1` trong đồ thị tĩnh:** Khi `for i in range(10): x += 1` được chuyển đổi thành đồ thị tĩnh, nó thực chất là một chuỗi 10 phép toán cộng:

$x_\{new\} = ⊢(⊢(⊢(⊢(⊢(⊢(⊢(⊢(⊢(x_\{initial\} + 1⊣) + 1⊣) + 1⊣) + 1⊣) + 1⊣) + 1⊣) + 1⊣) + 1⊣) + 1⊣) + 1⊣)$

Đây là một phép toán đơn giản: $x_new = x_initial + 10$.

Các khái niệm về `FuncGraph`, `Operation`, `Tensor` đều là các thành phần của một biểu đồ tính toán. Biểu đồ này về cơ bản biểu diễn một chuỗi các phép toán toán học (ví dụ: lũy thừa, cộng, nhân ma trận) và luồng dữ liệu (tensor) giữa chúng. Mục tiêu của việc chuyển đổi sang đồ thị là để tối ưu hóa việc thực thi các phép toán này, ví dụ như song song hóa, tối ưu hóa bộ nhớ, hoặc biên dịch thành mã máy hiệu quả hơn.